

AlgoMaster Exclusives

344 questions not in Set 1 or NeetCode ·

Priority-Grouped · 1×1 Portrait

Python Only · Priority-Grouped ·

Difficulty-First · 2×1 Landscape

**344 questions · 9 rounds · Inline Quick Review
· Chapter 2 Summary**

**High Easy → High Med → High Hard → Mid Easy
→ Mid Med → Mid Hard → Low Easy → Low Med
→ Low Hard**

Contents

★ = LeetCode Premium (Premium 98 list)

Round 1 | High | E Easy (38)

Arrays (5)

- ☐ ● #26 Remove Duplicates from Sorted Array
- ☐ ● #27 Remove Element
- ☐ ● #414 Third Maximum Number
- ☐ ● #485 Max Consecutive Ones
- ☐ ● #1470 Shuffle the Array

Strings (6)

- ☐ ● #28 Find the Index of the First Occurrence in a String
- ☐ ● #58 Length of Last Word
- ☐ ● #344 Reverse String
- ☐ ● #392 Is Subsequence
- ☐ ● #680 Valid Palindrome II
- ☐ ● #796 Rotate String

Hash Tables (8)

- ☐ ● #205 Isomorphic Strings ↗
- ☐ ● #219 Contains Duplicate II ↗
- ☐ ● #290 Word Pattern ↗
- ☐ ● #350 Intersection of Two Arrays II ↗
- ☐ ● #387 First Unique Character in a String ↗
- ☐ ● #448 Find All Numbers Disappeared in an Array ↗
- ☐ ● #1189 Maximum Number of Balloons ↗
- ☐ ● #1512 Number of Good Pairs ↗

Two Pointers (2) ↗

- ☐ ● #696 Count Binary Substrings ↗
- ☐ ● #1768 Merge Strings Alternately ↗

Sliding Window – Fixed Size (1) ↗

- ☐ ● #643 Maximum Average Subarray I ↗

Binary Search (2) ↗

- ☐ ● #367 Valid Perfect Square ↗
- ☐ ● #744 Find Smallest Letter Greater Than Target ↗

Stacks (3) ↗

- ☐ ● #682 Baseball Game ↗
- ☐ ● #1047 Remove All Adjacent Duplicates In String ↗
- ☐ ● #1614 Maximum Nesting Depth of the Parentheses ↗

Tree Traversal – Level Order (1) ↗

- ☐ ● #637 Average of Levels in Binary Tree ↗

Tree Traversal – Pre Order (4) ↗

- ☐ ● #144 Binary Tree Preorder Traversal ↗
- ☐ ● #222 Count Complete Tree Nodes ↗
- ☐ ● #257 Binary Tree Paths ↗
- ☐ ● #404 Sum of Left Leaves ↗

Tree Traversal – In Order (3) ↗

- ☐ ● #94 Binary Tree Inorder Traversal ↗
- ☐ ● #530 Minimum Absolute Difference in BST ↗
- ☐ ● #783 Minimum Distance Between BST Nodes ↗

Tree Traversal – Post-Order (1) ↗

- ☐ ● #145 Binary Tree Postorder Traversal ↗

Linked List (2) ↗

☐ ● #83 Remove Duplicates from Sorted List ↗

☐ ● #160 Intersection of Two Linked Lists ↗

Round 2 | High | M Medium (67) ↗

Arrays (5) ↗

☐ ● #80 Remove Duplicates from Sorted Array ↗
II

☐ ● #122 Best Time to Buy and Sell Stock II ↗

☐ ● #229 Majority Element II ↗

☐ ● #334 Increasing Triplet Subsequence ↗

☐ ● #2348 Number of Zero-Filled Subarrays ↗

Strings (4) ↗

☐ ● #6 Zigzag Conversion ↗

☐ ● #38 Count and Say ↗

☐ ● #151 Reverse Words in a String ↗

☐ ● #1657 Determine if Two Strings Are Close ↗

Hash Tables (6) ↗

☐ ● #535 Encode and Decode TinyURL ↗

- ☐ **● #659 Split Array into Consecutive Subsequences ↗**
- ☐ **● #767 Reorganize String ↗**
- ☐ **● #792 Number of Matching Subsequences ↗**
- ☐ **● #1525 Number of Good Ways to Split a String ↗**
- ☐ **● #1647 Minimum Deletions to Make Character Frequencies Unique ↗**

Two Pointers (3) ↗

- ☐ **● #18 4Sum ↗**
- ☐ **● #443 String Compression ↗**
- ☐ **● #861 Boats to Save People ↗**

Sliding Window – Fixed Size (2) ↗

- ☐ **● #1456 Maximum Number of Vowels in a Substring of Given Length ↗**
- ☐ **● #2461 Maximum Sum of Distinct Subarrays With Length K ↗**

Sliding Window – Dynamic Size (7) ↗

- ☐ **● #209 Minimum Size Subarray Sum ↗**

- ☐ ● #395 Longest Substring with At Least K Repeating Characters ↗
- ☐ ● #713 Subarray Product Less Than K ↗
- ☐ ● #904 Fruit Into Baskets ↗
- ☐ ● #1004 Max Consecutive Ones III ↗
- ☐ ● #1423 Maximum Points You Can Obtain from Cards ↗
- ☐ ● #1838 Frequency of the Most Frequent Element ↗

Binary Search (6) ↗

- ☐ ● #81 Search in Rotated Sorted Array II ↗
- ☐ ● #162 Find Peak Element ↗
- ☐ ● #240 Search a 2D Matrix II ↗
- ☐ ● #475 Heaters ↗
- ☐ ● #1482 Minimum Number of Days to Make m Bouquets ↗
- ☐ ● #1552 Magnetic Force Between Two Balls ↗

Stacks (6) ↗

- ☐ ● #71 Simplify Path ↗

- ☐ ● #316 Remove Duplicate Letters ↗
- ☐ ● #636 Exclusive Time of Functions ↗
- ☐ ● #946 Validate Stack Sequences ↗
- ☐ ● #1249 Minimum Remove to Make Valid Parentheses ↗
- ☐ ● #2390 Removing Stars From a String ↗
- Tree Traversal – Level Order (3) ↗**
- ☐ ● #116 Populating Next Right Pointers in Each Node ↗
- ☐ ● #117 Populating Next Right Pointers in Each Node II ↗
- ☐ ● #958 Check Completeness of a Binary Tree ↗
- Tree Traversal – Pre Order (3) ↗**
- ☐ ● #106 Construct Binary Tree from Inorder and Postorder Traversal ↗
- ☐ ● #687 Longest Univalue Path ↗
- ☐ ● #1026 Maximum Difference Between Node and Ancestor ↗
- Tree Traversal – In Order (1) ↗**
- ☐ ● #1382 Balance a Binary Search Tree ↗

Tree Traversal – Post-Order (6) ↗

- ☐ ● #114 Flatten Binary Tree to Linked List ↗
- ☐ ● #129 Sum Root to Leaf Numbers ↗
- ☐ ● #652 Find Duplicate Subtrees ↗
- ☐ ● #979 Distribute Coins in Binary Tree ↗
- ☐ ● #1110 Delete Nodes And Return Forest ↗
- ☐ ● #2096 Step-By-Step Directions From a Binary Tree Node to Another ↗

Depth First Search (DFS) (4) ↗

- ☐ ● #690 Employee Importance ↗
- ☐ ● #797 All Paths From Source to Target ↗
- ☐ ● #1020 Number of Enclaves ↗
- ☐ ● #1376 Time Needed to Inform All Employees ↗

Breadth First Search (BFS) (5) ↗

- ☐ ● #433 Minimum Genetic Mutation ↗
- ☐ ● #752 Open the Lock ↗
- ☐ ● #909 Snakes and Ladders ↗
- ☐ ● #1091 Shortest Path in Binary Matrix ↗

- ☐ **● #1102 Path With Maximum Minimum Value** ↗
Linked List (6) ↗
- ☐ **● #82 Remove Duplicates from Sorted List II** ↗
- ☐ **● #86 Partition List** ↗
- ☐ **● #237 Delete Node in a Linked List** ↗
- ☐ **● #445 Add Two Numbers II** ↗
- ☐ **● #1669 Merge In Between Linked Lists** ↗
- ☐ **● #2095 Delete the Middle Node of a Linked List** ↗

Round 3 | High | H Hard (11) ↗

Strings (1) ↗

- ☐ **● #843 Guess the Word** ↗

Two Pointers (1) ↗

- ☐ **● #2444 Count Subarrays With Fixed Bounds** ↗

Sliding Window – Fixed Size (1) ↗

- ☐ **● #30 Substring with Concatenation of All Words** ↗

Sliding Window – Dynamic Size (2) ↗

- ☐ **● #727 Minimum Window Subsequence** ↗

☐ ● #992 Subarrays with K Different Integers ↗

Binary Search (2) ↗

☐ ● #719 Find K-th Smallest Pair Distance ↗

☐ ● #1095 Find in Mountain Array ↗

Depth First Search (DFS) (2) ↗

☐ ● #827 Making A Large Island ↗

☐ ● #2246 Longest Path With Different Adjacent Characters ↗

Breadth First Search (BFS) (2) ↗

☐ ● #1293 Shortest Path in a Grid with Obstacles Elimination ↗

☐ ● #2290 Minimum Obstacle Removal to Reach Corner ↗

Round 4 | Mid | E Easy (20) ↗

Bit Manipulation (3) ↗

☐ ● #231 Power of Two ↗

☐ ● #461 Hamming Distance ↗

☐ ● #645 Set Mismatch ↗

Prefix Sum (4) ↗

☐ ● #303 Range Sum Query – Immutable ↗

☐ ● #724 Find Pivot Index ↗

☐ ● #1480 Running Sum of 1d Array ↗

☐ ● #1854 Maximum Population Year ↗

Monotonic Stack (2) ↗

☐ ● #496 Next Greater Element I ↗

☐ ● #1475 Final Prices With a Special Discount in a Shop ↗

Queues (2) ↗

☐ ● #933 Number of Recent Calls ↗

☐ ● #2073 Time Needed to Buy Tickets ↗

Divide and Conquer (1) ↗

☐ ● #1763 Longest Nice Substring ↗

BST / Ordered Set (3) ↗

☐ ● #653 Two Sum IV – Input is a BST ↗

☐ ● #700 Search in a Binary Search Tree ↗

☐ ● #938 Range Sum of BST ↗

Intervals (1) ↗

☐ ● #228 Summary Ranges ↗

Data Structure Design (2) ↗

☐ ● #225 Implement Stack using Queues ↗

☐ ● #359 Logger Rate Limiter ↗

Greedy (2) ↗

☐ ● #455 Assign Cookies ↗

☐ ● #3074 Apple Redistribution into Boxes ↗

Round 5 | Mid | M Medium (94) ↗

Bit Manipulation (3) ↗

☐ ● #137 Single Number II ↗

☐ ● #201 Bitwise AND of Numbers Range ↗

☐ ● #260 Single Number III ↗

Prefix Sum (6) ↗

☐ ● #304 Range Sum Query 2D – Immutable ↗

☐ ● #370 Range Addition ↗

☐ ● #523 Continuous Subarray Sum ↗

☐ ● #974 Subarray Sums Divisible by K ↗

☐ ● #1314 Matrix Block Sum ↗

☐ ● #2536 Increment Submatrices by One ↗

Kadane's Algorithm (5) ↗

- ☐ ● #918 Maximum Sum Circular Subarray ↗
- ☐ ● #978 Longest Turbulent Subarray ↗
- ☐ ● #1014 Best Sightseeing Pair ↗
- ☐ ● #1186 Maximum Subarray Sum with One Deletion ↗
- ☐ ● #1749 Maximum Absolute Sum of Any Subarray ↗

Matrix (2D Array) (1) ↗

- ☐ ● #289 Game of Life ↗

LinkedList In-place Reversal (1) ↗

- ☐ ● #92 Reverse Linked List II ↗

Monotonic Stack (9) ↗

- ☐ ● #402 Remove K Digits ↗
- ☐ ● #456 132 Pattern ↗
- ☐ ● #503 Next Greater Element II ↗
- ☐ ● #581 Shortest Unsorted Continuous Subarray ↗
- ☐ ● #769 Max Chunks To Make Sorted ↗

- ☐ ● #901 Online Stock Span ↗
- ☐ ● #907 Sum of Subarray Minimums ↗
- ☐ ● #1019 Next Greater Node In Linked List ↗
- ☐ ● #2104 Sum of Subarray Ranges ↗

Queues (3) ↗

- ☐ ● #950 Reveal Cards In Increasing Order ↗
- ☐ ● #1823 Find the Winner of the Circular Game ↗
- ☐ ● #2327 Number of People Aware of a Secret ↗

Monotonic Queue (5) ↗

- ☐ ● #1438 Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit ↗
- ☐ ● #1673 Find the Most Competitive Subsequence ↗
- ☐ ● #1696 Jump Game VI ↗
- ☐ ● #2762 Continuous Subarrays ↗
- ☐ ● #3578 Count Partitions With Max-Min Difference at Most K ↗

Divide and Conquer (3) ↗

☐ **● #109 Convert Sorted List to Binary Search Tree ↗**

☐ **● #427 Construct Quad Tree ↗**

☐ **● #654 Maximum Binary Tree ↗**

Merge Sort (1) ↗

☐ **● #912 Sort an Array ↗**

QuickSort / QuickSelect (1) ↗

☐ **● #1985 Find the Kth Largest Integer in the Array ↗**

Backtracking (4) ↗

☐ **● #47 Permutations II ↗**

☐ **● #77 Combinations ↗**

☐ **● #93 Restore IP Addresses ↗**

☐ **● #216 Combination Sum III ↗**

BST / Ordered Set (10) ↗

☐ **● #99 Recover Binary Search Tree ↗**

☐ **● #426 Convert Binary Search Tree to Sorted Doubly Linked List ↗**

☐ **● #450 Delete Node in a BST ↗**

- ☐ ● #510 Inorder Successor in BST II ↗
- ☐ ● #669 Trim a Binary Search Tree ↗
- ☐ ● #701 Insert into a Binary Search Tree ↗
- ☐ ● #729 My Calendar I ↗
- ☐ ● #731 My Calendar II ↗
- ☐ ● #1008 Construct Binary Search Tree from Preorder Traversal ↗
- ☐ ● #2034 Stock Price Fluctuation ↗
- Tries (6)** ↗
- ☐ ● #421 Maximum XOR of Two Numbers in an Array ↗
- ☐ ● #648 Replace Words ↗
- ☐ ● #1233 Remove Sub-Folders from the Filesystem ↗
- ☐ ● #1268 Search Suggestions System ↗
- ☐ ● #2452 Words Within Two Edits of Dictionary ↗
- ☐ ● #3043 Find the Length of the Longest Common Prefix ↗

Heaps (4) ↗

- ☐ ● #1405 Longest Happy String ↗
- ☐ ● #1642 Furthest Building You Can Reach ↗
- ☐ ● #1834 Single-Threaded CPU ↗
- ☐ ● #1882 Process Tasks Using Servers ↗

Intervals (6) ↗

- ☐ ● #452 Minimum Number of Arrows to Burst Balloons ↗
- ☐ ● #1094 Car Pooling ↗
- ☐ ● #1229 Meeting Scheduler ↗
- ☐ ● #1288 Remove Covered Intervals ↗
- ☐ ● #1353 Maximum Number of Events That Can Be Attended ↗
- ☐ ● #2054 Two Best Non-Overlapping Events ↗

K-Way Merge (2) ↗

- ☐ ● #373 Find K Pairs with Smallest Sums ↗
- ☐ ● #378 Kth Smallest Element in a Sorted Matrix ↗

Data Structure Design (4) ↗

☐ ● #641 Design Circular Deque ↗

☐ ● #1146 Snapshot Array ↗

☐ ● #1472 Design Browser History ↗

☐ ● #2353 Design a Food Rating System ↗

Greedy (8) ↗

☐ ● #406 Queue Reconstruction by Height ↗

☐ ● #670 Maximum Swap ↗

☐ ● #826 Most Profit Assigning Work ↗

☐ ● #921 Minimum Add to Make Parentheses Valid ↗

☐ ● #1488 Avoid Flood in The City ↗

☐ ● #1717 Maximum Score From Removing Substrings ↗

☐ ● #1871 Jump Game VII ↗

☐ ● #1975 Maximum Matrix Sum ↗

Topological Sort (3) ↗

☐ ● #802 Find Eventual Safe States ↗

☐ ● #1462 Course Schedule IV ↗

☐ ● #2115 Find All Possible Recipes from Given Supplies ↗

Union Find (3) ↗

☐ ● #399 Evaluate Division ↗

☐ ● #547 Number of Provinces ↗

☐ ● #947 Most Stones Removed with Same Row or Column ↗

Minimum Spanning Tree (1) ↗

☐ ● #1135 Connecting Cities With Minimum Cost ↗

Shortest Path (5) ↗

☐ ● #1514 Path with Maximum Probability ↗

☐ ● #1631 Path With Minimum Effort ↗

☐ ● #2976 Minimum Cost to Convert String I ↗

☐ ● #3341 Find Minimum Time to Reach Last Room I ↗

☐ ● #3650 Minimum Cost Path with Edge Reversals ↗

Round 6 | Mid | H Hard (30) ↗

Monotonic Stack (2) ↗

- ☐ ● #321 Create Maximum Number ↗
- ☐ ● #1944 Number of Visible People in a Queue ↗
Monotonic Queue (2) ↗
- ☐ ● #862 Shortest Subarray with Sum at Least K ↗
- ☐ ● #1499 Max Value of Equation ↗
Recursion (2) ↗
- ☐ ● #273 Integer to English Words ↗
- ☐ ● #761 Special Binary String ↗
Merge Sort (2) ↗
- ☐ ● #327 Count of Range Sum ↗
- ☐ ● #493 Reverse Pairs ↗
Backtracking (2) ↗
- ☐ ● #301 Remove Invalid Parentheses ↗
- ☐ ● #980 Unique Paths III ↗
Tries (3) ↗
- ☐ ● #425 Word Squares ↗
- ☐ ● #472 Concatenated Words ↗
- ☐ ● #1032 Stream of Characters ↗

Heaps (1) ↗

- ☐ ● #1383 Maximum Performance of a Team ↗

Two Heaps (2) ↗

- ☐ ● #480 Sliding Window Median ↗
- ☐ ● #502 IPO ↗

Intervals (1) ↗

- ☐ ● #2402 Meeting Rooms III ↗

Data Structure Design (2) ↗

- ☐ ● #715 Range Module ↗
- ☐ ● #2296 Design a Text Editor ↗

Greedy (4) ↗

- ☐ ● #135 Candy ↗
- ☐ ● #757 Set Intersection Size At Least Two ↗
- ☐ ● #857 Minimum Cost to Hire K Workers ↗
- ☐ ● #871 Minimum Number of Refueling Stops ↗

Topological Sort (1) ↗

- ☐ ● #1203 Sort Items by Groups Respecting Dependencies

Union Find (1) ↗

☐ ● #924 Minimize Malware Spread ↗

Minimum Spanning Tree (3) ↗

☐ ● #1168 Optimize Water Distribution in a Village ↗

☐ ● #1489 Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree ↗

☐ ● #3600 Maximize Spanning Tree Stability with Upgrades ↗

Shortest Path (2) ↗

☐ ● #1368 Minimum Cost to Make at Least One Valid Path in a Grid ↗

☐ ● #2203 Minimum Weighted Subgraph With the Required Paths ↗

Round 7 | Low | E Easy (6) ↗

1-D DP (1) ↗

☐ ● #509 Fibonacci Number ↗

2D Grid DP (1) ↗

☐ ● #118 Pascal's Triangle ↗

Maths / Geometry (3) ↗

☐ ● #168 Excel Sheet Column Title ↗

☐ ● #263 Ugly Number ↗

☐ ● #1071 Greatest Common Divisor of Strings ↗

String Matching (1) ↗

☐ ● #459 Repeated Substring Pattern ↗

Round 8 | Low | M Medium (42) ↗

Bucket Sort (2) ↗

☐ ● #164 Maximum Gap ↗

☐ ● #451 Sort Characters By Frequency ↗

1-D DP (4) ↗

☐ ● #343 Integer Break ↗

☐ ● #646 Maximum Length of Pair Chain ↗

☐ ● #740 Delete and Earn ↗

☐ ● #1262 Greatest Sum Divisible by Three ↗

0/1 Knapsack (2) ↗

☐ ● #474 Ones and Zeroes ↗

☐ ● #1049 Last Stone Weight II ↗

Unbounded Knapsack (2) ↗

☐ ● #279 Perfect Squares ↗

☐ ● #983 Minimum Cost For Tickets ↗

Longest Increasing Subsequence (LIS) (3) ↗

☐ ● #673 Number of Longest Increasing Subsequence ↗

☐ ● #1027 Longest Arithmetic Subsequence ↗

☐ ● #1048 Longest String Chain ↗

2D Grid DP (5) ↗

☐ ● #63 Unique Paths II ↗

☐ ● #120 Triangle ↗

☐ ● #931 Minimum Falling Path Sum ↗

☐ ● #1277 Count Square Submatrices with All Ones ↗

☐ ● #1937 Maximum Number of Points with Cost ↗

String DP (1) ↗

☐ ● #1653 Minimum Deletions to Make String Balanced ↗

Tree / Graph DP (3) ↗

☐ ● #95 Unique Binary Search Trees II ↗

☐ ● #337 House Robber III ↗

☐ ● #1976 Number of Ways to Arrive at Destination ↗

Bitmask DP (4) ↗

☐ ● #698 Partition to K Equal Sum Subsets ↗

☐ ● #1066 Campus Bikes II ↗

☐ ● #1986 Minimum Number of Work Sessions to Finish the Tasks ↗

☐ ● #2305 Fair Distribution of Cookies ↗

Digit DP (1) ↗

☐ ● #357 Count Numbers with Unique Digits ↗

Probability DP (3) ↗

☐ ● #688 Knight Probability in Chessboard ↗

☐ ● #808 Soup Servings ↗

☐ ● #837 New 21 Game ↗

State Machine DP (1) ↗

☐ ● #714 Best Time to Buy and Sell Stock with Transaction Fee ↗

Maths / Geometry (8) ↗

☐ ● #172 Factorial Trailing Zeroes ↗

☐ ● #204 Count Primes ↗

☐ ● #264 Ugly Number II ↗

☐ ● #593 Valid Square ↗

☐ ● #611 Valid Triangle Number ↗

☐ ● #939 Minimum Area Rectangle ↗

☐ ● #963 Minimum Area Rectangle II ↗

☐ ● #1922 Count Good Numbers ↗

String Matching (2) ↗

☐ ● #686 Repeated String Match ↗

☐ ● #1698 Number of Distinct Substrings in a String ↗

Binary Indexed Tree / Segment Tree (1) ↗

☐ ● #308 Range Sum Query 2D – Mutable ↗

Round 9 | Low | H Hard (36) ↗

Bucket Sort (1) ↗

☐ ● #220 Contains Duplicate III ↗

Eulerian Circuit (2) ↗

☐ ● #753 Cracking the Safe ↗

- ☐ ● #2097 Valid Arrangement of Pairs ↗
1-D DP (1) ↗
- ☐ ● #1411 Number of Ways to Paint $N \times 3$ Grid ↗
0/1 Knapsack (1) ↗
- ☐ ● #879 Profitable Schemes ↗
Longest Increasing Subsequence (LIS) (1) ↗
- ☐ ● #354 Russian Doll Envelopes ↗
2D Grid DP (10) ↗
- ☐ ● #85 Maximal Rectangle ↗
- ☐ ● #174 Dungeon Game ↗
- ☐ ● #403 Frog Jump ↗
- ☐ ● #465 Optimal Account Balancing ↗
- ☐ ● #546 Remove Boxes ↗
- ☐ ● #741 Cherry Pickup ↗
- ☐ ● #1335 Minimum Difficulty of a Job Schedule ↗
- ☐ ● #1547 Minimum Cost to Cut a Stick ↗
- ☐ ● #1931 Painting a Grid With Three Different Colors ↗

☐ ● #3651 Minimum Cost Path with Teleportations ↗

String DP (4) ↗

☐ ● #44 Wildcard Matching ↗

☐ ● #140 Word Break II ↗

☐ ● #664 Strange Printer ↗

☐ ● #1092 Shortest Common Supersequence ↗

Tree / Graph DP (2) ↗

☐ ● #834 Sum of Distances in Tree ↗

☐ ● #968 Binary Tree Cameras ↗

Bitmask DP (1) ↗

☐ ● #847 Shortest Path Visiting All Nodes ↗

Digit DP (2) ↗

☐ ● #233 Number of Digit One ↗

☐ ● #902 Numbers At Most N Given Digit Set ↗

State Machine DP (1) ↗

☐ ● #188 Best Time to Buy and Sell Stock IV ↗

Maths / Geometry (1) ↗

☐ ● #149 Max Points on a Line ↗

String Matching (3) ↗

☐ ● #214 Shortest Palindrome ↗

☐ ● #1044 Longest Duplicate Substring ↗

☐ ● #1392 Longest Happy Prefix ↗

Binary Indexed Tree / Segment Tree (4) ↗

☐ ● #699 Falling Squares ↗

☐ ● #2426 Number of Pairs Satisfying
Inequality

☐ ● #3161 Block Placement Queries ↗

☐ ● #3721 Longest Balanced Subarray II ↗

Line Sweep (2) ↗

☐ ● #391 Perfect Rectangle ↗

☐ ● #850 Rectangle Area II ↗

Round 1 · High · Easy

Round 1

High Priority · Easy

38 questions · 12 patterns

Arrays #26 #27 #414 #485 #1470

Strings #28 #58 #344 #392 #680 #796

**Hash Tables #205 #219 #290 #350 #387 #448
#1189 #1512**

Two Pointers #696 #1768

Sliding Window – Fixed Size #643

Binary Search #367 #744

Stacks #682 #1047 #1614

Tree Traversal – Level Order #637

**Tree Traversal – Pre Order #144 #222 #257
#404**

Tree Traversal – In Order #94 #530 #783

Tree Traversal – Post-Order #145

Linked List #83 #160

Arrays

Round 1 · High · Easy · 5 questions

Arrays

□ #26 Remove Duplicates from Sorted Array

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers
Lists: AlgoMaster 600**

Problem

Given an integer array `nums` sorted in non-decreasing order, remove the duplicates in-place such that each unique element appears only once. The relative order of the elements should be kept the same.

Consider the number of unique elements in `nums` to be `k` . After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in sorted order. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be accepted.

Example 1:

Input: nums = [1,1,2]
Output: 2, nums = [1,2,_]
Explanation: Your function should return k = 2, with the first two elements of nums being 1 and 2 respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).

Example 2:

Input: nums = [0,0,1,1,1,2,2,3,3,4]
Output: 5, nums = [0,1,2,3,4,_,_,_,_,_]
Explanation: Your function should return k = 5, with the first five elements of nums being 0, 1, 2, 3, and 4 respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-100 \leq \text{nums}[i] \leq 100$
- nums is sorted in non-decreasing order.

Community Solutions (Python)

- WalkCC


```
class Solution:
    def removeDuplicates(self, nums: list[int]) -> int:
        i = 0

        for num in nums:
            if i < 1 or num > nums[i - 1]:
                nums[i] = num
                i += 1

        return i
```

• LeetCode

```
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        k = 0
        for x in nums:
            if k == 0 or x != nums[k - 1]:
                nums[k] = x
                k += 1
        return k
```

• SimplyLeet

```
# Function to remove duplicates and return count of unique elements.
def removeDuplicates(nums):
    if not nums:
        return 0
    unique_index = 1 # Pointer for the next unique element.
    # Iterate over the array starting from the second element.
    for i in range(1, len(nums)):
        # If the current element is different from the previous.
        if nums[i] != nums[i - 1]:
            nums[unique_index] = nums[i] # Place the unique element at the
            unique_index.
```

```
        unique_index += 1
    return unique_index

# Example usage:
# nums = [0,0,1,1,1,2,2,3,3,4]
# k = removeDuplicates(nums)
# print(k, nums[:k])
```

◆ Quick Review

Key Insights

- The array is sorted in non-decreasing order, meaning duplicate values appear consecutively.
- Use a two-pointer approach:
- One pointer to iterate through the array.
- Another pointer to track the position to place the next unique element.
- The solution modifies the array in place with $O(1)$ extra space.

Space & Time

Time Complexity: $O(n)$, where n is the length of the input array.

Space Complexity: $O(1)$ since the modification is done in-place.

Solution

The approach uses two pointers. The first pointer (`uniqueIndex`) tracks the position for the next unique element, starting from index 1. The second pointer iterates over the array from index 1 to the end. Whenever a new element (different from the previous element) is encountered, it is placed at `uniqueIndex`, and `uniqueIndex` is incremented. Finally, `uniqueIndex` represents the count of unique elements and denotes how many elements are valid in the modified array.

Arrays

□ #27 Remove Element

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers
Lists: AlgoMaster 600**

Problem

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` in-place. The order of the elements may be changed. Then return the number of elements in `nums` which are not equal to `val`.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

- **Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.**
- **Return `k`.**

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
// It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
```

If all assertions pass, then your solution will be accepted.

Example 1:

```
Input: nums = [3,2,2,3], val = 3
Output: 2, nums = [2,2,_,_]
Explanation: Your function should return k = 2, with the first two elements of
nums being 2.
It does not matter what you leave beyond the returned k (hence they are
underscores).
```

Example 2:

```
Input: nums = [0,1,2,2,3,0,4,2], val = 2
Output: 5, nums = [0,1,4,0,3,_,_,_]
Explanation: Your function should return k = 5, with the first five elements of
nums containing 0, 0, 1, 3, and 4.
Note that the five elements can be returned in any order.
It does not matter what you leave beyond the returned k (hence they are
underscores).
```

Constraints:

- $0 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 50$
- $0 \leq \text{val} \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeElement(self, nums: list[int], val: int) -> int:
        i = 0

        for num in nums:
            if num != val:
                nums[i] = num
                i += 1

        return i
```

• LeetCode

```
class Solution:
    def removeElement(self, nums: List[int], val: int) -> int:
        k = 0
        for x in nums:
            if x != val:
                nums[k] = x
                k += 1
        return k
```

• SimplyLeet

```
def removeElement(nums, val):
    # Pointer for the position of the next valid element
    valid_index = 0
    # Iterate through each element in the array
    for num in nums:
        # If the current element is not equal to val, it is valid
        if num != val:
            # Write it to the position indicated by valid_index
            nums[valid_index] = num
            # Move the valid_index to the next position
```

```
        valid_index += 1
    # Return the count of valid elements
    return valid_index

# Example usage
nums = [3,2,2,3]
val = 3
k = removeElement(nums, val)
print(k, nums[:k])
```

◆ Quick Review

Key Insights

- Use a two–pointer approach: one pointer iterates through the array while the other keeps track of the position for the next valid element.
- Modify the array in–place, which ensures constant extra space usage.
- The order of elements is not preserved, which allows swapping elements or simply overwriting the unwanted ones.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The idea is to maintain a write pointer that indicates where the next valid element (not equal to val) should be placed. Iterate through the array with a read pointer. For every element that is not equal to val, write it to the position indicated by the write pointer and increment the write pointer. Once the loop completes, the write pointer will represent the number of valid elements (k) and the array's first k elements will be the desired result.

Arrays

□ #414 Third Maximum Number

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting

Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the third distinct maximum number in this array. If the third maximum does not exist, return the maximum number.

Example 1:

```
Input: nums = [3,2,1]
Output: 1
Explanation:
The first distinct maximum is 3.
The second distinct maximum is 2.
The third distinct maximum is 1.
```

Example 2:

```
Input: nums = [1,2]
Output: 2
Explanation:
The first distinct maximum is 2.
The second distinct maximum is 1.
The third distinct maximum does not exist, so the maximum (2) is returned instead.
```

Example 3:

Input: nums = [2,2,3,1]

Output: 1

Explanation:

The first distinct maximum is 3.

The second distinct maximum is 2 (both 2's are counted together since they have the same value).

The third distinct maximum is 1.

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-231 \leq \text{nums}[i] \leq 231 - 1$

Follow up: Can you find an $O(n)$ solution?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def thirdMax(self, nums: list[int]) -> int:
        max1 = -math.inf # the maximum
        max2 = -math.inf # the second maximum
        max3 = -math.inf # the third maximum

        for num in nums:
            if num > max1:
                max3 = max2
                max2 = max1

                max1 = num
            elif max1 > num and num > max2:
                max3 = max2
                max2 = num
            elif max2 > num and num > max3:
                max3 = num

        return max1 if max3 == -math.inf else max3
```

```

class Solution:
    def thirdMax(self, nums: list[int]) -> int:
        minHeap = []
        seen = set()

        for num in nums:
            if num not in seen:
                seen.add(num)
                heapq.heappush(minHeap, num)
                if len(minHeap) > 3:

                    heapq.heappop(minHeap)

        if len(minHeap) == 2:
            heapq.heappop(minHeap)

        return minHeap[0]

```

• LeetCode

```

class Solution:
    def thirdMax(self, nums: List[int]) -> int:
        m1 = m2 = m3 = -inf
        for num in nums:
            if num in [m1, m2, m3]:
                continue
            if num > m1:
                m3, m2, m1 = m2, m1, num
            elif num > m2:
                m3, m2 = m2, num
            elif num > m3:
                m3 = num
        return m3 if m3 != -inf else m1

```

• SimplyLeet

```
# Define function thirdMax that returns the third distinct maximum number in
the array.
def thirdMax(nums):
    # Initialize three maximum variables to None to represent their absence.
    first = second = third = None

    # Iterate over each number in the nums array.
    for num in nums:
        # Skip the current number if it is already one of the three maximums.
        if num == first or num == second or num == third:
            continue

        # Update the maximums based on the current number's value.
        if first is None or num > first:
            # Shift down the values: first becomes second, second becomes
third.
            third, second, first = second, first, num
        elif second is None or num > second:
            # Update second and shift second to third.
            third, second = second, num
        elif third is None or num > third:
            # Update third maximum.

            third = num

    # If there is no third distinct maximum, return the first maximum.
    return first if third is None else third

# Example usage:
print(thirdMax([3,2,1])) # Output: 1
print(thirdMax([1,2]))  # Output: 2
print(thirdMax([2,2,3,1])) # Output: 1
```

◆ Quick Review

Key Insights

- We need to consider only distinct values in the array.

- Use variables or a data structure to track the top 3 distinct maximum numbers.
- We need an $O(n)$ solution, so we should avoid unnecessary sorting.
- Edge-case: If there are fewer than 3 distinct numbers, the maximum among them should be returned.

Space & Time

Time Complexity: $O(n)$ because we iterate through the array once.

Space Complexity: $O(1)$ since we only maintain a fixed number of variables.

Solution

We iterate through the `nums` array and maintain three variables to store the first, second, and third distinct maximum numbers. For each number in the array, we check whether it is already one of the three maximums (to handle duplicates). If not, we update the maximums accordingly:

- If the current number is greater than the first maximum, then update the first, second, and third maximums.

- Else if the current number is between the first maximum and the second, update the second and third maximums accordingly.
- Else if the current number is between the second maximum and the third, update the third maximum.

After processing all elements, if the third maximum is still unassigned (i.e., there are less than three distinct numbers), return the first maximum. This approach uses constant extra space and only one pass through the input array.

Arrays

□ #485 Max Consecutive Ones

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array

Lists: AlgoMaster 600

Problem

Given a binary array `nums`, return the maximum number of consecutive 1's in the array.

Example 1:

Input: `nums = [1,1,0,1,1,1]`

Output: 3

Explanation: The first two digits or the last three digits are consecutive 1s.
The maximum number of consecutive 1s is 3.

Example 2:

Input: `nums = [1,0,1,1,0,1]`

Output: 2

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- `nums[i]` is either 0 or 1.

Community Solutions (Python)

- • WalkCC

```
class Solution:
    def findMaxConsecutiveOnes(self, nums: list[int]) -> int:
        ans = 0
        summ = 0

        for num in nums:
            if num == 0:
                summ = 0
            else:
                summ += num

        ans = max(ans, summ)

        return ans
```

• LeetCode

```
class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        ans = cnt = 0
        for x in nums:
            if x:
                cnt += 1
                ans = max(ans, cnt)
            else:
                cnt = 0
        return ans
```

• SimplyLeet

```
# Function to return maximum consecutive 1s in an array
def findMaxConsecutiveOnes(nums):
    # Initialize counters for consecutive ones
    max_consecutive = 0
    current_count = 0
    # Iterate through each number in the list
    for num in nums:
        if num == 1:
            # Increase the count when the current number is 1
            current_count += 1
```



```
        # Update maximum if the current count is higher
        max_consecutive = max(max_consecutive, current_count)
    else:
        # Reset current count when a 0 is encountered
        current_count = 0
    return max_consecutive

# Example usage:
if __name__ == "__main__":
    example_nums = [1, 1, 0, 1, 1, 1]

    print(findMaxConsecutiveOnes(example_nums)) # Expected output: 3
```

◆ Quick Review

Key Insights

- Iterate over the array while keeping track of the current sequence length of 1's.
- Reset the counter when a 0 is encountered.
- Update the maximum value during the iteration.
- This direct scanning results in a simple $O(n)$ solution with constant space.

Space & Time

Time Complexity: $O(n)$ – We traverse the array once.

Space Complexity: $O(1)$ – Only constant extra variables are used.

Solution

We iterate through the binary array and maintain two counters: one for the current sequence of consecutive 1's (currentCount) and one for the maximum sequence found so far (maxConsecutive). As we scan the array, we increment currentCount when we see a 1 and update maxConsecutive. When a 0 is found, we reset currentCount to 0. This approach leverages a simple linear pass through the array and uses a constant amount of extra space.

Arrays

□ #1470 Shuffle the Array

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array

Lists: AlgoMaster 600

Problem

Given the array `nums` consisting of $2n$ elements in the form `[x1,x2,...,xn,y1,y2,...,yn]`.

Return the array in the form `[x1,y1,x2,y2,...,xn,yn]`.

Example 1:

Input: `nums = [2,5,1,3,4,7]`, `n = 3`

Output: `[2,3,5,4,1,7]`

Explanation: Since `x1=2`, `x2=5`, `x3=1`, `y1=3`, `y2=4`, `y3=7` then the answer is `[2,3,5,4,1,7]`.

Example 2:

Input: `nums = [1,2,3,4,4,3,2,1]`, `n = 4`

Output: `[1,4,2,3,3,2,4,1]`

Example 3:

Input: `nums = [1,1,2,2]`, `n = 2`

Output: `[1,2,1,2]`

Constraints:

- $1 \leq n \leq 500$
- `nums.length == 2n`
- $1 \leq \text{nums}[i] \leq 10^3$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def shuffle(self, nums: list[int], n: int) -> list[int]:
        ans = []
        for a, b in zip(nums[:n], nums[n:]):
            ans.append(a)
            ans.append(b)
        return ans
```

• LeetDoocs

```
class Solution:
    def shuffle(self, nums: List[int], n: int) -> List[int]:
        return [x for pair in zip(nums[:n], nums[n:]) for x in pair]
```

• SimplyLeet

```
def shuffle(nums, n):
    # Create an empty list to store the result
    result = []
    # Loop through the first half of the array
    for i in range(n):
        # Append the element from the first half
        result.append(nums[i])
        # Append the corresponding element from the second half
        result.append(nums[i + n])
    return result
```

```
# Example usage:  
# nums = [2,5,1,3,4,7]  
# print(shuffle(nums, 3))
```

◆ Quick Review

Key Insights

- The array is split into two halves: the first n elements and the last n elements.
- Each element from the first half pairs with the corresponding element in the second half.
- A single iteration from 0 to $n-1$ is sufficient to interleave the elements correctly.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution involves iterating over the first half of the array and interleaving elements from both halves. Create an empty result array and for each index i from 0 to $n-1$, add $\text{nums}[i]$ from the first half and $\text{nums}[i+n]$ from the second half to the result array. This approach leverages a simple loop ensuring a linear time

complexity while using extra space to store the final array.

Strings

Round 1 · High · Easy · 6 questions

Strings

□ #28 Find the Index of the First Occurrence in a String

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String · String Matching
Lists: AlgoMaster 600

Problem

Given two strings *needle* and *haystack*, return the index of the first occurrence of *needle* in *haystack*, or -1 if *needle* is not part of *haystack*.

Example 1:

Input: *haystack* = "sadbutsad", *needle* = "sad"
Output: 0
Explanation: "sad" occurs at index 0 and 6.
The first occurrence is at index 0, so we return 0.

Example 2:

Input: *haystack* = "leetcode", *needle* = "leeto"
Output: -1
Explanation: "leeto" did not occur in "leetcode", so we return -1.

Constraints:

- $1 \leq \text{haystack.length}, \text{needle.length} \leq 104$
- *haystack* and *needle* consist of only lowercase English characters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:
        m = len(haystack)
        n = len(needle)

        for i in range(m - n + 1):
            if haystack[i:i + n] == needle:
                return i

        return -1
```

• LeetDoocs

```
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:
        n, m = len(haystack), len(needle)
        for i in range(n - m + 1):
            if haystack[i : i + m] == needle:
                return i
        return -1
```

• SimplyLeet

```
# Function to find the first occurrence of needle in haystack
def strStr(haystack: str, needle: str) -> int:
    haystack_length = len(haystack)
    needle_length = len(needle)
    # Edge: if needle is longer than haystack, it cannot be a substring
    if needle_length > haystack_length:
        return -1

    # Loop through each index in haystack where needle could fit
    for i in range(haystack_length - needle_length + 1):
```

```
# Check if the substring of haystack starting at i matches needle
match = True
for j in range(needle_length):
    if haystack[i + j] != needle[j]:
        match = False
        break # break early if mismatch found
if match:
    return i # return the starting index of the match
return -1 # return -1 if needle is not found in haystack
```

```
# Example usage:
# print(strStr("sadbutsad", "sad")) # Expected output: 0
```

◆ Quick Review

Key Insights

- Search for a substring (needle) within a larger string (haystack).
- A straightforward approach is to iterate and compare potential starting positions.
- The naive approach checks each possible starting index in haystack, while more advanced techniques (like KMP) can improve efficiency, but the naive approach is often sufficient given the constraints.
- Special edge-case: if needle is an empty string (as typically defined in some similar problems) but here constraints guarantee positive length.

Space & Time

Time Complexity: $O(n * m)$ in the worst case, where n is the length of haystack and m is the length of needle.

Space Complexity: $O(1)$ as no additional space is used.

Solution

The solution uses a brute force method with two pointers. We iterate over each possible starting position in haystack and check if the substring starting there matches the needle. If we find a match, we return the current index as the first occurrence. If no match is found after checking all possible positions, we return -1 . This approach is simple and directly implements the problem's requirement.

Strings

□ #58 Length of Last Word

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String

Lists: AlgoMaster 600

Problem

Given a string *s* consisting of words and spaces, return the length of the last word in the string.

A word is a maximal substring consisting of non-space characters only.

Example 1:

Input: *s* = "Hello World"

Output: 5

Explanation: The last word is "World" with length 5.

Example 2:

Input: *s* = " fly me to the moon "

Output: 4

Explanation: The last word is "moon" with length 4.

Example 3:

Input: *s* = "luffy is still joyboy"

Output: 6

Explanation: The last word is "joyboy" with length 6.

Constraints:

- $1 \leq s.length \leq 104$

- s consists of only English letters and spaces ' '.
- There will be at least one word in s.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lengthOfLastWord(self, s: str) -> int:
        i = len(s) - 1

        while i >= 0 and s[i] == ' ':
            i -= 1
        lastIndex = i
        while i >= 0 and s[i] != ' ':
            i -= 1

        return lastIndex - i
```

• LeetDoocs

```
class Solution:
    def lengthOfLastWord(self, s: str) -> int:
        i = len(s) - 1
        while i >= 0 and s[i] == ' ':
            i -= 1
        j = i
        while j >= 0 and s[j] != ' ':
            j -= 1
        return i - j
```

• SimplyLeet

```
# Function to compute the length of the last word in a string
def length_of_last_word(s):
    # Remove trailing spaces from the string
    s = s.rstrip()
    # Find the index of the last space in the string
    last_space_index = s.rfind(' ')
    # The length of the last word is the total length minus the position of
    last space - 1
    return len(s) - last_space_index - 1

# Test example

print(length_of_last_word("Hello World")) # Expected output: 5
```

◆ Quick Review

Key Insights

- Remove any trailing spaces to handle edge cases.
- Find the last space in the string to isolate the last word.
- The length of the last word is the difference between the total string length (after trimming) and the position of the last space.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string (single pass to trim and another pass to count last word characters).

Space Complexity: $O(1)$, using constant extra space.

Solution

We first trim the string to remove trailing spaces which may cause incorrect identification of the last word. Then, we locate the last space character in the trimmed string. The length of the last word is calculated as the difference between the total string length and the index of the last space minus one. This approach uses basic string operations and requires constant space while processing the string in linear time.

Strings

□ #344 Reverse String

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String
Lists: AlgoMaster 600

Problem

Write a function that reverses a string. The input string is given as an array of characters `s`.

You must do this by modifying the input array in-place with $O(1)$ extra memory.

Example 1:

```
Input: s = ["h","e","l","l","o"]  
Output: ["o","l","l","e","h"]
```

Example 2:

```
Input: s = ["H","a","n","n","a","h"]  
Output: ["h","a","n","n","a","H"]
```

Constraints:

- $1 \leq s.length \leq 105$
- `s[i]` is a printable ascii character.

Community Solutions (Python)

- • WalkCC


```
class Solution:
    def reverseString(self, s: list[str]) -> None:
        l = 0
        r = len(s) - 1

        while l < r:
            s[l], s[r] = s[r], s[l]
            l += 1
            r -= 1
```

• LeetCode

```
class Solution:
    def reverseString(self, s: List[str]) -> None:
        i, j = 0, len(s) - 1
        while i < j:
            s[i], s[j] = s[j], s[i]
            i, j = i + 1, j - 1
```

• SimplyLeet

```
def reverseString(s):
    # Initialize two pointers: one at the beginning (left) and one at the end (right)
    left, right = 0, len(s) - 1
    # Swap characters until the two pointers meet or cross
    while left < right:
        s[left], s[right] = s[right], s[left] # Swap the characters at the left and right positions
        left += 1 # Move the left pointer to the right
        right -= 1 # Move the right pointer to the left
```

◆ Quick Review

Key Insights

- Use the two–pointer technique, with one pointer starting at the beginning and the other at the end of the array.

- Swap the elements at these pointers and move them towards the center.
- Stop when the pointers cross (or meet in the middle).

Space & Time

Time Complexity: $O(n)$, where n is the number of characters in the array.

Space Complexity: $O(1)$ since only a constant amount of extra memory is used.

Solution

The solution uses the two-pointer approach to reverse the string in-place. By initializing one pointer at the start and one at the end of the array, we repeatedly swap the characters at these positions and then move them towards the center. This continues until the pointers meet or cross, ensuring the string is completely reversed without using additional space.

Strings

□ #392 Is Subsequence

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String · Dynamic Programming
Lists: AlgoMaster 600

Problem

Given two strings *s* and *t*, return true if *s* is a subsequence of *t*, or false otherwise.

A subsequence of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

Example 1:

Input: *s* = "abc", *t* = "ahbgdc"
Output: true

Example 2:

Input: *s* = "axc", *t* = "ahbgdc"
Output: false

Constraints:

- $0 \leq s.length \leq 100$

- $0 \leq t.length \leq 104$
- s and t consist only of lowercase English letters.

Follow up: Suppose there are lots of incoming s , say $s_1, s_2, \dots, s_k$ where $k \geq 109$, and you want to check one by one to see if t has its subsequence. In this scenario, how would you change your code?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isSubsequence(self, s: str, t: str) -> bool:
        if not s:
            return True

        i = 0
        for c in t:
            if s[i] == c:
                i += 1
            if i == len(s):
                return True

        return False
```

• LeetDoocs

```
class Solution:
    def isSubsequence(self, s: str, t: str) -> bool:
        i = j = 0
        while i < len(s) and j < len(t):
            if s[i] == t[j]:
                i += 1
            j += 1
        return i == len(s)
```

• SimplyLeet

Python solution with two pointers

```
def isSubsequence(s, t):
    # Initialize pointer for s
    s_index = 0
    # Iterate over each character in t
    for char in t:
        # If the pointer hasn't reached the end of s and the character matches
        if s_index < len(s) and s[s_index] == char:
            s_index += 1 # move pointer in s to next character

    # If all characters in s are found, return True
    if s_index == len(s):
        return True

    # Return whether the pointer reached the end of s after traversing t
    return s_index == len(s)

# Example usage:
print(isSubsequence("abc", "ahbgdc")) # Output: True
print(isSubsequence("axc", "ahbgdc")) # Output: False
```

◆ Quick Review

Key Insights

- Use a two–pointer approach: one pointer traversing s and one traversing t.

- As you iterate through t , if the current character matches the current character in s , move the pointer in s .
- If the pointer in s reaches the end, all characters were matched in order, so s is a subsequence of t .
- In the follow-up scenario with many s strings and one t , preprocess t by mapping each character to its indices, enabling binary search for the next occurrence in t for each character of s .

Space & Time

Time Complexity: $O(n + m)$, where n is the length of t and m is the length of s .

Space Complexity: $O(1)$ for the simple two-pointer approach. The preprocessing method (follow-up) requires extra space for the mapping, i.e., $O(n)$.

Solution

We use a two-pointer technique to solve the problem. One pointer goes through the string s while the other goes through t . Every time a matching character is found, move the pointer in s . If by the end of t the pointer in s has

reached the end, then s is a subsequence of t . For the follow-up scenario with many s queries, pre-process t by creating a mapping from each character to a list of its positions in t . Then for each s , use binary search to quickly find a valid next position for each character.

Strings

□ #680 Valid Palindrome II

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String · Greedy
Lists: AlgoMaster 600

Problem

Given a string *s*, return true if the *s* can be palindrome after deleting at most one character from it.

Example 1:

Input: *s* = "aba"
Output: true

Example 2:

Input: *s* = "abca"
Output: true
Explanation: You could delete the character 'c'.

Example 3:

Input: *s* = "abc"
Output: false

Constraints:

- $1 \leq s.length \leq 105$
- *s* consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def validPalindrome(self, s: str) -> bool:
        def validPalindrome(l: int, r: int) -> bool:
            return all(s[i] == s[r - i + l] for i in range(l, (l + r) // 2 + 1))

        n = len(s)

        for i in range(n // 2):
            if s[i] != s[~i]:
                return validPalindrome(i + 1, n - 1 - i) or validPalindrome(i, n - 2 - i)

        return True
```

• LeetDoocs

```
class Solution:
    def validPalindrome(self, s: str) -> bool:
        def check(i, j):
            while i < j:
                if s[i] != s[j]:
                    return False
                i, j = i + 1, j - 1
            return True

        i, j = 0, len(s) - 1

        while i < j:
            if s[i] != s[j]:
                return check(i, j - 1) or check(i + 1, j)
            i, j = i + 1, j - 1
        return True
```

• SimplyLeet

```

# Define the Solution class
class Solution:
    def validPalindrome(self, s: str) -> bool:
        # Helper function to check if the substring s[left:right+1] is a
        # palindrome.
        def isPalindromeRange(left, right):
            while left < right:
                if s[left] != s[right]:
                    return False
                left += 1
                right -= 1

            return True

        # Initialize two pointers.
        left, right = 0, len(s) - 1

        # Use two pointers to compare characters from both ends.
        while left < right:
            # If characters don't match, try skipping one character
            if s[left] != s[right]:
                # Check if either skipping the left or the right character
                # results in a palindrome.
                return isPalindromeRange(left + 1, right) or isPalindromeRange(
                    left, right - 1)
            left += 1
            right -= 1
        return True

```

◆ Quick Review

Key Insights

- Use two pointers starting from the beginning and end of the string.
- Compare characters; if they match, move both pointers inward.

- When a mismatch is found, you have two possibilities: skip the left character or the right character.
- Check if either of the resulting substrings is a palindrome.
- The approach effectively leverages a greedy method and operates in linear time.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(1)$, using only constant extra space.

Solution

The solution uses a two–pointer technique—one pointer starts at the beginning of the string and another at the end. They move towards each other while comparing characters. If a mismatch occurs, the algorithm tries skipping either the left or right character and then checks if the remaining substring forms a palindrome. This additional check is done using a helper function that verifies if a given substring is a palindrome. This strategy ensures that the algorithm only allows one

deletion, conforming to the constraint.

Strings

□ #796 Rotate String

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · String Matching

Lists: AlgoMaster 600

Problem

Given two strings *s* and *goal*, return true if and only if *s* can become *goal* after some number of shifts on *s*.

A shift on *s* consists of moving the leftmost character of *s* to the rightmost position.

- For example, if *s* = "abcde", then it will be "bcdea" after one shift.

Example 1:

Input: *s* = "abcde", *goal* = "cdeab"
Output: true

Example 2:

Input: *s* = "abcde", *goal* = "abced"
Output: false

Constraints:

- $1 \leq s.length, goal.length \leq 100$

- s and goal consist of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rotateString(self, s: str, goal: str) -> bool:
        return len(s) == len(goal) and goal in s + s
```

• LeetDoocs

```
class Solution:
    def rotateString(self, s: str, goal: str) -> bool:
        return len(s) == len(goal) and goal in s + s
```

• SimplyLeet

```
# Function to determine if s can become goal after some number of shifts
def rotateString(s, goal):
    # If lengths are different, s cannot be rotated to form goal
    if len(s) != len(goal):
        return False
    # Concatenate s with itself to cover all possible rotations
    doubled = s + s
    # Check if goal is a substring of the concatenated string
    return goal in doubled
```

```
# Example usage:
s = "abcde"
goal = "cdeab"
print(rotateString(s, goal)) # Output: True
```

◆ Quick Review

Key Insights

- Both strings must be of equal length for s to be rotated into goal.
- By concatenating s with itself ($s + s$), all possible rotations of s are represented as continuous substrings.
- We simply need to check if goal is a substring of $s + s$.

Space & Time

Time Complexity: $O(n)$ on average (n is the length of s) assuming an efficient substring search.

Space Complexity: $O(n)$ due to storing the concatenated string.

Solution

The solution leverages string concatenation. If s and goal have the same length, concatenating s with itself yields a new string that contains every rotation of s as a contiguous substring. Therefore, we only need to check if goal exists within this concatenated string. This solution uses simple string operations and takes advantage of built-in substring searching.

Hash Tables

Round 1 · High · Easy · 8 questions

Hash Tables

□ #205 Isomorphic Strings

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String

Lists: AlgoMaster 600

Problem

Given two strings s and t , determine if they are isomorphic.

Two strings s and t are isomorphic if the characters in s can be replaced to get t .

All occurrences of a character must be replaced with another character while preserving the order of characters. No two characters may map to the same character, but a character may map to itself.

Example 1:

Input: $s = \text{"egg"}, t = \text{"add"}$

Output: true

Explanation:

The strings s and t can be made identical by:

- **Mapping 'e' to 'a'.**

- Mapping 'g' to 'd'.

Example 2:

Input: s = "f11", t = "b23"

Output: false

Explanation:

The strings s and t can not be made identical as '1' needs to be mapped to both '2' and '3'.

Example 3:

Input: s = "paper", t = "title"

Output: true

Constraints:

- $1 \leq s.length \leq 5 * 10^4$
- $t.length == s.length$
- s and t consist of any valid ascii character.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isIsomorphic(self, s: str, t: str) -> bool:
        return [*map(s.index, s)] == [*map(t.index, t)]
```

• LeetDoocs

```
class Solution:
    def isIsomorphic(self, s: str, t: str) -> bool:
        d1 = {}
        d2 = {}
        for a, b in zip(s, t):
            if (a in d1 and d1[a] != b) or (b in d2 and d2[b] != a):
                return False
            d1[a] = b
            d2[b] = a
        return True
```

```
class Solution:
    def isIsomorphic(self, s: str, t: str) -> bool:
        d1, d2 = [0] * 256, [0] * 256
        for i, (a, b) in enumerate(zip(s, t), 1):
            a, b = ord(a), ord(b)
            if d1[a] != d2[b]:
                return False
            d1[a] = d2[b] = i
        return True
```

• SimplyLeet

```
# Function to check if s and t are isomorphic
def is_isomorphic(s, t):
    # Dictionary to store mapping of characters from s to t
    mapping_st = {}
    # Dictionary to store mapping of characters from t to s
    mapping_ts = {}

    # Iterate over characters in s and t concurrently
    for char_s, char_t in zip(s, t):
        # Check mapping from s to t
```

```
if char_s in mapping_st:
    if mapping_st[char_s] != char_t:
        return False
    else:
        mapping_st[char_s] = char_t

# Check mapping from t to s
if char_t in mapping_ts:
    if mapping_ts[char_t] != char_s:
        return False
    else:
        mapping_ts[char_t] = char_s

return True

# Example usage:
print(is_isomorphic("egg", "add"))    # Output: True
print(is_isomorphic("foo", "bar"))    # Output: False
print(is_isomorphic("paper", "title")) # Output: True
```

◆ Quick Review

Key Insights

- Use a mapping (dictionary/hash table) from characters in *s* to characters in *t*.
- Use a second mapping from characters in *t* back to *s* to ensure a one-to-one correspondence.
- If any mapping is inconsistent, the strings are not isomorphic.
- Traverse both strings simultaneously.

Space & Time

Time Complexity: $O(n)$, where n is the length of the strings, as we iterate through each character once.

Space Complexity: $O(1)$ under the assumption that the character set is fixed (ASCII), otherwise $O(n)$ in the worst case.

Solution

The algorithm iterates over each character pair from s and t . For each pair, it checks two mappings:

- If the current character from s has been previously mapped, it must map to the current character in t .
- If the current character from t has been previously mapped, it must map to the current character in s . If both conditions are satisfied for all character pairs, then the strings are isomorphic. Data structures used include two dictionaries (or hash tables) to keep track of the mappings from s to t and t to s .

Hash Tables

□ #219 Contains Duplicate II

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sliding Window
Lists: AlgoMaster 600

Problem

Given an integer array `nums` and an integer `k`, return `true` if there are two distinct indices `i` and `j` in the array such that `nums[i] == nums[j]` and `abs(i - j) <= k`.

Example 1:

Input: `nums = [1,2,3,1]`, `k = 3`
Output: `true`

Example 2:

Input: `nums = [1,0,1,1]`, `k = 1`
Output: `true`

Example 3:

Input: `nums = [1,2,3,1,2,3]`, `k = 2`
Output: `false`

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

- $0 \leq k \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def containsNearbyDuplicate(self, nums: list[int], k: int) -> bool:
        seen = set()

        for i, num in enumerate(nums):
            if i > k:
                seen.remove(nums[i - k - 1])
            if num in seen:
                return True
            seen.add(num)

        return False
```

• LeetDoocs

```
class Solution:
    def containsNearbyDuplicate(self, nums: List[int], k: int) -> bool:
        d = {}
        for i, x in enumerate(nums):
            if x in d and i - d[x] <= k:
                return True
            d[x] = i
        return False
```

• SimplyLeet

```
# Python solution with detailed comments
def containsNearbyDuplicate(nums, k):
    # Create a dictionary to store the number and its most recent index
    index_map = {}
    # Loop over every index and corresponding number in nums
    for i, num in enumerate(nums):
        # Check if the number is already in the dictionary
        if num in index_map:
            # If the difference between the current index and the stored index
            # is <= k, return True
            if i - index_map[num] <= k:
                return True
        # Update the dictionary with the current index for the number
        index_map[num] = i
    # If no such duplicate pair is found, return False
    return False

# Example usage:
# print(containsNearbyDuplicate([1,2,3,1], 3))
```

◆ Quick Review

Key Insights

- Use a hash table (or a sliding window) to keep track of the most recent index where each number appears.
- For every element, if it is already in the hash table, check the difference between the current index and its previously stored index.
- If the difference is within k, return true immediately.
- Otherwise, update the index for the element.

- Alternatively, maintain a sliding window (using a set) of size at most k to check for duplicates in the current window.

Space & Time

Time Complexity: $O(n)$ — where n is the number of elements in `nums`, since each element is processed once.

Space Complexity: $O(\min(n, k))$ — using a hash table or set that stores at most k elements at any time.

Solution

We leverage a hash table to store numbers and their most recent indices. As we iterate through the array, we check if the current number is already in the table. If so, we examine the distance between the current index and the saved index. Should this distance be at most k , we can immediately return true. If not, we update the stored index for that number. This method ensures only a single pass through the array is required. An alternative approach is to use a sliding window implemented via a set, where we maintain a window of size k . If a duplicate is

encountered within the window, we return true.

Hash Tables

□ #290 Word Pattern

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String
Lists: AlgoMaster 600

Problem

Given a pattern and a string s , find if s follows the same pattern.

Here follow means a full match, such that there is a bijection between a letter in pattern and a non-empty word in s . Specifically:

- **Each letter in pattern maps to exactly one unique word in s .**
- **Each unique word in s maps to exactly one letter in pattern.**
- **No two letters map to the same word, and no two words map to the same letter.**

Example 1:

Input: pattern = "abba", s = "dog cat cat dog"

Output: true

Explanation:

The bijection can be established as:

- 'a' maps to "dog".
- 'b' maps to "cat".

Example 2:

Input: pattern = "abba", s = "dog cat cat fish"

Output: false

Example 3:

Input: pattern = "aaaa", s = "dog cat cat dog"

Output: false

Constraints:

- $1 \leq \text{pattern.length} \leq 300$
- pattern contains only lower-case English letters.
- $1 \leq \text{s.length} \leq 3000$
- s contains only lowercase English letters and spaces ' '.
- s does not contain any leading or trailing spaces.
- All the words in s are separated by a single space.

Community Solutions (Python)

- ☐
- WalkCC

```
class Solution:
    def wordPattern(self, pattern: str, str: str) -> bool:
        t = str.split()
        return [*map(pattern.index, pattern)] == [*map(t.index, t)]
```

• LeetCode

```
class Solution:
    def wordPattern(self, pattern: str, s: str) -> bool:
        ws = s.split()
        if len(pattern) != len(ws):
            return False
        d1 = {}
        d2 = {}
        for a, b in zip(pattern, ws):
            if (a in d1 and d1[a] != b) or (b in d2 and d2[b] != a):
                return False

            d1[a] = b
            d2[b] = a
        return True
```

• SimplyLeet

```
# Python solution with line-by-line comments

def wordPattern(pattern: str, s: str) -> bool:
    # Split the string s based on spaces to get the list of words
    words = s.split()

    # If the number of words is not equal to the length of the pattern, the
    # mapping is impossible
    if len(pattern) != len(words):
        return False
```

```
# Initialize two dictionaries to maintain the mapping
char_to_word = {}
word_to_char = {}

# Traverse each character from the pattern along with the corresponding
word
for ch, word in zip(pattern, words):
    # If the mapping for character doesn't exist, check and assign
    if ch not in char_to_word:
        if word in word_to_char:
            # Word is already mapped to a different character, hence return
            False

        return False
    # Create mapping in both dictionaries
    char_to_word[ch] = word
    word_to_char[word] = ch
else:
    # If the character already has a mapping, validate it against the
    current word
    if char_to_word[ch] != word:
        return False
# If all mappings are consistent, return True
return True

# Example usage:
print(wordPattern("abba", "dog cat cat dog")) # Expected output: True
print(wordPattern("abba", "dog cat cat fish")) # Expected output: False
```

◆ Quick Review

Key Insights

- We need a one-to-one mapping between characters in the pattern and words in s.
- Use two hash maps (or dictionaries): one to map characters to words and another to map

words to characters.

- Ensure that the number of words in s equals the length of the pattern.
- Check for any conflict during the mapping process.

Space & Time

Time Complexity: $O(n)$ where n is the number of words (or the length of the pattern), as we iterate through both simultaneously.

Space Complexity: $O(n)$ for the mappings stored in the hash maps.

Solution

The approach is to first split the string s into words and verify that its length matches the length of the pattern. Then, iterate through each letter of the pattern along with the corresponding word, and do the following:

- If the pattern character is not in the char-to-word mapping, assign it, but ensure that the corresponding word is also not already assigned to a different character in the word-to-char mapping.
- If the pattern character already exists in the mapping, check that the mapped word

matches the current word.

– Return false at any point a conflict is found; otherwise, return true if the entire pattern is processed without error.

This dual-check via two hash maps ensures the bijection property is maintained.

Hash Tables

□ #350 Intersection of Two Arrays II

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Two Pointers · Binary
Search · Sorting

Lists: AlgoMaster 600

Problem

Given two integer arrays `nums1` and `nums2`, return an array of their intersection. Each element in the result must appear as many times as it shows in both arrays and you may return the result in any order.

Example 1:

Input: `nums1 = [1,2,2,1]`, `nums2 = [2,2]`
Output: `[2,2]`

Example 2:

Input: `nums1 = [4,9,5]`, `nums2 = [9,4,9,8,4]`
Output: `[4,9]`
Explanation: `[9,4]` is also accepted.

Constraints:

- $1 \leq \text{nums1.length}, \text{nums2.length} \leq 1000$
- $0 \leq \text{nums1}[i], \text{nums2}[i] \leq 1000$

Follow up:

- What if the given array is already sorted? How would you optimize your algorithm?
- What if nums1's size is small compared to nums2's size? Which algorithm is better?
- What if elements of nums2 are stored on disk, and the memory is limited such that you cannot load all elements into the memory at once?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def intersect(self, nums1: list[int], nums2: list[int]) -> list[int]:
        if len(nums1) > len(nums2):
            return self.intersect(nums2, nums1)

        ans = []
        count = collections.Counter(nums1)

        for num in nums2:
            if count[num] > 0:

                ans.append(num)
                count[num] -= 1

        return ans
```

• LeetDoocs

```
class Solution:
    def intersect(self, nums1: List[int], nums2: List[int]) -> List[int]:
        cnt = Counter(nums1)
        ans = []
        for x in nums2:
            if cnt[x]:
                ans.append(x)
                cnt[x] -= 1
        return ans
```

• SimplyLeet

```
# Function to compute the intersection of two arrays using a hash table
def intersect(nums1, nums2):
    # Create a dictionary to store frequency of elements in nums1
    frequency = {}
    for num in nums1:
        frequency[num] = frequency.get(num, 0) + 1

    # List to store the intersection result
    result = []
    # Iterate through nums2, and check if the element is in frequency with a
    # positive count

    for num in nums2:
        if frequency.get(num, 0) > 0:
            result.append(num)
            frequency[num] -= 1
    return result

# Example usage:
# print(intersect([1,2,2,1], [2,2]))
```

◆ Quick Review

Key Insights

- Use a hash table to count the occurrences of each number in one array.

- Iterate through the second array, and for each number, check if it exists in the hash table with a non-zero count.
- If it does, add the number to the result and decrement the count in the hash table.
- For sorted arrays, a two-pointer approach can be used for optimal performance.
- Consider memory-efficient methods when one of the arrays is too large to fit in memory.

Space & Time

Time Complexity: $O(n + m)$, where n and m are the lengths of the two arrays.

Space Complexity: $O(\min(n, m))$, since the hash table stores counts for the smaller array.

Solution

We solve the problem by first storing the frequency of each element from one input array using a hash table (dictionary). Then, we iterate through the second array and check if the element is present in the hash table with a positive count. If it is, we add it to the result array and reduce the count. This ensures that each common element is added as many times as it appears in both arrays. If the arrays are

sorted, a two–pointer technique can be used where both arrays are traversed simultaneously, providing an alternative approach with potentially lower overhead in specific scenarios.

Hash Tables

□ #387 First Unique Character in a String

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Queue · Counting
Lists: AlgoMaster 600

Problem

Given a string *s*, find the first non-repeating character in it and return its index. If it does not exist, return -1 .

Example 1:

Input: *s* = "leetcode"

Output: 0

Explanation:

The character 'l' at index 0 is the first character that does not occur at any other index.

Example 2:

Input: *s* = "loveleetcode"

Output: 2

Example 3:

Input: *s* = "aabb"

Output: -1

Constraints:

- $1 \leq s.length \leq 105$
- s consists of only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def firstUniqChar(self, s: str) -> int:
        count = collections.Counter(s)

        for i, c in enumerate(s):
            if count[c] == 1:
                return i

        return -1
```

• LeetDoocs

```
class Solution:
    def firstUniqChar(self, s: str) -> int:
        cnt = Counter(s)
        for i, c in enumerate(s):
            if cnt[c] == 1:
                return i
        return -1
```

• SimplyLeet

```
def firstUniqChar(s):
    # Create a frequency dictionary to count occurrences of each character
    char_count = {}
    # First pass: count every character in the string
    for char in s:
        if char in char_count:
            char_count[char] += 1
        else:
            char_count[char] = 1
    # Second pass: find the first character that occurs exactly once
```

```
for index, char in enumerate(s):
    if char_count[char] == 1:
        return index
# Return -1 if no unique character is found
return -1

# Example usage:
print(firstUniqChar("leetcode"))    # Expected output: 0
print(firstUniqChar("loveleetcode")) # Expected output: 2
```

◆ Quick Review

Key Insights

- Use a hash table (or fixed-size array) to count the frequency of each character in the string.
- Iterate over the string a second time to find the first character with a frequency of one.
- The solution handles the case where no unique character exists by returning -1.

Space & Time

Time Complexity: $O(n)$ – two passes over the string.

Space Complexity: $O(1)$ – only 26 lowercase letters are stored, which is a constant space.

Solution

This solution leverages a frequency counting approach with a hash table (or fixed-size array for the 26 lowercase letters). In the first pass, the frequency of each character in the string is recorded. In the second pass, the string is iterated again to locate the first character with a count of one. If found, its index is returned; if not, -1 is returned. This strategy provides an efficient way to solve the problem with just two passes over the input string.

Hash Tables

□ #448 Find All Numbers Disappeared in an Array

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table
Lists: AlgoMaster 600

Problem

Given an array `nums` of n integers where `nums[i]` is in the range $[1, n]$, return an array of all the integers in the range $[1, n]$ that do not appear in `nums`.

Example 1:

Input: `nums = [4,3,2,7,8,2,3,1]`
Output: `[5,6]`

Example 2:

Input: `nums = [1,1]`
Output: `[2]`

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 105$
- $1 \leq \text{nums}[i] \leq n$

Follow up: Could you do it without extra space and in $O(n)$ runtime? You may assume the returned list does not count as extra space.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findDisappearedNumbers(self, nums: list[int]) -> list[int]:
        for num in nums:
            index = abs(num) - 1
            nums[index] = -abs(nums[index])

        return [i + 1 for i, num in enumerate(nums) if num > 0]
```

• LeetDoocs

```
class Solution:
    def findDisappearedNumbers(self, nums: List[int]) -> List[int]:
        s = set(nums)
        return [x for x in range(1, len(nums) + 1) if x not in s]
```

```
class Solution:
    def findDisappearedNumbers(self, nums: List[int]) -> List[int]:
        for x in nums:
            i = abs(x) - 1
            if nums[i] > 0:
                nums[i] *= -1
        return [i + 1 for i in range(len(nums)) if nums[i] > 0]
```

• SimplyLeet

```
# Function to find all disappeared numbers in an array
def findDisappearedNumbers(nums):
    # Iterate over each number in the list
    for i in range(len(nums)):
        # Get the index corresponding to the current number (adjusting for
        # 0-indexed array)
        index = abs(nums[i]) - 1
        # Mark the number at this index as negative if it is not already marked
        if nums[index] > 0:
            nums[index] = -nums[index]

    # Initialize the result list
    result = []
    # Iterate over the array to collect indices of numbers that did not appear
    for i, num in enumerate(nums):
        # If the number is positive, then index + 1 is missing in the original
        # array
        if num > 0:
            result.append(i + 1)

    # Return the list of missing numbers
    return result

# Example usage:
# print(findDisappearedNumbers([4,3,2,7,8,2,3,1])) # Output: [5,6]
```

◆ Quick Review

Key Insights

- The input array contains integers from 1 to n, so each value can be mapped to an index in the array.
- In-place modification can be achieved by marking the values at these mapped indices to

indicate presence.

- By iterating over the array after marking, the indices that were not marked (i.e., still positive) correspond to the missing numbers.
- This solution avoids using extra space beyond the output array.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$ (excluding the space for the output array)

Solution

We iterate through the array and for each number, we determine its corresponding index (by taking the absolute value of the number and subtracting one). We then mark the number at that index as negative to indicate that the number (index + 1) exists in the array. Finally, we iterate through the array once more and collect all indices that still hold positive values; these indices correspond to numbers that did not appear in the original array.

Hash Tables

□ #1189 Maximum Number of Balloons

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Counting
Lists: AlgoMaster 600

Problem

Given a string text, you want to use the characters of text to form as many instances of the word "balloon" as possible.

You can use each character in text at most once. Return the maximum number of instances that can be formed.

Example 1:

```
Input: text = "nlaebolko"
Output: 1
```

Example 2:

```
Input: text = "loonbalxballpoon"
Output: 2
```

Example 3:

```
Input: text = "leetcode"
Output: 0
```

Constraints:

- $1 \leq \text{text.length} \leq 104$
- text consists of lower case English letters only.

Note: This question is the same as 2287: Rearrange Characters to Make Target String.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def maxNumberOfBalloons(self, text: str) -> int:
        cnt = Counter(text)
        cnt['o'] >= 1
        cnt['l'] >= 1
        return min(cnt[c] for c in 'balon')
```

• SimplyLeet

```
from collections import Counter

def maxNumberOfBalloons(text: str) -> int:
    # Count frequency of each character in the text.
    char_count = Counter(text)
    # Calculate the number of possible "balloon" words for each character,
    # noting that 'l' and 'o' are needed twice.
    count_b = char_count.get('b', 0)
    count_a = char_count.get('a', 0)
    count_l = char_count.get('l', 0) // 2
    count_o = char_count.get('o', 0) // 2

    count_n = char_count.get('n', 0)
    # The maximum number of balloons is limited by the smallest count from
    # required characters.
    return min(count_b, count_a, count_l, count_o, count_n)

# Example usage:
print(maxNumberOfBalloons("nlaebolko")) # Output: 1
```

◆ Quick Review

Key Insights

- Count the frequency of each character in the text.
- The target word "balloon" requires:
 - b: 1
 - a: 1
 - l: 2
 - o: 2
 - n: 1
- The number of times "balloon" can be formed is limited by the minimum value among these frequencies (taking into account that l and o are needed twice).

Space & Time

Time Complexity: $O(n)$, where n is the length of text.

Space Complexity: $O(1)$, as the frequency count uses constant space (only lower-case English letters).

Solution

We first build a frequency map of characters in the input text using a hash table (or dictionary). For the word "balloon", we calculate the possible counts by taking:

- The count of 'b'
- The count of 'a'
- The count of 'n'
- The count of 'l' divided by 2 (since two are needed)
- The count of 'o' divided by 2 (since two are needed)

The answer is the minimum of all these values, ensuring that all necessary characters are available in the required quantities. This approach is efficient due to single-pass counting and constant extra space.

Hash Tables

□ #1512 Number of Good Pairs

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Counting
Lists: AlgoMaster 600

Problem

Given an array of integers `nums`, return the number of good pairs.

A pair (i, j) is called good if `nums[i] == nums[j]` and $i < j$.

Example 1:

Input: `nums = [1,2,3,1,1,3]`

Output: 4

Explanation: There are 4 good pairs $(0,3)$, $(0,4)$, $(3,4)$, $(2,5)$ 0-indexed.

Example 2:

Input: `nums = [1,1,1,1]`

Output: 6

Explanation: Each pair in the array are good.

Example 3:

Input: `nums = [1,2,3]`

Output: 0

Constraints:

- $1 \leq \text{nums.length} \leq 100$

- $1 \leq \text{nums}[i] \leq 100$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def numIdenticalPairs(self, nums: List[int]) -> int:
        ans = 0
        cnt = Counter()
        for x in nums:
            ans += cnt[x]
            cnt[x] += 1
        return ans
```

• SimplyLeet

```
# Python solution to count the number of good pairs in an array

def num_identical_pairs(nums):
    # Dictionary to store the frequency of each number
    frequency = {}
    count = 0
    # Iterate over each number in the list
    for num in nums:
        # If the number is already seen, add its current frequency to count
        if num in frequency:
            count += frequency[num]
            frequency[num] += 1 # Increment the frequency
        else:
            frequency[num] = 1 # Initialize frequency for new number
    return count

# Example usage:
nums = [1, 2, 3, 1, 1, 3]
print(num_identical_pairs(nums)) # Expected output: 4
```

◆ Quick Review

Key Insights

- Use counting to determine how many times each number appears.
- For every occurrence of a number, the number of good pairs contributed by that occurrence is the count of that number seen so far.
- A hash table is ideal for tracking frequencies quickly.
- The pair counting can be done in one pass through the array.

Space & Time

Time Complexity: $O(n)$ where n is the length of the input array.

Space Complexity: $O(n)$ in the worst case due to the frequency hash table.

Solution

We utilize a hash table (or dictionary) to map each number to its frequency of occurrence as we iterate through the list. Each time we encounter a number, we add its current frequency (which represents the number of times it has appeared before) to the overall count of good pairs. Then, we increment its

frequency in the hash table. This method counts the pairs in a single pass, making the solution efficient in both time and space.

Two Pointers

Round 1 · High · Easy · 2 questions

Two Pointers

□ #696 Count Binary Substrings

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String
Lists: AlgoMaster 600

Problem

Given a binary string *s*, return the number of non-empty substrings that have the same number of 0's and 1's, and all the 0's and all the 1's in these substrings are grouped consecutively.

Substrings that occur multiple times are counted the number of times they occur.

Example 1:

Input: *s* = "00110011"

Output: 6

Explanation: There are 6 substrings that have equal number of consecutive 1's and 0's: "0011", "01", "1100", "10", "0011", and "01".

Notice that some of these substrings repeat and are counted the number of times they occur.

Also, "00110011" is not a valid substring because all the 0's (and 1's) are not grouped together.

Example 2:

Input: s = "10101"

Output: 4

Explanation: There are 4 substrings: "10", "01", "10", "01" that have equal number of consecutive 1's and 0's.

Constraints:

- $1 \leq s.length \leq 105$
- s[i] is either '0' or '1'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countBinarySubstrings(self, s: str) -> int:
        ans = 0
        prevCount = 0
        equals = 1

        for i in range(len(s) - 1):
            if s[i] == s[i + 1]:
                equals += 1
            else:
                ans += min(prevCount, equals)
                prevCount = equals
                equals = 1

        return ans + min(prevCount, equals)
```

• LeetDoocs


```

class Solution:
    def countBinarySubstrings(self, s: str) -> int:
        n = len(s)
        ans = i = 0
        pre = 0
        while i < n:
            j = i + 1
            while j < n and s[j] == s[i]:
                j += 1
            cur = j - i

            ans += min(pre, cur)
            pre = cur
            i = j
        return ans

```

• SimplyLeet

```

# Python solution with clear comments
def countBinarySubstrings(s: str) -> int:
    # List to store counts of consecutive groups
    groups = []
    count = 1 # Start count for the first character

    # Iterate through the string starting from the second character
    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            # Same character as previous, increase count

            count += 1
        else:
            # Character changed, append the count of the previous group and
            # reset count
            groups.append(count)
            count = 1

    # Append the count for the last group
    groups.append(count)

    # Now count valid substrings using adjacent group counts
    validSubstrings = 0

```

```
for i in range(1, len(groups)):
    # The number of valid substrings is the minimum of current and previous
    group counts
    validSubstrings += min(groups[i - 1], groups[i])
return validSubstrings

# Example usage:
s = "00110011"
print(countBinarySubstrings(s)) # Output: 6
```

◆ Quick Review

Key Insights

- The substrings we are looking for are periods where a block of 0's is directly followed by a block of 1's (or vice versa).
- The valid substrings are determined by the minimum length of two consecutive groups.
- We can solve this problem by compressing the string into a list of counts representing consecutive groups of the same character.
- The answer is the sum of the $\min(\text{count}[i], \text{count}[i+1])$ for all valid consecutive groups.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string.

Space Complexity: $O(n)$ in the worst-case scenario (when all characters are alternating)

which can be optimized to $O(1)$ by processing on the fly.

Solution

We first iterate over the string to count consecutive character groups. Once we have these counts, for each pair of adjacent groups, the number of valid substrings that can be formed is the minimum of the two counts. This works because if a group of 0's is of size x and the following group of 1's is of size y , then there are $\min(x, y)$ valid substrings that start in one group and extend into the next.

The solution leverages a two-pass approach (or even a single-pass approach by keeping track of the previous group count) and uses simple arithmetic to compute the final answer.

Two Pointers

□ #1768 Merge Strings Alternately

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String
Lists: AlgoMaster 600

Problem

You are given two strings `word1` and `word2`. Merge the strings by adding letters in alternating order, starting with `word1`. If a string is longer than the other, append the additional letters onto the end of the merged string. Return the merged string.

Example 1:

Input: `word1 = "abc", word2 = "pqr"`

Output: `"apbqcr"`

Explanation: The merged string will be merged as so:

`word1: a b c`

Example 2:

Input: word1 = "ab", word2 = "pqrs"

Output: "apbqrs"

Explanation: Notice that as word2 is longer, "rs" is appended to the end.

word1: a b

Example 3:

Input: word1 = "abcd", word2 = "pq"

Output: "apbqcd"

Explanation: Notice that as word1 is longer, "cd" is appended to the end.

word1: a b c d

Constraints:

- $1 \leq \text{word1.length}, \text{word2.length} \leq 100$
- word1 and word2 consist of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mergeAlternately(self, word1: str, word2: str) -> str:
        return ''.join(a + b for a, b in zip_longest(word1, word2, fillvalue=''))
```

• LeetDoocs

```
class Solution:
    def mergeAlternately(self, word1: str, word2: str) -> str:
        return ''.join(a + b for a, b in zip_longest(word1, word2, fillvalue=''))
```

• SimplyLeet

```
# Define function to merge two strings alternately
def merge_alternately(word1: str, word2: str) -> str:
    # Initialize an empty list to collect merged characters
    merged = []
    # Determine the minimum length to iterate for both strings
    min_length = min(len(word1), len(word2))

    # Iterate over both strings up to the minimum length
    for i in range(min_length):
        # Append character from word1
        merged.append(word1[i])
        # Append character from word2
        merged.append(word2[i])

    # After the loop, append the remaining part of word1 if any
    if min_length < len(word1):
        merged.append(word1[min_length:])
    # Append the remaining part of word2 if any
    if min_length < len(word2):
        merged.append(word2[min_length:])

    # Join all parts into a final string and return
    return ''.join(merged)

# Example usage:
print(merge_alternately("abc", "pqr")) # Output: "apbqcr"
```

◆ Quick Review

Key Insights

- Use two pointers to iterate through both strings.
- Append characters alternately until one string is exhausted.

- After the loop, append any remaining characters from the longer string.
- Ensure an $O(n + m)$ time complexity where n and m are the lengths of the two strings.

Space & Time

Time Complexity: $O(n + m)$

Space Complexity: $O(n + m)$

Solution

The solution involves initializing two pointers (or indices) for the two strings and iterating over them simultaneously. At each iteration, add the character from word1 followed by the character from word2. When one pointer reaches the end of its string, append the remainder of the other string to the result. This approach employs a straightforward two-pointer technique and string concatenation.

Sliding Window – Fixed Size

Round 1 · High · Easy · 1 question

Sliding Window – Fixed Size

□ #643 Maximum Average Subarray I

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sliding Window

Lists: AlgoMaster 600

Problem

You are given an integer array `nums` consisting of n elements, and an integer k .

Find a contiguous subarray whose length is equal to k that has the maximum average value and return this value. Any answer with a calculation error less than 10^{-5} will be accepted.

Example 1:

Input: `nums = [1,12,-5,-6,50,3]`, `k = 4`

Output: `12.75000`

Explanation: Maximum average is $(12 - 5 - 6 + 50) / 4 = 51 / 4 = 12.75$

Example 2:

Input: `nums = [5]`, `k = 1`

Output: `5.00000`

Constraints:

- $n == \text{nums.length}$

- $1 \leq k \leq n \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMaxAverage(self, nums: list[int], k: int) -> float:
        summ = sum(nums[:k])
        ans = summ

        for i in range(k, len(nums)):
            summ += nums[i] - nums[i - k]
            ans = max(ans, summ)

        return ans / k
```

• LeetDoocs

```
class Solution:
    def findMaxAverage(self, nums: List[int], k: int) -> float:
        ans = s = sum(nums[:k])
        for i in range(k, len(nums)):
            s += nums[i] - nums[i - k]
            ans = max(ans, s)
        return ans / k
```

• SimplyLeet

```
# Python solution using the sliding window technique

def findMaxAverage(nums, k):
    # Calculate the initial sum of the first window of size k
    window_sum = sum(nums[:k])
    max_sum = window_sum

    # Slide the window from index k to the end of the array
    for i in range(k, len(nums)):
        # Update the sum by subtracting the element that is sliding out
```

```
# and adding the new element
window_sum += nums[i] - nums[i - k]
# Update max_sum if the new window_sum is higher
max_sum = max(max_sum, window_sum)

# Return the maximum average found
return max_sum / k

# Example usage:
# nums = [1, 12, -5, -6, 50, 3]

# k = 4
# print(findMaxAverage(nums, k)) # Output: 12.75
```

◆ Quick Review

Key Insights

- Use a sliding window to efficiently compute the sum of every subarray of length k .
- Maintain the maximum sum encountered while sliding the window across the array.
- Finally, calculate the average of the subarray with the maximum sum by dividing by k .

Space & Time

Time Complexity: $O(n)$ – We compute the initial sum in $O(k)$ and then update the sum in $O(n-k)$.

Space Complexity: $O(1)$ – Only a few extra variables are used regardless of input size.

Solution

Use the sliding window technique. Start by computing the sum of the first k elements. As you slide the window forward one index at a time, subtract the element that is leaving the window and add the new element that enters the window. Maintain a variable to track the maximum sum seen so far. Finally, the maximum average is the maximum sum divided by k .

Binary Search

Round 1 · High · Easy · 2 questions

Binary Search

□ #367 Valid Perfect Square

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Binary Search
Lists: AlgoMaster 600

Problem

Given a positive integer num, return true if num is a perfect square or false otherwise.

A perfect square is an integer that is the square of an integer. In other words, it is the product of some integer with itself.

You must not use any built-in library function, such as sqrt.

Example 1:

Input: num = 16
Output: true
Explanation: We return true because $4 * 4 = 16$ and 4 is an integer.

Example 2:

Input: num = 14
Output: false
Explanation: We return false because $3.742 * 3.742 = 14$ and 3.742 is not an integer.

Constraints:

- $1 \leq \text{num} \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isPerfectSquare(self, num: int) -> bool:
        l = bisect.bisect_left(range(num), num, key=lambda m: m * m)
        return l**2 == num
```

• LeetDoocs

```
class Solution:
    def isPerfectSquare(self, num: int) -> bool:
        l = bisect_left(range(1, num + 1), num, key=lambda x: x * x) + 1
        return l * l == num
```

```
class Solution:
    def isPerfectSquare(self, num: int) -> bool:
        i = 1
        while num > 0:
            num -= i
            i += 2
        return num == 0
```

• SimplyLeet

```
# Function to determine if a number is a perfect square using binary search.
def isPerfectSquare(num):
    # If num is less than 2, it is a perfect square (e.g., 1).
    if num < 2:
        return True
    # Set initial boundaries for binary search.
    left, right = 2, num // 2
    while left <= right:
        # Calculate mid point.
        mid = left + (right - left) // 2
```

```
    guessed_square = mid * mid
    # Check if the mid value squares to num.
    if guessed_square == num:
        return True
    # If guessed square is too high, discard right half.
    elif guessed_square > num:
        right = mid - 1
    # Otherwise, discard left half.
    else:
        left = mid + 1

return False

# Example usage:
print(isPerfectSquare(16)) # True
print(isPerfectSquare(14)) # False
```

◆ Quick Review

Key Insights

- Recognize that if num is a perfect square, it will have an exact integer square root.
- Use binary search between 1 and num/2 (when num ≥ 4) to efficiently determine if there is an integer mid such that mid * mid == num.
- The binary search stops when the search space is exhausted, meaning num is not a perfect square if no integer mid found.

Space & Time

Time Complexity: $O(\log(\text{num}))$

Space Complexity: $O(1)$

Solution

We solve the problem using binary search. By checking $\text{mid} * \text{mid}$ at each step, we narrow down the interval where the square root could lie, eventually determining if an exact integer square root exists. The trick is setting the correct search boundaries: numbers less than 4 can be handled directly, and for larger numbers, the square root can never be more than $\text{num}/2$. This avoids using built-in functions and deals gracefully with large inputs.

Binary Search

□ #744 Find Smallest Letter Greater Than Target

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: AlgoMaster 600

Problem

You are given an array of characters letters that is sorted in non-decreasing order, and a character target. There are at least two different characters in letters.

Return the smallest character in letters that is lexicographically greater than target. If such a character does not exist, return the first character in letters.

Example 1:

```
Input: letters = ["c","f","j"], target = "a"
Output: "c"
Explanation: The smallest character that is lexicographically greater than 'a'
in letters is 'c'.
```

Example 2:

```
Input: letters = ["c","f","j"], target = "c"
Output: "f"
Explanation: The smallest character that is lexicographically greater than 'c'
in letters is 'f'.
```

Example 3:

Input: letters = ["x","x","y","y"], target = "z"

Output: "x"

Explanation: There are no characters in letters that is lexicographically greater than 'z' so we return letters[0].

Constraints:

- $2 \leq \text{letters.length} \leq 104$
- letters[i] is a lowercase English letter.
- letters is sorted in non-decreasing order.
- letters contains at least two different characters.
- target is a lowercase English letter.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def nextGreatestLetter(self, letters: list[str], target: str) -> str:
        l = bisect.bisect_right(range(len(letters)), target,
                                key=lambda m: letters[m])
        return letters[l % len(letters)]
```

• LeetDoocs

```
class Solution:
    def nextGreatestLetter(self, letters: List[str], target: str) -> str:
        i = bisect_right(letters, ord(target), key=lambda c: ord(c))
        return letters[i % len(letters)]
```

• SimplyLeet

```
# Function to find the smallest letter greater than target using binary search.
def nextGreatestLetter(letters, target):
    # Initialize left and right pointers.
    left, right = 0, len(letters) - 1

    # Perform binary search.
    while left <= right:
        # Calculate the middle index.
        mid = left + (right - left) // 2

        # If the middle element is less than or equal to target, search in the
        # right half.
        if letters[mid] <= target:
            left = mid + 1
        else:
            # Otherwise, search in the left half.
            right = mid - 1

    # Use modulo to handle wrap-around and return the next greatest letter.
    return letters[left % len(letters)]

# Example usage:
# letters = ['c', 'f', 'j']
# target = 'a'
# print(nextGreatestLetter(letters, target)) # Output: 'c'
```

◆ Quick Review

Key Insights

- The array is sorted in non-decreasing order, which enables an efficient binary search solution.
- The binary search is used to identify the first character greater than the target.

- Account for the wrap-around scenario by returning the first character when the target is greater than or equal to all elements in the array.

Space & Time

Time Complexity: $O(\log n)$ where n is the length of the letters array

Space Complexity: $O(1)$

Solution

We can solve this problem using binary search. Initialize two pointers, left and right, to define the search boundaries. Compute the middle index and compare the character at that index with the target. If the current character is less than or equal to the target, move the left pointer to $\text{mid} + 1$; otherwise, move the right pointer to $\text{mid} - 1$. After the loop, use $\text{left} \% n$ (where n is the size of the array) to handle the wrap-around case and return the correct character.

Stacks

Round 1 · High · Easy · 3 questions

Stacks

□ #682 Baseball Game

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Simulation
Lists: AlgoMaster 600

Problem

You are keeping the scores for a baseball game with strange rules. At the beginning of the game, you start with an empty record.

You are given a list of strings operations, where operations[i] is the ith operation you must apply to the record and is one of the following:

- An integer x. Record a new score of x.
- Record a new score that is the sum of the previous two scores.
- Record a new score that is the double of the previous score.
- Invalidate the previous score, removing it from the record.

Return the sum of all the scores on the record after applying all the operations.

The test cases are generated such that the answer and all intermediate calculations fit in a 32-bit integer and that all operations are valid.

Example 1:

Input: ops = ["5","2","C","D","+"]

Output: 30

Explanation:

"5" - Add 5 to the record, record is now [5].

"2" - Add 2 to the record, record is now [5, 2].

"C" - Invalidate and remove the previous score, record is now [5].

"D" - Add $2 * 5 = 10$ to the record, record is now [5, 10].

"+" - Add $5 + 10 = 15$ to the record, record is now [5, 10, 15].

Example 2:

Input: ops = ["5","-2","4","C","D","9","+","+"]

Output: 27

Explanation:

"5" - Add 5 to the record, record is now [5].

"-2" - Add -2 to the record, record is now [5, -2].

"4" - Add 4 to the record, record is now [5, -2, 4].

"C" - Invalidate and remove the previous score, record is now [5, -2].

"D" - Add $2 * -2 = -4$ to the record, record is now [5, -2, -4].

Example 3:

Input: ops = ["1","C"]

Output: 0

Explanation:

"1" - Add 1 to the record, record is now [1].

"C" - Invalidate and remove the previous score, record is now [].

Since the record is empty, the total sum is 0.

Constraints:

- $1 \leq \text{operations.length} \leq 1000$
- `operations[i]` is "C", "D", "+", or a string representing an integer in the range $[-3 * 10^4, 3 * 10^4]$.

- For operation "+", there will always be at least two previous scores on the record.
- For operations "C" and "D", there will always be at least one previous score on the record.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def calPoints(self, operations: list[str]) -> int:
        scores = []

        for operation in operations:
            match operation:
                case '+':
                    scores.append(scores[-1] + scores[-2])
                case 'D':
                    scores.append(scores[-1] * 2)

                case 'C':
                    scores.pop()
                case default:
                    scores.append(int(operation))

        return sum(scores)
```

• LeetDoocs

```
class Solution:
    def calPoints(self, operations: List[str]) -> int:
        stk = []
        for op in operations:
            if op == "+":
                stk.append(stk[-1] + stk[-2])
            elif op == "D":
                stk.append(stk[-1] << 1)
            elif op == "C":
                stk.pop()
```

```
        else:
            stk.append(int(op))
    return sum(stk)
```

• SimplyLeet

```
# Define the function to calculate total score for the baseball game
def calPoints(ops):
    # Initialize a list to be used as a stack for scores
    score_stack = []

    # Iterate over each operation in the list
    for op in ops:
        # If the operation is "C", remove the last score from the stack
        if op == "C":
            score_stack.pop()

        # If the operation is "D", double the last score and append it
        elif op == "D":
            score_stack.append(2 * score_stack[-1])

        # If the operation is "+", sum the last two scores and append the
        # result
        elif op == "+":
            score_stack.append(score_stack[-1] + score_stack[-2])

        # Otherwise, the operation is a number in string form, convert to
        # integer and append
        else:
            score_stack.append(int(op))

    # Return the total sum of all scores in the stack
    return sum(score_stack)

# Example usage:
print(calPoints(["5", "2", "C", "D", "+"])) # Expected output: 30
```

◆ Quick Review

Key Insights

- Use a stack (list) to keep track of valid scores.
- Process operations sequentially, updating the stack accordingly.
- Handle different operations by manipulating the stack: push new scores, pop invalid scores.
- Final result is the sum of scores left in the stack.

Space & Time

Time Complexity: $O(n)$, where n is the number of operations.

Space Complexity: $O(n)$, in the worst-case the stack stores all operation results.

Solution

The solution uses a stack-based approach.

Iterate through each operation and:

- If the operation is a numeric value, convert it to an integer and push it onto the stack.
- If it is "C", remove the most recent score by popping from the stack.
- If it is "D", double the most recent score and push the result.

– If it is "+", calculate the sum of the two most recent scores and push that value. Finally, sum all the values in the stack to obtain the final score.

Stacks

□ #1047 Remove All Adjacent Duplicates In String

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack

Lists: AlgoMaster 600

Problem

You are given a string *s* consisting of lowercase English letters. A duplicate removal consists of choosing two adjacent and equal letters and removing them.

We repeatedly make duplicate removals on *s* until we no longer can.

Return the final string after all such duplicate removals have been made. It can be proven that the answer is unique.

Example 1:

Input: *s* = "abbaca"

Output: "ca"

Explanation:

For example, in "abbaca" we could remove "bb" since the letters are adjacent and equal, and this is the only possible move. The result of this move is that the string is "aaca", of which only "aa" is possible, so the final string is "ca".

Example 2:

Input: s = "azxxzy"
Output: "ay"

Constraints:

- $1 \leq s.length \leq 105$
- s consists of lowercase English letters.

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def removeDuplicates(self, s: str) -> str:
        stk = []
        for c in s:
            if stk and stk[-1] == c:
                stk.pop()
            else:
                stk.append(c)
        return ''.join(stk)
```

• SimplyLeet

```
# Function to remove all adjacent duplicates in string
def remove_duplicates(s: str) -> str:
    # Initialize an empty list to use as a stack
    stack = []
    # Iterate through each character in the input string
    for char in s:
        # If stack is not empty and the top element equals the current
        # character, pop the top element
        if stack and stack[-1] == char:
            stack.pop()
        else:
```

```
        # Otherwise, push the current character onto the stack
        stack.append(char)
    # Join the stack elements to form the final string result and return it
    return ''.join(stack)

# Example usage:
# result = remove_duplicates("abbaca")
# print(result) # Output: "ca"
```

◆ Quick Review

Key Insights

- Utilize a stack to efficiently process and remove adjacent duplicates.
- Iterate through each character in the string, comparing it with the top of the stack.
- If the current character matches the top of the stack, pop the stack (i.e., remove the duplicate pair).
- If there is no match, push the current character onto the stack.
- Finally, join the characters remaining in the stack to obtain the result.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, because each character is processed at most once.

Space Complexity: $O(n)$, in the worst-case scenario where no duplicates are found and all characters are stored in the stack.

Solution

The solution uses a stack-based approach. As we iterate through the input string, each character is compared with the top element of the stack. If they are the same, it means we have found an adjacent duplicate, so we remove the top element. Otherwise, we push the current character onto the stack. This method ensures that duplicate adjacent pairs are removed in a single pass through the string. Finally, the remaining characters in the stack are combined to form the final string, which is then returned as the unique answer.

Stacks

□ #1614 Maximum Nesting Depth of the Parentheses

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack

Lists: AlgoMaster 600

Problem

Given a valid parentheses string s , return the nesting depth of s . The nesting depth is the maximum number of nested parentheses.

Example 1:

Input: $s = "(1+(2*3)+((8)/4))+1"$

Output: 3

Explanation:

Digit 8 is inside of 3 nested parentheses in the string.

Example 2:

Input: $s = "(1)+((2))+(((3)))"$

Output: 3

Explanation:

Digit 3 is inside of 3 nested parentheses in the string.

Example 3:

Input: $s = "()()((()()))"$

Output: 3

Constraints:

- $1 \leq s.length \leq 100$
- s consists of digits 0–9 and characters '+', '-', '*', '/', '(', and ')'. It is guaranteed that parentheses expression s is a VPS.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxDepth(self, s: str) -> int:
        ans = 0
        opened = 0

        for c in s:
            if c == '(':
                opened += 1
                ans = max(ans, opened)
            elif c == ')':
                opened -= 1

        return ans
```

• LeetDoocs

```
class Solution:
    def maxDepth(self, s: str) -> int:
        ans = d = 0
        for c in s:
            if c == '(':
                d += 1
                ans = max(ans, d)
            elif c == ')':
                d -= 1
        return ans
```

• SimplyLeet

```
# Function to calculate the maximum nesting depth of the parentheses in the
string
def maxDepth(s: str) -> int:
    currentDepth = 0 # To keep track of current nesting level
    maxDepth = 0     # To record the maximum nesting depth encountered

    # Iterate over each character in the string s
    for char in s:
        if char == '(':
            currentDepth += 1 # Increase nesting level for '('
            maxDepth = max(maxDepth, currentDepth) # Update maxDepth if
needed

        elif char == ')':
            currentDepth -= 1 # Decrease nesting level for ')'
    return maxDepth

# Example usage
s = "(1+(2*3)+((8)/4))+1"
print(maxDepth(s)) # Output: 3
```

◆ Quick Review

Key Insights

- Use a counter to track the current nesting level.

- Increment the counter when encountering an opening parenthesis '('.
- Update the maximum depth if the current counter exceeds the previous maximum.
- Decrement the counter when encountering a closing parenthesis ')'.
- Since the input is guaranteed to be a valid parentheses string, no additional checks are required.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(1)$

Solution

The approach involves iterating over every character in the string once. We maintain two variables: one to count the current level of nesting (`currentDepth`) and one to store the maximum depth encountered (`maxDepth`). For every character:

- If it is an opening parenthesis '(', increment `currentDepth`. Then check and update `maxDepth` if `currentDepth` is greater.

- If it is a closing parenthesis ')', decrement `currentDepth`. This simple counter approach ensures that we determine the maximum nesting depth efficiently using constant extra space.

Tree Traversal – Level Order

Round 1 · High · Easy · 1 question

Tree Traversal – Level Order

□ #637 Average of Levels in Binary Tree

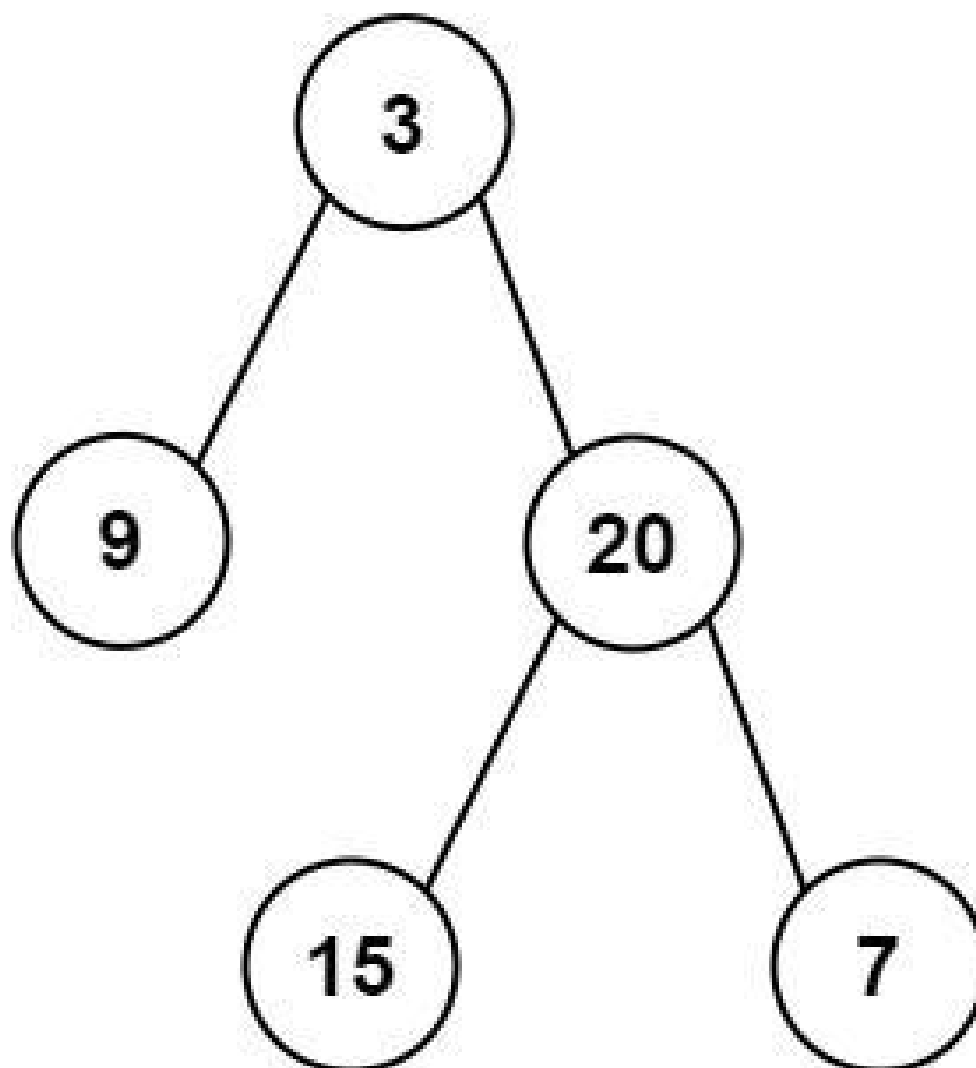
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Breadth-First
Search · Binary Tree
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, return the average value of the nodes on each level in the form of an array. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:



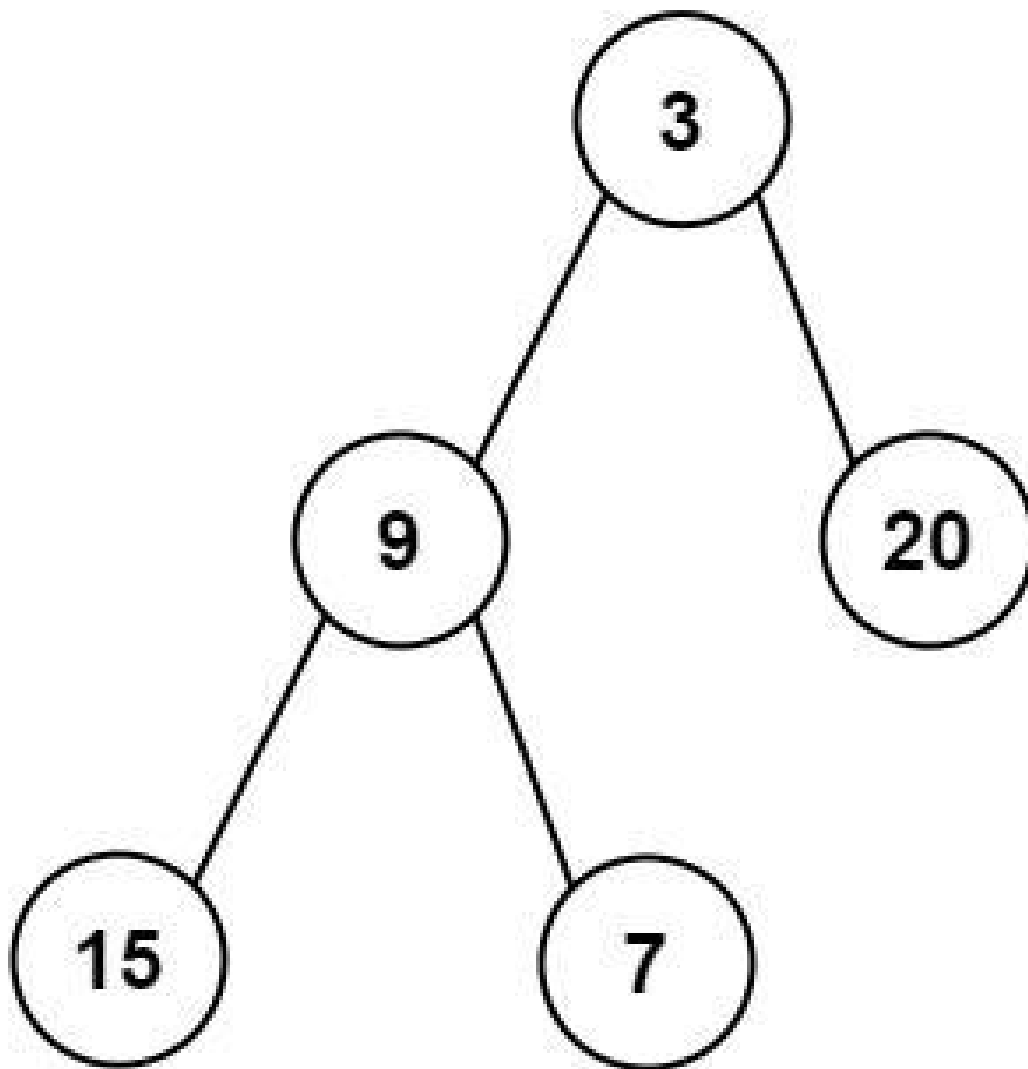
Input: root = [3,9,20,null,null,15,7]

Output: [3.00000,14.50000,11.00000]

Explanation: The average value of nodes on level 0 is 3, on level 1 is 14.5, and on level 2 is 11.

Hence return [3, 14.5, 11].

Example 2:



Input: root = [3,9,20,15,7]

Output: [3.00000,14.50000,11.00000]

Constraints:

- The number of nodes in the tree is in the range [1, 10⁴].
- $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

Community Solutions (Python)

- LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def averageOfLevels(self, root: Optional[TreeNode]) -> List[float]:
        q = deque([root])
        ans = []

        while q:
            s, n = 0, len(q)
            for _ in range(n):
                root = q.popleft()
                s += root.val
                if root.left:
                    q.append(root.left)
                if root.right:
                    q.append(root.right)
            ans.append(s / n)

        return ans

```

• SimplyLeet

```

# Python solution using BFS (level-order traversal)
from collections import deque

def averageOfLevels(root):
    # List to store the average of each level
    result = []
    # Deque for efficient popping from the left
    queue = deque([root])

    # Process until no more nodes exist in the current level

```

```
while queue:
    level_size = len(queue)
    level_sum = 0
    # Iterate through nodes on the current level
    for _ in range(level_size):
        node = queue.popleft() # Remove node from the queue
        level_sum += node.val # Add node's value to current level sum
        # Enqueue left and right children if they exist
        if node.left:
            queue.append(node.left)

        if node.right:
            queue.append(node.right)
    # Compute the average and add it to the result list
    result.append(level_sum / level_size)
return result
```

◆ Quick Review

Key Insights

- Use level-order traversal (BFS) to process each level of the tree.
- A queue helps to efficiently traverse the tree level by level.
- For each level, track the number of nodes and the cumulative sum to compute the average.
- Edge cases include handling an empty tree (though given constraints confirm at least one node).

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree because each node is processed exactly once.

Space Complexity: $O(n)$ in the worst-case (e.g., when the tree is completely balanced, the last level might contain $O(n/2)$ nodes).

Solution

The solution employs a breadth-first search (BFS) using a queue to traverse the tree level by level. For each level, it calculates the sum of node values and the count of nodes. By dividing the cumulative sum by the count, it obtains the average for that level, which is then added to the result list. This approach efficiently computes the required averages with a clear focus on level-wise processing.

Tree Traversal – Pre Order

Round 1 · High · Easy · 4 questions

Tree Traversal – Pre Order

□ #144 Binary Tree Preorder Traversal

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Tree · Depth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

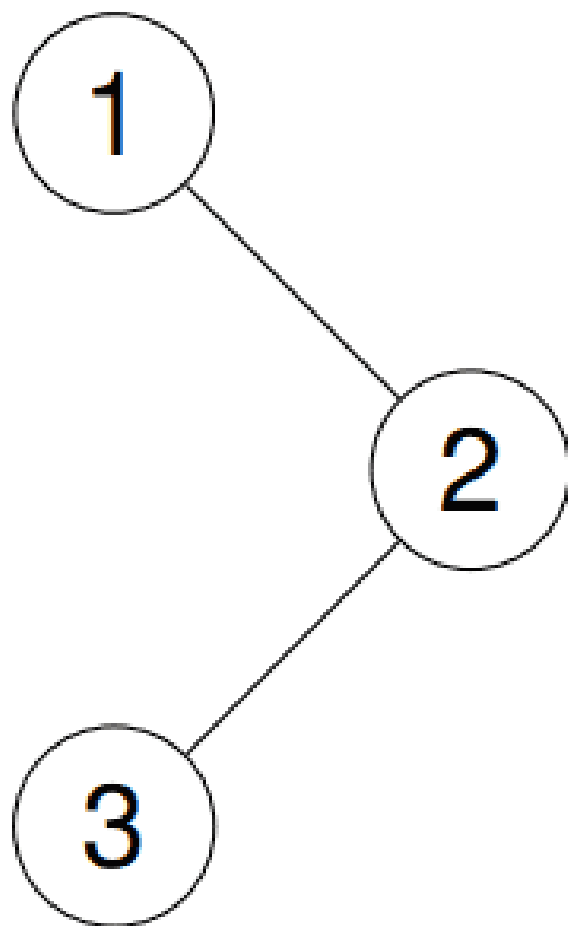
Given the root of a binary tree, return the preorder traversal of its nodes' values.

Example 1:

Input: root = [1,null,2,3]

Output: [1,2,3]

Explanation:

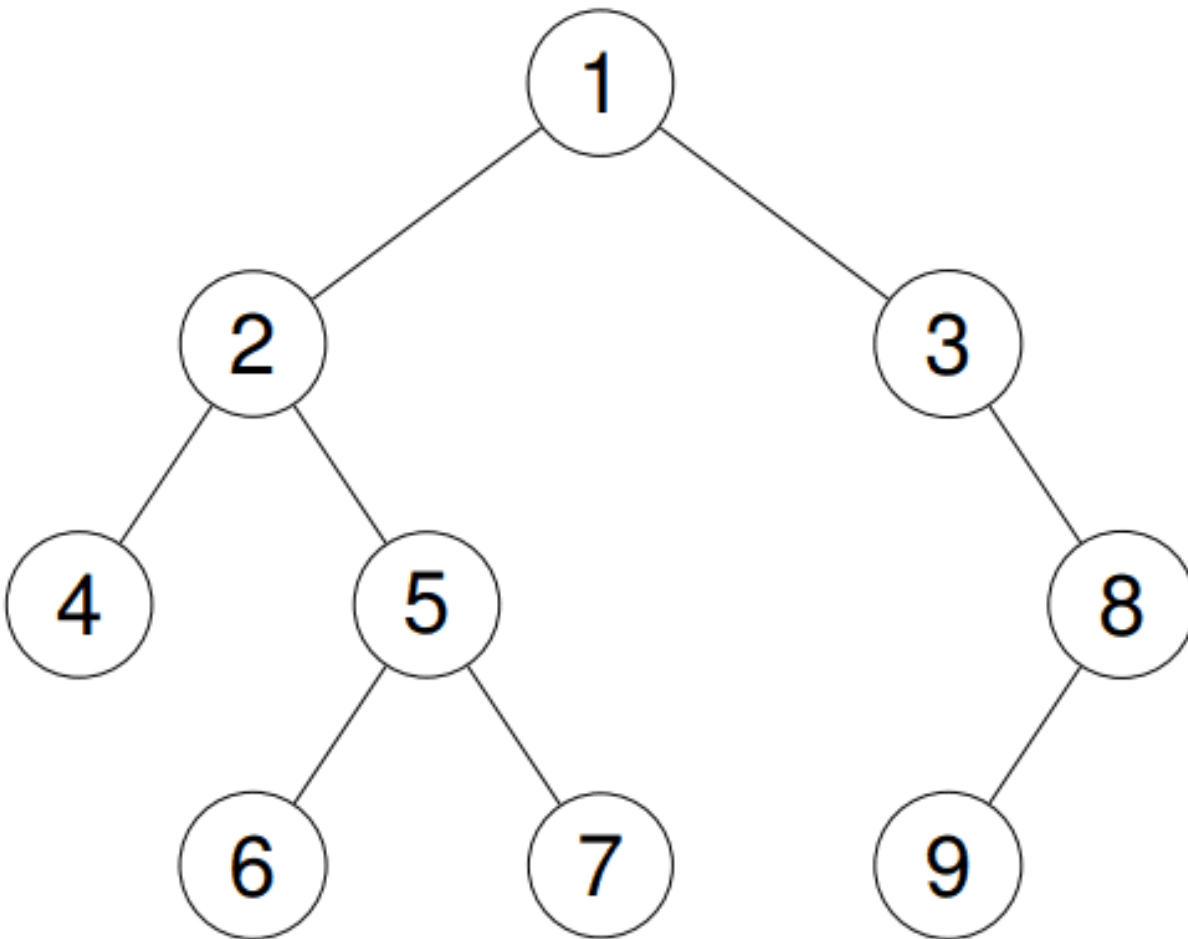


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [1,2,4,5,6,7,3,8,9]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of nodes in the tree is in the range [0, 100].

- $-100 \leq \text{Node.val} \leq 100$

Follow up: Recursive solution is trivial, could you do it iteratively?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def preorderTraversal(self, root: TreeNode | None) -> list[int]:
        ans = []

        def preorder(root: TreeNode | None) -> None:
            if not root:
                return

            ans.append(root.val)
            preorder(root.left)

            preorder(root.right)

        preorder(root)
        return ans
```

```
class Solution:
    def preorderTraversal(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = []
        stack = [root]

        while stack:
            node = stack.pop()
```

```

    ans.append(node.val)
    if node.right:
        stack.append(node.right)
    if node.left:
        stack.append(node.left)

    return ans

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def preorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(root):
            if root is None:
                return
            ans.append(root.val)
            dfs(root.left)
            dfs(root.right)

        ans = []
        dfs(root)
        return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def preorderTraversal(root):
    # List to store the preorder traversal
    result = []

```

```
# Edge case: if the tree is empty, return an empty list
if not root:
    return result

# Initialize stack with the root node
stack = [root]

# Iterate while the stack is not empty
while stack:

    # Pop the top node from the stack
    node = stack.pop()
    # Add the node's value to the result list
    result.append(node.val)

    # Push the right child first if it exists (stack LIFO ensures left is
    processed next)
    if node.right:
        stack.append(node.right)
    # Push the left child if it exists
    if node.left:
        stack.append(node.left)

# Return the result list containing preorder traversal
return result
```

◆ Quick Review

Key Insights

- Preorder traversal follows the order: Root → Left → Right.
- A recursive solution is straightforward but the problem asks for an iterative solution.

- Use a stack to simulate the recursive function call behavior.
- When using a stack, push the right child first so that left is processed before right.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes, because every node is visited once.

Space Complexity: $O(N)$ in the worst case due to the stack when the tree is unbalanced.

Solution

We use an iterative approach with a stack. Start by pushing the root to the stack. While the stack is not empty, pop the top node from the stack, record its value, and push its right child (if exists) followed by its left child (if exists) to the stack. This ensures that the left child is processed before the right child, adhering to the preorder traversal order.

Tree Traversal – Pre Order

□ #222 Count Complete Tree Nodes

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Binary Search · Bit Manipulation · Tree · Binary Tree

Lists: AlgoMaster 600

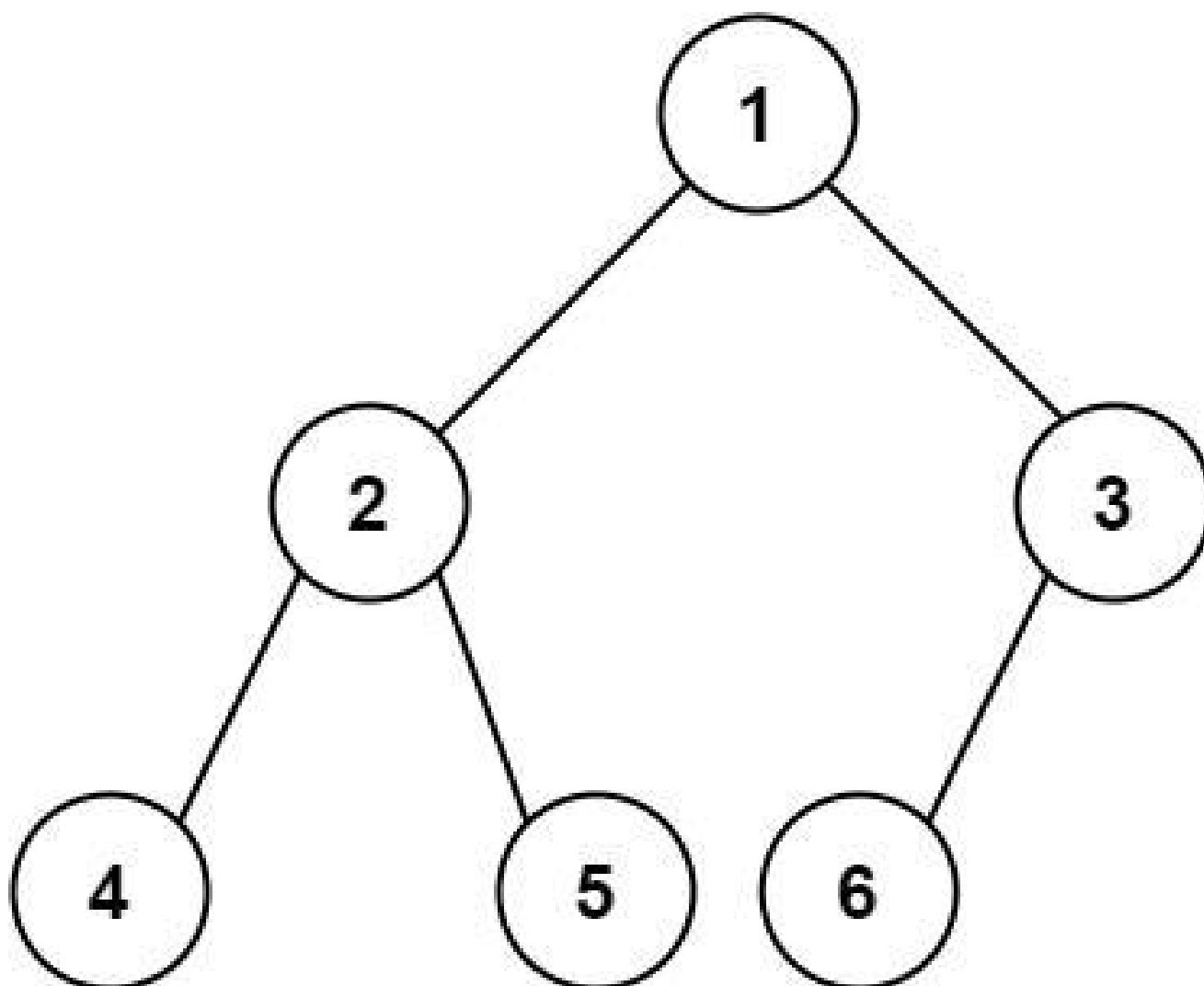
Problem

Given the root of a complete binary tree, return the number of the nodes in the tree.

According to Wikipedia, every level, except possibly the last, is completely filled in a complete binary tree, and all nodes in the last level are as far left as possible. It can have between 1 and 2^h nodes inclusive at the last level h .

Design an algorithm that runs in less than $O(n)$ time complexity.

Example 1:



Input: root = [1,2,3,4,5,6]

Output: 6

Example 2:

Input: root = []

Output: 0

Example 3:

Input: root = [1]

Output: 1

Constraints:

- The number of nodes in the tree is in the range $[0, 5 * 10^4]$.
- $0 \leq \text{Node.val} \leq 5 * 10^4$
- The tree is guaranteed to be complete.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countNodes(self, root: TreeNode | None) -> int:
        if not root:
            return 0
        return 1 + self.countNodes(root.left) + self.countNodes(root.right)
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def countNodes(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0

        return 1 + self.countNodes(root.left) + self.countNodes(root.right)
```

• SimplyLeet

```
# Python Solution

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def countNodes(root: TreeNode) -> int:

    # Helper function to compute the left depth of the tree.
    def leftDepth(node: TreeNode) -> int:
        depth = 0
        while node:
            depth += 1
            node = node.left
        return depth

    # Helper function to compute the right depth of the tree.
    def rightDepth(node: TreeNode) -> int:
        depth = 0
        while node:
            depth += 1
            node = node.right
        return depth

    if not root:
        return 0

    left_depth = leftDepth(root)

    right_depth = rightDepth(root)

    # If left and right depth are the same, tree is perfect.
    if left_depth == right_depth:
        return (1 << left_depth) - 1 # equivalent to 2^left_depth - 1
    else:
        # Recursively count nodes in left and right subtree plus one for root.
        return 1 + countNodes(root.left) + countNodes(root.right)

# Example usage:
```



```
# root = TreeNode(1, TreeNode(2), TreeNode(3))  
# print(countNodes(root))
```

◆ Quick Review

Key Insights

- A complete binary tree has properties that allow us to calculate parts of the tree quickly.
- Compute the depth of the left-most path and the right-most path:
- If they are equal, the tree is perfect and the number of nodes is $2^{\text{depth}} - 1$.
- If not equal, recursively count nodes in the left and right subtrees.
- This approach reduces the time complexity to $O((\log n)^2)$ versus a straightforward $O(n)$ traversal.

Space & Time

Time Complexity: $O((\log n)^2)$ – each level requires $O(\log n)$ comparisons and the height is $O(\log n)$.

Space Complexity: $O(\log n)$ – due to recursive stack if using recursion.

Solution

We use a recursive strategy:

- Define two helper functions to determine the depth along the left-most and right-most branches.**
- Compare these depths:**
- If equal, the tree is a perfect binary tree and the number of nodes can be calculated using the formula $2^{\text{depth}} - 1$.**
- If not equal, the tree is not fully perfect, so we recursively count the nodes in the left and right subtrees and add one for the root.**
- This algorithm uses the properties of a complete binary tree to avoid traversing every node, achieving a faster solution.**

Data Structures and Algorithms:

- Binary tree traversal (depth-first search style).**
- Recursion.**
- Bit manipulation (via power of two computation when the tree is perfect).**

Key trick: Quickly determine subtree perfection by comparing leftmost and rightmost depths.

Tree Traversal – Pre Order

□ #257 Binary Tree Paths

EAS

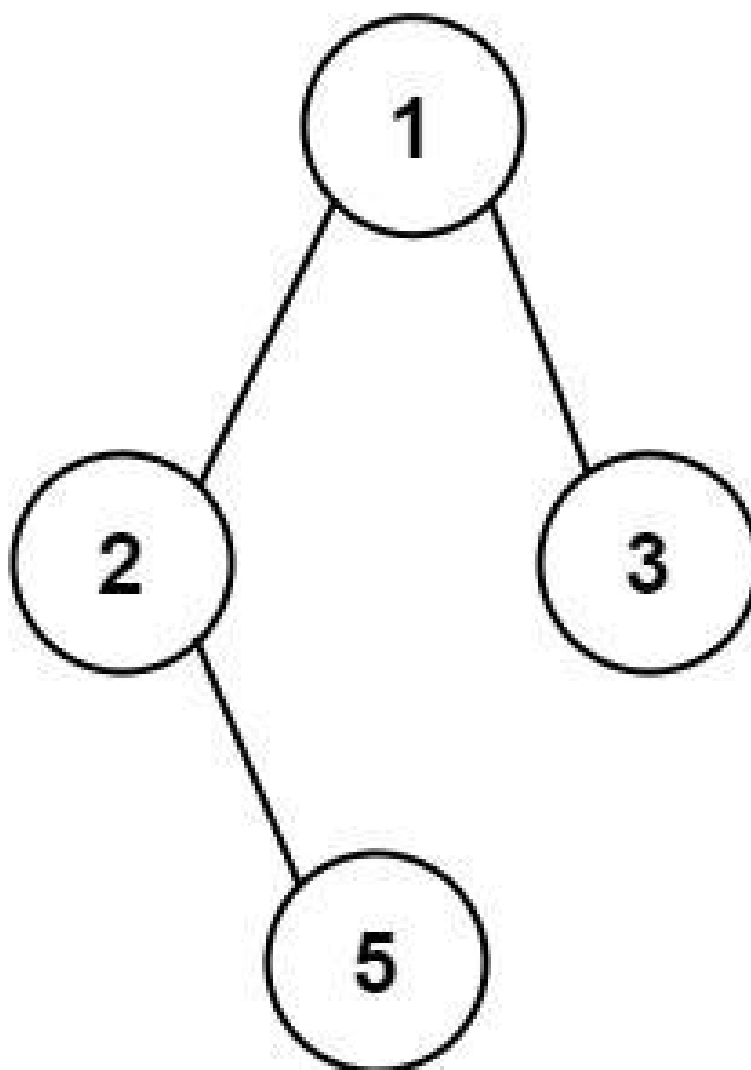
LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Backtracking · Tree · Depth-First
Search · Binary Tree
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, return all root-to-leaf paths in any order.

A leaf is a node with no children.

Example 1:



Input: root = [1,2,3,null,5]
Output: ["1->2->5","1->3"]

Example 2:

Input: root = [1]
Output: ["1"]

Constraints:

- The number of nodes in the tree is in the range [1, 100].
- $-100 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def binaryTreePaths(self, root: TreeNode | None) -> list[str]:
        ans = []

        def dfs(root: TreeNode | None, path: list[str]) -> None:
            if not root:
                return
            if not root.left and not root.right:
                ans.append(''.join(path) + str(root.val))
                return

            path.append(str(root.val) + '->')
            dfs(root.left, path)
            dfs(root.right, path)
            path.pop()

        dfs(root, [])
        return ans
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def binaryTreePaths(self, root: Optional[TreeNode]) -> List[str]:
        def dfs(root: Optional[TreeNode]):
            if root is None:
```

```

        return
    t.append(str(root.val))
    if root.left is None and root.right is None:
        ans.append("->".join(t))
    else:
        dfs(root.left)
        dfs(root.right)
    t.pop()

ans = []

t = []
dfs(root)
return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def binaryTreePaths(self, root: TreeNode) -> List[str]:
        # List to store the final root-to-leaf paths.

result = []

# Helper function to perform DFS on the tree.
def dfs(node, current_path):
    if not node:
        return
    # Append current node's value to the path.
    if current_path:
        current_path += "->" + str(node.val)
    else:

```

```
        current_path = str(node.val)
        # If the node is a leaf, add the path to result.
        if not node.left and not node.right:
            result.append(current_path)
            return
        # Continue DFS on the left and right children.
        if node.left:
            dfs(node.left, current_path)
        if node.right:
            dfs(node.right, current_path)

# Initiate DFS from the root.
dfs(root, "")
return result
```

◆ Quick Review

Key Insights

- Use a Depth-First Search (DFS) or backtracking approach to explore all paths from the root to leaves.
- Build the path as you traverse the tree.
- Once a leaf node is reached, append the constructed path to the result list.
- The problem requires returning all paths in any order.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes in the tree, since every node is visited once.

Space Complexity: $O(H)$, where H is the height of the tree, which is the space used on the recursion stack. In the worst case (skewed tree), H can be $O(N)$.

Solution

We perform a DFS traversal starting from the root. At each node, we append the node's value to the current path. If the current node is a leaf (i.e. it does not have left or right children), we add the current path (constructed as a string separated by " $->$ ") to the result. Otherwise, we recursively process the left and right children. The recursion naturally handles backtracking since the current path is passed (or extended) at each recursive call.

Tree Traversal – Pre Order

□ #404 Sum of Left Leaves

EAS

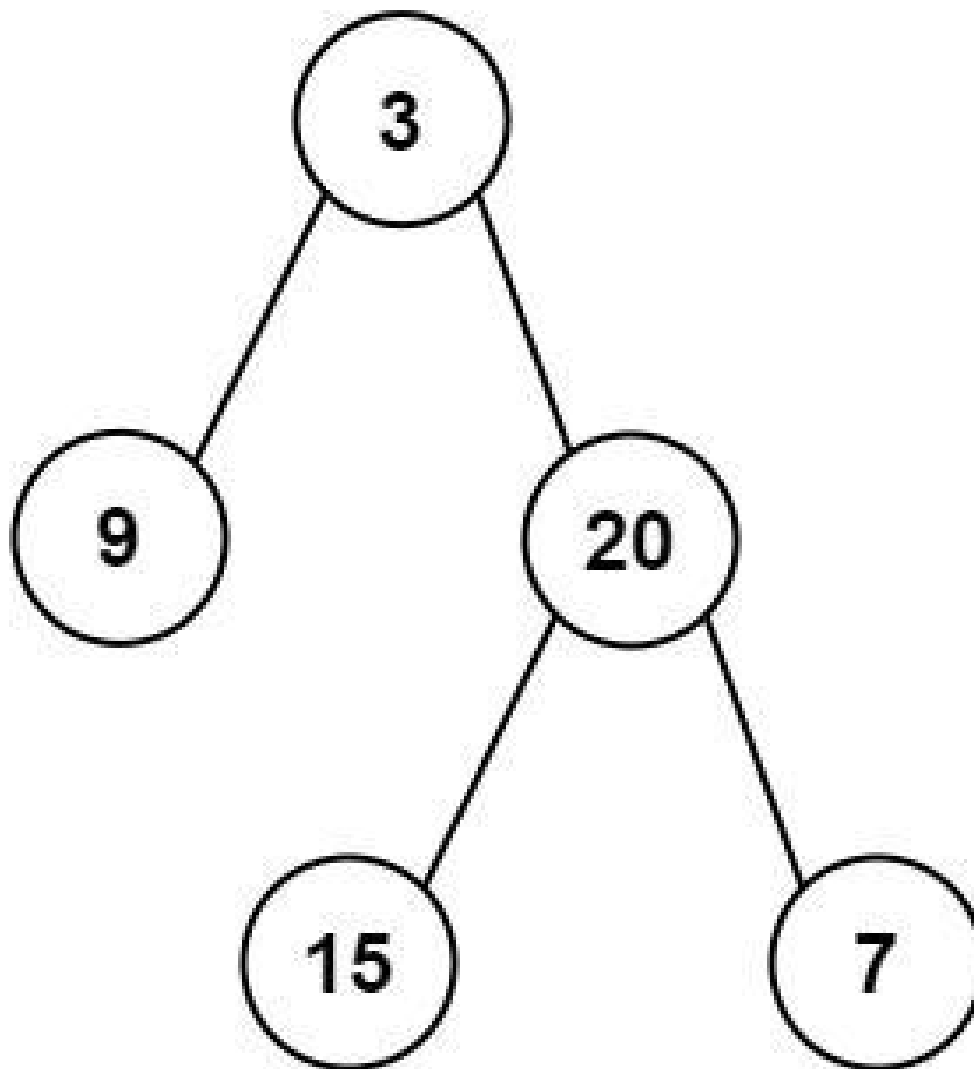
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Breadth-First
Search · Binary Tree
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, return the sum of all left leaves.

A leaf is a node with no children. A left leaf is a leaf that is the left child of another node.

Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: 24

Explanation: There are two left leaves in the binary tree, with values 9 and 15 respectively.

Example 2:

Input: root = [1]

Output: 0

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- $-1000 \leq \text{Node.val} \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sumOfLeftLeaves(self, root: TreeNode | None) -> int:
        if not root:
            return 0

        ans = 0

        if root.left:
            if not root.left.left and not root.left.right:
                ans += root.left.val

        else:
            ans += self.sumOfLeftLeaves(root.left)
            ans += self.sumOfLeftLeaves(root.right)

        return ans
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sumOfLeftLeaves(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0

        ans = self.sumOfLeftLeaves(root.right)
        if root.left:
            if root.left.left == root.left.right:
                ans += root.left.val
            else:
                ans += self.sumOfLeftLeaves(root.left)
        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def sumOfLeftLeaves(root):
    # If the node is None, return 0 as there is nothing to add.
    if not root:
        return 0
    total = 0
    # Check if the left child exists and is a leaf.
    if root.left:
        if not root.left.left and not root.left.right:
            # If it's a left leaf, add its value.
            total += root.left.val
        else:
            # Otherwise, recursively check the left subtree.
            total += sumOfLeftLeaves(root.left)

    # Always traverse the right subtree.
    total += sumOfLeftLeaves(root.right)
    return total
```

◆ Quick Review

Key Insights

- Use recursion or iteration (DFS/BFS) to traverse the binary tree.
- At each node, check if the left child is a leaf (has no children).
- Sum up the values of all left leaves.

- Ensure all nodes are checked, even if the immediate left child is not a leaf.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes in the tree, because every node is visited exactly once.

Space Complexity: $O(n)$ in the worst case, due to recursion stack space if the tree is skewed.

Solution

We use a Depth-First Search (DFS) recursive approach. At each node, check if the left child exists and is a leaf. If so, add its value to the sum. Regardless, recursively call the function for both the left and right children (if applicable) to explore the entire tree. The base case for recursion is when the current node is null. This method ensures that all nodes are properly evaluated while maintaining clarity and simplicity.

Tree Traversal – In Order

Round 1 · High · Easy · 3 questions

Tree Traversal – In Order

□ #94 Binary Tree Inorder Traversal

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Tree · Depth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

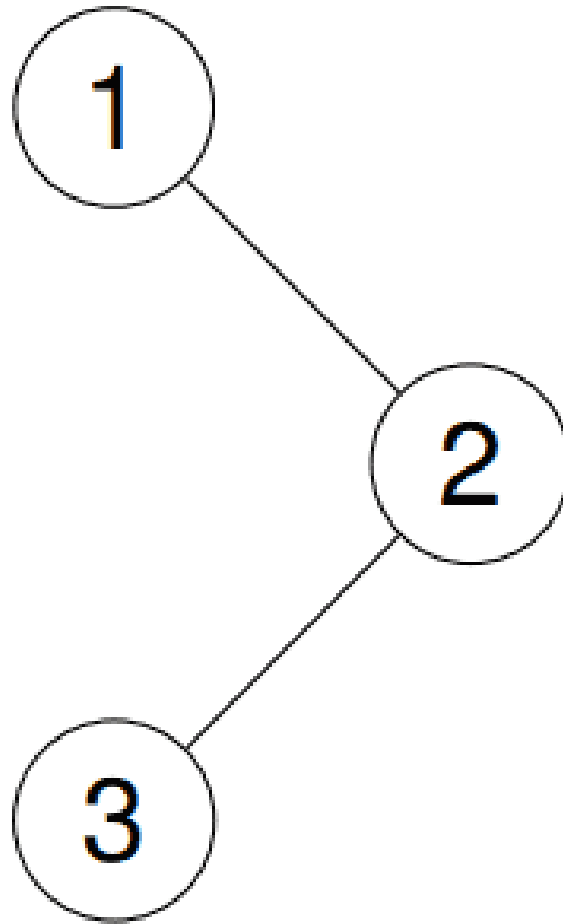
Given the root of a binary tree, return the inorder traversal of its nodes' values.

Example 1:

Input: root = [1,null,2,3]

Output: [1,3,2]

Explanation:

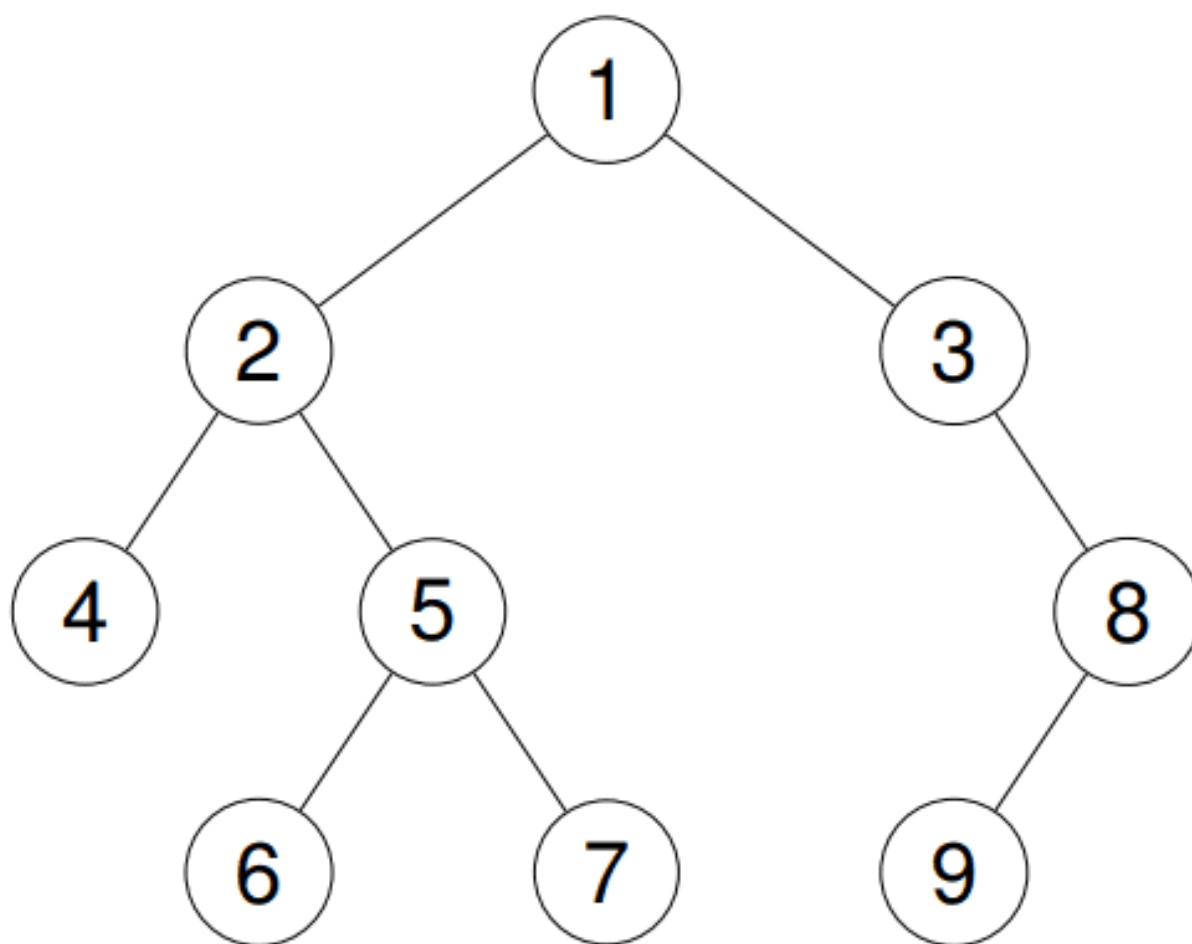


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,2,6,5,7,1,3,9,8]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of nodes in the tree is in the range [0, 100].

- $-100 \leq \text{Node.val} \leq 100$

Follow up: Recursive solution is trivial, could you do it iteratively?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def inorderTraversal(self, root: TreeNode | None) -> list[int]:
        ans = []
        stack = []

        while root or stack:
            while root:
                stack.append(root)
                root = root.left
            root = stack.pop()

            ans.append(root.val)
            root = root.right

        return ans
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(root):
            if root is None:
                return
            dfs(root.left)
            ans.append(root.val)
            dfs(root.right)
```

```

        return
    dfs(root.left)
    ans.append(root.val)
    dfs(root.right)

ans = []
dfs(root)
return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # Node value
        self.left = left        # Left child
        self.right = right       # Right child

class Solution:
    def inorderTraversal(self, root: TreeNode) -> list:
        result = []             # List to store inorder traversal

        stack = []               # Stack to simulate recursion
        current = root           # Start with the root node

        # Iterate while there are nodes to visit or process
        while current or stack:
            # Reach the left most node of the current subtree
            while current:
                stack.append(current)
                current = current.left

            # Current is null; backtrack via the stack
            current = stack.pop()
            result.append(current.val)    # Process current node
            current = current.right      # Consider right subtree

        return result

```

◆ Quick Review

Key Insights

- Inorder traversal pattern: left subtree → node → right subtree.
- An iterative solution uses a stack to simulate the recursive process.
- Continuously move to the leftmost node and store nodes in the stack.
- After reaching a null, backtrack by popping from the stack, process the node, and then explore its right subtree.
- Handle edge cases such as an empty tree.

Space & Time

Time Complexity: $O(n)$ – Every node is visited once.

Space Complexity: $O(n)$ – In the worst-case scenario (skewed tree), the stack may store all nodes.

Solution

The problem can be solved using an iterative approach that simulates the behavior of recursion with a stack. Start with the root node, then push each left child onto the stack until reaching a null node. Once you hit null,

pop the last node from the stack, process it, and then move to its right child. Repeat this process until there are no nodes left to process. This method ensures that nodes are processed in their correct inorder sequence.

Tree Traversal – In Order

□ #530 Minimum Absolute Difference in BST

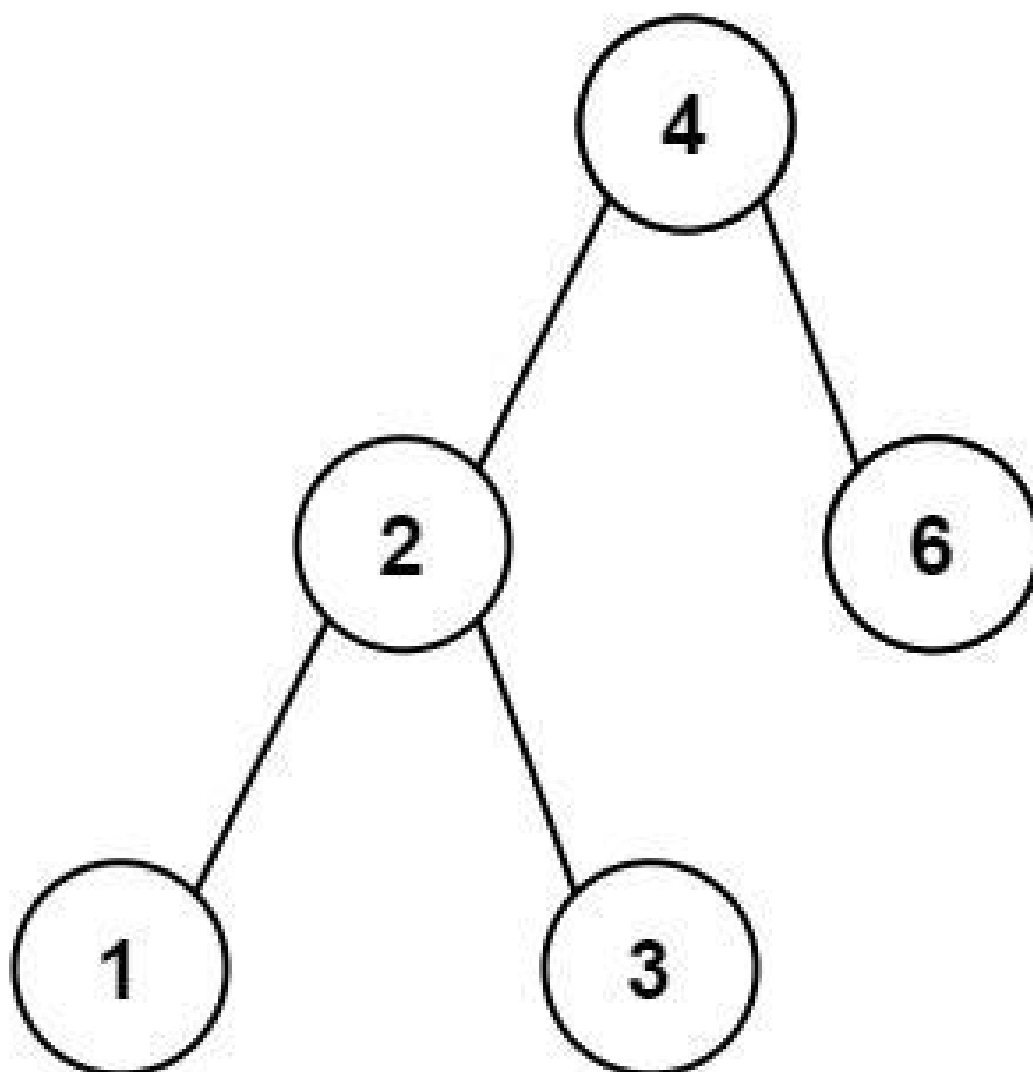
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Breadth-First
Search · Binary Search Tree · Binary Tree
Lists: AlgoMaster 600

Problem

Given the root of a Binary Search Tree (BST), return the minimum absolute difference between the values of any two different nodes in the tree.

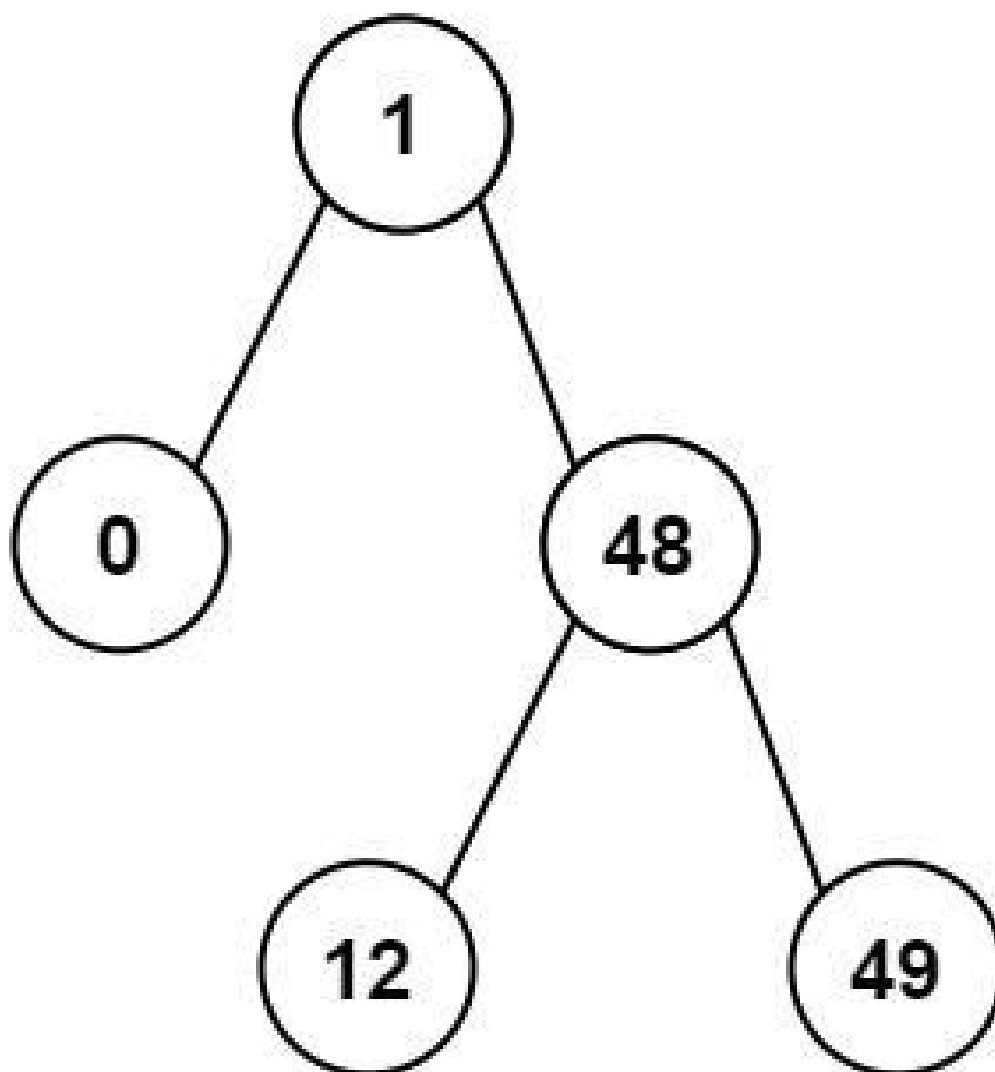
Example 1:



Input: root = [4,2,6,1,3]

Output: 1

Example 2:



Input: root = [1,0,48,null,null,12,49]

Output: 1

Constraints:

- The number of nodes in the tree is in the range [2, 104].
- $0 \leq \text{Node.val} \leq 105$

Note: This question is the same as 783: <https://leetcode.com/problems/minimum-distance-between-bst-nodes/>

Community Solutions (Python)

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def getMinimumDifference(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]):
            if root is None:
                return
            dfs(root.left)
            nonlocal pre, ans
            ans = min(ans, root.val - pre)
            pre = root.val
            dfs(root.right)

        pre = -inf
        ans = inf
        dfs(root)

        return ans
```

• SimplyLeet

```
# Define the TreeNode class for clarity
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def getMinimumDifference(self, root: TreeNode) -> int:
        # Initialize previous node value (None initially) and the minimum
        # difference as infinity
```

```
self.prev = None
self.min_diff = float('inf')

# Inorder traversal function to process nodes in sorted order
def inorder(node):
    if not node:
        return
    # Process left subtree
    inorder(node.left)
    # If there is a previous node, update the minimum difference

    if self.prev is not None:
        self.min_diff = min(self.min_diff, node.val - self.prev)
    # Update the previous node value to the current node's value
    self.prev = node.val
    # Process right subtree
    inorder(node.right)

inorder(root)
return self.min_diff
```

◆ Quick Review

Key Insights

- Inorder traversal of a BST yields the node values in sorted order.
- The minimum absolute difference will always be between two consecutive values in this sorted order.
- A single traversal (inorder) is sufficient to compute the answer by comparing the current node's value with the previous node's value.

- Using an extra variable to store the previous node's value avoids the need for extra space to store the sorted list.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the BST.

Space Complexity: $O(n)$ in the worst-case for the call stack (tree height) for a skewed tree; $O(\log n)$ on average for a balanced tree.

Solution

The solution performs an inorder traversal of the BST to retrieve the node values in sorted order. During the traversal, it keeps track of the previously visited node's value. At each node, the difference between the current node's value and the previous node's value is computed, and the minimum difference is updated accordingly. This method leverages the BST's inherent property of ordered traversal to efficiently calculate the desired result without additional data structures.

Tree Traversal – In Order

□ #783 Minimum Distance Between BST Nodes

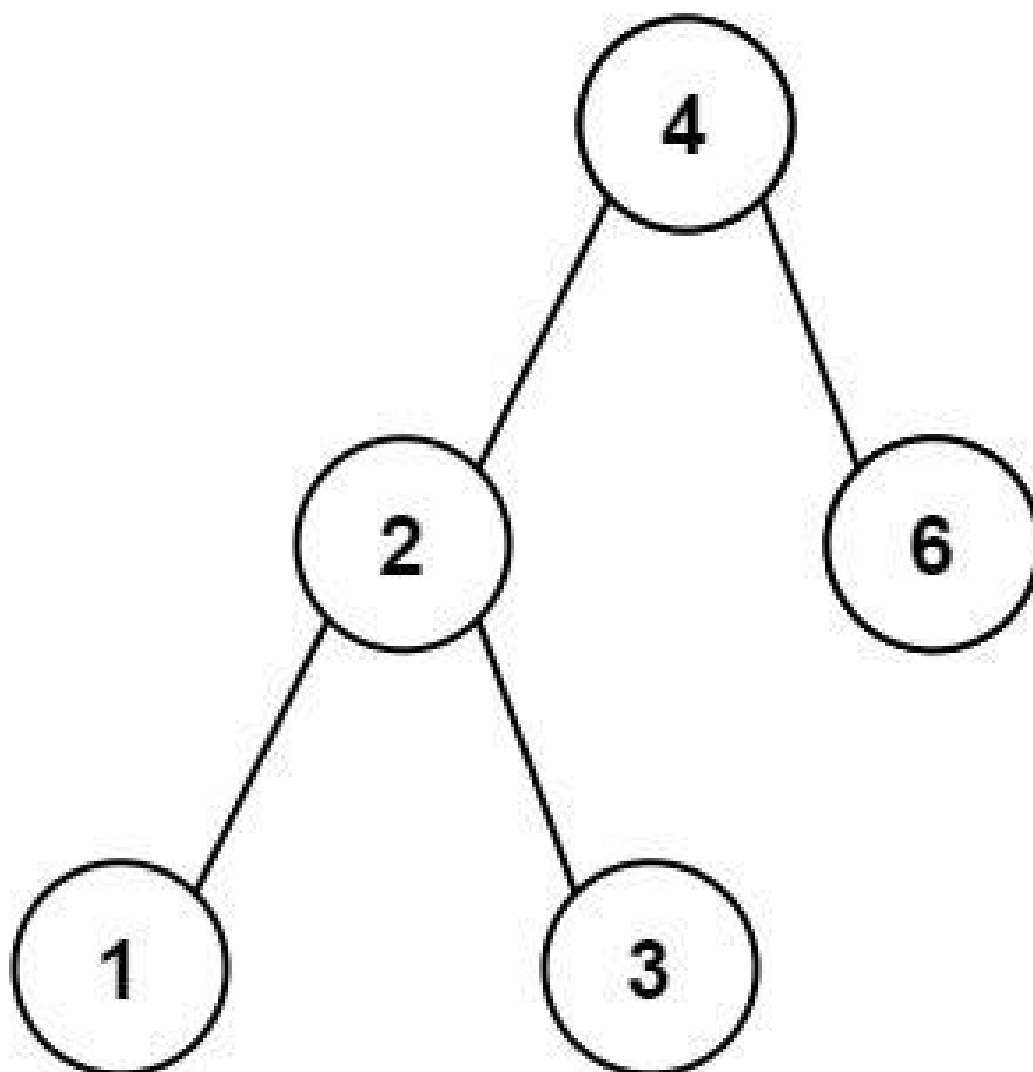
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Breadth-First
Search · Binary Search Tree · Binary Tree
Lists: AlgoMaster 600

Problem

Given the root of a Binary Search Tree (BST), return the minimum difference between the values of any two different nodes in the tree.

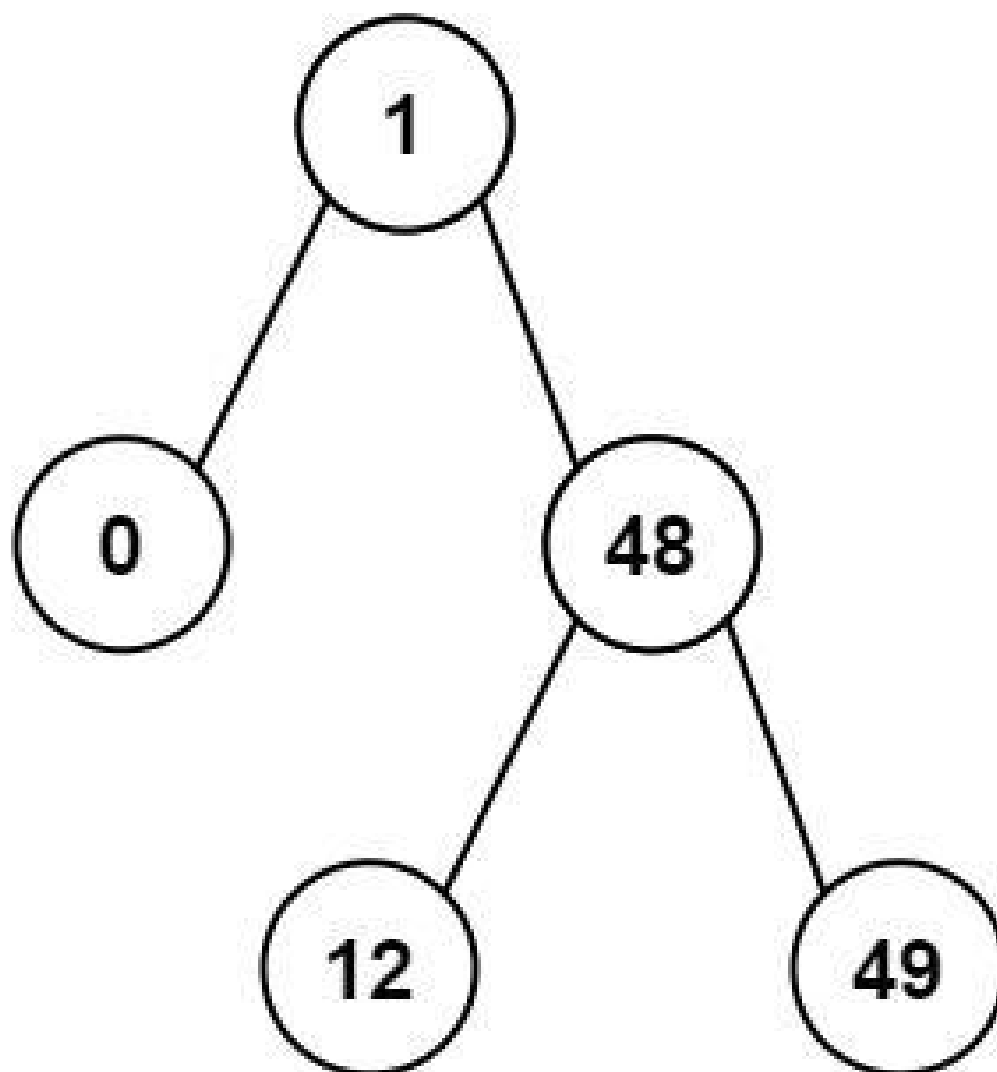
Example 1:



Input: root = [4,2,6,1,3]

Output: 1

Example 2:



Input: root = [1,0,48,null,null,12,49]

Output: 1

Constraints:

- The number of nodes in the tree is in the range [2, 100].
- $0 \leq \text{Node.val} \leq 105$

Note: This question is the same as 530: <https://leetcode.com/problems/minimum-absolute-difference-in-bst/>

Community Solutions (Python)

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def minDiffInBST(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]):
            if root is None:
                return
            dfs(root.left)
            nonlocal pre, ans
            ans = min(ans, root.val - pre)
            pre = root.val
            dfs(root.right)

        pre = -inf
        ans = inf
        dfs(root)

        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def minDiffInBST(self, root: TreeNode) -> int:
        # Initialize variables:
```

```
# prev_val keeps track of the previous node's value.
# min_diff stores the minimum difference found so far, initialized to a
large number.
self.prev_val = None
self.min_diff = float('inf')

# Define the in-order traversal function.
def in_order(node):
    if not node:
        return
    # Traverse left subtree.

    in_order(node.left)

    # Process current node:
    if self.prev_val is not None:
        # Update min_diff using the difference with previous node.
        self.min_diff = min(self.min_diff, node.val - self.prev_val)
    # Update the previous value to current node's value.
    self.prev_val = node.val

    # Traverse right subtree.

    in_order(node.right)

# Start in-order traversal from the root.
in_order(root)
return self.min_diff
```

◆ Quick Review

Key Insights

- The in-order traversal of a BST produces a sorted list of node values.
- The minimum difference will be found between two consecutive values in this sorted sequence.

- Use a variable to track the previous node value during traversal.
- Update the minimum difference as you visit each node.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the BST, since every node is visited once.

Space Complexity: $O(n)$ in the worst-case scenario due to recursion stack (or $O(h)$ where h is the tree height for balanced trees).

Solution

The solution uses an in-order traversal of the BST to get the nodes in a sorted order. As we traverse the tree, we maintain a variable to store the previous node's value. The absolute difference between the current node's value and the previous node's value is computed, and if it is smaller than the current minimum difference, it is updated. This approach leverages the BST's inherent characteristics and ensures that the minimum difference is found efficiently.

Tree Traversal – Post-Order

Round 1 · High · Easy · 1 question

Tree Traversal – Post-Order

□ #145 Binary Tree Postorder Traversal

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Tree · Depth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

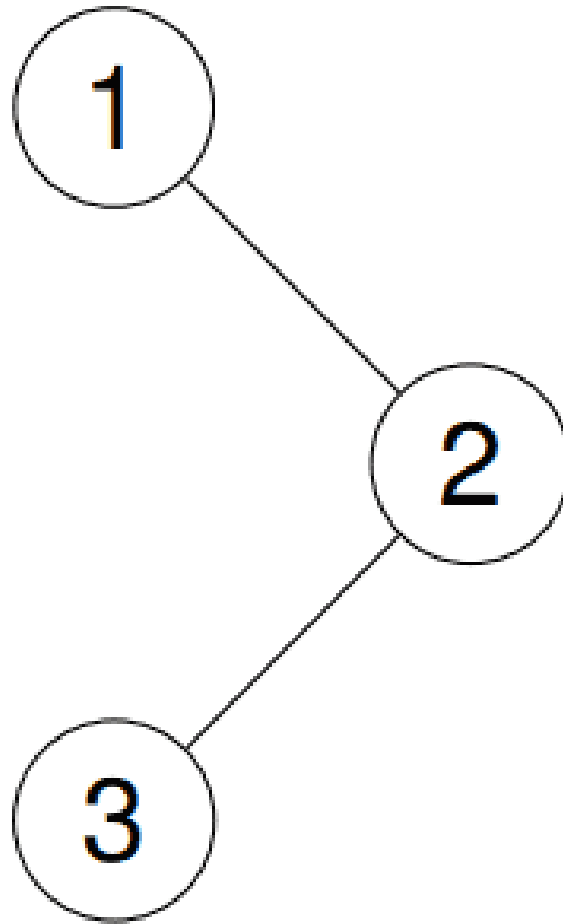
Given the root of a binary tree, return the postorder traversal of its nodes' values.

Example 1:

Input: root = [1,null,2,3]

Output: [3,2,1]

Explanation:

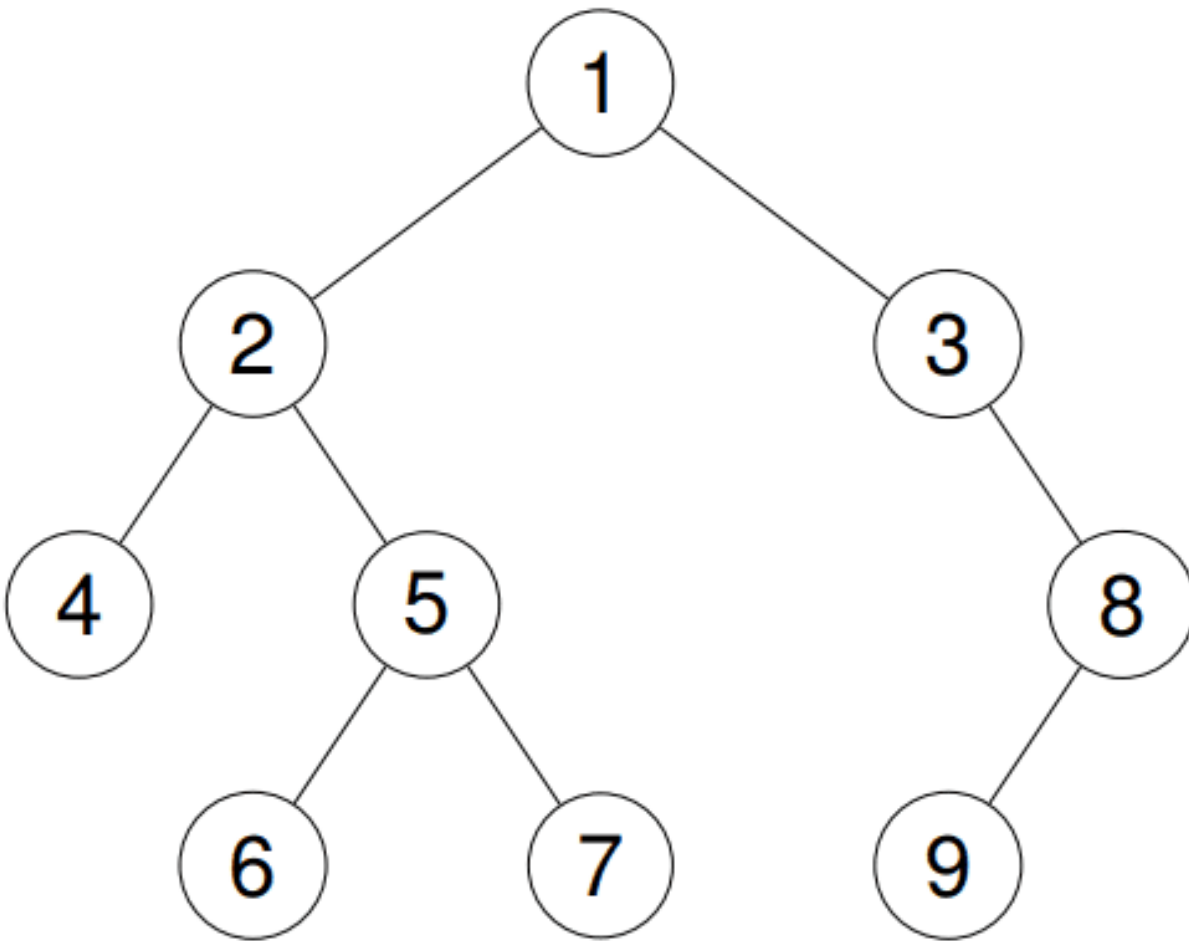


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,6,7,5,2,9,8,3,1]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of the nodes in the tree is in the range [0, 100].

- $-100 \leq \text{Node.val} \leq 100$

Follow up: Recursive solution is trivial, could you do it iteratively?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def postorderTraversal(self, root: TreeNode | None) -> list[int]:
        ans = []

        def postorder(root: TreeNode | None) -> None:
            if not root:
                return

            postorder(root.left)
            postorder(root.right)

            ans.append(root.val)

        postorder(root)
        return ans
```

```
class Solution:
    def postorderTraversal(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = []
        stack = [root]

        while stack:
            node = stack.pop()
```

```

    ans.append(node.val)
    if node.left:
        stack.append(node.left)
    if node.right:
        stack.append(node.right)

    return ans[::-1]

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def postorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(root):
            if root is None:
                return
            dfs(root.left)
            dfs(root.right)
            ans.append(root.val)

        ans = []
        dfs(root)
        return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def postorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        # If the tree is empty, return an empty list.

```

```
if not root:
    return []

# Stack for processing nodes.
stack1 = [root]
# Stack to collect nodes in reverse postorder.
stack2 = []
# Loop until all nodes are processed.
while stack1:
    # Pop a node from the first stack.

    node = stack1.pop()
    # Push the node onto the second stack.
    stack2.append(node)
    # Push the left child first (if exists).
    if node.left:
        stack1.append(node.left)
    # Push the right child (if exists).
    if node.right:
        stack1.append(node.right)

# The second stack now contains nodes in reverse of postorder.
# Extract the node values in proper postorder.
result = []
while stack2:
    result.append(stack2.pop().val)
return result
```

◆ Quick Review

Key Insights

- Use an iterative approach to avoid recursion.
- A two-stack strategy can reverse the order of node processing to simulate postorder traversal.

- One stack can be used too, though the two-stack method is intuitive: first stack for processing nodes, second stack to reverse the order.
- Edge case: if the tree is empty, return an empty list.

Space & Time

Time Complexity: $O(n)$ – Each node is processed once.

Space Complexity: $O(n)$ – In worst case, both stacks can contain up to all the nodes in the tree.

Solution

The solution uses a two-stack approach. The first stack is used for traversing the tree in a modified pre-order (root, right, left) such that when nodes are transferred into the second stack, they will be in the postorder (left, right, root) order. Finally, the values in the second stack are collected and returned as the result.

Linked List

Round 1 · High · Easy · 2 questions

Linked List

#83 Remove Duplicates from Sorted List **EAS**

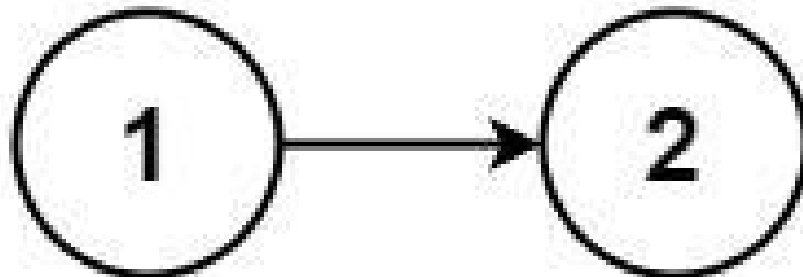
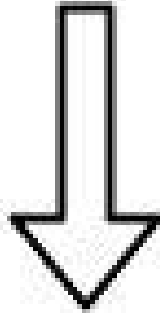
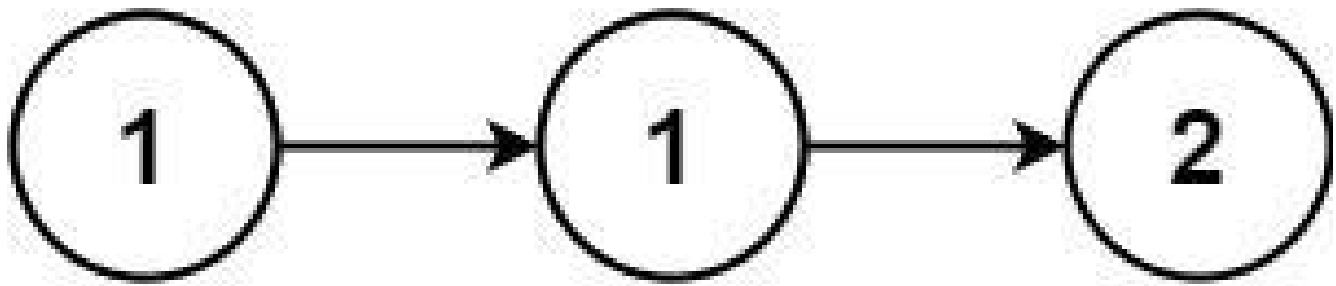
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List

Lists: AlgoMaster 600

Problem

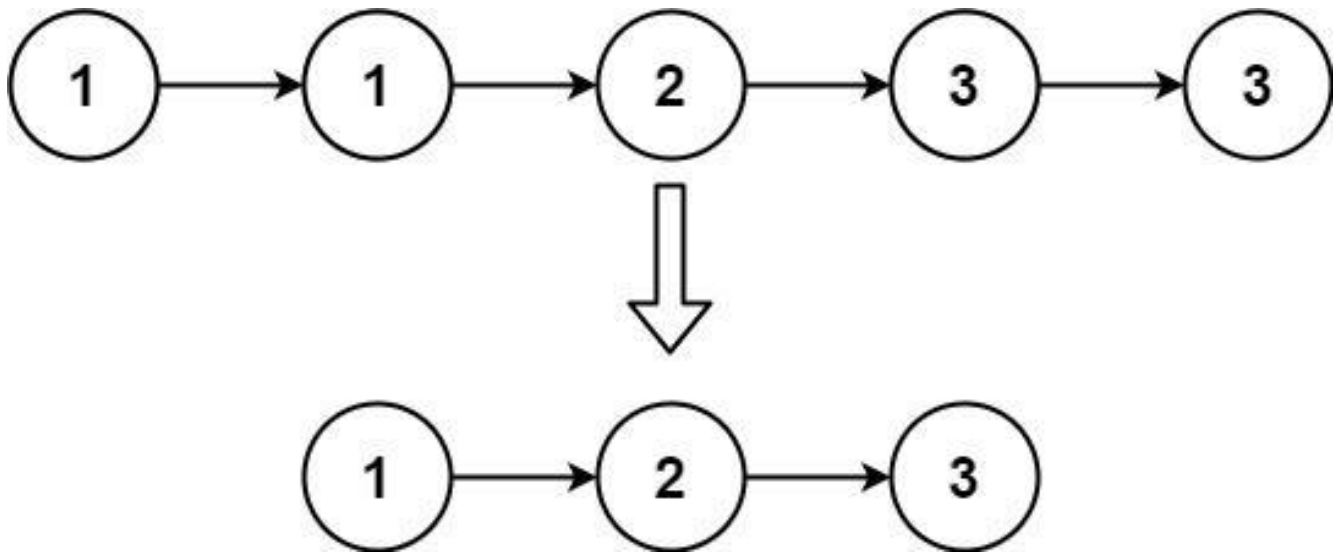
Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well.

Example 1:



Input: head = [1,1,2]
Output: [1,2]

Example 2:



Input: head = [1,1,2,3,3]

Output: [1,2,3]

Constraints:

- The number of nodes in the list is in the range [0, 300].
- $-100 \leq \text{Node.val} \leq 100$
- The list is guaranteed to be sorted in ascending order.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        curr = head

        while curr:
            while curr.next and curr.val == curr.next.val:
                curr.next = curr.next.next
            curr = curr.next

        return head
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteDuplicates(self, head: Optional[ListNode]) -> Optional[ListNode]:
        cur = head
        while cur and cur.next:
            if cur.val == cur.next.val:
                cur.next = cur.next.next
            else:
                cur = cur.next
        return head
```

• SimplyLeet

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        # Initialize current pointer to head of the list
        current = head

        # Traverse until we reach the end of the list
        while current and current.next:
            # If duplicate is found, skip the next node
            if current.val == current.next.val:
                current.next = current.next.next # Remove duplicate by
linking to the next distinct node
            else:
                current = current.next # Move to next node if current and next
values are distinct
        return head
```

◆ Quick Review

Key Insights

- The linked list is already sorted, so any duplicates will be adjacent.
- A single pass through the list is sufficient to remove duplicates.
- The algorithm modifies the list in place without needing extra memory.

Space & Time

Time Complexity: $O(n)$ – where n is the number of nodes in the list

Space Complexity: $O(1)$ – in-place modification with constant extra space

Solution

We traverse the linked list using a pointer. For each node, we check if the next node has the same value. If it does, we adjust the pointer to skip the duplicate node. Otherwise, we move our pointer to the next node. This approach efficiently removes duplicates in a single pass without requiring additional data structures.

Linked List

#160 Intersection of Two Linked Lists

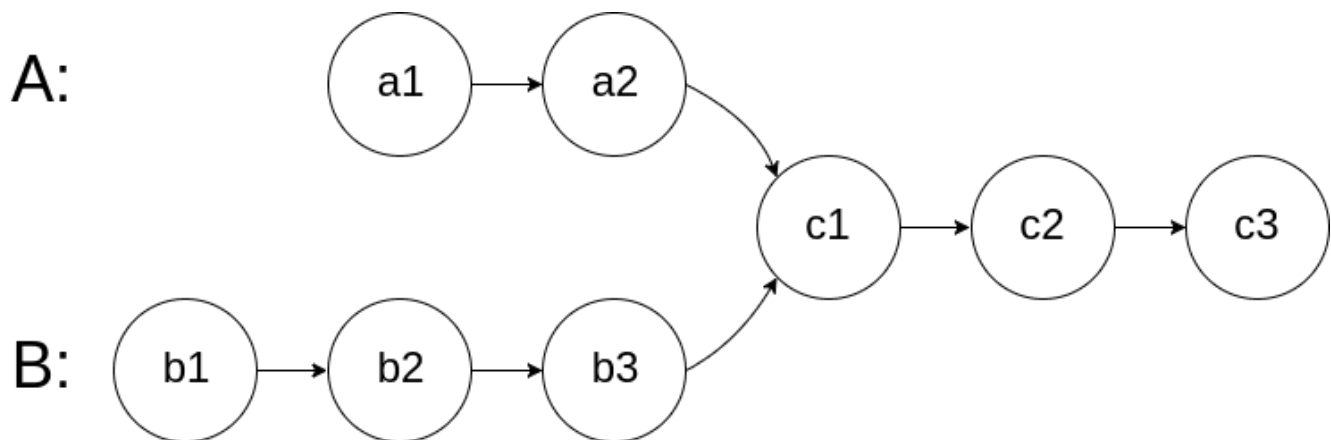
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Two Pointers
Lists: AlgoMaster 600

Problem

Given the heads of two singly linked-lists headA and headB, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return null.

For example, the following two linked lists begin to intersect at node c1:



The test cases are generated such that there are no cycles anywhere in the entire linked structure.

Note that the linked lists must retain their original structure after the function returns.

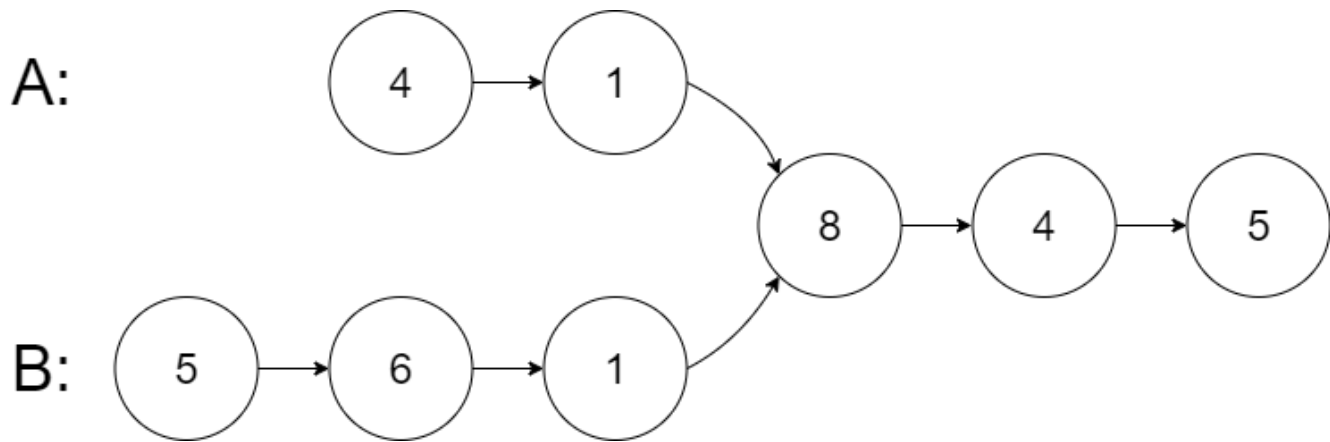
Custom Judge:

The inputs to the judge are given as follows (your program is not given these inputs):

- **intersectVal** – The value of the node where the intersection occurs. This is 0 if there is no intersected node.
- **listA** – The first linked list.
- **listB** – The second linked list.
- **skipA** – The number of nodes to skip ahead in listA (starting from the head) to get to the intersected node.
- **skipB** – The number of nodes to skip ahead in listB (starting from the head) to get to the intersected node.

The judge will then create the linked structure based on these inputs and pass the two heads, headA and headB to your program. If you correctly return the intersected node, then your solution will be accepted.

Example 1:



Input: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,6,1,8,4,5], skipA = 2, skipB = 3

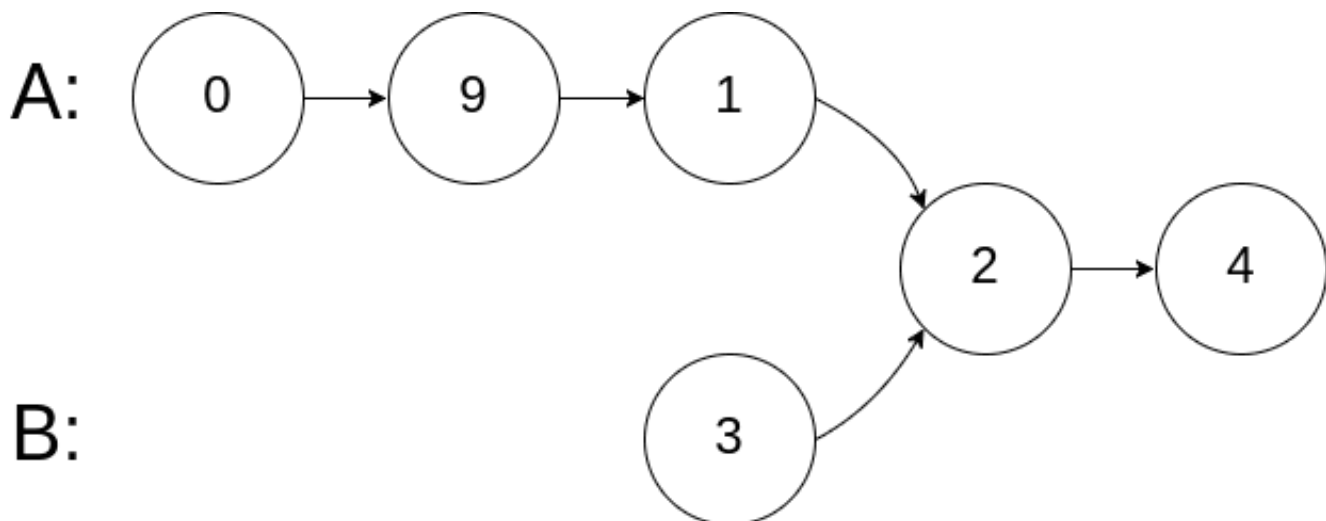
Output: Intersected at '8'

Explanation: The intersected node's value is 8 (note that this must not be 0 if the two lists intersect).

From the head of A, it reads as [4,1,8,4,5]. From the head of B, it reads as [5,6,1,8,4,5]. There are 2 nodes before the intersected node in A; There are 3 nodes before the intersected node in B.

- Note that the intersected node's value is not 1 because the nodes with value 1 in A and B (2nd node in A and 3rd node in B) are different node references. In other words, they point to two different locations in memory, while the nodes with value 8 in A and B (3rd node in A and 4th node in B) point to the same location in memory.

Example 2:



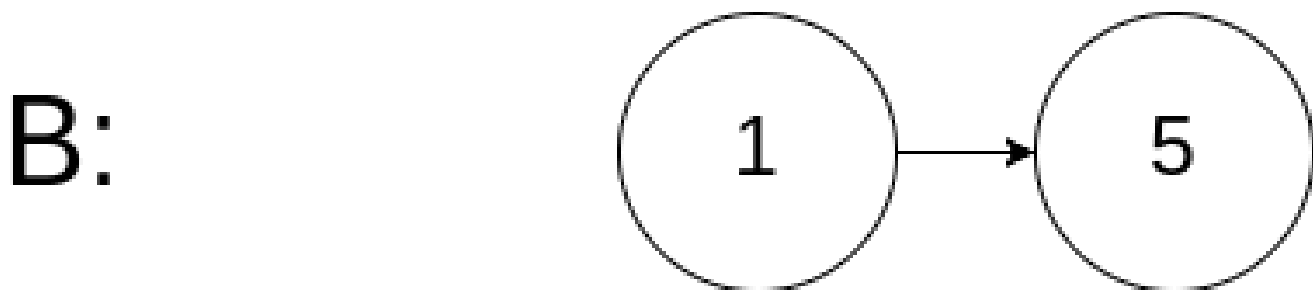
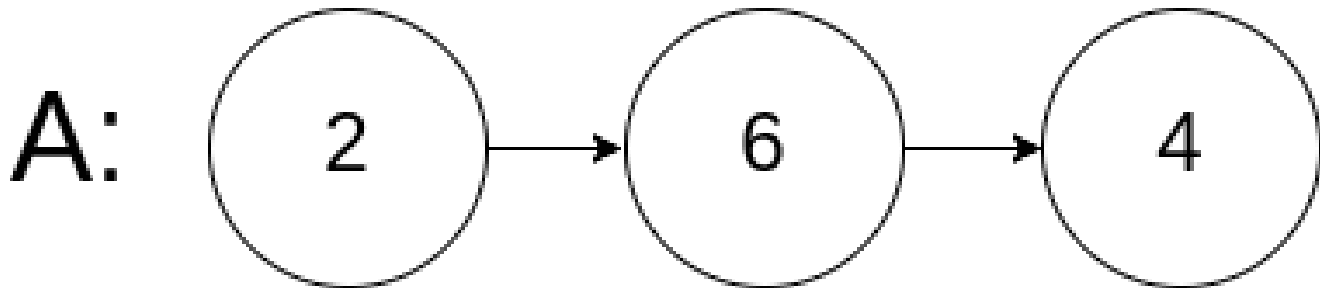
Input: intersectVal = 2, listA = [1,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1

Output: Intersected at '2'

Explanation: The intersected node's value is 2 (note that this must not be 0 if the two lists intersect).

From the head of A, it reads as [1,9,1,2,4]. From the head of B, it reads as [3,2,4]. There are 3 nodes before the intersected node in A; There are 1 node before the intersected node in B.

Example 3:



Input: intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2

Output: No intersection

Explanation: From the head of A, it reads as [2,6,4]. From the head of B, it reads as [1,5]. Since the two lists do not intersect, intersectVal must be 0, while skipA and skipB can be arbitrary values.

Explanation: The two lists do not intersect, so return null.

Constraints:

- The number of nodes of listA is in the m.
- The number of nodes of listB is in the n.

- $1 \leq m, n \leq 3 * 10^4$
- $1 \leq \text{Node.val} \leq 10^5$
- $0 \leq \text{skipA} \leq m$
- $0 \leq \text{skipB} \leq n$
- `intersectVal` is 0 if `listA` and `listB` do not intersect.
- `intersectVal == listA[skipA] == listB[skipB]` if `listA` and `listB` intersect.

Follow up: Could you write a solution that runs in $O(m + n)$ time and use only $O(1)$ memory?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def getIntersectionNode(
        self,
        headA: ListNode,
        headB: ListNode,
    ) -> ListNode | None:
        a = headA
        b = headB

        while a != b:

            a = a.next if a else headB
            b = b.next if b else headA

        return a
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) -> ListNode:
        a, b = headA, headB

        while a != b:
            a = a.next if a else headB
            b = b.next if b else headA
        return a
```

• SimplyLeet

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) -> ListNode:
        # If either list is empty, there is no intersection.
        if not headA or not headB:
            return None

        ptrA = headA # Pointer for list A
        ptrB = headB # Pointer for list B

        # Loop until the two pointers are the same.
        while ptrA != ptrB:
            # If pointer A reaches the end, switch to headB; otherwise move to the next node.
            ptrA = headB if ptrA is None else ptrA.next
            # If pointer B reaches the end, switch to headA; otherwise move to the next node.
```

```
ptrB = headA if ptrB is None else ptrB.next

# Either both pointers point to the intersection node or are both None.
return ptrA
```

◆ Quick Review

Key Insights

- The intersection point is defined by reference (i.e., the same memory location), not by the node value.
- Using two pointers that traverse both lists can help align the pointers so they traverse an equal number of nodes.
- When one pointer reaches the end of its list, it switches to the head of the other list.
- If the linked lists intersect, the pointers will eventually meet at the intersection node; if not, both will become null.

Space & Time

Time Complexity: $O(m + n)$

Space Complexity: $O(1)$

Solution

We use a two–pointer approach. Initialize two pointers, one at headA and one at headB.

Traverse the lists by moving each pointer one step at a time. When a pointer reaches the end of its list, reset it to the head of the other list. This aligns the pointers so that if there is an intersection, they will meet at the intersecting node. If not, both pointers will eventually become null, indicating no intersection. This approach runs in $O(m + n)$ time and uses constant extra space.

Round 2 · High · Medium

Round 2

High Priority · Medium

67 questions · 15 patterns

Arrays #80 #122 #229 #334 #2348

Strings #6 #38 #151 #1657

**Hash Tables #535 #659 #767 #792 #1525
#1647**

Two Pointers #18 #443 #861

Sliding Window – Fixed Size #1456 #2461

**Sliding Window – Dynamic Size #209 #395 #713
#904 #1004 #1423 #1838**

**Binary Search #81 #162 #240 #475 #1482
#1552**

Stacks #71 #316 #636 #946 #1249 #2390

Tree Traversal – Level Order #116 #117 #958

Tree Traversal – Pre Order #106 #687 #1026

Round 1 · High · Easy

p.236

- ☐ ● Tree Traversal – In Order #1382 ↗
- ☐ ● Tree Traversal – Post-Order #114 #129 #652 ↗
- ☐ ● #979 #1110 #2096 ↗
- ☐ ● Depth First Search (DFS) #690 #797 #1020 ↗
- ☐ ● #1376 ↗
- ☐ ● Breadth First Search (BFS) #433 #752 #909 ↗
- ☐ ● #1091 #1102 ↗
- ☐ ● Linked List #82 #86 #237 #445 #1669 #2095 ↗

Arrays

Round 2 · High · Medium · 5 questions

Arrays

□ #80 Remove Duplicates from Sorted Array II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers
Lists: AlgoMaster 600

Problem

Given an integer array `nums` sorted in non-decreasing order, remove some duplicates in-place such that each unique element appears at most twice. The relative order of the elements should be kept the same.

Since it is impossible to change the length of the array in some languages, you must instead have the result be placed in the first part of the array `nums`. More formally, if there are `k` elements after removing the duplicates, then the first `k` elements of `nums` should hold the final result. It does not matter what you leave beyond the first `k` elements.

Return `k` after placing the final result in the first `k` slots of `nums`.

Do not allocate extra space for another array. You must do this by modifying the input array in-place with $O(1)$ extra memory.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be accepted.

Example 1:

```
Input: nums = [1,1,1,2,2,3]
Output: 5, nums = [1,1,2,2,3,_]
Explanation: Your function should return k = 5, with the first five elements of
nums being 1, 1, 2, 2 and 3 respectively.
It does not matter what you leave beyond the returned k (hence they are
underscores).
```

Example 2:

```
Input: nums = [0,0,1,1,1,1,2,3,3]
Output: 7, nums = [0,0,1,1,2,3,3,_,_]
Explanation: Your function should return k = 7, with the first seven elements
of nums being 0, 0, 1, 1, 2, 3 and 3 respectively.
It does not matter what you leave beyond the returned k (hence they are
underscores).
```

Constraints:

- $1 \leq \text{nums.length} \leq 3 \times 10^4$

- $-104 \leq \text{nums}[i] \leq 104$
- `nums` is sorted in non-decreasing order.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeDuplicates(self, nums: list[int]) -> int:
        i = 0

        for num in nums:
            if i < 2 or num != nums[i - 2]:
                nums[i] = num
                i += 1

        return i
```

• LeetDoocs

```
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        k = 0
        for x in nums:
            if k < 2 or x != nums[k - 2]:
                nums[k] = x
                k += 1
        return k
```

• SimplyLeet

```
# Python solution for removing duplicates allowing at most two occurrences.
def removeDuplicates(nums):
    # slow pointer for the placement of the element
    slow = 0
    # iterate through each element using fast pointer
    for num in nums:
        # if less than two elements have been placed or the current number
        # is greater than the element at slow-2 (which ensures it is allowed)
        if slow < 2 or num > nums[slow - 2]:
            nums[slow] = num # place the current number at the slow pointer
            slow += 1 # increment the slow pointer
    return slow # slow is the length of the valid array

# Example usage:
# nums = [1,1,1,2,2,3]
# new_length = removeDuplicates(nums)
# print(new_length, nums[:new_length])
```

◆ Quick Review

Key Insights

- The array is already sorted, which allows for efficient duplicate detection.
- Use a two-pointer technique: one pointer (slow) to track the position for the next valid element and another pointer (fast) to iterate through the array.
- Since each element is allowed up to two occurrences, compare the current element with the element two positions before in the processed section.

- **In-place modifications ensure $O(1)$ extra space usage.**

Space & Time

Time Complexity: $O(n)$, where n is the length of the array, as we traverse the array once.

Space Complexity: $O(1)$, because the modifications are done in-place with constant extra space.

Solution

We use two pointers to solve the problem in one pass. The slow pointer holds the index where the next valid element should be placed. The fast pointer scans each element in the array.

- **For indices less than 2, we simply copy them, since they are automatically valid.**
- **For each subsequent element at index i , we compare it with the element at index $(\text{slow} - 2)$. If they are not equal, it means adding the current element will not exceed the allowed two occurrences.**
- **We then update the array in-place by writing the element at the slow pointer and incrementing the slow pointer.**

- Finally, all valid elements are stored in the first part of the array, and the slow pointer's value represents the new length.

Arrays

□ #122 Best Time to Buy and Sell Stock II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy
Lists: AlgoMaster 600

Problem

You are given an integer array `prices` where `prices[i]` is the price of a given stock on the *i*th day.

On each day, you may decide to buy and/or sell the stock. You can only hold at most one share of the stock at any time. However, you can sell and buy the stock multiple times on the same day, ensuring you never hold more than one share of the stock.

Find and return the maximum profit you can achieve.

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: 7

Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = 5-1 = 4.

Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = 6-3 = 3.

Total profit is 4 + 3 = 7.

Example 2:

Input: prices = [1,2,3,4,5]

Output: 4

Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.

Total profit is 4.

Example 3:

Input: prices = [7,6,4,3,1]

Output: 0

Explanation: There is no way to make a positive profit, so we never buy the stock to achieve the maximum profit of 0.

Constraints:

- $1 \leq \text{prices.length} \leq 3 * 10^4$
- $0 \leq \text{prices}[i] \leq 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        sell = 0
        hold = -math.inf

        for price in prices:
            sell = max(sell, hold + price)
            hold = max(hold, sell - price)

        return sell
```

• LeetDoocs

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        return sum(max(0, b - a) for a, b in pairwise(prices))
```

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        n = len(prices)
        f = [[0] * 2 for _ in range(n)]
        f[0][0] = -prices[0]
        for i in range(1, n):
            f[i][0] = max(f[i - 1][0], f[i - 1][1] - prices[i])
            f[i][1] = max(f[i - 1][1], f[i - 1][0] + prices[i])
        return f[n - 1][1]
```

• SimplyLeet

```
# Function to calculate maximum profit
def maxProfit(prices):
    profit = 0
    # Iterate over the price list starting from the first day
    for i in range(1, len(prices)):
        # If there is a rise in price from previous day, add up the profit
        if prices[i] > prices[i - 1]:
            profit += prices[i] - prices[i - 1]
    return profit
```

```
# Example usage:
prices = [7,1,5,3,6,4]
print(maxProfit(prices)) # Output: 7
```

◆ Quick Review

Key Insights

- You can make as many transactions as needed as long as you are not holding more than one share.
- The best strategy is to capture every upward movement in price.

- Instead of finding the best pair of days for each transaction, sum the differences between consecutive days that indicate an increase.
- This greedy approach avoids complex dynamic programming structures while still ensuring maximum profit.

Space & Time

Time Complexity: $O(n)$, where n is the length of the prices array, since we traverse the list once.

Space Complexity: $O(1)$, as we only need a few integer variables for tracking profit.

Solution

The solution uses a greedy approach by iterating over the prices list and adding up profits each time there is a rise from one day to the next. If $\text{prices}[i]$ is lower than $\text{prices}[i+1]$, buying on day i and selling on day $i+1$ will add to the overall profit. The idea is that accumulating all these small profits yields the maximum profit possible by making as many transactions as necessary. The approach avoids explicit tracking of all transactions, requiring only constant extra space.

Arrays

□ #229 Majority Element II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sorting · Counting
Lists: AlgoMaster 600

Problem

Given an integer array of size n , find all elements that appear more than $n/3$ times.

Example 1:

```
Input: nums = [3,2,3]
Output: [3]
```

Example 2:

```
Input: nums = [1]
Output: [1]
```

Example 3:

```
Input: nums = [1,2]
Output: [1,2]
```

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^4$
- $-109 \leq \text{nums}[i] \leq 109$

Follow up: Could you solve the problem in linear time and in $O(1)$ space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def majorityElement(self, nums: list[int]) -> list[int]:
        ans1 = 0
        ans2 = 1
        count1 = 0
        count2 = 0

        for num in nums:
            if num == ans1:
                count1 += 1

            elif num == ans2:
                count2 += 1
            elif count1 == 0:
                ans1 = num
                count1 = 1
            elif count2 == 0:
                ans2 = num
                count2 = 1
            else:
                count1 -= 1

        count2 -= 1

        return [ans for ans in (ans1, ans2) if nums.count(ans) > len(nums) // 3]
```

• LeetDoocs

```
class Solution:
    def majorityElement(self, nums: List[int]) -> List[int]:
        n1 = n2 = 0
        m1, m2 = 0, 1
        for m in nums:
            if m == m1:
                n1 += 1
            elif m == m2:
                n2 += 1
            elif n1 == 0:
```

```

        m1, n1 = m, 1
    elif n2 == 0:
        m2, n2 = m, 1
    else:
        n1, n2 = n1 - 1, n2 - 1
    return [m for m in [m1, m2] if nums.count(m) > len(nums) // 3]

```

• SimplyLeet

```

# Function to find all majority elements appearing more than n/3 times
def majorityElement(nums):
    # Initialize candidate and counters for two possible majority elements
    candidate1, candidate2 = None, None
    count1, count2 = 0, 0

    # First pass: find potential candidates
    for num in nums:
        if candidate1 == num:
            count1 += 1
        elif candidate2 == num:
            count2 += 1
        elif count1 == 0:
            candidate1 = num
            count1 = 1
        elif count2 == 0:
            candidate2 = num
            count2 = 1
        else:
            count1 -= 1
            count2 -= 1

    # Second pass: validate the candidates
    result = []
    count1 = count2 = 0
    for num in nums:
        if num == candidate1:
            count1 += 1
        elif num == candidate2:
            count2 += 1

```

```
n = len(nums)
if count1 > n // 3:
    result.append(candidate1)
if count2 > n // 3:
    result.append(candidate2)
return result

# Example usage:
print(majorityElement([3,2,3])) # Output: [3]
```

◆ Quick Review

Key Insights

- There can be at most 2 elements that appear more than $n/3$ times.
- A modified Boyer–Moore Voting Algorithm can be applied to track at most two potential candidates in one pass.
- A second pass is needed to validate the candidates count.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution uses a modified Boyer–Moore Voting Algorithm. Since an element must appear more than $n/3$ times to be valid,

there can be at most two such candidates.

- First, iterate through the array to select up to two candidates based on vote counts.**
- Then, re-iterate through the array to count the frequency of the selected candidates.**
- Finally, output the candidates that meet the threshold.**

This approach ensures linear time complexity and constant extra space.

Arrays

□ #334 Increasing Triplet Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy
Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return `true` if there exists a triple of indices (i, j, k) such that $i < j < k$ and $nums[i] < nums[j] < nums[k]$. If no such indices exists, return `false`.

Example 1:

Input: `nums = [1,2,3,4,5]`
Output: `true`
Explanation: Any triplet where $i < j < k$ is valid.

Example 2:

Input: `nums = [5,4,3,2,1]`
Output: `false`
Explanation: No triplet exists.

Example 3:

Input: `nums = [2,1,5,0,4,6]`
Output: `true`
Explanation: One of the valid triplet is $(1, 4, 5)$, because `nums[1] == 1 < nums[4] == 4 < nums[5] == 6`.

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^5$
- $-231 \leq \text{nums}[i] \leq 231 - 1$

Follow up: Could you implement a solution that runs in $O(n)$ time complexity and $O(1)$ space complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def increasingTriplet(self, nums: list[int]) -> bool:
        first = math.inf
        second = math.inf

        for num in nums:
            if num <= first:
                first = num
            elif num <= second: # first < num <= second
                second = num

        else:
            return True # first < second < num (third)

        return False
```

• LeetDoocs

```
class Solution:
    def increasingTriplet(self, nums: List[int]) -> bool:
        mi, mid = inf, inf
        for num in nums:
            if num > mid:
                return True
            if num <= mi:
                mi = num
            else:
                mid = num
```

```
return False
```

• SimplyLeet

```
def increasingTriplet(nums):  
    # Initialize first and second to be infinity  
    first = float('inf')  
    second = float('inf')  
  
    # Loop through each number in the list  
    for num in nums:  
        # Update first if current num is smaller than or equal first  
        if num <= first:  
            first = num  
  
        # Update second if current num is smaller than or equal second but  
        # greater than first  
        elif num <= second:  
            second = num  
        # If current num is greater than both first and second, a valid triplet  
        # is found  
        else:  
            return True  
    return False  
  
# Sample Test  
print(increasingTriplet([2,1,5,0,4,6])) # Expected output: True
```

◆ Quick Review

Key Insights

- You are asked to find three numbers in increasing order.
- The goal is to achieve this in $O(n)$ time and $O(1)$ space.

- Maintaining two variables for the smallest and second smallest numbers found so far allows checking if a greater third number exists.
- Each element is compared to update these candidates based on a greedy approach.

Space & Time

Time Complexity: $O(n)$ — traverse the list once

Space Complexity: $O(1)$ — only constant extra space is used

Solution

The algorithm scans through the array while keeping two variables: "first" representing the smallest value encountered so far and "second" for the next smallest value greater than "first". For each number:

- If it is less than or equal to "first", update "first".
- Else if it is less than or equal to "second", update "second".
- Otherwise, a valid triplet is found and the function returns true. This efficient approach uses the greedy method to maintain candidates for the first two numbers and

automatically finds a valid third number if it exists.

Arrays

□ #2348 Number of Zero-Filled Subarrays

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math

Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the number of subarrays filled with 0.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,3,0,0,2,0,0,4]`

Output: 6

Explanation:

There are 4 occurrences of `[0]` as a subarray.

There are 2 occurrences of `[0,0]` as a subarray.

There is no occurrence of a subarray with a size more than 2 filled with 0.

Therefore, we return 6.

Example 2:

Input: `nums = [0,0,0,2,0,0]`

Output: 9

Explanation:

There are 5 occurrences of `[0]` as a subarray.

There are 3 occurrences of `[0,0]` as a subarray.

There is 1 occurrence of `[0,0,0]` as a subarray.

There is no occurrence of a subarray with a size more than 3 filled with 0.

Therefore, we return 9.

Example 3:

Input: nums = [2,10,2019]

Output: 0

Explanation: There is no subarray filled with 0. Therefore, we return 0.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def zeroFilledSubarray(self, nums: list[int]) -> int:
        ans = 0
        indexBeforeZero = -1

        for i, num in enumerate(nums):
            if num:
                indexBeforeZero = i
            else:
                ans += i - indexBeforeZero

        return ans
```

• LeetDoocs

```
class Solution:
    def zeroFilledSubarray(self, nums: List[int]) -> int:
        ans = cnt = 0
        for x in nums:
            if x == 0:
                cnt += 1
                ans += cnt
            else:
                cnt = 0
        return ans
```


• SimplyLeet

```
# Python solution for counting zero-filled subarrays.
def zeroFilledSubarray(nums):
    total_subarrays = 0 # To hold the total count of zero-filled subarrays.
    consecutive_zeros = 0 # To count the current consecutive zeros.

    # Iterate through each number in the array.
    for num in nums:
        if num == 0:
            consecutive_zeros += 1 # Increase count if the current element is
            0.
        else:
            # When the streak of zeros ends, add the number of subarrays from
            the streak.
            total_subarrays += consecutive_zeros * (consecutive_zeros + 1) //
            2
            consecutive_zeros = 0 # Reset the counter.

    # If the array ends with a streak of zeros, count them as well.
    total_subarrays += consecutive_zeros * (consecutive_zeros + 1) // 2
    return total_subarrays

# Example usage:
print(zeroFilledSubarray([1,3,0,0,2,0,0,4])) # Output should be 6
```

◆ Quick Review

Key Insights

- A subarray is contiguous. For every sequence of consecutive 0's of length L , there are $L * (L + 1) / 2$ subarrays.
- Iterating through the array and counting consecutive zeros is an efficient approach.

- Once a non-zero element is encountered, compute the contributions from the current streak and then reset the count.
- Handle the edge case where the array ends with a sequence of zeros.

Space & Time

Time Complexity: $O(n)$, where n is the length of the array.

Space Complexity: $O(1)$, as only a few counter variables are used.

Solution

We use a simple linear scan with an integer counter to track consecutive zeros. Every time we see a zero, we increase the count. When a non-zero is encountered (or at the end of the array), we calculate the number of subarrays for that block using the formula $L * (L + 1) / 2$ and add it to our total. This way, we ensure all zero-filled subarrays are considered efficiently.

Strings

Round 2 · High · Medium · 4 questions

Strings

□ #6 Zigzag Conversion

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String

Lists: AlgoMaster 600

Problem

The string "PAYPALISHIRING" is written in a zigzag pattern on a given number of rows like this: (you may want to display this pattern in a fixed font for better legibility)

```
P A H N
A P L S I I G
Y I R
```

And then read line by line: "PAHNAPLSIIGYIR"

Write the code that will take a string and make this conversion given a number of rows:

```
string convert(string s, int numRows);
```

Example 1:

```
Input: s = "PAYPALISHIRING", numRows = 3
Output: "PAHNAPLSIIGYIR"
```

Example 2:

Input: s = "PAYPALISHIRING", numRows = 4
 Output: "PINALSIGYAHRPI"
 Explanation:
 P I N
 A L S I G
 Y A H R
 P I

Example 3:

Input: s = "A", numRows = 1
 Output: "A"

Constraints:

- $1 \leq s.length \leq 1000$
- s consists of English letters (lower-case and upper-case), ',' and '.'.
- $1 \leq numRows \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def convert(self, s: str, numRows: int) -> str:
        rows = [''] * numRows
        k = 0
        direction = (numRows == 1) - 1

        for c in s:
            rows[k] += c
            if k == 0 or k == numRows - 1:
                direction *= -1

            k += direction

        return ''.join(rows)
```

• LeetDoocs

```
class Solution:
    def convert(self, s: str, numRows: int) -> str:
        if numRows == 1:
            return s
        g = [[] for _ in range(numRows)]
        i, k = 0, -1
        for c in s:
            g[i].append(c)
            if i == 0 or i == numRows - 1:
                k = -k
            i += k
        return ''.join(chain(*g))
```

• SimplyLeet

```
def convert(s: str, numRows: int) -> str:
    # Handle the edge case when the zigzag pattern is not applicable.
    if numRows == 1 or numRows >= len(s):
        return s

    # Initialize a list to hold strings for each row.
    rows = [''] * numRows
    current_row = 0
    going_down = False

    # Loop over each character in the input string.
    for char in s:
        rows[current_row] += char
        # Reverse direction at the first or last row.
        if current_row == 0 or current_row == numRows - 1:
            going_down = not going_down
        current_row += 1 if going_down else -1

    # Concatenate all row strings to form the final result.
    return ''.join(rows)

# Example usage:
# print(convert("PAYPALISHIRING", 3))
```

◆ Quick Review

Key Insights

- The zigzag pattern can be simulated by iterating over the characters and appending them to rows.
- Maintain an index for the current row and a direction (up or down) for adding characters.
- When the direction reaches the top or bottom row, the traversal direction reverses.
- Concatenating all rows produces the final result.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(n)$, used to store characters in rows.

Solution

The solution involves simulating the zigzag writing process by iterating over the string once while keeping track of the current row and the direction of movement. An array (or list) of strings is used where each element corresponds to a row in the zigzag pattern. As

we iterate through the string:

- Append the current character to the string corresponding to the current row.**
- Reverse the direction if the current row is either the top or bottom row.**
- Adjust the current row index accordingly.**
- Finally, join all row strings to produce the output.**

This simulation approach efficiently arranges the characters and is intuitively easy to follow.

Strings

□ #38 Count and Say

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String

Lists: AlgoMaster 600

Problem

The count-and-say sequence is a sequence of digit strings defined by the recursive formula:

- $\text{countAndSay}(1) = "1"$
- $\text{countAndSay}(n)$ is the run-length encoding of $\text{countAndSay}(n - 1)$.

Run-length encoding (RLE) is a string compression method that works by replacing consecutive identical characters (repeated 2 or more times) with the concatenation of the character and the number marking the count of the characters (length of the run). For example, to compress the string "3322251" we replace "33" with "23", replace "222" with "32", replace "5" with "15" and replace "1" with "11". Thus the compressed string becomes "23321511".

Given a positive integer n , return the n th element of the count-and-say sequence.

Example 1:

Input: $n = 4$

Output: "1211"

Explanation:

```
countAndSay(1) = "1"
countAndSay(2) = RLE of "1" = "11"
countAndSay(3) = RLE of "11" = "21"
countAndSay(4) = RLE of "21" = "1211"
```

Example 2:

Input: $n = 1$

Output: "1"

Explanation:

This is the base case.

Constraints:

- $1 \leq n \leq 30$

Follow up: Could you solve it iteratively?

Community Solutions (Python)

- WalkCC

```

class Solution:
    def countAndSay(self, n: int) -> str:
        ans = '1'

        for _ in range(n - 1):
            nxt = ''
            i = 0
            while i < len(ans):
                count = 1
                while i + 1 < len(ans) and ans[i] == ans[i + 1]:

                    count += 1
                    i += 1
                nxt += str(count) + ans[i]
                i += 1
            ans = nxt

        return ans

```

• LeetCode

```

class Solution:
    def countAndSay(self, n: int) -> str:
        s = '1'
        for _ in range(n - 1):
            i = 0
            t = []
            while i < len(s):
                j = i
                while j < len(s) and s[j] == s[i]:
                    j += 1
                t.append(str(j - i))
                t.append(str(s[i]))
                i = j
            s = ''.join(t)
        return s

```

• SimplyLeet

```

# Function to generate the nth count-and-say sequence element
def countAndSay(n: int) -> str:
    # Base case: if n is 1, return "1"
    current_sequence = "1"

    # Iterate from 2 to n, building each sequence iteratively
    for _ in range(2, n + 1):
        next_sequence = "" # Initialize the next sequence
        i = 0
        # Process the current sequence using run-length encoding

        while i < len(current_sequence):
            count = 1 # We have at least one occurrence of current digit
            # Count the number of same consecutive digits
            while i + 1 < len(current_sequence) and current_sequence[i] == current_sequence[i + 1]:
                count += 1
                i += 1
            # Append the count and the digit to the next sequence
            next_sequence += str(count) + current_sequence[i]
            i += 1 # Move to the next distinct digit group
        current_sequence = next_sequence # Update current sequence for next iteration

    return current_sequence

# Example Usage:
print(countAndSay(4)) # Expected output: "1211"

```

◆ Quick Review

Key Insights

- The problem is essentially generating a sequence where each term is produced by describing the previous term.
- Use run-length encoding to count consecutive digits.

- The sequence is generated iteratively from 1 up to n .
- Iterative approach is straightforward and favored over recursion in this follow-up.

Space & Time

Time Complexity: $O(n * m)$ where m is the average length of the sequence string generated at each step.

Space Complexity: $O(m)$ for maintaining the current string and the next generated string.

Solution

We start with the base string "1" for $n=1$. For each subsequent term from 2 to n , iterate over the current string to count consecutive identical digits (run-length encoding). Append the count followed by the digit to form the next term. This iterative process continues until we reach the n th term.

Key data structures include:

- Strings for storing the current term.
 - Temporary string builders (or string concatenation) for constructing each new term.
- The algorithm uses a simple loop through the string and conditionally appends to the new

string when a digit changes, representing a group.

Strings

□ #151 Reverse Words in a String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String
Lists: AlgoMaster 600

Problem

Given an input string *s*, reverse the order of the words.

A word is defined as a sequence of non-space characters. The words in *s* will be separated by at least one space.

Return a string of the words in reverse order concatenated by a single space.

Note that *s* may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1:

```
Input: s = "the sky is blue"  
Output: "blue is sky the"
```

Example 2:

Input: `s = " hello world "`

Output: `"world hello"`

Explanation: Your reversed string should not contain leading or trailing spaces.

Example 3:

Input: `s = "a good example"`

Output: `"example good a"`

Explanation: You need to reduce multiple spaces between two words to a single space in the reversed string.

Constraints:

- $1 \leq s.length \leq 104$
- `s` contains English letters (upper-case and lower-case), digits, and spaces ' '.
- There is at least one word in `s`.

Follow-up: If the string data type is mutable in your language, can you solve it in-place with $O(1)$ extra space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverseWords(self, s: str) -> str:
        return ' '.join(reversed(s.split()))
```

• LeetDoocs


```

class Solution:
    def reverseWords(self, s: str) -> str:
        words = []
        i, n = 0, len(s)
        while i < n:
            while i < n and s[i] == " ":
                i += 1
            if i < n:
                j = i
                while j < n and s[j] != " ":
                    j += 1
                words.append(s[i:j])
                i = j
        return " ".join(words[::-1])

```

```

class Solution:
    def reverseWords(self, s: str) -> str:
        return " ".join(reversed(s.split()))

```

• SimplyLeet

```

# Python solution
def reverseWords(s: str) -> str:
    # Split the string by spaces; this automatically handles multiple spaces
    # and trimming
    words = s.split()
    # Reverse the list of words
    words.reverse()
    # Join the reversed words with a single space and return the result
    return " ".join(words)

# Example usage

if __name__ == "__main__":
    s = "  hello world "
    print(reverseWords(s)) # Expected output: "world hello"

```

◆ Quick Review

Key Insights

- Use built-in functions to trim and split the string by spaces.
- Reverse the list of words.
- Join the reversed list using a single space.
- Handle edge cases such as multiple spaces between words and leading/trailing spaces.
- In languages with mutable strings, consider an in-place algorithm for $O(1)$ extra space as a follow-up.

Space & Time

Time Complexity: $O(n)$, where n is the length of the input string.

Space Complexity: $O(n)$, to store the split words (unless an in-place algorithm is implemented).

Solution

The main idea is to clean up the string by removing extra spaces and then split the string into words. After obtaining a list of words, we simply reverse the list and join the words with a single space between them. This approach leverages string trimming, splitting, reversing, and joining, which are available in many programming languages. A potential

gotcha is ensuring that no extra spaces are left in the output, which is handled by trimming and using the join function appropriately.

Strings

□ #1657 Determine if Two Strings Are Close

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sorting · Counting
Lists: AlgoMaster 600

Problem

Two strings are considered close if you can attain one from the other using the following operations:

- Operation 1: Swap any two existing characters. For example, `abcde` → `aecdb`
- For example, `aacabb` → `bbcbaa` (all a's turn into b's, and all b's turn into a's)

You can use the operations on either string as many times as necessary.

Given two strings, `word1` and `word2`, return `true` if `word1` and `word2` are close, and `false` otherwise.

Example 1:

Input: word1 = "abc", word2 = "bca"
Output: true
Explanation: You can attain word2 from word1 in 2 operations.
Apply Operation 1: "abc" -> "acb"
Apply Operation 1: "acb" -> "bca"

Example 2:

Input: word1 = "a", word2 = "aa"
Output: false
Explanation: It is impossible to attain word2 from word1, or vice versa, in any number of operations.

Example 3:

Input: word1 = "cabbba", word2 = "abbccc"
Output: true
Explanation: You can attain word2 from word1 in 3 operations.
Apply Operation 1: "cabbba" -> "caabbb"
Apply Operation 2: "caabbb" -> "baaccc"
Apply Operation 2: "baaccc" -> "abbccc"

Constraints:

- $1 \leq \text{word1.length}, \text{word2.length} \leq 105$
- word1 and word2 contain only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def closeStrings(self, word1: str, word2: str) -> bool:
        if len(word1) != len(word2):
            return False

        count1 = collections.Counter(word1)
        count2 = collections.Counter(word2)
        if count1.keys() != count2.keys():
            return False
```

```
return sorted(count1.values()) == sorted(count2.values())
```

• LeetCode

```
class Solution:
    def closeStrings(self, word1: str, word2: str) -> bool:
        cnt1, cnt2 = Counter(word1), Counter(word2)
        return sorted(cnt1.values()) == sorted(cnt2.values()) and set(
            cnt1.keys()
        ) == set(cnt2.keys())
```

• SimplyLeet

```
# Python solution with line-by-line comments
from collections import Counter

def closeStrings(word1, word2):
    # If the lengths differ, they cannot be close.
    if len(word1) != len(word2):
        return False

    # Count frequency of each character in word1 and word2.
    counter1 = Counter(word1)

    counter2 = Counter(word2)

    # Check if both strings have the same set of characters.
    if set(counter1.keys()) != set(counter2.keys()):
        return False

    # Compare the sorted frequency distributions.
    if sorted(counter1.values()) != sorted(counter2.values()):
        return False

    return True

# Example usage:
print(closeStrings("cabbba", "abbccc")) # Expected output: True
```

◆ Quick Review

Key Insights

- The set of unique characters in both strings must be identical; if they differ, no series of transformations can make the strings equal.
- The frequency distribution (multiset of frequencies) of characters can be rearranged via transformations; hence, sorting the list of frequencies should yield the same result if the strings are close.
- Swapping can rearrange characters arbitrarily, so positions don't matter—only the counts do.
- Use hash maps (or frequency counters) to count occurrences and then compare.

Space & Time

Time Complexity: $O(n + m + k \log k)$ where n and m are the lengths of the two strings and k is the number of unique characters (at most 26).

Space Complexity: $O(k)$, for storing the frequencies.

Solution

The solution involves two main checks:

- Ensure both strings have exactly the same unique set of characters.
- Compare the sorted frequency counts of each string.

If both conditions are met, the strings can be transformed into each other using the allowed operations. Hash maps (or dictionaries) are used to count character frequencies, and sorting is applied on their values to compare frequency distributions.

Hash Tables

Round 2 · High · Medium · 6 questions

Hash Tables

□ #535 Encode and Decode TinyURL

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Design · Hash Function
Lists: AlgoMaster 600

Problem

TinyURL is a URL shortening service where you enter a URL such as

`https://leetcode.com/problems/design-tinyurl`
and it returns a short URL such as

`http://tinyurl.com/4e9iAk`. Design a class to encode a URL and decode a tiny URL.

There is no restriction on how your encode/decode algorithm should work. You just need to ensure that a URL can be encoded to a tiny URL and the tiny URL can be decoded to the original URL.

Implement the Solution class:

- **Solution()** Initializes the object of the system.

- **String encode(String longUrl)** Returns a tiny URL for the given longUrl.
- **String decode(String shortUrl)** Returns the original long URL for the given shortUrl. It is guaranteed that the given shortUrl was encoded by the same object.

Example 1:

Input: url = "https://leetcode.com/problems/design-tinyurl"
 Output: "https://leetcode.com/problems/design-tinyurl"

Explanation:

```
Solution obj = new Solution();
string tiny = obj.encode(url); // returns the encoded tiny url.
string ans = obj.decode(tiny); // returns the original url after decoding it.
```

Constraints:

- $1 \leq \text{url.length} \leq 104$
- url is guranteed to be a valid URL.

Community Solutions (Python)

• WalkCC

```
class Codec:
    alphabets = string.ascii_letters + '0123456789'
    urlToCode = {}
    codeToUrl = {}

    def encode(self, longUrl: str) -> str:
        while longUrl not in self.urlToCode:
            code = ''.join(random.choice(self.alphabets) for _ in range(6))
            if code not in self.codeToUrl:
                self.codeToUrl[code] = longUrl
```

```

        self.urlToCode[longUrl] = code
    return 'http://tinyurl.com/' + self.urlToCode[longUrl]

def decode(self, shortUrl: str) -> str:
    return self.codeToUrl[shortUrl[-6:]]

```

• LeetDoocs

```

class Codec:
    def __init__(self):
        self.m = defaultdict()
        self.idx = 0
        self.domain = 'https://tinyurl.com/'

    def encode(self, longUrl: str) -> str:
        """Encodes a URL to a shortened URL."""
        self.idx += 1
        self.m[str(self.idx)] = longUrl

        return f'{self.domain}{self.idx}'

    def decode(self, shortUrl: str) -> str:
        """Decodes a shortened URL to its original URL."""
        idx = shortUrl.split('/')[-1]
        return self.m[idx]

# Your Codec object will be instantiated and called as such:
# codec = Codec()

# codec.decode(codec.encode(url))

```

• SimplyLeet

```
import string
import random

class Solution:
    def __init__(self):
        # Dictionary to map short code to long URL
        self.url_map = {}
        # Base URL for the tiny URL
        self.base_url = "http://tinyurl.com/"

    def encode(self, longUrl: str) -> str:
        # Generate a random 6-character string
        code = ''.join(random.choices(string.ascii_letters + string.digits, k=
6))

        # Ensure the code is unique
        while code in self.url_map:
            code = ''.join(random.choices(string.ascii_letters + string.digits
, k=6))

        # Map the code to the long URL
        self.url_map[code] = longUrl
        # Return the complete tiny URL
        return self.base_url + code

    def decode(self, shortUrl: str) -> str:
        # Extract code from the short URL by removing the base URL prefix
        code = shortUrl.split(self.base_url)[-1]
        # Return the original long URL
        return self.url_map.get(code, "")
```

◆ Quick Review

Key Insights

- Use a hash table (or dictionary/map) to store the mapping between the tiny URL code and the original long URL.

- Generate a unique key (e.g., a random 6-character string) to serve as the shortened identifier.
- Handle potential collisions by regenerating the key when necessary.
- Both encoding and decoding operations should be efficient (preferably $O(1)$ time complexity).

Space & Time

Time Complexity: $O(1)$ for both encode and decode operations on average.

Space Complexity: $O(n)$, where n is the number of URLs stored.

Solution

We use a hash map to maintain the mapping between a randomly generated 6-character code and its corresponding long URL. For the encode function, a random code is generated and then concatenated with a base URL (e.g., `http://tinyurl.com/`). If the generated code already exists in the hash map (collision), we generate a new one until a unique code is obtained. The decode function simply extracts the code part from the tiny URL and looks up

the original URL in the hash map.

Hash Tables

□ #659 Split Array into Consecutive Subsequences

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Greedy · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given an integer array `nums` that is sorted in non-decreasing order.

Determine if it is possible to split `nums` into one or more subsequences such that both of the following conditions are true:

- Each subsequence is a consecutive increasing sequence (i.e. each integer is exactly one more than the previous integer).
- All subsequences have a length of 3 or more.

Return `true` if you can split `nums` according to the above conditions, or `false` otherwise.

A subsequence of an array is a new array that is formed from the original array by deleting some

(can be none) of the elements without disturbing the relative positions of the remaining elements. (i.e., [1,3,5] is a subsequence of [1,2,3,4,5] while [1,3,2] is not).

Example 1:

```
Input: nums = [1,2,3,3,4,5]
Output: true
Explanation: nums can be split into the following subsequences:
[1,2,3,3,4,5] --> 1, 2, 3
[1,2,3,3,4,5] --> 3, 4, 5
```

Example 2:

```
Input: nums = [1,2,3,3,4,4,5,5]
Output: true
Explanation: nums can be split into the following subsequences:
[1,2,3,3,4,4,5,5] --> 1, 2, 3, 4, 5
[1,2,3,3,4,4,5,5] --> 3, 4, 5
```

Example 3:

```
Input: nums = [1,2,3,4,4,5]
Output: false
Explanation: It is impossible to split nums into consecutive increasing
subsequences of length 3 or more.
```

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-1000 \leq \text{nums}[i] \leq 1000$
- `nums` is sorted in non-decreasing order.

Community Solutions (Python)

- [LeetDoocs](#)

```
class Solution:
    def isPossible(self, nums: List[int]) -> bool:
        d = defaultdict(list)
        for v in nums:
            if h := d[v - 1]:
                heappush(d[v], heappop(h) + 1)
            else:
                heappush(d[v], 1)
        return all(not v or v and v[0] > 2 for v in d.values())
```

• SimplyLeet

Python solution for splitting array into consecutive subsequences

```
from collections import Counter
```

```
def isPossible(nums):
```

```
    # Frequency map for all numbers in nums
```

```
    freq = Counter(nums)
```

```
    # Map to track the number of subsequences that can be extended (ending with a number)
```

```
    appendfreq = Counter()
```

```
    for num in nums:
```

```
        if freq[num] == 0:
```

```
            # This number has already been used completely, skip it
```

```
            continue
```

```
        # Decrement the frequency of current num as we are going to use it
```

```
        freq[num] -= 1
```

```
        if appendfreq[num] > 0:
```

```
            # If there's a subsequence ending with num-1, extend this subsequence with num
```

```

        appendfreq[num] -= 1
        # Now, this subsequence can be extended with num+1 in the future
        appendfreq[num + 1] += 1
    elif freq[num + 1] > 0 and freq[num + 2] > 0:
        # If num cannot extend a subsequence, try to create a new
subsequence of length at least 3
        freq[num + 1] -= 1
        freq[num + 2] -= 1
        # The new subsequence now can be extended with num+3
        appendfreq[num + 3] += 1
    else:

        # Neither extending nor starting a new valid subsequence is
possible
        return False
    return True

# Example usage:
print(isPossible([1,2,3,3,4,5])) # Expected output: True

```

◆ Quick Review

Key Insights

- Use a greedy approach to form the subsequences.
- Utilize a frequency map to track how many times each number appears.
- Use an additional map to track how many subsequences ending with a certain number need to be extended.
- For each number, either extend an existing subsequence or start a new one if possible.

- If neither is possible, then it is not possible to form valid subsequences.

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in `nums`

Space Complexity: $O(n)$ for storing the frequency and subsequence extension maps

Solution

We first count the frequency of every number in the array. Then, we iterate through the array and try to place each number:

- If a number is already part of an existing subsequence (tracked via the extension map), then extend that subsequence.
- Otherwise, check if the number can start a new subsequence (i.e., the next two consecutive numbers are available).
- If neither condition holds, return false immediately. This greedy algorithm ensures that subsequences are extended whenever possible, helping to avoid wasteful starts of new subsequences that might block later extensions.

Hash Tables

□ #767 Reorganize String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Greedy · Sorting · Heap
(Priority Queue) · Counting
Lists: AlgoMaster 600

Problem

Given a string s , rearrange the characters of s so that any two adjacent characters are not the same.

Return any possible rearrangement of s or return "" if not possible.

Example 1:

Input: $s = \text{"aab"}$
Output: "aba"

Example 2:

Input: $s = \text{"aaab"}$
Output: $\text{"}"$

Constraints:

- $1 \leq s.length \leq 500$
- s consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reorganizeString(self, s: str) -> str:
        count = collections.Counter(s)
        if max(count.values()) > (len(s) + 1) // 2:
            return ''

        ans = []
        maxHeap = [(-freq, c) for c, freq in count.items()]
        heapq.heapify(maxHeap)
        prevFreq = 0

        prevChar = '@'

        while maxHeap:
            # Get the letter with the maximum frequency.
            freq, c = heapq.heappop(maxHeap)
            ans.append(c)
            # Add the previous letter back s.t. any two adjacent characters are not
            # the same.
            if prevFreq < 0:
                heapq.heappush(maxHeap, (prevFreq, prevChar))

            prevFreq = freq + 1
            prevChar = c

        return ''.join(ans)

class Solution:
    def reorganizeString(self, s: str) -> str:
        n = len(s)
        count = collections.Counter(s)
        maxCount = max(count.values())

        if maxCount > (n + 1) // 2:
            return ''

        if maxCount == (n + 1) // 2:
```

```

maxLetter = max(count, key=count.get)
ans = [maxLetter if i % 2 == 0 else '' for i in range(n)]
del count[maxLetter]
i = 1
else:
    ans = [''] * n
    i = 0

for c, freq in count.items():
    for _ in range(freq):

```

```

        ans[i] = c
        i += 2
        if i >= n:
            i = 1

return ''.join(ans)

```

• LeetCode

```

class Solution:
    def reorganizeString(self, s: str) -> str:
        n = len(s)
        cnt = Counter(s)
        mx = max(cnt.values())
        if mx > (n + 1) // 2:
            return ''

        i = 0
        ans = [None] * n
        for k, v in cnt.most_common():

            while v:
                ans[i] = k
                v -= 1
                i += 2
                if i >= n:
                    i = 1
            return ''.join(ans)

```

```

class Solution:
    def reorganizeString(self, s: str) -> str:
        return self.rearrangeString(s, 2)

    def rearrangeString(self, s: str, k: int) -> str:
        h = [(-v, c) for c, v in Counter(s).items()]
        heapify(h)
        q = deque()
        ans = []
        while h:
            v, c = heappop(h)
            v *= -1
            ans.append(c)
            q.append((v - 1, c))
            if len(q) >= k:
                w, c = q.popleft()
                if w:
                    heappush(h, (-w, c))
        return "" if len(ans) != len(s) else "".join(ans)

```

• SimplyLeet

```

# Python solution using heapq. The heap is simulated as a max heap by storing
negative frequencies.
import heapq

def reorganizeString(s: str) -> str:
    # Count frequency of each character.
    frequency = {}
    for char in s:
        frequency[char] = frequency.get(char, 0) + 1

    # Check if reorganization is possible.

```



```

max_allowed = (len(s) + 1) // 2
for count in frequency.values():
    if count > max_allowed:
        return ""

# Build a max heap based on frequency.
max_heap = [(-count, char) for char, count in frequency.items()]
heapq.heapify(max_heap)

result = []

```

```

prev_count, prev_char = 0, ''

# Process the heap.
while max_heap:
    count, char = heapq.heappop(max_heap)
    # Append the current character to the result.
    result.append(char)

    # If there is a previous character leftover, push it back into the
    heap.
    if prev_count < 0:

```

```

        heapq.heappush(max_heap, (prev_count, prev_char))

    # Update previous character info: decrement the count.
    count += 1 # since count is negative, incrementing moves it towards
    zero.
    prev_count, prev_char = count, char

    return "".join(result)

# Example usage:
print(reorganizeString("aab")) # Expected Output: "aba"

print(reorganizeString("aaab")) # Expected Output: ""

```

◆ Quick Review

Key Insights

- Count the frequency of each character.
- A valid reorganization is impossible if any character's frequency exceeds half of the string length (rounded up).
- Use a max heap (priority queue) to always choose the character with the highest remaining frequency.
- Place the most frequent character and then pick the next most frequent; add the previous character back if it still has remaining frequency.
- This greedy strategy ensures that the most frequent characters are always spaced apart.

Space & Time

Time Complexity: $O(n \log k)$, where n is the length of the string and k is the number of unique characters.

Space Complexity: $O(k)$ for the frequency map and heap.

Solution

The solution begins by counting the frequency of each character in the string. If any character appears more than $(n + 1) / 2$ times, then a valid arrangement is impossible.

Next, use a max heap to always select the character with the highest remaining frequency. Repeatedly extract the top two characters from the heap, append them to the result, and then reduce their frequency. If a character still has a positive frequency after decrementing, push it back into the heap. This greedy strategy guarantees that no two adjacent characters are the same, because by always pairing the most frequent remaining characters, they are distributed as evenly as possible.

Hash Tables

□ #792 Number of Matching Subsequences **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Binary Search ·
Dynamic Programming · Trie · Sorting
Lists: AlgoMaster 600

Problem

Given a string *s* and an array of strings *words*, return the number of *words[i]* that is a subsequence of *s*.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

Example 1:

Input: *s* = "abcde", *words* = ["a","bb","acd","ace"]

Output: 3

Explanation: There are three strings in *words* that are a subsequence of *s*: "a", "acd", "ace".

Example 2:

Input: s = "dsahjppjauf", words = ["ahjppjau","ja","ahbwzggqnuuk","tnmlanowax"]
 Output: 2

Constraints:

- $1 \leq s.length \leq 5 * 10^4$
- $1 \leq words.length \leq 5000$
- $1 \leq words[i].length \leq 50$
- s and words[i] consist of only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numMatchingSubseq(self, s: str, words: list[str]) -> int:
        root = {}

        def insert(word: str) -> None:
            node = root
            for c in word:
                if c not in node:
                    node[c] = {'count': 0}
                node = node[c]

            node['count'] += 1

        for word in words:
            insert(word)

        def dfs(s: str, i: int, node: dict) -> int:
            ans = node['count'] if 'count' in node else 0

            if i >= len(s):
                return ans
```

```

        for c in string.ascii_lowercase:
            if c in node:
                try:
                    index = s.index(c, i)
                    ans += dfs(s, index + 1, node[c])
                except ValueError:
                    continue

        return ans

return dfs(s, 0, root)

class Solution:
    def numMatchingSubseq(self, s: str, words: list[str]) -> int:
        ans = 0
        # [(i, j)] := words[i] and the letter words[i][j] is waiting for
        bucket = [[] for _ in range(26)]

        # For each word, it's waiting for word[0].
        for i, word in enumerate(words):
            bucket[ord(word[0]) - ord('a')].append((i, 0))

        for c in s:
            # Let prevBucket = bucket[c] and clear bucket[c].
            index = ord(c) - ord('a')
            prevBucket = bucket[index]
            bucket[index] = []
            for i, j in prevBucket:
                j += 1
                if j == len(words[i]): # All the letters in words[i] are matched.
                    ans += 1
                else:
                    bucket[ord(words[i][j]) - ord('a')].append((i, j))

        return ans

```

• LeetDoocs

```

class Solution:
    def numMatchingSubseq(self, s: str, words: List[str]) -> int:
        d = defaultdict(deque)
        for w in words:
            d[w[0]].append(w)
        ans = 0
        for c in s:
            for _ in range(len(d[c])):
                t = d[c].popleft()
                if len(t) == 1:
                    ans += 1
            else:
                d[t[1]].append(t[1:])
        return ans

```

```

class Solution:
    def numMatchingSubseq(self, s: str, words: List[str]) -> int:
        def check(w):
            i = -1
            for c in w:
                j = bisect_right(d[c], i)
                if j == len(d[c]):
                    return False
                i = d[c][j]
            return True

        d = defaultdict(list)
        for i, c in enumerate(s):
            d[c].append(i)
        return sum(check(w) for w in words)

```

• SimplyLeet

```
# Python solution with line-by-line comments
from collections import defaultdict, deque

def numMatchingSubseq(s, words):
    # Dictionary mapping from character to a deque of iterators over words
    waiting = defaultdict(deque)

    # Initialize waiting lists with an iterator for each word starting with its
    # first character
    for word in words:
        # Append the word itself as the sequence, waiting for its first
        # character (or next character pointer)

        waiting[word[0]].append(word)

    count = 0 # To count the number of words that are subsequences
    # Iterate over each character in s
    for char in s:
        # Access the list of words waiting for the current character
        current_bucket = waiting[char]
        # Clear the bucket as we are going to process all the current waiting
        # words
        waiting[char] = deque()

        # Process each word that was waiting for the current character
        while current_bucket:
            word = current_bucket.popleft()
            # If the word length is 1, then current character was the final
            # needed character
            if len(word) == 1:
                count += 1 # Word is fully matched
            else:
                # Move to the next character in the word and add it to the
                # appropriate bucket
                waiting[word[1]].append(word[1:])
```



```
    return count

# Example usage:
s = "abcde"
words = ["a", "bb", "acd", "ace"]
print(numMatchingSubseq(s, words)) # Expected output: 3
```

◆ Quick Review

Key Insights

- Instead of checking each word individually against s , leverage the fact that s is fixed and iterate through it to "advance" pointers for each word.
- Use a mapping (or bucket) from characters to lists of iterators/pointers representing the next character needed by each word.
- As we iterate over s , update the waiting lists and count when a word is fully matched.
- This approach avoids redundant scanning and significantly reduces time complexity.

Space & Time

Time Complexity: $O(n + m)$ where n is the length of s and m is the total number of characters in all words

Space Complexity: $O(m)$ for storing the waiting pointers for each character in all words

Solution

We initialize a dictionary (hash map) that maps each letter (a–z) to a list of word iterators or pointers. Each word in words is associated with its starting character. Then, iterate through each character in s. For the current character, retrieve and clear its waiting list and advance each word's pointer. If a word is completely matched (i.e., all its characters have been processed), increment the result counter. Otherwise, push the word pointer into the bucket corresponding to its next required character. This technique efficiently checks for subsequences in a single pass over s with minimal repeated scanning.

Hash Tables

□ #1525 Number of Good Ways to Split a String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Dynamic Programming ·
Bit Manipulation · Prefix Sum
Lists: AlgoMaster 600

Problem

You are given a string s .

A split is called good if you can split s into two non-empty strings s_{left} and s_{right} where their concatenation is equal to s (i.e., $s_{left} + s_{right} = s$) and the number of distinct letters in s_{left} and s_{right} is the same.

Return the number of good splits you can make in s .

Example 1:

Input: s = "aacaba"

Output: 2

Explanation: There are 5 ways to split "aacaba" and 2 of them are good.

("a", "acaba") Left string and right string contains 1 and 3 different letters respectively.

("aa", "caba") Left string and right string contains 1 and 3 different letters respectively.

("aac", "aba") Left string and right string contains 2 and 2 different letters respectively (good split).

("aaca", "ba") Left string and right string contains 2 and 2 different letters respectively (good split).

("aacab", "a") Left string and right string contains 3 and 1 different letters respectively.

Example 2:

Input: s = "abcd"

Output: 1

Explanation: Split the string as follows ("ab", "cd").

Constraints:

- $1 \leq s.length \leq 105$
- s consists of only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numSplits(self, s: str) -> int:
        n = len(s)
        ans = 0
        seen = set()
        prefix = [0] * n
        suffix = [0] * n

        for i in range(n):
            seen.add(s[i])
```

```

    prefix[i] = len(seen)

    seen.clear()

    for i in reversed(range(n)):
        seen.add(s[i])
        suffix[i] = len(seen)

    for i in range(n - 1):
        if prefix[i] == suffix[i + 1]:

            ans += 1

    return ans

```

• LeetCode

```

class Solution:
    def numSplits(self, s: str) -> int:
        cnt = Counter(s)
        vis = set()
        ans = 0
        for c in s:
            vis.add(c)
            cnt[c] -= 1
            if cnt[c] == 0:
                cnt.pop(c)

        ans += len(vis) == len(cnt)
        return ans

```

• SimplyLeet

```
# Python solution
def numSplits(s: str) -> int:
    n = len(s)

    # leftDistinct[i] represents the number of unique characters in s[:i+1]
    leftDistinct = [0] * n
    seen_left = set()

    for i in range(n):
        seen_left.add(s[i])

        leftDistinct[i] = len(seen_left)

    # rightDistinct[i] represents the number of unique characters in s[i:]
    rightDistinct = [0] * n
    seen_right = set()

    for i in range(n-1, -1, -1):
        seen_right.add(s[i])
        rightDistinct[i] = len(seen_right)

    goodSplits = 0
    # Iterate through all possible split points
    for i in range(n-1):
        if leftDistinct[i] == rightDistinct[i+1]:
            goodSplits += 1

    return goodSplits

# For testing
if __name__ == "__main__":

    test_string = "aacaba"
    print(numSplits(test_string)) # Expected output: 2
```

◆ Quick Review

Key Insights

- Use two passes: one forward to compute the running count of distinct characters from the beginning, and one backward to compute the running count from the end.
- By precomputing the distinct letter count for every possible split point, the count for each potential split can be quickly compared.
- A simple iteration through potential split indices lets us count the number of valid splits.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, as we traverse the string twice.

Space Complexity: $O(n)$, used for storing the distinct character counts for each prefix and suffix.

Solution

We solve the problem in multiple steps. First, we create an array, `leftDistinct`, where `leftDistinct[i]` represents the number of unique characters from the start of the string to index i . We then create a similar array, `rightDistinct`, where `rightDistinct[i]` represents the number of unique characters from index i to the end of

the string. Finally, we iterate over all valid split positions (from 0 to $n-2$) and count the splits where `leftDistinct[i]` equals `rightDistinct[i+1]`. The main trick here is the efficient counting of distinct characters using a set in a single pass, making the solution linear in time complexity.

Hash Tables

□ #1647 Minimum Deletions to Make Character Frequencies Unique

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Greedy · Sorting
Lists: AlgoMaster 600

Problem

A string s is called good if there are no two different characters in s that have the same frequency.

Given a string s , return the minimum number of characters you need to delete to make s good.

The frequency of a character in a string is the number of times it appears in the string. For example, in the string "aab", the frequency of 'a' is 2, while the frequency of 'b' is 1.

Example 1:

Input: $s = \text{"aab"}$

Output: 0

Explanation: s is already good.

Example 2:

Input: s = "aaabbbcc"

Output: 2

Explanation: You can delete two 'b's resulting in the good string "aaabcc".
Another way it to delete one 'b' and one 'c' resulting in the good string "aaabbc".

Example 3:

Input: s = "ceabaacb"

Output: 2

Explanation: You can delete both 'c's resulting in the good string "eabaab".
Note that we only care about characters that are still in the string at the end (i.e. frequency of 0 is ignored).

Constraints:

- $1 \leq s.length \leq 105$
- s contains only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minDeletions(self, s: str) -> int:
        ans = 0
        count = collections.Counter(s)
        usedFreq = set()

        for freq in count.values():
            while freq > 0 and freq in usedFreq:
                freq -= 1 # Delete ('a' + i).
                ans += 1

            usedFreq.add(freq)

        return ans
```

• LeetDoocs

```

class Solution:
    def minDeletions(self, s: str) -> int:
        cnt = Counter(s)
        ans, pre = 0, inf
        for v in sorted(cnt.values(), reverse=True):
            if pre == 0:
                ans += v
            elif v >= pre:
                ans += v - pre + 1
                pre -= 1
            else:
                pre = v
        return ans

```

```

class Solution:
    def minDeletions(self, s: str) -> int:
        cnt = Counter(s)
        vals = sorted(cnt.values(), reverse=True)
        ans = 0
        for i in range(1, len(vals)):
            while vals[i] >= vals[i - 1] and vals[i] > 0:
                vals[i] -= 1
            ans += 1
        return ans

```

• SimplyLeet

```

def minDeletions(s: str) -> int:
    # Count the frequency of each character in the string.
    from collections import Counter
    freq_counter = Counter(s)

    # Initialize a set to track frequencies that have been used.
    used_frequencies = set()
    deletions = 0

    # Iterate over each character's frequency.

```

```
for char, freq in freq_counter.items():
    # Adjust the frequency until it becomes unique (or zero)
    while freq > 0 and freq in used_frequencies:
        freq -= 1 # simulate deletion of a character
        deletions += 1
    # Add the (now unique) frequency to the set if it is positive.
    used_frequencies.add(freq)

return deletions
```

```
# Example usage:
print(minDeletions("aaabbbcc")) # Output: 2
```

◆ Quick Review

Key Insights

- Count the frequency of each character in the string.
- Use a greedy approach to ensure that each frequency is unique.
- Utilize a set (or similar data structure) to keep track of frequencies already used.
- For each character's frequency, if it conflicts with an already used frequency, reduce the frequency (simulate deletion) until it becomes unique or zero.
- The problem leverages sorting or iterating over a small fixed alphabet (26 letters) for efficiency.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string (plus a negligible factor for iterating over at most 26 unique characters).

Space Complexity: $O(1)$ since the frequency dictionary and set have at most 26 keys.

Solution

The solution begins by counting the frequency of every character in the string. Then, iterate over these frequencies and use a set to check for uniqueness. If a frequency already exists in the set, decrement the frequency (and count a deletion) until you either find a unique frequency or drop to 0. This greedy adjustment minimizes the total number of deletions required. The main data structures used are a hash map (or dictionary) for frequency counting and a set to track used frequencies.

Two Pointers

Round 2 · High · Medium · 3 questions

Two Pointers

□ #18 4Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: AlgoMaster 600

Problem

Given an array `nums` of `n` integers, return an array of all the unique quadruplets `[nums[a], nums[b], nums[c], nums[d]]` such that:

- $0 \leq a, b, c, d < n$
- `a`, `b`, `c`, and `d` are distinct.
- `nums[a] + nums[b] + nums[c] + nums[d] == target`

You may return the answer in any order.

Example 1:

```
Input: nums = [1,0,-1,0,-2,2], target = 0  
Output: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]
```

Example 2:

```
Input: nums = [2,2,2,2,2], target = 8  
Output: [[2,2,2,2]]
```

Constraints:

- $1 \leq \text{nums.length} \leq 200$

- $-109 \leq \text{nums}[i] \leq 109$
- $-109 \leq \text{target} \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def fourSum(self, nums: list[int], target: int):
        ans = []

        def nSum(
            l: int, r: int, target: int, n: int, path: list[int],
            ans: list[list[int]]) -> None:
            """Finds n numbers that add up to the target in [l, r]."""
            if r - l + 1 < n or n < 2 or target < nums[l] * n or target > nums[r] *
n:
                return

            if n == 2:
                while l < r:
                    summ = nums[l] + nums[r]
                    if summ == target:
                        ans.append(path + [nums[l], nums[r]])
                        l += 1
                        while nums[l] == nums[l - 1] and l < r:
                            l += 1
                    elif summ < target:
                        l += 1

                else:
                    r -= 1
                return

            for i in range(l, r + 1):
                if i > l and nums[i] == nums[i - 1]:
                    continue

                nSum(i + 1, r, target - nums[i], n - 1, path + [nums[i]], ans)
```



```

nums.sort()
nSum(0, len(nums) - 1, target, 4, [], ans)
return ans

```

• LeetCode

```

class Solution:
    def fourSum(self, nums: List[int], target: int) -> List[List[int]]:
        n = len(nums)
        ans = []
        if n < 4:
            return ans
        nums.sort()
        for i in range(n - 3):
            if i and nums[i] == nums[i - 1]:
                continue

            for j in range(i + 1, n - 2):
                if j > i + 1 and nums[j] == nums[j - 1]:
                    continue
                k, l = j + 1, n - 1
                while k < l:
                    x = nums[i] + nums[j] + nums[k] + nums[l]
                    if x < target:
                        k += 1
                    elif x > target:
                        l -= 1
                    else:
                        ans.append([nums[i], nums[j], nums[k], nums[l]])
                        k, l = k + 1, l - 1
                        while k < l and nums[k] == nums[k - 1]:
                            k += 1
                        while k < l and nums[l] == nums[l + 1]:
                            l -= 1

            return ans

```

• SimplyLeet

```

# Define a function to find all quadruplets that sum to the target.
def fourSum(nums, target):
    # Sort the input list to facilitate the two-pointer technique and duplicate
    # handling.
    nums.sort()
    quadruplets = []
    n = len(nums)

    # First loop: iterate through the list for the first element.
    for i in range(n - 3):
        # Skip duplicate values for the first element.

        if i > 0 and nums[i] == nums[i - 1]:
            continue
        # Second loop: iterate for the second element.
        for j in range(i + 1, n - 2):
            # Skip duplicate values for the second element.
            if j > i + 1 and nums[j] == nums[j - 1]:
                continue
            # Two pointers for the third and fourth element.
            left = j + 1
            right = n - 1

            while left < right:
                current_sum = nums[i] + nums[j] + nums[left] + nums[right]
                # If quadruplet sum matches target, add to result.
                if current_sum == target:
                    quadruplets.append([nums[i], nums[j], nums[left], nums[right]])

                    # Skip duplicates for the third element.
                    while left < right and nums[left] == nums[left + 1]:
                        left += 1
                    # Skip duplicates for the fourth element.
                    while left < right and nums[right] == nums[right - 1]:

```

```
        right -= 1
        left += 1
        right -= 1
        # Move left pointer if sum is less.
        elif current_sum < target:
            left += 1
        # Move right pointer if sum is more.
        else:
            right -= 1
    return quadruplets
```

```
# Example usage:
print(fourSum([1, 0, -1, 0, -2, 2], 0))
```

◆ Quick Review

Key Insights

- Sort the array to simplify handling duplicates and employ two pointers efficiently.
- Use nested loops to fix the first two numbers and then use the two-pointer technique for the remaining two numbers.
- Carefully skip duplicates at each step to ensure all quadruplets in the result are unique.
- Manage potential overflow or large input sizes by careful index manipulation.

Space & Time

Time Complexity: $O(n^3)$

Space Complexity: $O(n)$ for the output space assuming sorting space does not count.

Solution

The solution begins by sorting the input array to make duplicate elimination easier. Two nested loops are used to select the first and second elements of the quadruplet. For the remaining two numbers, the two-pointer approach is applied by initializing pointers at the start and the end of the remaining subarray. The sum of the four numbers is compared to the target:

- If the sum equals the target, the quadruplet is added to the results and the pointers move inward while skipping duplicate numbers.**
- If the sum is less than the target, the left pointer is moved right to increase the sum.**
- If the sum is greater than the target, the right pointer is moved left to decrease the sum. This strategy efficiently finds all unique quadruplets that meet the condition.**

Two Pointers

□ #443 String Compression

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String
Lists: AlgoMaster 600

Problem

Given an array of characters `chars`, compress it using the following algorithm:

Begin with an empty string `s`. For each group of consecutive repeating characters in `chars`:

- If the group's length is 1, append the character to `s`.
- Otherwise, append the character followed by the group's length.

The compressed string `s` should not be returned separately, but instead, be stored in the input character array `chars`. Note that group lengths that are 10 or longer will be split into multiple characters in `chars`.

After you are done modifying the input array, return the new length of the array.

You must write an algorithm that uses only constant extra space.

Note: The characters in the array beyond the returned length do not matter and should be ignored.

Example 1:

Input: chars = ["a","a","b","b","c","c","c"]

Output: 6

Explanation: The groups are "aa", "bb", and "ccc". This compresses to "a2b2c3".

Example 2:

Input: chars = ["a"]

Output: 1

Explanation: The only group is "a", which remains uncompressed since it's a single character.

Example 3:

Input: chars = ["a","b","b","b","b","b","b","b","b","b","b","b","b","b"]

Output: 4

Explanation: The groups are "a" and "bbbbbbbbbbbb". This compresses to "ab12".

Constraints:

- $1 \leq \text{chars.length} \leq 2000$
- `chars[i]` is a lowercase English letter, uppercase English letter, digit, or symbol.

Community Solutions (Python)

- WalkCC

```

class Solution:
    def compress(self, chars: list[str]) -> int:
        ans = 0
        i = 0

        while i < len(chars):
            letter = chars[i]
            count = 0
            while i < len(chars) and chars[i] == letter:
                count += 1

            i += 1
            chars[ans] = letter
            ans += 1
            if count > 1:
                for c in str(count):
                    chars[ans] = c
                    ans += 1

        return ans

```

• LeetDoocs

```

class Solution:
    def compress(self, chars: List[str]) -> int:
        i, k, n = 0, 0, len(chars)
        while i < n:
            j = i + 1
            while j < n and chars[j] == chars[i]:
                j += 1
            chars[k] = chars[i]
            k += 1
            if j - i > 1:
                cnt = str(j - i)
                for c in cnt:
                    chars[k] = c
                    k += 1
            i = j
        return k

```

• SimplyLeet

```
# Function to compress the list of characters in-place.
def compress(chars):
    # Initialize read and write pointers.
    read, write = 0, 0
    # Loop until the read pointer reaches the end of the array.
    while read < len(chars):
        # Current character to be processed.
        current_char = chars[read]
        count = 0
        # Count duplicates of current_char.

        while read < len(chars) and chars[read] == current_char:
            read += 1
            count += 1
        # Write the current character to the result.
        chars[write] = current_char
        write += 1
        # If count is greater than 1, write each digit of the count.
        if count > 1:
            for digit in str(count):
                chars[write] = digit
                write += 1

    # Return the new length of the array.
    return write

# Example usage:
# chars = ["a","a","b","b","c","c","c"]
# new_length = compress(chars)
# print(new_length, chars[:new_length])
```

◆ Quick Review

Key Insights

- Use a two–pointer approach: one pointer for reading the original characters and one for writing the compressed result.

- Iterate through the array, counting consecutive duplicates.
- Write the character to the result followed by its count (if greater than 1), converting the count to its string representation.
- The algorithm works in-place with constant extra space aside from the counter and temporary variables.

Space & Time

Time Complexity: $O(n)$, where n is the number of characters in the array.

Space Complexity: $O(1)$ (in-place compression with only a few extra variables).

Solution

The solution leverages a two-pointer technique:

- A "read" pointer traverses the array to identify groups of repeated characters.
- A "write" pointer updates the array with the compressed characters.
- For each group, after writing the character, if the group size is more than 1, the count is converted to a string and each digit is written

sequentially.

- **This in-place modification ensures constant extra space and the entire array is scanned once, resulting in $O(n)$ time complexity.**

Two Pointers

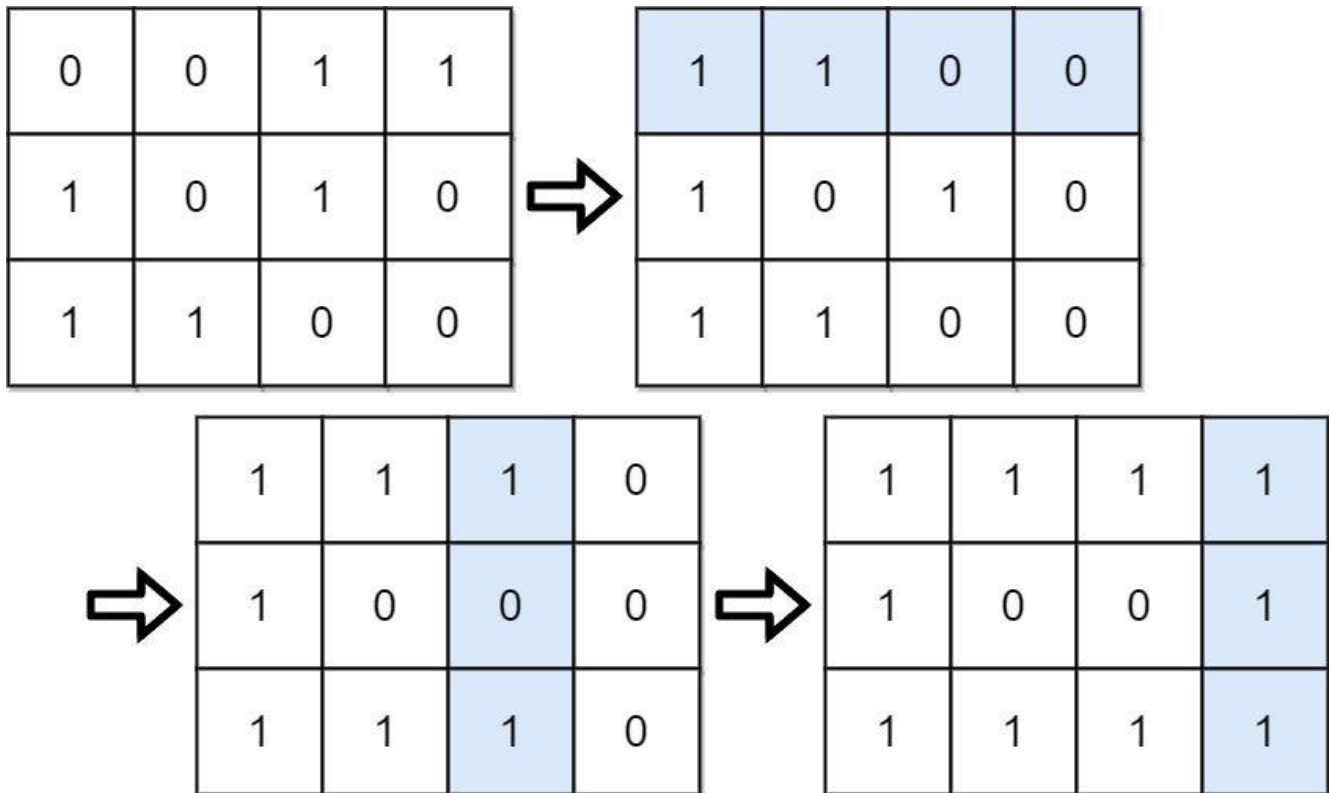
□ #861 Boats to Save People

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Greedy · Sorting
Lists: AlgoMaster 600

Problem

You are given an $m \times n$ binary matrix grid.
A move consists of choosing any row or column and toggling each value in that row or column (i.e., changing all 0's to 1's, and all 1's to 0's).
Every row of the matrix is interpreted as a binary number, and the score of the matrix is the sum of these numbers.
Return the highest possible score after making any number of moves (including zero moves).
Example 1:



Input: grid = [[0,0,1,1],[1,0,1,0],[1,1,0,0]]

Output: 39

Explanation: 0b1111 + 0b1001 + 0b1111 = 15 + 9 + 15 = 39

Example 2:

Input: grid = [[0]]

Output: 1

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 20$
- $\text{grid}[i][j]$ is either 0 or 1.

Community Solutions (Python)

- WalkCC

```

class Solution:
    def matrixScore(self, grid: list[list[int]]) -> int:
        # Flip the rows with a leading 0.
        for row in grid:
            if row[0] == 0:
                self._flip(row)

        # Flip the columns with 1s < 0s.
        for j, col in enumerate(list(zip(*grid))):
            if sum(col) * 2 < len(grid):

                self._flipCol(grid, j)

        # Add a binary number for each row.
        return sum(self._binary(row) for row in grid)

    def _flip(self, row: list[int]) -> None:
        for i in range(len(row)):
            row[i] ^= 1

    def _flipCol(self, grid: list[list[int]], j: int) -> None:

        for i in range(len(grid)):
            grid[i][j] ^= 1

    def _binary(self, row: list[int]) -> int:
        res = row[0]
        for j in range(1, len(row)):
            res = res * 2 + row[j]
        return res

```

```

class Solution:
    def matrixScore(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        ans = m # All the cells in the first column are 1.

        for j in range(1, n):
            # The best strategy is flipping the rows with a leading 0..
            onesCount = sum(grid[i][j] == grid[i][0] for i in range(m))
            ans = ans * 2 + max(onesCount, m - onesCount)

```

```
return ans
```

• LeetCode

```
class Solution:
    def matrixScore(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        for i in range(m):
            if grid[i][0] == 0:
                for j in range(n):
                    grid[i][j] ^= 1
        ans = 0
        for j in range(n):
            cnt = sum(grid[i][j] for i in range(m))
            ans += max(cnt, m - cnt) * (1 << (n - j - 1))
        return ans
```

• SimplyLeet

```
# Function to calculate the minimum number of boats needed
def numRescueBoats(people, limit):
    # Sort the list of people's weights
    people.sort()
    # Initialize two pointers; left points to the lightest, right to the
    # heaviest
    left, right = 0, len(people) - 1
    boats = 0 # Count of boats used

    # Iterate until all persons are assigned to a boat
    while left <= right:
```

```
# If the lightest and heaviest person can share a boat, do so
if people[left] + people[right] <= limit:
    left += 1 # Move left pointer to the next lightest person
# Always move the right pointer as the heaviest person occupies a boat
now
right -= 1
boats += 1 # Increment boat count for each allocation

return boats

# Example usage:

# people = [3,2,2,1], limit = 3
# print(numRescueBoats(people, limit)) # Expected output: 3
```

◆ Quick Review

Key Insights

- Sort the list of people's weights.
- Use the two pointers technique: one pointer at the beginning (lightest) and one at the end (heaviest).
- If the sum of the weights at both pointers is within the boat's limit, pair them and move both pointers.
- Otherwise, assign the heavier person alone and move the pointer for the heaviest.
- Continue until all people are accounted for.

Space & Time

Time Complexity: $O(n \log n)$ due to the sorting step.

Space Complexity: $O(1)$ additional space (ignoring the space taken by sorting, which may be $O(\log n)$ depending on the implementation).

Solution

The solution uses a greedy approach combined with the two pointers technique. After sorting the array, we initialize two pointers: one at the start (pointing to the lightest person) and one at the end (pointing to the heaviest person). In each iteration, we check if the combined weight of the persons at these pointers is within the limit. If so, we pair them in a single boat and move both pointers. Otherwise, the heavier person boards a boat alone, and we move the pointer for the heaviest person. This process repeats until all people have been allocated a boat. This approach ensures that we use the minimum number of boats.

Sliding Window – Fixed Size

Round 2 · High · Medium · 2 questions

Sliding Window – Fixed Size

□ #1456 Maximum Number of Vowels in a Substring of Given Length

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Sliding Window
Lists: AlgoMaster 600

Problem

Given a string *s* and an integer *k*, return the maximum number of vowel letters in any substring of *s* with length *k*.

Vowel letters in English are 'a', 'e', 'i', 'o', and 'u'.

Example 1:

Input: *s* = "abciidef", *k* = 3
Output: 3
Explanation: The substring "iii" contains 3 vowel letters.

Example 2:

Input: *s* = "aeiou", *k* = 2
Output: 2
Explanation: Any substring of length 2 contains 2 vowels.

Example 3:

Input: *s* = "leetcode", *k* = 3
Output: 2
Explanation: "lee", "eet" and "ode" contain 2 vowels.

Constraints:

- $1 \leq s.length \leq 105$
- s consists of lowercase English letters.
- $1 \leq k \leq s.length$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxVowels(self, s: str, k: int) -> int:
        ans = 0
        mx = 0
        VOWELS = 'aeiou'

        for i, c in enumerate(s):
            if c in VOWELS:
                mx += 1
            if i >= k and s[i - k] in VOWELS:

                mx -= 1
            ans = max(ans, mx)

        return ans
```

• LeetDoocs

```
class Solution:
    def maxVowels(self, s: str, k: int) -> int:
        vowels = set("aeiou")
        ans = cnt = sum(c in vowels for c in s[:k])
        for i in range(k, len(s)):
            cnt += int(s[i] in vowels) - int(s[i - k] in vowels)
            ans = max(ans, cnt)
        return ans
```

• SimplyLeet

```
# Python solution using sliding window algorithm

def maxVowels(s: str, k: int) -> int:
    vowels = set("aeiou") # Set for quick vowel lookup
    current_count = 0

    # Count vowels in the initial window of size k
    for i in range(k):
        if s[i] in vowels:
            current_count += 1

    max_count = current_count # Initialize max_count with the count of the
    first window

    # Slide the window through the string
    for i in range(k, len(s)):
        if s[i - k] in vowels: # Remove letter going out of the window
            current_count -= 1
        if s[i] in vowels: # Add incoming letter
            current_count += 1
        max_count = max(max_count, current_count) # Update maximum found so
    far

    return max_count

# Example usage:
# print(maxVowels("abcciiidef", 3))
```

◆ Quick Review

Key Insights

- Use the sliding window technique to efficiently evaluate substrings of length k.
- Maintain a running count of vowels within the current window.

- When sliding the window, subtract the count if the exiting character is a vowel and add if the new incoming character is a vowel.
- The approach is efficient with a time complexity of $O(n)$.

Space & Time

Time Complexity: $O(n)$, where n is the length of string s .

Space Complexity: $O(1)$, since only constant extra space is used.

Solution

We solve the problem by implementing a sliding window. First, count the vowels in the initial window of size k . Then continue sliding the window one character at a time. For each slide, if the character that exits is a vowel, decrement the count; if the entering character is a vowel, increment the count. Track the maximum count during this process. This method ensures each character is processed once for an efficient solution.

Sliding Window – Fixed Size

□ #2461 Maximum Sum of Distinct Subarrays With Length K

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sliding Window
Lists: AlgoMaster 600

Problem

You are given an integer array `nums` and an integer `k`. Find the maximum subarray sum of all the subarrays of `nums` that meet the following conditions:

- The length of the subarray is `k`, and
- All the elements of the subarray are distinct.

Return the maximum subarray sum of all the subarrays that meet the conditions. If no subarray meets the conditions, return 0.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: nums = [1,5,4,2,9,9,9], k = 3

Output: 15

Explanation: The subarrays of nums with length 3 are:

- [1,5,4] which meets the requirements and has a sum of 10.
- [5,4,2] which meets the requirements and has a sum of 11.
- [4,2,9] which meets the requirements and has a sum of 15.
- [2,9,9] which does not meet the requirements because the element 9 is repeated.
- [9,9,9] which does not meet the requirements because the element 9 is repeated.

Example 2:

Input: nums = [4,4,4], k = 3

Output: 0

Explanation: The subarrays of nums with length 3 are:

- [4,4,4] which does not meet the requirements because the element 4 is repeated.

We return 0 because no subarrays meet the conditions.

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i] \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maximumSubarraySum(self, nums: list[int], k: int) -> int:
        ans = 0
        summ = 0
        distinct = 0
        count = collections.Counter()

        for i, num in enumerate(nums):
            summ += num
            count[num] += 1
```

```

if count[num] == 1:
    distinct += 1
if i >= k:
    count[nums[i - k]] -= 1
    if count[nums[i - k]] == 0:
        distinct -= 1
    summ -= nums[i - k]
if i >= k - 1 and distinct == k:
    ans = max(ans, summ)

```

```

return ans

```

• LeetCode

```

class Solution:
    def maximumSubarraySum(self, nums: List[int], k: int) -> int:
        cnt = Counter(nums[:k])
        s = sum(nums[:k])
        ans = s if len(cnt) == k else 0
        for i in range(k, len(nums)):
            cnt[nums[i]] += 1
            cnt[nums[i - k]] -= 1
            if cnt[nums[i - k]] == 0:
                cnt.pop(nums[i - k])

            s += nums[i] - nums[i - k]
            if len(cnt) == k:
                ans = max(ans, s)
        return ans

```

• SimplyLeet


```
# Python solution for Maximum Sum of Distinct Subarrays With Length K

def maximumSubarraySum(nums, k):
    # Dictionary to store frequencies of elements in current window
    freq = {}
    current_sum = 0
    max_sum = 0
    left = 0 # window left pointer

    # Loop through each element in the array using a sliding window approach

    for right in range(len(nums)):
        # Add the new element at the 'right' end of the window
        current = nums[right]
        freq[current] = freq.get(current, 0) + 1
        current_sum += current

        # Slide window if its size is greater than k
        if right - left + 1 > k:
            remove_val = nums[left]
            freq[remove_val] -= 1

            # Remove element from dictionary if its count becomes zero
            if freq[remove_val] == 0:
                del freq[remove_val]
            current_sum -= remove_val
            left += 1

        # Check the window only if it has exactly k elements and all are
        distinct
        if right - left + 1 == k and len(freq) == k:
            max_sum = max(max_sum, current_sum)

    return max_sum

# Example usage:
print(maximumSubarraySum([1,5,4,2,9,9,9], 3)) # Output: 15
```

◆ Quick Review

Key Insights

- Use a sliding window of fixed length k to iterate through the array.
- Maintain a frequency map (or hash table) for elements in the current window to easily check for duplicate elements.
- Instead of recalculating the sum for each window, update the current sum by subtracting the element that is exiting the window and adding the new element.
- Only consider the current window's sum if the frequency map size is equal to k (no duplicates).

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in `nums`, as each element is added and removed from the sliding window exactly once.

Space Complexity: $O(k)$ in the worst-case scenario for maintaining the frequency map.

Solution

The problem is solved using a sliding window technique. We maintain a window of size k along with a hash table (or dictionary) to count

the occurrence of elements in the window. For each move of the window:

- Add the new element and update the current window sum.**
- If the window size exceeds k , remove the leftmost element and update the frequency map and current sum.**
- Check if the current window contains distinct elements by verifying if the size of the frequency dictionary equals k .**
- If valid, update the maximum sum. This approach ensures that each element is processed in constant time on average, and the sliding window guarantees that we only look at contiguous segments.**

Sliding Window – Dynamic Size

Round 2 · High · Medium · 7 questions

Sliding Window – Dynamic Size

□ #209 Minimum Size Subarray Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Sliding Window · Prefix Sum

Lists: AlgoMaster 600

Problem

Given an array of positive integers `nums` and a positive integer `target`, return the minimal length of a subarray whose sum is greater than or equal to `target`. If there is no such subarray, return 0 instead.

Example 1:

Input: `target = 7, nums = [2,3,1,2,4,3]`
Output: 2
Explanation: The subarray `[4,3]` has the minimal length under the problem constraint.

Example 2:

Input: `target = 4, nums = [1,4,4]`
Output: 1

Example 3:

Input: `target = 11, nums = [1,1,1,1,1,1,1,1]`
Output: 0

Constraints:

- $1 \leq \text{target} \leq 109$
- $1 \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i] \leq 104$

Follow up: If you have figured out the $O(n)$ solution, try coding another solution of which the time complexity is $O(n \log(n))$.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minSubArrayLen(self, target: int, nums: list[int]) -> int:
        ans = math.inf
        summ = 0
        j = 0

        for i, num in enumerate(nums):
            summ += num
            while summ >= target:
                ans = min(ans, i - j + 1)
                summ -= nums[j]
                j += 1

        return 0 if ans == math.inf else ans
```

• LeetDoocs

```
class Solution:
    def minSubArrayLen(self, target: int, nums: List[int]) -> int:
        n = len(nums)
        s = list(accumulate(nums, initial=0))
        ans = n + 1
        for i, x in enumerate(s):
            j = bisect_left(s, x + target)
            if j <= n:
                ans = min(ans, j - i)
        return ans if ans <= n else 0
```

```
class Solution:
    def minSubArrayLen(self, target: int, nums: List[int]) -> int:
        l = s = 0
        ans = inf
        for r, x in enumerate(nums):
            s += x
            while s >= target:
                ans = min(ans, r - l + 1)
                s -= nums[l]
                l += 1
        return 0 if ans == inf else ans
```

• SimplyLeet

Python implementation with detailed comments

```
def minSubArrayLen(target, nums):
    # Initialize the window pointers and variables
    n = len(nums)
    left = 0 # Left pointer of the sliding window
    window_sum = 0 # Sum of the current window
    min_len = float('inf') # Track minimum length, start with infinity

    # Iterate over the array with right pointer
```

```
for right in range(n):
    window_sum += nums[right] # Expand the window by adding the current
    element

    # Contract the window as long as the current sum is large enough
    while window_sum >= target:
        # Update the minimum length found so far
        current_window_length = right - left + 1
        min_len = min(min_len, current_window_length)

        # Shrink the window from the left and subtract the value

    window_sum -= nums[left]
    left += 1

# If the min_len wasn't updated, no valid subarray was found. Return 0.
return 0 if min_len == float('inf') else min_len

# Example usage:
print(minSubArrayLen(7, [2,3,1,2,4,3])) # Expected output: 2
```

◆ Quick Review

Key Insights

- All numbers are positive, which allows for a sliding window approach.
- A sliding window maintains a dynamic range [left, right] that expands or contracts to find candidate subarrays.
- The window is expanded until the sum meets/exceeds the target, then contracted to seek the minimal candidate.

- An $O(n)$ solution is achievable with the sliding window technique; an alternative $O(n \log n)$ solution exists using prefix sums and binary search.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We use the sliding window approach:

- Initialize two pointers, left and right, and a variable to maintain the current window sum.
- As you iterate with the right pointer, add the current element to the window sum.
- While the window sum is greater than or equal to target, try to reduce the window size by moving the left pointer forward and update the minimal length found.
- If no valid window is found throughout the loop, return 0. This method works efficiently because adding an element increases the sum and removing it decreases the sum, so we maintain validity within a single pass.

Sliding Window – Dynamic Size

□ #395 Longest Substring with At Least K Repeating Characters

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Divide and Conquer ·
Sliding Window
Lists: AlgoMaster 600

Problem

Given a string s and an integer k , return the length of the longest substring of s such that the frequency of each character in this substring is greater than or equal to k .
if no such substring exists, return 0.

Example 1:

Input: $s = \text{"aaabb"}, k = 3$
Output: 3
Explanation: The longest substring is "aaa", as 'a' is repeated 3 times.

Example 2:

Input: $s = \text{"ababbc"}, k = 2$
Output: 5
Explanation: The longest substring is "ababb", as 'a' is repeated 2 times and 'b' is repeated 3 times.

Constraints:

- $1 \leq s.length \leq 104$

- s consists of only lowercase English letters.
- $1 \leq k \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestSubstring(self, s: str, k: int) -> int:
        def longestSubstringWithNUniqueLetters(n: int) -> int:
            res = 0
            uniqueLetters = 0 # the number of unique letters
            lettersHavingKFreq = 0 # the number of letters having frequency >= k
            count = collections.Counter()

            l = 0
            for r, c in enumerate(s):

                count[c] += 1
                if count[c] == 1:
                    uniqueLetters += 1
                if count[c] == k:
                    lettersHavingKFreq += 1
                while uniqueLetters > n:
                    if count[s[l]] == k:
                        lettersHavingKFreq -= 1
                    count[s[l]] -= 1
                    if count[s[l]] == 0:

                        uniqueLetters -= 1
                    l += 1
                # Since both the number of unique letters and the number of letters
                # having frequency >= k are equal to n, this is a valid window.
                if lettersHavingKFreq == n: # Implicit: uniqueLetters == n
                    res = max(res, r - l + 1)

            return res

        return max(longestSubstringWithNUniqueLetters(n)

                    for n in range(1, 27))
```

• LeetCode

```
class Solution:
    def longestSubstring(self, s: str, k: int) -> int:
        def dfs(l, r):
            cnt = Counter(s[l : r + 1])
            split = next((c for c, v in cnt.items() if v < k), '')
            if not split:
                return r - l + 1
            i = l
            ans = 0
            while i <= r:
                while i <= r and s[i] == split:
                    i += 1
                if i >= r:
                    break
                j = i
                while j <= r and s[j] != split:
                    j += 1
                t = dfs(i, j - 1)
                ans = max(ans, t)
                i = j

            return ans

        return dfs(0, len(s) - 1)
```

• SimplyLeet

```
# Python solution with line-by-line comments

def longestSubstring(s: str, k: int) -> int:
    max_len = 0
    n = len(s)
    # Check for each potential number of unique characters in the substring
    for target_unique in range(1, 27): # There are at most 26 unique
characters
        count = [0] * 26 # Frequency counter for current window
        start, end = 0, 0 # Sliding window pointers
        unique_count = 0 # Current number of unique characters in window
```

```

count_at_least_k = 0 # Number of characters meeting the k threshold

while end < n:
    # Add character at end pointer
    if count[ord(s[end]) - ord('a')] == 0:
        unique_count += 1 # New unique character encountered
    count[ord(s[end]) - ord('a')] += 1
    if count[ord(s[end]) - ord('a')] == k:
        count_at_least_k += 1 # Character frequency reached k
    end += 1

    # Shrink window from left if unique characters exceed target
    while unique_count > target_unique:
        if count[ord(s[start]) - ord('a')] == k:
            count_at_least_k -= 1
        count[ord(s[start]) - ord('a')] -= 1
        if count[ord(s[start]) - ord('a')] == 0:
            unique_count -= 1
        start += 1

    # If window is valid update max_len
    if unique_count == target_unique and unique_count == count_at_least_k:
        max_len = max(max_len, end - start)

return max_len

# Example test cases
if __name__ == "__main__":
    print(longestSubstring("aaabb", 3)) # Expected output: 3
    print(longestSubstring("ababbc", 2)) # Expected output: 5

```

◆ Quick Review

Key Insights

- The valid substring must have every character's frequency greater than or equal to k.

- A sliding window approach can be employed by controlling the number of unique characters within the window.
- Iterating over all possible unique character counts (from 1 to 26) allows us to systematically explore all valid substrings.
- Alternatively, a divide and conquer approach splits the string on characters that fail the k -occurrence criteria.

Space & Time

Time Complexity: $O(n * 26) = O(n)$, since we iterate over the string for each unique character count.

Space Complexity: $O(26) = O(1)$, for the frequency counter array.

Solution

We use a sliding window approach that iterates over the possible counts of unique characters (from 1 to 26). For each unique count:

- Expand the window while updating the frequency counts.
- Maintain the number of unique characters and the number meeting the k threshold.

- Contract the window when the number of unique characters exceeds the current target.
- If the number of unique characters equals the number meeting the k threshold, update the maximum length.

This approach efficiently explores all possible windows that can form a valid substring.

Sliding Window – Dynamic Size

□ #713 Subarray Product Less Than K

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Sliding Window · Prefix Sum

Lists: AlgoMaster 600

Problem

Given an array of integers `nums` and an integer `k`, return the number of contiguous subarrays where the product of all the elements in the subarray is strictly less than `k`.

Example 1:

Input: `nums = [10,5,2,6]`, `k = 100`

Output: 8

Explanation: The 8 subarrays that have product less than 100 are:

[10], [5], [2], [6], [10, 5], [5, 2], [2, 6], [5, 2, 6]

Note that [10, 5, 2] is not included as the product of 100 is not strictly less than `k`.

Example 2:

Input: `nums = [1,2,3]`, `k = 0`

Output: 0

Constraints:

- $1 \leq \text{nums.length} \leq 3 \times 10^4$
- $1 \leq \text{nums}[i] \leq 1000$

- $0 \leq k \leq 106$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numSubarrayProductLessThanK(self, nums: list[int], k: int) -> int:
        if k <= 1:
            return 0

        ans = 0
        prod = 1

        j = 0
        for i, num in enumerate(nums):

            prod *= num
            while prod >= k:
                prod /= nums[j]
                j += 1
            ans += i - j + 1

        return ans
```

• LeetDoocs

```
class Solution:
    def numSubarrayProductLessThanK(self, nums: List[int], k: int) -> int:
        ans = l = 0
        p = 1
        for r, x in enumerate(nums):
            p *= x
            while l <= r and p >= k:
                p //= nums[l]
                l += 1
            ans += r - l + 1

        return ans
```

• SimplyLeet

```
def numSubarrayProductLessThanK(nums, k):  
    # If k is less than or equal to 1, no valid subarray exists  
    if k <= 1:  
        return 0  
    prod = 1 # Running product of the current window  
    ans = 0 # Counter for valid subarrays  
    left = 0 # Left pointer of the sliding window  
    # Iterate over the array using the right pointer  
    for right, value in enumerate(nums):  
        prod *= value # Multiply the current value into the running product  
  
        # Shrink the window from the left until the product is less than k  
        while prod >= k:  
            prod //= nums[left]  
            left += 1  
        # All subarrays ending at 'right' with start indices between 'left' and  
        # 'right' are valid  
        ans += right - left + 1  
    return ans  
  
# Example usage  
print(numSubarrayProductLessThanK([10, 5, 2, 6], 100)) # Expected output: 8
```

◆ Quick Review

Key Insights

- Using a sliding window approach is effective when dealing with contiguous subarrays and maintaining a running product.
- Since the problem involves multiplying numbers, as soon as the running product exceeds or equals k , shrink the window from the left.

- The number of valid subarrays ending at the current right pointer is given by the length of the window.

Space & Time

Time Complexity: $O(n)$ because each element is visited at most twice.

Space Complexity: $O(1)$ as only a few extra variables are used.

Solution

The solution uses a sliding window technique to maintain a running product of the subarray.

- Initialize pointers for the window (left) and variables for the running product (prod) and count of valid subarrays (ans).
- Iterate through the array with a right pointer, multiplying the current element to the product.
- If the product becomes greater than or equal to k , move the left pointer rightwards while dividing the product by each element passed.
- The number of subarrays ending at the current index that satisfy the condition is equal to $(\text{right} - \text{left} + 1)$. Add this count to ans.

- Return the final count stored in ans.

This method efficiently tracks valid subarrays and avoids checking all possible combinations.

Sliding Window – Dynamic Size

□ #904 Fruit Into Baskets

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sliding Window
Lists: AlgoMaster 600

Problem

You are visiting a farm that has a single row of fruit trees arranged from left to right. The trees are represented by an integer array `fruits` where `fruits[i]` is the type of fruit the *i*th tree produces. You want to collect as much fruit as possible. However, the owner has some strict rules that you must follow:

- You only have two baskets, and each basket can only hold a single type of fruit. There is no limit on the amount of fruit each basket can hold.
- Starting from any tree of your choice, you must pick exactly one fruit from every tree (including the start tree) while moving to the right. The picked fruits must fit in one of your baskets.

- Once you reach a tree with fruit that cannot fit in your baskets, you must stop.

Given the integer array `fruits`, return the maximum number of fruits you can pick.

Example 1:

Input: `fruits = [1,2,1]`
Output: 3
Explanation: We can pick from all 3 trees.

Example 2:

Input: `fruits = [0,1,2,2]`
Output: 3
Explanation: We can pick from trees `[1,2,2]`.
If we had started at the first tree, we would only pick from trees `[0,1]`.

Example 3:

Input: `fruits = [1,2,3,2,2]`
Output: 4
Explanation: We can pick from trees `[2,3,2,2]`.
If we had started at the first tree, we would only pick from trees `[1,2]`.

Constraints:

- $1 \leq \text{fruits.length} \leq 105$
- $0 \leq \text{fruits}[i] < \text{fruits.length}$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def totalFruit(self, fruits: list[int]) -> int:
        ans = 0
        count = collections.defaultdict(int)

        l = 0
        for r, fruit in enumerate(fruits):
            count[fruit] += 1
            while len(count) > 2:
                count[fruits[l]] -= 1

                if count[fruits[l]] == 0:
                    del count[fruits[l]]
                l += 1
            ans = max(ans, r - l + 1)

        return ans

```

• LeetCode

```

class Solution:
    def totalFruit(self, fruits: List[int]) -> int:
        cnt = Counter()
        ans = j = 0
        for i, x in enumerate(fruits):
            cnt[x] += 1
            while len(cnt) > 2:
                y = fruits[j]
                cnt[y] -= 1
                if cnt[y] == 0:
                    cnt.pop(y)
                j += 1
            ans = max(ans, i - j + 1)
        return ans

```

• SimplyLeet

```
# Python solution for Fruit Into Baskets

def totalFruit(fruits):
    # Dictionary to track the count of each fruit type in the current window
    fruit_count = {}
    left = 0 # left index of our sliding window
    max_fruits = 0 # maximum number of fruits collected so far

    # Iterate over fruits with the index 'right'
    for right, fruit in enumerate(fruits):

        # Add/update the current fruit in the window
        fruit_count[fruit] = fruit_count.get(fruit, 0) + 1

        # If the window contains more than 2 distinct fruit types, shrink it
        # from the left
        while len(fruit_count) > 2:
            fruit_count[fruits[left]] -= 1
            # If the count drops to 0, remove that fruit type from the
            # dictionary
            if fruit_count[fruits[left]] == 0:
                del fruit_count[fruits[left]]
            left += 1

        # Update the maximum fruits picked
        max_fruits = max(max_fruits, right - left + 1)

    return max_fruits

# Example usage:
print(totalFruit([1, 2, 1])) # Expected Output: 3
```

◆ Quick Review

Key Insights

- Use a sliding window to maintain a contiguous subarray with at most two distinct fruit types.

- A hash table (or dictionary) is used to count occurrences of fruit types in the current window.
- Expand the window to include new fruits and shrink it from the left when more than two types are present.
- This approach is similar to solving the "longest substring with at most K distinct characters" for $K = 2$.

Space & Time

Time Complexity: $O(n)$, where n is the length of the fruits array.

Space Complexity: $O(1)$, as the hash table stores at most 3 keys in worst-case scenarios (effectively bounded by 2 unique types).

Solution

We use a sliding window technique. By iterating through the fruits array with a right pointer, we update a hash table that tracks the count of each fruit type in the window. When the window contains more than two unique fruit types, we move the left pointer to reduce the window size until it becomes valid again (only two types remain). The maximum valid

window size over all iterations gives the maximum number of fruits that can be picked.

Sliding Window – Dynamic Size

□ #1004 Max Consecutive Ones III

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Sliding Window · Prefix Sum

Lists: AlgoMaster 600

Problem

Given a binary array `nums` and an integer `k`, return the maximum number of consecutive 1's in the array if you can flip at most `k` 0's.

Example 1:

Input: `nums = [1,1,1,0,0,0,1,1,1,1,0]`, `k = 2`
Output: 6
Explanation: `[1,1,1,0,0,1,1,1,1,1,1]`
Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.

Example 2:

Input: `nums = [0,0,1,1,0,0,1,1,1,0,1,1,0,0,0,1,1,1,1]`, `k = 3`
Output: 10
Explanation: `[0,0,1,1,1,1,1,1,1,1,1,0,0,0,1,1,1,1]`
Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- `nums[i]` is either 0 or 1.
- $0 \leq k \leq \text{nums.length}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestOnes(self, nums: list[int], k: int) -> int:
        ans = 0

        l = 0
        for r, num in enumerate(nums):
            if num == 0:
                k -= 1
                while k < 0:
                    if nums[l] == 0:
                        k += 1
                        l += 1
                ans = max(ans, r - l + 1)

        return ans
```

• LeetDoocs

```
class Solution:
    def longestOnes(self, nums: List[int], k: int) -> int:
        l = cnt = 0
        for x in nums:
            cnt += x ^ 1
            if cnt > k:
                cnt -= nums[l] ^ 1
                l += 1
        return len(nums) - l
```

• SimplyLeet

```
# Python solution with line-by-line comments
def longestOnes(nums, k):
    # Initialize two pointers for the sliding window and zero_count to count
    # zeros in the window.
    left = 0
    max_length = 0
    zero_count = 0

    # Expand the window using the right pointer
    for right in range(len(nums)):
        # If the right element is 0, increment the zero counter

        if nums[right] == 0:
            zero_count += 1

        # If there are more than k zeros, shrink the window from the left
        while zero_count > k:
            if nums[left] == 0:
                zero_count -= 1
            left += 1

        # Update the maximum length with the current window size

        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
# print(longestOnes([1,1,1,0,0,0,1,1,1,1,0], 2))
```

◆ Quick Review

Key Insights

- The problem can be solved efficiently using the sliding window technique.
- Maintain a window that can contain at most k zeros.

- When the number of zeros in the window exceeds k , move the left pointer to shrink the window.
- Record the maximum window length during the iteration which gives the maximum consecutive ones.

Space & Time

Time Complexity: $O(n)$, where n is the length of the array, since we process each element at most once.

Space Complexity: $O(1)$, only constant extra space is used.

Solution

We use a sliding window approach:

- Use two pointers (left and right) that define the current window.
- Traverse the array using the right pointer.
- Count the number of zeros encountered in the current window.
- When the count exceeds k , increment the left pointer until there are at most k zeros in the window.

- Throughout the process, the length of the current valid window is recorded, and the maximum window length is updated.
- The maximum window length gives the maximum number of consecutive 1's achievable with at most k flips.

Sliding Window – Dynamic Size

□ #1423 Maximum Points You Can Obtain from Cards

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sliding Window · Prefix Sum
Lists: AlgoMaster 600

Problem

There are several cards arranged in a row, and each card has an associated number of points. The points are given in the integer array `cardPoints`.

In one step, you can take one card from the beginning or from the end of the row. You have to take exactly `k` cards.

Your score is the sum of the points of the cards you have taken.

Given the integer array `cardPoints` and the integer `k`, return the maximum score you can obtain.

Example 1:

Input: cardPoints = [1,2,3,4,5,6,1], k = 3

Output: 12

Explanation: After the first step, your score will always be 1. However, choosing the rightmost card first will maximize your total score. The optimal strategy is to take the three cards on the right, giving a final score of $1 + 6 + 5 = 12$.

Example 2:

Input: cardPoints = [2,2,2], k = 2

Output: 4

Explanation: Regardless of which two cards you take, your score will always be 4.

Example 3:

Input: cardPoints = [9,7,7,9,7,7,9], k = 7

Output: 55

Explanation: You have to take all the cards. Your score is the sum of points of all cards.

Constraints:

- $1 \leq \text{cardPoints.length} \leq 105$
- $1 \leq \text{cardPoints}[i] \leq 104$
- $1 \leq k \leq \text{cardPoints.length}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxScore(self, cardPoints: list[int], k: int) -> int:
        n = len(cardPoints)
        summ = sum(cardPoints)
        windowSum = sum(cardPoints[:n - k])
        ans = summ - windowSum

        for i in range(k):
            windowSum -= cardPoints[i]
            windowSum += cardPoints[i + n - k]
```

```

    ans = max(ans, summ - windowSum)

    return ans

```

• LeetDoocs

```

class Solution:
    def maxScore(self, cardPoints: List[int], k: int) -> int:
        ans = s = sum(cardPoints[-k:])
        for i, x in enumerate(cardPoints[:k]):
            s += x - cardPoints[-k + i]
            ans = max(ans, s)
        return ans

```

```

# @param {Integer[]} card_points
# @param {Integer} k
# @return {Integer}
def max_score(card_points, k)
  n = card_points.length
  s = card_points[-k..].sum
  ans = s
  k.times do |i|
    s += card_points[i] - card_points[n - k + i]
    ans = [ans, s].max
  end
  ans
end

```

• SimplyLeet

```

# Function to calculate the maximum points obtainable
def maxScore(cardPoints, k):
    n = len(cardPoints)
    # If k equals the total number of cards, return the total sum of cardPoints
    if k == n:
        return sum(cardPoints)

    total_sum = sum(cardPoints)
    window_size = n - k # Size of the subarray we are leaving behind

```

```
# Compute the sum of the first window
current_window_sum = sum(cardPoints[:window_size])
min_window_sum = current_window_sum

# Slide the window from index 1 to n - window_size
for i in range(window_size, n):
    # Subtract the element going out of the window and add the element
    # coming into the window
    current_window_sum += cardPoints[i] - cardPoints[i - window_size]
    # Keep track of the minimum sum we've seen in the window
    if current_window_sum < min_window_sum:

        min_window_sum = current_window_sum

# Maximum points is the total sum minus the minimum window sum
return total_sum - min_window_sum

# Example usage:
print(maxScore([1,2,3,4,5,6,1], 3)) # Output: 12
```

◆ Quick Review

Key Insights

- Instead of choosing which k cards to take, think in reverse: find the contiguous subarray of length $n - k$ (where n is the total number of cards) that has the minimum sum.
- The maximal points you can obtain is the total sum of the cards minus the sum of the subarray that you did not choose.
- A sliding window technique can efficiently find the minimum sum subarray of a given fixed length.

- This approach changes a two-ended selection problem into a single contiguous subarray problem.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The idea is to first compute the total sum of the `cardPoints` array. If k equals the number of cards, then the answer is simply the total sum. Otherwise, since you are required to take k cards and the remaining cards form a contiguous block of length $n - k$, find that contiguous block with the minimum sum using the sliding window technique. Subtract this minimum window sum from the total sum, which gives the maximum points possible.

The algorithm uses the following steps:

- Compute the total sum of the array.
- Compute the sum of the first window of size $n - k$.
- Slide the window across the array while updating the current window sum and tracking the minimum window sum encountered.

– Subtract the minimum window sum from the total sum to obtain the maximum points obtainable by taking k cards from the ends.

Data structures used: variables for sum tracking; no additional data structures beyond constant space is required.

Sliding Window – Dynamic Size

□ #1838 Frequency of the Most Frequent Element

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Greedy · Sliding Window
· Sorting · Prefix Sum
Lists: AlgoMaster 600

Problem

The frequency of an element is the number of times it occurs in an array.

You are given an integer array `nums` and an integer `k`. In one operation, you can choose an index of `nums` and increment the element at that index by 1.

Return the maximum possible frequency of an element after performing at most `k` operations.

Example 1:

Input: `nums = [1,2,4]`, `k = 5`

Output: 3

Explanation: Increment the first element three times and the second element two times to make `nums = [4,4,4]`.

4 has a frequency of 3.

Example 2:

Input: nums = [1,4,8,13], k = 5

Output: 2

Explanation: There are multiple optimal solutions:

- Increment the first element three times to make nums = [4,4,8,13]. 4 has a frequency of 2.
- Increment the second element four times to make nums = [1,8,8,13]. 8 has a frequency of 2.
- Increment the third element five times to make nums = [1,4,13,13]. 13 has a frequency of 2.

Example 3:

Input: nums = [3,9,6], k = 2

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i] \leq 105$
- $1 \leq k \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxFrequency(self, nums: list[int], k: int) -> int:
        ans = 0
        summ = 0

        nums.sort()

        l = 0
        for r, num in enumerate(nums):
            summ += num
```

```

while summ + k < num * (r - l + 1):
    summ -= nums[l]
    l += 1
ans = max(ans, r - l + 1)

return ans

```

• LeetDoocs

```

class Solution:
    def maxFrequency(self, nums: List[int], k: int) -> int:
        def check(m: int) -> bool:
            for i in range(m, n + 1):
                if nums[i - 1] * m - (s[i] - s[i - m]) <= k:
                    return True
            return False

        n = len(nums)
        nums.sort()

```

```

        s = list(accumulate(nums, initial=0))
        l, r = 1, n
        while l < r:
            mid = (l + r + 1) >> 1
            if check(mid):
                l = mid
            else:
                r = mid - 1
        return l

```

```

class Solution:
    def maxFrequency(self, nums: List[int], k: int) -> int:
        nums.sort()
        ans = 1
        s = j = 0
        for i in range(1, len(nums)):
            s += (nums[i] - nums[i - 1]) * (i - j)
            while s > k:
                s -= nums[i] - nums[j]
                j += 1

            ans = max(ans, i - j + 1)
        return ans

```


• SimplyLeet

```
# Python solution with line-by-line comments
class Solution:
    def maxFrequency(self, nums: List[int], k: int) -> int:
        # Sort the array to group similar values together
        nums.sort()
        left = 0 # left pointer of the sliding window
        total_sum = 0 # sum of elements in the current window
        max_freq = 0 # the maximum frequency found

        # Iterate through the array using right pointer

        for right in range(len(nums)):
            # Add the current element to the running sum
            total_sum += nums[right]
            # Calculate cost to make all elements in window equal to
            nums[right]
            # If cost exceeds k, move left pointer to shrink the window
            while nums[right] * (right - left + 1) - total_sum > k:
                total_sum -= nums[left]
                left += 1
            # Update maximum frequency if the current window is larger
            max_freq = max(max_freq, right - left + 1)

        return max_freq
```

◆ Quick Review

Key Insights

- Sorting the array simplifies the problem by allowing us to work with contiguous subarrays.
- A sliding window approach efficiently finds the largest window where all elements can be made equal with at most k operations.

- Maintain a running sum of the current window to compute the cost of increasing all elements to match the current maximum.
- If the cost exceeds k , shrink the window from the left until the cost is within limits.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting, plus $O(n)$ for the sliding window traversal.

Space Complexity: $O(1)$ if sorting is done in-place (or $O(n)$ otherwise).

Solution

Sort the input array and use a sliding window approach. For each new element considered at the right pointer, calculate the cost to convert all elements within the window to match the current element ($\text{nums}[\text{right}]$). This cost is computed as: $\text{cost} = \text{nums}[\text{right}] * \text{window_length} - \text{sum}(\text{window})$

If the cost is greater than k , move the left pointer to shrink the window until the cost is acceptable. The length of the window at any point reflects the frequency of the most frequent element that can be obtained. Update the maximum frequency accordingly.

Binary Search

Round 2 · High · Medium · 6 questions

Binary Search

□ #81 Search in Rotated Sorted Array II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search

Lists: AlgoMaster 600

Problem

There is an integer array `nums` sorted in non-decreasing order (not necessarily with distinct values).

Before being passed to your function, `nums` is rotated at an unknown pivot index k ($0 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (0-indexed). For example, `[0,1,2,4,4,4,5,6,6,7]` might be rotated at pivot index 5 and become `[4,5,6,6,7,0,1,2,4,4]`. Given the array `nums` after the rotation and an integer `target`, return `true` if `target` is in `nums`, or `false` if it is not in `nums`.

You must decrease the overall operation steps as much as possible.

Example 1:

Input: nums = [2,5,6,0,0,1,2], target = 0
Output: true

Example 2:

Input: nums = [2,5,6,0,0,1,2], target = 3
Output: false

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $-104 \leq \text{nums}[i] \leq 104$
- nums is guaranteed to be rotated at some pivot.
- $-104 \leq \text{target} \leq 104$

Follow up: This problem is similar to Search in Rotated Sorted Array, but nums may contain duplicates. Would this affect the runtime complexity? How and why?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def search(self, nums: list[int], target: int) -> bool:
        l = 0
        r = len(nums) - 1

        while l <= r:
            m = (l + r) // 2
            if nums[m] == target:
                return True
            if nums[l] == nums[m] == nums[r]:
```

```

    l += 1
    r -= 1
elif nums[l] <= nums[m]: # nums[l..m] are sorted
    if nums[l] <= target < nums[m]:
        r = m - 1
    else:
        l = m + 1
else: # nums[m..n - 1] are sorted
    if nums[m] < target <= nums[r]:
        l = m + 1

    else:
        r = m - 1

return False

```

• LeetDoocs

```

class Solution:
    def search(self, nums: List[int], target: int) -> bool:
        n = len(nums)
        l, r = 0, n - 1
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] > nums[r]:
                if nums[l] <= target <= nums[mid]:
                    r = mid
                else:
                    l = mid + 1
            elif nums[mid] < nums[r]:
                if nums[mid] < target <= nums[r]:
                    l = mid + 1
                else:
                    r = mid
            else:
                r -= 1
        return nums[l] == target

```

• SimplyLeet

```

# Function to search for the target in a rotated sorted array that may contain
duplicates.
class Solution:
    def search(self, nums: List[int], target: int) -> bool:
        low, high = 0, len(nums) - 1 # Initialize two pointers.

        while low <= high:
            mid = (low + high) // 2 # Find the mid-point.
            if nums[mid] == target: # If the mid element is the target, return
True.
                return True

            # If the values at low, mid, and high are equal, we cannot decide
which half is sorted.
            if nums[low] == nums[mid] == nums[high]:
                low += 1 # Shrink the search window.
                high -= 1
            # If the left half is sorted.
            elif nums[low] <= nums[mid]:
                if nums[low] <= target < nums[mid]: # Check if target is in
the left half.
                    high = mid - 1
                else:
                    low = mid + 1

            # Otherwise, the right half must be sorted.
            else:
                if nums[mid] < target <= nums[high]: # Check if target is in
the right half.
                    low = mid + 1
                else:
                    high = mid - 1

        return False # Target not found.

```

◆ Quick Review

Key Insights

- The array is rotated, so one of the two halves will always be sorted.
- Duplicates can complicate the binary search by obscuring which half is sorted.
- When duplicates cause ambiguity (i.e., when `nums[low]`, `nums[mid]`, and `nums[high]` are equal), the search space is reduced by incrementally moving the boundaries.
- A modified binary search can efficiently search the array on average, though the worst-case runtime can degrade to $O(n)$ due to duplicates.

Space & Time

Time Complexity: Worst-case $O(n)$,

Average-case $O(\log n)$

Space Complexity: $O(1)$

Solution

The solution employs a modified binary search technique. The main algorithm works by maintaining two pointers (low and high) and calculating a mid-point. If the target is equal to the element at the mid, the algorithm returns true. If duplicates are detected at the low, mid, and high indices, the boundaries are

adjusted to avoid ambiguity. Otherwise, the algorithm determines if the left or right side is sorted; if the target lies within the sorted portion, it moves the boundaries accordingly. This method accounts for potential duplicate values influencing the decision process.

Binary Search

□ #162 Find Peak Element

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search

Lists: AlgoMaster 600

Problem

A peak element is an element that is strictly greater than its neighbors.

Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks.

You may imagine that `nums[-1] = nums[n] =  $-\infty$` . In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 2

Explanation: 3 is a peak element and your function should return the index number 2.

Example 2:

Input: `nums = [1,2,1,3,5,6,4]`

Output: 5

Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $-231 \leq \text{nums}[i] \leq 231 - 1$
- $\text{nums}[i] \neq \text{nums}[i + 1]$ for all valid i .

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findPeakElement(self, nums: list[int]) -> int:
        l = 0
        r = len(nums) - 1

        while l < r:
            m = (l + r) // 2
            if nums[m] >= nums[m + 1]:
                r = m
            else:
                l = m + 1

        return l
```

• LeetDoocs

```
class Solution:
    def findPeakElement(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[mid] > nums[mid + 1]:
                right = mid
            else:
                left = mid + 1
        return left
```

• SimplyLeet

Python solution for Find Peak Element

```
def findPeakElement(nums):
    # Initialize pointers for binary search
    low, high = 0, len(nums) - 1

    # Continue binary search until low meets high
    while low < high:
        mid = (low + high) // 2 # Find the middle index

        # If the element at mid is less than its right neighbor,
        # then a peak exists in the right half.
        if nums[mid] < nums[mid + 1]:
            low = mid + 1
        else:
            # Otherwise, a peak exists in the left half,
            # including possibly mid itself.
            high = mid

    # The low pointer eventually points to a peak element.
    return low

# Example usage:
# result = findPeakElement([1,2,3,1])
# print(result)
```

◆ Quick Review

Key Insights

- Use a binary search strategy to achieve the required $O(\log n)$ time complexity.
- At each step, compare the middle element with its right neighbor to decide which half to search.
- Due to the $-\infty$ boundaries, there is always at least one peak in the array.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

We use a binary search approach by maintaining two pointers: low and high. We calculate a mid index and check if it is a peak by comparing it with its neighbors. If the element to the right of mid is greater, then a peak must exist in the right half of the array; otherwise, it will be in the left half. This greedy move is valid due to the boundary conditions where the ends are effectively $-\infty$. By continually narrowing the search space, we can efficiently find a peak element.

Binary Search

□ #240 Search a 2D Matrix II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Divide and Conquer ·
Matrix

Lists: AlgoMaster 600

Problem

Write an efficient algorithm that searches for a value target in an $m \times n$ integer matrix matrix.

This matrix has the following properties:

- Integers in each row are sorted in ascending from left to right.
- Integers in each column are sorted in ascending from top to bottom.

Example 1:

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

Input: matrix =
[[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]],
target = 5
Output: true

Example 2:

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

```
Input: matrix =  
[[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]],  
target = 20  
Output: false
```

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq n, m \leq 300$
- $-109 \leq \text{matrix}[i][j] \leq 109$

- All the integers in each row are sorted in ascending order.
- All the integers in each column are sorted in ascending order.
- $-109 \leq \text{target} \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def searchMatrix(self, matrix: list[list[int]], target: int) -> bool:
        r = 0
        c = len(matrix[0]) - 1

        while r < len(matrix) and c >= 0:
            if matrix[r][c] == target:
                return True
            if target < matrix[r][c]:
                c -= 1
            else:
                r += 1

        return False
```

• LeetDoocs

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        for row in matrix:
            j = bisect_left(row, target)
            if j < len(matrix[0]) and row[j] == target:
                return True
        return False
```

```

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        i, j = m - 1, 0
        while i >= 0 and j < n:
            if matrix[i][j] == target:
                return True
            if matrix[i][j] > target:
                i -= 1
            else:
                j += 1
        return False

```

• SimplyLeet

Python implementation with detailed comments

```

def searchMatrix(matrix, target):
    # If the matrix is empty, return False
    if not matrix or not matrix[0]:
        return False

    # Set initial position to top-right corner of the matrix
    row, col = 0, len(matrix[0]) - 1

    # Iterate until we either find the target or exhaust the matrix bounds
    while row < len(matrix) and col >= 0:
        # Store the current element
        current = matrix[row][col]
        # Check if the current element is equal to target
        if current == target:
            return True
        # If the current element is greater, move left
        elif current > target:
            col -= 1

```

```
# If smaller, move down
else:
    row += 1

# Target was not found
return False
```

◆ Quick Review

Key Insights

- Each row and column is sorted, which allows us to eliminate multiple rows or columns with one comparison.
- Starting from the top-right corner (or bottom-left) is beneficial because at this position, one can decide to move left or down based on how the current value compares with the target.
- This "staircase" search leads to a linear time algorithm relative to the sum of dimensions of the matrix.

Space & Time

Time Complexity: $O(m + n)$ where m is the number of rows and n is the number of columns.

Space Complexity: $O(1)$

Solution

The algorithm starts from the top-right corner of the matrix. At each step, compare the current element with the target:

- If the current element equals the target, return true.
- If the current element is greater than the target, move left because all elements below are larger.
- If the current element is smaller than the target, move down because all elements to the left are smaller. This method ensures that in each step one row or one column is eliminated from consideration, leading to an efficient search.

Binary Search

□ #475 Heaters

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search · Sorting
Lists: AlgoMaster 600

Problem

Winter is coming! During the contest, your first job is to design a standard heater with a fixed warm radius to warm all the houses.

Every house can be warmed, as long as the house is within the heater's warm radius range. Given the positions of houses and heaters on a horizontal line, return the minimum radius standard of heaters so that those heaters could cover all houses.

Notice that all the heaters follow your radius standard, and the warm radius will be the same.

Example 1:

Input: houses = [1,2,3], heaters = [2]

Output: 1

Explanation: The only heater was placed in the position 2, and if we use the radius 1 standard, then all the houses can be warmed.

Example 2:

Input: houses = [1,2,3,4], heaters = [1,4]

Output: 1

Explanation: The two heaters were placed at positions 1 and 4. We need to use a radius 1 standard, then all the houses can be warmed.

Example 3:

Input: houses = [1,5], heaters = [2]

Output: 3

Constraints:

- $1 \leq \text{houses.length}, \text{heaters.length} \leq 3 * 10^4$
- $1 \leq \text{houses}[i], \text{heaters}[i] \leq 10^9$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def findRadius(self, houses: List[int], heaters: List[int]) -> int:
        houses.sort()
        heaters.sort()

        def check(r):
            m, n = len(houses), len(heaters)
            i = j = 0
            while i < m:
                if j >= n:
                    return False
                mi = heaters[j] - r
                mx = heaters[j] + r
                if houses[i] < mi:
                    return False
                if houses[i] > mx:
                    j += 1
            else:
                i += 1
            return True
```

```

left, right = 0, int(1e9)
while left < right:
    mid = (left + right) >> 1
    if check(mid):
        right = mid
    else:
        left = mid + 1
return left

```

• SimplyLeet

```

# Function to compute the minimum radius required for heaters to warm all
houses
def findRadius(houses, heaters):
    # Sort both arrays for efficient searching
    houses.sort() # O(n log n)
    heaters.sort() # O(m log m)

    radius = 0
    # Iterate over each house and find the nearest heater using binary search
    for house in houses:
        low, high = 0, len(heaters) - 1

        # Binary search to find the right position in heaters array
        while low <= high:
            mid = (low + high) // 2
            if heaters[mid] < house:
                low = mid + 1
            else:
                high = mid - 1
        # low is the index of the first heater >= house (if exists)
        # high is the index of the last heater <= house (if exists)
        left_dist = abs(house - heaters[high]) if high >= 0 else float('inf')

```

```
        right_dist = abs(heaters[low] - house) if low < len(heaters) else float('inf')
        nearest = min(left_dist, right_dist)
        radius = max(radius, nearest)
    return radius

# Example usage:
houses = [1,5]
heaters = [2]
print(findRadius(houses, heaters)) # Expected Output: 3
```

◆ Quick Review

Key Insights

- Sort both the houses and the heaters arrays to efficiently search for the nearest heater for any given house.
- For each house, use binary search to locate the heater that is closest.
- The minimum required radius is the maximum distance a house is from its nearest heater.
- The problem leverages binary search for each house, leading to an overall efficient solution.

Space & Time

Time Complexity: $O(n \log m)$ where n is the number of houses and m is the number of

heaters.

Space Complexity: $O(1)$ (ignoring the space required for input sorting, which can be done in-place)

Solution

The solution first sorts the houses and heaters arrays. For each house, binary search is applied to the heaters array to find the position where the heater value is just greater than or equal to the house's position. The distance to the closest heater is then the minimum of the distance from the house to the heater just before or just after this position. Finally, the answer is the maximum of these minimum distances, ensuring that every house is within the coverage of some heater.

Binary Search

□ #1482 Minimum Number of Days to Make m Bouquets

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: AlgoMaster 600

Problem

You are given an integer array `bloomDay`, an integer `m` and an integer `k`.

You want to make `m` bouquets. To make a bouquet, you need to use `k` adjacent flowers from the garden.

The garden consists of `n` flowers, the `i`th flower will bloom in the `bloomDay[i]` and then can be used in exactly one bouquet.

Return the minimum number of days you need to wait to be able to make `m` bouquets from the garden. If it is impossible to make `m` bouquets return `-1`.

Example 1:

Input: bloomDay = [1,10,3,10,2], m = 3, k = 1

Output: 3

Explanation: Let us see what happened in the first three days. x means flower bloomed and _ means flower did not bloom in the garden.

We need 3 bouquets each should contain 1 flower.

After day 1: [x, _, _, _, _] // we can only make one bouquet.

After day 2: [x, _, _, _, x] // we can only make two bouquets.

After day 3: [x, _, x, _, x] // we can make 3 bouquets. The answer is 3.

Example 2:

Input: bloomDay = [1,10,3,10,2], m = 3, k = 2

Output: -1

Explanation: We need 3 bouquets each has 2 flowers, that means we need 6 flowers. We only have 5 flowers so it is impossible to get the needed bouquets and we return -1.

Example 3:

Input: bloomDay = [7,7,7,7,12,7,7], m = 2, k = 3

Output: 12

Explanation: We need 2 bouquets each should have 3 flowers.

Here is the garden after the 7 and 12 days:

After day 7: [x, x, x, x, _, x, x]

We can make one bouquet of the first three flowers that bloomed. We cannot make another bouquet from the last three flowers that bloomed because they are not adjacent.

After day 12: [x, x, x, x, x, x, x]

It is obvious that we can make two bouquets in different ways.

Constraints:

- bloomDay.length == n
- 1 <= n <= 105
- 1 <= bloomDay[i] <= 109
- 1 <= m <= 106
- 1 <= k <= n

Community Solutions (Python)

- WalkCC

```

class Solution:
    def minDays(self, bloomDay: list[int], m: int, k: int) -> int:
        if len(bloomDay) < m * k:
            return -1

        def getBouquetCount(waitingDays: int) -> int:
            """
            Returns the number of bouquets (k flowers needed) can be made after the
            `waitingDays`.
            """

            bouquetCount = 0
            requiredFlowers = k
            for day in bloomDay:
                if day > waitingDays:
                    # Reset `requiredFlowers` since there was not enough adjacent
                    flowers.
                    requiredFlowers = k
                else:
                    requiredFlowers -= 1
                    if requiredFlowers == 0:
                        # Use k adjacent flowers to make a bouquet.

                        bouquetCount += 1
                        requiredFlowers = k
            return bouquetCount

            l = min(bloomDay)
            r = max(bloomDay)

            while l < r:
                mid = (l + r) // 2
                if getBouquetCount(mid) >= m:

                    r = mid
                else:
                    l = mid + 1

            return l

```

• LeetDoocs

```

class Solution:
    def minDays(self, bloomDay: List[int], m: int, k: int) -> int:
        def check(days: int) -> int:
            cnt = cur = 0
            for x in bloomDay:
                cur = cur + 1 if x <= days else 0
                if cur == k:
                    cnt += 1
                    cur = 0
            return cnt >= m

        mx = max(bloomDay)
        l = bisect_left(range(mx + 2), True, key=check)
        return -1 if l > mx else l

```

• SimplyLeet

Python solution for the problem

```

def minDays(bloomDay, m, k):
    # If total flowers are less than required, return -1 early
    if m * k > len(bloomDay):
        return -1

    # Helper function to check if we can make required bouquets by day "day"
    def canMakeBouquets(day):
        bouquets = 0      # Number of bouquets made so far

        consecutive = 0    # Count of consecutive blooming flowers
        for bloom in bloomDay:
            # If the flower blooms by the given day, increase consecutive count
            if bloom <= day:
                consecutive += 1
                # If we reached k consecutive flowers, one bouquet is formed
                if consecutive == k:
                    bouquets += 1
                    consecutive = 0 # Reset for the next bouquet
            else:

```

```

        consecutive = 0 # Reset count if flower not bloomed
        # Early return if required bouquets have been made
        if bouquets >= m:
            return True
        return False

```

```

low, high = min(bloomDay), max(bloomDay)
result = -1

```

```

# Binary search on the day range

```

```

while low <= high:
    mid = (low + high) // 2 # middle day value
    if canMakeBouquets(mid):
        result = mid # candidate answer
        high = mid - 1 # try to find a smaller day
    else:
        low = mid + 1 # need more days
return result

```

```

# Example Test Cases

```

```

print(minDays([1,10,3,10,2], 3, 1)) # Expected output: 3
print(minDays([1,10,3,10,2], 3, 2)) # Expected output: -1
print(minDays([7,7,7,7,12,7,7], 2, 3)) # Expected output: 12

```

◆ Quick Review

Key Insights

- Use binary search on the range of possible days since waiting more days only increases the number of available flowers.
- At each day guess, simulate whether it is possible to form m bouquets by iterating over the `bloomDay` array.

- Count only bouquets formed by k consecutive flowers (reset count when encountering a flower that hasn't bloomed by the considered day).
- Early termination: if the total number of flowers is insufficient to form m bouquets (i.e. if $m * k > n$), return -1 immediately.

Space & Time

Time Complexity: $O(n * \log(\max(\text{bloomDay})))$ – For each binary search iteration, we scan the entire array.

Space Complexity: $O(1)$ – Constant extra space is used.

Solution

The problem can be efficiently solved using a binary search approach. The idea is to determine a minimum day such that there are enough adjacent flowers (k consecutive ones) to form m bouquets. We define a helper function to check if for a given day, at least m bouquets are possible.

The binary search is performed over the range of days $[\min(\text{bloomDay}), \max(\text{bloomDay})]$. For each mid value (representing a day), iterate

through the bloomDay array to count how many bouquets can be created if only flowers with bloomDay $\leq$ mid are considered bloomed. The count resets if a flower that hasn't bloomed is encountered. If the bouquet count meets or exceeds m, then this day is a potential answer, and we try to find a smaller day. Otherwise, we search in the higher day range.

Data structures used include simple counters and pointers for iterating over the array. The algorithmic approach is a combination of greedy checking and binary search.

Binary Search

□ #1552 Magnetic Force Between Two Balls

MED

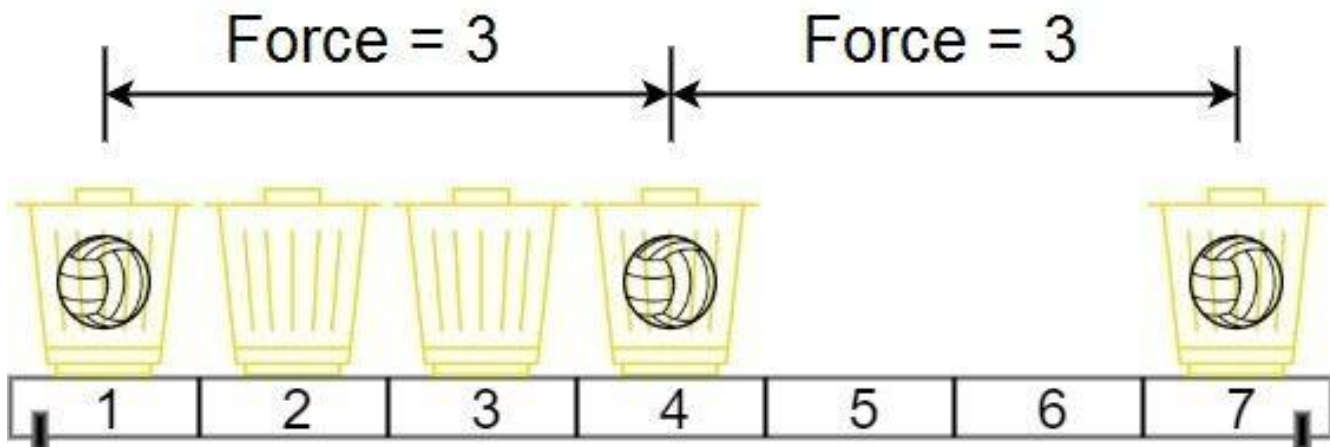
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Sorting
Lists: AlgoMaster 600

Problem

In the universe Earth C-137, Rick discovered a special form of magnetic force between two balls if they are put in his new invented basket. Rick has n empty baskets, the i th basket is at position $[i]$, Morty has m balls and needs to distribute the balls into the baskets such that the minimum magnetic force between any two balls is maximum.

Rick stated that magnetic force between two different balls at positions x and y is $|x - y|$. Given the integer array position and the integer m . Return the required force.

Example 1:



Input: position = [1,2,3,4,7], m = 3

Output: 3

Explanation: Distributing the 3 balls into baskets 1, 4 and 7 will make the magnetic force between ball pairs [3, 3, 6]. The minimum magnetic force is 3. We cannot achieve a larger minimum magnetic force than 3.

Example 2:

Input: position = [5,4,3,2,1,1000000000], m = 2

Output: 999999999

Explanation: We can use baskets 1 and 1000000000.

Constraints:

- $n == \text{position.length}$
- $2 \leq n \leq 105$
- $1 \leq \text{position}[i] \leq 10^9$
- All integers in position are distinct.
- $2 \leq m \leq \text{position.length}$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def maxDistance(self, position: List[int], m: int) -> int:
        position.sort()

        l = 1
        r = position[-1] - position[0]

        def numBalls(force: int) -> int:
            balls = 0
            prevPosition = -force

            for pos in position:
                if pos - prevPosition >= force:
                    balls += 1
                    prevPosition = pos
            return balls

        while l < r:
            mid = r - (r - l) // 2
            if numBalls(mid) >= m:
                l = mid
            else:
                r = mid - 1

        return l

```

• LeetDoocs

```

class Solution:
    def maxDistance(self, position: List[int], m: int) -> int:
        def check(f: int) -> bool:
            prev = -inf
            cnt = 0
            for curr in position:
                if curr - prev >= f:
                    prev = curr
                    cnt += 1
            return cnt < m

```

```

position.sort()
l, r = 1, position[-1]
return bisect_left(range(l, r + 1), True, key=check)

```

• SimplyLeet

Python solution for Magnetic Force Between Two Balls

```

def maxDistance(position, m):
    # Sort the basket positions
    position.sort()

    # Function to check if we can place m balls with at least min_force between
    # them
    def canPlace(min_force):
        count = 1 # First ball placed in the first basket
        last_position = position[0]
        # Try to place remaining m-1 balls

        for pos in position[1:]:
            if pos - last_position >= min_force:
                count += 1
                last_position = pos
                if count == m: # All m balls placed successfully
                    return True
        return False

    # Binary search boundaries
    low, high = 1, position[-1] - position[0]

    best_force = 0

    while low <= high:
        mid = (low + high) // 2 # Candidate minimum force
        if canPlace(mid):
            best_force = mid # mid is a possible answer, try to find a larger
            distance
            low = mid + 1
        else:
            high = mid - 1
    return best_force

```

```
# Example usage:  
print(maxDistance([1,2,3,4,7], 3)) # Output should be 3
```

◆ Quick Review

Key Insights

- Sort the basket positions to enable efficient checking.
- Use binary search on the “answer” (the minimum force) rather than iterating over all possible forces.
- For a given minimum force, use a greedy strategy to place balls sequentially, placing a ball whenever the distance from the last placed ball meets the minimum requirement.
- Adjust the search boundaries based on whether the placement was possible.

Space & Time

Time Complexity: $O(n \log(\text{max_position} - \text{min_position}))$

Space Complexity: $O(1)$ (ignoring the space used by sorting in-place)

Solution

The solution starts by sorting the basket positions. Then, the maximum possible minimum force is determined using binary search. The binary search explores all possible force values between 1 and the difference between the maximum and minimum basket positions. For each candidate force, a helper function checks if it is possible to place m balls such that each ball is at least the candidate force apart. This greedy placement starts from the first basket and places subsequent balls only when the current basket's position is at least the candidate force away from the last placed ball. Adjust the binary search boundaries based on whether the placement is possible or not, and finally, return the largest force for which placement is possible.

Stacks

Round 2 · High · Medium · 6 questions

Stacks

□ #71 Simplify Path

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack

Lists: AlgoMaster 600

Problem

You are given an absolute path for a Unix-style file system, which always begins with a slash '/'. Your task is to transform this absolute path into its simplified canonical path.

The rules of a Unix-style file system are as follows:

- A single period '.' represents the current directory.
- A double period '..' represents the previous/parent directory.
- Multiple consecutive slashes such as '/' and '//' are treated as a single slash '/'.
- Any sequence of periods that does not match the rules above should be treated as a valid directory or file name. For example, '...'

and '....' are valid directory or file names.
The simplified canonical path should follow these rules:

- The path must start with a single slash '/'.
- Directories within the path must be separated by exactly one slash '/'.
- The path must not end with a slash '/', unless it is the root directory.
- The path must not have any single or double periods ('.' and '..') used to denote current or parent directories.

Return the simplified canonical path.

Example 1:

Input: path = "/home/"

Output: "/home"

Explanation:

The trailing slash should be removed.

Example 2:

Input: path = "/home//foo/"

Output: "/home/foo"

Explanation:

Multiple consecutive slashes are replaced by a single one.

Example 3:

Input: path =

"/home/user/Documents/../Pictures"

Output: **"/home/user/Pictures"**

Explanation:

A double period **".."** refers to the directory up a level (the parent directory).

Example 4:

Input: path = **"/../"**

Output: **"/"**

Explanation:

Going one level up from the root directory is not possible.

Example 5:

Input: path = **"/.../a/../b/c/../d/./"**

Output: **"/.../b/d"**

Explanation:

"..." is a valid name for a directory in this problem.

Constraints:

- $1 \leq \text{path.length} \leq 3000$
- path consists of English letters, digits, period '.', slash '/' or '_'.

- path is a valid absolute Unix path.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def simplifyPath(self, path: str) -> str:
        stack = []

        for str in path.split('/'):
            if str in ('', '.'):
                continue
            if str == '..':
                if stack:
                    stack.pop()
            else:
                stack.append(str)

        return '/' + '/'.join(stack)
```

• LeetDoocs

```
class Solution:
    def simplifyPath(self, path: str) -> str:
        stk = []
        for s in path.split('/'):
            if not s or s == '.':
                continue
            if s == '..':
                if stk:
                    stk.pop()
            else:
                stk.append(s)
        return '/' + '/'.join(stk)
```

• SimplyLeet

```
def simplifyPath(path):  
    # Split the path by '/'  
    parts = path.split('/')  
    # Initialize a stack to store valid directory names  
    stack = []  
    # Process each part from the split operation  
    for part in parts:  
        # Skip empty parts or the current directory symbol  
        if not part or part == '.':  
            continue  
  
        # If encountering a parent directory symbol, pop if possible  
        if part == '..':  
            if stack:  
                stack.pop()  
        else:  
            # Otherwise, push the valid directory name onto the stack  
            stack.append(part)  
    # Construct the canonical path by joining the stack with '/'  
    return '/' + '/'.join(stack)  
  
# Example usage  
print(simplifyPath("/home//foo/"))
```

◆ Quick Review

Key Insights

- Use a stack to handle the directory names.
- Ignore empty strings or '.' as they represent no change in the directory.
- When encountering '..', pop the top of the stack if it's not empty.
- All other valid strings are treated as directory names and pushed onto the stack.

- Finally, join the elements in the stack with '/' and prepend a slash to form the canonical path.

Space & Time

Time Complexity: $O(n)$ where n is the length of the path string.

Space Complexity: $O(n)$ in the worst-case scenario where all directories are valid and stored in the stack.

Solution

We approach the problem by:

- Splitting the input path by the '/' delimiter.
- Iterating through the resulting tokens:
- Skip tokens that are empty or equal to '.'.
- When encountering '..', pop an element from the stack if it exists.
- Otherwise, push the token onto the stack as a valid directory name.
- After processing all tokens, the stack contains the valid path components. We then construct the simplified path by concatenating these components with '/' in between, making sure it starts with a '/'.

- Edge cases include handling the root directory where the stack might be empty.

Stacks

□ #316 Remove Duplicate Letters

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Greedy · Monotonic Stack
Lists: AlgoMaster 600

Problem

Given a string *s*, remove duplicate letters so that every letter appears once and only once. You must make sure your result is the smallest in lexicographical order among all possible results.

Example 1:

Input: *s* = "bcabc"
Output: "abc"

Example 2:

Input: *s* = "cbacdcbc"
Output: "acdb"

Constraints:

- $1 \leq s.length \leq 104$
- *s* consists of lowercase English letters.

Note: This question is the same as 1081: <https://leetcode.com/problems/smallest-subsequence-of-distinct-characters/>

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeDuplicateLetters(self, s: str) -> str:
        ans = []
        count = collections.Counter(s)
        used = [False] * 26

        for c in s:
            count[c] -= 1
            if used[ord(c) - ord('a')]:
                continue

            while ans and ans[-1] > c and count[ans[-1]] > 0:
                used[ord(ans[-1]) - ord('a')] = False
                ans.pop()
            ans.append(c)
            used[ord(ans[-1]) - ord('a')] = True

        return ''.join(ans)
```

• LeetDoocs

```
class Solution:
    def removeDuplicateLetters(self, s: str) -> str:
        last = {c: i for i, c in enumerate(s)}
        stk = []
        vis = set()
        for i, c in enumerate(s):
            if c in vis:
                continue
            while stk and stk[-1] > c and last[stk[-1]] > i:
                vis.remove(stk.pop())

            stk.append(c)
            vis.add(c)
        return ''.join(stk)
```



```

class Solution:
    def removeDuplicateLetters(self, s: str) -> str:
        count, in_stack = [0] * 128, [False] * 128
        stack = []
        for c in s:
            count[ord(c)] += 1
        for c in s:
            count[ord(c)] -= 1
            if in_stack[ord(c)]:
                continue

            while len(stack) and stack[-1] > c:
                peek = stack[-1]
                if count[ord(peek)] < 1:
                    break
                in_stack[ord(peek)] = False
                stack.pop()
            stack.append(c)
            in_stack[ord(c)] = True
        return ''.join(stack)

```

• SimplyLeet

```

class Solution:
    def removeDuplicateLetters(self, s: str) -> str:
        # Map each character to its last occurrence index
        last_occurrence = {c: i for i, c in enumerate(s)}
        stack = [] # Monotonic stack to build the result
        seen = set() # Set to record characters already in the stack

        # Iterate over each character and its index
        for i, char in enumerate(s):
            # Skip if character is already in the stack

```

```
        if char in seen:
            continue
        # Remove characters that are larger than current character and
        appear later
        while stack and char < stack[-1] and last_occurrence[stack[-1]] >
i:
            removed_char = stack.pop()
            seen.remove(removed_char)
            stack.append(char)
            seen.add(char)
        # Join the stack to form the final result
        return "".join(stack)
```

◆ Quick Review

Key Insights

- Use a monotonic stack to build the resulting string.
- Store the last occurrence of each character to know if it can be safely removed.
- Use a set (or boolean array) to track which characters have been added to avoid duplication.
- Greedily remove characters from the stack if the current character is smaller and the top of the stack appears again later in the string.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, because each character is

processed once.

Space Complexity: $O(n)$ for the stack and auxiliary data structures.

Solution

We iterate over the string while maintaining a stack and a record of whether each character is already in the result. For each character, if it is already in the result, we skip it. Otherwise, we check if the current character is smaller than the last character in the stack and if that last character appears later in the string. If both conditions are met, we pop it from the stack so that the overall result remains lexicographically minimal. Then we add the current character to the stack and mark it as seen.

Stacks

□ #636 Exclusive Time of Functions

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack

Lists: AlgoMaster 600

Problem

On a single-threaded CPU, we execute a program containing n functions. Each function has a unique ID between 0 and $n-1$.

Function calls are stored in a call stack: when a function call starts, its ID is pushed onto the stack, and when a function call ends, its ID is popped off the stack. The function whose ID is at the top of the stack is the current function being executed. Each time a function starts or ends, we write a log with the ID, whether it started or ended, and the timestamp.

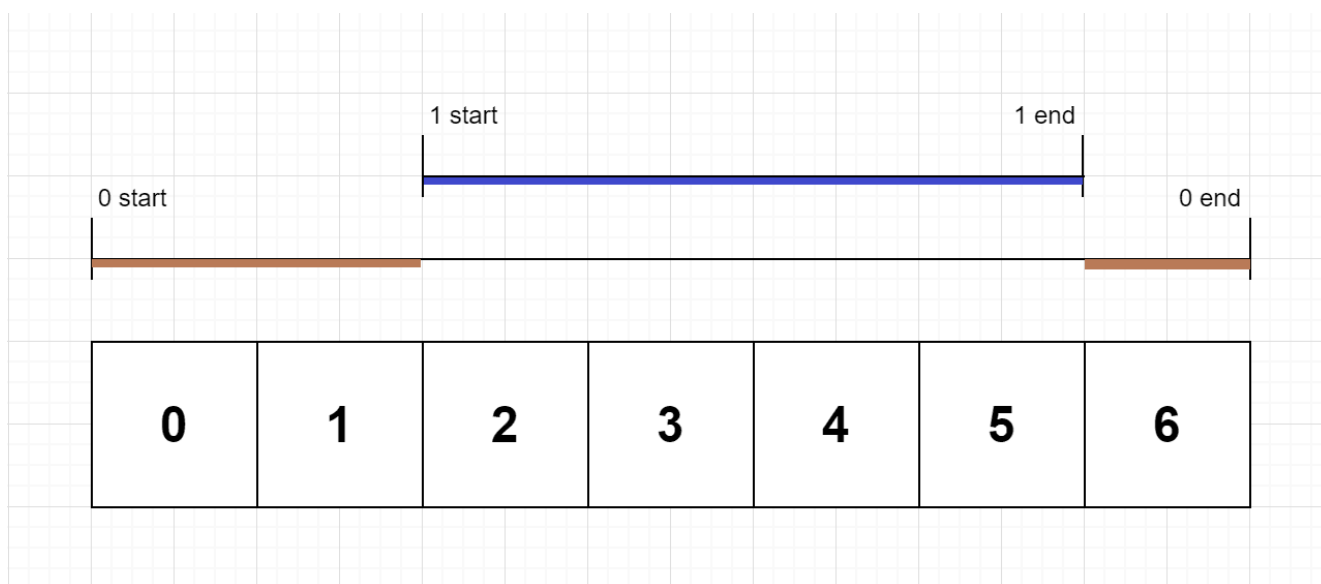
You are given a list `logs`, where `logs[i]` represents the i th log message formatted as a string `"{function_id}:{start | end}:{timestamp}"`. For example, `"0:start:3"` means a function call with function ID 0 started

at the beginning of timestamp 3, and "1:end:2" means a function call with function ID 1 ended at the end of timestamp 2. Note that a function can be called multiple times, possibly recursively.

A function's exclusive time is the sum of execution times for all function calls in the program. For example, if a function is called twice, one call executing for 2 time units and another call executing for 1 time unit, the exclusive time is $2 + 1 = 3$.

Return the exclusive time of each function in an array, where the value at the i th index represents the exclusive time for the function with ID i .

Example 1:



Input: $n = 2$, logs = ["0:start:0","1:start:2","1:end:5","0:end:6"]

Output: [3,4]

Explanation:

Function 0 starts at the beginning of time 0, then it executes 2 for units of time and reaches the end of time 1.

Function 1 starts at the beginning of time 2, executes for 4 units of time, and ends at the end of time 5.

Function 0 resumes execution at the beginning of time 6 and executes for 1 unit of time.

So function 0 spends $2 + 1 = 3$ units of total time executing, and function 1 spends 4 units of total time executing.

Example 2:

Input: $n = 1$, logs =

["0:start:0","0:start:2","0:end:5","0:start:6","0:end:6","0:end:7"]

Output: [8]

Explanation:

Function 0 starts at the beginning of time 0, executes for 2 units of time, and recursively calls itself.

Function 0 (recursive call) starts at the beginning of time 2 and executes for 4 units of time.

Function 0 (initial call) resumes execution then immediately calls itself again.

Function 0 (2nd recursive call) starts at the beginning of time 6 and executes for 1 unit of time.

Function 0 (initial call) resumes execution at the beginning of time 7 and executes for 1 unit of time.

Example 3:

Input: $n = 2$, logs =

["0:start:0","0:start:2","0:end:5","1:start:6","1:end:6","0:end:7"]

Output: [7,1]

Explanation:

Function 0 starts at the beginning of time 0, executes for 2 units of time, and recursively calls itself.

Function 0 (recursive call) starts at the beginning of time 2 and executes for 4 units of time.

Function 0 (initial call) resumes execution then immediately calls function 1.

Function 1 starts at the beginning of time 6, executes 1 unit of time, and ends at the end of time 6.

Function 0 resumes execution at the beginning of time 6 and executes for 2 units of time.

Constraints:

- $1 \leq n \leq 100$
- $2 \leq \text{logs.length} \leq 500$
- $0 \leq \text{function_id} < n$
- $0 \leq \text{timestamp} \leq 109$
- No two start events will happen at the same timestamp.
- No two end events will happen at the same timestamp.
- Each function has an "end" log for each "start" log.

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def exclusiveTime(self, n: int, logs: List[str]) -> List[int]:
        stk = []
        ans = [0] * n
        pre = 0
        for log in logs:
            i, op, t = log.split(":")
            i, cur = int(i), int(t)
            if op[0] == "s":
                if stk:
                    ans[stk[-1]] += cur - pre
                    stk.append(i)
                    pre = cur
            else:
                ans[stk.pop()] += cur - pre + 1
                pre = cur + 1
        return ans
```

• SimplyLeet

```
# Python solution for Exclusive Time of Functions

def exclusiveTime(n, logs):
    # Initialize result array with zeros for each function
    result = [0] * n
    # Stack to keep track of the function ids
    stack = []
    # Previous timestamp to calculate elapsed time
    prev_time = 0

    # Process each log entry
    for log in logs:
        # Split the log string into parts
        fn_id, typ, time_str = log.split(":")
        curr_time = int(time_str)
        fn_id = int(fn_id)

        if typ == "start":
            # If there is a function currently running, update its exclusive
            time
            if stack:
                result[stack[-1]] += curr_time - prev_time
            # Push the new function id onto the stack
            stack.append(fn_id)
            # Update previous time to current timestamp
            prev_time = curr_time
        else: # 'end' log
            # Pop the current function from the stack
            completed_fn = stack.pop()
            # Add the elapsed time including the current time unit
            result[completed_fn] += curr_time - prev_time + 1
```



```
# Update previous time to one unit after the current timestamp
prev_time = curr_time + 1

return result

# Example usage:
n = 2
logs = ["0:start:0", "1:start:2", "1:end:5", "0:end:6"]
print(exclusiveTime(n, logs)) # Output: [3, 4]
```

◆ Quick Review

Key Insights

- Use a stack to simulate function calls; the top of the stack represents the currently executing function.
- When a function starts while another is running, update the exclusive time of the currently running function.
- For an "end" log, add the elapsed time (including the current timestamp) to the function on top of the stack.
- Keep track of the previous timestamp to calculate elapsed durations correctly.

Space & Time

Time Complexity: $O(n)$ where n is the number of log entries.

Space Complexity: $O(n)$ in the worst-case scenario due to the stack.

Solution

We use a stack to track the active functions. As we iterate through each log:

- For a "start" log, if there is a function currently executing (top of the stack), update its exclusive time with the time elapsed since the last timestamp.**
- Then push the new function onto the stack and update the previous timestamp.**
- For an "end" log, pop the completed function from the stack and update its exclusive time by adding the elapsed time plus one (to account for the inclusive end timestamp). Then update the previous timestamp to one more than the current timestamp. This approach efficiently tracks the execution time intervals for each function.**

Stacks

□ #946 Validate Stack Sequences

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Simulation

Lists: AlgoMaster 600

Problem

Given two integer arrays pushed and popped each with distinct values, return true if this could have been the result of a sequence of push and pop operations on an initially empty stack, or false otherwise.

Example 1:

```
Input: pushed = [1,2,3,4,5], popped = [4,5,3,2,1]
Output: true
Explanation: We might do the following sequence:
push(1), push(2), push(3), push(4),
pop() -> 4,
push(5),
pop() -> 5, pop() -> 3, pop() -> 2, pop() -> 1
```

Example 2:

```
Input: pushed = [1,2,3,4,5], popped = [4,3,5,1,2]
Output: false
Explanation: 1 cannot be popped before 2.
```

Constraints:

- $1 \leq \text{pushed.length} \leq 1000$

- $0 \leq \text{pushed}[i] \leq 1000$
- All the elements of pushed are unique.
- `popped.length == pushed.length`
- popped is a permutation of pushed.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def validateStackSequences(
        self,
        pushed: list[int],
        popped: list[int],
    ) -> bool:
        stack = []
        i = 0 # popped's index

        for x in pushed:

            stack.append(x)
            while stack and stack[-1] == popped[i]:
                stack.pop()
                i += 1

        return not stack
```

• LeetDoocs

```
class Solution:
    def validateStackSequences(self, pushed: List[int], popped: List[int]) ->
bool:
    stk = []
    i = 0
    for x in pushed:
        stk.append(x)
        while stk and stk[-1] == popped[i]:
            stk.pop()
            i += 1
    return i == len(popped)
```

• SimplyLeet

```
# Function to validate stack sequences using simulation with a stack.
def validateStackSequences(pushed, popped):
    # Initialize an empty stack.
    stack = []
    # Initialize pointer for popped array.
    popped_index = 0

    # Iterate through each element in the pushed array.
    for num in pushed:
        stack.append(num) # Push the current element onto the stack.

        # Check if the current stack top matches the expected popped element.
        while stack and stack[-1] == popped[popped_index]:
            stack.pop() # Pop the element from the stack.
            popped_index += 1 # Move to the next element in the popped
sequence.

    # If the stack is empty, the sequence is valid.
    return not stack

# Example Usage
pushed = [1,2,3,4,5]

popped = [4,5,3,2,1]
print(validateStackSequences(pushed, popped)) # Expected output: True
```

◆ Quick Review

Key Insights

- Use a stack to simulate the push and pop operations.
- Iterate over the pushed array, pushing each element to the stack.
- After each push, check if the top of the stack equals the current element in the popped array. If so, perform pop operations.
- Continue this process until either all elements are processed or a mismatch occurs.
- The simulation confirms the validity of the popped sequence if the stack is empty at the end.

Space & Time

Time Complexity: $O(n)$ where n is the number of elements, as each element is pushed and popped at most once.

Space Complexity: $O(n)$ for the auxiliary stack used during the simulation.

Solution

We simulate the process using a stack:

- Initialize an empty stack and an index pointer for the popped array.

- Traverse the pushed sequence, pushing each element onto the stack.
- After each push, while the stack is not empty and the element on top of the stack equals the current element in popped (indicated by the index pointer), pop from the stack and increment the pointer.
- If all operations complete and the stack is empty, the sequence is valid; otherwise, it is not. The key trick is continuously comparing the top of the stack with the next element in popped and performing pop operations as long as they match.

Stacks

□ #1249 Minimum Remove to Make Valid Parentheses

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack

Lists: AlgoMaster 600

Problem

Given a string *s* of '(' , ')' and lowercase English characters.

Your task is to remove the minimum number of parentheses ('(' or ')', in any positions) so that the resulting parentheses string is valid and return any valid string.

Formally, a parentheses string is valid if and only if:

- It is the empty string, contains only lowercase characters, or
- It can be written as *AB* (*A* concatenated with *B*), where *A* and *B* are valid strings, or
- It can be written as (*A*), where *A* is a valid string.

Example 1:

Input: `s = "lee(t(c)o)de)"`

Output: `"lee(t(c)o)de"`

Explanation: `"lee(t(co)de)"` , `"lee(t(c)ode)"` would also be accepted.

Example 2:

Input: `s = "a)b(c)d"`

Output: `"ab(c)d"`

Example 3:

Input: `s = ") ) ( ( "`

Output: `""`

Explanation: An empty string is also valid.

Constraints:

- $1 \leq s.length \leq 105$
- `s[i]` is either `'('` , `')'`, or lowercase English letter.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minRemoveToMakeValid(self, s: str) -> str:
        stack = [] # unpaired '(' indices
        chars = list(s)

        for i, c in enumerate(chars):
            if c == '(':
                stack.append(i) # Record the unpaired '(' index.
            elif c == ')':
                if stack:
```

```

        stack.pop() # Find a pair
    else:
        chars[i] = '*' # Mark the unpaired ')' as '*'.

# Mark the unpaired '(' as '*'.
while stack:
    chars[stack.pop()] = '*'

return ''.join(chars).replace('*', '')

```

• LeetDoocs

```

class Solution:
    def minRemoveToMakeValid(self, s: str) -> str:
        stk = []
        x = 0
        for c in s:
            if c == ')' and x == 0:
                continue
            if c == '(':
                x += 1
            elif c == ')':
                x -= 1
            stk.append(c)
        x = 0
        ans = []
        for c in stk[::-1]:
            if c == '(' and x == 0:
                continue
            if c == ')':
                x += 1
            elif c == '(':
                x -= 1
            ans.append(c)
        return ''.join(ans[::-1])

```

• SimplyLeet

```
# Define a function to remove invalid parentheses and return a valid string.
def minRemoveToMakeValid(s: str) -> str:
    # Initialize a list to store the indices of '(' that haven't been matched
    yet.
    open_stack = []
    # Use a set to record indices to remove.
    indices_to_remove = set()

    # Loop over every character and its index in the string.
    for i, char in enumerate(s):
        # If the character is an open parenthesis, store its index.

        if char == '(':
            open_stack.append(i)
        # If the character is a closing parenthesis.
        elif char == ')':
            # If there is a matching open parenthesis, pop from the stack.
            if open_stack:
                open_stack.pop()
            # Otherwise, mark the index of this unmatched ')' for removal.
            else:
                indices_to_remove.add(i)

    # Add any remaining unmatched '(' indices from the stack to the removal
    set.
    indices_to_remove = indices_to_remove.union(set(open_stack))

    # Build and return the valid string by skipping characters at indices to be
    removed.
    result = []
    for i, char in enumerate(s):
        if i not in indices_to_remove:
            result.append(char)
    return "".join(result)

# Example usage:
print(minRemoveToMakeValid("lee(t(c)o)de"))
```

◆ Quick Review

Key Insights

- Use a stack to record indices of unmatched '('.
- Traverse the string once to identify unmatched ')' and push indices of '('.
- Mark indices that need removal and reconstruct the valid string.
- Greedy, one-pass validation is efficient.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string

Space Complexity: $O(n)$ in the worst case for the stack and additional storage

Solution

We can solve the problem by iterating over the string with a stack to match parentheses:

- For every character in the string:
- If the character is '(', push its index onto the stack.
- If it is ')', check if there is a matching '(' in the stack. If so, pop the top; otherwise, mark this index for removal.

- After the iteration, any indices left in the stack (representing unmatched '(') should also be marked for removal.
- Build the final string by skipping characters at marked indices. This approach ensures we only remove the minimum number of parentheses to achieve a valid string.

Stacks

□ #2390 Removing Stars From a String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Simulation

Lists: AlgoMaster 600

Problem

You are given a string s , which contains stars $*$.
In one operation, you can:

- Choose a star in s .
- Remove the closest non-star character to its left, as well as remove the star itself.

Return the string after all stars have been removed.

Note:

- The input will be generated such that the operation is always possible.
- It can be shown that the resulting string will always be unique.

Example 1:

Input: `s = "leet**cod*e"`

Output: `"lecoe"`

Explanation: Performing the removals from left to right:

- The closest character to the 1st star is 't' in `"leet**cod*e"`. `s` becomes `"lee*cod*e"`.
 - The closest character to the 2nd star is 'e' in `"lee*cod*e"`. `s` becomes `"lecod*e"`.
 - The closest character to the 3rd star is 'd' in `"lecod*e"`. `s` becomes `"lecoe"`.
- There are no more stars, so we return `"lecoe"`.

Example 2:

Input: `s = "erase*****"`

Output: `""`

Explanation: The entire string is removed, so we return an empty string.

Constraints:

- $1 \leq s.length \leq 105$
- `s` consists of lowercase English letters and stars `*`.
- The operation above can be performed on `s`.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeStars(self, s: str) -> str:
        ans = []
        for c in s:
            if c == '*':
                ans.pop()
            else:
                ans.append(c)
        return ''.join(ans)
```

• LeetDoocs

```
class Solution:
    def removeStars(self, s: str) -> str:
        ans = []
        for c in s:
            if c == '*':
                ans.pop()
            else:
                ans.append(c)
        return ''.join(ans)
```

• SimplyLeet

```
# Python solution for Removing Stars From a String
class Solution:
    def removeStars(self, s: str) -> str:
        result_stack = [] # Initialize a stack to hold characters
        # Iterate through each character in the string
        for char in s:
            if char == '*':
                # When a star is encountered, pop the last character off the
                # stack

                if result_stack:
                    result_stack.pop()

            else:
                # For any normal character, push it onto the stack
                result_stack.append(char)
        # Join the remaining characters to form the final result
        return "".join(result_stack)

# Example usage
# sol = Solution()
# print(sol.removeStars("leet**cod*e"))
```

◆ Quick Review

Key Insights

- Use a stack data structure to simulate the removal operation.

- When encountering a non-star character, push it onto the stack.
- When encountering a star, pop the top character from the stack (which represents the nearest non-star character to its left).
- The final answer is the concatenation of the characters remaining in the stack.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string s , since we process each character once.

Space Complexity: $O(n)$, in the worst case when no stars are encountered and all characters are stored in the stack.

Solution

The solution employs a stack to efficiently simulate the removal process. As you iterate through the string, push every non-star character onto the stack. When you encounter a star, pop the last character from the stack which represents removing the closest non-star character to its left along with the star. After processing the entire string, join the remaining characters in the stack to form the

final result. Critical points include ensuring the operation is performed only when there is a character to remove (guaranteed by problem constraints) and handling the string traversal in a single pass.

Tree Traversal – Level Order

Round 2 · High · Medium · 3 questions

Tree Traversal – Level Order

□ #116 Populating Next Right Pointers in Each Node

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Tree · Depth–First Search ·
Breadth–First Search · Binary Tree
Lists: AlgoMaster 600

Problem

You are given a perfect binary tree where all leaves are on the same level, and every parent has two children. The binary tree has the following definition:

```
struct Node {  
    int val;  
    Node *left;  
    Node *right;  
    Node *next;  
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL.

Initially, all next pointers are set to NULL.

Example 1:

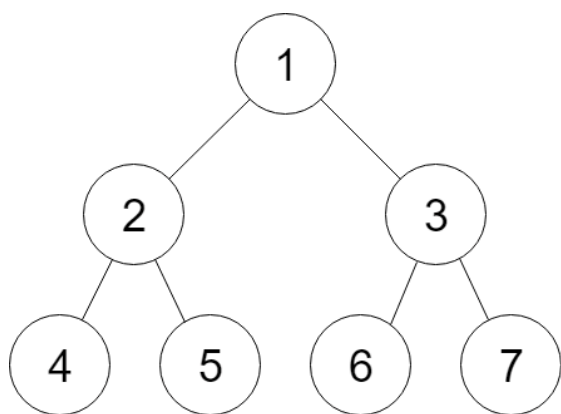


Figure A

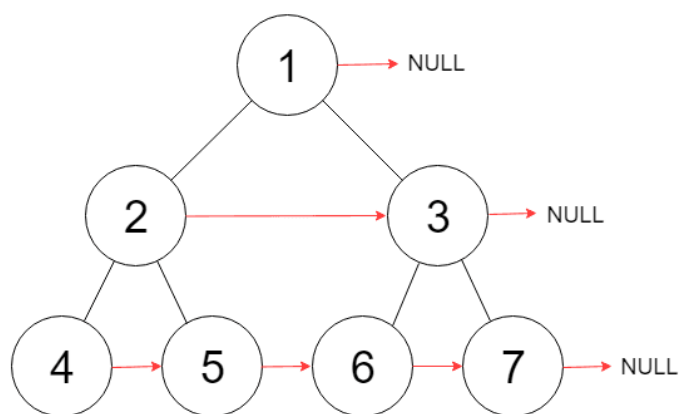


Figure B

Input: root = [1,2,3,4,5,6,7]

Output: [1,#,2,3,#,4,5,6,7,#]

Explanation: Given the above perfect binary tree (Figure A), your function should populate each next pointer to point to its next right node, just like in Figure B. The serialized output is in level order as connected by the next pointers, with '#' signifying the end of each level.

Example 2:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 2^{12} - 1]$.
- $-1000 \leq \text{Node.val} \leq 1000$

Follow-up:

- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def connect(self, root: 'Node | None') -> 'Node | None':
        if not root:
            return None

        def connectTwoNodes(p, q) -> None:
            if not p:
                return
            p.next = q
            connectTwoNodes(p.left, p.right)

            connectTwoNodes(q.left, q.right)
            connectTwoNodes(p.right, q.left)

        connectTwoNodes(root.left, root.right)
        return root

class Solution:
    def connect(self, root: 'Node') -> 'Node':
        node = root # the node that is above the current needling

        while node and node.left:
            dummy = Node(0) # a dummy node before needling
            # Needle the children of the node.
            needle = dummy
            while node:
                needle.next = node.left

                needle = needle.next
                needle.next = node.right
                needle = needle.next
                node = node.next
            node = dummy.next # Move the node to the next level.

        return root
```

• LeetDoocs

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val: int = 0, left: 'Node' = None, right: 'Node' = None,
next: 'Node' = None):
        self.val = val
        self.left = left
        self.right = right
        self.next = next
"""

```

```

class Solution:
    def connect(self, root: "Optional[Node]") -> "Optional[Node]":
        if root is None:
            return root
        q = deque([root])
        while q:
            p = None
            for _ in range(len(q)):
                node = q.popleft()
                if p:
                    p.next = node
                p = node
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            return root

```

• SimplyLeet

```

# Definition for a Node.
class Node:
    def __init__(self, val: int = 0, left: 'Node' = None, right: 'Node' = None
, next: 'Node' = None):
        self.val = val           # Node's value
        self.left = left         # Left child
        self.right = right       # Right child
        self.next = next        # Next right pointer

def connect(root: 'Node') -> 'Node':
    if not root:

        return root # If tree is empty, return null

    # Start with the root node
    level_start = root
    while level_start.left:
        # Traverse nodes at the current level
        current = level_start
        while current:
            # Connect left child to right child
            current.left.next = current.right

            # Connect right child to the next node's left child if available
            if current.next:
                current.right.next = current.next.left
            # Move to the next node on the same level using next pointer
            current = current.next
        # Move to the leftmost node of the next level
        level_start = level_start.left

    return root # Return the modified tree

```

◆ Quick Review

Key Insights

- The tree is perfect, meaning that if a node exists, both its left and right children exist (except at the leaf level).

- At each node, we can set:
- `node.left.next` to `node.right`.
- `node.right.next` to `node.next.left` (if `node.next` exists).
- We can use a level-order traversal without extra space by linking nodes from left to right on the same level.
- The approach only uses constant extra space (excluding recursion stack if used).

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree.

Space Complexity: $O(1)$ extra space (ignoring recursion stack or pointers used to traverse levels).

Solution

We can solve this problem by taking advantage of the perfect tree property. Starting from the root, we traverse each level using the already established "next" pointers. For each node at a given level, set the left child's "next" pointer to the right child. For the right child, if the node has a "next" pointer, link it to the left child of the node's next neighbor. After processing a

level, move to the leftmost node of the next level. This iterative process continues until we reach a level without children.

Tree Traversal – Level Order

□ #117 Populating Next Right Pointers in Each Node II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Tree · Depth–First Search ·
Breadth–First Search · Binary Tree
Lists: AlgoMaster 600

Problem

Given a binary tree

```
struct Node {  
    int val;  
    Node *left;  
    Node *right;  
    Node *next;  
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL.

Initially, all next pointers are set to NULL.

Example 1:

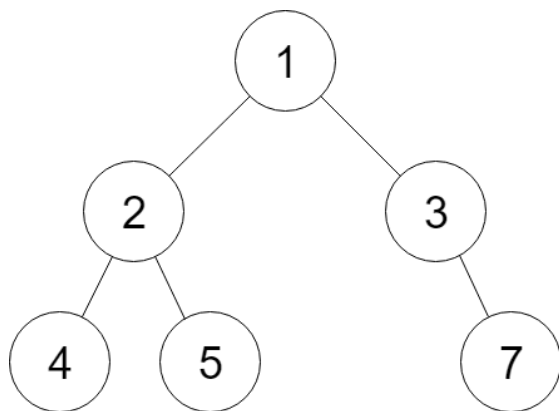


Figure A

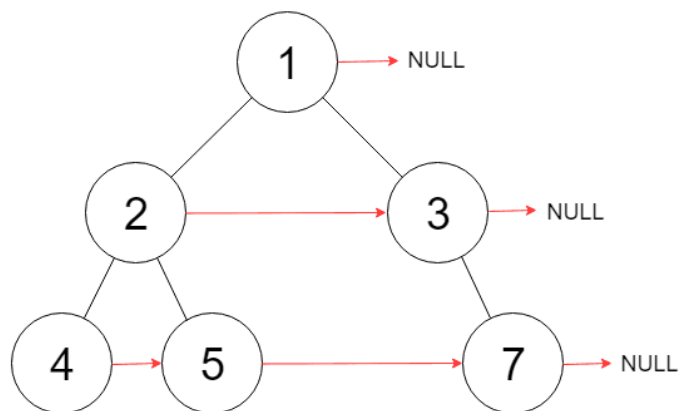


Figure B

Input: root = [1,2,3,4,5,null,7]

Output: [1,#,2,3,#,4,5,7,#]

Explanation: Given the above binary tree (Figure A), your function should populate each next pointer to point to its next right node, just like in Figure B. The serialized output is in level order as connected by the next pointers, with '#' signifying the end of each level.

Example 2:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 6000].
- $-100 \leq \text{Node.val} \leq 100$

Follow-up:

- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def connect(self, root: 'Node') -> 'Node':
        node = root # the node that is above the current needling

        while node:
            dummy = Node(0) # a dummy node before needling
            # Needle the children of the node.
            needle = dummy
            while node:
                if node.left: # Needle the left child.

                    needle.next = node.left
                    needle = needle.next
                if node.right: # Needle the right child.
                    needle.next = node.right
                    needle = needle.next
                node = node.next
            node = dummy.next # Move the node to the next level.

        return root
```

• LeetDoocs

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val: int = 0, left: 'Node' = None, right: 'Node' = None,
next: 'Node' = None):
        self.val = val
        self.left = left
        self.right = right
        self.next = next
"""
```

```

class Solution:
    def connect(self, root: "Node") -> "Node":
        if root is None:
            return root
        q = deque([root])
        while q:
            p = None
            for _ in range(len(q)):
                node = q.popleft()

                if p:
                    p.next = node
                p = node
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            return root

```

• SimplyLeet

```

# Definition for a Node.
# class Node:
# def __init__(self, val: int = 0, left: 'Node' = None, right: 'Node' = None,
# next: 'Node' = None):
# self.val = val
# self.left = left
# self.right = right
# self.next = next

class Solution:
    def connect(self, root: 'Node') -> 'Node':

```

```

# Start with the root node
current = root
# Process level by level until no nodes remain
while current:
    # Initialize pointers for the next level
    nextHead = None
    nextTail = None
    # Traverse nodes in the current level using next pointers
    while current:
        # Process current node's left child

        if current.left:
            if not nextHead:
                nextHead = current.left
                nextTail = current.left
            else:
                nextTail.next = current.left
                nextTail = nextTail.next
        # Process current node's right child
        if current.right:
            if not nextHead:
                nextHead = current.right
                nextTail = current.right
            else:
                nextTail.next = current.right
                nextTail = nextTail.next
        # Move to the next node in current level
        current = current.next
    # Move to next level
    current = nextHead
return root

```

◆ Quick Review

Key Insights

- The tree is not necessarily a perfect binary tree.

- The problem requires an algorithm using constant extra space.
- A level order traversal using established next pointers allows linking nodes on the next level without additional data structures.
- Use pointers to keep track of the current level, the head of the next level, and the tail to build the next connections.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes, because every node is processed once.

Space Complexity: $O(1)$ extra space (ignoring the recursion stack if a recursive approach were used) by leveraging the next pointers for level order traversal.

Solution

The solution uses a level order traversal without using an explicit queue. Instead, it uses three pointers:

- **current:** iterates over nodes in the current level.
- **nextHead:** points to the first node in the next level.

– **nextTail**: used to connect nodes in the next level. For each node in the current level, if it has a left child, attach it to nextTail. Similarly, if it has a right child, attach it to nextTail. After processing the entire level, move to nextHead and continue the process. This guarantees constant extra space usage while linking all nodes appropriately.

Tree Traversal – Level Order

□ #958 Check Completeness of a Binary Tree

MED

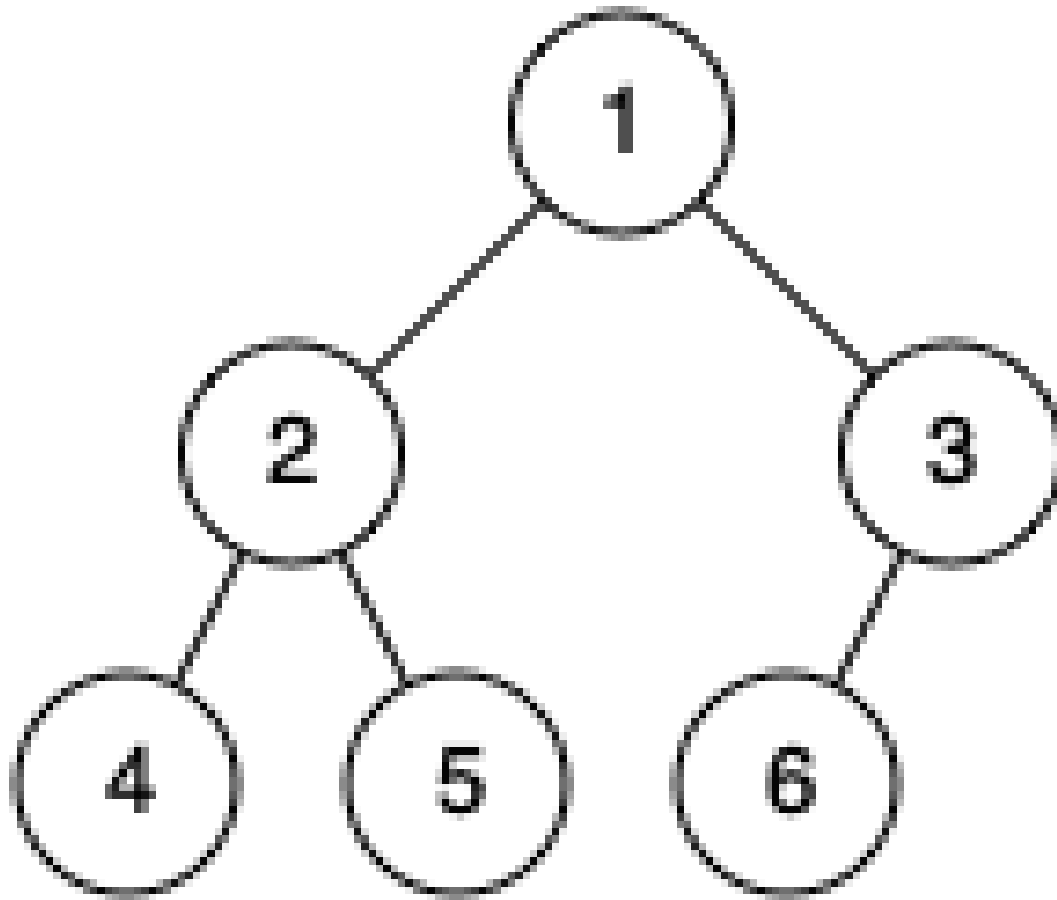
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Breadth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, determine if it is a complete binary tree.

In a complete binary tree, every level, except possibly the last, is completely filled, and all nodes in the last level are as far left as possible. It can have between 1 and 2^h nodes inclusive at the last level h .

Example 1:

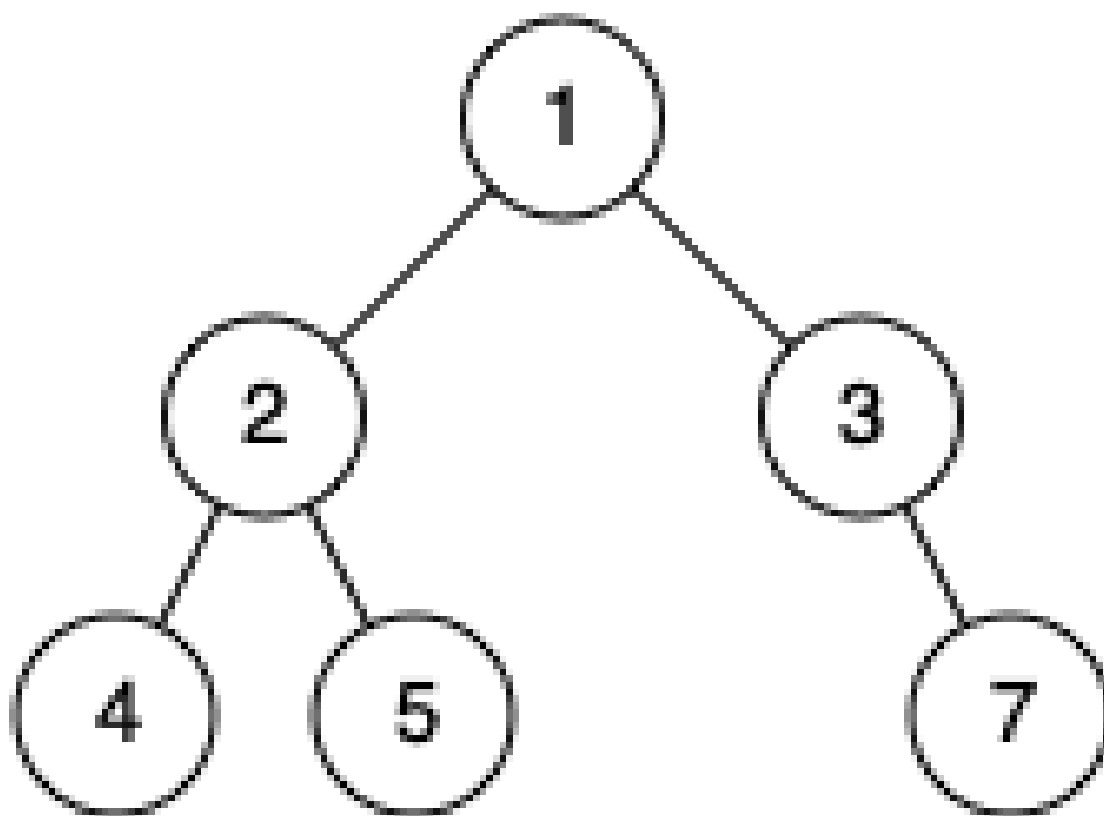


Input: root = [1,2,3,4,5,6]

Output: true

Explanation: Every level before the last is full (ie. levels with node-values {1} and {2, 3}), and all nodes in the last level ({4, 5, 6}) are as far left as possible.

Example 2:



Input: root = [1,2,3,4,5,null,7]

Output: false

Explanation: The node with value 7 isn't as far left as possible.

Constraints:

- The number of nodes in the tree is in the range [1, 100].
- $1 \leq \text{Node.val} \leq 1000$

Community Solutions (Python)

- LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isCompleteTree(self, root: TreeNode) -> bool:
        q = deque([root])
        while q:
            node = q.popleft()
            if node is None:
                break
            q.append(node.left)
            q.append(node.right)
        return all(node is None for node in q)
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# Function to check completeness of binary tree
def isCompleteTree(root: TreeNode) -> bool:
    if not root:
        return True

    # Initialize queue for BFS traversal
    queue = [root]
    # Flag to mark when a missing child is seen
    end = False

    while queue:
        node = queue.pop(0)
```

```
# Check left child
if node.left:
    if end:
        return False # Node with left child found after a missing
child has been encountered
    queue.append(node.left)
else:
    end = True # Future nodes must be leaves

# Check right child
if node.right:

    if end:
        return False # Node with right child found after a missing
child has been encountered
    queue.append(node.right)
else:
    end = True # Future nodes must be leaves

return True
```

◆ Quick Review

Key Insights

- A complete binary tree requires every level (except the last) to be fully occupied.
- In the last level, nodes must fill from the left.
- A level order traversal (BFS) effectively checks these conditions.
- Once a missing child is encountered during BFS, all subsequent nodes must be leaves.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, since each node is visited once.

Space Complexity: $O(n)$ in the worst-case scenario due to the queue used for BFS traversal.

Solution

We solve this problem using a breadth-first search (BFS) traversal. Traverse the tree level by level using a queue. When a node is encountered that does not have a left or right child, set a flag indicating that all following nodes must have no children. If any subsequent node has a child, the tree is not complete. This approach efficiently ensures the tree maintains the complete tree properties.

Tree Traversal – Pre Order

Round 2 · High · Medium · 3 questions

Tree Traversal – Pre Order

□ #106 Construct Binary Tree from Inorder and Postorder Traversal **MED**

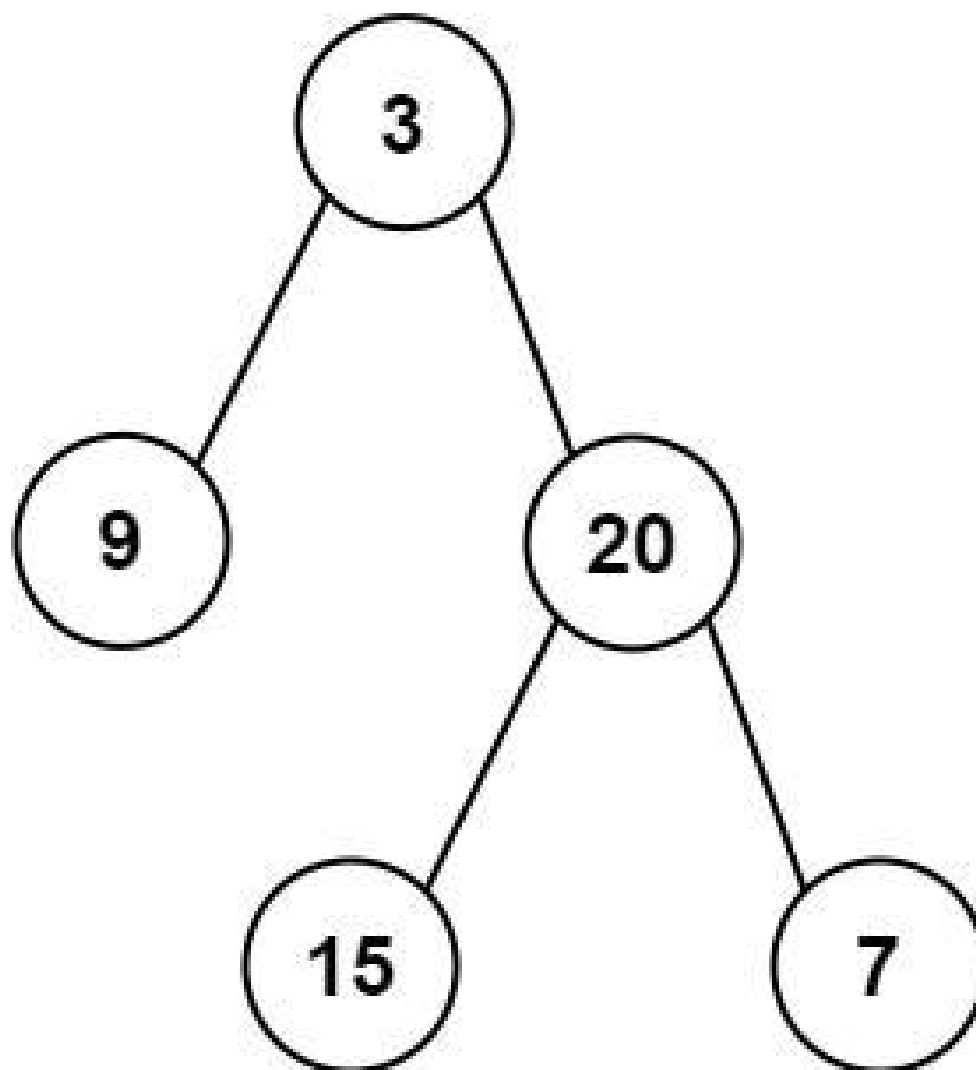
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Divide and Conquer · Tree
· Binary Tree

Lists: AlgoMaster 600

Problem

Given two integer arrays *inorder* and *postorder* where *inorder* is the inorder traversal of a binary tree and *postorder* is the postorder traversal of the same tree, construct and return the binary tree.

Example 1:



Input: inorder = [9,3,15,20,7], postorder = [9,15,7,20,3]
Output: [3,9,20,null,null,15,7]

Example 2:

Input: inorder = [-1], postorder = [-1]
Output: [-1]

Constraints:

- $1 \leq \text{inorder.length} \leq 3000$
- $\text{postorder.length} == \text{inorder.length}$
- $-3000 \leq \text{inorder}[i], \text{postorder}[i] \leq 3000$

- inorder and postorder consist of unique values.
- Each value of postorder also appears in inorder.
- inorder is guaranteed to be the inorder traversal of the tree.
- postorder is guaranteed to be the postorder traversal of the tree.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def buildTree(
        self,
        inorder: list[int],
        postorder: list[int],
    ) -> TreeNode | None:
        inToIndex = {num: i for i, num in enumerate(inorder)}

        def build(
            inStart: int,

            inEnd: int,
            postStart: int,
            postEnd: int,
        ) -> TreeNode | None:
            if inStart > inEnd:
                return None

            rootVal = postorder[postEnd]
            rootInIndex = inToIndex[rootVal]
            leftSize = rootInIndex - inStart
```

```

root = TreeNode(rootVal)
root.left = build(inStart, rootInIndex - 1, postStart,
                  postStart + leftSize - 1)
root.right = build(rootInIndex + 1, inEnd, postStart + leftSize,
                   postEnd - 1)

return root

return build(0, len(inorder) - 1, 0, len(postorder) - 1)

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def buildTree(self, inorder: List[int], postorder: List[int]) -> Optional[TreeNode]:
        def dfs(i: int, j: int, n: int) -> Optional[TreeNode]:
            if n <= 0:
                return None
            v = postorder[j + n - 1]
            k = d[v]
            l = dfs(i, j, k - i)
            r = dfs(k + 1, j + k - i, n - k + i - 1)
            return TreeNode(v, l, r)

            d = {v: i for i, v in enumerate(inorder)}
            return dfs(0, 0, len(inorder))

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # Node's value
        self.left = left         # Left child
        self.right = right       # Right child

class Solution:
    def buildTree(self, inorder: list[int], postorder: list[int]) -> TreeNode:
        # Create a dictionary to map value -> index in the inorder array for
        # O(1) lookup

        inorder_index_map = {value: idx for idx, value in enumerate(inorder)}

        # Recursive helper function that builds the tree
        def build(left_in, right_in, left_post, right_post):
            if left_in > right_in or left_post > right_post:
                return None

            # Last element in current postorder range is root
            root_val = postorder[right_post]
            root = TreeNode(root_val)

            # Find the index of the root in inorder traversal
            index = inorder_index_map[root_val]
            # Number of nodes in left subtree
            left_count = index - left_in

            # Construct left subtree using the corresponding subarrays
            root.left = build(left_in, index - 1, left_post, left_post + left_
count - 1)

            # Construct right subtree using the corresponding subarrays
            root.right = build(index + 1, right_in, left_post + left_count, ri
ght_post - 1)

```

```
        return root

    # Initiate the recursive function with proper indices
    return build(0, len(inorder) - 1, 0, len(postorder) - 1)

# Example usage:
# sol = Solution()
# tree = sol.buildTree([9,3,15,20,7], [9,15,7,20,3])
```

◆ Quick Review

Key Insights

- The last element of the postorder array is always the root.
- Finding the index of the root in the inorder array splits the tree into left and right subtrees.
- Recursively reconstruct the tree using the divided inorder arrays and corresponding segments of the postorder array.
- Use a hash table (or dictionary) to quickly locate the index of any value in the inorder array.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ (Space is used for the recursive call stack and extra hash table)

Solution

We can solve the problem by leveraging the observation that in postorder traversal, the last element is the root of the current subtree. After identifying the root, we locate its index in the inorder array which partitions the array into left and right subtrees. A hash table is used for fast lookup of the root index in the inorder sequence. We then recursively apply the same process for left and right subtrees, carefully managing indices to maintain alignment between the inorder and postorder arrays.

Tree Traversal – Pre Order

□ #687 Longest Univalue Path

MED

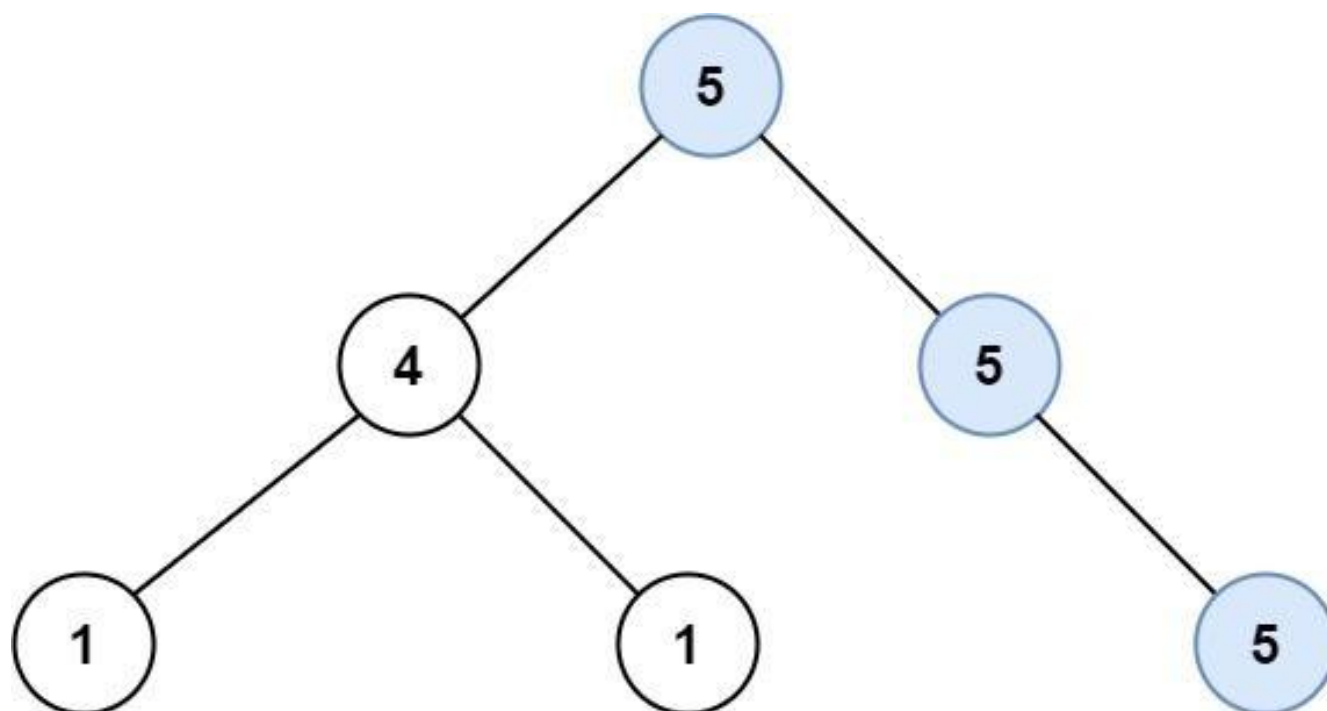
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, return the length of the longest path, where each node in the path has the same value. This path may or may not pass through the root.

The length of the path between two nodes is represented by the number of edges between them.

Example 1:

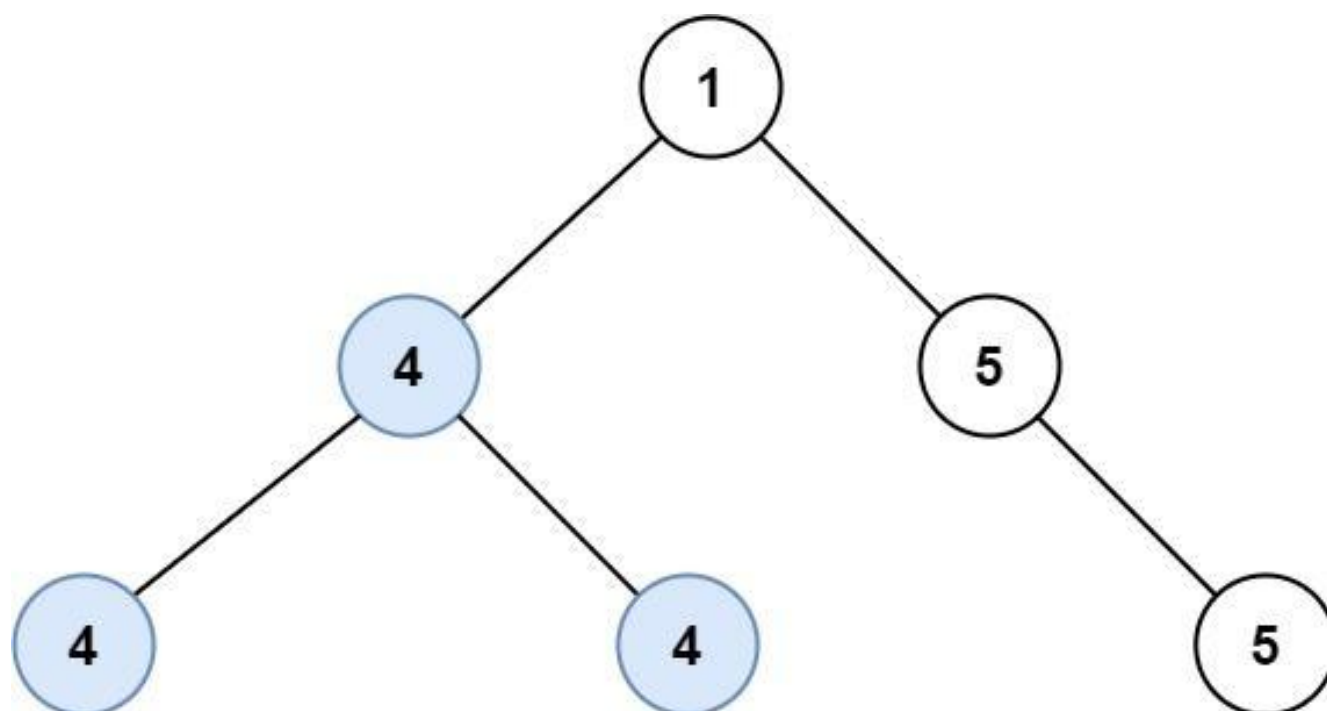


Input: root = [5,4,5,1,1,null,5]

Output: 2

Explanation: The shown image shows that the longest path of the same value (i.e. 5).

Example 2:



Input: root = [1,4,5,4,4,null,5]

Output: 2

Explanation: The shown image shows that the longest path of the same value (i.e. 4).

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-1000 \leq \text{Node.val} \leq 1000$
- The depth of the tree will not exceed 1000.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestUnivaluePath(self, root: TreeNode | None) -> int:
        ans = 0

        def longestUnivaluePathDownFrom(root: TreeNode | None) -> int:
            nonlocal ans
            if not root:
                return 0

            l = longestUnivaluePathDownFrom(root.left)

            r = longestUnivaluePathDownFrom(root.right)
            arrowLeft = l + 1 if root.left and root.left.val == root.val else 0
            arrowRight = r + 1 if root.right and root.right.val == root.val else 0
            ans = max(ans, arrowLeft + arrowRight)
            return max(arrowLeft, arrowRight)

        longestUnivaluePathDownFrom(root)
        return ans
```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def longestUnivaluePath(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:
                return 0
            l, r = dfs(root.left), dfs(root.right)
            l = l + 1 if root.left and root.left.val == root.val else 0
            r = r + 1 if root.right and root.right.val == root.val else 0
            nonlocal ans
            ans = max(ans, l + r)
            return max(l, r)

        ans = 0
        dfs(root)

        return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # The value of the node
        self.left = left        # Pointer to the left child
        self.right = right       # Pointer to the right child

class Solution:
    def longestUnivaluePath(self, root: TreeNode) -> int:
        self.max_path = 0 # Global variable to track the longest univalue path

```

```

def dfs(node: TreeNode) -> int:
    if not node:
        return 0 # Base case: no node means 0 path length
    # Recursively find the longest univalue paths in the left and right
    subtrees
    left_length = dfs(node.left)
    right_length = dfs(node.right)

    left_path = right_path = 0 # Initialize paths from left and right
    # If left child exists and value matches, extend the left path

    if node.left and node.left.val == node.val:
        left_path = left_length + 1
    # If right child exists and value matches, extend the right path
    if node.right and node.right.val == node.val:
        right_path = right_length + 1

    # Update the global maximum path: path may use both left and right
    extensions
    self.max_path = max(self.max_path, left_path + right_path)
    # Return the longer of one side which could be extended to parent's
    path
    return max(left_path, right_path)

dfs(root) # Start DFS from the root
return self.max_path # Return the longest univalue path

```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to traverse the tree in postorder.
- For each node, compute the longest path with the same value that starts from its left child and right child.

- Extend the path if the child node's value equals the current node's value.
- Combine the left and right paths at the current node to consider the situation where the path goes through the node.
- Maintain a global variable to track the maximum path length found.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree.

Space Complexity: $O(n)$ in the worst case due to the recursion stack.

Solution

Perform a DFS traversal of the tree. At each node, calculate the longest univalue path from its left and right children that can be connected with the current node (if the child's value is the same as the current node's value). Update a global maximum using the sum of the left and right potential paths at each node. Return the maximum edge count obtained amongst all nodes.

Tree Traversal – Pre Order

□ #1026 Maximum Difference Between Node and Ancestor

MED

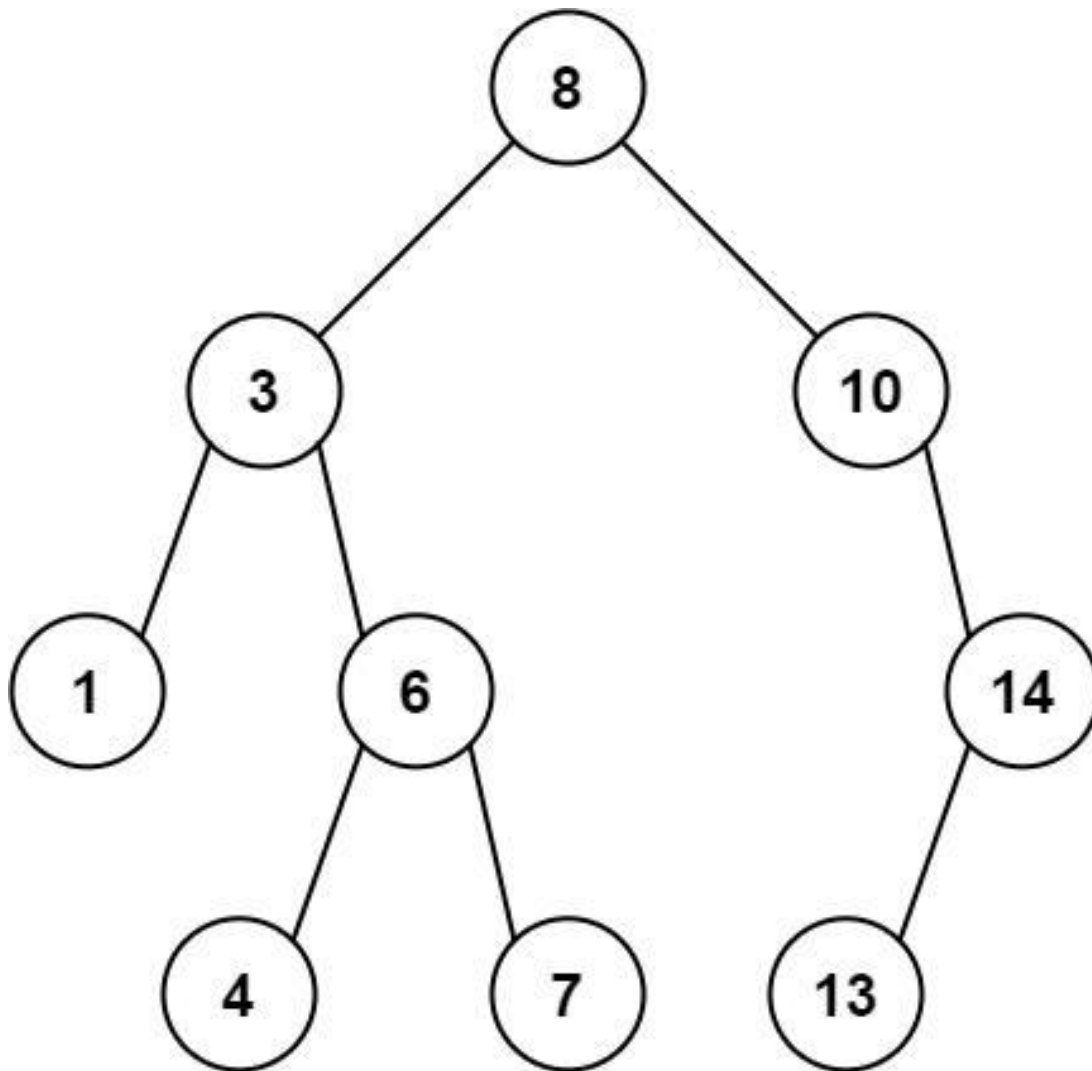
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, find the maximum value v for which there exist different nodes a and b where $v = |a.val - b.val|$ and a is an ancestor of b .

A node a is an ancestor of b if either: any child of a is equal to b or any child of a is an ancestor of b .

Example 1:



Input: root = [8,3,10,1,6,null,14,null,null,4,7,13]

Output: 7

Explanation: We have various ancestor-node differences, some of which are given below :

$$|8 - 3| = 5$$

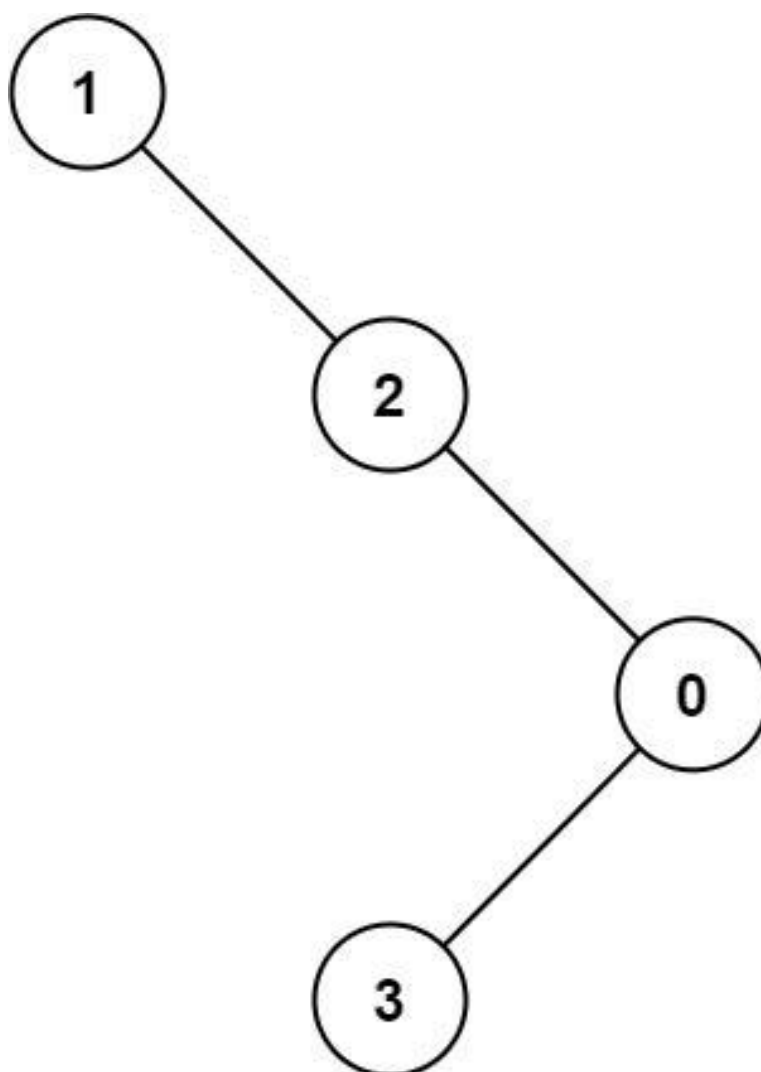
$$|3 - 7| = 4$$

$$|8 - 1| = 7$$

$$|10 - 13| = 3$$

Among all possible differences, the maximum value of 7 is obtained by $|8 - 1| = 7$.

Example 2:



Input: root = [1,null,2,null,0,3]

Output: 3

Constraints:

- The number of nodes in the tree is in the range [2, 5000].
- $0 \leq \text{Node.val} \leq 105$

Community Solutions (Python)

- LeetDoocs


```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxAncestorDiff(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode], mi: int, mx: int):
            if root is None:
                return
            nonlocal ans
            ans = max(ans, abs(mi - root.val), abs(mx - root.val))
            mi = min(mi, root.val)
            mx = max(mx, root.val)
            dfs(root.left, mi, mx)
            dfs(root.right, mi, mx)

        ans = 0
        dfs(root, root.val, root.val)

        return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def maxAncestorDiff(self, root: TreeNode) -> int:
        # Helper function for DFS traversal with current min and max values.

```

```
def dfs(node, cur_min, cur_max):  
    if not node:  
        # At a null node, return the difference encountered so far.  
        return cur_max - cur_min  
    # Update the current path's min and max values.  
    cur_min = min(cur_min, node.val)  
    cur_max = max(cur_max, node.val)  
    # Recursively get the maximum difference in both subtrees.  
    left_diff = dfs(node.left, cur_min, cur_max)  
    right_diff = dfs(node.right, cur_min, cur_max)  
  
    # Return the maximum difference found.  
    return max(left_diff, right_diff)  
  
# Start DFS with the root value as both current min and max.  
return dfs(root, root.val, root.val)
```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to traverse the tree.
- Track the minimum and maximum values encountered along the current path from the root.
- At each node, update the global maximum difference using the current node's value and the tracked minimum and maximum.
- The solution leverages recursion which implicitly uses the call stack to manage state.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree.

Space Complexity: $O(h)$, where h is the height of the tree. In the worst case (skewed tree) this could be $O(n)$.

Solution

The problem can be solved using a DFS traversal that passes along two values: the current minimum and maximum encountered on the path from the root to the current node. At each node, compute the potential differences from the current node's value with both the minimum and maximum. Update the overall maximum difference if these values yield a larger difference. Then, update the current path's minimum and maximum values, and recursively traverse the left and right children. When the traversal backtracks, the previously computed values guarantee the differences are correctly calculated for all ancestor–descendant relations.

Tree Traversal – In Order

Round 2 · High · Medium · 1 question

Tree Traversal – In Order

□ #1382 Balance a Binary Search Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Divide and Conquer · Greedy · Tree ·
Depth-First Search · Binary Search Tree · Binary
Tree

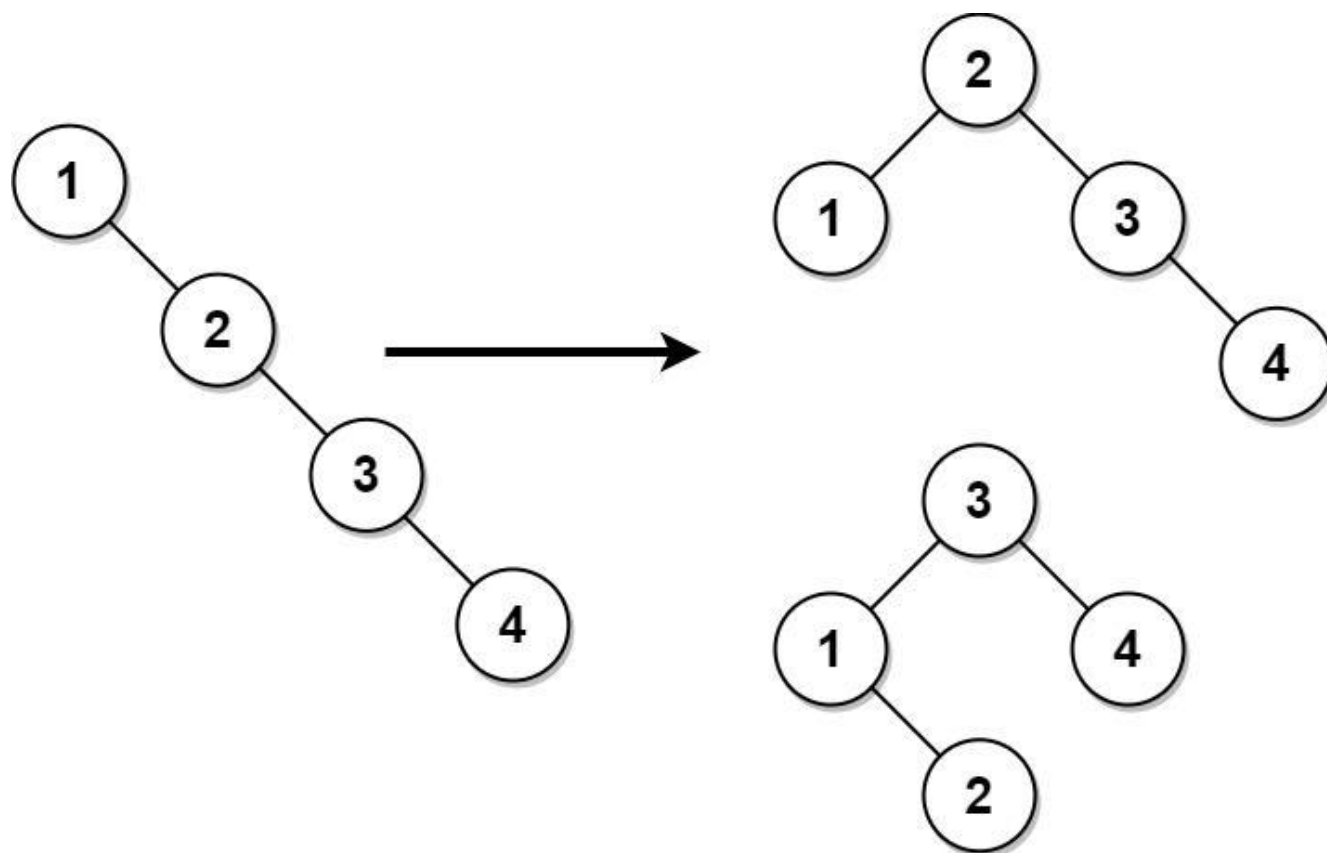
Lists: AlgoMaster 600

Problem

Given the root of a binary search tree, return a balanced binary search tree with the same node values. If there is more than one answer, return any of them.

A binary search tree is balanced if the depth of the two subtrees of every node never differs by more than 1.

Example 1:

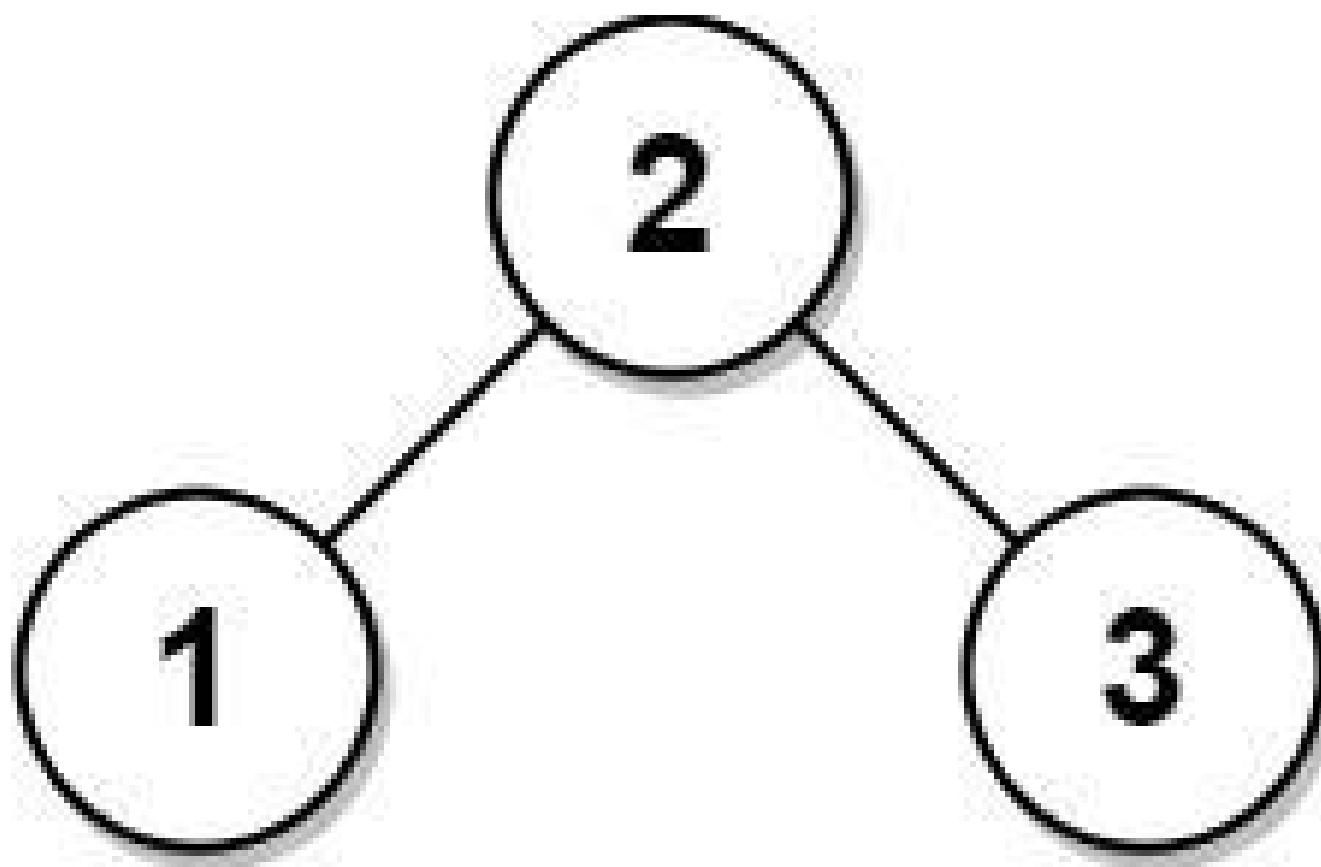


Input: root = [1,null,2,null,3,null,4,null,null]

Output: [2,1,3,null,null,null,4]

Explanation: This is not the only correct answer, [3,1,4,null,2] is also correct.

Example 2:



Input: root = [2,1,3]

Output: [2,1,3]

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $1 \leq \text{Node.val} \leq 105$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def balanceBST(self, root: TreeNode | None) -> TreeNode | None:
        nums = []

        def inorder(root: TreeNode | None) -> None:
            if not root:
                return
            inorder(root.left)
            nums.append(root.val)
            inorder(root.right)

        inorder(root)

        # Same as 108. Convert Sorted Array to Binary Search Tree
        def build(l: int, r: int) -> TreeNode | None:
            if l > r:
                return None
            m = (l + r) // 2
            return TreeNode(nums[m],
                            build(l, m - 1),

                            build(m + 1, r))

        return build(0, len(nums) - 1)

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def balanceBST(self, root: TreeNode) -> TreeNode:
        def dfs(root: TreeNode):
            if root is None:

```



```

        return
    dfs(root.left)
    nums.append(root.val)
    dfs(root.right)

def build(i: int, j: int) -> TreeNode:
    if i > j:
        return None
    mid = (i + j) >> 1
    left = build(i, mid - 1)

    right = build(mid + 1, j)
    return TreeNode(nums[mid], left, right)

nums = []
dfs(root)
return build(0, len(nums) - 1)

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val      # Node's value.
        self.left = left    # Left child.
        self.right = right  # Right child.

class Solution:
    def balanceBST(self, root: TreeNode) -> TreeNode:
        # In-order traversal to collect nodes in sorted order.

        def inorder(node):
            if not node:
                return []
            return inorder(node.left) + [node.val] + inorder(node.right)

        sorted_vals = inorder(root) # Sorted list of node values.

        # Recursively build balanced BST from the sorted list.
        def buildBST(left, right):
            if left > right:

```

```
        return None
    mid = (left + right) // 2
    node = TreeNode(sorted_vals[mid])
    node.left = buildBST(left, mid - 1)
    node.right = buildBST(mid + 1, right)
    return node

return buildBST(0, len(sorted_vals) - 1)
```

◆ Quick Review

Key Insights

- Perform an in-order traversal to obtain a sorted list of node values.
- Use the sorted list to reconstruct the BST by choosing the middle element as the root, ensuring balanced subtrees.
- Recursively build left and right subtrees from the corresponding halves of the array.
- Leveraging the BST's in-order property guarantees a sorted array, making the reconstruction straightforward.

Space & Time

Time Complexity: $O(n)$, where n is the total number of nodes, due to the in-order traversal and balanced BST construction.

Space Complexity: $O(n)$, for storing node values in an array and the recursion stack.

Solution

The problem is solved in two primary steps. First, perform an in-order traversal of the BST and store the node values in a sorted list. Then, build a balanced BST from this sorted list by recursively picking the middle element as the root. This ensures that the tree remains balanced because the heights of the left and right subtrees are kept as equal as possible.

Tree Traversal – Post-Order

Round 2 · High · Medium · 6 questions

Tree Traversal – Post-Order

□ #114 Flatten Binary Tree to Linked List

MED

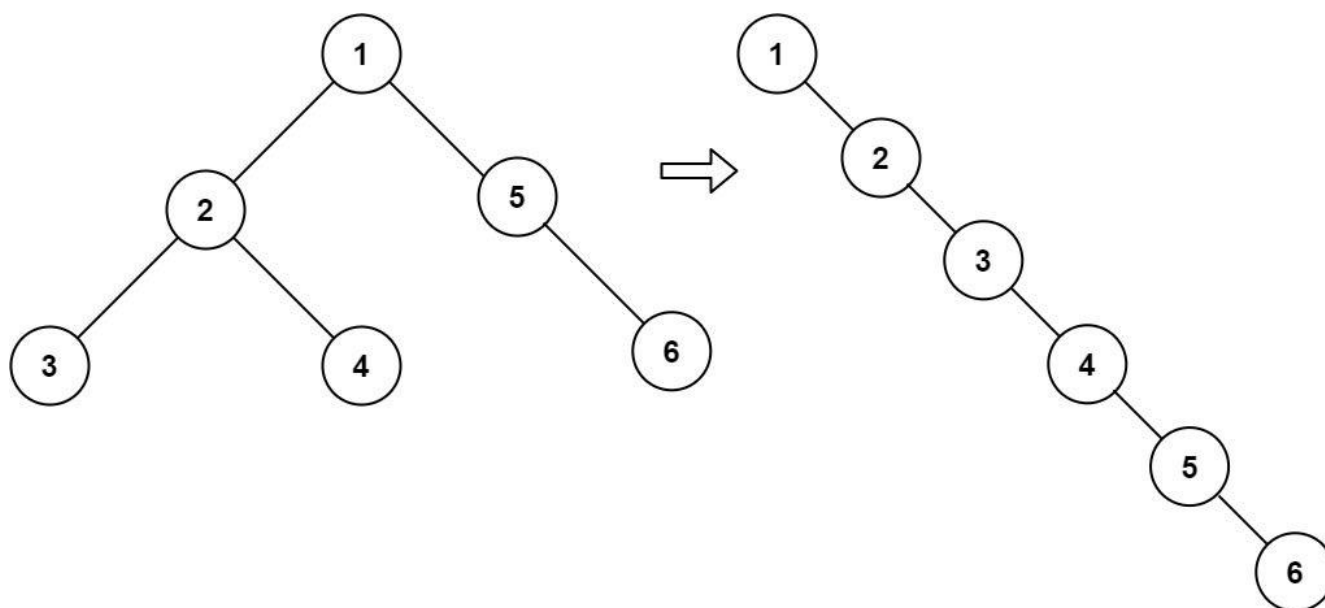
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Stack · Tree · Depth-First Search
· Binary Tree
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, flatten the tree into a "linked list":

- The "linked list" should use the same `TreeNode` class where the right child pointer points to the next node in the list and the left child pointer is always null.
- The "linked list" should be in the same order as a pre-order traversal of the binary tree.

Example 1:



Input: root = [1,2,5,3,4,null,6]
 Output: [1,null,2,null,3,null,4,null,5,null,6]

Example 2:

Input: root = []
 Output: []

Example 3:

Input: root = [0]
 Output: [0]

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Can you flatten the tree in-place (with $O(1)$ extra space)?

Community Solutions (Python)

- WalkCC

```

class Solution:
    def flatten(self, root: TreeNode | None) -> None:
        if not root:
            return

        self.flatten(root.left)
        self.flatten(root.right)

        left = root.left # flattened left
        right = root.right # flattened right

        root.left = None
        root.right = left

        # Connect the original right subtree to the end of the new right subtree.
        rightmost = root
        while rightmost.right:
            rightmost = rightmost.right
        rightmost.right = right

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def flatten(self, root: Optional[TreeNode]) -> None:
        """
        Do not return anything, modify root in-place instead.

```

```

    while root:
        if root.left:
            pre = root.left
            while pre.right:
                pre = pre.right
            pre.right = root.right
            root.right = root.left
            root.left = None
        root = root.right

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def flatten(root):
    # Start from the root node
    current = root

    # Iterate until we have processed all nodes
    while current:
        # If current node has a left subtree
        if current.left:
            # Find the rightmost node in the left subtree
            rightmost = current.left
            while rightmost.right:
                rightmost = rightmost.right
            # Connect the original right subtree to the rightmost node
            rightmost.right = current.right

        # Move the left subtree to become the right subtree
        current.right = current.left
        # Set the left child to None as per linked list requirement
        current.left = None
        # Move to the next right node in the flattened list
        current = current.right

```


◆ Quick Review

Key Insights

- The goal is to rearrange the tree without using extra space for new nodes or data structures.
- A pre-order traversal order must be maintained, meaning the current node is processed before its left and right children.
- One in-place solution involves iterating over nodes, finding the rightmost node in the left subtree (if it exists), attaching the original right subtree to this node, then moving the left subtree to the right.
- Using an iterative approach can achieve $O(1)$ extra space, while a recursive solution may use stack space proportional to the tree height.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes (each node is processed a constant number of times).

Space Complexity: $O(1)$ extra space in the iterative solution (ignoring recursion stack in the recursive approach).

Solution

We use an iterative approach to flatten the tree. The idea is to start from the root and for each node:

- If a left child exists, locate the rightmost node of the left subtree.
- Attach the node's existing right subtree to the right pointer of this rightmost node.
- Move the left subtree to the right pointer of the current node and set the left pointer to null.
- Move to the next node (which is now the right child) and repeat until all nodes have been processed.

This approach effectively re-links all nodes following a pre-order traversal and avoids the overhead of extra space (aside from loop variables).

Tree Traversal – Post-Order

□ #129 Sum Root to Leaf Numbers

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

You are given the root of a binary tree containing digits from 0 to 9 only.

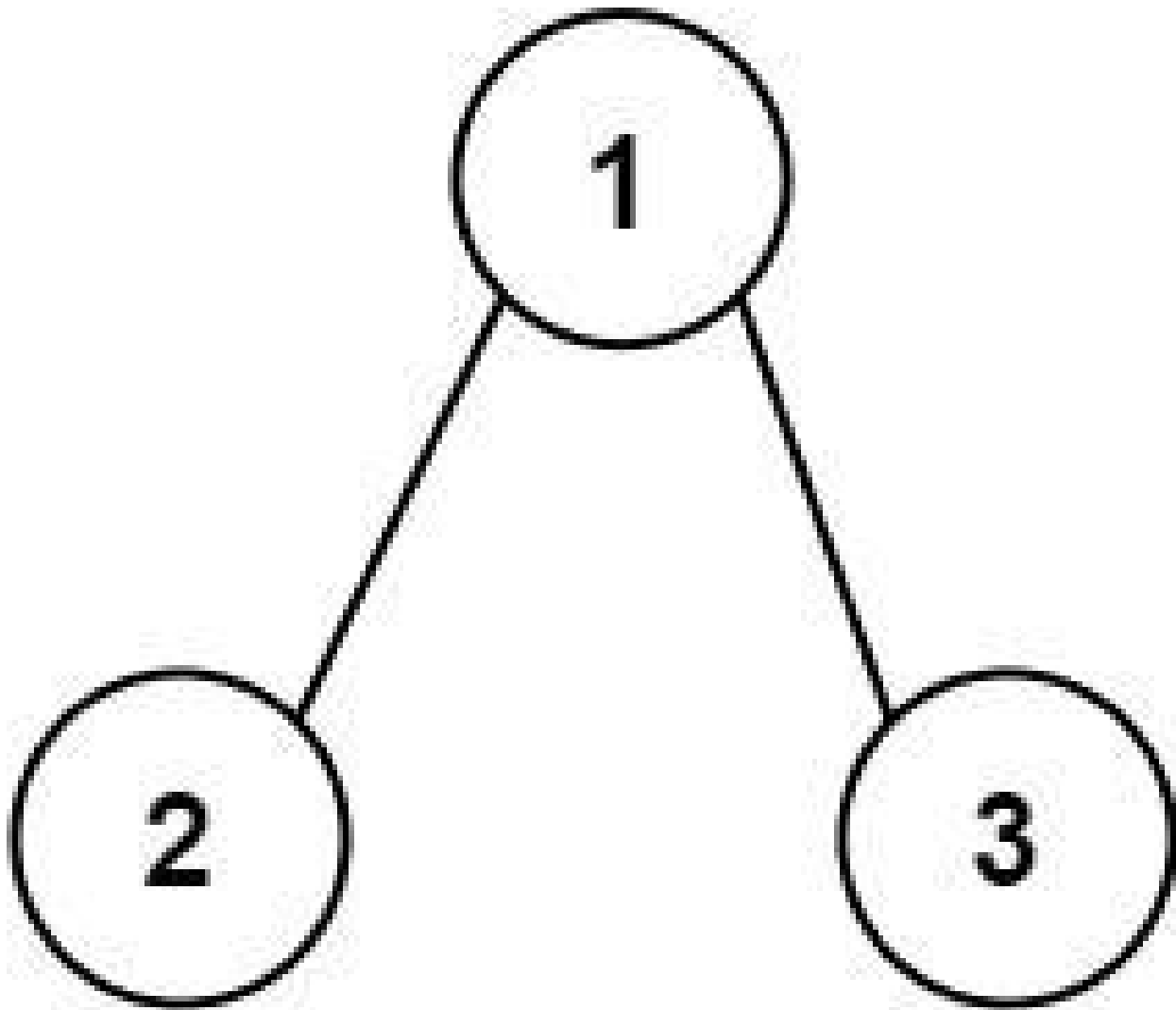
Each root-to-leaf path in the tree represents a number.

- For example, the root-to-leaf path 1 -> 2 -> 3 represents the number 123.

Return the total sum of all root-to-leaf numbers. Test cases are generated so that the answer will fit in a 32-bit integer.

A leaf node is a node with no children.

Example 1:



Input: root = [1,2,3]

Output: 25

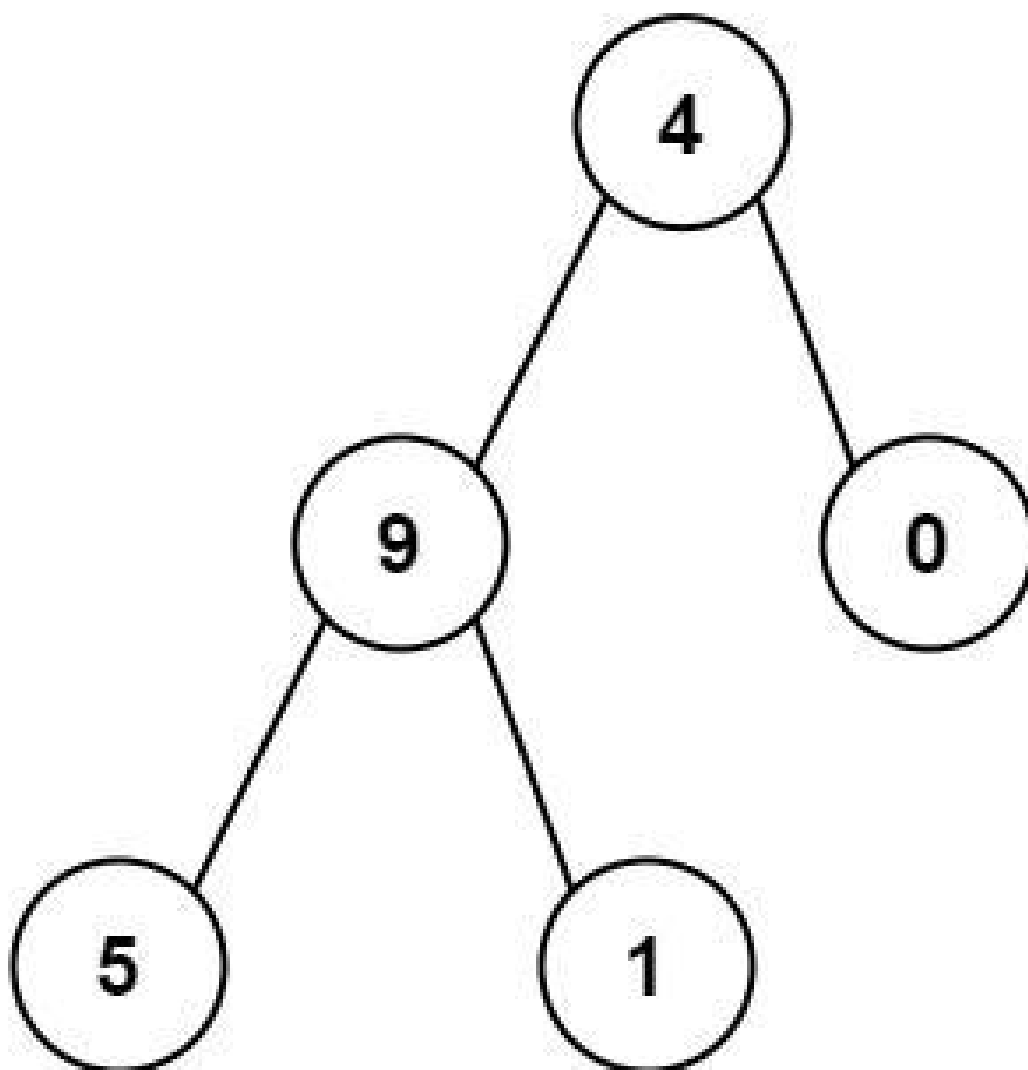
Explanation:

The root-to-leaf path 1->2 represents the number 12.

The root-to-leaf path 1->3 represents the number 13.

Therefore, sum = 12 + 13 = 25.

Example 2:



Input: root = [4,9,0,5,1]

Output: 1026

Explanation:

The root-to-leaf path 4->9->5 represents the number 495.

The root-to-leaf path 4->9->1 represents the number 491.

The root-to-leaf path 4->0 represents the number 40.

Therefore, sum = 495 + 491 + 40 = 1026.

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- $0 \leq \text{Node.val} \leq 9$
- The depth of the tree will not exceed 10.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sumNumbers(self, root: TreeNode | None) -> int:
        ans = 0

        def dfs(root: TreeNode | None, path: int) -> None:
            nonlocal ans
            if not root:
                return
            if not root.left and not root.right:
                ans += path * 10 + root.val

            return

            dfs(root.left, path * 10 + root.val)
            dfs(root.right, path * 10 + root.val)

        dfs(root, 0)
        return ans
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sumNumbers(self, root: Optional[TreeNode]) -> int:
        def dfs(root, s):
            if root is None:
```

```

        return 0
    s = s * 10 + root.val
    if root.left is None and root.right is None:
        return s
    return dfs(root.left, s) + dfs(root.right, s)

return dfs(root, 0)

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def sumNumbers(self, root: TreeNode) -> int:
        # Define a helper function for DFS traversal.

```

```

    def dfs(node, current_sum):
        if not node:
            return 0
        # Update the current sum (forming the number by appending current
        # node's value).
        current_sum = current_sum * 10 + node.val
        # If the node is a leaf, return the current sum.
        if not node.left and not node.right:
            return current_sum
        # Recursively compute the sum for left and right subtrees.
        left_sum = dfs(node.left, current_sum)

        right_sum = dfs(node.right, current_sum)
        return left_sum + right_sum

    # Start DFS from the root with an initial sum of 0.
    return dfs(root, 0)

```

◆ Quick Review

Key Insights

- Utilize a Depth-First Search (DFS) or recursion strategy to traverse the tree.
- Carry forward a running value that represents the number formed from the root to the current node.
- When a leaf node is reached, the currently formed number is complete and can be added to the total sum.
- Use the mathematical transformation $\text{current_value} * 10 + \text{node.val}$ to update the number along the path.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes in the tree.

Space Complexity: $O(h)$ where h is the height of the tree because of the recursion call stack.

Solution

The problem can be solved using recursive DFS. Start at the root with an initial value of 0. For each node, update the accumulated number by multiplying the current sum by 10 and adding the node's digit. When a leaf is encountered (a node with no children), add the

current number to the total sum. Finally, return the sum of numbers formed from all root-to-leaf paths. This approach guarantees that every node is visited exactly once and uses only the call stack proportional to the height of the tree.

Tree Traversal – Post-Order

□ #652 Find Duplicate Subtrees

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Tree · Depth-First Search · Binary
Tree

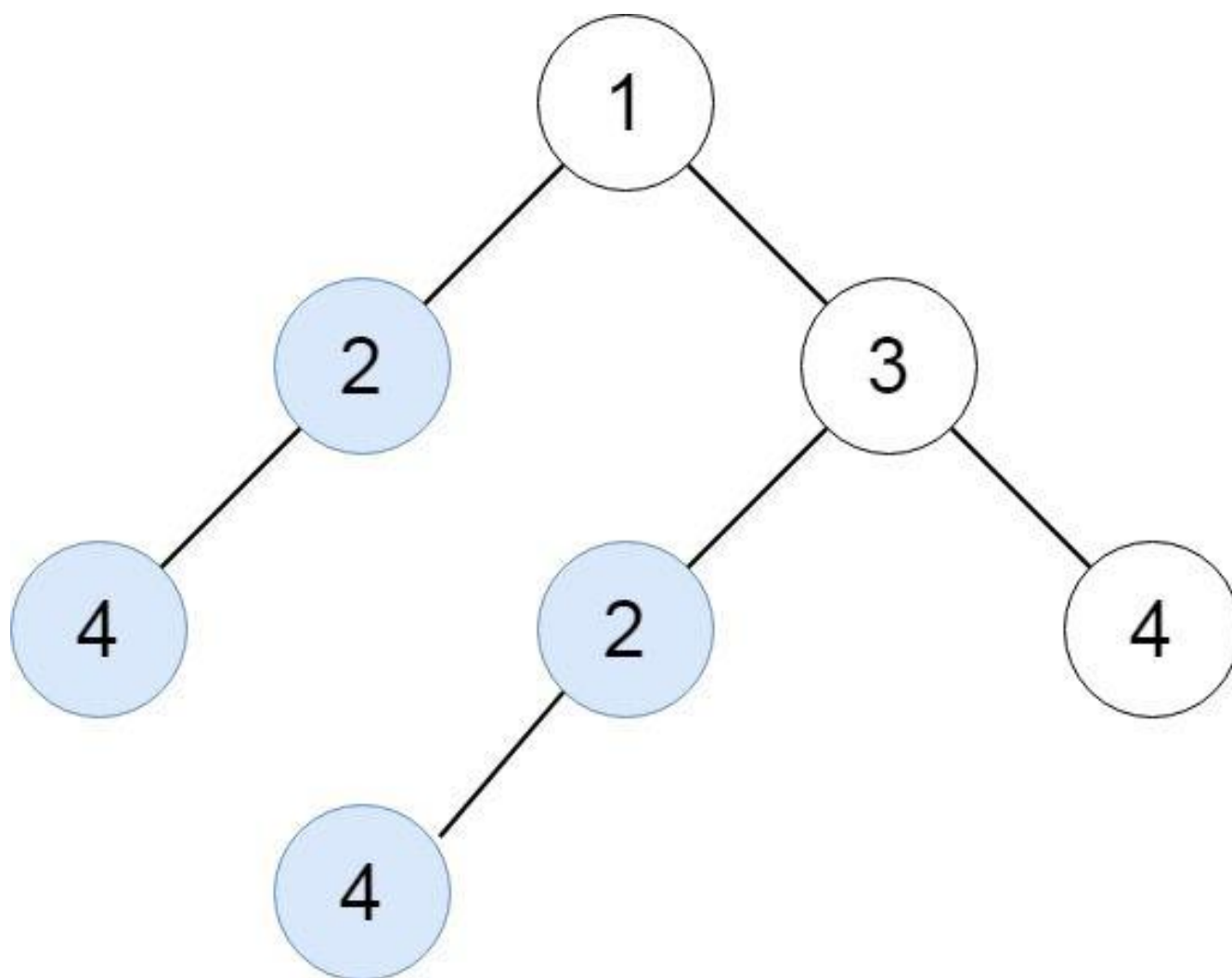
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, return all duplicate subtrees.

For each kind of duplicate subtrees, you only need to return the root node of any one of them. Two trees are duplicate if they have the same structure with the same node values.

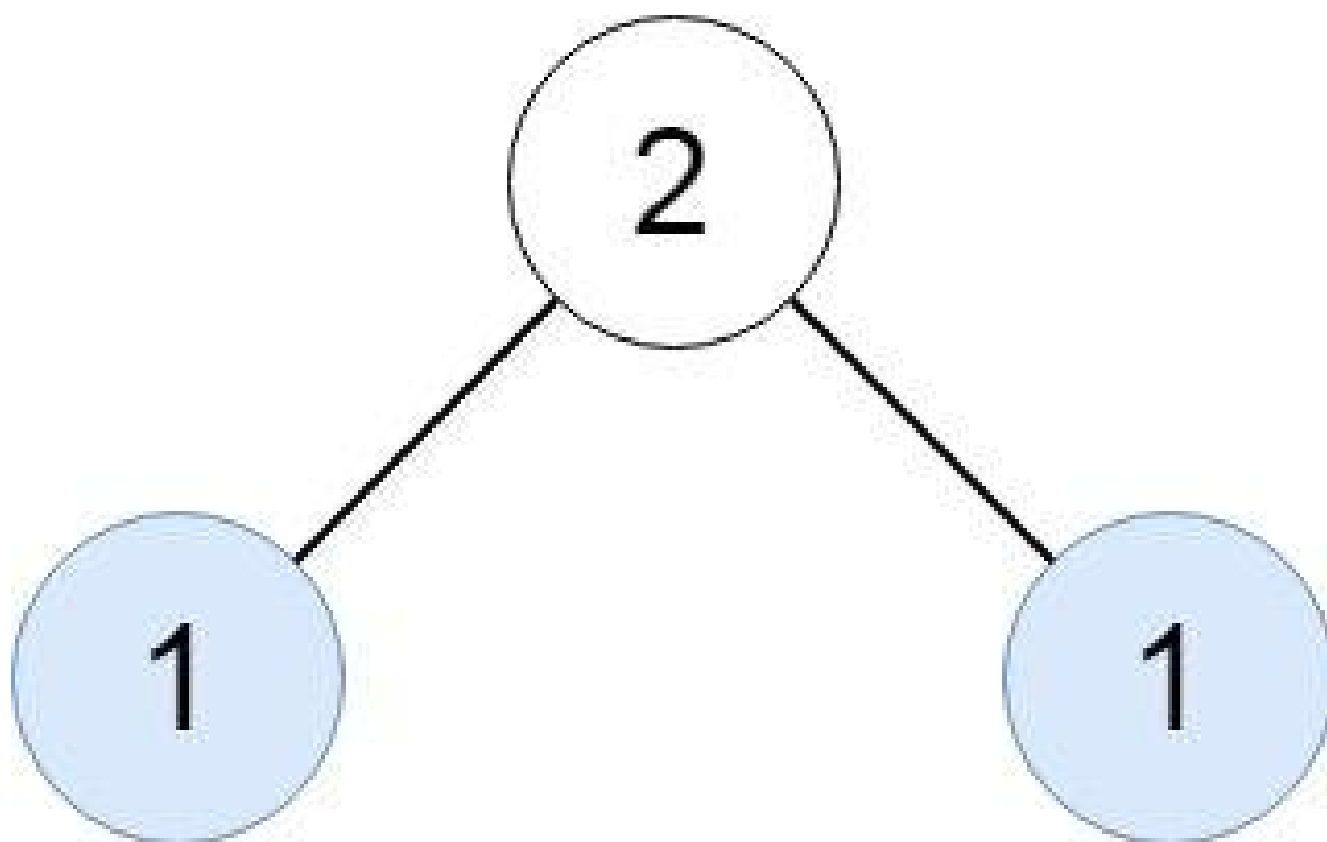
Example 1:



Input: root = [1,2,3,4,null,2,4,null,null,4]

Output: [[2,4],[4]]

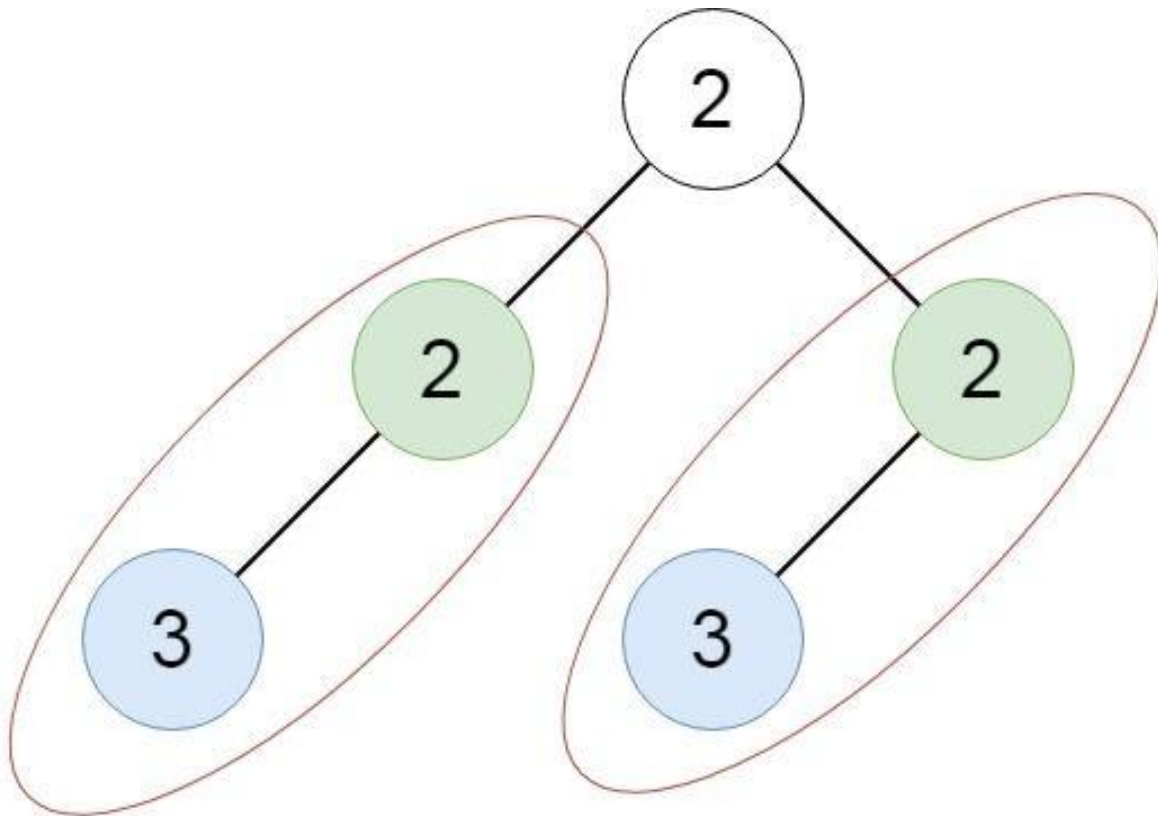
Example 2:



Input: root = [2,1,1]

Output: [[1]]

Example 3:



Input: root = [2,2,2,3,null,3,null]
Output: [[2,3],[3]]

Constraints:

- The number of the nodes in the tree will be in the range [1, 5000]
- $-200 \leq \text{Node.val} \leq 200$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def findDuplicateSubtrees(self, root: TreeNode | None) -> list[TreeNode | None]:
        ans = []
        count = collections.Counter()

        def encode(root: TreeNode | None) -> str:
            if not root:
                return ''

            encoded = (str(root.val) + '#' +
                      encode(root.left) + '#' +
                      encode(root.right))
            count[encoded] += 1
            if count[encoded] == 2:
                ans.append(root)
            return encoded

        encode(root)
        return ans

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findDuplicateSubtrees(
        self, root: Optional[TreeNode]
    ) -> List[Optional[TreeNode]]:

```

```
def dfs(root):
    if root is None:
        return '#'
    v = f'{root.val},{dfs(root.left)},{dfs(root.right)}'
    counter[v] += 1
    if counter[v] == 2:
        ans.append(root)
    return v
```

```
ans = []
```

```
counter = Counter()
dfs(root)
return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def findDuplicateSubtrees(self, root: TreeNode) -> List[TreeNode]:
        # Dictionary to store the frequency of each subtree serialization.
```

```
        subtree_count = {}
        # List to store the root nodes of duplicate subtrees.
        duplicates = []

        # Define a helper function for postorder traversal and serialization.
        def serialize(node):
            # If the current node is None, use a special marker.
            if not node:
                return "#"
            # Recursively serialize the left and right subtree.
```

```
        left_serial = serialize(node.left)
        right_serial = serialize(node.right)
        # Create a unique serialization for the subtree rooted at the
        current node.
        serial = str(node.val) + "," + left_serial + "," + right_serial
        # Update the count of this serialization in our dictionary.
        subtree_count[serial] = subtree_count.get(serial, 0) + 1
        # When a duplicate is found for the first time, add the node to the
        duplicates list.
        if subtree_count[serial] == 2:
            duplicates.append(node)
        # Return the serialization to the parent caller.

    return serial

serialize(root)
return duplicates
```

◆ Quick Review

Key Insights

- Use a postorder traversal (DFS) so that each subtree is serialized after processing its children.
- Represent each subtree uniquely (e.g., as a string serialization) to detect duplicates.
- Use a hash table (dictionary) to count the occurrence of each serialized subtree.
- When a serialized subtree's count becomes two, add its root node to the result list (to ensure each duplicate subtree is reported only once).

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree. (In the worst-case scenario, serialization of each subtree might lead to higher complexity, but typically each node is processed once.)

Space Complexity: $O(n)$, for storing the serialization of each subtree and the hash table.

Solution

The solution adopts a DFS (postorder traversal) approach where each subtree is serialized into a string based on its node value and the serializations of its left and right children. This unique serialization acts as a key in a hash table that records how many times a particular subtree has been seen. When a subtree's serialization appears for the second time, we record its root in the result array. This method ensures that we only return one instance for each group of duplicate subtrees.

Tree Traversal – Post-Order

□ #979 Distribute Coins in Binary Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Binary Tree
Lists: AlgoMaster 600

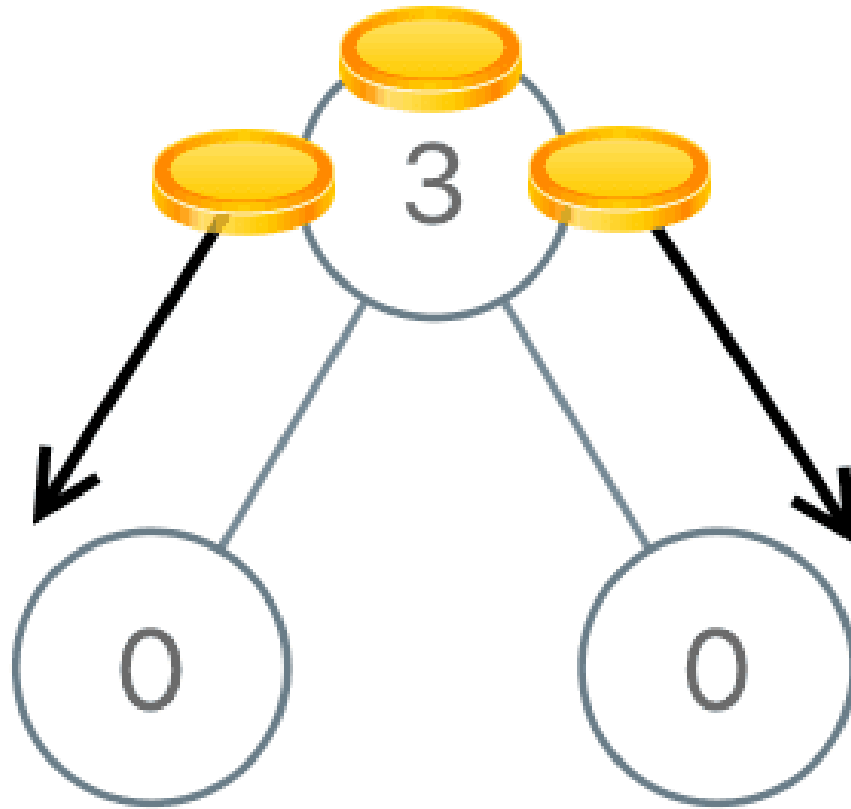
Problem

You are given the root of a binary tree with n nodes where each node in the tree has `node.val` coins. There are n coins in total throughout the whole tree.

In one move, we may choose two adjacent nodes and move one coin from one node to another. A move may be from parent to child, or from child to parent.

Return the minimum number of moves required to make every node have exactly one coin.

Example 1:

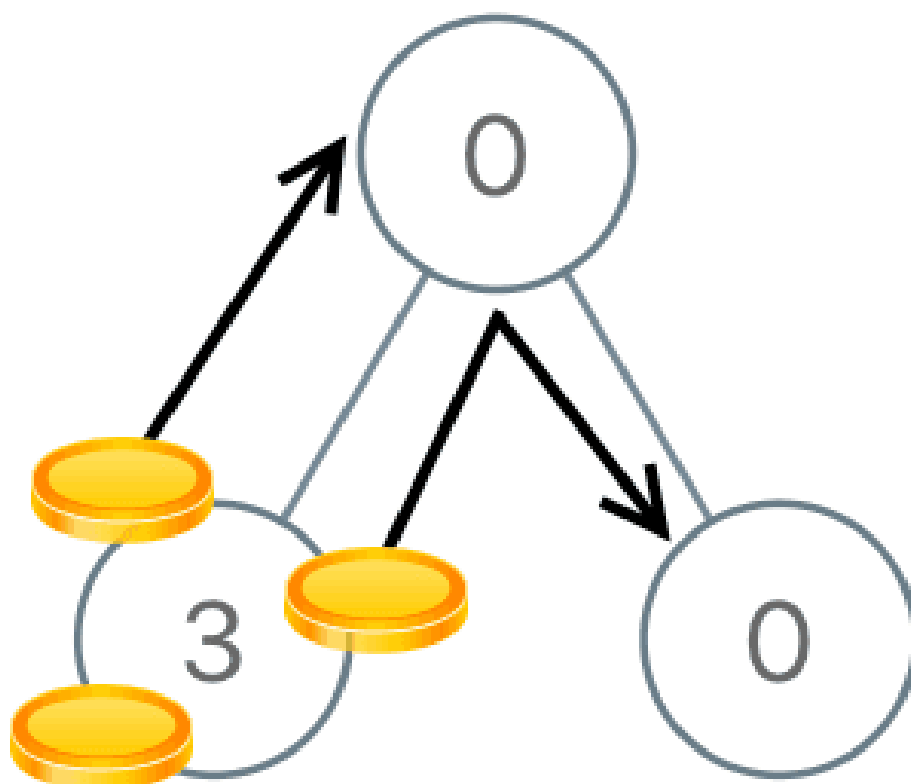


Input: root = [3,0,0]

Output: 2

Explanation: From the root of the tree, we move one coin to its left child, and one coin to its right child.

Example 2:



Input: root = [0,3,0]

Output: 3

Explanation: From the left child of the root, we move two coins to the root [taking two moves]. Then, we move one coin from the root of the tree to the right child.

Constraints:

- The number of nodes in the tree is n .
- $1 \leq n \leq 100$
- $0 \leq \text{Node.val} \leq n$
- The sum of all Node.val is n .

Community Solutions (Python)

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def distributeCoins(self, root: Optional[TreeNode]) -> int:
        def dfs(root):
            if root is None:
                return 0
            left, right = dfs(root.left), dfs(root.right)
            nonlocal ans
            ans += abs(left) + abs(right)
            return left + right + root.val - 1

        ans = 0
        dfs(root)
        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val # number of coins at the node
        self.left = left # left child
        self.right = right # right child

class Solution:
    def distributeCoins(self, root: TreeNode) -> int:
        self.moves = 0 # initialize the move counter
```

```
def dfs(node):
    if not node:
        return 0 # base case: null nodes contribute 0 balance
    # Recursively process left and right children
    left_balance = dfs(node.left)
    right_balance = dfs(node.right)
    # Accumulate moves: absolute coin imbalances from both children
    self.moves += abs(left_balance) + abs(right_balance)
    # Return the current node's net balance

    return node.val + left_balance + right_balance - 1

dfs(root)
return self.moves
```

◆ Quick Review

Key Insights

- Each node's balance is defined as (number of coins at the node) – 1, which indicates the surplus (if positive) or deficit (if negative).
- Use a Depth-First Search (DFS) traversal (post-order) to process children before handling the parent.
- At each node, the number of moves required is the sum of the absolute balances from its left and right subtrees.
- The DFS returns the net balance for each subtree so that these balances can be accumulated at the parent.

- The total moves required is the sum of the absolute values of the surplus/deficits transferred across each edge.

Space & Time

Time Complexity: $O(n)$ since each node is processed once.

Space Complexity: $O(h)$, where h is the height of the tree (due to recursion stack).

Solution

We solve the problem by performing a post-order DFS traversal on the tree. At every node, compute the net coin balance ($\text{node.val} + \text{left balance} + \text{right balance} - 1$). The moves needed at that node are the absolute values of the left and right balances, reflecting the coins that must be moved into or out of that subtree. Accumulate these moves during the traversal. Finally, the sum of all moves gives the minimum number of moves required to achieve the desired coin distribution.

Tree Traversal – Post-Order

□ #1110 Delete Nodes And Return Forest

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Tree · Depth-First Search
· Binary Tree

Lists: AlgoMaster 600

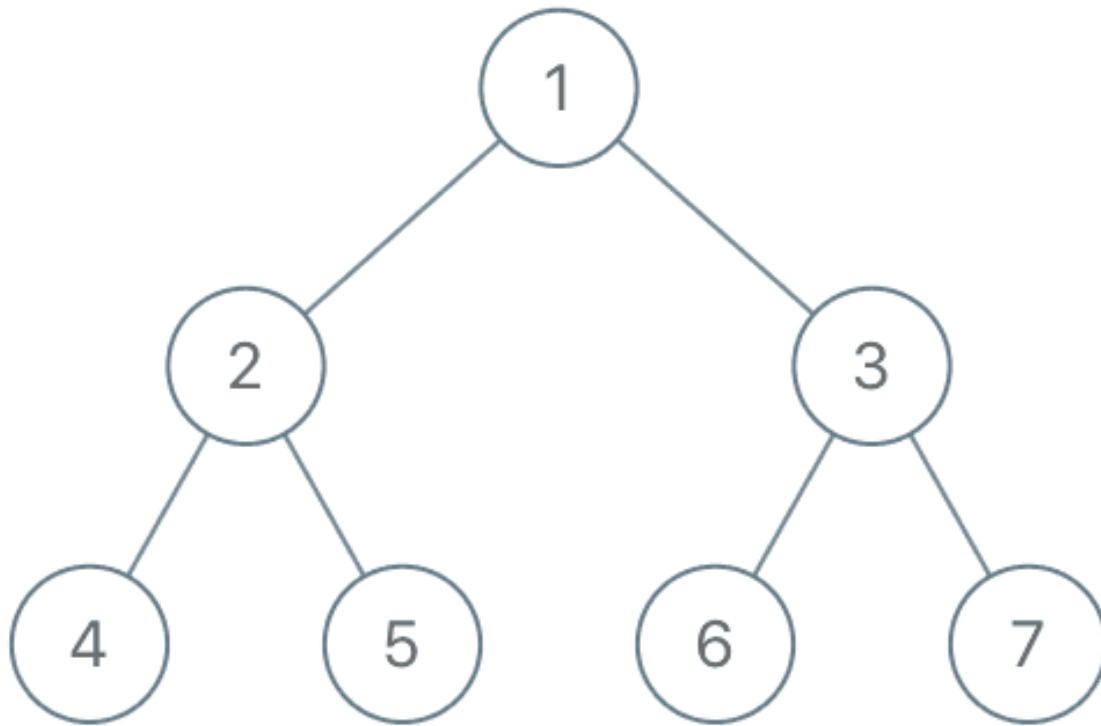
Problem

Given the root of a binary tree, each node in the tree has a distinct value.

After deleting all nodes with a value in `to_delete`, we are left with a forest (a disjoint union of trees).

Return the roots of the trees in the remaining forest. You may return the result in any order.

Example 1:



Input: root = [1,2,3,4,5,6,7], to_delete = [3,5]
Output: [[1,2,null,4],[6],[7]]

Example 2:

Input: root = [1,2,4,null,3], to_delete = [3]
Output: [[1,2,4]]

Constraints:

- The number of nodes in the given tree is at most 1000.
- Each node has a distinct value between 1 and 1000.
- `to_delete.length` $\leq$ 1000
- `to_delete` contains distinct values between 1 and 1000.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def delNodes(self, root: TreeNode, to_delete: list[int]) -> list[TreeNode]:
        ans = []
        toDeleteSet = set(to_delete)

        def dfs(root: TreeNode, isRoot: bool) -> TreeNode:
            if not root:
                return None

            deleted = root.val in toDeleteSet

            if isRoot and not deleted:
                ans.append(root)

            # If root is deleted, both children have the possibility to be a new root
            root.left = dfs(root.left, deleted)
            root.right = dfs(root.right, deleted)
            return None if deleted else root

        dfs(root, True)
        return ans
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def delNodes(
        self, root: Optional[TreeNode], to_delete: List[int]
    ) -> List[TreeNode]:
```

```

def dfs(root: Optional[TreeNode]) -> Optional[TreeNode]:
    if root is None:
        return None
    root.left, root.right = dfs(root.left), dfs(root.right)
    if root.val not in s:
        return root
    if root.left:
        ans.append(root.left)
    if root.right:
        ans.append(root.right)

    return None

s = set(to_delete)
ans = []
if dfs(root):
    ans.append(root)
return ans

```

• SimplyLeet

```

# Python solution with line-by-line comments

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:

    def delNodes(self, root: TreeNode, to_delete: list) -> list:
        # Convert the to_delete list to a set for faster lookup
        to_delete_set = set(to_delete)
        # Result list to hold roots of the remaining forest
        remaining_forest = []

        def dfs(node, is_root):
            # Base condition: if the current node is null, return None
            if not node:
                return None

            if node.val in to_delete_set:
                # If the current node is to be deleted, delete its children
                dfs(node.left, True)
                dfs(node.right, True)
                return None

            # If the current node is not to be deleted, add it to the forest
            if is_root:
                remaining_forest.append(node)

            dfs(node.left, False)
            dfs(node.right, False)

        dfs(root, True)
        return remaining_forest

```

```
# Determine if current node needs deletion
node_deleted = node.val in to_delete_set

# If this node is a root (or newly added) and is not deleted,
# add it to the remaining forest.
if is_root and not node_deleted:
    remaining_forest.append(node)

# Recursively process the left and right subtrees

node.left = dfs(node.left, node_deleted)
node.right = dfs(node.right, node_deleted)

# Return None if node is deleted, else return the node itself.
return None if node_deleted else node

# Start DFS from the root, the root is considered as a starting root.
dfs(root, True)
return remaining_forest
```

◆ Quick Review

Key Insights

- Use a recursive depth–first search (DFS) to traverse the tree.
- At each node, determine if it should be deleted based on the `to_delete` set.
- When deleting a node, ensure that its non–deleted children are added as new roots in the forest.
- Postorder traversal (processing children before the current node) is beneficial so that

children are processed before potentially deleting the parent.

- Using a set for `to_delete` items allows for fast lookup.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes since each node is visited once.

Space Complexity: $O(N)$ for the recursion stack and the set used to store deletion values and resulting forest.

Solution

We employ a DFS approach to traverse the tree in a postorder manner, processing left and right children first. For each node:

- Check if the node should be deleted by looking it up in the `to_delete` set.
- If the node is flagged for deletion, add its non-null children as potential new roots to the forest.
- Return null for a deleted node and the node itself if it is kept. This recursive process ensures that any subtree whose root gets deleted properly adds any potential subtrees to the answer list before the node is removed.

Tree Traversal – Post-Order

□ #2096 Step-By-Step Directions From a Binary Tree Node to Another

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Tree · Depth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

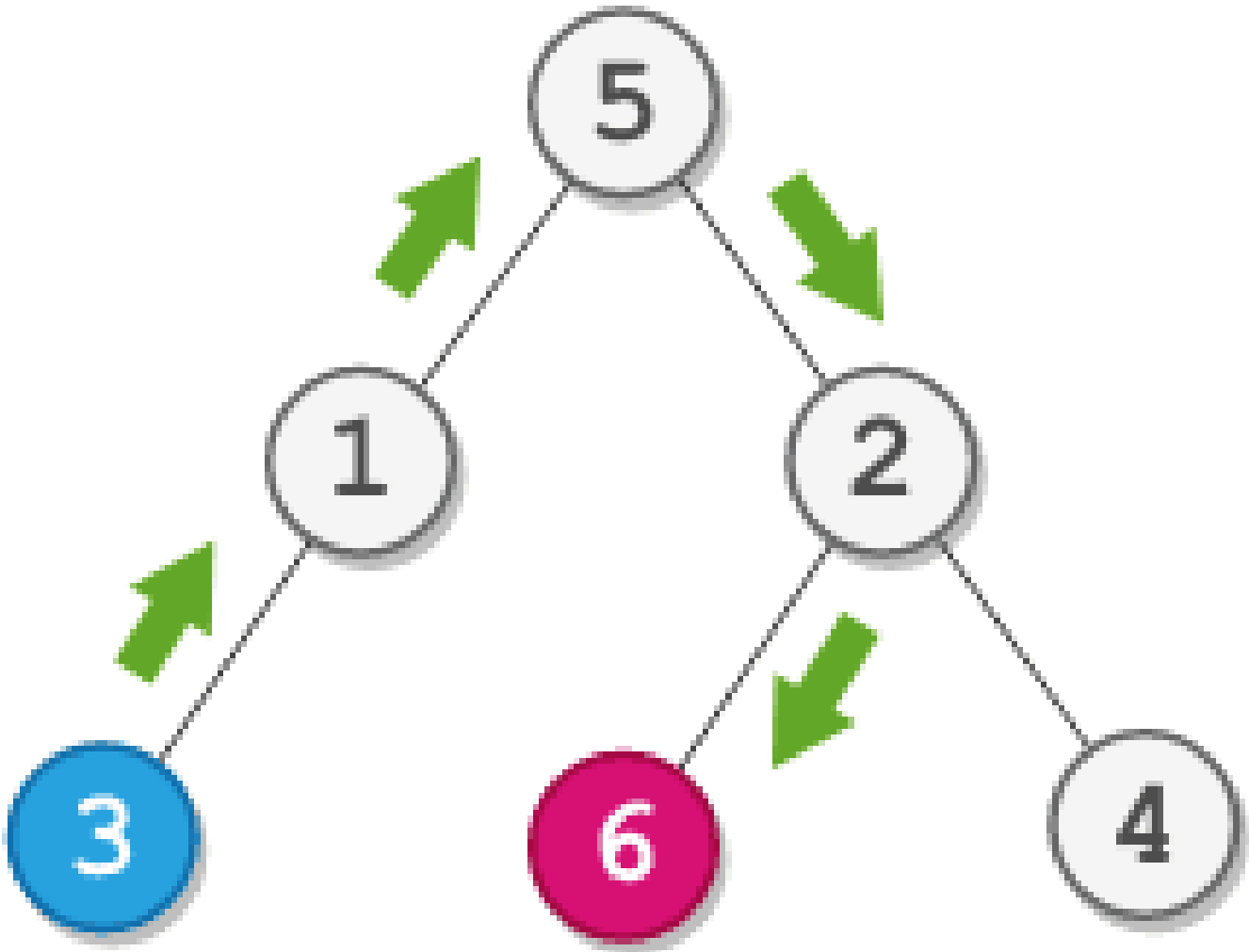
You are given the root of a binary tree with n nodes. Each node is uniquely assigned a value from 1 to n . You are also given an integer `startValue` representing the value of the start node s , and a different integer `destValue` representing the value of the destination node t . Find the shortest path starting from node s and ending at node t . Generate step-by-step directions of such path as a string consisting of only the uppercase letters 'L', 'R', and 'U'. Each letter indicates a specific direction:

- 'L' means to go from a node to its left child node.
- 'R' means to go from a node to its right child node.

- 'U' means to go from a node to its parent node.

Return the step-by-step directions of the shortest path from node *s* to node *t*.

Example 1:

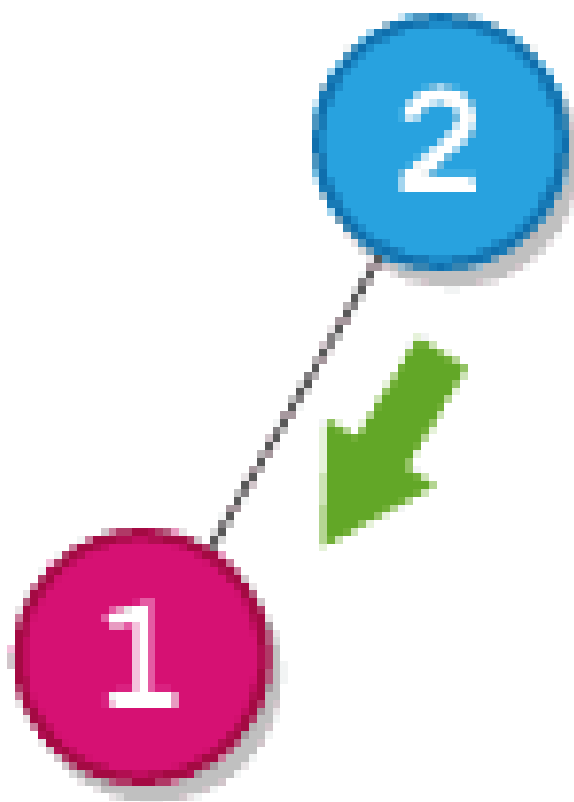


Input: root = [5,1,2,3,null,6,4], startValue = 3, destValue = 6

Output: "UURL"

Explanation: The shortest path is: 3 → 1 → 5 → 2 → 6.

Example 2:



Input: root = [2,1], startValue = 2, destValue = 1

Output: "L"

Explanation: The shortest path is: 2 → 1.

Constraints:

- The number of nodes in the tree is n.
- $2 \leq n \leq 105$
- $1 \leq \text{Node.val} \leq n$
- All the values in the tree are unique.
- $1 \leq \text{startValue}, \text{destValue} \leq n$
- $\text{startValue} \neq \text{destValue}$

Community Solutions (Python)

- WalkCC


```

class Solution:
    def getDirections(
        self,
        root: TreeNode | None,
        startValue: int,
        destValue: int,
    ) -> str:
        def lca(root: TreeNode | None) -> TreeNode | None:
            if not root or root.val in (startValue, destValue):
                return root

            left = lca(root.left)
            right = lca(root.right)
            if left and right:
                return root
            return left or right

        def dfs(root: TreeNode | None, path: list[str]) -> None:
            if not root:
                return
            if root.val == startValue:
                self.pathToStart = ''.join(path)
                if root.val == destValue:
                    self.pathToDest = ''.join(path)
                path.append('L')
                dfs(root.left, path)
                path.pop()
                path.append('R')
                dfs(root.right, path)
                path.pop()

        dfs(lca(root), []) # Only this subtree matters.
        return 'U' * len(self.pathToStart) + ''.join(self.pathToDest)

```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def getDirections(
        self, root: Optional[TreeNode], startValue: int, destValue: int
    ) -> str:
        def lca(node: Optional[TreeNode], p: int, q: int):
            if node is None or node.val in (p, q):
                return node
            left = lca(node.left, p, q)
            right = lca(node.right, p, q)
            if left and right:
                return node
            return left or right

        def dfs(node: Optional[TreeNode], x: int, path: List[str]):
            if node is None:
                return False
            if node.val == x:
                return True
            path.append("L")
            if dfs(node.left, x, path):
                return True
            path[-1] = "R"

            if dfs(node.right, x, path):
                return True
            path.pop()
            return False

        node = lca(root, startValue, destValue)

        path_to_start = []
        path_to_dest = []
```

```

dfs(node, startValue, path_to_start)
dfs(node, destValue, path_to_dest)

return "U" * len(path_to_start) + "".join(path_to_dest)

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# Function to perform DFS and find the path from the root to a target node.
# The path list will record 'L' or 'R' for each move from the current node.
def dfs(node, target, path):

    if node is None:
        return False
    if node.val == target:
        return True
    # Try left subtree
    path.append('L')
    if dfs(node.left, target, path):
        return True
    path.pop()
    # Try right subtree

    path.append('R')
    if dfs(node.right, target, path):
        return True
    path.pop()
    return False

def getDirections(root, startValue, destValue):
    # Get the paths from root to start and destination
    path_to_start = []
    path_to_dest = []

```

```

dfs(root, startValue, path_to_start)
dfs(root, destValue, path_to_dest)

# Find the common prefix length (i.e. the LCA)
i = 0
while i < len(path_to_start) and i < len(path_to_dest) and path_to_start[i] == path_to_dest[i]:
    i += 1

# Steps to move up from start to the LCA

upward_moves = "U" * (len(path_to_start) - i)
# Steps to move down from the LCA to the destination
downward_moves = "".join(path_to_dest[i:])

return upward_moves + downward_moves

# Example usage:
# Suppose we have a tree constructed and we need to call getDirections(root,
startValue, destValue)

```

◆ Quick Review

Key Insights

- Find the path from the root to the start node and the root to the destination node using DFS.
- Identify the last common node (i.e. the lowest common ancestor, LCA) by comparing the two paths.
- The moves from the start node to the LCA are represented by ‘U’s.

- The remaining part of the destination path (from the LCA to the destination node) provides the directions ('L' or 'R').
- Concatenate the appropriate number of 'U's with the unique portion of the destination path to obtain the final direction string.

Space & Time

Time Complexity: $O(n)$ – Traversing the tree using DFS and comparing paths takes linear time.

Space Complexity: $O(n)$ – In the worst-case scenario, the recursion stack and path storage will use linear space.

Solution

We solve the problem by:

- Using DFS to obtain the path from the root to the start node and from the root to the destination node. During DFS, we accumulate the direction taken (either 'L' or 'R').
- Comparing the two generated paths from the beginning to determine where they diverge. This divergence point corresponds to the lowest common ancestor (LCA).

- For the portion of the start path after the common part, each step is replaced by a ‘U’ (to move upward from the start node to the LCA).
- After reaching the LCA, append the remaining steps from the destination path to reach the destination node.
- Finally, return the concatenated instruction string.

The key data structures used are:

- Vectors/arrays (or lists) to store the path for each target node.
- Recursive DFS for tree traversal.

This approach ensures that we correctly translate the downward path information into the required “upward then downward” directions.

Depth First Search (DFS)

Round 2 · High · Medium · 4 questions

Depth First Search (DFS)

□ #690 Employee Importance

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Tree · Depth-First Search
· Breadth-First Search
Lists: AlgoMaster 600

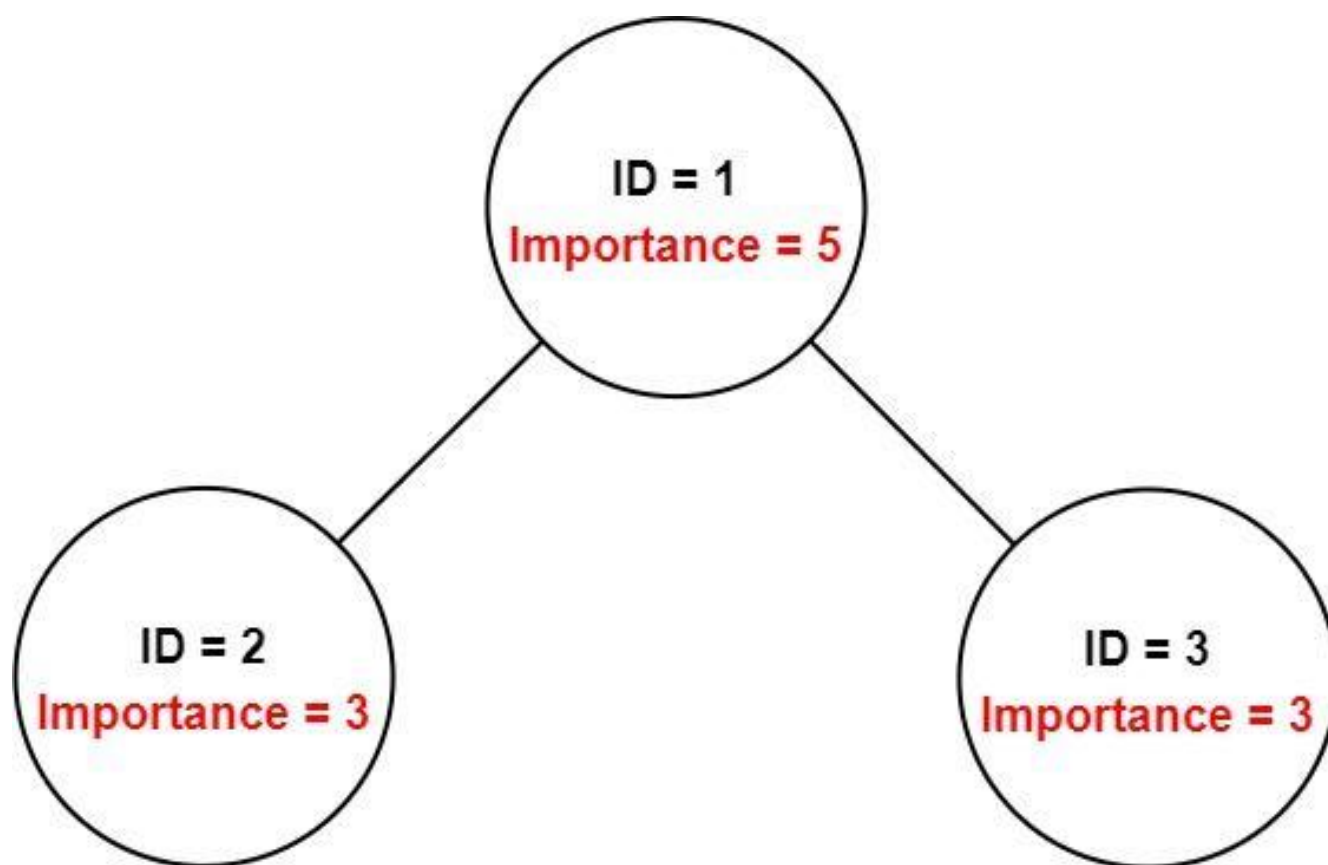
Problem

You have a data structure of employee information, including the employee's unique ID, importance value, and direct subordinates' IDs. You are given an array of employees `employees` where:

- `employees[i].id` is the ID of the *i*th employee.
- `employees[i].importance` is the importance value of the *i*th employee.
- `employees[i].subordinates` is a list of the IDs of the direct subordinates of the *i*th employee.

Given an integer `id` that represents an employee's ID, return the total importance value of this employee and all their direct and indirect subordinates.

Example 1:



Input: employees = [[1,5,[2,3]],[2,3,[]],[3,3,[]]], id = 1

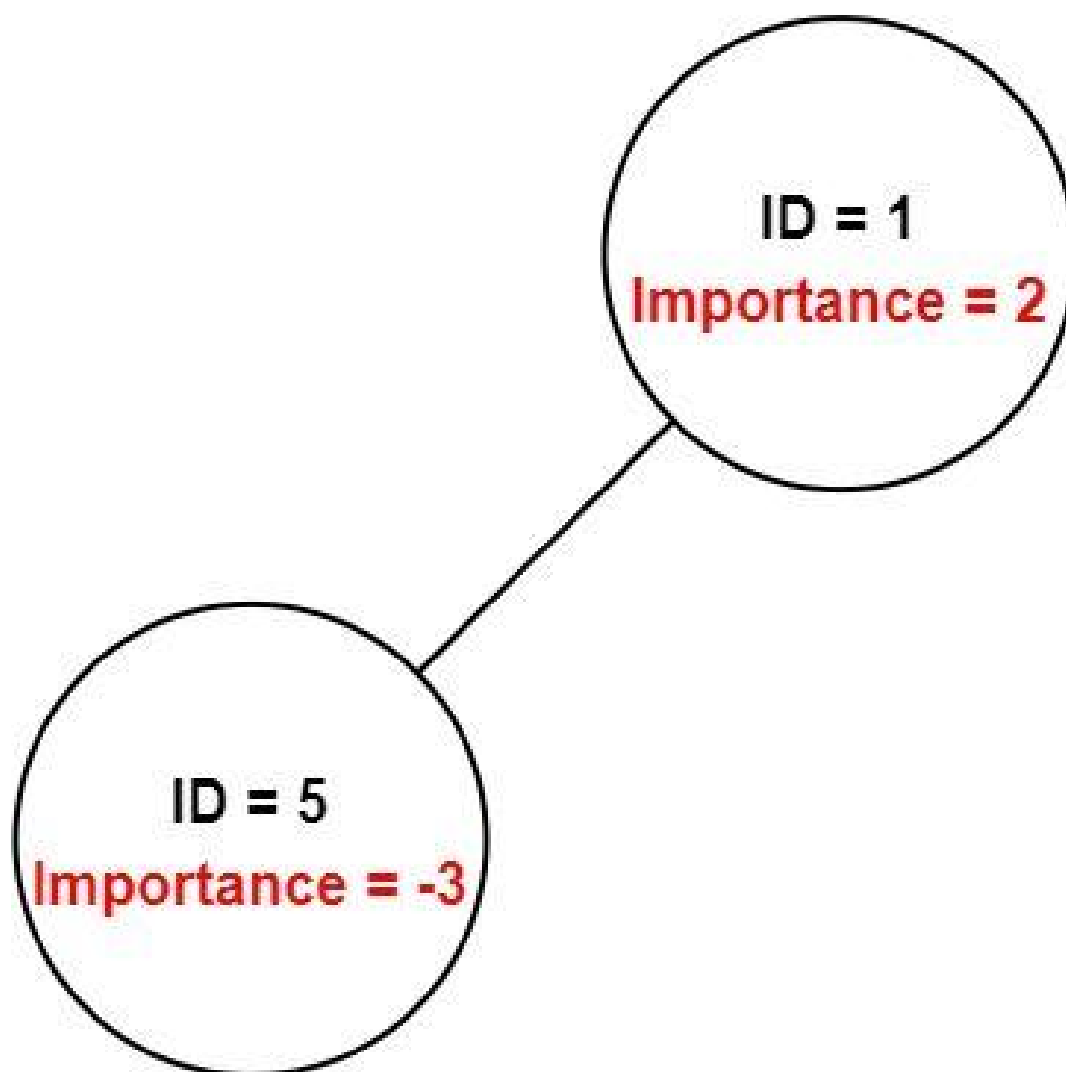
Output: 11

Explanation: Employee 1 has an importance value of 5 and has two direct subordinates: employee 2 and employee 3.

They both have an importance value of 3.

Thus, the total importance value of employee 1 is $5 + 3 + 3 = 11$.

Example 2:



Input: employees = [[1,2,[5]],[5,-3,[]]], id = 5

Output: -3

Explanation: Employee 5 has an importance value of -3 and has no direct subordinates.

Thus, the total importance value of employee 5 is -3.

Constraints:

- $1 \leq \text{employees.length} \leq 2000$
- $1 \leq \text{employees}[i].\text{id} \leq 2000$
- All $\text{employees}[i].\text{id}$ are unique.
- $-100 \leq \text{employees}[i].\text{importance} \leq 100$

- One employee has at most one direct leader and may have several subordinates.
- The IDs in `employees[i].subordinates` are valid IDs.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def getImportance(self, employees: list['Employee'], id: int) -> int:
        idToEmployee = {employee.id: employee for employee in employees}

        def dfs(id: int) -> int:
            values = idToEmployee[id].importance
            for subId in idToEmployee[id].subordinates:
                values += dfs(subId)
            return values

        return dfs(id)
```

• LeetDoocs

```
"""
# Definition for Employee.
class Employee:
    def __init__(self, id: int, importance: int, subordinates: List[int]):
        self.id = id
        self.importance = importance
        self.subordinates = subordinates
"""
```

```
class Solution:
    def getImportance(self, employees: List["Employee"], id: int) -> int:
        def dfs(i: int) -> int:
            return d[i].importance + sum(dfs(j) for j in d[i].subordinates)

        d = {e.id: e for e in employees}
        return dfs(id)
```

• SimplyLeet

```
# Define the Employee class (for context)
class Employee:
    def __init__(self, id, importance, subordinates):
        self.id = id                # Unique employee ID
        self.importance = importance # Importance value of the employee
        self.subordinates = subordinates # List of subordinate IDs

def getImportance(employees, id):
    # Create a mapping from employee id to the employee object for quick lookup
    employee_map = {employee.id: employee for employee in employees}

    # Define a recursive DFS function
    def dfs(emp_id):
        current = employee_map[emp_id] # Get current employee object using
employee_map
        total = current.importance      # Start with current employee's
importance

        # Recursively add importance from all direct subordinates
        for sub_id in current.subordinates:
            total += dfs(sub_id)        # Add the importance from subordinate
subtree

        return total

    return dfs(id) # Initiate DFS from the given employee id

# Example usage:
# employees = [Employee(1, 5, [2,3]), Employee(2, 3, []), Employee(3, 3, [])]
# print(getImportance(employees, 1)) # Output: 11
```

◆ Quick Review

Key Insights

- Map employee IDs to their corresponding employee object for quick lookup.
- The problem can be modeled as a tree or graph where nodes represent employees.
- Traverse the tree (using DFS or BFS) starting from the given employee ID.
- Sum the importance values along the traversal path.
- Handle potential negative importance values.

Space & Time

Time Complexity: $O(n)$, where n is the number of employees, since in the worst case every employee will be visited.

Space Complexity: $O(n)$ in the worst case due to recursion call stack (DFS) or queue (BFS) and additional hashmap for storing employee lookup.

Solution

We first build a hashmap (or dictionary) mapping each employee's ID to the employee object for $O(1)$ access during traversal. Then,

using either Depth First Search (DFS) or Breadth First Search (BFS), we traverse from the target employee, and at each node we add its importance to a running total. The DFS approach can be implemented recursively or iteratively with a stack, while the BFS approach makes use of a queue. The key idea is ensuring that each employee is processed exactly once.

Depth First Search (DFS)

□ #797 All Paths From Source to Target

MED

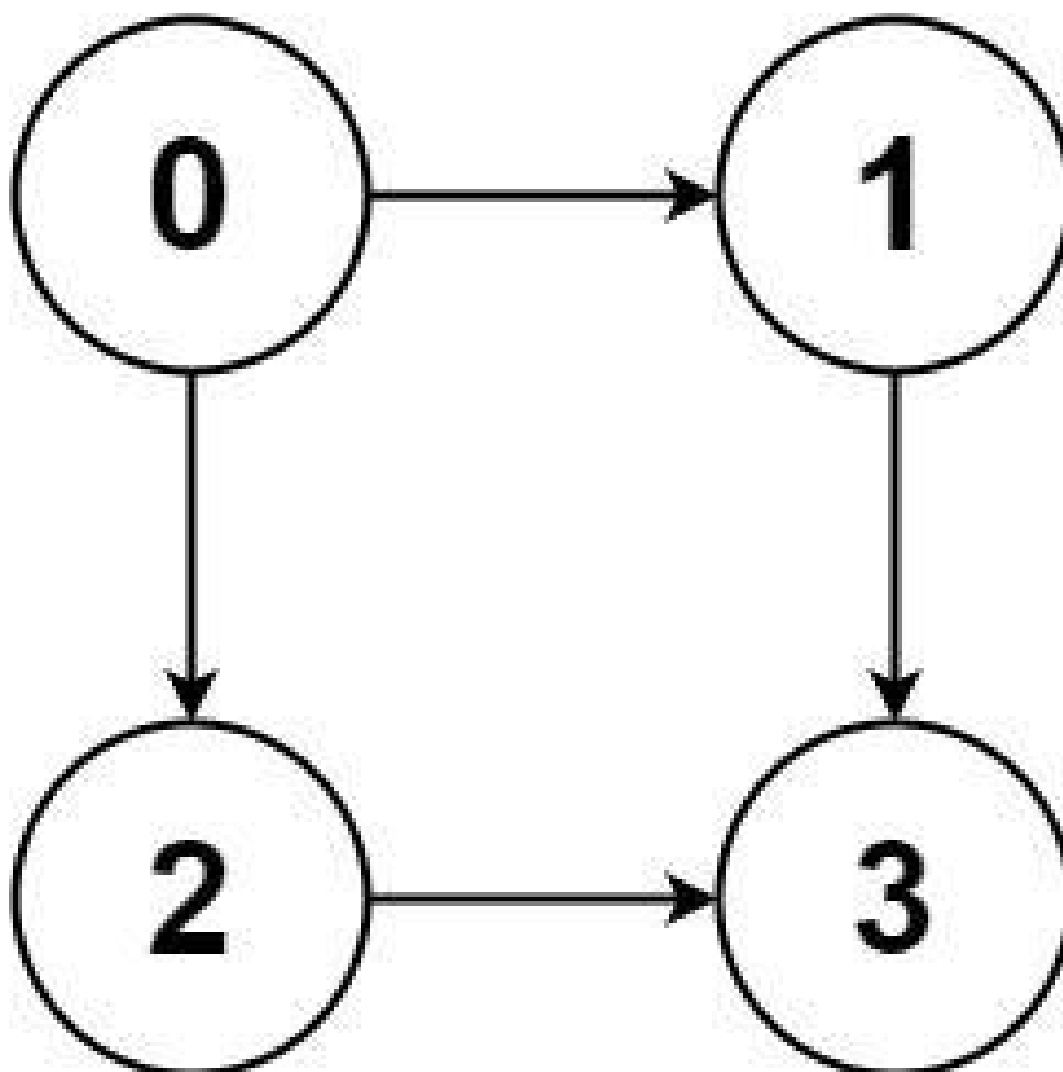
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Backtracking · Depth-First Search ·
Breadth-First Search · Graph Theory
Lists: AlgoMaster 600

Problem

Given a directed acyclic graph (DAG) of n nodes labeled from 0 to $n - 1$, find all possible paths from node 0 to node $n - 1$ and return them in any order.

The graph is given as follows: $\text{graph}[i]$ is a list of all nodes you can visit from node i (i.e., there is a directed edge from node i to node $\text{graph}[i][j]$).

Example 1:

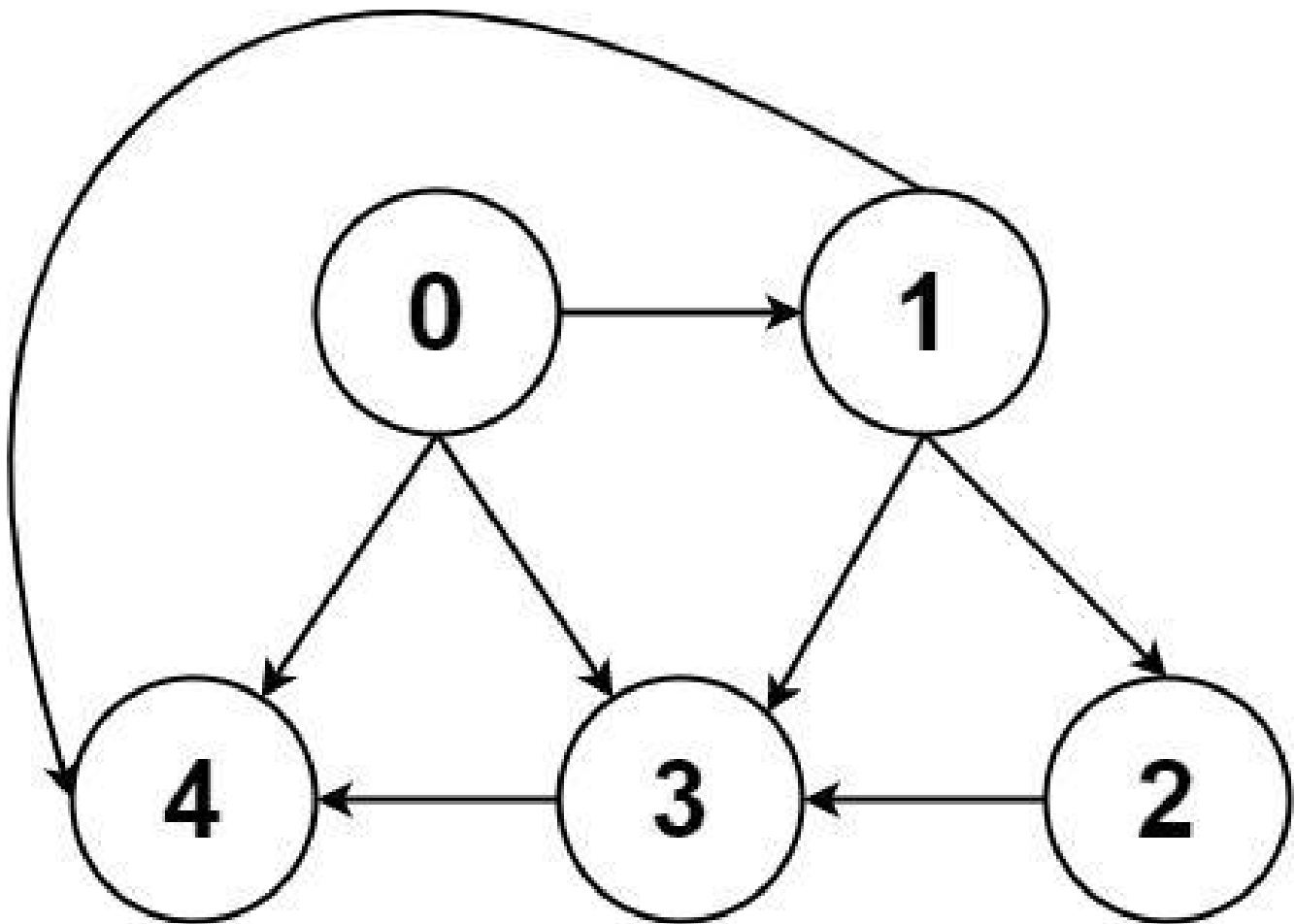


Input: graph = [[1,2],[3],[3],[]]

Output: [[0,1,3],[0,2,3]]

Explanation: There are two paths: 0 -> 1 -> 3 and 0 -> 2 -> 3.

Example 2:



Input: graph = [[4,3,1],[3,2,4],[3],[4],[]]

Output: [[0,4],[0,3,4],[0,1,3,4],[0,1,2,3,4],[0,1,4]]

Constraints:

- $n == \text{graph.length}$
- $2 \leq n \leq 15$
- $0 \leq \text{graph}[i][j] < n$
- $\text{graph}[i][j] \neq i$ (i.e., there will be no self-loops).
- All the elements of $\text{graph}[i]$ are unique.
- The input graph is guaranteed to be a DAG.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def allPathsSourceTarget(self, graph: list[list[int]]) -> list[list[int]]:
        ans = []

        def dfs(u: int, path: list[int]) -> None:
            if u == len(graph) - 1:
                ans.append(path)
                return

            for v in graph[u]:
                dfs(v, path + [v])

        dfs(0, [0])
        return ans
```

• LeetDoocs

```
class Solution:
    def allPathsSourceTarget(self, graph: List[List[int]]) -> List[List[int]]:
        n = len(graph)
        q = deque([[0]])
        ans = []
        while q:
            path = q.popleft()
            u = path[-1]
            if u == n - 1:
                ans.append(path)
                continue
            for v in graph[u]:
                q.append(path + [v])
        return ans
```

• SimplyLeet

```
def allPathsSourceTarget(graph):  
    # List to store all valid paths from source to target  
    result = []  
    # Current path tracker  
    path = []  
  
    def dfs(node):  
        # Add current node to the path  
        path.append(node)  
        # If target node is reached, add a copy of the current path to result  
  
        if node == len(graph) - 1:  
            result.append(list(path))  
        else:  
            # Explore all neighbors of the current node  
            for neighbor in graph[node]:  
                dfs(neighbor)  
        # Backtrack: remove the current node before exploring a new branch  
        path.pop()  
  
    # Start DFS from node 0  
  
    dfs(0)  
    return result  
  
# Example usage:  
# print(allPathsSourceTarget([[1,2],[3],[3],[[]]]))
```

◆ Quick Review

Key Insights

- The graph is a DAG, so there are no cycles. This makes a recursive Depth-First Search (DFS) or backtracking approach viable without needing to worry about infinite loops.

- Every decision (choosing a neighbor) is independent because paths do not cycle back.
- Backtracking is essential to explore different branches, adding a node, and later removing it to reset the current path.
- Since the output requires listing complete paths from the source (0) to the target ($n - 1$), we need to store and eventually copy the current path once a valid path is found.

Space & Time

Time Complexity: $O(2^n * n)$ in the worst-case scenario where exponential number of paths may exist, each path requiring $O(n)$ to copy.

Space Complexity: $O(n)$ for the recursion stack and current path, excluding the space required for the output.

Solution

We can solve this problem using a Depth-First Search (DFS) algorithm with backtracking. The basic idea is to start at node 0 and explore each neighbor recursively. When the target node ($n - 1$) is reached, a copy of the current path is added to the result list. Once the recursive call returns, we backtrack by

removing the last node so that we can continue exploring other possible paths from the previous node.

The following solutions illustrate how to implement this approach in various programming languages.

Depth First Search (DFS)

□ #1020 Number of Enclaves

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Depth-First Search · Breadth-First
Search · Union-Find · Matrix
Lists: AlgoMaster 600

Problem

You are given an $m \times n$ binary matrix grid, where 0 represents a sea cell and 1 represents a land cell.

A move consists of walking from one land cell to another adjacent (4-directionally) land cell or walking off the boundary of the grid.

Return the number of land cells in grid for which we cannot walk off the boundary of the grid in any number of moves.

Example 1:

0	0	0	0
1	0	1	0
0	1	1	0
0	0	0	0

Input: grid = [[0,0,0,0],[1,0,1,0],[0,1,1,0],[0,0,0,0]]

Output: 3

Explanation: There are three 1s that are enclosed by 0s, and one 1 that is not enclosed because its on the boundary.

Example 2:

0	1	1	0
0	0	1	0
0	0	1	0
0	0	0	0

Input: grid = [[0,1,1,0],[0,0,1,0],[0,0,1,0],[0,0,0,0]]

Output: 0

Explanation: All 1s are either on the boundary or can reach the boundary.

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 500$
- $\text{grid}[i][j]$ is either 0 or 1.

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def numEnclaves(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int):
            grid[i][j] = 0
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and grid[x][y]:
                    dfs(x, y)

        m, n = len(grid), len(grid[0])

        dirs = (-1, 0, 1, 0, -1)
        for j in range(n):
            if grid[0][j]:
                dfs(0, j)
            if grid[m - 1][j]:
                dfs(m - 1, j)
        for i in range(m):
            if grid[i][0]:
                dfs(i, 0)
            if grid[i][n - 1]:
                dfs(i, n - 1)

        return sum(sum(row) for row in grid)

class Solution:
    def numEnclaves(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        q = deque()
        for i in range(m):
            for j in range(n):
                if grid[i][j] and (i == 0 or i == m - 1 or j == 0 or j == n - 1):
                    q.append((i, j))
                    grid[i][j] = 0
        dirs = (-1, 0, 1, 0, -1)
```

```

while q:
    i, j = q.popleft()
    for a, b in pairwise(dirs):
        x, y = i + a, j + b
        if x >= 0 and x < m and y >= 0 and y < n and grid[x][y]:
            q.append((x, y))
            grid[x][y] = 0
return sum(v for row in grid for v in row)

```

```

class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

```

```

def union(self, a, b):
    pa, pb = self.find(a), self.find(b)
    if pa != pb:
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]

```

```

class Solution:
    def numEnclaves(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        uf = UnionFind(m * n + 1)
        dirs = (-1, 0, 1, 0, -1)
        for i, row in enumerate(grid):
            for j, v in enumerate(row):
                if v:
                    if i == 0 or i == m - 1 or j == 0 or j == n - 1:

```

```

        uf.union(i * n + j, m * n)
    else:
        for a, b in pairwise(dirs):
            x, y = i + a, j + b
            if x >= 0 and x < m and y >= 0 and y < n and grid[
x][y]:
                uf.union(i * n + j, x * n + y)
    return sum(
        grid[i][j] == 1 and uf.find(i * n + j) != uf.find(m * n)
        for i in range(m)
        for j in range(n)
    )

```

• SimplyLeet

```

class Solution:
    def numEnclaves(self, grid: List[List[int]]) -> int:
        # Calculate number of rows and columns
        m, n = len(grid), len(grid[0])

        # Helper function using DFS to mark connected land as sea.
        def dfs(r, c):
            # Base case: if out of bounds or already water, return
            if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] == 0:
                return

            # Mark the current cell as water to avoid revisiting it
            grid[r][c] = 0
            # Explore all 4 directions (up, down, left, right)
            dfs(r + 1, c)
            dfs(r - 1, c)
            dfs(r, c + 1)
            dfs(r, c - 1)

        # Remove boundary-connected land cells by traversing first and last
        # columns for each row.
        for r in range(m):

```

```
        dfs(r, 0)
        dfs(r, n - 1)
    # Traverse first and last rows for each column.
    for c in range(n):
        dfs(0, c)
        dfs(m - 1, c)

    # Count remaining land cells that are enclaves.
    enclaveCount = sum(grid[r][c] for r in range(m) for c in range(n))
    return enclaveCount
```

◆ Quick Review

Key Insights

- Any land cell that is connected to the boundary can reach the edge and should not be counted.
- Use DFS/BFS to traverse and mark all land cells connected to the boundary as water.
- Once the boundary-connected cells are removed, count the remaining land cells in the grid.

Space & Time

Time Complexity: $O(m * n)$ – Each cell is processed at most once.

Space Complexity: $O(m * n)$ – In worst-case scenarios (e.g., deep recursion in DFS), the auxiliary space used can be proportional to the

number of cells.

Solution

The approach involves a two-step process:

- Traverse the boundaries of the grid and perform a DFS/BFS from each boundary cell that is a land cell. During traversal, mark all reachable land cells (i.e., cells that can potentially reach the boundary) as sea.**
- After processing the boundaries, count all remaining land cells in the grid. These are the enclaves that cannot reach the boundary.**

Key data structures include the grid itself and, depending on the implementation, either a recursion stack (DFS) or an explicit queue (BFS) for traversal. The main trick is to remove (or mark) all boundary-connected land cells before counting, effectively isolating the enclaves.

Depth First Search (DFS)

□ #1376 Time Needed to Inform All Employees

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Breadth-First
Search

Lists: AlgoMaster 600

Problem

A company has n employees with a unique ID for each employee from 0 to $n - 1$. The head of the company is the one with `headID`.

Each employee has one direct manager given in the `manager` array where `manager[i]` is the direct manager of the i -th employee, `manager[headID] = -1`. Also, it is guaranteed that the subordination relationships have a tree structure.

The head of the company wants to inform all the company employees of an urgent piece of news. He will inform his direct subordinates, and they will inform their subordinates, and so on until all employees know about the urgent news.

The i -th employee needs `informTime[i]` minutes to inform all of his direct subordinates (i.e., After `informTime[i]` minutes, all his direct subordinates can start spreading the news). Return the number of minutes needed to inform all the employees about the urgent news.

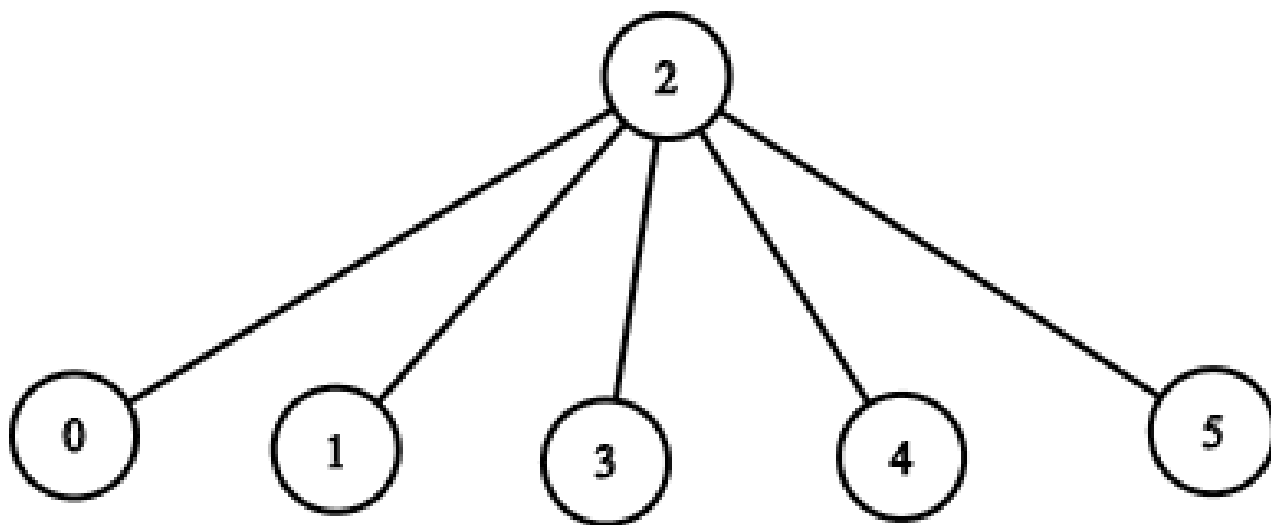
Example 1:

Input: `n = 1, headID = 0, manager = [-1], informTime = [0]`

Output: 0

Explanation: The head of the company is the only employee in the company.

Example 2:



Input: `n = 6, headID = 2, manager = [2,2,-1,2,2,2], informTime = [0,0,1,0,0,0]`

Output: 1

Explanation: The head of the company with `id = 2` is the direct manager of all the employees in the company and needs 1 minute to inform them all.

The tree structure of the employees in the company is shown.

Constraints:

- $1 \leq n \leq 105$

- $0 \leq \text{headID} < n$
- `manager.length == n`
- $0 \leq \text{manager}[i] < n$
- `manager[headID] == -1`
- `informTime.length == n`
- $0 \leq \text{informTime}[i] \leq 1000$
- `informTime[i] == 0` if employee *i* has no subordinates.
- It is guaranteed that all the employees can be informed.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def numOfMinutes(
        self, n: int, headID: int, manager: List[int], informTime: List[int]
    ) -> int:
        def dfs(i: int) -> int:
            ans = 0
            for j in g[i]:
                ans = max(ans, dfs(j) + informTime[i])
            return ans

        g = defaultdict(list)
        for i, x in enumerate(manager):
            g[x].append(i)
        return dfs(headID)
```

• SimplyLeet


```
# Python solution using DFS and an adjacency list to represent the tree.
from collections import defaultdict

def numOfMinutes(n, headID, manager, informTime):
    # Build the tree: map each manager to its direct subordinates.
    tree = defaultdict(list)
    for employee, mgr in enumerate(manager):
        if mgr != -1:
            tree[mgr].append(employee)

    # DFS function to compute the time to inform all subordinates under 'emp'
    def dfs(emp):
        max_subordinate_time = 0
        # Traverse all direct subordinates of the employee
        for subordinate in tree[emp]:
            max_subordinate_time = max(max_subordinate_time, dfs(subordinate))
        # Total time is the current employee's inform time plus max time
        # required by subordinates.
        return informTime[emp] + max_subordinate_time

    return dfs(headID)

# Example usage:
# n = 6, headID = 2, manager = [2,2,-1,2,2,2], informTime = [0,0,1,0,0,0]
# print(numOfMinutes(6, 2, [2,2,-1,2,2,2], [0,0,1,0,0,0]))
```

◆ Quick Review

Key Insights

- The company structure forms a tree with the head as the root.
- We need to propagate the news from the head down through all branches.

- The time taken for the news to fully propagate is determined by the longest path from the head to any leaf in the tree.
- A Depth-First Search (DFS) efficiently traverses the tree, accumulating inform times along each branch.

Space & Time

Time Complexity: $O(n)$ – Each employee (node) is visited once.

Space Complexity: $O(n)$ – In the worst-case, the recursion stack may hold all n nodes.

Solution

We construct an adjacency list (or tree) that maps each manager to their list of direct subordinates. Then, we perform a DFS starting from the head. For each employee, we calculate the time required to inform all their subordinates by taking the maximum time among all children and adding the current employee's `informTime`. This approach correctly computes the time to disseminate the news along the longest chain in the tree.

Breadth First Search (BFS)

Round 2 · High · Medium · 5 questions

Breadth First Search (BFS)

□ #433 Minimum Genetic Mutation

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Breadth-First Search
Lists: AlgoMaster 600

Problem

A gene string can be represented by an 8-character long string, with choices from 'A', 'C', 'G', and 'T'.

Suppose we need to investigate a mutation from a gene string `startGene` to a gene string `endGene` where one mutation is defined as one single character changed in the gene string.

- For example, "AACCGGTT" --> "AACCGGTA" is one mutation.

There is also a gene bank `bank` that records all the valid gene mutations. A gene must be in `bank` to make it a valid gene string.

Given the two gene strings `startGene` and `endGene` and the gene bank `bank`, return the minimum number of mutations needed to

mutate from startGene to endGene. If there is no such a mutation, return -1.

Note that the starting point is assumed to be valid, so it might not be included in the bank.

Example 1:

```
Input: startGene = "AACCGGTT", endGene = "AACCGGTA", bank = ["AACCGGTA"]  
Output: 1
```

Example 2:

```
Input: startGene = "AACCGGTT", endGene = "AAACGGTA", bank =  
["AACCGGTA", "AACCGCTA", "AAACGGTA"]  
Output: 2
```

Constraints:

- $0 \leq \text{bank.length} \leq 10$
- $\text{startGene.length} == \text{endGene.length} == \text{bank}[i].\text{length} == 8$
- startGene, endGene, and bank[i] consist of only the characters ['A', 'C', 'G', 'T'].

Community Solutions (Python)

- ☐
- WalkCC

```

class Solution:
    def minMutation(self, startGene: str, endGene: str, bank: list[str]) -> int:
        bankSet = set(bank)
        if endGene not in bankSet:
            return -1

        GENES = 'ACGT'
        q = collections.deque([startGene])

        step = 1

        while q:
            for _ in range(len(q)):
                wordList = list(q.popleft())
                for j, cache in enumerate(wordList):
                    for c in GENES:
                        wordList[j] = c
                        word = ''.join(wordList)
                        if word == endGene:
                            return step
                        if word in bankSet:

                            bankSet.remove(word)
                            q.append(word)
                            wordList[j] = cache
                step += 1

        return -1

```

• LeetDoocs

```

class Solution:
    def minMutation(self, startGene: str, endGene: str, bank: List[str]) -> int:
        q = deque([(startGene, 0)])
        vis = {startGene}
        while q:
            gene, depth = q.popleft()
            if gene == endGene:
                return depth
            for nxt in bank:
                diff = sum(a != b for a, b in zip(gene, nxt))

```

```

        if diff == 1 and nxt not in vis:
            q.append((nxt, depth + 1))
            vis.add(nxt)

    return -1

```

• SimplyLeet

```

# Python implementation for Minimum Genetic Mutation
from collections import deque

def minMutation(startGene, endGene, bank):
    # Convert bank list to set for quick lookup
    bankSet = set(bank)

    # Early exit if endGene is not in the bank
    if endGene not in bankSet:
        return -1

    # Queue holds tuples (current_gene, mutations_count)
    queue = deque([(startGene, 0)])
    # Valid characters for mutation
    validGenes = ['A', 'C', 'G', 'T']

    while queue:
        current, mutations = queue.popleft()
        # If the current gene equals the endGene, we found the mutation path
        if current == endGene:
            return mutations

        # Try all possible single-character mutations
        for i in range(len(current)):
            for char in validGenes:
                if char == current[i]:
                    continue

                # Create a new gene string mutation
                mutated = current[:i] + char + current[i+1:]
                # If this mutation is valid and in the bank

```

```
if mutated in bankSet:
    # Remove to avoid revisiting and add to the queue
    bankSet.remove(mutated)
    queue.append((mutated, mutations + 1))

# No valid mutation sequence was found
return -1

# Example usage:
# print(minMutation("AACCGGTT", "AAACGGTA",
# ["AACCGGTA", "AACCGCTA", "AAACGGTA"]))
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) as it finds the shortest path (minimum mutations) in an unweighted graph.
- Mutations are generated by changing each character of the gene string one by one to one of the valid characters: A, C, G, or T.
- Use a set for the bank for $O(1)$ membership checks.
- Avoid revisiting the same gene by marking mutations as visited (or removing them from the bank).

Space & Time

Time Complexity: $O(N * L * 4)$ where N is the number of genes in the bank and L is the

length of the gene (here $L=8$), due to checking each position with 4 possible mutations.

Space Complexity: $O(N)$ for the queue and visited set, with additional $O(N)$ for the bank set.

Solution

We solve the problem using a Breadth-First Search (BFS) algorithm. The idea is to start from the initial gene and generate all possible one-character mutations. Each valid mutation (i.e., one that exists in the bank) is added to the BFS queue. For each level of BFS, we increment the mutation count. When the end gene is reached, we return the current mutation count.

We store the bank in a set to ensure constant time membership checks and avoid revisiting states. This solution guarantees finding the minimum number of mutations needed if a valid sequence exists, otherwise, it returns -1 .

Breadth First Search (BFS)

□ #752 Open the Lock

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Breadth-First
Search

Lists: AlgoMaster 600

Problem

You have a lock in front of you with 4 circular wheels. Each wheel has 10 slots: '0', '1', '2', '3', '4', '5', '6', '7', '8', '9'. The wheels can rotate freely and wrap around: for example we can turn '9' to be '0', or '0' to be '9'. Each move consists of turning one wheel one slot.

The lock initially starts at '0000', a string representing the state of the 4 wheels.

You are given a list of deadends dead ends, meaning if the lock displays any of these codes, the wheels of the lock will stop turning and you will be unable to open it.

Given a target representing the value of the wheels that will unlock the lock, return the minimum total number of turns required to

open the lock, or -1 if it is impossible.

Example 1:

Input: deadends = ["0201","0101","0102","1212","2002"], target = "0202"

Output: 6

Explanation:

A sequence of valid moves would be "0000" -> "1000" -> "1100" -> "1200" -> "1201" -> "1202" -> "0202".

Note that a sequence like "0000" -> "0001" -> "0002" -> "0102" -> "0202" would be invalid, because the wheels of the lock become stuck after the display becomes the dead end "0102".

Example 2:

Input: deadends = ["8888"], target = "0009"

Output: 1

Explanation: We can turn the last wheel in reverse to move from "0000" -> "0009".

Example 3:

Input: deadends = ["8887","8889","8878","8898","8788","8988","7888","9888"], target = "8888"

Output: -1

Explanation: We cannot reach the target without getting stuck.

Constraints:

- $1 \leq \text{deadends.length} \leq 500$
- $\text{deadends}[i].\text{length} == 4$
- $\text{target.length} == 4$
- target will not be in the list deadends.
- target and deadends[i] consist of digits only.

Community Solutions (Python)

- [LeetDoocs](#)

```

class Solution:
    def openLock(self, deadends: List[str], target: str) -> int:
        def next(s):
            res = []
            s = list(s)
            for i in range(4):
                c = s[i]
                s[i] = '9' if c == '0' else str(int(c) - 1)
                res.append(''.join(s))
                s[i] = '0' if c == '9' else str(int(c) + 1)

            res.append(''.join(s))
            s[i] = c
            return res

        if target == '0000':
            return 0
        s = set(deadends)
        if '0000' in s:
            return -1
        q = deque(['0000'])

        s.add('0000')
        ans = 0
        while q:
            ans += 1
            for _ in range(len(q)):
                p = q.popleft()
                for t in next(p):
                    if t == target:
                        return ans
                    if t not in s:

                        q.append(t)
                        s.add(t)

        return -1

```

```

class Solution {
public:
    string target;

    int openLock(vector<string>& deadends, string target) {
        if (target == "0000") return 0;
        unordered_set<string> s(deadends.begin(), deadends.end());
        if (s.count("0000")) return -1;
        string start = "0000";
        this->target = target;

        typedef pair<int, string> PIS;
        priority_queue<PIS, vector<PIS>, greater<PIS>> q;
        unordered_map<string, int> dist;
        dist[start] = 0;
        q.push({f(start), start});
        while (!q.empty()) {
            PIS t = q.top();
            q.pop();
            string state = t.second;
            int step = dist[state];

            if (state == target) return step;
            for (string& t : next(state)) {
                if (s.count(t)) continue;
                if (!dist.count(t) || dist[t] > step + 1) {
                    dist[t] = step + 1;
                    q.push({step + 1 + f(t), t});
                }
            }
        }
        return -1;
    }

    int f(string s) {
        int ans = 0;
        for (int i = 0; i < 4; ++i) {
            int a = s[i] - '0';
            int b = target[i] - '0';
            if (a < b) {
                int t = a;
                a = b;
            }
        }
    }
}

```

```

        b = t;
    }
    ans += min(b - a, a + 10 - b);
}
return ans;
}

vector<string> next(string& t) {
    vector<string> res;
    for (int i = 0; i < 4; ++i) {

        char c = t[i];
        t[i] = c == '0' ? '9' : (char) (c - 1);
        res.push_back(t);
        t[i] = c == '9' ? '0' : (char) (c + 1);
        res.push_back(t);
        t[i] = c;
    }
    return res;
}
};

```

• SimplyLeet

```

# Python implementation
from collections import deque

def openLock(deadends, target):
    dead_set = set(deadends)
    # Early check: if starting point is a deadend, no solution exists.
    if "0000" in dead_set:
        return -1

    queue = deque(["0000", 0]) # (state, moves)

```

```

visited = {"0000"}

# Function to generate neighbors (all possible states from current state)
def get_neighbors(state):
    neighbors = []
    for i in range(4):
        digit = int(state[i])
        # Two possible moves: increment or decrement with wrap-around
        for move in (-1, 1):
            new_digit = (digit + move) % 10

            new_state = state[:i] + str(new_digit) + state[i+1:]
            neighbors.append(new_state)
    return neighbors

while queue:
    state, moves = queue.popleft()
    # Check if we reached the target combination
    if state == target:
        return moves

    for neighbor in get_neighbors(state):
        if neighbor not in visited and neighbor not in dead_set:
            visited.add(neighbor)
            queue.append((neighbor, moves + 1))

# If target is not reached, return -1
return -1

# Example test case
print(openLock(["0201","0101","0102","1212","2002"], "0202"))

```

◆ Quick Review

Key Insights

- This problem can be modeled as a graph where each node represents a 4-digit combination.

- Each valid move from one node generates up to 8 neighboring combinations (one for each wheel moving one step forward or backward).
- Breadth-First Search (BFS) is used to explore the graph level by level, which naturally finds the shortest path.
- Deadends must be avoided during the traversal.
- The initial state "0000" must be checked against the deadends.

Space & Time

Time Complexity: $O(10^N)$ where N is the number of wheels ($N = 4$ gives 10^4 combinations in worst-case scenario).

Space Complexity: $O(10^N)$ due to storing visited states and the queue in BFS.

Solution

We use a Breadth-First Search (BFS) to systematically explore all potential combinations starting from "0000". Each combination represents a state or node in our search space. From each state, we generate all valid next states by incrementing or decrementing each digit (with wraparound

behavior). We keep track of combinations that are either deadends or already visited to avoid cycles and unnecessary processing.

Key steps:

- Start at "0000" if it is not a deadend.
- Use a queue to process one level of combinations at a time, representing the number of moves.
- For each combination, generate all possible next moves (neighbors). Skip any that are deadends or that have been visited.
- If the target combination is reached, return the level (move count).
- If the queue empties without finding the target, return -1.

Breadth First Search (BFS)

□ #909 Snakes and Ladders

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Breadth-First Search · Matrix
Lists: AlgoMaster 600

Problem

You are given an $n \times n$ integer matrix board where the cells are labeled from 1 to n^2 in a Boustrophedon style starting from the bottom left of the board (i.e. `board[n - 1][0]`) and alternating direction each row.

You start on square 1 of the board. In each move, starting from square `curr`, do the following:

- Choose a destination square next with a label in the range `[curr + 1, min(curr + 6,  $n^2$ )]`. This choice simulates the result of a standard 6-sided die roll: i.e., there are always at most 6 destinations, regardless of the size of the board.

A board square on row `r` and column `c` has a snake or ladder if `board[r][c] != -1`. The

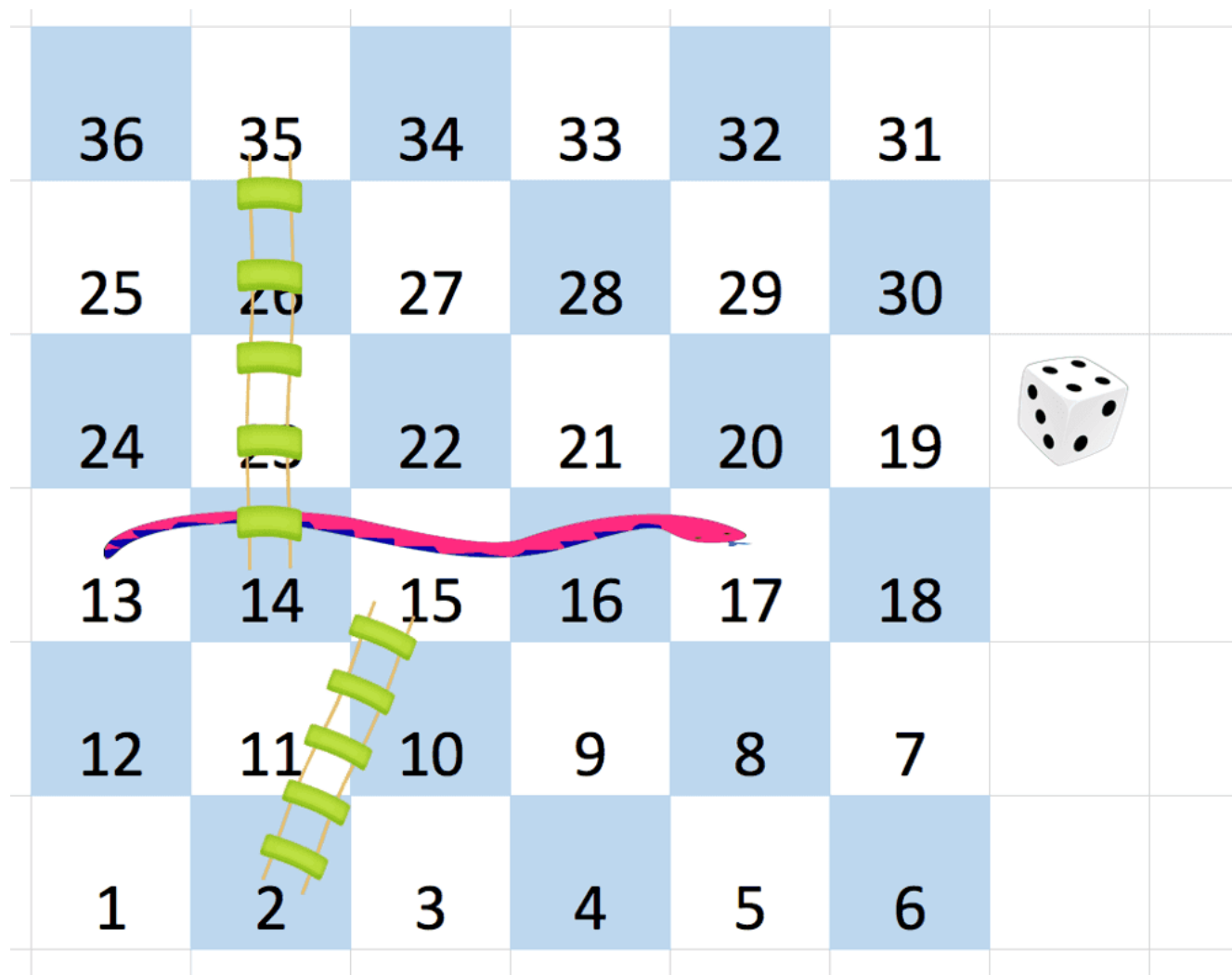
destination of that snake or ladder is `board[r][c]`. Squares 1 and `n2` are not the starting points of any snake or ladder.

Note that you only take a snake or ladder at most once per dice roll. If the destination to a snake or ladder is the start of another snake or ladder, you do not follow the subsequent snake or ladder.

- For example, suppose the board is `[[-1,4],[-1,3]]`, and on the first move, your destination square is 2. You follow the ladder to square 3, but do not follow the subsequent ladder to 4.

Return the least number of dice rolls required to reach the square `n2`. If it is not possible to reach the square, return `-1`.

Example 1:



Input: board = [[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],[-1,35,-1,-1,13,-1],[-1,-1,-1,-1,-1,-1],[-1,15,-1,-1,-1,-1]]

Output: 4

Explanation:

In the beginning, you start at square 1 (at row 5, column 0).

You decide to move to square 2 and must take the ladder to square 15.

You then decide to move to square 17 and must take the snake to square 13.

You then decide to move to square 14 and must take the ladder to square 35.

You then decide to move to square 36, ending the game.

Example 2:

Input: board = [[-1,-1],[-1,3]]

Output: 1

Constraints:

- $n == \text{board.length} == \text{board}[i].\text{length}$

- $2 \leq n \leq 20$
- `board[i][j]` is either `-1` or in the range `[1, n2]`.
- The squares labeled 1 and `n2` are not the starting points of any snake or ladder.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def snakesAndLadders(self, board: list[list[int]]) -> int:
        n = len(board)
        q = collections.deque([1])
        seen = set()
        arr = [0] * (1 + n * n)  # 2D -> 1D

        for i in range(n):
            for j in range(n):
                arr[(n - 1 - i) * n + (n - j if (n - i) % 2 == 0 else j + 1)] = board[i][j]

        step = 1
        while q:
            for _ in range(len(q)):
                curr = q.popleft()
                for next in range(curr + 1, min(curr + 6, n * n) + 1):
                    dest = arr[next] if arr[next] > 0 else next
                    if dest == n * n:
                        return step
                    if dest in seen:
                        continue
                    q.append(dest)
                    seen.add(dest)
            step += 1

        return -1
```

• LeetCode

```
class Solution:
    def snakesAndLadders(self, board: List[List[int]]) -> int:
        n = len(board)
        q = deque([1])
        vis = {1}
        ans = 0
        m = n * n
        while q:
            for _ in range(len(q)):
                x = q.popleft()

                if x == m:
                    return ans
                for y in range(x + 1, min(x + 6, m) + 1):
                    i, j = divmod(y - 1, n)
                    if i & 1:
                        j = n - j - 1
                    i = n - i - 1
                    z = y if board[i][j] == -1 else board[i][j]
                    if z not in vis:
                        vis.add(z)

                q.append(z)

            ans += 1
        return -1
```

• SimplyLeet

```
from collections import deque

def snakesAndLadders(board):
    n = len(board)

    # Helper function to convert a square number to board coordinates
    def get_position(square):
        # Determine row index from bottom to top
        quot, rem = divmod(square - 1, n)
        row = n - 1 - quot
```

```

        # For even rows (from bottom), direction is left->right; for odd rows,
        right->left.
        if quot % 2 == 0:
            col = rem
        else:
            col = n - 1 - rem
        return row, col

visited = set()
queue = deque([(1, 0)]) # tuple: (current square, moves)
visited.add(1)

```

```

while queue:
    curr, moves = queue.popleft()
    if curr == n * n:
        return moves
    for i in range(1, 7): # simulate die roll
        next_sq = curr + i
        if next_sq > n * n:
            continue
        r, c = get_position(next_sq)

```

```

        # if there's a snake or ladder, take the jump
        if board[r][c] != -1:
            next_sq = board[r][c]
        if next_sq not in visited:
            visited.add(next_sq)
            queue.append((next_sq, moves + 1))
    return -1

```

```

# Example usage:
#print(snakesAndLadders([[ -1, -1, -1, -1, -1, -1],

```

```

# [ -1, -1, -1, -1, -1, -1],
# [ -1, -1, -1, -1, -1, -1],
# [ -1, 35, -1, -1, 13, -1],
# [ -1, -1, -1, -1, -1, -1],
# [ -1, 15, -1, -1, -1, -1]]))

```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to explore all possible moves from the start while tracking the number of dice rolls.
- Convert the one-dimensional square number into board coordinates accounting for the alternating direction in each row.
- When landing on a snake or ladder, follow it immediately, but only take one jump per roll.
- Maintain a visited set or array to avoid re-processing cells and prevent infinite loops.

Space & Time

Time Complexity: $O(n^2)$ – each of the n^2 cells is processed at most once during the BFS.

Space Complexity: $O(n^2)$ – used for the queue and visited set to store positions.

Solution

The problem is solved via a Breadth-First Search (BFS) approach. Starting from square 1, we explore all reachable squares (up to six moves ahead) for each dice roll. For each move, we convert the square number into board coordinates while considering the alternating order of rows. When a move

encounters a snake or ladder (board value not equal to -1), we move directly to its destination. By using BFS, we ensure that the first time we reach the final square (n^2) it is with the fewest moves possible. To guarantee no repeated work, we track visited squares.

Breadth First Search (BFS)

□ #1091 Shortest Path in Binary Matrix

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Breadth-First Search · Matrix
Lists: AlgoMaster 600

Problem

Given an $n \times n$ binary matrix grid, return the length of the shortest clear path in the matrix. If there is no clear path, return -1 .

A clear path in a binary matrix is a path from the top-left cell (i.e., $(0, 0)$) to the bottom-right cell (i.e., $(n - 1, n - 1)$) such that:

- All the visited cells of the path are 0.
- All the adjacent cells of the path are 8-directionally connected (i.e., they are different and they share an edge or a corner).

The length of a clear path is the number of visited cells of this path.

Example 1:

0	1
1	0

Input: grid = [[0,1],[1,0]]

Output: 2

Example 2:

0	0	0
1	1	0
1	1	0

Input: grid = [[0,0,0],[1,1,0],[1,1,0]]
Output: 4

Example 3:

Input: grid = [[1,0,0],[1,1,0],[1,1,0]]
Output: -1

Constraints:

- $n == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq n \leq 100$
- $\text{grid}[i][j]$ is 0 or 1

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def shortestPathBinaryMatrix(self, grid: List[List[int]]) -> int:
        if grid[0][0]:
            return -1
        n = len(grid)
        grid[0][0] = 1
        q = deque([(0, 0)])
        ans = 1
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                if i == j == n - 1:
                    return ans
                for x in range(i - 1, i + 2):
                    for y in range(j - 1, j + 2):
                        if 0 <= x < n and 0 <= y < n and grid[x][y] == 0:
                            grid[x][y] = 1
                            q.append((x, y))
            ans += 1
        return -1
```

• SimplyLeet

```
from collections import deque

def shortestPathBinaryMatrix(grid):
    n = len(grid)
    if grid[0][0] != 0 or grid[n-1][n-1] != 0:
        return -1

    # Directions: 8 directions (vertical, horizontal, diagonal)
    directions = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1)]
```

```
# BFS initialization
queue = deque([(0, 0, 1)]) # (row, column, path_length)
grid[0][0] = 1 # mark as visited

while queue:
    row, col, path_length = queue.popleft()
    if row == n - 1 and col == n - 1:
        return path_length

    # Explore all neighbors

    for dr, dc in directions:
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < n and 0 <= new_col < n and grid[new_row][new_col]
] == 0:
            queue.append((new_row, new_col, path_length + 1))
            grid[new_row][new_col] = 1 # mark as visited

    return -1

# Example usage:
# grid = [[0,1],[1,0]]

# print(shortestPathBinaryMatrix(grid))
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to guarantee the shortest path in an unweighted grid.
- Movement is allowed in 8 directions, so each cell has up to 8 neighbors.
- Check boundary conditions and ensure only unvisited cells with value 0 are processed.

- Mark cells as visited to avoid reprocessing and infinite loops.
- If the start or finish is blocked ($==1$), the answer is immediately -1 .

Space & Time

Time Complexity: $O(n^2)$, as in the worst-case scenario every cell in the grid is visited once.

Space Complexity: $O(n^2)$, due to the queue used for BFS and storage for visited cells.

Solution

We approach this problem using BFS, which is ideal for finding the shortest path in a grid when each move has the same cost. Initialize the BFS from the top-left cell if it is 0. For every cell dequeued, explore its 8 neighboring cells. If a neighboring cell is within grid bounds, has a value of 0, and hasn't been visited, mark it as visited and enqueue it with an incremented path length. Continue this process until the bottom-right cell is reached. If the queue is exhausted without reaching the destination, return -1 .

Breadth First Search (BFS)

□ #1102 Path With Maximum Minimum Value

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Depth-First Search ·
Breadth-First Search · Union-Find · Heap
(Priority Queue) · Matrix

Lists: AlgoMaster 600

Problem

Given an $m \times n$ integer matrix grid, return the maximum score of a path starting at $(0, 0)$ and ending at $(m - 1, n - 1)$ moving in the 4 cardinal directions.

The score of a path is the minimum value in that path.

- For example, the score of the path $8 \rightarrow 4 \rightarrow 5 \rightarrow 9$ is 4.

Example 1:

5	4	5
1	2	6
7	4	6

Input: grid = [[5,4,5],[1,2,6],[7,4,6]]

Output: 4

Explanation: The path with the maximum score is highlighted in yellow.

Example 2:

2	2	1	2	2	2
1	2	2	2	1	2

Input: grid = [[2,2,1,2,2,2],[1,2,2,2,1,2]]
 Output: 2

Example 3:

3	4	6	3	4
0	2	1	1	7
8	8	3	2	7
3	2	4	9	8
4	1	2	0	0
4	6	5	4	3

Input: grid =
 [[3,4,6,3,4],[0,2,1,1,7],[8,8,3,2,7],[3,2,4,9,8],[4,1,2,0,0],[4,6,5,4,3]]
 Output: 3

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 100$

- $0 \leq \text{grid}[i][j] \leq 109$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def maximumMinimumPath(self, grid: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        m, n = len(grid), len(grid[0])
        p = list(range(m * n))
        q = [(v, i, j) for i, row in enumerate(grid) for j, v in enumerate(row)]

        q.sort()
        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        vis = set()
        while find(0) != find(m * n - 1):
            v, i, j = q.pop()
            ans = v
            vis.add((i, j))
            for a, b in pairwise(dirs):
                x, y = i + a, j + b

                if (x, y) in vis:
                    p[find(i * n + j)] = find(x * n + y)
        return ans
```

```
class UnionFind:
    __slots__ = ("p", "size")

    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])

        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True

class Solution:
    def maximumMinimumPath(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        uf = UnionFind(m * n)
        q = [(v, i, j) for i, row in enumerate(grid) for j, v in enumerate(row
)]
```

```

q.sort()
ans = 0
vis = set()
dirs = (-1, 0, 1, 0, -1)
while uf.find(0) != uf.find(m * n - 1):
    v, i, j = q.pop()
    ans = v
    vis.add((i, j))
    for a, b in pairwise(dirs):
        x, y = i + a, j + b

        if (x, y) in vis:
            uf.union(x * n + y, i * n + j)
return ans

```

• SimplyLeet

```

import heapq

def maximumMinimumPath(grid):
    m, n = len(grid), len(grid[0])
    # Max-heap: use negative values because Python's heapq is a min-heap.
    heap = [(-grid[0][0], 0, 0)]
    visited = [[False] * n for _ in range(m)]
    visited[0][0] = True

    # Directions: up, down, left, right.

    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
    # Current answer is the start value.
    current_min = grid[0][0]

    while heap:
        # Get the cell with the maximum value (using negative because of
        # min-heap).
        val, i, j = heapq.heappop(heap)
        # Reverse the sign to get the actual value.
        val = -val
        # Update the current path min.

```

```

current_min = min(current_min, val)
# If destination is reached, return current path's min value.
if i == m - 1 and j == n - 1:
    return current_min

for di, dj in directions:
    ni, nj = i + di, j + dj
    if 0 <= ni < m and 0 <= nj < n and not visited[ni][nj]:
        visited[ni][nj] = True
        # Push neighbor cell into heap.

        heapq.heappush(heap, (-grid[ni][nj], ni, nj))

return -1 # Should never reach here if input grid is valid.

# Example test
print(maximumMinimumPath([[5,4,5],[1,2,6],[7,4,6]]))

```

◆ Quick Review

Key Insights

- The score of a path is determined by its lowest value. Therefore, the objective is to maximize this minimum value over all possible paths.
- A greedy strategy that always considers the next step with the highest value works well.
- Using a max-heap (priority queue) allows for exploring high-value paths first.
- Once the destination is reached, the minimum value along the chosen path is the answer.

- Alternative methods include binary search over the possible answer space with a connectivity check (using DFS/BFS) or Union-Find to simulate connectivity with a threshold.

Space & Time

Time Complexity: $O(m * n * \log(m * n))$ because each cell might be pushed into the heap once and each heap operation costs $\log(m * n)$.

Space Complexity: $O(m * n)$ for the heap and visited cells tracking.

Solution

The solution uses a greedy algorithm implemented with a max-heap:

- Start at cell (0, 0) and initialize the score as the value at the starting cell.
- Use a max-heap to store cells with their corresponding values in descending order.
- Repeatedly pop the cell with the current highest value. Update the score as the minimum value encountered so far along the path.
- If the destination cell is reached, return the current score.

– Otherwise, push all valid neighboring cells that have not been visited into the heap. This method ensures that paths that are more promising (i.e., with higher current minimum values) are explored first, thus guaranteeing the maximum possible score upon reaching the destination.

Linked List

Round 2 · High · Medium · 6 questions

Linked List

□ #82 Remove Duplicates from Sorted List II

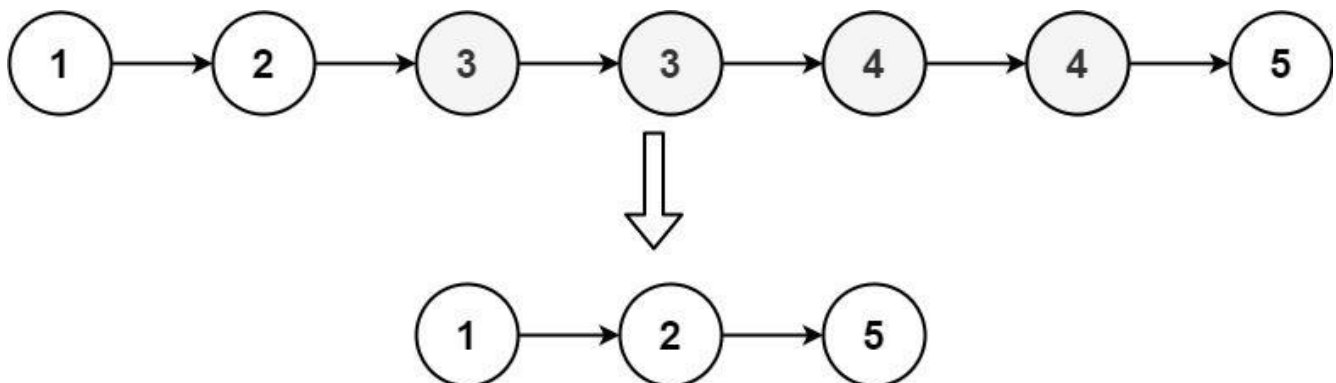
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Two Pointers
Lists: AlgoMaster 600

Problem

Given the head of a sorted linked list, delete all nodes that have duplicate numbers, leaving only distinct numbers from the original list. Return the linked list sorted as well.

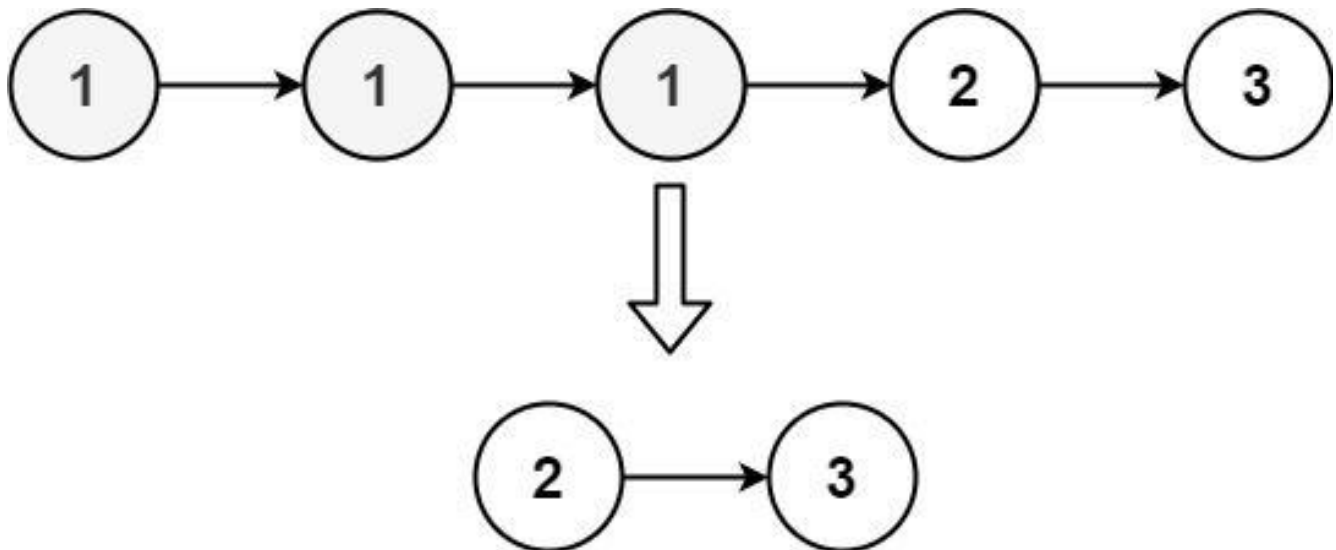
Example 1:



Input: head = [1,2,3,3,4,4,5]

Output: [1,2,5]

Example 2:



Input: head = [1,1,1,2,3]
Output: [2,3]

Constraints:

- The number of nodes in the list is in the range [0, 300].
- $-100 \leq \text{Node.val} \leq 100$
- The list is guaranteed to be sorted in ascending order.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        dummy = ListNode(0, head)
        prev = dummy

        while head:
            while head.next and head.val == head.next.val:
                head = head.next
            if prev.next == head:
                prev = prev.next
```

```

    else:
        prev.next = head.next
        head = head.next

    return dummy.next

```

• LeetDoocs

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteDuplicates(self, head: Optional[ListNode]) -> Optional[ListNode]
    :
        dummy = pre = ListNode(next=head)
        cur = head
        while cur:

            while cur.next and cur.next.val == cur.val:
                cur = cur.next
            if pre.next == cur:
                pre = cur
            else:
                pre.next = cur.next
            cur = cur.next
        return dummy.next

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        # Create a dummy node to handle edge cases.
        dummy = ListNode(0)

```

```
dummy.next = head

# Initialize pointers: prev for the last confirmed non-duplicate node,
current for traversal.
prev = dummy
current = head

while current:
    # If the current node is the start of duplicates.
    if current.next and current.val == current.next.val:
        duplicate_val = current.val

        # Skip all nodes with this duplicate value.
        while current and current.val == duplicate_val:
            current = current.next
        # Link the last non-duplicate node to the next distinct node.
        prev.next = current
    else:
        # No duplicate: move prev pointer to current.
        prev = current
        current = current.next

# Return the modified linked list.
return dummy.next
```

◆ Quick Review

Key Insights

- The list is sorted, so duplicate nodes appear consecutively.
- Use a dummy node to simplify edge cases (e.g., when the first node is a duplicate).
- Two pointers (prev and current) can be used to detect and skip over duplicate nodes.

- If a node has duplicates, skip all nodes with that value and connect the previous valid node to the next distinct node.

Space & Time

Time Complexity: $O(n)$ – We traverse the list once.

Space Complexity: $O(1)$ – Only a few pointers are used, regardless of the input size.

Solution

We iterate through the linked list with a pointer called **current** while maintaining a previous pointer (**prev**) initialized to a dummy node. Whenever we detect that the current node has a duplicate (i.e., its next node has the same value), we record that value and advance **current** until all nodes with that duplicate value are skipped. Then, we link the previous node to the current node (which is now the next distinct node). If no duplicate is found, we simply move both pointers forward. This approach efficiently removes all duplicates in one pass with constant extra space.

Linked List

□ #86 Partition List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Two Pointers

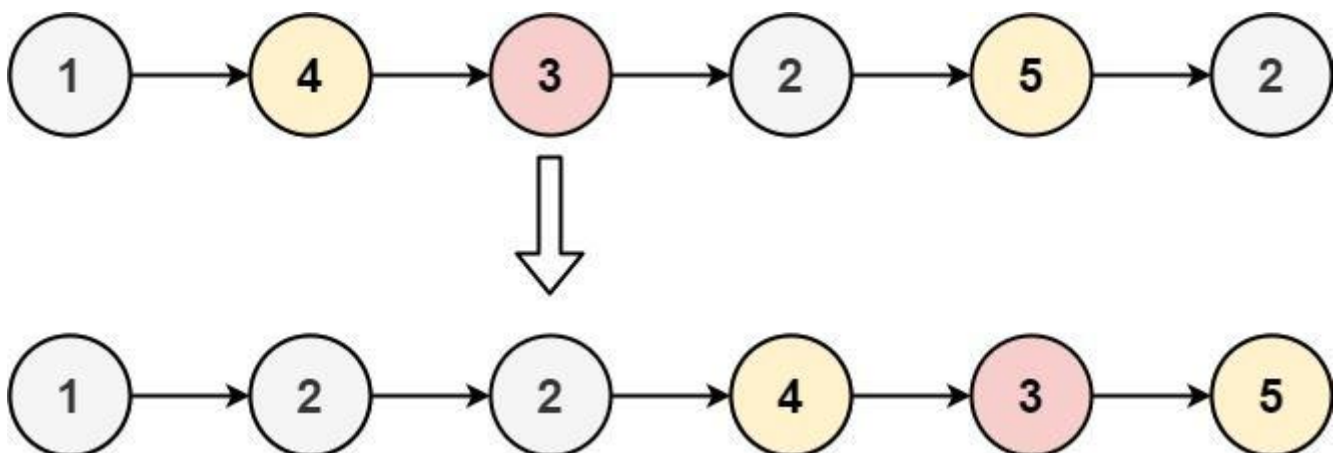
Lists: AlgoMaster 600

Problem

Given the head of a linked list and a value x , partition it such that all nodes less than x come before nodes greater than or equal to x .

You should preserve the original relative order of the nodes in each of the two partitions.

Example 1:



Input: head = [1,4,3,2,5,2], x = 3
Output: [1,2,2,4,3,5]

Example 2:

Input: head = [2,1], x = 2

Output: [1,2]

Constraints:

- The number of nodes in the list is in the range [0, 200].
- $-100 \leq \text{Node.val} \leq 100$
- $-200 \leq x \leq 200$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def partition(self, head: ListNode, x: int) -> ListNode:
        beforeHead = ListNode(0)
        afterHead = ListNode(0)
        before = beforeHead
        after = afterHead

        while head:
            if head.val < x:
                before.next = head
                before = head
            else:
                after.next = head
                after = head
            head = head.next

        after.next = None
        before.next = afterHead.next

        return beforeHead.next
```

• LeetDoocs


```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def partition(self, head: Optional[ListNode], x: int) -> Optional[ListNode]:
        l = ListNode()
        r = ListNode()
        tl, tr = l, r

        while head:
            if head.val < x:
                tl.next = head
                tl = tl.next
            else:
                tr.next = head
                tr = tr.next
            head = head.next
        tr.next = None
        tl.next = r.next

        return l.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def partition(self, head: ListNode, x: int) -> ListNode:
        # Initialize two dummy nodes for less and greater lists
        less_dummy = ListNode(0)
```

```
greater_dummy = ListNode(0)
less = less_dummy # pointer for less-than list
greater = greater_dummy # pointer for greater or equal list

# Traverse through original list and partition nodes
while head:
    if head.val < x:
        less.next = head # add node to less list
        less = less.next # move pointer forward
    else:
        greater.next = head # add node to greater list
        greater = greater.next # move pointer forward
    head = head.next # advance to the next node in the list

# Terminate the greater list to avoid cycles
greater.next = None
# Link the less list with the greater list
less.next = greater_dummy.next

# Return the start of the new list

return less_dummy.next
```

◆ Quick Review

Key Insights

- Use two separate lists: one for nodes less than x and one for nodes greater than or equal to x.
- Create dummy nodes for each list to simplify edge case handling.
- Traverse the original list once, partitioning nodes into the two lists.

- **Connect the two lists at the end to obtain the final partitioned list.**

Space & Time

Time Complexity: $O(n)$ because we traverse the list once.

Space Complexity: $O(1)$ since we only use a constant number of pointers.

Solution

We maintain two dummy head nodes to start the lists for nodes less than x and nodes greater than or equal to x . As we traverse the original list, we append each node to the appropriate list. After the traversal, we link the end of the less-than list to the head of the greater-than or equal list and set the tail of the greater list to None (or null) to avoid forming a cycle. Finally, we return the new head from the less-than list.

Linked List

□ #237 Delete Node in a Linked List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List

Lists: AlgoMaster 600

Problem

There is a singly-linked list head and we want to delete a node node in it.

You are given the node to be deleted node. You will not be given access to the first node of head.

All the values of the linked list are unique, and it is guaranteed that the given node node is not the last node in the linked list.

Delete the given node. Note that by deleting the node, we do not mean removing it from memory. We mean:

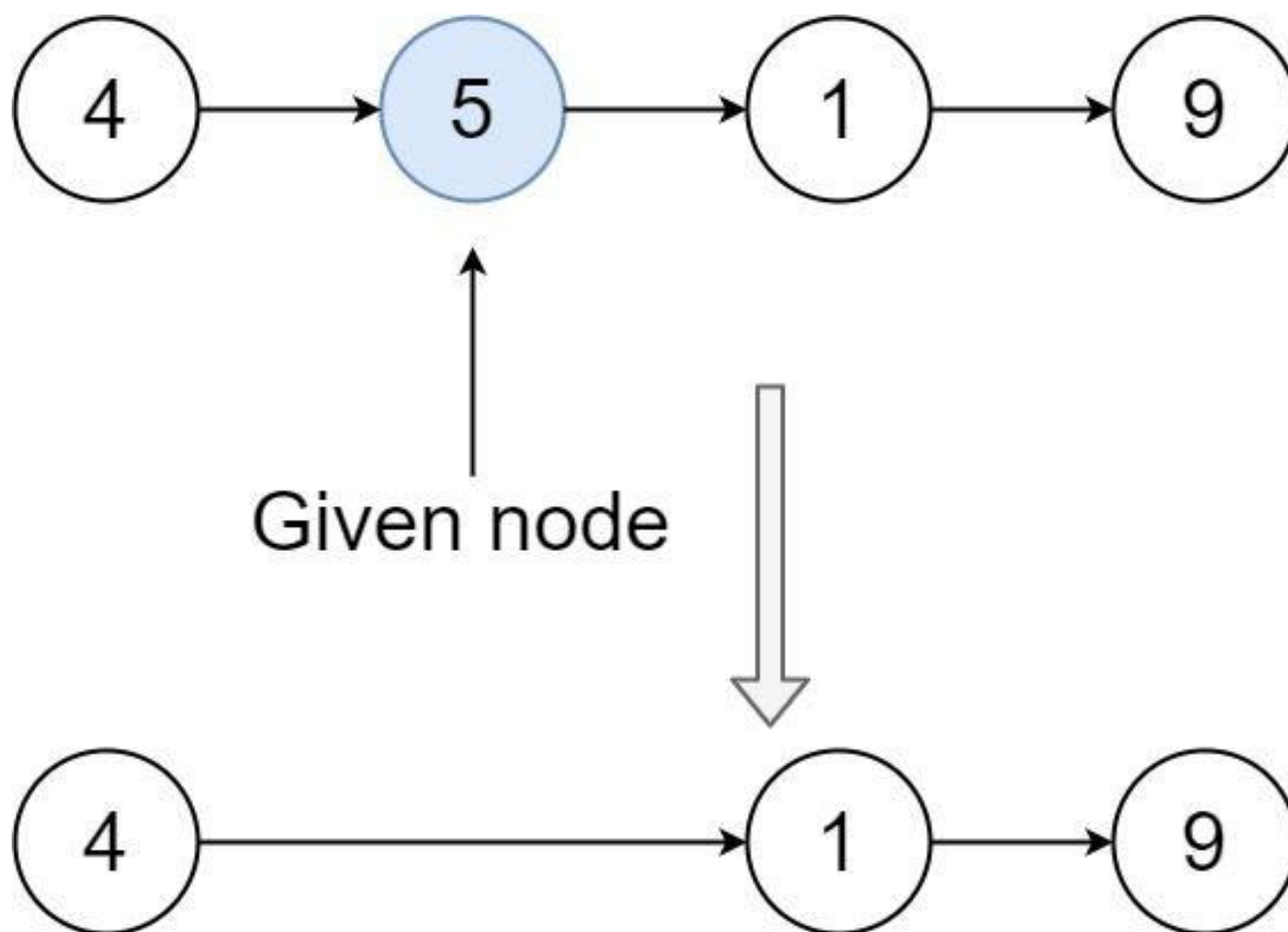
- The value of the given node should not exist in the linked list.
- The number of nodes in the linked list should decrease by one.

- All the values before node should be in the same order.
- All the values after node should be in the same order.

Custom testing:

- For the input, you should provide the entire linked list head and the node to be given node. node should not be the last node of the list and should be an actual node in the list.
- We will build the linked list and pass the node to your function.
- The output will be the entire list after calling your function.

Example 1:

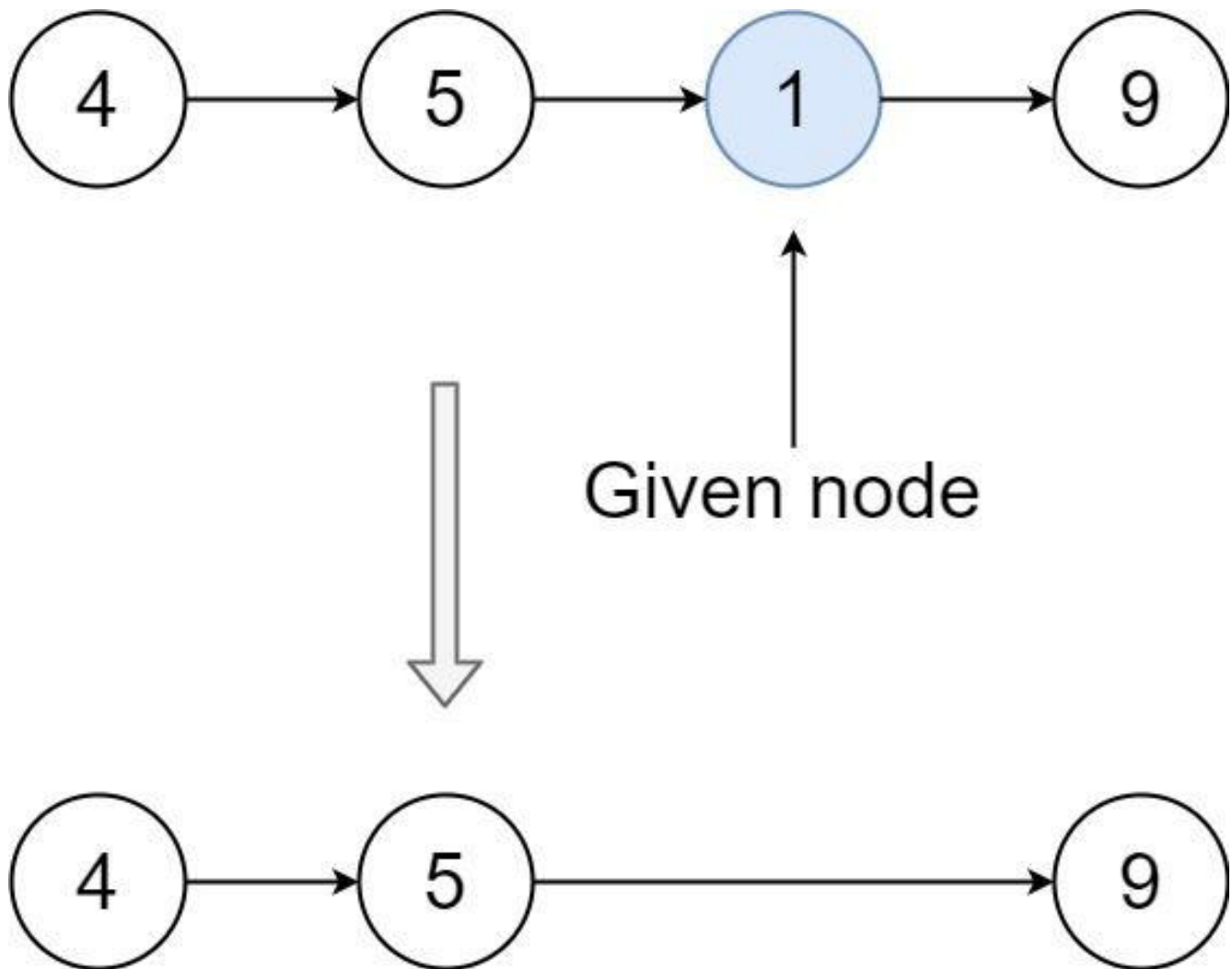


Input: head = [4,5,1,9], node = 5

Output: [4,1,9]

Explanation: You are given the second node with value 5, the linked list should become 4 -> 1 -> 9 after calling your function.

Example 2:



Input: head = [4,5,1,9], node = 1

Output: [4,5,9]

Explanation: You are given the third node with value 1, the linked list should become 4 -> 5 -> 9 after calling your function.

Constraints:

- The number of the nodes in the given list is in the range [2, 1000].
- $-1000 \leq \text{Node.val} \leq 1000$
- The value of each node in the list is unique.
- The node to be deleted is in the list and is not a tail node.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def deleteNode(self, node):
        node.val = node.next.val
        node.next = node.next.next
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None
```

```
class Solution:
    def deleteNode(self, node):
        """
```

```
:type node: ListNode
:rtype: void Do not return anything, modify node in-place instead.
        """
```

```
node.val = node.next.val
node.next = node.next.next
```

• SimplyLeet

```
# Function to delete a node in a linked list.
def deleteNode(node):
    # Copy the next node's value into the current node.
    node.val = node.next.val
    # Bypass the next node by pointing to the next node's next.
    node.next = node.next.next
```

◆ Quick Review

Key Insights

- You are given direct access to the node to be deleted, not the head.
- The node to be deleted is guaranteed not to be the tail.
- The solution involves copying the next node's value into the current node.
- Then, bypass the next node by updating the pointer of the current node to point to the next node's next pointer.

Space & Time

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Solution

The key idea is to simulate deletion by copying the next node's data into the current node and then deleting the next node. Since you don't have access to the node before the one to be deleted, you can't adjust its pointer. Instead, you overwrite the target node's value with the next node's value and update the pointer to skip the next node. This in-place modification effectively removes the original node value from the list without requiring traversal from

the head.

Linked List

□ #445 Add Two Numbers II

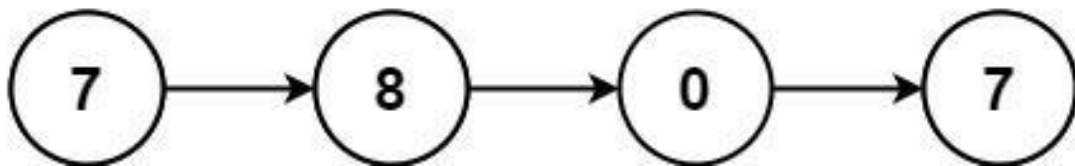
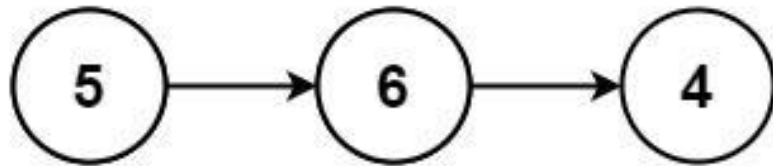
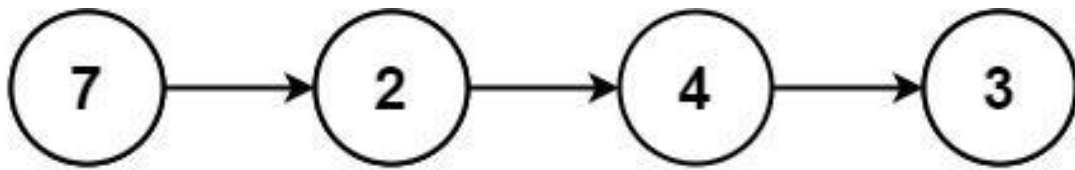
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Math · Stack
Lists: AlgoMaster 600

Problem

You are given two non-empty linked lists representing two non-negative integers. The most significant digit comes first and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list. You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:



Input: l1 = [7,2,4,3], l2 = [5,6,4]
Output: [7,8,0,7]

Example 2:

Input: l1 = [2,4,3], l2 = [5,6,4]
Output: [8,0,7]

Example 3:

Input: l1 = [0], l2 = [0]
Output: [0]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$

- It is guaranteed that the list represents a number that does not have leading zeros.

Follow up: Could you solve it without reversing the input lists?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def addTwoNumbers(self, l1: ListNode, l2: ListNode) -> ListNode:
        stack1 = []
        stack2 = []

        while l1:
            stack1.append(l1.val)
            l1 = l1.next

        while l2:
            stack2.append(l2.val)
            l2 = l2.next

        head = None
        carry = 0

        while carry or stack1 or stack2:
            if stack1:
                carry += stack1.pop().val
            if stack2:
                carry += stack2.pop().val
            node = ListNode(carry % 10)
            node.next = head
            head = node
            carry //= 10

        return head
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def addTwoNumbers(
        self, l1: Optional[ListNode], l2: Optional[ListNode]
    ) -> Optional[ListNode]:
        s1, s2 = [], []

        while l1:
            s1.append(l1.val)
            l1 = l1.next
        while l2:
            s2.append(l2.val)
            l2 = l2.next
        dummy = ListNode()
        carry = 0
        while s1 or s2 or carry:
            s = (0 if not s1 else s1.pop()) + (0 if not s2 else s2.pop()) + ca
            carry, val = divmod(s, 10)
            # node = ListNode(val, dummy.next)
            # dummy.next = node
            dummy.next = ListNode(val, dummy.next)
        return dummy.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def addTwoNumbers(l1: ListNode, l2: ListNode) -> ListNode:
    # Initialize two stacks for the two numbers.
    stack1 = []
    stack2 = []
```

```
# Push all nodes' values of l1 onto stack1.
while l1:
    stack1.append(l1.val)
    l1 = l1.next

# Push all nodes' values of l2 onto stack2.
while l2:
    stack2.append(l2.val)
    l2 = l2.next

carry = 0
result = None # This will be the head of the resulting linked list.

# Process stacks until both are empty.
while stack1 or stack2 or carry:
    # Get the top values from stacks if available.
    val1 = stack1.pop() if stack1 else 0
    val2 = stack2.pop() if stack2 else 0

    # Calculate the total sum of values and carry.
    total = val1 + val2 + carry
    carry = total // 10
    newNode = ListNode(total % 10)

    # Prepend the new node to the result list.
    newNode.next = result
    result = newNode

return result

# Example usage:
# l1 = ListNode(7, ListNode(2, ListNode(4, ListNode(3))))
# l2 = ListNode(5, ListNode(6, ListNode(4)))
# head = addTwoNumbers(l1, l2)
```

◆ Quick Review

Key Insights

- Use stacks to simulate processing the digits from least significant (end of the list) to most significant (start of the list).
- By pushing each list's values onto a stack, you can pop them off in reverse order and perform addition with a carry.
- Create new nodes for the resulting linked list as you compute the sum, linking them appropriately in reverse order.
- Careful handling of the carry value is critical for correct summation.

Space & Time

Time Complexity: $O(n + m)$ where n and m are the lengths of the two linked lists.

Space Complexity: $O(n + m)$ due to the use of stacks to hold the nodes' values and the space needed for the result list.

Solution

We begin by pushing the values of each linked list onto two separate stacks, which allows us to retrieve the digits in reverse order for addition. Then we pop elements from both stacks, add them along with any carry from the

previous addition, and create a new node for each digit of the resultant sum. Finally, the new nodes are prepended to the resultant list. This approach avoids modifying the input lists (i.e., no reversal is performed on the original linked lists) and handles different list lengths and carry propagation gracefully.

Linked List

□ #1669 Merge In Between Linked Lists

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List

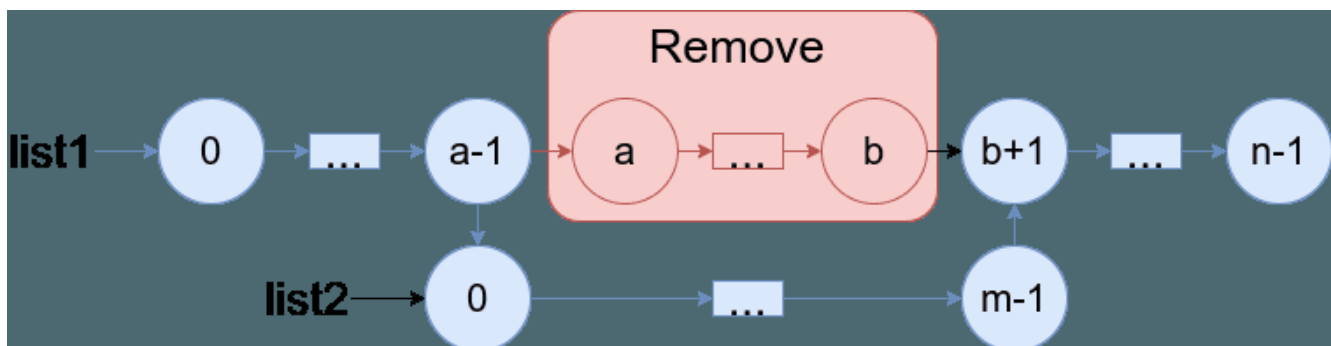
Lists: AlgoMaster 600

Problem

You are given two linked lists: list1 and list2 of sizes n and m respectively.

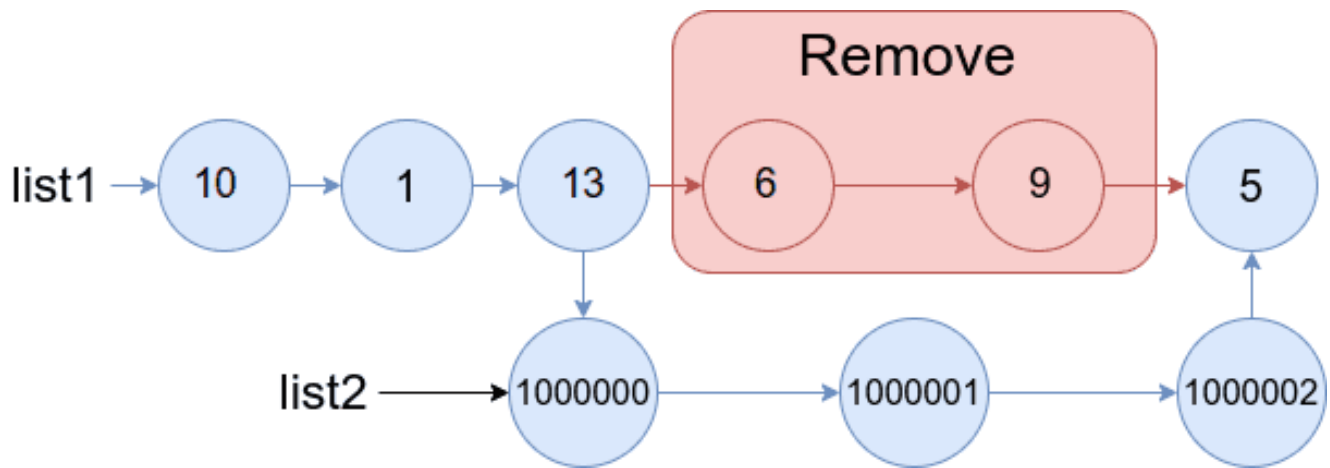
Remove list1's nodes from the a th node to the b th node, and put list2 in their place.

The blue edges and nodes in the following figure indicate the result:



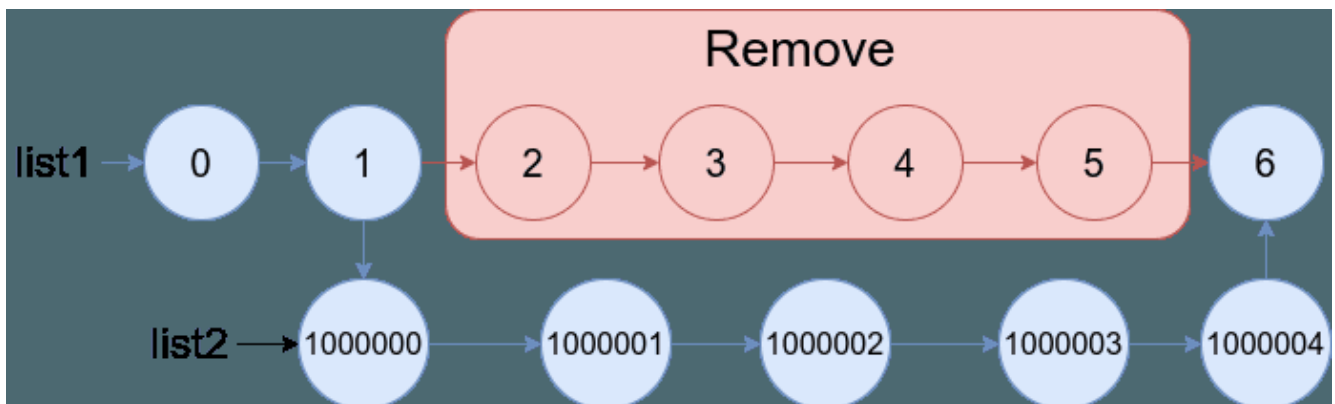
Build the result list and return its head.

Example 1:



Input: list1 = [10,1,13,6,9,5], a = 3, b = 4, list2 = [1000000,1000001,1000002]
 Output: [10,1,13,1000000,1000001,1000002,5]
 Explanation: We remove the nodes 3 and 4 and put the entire list2 in their place. The blue edges and nodes in the above figure indicate the result.

Example 2:



Input: list1 = [0,1,2,3,4,5,6], a = 2, b = 5, list2 = [1000000,1000001,1000002,1000003,1000004]
 Output: [0,1,1000000,1000001,1000002,1000003,1000004,6]
 Explanation: The blue edges and nodes in the above figure indicate the result.

Constraints:

- $3 \leq \text{list1.length} \leq 104$
- $1 \leq a \leq b < \text{list1.length} - 1$
- $1 \leq \text{list2.length} \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mergeInBetween(
        self,
        list1: ListNode,
        a: int,
        b: int,
        list2: ListNode,
    ) -> ListNode:
        nodeBeforeA = list1
        for i in range(a - 1):

            nodeBeforeA = nodeBeforeA.next

        nodeB = nodeBeforeA.next
        for i in range(b - a):
            nodeB = nodeB.next

        nodeBeforeA.next = list2
        lastNodeInList2 = list2

        while lastNodeInList2.next:

            lastNodeInList2 = lastNodeInList2.next

        lastNodeInList2.next = nodeB.next
        nodeB.next = None
        return list1
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeInBetween(
        self, list1: ListNode, a: int, b: int, list2: ListNode
    ) -> ListNode:
        p = q = list1

        for _ in range(a - 1):
            p = p.next
        for _ in range(b):
            q = q.next
        p.next = list2
        while p.next:
            p = p.next
        p.next = q.next
        q.next = None
        return list1
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def mergeInBetween(list1: ListNode, a: int, b: int, list2: ListNode) -> ListNo
de:
    # Step 1: Traverse list1 to reach node just before index a (prevA)
    current = list1
    for i in range(a - 1):
```

```
        current = current.next
    prevA = current

    # Step 2: Continue traversing to reach node after index b (postB)
    current = current.next
    for i in range(b - a + 1):
        current = current.next
    postB = current

    # Step 3: Find the tail of list2

    tail2 = list2
    while tail2.next:
        tail2 = tail2.next

    # Step 4: Connect list2 between list1 parts
    prevA.next = list2          # Connect node before index a to head of list2
    tail2.next = postB         # Connect tail of list2 to node after index b

    return list1               # Return the modified list1
```

◆ Quick Review

Key Insights

- Traverse list1 to find the node right before index a.
- Continue traversal to locate the node immediately after index b.
- Find the tail of list2.
- Reconnect these parts: connect the node before a to the head of list2 and connect list2's tail to the node after b.

Space & Time

Time Complexity: $O(n_1 + n_2)$ where n_1 is the length of list1, and n_2 is the length of list2.

Space Complexity: $O(1)$ assuming we perform pointer manipulation in-place.

Solution

We solve the problem by performing the following steps:

- Traverse list1 to obtain the node just before the ath node (let's call it prevA).
- Continue traversing list1 from prevA to get to the node right after the bth node (postB).
- Traverse list2 to locate its tail.
- Link prevA.next to list2's head.
- Link list2's tail to postB. This reconnects list1 with list2 in-between, effectively removing the nodes from a to b.

Linked List

□ #2095 Delete the Middle Node of a Linked List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Two Pointers
Lists: AlgoMaster 600

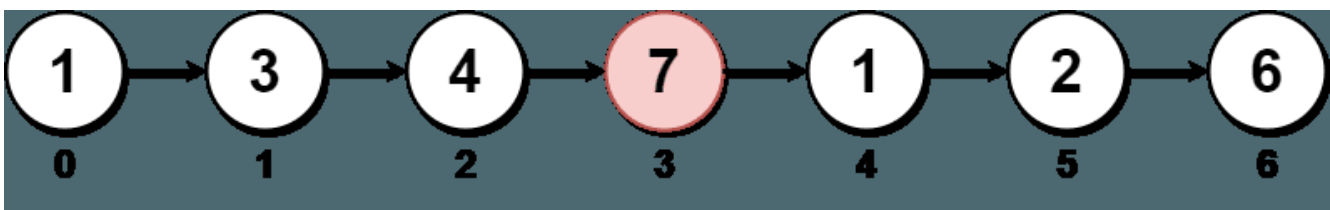
Problem

You are given the head of a linked list. Delete the middle node, and return the head of the modified linked list.

The middle node of a linked list of size n is the $n / 2$ th node from the start using 0-based indexing, where x denotes the largest integer less than or equal to x .

- For $n = 1, 2, 3, 4$, and 5 , the middle nodes are $0, 1, 1, 2$, and 2 , respectively.

Example 1:



Input: head = [1,3,4,7,1,2,6]

Output: [1,3,4,1,2,6]

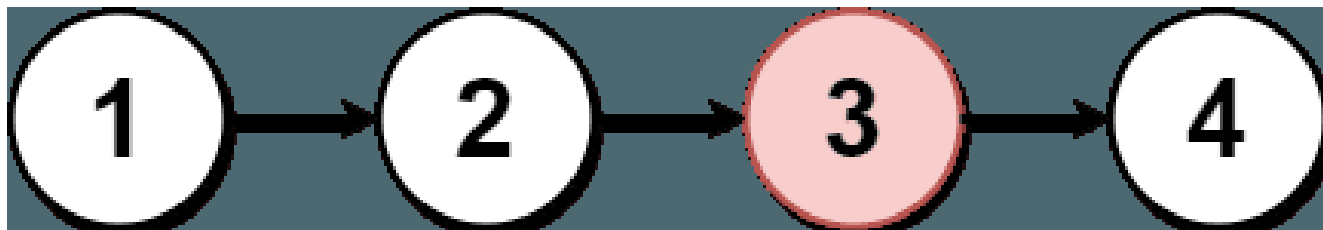
Explanation:

The above figure represents the given linked list. The indices of the nodes are written below.

Since $n = 7$, node 3 with value 7 is the middle node, which is marked in red.

We return the new list after removing this node.

Example 2:



Input: head = [1,2,3,4]

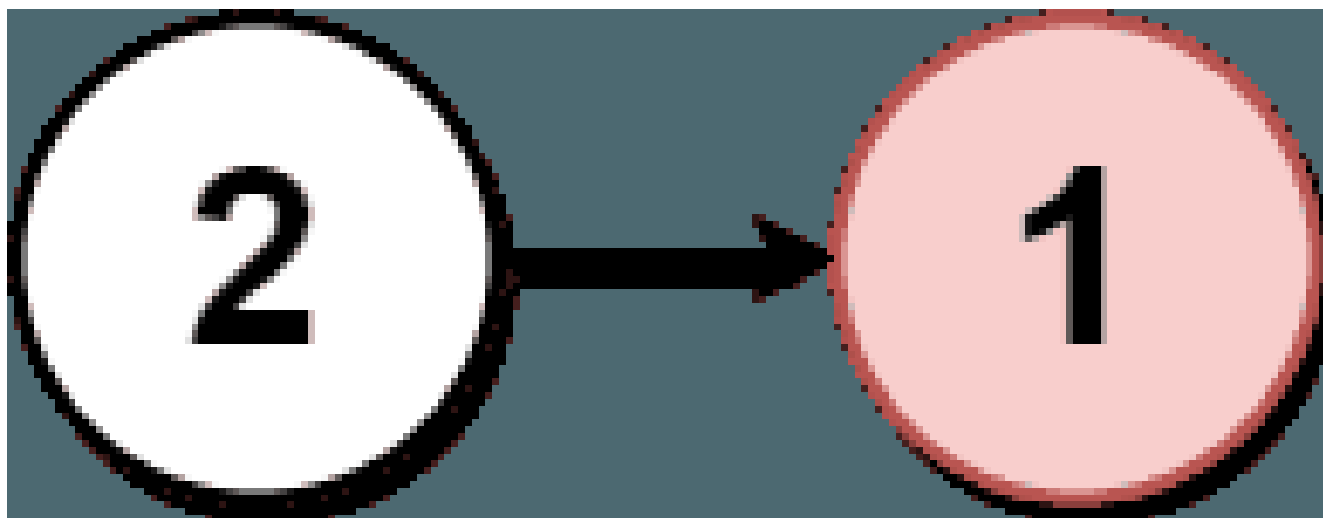
Output: [1,2,4]

Explanation:

The above figure represents the given linked list.

For $n = 4$, node 2 with value 3 is the middle node, which is marked in red.

Example 3:



Input: head = [2,1]

Output: [2]

Explanation:

The above figure represents the given linked list.

For $n = 2$, node 1 with value 1 is the middle node, which is marked in red.

Node 0 with value 2 is the only node remaining after removing node 1.

Constraints:

- The number of nodes in the list is in the range [1, 105].
- $1 \leq \text{Node.val} \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def deleteMiddle(self, head: ListNode | None) -> ListNode | None:
        dummy = ListNode(0, head)
        slow = dummy
        fast = dummy

        while fast.next and fast.next.next:
            slow = slow.next
            fast = fast.next.next

        # Delete the middle node.
        slow.next = slow.next.next
        return dummy.next
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteMiddle(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        slow, fast = dummy, head
        while fast and fast.next:
```

```

        slow = slow.next
        fast = fast.next.next
    slow.next = slow.next.next
    return dummy.next

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def deleteMiddle(self, head: ListNode) -> ListNode:
        # If the list is empty or contains only one node, return None
        if not head or not head.next:
            return None

        # Initialize two pointers for the two-pointer approach
        slow = head           # slow pointer starts at head
        fast = head           # fast pointer starts at head
        prev = None           # previous pointer to track node before slow

        # Traverse the list until fast reaches the end
        while fast and fast.next:
            prev = slow        # keep track of the node before slow
            slow = slow.next    # move slow by one step
            fast = fast.next.next # move fast by two steps

        # Now, slow is at the middle node.
        # Remove the middle node by linking the previous node to slow.next.
        prev.next = slow.next

        return head

```

◆ Quick Review

Key Insights

- Use the two pointers technique (slow and fast pointers) to find the middle node in a single pass.
- Track the node previous to the slow pointer node to efficiently remove the middle node by changing the 'next' pointer.
- Handle the edge case when the list contains only one node, in which case the result is an empty list.

Space & Time

Time Complexity: $O(n)$ because we traverse the list once.

Space Complexity: $O(1)$ as only a constant amount of extra space is used.

Solution

We use the two pointers technique where the slow pointer moves one step at a time while the fast pointer moves two steps. When the fast pointer reaches the end, the slow pointer will be at the middle node. We also maintain a pointer to the previous node of the slow pointer so we can remove the middle node by linking the previous node directly to the next node of the slow pointer. If the list only has

one node, we return null since the single node is the middle node to be removed.

Round 3 · High · Hard

Round 3

High Priority · Hard

11 questions · 7 patterns

- ☐ ● **Strings #843** ↗
- ☐ ● **Two Pointers #2444** ↗
- ☐ ● **Sliding Window – Fixed Size #30** ↗
- ☐ ● **Sliding Window – Dynamic Size #727 #992** ↗
- ☐ ● **Binary Search #719 #1095** ↗
- ☐ ● **Depth First Search (DFS) #827 #2246** ↗
- ☐ ● **Breadth First Search (BFS) #1293 #2290** ↗

Strings

Round 3 · High · Hard · 1 question

Strings

□ #843 Guess the Word

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · String · Interactive · Game
Theory

Lists: AlgoMaster 600

Problem

You are given an array of unique strings `words` where `words[i]` is six letters long. One word of `words` was chosen as a secret word.

You are also given the helper object `Master`. You may call `Master.guess(word)` where `word` is a six-letter-long string, and it must be from `words`. `Master.guess(word)` returns:

- `-1` if `word` is not from `words`, or
- an integer representing the number of exact matches (value and position) of your guess to the secret word.

There is a parameter `allowedGuesses` for each test case where `allowedGuesses` is the maximum number of times you can call

Master.guess(word).

For each test case, you should call **Master.guess** with the secret word without exceeding the maximum number of allowed guesses. You will get:

- "Either you took too many guesses, or you did not find the secret word." if you called **Master.guess** more than **allowedGuesses** times or if you did not call **Master.guess** with the secret word, or
- "You guessed the secret word correctly." if you called **Master.guess** with the secret word with the number of calls to **Master.guess** less than or equal to **allowedGuesses**.

The test cases are generated such that you can guess the secret word with a reasonable strategy (other than using the brute force method).

Example 1:

```
Input: secret = "acckzz", words = ["acckzz","ccbazz","eiowzz","abcczz"],
allowedGuesses = 10
Output: You guessed the secret word correctly.
Explanation:
master.guess("aaaaaa") returns -1, because "aaaaaa" is not in words.
master.guess("acckzz") returns 6, because "acckzz" is secret and has all 6
matches.
master.guess("ccbazz") returns 3, because "ccbazz" has 3 matches.
master.guess("eiowzz") returns 2, because "eiowzz" has 2 matches.
master.guess("abcczz") returns 4, because "abcczz" has 4 matches.
```

Example 2:

Input: secret = "hamada", words = ["hamada","khaled"], allowedGuesses = 10
 Output: You guessed the secret word correctly.
 Explanation: Since there are two words, you can guess both.

Constraints:

- $1 \leq \text{words.length} \leq 100$
- $\text{words}[i].\text{length} == 6$
- $\text{words}[i]$ consist of lowercase English letters.
- All the strings of words are unique.
- secret exists in words.
- $10 \leq \text{allowedGuesses} \leq 30$

Community Solutions (Python)

• WalkCC

```
# """
# This is Master's API interface.
# You should not implement it, or speculate about its implementation
# """
# class Master:
#     def guess(self, word: str) -> int:

class Solution:
    def findSecretWord(self, words: list[str], master: 'Master') -> None:
        for _ in range(10):

            guessedWord = words[random.randint(0, len(words) - 1)]
            matches = master.guess(guessedWord)
            if matches == 6:
                break
            words = [
                word for word in words
                if sum(c1 == c2 for c1, c2 in zip(guessedWord, word)) == matches]
```

• SimplyLeet

```
# Python solution for "Guess the Word" problem
```

```
class Solution:
    def findSecretWord(self, wordlist, master):
        # Helper function to count matching positions between two words
        def countMatches(word1, word2):
            matches = 0
            for c1, c2 in zip(word1, word2):
                if c1 == c2:
                    matches += 1

            return matches

        # Initially, all words are candidates
        candidates = wordlist[:]

        # Iterate until we find the secret word or run out of allowed guesses
        for _ in range(10): # allowed guesses is at most 10-30, using 10 as a
            # safe upper bound
            # pick a candidate word – a simple strategy is to select the first
            # word
            guess_word = candidates[0]

            # Make a guess using Master.guess; it returns the count of matching
            # characters
            matchCount = master.guess(guess_word)

            # If the secret word is found (6 matches for a 6-letter word),
            # return immediately
            if matchCount == 6:
                return

            # Otherwise filter the candidates based on the match count
            new_candidates = []
            for word in candidates:

                if countMatches(guess_word, word) == matchCount:
                    new_candidates.append(word)
            candidates = new_candidates
```

◆ Quick Review

Key Insights

- Use a strategy based on eliminating words from the candidate set based on the number of matching letters returned by the `Master.guess` call.
- Define a helper function to calculate the number of matching character positions between any two words.
- Each guess helps you narrow down the candidate words: you only keep words in the list that would give the same matching count with the guessed word as the `Master.guess` output.
- Although brute-force could work given a small wordlist, a strategy such as minimax or simply narrowing down the candidate set by eliminating non-conforming words is sufficient.
- This problem is interactive in nature; careful management of the guess count and candidate filtering is paramount.

Space & Time

Time Complexity: In the worst-case scenario, each guess requires checking against all candidates with $O(n)$ comparisons and each comparison takes $O(\text{word_length})$, leading to $O(n^2 * \text{word_length})$.

Space Complexity: $O(n)$ for storing the candidate list.

Solution

The solution involves iteratively guessing a word from the current candidate list and then filtering the candidate list based on the number of matching letters reported by `Master.guess`. A helper function computes the number of matching letters between two words. After each guess, the candidate list is reduced to only those words that would have produced the same matching score with the guessed word. This process continues until the secret word is found or the allowed number of guesses is exceeded. The use of candidate elimination is a key trick here, ensuring that each guess reduces the possible search space.

Two Pointers

Round 3 · High · Hard · 1 question

Two Pointers

□ #2444 Count Subarrays With Fixed Bounds

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Queue · Sliding Window · Monotonic
Queue

Lists: AlgoMaster 600

Problem

You are given an integer array `nums` and two integers `minK` and `maxK`.

A fixed-bound subarray of `nums` is a subarray that satisfies the following conditions:

- The minimum value in the subarray is equal to `minK`.
- The maximum value in the subarray is equal to `maxK`.

Return the number of fixed-bound subarrays.

A subarray is a contiguous part of an array.

Example 1:

Input: `nums = [1,3,5,2,7,5]`, `minK = 1`, `maxK = 5`

Output: 2

Explanation: The fixed-bound subarrays are `[1,3,5]` and `[1,3,5,2]`.

Example 2:

Input: nums = [1,1,1,1], minK = 1, maxK = 1

Output: 10

Explanation: Every subarray of nums is a fixed-bound subarray. There are 10 possible subarrays.

Constraints:

- $2 \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i], \text{minK}, \text{maxK} \leq 106$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countSubarrays(self, nums: list[int], minK: int, maxK: int) -> int:
        ans = 0
        j = -1
        prevMinKIndex = -1
        prevMaxKIndex = -1

        for i, num in enumerate(nums):
            if num < minK or num > maxK:
                j = i

            if num == minK:
                prevMinKIndex = i
            if num == maxK:
                prevMaxKIndex = i
            # Any index k in [j + 1, min(prevMinKIndex, prevMaxKIndex)] can be the
            # start of the subarray s.t. nums[k..i] satisfies the conditions.
            ans += max(0, min(prevMinKIndex, prevMaxKIndex) - j)

        return ans
```

• LeetDoocs


```

class Solution:
    def countSubarrays(self, nums: List[int], minK: int, maxK: int) -> int:
        j1 = j2 = k = -1
        ans = 0
        for i, v in enumerate(nums):
            if v < minK or v > maxK:
                k = i
            if v == minK:
                j1 = i
            if v == maxK:

                j2 = i
            ans += max(0, min(j1, j2) - k)
        return ans

```

• SimplyLeet

Python implementation with line-by-line comments

```

def countSubarrays(nums, minK, maxK):
    # Initialize pointers for last indices of minK, maxK, and invalid element
    last_invalid = -1
    last_min = -1
    last_max = -1
    count = 0

    # Loop through all indices in the nums array

    for index, num in enumerate(nums):
        # Check if current number is out of valid range
        if num < minK or num > maxK:
            last_invalid = index # Reset the last invalid position to current
index

        # Update last position where minK is found
        if num == minK:
            last_min = index

        # Update last position where maxK is found

```

```
    if num == maxK:
        last_max = index

    # Calculate valid subarray count ending at the current index
    # Only consider subarray if both minK and maxK have been seen after the
    last invalid index
    valid_start = min(last_min, last_max)
    if valid_start > last_invalid:
        count += valid_start - last_invalid

    return count

# Example usage:
nums = [1, 3, 5, 2, 7, 5]
minK = 1
maxK = 5
print(countSubarrays(nums, minK, maxK)) # Expected output: 2
```

◆ Quick Review

Key Insights

- Use a sliding window to consider only valid segments where every element is between minK and maxK.
- Keep track of the most recent indices where minK and maxK appeared.
- Reset the window when an invalid element (less than minK or greater than maxK) is encountered.
- The number of valid subarrays ending at a position can be determined by the distance

from the most recent invalid index to the minimum of the last occurrences of minK and maxK .

Space & Time

Time Complexity: $O(n)$, where n is the length of the array since we make a single pass.

Space Complexity: $O(1)$, as we use only a constant amount of extra space.

Solution

Use a single traversal of the array while maintaining three pointers:

- **lastInvalid:** index of the most recent element that is outside the $[\text{minK}, \text{maxK}]$ range.
- **lastMin:** index of the most recent occurrence of minK .
- **lastMax:** index of the most recent occurrence of maxK .

For each element:

- If it is invalid (not in $[\text{minK}, \text{maxK}]$), update **lastInvalid**.
- Update **lastMin** or **lastMax** if the element is equal to minK or maxK .

– The number of valid subarrays ending at the current index is $\max(0, \min(\text{lastMin}, \text{lastMax}) - \text{lastInvalid})$.

Accumulate this count over the array.

Sliding Window – Fixed Size

Round 3 · High · Hard · 1 question

Sliding Window – Fixed Size

□ #30 Substring with Concatenation of All Words

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: AlgoMaster 600

Problem

You are given a string *s* and an array of strings *words*. All the strings of *words* are of the same length.

A concatenated string is a string that exactly contains all the strings of any permutation of words concatenated.

- For example, if *words* = ["ab","cd","ef"], then "abcdef", "abefcd", "cdabef", "cdefab", "efabcd", and "efcdab" are all concatenated strings. "acdbef" is not a concatenated string because it is not the concatenation of any permutation of words.

Return an array of the starting indices of all the concatenated substrings in *s*. You can return the answer in any order.

Example 1:

Input: s = "barfoothefoobarman", words = ["foo","bar"]

Output: [0,9]

Explanation:

The substring starting at 0 is "barfoo". It is the concatenation of ["bar","foo"] which is a permutation of words. The substring starting at 9 is "foobar". It is the concatenation of ["foo","bar"] which is a permutation of words.

Example 2:

Input: s = "wordgoodgoodgoodbestword", words = ["word","good","best","word"]

Output: []

Explanation:

There is no concatenated substring.

Example 3:

Input: s = "barfoofoobarthefoobarman", words = ["bar","foo","the"]

Output: [6,9,12]

Explanation:

The substring starting at 6 is "foobarthe". It is the concatenation of ["foo","bar","the"]. The substring starting at 9 is "barthefoo". It is the

concatenation of ["bar","the","foo"]. The substring starting at 12 is "thefoobar". It is the concatenation of ["the","foo","bar"].

Constraints:

- $1 \leq s.length \leq 104$
- $1 \leq words.length \leq 5000$
- $1 \leq words[i].length \leq 30$
- s and $words[i]$ consist of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findSubstring(self, s: str, words: list[str]) -> list[int]:
        if len(s) == 0 or words == []:
            return []

        k = len(words)
        n = len(words[0])
        ans = []
        count = collections.Counter(words)

        for i in range(len(s) - k * n + 1):
            seen = collections.defaultdict(int)
            j = 0
            while j < k:
                word = s[i + j * n: i + j * n + n]
                seen[word] += 1
                if seen[word] > count[word]:
                    break
                j += 1
            if j == k:
                ans.append(i)
```



```
ans.append(i)

return ans
```

• LeetDoocs

```
class Solution:
    def findSubstring(self, s: str, words: List[str]) -> List[int]:
        cnt = Counter(words)
        m, n = len(s), len(words)
        k = len(words[0])
        ans = []
        for i in range(k):
            l = r = i
            cnt1 = Counter()
            while r + k <= m:

                t = s[r : r + k]
                r += k
                if cnt[t] == 0:
                    l = r
                    cnt1.clear()
                    continue
                cnt1[t] += 1
                while cnt1[t] > cnt[t]:
                    rem = s[l : l + k]
                    l += k

                cnt1[rem] -= 1
                if r - l == n * k:
                    ans.append(l)

        return ans
```

• SimplyLeet

```

# Python solution with detailed comments
class Solution:
    def findSubstring(self, s: str, words: [str]) -> [int]:
        if not s or not words:
            return []
        word_length = len(words[0])
        total_words = len(words)
        window_size = word_length * total_words
        if len(s) < window_size:
            return []

        # Build the frequency map of words
        word_freq = {}
        for word in words:
            word_freq[word] = word_freq.get(word, 0) + 1

        results = []
        # Check each offset to account for alignment
        for i in range(word_length):
            left = i

            current_count = {}
            count = 0 # current number of words matched
            # Slide the window in increments of word_length
            for right in range(i, len(s) - word_length + 1, word_length):
                curr_word = s[right:right+word_length]
                if curr_word in word_freq:
                    current_count[curr_word] = current_count.get(curr_word, 0)
                    count += 1
                # Shrink window if frequency is exceeded
                while current_count[curr_word] > word_freq[curr_word]:
                    left = left + word_length
                    curr_word = s[left:left+word_length]
                    if curr_word in word_freq:
                        current_count[curr_word] -= 1
                    else:
                        break
            if count == total_words:
                results.append(left)
                left = left + word_length

```

```
        left_word = s[left:left+word_length]
        current_count[left_word] -= 1
        left += word_length
        count -= 1

        # If window matches all words, record the index
        if count == total_words:
            results.append(left)
    else:
        # Reset the window if the word is not in the words list
        current_count.clear()

    count = 0
    left = right + word_length
    return results
```

◆ Quick Review

Key Insights

- Use the fact that all words are of the same length to slide through *s* in fixed-size segments.
- Create a frequency map for the words in the array to know the required counts.
- Use a sliding window approach with different starting offsets (0 to *wordLength*–1) to maintain alignment.
- Expand the window by checking word-sized chunks and adjust (slide) the window when the frequency of a word exceeds its allowed count.
- When the window contains exactly all words, record the starting index.

Space & Time

Time Complexity: $O(n * m)$ where n is the length of s and m is the number of words (each word check takes constant time due to fixed word length splitting).

Space Complexity: $O(m)$ for the frequency map used for the words.

Solution

The solution uses a sliding window exhibiting multiple starting offsets to ensure each word chunk is properly aligned. For each offset, the window slides in increments of `wordLength`. A hash map tracks the current counts of words found. When a word's count exceeds its allowed frequency (from the input words array), the window is shrunk from the left until the condition is met. If the window size matches total words, the starting index is recorded. This approach efficiently checks each possible concatenation substring.

Sliding Window – Dynamic Size

Round 3 · High · Hard · 2 questions

Sliding Window – Dynamic Size

□ #727 Minimum Window Subsequence

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Sliding
Window

Lists: AlgoMaster 600

Problem

Given strings s_1 and s_2 , return the minimum contiguous substring part of s_1 , so that s_2 is a subsequence of the part.

If there is no such window in s_1 that covers all characters in s_2 , return the empty string `""`. If there are multiple such minimum-length windows, return the one with the left-most starting index.

Example 1:

Input: $s_1 = \text{"abcdebddde"} , s_2 = \text{"bde"}$

Output: `"bcde"`

Explanation:

`"bcde"` is the answer because it occurs before `"bdde"` which has the same length.
`"deb"` is not a smaller window because the elements of s_2 in the window must occur in order.

Example 2:

Input: s1 = "jmeqksfrsdcmsiwvaovztaqenprpvnbstl", s2 = "u"
 Output: ""

Constraints:

- $1 \leq s1.length \leq 2 * 10^4$
- $1 \leq s2.length \leq 100$
- s1 and s2 consist of lowercase English letters.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def minWindow(self, s1: str, s2: str) -> str:
        m, n = len(s1), len(s2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i, a in enumerate(s1, 1):
            for j, b in enumerate(s2, 1):
                if a == b:
                    f[i][j] = i if j == 1 else f[i - 1][j - 1]
                else:
                    f[i][j] = f[i - 1][j]

        p, k = 0, m + 1
        for i, a in enumerate(s1, 1):
            if a == s2[n - 1] and f[i][n]:
                j = f[i][n] - 1
                if i - j < k:
                    k = i - j
                    p = j
        return "" if k > m else s1[p : p + k]
```

• SimplyLeet

```

# Python solution for Minimum Window Subsequence

def minWindow(s1: str, s2: str) -> str:
    s1_len, s2_len = len(s1), len(s2)
    min_len = float('inf')
    start_index = -1

    # Iterate through s1
    for i in range(s1_len):
        # When the current character matches the first character of s2, try to
        # find a valid window

        if s1[i] == s2[0]:
            j = i
            k = 0
            # Move forward as long as characters match s2
            while j < s1_len and k < s2_len:
                if s1[j] == s2[k]:
                    k += 1
                j += 1
            # If entire s2 is matched, perform a backward scan
            if k == s2_len:
                end = j - 1 # end index of the valid window
                k -= 1
                # Move backward to find the start of the window
                while k >= 0:
                    if s1[j-1] == s2[k]:
                        k -= 1
                    j -= 1
                j += 1 # adjust j to the actual start index
                # Update result if the current window is smaller
                if end - j + 1 < min_len:

                    min_len = end - j + 1
                    start_index = j
            return "" if start_index == -1 else s1[start_index:start_index+min_len]

# Example usage:
# result = minWindow("abcdebdde", "bde")
# print(result) # Expected "bcde"

```


◆ Quick Review

Key Insights

- The problem is different from the classic "minimum window substring" because the characters of s_2 must appear in order.
- A two-pointer approach can be used to first "fast-forward" to find a valid window end where the last character of s_2 is matched.
- Once the end is found, a reverse scan is applied to find the beginning of the window by matching s_2 in reverse order.
- Updating the result when a smaller window is found is crucial.
- The approach runs in $O(n * m)$ time, with n and m being the lengths of s_1 and s_2 respectively.

Space & Time

Time Complexity: $O(n * m)$, where n is the length of s_1 and m is the length of s_2 .

Space Complexity: $O(1)$ additional space is used apart from the input strings.

Solution

The solution uses a two-phase approach for each candidate window:

- Use a forward pointer to traverse s1 until we find a valid window ending at an index where the last character of s2 is matched.**
- Once a valid end is detected, use a backward pointer starting at that index and move in reverse to find the start index of the window where s2 fully matches as a subsequence.**

The algorithm repeatedly checks every possible window end where the match is complete and updates the minimal window to return the left-most minimal substring if multiple windows of the same length exist.

Sliding Window – Dynamic Size

□ #992 Subarrays with K Different Integers **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sliding Window · Counting
Lists: AlgoMaster 600

Problem

Given an integer array `nums` and an integer `k`, return the number of good subarrays of `nums`. A good array is an array where the number of different integers in that array is exactly `k`.

- For example, `[1,2,3,1,2]` has 3 different integers: 1, 2, and 3.

A subarray is a contiguous part of an array.

Example 1:

Input: `nums = [1,2,1,2,3]`, `k = 2`

Output: 7

Explanation: Subarrays formed with exactly 2 different integers: `[1,2]`, `[2,1]`, `[1,2]`, `[2,3]`, `[1,2,1]`, `[2,1,2]`, `[1,2,1,2]`

Example 2:

Input: `nums = [1,2,1,3,4]`, `k = 3`

Output: 3

Explanation: Subarrays formed with exactly 3 different integers: `[1,2,1,3]`, `[2,1,3]`, `[1,3,4]`.

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $1 \leq \text{nums}[i], k \leq \text{nums.length}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def subarraysWithKDistinct(self, nums: list[int], k: int) -> int:
        def subarraysWithAtMostKDistinct(k: int) -> int:
            res = 0
            count = collections.Counter()

            l = 0
            for r, num in enumerate(nums):
                count[num] += 1
                if count[num] == 1:
                    k -= 1
                while k < 0:
                    count[nums[l]] -= 1
                    if count[nums[l]] == 0:
                        k += 1
                    l += 1
                res += r - l + 1 # nums[l..r], nums[l + 1..r], ..., nums[r]

            return res

        return subarraysWithAtMostKDistinct(k) - subarraysWithAtMostKDistinct(k - 1)
```

• LeetDoocs

```

class Solution:
    def subarraysWithKDistinct(self, nums: List[int], k: int) -> int:
        def f(k):
            pos = [0] * len(nums)
            cnt = Counter()
            j = 0
            for i, x in enumerate(nums):
                cnt[x] += 1
                while len(cnt) > k:
                    cnt[nums[j]] -= 1
                    if cnt[nums[j]] == 0:
                        cnt.pop(nums[j])
                    j += 1
                pos[i] = j
            return pos

        return sum(a - b for a, b in zip(f(k - 1), f(k)))

```

• SimplyLeet

```

# Python code solution with line-by-line comments
def subarraysWithKDistinct(nums, k):
    # Helper function to count subarrays with at most K distinct integers
    def atMostKDistinct(K):
        count = {} # dictionary to hold frequency of elements
        left = result = 0
        # Iterate over nums using the right pointer
        for right, value in enumerate(nums):
            count[value] = count.get(value, 0) + 1 # update frequency
            # If distinct integers exceed K, move left pointer to adjust window

            while len(count) > K:
                count[nums[left]] -= 1 # reduce frequency for element at left
                if count[nums[left]] == 0:
                    del count[nums[left]] # remove element if frequency is
zero
                left += 1 # shrink window from left
            # All subarrays ending at right with at most K distinct integers
            result += right - left + 1
        return result

    # The number of subarrays with exactly k distinct integers

```

```
return atMostKDistinct(k) - atMostKDistinct(k - 1)
```

```
# Example usage:
```

```
#print(subarraysWithKDistinct([1,2,1,2,3], 2)) # Output: 7
```

◆ Quick Review

Key Insights

- Count subarrays with exactly k distinct integers by leveraging the sliding window technique.
- Use the trick: the number of subarrays with exactly k distinct = subarrays with at most k distinct – subarrays with at most $(k-1)$ distinct.
- Maintain a hash map (or dictionary) to count frequency of array elements as you expand and contract the sliding window.
- Carefully update your window boundaries to ensure the count is maintained correctly.

Space & Time

Time Complexity: $O(n)$ for each helper sliding window (total $O(n)$ since each element is processed a constant number of times)

Space Complexity: $O(n)$ in the worst case, due to the hash map storing frequencies of unique

numbers.

Solution

To solve the problem, we use a sliding window technique combined with a helper function that counts subarrays with at most k distinct integers. The idea is:

- Create a helper function `countAtMost(K)` that returns the number of subarrays with at most K distinct integers.**
- In the helper, maintain two pointers (left and right) for the sliding window and a frequency map.**
- For each position of right pointer, update the frequency map. If the number of distinct integers exceeds K, move the left pointer to reduce it.**
- The total subarrays ending at the current right are added to the answer.**
- The final answer is: `countAtMost(k) – countAtMost(k – 1)`.**

This approach ensures we efficiently count valid subarrays without needing to re-calculate subarray properties repeatedly.

Binary Search

Round 3 · High · Hard · 2 questions

Binary Search

□ #719 Find K-th Smallest Pair Distance

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search · Sorting
Lists: AlgoMaster 600

Problem

The distance of a pair of integers a and b is defined as the absolute difference between a and b .

Given an integer array `nums` and an integer k , return the k th smallest distance among all the pairs `nums[i]` and `nums[j]` where $0 \leq i < j < \text{nums.length}$.

Example 1:

```
Input: nums = [1,3,1], k = 1
Output: 0
Explanation: Here are all the pairs:
(1,3) -> 2
(1,1) -> 0
(3,1) -> 2
Then the 1st smallest distance pair is (1,1), and its distance is 0.
```

Example 2:

```
Input: nums = [1,1,1], k = 2
Output: 0
```

Example 3:

Input: `nums = [1,6,1]`, `k = 3`

Output: 5

Constraints:

- `n == nums.length`
- `2 <= n <= 104`
- `0 <= nums[i] <= 106`
- `1 <= k <= n * (n - 1) / 2`

Community Solutions (Python)

• WalkCC

```
class Solution:
    def smallestDistancePair(self, nums: list[int], k: int) -> int:
        nums.sort()

        def numPairDistancesNoGreaterThan(m: int) -> int:
            count = 0
            j = 1
            # For each index i, find the first index j s.t. nums[j] > nums[i] + m,
            # so numPairDistancesNoGreaterThan for the index i will be j - i - 1.
            for i, num in enumerate(nums):
                while j < len(nums) and nums[j] <= num + m:
                    j += 1
                count += j - i - 1
            return count

        return bisect.bisect_left(range(nums[-1] - nums[0]), k,
                                   key=numPairDistancesNoGreaterThan)
```

• LeetDoocs

```

class Solution:
    def smallestDistancePair(self, nums: List[int], k: int) -> int:
        def count(dist):
            cnt = 0
            for i, b in enumerate(nums):
                a = b - dist
                j = bisect_left(nums, a, 0, i)
                cnt += i - j
            return cnt

        nums.sort()
        return bisect_left(range(nums[-1] - nums[0]), k, key=count)

```

• SimplyLeet

```

# Function to find the kth smallest pair distance using binary search and two pointers
def smallestDistancePair(nums, k):
    # Sort the array to simplify finding pair distances.
    nums.sort()

    # Define the search range for possible distances
    low, high = 0, nums[-1] - nums[0]

    # Helper function: count number of pairs with difference <= mid.
    def count_pairs(mid):
        count, left = 0, 0
        # Use two pointers to count valid pairs.
        for right in range(len(nums)):
            while nums[right] - nums[left] > mid:
                left += 1
            count += right - left
        return count

    # Binary search over the possible distances.
    while low < high:

```

```
mid = (low + high) // 2 # Midpoint of current distance range.
if count_pairs(mid) < k:
    low = mid + 1 # Not enough pairs, increase low bound.
else:
    high = mid # Enough pairs, try a smaller distance.
return low
```

```
# Example usage:
```

```
print(smallestDistancePair([1, 3, 1], 1)) # Expected Output: 0
```

◆ Quick Review

Key Insights

- Sorting the array simplifies the calculation since the absolute difference is monotonic when traversing sequentially.
- Instead of generating all pair distances, use binary search on the range of possible distances.
- A two–pointer technique can efficiently count the number of pairs with a distance less than or equal to a given middle value.
- The binary search converges on the kth smallest distance based on the count of valid pairs.

Space & Time

Time Complexity: $O(n \log(\max - \min) + n \log n)$ where n is the number of elements. $O(n \log$

n) for sorting and $O(n \log(\max - \min))$ for binary search across the distance range.

Space Complexity: $O(1)$ extra space (ignoring the space required for sorting).

Solution

The approach starts by sorting the input array to leverage the ordered property of the numbers. The possible pair distances range between 0 and $(\max(\text{nums}) - \min(\text{nums}))$. Use binary search over this range to find the smallest distance d such that there are at least k pairs with a distance less than or equal to d . A helper function uses a two-pointer method on the sorted array to count the number of pairs satisfying the condition. Adjust the binary search boundaries based on this count until convergence.

Binary Search

□ #1095 Find in Mountain Array

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Interactive
Lists: AlgoMaster 600

Problem

(This problem is an interactive problem.)

You may recall that an array `arr` is a mountain array if and only if:

- `arr.length >= 3`
- There exists some `i` with $0 < i < \text{arr.length} - 1$ such that: `arr[0] < arr[1] < ... < arr[i - 1] < arr[i]`
- `arr[i] > arr[i + 1] > ... > arr[arr.length - 1]`

Given a mountain array `mountainArr`, return the minimum index such that `mountainArr.get(index) == target`. If such an index does not exist, return `-1`.

You cannot access the mountain array directly. You may only access the array using a `MountainArray` interface:

- `MountainArray.get(k)` returns the element of the array at index `k` (0-indexed).
- `MountainArray.length()` returns the length of the array.

Submissions making more than 100 calls to `MountainArray.get` will be judged Wrong Answer. Also, any solutions that attempt to circumvent the judge will result in disqualification.

Example 1:

```
Input: mountainArr = [1,2,3,4,5,3,1], target = 3
Output: 2
Explanation: 3 exists in the array, at index=2 and index=5. Return the minimum index, which is 2.
```

Example 2:

```
Input: mountainArr = [0,1,2,4,2,1], target = 3
Output: -1
Explanation: 3 does not exist in the array, so we return -1.
```

Constraints:

- $3 \leq \text{mountainArr.length()} \leq 104$
- $0 \leq \text{target} \leq 109$
- $0 \leq \text{mountainArr.get(index)} \leq 109$

Community Solutions (Python)

- WalkCC

```
# """
# This is MountainArray's API interface.
# You should not implement it, or speculate about its implementation
# """
# Class MountainArray:
# def get(self, index: int) -> int:
# def length(self) -> int:

class Solution:
    def findInMountainArray(

        self,
        target: int,
        mountain_arr: 'MountainArray',
    ) -> int:
        n = mountain_arr.length()
        peakIndex = self.peakIndexInMountainArray(mountain_arr, 0, n - 1)

        leftIndex = self.searchLeft(mountain_arr, target, 0, peakIndex)
        if mountain_arr.get(leftIndex) == target:
            return leftIndex

        rightIndex = self.searchRight(mountain_arr, target, peakIndex + 1, n - 1)
        if mountain_arr.get(rightIndex) == target:
            return rightIndex

        return -1

# 852. Peak Index in a Mountain Array
def peakIndexInMountainArray(self, A: 'MountainArray', l: int, r: int) -> in
t:
    while l < r:
```



```

        m = (l + r) // 2
        if A.get(m) < A.get(m + 1):
            l = m + 1
        else:
            r = m
        return l

    def searchLeft(self, A: 'MountainArray', target: int, l: int, r: int) -> int:
        while l < r:
            m = (l + r) // 2

            if A.get(m) < target:
                l = m + 1
            else:
                r = m
            return l

    def searchRight(self, A: 'MountainArray', target: int, l: int, r: int) -> int:
        while l < r:
            m = (l + r) // 2
            if A.get(m) > target:
                r = m
            else:
                l = m + 1
            return l

```

• LeetDoocs

```

# """
# This is MountainArray's API interface.
# You should not implement it, or speculate about its implementation
# """
# class MountainArray:
#     def get(self, index: int) -> int:
#     def length(self) -> int:

class Solution:

```

```

def findInMountainArray(self, target: int, mountain_arr: 'MountainArray')
-> int:
    def search(l: int, r: int, k: int) -> int:
        while l < r:
            mid = (l + r) >> 1
            if k * mountain_arr.get(mid) >= k * target:
                r = mid
            else:
                l = mid + 1
        return -1 if mountain_arr.get(l) != target else l

    n = mountain_arr.length()
    l, r = 0, n - 1
    while l < r:
        mid = (l + r) >> 1
        if mountain_arr.get(mid) > mountain_arr.get(mid + 1):
            r = mid
        else:
            l = mid + 1
    ans = search(0, l, 1)
    return search(l + 1, n - 1, -1) if ans == -1 else ans

```

• SimplyLeet

```

# Python solution with detailed comments.
class Solution:
    def findInMountainArray(self, target: int, mountain_arr: 'MountainArray')
-> int:
        # Find the peak of the mountain array using binary search.
        n = mountain_arr.length()
        left, right = 0, n - 1
        while left < right:
            mid = (left + right) // 2
            # Compare current value with the next value to decide direction.
            if mountain_arr.get(mid) < mountain_arr.get(mid + 1):

```

```

        left = mid + 1
    else:
        right = mid
peak = left # The index of the peak

# Binary search on the ascending part of the array.
def binarySearchAsc(low, high):
    while low <= high:
        mid = (low + high) // 2
        value = mountain_arr.get(mid)

        if value == target:
            return mid
        elif value < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

# Binary search on the descending part of the array.
def binarySearchDesc(low, high):
    while low <= high:
        mid = (low + high) // 2
        value = mountain_arr.get(mid)
        if value == target:
            return mid
        elif value > target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

# Try searching in the ascending part.
index = binarySearchAsc(0, peak)
if index != -1:
    return index
# Otherwise, search in the descending part.
return binarySearchDesc(peak + 1, n - 1)

```

◆ Quick Review

Key Insights

- Use binary search to minimize the number of `MountainArray.get` calls.
- First, identify the peak (maximum element) using a modified binary search.
- Once the peak is found, perform two separate binary searches: one on the increasing part (left side) and one on the decreasing part (right side).
- Return the index from the left side search if found (since we want the minimum index); otherwise, check the right side.
- Be mindful of the interactive constraint where total number of get calls must be under 100.

Space & Time

Time Complexity: $O(\log n)$ – binary searches are used to identify the peak and target index.

Space Complexity: $O(1)$ – only a constant amount of extra memory is used.

Solution

The solution consists of three main steps:

- **Peak Finding:** Use binary search over the mountain array to locate the peak. In each step, compare element at mid and mid+1 to determine if the peak is to the right or left.
- **Target Search in the Ascending Part:** Conduct a standard binary search in the increasing segment (from index 0 to peak). Since the sequence is sorted in ascending order, a regular binary search works.
- **Target Search in the Descending Part:** If the target is not found on the left, perform a binary search on the decreasing segment (from peak+1 to the end) where comparisons are inverted (i.e., larger values come before smaller ones).
- Return the index from the left search if found, otherwise if found on the right, return that index. If not found in either part, return -1.

Depth First Search (DFS)

Round 3 · High · Hard · 2 questions

Depth First Search (DFS)

□ #827 Making A Large Island

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Depth-First Search · Breadth-First
Search · Union-Find · Matrix
Lists: AlgoMaster 600

Problem

You are given an $n \times n$ binary matrix grid. You are allowed to change at most one 0 to be 1. Return the size of the largest island in grid after applying this operation.

An island is a 4-directionally connected group of 1s.

Example 1:

Input: grid = [[1,0],[0,1]]

Output: 3

Explanation: Change one 0 to 1 and connect two 1s, then we get an island with area = 3.

Example 2:

Input: grid = [[1,1],[1,0]]

Output: 4

Explanation: Change the 0 to 1 and make the island bigger, only one island with area = 4.

Example 3:

Input: grid = [[1,1],[1,1]]

Output: 4

Explanation: Can't change any 0 to 1, only one island with area = 4.

Constraints:

- $n == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq n \leq 500$
- $\text{grid}[i][j]$ is either 0 or 1.

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def largestIsland(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int):
            p[i][j] = root
            cnt[root] += 1
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < n and 0 <= y < n and grid[x][y] and p[x][y] == 0:
                    dfs(x, y)

        n = len(grid)
        cnt = Counter()
        p = [[0] * n for _ in range(n)]
        dirs = (-1, 0, 1, 0, -1)
        root = 0
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                if x and p[i][j] == 0:
                    root += 1
                    dfs(i, j)
```



```

ans = max(cnt.values() or [0])
for i, row in enumerate(grid):
    for j, x in enumerate(row):
        if x == 0:
            s = set()
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < n and 0 <= y < n:
                    s.add(p[x][y])
            ans = max(ans, sum(cnt[root] for root in s) + 1)

return ans

```

• SimplyLeet

```

# Python solution for Making A Large Island
class Solution:
    def largestIsland(self, grid: List[List[int]]) -> int:
        n = len(grid)
        island_size = {} # Map island id to island area
        island_id = 2 # Start island ids from 2

        # Helper function: DFS to label the islands
        def dfs(i, j, id):
            if i < 0 or i >= n or j < 0 or j >= n or grid[i][j] != 1:
                return 0
            grid[i][j] = id # Label the cell with the island id
            size = 1
            # Visit neighbors (4-directionally)
            size += dfs(i + 1, j, id)
            size += dfs(i - 1, j, id)
            size += dfs(i, j + 1, id)
            size += dfs(i, j - 1, id)
            return size

```

```
# First pass: identify each island and record its size.
for i in range(n):
    for j in range(n):
        if grid[i][j] == 1: # Found an unlabeled island cell
            size = dfs(i, j, island_id)
            island_size[island_id] = size
            island_id += 1

# If no 0 cell exists, then the entire grid is already one island.
max_area = max(island_size.values()) if island_size else 0
```

```
# Second pass: Try changing each 0 to 1 and calculate the connected
island area.
```

```
for i in range(n):
    for j in range(n):
        if grid[i][j] == 0:
            seen_islands = set()
            current_area = 1 # Count the flip (0 -> 1)
            for di, dj in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                ni, nj = i + di, j + dj
                if 0 <= ni < n and 0 <= nj < n:
```

```
                    id = grid[ni][nj]
                    if id > 1 and id not in seen_islands:
                        seen_islands.add(id)
                        current_area += island_size[id]
            max_area = max(max_area, current_area)
```

```
return max_area
```

◆ Quick Review

Key Insights

- First, identify and label every existing island using DFS/BFS.
- Maintain a mapping from island id to its size.

- For each 0, examine its four neighbors. Sum the size of distinct adjacent islands and add 1 (for the flipped cell) to compute a candidate island size.
- Be careful to avoid counting the same island more than once when summing.
- If the grid is full of 1s (i.e. no 0 to flip), then the largest island is the whole grid.

Space & Time

Time Complexity: $O(n^2)$ because each cell is visited a constant number of times.

Space Complexity: $O(n^2)$ for the recursion/queue in DFS/BFS in the worst case and for storing island information.

Solution

The solution uses a two-phase algorithm. In the first phase, we use DFS to label each island with a unique id (starting from 2 to avoid confusion with 0 and 1) and record its area in a mapping (dictionary). In the second phase, for each cell that is 0, we look at its 4-directional neighbors to find distinct islands. We then sum the sizes of these islands and add 1 (for converting the current 0 into a

1), tracking the maximum island size possible. This approach efficiently calculates the largest island that can be created after a single modification.

Depth First Search (DFS)

□ #2246 Longest Path With Different Adjacent Characters

HAR

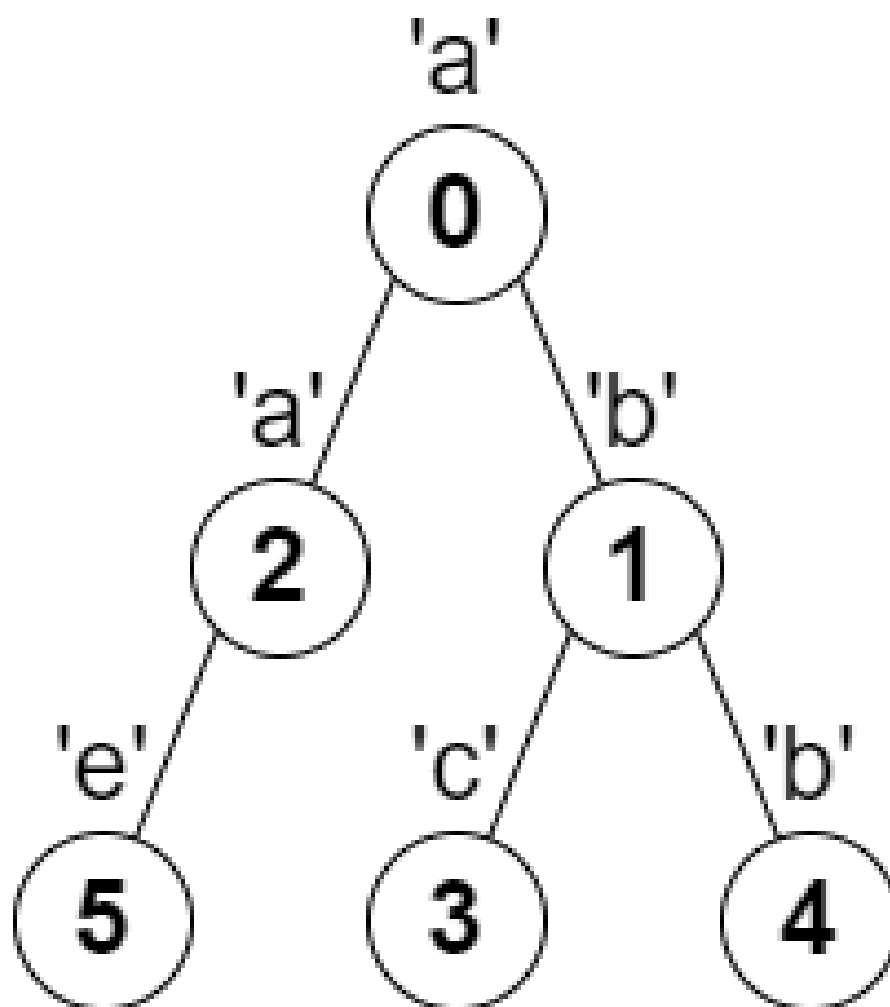
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Tree · Depth-First Search ·
Graph Theory · Topological Sort
Lists: AlgoMaster 600

Problem

You are given a tree (i.e. a connected, undirected graph that has no cycles) rooted at node 0 consisting of n nodes numbered from 0 to $n - 1$. The tree is represented by a 0-indexed array `parent` of size n , where `parent[i]` is the parent of node i . Since node 0 is the root, `parent[0] == -1`. You are also given a string `s` of length n , where `s[i]` is the character assigned to node i .

Return the length of the longest path in the tree such that no pair of adjacent nodes on the path have the same character assigned to them.

Example 1:



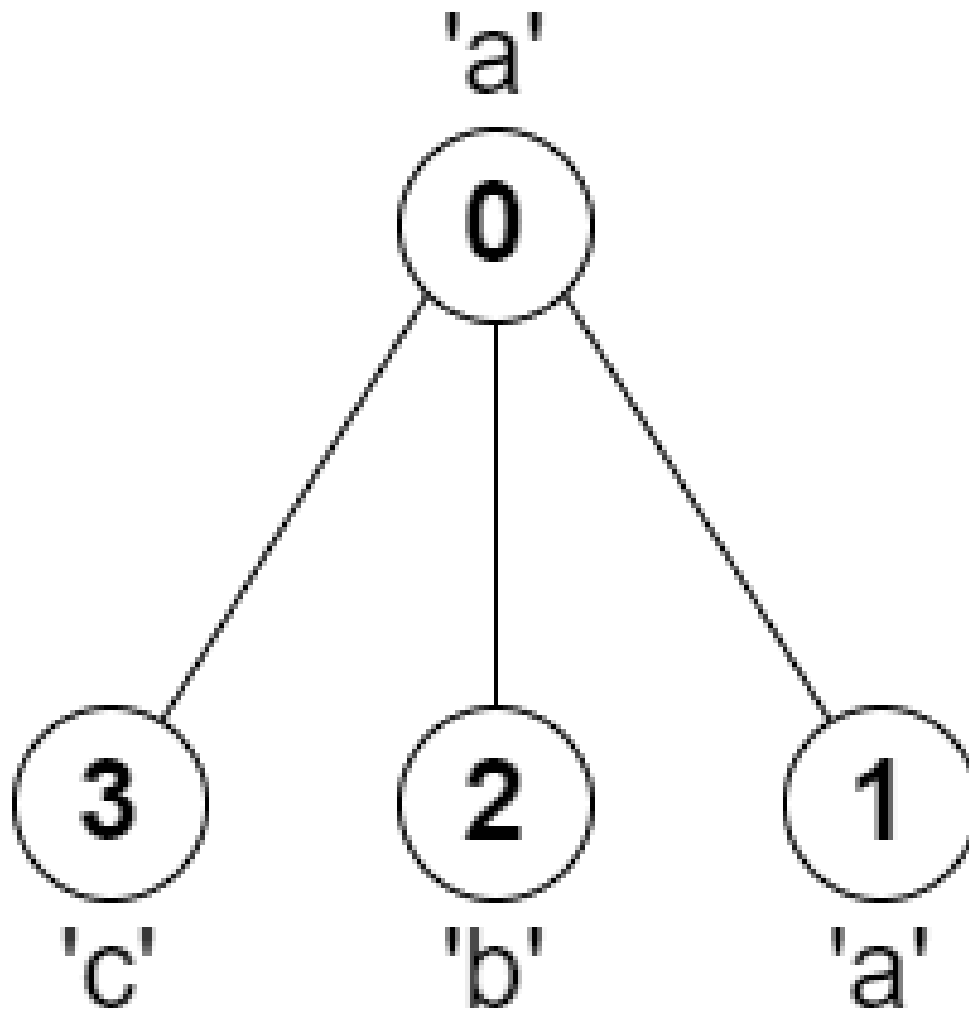
Input: parent = [-1,0,0,1,1,2], s = "abacbe"

Output: 3

Explanation: The longest path where each two adjacent nodes have different characters in the tree is the path: 0 -> 1 -> 3. The length of this path is 3, so 3 is returned.

It can be proven that there is no longer path that satisfies the conditions.

Example 2:



Input: `parent = [-1,0,0,0]`, `s = "aabc"`

Output: 3

Explanation: The longest path where each two adjacent nodes have different characters is the path: 2 → 0 → 3. The length of this path is 3, so 3 is returned.

Constraints:

- `n == parent.length == s.length`
- `1 <= n <= 105`
- `0 <= parent[i] <= n - 1` for all `i >= 1`
- `parent[0] == -1`
- `parent` represents a valid tree.

- s consists of only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestPath(self, parent: list[int], s: str) -> int:
        n = len(parent)
        ans = 0
        graph = [[] for _ in range(n)]

        for i in range(1, n):
            graph[parent[i]].append(i)

        def longestPathDownFrom(u: int) -> int:

            nonlocal ans
            max1 = 0
            max2 = 0

            for v in graph[u]:
                res = longestPathDownFrom(v)
                if s[u] == s[v]:
                    continue
                if res > max1:
                    max2 = max1

            max1 = res
            elif res > max2:
                max2 = res

            ans = max(ans, 1 + max1 + max2)
            return 1 + max1

        longestPathDownFrom(0)
        return ans
```

• LeetDoocs


```

class Solution:
    def longestPath(self, parent: List[int], s: str) -> int:
        def dfs(i: int) -> int:
            mx = 0
            nonlocal ans
            for j in g[i]:
                x = dfs(j) + 1
                if s[i] != s[j]:
                    ans = max(ans, mx + x)
                    mx = max(mx, x)

            return mx

        g = defaultdict(list)
        for i in range(1, len(parent)):
            g[parent[i]].append(i)
        ans = 0
        dfs(0)
        return ans + 1

```

• SimplyLeet

Python solution for "Longest Path With Different Adjacent Characters"

```

def longestPath(parent, s):
    from collections import defaultdict

    # Build the adjacency list for the tree
    tree = defaultdict(list)
    n = len(parent)
    for node in range(1, n):
        tree[parent[node]].append(node)

```

```

# Global variable to store the longest valid path length
max_path = 0

# DFS function to compute the longest valid branch starting from the
current node
def dfs(node):
    nonlocal max_path
    # The two longest valid branch lengths from the children (without
current node)
    longest, second_longest = 0, 0

    # Explore each child
    for child in tree[node]:
        branch_length = dfs(child) # Compute branch length for child
subtree

        # Only extend the branch if the character at child is different
        if s[child] != s[node]:
            if branch_length > longest:
                second_longest = longest
                longest = branch_length
            elif branch_length > second_longest:
                second_longest = branch_length

    # Update the global maximum with the sum of the two best branches from
the node plus itself
    max_path = max(max_path, longest + second_longest + 1)
    # Return the length of the longest branch including the current node
    return longest + 1

dfs(0)
return max_path

# Example usage:

# parent = [-1,0,0,1,1,2]
# s = "abacbe"
# print(longestPath(parent, s)) # Output: 3

```

◆ Quick Review

Round 3 · High · Hard

p.718

Key Insights

- The tree structure guarantees a unique path between any two nodes.
- A depth-first search (DFS) can be used to explore each subtree and aggregate the maximum valid branch lengths.
- At each node, only child paths with a different character than the current node can be extended.
- Combining the two longest valid branches stemming from a node gives the candidate longest path that passes through that node.
- A global variable is maintained to track the overall longest valid path found during traversal.

Space & Time

Time Complexity: $O(n)$ – Each node is visited once.

Space Complexity: $O(n)$ – For recursion stack and storage of the tree structure.

Solution

We solve the problem using a DFS traversal. First, we build an adjacency list (child list)

from the parent array to represent the tree. For each node, we perform a DFS that returns the length of the longest valid path starting from that node (including itself) where the adjacent characters differ. For every node, we consider all its children and update the two longest valid branch lengths only if their characters differ from the current node. The candidate longest path passing through a node is the sum of the two longest valid branch lengths plus 1 (for the current node). We update a global maximum with this value at every node. Finally, the DFS returns the overall maximum path length.

Breadth First Search (BFS)

Round 3 · High · Hard · 2 questions

Breadth First Search (BFS)

□ #1293 Shortest Path in a Grid with Obstacles Elimination

HAR

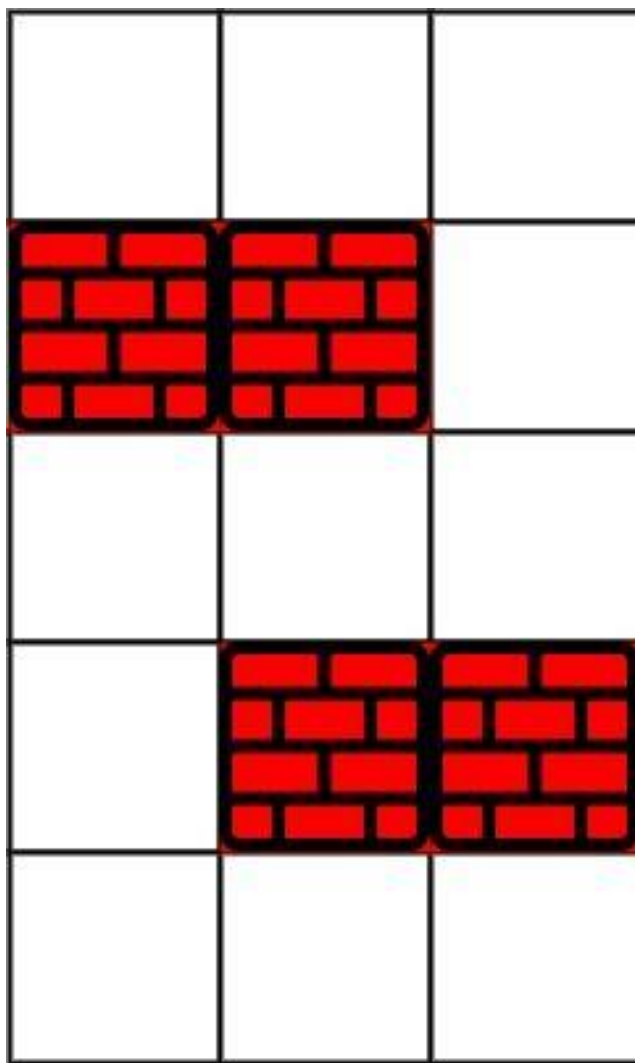
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Breadth-First Search · Matrix
Lists: AlgoMaster 600

Problem

You are given an $m \times n$ integer matrix grid where each cell is either 0 (empty) or 1 (obstacle). You can move up, down, left, or right from and to an empty cell in one step.

Return the minimum number of steps to walk from the upper left corner (0, 0) to the lower right corner ($m - 1$, $n - 1$) given that you can eliminate at most k obstacles. If it is not possible to find such walk return -1.

Example 1:



Input: grid = [[0,0,0],[1,1,0],[0,0,0],[0,1,1],[0,0,0]], k = 1

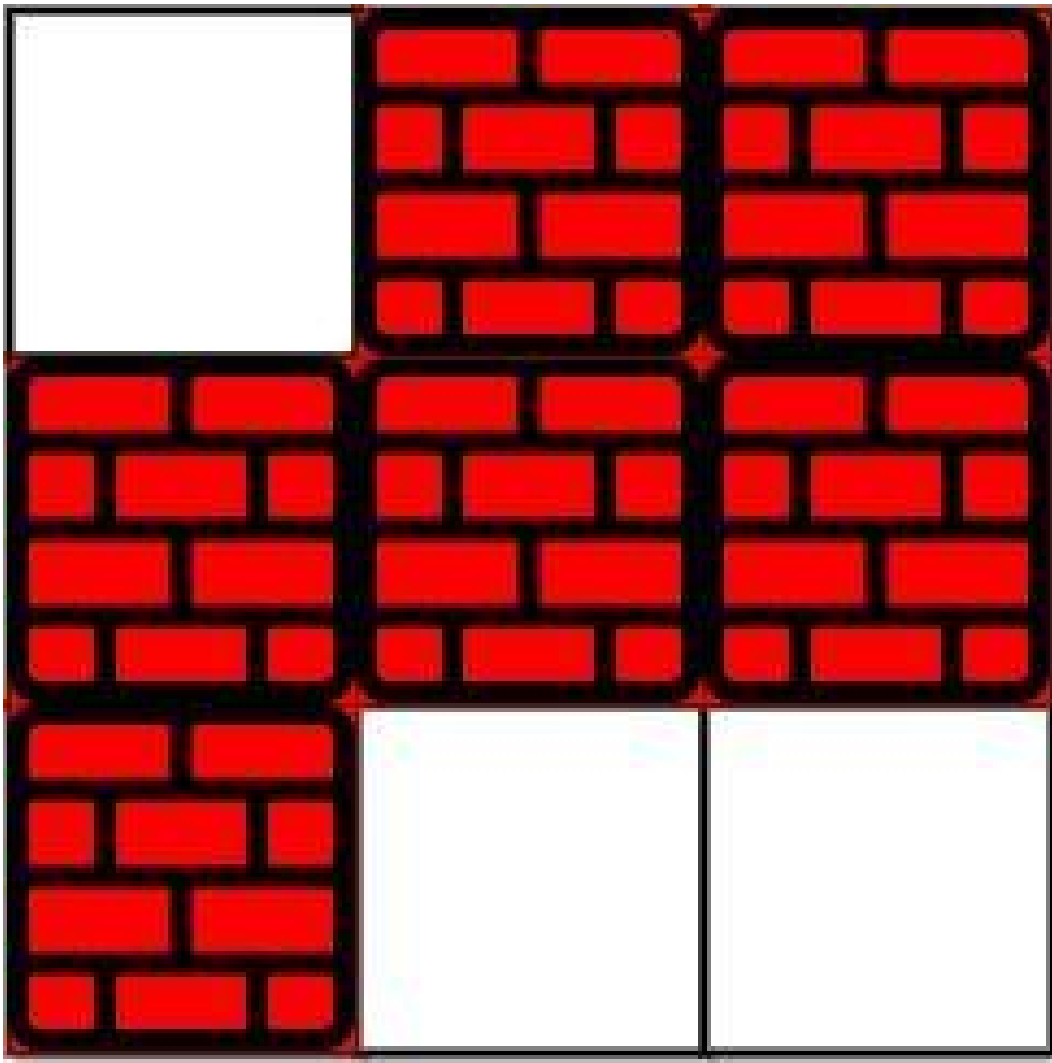
Output: 6

Explanation:

The shortest path without eliminating any obstacle is 10.

The shortest path with one obstacle elimination at position (3,2) is 6. Such path is (0,0) -> (0,1) -> (0,2) -> (1,2) -> (2,2) -> (3,2) -> (4,2).

Example 2:



Input: grid = [[0,1,1],[1,1,1],[1,0,0]], k = 1

Output: -1

Explanation: We need to eliminate at least two obstacles to find such a walk.

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 40$
- $1 \leq k \leq m * n$
- $\text{grid}[i][j]$ is either 0 or 1.
- $\text{grid}[0][0] == \text{grid}[m - 1][n - 1] == 0$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def shortestPath(self, grid: list[list[int]], k: int) -> int:
        m = len(grid)
        n = len(grid[0])
        if m == 1 and n == 1:
            return 0

        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        q = collections.deque([(0, 0, k)])
        seen = {(0, 0, k)}

        step = 0
        while q:
            step += 1
            for _ in range(len(q)):
                i, j, eliminate = q.popleft()
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if x == m - 1 and y == n - 1:
                        return step
                    if grid[x][y] == 1 and eliminate == 0:
                        continue
                    newEliminate = eliminate - grid[x][y]
                    if (x, y, newEliminate) in seen:
                        continue
                    q.append((x, y, newEliminate))
                    seen.add((x, y, newEliminate))

        return -1
```

• LeetDoocs

```

class Solution:
    def shortestPath(self, grid: List[List[int]], k: int) -> int:
        m, n = len(grid), len(grid[0])
        if k >= m + n - 3:
            return m + n - 2
        q = deque([(0, 0, k)])
        vis = {(0, 0, k)}
        ans = 0
        while q:
            ans += 1

            for _ in range(len(q)):
                i, j, k = q.popleft()
                for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n:
                        if x == m - 1 and y == n - 1:
                            return ans
                        if grid[x][y] == 0 and (x, y, k) not in vis:
                            q.append((x, y, k))
                            vis.add((x, y, k))

                        if grid[x][y] == 1 and k > 0 and (x, y, k - 1) not in vis:
                            q.append((x, y, k - 1))
                            vis.add((x, y, k - 1))

        return -1

```

• SimplyLeet

```

from collections import deque

def shortestPath(grid, k):
    # Dimensions of the grid
    rows, cols = len(grid), len(grid[0])

    # If k is large enough, we can take a direct Manhattan route
    if k >= rows + cols - 2:
        return rows + cols - 2

```

```

# Queue for BFS: each element is (row, col, obstaclesEliminated, steps)
queue = deque([(0, 0, 0, 0)])

# 3D visited: visited[row][col] stores the smallest number of obstacles
eliminated so far
visited = [[[False] * (k + 1) for _ in range(cols)] for _ in range(rows)]
visited[0][0][0] = True

# Possible directions: right, left, down, up
directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

while queue:
    row, col, eliminated, steps = queue.popleft()

    # If reached destination, return the number of steps
    if row == rows - 1 and col == cols - 1:
        return steps

    # Explore neighbors in all four directions
    for dr, dc in directions:
        newRow, newCol = row + dr, col + dc

        # Check if new position is inside the grid bounds
        if 0 <= newRow < rows and 0 <= newCol < cols:
            newEliminated = eliminated + grid[newRow][newCol]
            # Only proceed if the new elimination count is within limit and
            state not visited
            if newEliminated <= k and not visited[newRow][newCol][newEliminated]:
                visited[newRow][newCol][newEliminated] = True
                queue.append((newRow, newCol, newEliminated, steps + 1))

    # If destination is unreachable, return -1
    return -1

# Example Usage:
print(shortestPath([[0,0,0],[1,1,0],[0,0,0],[0,1,1],[0,0,0]], 1))

```

◆ Quick Review

Key Insights

- Use a Breadth-First Search (BFS) approach to explore the grid level by level.
- Maintain a state that includes the current position (row and column) and the number of obstacles eliminated so far.
- Use a 3-dimensional visited data structure (or a 2D array with obstacle elimination count) to avoid reprocessing states that have been reached with fewer eliminations.
- The BFS ensures that the first time you reach the target, it is via the shortest path.

Space & Time

Time Complexity: $O(m * n * k)$, where m and n are grid dimensions and k is the maximum obstacles that can be eliminated.

Space Complexity: $O(m * n * k)$, due to storing the state (position and obstacles eliminated).

Solution

The solution utilizes a modified BFS that tracks each state as a tuple (row, column, obstaclesEliminated). Starting from (0,0) with 0 obstacles eliminated, we explore all possible moves (up, down, left, right). When moving to

an adjacent cell:

- If the cell is empty, continue without increasing the elimination count.**
- If the cell is an obstacle and you have remaining eliminations (i.e., $obstaclesEliminated \leq k$), increase the elimination count. Only states that offer a better (i.e., lower elimination count) path to a cell compared to any previously recorded state are added to the BFS queue. This prevents unnecessary reprocessing and reduces time complexity. When the destination is reached, the current step count from the BFS is returned. If the queue depletes without reaching the destination, the function returns -1 .**

Breadth First Search (BFS)

□ #2290 Minimum Obstacle Removal to Reach Corner

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Breadth-First Search · Graph Theory ·
Heap (Priority Queue) · Matrix · Shortest Path
Lists: AlgoMaster 600

Problem

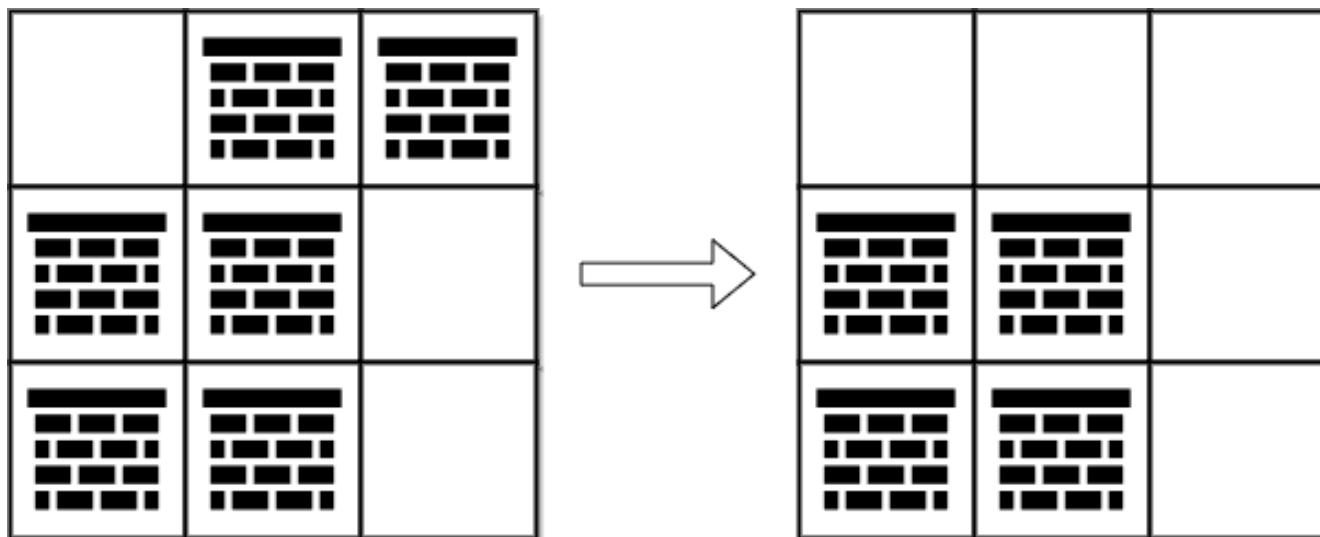
You are given a 0-indexed 2D integer array grid of size $m \times n$. Each cell has one of two values:

- 0 represents an empty cell,
- 1 represents an obstacle that may be removed.

You can move up, down, left, or right from and to an empty cell.

Return the minimum number of obstacles to remove so you can move from the upper left corner $(0, 0)$ to the lower right corner $(m - 1, n - 1)$.

Example 1:



Input: grid = [[0,1,1],[1,1,0],[1,1,0]]

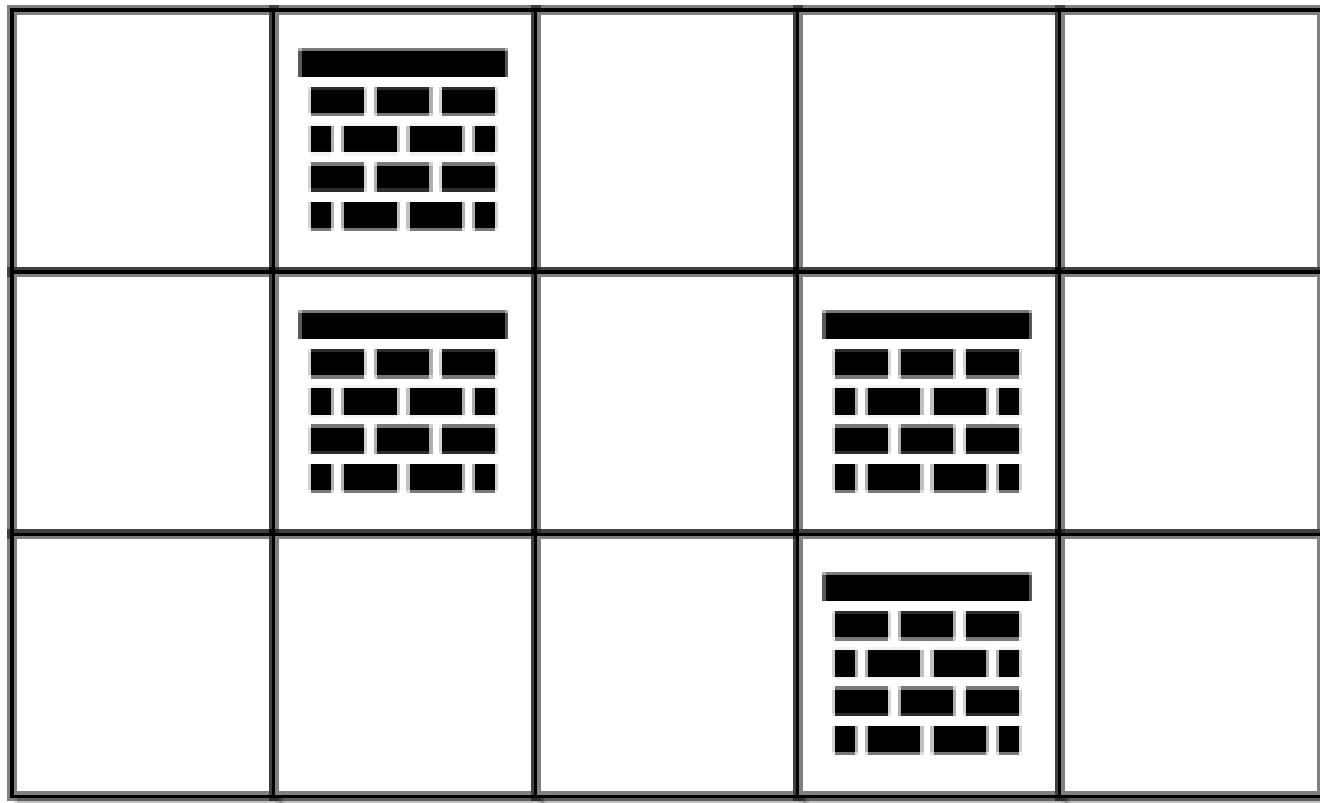
Output: 2

Explanation: We can remove the obstacles at (0, 1) and (0, 2) to create a path from (0, 0) to (2, 2).

It can be shown that we need to remove at least 2 obstacles, so we return 2.

Note that there may be other ways to remove 2 obstacles to create a path.

Example 2:



Input: `grid = [[0,1,0,0,0],[0,1,0,1,0],[0,0,0,1,0]]`

Output: `0`

Explanation: We can move from (0, 0) to (2, 4) without removing any obstacles, so we return 0.

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 105`
- `2 <= m * n <= 105`
- `grid[i][j]` is either 0 or 1.
- `grid[0][0] == grid[m - 1][n - 1] == 0`

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minimumObstacles(self, grid: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        minHeap = [(grid[0][0], 0, 0)] # (d, i, j)
        dist = [[math.inf] * n for _ in range(m)]
        dist[0][0] = grid[0][0]

        while minHeap:
```



```

d, i, j = heapq.heappop(minHeap)
if i == m - 1 and j == n - 1:
    return d
for dx, dy in DIRS:
    x = i + dx
    y = j + dy
    if x < 0 or x == m or y < 0 or y == n:
        continue
    newDist = d + grid[i][j]
    if newDist < dist[x][y]:

        dist[x][y] = newDist
        heapq.heappush(minHeap, (newDist, x, y))

return dist[m - 1][n - 1]

```

• LeetDoocs

```

class Solution:
    def minimumObstacles(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        q = deque([(0, 0, 0)])
        vis = set()
        dirs = (-1, 0, 1, 0, -1)
        while 1:
            i, j, k = q.popleft()
            if i == m - 1 and j == n - 1:
                return k

            if (i, j) in vis:
                continue
            vis.add((i, j))
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n:
                    if grid[x][y] == 0:
                        q.appendleft((x, y, k))
                    else:
                        q.append((x, y, k + 1))

```

• SimplyLeet

```

from collections import deque

def minimumObstacles(grid):
    # Number of rows and columns
    m, n = len(grid), len(grid[0])
    # Initialize deque with the starting position and initial cost 0
    dq = deque([(0, 0, 0)]) # (row, col, obstacles_removed)

    # Create a 2D list to track the minimum obstacles removed to reach each
    cell
    min_removed = [[float('inf')] * n for _ in range(m)]

    min_removed[0][0] = 0

    # Directions: up, down, left, right
    directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

    # Process the deque BFS style using 0-1 BFS approach
    while dq:
        r, c, cost = dq.popleft()
        # If destination is reached, return the cost
        if r == m - 1 and c == n - 1:
            return cost

        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            # Check bounds
            if 0 <= nr < m and 0 <= nc < n:
                # Calculate new_cost: add 1 if there is an obstacle, else add 0
                new_cost = cost + grid[nr][nc]
                # Only proceed if this new_cost is better than any previously
                found
                if new_cost < min_removed[nr][nc]:

```

```
min_removed[nr][nc] = new_cost
# For cost 0, append left; for cost 1, append right
if grid[nr][nc] == 0:
    dq.appendleft((nr, nc, new_cost))
else:
    dq.append((nr, nc, new_cost))

# If destination is unreachable, though the problem guarantees it is
# reachable.
return -1

# Example usage:

grid_example = [[0, 1, 1], [1, 1, 0], [1, 1, 0]]
print(minimumObstacles(grid_example))
```

◆ Quick Review

Key Insights

- The grid can be interpreted as a weighted graph where moving through an empty cell (0) has cost 0 and moving through an obstacle (1) has cost 1.
- The goal is to find the path with the minimum total cost.
- Use a 0–1 BFS or a modified Dijkstra's algorithm to efficiently process the grid.
- 0–1 BFS optimizes the traversal by using a deque: pushing 0–cost moves to the front and 1–cost moves to the back.

Space & Time

Time Complexity: $O(m * n)$ since each cell is processed at most once.

Space Complexity: $O(m * n)$ for the deque and the visited/cost matrix.

Solution

We solve the problem by leveraging 0-1 BFS. The grid is modeled as a weighted graph where:

- Moving to a neighbor cell with value 0 has no additional cost.
- Moving to a neighbor cell with value 1 incurs a cost of 1 (obstacle removal).

A deque (double-ended queue) is used:

- When moving to an empty cell (0), push the new position to the front of the deque.
- When moving to an obstacle (1), push the new position to the back.

This ensures positions that cost 0 are prioritized, and we maintain an array or matrix to keep track of the minimum obstacles removed to reach each cell.

The algorithm terminates when the destination cell is dequeued with the minimum cost, or when all cells are processed.

Round 4 · Mid · Easy

Round 4

Mid Priority · Easy

20 questions · 9 patterns

Bit Manipulation #231 #461 #645

Prefix Sum #303 #724 #1480 #1854

Monotonic Stack #496 #1475

Queues #933 #2073

Divide and Conquer #1763

BST / Ordered Set #653 #700 #938

Intervals #228

Data Structure Design #225 #359

Greedy #455 #3074

Bit Manipulation

Round 4 · Mid · Easy · 3 questions

Bit Manipulation

□ #231 Power of Two

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Bit Manipulation · Recursion
Lists: AlgoMaster 600

Problem

Given an integer n , return true if it is a power of two. Otherwise, return false.

An integer n is a power of two, if there exists an integer x such that $n == 2^x$.

Example 1:

Input: $n = 1$
Output: true
Explanation: $2^0 = 1$

Example 2:

Input: $n = 16$
Output: true
Explanation: $2^4 = 16$

Example 3:

Input: $n = 3$
Output: false

Constraints:

- $-2^{31} \leq n \leq 2^{31} - 1$

Follow up: Could you solve it without loops/recursion?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isPowerOfTwo(self, n: int) -> bool:
        return n >= 0 and n.bit_count() == 1
```

• LeetDoocs

```
class Solution:
    def isPowerOfTwo(self, n: int) -> bool:
        return n > 0 and (n & (n - 1)) == 0
```

```
class Solution:
    def isPowerOfTwo(self, n: int) -> bool:
        return n > 0 and n == n & (-n)
```

• SimplyLeet

```
# Function to determine if a number is a power of two
def isPowerOfTwo(n):
    # n must be positive and only one bit should be set in its binary
    # representation.
    return n > 0 and (n & (n - 1)) == 0

# Example usage:
print(isPowerOfTwo(1)) # True, because 2^0 = 1
print(isPowerOfTwo(16)) # True, because 2^4 = 16
print(isPowerOfTwo(3)) # False, 3 is not a power of two
```

◆ Quick Review

Key Insights

- A power of two in binary representation contains exactly one set bit.
- Negative numbers and zero cannot be powers of two.
- The bit manipulation trick $n \&\#x26; (n - 1)$ can be used to determine if there's exactly one set bit (for $n > 0$).

Space & Time

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Solution

The algorithm checks whether n is a power of two by leveraging the fact that powers of two have exactly one set bit. First, the check $n > 0$ ensures the number is positive because only positive numbers can be powers of two. Then, by performing $n \&\#x26; (n - 1)$, we verify that the number has only one set bit. If the result is 0, then n is a power of two; otherwise, it is not. This solution executes in constant time and uses constant space.

Bit Manipulation

□ #461 Hamming Distance

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Bit Manipulation

Lists: AlgoMaster 600

Problem

The Hamming distance between two integers is the number of positions at which the corresponding bits are different.

Given two integers x and y , return the Hamming distance between them.

Example 1:

Input: $x = 1, y = 4$

Output: 2

Explanation:

1 (0 0 0 1)

4 (0 1 0 0)

↑ ↑

The above arrows point to positions where the corresponding bits are different.

Example 2:

Input: $x = 3, y = 1$

Output: 1

Constraints:

- $0 \leq x, y \leq 2^{31} - 1$

Note: This question is the same as 2220: Minimum Bit Flips to Convert Number.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def hammingDistance(self, x: int, y: int) -> int:
        ans = 0

        while x > 0 or y > 0:
            ans += (x & 1) ^ (y & 1)
            x >>= 1
            y >>= 1

        return ans
```

• LeetDoocs

```
class Solution:
    def hammingDistance(self, x: int, y: int) -> int:
        return (x ^ y).bit_count()
```

• SimplyLeet

```
# Function to calculate Hamming distance between two integers.
def hammingDistance(x, y):
    # Compute XOR of x and y to highlight differing bits.
    xor_result = x ^ y
    count = 0
    # Count the number of set bits in xor_result.
    while xor_result:
        # Remove the lowest set bit.
        xor_result &= xor_result - 1
        count += 1

    return count
```

```
# Example usage:
print(hammingDistance(1, 4)) # Output: 2
```

◆ Quick Review

Key Insights

- The Hamming distance between two integers can be obtained by computing the XOR of x and y .
- The XOR operation highlights the bits that differ between the two numbers.
- Counting the number of set bits (1s) in the XOR result gives the required Hamming distance.
- An efficient method to count set bits is to repeatedly clear the least significant bit set using bit manipulation ($n = n \&\#x26; (n - 1)$).

Space & Time

Time Complexity: $O(1)$ (since the numbers are 32-bit, the operation takes constant time)

Space Complexity: $O(1)$

Solution

The solution involves computing the XOR of the two given integers. The resulting number will have bits set only at the positions where x and y differ. The next step is to count the

number of 1s in this XOR result. This count directly represents the Hamming distance. We can count the set bits by iteratively removing the lowest set bit (using the operation $n = n \& (n - 1)$) until the number becomes zero. This approach is efficient and leverages basic bit manipulation techniques.

Bit Manipulation

□ #645 Set Mismatch

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Bit Manipulation · Sorting
Lists: AlgoMaster 600**

Problem

You have a set of integers s , which originally contains all the numbers from 1 to n .

Unfortunately, due to some error, one of the numbers in s got duplicated to another number in the set, which results in repetition of one number and loss of another number.

You are given an integer array $nums$ representing the data status of this set after the error.

Find the number that occurs twice and the number that is missing and return them in the form of an array.

Example 1:

Input: $nums = [1,2,2,4]$ Output: $[2,3]$
--

Example 2:

Input: nums = [1,1]
Output: [1,2]

Constraints:

- $2 \leq \text{nums.length} \leq 104$
- $1 \leq \text{nums}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findErrorNums(self, nums: list[int]) -> list[int]:
        for num in nums:
            if nums[abs(num) - 1] < 0:
                duplicate = abs(num)
            else:
                nums[abs(num) - 1] *= -1

        for i, num in enumerate(nums):
            if num > 0:

                return [duplicate, i + 1]
```

• SimplyLeet

```
# Python solution with clear line-by-line comments.
class Solution:
    def findErrorNums(self, nums: List[int]) -> List[int]:
        # initialize a list to count frequency; size n+1 for 1-indexing
        n = len(nums)
        count = [0] * (n + 1)

        duplicate = -1
        missing = -1
```

```
# count frequency for each number in the array
for num in nums:
    count[num] += 1
    # if count goes to 2, it's the duplicate
    if count[num] == 2:
        duplicate = num

# check for the missing number in range 1 to n
for i in range(1, n + 1):
    if count[i] == 0:
        missing = i
        break

# return the duplicate and missing numbers as required
return [duplicate, missing]
```

◆ Quick Review

Key Insights

- The array is of length n and is intended to contain numbers from 1 to n .
- One number appears twice while another number is missing entirely.
- A simple way is to use a frequency count (often implemented via an array or hash table) to detect the duplicate.
- Alternatively, arithmetic properties (sum and sum of squares) or XOR can be exploited to solve the problem, but using a hash table is straightforward for an easy constraint.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ (due to using an extra data structure for frequency count)

Solution

We leverage a hash table (or an auxiliary array) to count the occurrences of each number in the array. As we iterate through the array, we detect the number that occurs twice. Then, by iterating through the numbers from 1 to n , we can determine which number is missing since it will have a count of 0.

Algorithm:

- Initialize an empty hash table (or an auxiliary list) to keep track of counts.
- Iterate through `nums` and update the count of each number.
- While updating counts, identify the number that appears twice.
- Iterate from 1 through n to find the missing number (the one with 0 count).
- Return the duplicate and missing numbers as an array.

Prefix Sum

Round 4 · Mid · Easy · 4 questions

Prefix Sum

□ #303 Range Sum Query – Immutable

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Design · Prefix Sum
Lists: AlgoMaster 600

Problem

Given an integer array `nums`, handle multiple queries of the following type:

1. Calculate the sum of the elements of `nums` between indices `left` and `right` inclusive where `left <= right`.

Implement the `NumArray` class:

- `NumArray(int[] nums)` Initializes the object with the integer array `nums`.
- `int sumRange(int left, int right)` Returns the sum of the elements of `nums` between indices `left` and `right` inclusive (i.e. `nums[left] + nums[left + 1] + ... + nums[right]`).

Example 1:

Input

```
["NumArray", "sumRange", "sumRange", "sumRange"]
[[-2, 0, 3, -5, 2, -1]], [0, 2], [2, 5], [0, 5]]
```

Output

```
[null, 1, -1, -3]
```

Explanation

```
NumArray numArray = new NumArray([-2, 0, 3, -5, 2, -1]);
```

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-105 \leq \text{nums}[i] \leq 105$
- $0 \leq \text{left} \leq \text{right} < \text{nums.length}$
- At most 104 calls will be made to `sumRange`.

Community Solutions (Python)

☐

• WalkCC

```
class NumArray:
    def __init__(self, nums: list[int]):
        self.prefix = list(itertools.accumulate(nums, initial=0))

    def sumRange(self, left: int, right: int) -> int:
        return self.prefix[right + 1] - self.prefix[left]
```

☐

• LeetDoocs

```
class NumArray:
    def __init__(self, nums: List[int]):
        self.s = list(accumulate(nums, initial=0))

    def sumRange(self, left: int, right: int) -> int:
        return self.s[right + 1] - self.s[left]

# Your NumArray object will be instantiated and called as such:
# obj = NumArray(nums)
```

```
# param_1 = obj.sumRange(left, right)
```

```
typedef struct {
    int* s;
} NumArray;
```

```
NumArray* numArrayCreate(int* nums, int n) {
    int* s = malloc(sizeof(int) * (n + 1));
    s[0] = 0;
    for (int i = 0; i < n; i++) {
        s[i + 1] = s[i] + nums[i];
    }
}
```

```
NumArray* obj = malloc(sizeof(NumArray));
obj->s = s;
return obj;
}
```

```
int numArraySumRange(NumArray* obj, int left, int right) {
    return obj->s[right + 1] - obj->s[left];
}
```

```
void numArrayFree(NumArray* obj) {
```

```

    free(obj->s);
    free(obj);
}

/**
 * Your NumArray struct will be instantiated and called as such:
 * NumArray* obj = numArrayCreate(nums, numsSize);
 * int param_1 = numArraySumRange(obj, left, right);
 *
 * numArrayFree(obj);
 */

```

• SimplyLeet

```

class NumArray:
    def __init__(self, nums: list[int]):
        # Create prefix sum array with an extra 0 at the beginning.
        self.prefix = [0] * (len(nums) + 1)
        for i in range(len(nums)):
            # Each element at prefix[i+1] is the sum of nums[0] to nums[i].
            self.prefix[i + 1] = self.prefix[i] + nums[i]

    def sumRange(self, left: int, right: int) -> int:
        # Return sum of elements from index left to right inclusive.

        return self.prefix[right + 1] - self.prefix[left]

# Example Usage:
# numArray = NumArray([-2, 0, 3, -5, 2, -1])
# print(numArray.sumRange(0, 2)) # Output: 1
# print(numArray.sumRange(2, 5)) # Output: -1
# print(numArray.sumRange(0, 5)) # Output: -3

```

◆ Quick Review

Key Insights

- Precompute a prefix sum array during initialization.

- The sum for any given range can be computed in $O(1)$ by subtracting the prefix sum at the starting index from the prefix sum at the ending index + 1.
- This approach makes use of extra $O(n)$ space but provides fast queries.

Space & Time

Time Complexity: $O(n)$ for preprocessing and $O(1)$ per query.

Space Complexity: $O(n)$ for storing the prefix sum array.

Solution

We build a prefix sum array such that each element at index $i+1$ represents the sum of the first i elements. For any query (left, right), the sum of elements between left and right is given by $\text{prefix}[\text{right}+1] - \text{prefix}[\text{left}]$. This approach leverages preprocessing to optimize query performance.

Prefix Sum

□ #724 Find Pivot Index

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Prefix Sum
Lists: AlgoMaster 600**

Problem

Given an array of integers `nums`, calculate the pivot index of this array.

The pivot index is the index where the sum of all the numbers strictly to the left of the index is equal to the sum of all the numbers strictly to the index's right.

If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array.

Return the leftmost pivot index. If no such index exists, return `-1`.

Example 1:


```
Input: nums = [1,7,3,6,5,6]
Output: 3
Explanation:
The pivot index is 3.
Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11
Right sum = nums[4] + nums[5] = 5 + 6 = 11
```

Example 2:

```
Input: nums = [1,2,3]
Output: -1
Explanation:
There is no index that satisfies the conditions in the problem statement.
```

Example 3:

```
Input: nums = [2,1,-1]
Output: 0
Explanation:
The pivot index is 0.
Left sum = 0 (no elements to the left of index 0)
Right sum = nums[1] + nums[2] = 1 + -1 = 0
```

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-1000 \leq \text{nums}[i] \leq 1000$

Note: This question is the same as 1991: <https://leetcode.com/problems/find-the-middle-index-in-array/>

Community Solutions (Python)

- WalkCC

```
class Solution:
    def pivotIndex(self, nums: list[int]) -> int:
        summ = sum(nums)
        prefix = 0

        for i, num in enumerate(nums):
            if prefix == summ - prefix - num:
                return i
            prefix += num

        return -1
```

• LeetCode

```
class Solution:
    def pivotIndex(self, nums: List[int]) -> int:
        left, right = 0, sum(nums)
        for i, x in enumerate(nums):
            right -= x
            if left == right:
                return i
            left += x
        return -1
```

• SimplyLeet

```
# Python solution with line-by-line comments

def pivotIndex(nums):
    # Calculate the total sum of the array
    total_sum = sum(nums)

    # Initialize left sum to 0
    left_sum = 0

    # Iterate over each index and element in the array
```

```
for i, num in enumerate(nums):
    # Right sum is total sum minus left sum minus the current element
    right_sum = total_sum - left_sum - num

    # Check if left sum equals right sum
    if left_sum == right_sum:
        return i # Return the pivot index

    # Update left sum by adding the current element
    left_sum += num

# No pivot index found
return -1

# Example usage
print(pivotIndex([1,7,3,6,5,6])) # Output: 3
```

◆ Quick Review

Key Insights

- Use a prefix sum strategy by first calculating the total sum of the array.
- As you iterate over the array, maintain the sum of numbers seen so far (left sum).
- At any index, the right sum can be derived by subtracting the left sum and the current element from total sum.
- When left sum equals right sum, you've found the pivot index.
- Only one pass through the array is needed, resulting in $O(n)$ time complexity.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We start by computing the total sum of the array. Then, we use a single loop to traverse the array and maintain a running sum (left sum). For each index, the right sum is computed as total sum minus left sum minus the current element. If the left sum equals the right sum, we return the current index as the pivot index. If no index satisfies this condition, we return -1 .

Prefix Sum

□ #1480 Running Sum of 1d Array

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Prefix Sum

Lists: AlgoMaster 600

Problem

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]...nums[i])`.

Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`

Output: `[1,3,6,10]`

Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`

Output: `[1,2,3,4,5]`

Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

Input: `nums = [3,1,2,10,1]`

Output: `[3,4,6,16,17]`

Constraints:

- $1 \leq \text{nums.length} \leq 1000$

- $-10^6 \leq \text{nums}[i] \leq 10^6$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def runningSum(self, nums: List[int]) -> List[int]:
        return itertools.accumulate(nums)
```

• LeetDoocs

```
class Solution:
    def runningSum(self, nums: List[int]) -> List[int]:
        return list(accumulate(nums))
```

• SimplyLeet

```
# Function to compute running sum of a 1d array.
def running_sum(nums):
    # Initialize an empty list to store the running sum.
    result = []
    # Initialize the cumulative sum as 0.
    cumulative = 0
    # Iterate over each value in the input list.
    for num in nums:
        # Update the cumulative sum with the current number.
        cumulative += num

    # Append the updated cumulative sum to the result list.
    result.append(cumulative)
    # Return the list containing the running sums.
    return result

# Example usage:
print(running_sum([1, 2, 3, 4])) # Expected output: [1, 3, 6, 10]
```

◆ Quick Review

Key Insights

- The problem requires computing cumulative sums, also known as prefix sums.
- Iterating over the array while maintaining a running total yields the answer.
- In-place modification is possible if allowed, but creating a new array maintains immutability.
- The constraints are small enough to allow a simple $O(n)$ solution.

Space & Time

Time Complexity: $O(n)$ because we process each element once.

Space Complexity: $O(n)$ if we store the running sum in a new array; $O(1)$ extra space if modified in-place.

Solution

The solution uses a simple iterative approach. Initialize a variable to keep track of the cumulative sum. Then iterate over the input array, updating the cumulative sum with each element and storing it in the resulting array. This approach uses the array itself (or an additional output array) to build the running

sum in a single pass through the data. The algorithm leverages the concept of prefix sum which allows for an efficient calculation with minimal additional memory overhead.

Prefix Sum

□ #1854 Maximum Population Year

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Counting · Prefix Sum
Lists: AlgoMaster 600

Problem

You are given a 2D integer array `logs` where each `logs[i] = [birthi, deathi]` indicates the birth and death years of the *i*th person.

The population of some year *x* is the number of people alive during that year. The *i*th person is counted in year *x*'s population if *x* is in the inclusive range `[birthi, deathi - 1]`. Note that the person is not counted in the year that they die. Return the earliest year with the maximum population.

Example 1:

Input: `logs = [[1993,1999],[2000,2010]]`

Output: 1993

Explanation: The maximum population is 1, and 1993 is the earliest year with this population.

Example 2:

Input: logs = [[1950,1961],[1960,1971],[1970,1981]]

Output: 1960

Explanation:

The maximum population is 2, and it had happened in years 1960 and 1970.
The earlier year between them is 1960.

Constraints:

- $1 \leq \text{logs.length} \leq 100$
- $1950 \leq \text{birth}_i < \text{death}_i \leq 2050$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maximumPopulation(self, logs: list[list[int]]) -> int:
        MIN_YEAR = 1950
        MAX_YEAR = 2050
        ans = 0
        maxPopulation = 0
        runningPopulation = 0
        # population[i] := the population of year i
        population = [0] * (MAX_YEAR + 1)

        for birth, death in logs:
            population[birth] += 1
            population[death] -= 1

        for year in range(MIN_YEAR, MAX_YEAR + 1):
            runningPopulation += population[year]
            if runningPopulation > maxPopulation:
                maxPopulation = runningPopulation
                ans = year

        return ans
```

• LeetDoocs

```

class Solution:
    def maximumPopulation(self, logs: List[List[int]]) -> int:
        d = [0] * 101
        offset = 1950
        for a, b in logs:
            a, b = a - offset, b - offset
            d[a] += 1
            d[b] -= 1
        s = mx = j = 0
        for i, x in enumerate(d):
            s += x
            if mx < s:
                mx, j = s, i
        return j + offset

```

• SimplyLeet

Python solution using a difference array approach

```

def maximumPopulation(logs):
    # Define the start and end years based on problem constraints.
    start_year = 1950
    end_year = 2050
    # Create an array to hold the changes in population for each year.
    # We need an extra space to handle the last decrement.
    population_changes = [0] * (end_year + 2)

    # For each log, increment count at birth year and decrement count at death
    year.
    for birth, death in logs:
        population_changes[birth] += 1 # Person becomes alive in the birth
        year
        population_changes[death] -= 1 # Person is not alive in the death year

    max_population = 0 # To store maximum observed population
    max_year = start_year # To store the corresponding earliest year
    current_population = 0 # Running total of the population

    # Iterate from start_year to end_year to compute population count per year.

```

```
for year in range(start_year, end_year + 1):
    current_population += population_changes[year]
    # Update maximum and earliest year if a new maximum population is
    found.
    if current_population > max_population:
        max_population = current_population
        max_year = year

return max_year

# Example usage:

logs = [[1950,1961],[1960,1971],[1970,1981]]
print(maximumPopulation(logs)) # Expected output: 1960
```

◆ Quick Review

Key Insights

- Use a difference array (or prefix sum technique) to mark changes in population over the years.
- Increment the population counter at the birth year and decrement at the death year.
- Traverse through the years (from 1950 to 2050) to compute the prefix sum, which gives the population for each year.
- Track the maximum population and update the earliest year whenever a new maximum is found.

Space & Time

Time Complexity: $O(R)$ where R is the range of years (a constant range from 1950 to 2050, so effectively $O(1)$).

Space Complexity: $O(R)$ for the difference array (again constant space with respect to input size).

Solution

We use a difference array technique to track the population changes. For each log, we increase the population count at the birth year and decrease it at the death year (since the person is not alive during the death year). We then iterate through the years from 1950 to 2050, accumulating the changes to compute the population per year. By tracking the maximum population encountered and choosing the earliest year with that population, we can determine the required answer.

Monotonic Stack

Round 4 · Mid · Easy · 2 questions

Monotonic Stack

□ #496 Next Greater Element I

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Stack · Monotonic Stack
Lists: AlgoMaster 600

Problem

The next greater element of some element x in an array is the first greater element that is to the right of x in the same array.

You are given two distinct 0-indexed integer arrays `nums1` and `nums2`, where `nums1` is a subset of `nums2`.

For each $0 \leq i < \text{nums1.length}$, find the index j such that `nums1[i] == nums2[j]` and determine the next greater element of `nums2[j]` in `nums2`. If there is no next greater element, then the answer for this query is `-1`.

Return an array `ans` of length `nums1.length` such that `ans[i]` is the next greater element as described above.

Example 1:

Input: nums1 = [4,1,2], nums2 = [1,3,4,2]

Output: [-1,3,-1]

Explanation: The next greater element for each value of nums1 is as follows:

- 4 is underlined in nums2 = [1,3,4,2]. There is no next greater element, so the answer is -1.
- 1 is underlined in nums2 = [1,3,4,2]. The next greater element is 3.
- 2 is underlined in nums2 = [1,3,4,2]. There is no next greater element, so the answer is -1.

Example 2:

Input: nums1 = [2,4], nums2 = [1,2,3,4]

Output: [3,-1]

Explanation: The next greater element for each value of nums1 is as follows:

- 2 is underlined in nums2 = [1,2,3,4]. The next greater element is 3.
- 4 is underlined in nums2 = [1,2,3,4]. There is no next greater element, so the answer is -1.

Constraints:

- $1 \leq \text{nums1.length} \leq \text{nums2.length} \leq 1000$
- $0 \leq \text{nums1}[i], \text{nums2}[i] \leq 10^4$
- All integers in nums1 and nums2 are unique.
- All the integers of nums1 also appear in nums2.

Follow up: Could you find an $O(\text{nums1.length} + \text{nums2.length})$ solution?

Community Solutions (Python)

- WalkCC


```
class Solution:
    def nextGreaterElement(self, nums1: list[int], nums2: list[int]) -> list[int]:
        numToNextGreater = {}
        stack = [] # a decreasing stack

        for num in nums2:
            while stack and stack[-1] < num:
                numToNextGreater[stack.pop()] = num
            stack.append(num)

        return [numToNextGreater.get(num, -1) for num in nums1]
```

• LeetCode

```
class Solution:
    def nextGreaterElement(self, nums1: List[int], nums2: List[int]) -> List[int]:
        stk = []
        d = {}
        for x in nums2[::-1]:
            while stk and stk[-1] < x:
                stk.pop()
            if stk:
                d[x] = stk[-1]
            stk.append(x)

        return [d.get(x, -1) for x in nums1]
```

• SimplyLeet

```
# Python solution for Next Greater Element I

def nextGreaterElement(nums1, nums2):
    # Create a dictionary to hold the next greater element for each number in
    # nums2
    next_greater = {}
    # Initialize an empty stack to keep track of numbers for which we haven't
    # found the next greater element yet
    stack = []

    # Iterate over every number in nums2
    for num in nums2:
        # While the current number is greater than the last number in stack,
        # it is the next greater element for that number.
        while stack and num > stack[-1]:
            previous_num = stack.pop()
            next_greater[previous_num] = num
        # Push the current number onto the stack
        stack.append(num)

    # For numbers left in the stack, no next greater element exists so assign
    -1
    while stack:
        previous_num = stack.pop()
        next_greater[previous_num] = -1

    # Build the result array for nums1 using the hash map
    return [next_greater[x] for x in nums1]

# Example usage
nums1 = [4,1,2]
nums2 = [1,3,4,2]
print(nextGreaterElement(nums1, nums2))
```

◆ Quick Review

Key Insights

- Use a monotonic decreasing stack to precompute the next greater element for every number in `nums2`.
- Use a hash map to store the mapping from element to its next greater element.
- For each element in `nums1`, look up the next greater element from the hash map.
- This approach avoids a nested loop by processing `nums2` only once.

Space & Time

Time Complexity: $O(n)$ where n is the length of `nums2` (plus $O(m)$ for `nums1`, overall $O(n + m)$).

Space Complexity: $O(n)$ for the stack and hash map.

Solution

We iterate through `nums2` once while maintaining a stack that keeps numbers in decreasing order. When a new number is encountered, it is the next greater element for all numbers in the stack that are smaller than it. These elements are popped from the stack and the mapping is recorded in a hash map. Afterwards, for each element in `nums1`, we

**simply fetch the answer from the hash map.
This ensures an optimal $O(n + m)$ solution.**

Monotonic Stack

□ #1475 Final Prices With a Special Discount in a Shop

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: AlgoMaster 600

Problem

You are given an integer array `prices` where `prices[i]` is the price of the i th item in a shop. There is a special discount for items in the shop. If you buy the i th item, then you will receive a discount equivalent to `prices[j]` where j is the minimum index such that $j > i$ and `prices[j] ≤ prices[i]`. Otherwise, you will not receive any discount at all.

Return an integer array `answer` where `answer[i]` is the final price you will pay for the i th item of the shop, considering the special discount.

Example 1:

Input: prices = [8,4,6,2,3]

Output: [4,2,4,2,3]

Explanation:

For item 0 with price[0]=8 you will receive a discount equivalent to prices[1]=4, therefore, the final price you will pay is $8 - 4 = 4$.

For item 1 with price[1]=4 you will receive a discount equivalent to prices[3]=2, therefore, the final price you will pay is $4 - 2 = 2$.

For item 2 with price[2]=6 you will receive a discount equivalent to prices[3]=2, therefore, the final price you will pay is $6 - 2 = 4$.

For items 3 and 4 you will not receive any discount at all.

Example 2:

Input: prices = [1,2,3,4,5]

Output: [1,2,3,4,5]

Explanation: In this case, for all items, you will not receive any discount at all.

Example 3:

Input: prices = [10,1,1,6]

Output: [9,0,1,6]

Constraints:

- $1 \leq \text{prices.length} \leq 500$
- $1 \leq \text{prices}[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def finalPrices(self, prices: list[int]) -> list[int]:
        ans = prices.copy()
        stack = []

        for i, price in enumerate(prices):
            # stack[-1] := i in the problem description.
            while stack and prices[stack[-1]] >= price:
                ans[stack.pop()] -= price
            stack.append(i)
```

```
return ans
```

• LeetCode

```
class Solution:
    def finalPrices(self, prices: List[int]) -> List[int]:
        stk = []
        for i in reversed(range(len(prices))):
            x = prices[i]
            while stk and x < stk[-1]:
                stk.pop()
            if stk:
                prices[i] -= stk[-1]
            stk.append(x)

        return prices
```

• SimplyLeet

```
# Python solution using a monotonic stack.

def finalPrices(prices):
    # Stack to store the indices of prices which are waiting for a discount.
    stack = []
    # Iterate over each price by its index.
    for i, price in enumerate(prices):
        # While there are indices in stack and the current price qualifies as a
        # discount
        while stack and prices[stack[-1]] >= price:
            # Pop index from the stack; current price is the discount for that
            # price.

            index = stack.pop()
            prices[index] -= price    # Update the price at that index by
            # applying the discount.
            # Push the current index onto the stack.
            stack.append(i)
    return prices

# Example usage:
# print(finalPrices([8,4,6,2,3])) would output [4,2,4,2,3].
```

◆ Quick Review

Key Insights

- A brute-force approach would check each subsequent item for every index which can be inefficient.
- Using a monotonic stack helps in efficiently finding the next smaller or equal price for each item.
- Traverse the array once, while maintaining a stack of indices where the discount has not yet been applied.
- When a price qualifies as the discount for the items stored in the stack, update those items.

Space & Time

Time Complexity: $O(n)$ – Each element is pushed and popped from the stack at most once.

Space Complexity: $O(n)$ – In the worst-case scenario, all elements are stored in the stack.

Solution

The solution uses a monotonic stack to track the indices of prices that have not yet found

their discount. As we iterate over the prices:

- Check if the current price can act as a discount for any previously seen elements (stored in the stack) by comparing it with the price at the index on the top of the stack.**
- If the current price is less than or equal to the price at the top of the stack, it qualifies as a discount; subtract it from that price and pop that index from the stack.**
- Push the current index onto the stack. Once the iteration is complete, any index left in the stack did not encounter a valid discount and remains at its original price.**

Queues

Round 4 · Mid · Easy · 2 questions

Queues

□ #933 Number of Recent Calls

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Design · Queue · Data Stream
Lists: AlgoMaster 600

Problem

You have a `RecentCounter` class which counts the number of recent requests within a certain time frame.

Implement the `RecentCounter` class:

- `RecentCounter()` Initializes the counter with zero recent requests.
- `int ping(int t)` Adds a new request at time `t`, where `t` represents some time in milliseconds, and returns the number of requests that has happened in the past 3000 milliseconds (including the new request). Specifically, return the number of requests that have happened in the inclusive range `[t - 3000, t]`.

It is guaranteed that every call to `ping` uses a strictly larger value of `t` than the previous call.

Example 1:

Input

```
["RecentCounter", "ping", "ping", "ping", "ping"]
```

```
[[], [1], [100], [3001], [3002]]
```

Output

```
[null, 1, 2, 3, 3]
```

Explanation

```
RecentCounter recentCounter = new RecentCounter();
```

Constraints:

- $1 \leq t \leq 109$
- Each test case will call ping with strictly increasing values of t.
- At most 104 calls will be made to ping.

Community Solutions (Python)

• WalkCC

```
class RecentCounter:
    def __init__(self):
        self.q = collections.deque()

    def ping(self, t: int) -> int:
        self.q.append(t)
        while self.q[0] < t - 3000:
            self.q.popleft()
        return len(self.q)
```

• LeetDoocs

```
class RecentCounter:
    def __init__(self):
        self.q = deque()

    def ping(self, t: int) -> int:
        self.q.append(t)
        while self.q[0] < t - 3000:
            self.q.popleft()
        return len(self.q)
```

```
# Your RecentCounter object will be instantiated and called as such:
# obj = RecentCounter()
# param_1 = obj.ping(t)
```

```
class RecentCounter:
    def __init__(self):
        self.s = []

    def ping(self, t: int) -> int:
        self.s.append(t)
        return len(self.s) - bisect_left(self.s, t - 3000)
```

```
# Your RecentCounter object will be instantiated and called as such:
```

```
# obj = RecentCounter()
# param_1 = obj.ping(t)
```

• SimplyLeet

```
# Define the RecentCounter class
class RecentCounter:
    def __init__(self):
        # Initialize a list to act as the queue for storing timestamps
        self.queue = []

    def ping(self, t: int) -> int:
        # Append the new timestamp to the queue
        self.queue.append(t)
        # Remove timestamps from the queue that are older than (t - 3000)
```

```
while self.queue[0] < t - 3000:
    self.queue.pop(0) # Remove the oldest timestamp
# Return the count of timestamps in the current valid window
return len(self.queue)

# Example usage:
# recentCounter = RecentCounter()
# print(recentCounter.ping(1)) # returns 1
# print(recentCounter.ping(100)) # returns 2
# print(recentCounter.ping(3001)) # returns 3

# print(recentCounter.ping(3002)) # returns 3
```

◆ Quick Review

Key Insights

- Use a queue (or deque) to store the timestamps of the recent pings.
- Since the ping times are guaranteed to be strictly increasing, the oldest time is always at the front of the queue.
- For each new ping, remove elements from the front of the queue that fall outside the desired time window.
- The remaining elements in the queue represent the pings in the time range $[t - 3000, t]$.

Space & Time

Time Complexity: $O(1)$ amortized per ping, since each timestamp is added and removed at most once.

Space Complexity: $O(n)$, where n is the number of pings stored (at most the number of pings within any 3000 ms window).

Solution

The solution uses a queue data structure to efficiently track and manage the timestamps of the ping requests. When a ping is made:

- Add the new timestamp to the queue.
- Remove all timestamps from the front of the queue that are less than $t - 3000$.
- The size of the queue then gives the count of recent pings.

This approach leverages the fact that the incoming timestamps are strictly increasing. The removal process ensures that no outdated ping remains in the queue, keeping the data structure clean and efficient.

Queues

□ #2073 Time Needed to Buy Tickets

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Queue · Simulation
Lists: AlgoMaster 600**

Problem

There are n people in a line queuing to buy tickets, where the 0th person is at the front of the line and the $(n - 1)$ th person is at the back of the line.

You are given a 0-indexed integer array `tickets` of length n where the number of tickets that the i th person would like to buy is `tickets[i]`.

Each person takes exactly 1 second to buy a ticket. A person can only buy 1 ticket at a time and has to go back to the end of the line (which happens instantaneously) in order to buy more tickets. If a person does not have any tickets left to buy, the person will leave the line.

Return the time taken for the person initially at position k (0-indexed) to finish buying tickets.

Example 1:

Input: tickets = [2,3,2], k = 2

Output: 6

Explanation:

- The queue starts as [2,3,2], where the kth person is underlined.
- After the person at the front has bought a ticket, the queue becomes [3,2,1] at 1 second.
- Continuing this process, the queue becomes [2,1,2] at 2 seconds.
- Continuing this process, the queue becomes [1,2,1] at 3 seconds.
- Continuing this process, the queue becomes [2,1] at 4 seconds. Note: the person at the front left the queue.
- Continuing this process, the queue becomes [1,1] at 5 seconds.
- Continuing this process, the queue becomes [1] at 6 seconds. The kth person has bought all their tickets, so return 6.

Example 2:

Input: tickets = [5,1,1,1], k = 0

Output: 8

Explanation:

- The queue starts as [5,1,1,1], where the k th person is underlined.
- After the person at the front has bought a ticket, the queue becomes [1,1,1,4] at 1 second.
- Continuing this process for 3 seconds, the queue becomes [4] at 4 seconds.
- Continuing this process for 4 seconds, the queue becomes [] at 8 seconds. The k th person has bought all their tickets, so return 8.

Constraints:

- $n == \text{tickets.length}$
- $1 \leq n \leq 100$
- $1 \leq \text{tickets}[i] \leq 100$
- $0 \leq k < n$

Community Solutions (Python)

- ☐
- WalkCC

```
class Solution:
    def timeRequiredToBuy(self, tickets: list[int], k: int) -> int:
        ans = 0

        for i, ticket in enumerate(tickets):
            if i <= k:
                ans += min(ticket, tickets[k])
            else:
                ans += min(ticket, tickets[k] - 1)

        return ans
```

• LeetDoocs

```
class Solution:
    def timeRequiredToBuy(self, tickets: List[int], k: int) -> int:
        ans = 0
        for i, x in enumerate(tickets):
            ans += min(x, tickets[k] if i <= k else tickets[k] - 1)
        return ans
```

• SimplyLeet

```
# Function to calculate the time needed for kth person to finish buying tickets.
def time_required_to_buy(tickets, k):
    kth_tickets = tickets[k]
    time = 0
    # Loop over each person in the queue.
    for i, ticket in enumerate(tickets):
        # If the person is after kth, they only get up to kth_tickets - 1
        turns.
        if i > k:
            time += min(ticket, kth_tickets - 1)
        else:
            time += min(ticket, kth_tickets)
    return time

# Example usage:
print(time_required_to_buy([2,3,2], 2)) # Expected output: 6
```

◆ Quick Review

Key Insights

- The problem simulates a round-robin process where every person buys one ticket per turn.
- Instead of simulating the entire queue operation, it is possible to compute the time using counts of rounds each person goes through.
- For each person, compare the number of tickets they need with the tickets needed by the k th person to determine if they buy a ticket in the final round.
- The k th person might only receive as many turns as $\text{tickets}[k]$ while others ahead or after might receive an extra turn if they still have tickets left when k th person finishes in the round.

Space & Time

Time Complexity: $O(n)$ where n is the number of people in the queue.

Space Complexity: $O(1)$ as only constant extra space is used.

Solution

The idea is to count how many seconds (or turns) each person in the queue spends buying tickets. For every person i in the queue, the number of turns they get to buy a ticket is the minimum between their required tickets and $\text{tickets}[k]$ (the k th person's tickets). However, for person i where the index is less than or equal to k , if they require exactly $\text{tickets}[k]$ tickets, they get to buy a ticket in the final round; otherwise, if i is after k , they only get turns up to $\text{tickets}[k] - 1$ since the k th person finishes during the k th round.

This insight leads to a formula. The total time is the sum over all i of: $\min(\text{tickets}[i], \text{tickets}[k] - (i > k ? 1 : 0))$

Using this, we avoid explicit simulation and compute the answer directly.

Divide and Conquer

Round 4 · Mid · Easy · 1 question

Divide and Conquer

□ #1763 Longest Nice Substring

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Divide and Conquer · Bit
Manipulation · Sliding Window
Lists: AlgoMaster 600

Problem

A string s is nice if, for every letter of the alphabet that s contains, it appears both in uppercase and lowercase. For example, "abABB" is nice because 'A' and 'a' appear, and 'B' and 'b' appear. However, "abA" is not because 'b' appears, but 'B' does not.

Given a string s , return the longest substring of s that is nice. If there are multiple, return the substring of the earliest occurrence. If there are none, return an empty string.

Example 1:

Input: $s = \text{"YazaAay"}$

Output: "aAa"

Explanation: "aAa" is a nice string because 'A/a' is the only letter of the alphabet in s , and both 'A' and 'a' appear.

"aAa" is the longest nice substring.

Example 2:

Input: s = "Bb"

Output: "Bb"

Explanation: "Bb" is a nice string because both 'B' and 'b' appear. The whole string is a substring.

Example 3:

Input: s = "c"

Output: ""

Explanation: There are no nice substrings.

Constraints:

- $1 \leq s.length \leq 100$
- s consists of uppercase and lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestNiceSubstring(self, s: str) -> str:
        if len(s) < 2:
            return ''

        seen = set(s)

        for i, c in enumerate(s):
            # If both upper and lower case letters exists in the string, keep moving,
            # else take the erroneous character as a partition and check for its left

            # and right parts to be nice strings.
            if c.swapcase() not in seen:
                prefix = self.longestNiceSubstring(s[:i])
                suffix = self.longestNiceSubstring(s[i + 1:])
                return max(prefix, suffix, key=len)

        return s
```


• LeetCode

```
class Solution:
    def longestNiceSubstring(self, s: str) -> str:
        n = len(s)
        ans = ''
        for i in range(n):
            ss = set()
            for j in range(i, n):
                ss.add(s[j])
                if (
                    all(c.lower() in ss and c.upper() in ss for c in ss)
                    and len(ans) < j - i + 1
                ):
                    ans = s[i : j + 1]
        return ans
```

• SimplyLeet

```
# Python solution for Longest Nice Substring
def longestNiceSubstring(s: str) -> str:
    # Helper function to check if a substring is nice
    def is_nice(sub: str) -> bool:
        # Create a set of characters in the substring
        char_set = set(sub)
        # For each character, check if its opposite case exists
        for ch in char_set:
            if ch.swapcase() not in char_set:
                return False
        return True

    # Recursive function using divide and conquer approach
    def helper(sub: str) -> str:
        # Base case: if the substring is too short, it cannot be nice
        if len(sub) < 2:
            return ""
        # For each character, if it's offending (not forming a nice pair),
        # split and solve recursively
        for i, ch in enumerate(sub):
            if ch.swapcase() not in sub:
                return helper(sub[:i]) + helper(sub[i+1:])
        return sub
```

```
# Recurse on the left part and right part
left = helper(sub[:i])
right = helper(sub[i+1:])
# Return the longer substring; if same length, left portion
appears earlier
return left if len(left) >= len(right) else right
# If the loop finishes, the substring is nice
return sub

return helper(s)
```

```
# Example usage:
print(longestNiceSubstring("YazaAay")) # Expected output: "aAa"
```

◆ Quick Review

Key Insights

- A substring is considered nice if for every character, both its uppercase and lowercase forms are present.
- To check if a substring is nice, using a set to quickly verify the presence of both cases is effective.
- Due to the small constraint ($s.length \leq 100$), an $O(n^3)$ brute-force solution is acceptable, but an efficient divide-and-conquer method can also be applied.
- The divide-and-conquer technique works by identifying an offending character (one

missing its counterpart) and then recursively solving for the segments on both sides.

- Keeping track of the earliest occurrence for substrings of the same length is necessary when multiple valid candidates exist.

Space & Time

Time Complexity: $O(n^2)$ to $O(n^3)$ in the worst-case scenario if using brute force; however, the divide and conquer approach typically performs faster in practice.

Space Complexity: $O(n)$ due to recursion and auxiliary data structures (sets).

Solution

We can solve the problem using a recursive divide-and-conquer approach:

- Check if the current string is nice. We do this by creating a set of characters in the string and confirming for each character that its opposite case also exists.
- If the string is nice, return it as a candidate.
- If not, find the first offending character that violates the niceness condition.

- Recursively call the function for the two substrings split at the offending character, and return the longer of the two valid nice substrings.
- This approach ensures that we consider every possibility while naturally skipping segments that cannot form a nice substring.

BST / Ordered Set

Round 4 · Mid · Easy · 3 questions

BST / Ordered Set

□ #653 Two Sum IV – Input is a BST

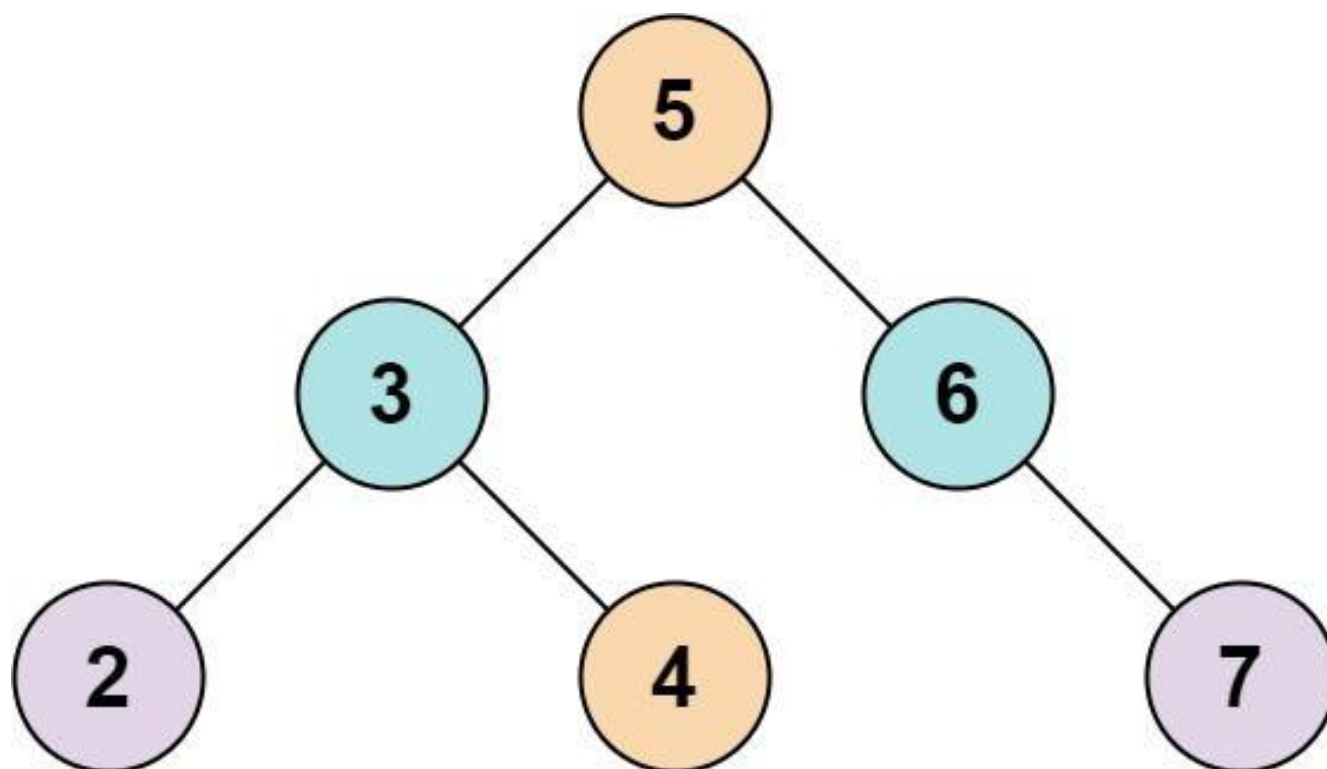
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Two Pointers · Tree · Depth-First
Search · Breadth-First Search · Binary Search
Tree · Binary Tree
Lists: AlgoMaster 600

Problem

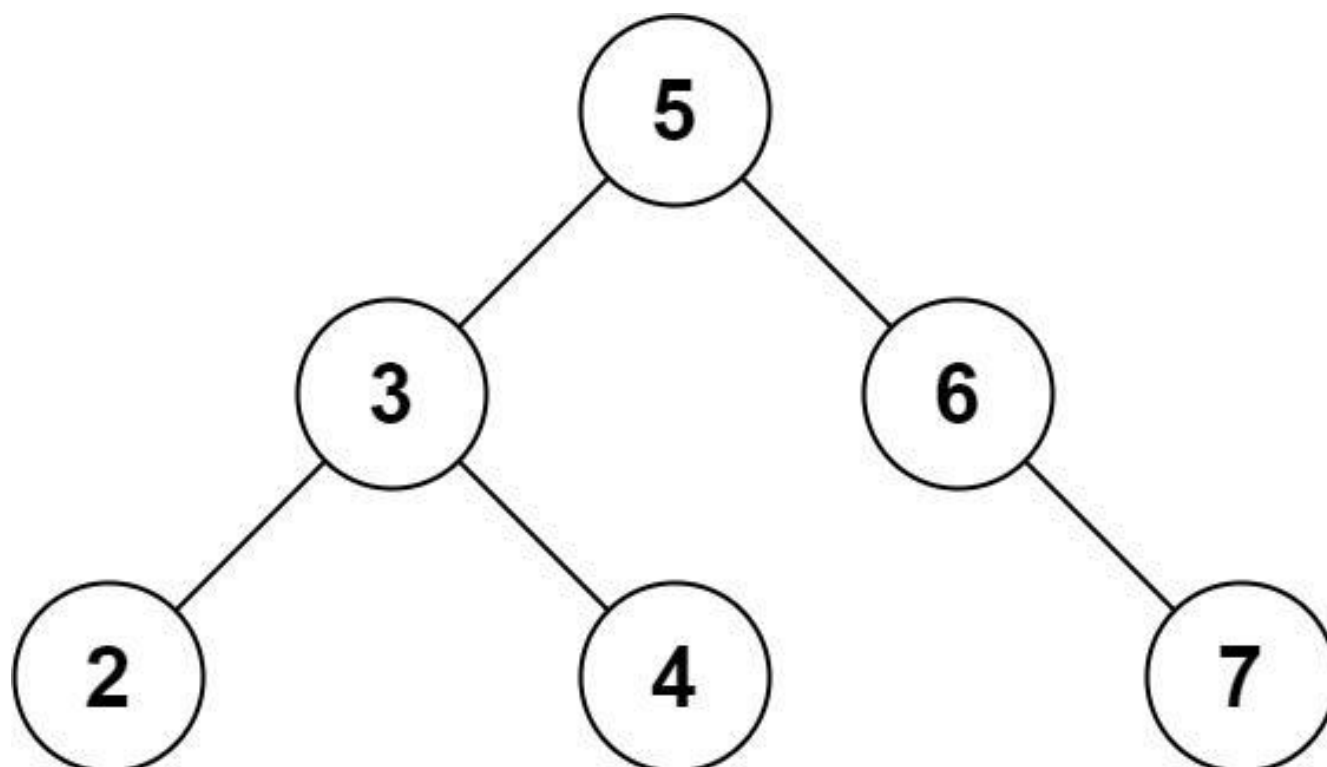
Given the root of a binary search tree and an integer k , return true if there exist two elements in the BST such that their sum is equal to k , or false otherwise.

Example 1:



Input: root = [5,3,6,2,4,null,7], k = 9
Output: true

Example 2:



Input: root = [5,3,6,2,4,null,7], k = 28
 Output: false

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-104 \leq \text{Node.val} \leq 104$
- root is guaranteed to be a valid binary search tree.
- $-105 \leq k \leq 105$

Community Solutions (Python)

• WalkCC

```
class BSTIterator:
    def __init__(self, root: TreeNode | None, leftToRight: bool):
        self.stack = []
        self.leftToRight = leftToRight
        self._pushUntilNone(root)

    def next(self) -> int:
        node = self.stack.pop()
        if self.leftToRight:
            self._pushUntilNone(node.right)

        else:
            self._pushUntilNone(node.left)
        return node.val

    def _pushUntilNone(self, root: TreeNode | None):
        while root:
            self.stack.append(root)
            root = root.left if self.leftToRight else root.right
```



```

class Solution:
    def findTarget(self, root: TreeNode | None, k: int) -> bool:
        if not root:
            return False

        left = BSTIterator(root, True)
        right = BSTIterator(root, False)

        l = left.next()
        r = right.next()

        while l < r:
            summ = l + r
            if summ == k:
                return True
            if summ < k:
                l = left.next()
            else:
                r = right.next()

        return False

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findTarget(self, root: Optional[TreeNode], k: int) -> bool:
        def dfs(root):
            if root is None:
                return False
            if k - root.val in vis:
                return True
            vis.add(root.val)
            return dfs(root.left) or dfs(root.right)

        vis = set()
        return dfs(root)

```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def findTarget(self, root: TreeNode, k: int) -> bool:
        seen = set() # Set to store the values of visited nodes

        def dfs(node):
            if not node: # Base case: if the node is null, return False
                return False
            # Check if the complement of the current node's value exists in the set
            if (k - node.val) in seen:
                return True
            # Add the current node's value to the set
            seen.add(node.val)
            # Continue DFS traversal on left and right children

            return dfs(node.left) or dfs(node.right)

        return dfs(root) # Start DFS from the root node
```

◆ Quick Review

Key Insights

- Utilize the BST property to traverse the tree while checking for the complementary value ($k - \text{current node's value}$).
- A hash set can be used to store seen node values which facilitates constant time lookups.

- Alternatively, an in-order traversal converts the BST into a sorted array, allowing a two-pointer approach.
- The recursive DFS solution is both intuitive and easy to implement for this problem.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes, since each node is visited once.

Space Complexity: $O(n)$ in the worst case due to the extra space used by the hash set (or recursion stack in the worst-case scenario of an unbalanced tree).

Solution

The solution uses a depth-first search (DFS) traversal while maintaining a hash set to remember the values of visited nodes. For each node, check if k minus the node's value exists in the hash set. If it does, return true. Otherwise, add the node's value to the set and continue the DFS on both the left and right children. This approach efficiently finds a pair of nodes that sum up to k without needing to traverse the tree more than once.

BST / Ordered Set

□ #700 Search in a Binary Search Tree

EAS

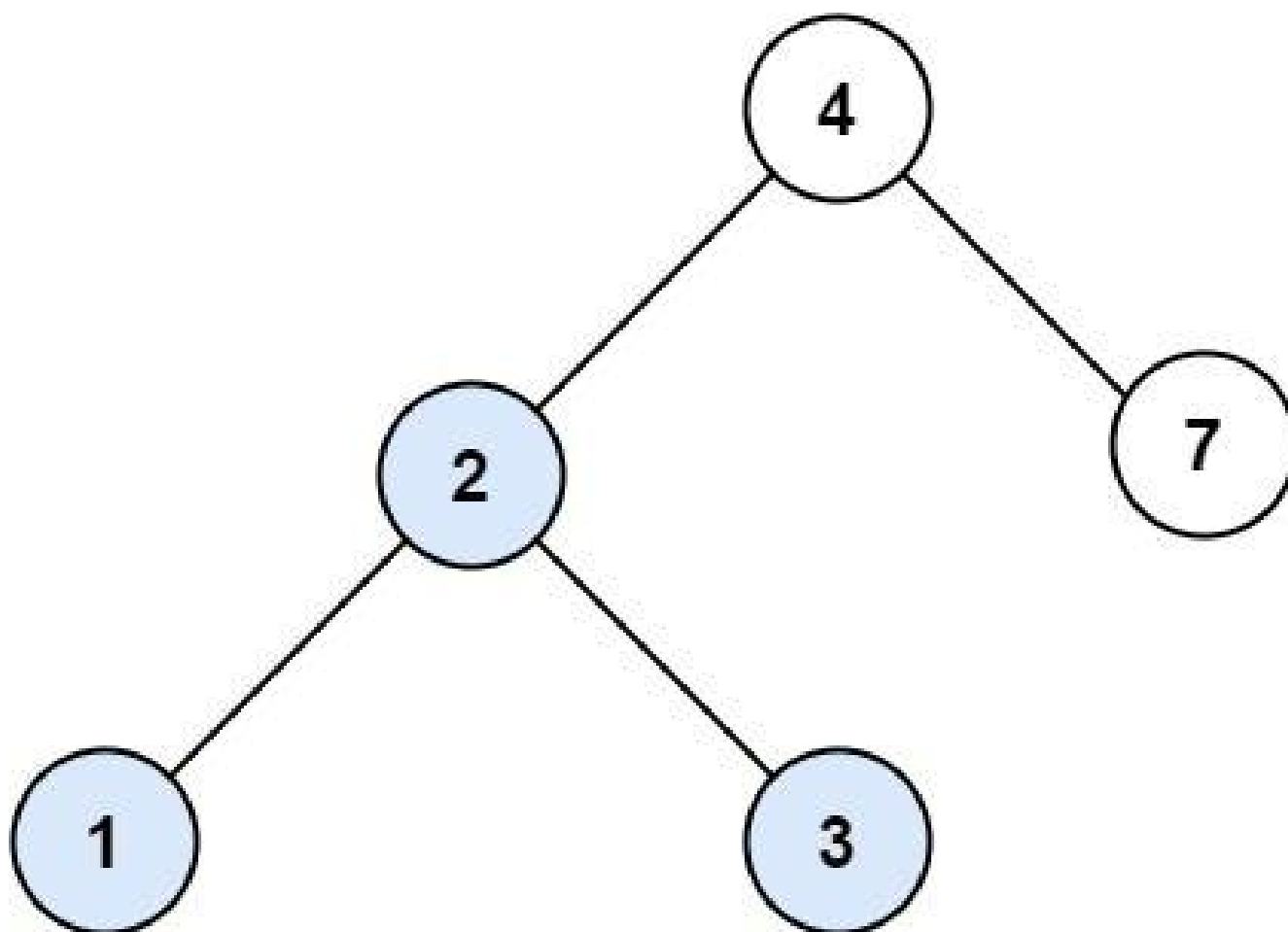
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Binary Search Tree · Binary Tree
Lists: AlgoMaster 600

Problem

You are given the root of a binary search tree (BST) and an integer val.

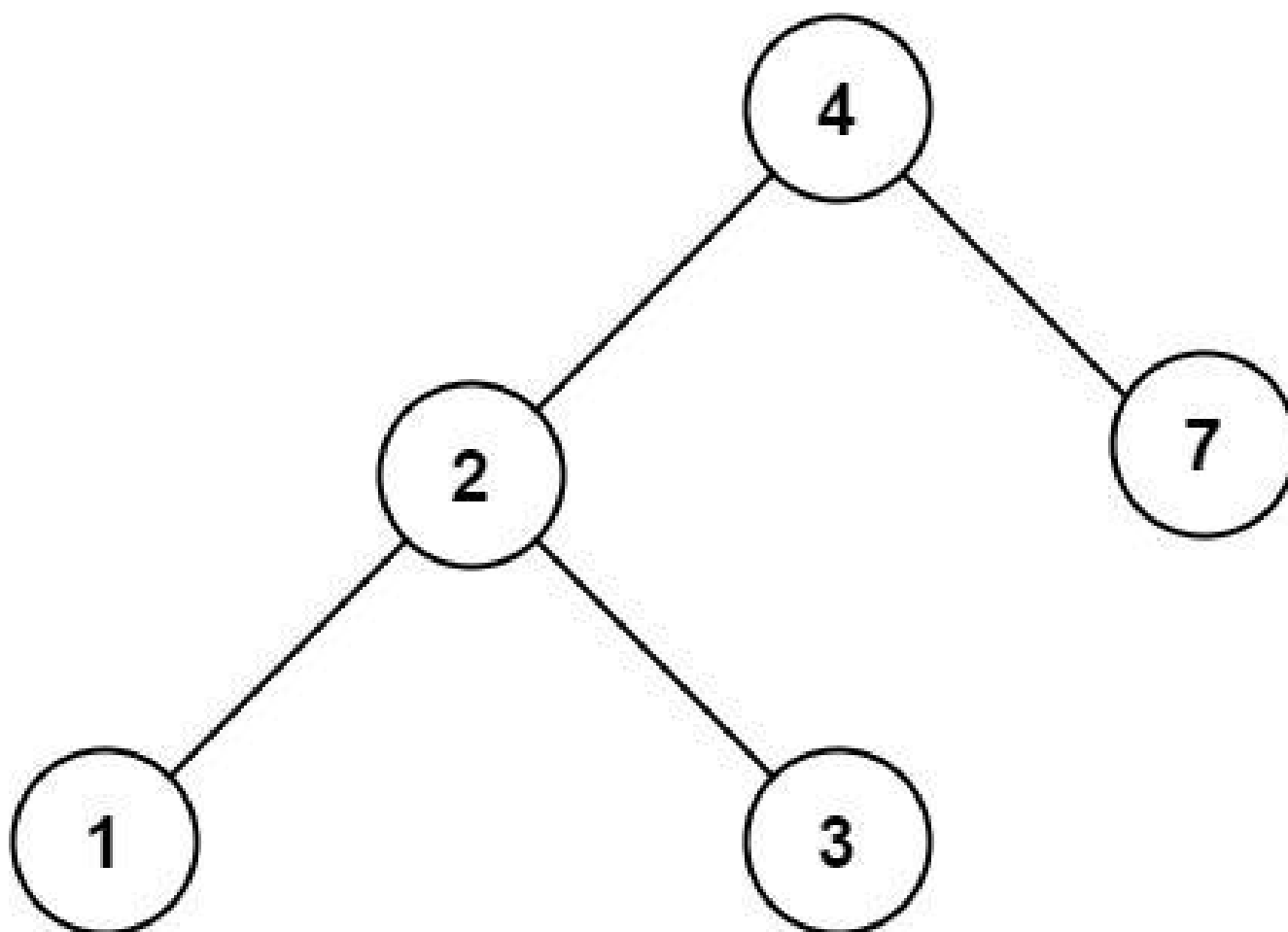
Find the node in the BST that the node's value equals val and return the subtree rooted with that node. If such a node does not exist, return null.

Example 1:



Input: root = [4,2,7,1,3], val = 2
Output: [2,1,3]

Example 2:



Input: root = [4,2,7,1,3], val = 5
Output: []

Constraints:

- The number of nodes in the tree is in the range [1, 5000].
- $1 \leq \text{Node.val} \leq 10^7$
- root is a binary search tree.
- $1 \leq \text{val} \leq 10^7$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def searchBST(self, root: TreeNode | None, val: int) -> TreeNode | None:
        if not root:
            return None
        if root.val == val:
            return root
        if root.val > val:
            return self.searchBST(root.left, val)
        return self.searchBST(root.right, val)
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def searchBST(self, root: Optional[TreeNode], val: int) -> Optional[TreeNode]:
        if root is None or root.val == val:
            return root

        return (
            self.searchBST(root.left, val)
            if root.val > val
            else self.searchBST(root.right, val)
        )
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def searchBST(root: TreeNode, val: int) -> TreeNode:
    # Base case: if root is null or found the target value, return the root
    if not root or root.val == val:
```

```
        return root
    # If target value is less than current node's value, search in the left
    subtree
    if val < root.val:
        return searchBST(root.left, val)
    # Otherwise, search in the right subtree
    return searchBST(root.right, val)
```

◆ Quick Review

Key Insights

- Utilize the BST property: for any node, all values in the left subtree are smaller and all values in the right subtree are larger.
- This property allows efficient searching by choosing the correct subtree to continue the search.
- The problem can be recursively or iteratively solved.
- Recursion typically leads to a cleaner, more intuitive implementation.

Space & Time

Time Complexity: $O(h)$, where h is the height of the tree.

Space Complexity: $O(h)$ due to recursion stack in the recursive approach, or $O(1)$ if an iterative approach is used.

Solution

We traverse the BST starting at the root. For each node, we compare the target value with the current node's value:

- If the values match, we return the current node.
- If the target value is less, we recursively (or iteratively) search the left subtree.
- If the target value is greater, we proceed to search the right subtree. By making use of the BST properties, we avoid checking every node, which makes the approach efficient.

BST / Ordered Set

□ #938 Range Sum of BST

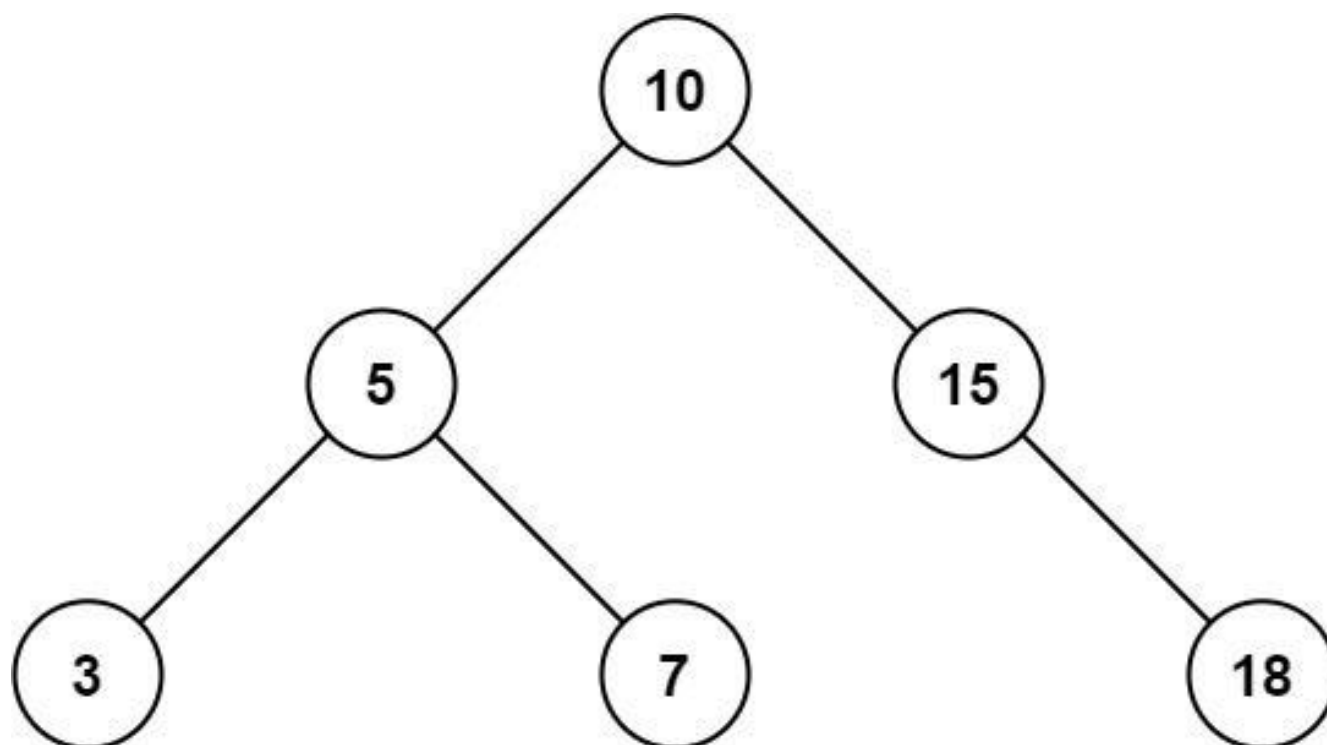
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Binary Search Tree ·
Binary Tree
Lists: AlgoMaster 600

Problem

Given the root node of a binary search tree and two integers low and high, return the sum of values of all nodes with a value in the inclusive range [low, high].

Example 1:

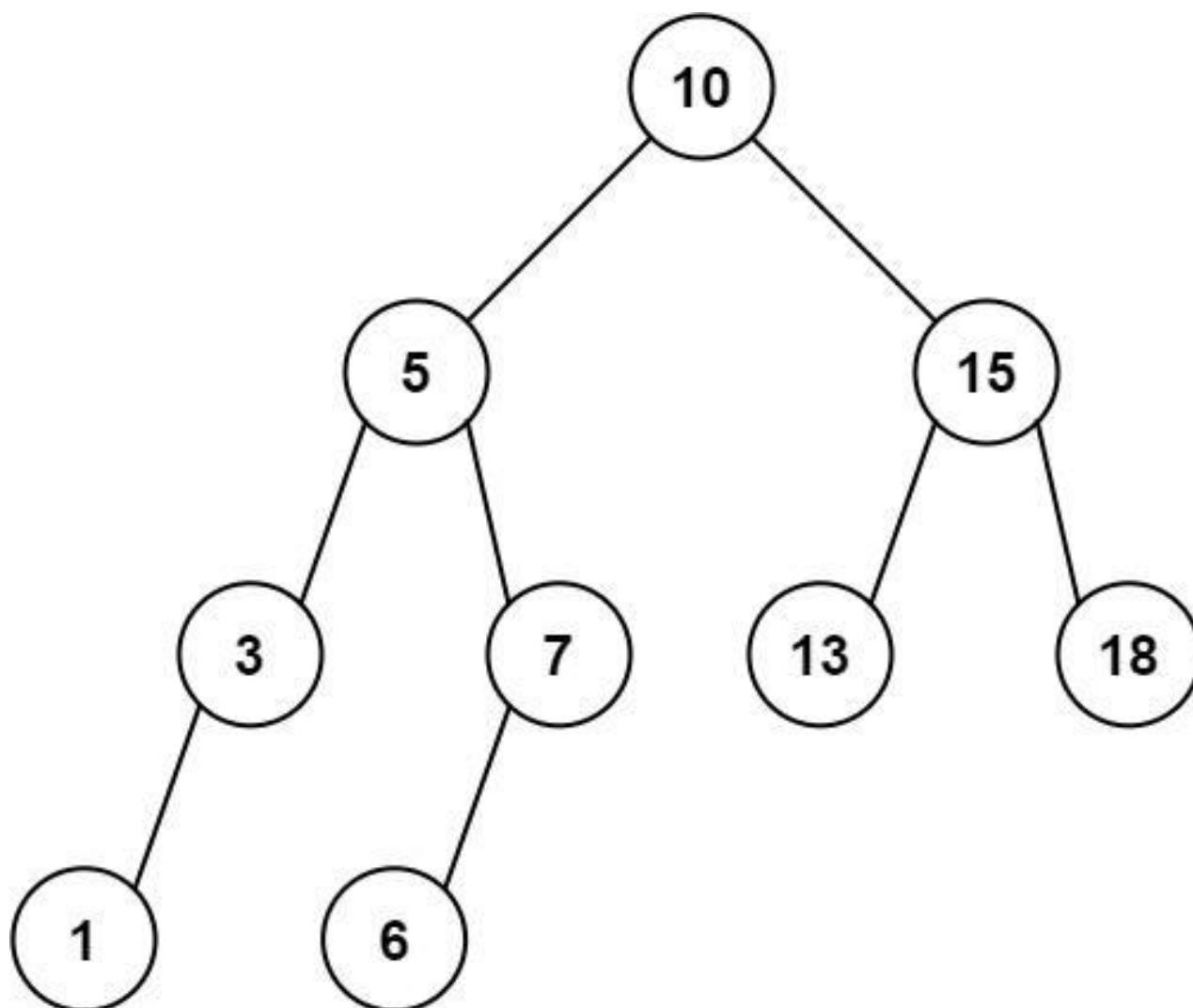


Input: root = [10,5,15,3,7,null,18], low = 7, high = 15

Output: 32

Explanation: Nodes 7, 10, and 15 are in the range [7, 15]. $7 + 10 + 15 = 32$.

Example 2:



Input: root = [10,5,15,3,7,13,18,1,null,6], low = 6, high = 10

Output: 23

Explanation: Nodes 6, 7, and 10 are in the range [6, 10]. $6 + 7 + 10 = 23$.

Constraints:

- The number of nodes in the tree is in the range $[1, 2 * 10^4]$.
- $1 \leq \text{Node.val} \leq 105$
- $1 \leq \text{low} \leq \text{high} \leq 105$
- All Node.val are unique.

Community Solutions (Python)

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rangeSumBST(self, root: Optional[TreeNode], low: int, high: int) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:
                return 0
            x = root.val
            ans = x if low <= x <= high else 0
            if x > low:
                ans += dfs(root.left)
            if x < high:
                ans += dfs(root.right)
            return ans
        return dfs(root)
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rangeSumBST(self, root: TreeNode, low: int, high: int) -> int:
        # Base case: if the current node is None, return sum 0.
```

```
if not root:
    return 0

# Initialize sum for the current node.
sum_val = 0

# If the current node's value is within [low, high], add its value.
if low <= root.val <= high:
    sum_val += root.val

# If the current node's value is greater than low, explore left
subtree.
if root.val > low:
    sum_val += self.rangeSumBST(root.left, low, high)

# If the current node's value is less than high, explore right subtree.
if root.val < high:
    sum_val += self.rangeSumBST(root.right, low, high)

return sum_val
```

◆ Quick Review

Key Insights

- Utilize the BST property to prune unnecessary branches (i.e., if a node's value is less than low, then its left subtree can be skipped; if greater than high, its right subtree can be skipped).
- A Depth-First Search (DFS) traversal (recursive or iterative) is appropriate to visit only the relevant nodes.

- Keep a running sum as you traverse nodes that fall within the range [low, high].

Space & Time

Time Complexity: $O(n)$ in the worst-case scenario where every node is visited.

Space Complexity: $O(n)$ in the worst-case due to recursive call stack or if an iterative approach uses a stack, especially in the case of a skewed tree.

Solution

We solve the problem using DFS traversal (recursion) leveraging the BST properties. At each node:

- If the node is null, return 0.
 - If the node's value is within the range [low, high], add it to the sum.
 - If the node's value is greater than low, explore the left subtree because it might contain nodes in range.
 - If the node's value is less than high, explore the right subtree for potential nodes in range.
- This algorithm reduces unnecessary traversals by pruning off subtrees that cannot contribute to the result.

Intervals

Round 4 · Mid · Easy · 1 question

Intervals

□ #228 Summary Ranges

EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array**

Lists: AlgoMaster 600

Problem

**You are given a sorted unique integer array
nums.**

**A range $[a,b]$ is the set of all integers from a to b
(inclusive).**

**Return the smallest sorted list of ranges that
cover all the numbers in the array exactly. That
is, each element of nums is covered by exactly
one of the ranges, and there is no integer x such
that x is in one of the ranges but not in nums.**

Each range $[a,b]$ in the list should be output as:

- **"a->b" if $a \neq b$**
- **"a" if $a == b$**

Example 1:

```
Input: nums = [0,1,2,4,5,7]
Output: ["0->2","4->5","7"]
Explanation: The ranges are:
[0,2] --> "0->2"
[4,5] --> "4->5"
[7,7] --> "7"
```

Example 2:

```
Input: nums = [0,2,3,4,6,8,9]
Output: ["0","2->4","6","8->9"]
Explanation: The ranges are:
[0,0] --> "0"
[2,4] --> "2->4"
[6,6] --> "6"
[8,9] --> "8->9"
```

Constraints:

- $0 \leq \text{nums.length} \leq 20$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$
- All the values of `nums` are unique.
- `nums` is sorted in ascending order.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def summaryRanges(self, nums: list[int]) -> list[str]:
        ans = []

        i = 0
        while i < len(nums):
            begin = nums[i]
            while i < len(nums) - 1 and nums[i] == nums[i + 1] - 1:
                i += 1
            end = nums[i]
```

```

    if begin == end:
        ans.append(str(begin))
    else:
        ans.append(str(begin) + "->" + str(end))
    i += 1

return ans

```

• LeetDoocs

```

class Solution:
    def summaryRanges(self, nums: List[int]) -> List[str]:
        def f(i: int, j: int) -> str:
            return str(nums[i]) if i == j else f'{nums[i]}->{nums[j]}'

        i = 0
        n = len(nums)
        ans = []
        while i < n:
            j = i

            while j + 1 < n and nums[j + 1] == nums[j] + 1:
                j += 1
            ans.append(f(i, j))
            i = j + 1
        return ans

```

• SimplyLeet

```

# Function to summarize ranges in a sorted unique integer array.
def summary_ranges(nums):
    ranges = [] # List to store the resulting range strings
    n = len(nums)
    i = 0 # Pointer to iterate through nums

    # Iterate over the numbers in the array
    while i < n:
        start = nums[i] # Start of a new range
        # Traverse the consecutive sequence

```

```
while i + 1 < n and nums[i + 1] == nums[i] + 1:
    i += 1
end = nums[i] # End of the current range
# If start and end are the same, it's a single element range
if start == end:
    ranges.append(str(start))
else:
    ranges.append(f"{start}->{end}")
i += 1 # Move to the next number after the current range
return ranges
```

```
# Example test cases
```

```
print(summary_ranges([0,1,2,4,5,7])) # Output: ["0->2", "4->5", "7"]
```

```
print(summary_ranges([0,2,3,4,6,8,9])) # Output: ["0", "2->4", "6", "8->9"]
```

◆ Quick Review

Key Insights

- Since the array is sorted and unique, we can simply iterate through it.
- Track the start of a new range, and expand the range as long as the next number is consecutive.
- When a break in consecutiveness is noticed, format the current range accordingly and start a new range.
- Finalize the last range after exiting the loop.

Space & Time

Time Complexity: $O(n)$ where n is the length of the `nums` array.

Space Complexity: $O(1)$ additional space (excluding the space for the output list).

Solution

We iterate through the array while maintaining a start pointer for the current range. For each element, check if the next element is exactly one greater. If not, the current range ends here. Format the range as "start->end" if there is more than one element in the range, otherwise just as "start". This algorithm leverages the sorted property of the array to accomplish the task in one pass using constant extra space.

Data Structure Design

Round 4 · Mid · Easy · 2 questions

Data Structure Design

□ #225 Implement Stack using Queues

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Design · Queue
Lists: AlgoMaster 600

Problem

Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (push, top, pop, and empty).

Implement the MyStack class:

- **void push(int x)** Pushes element x to the top of the stack.
- **int pop()** Removes the element on the top of the stack and returns it.
- **int top()** Returns the element on the top of the stack.
- **boolean empty()** Returns true if the stack is empty, false otherwise.

Notes:

- You must use only standard operations of a queue, which means that only push to back, peek/pop from front, size and is empty operations are valid.
- Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations.

Example 1:

Input

```
["MyStack", "push", "push", "top", "pop", "empty"]
```

```
[[], [1], [2], [], [], []]
```

Output

```
[null, null, null, 2, 2, false]
```

Explanation

```
MyStack myStack = new MyStack();
```

Constraints:

- $1 \leq x \leq 9$
- At most 100 calls will be made to push, pop, top, and empty.
- All the calls to pop and top are valid.

Follow-up: Can you implement the stack using only one queue?

Community Solutions (Python)

- WalkCC


```

class MyStack:
    def __init__(self):
        self.q = collections.deque()

    def push(self, x: int) -> None:
        self.q.append(x)
        for _ in range(len(self.q) - 1):
            self.q.append(self.q.popleft())

    def pop(self) -> int:

        return self.q.popleft()

    def top(self) -> int:
        return self.q[0]

    def empty(self) -> bool:
        return not self.q

```

• LeetCode

```

class MyStack:
    def __init__(self):
        self.q1 = deque()
        self.q2 = deque()

    def push(self, x: int) -> None:
        self.q2.append(x)
        while self.q1:
            self.q2.append(self.q1.popleft())
        self.q1, self.q2 = self.q2, self.q1

    def pop(self) -> int:
        return self.q1.popleft()

    def top(self) -> int:
        return self.q1[0]

    def empty(self) -> bool:
        return len(self.q1) == 0

```

```
# Your MyStack object will be instantiated and called as such:
# obj = MyStack()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.top()
# param_4 = obj.empty()
```

• SimplyLeet

```
from collections import deque

class MyStack:
    def __init__(self):
        # Initialize two queues using deque
        self.q1 = deque()
        self.q2 = deque()

    def push(self, x: int) -> None:
        # Push x into q2 (auxiliary queue)

        self.q2.append(x)
        # Transfer all elements from q1 to q2 to reverse the order
        while self.q1:
            self.q2.append(self.q1.popleft())
        # Swap q1 and q2 so that q1 always holds the current elements
        self.q1, self.q2 = self.q2, self.q1

    def pop(self) -> int:
        # Pop element from the front of q1 which is the top of the stack
        return self.q1.popleft()

    def top(self) -> int:
        # Peek at the front of q1 to get the top element
        return self.q1[0]

    def empty(self) -> bool:
        # Return True if q1 is empty, else False
        return not self.q1
```

◆ Quick Review

Key Insights

- We simulate a stack (LIFO) behavior using two queues (FIFO).
- For the push operation, use one queue (q2) to hold the new element and then transfer all elements from the other queue (q1) to q2 so that the new element is at the front.
- Swap the roles of the two queues after each push so that q1 always holds the stack in the correct order.
- The pop and top operations can then be implemented by simply removing or peeking at the front of q1.

Space & Time

Time Complexity:

- push: $O(n)$, where n is the number of elements in the stack (due to transferring all elements between queues).
- pop: $O(1)$
- top: $O(1)$

Space Complexity: $O(n)$, as we store all elements in the two queues.

Solution

We implement the stack using two queues. During each push operation, we enqueue the new element into an empty auxiliary queue (q2) and then move all elements from the primary queue (q1) to q2. This way, the new element comes to the front of the queue, mimicking stack behavior. Finally, we swap q1 and q2 so that q1 always represents the current stack. For pop, we simply dequeue from q1; for top, we peek at the front of q1; and for empty, we check if q1 is empty.

Data Structure Design

□ #359 Logger Rate Limiter

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Design · Data Stream
Lists: AlgoMaster 600

Problem

Design a logger system that receives a stream of messages along with their timestamps. Each unique message should only be printed at most every 10 seconds (i.e. a message printed at timestamp t will prevent other identical messages from being printed until timestamp $t + 10$).

All messages will come in chronological order. Several messages may arrive at the same timestamp.

Implement the Logger class:

- **Logger()** Initializes the logger object.
- **bool shouldPrintMessage(int timestamp, string message)** Returns true if the message should be printed in the given timestamp,

otherwise returns false.

Example 1:

Input

```
["Logger", "shouldPrintMessage", "shouldPrintMessage", "shouldPrintMessage",
"shouldPrintMessage", "shouldPrintMessage", "shouldPrintMessage"]
[[], [1, "foo"], [2, "bar"], [3, "foo"], [8, "bar"], [10, "foo"], [11, "foo"]]
```

Output

```
[null, true, true, false, false, false, true]
```

Explanation

```
Logger logger = new Logger();
```

Constraints:

- $0 \leq \text{timestamp} \leq 109$
- Every timestamp will be passed in non-decreasing order (chronological order).
- $1 \leq \text{message.length} \leq 30$
- At most 104 calls will be made to `shouldPrintMessage`.

Community Solutions (Python)

• WalkCC

```
class Logger:
    def __init__(self):
        # [(timestamp, message)]
        self.messageQueue = collections.deque()
        self.messageSet = set()

    def shouldPrintMessage(self, timestamp: int, message: str) -> bool:
        # Remove the messages that are 10 seconds from the current timestamp.
        while self.messageQueue:
            headTimestamp, headMessage = self.messageQueue[0]
```

```

        if timestamp < headTimestamp + 10:
            break
        self.messageQueue.popleft()
        self.messageSet.remove(headMessage)

    if message in self.messageSet:
        return False

    self.messageQueue.append((timestamp, message))
    self.messageSet.add(message)

    return True

```

```

class Logger:
    def __init__(self):
        self.okTime = {} # {message: ok time}

    def shouldPrintMessage(self, timestamp: int, message: str) -> bool:
        if timestamp < self.okTime.get(message, 0):
            return False

        self.okTime[message] = timestamp + 10
        return True

```

• LeetDoocs

```

class Logger:

    def __init__(self):
        self.ts = {}

    def shouldPrintMessage(self, timestamp: int, message: str) -> bool:
        t = self.ts.get(message, 0)
        if t > timestamp:
            return False
        self.ts[message] = timestamp + 10

    return True

```

```

# Your Logger object will be instantiated and called as such:
# obj = Logger()
# param_1 = obj.shouldPrintMessage(timestamp,message)

```

• SimplyLeet

```
# Python code with detailed comments
```

```
class Logger:
    def __init__(self):
        # Dictionary to record the next allowed timestamp for each message.
        self.message_dict = {}

    def shouldPrintMessage(self, timestamp: int, message: str) -> bool:
        # If message not seen before or current timestamp is >= expected
        # timestamp.
        if message not in self.message_dict or timestamp >= self.message_dict[
            message]:

            # Update next allowed time to current timestamp + 10.
            self.message_dict[message] = timestamp + 10
            # Return True since message should be printed.
            return True

        # Else, the message cannot be printed yet.
        return False

# Example usage:
# logger = Logger()
# print(logger.shouldPrintMessage(1, "foo")) # True

# print(logger.shouldPrintMessage(3, "foo")) # False
```

◆ Quick Review

Key Insights

- Use a hash table (dictionary) to store the most recent timestamp when each message was printed.
- Check if the current timestamp is at least 10 seconds later than the stored timestamp before printing the message.

- The messages are in chronological order, so no need for extra data structures to handle reordering.
- The approach efficiently handles streaming input with constant time lookups.

Space & Time

Time Complexity: $O(1)$ average per message (dictionary lookup and update)

Space Complexity: $O(n)$ in the worst case, where n is the number of unique messages

Solution

Maintain a hash table where keys are messages and values are the next allowed timestamp when the message can be printed. When a new message arrives with a given timestamp, check:

- If the message is not stored in the hash table, print it and set the allowed timestamp as current timestamp + 10.
- If the current timestamp is greater than or equal to the stored allowed timestamp, update the allowed timestamp and print the message.
- Otherwise, do not print the message. This approach is efficient because it only requires

constant time operations per call and uses minimal additional space.

Greedy

Round 4 · Mid · Easy · 2 questions

Greedy

□ #455 Assign Cookies

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Greedy · Sorting
Lists: AlgoMaster 600

Problem

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor $g[i]$, which is the minimum size of a cookie that the child will be content with; and each cookie j has a size $s[j]$. If $s[j] \geq g[i]$, we can assign the cookie j to the child i , and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input: $g = [1,2,3]$, $s = [1,1]$

Output: 1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Example 2:

Input: `g = [1,2], s = [1,2,3]`

Output: 2

Explanation: You have 2 children and 3 cookies. The greed factors of 2 children are 1, 2.

You have 3 cookies and their sizes are big enough to gratify all of the children,

You need to output 2.

Constraints:

- $1 \leq g.length \leq 3 * 10^4$
- $0 \leq s.length \leq 3 * 10^4$
- $1 \leq g[i], s[j] \leq 2^{31} - 1$

Note: This question is the same as 2410: Maximum Matching of Players With Trainers.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findContentChildren(self, g: list[int], s: list[int]) -> int:
        g.sort()
        s.sort()

        i = 0
        for cookie in s:
            if i < len(g) and g[i] <= cookie:
                i += 1

        return i
```

• LeetDoocs

```

class Solution:
    def findContentChildren(self, g: List[int], s: List[int]) -> int:
        g.sort()
        s.sort()
        j = 0
        for i, x in enumerate(g):
            while j < len(s) and s[j] < g[i]:
                j += 1
            if j >= len(s):
                return i

        j += 1
        return len(g)

```

• SimplyLeet

Python solution for assigning cookies

```

def findContentChildren(g, s):
    # Sort greed factors and cookie sizes
    g.sort() # Sorting children by greed factor
    s.sort() # Sorting cookies by size

    child = 0 # Pointer for children
    cookie = 0 # Pointer for cookies

    # Iterate until we exhaust either cookies or children
    while child < len(g) and cookie < len(s):
        # If current cookie satisfies current child's greed factor
        if s[cookie] >= g[child]:
            child += 1 # Assign cookie to child, move to next child
            cookie += 1 # Move to next cookie regardless of assignment

    return child # Number of content children

```

Example usage:

```

print(findContentChildren([1,2,3], [1,1])) # Expected output: 1
print(findContentChildren([1,2], [1,2,3])) # Expected output: 2

```

◆ Quick Review

Key Insights

- Sort both the greed factor array and the cookie sizes array.
- Use a two–pointer approach to iterate through both arrays.
- If the current cookie can satisfy the current child's greed, assign it and move to the next child.
- Continue until either all children are checked or all cookies are used.

Space & Time

Time Complexity: $O(n \log n + m \log m)$ where n is the number of children and m is the number of cookies (due to sorting).

Space Complexity: $O(1)$ if sorting is done in–place, otherwise $O(n + m)$ if extra space is used to copy lists.

Solution

The approach is based on greedy allocation. By sorting the children based on their greed factors and sorting the cookies by size, we can efficiently match the smallest available cookie

that is sufficient for a given child. The two-pointer technique helps in traversing both arrays simultaneously, ensuring that we don't waste a potentially larger cookie on a child with lower greed if a smaller cookie would suffice. This strategy maximizes the number of content children.

Greedy

□ #3074 Apple Redistribution into Boxes

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Sorting
Lists: AlgoMaster 600

Problem

You are given an array `apple` of size `n` and an array `capacity` of size `m`.

There are `n` packs where the `i`th pack contains `apple[i]` apples. There are `m` boxes as well, and the `i`th box has a capacity of `capacity[i]` apples. Return the minimum number of boxes you need to select to redistribute these `n` packs of apples into boxes.

Note that, apples from the same pack can be distributed into different boxes.

Example 1:

Input: `apple = [1,3,2]`, `capacity = [4,3,1,5,2]`

Output: 2

Explanation: We will use boxes with capacities 4 and 5.

It is possible to distribute the apples as the total capacity is greater than or equal to the total number of apples.

Example 2:

Input: apple = [5,5,5], capacity = [2,4,2,7]
Output: 4
Explanation: We will need to use all the boxes.

Constraints:

- $1 \leq n \leq \text{apple.length} \leq 50$
- $1 \leq m \leq \text{capacity.length} \leq 50$
- $1 \leq \text{apple}[i], \text{capacity}[i] \leq 50$
- The input is generated such that it's possible to redistribute packs of apples into boxes.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minimumBoxes(self, apple: list[int], capacity: list[int]) -> int:
        appleSum = sum(apple)
        capacitySum = 0

        for i, c in enumerate(sorted(capacity, reverse=True)):
            capacitySum += c
            if capacitySum >= appleSum:
                return i + 1

        return len(capacity)
```

• LeetDoocs

```
class Solution:
    def minimumBoxes(self, apple: List[int], capacity: List[int]) -> int:
        capacity.sort(reverse=True)
        s = sum(apple)
        for i, c in enumerate(capacity, 1):
            s -= c
            if s <= 0:
                return i
```

• SimplyLeet

```
def min_boxes(apple, capacity):
    # Calculate total number of apples
    total_apples = sum(apple)
    # Sort capacities in descending order for optimal box selection
    capacity.sort(reverse=True)
    cumulative_capacity = 0
    box_count = 0
    # Iterate over sorted capacity list
    for cap in capacity:
        cumulative_capacity += cap    # Add the current box capacity

        box_count += 1                # Count the box
        if cumulative_capacity >= total_apples:
            break                    # Enough capacity reached
    return box_count

# Example usage:
print(min_boxes([1, 3, 2], [4, 3, 1, 5, 2])) # Output: 2
print(min_boxes([5, 5, 5], [2, 4, 2, 7]))    # Output: 4
```

◆ Quick Review

Key Insights

- Compute the total number of apples by summing the values in the apple array.

- To minimize the number of boxes, choose boxes with the largest capacities first.
- Sort the capacity array in descending order.
- Recursively add box capacities until the cumulative capacity is greater than or equal to the total apples.
- This greedy approach ensures the optimal solution using minimal boxes.

Space & Time

Time Complexity: $O(m \log m)$ due to sorting the capacity array, where m is the number of boxes.

Space Complexity: $O(1)$ extra space if sorting is done in-place.

Solution

The solution starts by summing all apples in the input array `apple`. The capacity array is then sorted in descending order, allowing the selection of the largest boxes first. Iterate through this sorted capacity list, accumulating the capacity until it is at least equal to the total apple count. The number of iterations (boxes) used will be the answer. The key data structures include simple integer variables

and an array. The greedy algorithm is optimal here because it ensures that the highest capacities are chosen to reach the required sum as quickly as possible.

Round 5 · Mid · Medium

Round 5

Mid Priority · Medium

94 questions · 23 patterns

Bit Manipulation #137 #201 #260

Prefix Sum #304 #370 #523 #974 #1314 #2536

**Kadane's Algorithm #918 #978 #1014 #1186
#1749**

Matrix (2D Array) #289

LinkedList In-place Reversal #92

**Monotonic Stack #402 #456 #503 #581 #769
#901 #907 #1019 #2104**

Queues #950 #1823 #2327

**Monotonic Queue #1438 #1673 #1696 #2762
#3578**

Divide and Conquer #109 #427 #654

Merge Sort #912

- ☐ ● QuickSort / QuickSelect #1985 ↗
- ☐ ● Backtracking #47 #77 #93 #216 ↗
- ☐ ● BST / Ordered Set #99 #426 #450 #510 #669 ↗
- ☐ ● #701 #729 #731 #1008 #2034 ↗
- ☐ ● Tries #421 #648 #1233 #1268 #2452 #3043 ↗
- ☐ ● Heaps #1405 #1642 #1834 #1882 ↗
- ☐ ● Intervals #452 #1094 #1229 #1288 #1353 ↗
- ☐ ● #2054 ↗
- ☐ ● K-Way Merge #373 #378 ↗
- ☐ ● Data Structure Design #641 #1146 #1472 #2353 ↗
- ☐ ● Greedy #406 #670 #826 #921 #1488 #1717 ↗
- ☐ ● #1871 #1975 ↗
- ☐ ● Topological Sort #802 #1462 #2115 ↗
- ☐ ● Union Find #399 #547 #947 ↗
- ☐ ● Minimum Spanning Tree #1135 ↗
- ☐ ● Shortest Path #1514 #1631 #2976 #3341 #3650 ↗

Bit Manipulation

Round 5 · Mid · Medium · 3 questions

Bit Manipulation

□ #137 Single Number II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Bit Manipulation
Lists: AlgoMaster 600

Problem

Given an integer array `nums` where every element appears three times except for one, which appears exactly once. Find the single element and return it.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: `nums = [2,2,3,2]`
Output: `3`

Example 2:

Input: `nums = [0,1,0,1,0,1,99]`
Output: `99`

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

- Each element in `nums` appears exactly three times except for one element which appears once.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        ones = 0
        twos = 0

        for num in nums:
            ones ^= num & ~twos
            twos ^= num & ~ones

        return ones
```

• LeetDoocs

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        ans = 0
        for i in range(32):
            cnt = sum(num >> i & 1 for num in nums)
            if cnt % 3:
                if i == 31:
                    ans -= 1 << i
                else:
                    ans |= 1 << i

        return ans
```

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        a = b = 0
        for c in nums:
            aa = (~a & b & c) | (a & ~b & ~c)
            bb = ~a & (b ^ c)
            a, b = aa, bb
        return b
```

• SimplyLeet

```
# Python solution for Single Number II
def singleNumber(nums):
    # Variables to store bits seen once and twice
    ones = 0
    twos = 0
    # Process each number in the input list
    for num in nums:
        # Update ones with the current number and clear bits already in twos
        new_ones = (ones ^ num) & ~twos
        # Update twos with the current number and clear bits just added to
        new_twos = (twos ^ num) & ~new_ones
        ones, twos = new_ones, new_twos
    return ones

# Example usage:
print(singleNumber([2,2,3,2])) # Output: 3
```

◆ Quick Review

Key Insights

- Use bit manipulation to solve the problem in $O(n)$ time with $O(1)$ extra space.
- The core idea is to track the bit counts across all numbers and use modulo operations

to cancel out bits that appear three times.

- Instead of using extra arrays for bit counts, maintain two accumulator variables to represent bits seen once and twice.
- Leverage bitwise XOR, AND, and NOT operations to update and clear bits accordingly.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The approach uses two accumulators, named ones and twos, to track the bits that have appeared once and twice, respectively. For each number in the array, we update these accumulators:

- Calculate new values for ones by XOR-ing with the current number and removing any bits that are already in twos.
- Update twos accordingly using bits that appear and are not newly reintroduced into ones. After processing all numbers, the variable ones will contain the unique single number because the bits corresponding to

numbers that appear three times cancel out.

Bit Manipulation

□ #201 Bitwise AND of Numbers Range

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Bit Manipulation

Lists: AlgoMaster 600

Problem

Given two integers `left` and `right` that represent the range `[left, right]`, return the bitwise AND of all numbers in this range, inclusive.

Example 1:

Input: `left = 5, right = 7`
Output: `4`

Example 2:

Input: `left = 0, right = 0`
Output: `0`

Example 3:

Input: `left = 1, right = 2147483647`
Output: `0`

Constraints:

- $0 \leq \text{left} \leq \text{right} \leq 2^{31} - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rangeBitwiseAnd(self, m: int, n: int) -> int:
        return self.rangeBitwiseAnd(m >> 1, n >> 1) << 1 if m < n else m
```

• LeetCode

```
class Solution:
    def rangeBitwiseAnd(self, left: int, right: int) -> int:
        while left < right:
            right &= right - 1
        return right
```

• SimplyLeet

```
# Function to calculate bitwise AND of numbers in range [left, right]
def rangeBitwiseAnd(left: int, right: int) -> int:
    shift = 0 # Counter for number of shifts performed
    # Shift both numbers right until they are equal
    while left != right:
        left >>= 1 # Right shift left by 1
        right >>= 1 # Right shift right by 1
        shift += 1 # Increment shift counter
    # Shift left to restore the common prefix in its original position
    return left << shift # Left shift by the number of shifts
```

```
# Example usage
print(rangeBitwiseAnd(5, 7)) # Expected output: 4
```

◆ Quick Review

Key Insights

- The bitwise AND operation only retains a bit if that bit is 1 in every number in the given range.

- The key observation is that the range of numbers will eventually force a bit position to flip from 1 to 0; hence, only the common prefix (the leftmost bits that do not change when moving from left to right) contributes to the final result.
- The solution involves shifting both left and right until they match, tracking the number of shifts, and then shifting back to obtain the common bits.

Space & Time

Time Complexity: $O(\log n)$, where n is the number represented by right, since we process a fixed number of bits (typically 32 bits for 32-bit integers).

Space Complexity: $O(1)$, since only a few integer variables are used.

Solution

The main idea is to discard the differing bits between left and right. We do this by shifting both numbers to the right until they become equal, which effectively removes the non-common bits at the right. The number of shifts performed tells us how many trailing

zeros must be added back by shifting left. This approach efficiently finds the common prefix shared by both numbers, which represents the bitwise AND of all numbers in the given range.

Data Structures Used:

- Only integer variables are used.

Algorithmic Approach:

- While left is not equal to right, right-shift both left and right (i.e., remove the least significant bit).
- Count the number of shifts performed.
- Once left equals right, shift back to the left by the counted amount to restore the common prefix in its original bit positions.
- The final number is the bitwise AND of all numbers in the range.

Bit Manipulation

□ #260 Single Number III

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Bit Manipulation
Lists: AlgoMaster 600

Problem

Given an integer array `nums`, in which exactly two elements appear only once and all the other elements appear exactly twice. Find the two elements that appear only once. You can return the answer in any order.

You must write an algorithm that runs in linear runtime complexity and uses only constant extra space.

Example 1:

```
Input: nums = [1,2,1,3,2,5]
Output: [3,5]
Explanation: [5, 3] is also a valid answer.
```

Example 2:

```
Input: nums = [-1,0]
Output: [-1,0]
```

Example 3:

Input: nums = [0,1]
Output: [1,0]

Constraints:

- $2 \leq \text{nums.length} \leq 3 * 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$
- Each integer in nums will appear twice, only two integers will appear once.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def singleNumber(self, nums: list[int]) -> list[int]:
        xors = functools.reduce(operator.xor, nums)
        lowbit = xors & -xors
        ans = [0, 0]

        # Seperate `nums` into two groups by `lowbit`.
        for num in nums:
            if num & lowbit:
                ans[0] ^= num

            else:
                ans[1] ^= num

        return ans
```

• LeetDoocs

```
class Solution:
    def singleNumber(self, nums: List[int]) -> List[int]:
        xs = reduce(xor, nums)
        a = 0
        lb = xs & -xs
        for x in nums:
            if x & lb:
                a ^= x
        b = xs ^ a
        return [a, b]
```

• SimplyLeet

```
# Python implementation for "Single Number III"
def singleNumber(nums):
    # Step 1: XOR all numbers to get xorTotal = unique1 XOR unique2
    xorTotal = 0
    for num in nums:
        xorTotal ^= num

    # Step 2: Find a set bit (rightmost set bit) in xorTotal
    diff_bit = xorTotal & -xorTotal

    # Step 3: Partition numbers into two groups and XOR separately
    unique1, unique2 = 0, 0
    for num in nums:
        if num & diff_bit:
            unique1 ^= num
        else:
            unique2 ^= num

    return [unique1, unique2]

# Example usage:
# result = singleNumber([1,2,1,3,2,5])
# print(result) # Output might be [3, 5] or [5, 3]
```

◆ Quick Review

Key Insights

- XOR of two identical numbers is 0.
- XOR of all numbers results in the XOR of the two unique numbers.
- A set bit in the XOR results (xorTotal) indicates a bit position where the two unique numbers differ.
- Partitioning the array based on this set bit allows isolation of each unique number.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

Use XOR to combine all numbers, which cancels out the pairs and leaves the XOR of the two unique numbers. Identify any set bit in this XOR to differentiate between the two unique numbers. Partition the numbers based on whether they have that bit set and XOR each group separately. The result are the two unique numbers. This approach leverages bit manipulation with constant extra space and a single pass (or two passes) over the input.

Prefix Sum

Round 5 · Mid · Medium · 6 questions

Prefix Sum

□ #304 Range Sum Query 2D – Immutable

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Design · Matrix · Prefix Sum
Lists: AlgoMaster 600

Problem

Given a 2D matrix `matrix`, handle multiple queries of the following type:

- Calculate the sum of the elements of matrix inside the rectangle defined by its upper left corner `(row1, col1)` and lower right corner `(row2, col2)`.

Implement the `NumMatrix` class:

- `NumMatrix(int[][] matrix)` Initializes the object with the integer matrix `matrix`.
- `int sumRegion(int row1, int col1, int row2, int col2)` Returns the sum of the elements of matrix inside the rectangle defined by its upper left corner `(row1, col1)` and lower right corner `(row2, col2)`.

You must design an algorithm where `sumRegion` works on $O(1)$ time complexity.

Example 1:

3	0	1	4	2
5	6	3	2	1
1	2	0	1	5
4	1	0	1	7
1	0	3	0	5

Input

```
["NumMatrix", "sumRegion", "sumRegion", "sumRegion"]
[[[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]], [2, 1, 4, 3], [1, 1, 2, 2], [1, 2, 2, 4]]
```

Output

```
[null, 8, 11, 12]
```

Explanation

```
NumMatrix numMatrix = new NumMatrix([[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]);
```


Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 200$
- $-104 \leq \text{matrix}[i][j] \leq 104$
- $0 \leq \text{row1} \leq \text{row2} < m$
- $0 \leq \text{col1} \leq \text{col2} < n$
- At most 104 calls will be made to `sumRegion`.

Community Solutions (Python)

• WalkCC

```
class NumMatrix:
    def __init__(self, matrix: list[list[int]]):
        if not matrix:
            return

        m = len(matrix)
        n = len(matrix[0])
        # prefix[i][j] := the sum of matrix[0..i][0..j]
        self.prefix = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m):
            for j in range(n):
                self.prefix[i + 1][j + 1] = (matrix[i][j] + self.prefix[i][j + 1] +
                                             self.prefix[i + 1][j] - self.prefix[i][j])

        def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
            return (self.prefix[row2 + 1][col2 + 1] - self.prefix[row1][col2 + 1] -
                    self.prefix[row2 + 1][col1] + self.prefix[row1][col1])
```

• LeetDoocs

```
class NumMatrix:
    def __init__(self, matrix: List[List[int]]):
        m, n = len(matrix), len(matrix[0])
        self.s = [[0] * (n + 1) for _ in range(m + 1)]
        for i, row in enumerate(matrix):
            for j, v in enumerate(row):
                self.s[i + 1][j + 1] = (
                    self.s[i][j + 1] + self.s[i + 1][j] - self.s[i][j] + v
                )

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
        return (
            self.s[row2 + 1][col2 + 1]
            - self.s[row2 + 1][col1]
            - self.s[row1][col2 + 1]
            + self.s[row1][col1]
        )

# Your NumMatrix object will be instantiated and called as such:

# obj = NumMatrix(matrix)
# param_1 = obj.sumRegion(row1,col1,row2,col2)
```

• SimplyLeet

```
# Class definition for NumMatrix using prefix sum 2D array.
class NumMatrix:
    def __init__(self, matrix):
        # Get dimensions of the matrix.
        m = len(matrix)
        n = len(matrix[0]) if m > 0 else 0
        # Create prefix sum matrix with extra row and column.
        self.dp = [[0] * (n + 1) for _ in range(m + 1)]
        # Precompute the prefix sum values.
        for i in range(m):
```

```

        for j in range(n):
            # Sum of current cell, top, left, and subtract overlapping
            top-left.
            self.dp[i+1][j+1] = self.dp[i][j+1] + self.dp[i+1][j] - self.d
            p[i][j] + matrix[i][j]

        def sumRegion(self, row1, col1, row2, col2):
            # Calculate the sum using the inclusion-exclusion principle.
            return self.dp[row2+1][col2+1] - self.dp[row1][col2+1] - self.dp[row2+
            1][col1] + self.dp[row1][col1]

```

◆ Quick Review

Key Insights

- Precompute a prefix sum matrix where each cell accumulates the sum of elements above and to the left.
- Use the inclusion-exclusion principle to efficiently calculate the sum of any submatrix using precomputed sums.
- Preprocessing the matrix takes $O(m * n)$ time, but each query is answered in constant time ($O(1)$).

Space & Time

Time Complexity:

- Preprocessing: $O(m * n)$
- Each Query: $O(1)$

Space Complexity: $O(m * n)$ for storing the prefix sum matrix.

Solution

The solution involves building a prefix sum matrix, dp , where $dp[i+1][j+1]$ holds the sum of elements in the submatrix from $(0, 0)$ to (i, j) . This allows summing any rectangular region using the inclusion–exclusion principle.

Specifically, for a query defined by $(row1, col1)$ to $(row2, col2)$, the region sum can be computed as:

$$\text{dp}[row2+1][col2+1] - \text{dp}[row1][col2+1] - \text{dp}[row2+1][col1] + \text{dp}[row1][col1]$$

This approach ensures that after an $O(m * n)$ preprocessing step, each query is computed in constant time.

Prefix Sum

□ #370 Range Addition

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Prefix Sum

Lists: AlgoMaster 600

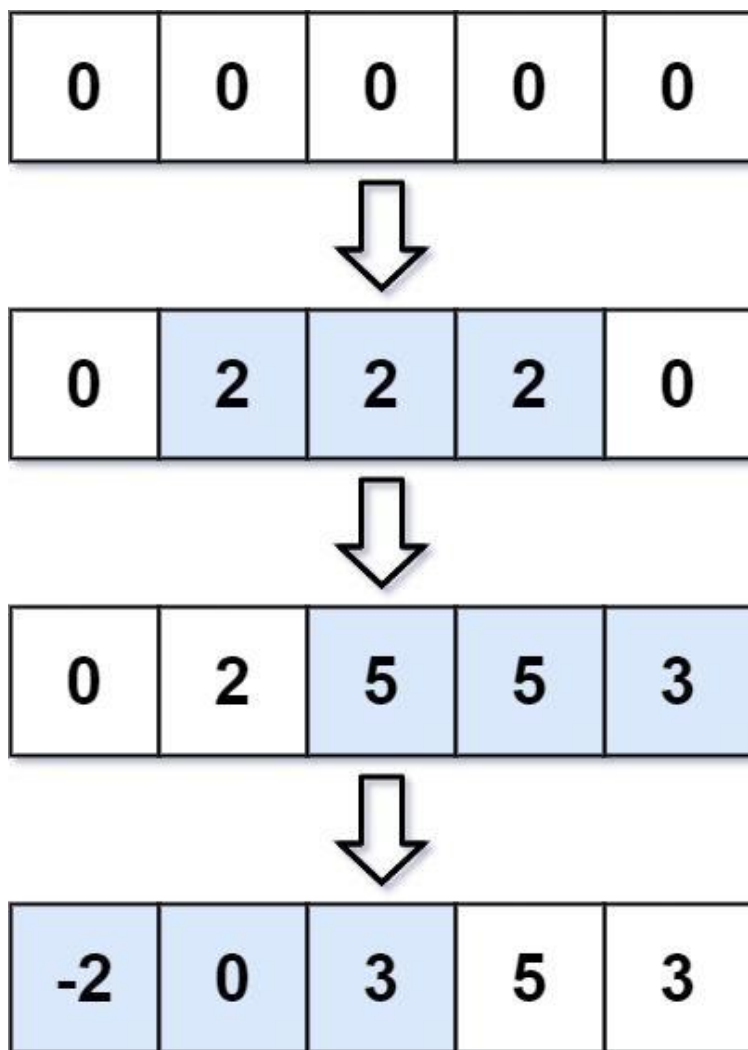
Problem

You are given an integer length and an array updates where $\text{updates}[i] = [\text{startIdxi}, \text{endIdxi}, \text{inci}]$.

You have an array arr of length length with all zeros, and you have some operation to apply on arr. In the ith operation, you should increment all the elements $\text{arr}[\text{startIdxi}]$, $\text{arr}[\text{startIdxi} + 1]$, ..., $\text{arr}[\text{endIdxi}]$ by inci.

Return arr after applying all the updates.

Example 1:



Input: length = 5, updates = [[1,3,2],[2,4,3],[0,2,-2]]
Output: [-2,0,3,5,3]

Example 2:

Input: length = 10, updates = [[2,4,6],[5,6,8],[1,9,-4]]
Output: [0,-4,2,2,2,4,4,-4,-4,-4]

Constraints:

- $1 \leq \text{length} \leq 105$
- $0 \leq \text{updates.length} \leq 104$
- $0 \leq \text{startIdx} \leq \text{endIdx} < \text{length}$
- $-1000 \leq \text{inci} \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def getModifiedArray(
        self,
        length: int,
        updates: list[list[int]],
    ) -> list[int]:
        line = [0] * length

        for start, end, inc in updates:
            line[start] += inc

            if end + 1 < length:
                line[end + 1] -= inc

        return itertools.accumulate(line)
```

• LeetDoocs

```
class Solution:
    def getModifiedArray(self, length: int, updates: List[List[int]]) -> List[
int]:
        d = [0] * length
        for l, r, c in updates:
            d[l] += c
            if r + 1 < length:
                d[r + 1] -= c
        return list(accumulate(d))

class BinaryIndexedTree:
    __slots__ = "n", "c"

    def __init__(self, n: int):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, delta: int) -> None:
        while x <= self.n:
            self.c[x] += delta
```

```

        x += x & -x

    def query(self, x: int) -> int:
        s = 0
        while x:
            s += self.c[x]
            x -= x & -x
        return s

```

```

class Solution:
    def getModifiedArray(self, length: int, updates: List[List[int]]) -> List[int]:
        tree = BinaryIndexedTree(length)
        for l, r, c in updates:
            tree.update(l + 1, c)
            tree.update(r + 2, -c)
        return [tree.query(i + 1) for i in range(length)]

```

• SimplyLeet

Python solution for Range Addition

```

def getModifiedArray(length, updates):
    # Initialize the result list with zeros
    result = [0] * length

    # Process each update by marking the start and end+1 positions
    for update in updates:
        start, end, inc = update
        result[start] += inc # Add inc at the start index

        if end + 1 < length:
            result[end + 1] -= inc # Subtract inc just after the end index

    # Convert the difference array to the final result using a running sum
    for i in range(1, length):
        result[i] += result[i - 1]

    return result

# Example usage:

```



```
length = 5
updates = [[1, 3, 2], [2, 4, 3], [0, 2, -2]]
print(getModifiedArray(length, updates)) # Output: [-2, 0, 3, 5, 3]
```

◆ Quick Review

Key Insights

- Use a difference array (prefix sum technique) to update ranges efficiently.
- Instead of updating every element for each range (which could be slow), update the start index and subtract after the end index.
- Compute the final array by taking the cumulative sum of the difference array.
- This approach reduces the time complexity from potentially $O(n * \text{number_of_updates})$ to $O(n + \text{number_of_updates})$.

Space & Time

Time Complexity: $O(n + k)$, where n is the length of the array and k is the number of updates.

Space Complexity: $O(n)$ for the difference array (and output array).

Solution

We use a difference array to efficiently perform range updates. The idea is to add the increment at the start index and subtract the increment just after the end index. After processing all updates, a single pass of cumulative sum across the array gives the final result. This method avoids updating every element within the range for each query, making it efficient for large inputs.

Prefix Sum

□ #523 Continuous Subarray Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Prefix Sum
Lists: AlgoMaster 600

Problem

Given an integer array `nums` and an integer `k`, return `true` if `nums` has a good subarray or `false` otherwise.

A good subarray is a subarray where:

- its length is at least two, and
- the sum of the elements of the subarray is a multiple of `k`.

Note that:

- A subarray is a contiguous part of the array.
- An integer `x` is a multiple of `k` if there exists an integer `n` such that $x = n * k$. 0 is always a multiple of `k`.

Example 1:

Input: `nums = [23,2,4,6,7]`, `k = 6`

Output: `true`

Explanation: `[2, 4]` is a continuous subarray of size 2 whose elements sum up to 6.

Example 2:

Input: `nums = [23,2,6,4,7]`, `k = 6`

Output: `true`

Explanation: `[23, 2, 6, 4, 7]` is an continuous subarray of size 5 whose elements sum up to 42.

42 is a multiple of 6 because $42 = 7 * 6$ and 7 is an integer.

Example 3:

Input: `nums = [23,2,6,4,7]`, `k = 13`

Output: `false`

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $0 \leq \text{nums}[i] \leq 109$
- $0 \leq \text{sum}(\text{nums}[i]) \leq 231 - 1$
- $1 \leq k \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def checkSubarraySum(self, nums: list[int], k: int) -> bool:
        prefix = 0
        prefixToIndex = {0: -1}

        for i, num in enumerate(nums):
            prefix += num
            if k != 0:
                prefix %= k
            if prefix in prefixToIndex:
```

```

        if i - prefixToIndex[prefix] > 1:
            return True
    else:
        # Set a new key if it's absent because the previous index is better.
        prefixToIndex[prefix] = i

    return False

```

• LeetCode

```

class Solution:
    def checkSubarraySum(self, nums: List[int], k: int) -> bool:
        d = {0: -1}
        s = 0
        for i, x in enumerate(nums):
            s = (s + x) % k
            if s not in d:
                d[s] = i
            elif i - d[s] > 1:
                return True

        return False

```

• SimplyLeet

```

# Python solution using prefix sum with modulo and a dictionary to store
remainders.

def checkSubarraySum(nums, k):
    # Dictionary to store remainder and its earliest index.
    remainder_index = {0: -1}
    cumulative_sum = 0

    # Iterate over the array elements.
    for index, num in enumerate(nums):
        cumulative_sum += num

```

```
remainder = cumulative_sum % k

# If the remainder is seen before, check the subarray length.
if remainder in remainder_index:
    # If subarray length is at least 2 (current index - previous index)
    if index - remainder_index[remainder] > 1:
        return True
else:
    # Store the index for this remainder the first time it is seen.
    remainder_index[remainder] = index

return False

# Example usage:
# print(checkSubarraySum([23,2,4,6,7], 6)) # Expected output: True
```

◆ Quick Review

Key Insights

- Use the prefix sum technique to accumulate sums.
- Instead of storing full sums, store the remainder (mod k) when divided by k .
- If two prefix sums have the same remainder, the subarray between these indices sums to a multiple of k .
- Ensure that the subarray has at least two elements; thus, check that the distance between the indices is at least 2.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(\min(n, k))$

Solution

The approach involves using prefix sums and taking advantage of the modulo property. As you compute the cumulative sum for each element, calculate the remainder of the sum modulo k . If the same remainder has been seen before and the subarray between the two indices is of length at least 2, then the subarray's sum is a multiple of k .

We store these remainders in a hash table (dictionary/map) mapping the remainder to the earliest index where this remainder was encountered. Special care is taken to initialize the hash table with the remainder 0 mapped to index -1 to handle cases where the subarray starting at index 0 has a valid sum.

Prefix Sum

□ #974 Subarray Sums Divisible by K

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Prefix Sum
Lists: AlgoMaster 600

Problem

Given an integer array `nums` and an integer `k`, return the number of non-empty subarrays that have a sum divisible by `k`.

A subarray is a contiguous part of an array.

Example 1:

Input: `nums = [4,5,0,-2,-3,1]`, `k = 5`

Output: 7

Explanation: There are 7 subarrays with a sum divisible by `k = 5`:

`[4, 5, 0, -2, -3, 1]`, `[5]`, `[5, 0]`, `[5, 0, -2, -3]`, `[0]`, `[0, -2, -3]`, `[-2, -3]`

Example 2:

Input: `nums = [5]`, `k = 9`

Output: 0

Constraints:

- $1 \leq \text{nums.length} \leq 3 \times 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- $2 \leq k \leq 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def subarraysDivByK(self, nums: list[int], k: int) -> int:
        ans = 0
        prefix = 0
        count = [0] * k
        count[0] = 1

        for num in nums:
            prefix = (prefix + num % k + k) % k
            ans += count[prefix]

            count[prefix] += 1

        return ans
```

• LeetDoocs

```
class Solution:
    def subarraysDivByK(self, nums: List[int], k: int) -> int:
        cnt = Counter({0: 1})
        ans = s = 0
        for x in nums:
            s = (s + x) % k
            ans += cnt[s]
            cnt[s] += 1
        return ans
```

• SimplyLeet

```
# Python Solution

def subarraysDivByK(nums, k):
    # Dictionary to store frequency of each remainder
    remainder_count = {0: 1} # initialize with remainder 0 for subarrays
    # starting at index 0
    current_sum = 0 # current prefix sum
    count = 0 # counter for valid subarrays

    # Iterate through the given array
    for num in nums:

        current_sum += num # update the prefix sum
        # Compute the modulo while handling negative values
        remainder = current_sum % k

        # If this remainder has been seen before, add its count to our answer
        count += remainder_count.get(remainder, 0)

        # Update the frequency count for this remainder
        remainder_count[remainder] = remainder_count.get(remainder, 0) + 1

    return count

# Example Usage:
# print(subarraysDivByK([4,5,0,-2,-3,1], 5))
```

◆ Quick Review

Key Insights

- Use the concept of prefix sums where $\text{prefix}[i]$ represents the sum of elements from the start up to index i .
- A subarray sum from index $i+1$ to j is divisible by k if and only if $\text{prefix}[j] \% k$ equals

$\text{prefix}[i] \% k$.

- Use a hash table (or dictionary/map) to count occurrences of each remainder.
- Initialize the hash table with the remainder 0 count set to 1 to account for subarrays that start from index 0.
- Handle negative numbers by adjusting mod results to ensure they are non-negative.

Space & Time

Time Complexity: $O(n)$ where n is the length of the array, since we traverse the array once.

Space Complexity: $O(k)$ in the worst-case scenario, due to storing frequency counts of remainders.

Solution

We solve the problem using prefix sums and a hash table (or map) to track the frequency of each modulo value (prefix sum mod k). For each element in the array, we update the cumulative sum and compute its modulo with k . The number of valid subarrays ending at the current index is equal to the number of times this modulo value has been seen before (since the difference between two prefix sums with

the same modulo is divisible by k). Finally, sum up these counts to obtain the answer.

Prefix Sum

□ #1314 Matrix Block Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Prefix Sum
Lists: AlgoMaster 600

Problem

Given a $m \times n$ matrix `mat` and an integer k , return a matrix `answer` where each `answer[i][j]` is the sum of all elements `mat[r][c]` for:

- $i - k \leq r \leq i + k$,
- $j - k \leq c \leq j + k$, and
- (r, c) is a valid position in the matrix.

Example 1:

```
Input: mat = [[1,2,3],[4,5,6],[7,8,9]], k = 1  
Output: [[12,21,16],[27,45,33],[24,39,28]]
```

Example 2:

```
Input: mat = [[1,2,3],[4,5,6],[7,8,9]], k = 2  
Output: [[45,45,45],[45,45,45],[45,45,45]]
```

Constraints:

- $m == \text{mat.length}$
- $n == \text{mat}[i].\text{length}$
- $1 \leq m, n, k \leq 100$

- $1 \leq \text{mat}[i][j] \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def matrixBlockSum(self, mat: list[list[int]], k: int) -> list[list[int]]:
        m = len(mat)
        n = len(mat[0])
        ans = [[0] * n for _ in range(m)]
        prefix = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m):
            for j in range(n):
                prefix[i + 1][j + 1] = (mat[i][j] + prefix[i][j + 1] +

                prefix[i + 1][j] - prefix[i][j])

        for i in range(m):
            for j in range(n):
                r1 = max(0, i - k) + 1
                c1 = max(0, j - k) + 1
                r2 = min(m - 1, i + k) + 1
                c2 = min(n - 1, j + k) + 1
                ans[i][j] = (prefix[r2][c2] - prefix[r2][c1 - 1] -
                prefix[r1 - 1][c2] + prefix[r1 - 1][c1 - 1])

        return ans
```

• LeetDoocs

```

class Solution:
    def matrixBlockSum(self, mat: List[List[int]], k: int) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        s = [[0] * (n + 1) for _ in range(m + 1)]
        for i, row in enumerate(mat, 1):
            for j, x in enumerate(row, 1):
                s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + x
        ans = [[0] * n for _ in range(m)]
        for i in range(m):
            for j in range(n):
                x1, y1 = max(i - k, 0), max(j - k, 0)
                x2, y2 = min(m - 1, i + k), min(n - 1, j + k)
                ans[i][j] = (
                    s[x2 + 1][y2 + 1] - s[x1][y2 + 1] - s[x2 + 1][y1] + s[x1][y1]
                )
        return ans

```

• SimplyLeet

```

# Python solution using prefix sum
def matrixBlockSum(mat, k):
    # dimensions of the matrix
    m = len(mat)
    n = len(mat[0])

    # create a prefix sum matrix with extra row and column for easier
    # calculations
    prefix = [[0] * (n + 1) for _ in range(m + 1)]

    # Build prefix sum matrix

```

```

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            prefix[i][j] = mat[i-1][j-1] + prefix[i-1][j] + prefix[i][j-1] - p
            prefix[i][j] = prefix[i-1][j] + prefix[i][j-1] - prefix[i-1][j-1]

    # function to get sum of submatrix from (r1, c1) to (r2, c2) inclusive
    def get_sum(r1, c1, r2, c2):
        return prefix[r2+1][c2+1] - prefix[r1][c2+1] - prefix[r2+1][c1] + prefix[r1][c1]

    # Build result matrix
    result = [[0] * n for _ in range(m)]

    for i in range(m):
        for j in range(n):
            # Determine block boundaries considering matrix limits
            r1 = max(0, i - k)
            c1 = max(0, j - k)
            r2 = min(m - 1, i + k)
            c2 = min(n - 1, j + k)
            result[i][j] = get_sum(r1, c1, r2, c2)
    return result

# Example usage:
# print(matrixBlockSum([[1,2,3],[4,5,6],[7,8,9]], 1))

```

◆ Quick Review

Key Insights

- Brute-force summing every block for each element is inefficient for larger matrices.
- Using a prefix sum (cumulative sum) matrix greatly optimizes the process.
- With a prefix sum matrix, we can compute the sum of any submatrix in constant time.

- **Boundary management is crucial since the k -neighborhood may extend beyond the matrix edges.**

Space & Time

Time Complexity: $O(m * n)$

Space Complexity: $O(m * n)$

Solution

We compute a prefix sum matrix where each element at (i, j) represents the sum of all elements in `mat` from $(0, 0)$ to (i, j) . With the prefix sum matrix, the sum of any submatrix defined by its corners can be efficiently computed using the inclusion-exclusion principle. For each element in the result matrix, the boundaries of the submatrix are determined by the limits imposed by k and the matrix size. This approach ensures that each matrix cell's block sum is calculated in constant time after an initial $O(m * n)$ preprocessing step for building the prefix sum matrix.

Prefix Sum

□ #2536 Increment Submatrices by One

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Prefix Sum
Lists: AlgoMaster 600

Problem

You are given a positive integer n , indicating that we initially have an $n \times n$ 0-indexed integer matrix `mat` filled with zeroes.

You are also given a 2D integer array `query`. For each `query[i] = [row1i, col1i, row2i, col2i]`, you should do the following operation:

- Add 1 to every element in the submatrix with the top left corner $(row1i, col1i)$ and the bottom right corner $(row2i, col2i)$. That is, add 1 to `mat[x][y]` for all $row1i \leq x \leq row2i$ and $col1i \leq y \leq col2i$.

Return the matrix `mat` after performing every query.

Example 1:

0	0	0				1	1	0
0	0	0				1	2	1
0	0	0				0	1	1

Input: $n = 3$, queries = $[[1,1,2,2],[0,0,1,1]]$

Output: $[[1,1,0],[1,2,1],[0,1,1]]$

Explanation: The diagram above shows the initial matrix, the matrix after the first query, and the matrix after the second query.

- In the first query, we add 1 to every element in the submatrix with the top left corner (1, 1) and bottom right corner (2, 2).
- In the second query, we add 1 to every element in the submatrix with the top left corner (0, 0) and bottom right corner (1, 1).

Example 2:

0	0			1	1
0	0			1	1

Input: $n = 2$, queries = $[[0,0,1,1]]$

Output: $[[1,1],[1,1]]$

Explanation: The diagram above shows the initial matrix and the matrix after the first query.

- In the first query we add 1 to every element in the matrix.

Constraints:

- $1 \leq n \leq 500$
- $1 \leq \text{queries.length} \leq 104$
- $0 \leq \text{row1i} \leq \text{row2i} < n$

- $0 \leq \text{col1i} \leq \text{col2i} < n$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def rangeAddQueries(self, n: int, queries: List[List[int]]) -> List[List[int]]:
        mat = [[0] * n for _ in range(n)]
        for x1, y1, x2, y2 in queries:
            mat[x1][y1] += 1
            if x2 + 1 < n:
                mat[x2 + 1][y1] -= 1
            if y2 + 1 < n:
                mat[x1][y2 + 1] -= 1
            if x2 + 1 < n and y2 + 1 < n:
                mat[x2 + 1][y2 + 1] += 1

        for i in range(n):
            for j in range(n):
                if i:
                    mat[i][j] += mat[i - 1][j]
                if j:
                    mat[i][j] += mat[i][j - 1]
                if i and j:
                    mat[i][j] -= mat[i - 1][j - 1]

        return mat
```

• SimplyLeet

```

# Python Solution
class Solution:
    def rangeAddQueries(self, n: int, queries: List[List[int]]) -> List[List[int]]:
        # Initialize a difference matrix of size (n+1) x (n+1) with zeros.
        diff = [[0] * (n + 1) for _ in range(n + 1)]

        # Process each query to update the difference matrix.
        for row1, col1, row2, col2 in queries:
            diff[row1][col1] += 1 # Increment top-left corner.
            if col2 + 1 < n:
                diff[row1][col2 + 1] -= 1 # Subtract right to the submatrix.
            if row2 + 1 < n:
                diff[row2 + 1][col1] -= 1 # Subtract below the submatrix.
            if row2 + 1 < n and col2 + 1 < n:
                diff[row2 + 1][col2 + 1] += 1 # Correct double subtraction.

        # Build the final matrix from the difference matrix using prefix sum.
        # First, process prefix sums row-wise.
        for i in range(n):
            for j in range(1, n):
                diff[i][j] += diff[i][j-1]

        # Then, process prefix sums column-wise.
        for j in range(n):
            for i in range(1, n):
                diff[i][j] += diff[i-1][j]

        # Extract the first n x n matrix as result.
        return [row[:n] for row in diff[:n]]

```

◆ Quick Review

Key Insights

- Instead of updating each element within a query's submatrix (which could be costly), use a 2D difference array (prefix sum technique)

for efficient range updates.

- For each query, update only the borders in the difference matrix:
- Add 1 at (row1, col1).
- Subtract 1 at (row1, col2 + 1) if within bounds.
- Subtract 1 at (row2 + 1, col1) if within bounds.
- Add 1 at (row2 + 1, col2 + 1) if within bounds.
- After processing all queries, convert the difference matrix into the final matrix using two-pass prefix sums (first row-wise then column-wise).

Space & Time

Time Complexity: $O(q + n^2)$, where q is the number of queries.

Space Complexity: $O(n^2)$ for the extra difference matrix used.

Solution

We use a 2D difference (or prefix sum) approach to handle the range updates efficiently. Instead of incrementing every

element in a queried submatrix directly, we update only the corners of the submatrix in a difference matrix. After processing every query, we compute prefix sums to convert the difference matrix to the actual updated matrix.

Steps:

- Initialize a 2D difference matrix of size $(n+1) \times (n+1)$ with zeros.
- For each query $[row1, col1, row2, col2]$:
- Add 1 at $diff[row1][col1]$.
- Subtract 1 at $diff[row1][col2 + 1]$ if $col2+1 \leq n$.
- Subtract 1 at $diff[row2 + 1][col1]$ if $row2+1 \leq n$.
- Add 1 at $diff[row2 + 1][col2 + 1]$ if both $row2+1$ and $col2+1$ are $\leq n$.
- Compute the prefix sum row-wise and then column-wise to form the final matrix.
- Return the updated matrix up to $n \times n$.

This approach allows each query to be processed in constant time and handles large inputs efficiently.

Kadane's Algorithm

Round 5 · Mid · Medium · 5 questions

Kadane's Algorithm

□ #918 Maximum Sum Circular Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Divide and Conquer · Dynamic
Programming · Queue · Monotonic Queue
Lists: AlgoMaster 600

Problem

Given a circular integer array `nums` of length `n`, return the maximum possible sum of a non-empty subarray of `nums`.

A circular array means the end of the array connects to the beginning of the array. Formally, the next element of `nums[i]` is `nums[(i + 1) % n]` and the previous element of `nums[i]` is `nums[(i - 1 + n) % n]`.

A subarray may only include each element of the fixed buffer `nums` at most once. Formally, for a subarray `nums[i], nums[i + 1], ..., nums[j]`, there does not exist $i \leq k_1, k_2 \leq j$ with $k_1 \% n \neq k_2 \% n$.

Example 1:

Input: `nums = [1,-2,3,-2]`
 Output: 3
 Explanation: Subarray `[3]` has maximum sum 3.

Example 2:

Input: `nums = [5,-3,5]`
 Output: 10
 Explanation: Subarray `[5,5]` has maximum sum $5 + 5 = 10$.

Example 3:

Input: `nums = [-3,-2,-3]`
 Output: -2
 Explanation: Subarray `[-2]` has maximum sum -2.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 3 * 10^4$
- $-3 * 10^4 \leq \text{nums}[i] \leq 3 * 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxSubarraySumCircular(self, nums: list[int]) -> int:
        totalSum = 0
        currMaxSum = 0
        currMinSum = 0
        maxSum = -math.inf
        minSum = math.inf

        for num in nums:
            totalSum += num
```

```

currMaxSum = max(currMaxSum + num, num)
currMinSum = min(currMinSum + num, num)
maxSum = max(maxSum, currMaxSum)
minSum = min(minSum, currMinSum)

return maxSum if maxSum < 0 else max(maxSum, totalSum - minSum)

```

• LeetCode

```

class Solution:
    def maxSubarraySumCircular(self, nums: List[int]) -> int:
        pmi, pmx = 0, -inf
        ans, s, smi = -inf, 0, inf
        for x in nums:
            s += x
            ans = max(ans, s - pmi)
            smi = min(smi, s - pmx)
            pmi = min(pmi, s)
            pmx = max(pmx, s)

        return max(ans, s - smi)

```

• SimplyLeet

Python solution

```

def maxSubarraySumCircular(nums):
    # Initialize variables for Kadane's algorithm for maximum subarray sum
    max_current = max_global = nums[0]
    # Initialize variables for finding the minimum subarray sum
    min_current = min_global = nums[0]
    # Calculate total sum starting with the first element
    total_sum = nums[0]

```

```
# Iterate over the array from the second element
for num in nums[1:]:
    total_sum += num # Update total sum

    # Update maximum subarray sum ending at current position
    max_current = max(num, max_current + num)
    max_global = max(max_global, max_current)

    # Update minimum subarray sum ending at current position
    min_current = min(num, min_current + num)

    min_global = min(min_global, min_current)

# If all numbers are negative, max_global is the maximum
if max_global < 0:
    return max_global
else:
    # Otherwise, the answer is the maximum of non-circular and circular
    case
    return max(max_global, total_sum - min_global)

# Example usage:

print(maxSubarraySumCircular([1, -2, 3, -2]))
```

◆ Quick Review

Key Insights

- Use Kadane's algorithm to find the maximum sum of a contiguous non-circular subarray.
- Apply a modified Kadane's algorithm to find the minimum subarray sum.
- The maximum circular subarray sum can be computed as (total sum of array – minimum subarray sum).

- Handle the edge case where all numbers are negative by returning the non-circular maximum.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution involves two passes over the array. In the first pass, we apply Kadane's algorithm to find the maximum subarray sum. In the second pass, we compute the minimum subarray sum using a similar approach. The maximum circular subarray sum is then the maximum of the non-circular (standard) maximum or (total sum - minimum subarray sum). However, if all numbers are negative, the non-circular maximum is returned to avoid an incorrect result caused by subtracting the minimum subarray sum.

Kadane's Algorithm

□ #978 Longest Turbulent Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Sliding Window
Lists: AlgoMaster 600

Problem

Given an integer array `arr`, return the length of a maximum size turbulent subarray of `arr`.

A subarray is turbulent if the comparison sign flips between each adjacent pair of elements in the subarray.

More formally, a subarray `[arr[i], arr[i + 1], ..., arr[j]]` of `arr` is said to be turbulent if and only if:

- For $i \leq k < j$: $arr[k] > arr[k + 1]$ when k is odd, and
- $arr[k] < arr[k + 1]$ when k is even.
- $arr[k] > arr[k + 1]$ when k is even, and
- $arr[k] < arr[k + 1]$ when k is odd.

Example 1:

Input: `arr = [9,4,2,10,7,8,8,1,9]`

Output: 5

Explanation: `arr[1] > arr[2] < arr[3] > arr[4] < arr[5]`

Example 2:

Input: arr = [4,8,12,16]
Output: 2

Example 3:

Input: arr = [100]
Output: 1

Constraints:

- $1 \leq \text{arr.length} \leq 4 * 10^4$
- $0 \leq \text{arr}[i] \leq 10^9$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxTurbulenceSize(self, arr: list[int]) -> int:
        ans = 1
        increasing = 1
        decreasing = 1

        for i in range(1, len(arr)):
            if arr[i] > arr[i - 1]:
                increasing = decreasing + 1
                decreasing = 1

            elif arr[i] < arr[i - 1]:
                decreasing = increasing + 1
                increasing = 1
            else:
                increasing = 1
                decreasing = 1
            ans = max(ans, max(increasing, decreasing))

        return ans
```

• LeetDoocs

```
class Solution:
    def maxTurbulenceSize(self, arr: List[int]) -> int:
        ans = f = g = 1
        for a, b in pairwise(arr):
            ff = g + 1 if a < b else 1
            gg = f + 1 if a > b else 1
            f, g = ff, gg
            ans = max(ans, f, g)
        return ans
```

• SimplyLeet

```
# Function to find the length of the longest turbulent subarray.
def maxTurbulenceSize(arr):
    n = len(arr)
    if n == 1:
        return 1

    # Initialize maximum turbulent subarray length and current subarray length.
    maxLength = 1
    currentLength = 1

    # Iterate through the array.
    for i in range(1, n):
        # If current and previous elements are equal, reset the current
        # sequence.
        if arr[i] == arr[i - 1]:
            currentLength = 1
        else:
            # For the first pair or if alternating condition holds with
            # previous comparison.
            if i == 1 or (arr[i - 1] - arr[i - 2]) * (arr[i] - arr[i - 1]) < 0:
                currentLength += 1
            else:
```



```
        # Reset: the current pair forms a new turbulent subarray of
length 2.
        currentLength = 2
        # Update the maximum observed length.
        maxLength = max(maxLength, currentLength)
    return maxLength

# Example usage:
print(maxTurbulenceSize([9,4,2,10,7,8,8,1,9])) # Expected output: 5
```

◆ Quick Review

Key Insights

- A turbulent subarray requires alternating "greater than" and "less than" comparisons.
- A sliding window or dynamic programming approach can be used to solve the problem in $O(n)$ time.
- Reset the subarray length when encountering equal adjacent elements or when the alternating condition fails.
- Handle the case where the array has only one element.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We can solve the problem using a dynamic programming / sliding window approach:

- If the array has only one element, return 1.**
- Initialize a variable (currentLength) to keep track of the current turbulent subarray length.**
- Traverse the array from the second element and:**
 - If the current element is equal to the previous element, reset currentLength to 1.**
 - Otherwise, if the current and previous comparisons create a valid alternating pattern when compared to the one before them, increment currentLength.**
 - If the alternating pattern breaks, reset currentLength to 2 (since the current two elements may start a new turbulent subarray).**
 - Track the maximum length seen during the traversal.**

Key trick: Instead of precomputing comparison signs, we check the alternating condition on the fly using the relationship between three consecutive elements.

Kadane's Algorithm

□ #1014 Best Sightseeing Pair

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: AlgoMaster 600

Problem

You are given an integer array `values` where `values[i]` represents the value of the i th sightseeing spot. Two sightseeing spots i and j have a distance $j - i$ between them.

The score of a pair ($i < j$) of sightseeing spots is `values[i] + values[j] + i - j`: the sum of the values of the sightseeing spots, minus the distance between them.

Return the maximum score of a pair of sightseeing spots.

Example 1:

Input: `values = [8,1,5,2,6]`

Output: 11

Explanation: $i = 0, j = 2, \text{values}[i] + \text{values}[j] + i - j = 8 + 5 + 0 - 2 = 11$

Example 2:

Input: values = [1,2]

Output: 2

Constraints:

- $2 \leq \text{values.length} \leq 5 * 10^4$
- $1 \leq \text{values}[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxScoreSightseeingPair(self, values: list[int]) -> int:
        ans = 0
        bestPrev = 0

        for value in values:
            ans = max(ans, value + bestPrev)
            bestPrev = max(bestPrev, value) - 1

        return ans
```

• LeetDoocs

```
class Solution:
    def maxScoreSightseeingPair(self, values: List[int]) -> int:
        ans = mx = 0
        for j, x in enumerate(values):
            ans = max(ans, mx + x - j)
            mx = max(mx, x + j)
        return ans
```

• SimplyLeet

```
def maxScoreSightseeing(values):
    # best_left holds the maximum value of (values[i] + i) encountered up to
    the current index
    best_left = values[0] + 0 # Initialize with the first element and index
    max_score = 0 # Track the maximum score

    # Iterate through the array starting from the second element
    for j in range(1, len(values)):
        # Calculate current score using the best value so far and the current
        spot's contribution
        current_score = best_left + values[j] - j
        # Update max_score if the current score is greater

        max_score = max(max_score, current_score)
        # Update best_left with the maximum of itself and (values[j] + j)
        best_left = max(best_left, values[j] + j)
    return max_score

# Example usage:
print(maxScoreSightseeing([8, 1, 5, 2, 6])) # Expected output: 11
```

◆ Quick Review

Key Insights

- The score can be reformulated as $(\text{values}[i] + i) + (\text{values}[j] - j)$, decoupling the contributions from the two indices.
- While iterating through the array, maintain the maximum value of $(\text{values}[i] + i)$ seen so far.
- For every new spot j , combine the previously stored maximum with $(\text{values}[j] - j)$ to update the answer.

- This one-pass strategy ensures an $O(n)$ time complexity.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution involves a single pass through the array, maintaining a variable (`best_left`) to record the maximum value of $(\text{values}[i] + i)$ encountered so far. For each index j (starting from 1), calculate the score by adding `best_left` and $(\text{values}[j] - j)$. Update the max score whenever a higher score is found, and also update the `best_left` for future calculations. This efficient approach leverages dynamic programming by keeping track of cumulative optimal sub-results.

Kadane's Algorithm

□ #1186 Maximum Subarray Sum with One Deletion **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: AlgoMaster 600

Problem

Given an array of integers, return the maximum sum for a non-empty subarray (contiguous elements) with at most one element deletion. In other words, you want to choose a subarray and optionally delete one element from it so that there is still at least one element left and the sum of the remaining elements is maximum possible.

Note that the subarray needs to be non-empty after deleting one element.

Example 1:

Input: arr = [1,-2,0,3]

Output: 4

Explanation: Because we can choose [1, -2, 0, 3] and drop -2, thus the subarray [1, 0, 3] becomes the maximum value.

Example 2:

Input: arr = [1,-2,-2,3]

Output: 3

Explanation: We just choose [3] and it's the maximum sum.

Example 3:

Input: arr = [-1,-1,-1,-1]

Output: -1

Explanation: The final subarray needs to be non-empty. You can't choose [-1] and delete -1 from it, then get an empty subarray to make the sum equals to 0.

Constraints:

- $1 \leq \text{arr.length} \leq 105$
- $-104 \leq \text{arr}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    # Similar to 53. Maximum Subarray
    def maximumSum(self, arr: list[int]) -> int:
        # dp[0][i] := the maximum sum subarray ending in i (no deletion)
        # dp[1][i] := the maximum sum subarray ending in i (at most 1 deletion)
        dp = [[0] * len(arr) for _ in range(2)]

        dp[0][0] = arr[0]
        dp[1][0] = arr[0]
        for i in range(1, len(arr)):

            dp[0][i] = max(arr[i], dp[0][i - 1] + arr[i])
            dp[1][i] = max(arr[i], dp[1][i - 1] + arr[i], dp[0][i - 1])

        return max(dp[1])
```

• LeetDoocs


```

class Solution:
    def maximumSum(self, arr: List[int]) -> int:
        n = len(arr)
        left = [0] * n
        right = [0] * n
        s = 0
        for i, x in enumerate(arr):
            s = max(s, 0) + x
            left[i] = s
        s = 0

        for i in range(n - 1, -1, -1):
            s = max(s, 0) + arr[i]
            right[i] = s
        ans = max(left)
        for i in range(1, n - 1):
            ans = max(ans, left[i - 1] + right[i + 1])
        return ans

```

• SimplyLeet

```

# Python solution for Maximum Subarray Sum with One Deletion

def maximumSum(arr):
    n = len(arr)
    # Initialize dp arrays
    dp_no_del = [0] * n # Maximum subarray sum ending here without deletion
    dp_del = [0] * n    # Maximum subarray sum ending here with one deletion

    dp_no_del[0] = arr[0]
    dp_del[0] = 0 # Cannot delete the first element (subarray must be
non-empty)

```

```
max_sum = arr[0]

# Iterate through the array, updating dp values
for i in range(1, n):
    # Maximum no deletion: either extend the previous subarray or start
    fresh
    dp_no_del[i] = max(arr[i], dp_no_del[i-1] + arr[i])
    # Maximum with one deletion: either delete current element using
    previous dp_no_del,
    # or extend a subarray that already had one deletion
    dp_del[i] = max(dp_no_del[i-1], dp_del[i-1] + arr[i])

    # Update overall maximum sum considering both states
    max_sum = max(max_sum, dp_no_del[i], dp_del[i])

return max_sum

# Example usage:
print(maximumSum([1, -2, 0, 3])) # Expected output: 4
```

◆ Quick Review

Key Insights

- Use dynamic programming to track two states: the maximum subarray sum ending at index i without deletion, and the maximum subarray sum ending at index i with one deletion.
- The state with no deletion uses the classic Kadane's algorithm.
- For the state with one deletion, either use the current element added to a previously

"one-deleted" state or delete the current element by taking the previous "no deletion" sum.

- **Careful handling is required when all numbers are negative, in which case no deletion might yield the maximum sum.**

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ (Can be reduced to $O(1)$ with optimized variables)

Solution

We solve this problem using dynamic programming with two arrays/variables. One (dp_no_del) tracks the maximum subarray sum ending at the current index without any deletion, while the other (dp_del) tracks the maximum subarray sum ending at the current index with one deletion allowed.

At each index:

- **For dp_no_del: We either continue the previous subarray or start a new subarray with the current element.**
- **For dp_del: We either delete the current element (thus carry over the previous**

dp_no_del) or add the current element to the previous dp_del.

The answer is the maximum value found in both dp_no_del and dp_del across all indices.

Kadane's Algorithm

□ #1749 Maximum Absolute Sum of Any Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: AlgoMaster 600

Problem

You are given an integer array `nums`. The absolute sum of a subarray `[numsl, numsl+1, ..., numsr-1, numsr]` is $\text{abs}(\text{numsl} + \text{numsl}+1 + \dots + \text{numsr}-1 + \text{numsr})$.

Return the maximum absolute sum of any (possibly empty) subarray of `nums`.

Note that $\text{abs}(x)$ is defined as follows:

- If x is a negative integer, then $\text{abs}(x) = -x$.
- If x is a non-negative integer, then $\text{abs}(x) = x$.

Example 1:

Input: `nums = [1,-3,2,3,-4]`

Output: 5

Explanation: The subarray `[2,3]` has absolute sum = $\text{abs}(2+3) = \text{abs}(5) = 5$.

Example 2:

Input: `nums = [2,-5,1,-4,3,-2]`

Output: 8

Explanation: The subarray `[-5,1,-4]` has absolute sum = `abs(-5+1-4) = abs(-8) = 8`.

Constraints:

- `1 <= nums.length <= 105`
- `-104 <= nums[i] <= 104`

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxAbsoluteSum(self, nums):
        ans = -math.inf
        maxSum = 0
        minSum = 0

        for num in nums:
            maxSum = max(num, maxSum + num)
            minSum = min(num, minSum + num)
            ans = max(ans, maxSum, -minSum)

        return ans
```

```
class Solution:
    def maxAbsoluteSum(self, nums):
        summ = 0
        maxPrefix = 0
        minPrefix = 0

        for num in nums:
            summ += num
            maxPrefix = max(maxPrefix, summ)
            minPrefix = min(minPrefix, summ)

        return maxPrefix - minPrefix
```

• LeetCode

```
class Solution:
    def maxAbsoluteSum(self, nums: List[int]) -> int:
        f = g = 0
        ans = 0
        for x in nums:
            f = max(f, 0) + x
            g = min(g, 0) + x
            ans = max(ans, f, abs(g))
        return ans
```

• SimplyLeet

```
# Python solution using Kadane's algorithm for both max and min subarray sums.
```

```
def maxAbsoluteSum(nums):
```

```
    # Initialize variables for the maximum subarray sum
```

```
    max_ending_here = 0
```

```
    max_sum = 0
```

```
    # Initialize variables for the minimum subarray sum
```

```
    min_ending_here = 0
```

```
    min_sum = 0
```

```
    # Iterate over each number in the list
```

```
    for num in nums:
```

```
        # Update the maximum sum subarray ending at current position
```

```
        max_ending_here = max(0, max_ending_here + num)
```

```
        max_sum = max(max_sum, max_ending_here)
```

```
        # Update the minimum sum subarray ending at current position
```

```
        min_ending_here = min(0, min_ending_here + num)
```

```
        min_sum = min(min_sum, min_ending_here)
```

```
    # The maximum absolute sum is the maximum of the absolute min sum and max sum
```

```
    return max(max_sum, abs(min_sum))
```

```
# Example usage:
```

```
nums = [1, -3, 2, 3, -4]
```

```
print(maxAbsoluteSum(nums)) # Output: 5
```

◆ Quick Review

Key Insights

- The problem can be reduced to finding two quantities: the maximum subarray sum and the minimum subarray sum.
- The maximum absolute sum will be the maximum of these two values, since taking the absolute ensures both positive and negative extremes are accounted for.
- Kadane's algorithm can be used to efficiently find both the maximum subarray sum and the minimum subarray sum in $O(n)$ time.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in the array.

Space Complexity: $O(1)$, since only a constant amount of extra space is used.

Solution

The solution uses Kadane's algorithm twice. First, we calculate the maximum subarray sum by iteratively updating the current maximum and overall maximum. Then, we can modify the

algorithm to get the minimum subarray sum by tracking the current minimum and overall minimum. The answer is then the maximum of the overall maximum and the absolute value of the overall minimum. This approach is efficient and uses constant extra space.

Matrix (2D Array)

Round 5 · Mid · Medium · 1 question

Matrix (2D Array)

□ #289 Game of Life

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation
Lists: AlgoMaster 600

Problem

According to Wikipedia's article: "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970."

The board is made up of an $m \times n$ grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules (taken from the above Wikipedia article):

1. Any live cell with fewer than two live neighbors dies as if caused by under-population.
2. Any live cell with two or three live neighbors lives on to the next generation.

3. Any live cell with more than three live neighbors dies, as if by over-population.
4. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

The next state of the board is determined by applying the above rules simultaneously to every cell in the current state of the $m \times n$ grid board. In this process, births and deaths occur simultaneously.

Given the current state of the board, update the board to reflect its next state.

Note that you do not need to return anything.

Example 1:

0	1	0		0	0	0
0	0	1		1	0	1
1	1	1	→	0	1	1
0	0	0		0	1	0

Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
 Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]

Example 2:

1	1		1	1
1	0	→	1	1

Input: board = [[1,1],[1,0]]
 Output: [[1,1],[1,1]]

Constraints:

- `m == board.length`
- `n == board[i].length`
- `1 <= m, n <= 25`
- `board[i][j]` is 0 or 1.

Follow up:

- Could you solve it in-place? Remember that the board needs to be updated simultaneously: You cannot update some cells first and then use their updated values to update other cells.
- In this question, we represent the board using a 2D array. In principle, the board is infinite, which would cause problems when the active area encroaches upon the border of the array (i.e., live cells reach the border). How would you address these problems?

Community Solutions (Python)

- ☐
- WalkCC

```

class Solution:
    def gameOfLife(self, board: list[list[int]]) -> None:
        m = len(board)
        n = len(board[0])

        for i in range(m):
            for j in range(n):
                ones = 0
                for x in range(max(0, i - 1), min(m, i + 2)):
                    for y in range(max(0, j - 1), min(n, j + 2)):

                        ones += board[x][y] & 1
                # Any live cell with two or three live neighbors lives on to the next
                # generation.
                if board[i][j] == 1 and (ones == 3 or ones == 4):
                    board[i][j] |= 0b10
                # Any dead cell with exactly three live neighbors becomes a live cell,
                # as if by reproduction.
                if board[i][j] == 0 and ones == 3:
                    board[i][j] |= 0b10

        for i in range(m):
            for j in range(n):
                board[i][j] >>= 1

```

• LeetDoocs

```

class Solution:
    def gameOfLife(self, board: List[List[int]]) -> None:
        m, n = len(board), len(board[0])
        for i in range(m):
            for j in range(n):
                live = -board[i][j]
                for x in range(i - 1, i + 2):
                    for y in range(j - 1, j + 2):
                        if 0 <= x < m and 0 <= y < n and board[x][y] > 0:
                            live += 1

```

```

        if board[i][j] and (live < 2 or live > 3):
            board[i][j] = 2
        if board[i][j] == 0 and live == 3:
            board[i][j] = -1
    for i in range(m):
        for j in range(n):
            if board[i][j] == 2:
                board[i][j] = 0
            elif board[i][j] == -1:
                board[i][j] = 1

```

• SimplyLeet

```

# Python solution
class Solution:
    def gameOfLife(self, board: list[list[int]]) -> None:
        # Dimensions of the board
        rows, cols = len(board), len(board[0])

        # Helper function to count live neighbors
        def count_live_neighbors(r, c):
            live = 0
            for i in range(r-1, r+2): # Loop over row neighbors

                for j in range(c-1, c+2): # Loop over column neighbors
                    if (i == r and j == c) or i < 0 or j < 0 or i >= rows or j
                    >= cols:
                        continue # Skip the cell itself and out-of-bound
                    indices

                    # If neighbor was originally live (1 or -1)
                    if abs(board[i][j]) == 1:
                        live += 1
            return live

        # First pass: update board using intermediate states
        for r in range(rows):

```



```

        for c in range(cols):
            live_neighbors = count_live_neighbors(r, c)
            # Rule 1 or Rule 3: live cell dies if fewer than 2 or more than
3 neighbors
            if board[r][c] == 1 and (live_neighbors < 2 or live_neighbors
> 3):
                board[r][c] = -1 # Mark as cell that was live but now
becomes dead
            # Rule 4: dead cell becomes live if exactly 3 live neighbors
            if board[r][c] == 0 and live_neighbors == 3:
                board[r][c] = 2 # Mark as cell that was dead but now
becomes live

        # Second pass: finalize the board by converting intermediate states to
final states

        for r in range(rows):
            for c in range(cols):
                # If cell has positive value, set to live; otherwise, set to
dead
                board[r][c] = 1 if board[r][c] > 0 else 0

```

◆ Quick Review

Key Insights

- The update must happen simultaneously, so changes in one cell cannot immediately affect its neighbors.
- Use an in-place strategy by encoding the transition states within the board.
- Introduce temporary values to represent state changes: for example, mark a cell that was live but now dead, and a cell that was dead but now live.

- When counting live neighbors, consider the cell's original state, which can be recovered using the absolute value or a defined mapping.

Space & Time

Time Complexity: $O(m * n)$ – we check each cell and its at most 8 neighbors.

Space Complexity: $O(1)$ – updates are performed in-place without extra auxiliary storage.

Solution

We can solve the problem in-place by using intermediate states:

- Use -1 to represent a cell that was originally live (1) but becomes dead (0).
- Use 2 to represent a cell that was originally dead (0) but becomes live (1).

During the first pass, for each cell count the live neighbors (treating a cell as originally live if its value is 1 or -1). Then, update the cell using the Game of Life rules:

- If a cell is dead (0) and has exactly 3 live neighbors, mark it as 2 (becomes live).

- If a cell is live (1) and has fewer than 2 or more than 3 live neighbors, mark it as -1 (dies).

In a second pass, finalize the board by converting cells:

- If the value is > 0 (either 1 or 2), set the cell to 1.
- Otherwise, set the cell to 0.

This method leverages in-place modifications while correctly computing neighbor counts based on original states.

LinkedList In-place Reversal

Round 5 · Mid · Medium · 1 question

LinkedList In-place Reversal

□ #92 Reverse Linked List II

MED

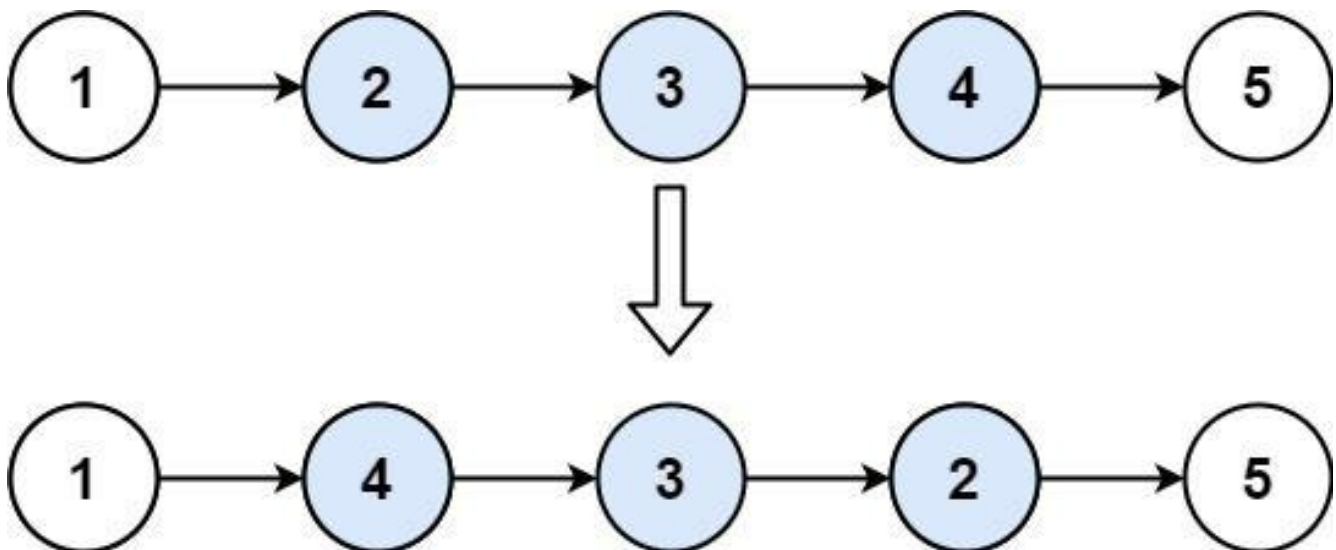
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List

Lists: AlgoMaster 600

Problem

Given the head of a singly linked list and two integers $left$ and $right$ where $left \leq right$, reverse the nodes of the list from position $left$ to position $right$, and return the reversed list.

Example 1:



Input: head = [1,2,3,4,5], left = 2, right = 4
Output: [1,4,3,2,5]

Example 2:

Input: head = [5], left = 1, right = 1
 Output: [5]

Constraints:

- The number of nodes in the list is n .
- $1 \leq n \leq 500$
- $-500 \leq \text{Node.val} \leq 500$
- $1 \leq \text{left} \leq \text{right} \leq n$

Follow up: Could you do it in one pass?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverseBetween(
        self,
        head: ListNode | None,
        left: int,
        right: int,
    ) -> ListNode | None:
        if left == 1:
            return self.reverseN(head, right)

        head.next = self.reverseBetween(head.next, left - 1, right - 1)
        return head

    def reverseN(self, head: ListNode | None, n: int) -> ListNode | None:
        if n == 1:
            return head

        newHead = self.reverseN(head.next, n - 1)
        headNext = head.next
        head.next = headNext.next
```

```
headNext.next = head
return newHead
```

```
class Solution:
    def reverseBetween(self, head: ListNode, m: int, n: int) -> ListNode:
        if not head and m == n:
            return head

        dummy = ListNode(0, head)
        prev = dummy

        for _ in range(m - 1):
            prev = prev.next # Point to the node before the sublist [m, n].

        tail = prev.next # Be the tail of the sublist [m, n].

        # Reverse the sublist [m, n] one by one.
        for _ in range(n - m):
            cache = tail.next
            tail.next = cache.next
            cache.next = prev.next
            prev.next = cache

        return dummy.next
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseBetween(
        self, head: Optional[ListNode], left: int, right: int
    ) -> Optional[ListNode]:
        if head.next is None or left == right:
```

```

        return head
    dummy = ListNode(0, head)
    pre = dummy
    for _ in range(left - 1):
        pre = pre.next
    p, q = pre, pre.next
    cur = q
    for _ in range(right - left + 1):
        t = cur.next
        cur.next = pre

        pre, cur = cur, t
    p.next = pre
    q.next = cur
    return dummy.next

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reverseBetween(self, head: ListNode, left: int, right: int) -> ListNode:
        # Create a dummy node that points to the head to simplify edge cases.
        dummy = ListNode(0)

        dummy.next = head
        # Initialize pointer 'prev' to the dummy node.
        prev = dummy
        # Move 'prev' to the node immediately before the left-th node.
        for i in range(left - 1):
            prev = prev.next
        # 'curr' will point to the first node of the sublist to be reversed.
        curr = prev.next
        # Reverse the sublist from left to right.
        for i in range(right - left):

```



```
temp = curr.next          # 'temp' is the node to be moved.
curr.next = temp.next     # Bypass 'temp' in the current sublist.
temp.next = prev.next     # Insert 'temp' right after 'prev'.
prev.next = temp          # Update 'prev' to point to the moved
node.
# Return the new head, which is dummy.next.
return dummy.next
```

◆ Quick Review

Key Insights

- Use a dummy node to simplify edge cases.
- Traverse the list to locate the node immediately before the reversal section.
- Reverse the sublist in-place by adjusting pointers iteratively.
- Reconnect the reversed sublist back to the original list.
- Achieve the reversal in one pass through the list.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution uses an in-place reversal technique. A dummy node is created to handle

cases where the reversal starts at the head. First, traverse to the node immediately before the left-th node. Then, reverse the sublist by iteratively moving the next node in the sublist to the front of the reversed portion. Finally, reconnect the reversed section with the remainder of the list. The key trick here is the iterative reversal within the sublist which ensures that the overall reversal is done in a single pass with constant extra space.

Monotonic Stack

Round 5 · Mid · Medium · 9 questions

Monotonic Stack

□ #402 Remove K Digits

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Greedy · Monotonic Stack
Lists: AlgoMaster 600

Problem

Given string `num` representing a non-negative integer `num`, and an integer `k`, return the smallest possible integer after removing `k` digits from `num`.

Example 1:

```
Input: num = "1432219", k = 3
Output: "1219"
Explanation: Remove the three digits 4, 3, and 2 to form the new number 1219
which is the smallest.
```

Example 2:

```
Input: num = "10200", k = 1
Output: "200"
Explanation: Remove the leading 1 and the number is 200. Note that the output
must not contain leading zeroes.
```

Example 3:

```
Input: num = "10", k = 2
Output: "0"
Explanation: Remove all the digits from the number and it is left with nothing
which is 0.
```

Constraints:

- $1 \leq k \leq \text{num.length} \leq 105$
- num consists of only digits.
- num does not have any leading zeros except for the zero itself.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeKdigits(self, num: str, k: int) -> str:
        if len(num) == k:
            return '0'

        ans = []
        stack = []

        for i, digit in enumerate(num):
            while k > 0 and stack and stack[-1] > digit:

                stack.pop()
                k -= 1
            stack.append(digit)

        for _ in range(k):
            stack.pop()

        for c in stack:
            if c == '0' and not ans:
                continue
            ans.append(c)

        return ''.join(ans) if ans else '0'
```

• LeetDoocs

```
class Solution:
    def removeKdigits(self, num: str, k: int) -> str:
        stk = []
        remain = len(num) - k
        for c in num:
            while k and stk and stk[-1] > c:
                stk.pop()
                k -= 1
            stk.append(c)
        return ''.join(stk[:remain]).lstrip('0') or '0'
```

• SimplyLeet

```
# Python solution with line-by-line comments
def removeKdigits(num: str, k: int) -> str:
    # Initialize a stack to build the final number.
    stack = []

    # Traverse each digit in the input number.
    for digit in num:
        # Remove digits from the stack if the current digit is smaller,
        # making sure we still have removals available.
        while k > 0 and stack and stack[-1] > digit:

            stack.pop()
            k -= 1
        # Append the current digit to the stack.
        stack.append(digit)

    # If there are still removals left, remove digits from the end.
    while k > 0:
        stack.pop()
        k -= 1

    # Construct the result string and remove any leading zeros.
    result = ''.join(stack).lstrip('0')

    # Return "0" if the resulting string is empty.
    return result if result else "0"

# Example usage:
# print(removeKdigits("1432219", 3)) # expected output: "1219"
```

◆ Quick Review

Key Insights

- Use a greedy approach that removes digits to make the number as small as possible.
- Utilize a stack (monotonic increasing sequence) to decide whether to keep or remove a digit.
- Remove any excess digits from the end if $k > 0$ after the primary loop.
- Handle edge cases including leading zeros and the possibility of removing all digits.

Space & Time

Time Complexity: $O(n)$, where n is the length of the input string.

Space Complexity: $O(n)$, needed for the stack.

Solution

We apply a greedy algorithm using a monotonic stack. For each digit in the number, if the current digit is smaller than the last digit on the stack and we still have removals ($k > 0$), we pop from the stack. This removal helps in making the constructed number smaller.

After iterating over the entire number, if any removals remain, we trim the number from the end. Finally, we join the resulting digits, strip leading zeros, and if the result is empty, we return "0".

Monotonic Stack

□ #456 132 Pattern

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Stack · Monotonic Stack
· Ordered Set

Lists: AlgoMaster 600

Problem

Given an array of n integers `nums`, a 132 pattern is a subsequence of three integers `nums[i]`, `nums[j]` and `nums[k]` such that $i < j < k$ and $nums[i] < nums[k] < nums[j]$.

Return `true` if there is a 132 pattern in `nums`, otherwise, return `false`.

Example 1:

Input: `nums = [1,2,3,4]`
Output: `false`
Explanation: There is no 132 pattern in the sequence.

Example 2:

Input: `nums = [3,1,4,2]`
Output: `true`
Explanation: There is a 132 pattern in the sequence: `[1, 4, 2]`.

Example 3:

Input: `nums = [-1,3,2,0]`

Output: `true`

Explanation: There are three 132 patterns in the sequence: `[-1, 3, 2]`, `[-1, 3, 0]` and `[-1, 2, 0]`.

Constraints:

- `n == nums.length`
- `1 <= n <= 2 * 105`
- `-109 <= nums[i] <= 109`

Community Solutions (Python)

• WalkCC

```
class Solution:
    def find132pattern(self, nums: list[int]) -> bool:
        stack = [] # a decreasing stack
        ak = -math.inf # Find a seq, where ai < ak < aj.

        for num in reversed(nums):
            # If ai < ak, done because ai must < aj.
            if num < ak:
                return True
            while stack and stack[-1] < num:

                ak = stack[-1]
                stack.pop()
            stack.append(num) # `nums[i]` is a candidate of aj.

        return False
```

• LeetDoocs

```
class Solution:
    def find132pattern(self, nums: List[int]) -> bool:
        vk = -inf
        stk = []
        for x in nums[::-1]:
            if x < vk:
                return True
            while stk and stk[-1] < x:
                vk = stk.pop()
            stk.append(x)
```

```
        return False
```

```
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x, delta):
        while x <= self.n:
            self.c[x] += delta
            x += x & -x
```

```
    def query(self, x):
        s = 0
        while x:
            s += self.c[x]
            x -= x & -x
        return s
```

```
class Solution:
    def find132pattern(self, nums: List[int]) -> bool:
```

```

s = sorted(set(nums))
n = len(nums)
left = [inf] * (n + 1)
for i, x in enumerate(nums):
    left[i + 1] = min(left[i], x)
tree = BinaryIndexedTree(len(s))
for i in range(n - 1, -1, -1):
    x = bisect_left(s, nums[i]) + 1
    y = bisect_left(s, left[i]) + 1
    if x > y and tree.query(x - 1) > tree.query(y):

        return True
    tree.update(x, 1)
return False

```

• SimplyLeet

Python solution for the 132 Pattern problem

```

def find132pattern(nums):
    # Initialize candidate for nums[k] with a very small value
    candidate = float('-inf')
    stack = [] # Stack to keep potential nums[j] elements

    # Traverse the array in reverse order
    for num in reversed(nums):
        # If current number is less than candidate, we found a valid 132
        pattern

        if num < candidate:
            return True

        # Update the candidate while current number is greater than the element
        at stack's top
        while stack and num > stack[-1]:
            candidate = stack.pop()

        # Push the current number onto the stack for potential future use
        stack.append(num)
    return False

# Example usage:

# print(find132pattern([3, 1, 4, 2])) # Output: True

```

◆ Quick Review

Key Insights

- The challenge is to efficiently detect a subsequence that satisfies $\text{nums}[i] \leq \text{nums}[k] \leq \text{nums}[j]$.
- A brute force approach checking all triples would be too slow given the constraints.
- Using a monotonic stack in reverse order allows us to keep track of potential candidates for $\text{nums}[k]$ and check for a valid pattern.
- We maintain a variable to store the possible "2" element (candidate for $\text{nums}[k]$) as we move from right to left.

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in the array.

Space Complexity: $O(n)$ in the worst case for the stack.

Solution

We solve the problem by iterating through the list in reverse order while using a stack to maintain potential candidates for the "middle" element ($\text{nums}[j]$). We also use a variable (let's

call it candidate) to represent the best candidate for `nums[k]` that we have seen so far. For each element in the reversed array:

- If the current element is less than the candidate, we have found a valid 132 pattern.
- Otherwise, while the stack is not empty and the current element is greater than the top of the stack, we update the candidate by popping from the stack.
- Push the current element onto the stack as a potential future candidate. This approach efficiently identifies whether a 132 pattern exists.

Monotonic Stack

□ #503 Next Greater Element II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: AlgoMaster 600

Problem

Given a circular integer array `nums` (i.e., the next element of `nums[nums.length - 1]` is `nums[0]`), return the next greater number for every element in `nums`.

The next greater number of a number `x` is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return `-1` for this number.

Example 1:

Input: `nums = [1,2,1]`

Output: `[2,-1,2]`

Explanation: The first 1's next greater number is 2;

The number 2 can't find next greater number.

The second 1's next greater number needs to search circularly, which is also 2.

Example 2:

Input: `nums = [1,2,3,4,3]`

Output: `[2,3,4,-1,4]`

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-109 \leq \text{nums}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def nextGreaterElements(self, nums: list[int]) -> list[int]:
        n = len(nums)
        ans = [-1] * n
        stack = [] # a decreasing stack storing indices

        for i in range(n * 2):
            num = nums[i % n]
            while stack and nums[stack[-1]] < num:
                ans[stack.pop()] = num

            if i < n:
                stack.append(i)

        return ans
```

• LeetDoocs

```
class Solution:
    def nextGreaterElements(self, nums: List[int]) -> List[int]:
        n = len(nums)
        ans = [-1] * n
        stk = []
        for i in range(n * 2 - 1, -1, -1):
            i %= n
            while stk and stk[-1] <= nums[i]:
                stk.pop()
            if stk:
                ans[i] = stk[-1]
            stk.append(nums[i])
        return ans
```


• SimplyLeet

```
# Python solution
def nextGreaterElements(nums):
    n = len(nums)
    result = [-1] * n # Initialize result list with -1
    stack = [] # Stack to store indices

    # Traverse the list twice to account for circularity
    for i in range(2 * n - 1, -1, -1):
        current_index = i % n # Get current index in circular manner
        # Remove elements that are less than or equal to nums[current_index]

        while stack and nums[stack[-1]] <= nums[current_index]:
            stack.pop()
        # If stack is not empty, the top element is the next greater element
        if stack:
            result[current_index] = nums[stack[-1]]
        # Push current index to the stack for future comparisons
        stack.append(current_index)

    return result

# Example usage:
# nums = [1, 2, 1]
# print(nextGreaterElements(nums)) # Output: [2, -1, 2]
```

◆ Quick Review

Key Insights

- Because the array is circular, we simulate the wrap-around by iterating over the array twice.
- A monotonic stack (decreasing order) is used to efficiently track the next greater element.

- Process elements in reverse order to handle dependencies correctly.
- Each element is pushed and popped at most once, yielding an efficient solution.

Space & Time

Time Complexity: $O(n)$, since each element is processed a constant number of times.

Space Complexity: $O(n)$, used for the result array and the stack.

Solution

This solution employs a monotonic stack to find the next greater element efficiently. We traverse the array twice (using modulo arithmetic to simulate circular behavior) from right to left. For each element, we pop elements from the stack until we find one that is greater than the current element. If such an element exists, it is the next greater element; otherwise, the result remains -1 . Finally, we push the current index to the stack. This approach guarantees that every element is considered only a few times.

Monotonic Stack

□ #581 Shortest Unsorted Continuous Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Stack · Greedy · Sorting
· Monotonic Stack
Lists: AlgoMaster 600

Problem

Given an integer array `nums`, you need to find one continuous subarray such that if you only sort this subarray in non-decreasing order, then the whole array will be sorted in non-decreasing order.

Return the shortest such subarray and output its length.

Example 1:

Input: `nums = [2,6,4,8,10,9,15]`

Output: 5

Explanation: You need to sort `[6, 4, 8, 10, 9]` in ascending order to make the whole array sorted in ascending order.

Example 2:

Input: `nums = [1,2,3,4]`

Output: 0

Example 3:

Input: `nums = [1]`

Output: `0`

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-105 \leq \text{nums}[i] \leq 105$

Follow up: Can you solve it in $O(n)$ time complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findUnsortedSubarray(self, nums: list[int]) -> int:
        mn = math.inf
        mx = -math.inf
        flag = False

        for i in range(1, len(nums)):
            if nums[i] < nums[i - 1]:
                flag = True
            if flag:
                mn = min(mn, nums[i])

        flag = False

        for i in reversed(range(len(nums) - 1)):
            if nums[i] > nums[i + 1]:
                flag = True
            if flag:
                mx = max(mx, nums[i])
```

```

for l in range(len(nums)):
    if nums[l] > mn:
        break

for r, num in reversed(list(enumerate(nums))):
    if num < mx:
        break

return 0 if l >= r else r - l + 1

```

• LeetDoocs

```

class Solution:
    def findUnsortedSubarray(self, nums: List[int]) -> int:
        arr = sorted(nums)
        l, r = 0, len(nums) - 1
        while l <= r and nums[l] == arr[l]:
            l += 1
        while l <= r and nums[r] == arr[r]:
            r -= 1
        return r - l + 1

```

```

class Solution:
    def findUnsortedSubarray(self, nums: List[int]) -> int:
        mi, mx = inf, -inf
        l = r = -1
        n = len(nums)
        for i, x in enumerate(nums):
            if mx > x:
                r = i
            else:
                mx = x

        if mi < nums[n - i - 1]:
            l = n - i - 1
        else:
            mi = nums[n - i - 1]
        return 0 if r == -1 else r - l + 1

```

• SimplyLeet

```
# Python solution with detailed comments
def findUnsortedSubarray(nums):
    n = len(nums)
    left, right = -1, -1
    max_seen = nums[0]
    min_seen = nums[-1]

    # Scan from left to right to find the right boundary
    for i in range(1, n):
        max_seen = max(max_seen, nums[i])

    # if current element is less than the max seen so far, update right
    # boundary
    if nums[i] < max_seen:
        right = i

    # Scan from right to left to find the left boundary
    for i in range(n - 2, -1, -1):
        min_seen = min(min_seen, nums[i])
        # if current element is greater than the min seen so far, update left
        # boundary
        if nums[i] > min_seen:
            left = i

    # If right boundary was never updated, array is sorted.
    if right == -1:
        return 0
    return right - left + 1

# Example usage:
print(findUnsortedSubarray([2,6,4,8,10,9,15])) # Output: 5
```

◆ Quick Review

Key Insights

- The sorted order of the array determines the positions where the actual array deviates from the sorted order.

- A straightforward solution is to compare the array with a sorted copy and identify the first and last indices where they differ.
- A more optimal $O(n)$ approach involves traversing the array from left-to-right and right-to-left:
 - Left-to-right pass: Track the maximum value seen. Whenever the current number is less than this maximum, it indicates the elements are not in order; update the right boundary accordingly.
 - Right-to-left pass: Track the minimum value seen. Whenever the current number is greater than this minimum, update the left boundary.
- If the array is already sorted, the boundaries never update, and the result is 0.
- This approach uses constant extra space (besides a few variables), making it efficient for large inputs.

Space & Time

Time Complexity: $O(n)$ – The array is traversed twice.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

The solution uses a two-pointer greedy approach:

- In the first pass (left-to-right), track the maximum value encountered so far. When a current element is found smaller than this maximum, mark its position as a potential right boundary.
 - In the second pass (right-to-left), track the minimum value encountered so far. When a current element is larger than this minimum, mark its position as a potential left boundary.
 - The difference between these boundary indices (plus one) yields the length of the shortest unsorted subarray.
 - If no such boundaries are found, then the array is already sorted, and the answer is 0.
- This method avoids extra space for a sorted copy and achieves optimal time performance.

Monotonic Stack

□ #769 Max Chunks To Make Sorted

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Greedy · Sorting · Monotonic
Stack

Lists: AlgoMaster 600

Problem

You are given an integer array `arr` of length `n` that represents a permutation of the integers in the range `[0, n - 1]`.

We split `arr` into some number of chunks (i.e., partitions), and individually sort each chunk. After concatenating them, the result should equal the sorted array.

Return the largest number of chunks we can make to sort the array.

Example 1:

Input: `arr = [4,3,2,1,0]`

Output: 1

Explanation:

Splitting into two or more chunks will not return the required result.

For example, splitting into `[4, 3]`, `[2, 1, 0]` will result in `[3, 4, 0, 1, 2]`, which isn't sorted.

Example 2:

Input: arr = [1,0,2,3,4]

Output: 4

Explanation:

We can split into two chunks, such as [1, 0], [2, 3, 4].

However, splitting into [1, 0], [2], [3], [4] is the highest number of chunks possible.

Constraints:

- $n == \text{arr.length}$
- $1 \leq n \leq 10$
- $0 \leq \text{arr}[i] < n$
- All the elements of arr are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxChunksToSorted(self, arr: list[int]) -> int:
        ans = 0
        mx = -math.inf

        for i, a in enumerate(arr):
            mx = max(mx, a)
            if mx == i:
                ans += 1

        return ans
```

• LeetDoocs

```
class Solution:
    def maxChunksToSorted(self, arr: List[int]) -> int:
        mx = ans = 0
        for i, v in enumerate(arr):
            mx = max(mx, v)
            if i == mx:
                ans += 1
        return ans
```

```
class Solution:
    def maxChunksToSorted(self, arr: List[int]) -> int:
        stk = []
        for v in arr:
            if not stk or v >= stk[-1]:
                stk.append(v)
            else:
                mx = stk.pop()
                while stk and stk[-1] > v:
                    stk.pop()
                stk.append(mx)
        return len(stk)
```

• SimplyLeet

```
# Define the function
def max_chunks_to_sorted(arr):
    # Initialize count of chunks and the maximum value seen so far
    chunk_count = 0
    max_so_far = 0

    # Iterate over the array with index
    for i, num in enumerate(arr):
        # Update the maximum value encountered
        max_so_far = max(max_so_far, num)
```

```
# Check if the current max equals the index
if max_so_far == i:
    # Valid chunk identified, increment count
    chunk_count += 1

# Return the total number of chunks
return chunk_count

# Sample test
print(max_chunks_to_sorted([1,0,2,3,4])) # Expected output: 4
```

◆ Quick Review

Key Insights

- Since the input is a permutation of 0 through $n-1$, the correctly sorted array is exactly $[0, 1, 2, \dots, n-1]$.
- While iterating through the array, if the maximum value seen so far is equal to the current index, then all elements up to the current index can form one chunk.
- A greedy approach by iterating and making a cut whenever `max_so_far` equals the current index ensures the maximum number of chunks.

Space & Time

Time Complexity: $O(n)$ – Single pass through the array.

Space Complexity: $O(1)$ – Only a few auxiliary variables are used.

Solution

The algorithm uses a greedy approach with a single linear scan of the array. A variable is maintained to track the maximum value seen so far during the iteration. At each index, if the maximum value equals the current index, this indicates that all numbers up to that index are exactly the set $\{0, 1, \dots, i\}$, thus permitting a chunk division. The process continues until the end of the array, and the count of chunk boundaries is returned as the maximum number of chunks.

Monotonic Stack

□ #901 Online Stock Span

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Design · Monotonic Stack · Data Stream
Lists: AlgoMaster 600

Problem

Design an algorithm that collects daily price quotes for some stock and returns the span of that stock's price for the current day.

The span of the stock's price in one day is the maximum number of consecutive days (starting from that day and going backward) for which the stock price was less than or equal to the price of that day.

- For example, if the prices of the stock in the last four days is [7,2,1,2] and the price of the stock today is 2, then the span of today is 4 because starting from today, the price of the stock was less than or equal 2 for 4 consecutive days.
- Also, if the prices of the stock in the last four days is [7,34,1,2] and the price of the

stock today is 8, then the span of today is 3 because starting from today, the price of the stock was less than or equal 8 for 3 consecutive days.

Implement the StockSpanner class:

- **StockSpanner()** Initializes the object of the class.
- **int next(int price)** Returns the span of the stock's price given that today's price is price.

Example 1:

Input

```
["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]  
[[], [100], [80], [60], [70], [60], [75], [85]]
```

Output

```
[null, 1, 1, 1, 2, 1, 4, 6]
```

Explanation

```
StockSpanner stockSpanner = new StockSpanner();
```

Constraints:

- $1 \leq \text{price} \leq 105$
- At most 104 calls will be made to next.

Community Solutions (Python)

- WalkCC

```
class StockSpanner:
    def __init__(self):
        self.stack = [] # (price, span)

    def next(self, price: int) -> int:
        span = 1
        while self.stack and self.stack[-1][0] <= price:
            span += self.stack.pop()[1]
        self.stack.append((price, span))
        return span
```

• LeetCode

```
class StockSpanner:
    def __init__(self):
        self.stk = []

    def next(self, price: int) -> int:
        cnt = 1
        while self.stk and self.stk[-1][0] <= price:
            cnt += self.stk.pop()[1]
        self.stk.append((price, cnt))
        return cnt
```

```
# Your StockSpanner object will be instantiated and called as such:
# obj = StockSpanner()
# param_1 = obj.next(price)
```

• SimplyLeet


```
class StockSpanner:
    def __init__(self):
        # Stack will store pairs of (price, span)
        self.stack = []

    def next(self, price: int) -> int:
        span = 1 # Start with a span of 1 for the current price
        # While the stack is not empty and the current price is greater than or
        # equal to the price on top of the stack
        while self.stack and self.stack[-1][0] <= price:
            # Add the span from the popped element to the current span

            span += self.stack.pop()[1]
        # Push the current price and its span onto the stack
        self.stack.append((price, span))
        return span

# Example usage:
# stockSpanner = StockSpanner()
# print(stockSpanner.next(100)) # Output: 1
# print(stockSpanner.next(80)) # Output: 1
# print(stockSpanner.next(60)) # Output: 1

# print(stockSpanner.next(70)) # Output: 2
# print(stockSpanner.next(60)) # Output: 1
# print(stockSpanner.next(75)) # Output: 4
# print(stockSpanner.next(85)) # Output: 6
```

◆ Quick Review

Key Insights

- The problem can be solved efficiently using a monotonic stack.
- Store pairs of (price, span) in the stack, where span is the number of consecutive days whose prices are less than or equal to that

price.

- For every new price, pop elements from the stack while they are less than or equal to the current price, and accumulate their spans.
- This approach ensures that each price is processed only once, leading to an amortized $O(1)$ time per operation.

Space & Time

Time Complexity: Amortized $O(1)$ per call to next

Space Complexity: $O(n)$ where n is the number of price quotes stored in the stack

Solution

We use a monotonic stack to keep track of prices and their associated spans. Each time the next price is introduced, we:

- Initialize a span value of 1 for the current day.
- While the stack is not empty and the price at the top of the stack is less than or equal to the current price, pop the pair and add its span to the current span.

- Push the current price and its computed span onto the stack.
- Return the span.

This works because the popped elements represent a sequence of consecutive days where the stock prices were less than or equal to the current price.

Monotonic Stack

□ #907 Sum of Subarray Minimums

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Stack ·
Monotonic Stack
Lists: AlgoMaster 600

Problem

Given an array of integers `arr`, find the sum of `min(b)`, where `b` ranges over every (contiguous) subarray of `arr`. Since the answer may be large, return the answer modulo $10^9 + 7$.

Example 1:

```
Input: arr = [3,1,2,4]
Output: 17
Explanation:
Subarrays are [3], [1], [2], [4], [3,1], [1,2], [2,4], [3,1,2], [1,2,4],
[3,1,2,4].
Minimums are 3, 1, 2, 4, 1, 1, 2, 1, 1, 1.
Sum is 17.
```

Example 2:

```
Input: arr = [11,81,94,43,3]
Output: 444
```

Constraints:

- $1 \leq \text{arr.length} \leq 3 \times 10^4$

- $1 \leq \text{arr}[i] \leq 3 * 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sumSubarrayMins(self, arr: list[int]) -> int:
        MOD = 1_000_000_007
        n = len(arr)
        ans = 0
        # prevMin[i] := index k s.t. arr[k] is the previous minimum in arr[:i]
        prevMin = [-1] * n
        # nextMin[i] := index k s.t. arr[k] is the next minimum in arr[i + 1:]
        nextMin = [n] * n
        stack = []

        for i, a in enumerate(arr):
            while stack and arr[stack[-1]] > a:
                index = stack.pop()
                nextMin[index] = i
            if stack:
                prevMin[i] = stack[-1]
            stack.append(i)

        for i, a in enumerate(arr):
            ans += a * (i - prevMin[i]) * (nextMin[i] - i)
            ans %= MOD

        return ans
```

• LeetDoocs

```

class Solution:
    def sumSubarrayMins(self, arr: List[int]) -> int:
        n = len(arr)
        left = [-1] * n
        right = [n] * n
        stk = []
        for i, v in enumerate(arr):
            while stk and arr[stk[-1]] >= v:
                stk.pop()
            if stk:
                left[i] = stk[-1]
            stk.append(i)

        stk = []
        for i in range(n - 1, -1, -1):
            while stk and arr[stk[-1]] > arr[i]:
                stk.pop()
            if stk:
                right[i] = stk[-1]
            stk.append(i)

        mod = 10**9 + 7
        return sum((i - left[i]) * (right[i] - i) * v for i, v in enumerate(arr)) % mod

```

• SimplyLeet

```

# Python solution with detailed comments
class Solution:
    def sumSubarrayMins(self, arr):
        MOD = 10**9 + 7 # modulo value
        n = len(arr)
        left = [0] * n # stores the number of contiguous subarrays ending at
        # i where arr[i] is the minimum
        right = [0] * n # stores the number of contiguous subarrays starting
        # at i where arr[i] is the minimum

        stack = []
        # Calculate left spans

```

```
for i in range(n):
    count = 1 # each element stands as a subarray itself
    # Pop while top element is greater than arr[i]
    while stack and arr[stack[-1][0]] > arr[i]:
        count += stack.pop()[1]
    stack.append((i, count))
    left[i] = count

stack = []
# Calculate right spans by iterating from right to left

for i in range(n - 1, -1, -1):
    count = 1
    # Use >= here for the right side to correctly handle duplicates
    while stack and arr[stack[-1][0]] >= arr[i]:
        count += stack.pop()[1]
    stack.append((i, count))
    right[i] = count

total = 0
# Combine the contributions of each element

for i in range(n):
    total = (total + arr[i] * left[i] * right[i]) % MOD
return total
```

◆ Quick Review

Key Insights

- Leverage a monotonic stack to determine the span over which an element is the minimum.
- For each element, compute its "left" span (number of subarrays ending at the element where it is the minimum) and "right" span (number of subarrays starting at the element where it remains the minimum).

- The contribution of an element is its value multiplied by its left and right spans.
- Use modular arithmetic to handle large numbers.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The approach involves two passes using a monotonic stack. In the first pass, for each element, we count how many contiguous elements on the left (including the current one) are greater than (or equal, with a twist for handling duplicates) this element; this gives the left span. In the second pass, we do a similar traversal from the right to obtain the right span, ensuring that while moving right the criteria for maintaining the minimum includes strict comparisons in order to correctly handle duplicates. Finally, the element's overall contribution is the product of its value, its left span, and its right span. Summing up these contributions for all elements yields the answer modulo $10^9 + 7$.

Monotonic Stack

□ #1019 Next Greater Node In Linked List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Linked List · Stack · Monotonic Stack
Lists: AlgoMaster 600

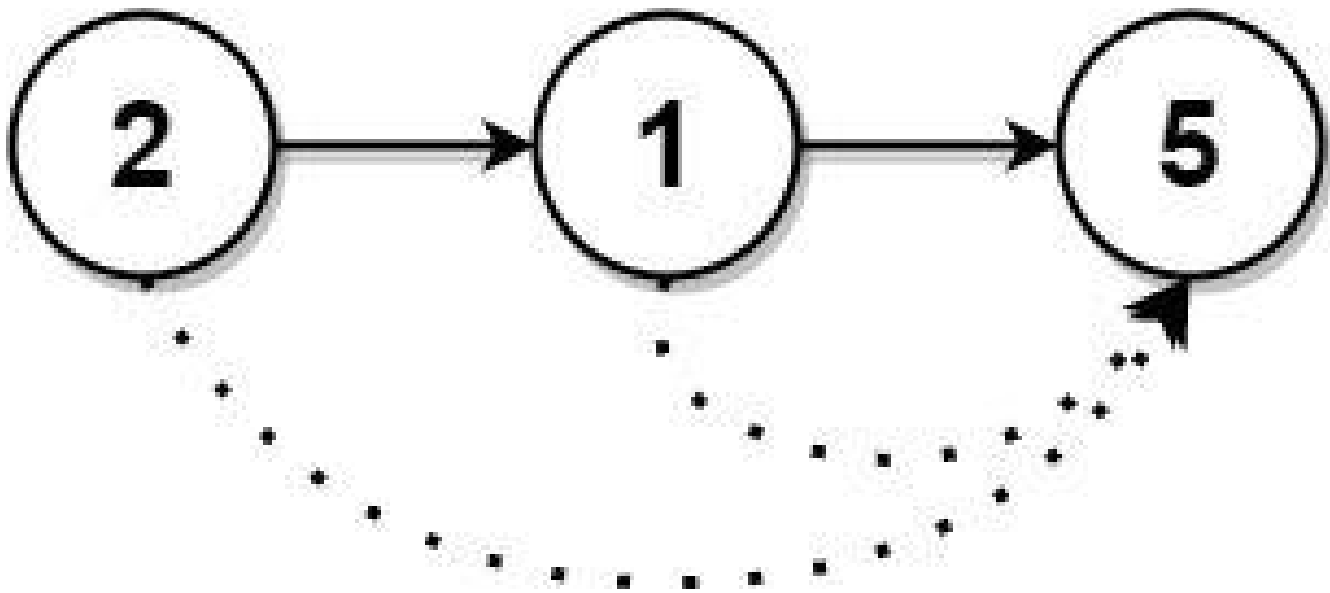
Problem

You are given the head of a linked list with n nodes.

For each node in the list, find the value of the next greater node. That is, for each node, find the value of the first node that is next to it and has a strictly larger value than it.

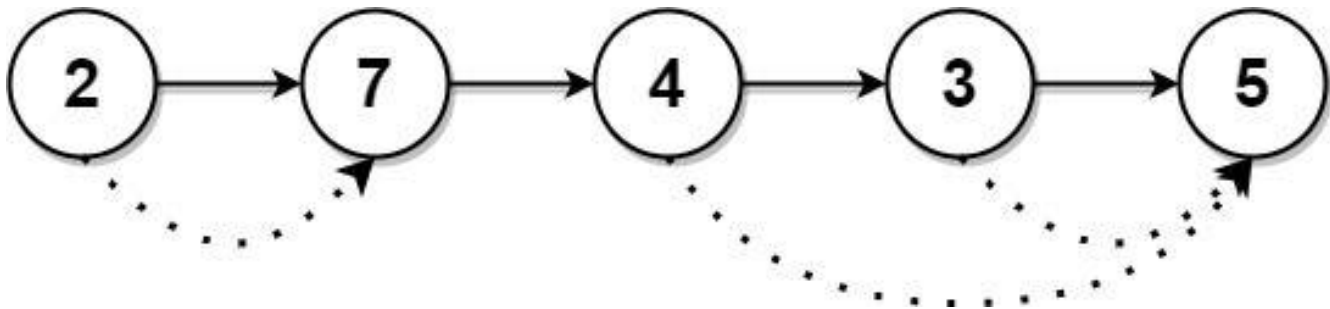
Return an integer array `answer` where `answer[i]` is the value of the next greater node of the i th node (1-indexed). If the i th node does not have a next greater node, set `answer[i] = 0`.

Example 1:



Input: head = [2,1,5]
Output: [5,5,0]

Example 2:



Input: head = [2,7,4,3,5]
Output: [7,0,5,5,0]

Constraints:

- The number of nodes in the list is n .
- $1 \leq n \leq 104$
- $1 \leq \text{Node.val} \leq 109$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def nextLargerNodes(self, head: ListNode) -> list[int]:
        ans = []
        stack = []

        while head:
            while stack and head.val > ans[stack[-1]]:
                index = stack.pop()
                ans[index] = head.val
                stack.append(len(ans))

            ans.append(head.val)
            head = head.next

        for i in stack:
            ans[i] = 0

        return ans

```

• LeetDoocs

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def nextLargerNodes(self, head: Optional[ListNode]) -> List[int]:
        nums = []
        while head:
            nums.append(head.val)

            head = head.next

        stk = []
        n = len(nums)
        ans = [0] * n
        for i in range(n - 1, -1, -1):
            while stk and stk[-1] <= nums[i]:
                stk.pop()
            if stk:
                ans[i] = stk[-1]
            stk.append(nums[i])

```

```
return ans
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def nextLargerNodes(self, head: ListNode) -> list:
        # Convert linked list to array for easy access by index.
        values = []

        current = head
        while current:
            values.append(current.val)
            current = current.next

        # Initialize result array with zeros.
        result = [0] * len(values)
        stack = [] # Stack to store indices of nodes.

        # Iterate through the list using the monotonic stack approach.

        for i, value in enumerate(values):
            # Find all nodes in the stack that have a lesser value than the
            current node.
            while stack and values[stack[-1]] < value:
                index = stack.pop()
                result[index] = value
            stack.append(i)

        return result
```

◆ Quick Review

Key Insights

- Convert the linked list into an array for easier index manipulation.
- Use a monotonic (decreasing) stack to store indices of nodes that haven't found their next greater element.
- As you traverse the array, compare the current value with values corresponding to indices stored in the stack.
- When the current value is larger, update the result for those indices and pop them off the stack.
- Unresolved indices in the stack will have a next greater value of 0.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution begins by converting the linked list into an array, which allows for straightforward index-based traversal. A stack is used to keep track of indices for which we have not yet found a next greater element. As we iterate through the array, if the current element's value is greater than the value at the

index on the top of the stack, it means the current element is the next greater element for that index. We update the result at that index and pop it from the stack. This process continues until the stack is either empty or the current element is not greater than the element referenced by the stack's top index. Finally, any indices remaining in the stack are assigned a value of 0 as there is no greater element after them.

Monotonic Stack

□ #2104 Sum of Subarray Ranges

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: AlgoMaster 600

Problem

You are given an integer array `nums`. The range of a subarray of `nums` is the difference between the largest and smallest element in the subarray.

Return the sum of all subarray ranges of `nums`.
A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

```
Input: nums = [1,2,3]
Output: 4
Explanation: The 6 subarrays of nums are the following:
[1], range = largest - smallest = 1 - 1 = 0
[2], range = 2 - 2 = 0
[3], range = 3 - 3 = 0
[1,2], range = 2 - 1 = 1
[2,3], range = 3 - 2 = 1
```

Example 2:

Input: nums = [1,3,3]

Output: 4

Explanation: The 6 subarrays of nums are the following:

[1], range = largest - smallest = 1 - 1 = 0

[3], range = 3 - 3 = 0

[3], range = 3 - 3 = 0

[1,3], range = 3 - 1 = 2

[3,3], range = 3 - 3 = 0

Example 3:

Input: nums = [4,-2,-3,4,1]

Output: 59

Explanation: The sum of all subarray ranges of nums is 59.

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $-109 \leq \text{nums}[i] \leq 109$

Follow-up: Could you find a solution with $O(n)$ time complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def subArrayRanges(self, nums: list[int]) -> int:
        prevGt, nextGt = self._getPrevNext(nums, operator.lt)
        prevLt, nextLt = self._getPrevNext(nums, operator.gt)
        return sum(num * (i - prevGt[i]) * (nextGt[i] - i) -
                   num * (i - prevLt[i]) * (nextLt[i] - i)
                   for i, num in enumerate(nums))

    def _getPrevNext(
        self,
```



```

    nums: list[int],
    op: callable
) -> tuple[list[int], list[int]]:
    """
    Returns `prev` and `next`, that store the indices of the nearest numbers
    that are smaller or larger than the current number depending on `op`.
    """
    n = len(nums)
    prev = [-1] * n
    next = [n] * n

```

```

    stack = []
    for i, num in enumerate(nums):
        while stack and op(nums[stack[-1]], num):
            next[stack.pop()] = i
        if stack:
            prev[i] = stack[-1]
        stack.append(i)
    return prev, next

```

• LeetCode

```

class Solution:
    def subArrayRanges(self, nums: List[int]) -> int:
        ans, n = 0, len(nums)
        for i in range(n - 1):
            mi = mx = nums[i]
            for j in range(i + 1, n):
                mi = min(mi, nums[j])
                mx = max(mx, nums[j])
                ans += mx - mi
        return ans

```

```

class Solution:
    def subArrayRanges(self, nums: List[int]) -> int:
        def f(nums):
            stk = []
            n = len(nums)
            left = [-1] * n
            right = [n] * n
            for i, v in enumerate(nums):
                while stk and nums[stk[-1]] <= v:
                    stk.pop()

```

```

        if stk:
            left[i] = stk[-1]
        stk.append(i)
    stk = []
    for i in range(n - 1, -1, -1):
        while stk and nums[stk[-1]] < nums[i]:
            stk.pop()
        if stk:
            right[i] = stk[-1]
        stk.append(i)

    return sum((i - left[i]) * (right[i] - i) * v for i, v in enumerate(nums))

mx = f(nums)
mi = f([-v for v in nums])
return mx + mi

```

• SimplyLeet

```

def subArrayRanges(nums):
    n = len(nums)
    # Initialize arrays to store distance contributions for min and max
    left_min = [0] * n
    right_min = [0] * n
    left_max = [0] * n
    right_max = [0] * n

    # Compute left_min: distance to previous less element (strictly less)
    stack = []

    for i in range(n):
        # Maintain an increasing stack for minimums
        while stack and nums[stack[-1]] > nums[i]:
            stack.pop()
        left_min[i] = i - stack[-1] if stack else i + 1
        stack.append(i)

    # Compute right_min: distance to next less or equal element
    stack = []
    for i in range(n - 1, -1, -1):

```

```

# Use >= to handle duplicates correctly
while stack and nums[stack[-1]] >= nums[i]:
    stack.pop()
right_min[i] = stack[-1] - i if stack else n - i
stack.append(i)

# Compute left_max: distance to previous greater element (strictly greater)
stack = []
for i in range(n):
    # Maintain a decreasing stack for maximums

    while stack and nums[stack[-1]] < nums[i]:
        stack.pop()
    left_max[i] = i - stack[-1] if stack else i + 1
    stack.append(i)

# Compute right_max: distance to next greater or equal element
stack = []
for i in range(n - 1, -1, -1):
    # Use <= to handle duplicates correctly
    while stack and nums[stack[-1]] <= nums[i]:

        stack.pop()
    right_max[i] = stack[-1] - i if stack else n - i
    stack.append(i)

# Calculate the final answer using contributions from max and min roles
total = 0
for i in range(n):
    total += nums[i] * left_max[i] * right_max[i]
    total -= nums[i] * left_min[i] * right_min[i]
return total

# Example usage:
print(subArrayRanges([1, 2, 3])) # Output: 4

```

◆ Quick Review

Key Insights

- Instead of enumerating all subarrays (which is $O(n^2)$), calculate the contribution of each element as the maximum and minimum in subarrays.
- Use monotonic stacks to compute, for each element, the number of subarrays where it is the minimum and where it is the maximum.
- Specifically, determine the distance to previous and next less elements (for minimum) and previous and next greater elements (for maximum).
- The overall sum is the aggregated difference of the contributions: $(\text{element} * \text{count_as_max}) - (\text{element} * \text{count_as_min})$.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

We solve the problem by breaking it into two parts: calculating the total contribution when each element acts as the maximum and as the minimum in subarrays. For each index i :

- Compute left and right boundaries indicating how far we can extend while keeping $\text{nums}[i]$

as the unique maximum or minimum using monotonic stacks.

- Multiply these boundaries to determine the number of subarrays in which `nums[i]` is the maximum or minimum.
- Accumulate the aggregate sum by adding `nums[i] * (number of subarrays where it is maximum)` and subtracting `nums[i] * (number of subarrays where it is minimum)`.

This approach leverages monotonic stacks to avoid an $O(n^2)$ enumeration, delivering an $O(n)$ time solution.

Queues

Round 5 · Mid · Medium · 3 questions

Queues

□ #950 Reveal Cards In Increasing Order

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Queue · Sorting · Simulation
Lists: AlgoMaster 600

Problem

You are given an integer array `deck`. There is a deck of cards where every card has a unique integer. The integer on the i th card is `deck[i]`. You can order the deck in any order you want. Initially, all the cards start face down (unrevealed) in one deck.

You will do the following steps repeatedly until all cards are revealed:

1. Take the top card of the deck, reveal it, and take it out of the deck.
2. If there are still cards in the deck then put the next top card of the deck at the bottom of the deck.
3. If there are still unrevealed cards, go back to step 1. Otherwise, stop.

Return an ordering of the deck that would reveal the cards in increasing order.

Note that the first entry in the answer is considered to be the top of the deck.

Example 1:

Input: deck = [17,13,11,2,3,5,7]

Output: [2,13,3,11,5,17,7]

Explanation:

We get the deck in the order [17,13,11,2,3,5,7] (this order does not matter), and reorder it.

After reordering, the deck starts as [2,13,3,11,5,17,7], where 2 is the top of the deck.

We reveal 2, and move 13 to the bottom. The deck is now [3,11,5,17,7,13].

We reveal 3, and move 11 to the bottom. The deck is now [5,17,7,13,11].

We reveal 5, and move 17 to the bottom. The deck is now [7,13,11,17].

Example 2:

Input: deck = [1,1000]

Output: [1,1000]

Constraints:

- $1 \leq \text{deck.length} \leq 1000$
- $1 \leq \text{deck}[i] \leq 10^6$
- All the values of deck are unique.

Community Solutions (Python)

- WalkCC


```
class Solution:
    def deckRevealedIncreasing(self, deck: list[int]) -> list[int]:
        dq = collections.deque()

        for card in reversed(sorted(deck)):
            dq.rotate()
            dq.appendleft(card)

        return list(dq)
```

• LeetCode

```
class Solution:
    def deckRevealedIncreasing(self, deck: List[int]) -> List[int]:
        q = deque()
        for v in sorted(deck, reverse=True):
            if q:
                q.appendleft(q.pop())
            q.appendleft(v)
        return list(q)
```

• SimplyLeet

Python solution using collections.deque for efficient O(1) pop and append operations on both ends.

```
from collections import deque

def deckRevealedIncreasing(deck):
    # Sort the deck in ascending order.
    sorted_deck = sorted(deck)
    # Initialize an empty deque to simulate the reverse process.
    dq = deque()
```

```
# Process each card from the largest to smallest
for card in reversed(sorted_deck):
    # If the deque is not empty, simulate moving the last element to the
    front.
    if dq:
        dq.appendleft(dq.pop())
    # Place the current card at the front of the deque.
    dq.appendleft(card)

# Convert deque back to list which is the required order.
return list(dq)
```

```
# Example usage
deck = [17, 13, 11, 2, 3, 5, 7]
print(deckRevealedIncreasing(deck)) # Expected output: [2, 13, 3, 11, 5, 17,
7]
```

◆ Quick Review

Key Insights

- The final revealed sequence must be the sorted order of the deck.
- Instead of simulating the process forward, we simulate it in reverse.
- Use a queue (or deque) to efficiently perform operations on both ends.
- Start from the largest card and reverse the process to reconstruct the initial order.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the deck.

Space Complexity: $O(n)$ for storing the recreated ordering and using a deque.

Solution

To solve the problem, sort the deck in ascending order. Then, simulate the revealing process in reverse: Initialize an empty deque. Iterate over the sorted deck in descending order. For each card, if the deque is non-empty, remove the last element and insert it at the front to simulate reversing the "move next card to bottom" step. Then, insert the current card at the front. The resulting deque represents the required deck order that, when processed by the given steps, reveals the cards in increasing order.

Queues

□ #1823 Find the Winner of the Circular Game

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Recursion · Queue · Simulation
Lists: AlgoMaster 600

Problem

There are n friends that are playing a game. The friends are sitting in a circle and are numbered from 1 to n in clockwise order. More formally, moving clockwise from the i th friend brings you to the $(i+1)$ th friend for $1 \leq i < n$, and moving clockwise from the n th friend brings you to the 1st friend.

The rules of the game are as follows:

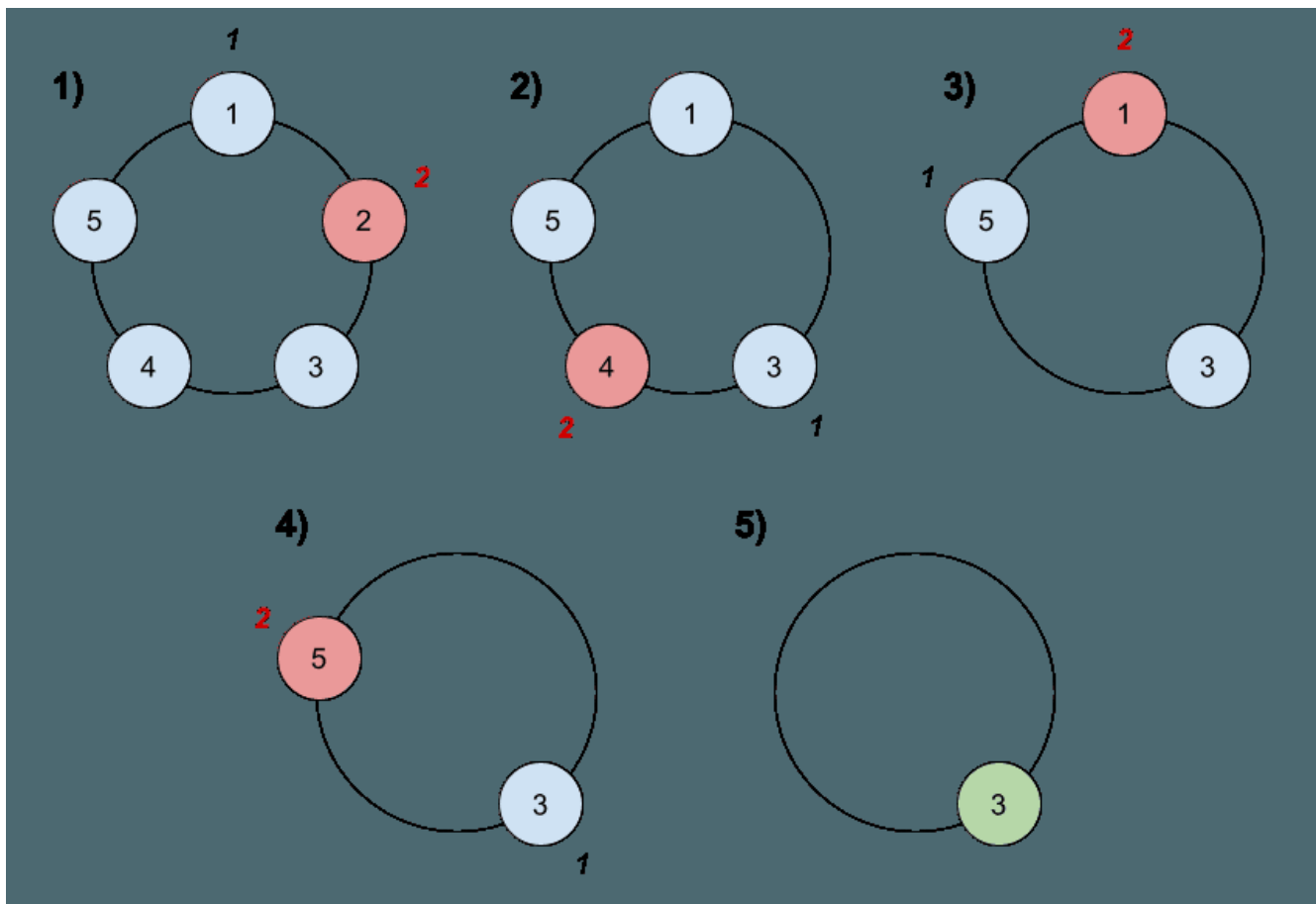
1. Start at the 1st friend.
2. Count the next k friends in the clockwise direction including the friend you started at. The counting wraps around the circle and may count some friends more than once.
3. The last friend you counted leaves the circle and loses the game.

4. If there is still more than one friend in the circle, go back to step 2 starting from the friend immediately clockwise of the friend who just lost and repeat.

5. Else, the last friend in the circle wins the game.

Given the number of friends, n , and an integer k , return the winner of the game.

Example 1:



Input: $n = 5, k = 2$

Output: 3

Explanation: Here are the steps of the game:

- 1) Start at friend 1.
- 2) Count 2 friends clockwise, which are friends 1 and 2.
- 3) Friend 2 leaves the circle. Next start is friend 3.
- 4) Count 2 friends clockwise, which are friends 3 and 4.
- 5) Friend 4 leaves the circle. Next start is friend 5.

Example 2:

Input: $n = 6, k = 5$

Output: 1

Explanation: The friends leave in this order: 5, 4, 6, 2, 3. The winner is friend 1.

Constraints:

- $1 \leq k \leq n \leq 500$

Follow up:

Could you solve this problem in linear time with constant space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findTheWinner(self, n: int, k: int) -> int:
        # True if i-th friend is left
        friends = [False] * n

        friendCount = n
        fp = 0 # friends' index

        while friendCount > 1:
            for _ in range(k):
```

```

        while friends[fp % n]: # The friend is not there.
            fp += 1 # Point to the next one.
            fp += 1
            friends[(fp - 1) % n] = True
            friendCount -= 1

```

```

fp = 0
while friends[fp]:
    fp += 1

```

```

return fp + 1

```

```

class Solution:
    def findTheWinner(self, n: int, k: int) -> int:
        # e.g. n = 4, k = 2.
        # By using 0-indexed notation, we have the following circle:
        #
        # 0 -> 1 -> 2 -> 3 -> 0
        # x
        # 0 -> 1 -> 2 -> 0
        #
        # After the first round, 1 is removed.

```

```

# So, 2 becomes 0, 3 becomes 1, and 0 becomes 2.
# Let's denote that oldIndex = f(n, k) and newIndex = f(n - 1, k).
# By observation, we know f(n, k) = (f(n - 1, k) + k) % n.
def f(n: int, k: int) -> int:
    if n == 1:
        return 0
    return (f(n - 1, k) + k) % n

# Converts back to 1-indexed.
return f(n, k) + 1

```

• LeetDoocs

```

class Solution:
    def findTheWinner(self, n: int, k: int) -> int:
        if n == 1:
            return 1
        ans = (k + self.findTheWinner(n - 1, k)) % n
        return n if ans == 0 else ans

```

• SimplyLeet

```
# Python implementation for finding the winner in the circular game

def findWinner(n, k):
    # Initialize winner to 0 for the base case of 1 friend (0-indexed)
    winner = 0
    # Process from 2 friends to n friends
    for i in range(2, n + 1):
        # Update winner index based on recurrence relation
        winner = (winner + k) % i
    # Convert the 0-indexed winner to 1-indexed result

    return winner + 1

# Example usage:
n = 5
k = 2
print(findWinner(n, k)) # Expected output: 3
```

◆ Quick Review

Key Insights

- The problem is a variation of the Josephus Problem.
- A recurrence relation exists: $f(1, k) = 0$ and $f(n, k) = (f(n - 1, k) + k) \bmod n$.
- The final answer must be adjusted to 1-indexed by adding 1 to the result.
- The mathematical recurrence yields an $O(n)$ time solution with $O(1)$ space.
- While simulation using a list or queue can solve the problem, it may be less efficient.

Space & Time

Time Complexity: $O(n)$ using the mathematical recurrence.

Space Complexity: $O(1)$ as it only requires a few variables.

Solution

The optimal solution leverages the mathematical recurrence relation of the Josephus problem. The recurrence relation is defined as:

- **Base case:** When there is only one friend, the winning index is 0.
- **Recurrence:** For a circle of size n , the new winning index can be computed by $(\text{previous winner} + k) \bmod n$. After computing the solution using zero-based indexing, add 1 to convert it to one-based indexing.

We maintain a running winner position and update it as we iterate from 2 to n . This approach requires constant space and runs in linear time.

Queues

□ #2327 Number of People Aware of a Secret

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Queue · Simulation
Lists: AlgoMaster 600

Problem

On day 1, one person discovers a secret.

You are given an integer `delay`, which means that each person will share the secret with a new person every day, starting from `delay` days after discovering the secret. You are also given an integer `forget`, which means that each person will forget the secret `forget` days after discovering it. A person cannot share the secret on the same day they forgot it, or on any day afterwards.

Given an integer `n`, return the number of people who know the secret at the end of day `n`. Since the answer may be very large, return it modulo $10^9 + 7$.

Example 1:

Input: $n = 6$, $\text{delay} = 2$, $\text{forget} = 4$

Output: 5

Explanation:

Day 1: Suppose the first person is named A. (1 person)

Day 2: A is the only person who knows the secret. (1 person)

Day 3: A shares the secret with a new person, B. (2 people)

Day 4: A shares the secret with a new person, C. (3 people)

Day 5: A forgets the secret, and B shares the secret with a new person, D. (3 people)

Example 2:

Input: $n = 4$, $\text{delay} = 1$, $\text{forget} = 3$

Output: 6

Explanation:

Day 1: The first person is named A. (1 person)

Day 2: A shares the secret with B. (2 people)

Day 3: A and B share the secret with 2 new people, C and D. (4 people)

Day 4: A forgets the secret. B, C, and D share the secret with 3 new people. (6 people)

Constraints:

- $2 \leq n \leq 1000$
- $1 \leq \text{delay} < \text{forget} \leq n$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def peopleAwareOfSecret(self, n: int, delay: int, forget: int) -> int:
        MOD = 1_000_000_007
        share = 0
        # dp[i] := the number of people know the secret at day i
        dp = [0] * n # Maps day i to i + 1.
        dp[0] = 1

        for i in range(1, n):
            if i - delay >= 0:
```

```

        share += dp[i - delay]
    if i - forget >= 0:
        share -= dp[i - forget]
    share += MOD
    share %= MOD
    dp[i] = share

# People before day `n - forget - 1` already forget the secret.
return sum(dp[-forget:]) % MOD

```

• LeetDoocs

```

class Solution:
    def peopleAwareOfSecret(self, n: int, delay: int, forget: int) -> int:
        m = (n <= 1) + 10
        d = [0] * m
        cnt = [0] * m
        cnt[1] = 1
        for i in range(1, n + 1):
            if cnt[i]:
                d[i] += cnt[i]
                d[i + forget] -= cnt[i]

            nxt = i + delay
            while nxt < i + forget:
                cnt[nxt] += cnt[i]
                nxt += 1
        mod = 10**9 + 7
        return sum(d[: n + 1]) % mod

```

• SimplyLeet

```

MOD = 10**9 + 7

def peopleAwareOfSecret(n, delay, forget):
    # Initialize dp and prefix arrays.
    dp = [0] * (n + 1)
    prefix = [0] * (n + 1)

    # On day 1, one person discovers the secret.
    dp[1] = 1
    prefix[1] = 1

```

```

# For each day from 2 to n, determine how many new people learn the secret.
for day in range(2, n + 1):
    # Calculate the start contribution: people eligible to share from day
    "day-delay".
    if day - delay >= 1:
        share_start = prefix[day - delay]
    else:
        share_start = 0

    # Subtract those who have forgotten the secret.

    if day - forget >= 1:
        forget_end = prefix[day - forget]
    else:
        forget_end = 0

    dp[day] = (share_start - forget_end) % MOD
    prefix[day] = (prefix[day - 1] + dp[day]) % MOD

# Sum up individuals who have not forgotten the secret by day n.
total = 0

# People that discovered the secret on day d will still remember if d > n -
forget.
for day in range(n - forget + 1, n + 1):
    if day >= 1:
        total = (total + dp[day]) % MOD
return total

# Example usage:
print(peopleAwareOfSecret(6, 2, 4))

```

◆ Quick Review

Key Insights

- Use dynamic programming to simulate the number of new people who learn the secret each day.

- Track when people become eligible to share (after delay days) and when they forget the secret (after forget days).
- Utilize prefix sums (cumulative sums) to efficiently compute the range sum of contributions for each day.
- The final answer is the sum of new persons who still remember the secret (i.e., those whose discovery day is $\geq n - \text{forget} + 1$).

Space & Time

Time Complexity: $O(n)$ – We iterate through days 1 to n once.

Space Complexity: $O(n)$ – We store dp and prefix arrays for days 1 through n .

Solution

We simulate day by day using two arrays:

- $\text{dp}[i]$: the number of people who learn the secret on day i .
- $\text{prefix}[i]$: the cumulative sum of dp from day 1 to day i .

For each day i (starting from day 2), the number of new people who learn the secret, $\text{dp}[i]$, is calculated as the sum of contributions

from those who learned the secret between day $i - \text{delay}$ and $i - 1$ (inclusive), minus those who have already forgotten (i.e., those from day $i - \text{forget}$ and earlier). The prefix array allows us to quickly compute these segment sums. Finally, the answer is the sum of dp values for days when individuals still remember the secret at day n (i.e., those from day $n - \text{forget} + 1$ to day n).

Monotonic Queue

Round 5 · Mid · Medium · 5 questions

Monotonic Queue

□ #1438 Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Queue · Sliding Window · Heap (Priority Queue) · Ordered Set · Monotonic Queue
Lists: AlgoMaster 600

Problem

Given an array of integers `nums` and an integer `limit`, return the size of the longest non-empty subarray such that the absolute difference between any two elements of this subarray is less than or equal to `limit`.

Example 1:

```
Input: nums = [8,2,4,7], limit = 4
Output: 2
Explanation: All subarrays are:
[8] with maximum absolute diff  $|8-8| = 0 \leq 4$ .
[8,2] with maximum absolute diff  $|8-2| = 6 > 4$ .
[8,2,4] with maximum absolute diff  $|8-2| = 6 > 4$ .
[8,2,4,7] with maximum absolute diff  $|8-2| = 6 > 4$ .
[2] with maximum absolute diff  $|2-2| = 0 \leq 4$ .
```

Example 2:

Input: `nums = [10,1,2,4,7,2]`, `limit = 5`

Output: 4

Explanation: The subarray `[2,4,7,2]` is the longest since the maximum absolute diff is $|2-7| = 5 \leq 5$.

Example 3:

Input: `nums = [4,2,2,2,4,4,2,2]`, `limit = 0`

Output: 3

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i] \leq 109$
- $0 \leq \text{limit} \leq 109$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def longestSubarray(self, nums: List[int], limit: int) -> int:
        sl = SortedList()
        ans = j = 0
        for i, x in enumerate(nums):
            sl.add(x)
            while sl[-1] - sl[0] > limit:
                sl.remove(nums[j])
                j += 1
            ans = max(ans, i - j + 1)

        return ans
```

```

class Solution:
    def longestSubarray(self, nums: List[int], limit: int) -> int:
        def check(k: int) -> bool:
            min_q = deque()
            max_q = deque()
            for i, x in enumerate(nums):
                if min_q and i - min_q[0] + 1 > k:
                    min_q.popleft()
                if max_q and i - max_q[0] + 1 > k:
                    max_q.popleft()

                while min_q and nums[min_q[-1]] >= x:
                    min_q.pop()
                while max_q and nums[max_q[-1]] <= x:
                    max_q.pop()
                min_q.append(i)
                max_q.append(i)
                if i >= k - 1 and nums[max_q[0]] - nums[min_q[0]] <= limit:
                    return True
            return False

        l, r = 1, len(nums)
        while l < r:
            mid = (l + r + 1) >> 1
            if check(mid):
                l = mid
            else:
                r = mid - 1
        return l

```

```

class Solution:
    def longestSubarray(self, nums: List[int], limit: int) -> int:
        maxq = deque()
        minq = deque()
        l, n = 0, len(nums)
        for r, x in enumerate(nums):
            while maxq and nums[maxq[-1]] < x:
                maxq.pop()
            while minq and nums[minq[-1]] > x:
                minq.pop()

```

```

maxq.append(r)
minq.append(r)
if nums[maxq[0]] - nums[minq[0]] > limit:
    l += 1
    if maxq[0] < l:
        maxq.popleft()
    if minq[0] < l:
        minq.popleft()
return n - l

```

• SimplyLeet

```

from collections import deque

def longestSubarray(nums, limit):
    maxDeque = deque() # stores indices for the maximum values
    minDeque = deque() # stores indices for the minimum values
    left = 0
    maxLength = 0

    # Iterate over each index with right pointer
    for right in range(len(nums)):

        # Maintain decreasing order in maxDeque
        while maxDeque and nums[right] > nums[maxDeque[-1]]:
            maxDeque.pop()
        maxDeque.append(right)

        # Maintain increasing order in minDeque
        while minDeque and nums[right] < nums[minDeque[-1]]:
            minDeque.pop()
        minDeque.append(right)

```

```
# If window is invalid, shrink from the left
while nums[maxDeque[0]] - nums[minDeque[0]] > limit:
    # Move the left pointer forward and remove indices if they are out
    # of the new window
    if maxDeque[0] == left:
        maxDeque.popleft()
    if minDeque[0] == left:
        minDeque.popleft()
    left += 1

# Update maxLength with the valid window size

maxLength = max(maxLength, right - left + 1)

return maxLength

# Example usage:
print(longestSubarray([8,2,4,7], 4))
```

◆ Quick Review

Key Insights

- Use a sliding window approach to efficiently find the longest valid subarray.
- Maintain the current window's minimum and maximum values using two monotonic deques.
- One deque (minDeque) will keep track of elements in increasing order to quickly access the minimum.
- The other deque (maxDeque) will track elements in decreasing order to quickly access the maximum.

- Slide the right pointer to expand the window and when the condition ($\text{max} - \text{min} > \text{limit}$) is violated, move the left pointer to shrink the window.
- The dequeues are updated by removing indices that are no longer in the window or are not useful for maintaining the monotonic order.

Space & Time

Time Complexity: $O(n)$ – Each element is processed at most twice (once when added and once when removed from the window).

Space Complexity: $O(n)$ – In the worst case, the dequeues may hold all the elements.

Solution

We use the sliding window technique with two dequeues to store indices for the current window's maximum and minimum values. As we iterate through `nums` with a right pointer, we update:

- The `maxDeque` by removing elements from the back that are less than the current element.
- The `minDeque` by removing elements from the back that are greater than the current

element. Then, if the current window (from left to right) does not satisfy the condition (i.e., the difference between the front values of `maxDeque` and `minDeque` is greater than `limit`), we increment the left pointer and update the deques by discarding indices that fall out of the window. The maximum window length encountered is the answer.

Monotonic Queue

□ #1673 Find the Most Competitive Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Greedy · Monotonic Stack
Lists: AlgoMaster 600

Problem

Given an integer array `nums` and a positive integer `k`, return the most competitive subsequence of `nums` of size `k`.

An array's subsequence is a resulting sequence obtained by erasing some (possibly zero) elements from the array.

We define that a subsequence `a` is more competitive than a subsequence `b` (of the same length) if in the first position where `a` and `b` differ, subsequence `a` has a number less than the corresponding number in `b`. For example, `[1,3,4]` is more competitive than `[1,3,5]` because the first position they differ is at the final number, and 4 is less than 5.

Example 1:

Input: `nums = [3,5,2,6]`, `k = 2`

Output: `[2,6]`

Explanation: Among the set of every possible subsequence: `{[3,5], [3,2], [3,6], [5,2], [5,6], [2,6]}`, `[2,6]` is the most competitive.

Example 2:

Input: `nums = [2,4,3,3,5,4,9,6]`, `k = 4`

Output: `[2,3,3,4]`

Constraints:

- `1 <= nums.length <= 105`
- `0 <= nums[i] <= 109`
- `1 <= k <= nums.length`

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mostCompetitive(self, nums: list[int], k: int) -> list[int]:
        ans = []

        for i, num in enumerate(nums):
            # If |ans| - 1 + |nums[i..n)| >= k, then it means we still have enough
            # numbers, and we can safely pop an element from ans.
            while ans and ans[-1] > nums[i] and len(ans) - 1 + len(nums) - i >= k:
                ans.pop()
            if len(ans) < k:
                ans.append(nums[i])

        return ans
```

• LeetDoocs

```
class Solution:
    def mostCompetitive(self, nums: List[int], k: int) -> List[int]:
        stk = []
        n = len(nums)
        for i, v in enumerate(nums):
            while stk and stk[-1] > v and len(stk) + n - i > k:
                stk.pop()
            if len(stk) < k:
                stk.append(v)
        return stk
```

• SimplyLeet

```
# Function to find the most competitive subsequence using a greedy monotonic
stack.
def mostCompetitive(nums, k):
    stack = [] # Stack to hold the resulting subsequence
    n = len(nums) # Total number of elements in the input array

    # Iterate over each number with its index.
    for i, num in enumerate(nums):
        # While the stack is not empty, and the last element in the stack is
        greater than the current element,
        # and we have enough remaining elements to fill the subsequence of size
        k:
            while stack and num < stack[-1] and (len(stack) - 1 + (n - i)) >= k:
                stack.pop() # Remove the last element from the stack
            # If the current subsequence is not yet of length k, add the current
            element.
            if len(stack) < k:
                stack.append(num)
    return stack

# Example usage:
print(mostCompetitive([3, 5, 2, 6], 2)) # Expected Output: [2, 6]
```

◆ Quick Review

Key Insights

- Use a greedy algorithm with a monotonic stack to continuously build the smallest lexicographical sequence.
- Ensure that after potential removals, there are still enough elements remaining to complete a subsequence of size k .
- At each step, compare the current number with the last element in the stack and decide whether to pop from the stack.

Space & Time

Time Complexity: $O(n)$, where n is the length of `nums`.

Space Complexity: $O(k)$, since the stack holds at most k elements.

Solution

The solution iterates over the array while maintaining a monotonic stack. For each element, if the current element is smaller than the last element in the stack and there are enough elements left to reach a total of k , the algorithm pops from the stack to remove suboptimal choices. This greedy process guarantees that the resulting subsequence is the most competitive (i.e., lexicographically

smallest possible). Once the iteration is finished, the stack contains the desired subsequence.

Monotonic Queue

□ #1696 Jump Game VI

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Queue · Heap
(Priority Queue) · Monotonic Queue
Lists: AlgoMaster 600

Problem

You are given a 0-indexed integer array `nums` and an integer `k`.

You are initially standing at index 0. In one move, you can jump at most `k` steps forward without going outside the boundaries of the array. That is, you can jump from index `i` to any index in the range `[i + 1, min(n - 1, i + k)]` inclusive.

You want to reach the last index of the array (index `n - 1`). Your score is the sum of all `nums[j]` for each index `j` you visited in the array. Return the maximum score you can get.

Example 1:

Input: nums = [1,-1,-2,4,-7,3], k = 2

Output: 7

Explanation: You can choose your jumps forming the subsequence [1,-1,4,3] (underlined above). The sum is 7.

Example 2:

Input: nums = [10,-5,-2,4,0,3], k = 3

Output: 17

Explanation: You can choose your jumps forming the subsequence [10,4,3] (underlined above). The sum is 17.

Example 3:

Input: nums = [1,-5,-20,4,-1,3,-6,-3], k = 2

Output: 0

Constraints:

- $1 \leq \text{nums.length}, k \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxResult(self, nums: list[int], k: int) -> int:
        # Stores dp[i] within the bounds.
        maxQ = collections.deque([0])
        # dp[i] := the maximum score to consider nums[0..i]
        dp = [0] * len(nums)
        dp[0] = nums[0]

        for i in range(1, len(nums)):
            # Pop the index if it's out-of-bounds.
```

```

    if maxQ[0] + k < i:
        maxQ.popleft()
    dp[i] = dp[maxQ[0]] + nums[i]
    # Pop indices that won't be chosen in the future.
    while maxQ and dp[maxQ[-1]] <= dp[i]:
        maxQ.pop()
    maxQ.append(i)

    return dp[-1]

```

• LeetDoocs

```

class Solution:
    def maxResult(self, nums: List[int], k: int) -> int:
        n = len(nums)
        f = [0] * n
        q = deque([0])
        for i in range(n):
            if i - q[0] > k:
                q.popleft()
            f[i] = nums[i] + f[q[0]]
            while q and f[q[-1]] <= f[i]:
                q.pop()
            q.append(i)
        return f[-1]

```

• SimplyLeet

```

# Python solution using deque for the sliding window maximum
from collections import deque

def maxResult(nums, k):
    n = len(nums)
    # Initialize dp array where dp[i] represents the maximum score to reach
    # index i
    dp = [0] * n
    dp[0] = nums[0]
    # Use deque to keep track of indices with maximum dp values in the current
    # window
    dq = deque([0])

```

```

# Process each index from 1 to n-1
for i in range(1, n):
    # Remove indices that are out of the window (i - k)
    while dq and dq[0] < i - k:
        dq.popleft()
    # dp[i] equals nums[i] plus the maximum score within the window
    dp[i] = nums[i] + dp[dq[0]]
    # Maintain deque in decreasing order of dp values
    while dq and dp[i] >= dp[dq[-1]]:
        dq.pop()
    dq.append(i)
return dp[-1]

# Example usage:
if __name__ == "__main__":
    print(maxResult([1, -1, -2, 4, -7, 3], 2)) # Output: 7

```

◆ Quick Review

Key Insights

- The problem can be modeled using dynamic programming where $dp[i]$ represents the maximum score to reach index i .
- The recurrence becomes $dp[i] = \text{nums}[i] + \max(dp[j])$ for all indices j such that $i - k \leq j < i$.
- A monotonic queue can be used to maintain the maximum dp value in the sliding window efficiently.

- Removing indices that fall outside the current window is crucial to ensure consistent window size.

Space & Time

Time Complexity: $O(n)$ because each element is processed at most twice.

Space Complexity: $O(n)$ for the dp array and the deque, though the deque holds at most k elements.

Solution

We use a dynamic programming approach where we define $dp[i]$ as the maximum score achievable when reaching index i . The transition is defined as $dp[i] = \text{nums}[i] + \max(dp[i-1], dp[i-2], \dots, dp[i-k])$.

To efficiently get the maximum in the sliding window of size k , we use a monotonic deque that stores indices in decreasing order of their dp values. For every index i :

- Remove indices from the front if they are out-of-window (i.e., index $\leq i - k$).
- The front of the deque gives the index of the maximum dp value in the current window.

- Calculate $dp[i]$ using $dp[\text{deque.front()}] + \text{nums}[i]$.
- Before adding i to the deque, remove all indices from the back whose dp values are less than $dp[i]$ to maintain a decreasing order. This method ensures that each dp value is computed in constant amortized time.

Monotonic Queue

□ #2762 Continuous Subarrays

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Queue · Sliding Window · Heap (Priority Queue) · Ordered Set · Monotonic Queue
Lists: AlgoMaster 600

Problem

You are given a 0-indexed integer array `nums`. A subarray of `nums` is called continuous if:

- Let $i, i + 1, \dots, j$ be the indices in the subarray. Then, for each pair of indices $i \leq i_1, i_2 \leq j$, $0 \leq |\text{nums}[i_1] - \text{nums}[i_2]| \leq 2$.

Return the total number of continuous subarrays.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [5,4,2,4]`

Output: 8

Explanation:

Continuous subarray of size 1: `[5]`, `[4]`, `[2]`, `[4]`.

Continuous subarray of size 2: `[5,4]`, `[4,2]`, `[2,4]`.

Continuous subarray of size 3: `[4,2,4]`.

There are no subarrays of size 4.

Total continuous subarrays = $4 + 3 + 1 = 8$.

Example 2:

Input: nums = [1,2,3]
 Output: 6
 Explanation:
 Continuous subarray of size 1: [1], [2], [3].
 Continuous subarray of size 2: [1,2], [2,3].
 Continuous subarray of size 3: [1,2,3].
 Total continuous subarrays = 3 + 2 + 1 = 6.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def continuousSubarrays(self, nums: list[int]) -> int:
        ans = 1 # [nums[0]]
        left = nums[0] - 2
        right = nums[0] + 2
        l = 0

        # nums[l..r] is a valid window.
        for r in range(1, len(nums)):
            if left <= nums[r] <= right:

                left = max(left, nums[r] - 2)
                right = min(right, nums[r] + 2)
            else:
                # nums[r] is out-of-bounds, so reconstruct the window.
                left = nums[r] - 2
                right = nums[r] + 2
                l = r

        # If we consistently move leftward in each iteration, it implies that
        # the entire left subarray satisfies the given condition. For every
        # subarray with l in the range [0, r], the condition is met, preventing
```

```

# the code from reaching the final "else" condition. Instead, it stops
# at the "if" condition.
while nums[r] - 2 <= nums[l] <= nums[r] + 2:
    left = max(left, nums[l] - 2)
    right = min(right, nums[l] + 2)
    l -= 1
    l += 1
# nums[l..r], num[l + 1..r], ..., nums[r]
ans += r - l + 1

return ans

```

• LeetCode

```

class Solution:
    def continuousSubarrays(self, nums: List[int]) -> int:
        ans = i = 0
        sl = SortedList()
        for x in nums:
            sl.add(x)
            while sl[-1] - sl[0] > 2:
                sl.remove(nums[i])
                i += 1
            ans += len(sl)

        return ans

```

• SimplyLeet

```

from collections import deque

def continuousSubarrays(nums):
    # Deques for storing indices for minimum and maximum values
    min_deque = deque()
    max_deque = deque()
    left = 0
    count = 0

    # Iterate over each element as the right end of the window

```

```
for right, num in enumerate(nums):
    # Maintain min_deque: remove elements greater than current num
    while min_deque and nums[min_deque[-1]] > num:
        min_deque.pop()
    min_deque.append(right)

    # Maintain max_deque: remove elements smaller than current num
    while max_deque and nums[max_deque[-1]] < num:
        max_deque.pop()
    max_deque.append(right)

    # If current window doesn't satisfy condition, slide left pointer
    while nums[max_deque[0]] - nums[min_deque[0]] > 2:
        # Move left pointer and remove if it is out of window
        if min_deque[0] == left:
            min_deque.popleft()
        if max_deque[0] == left:
            max_deque.popleft()
        left += 1

    # Count all subarrays ending at right index
    count += (right - left + 1)

return count

# Example usage:
nums = [5,4,2,4]
print(continuousSubarrays(nums)) # Expected output: 8
```

◆ Quick Review

Key Insights

- The condition on the subarray can be rephrased as ensuring that the difference between the maximum and minimum value in the subarray is at most 2.

- A sliding window technique is ideal because subarrays are contiguous.
- Use two monotonic queues (or dequeues) to efficiently track the minimum and maximum values in the current window.
- When the condition breaks (i.e., $\text{max} - \text{min} > 2$), adjust the window by moving the left pointer appropriately.
- The number of subarrays ending at the current right pointer is the window's length ($\text{right} - \text{left} + 1$).

Space & Time

Time Complexity: $O(n)$ because each element is processed at most twice (once when added and once when removed).

Space Complexity: $O(n)$ in the worst case for the dequeues, though typical use is much lower.

Solution

We use a sliding window approach where we maintain two dequeues: one that keeps track of the maximum values (in decreasing order) and one for the minimum values (in increasing order) within the current window. For each new element at the right pointer, we update

both deques. If the difference between the front elements of the maximum and minimum deques exceeds 2, we slide the left pointer until the condition is restored. The count of valid subarrays ending at the current right index is the size of the window, which is then added to the overall answer.

Monotonic Queue

□ #3578 Count Partitions With Max–Min Difference at Most K

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Queue ·
Sliding Window · Prefix Sum · Monotonic Queue
Lists: AlgoMaster 600

Problem

You are given an integer array `nums` and an integer `k`. Your task is to partition `nums` into one or more non-empty contiguous segments such that in each segment, the difference between its maximum and minimum elements is at most `k`.

Return the total number of ways to partition `nums` under this condition.

Since the answer may be too large, return it modulo $10^9 + 7$.

Example 1:

Input: `nums = [9,4,1,3,7]`, `k = 4`

Output: 6

Explanation:

There are 6 valid partitions where the difference between the maximum and minimum elements in each segment is at most $k = 4$:

- `[[9], [4], [1], [3], [7]]`
- `[[9], [4], [1], [3, 7]]`
- `[[9], [4], [1, 3], [7]]`
- `[[9], [4, 1], [3], [7]]`
- `[[9], [4, 1], [3, 7]]`
- `[[9], [4, 1, 3], [7]]`

Example 2:

Input: `nums = [3,3,4]`, $k = 0$

Output: 2

Explanation:

There are 2 valid partitions that satisfy the given conditions:

- `[[3], [3], [4]]`
- `[[3, 3], [4]]`

Constraints:

- $2 \leq \text{nums.length} \leq 5 * 10^4$
- $1 \leq \text{nums}[i] \leq 10^9$
- $0 \leq k \leq 10^9$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def countPartitions(self, nums: List[int], k: int) -> int:
        mod = 10**9 + 7
        sl = SortedList()
        n = len(nums)
        f = [1] + [0] * n
        g = [1] + [0] * n
        l = 1
        for r, x in enumerate(nums, 1):
            sl.add(x)

            while sl[-1] - sl[0] > k:
                sl.remove(nums[l - 1])
                l += 1
            f[r] = (g[r - 1] - (g[l - 2] if l >= 2 else 0) + mod) % mod
            g[r] = (g[r - 1] + f[r]) % mod
        return f[n]
```

Divide and Conquer

Round 5 · Mid · Medium · 3 questions

Divide and Conquer

□ #109 Convert Sorted List to Binary Search Tree

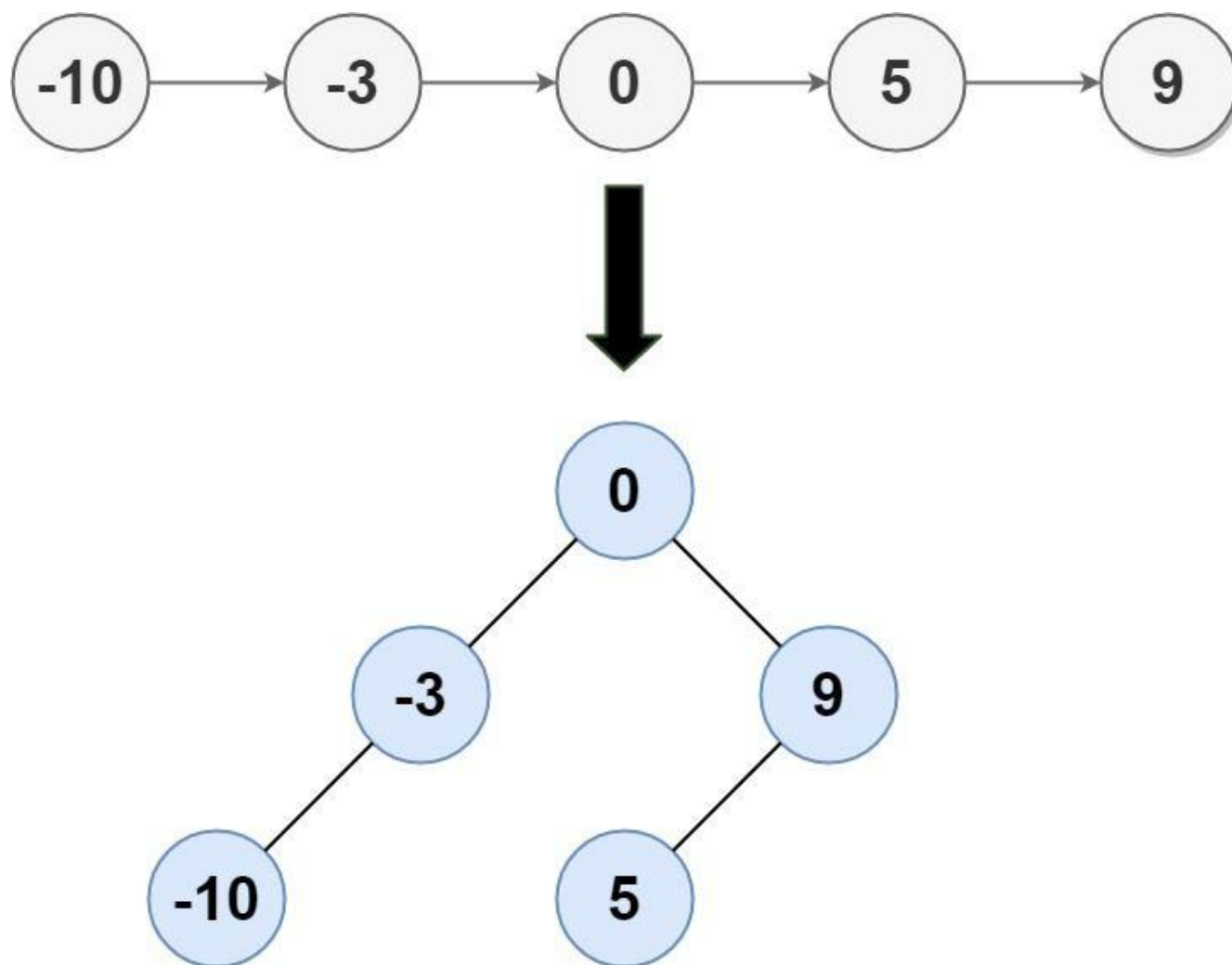
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Divide and Conquer · Tree ·
Binary Search Tree · Binary Tree
Lists: AlgoMaster 600

Problem

Given the head of a singly linked list where elements are sorted in ascending order, convert it to a height-balanced binary search tree.

Example 1:



Input: head = [-10,-3,0,5,9]

Output: [0,-3,9,-10,null,5]

Explanation: One possible answer is [0,-3,9,-10,null,5], which represents the shown height balanced BST.

Example 2:

Input: head = []

Output: []

Constraints:

- The number of nodes in head is in the range [0, 2 * 10⁴].
- -105 ≤ Node.val ≤ 105

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sortedListToBST(self, head: ListNode) -> TreeNode:
        def findMid(head: ListNode) -> ListNode:
            prev = None
            slow = head
            fast = head

            while fast and fast.next:
                prev = slow
                slow = slow.next
                fast = fast.next.next
            prev.next = None

            return slow

        if not head:
            return None
        if not head.next:
            return TreeNode(head.val)

        mid = findMid(head)
        root = TreeNode(mid.val)
        root.left = self.sortedListToBST(head)
        root.right = self.sortedListToBST(mid.next)
        return root

class Solution:
    def sortedListToBST(self, head: ListNode | None) -> TreeNode | None:
        arr = []

        # Construct the array.
        curr = head
        while curr:
            arr.append(curr.val)
            curr = curr.next
```

```
def helper(l: int, r: int) -> TreeNode | None:
    if l > r:
        return None
    m = (l + r) // 2
    root = TreeNode(arr[m])
    root.left = helper(l, m - 1)
    root.right = helper(m + 1, r)
    return root

return helper(0, len(arr) - 1)
```

```
class Solution:
    def sortedListToBST(self, head: ListNode | None) -> TreeNode | None:
        def helper(l: int, r: int) -> TreeNode | None:
            nonlocal head
            if l > r:
                return None

            m = (l + r) // 2

            # Simulate inorder traversal: recursively form the left half.
```

```
            left = helper(l, m - 1)

            # Once the left half is traversed, process the current node.
            root = TreeNode(head.val)
            root.left = left

            # Maintain the invariance.
            head = head.next

            # Simulate inorder traversal: recursively form the right half.
```

```
            root.right = helper(m + 1, r)
            return root

        return helper(0, self._getLength(head) - 1)

    def _getLength(self, head: ListNode | None) -> int:
        length = 0
        curr = head
        while curr:
            length += 1
```



```
curr = curr.next
return length
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def sortedListToBST(self, head: Optional[ListNode]) -> Optional[TreeNode]:
        def dfs(i: int, j: int) -> Optional[TreeNode]:
            if i > j:
                return None
            mid = (i + j) >> 1
            l, r = dfs(i, mid - 1), dfs(mid + 1, j)
            return TreeNode(nums[mid], l, r)

        nums = []
        while head:
            nums.append(head.val)
            head = head.next
        return dfs(0, len(nums) - 1)
```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val

        self.left = left
        self.right = right

class Solution:
    def sortedListToBST(self, head: ListNode) -> TreeNode:
        # Convert linked list to array for easier middle element access
        values = []
        while head:
            values.append(head.val)
            head = head.next

        # Helper function to build BST from sorted array
        def buildBST(left, right):
            if left > right:
                return None
            mid = (left + right) // 2 # Choose middle element as root for
balance
            node = TreeNode(values[mid])
            # Recursively build left subtree
            node.left = buildBST(left, mid - 1)
            # Recursively build right subtree

            node.right = buildBST(mid + 1, right)
            return node

        # Build BST using the helper function
        return buildBST(0, len(values) - 1)

```

◆ Quick Review

Key Insights

- The list is sorted in ascending order, which allows us to apply a divide and conquer approach.
- The middle element of the list should be the root of the BST to ensure the tree is balanced.
- Recursively repeating the process for the left and right segments of the list constructs the left and right subtrees.
- Two popular approaches are:
- Convert the list to an array and then use two pointers for recursion.
- Use slow and fast pointers to find the middle element without extra space.

Space & Time

Time Complexity: $O(n)$ if using the in-order simulation approach; $O(n \log n)$ if repeatedly finding the middle with two pointers.

Space Complexity: $O(\log n)$ for the recursion stack (assuming the conversion to array uses $O(n)$ extra space in that approach).

Solution

We solve the problem using a divide and conquer method by first converting the linked list into an array, which allows for easy access to the middle element. The middle element is chosen as the root of the BST, and the left half of the array is used to build the left subtree while the right half constructs the right subtree. This method guarantees that the height difference between the subtrees is minimized, resulting in a height-balanced BST.

Key data structures used:

- Array (to store linked list values)**
- Tree nodes to build the BST**

The algorithm recursively computes the middle index and creates tree nodes until the entire array is processed.

Divide and Conquer

□ #427 Construct Quad Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Divide and Conquer · Tree · Matrix
Lists: AlgoMaster 600

Problem

Given a $n * n$ matrix grid of 0's and 1's only. We want to represent grid with a Quad-Tree.

Return the root of the Quad-Tree representing grid.

A Quad-Tree is a tree data structure in which each internal node has exactly four children.

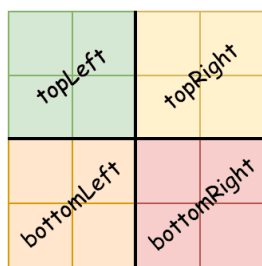
Besides, each node has two attributes:

- **val**: True if the node represents a grid of 1's or False if the node represents a grid of 0's. Notice that you can assign the val to True or False when isLeaf is False, and both are accepted in the answer.
- **isLeaf**: True if the node is a leaf node on the tree or False if the node has four children.

```
class Node {  
    public boolean val;  
    public boolean isLeaf;  
    public Node topLeft;  
    public Node topRight;  
    public Node bottomLeft;  
    public Node bottomRight;  
}
```

We can construct a Quad-Tree from a two-dimensional area using the following steps:

- 1. If the current grid has the same value (i.e all 1's or all 0's) set isLeaf True and set val to the value of the grid and set the four children to Null and stop.**
- 2. If the current grid has different values, set isLeaf to False and set val to any value and divide the current grid into four sub-grids as shown in the photo.**
- 3. Recurse for each of the children with the proper sub-grid.**



If you want to know more about the Quad-Tree, you can refer to the wiki.

Quad-Tree format:

You don't need to read this section for solving the problem. This is only if you want to understand the output format here. The output represents the serialized format of a Quad-Tree using level order traversal, where null signifies a path terminator where no node exists below. It is very similar to the serialization of the binary tree. The only difference is that the node is represented as a list [isLeaf, val].

If the value of isLeaf or val is True we represent it as 1 in the list [isLeaf, val] and if the value of isLeaf or val is False we represent it as 0.

Example 1:

0	1
1	0

Input: grid = [[0,1],[1,0]]

Output: [[0,1],[1,0],[1,1],[1,1],[1,0]]

Explanation: The explanation of this example is shown below:

Notice that 0 represents False and 1 represents True in the photo representing the Quad-Tree.

Example 2:

1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0

Input: grid = [[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,1,1,1,1],[1,1,1,1,1,1,1,1],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0]]

Output:

[[0,1],[1,1],[0,1],[1,1],[1,0],null,null,null,null,[1,0],[1,0],[1,1],[1,1]]

Explanation: All values in the grid are not the same. We divide the grid into four sub-grids.

The topLeft, bottomLeft and bottomRight each has the same value.

The topRight have different values so we divide it into 4 sub-grids where each has the same value.

Explanation is shown in the photo below:

Constraints:

- $n == \text{grid.length} == \text{grid}[i].\text{length}$
- $n == 2^x$ where $0 \leq x \leq 6$

Community Solutions (Python)

- WalkCC


```

class Solution:
    def construct(self, grid: list[list[int]]) -> 'Node':
        return self._helper(grid, 0, 0, len(grid))

    def _helper(self, grid: list[list[int]], i: int, j: int, w: int) -> 'Node':
        if self._allSame(grid, i, j, w):
            return Node(grid[i][j] == 1, True)
        half = w // 2
        return Node(True, False,
                    self._helper(grid, i, j, half),
                    self._helper(grid, i, j + half, half),
                    self._helper(grid, i + half, j, half),
                    self._helper(grid, i + half, j + half, half))

    def _allSame(self, grid: list[list[int]], i: int, j: int, w: int) -> bool:
        return all(grid[x][y] == grid[i][j]
                    for x in range(i, i + w)
                    for y in range(j, j + w))

```

• LeetDoocs

```

"""
# Definition for a QuadTree node.
class Node:
    def __init__(self, val, isLeaf, topLeft, topRight, bottomLeft,
bottomRight):
        self.val = val
        self.isLeaf = isLeaf
        self.topLeft = topLeft
        self.topRight = topRight
        self.bottomLeft = bottomLeft
        self.bottomRight = bottomRight

```

```

class Solution:
    def construct(self, grid: List[List[int]]) -> 'Node':
        def dfs(a, b, c, d):
            zero = one = 0
            for i in range(a, c + 1):
                for j in range(b, d + 1):
                    if grid[i][j] == 0:

                        zero = 1
                    else:
                        one = 1
            isLeaf = zero + one == 1
            val = isLeaf and one
            if isLeaf:
                return Node(grid[a][b], True)
            topLeft = dfs(a, b, (a + c) // 2, (b + d) // 2)
            topRight = dfs(a, (b + d) // 2 + 1, (a + c) // 2, d)
            bottomLeft = dfs((a + c) // 2 + 1, b, c, (b + d) // 2)

            bottomRight = dfs((a + c) // 2 + 1, (b + d) // 2 + 1, c, d)
            return Node(val, isLeaf, topLeft, topRight, bottomLeft, bottomRight)

        return dfs(0, 0, len(grid) - 1, len(grid[0]) - 1)

```

• SimplyLeet

```

# Definition for a Quad-Tree Node.
class Node:
    def __init__(self, val, isLeaf, topLeft=None, topRight=None, bottomLeft=None, bottomRight=None):
        self.val = val                # True for 1, False for 0.
        self.isLeaf = isLeaf          # Boolean indicating if this node is a
        leaf.
        self.topLeft = topLeft        # Child node for the top-left quadrant.
        self.topRight = topRight      # Child node for the top-right quadrant.
        self.bottomLeft = bottomLeft # Child node for the bottom-left
        quadrant.
        self.bottomRight = bottomRight # Child node for the bottom-right
        quadrant.

def construct(grid):
    # Helper function to determine if the current grid section is uniform.
    def is_uniform(r0, c0, length):
        value = grid[r0][c0]
        for i in range(r0, r0 + length):
            for j in range(c0, c0 + length):
                if grid[i][j] != value:
                    return False, value
            return True, value

    # Recursive function to build the Quad Tree.
    def helper(r0, c0, length):
        uniform, value = is_uniform(r0, c0, length)
        if uniform:
            # All cells are the same: create a leaf node.
            return Node(val=(value == 1), isLeaf=True)
        else:
            half = length // 2
            # Recursively construct each quadrant.
            topLeft = helper(r0, c0, half)

```

```

    topRight = helper(r0, c0 + half, half)
    bottomLeft = helper(r0 + half, c0, half)
    bottomRight = helper(r0 + half, c0 + half, half)
    # Create an internal node with four children.
    return Node(val=True, isLeaf=False, topLeft=topLeft, topRight=topR
ight,
                bottomLeft=bottomLeft, bottomRight=bottomRight)

n = len(grid)
return helper(0, 0, n)

```

◆ Quick Review

Key Insights

- Use a divide and conquer approach: recursively check if the current grid region is uniform.
- A region is uniform if all entries are the same; if so, create a leaf node.
- If not uniform, split the region into four equal quadrants and recursively construct the Quad-Tree for each.
- Careful handling of grid boundaries is required during recursion.

Space & Time

Time Complexity: $O(n^2)$ in the worst-case scenario since every cell might be inspected.

Space Complexity: $O(n^2)$ in the worst-case due to recursion stack and node storage.

Solution

We use recursion to examine each sub-grid of the matrix. First, we check if the current grid region has a uniform value. If it does, we return a leaf node with that value. If not, we partition the grid into four smaller quadrants (topLeft, topRight, bottomLeft, bottomRight) and recursively construct the corresponding Quad-Tree nodes for each quadrant. Finally, we create an internal node combining these four children. This strategy guarantees we process all parts of the grid and correctly form the Quad-Tree.

Divide and Conquer

□ #654 Maximum Binary Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Divide and Conquer · Stack · Tree ·
Monotonic Stack · Binary Tree
Lists: AlgoMaster 600

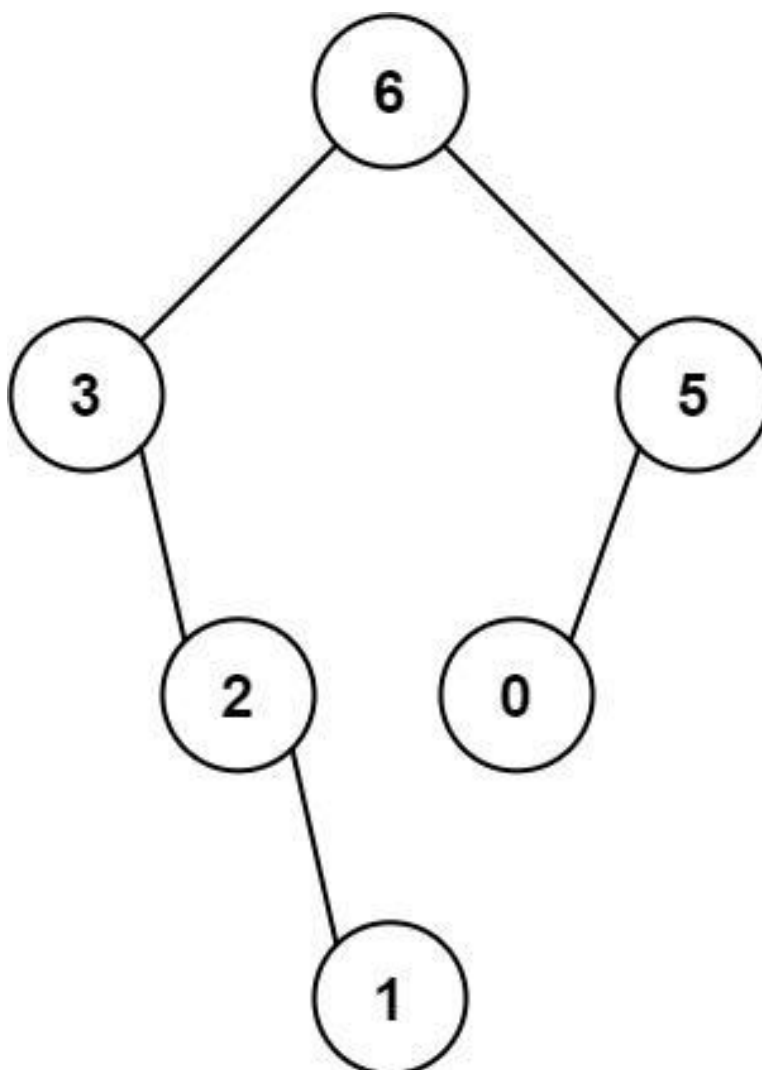
Problem

You are given an integer array `nums` with no duplicates. A maximum binary tree can be built recursively from `nums` using the following algorithm:

1. Create a root node whose value is the maximum value in `nums`.
2. Recursively build the left subtree on the subarray prefix to the left of the maximum value.
3. Recursively build the right subtree on the subarray suffix to the right of the maximum value.

Return the maximum binary tree built from `nums`.

Example 1:



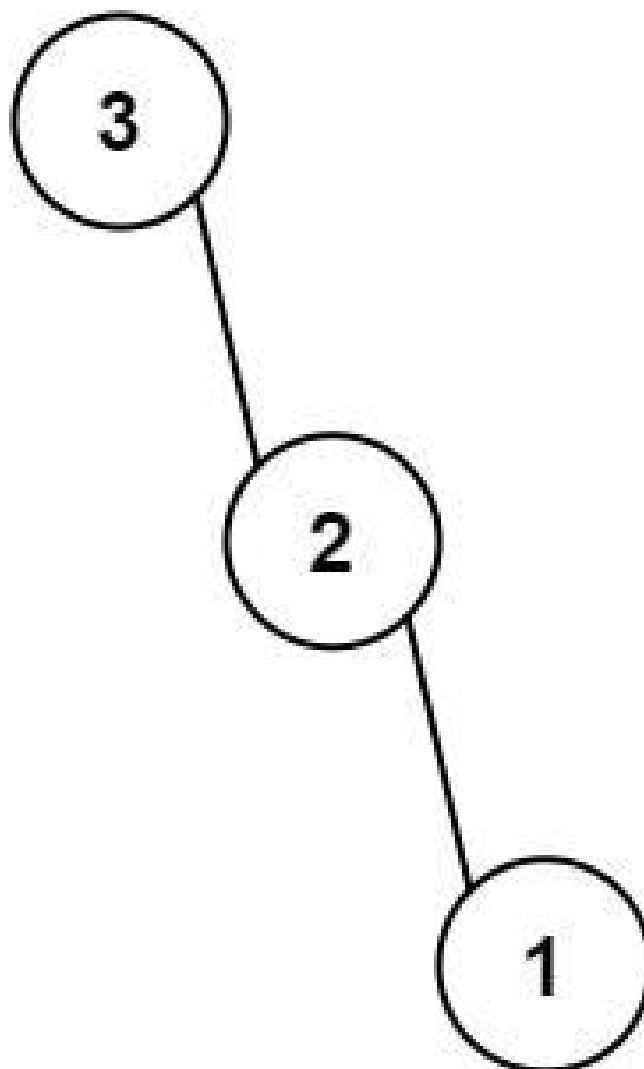
Input: nums = [3,2,1,6,0,5]

Output: [6,3,5,null,2,0,null,null,1]

Explanation: The recursive calls are as follow:

- The largest value in [3,2,1,6,0,5] is 6. Left prefix is [3,2,1] and right suffix is [0,5].
- The largest value in [3,2,1] is 3. Left prefix is [] and right suffix is [2,1].
- Empty array, so no child.
- The largest value in [2,1] is 2. Left prefix is [] and right suffix is [1].
- Empty array, so no child.

Example 2:



Input: nums = [3,2,1]
Output: [3,null,2,null,1]

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] \leq 1000$
- All integers in nums are unique.

Community Solutions (Python)

- WalkCC


```

class Solution:
    def constructMaximumBinaryTree(self, nums: list[int]) -> TreeNode | None:
        def build(i: int, j: int) -> TreeNode | None:
            if i > j:
                return None

            maxNum = max(nums[i:j + 1])
            maxIndex = nums.index(maxNum)

            root = TreeNode(maxNum)

            root.left = build(i, maxIndex - 1)
            root.right = build(maxIndex + 1, j)
            return root

        return build(0, len(nums) - 1)

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def constructMaximumBinaryTree(self, nums: List[int]) -> Optional[TreeNode]:
        def dfs(nums):
            if not nums:
                return None
            val = max(nums)
            i = nums.index(val)
            root = TreeNode(val)
            root.left = dfs(nums[:i])
            root.right = dfs(nums[i + 1 :])
            return root

        return dfs(nums)

```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # Node value
        self.left = left        # Left child
        self.right = right       # Right child

class Solution:
    def constructMaximumBinaryTree(self, nums: list[int]) -> TreeNode:
        # Base case: if nums is empty return None

        if not nums:
            return None
        # Find index of the maximum element in the list
        max_index = nums.index(max(nums))
        # Create a new TreeNode with the maximum element
        node = TreeNode(nums[max_index])
        # Recursively build the left subtree using elements before max_index
        node.left = self.constructMaximumBinaryTree(nums[:max_index])
        # Recursively build the right subtree using elements after max_index
        node.right = self.constructMaximumBinaryTree(nums[max_index + 1:])

        # Return the created tree node
        return node
```

◆ Quick Review

Key Insights

- The maximum element in any subarray becomes the root of the (sub)tree.
- The problem can be naturally solved using a recursive, divide and conquer approach.
- Alternatively, a monotonic stack can be used to achieve an $O(n)$ solution.

- Handling base cases, such as an empty array, is crucial to prevent errors.

Space & Time

Time Complexity: $O(n^2)$ in the worst-case (when the array is sorted in ascending or descending order) using the recursive approach; $O(n)$ using a monotonic stack approach.

Space Complexity: $O(n)$ due to recursion stack or explicit stack usage.

Solution

We solve the problem recursively:

- Identify the maximum element in the current array segment.
- Create a tree node with this maximum value.
- Recursively build the left subtree with the subarray left of the maximum element.
- Recursively build the right subtree with the subarray right of the maximum element. This approach leverages the divide and conquer technique to build the tree in a structured manner.

Merge Sort

Round 5 · Mid · Medium · 1 question

Merge Sort

□ #912 Sort an Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Divide and Conquer · Sorting · Heap
(Priority Queue) · Merge Sort · Bucket Sort ·
Radix Sort · Counting Sort

Lists: AlgoMaster 600

Problem

Given an array of integers `nums`, sort the array in ascending order and return it.

You must solve the problem without using any built-in functions in $O(n\log(n))$ time complexity and with the smallest space complexity possible.

Example 1:

Input: `nums = [5,2,3,1]`

Output: `[1,2,3,5]`

Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

Example 2:

Input: `nums = [5,1,1,2,0,0]`

Output: `[0,0,1,1,2,5]`

Explanation: Note that the values of `nums` are not necessarily unique.

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^4$
- $-5 * 10^4 \leq \text{nums}[i] \leq 5 * 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sortArray(self, nums: list[int]) -> list[int]:
        self._mergeSort(nums, 0, len(nums) - 1)
        return nums

    def _mergeSort(self, nums: list[int], l: int, r: int) -> None:
        if l >= r:
            return

        def merge(nums: list[int], l: int, m: int, r: int) -> None:

            sorted = [0] * (r - l + 1)
            k = 0 # sorted's index
            i = l # left's index
            j = m + 1 # right's index

            while i <= m and j <= r:
                if nums[i] < nums[j]:
                    sorted[k] = nums[i]
                    k += 1
                    i += 1
                else:
                    sorted[k] = nums[j]
                    k += 1
                    j += 1

            # Put the possible remaining left part into the sorted array.
            while i <= m:
                sorted[k] = nums[i]
                k += 1
                i += 1
```

```
# Put the possible remaining right part into the sorted array.
while j <= r:
    sorted[k] = nums[j]
    k += 1
    j += 1

nums[l:l + len(sorted)] = sorted

m = (l + r) // 2
```

```
self._mergeSort(nums, l, m)
self._mergeSort(nums, m + 1, r)
merge(nums, l, m, r)
```

```
class Solution:
    def sortArray(self, nums: list[int]) -> list[int]:
        self._heapSort(nums)
        return nums

    def _heapSort(self, nums: list[int]) -> None:
        def maxHeapify(nums: list[int], i: int, heapSize: int) -> None:
            l = 2 * i + 1
            r = 2 * i + 2
            largest = i

            if l < heapSize and nums[largest] < nums[l]:
                largest = l
            if r < heapSize and nums[largest] < nums[r]:
                largest = r
            if largest != i:
                nums[largest], nums[i] = nums[i], nums[largest]
                maxHeapify(nums, largest, heapSize)

        def buildMaxHeap(nums: list[int]) -> None:
            for i in range(len(nums) // 2, -1, -1):
```

```

        maxHeapify(nums, i, len(nums))

    buildMaxHeap(nums)
    heapSize = len(nums)
    for i in reversed(range(1, len(nums))):
        nums[i], nums[0] = nums[0], nums[i]
        heapSize -= 1
        maxHeapify(nums, 0, heapSize)

class Solution:
    def sortArray(self, nums: list[int]) -> list[int]:
        self._quickSort(nums, 0, len(nums) - 1)
        return nums

    def _quickSort(self, nums: list[int], l: int, r: int) -> None:
        if l >= r:
            return

        def partition(nums: list[int], l: int, r: int) -> int:
            randIndex = random.randint(0, r - l) + l
            nums[randIndex], nums[r] = nums[r], nums[randIndex]
            pivot = nums[r]
            nextSwapped = l
            for i in range(l, r):
                if nums[i] <= pivot:
                    nums[nextSwapped], nums[i] = nums[i], nums[nextSwapped]
                    nextSwapped += 1
            nums[nextSwapped], nums[r] = nums[r], nums[nextSwapped]
            return nextSwapped

        m = partition(nums, l, r)
        self._quickSort(nums, l, m - 1)
        self._quickSort(nums, m + 1, r)

```

• LeetDoocs


```
class Solution:
    def sortArray(self, nums: List[int]) -> List[int]:
        def quick_sort(l, r):
            if l >= r:
                return
            x = nums[randint(l, r)]
            i, j, k = l - 1, r + 1, l
            while k < j:
                if nums[k] < x:
                    nums[i + 1], nums[k] = nums[k], nums[i + 1]

                    i, k = i + 1, k + 1
                elif nums[k] > x:
                    j -= 1
                    nums[j], nums[k] = nums[k], nums[j]
                else:
                    k = k + 1
            quick_sort(l, i)
            quick_sort(j, r)

        quick_sort(0, len(nums) - 1)

    return nums
```

```
class Solution:
    def sortArray(self, nums: List[int]) -> List[int]:
        def merge_sort(l, r):
            if l >= r:
                return
            mid = (l + r) >> 1
            merge_sort(l, mid)
            merge_sort(mid + 1, r)
            i, j = l, mid + 1
            tmp = []
```

```

        while i <= mid and j <= r:
            if nums[i] <= nums[j]:
                tmp.append(nums[i])
                i += 1
            else:
                tmp.append(nums[j])
                j += 1
        if i <= mid:
            tmp.extend(nums[i : mid + 1])
        if j <= r:
            tmp.extend(nums[j : r + 1])
        for i in range(l, r + 1):
            nums[i] = tmp[i - l]

merge_sort(0, len(nums) - 1)
return nums

```

• SimplyLeet

```

def heapify(nums, heap_size, root_index):
    # Initialize largest as the root index
    largest = root_index
    left_child = 2 * root_index + 1 # left child index
    right_child = 2 * root_index + 2 # right child index

    # If left child exists and is larger than the current largest, update
    largest
    if left_child < heap_size and nums[left_child] > nums[largest]:
        largest = left_child

```

```

    # If right child exists and is larger than the current largest, update
    largest
    if right_child < heap_size and nums[right_child] > nums[largest]:
        largest = right_child

```

```

    # If largest is not the root, swap the values and heapify the affected
    subtree
    if largest != root_index:
        nums[root_index], nums[largest] = nums[largest], nums[root_index]
        heapify(nums, heap_size, largest)

```

```

def sortArray(nums):

```

```

    n = len(nums)
    # Build a max heap by heapifying each non-leaf node (from n//2 - 1 down to
    0)

```

```

    for i in range(n // 2 - 1, -1, -1):
        heapify(nums, n, i)

```

```

    # Extract elements one by one from the heap

```

```

    for i in range(n - 1, 0, -1):
        # Move current root to the end (swap)
        nums[0], nums[i] = nums[i], nums[0]
        # Heapify the reduced heap to maintain max heap invariant

```

```

        heapify(nums, i, 0)

```

```

    return nums

```

```

# Example usage:
# print(sortArray([5,2,3,1]))

```

◆ Quick Review

Key Insights

- Must not use built-in functions.
- Achieve $O(n \log(n))$ worst-case time complexity.

- Aim for minimum additional space; using an in-place algorithm is ideal.
- Heap sort is a good candidate because it offers $O(n \log(n))$ time with $O(1)$ additional space.

Space & Time

Time Complexity: $O(n \log(n))$

Space Complexity: $O(1)$ (in-place)

Solution

The chosen approach is Heap Sort. Heap sort works by first building a max heap from the input array. A max heap is a complete binary tree where the value in each node is greater than or equal to the values in its children.

After the max heap is constructed, the largest element is at the root of the heap. Then, we repeatedly swap the root of the heap with the last element of the heap and reduce the heap size by one. We then re-heapify the root to restore the max heap property. This process continues until the heap is empty and the array is sorted in ascending order.

Key steps in the algorithm:

- Build the max heap in-place by calling a helper function (heapify) for each non-leaf node.
 - Iteratively swap the first element (largest in the heap) with the last element and reduce the heap size.
 - Re-heapify the array from the root index to ensure the max heap property is maintained.
- This approach avoids the extra space required for algorithms like merge sort and guarantees $O(n \log(n))$ worst-case time.

QuickSort / QuickSelect

Round 5 · Mid · Medium · 1 question

QuickSort / QuickSelect

□ #1985 Find the Kth Largest Integer in the Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Divide and Conquer · Sorting ·
Heap (Priority Queue) · Quickselect
Lists: AlgoMaster 600

Problem

You are given an array of strings `nums` and an integer `k`. Each string in `nums` represents an integer without leading zeros.

Return the string that represents the `k`th largest integer in `nums`.

Note: Duplicate numbers should be counted distinctly. For example, if `nums` is `["1","2","2"]`, `"2"` is the first largest integer, `"2"` is the second-largest integer, and `"1"` is the third-largest integer.

Example 1:

Input: `nums = ["3","6","7","10"], k = 4`

Output: `"3"`

Explanation:

The numbers in `nums` sorted in non-decreasing order are `["3","6","7","10"]`.
The 4th largest integer in `nums` is `"3"`.

Example 2:

Input: nums = ["2","21","12","1"], k = 3

Output: "2"

Explanation:

The numbers in nums sorted in non-decreasing order are ["1","2","12","21"].

The 3rd largest integer in nums is "2".

Example 3:

Input: nums = ["0","0"], k = 2

Output: "0"

Explanation:

The numbers in nums sorted in non-decreasing order are ["0","0"].

The 2nd largest integer in nums is "0".

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 104$
- $1 \leq \text{nums}[i].\text{length} \leq 100$
- `nums[i]` consists of only digits.
- `nums[i]` will not have any leading zeros.

Community Solutions (Python)

• WalkCC

```
class Solution:
    # Similar to 215. Kth Largest Element in an Array
    def kthLargestNumber(self, nums: list[str], k: int) -> str:
        minHeap = []

        for num in nums:
            heapq.heappush(minHeap, int(num))
            if len(minHeap) > k:
                heapq.heappop(minHeap)

        return str(minHeap[0])
```


• LeetCode

```
class Solution:
    def kthLargestNumber(self, nums: List[str], k: int) -> str:
        return nlargest(k, nums, key=lambda x: int(x))[k - 1]
```

• SimplyLeet

```
# Define a function to find the kth largest integer in the array
def kthLargestNumber(nums, k):
    # Sort the list 'nums' in descending order by using a custom key:
    # 1. Compare by the length of the string (more digits means larger number)
    # 2. If lengths are equal, compare lexicographically.
    nums.sort(key=lambda x: (len(x), x), reverse=True)
    # Return the kth largest element (0-indexed array so k-1)
    return nums[k - 1]
```

```
# Example usage:
```

```
if __name__ == "__main__":
    nums = ["3", "6", "7", "10"]
    k = 4
    result = kthLargestNumber(nums, k)
    print(result) # Expected output: "3"
```

◆ Quick Review

Key Insights

- Since each number is represented as a string and can be very long (up to 100 digits), converting them to numeric types might be impractical in some languages; instead, compare using string properties.
- The key observation is that one integer is greater than another if its string length

(number of digits) is larger; if lengths are equal, lexicographical order gives the order.

- Sorting the array in descending order using a custom comparator (first comparing lengths, then lexicographically) allows direct access to the k th largest element.
- Alternatively, selection algorithms (e.g., Quickselect) or heaps can be used, but a sort is simple and effective given the input constraints.

Space & Time

Time Complexity: $O(n \log n * m)$ where m is the maximum string length (for comparing two numbers)

Space Complexity: $O(n)$ (space used by sorting)

Solution

The solution involves sorting the array of strings in descending order based on their numeric value. We define a custom comparator that:

- Compares the lengths of the strings. A longer string indicates a larger integer.
- If the lengths are equal, compares them lexicographically.

After sorting, the k th largest integer is simply found at the $k-1$ index in the sorted array. This approach avoids pitfalls with very large numbers since it relies only on string properties.

Backtracking

Round 5 · Mid · Medium · 4 questions

Backtracking

□ #47 Permutations II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking · Sorting
Lists: AlgoMaster 600

Problem

Given a collection of numbers, `nums`, that might contain duplicates, return all possible unique permutations in any order.

Example 1:

```
Input: nums = [1,1,2]
Output:
[[1,1,2],
 [1,2,1],
 [2,1,1]]
```

Example 2:

```
Input: nums = [1,2,3]
Output: [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]
```

Constraints:

- $1 \leq \text{nums.length} \leq 8$
- $-10 \leq \text{nums}[i] \leq 10$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def permuteUnique(self, nums: list[int]) -> list[list[int]]:
        ans = []
        used = [False] * len(nums)

        def dfs(path: list[int]) -> None:
            if len(path) == len(nums):
                ans.append(path.copy())
                return

        for i, num in enumerate(nums):
            if used[i]:
                continue
            if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                continue
            used[i] = True
            path.append(num)
            dfs(path)
            path.pop()
            used[i] = False

        nums.sort()
        dfs([])
        return ans
```

• LeetCode

```
class Solution:
    def permuteUnique(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == n:
                ans.append(t[:])
                return
            for j in range(n):
                if vis[j] or (j and nums[j] == nums[j - 1] and not vis[j - 1]):
                    continue
                t[i] = nums[j]
```

```

        vis[j] = True
        dfs(i + 1)
        vis[j] = False

```

```

n = len(nums)
nums.sort()
ans = []
t = [0] * n
vis = [False] * n
dfs(0)

```

```

return ans

```

• SimplyLeet

```

# Function to return all unique permutations of the input list nums.

```

```

def permuteUnique(nums):

```

```

    # Sort the numbers to facilitate duplicate checking.

```

```

    nums.sort()

```

```

    results = []

```

```

    permutation = []

```

```

    used = [False] * len(nums)

```

```

    # The helper function for backtracking.

```

```

    def backtrack():

```

```

        # If the current permutation is complete, add a copy to the results.

```

```

        if len(permutation) == len(nums):

```

```

            results.append(permutation.copy())

```

```

            return

```

```

        # Iterate over each number in nums.

```

```

        for i in range(len(nums)):

```

```

            # Skip used elements.

```

```

            if used[i]:

```

```

                continue

```

```
        # Skip duplicates: if the current element is the same as the
previous and
        # the previous element was not used, then skip the current element.
        if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
            continue
        # Mark the element as used.
        used[i] = True
        permutation.append(nums[i])
        # Recurse.
        backtrack()
        # Backtrack: unmark the element and remove it from the current
permutation.

        used[i] = False
        permutation.pop()

    backtrack()
    return results

# Example usage:
# print(permuteUnique([1, 1, 2]))
```

◆ Quick Review

Key Insights

- Sort the input array to identify and easily skip duplicates.
- Use backtracking to generate all candidate permutations.
- Maintain a visited or used array to track elements already added to the current permutation.
- When encountering a duplicate element, only use it if the previous identical element was

used in the current recursion branch.

- Backtrack by removing elements and resetting their used status when moving to another branch.

Space & Time

Time Complexity: $O(n * n!)$ in the worst-case scenario, where n is the length of the input array.

Space Complexity: $O(n)$ for the recursion stack and used data structure, plus $O(n * n!)$ to store the permutations.

Solution

The solution employs a backtracking algorithm. First, sort the `nums` array to ensure that duplicates are adjacent. Then, use a recursive function to build permutations by iterating through the array and choosing elements that have not been used yet. To handle duplicates, if the current element is the same as the previous one and the previous one was not used in the current recursive path, skip the current element. This ensures that duplicate permutations are never constructed. Data structures used include:

- A list (or array) to hold the current permutation path.
- A boolean visited (or used) array to keep track of whether a number is already in the current permutation.

After constructing a permutation of the same length as `nums`, add it to the results list, then backtrack to explore alternative possibilities.

Backtracking

□ #77 Combinations

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Backtracking

Lists: AlgoMaster 600

Problem

Given two integers n and k , return all possible combinations of k numbers chosen from the range $[1, n]$.

You may return the answer in any order.

Example 1:

Input: $n = 4, k = 2$

Output: $[[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]$

Explanation: There are $4 \text{ choose } 2 = 6$ total combinations.

Note that combinations are unordered, i.e., $[1,2]$ and $[2,1]$ are considered to be the same combination.

Example 2:

Input: $n = 1, k = 1$

Output: $[[1]]$

Explanation: There is $1 \text{ choose } 1 = 1$ total combination.

Constraints:

- $1 \leq n \leq 20$
- $1 \leq k \leq n$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def combine(self, n: int, k: int) -> list[list[int]]:
        ans = []

        def dfs(s: int, path: list[int]) -> None:
            if len(path) == k:
                ans.append(path.copy())
                return

            for i in range(s, n + 1):

                path.append(i)
                dfs(i + 1, path)
                path.pop()

        dfs(1, [])
        return ans
```

• LeetDoocs

```
class Solution:
    def combine(self, n: int, k: int) -> List[List[int]]:
        def dfs(i: int):
            if len(t) == k:
                ans.append(t[:])
                return
            if i > n:
                return
            t.append(i)
            dfs(i + 1)

            t.pop()
            dfs(i + 1)

        ans = []
        t = []
        dfs(1)
        return ans
```

• SimplyLeet

```
# Define the function to get combinations
def combine(n, k):
    # This list will store all the valid combinations
    result = []

    # Helper function for backtracking
    def backtrack(start, current):
        # If current combination is of the required length, add a copy to
        result
        if len(current) == k:
            result.append(current.copy())

        return

    # Iterate from the current start to n
    # Prune the loop: Stop if remaining numbers are less than needed
    for num in range(start, n + 1):
        # Add the current number to the combination
        current.append(num)
        # Move on to the next number; increment start to ensure increasing
        order

        backtrack(num + 1, current)
        # Remove the last added number to backtrack

    current.pop()

    # Initialize backtracking with starting number 1 and an empty combination
    backtrack(1, [])
    return result

# Example usage
print(combine(4, 2))
```

◆ Quick Review

Key Insights

- The problem requires generating all unique combinations (order does not matter) from 1

to n .

- Backtracking (depth-first search) is ideal for exploring all possible combinations.
- We build combinations incrementally and backtrack when the current combination reaches a dead end or a complete combination of k numbers is formed.
- Pruning the recursion early by limiting the candidate range helps improve efficiency.

Space & Time

Time Complexity: $O(k * C(n, k))$

Space Complexity: $O(k)$ for the recursion stack (not including space for the output)

Solution

We use a backtracking approach with depth-first search to generate all possible combinations. Start with an empty combination and recursively add numbers from a candidate range ensuring each new element is greater than the last added, which maintains the increasing order and avoids duplicates. When the length of the current combination equals k , add it to the result list. After exploring each path, backtrack by

removing the last element and try the next number. The candidate range can be pruned since if the remaining numbers are insufficient to reach k, no further recursive calls are made.

Backtracking

□ #93 Restore IP Addresses

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Backtracking
Lists: AlgoMaster 600

Problem

A valid IP address consists of exactly four integers separated by single dots. Each integer is between 0 and 255 (inclusive) and cannot have leading zeros.

- For example, "0.1.2.201" and "192.168.1.1" are valid IP addresses, but "0.011.255.245", "192.168.1.312" and "192.168@1.1" are invalid IP addresses.

Given a string *s* containing only digits, return all possible valid IP addresses that can be formed by inserting dots into *s*. You are not allowed to reorder or remove any digits in *s*. You may return the valid IP addresses in any order.

Example 1:

```
Input: s = "25525511135"  
Output: ["255.255.11.135", "255.255.111.35"]
```


Example 2:

Input: s = "0000"
Output: ["0.0.0.0"]

Example 3:

Input: s = "101023"
Output: ["1.0.10.23", "1.0.102.3", "10.1.0.23", "10.10.2.3", "101.0.2.3"]

Constraints:

- $1 \leq s.length \leq 20$
- s consists of digits only.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def restoreIpAddresses(self, s: str) -> list[str]:
        ans = []

        def dfs(start: int, path: list[int]) -> None:
            if len(path) == 4 and start == len(s):
                ans.append(path[0] + '.' + path[1] + '.' + path[2] + '.' + path[3])
                return
            if len(path) == 4 or start == len(s):
                return

        for length in range(1, 4):
            if start + length > len(s):
                return # out-of-bounds
            if length > 1 and s[start] == '0':
                return # leading '0'
            num = s[start: start + length]
            if int(num) > 255:
                return
            dfs(start + length, path + [num])
```

```
dfs(0, [])
return ans
```

• LeetCode

```
class Solution:
    def restoreIpAddresses(self, s: str) -> List[str]:
        def check(i: int, j: int) -> int:
            if s[i] == "0" and i != j:
                return False
            return 0 <= int(s[i : j + 1]) <= 255

        def dfs(i: int):
            if i >= n and len(t) == 4:
                ans.append(".".join(t))

            return
            if i >= n or len(t) >= 4:
                return
            for j in range(i, min(i + 3, n)):
                if check(i, j):
                    t.append(s[i : j + 1])
                    dfs(j + 1)
                    t.pop()

        n = len(s)

        ans = []
        t = []
        dfs(0)
        return ans
```

• SimplyLeet

```
# Python solution using backtracking

class Solution:
    def restoreIpAddresses(self, s: str) -> list:
        result = [] # List to store valid IP addresses

        def backtrack(start: int, path: list):
            # If 4 parts are collected and we've used all characters, record
            the result
            if len(path) == 4 and start == len(s):
                result.append(".".join(path))

            return

            # If either 4 parts are collected or not enough parts remain, prune
            the search
            if len(path) == 4 or start == len(s):
                return

            # Try each segment length from 1 to 3
            for length in range(1, 4):
                # Avoid index overflow
                if start + length > len(s):
                    break

                segment = s[start:start+length]
                # Check for leading zeros (segment "0" is valid)
                if len(segment) > 1 and segment[0] == '0':
                    continue
                # Convert segment to integer and check valid range [0, 255]
                if int(segment) > 255:
                    continue
                # Append segment to path and backtrack for next segment
                backtrack(start + length, path + [segment])

        backtrack(0, [])
        return result

# Example usage:
# sol = Solution()
# print(sol.restoreIpAddresses("25525511135"))
```

◆ Quick Review

Key Insights

- Use a backtracking approach to explore different segmentations of the input string.
- IP addresses contain exactly four segments; once four segments are generated, the entire string must have been used.
- Validate each segment to ensure it is within the range $[0, 255]$ and does not have leading zeros (unless the segment is "0").
- Prune the search if the remaining characters do not allow forming a valid IP address (too short or too long).

Space & Time

Time Complexity: $O(N^4)$ in the worst-case scenario where N is the length of the string, though N is bounded by string length constraints.

Space Complexity: $O(N)$ for the recursion call stack and temporary storage, where N is the length of the string.

Solution

We use a backtracking recursive approach to partition the string into four segments. At each recursive call, we try all possible segment lengths (1 to 3 characters) while ensuring that:

- The chosen segment does not exceed the string's bounds.**

- The segment does not have any invalid leading zeros (unless the segment is "0").**

- The numeric value of the segment lies between 0 and 255.**

Once four segments are collected, we check if all characters in the string have been consumed. If so, we join the segments with dots and add the resulting IP address to the final answer list.

Backtracking

□ #216 Combination Sum III

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking
Lists: AlgoMaster 600

Problem

Find all valid combinations of k numbers that sum up to n such that the following conditions are true:

- Only numbers 1 through 9 are used.
- Each number is used at most once.

Return a list of all possible valid combinations. The list must not contain the same combination twice, and the combinations may be returned in any order.

Example 1:

```
Input: k = 3, n = 7
Output: [[1,2,4]]
Explanation:
1 + 2 + 4 = 7
There are no other valid combinations.
```

Example 2:

Input: $k = 3, n = 9$
 Output: $[[1,2,6],[1,3,5],[2,3,4]]$
 Explanation:
 $1 + 2 + 6 = 9$
 $1 + 3 + 5 = 9$
 $2 + 3 + 4 = 9$
 There are no other valid combinations.

Example 3:

Input: $k = 4, n = 1$
 Output: $[]$
 Explanation: There are no valid combinations.
 Using 4 different numbers in the range $[1,9]$, the smallest sum we can get is $1+2+3+4 = 10$ and since $10 > 1$, there are no valid combination.

Constraints:

- $2 \leq k \leq 9$
- $1 \leq n \leq 60$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def combinationSum3(self, k: int, n: int) -> list[list[int]]:
        ans = []

        def dfs(k: int, n: int, s: int, path: list[int]) -> None:
            if k == 0 and n == 0:
                ans.append(path)
                return
            if k == 0 or n < 0:
                return

            for i in range(s, 10):
                dfs(k - 1, n - i, i + 1, path + [i])

        dfs(k, n, 1, [])
        return ans
```

• LeetCode

```
class Solution:
    def combinationSum3(self, k: int, n: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                if len(t) == k:
                    ans.append(t[:])
                return
            if i > 9 or i > s or len(t) >= k:
                return
            t.append(i)

            dfs(i + 1, s - i)
            t.pop()
            dfs(i + 1, s)

        ans = []
        t = []
        dfs(1, n)
        return ans

class Solution:
    def combinationSum3(self, k: int, n: int) -> List[List[int]]:
        ans = []
        for mask in range(1 << 9):
            if mask.bit_count() == k:
                t = [i + 1 for i in range(9) if mask >> i & 1]
                if sum(t) == n:
                    ans.append(t)
        return ans
```

• SimplyLeet


```
def combinationSum3(k, n):
    result = []

    def backtrack(start, path, current_sum):
        # If we've chosen k numbers, check the sum.
        if len(path) == k:
            if current_sum == n:
                result.append(path.copy())
            return

        # Iterate through numbers from 'start' to 9.
        for num in range(start, 10):
            if current_sum + num > n: # Early pruning.
                break
            path.append(num)
            backtrack(num + 1, path, current_sum + num)
            path.pop() # Backtrack.

    backtrack(1, [], 0)
    return result

# Example usage:
print(combinationSum3(3, 7)) # Output: [[1,2,4]]
```

◆ Quick Review

Key Insights

- Use a backtracking approach to explore all potential combinations.
- Only numbers from 1 to 9 are available.
- Each number can be used only once.
- The solution must consist of exactly k numbers summing to n.

- Early pruning is possible if the current sum plus the next candidate exceeds n .

Space & Time

Time Complexity: $O(2^9)$ in the worst-case scenario, since we are exploring subsets of 9 numbers.

Space Complexity: $O(k)$ for the recursion stack, where k is the number of elements in the combination.

Solution

We solve this problem using the backtracking algorithm. The solution involves a recursive helper function that tries to build combinations from numbers 1 through 9. At each recursive call, if the current path contains k numbers and their sum equals n , the path is added to the result. We use early pruning by checking if the running sum exceeds n , and we start with numbers greater than the last included to avoid duplicate combinations.

BST / Ordered Set

Round 5 · Mid · Medium · 10 questions

BST / Ordered Set

□ #99 Recover Binary Search Tree

MED

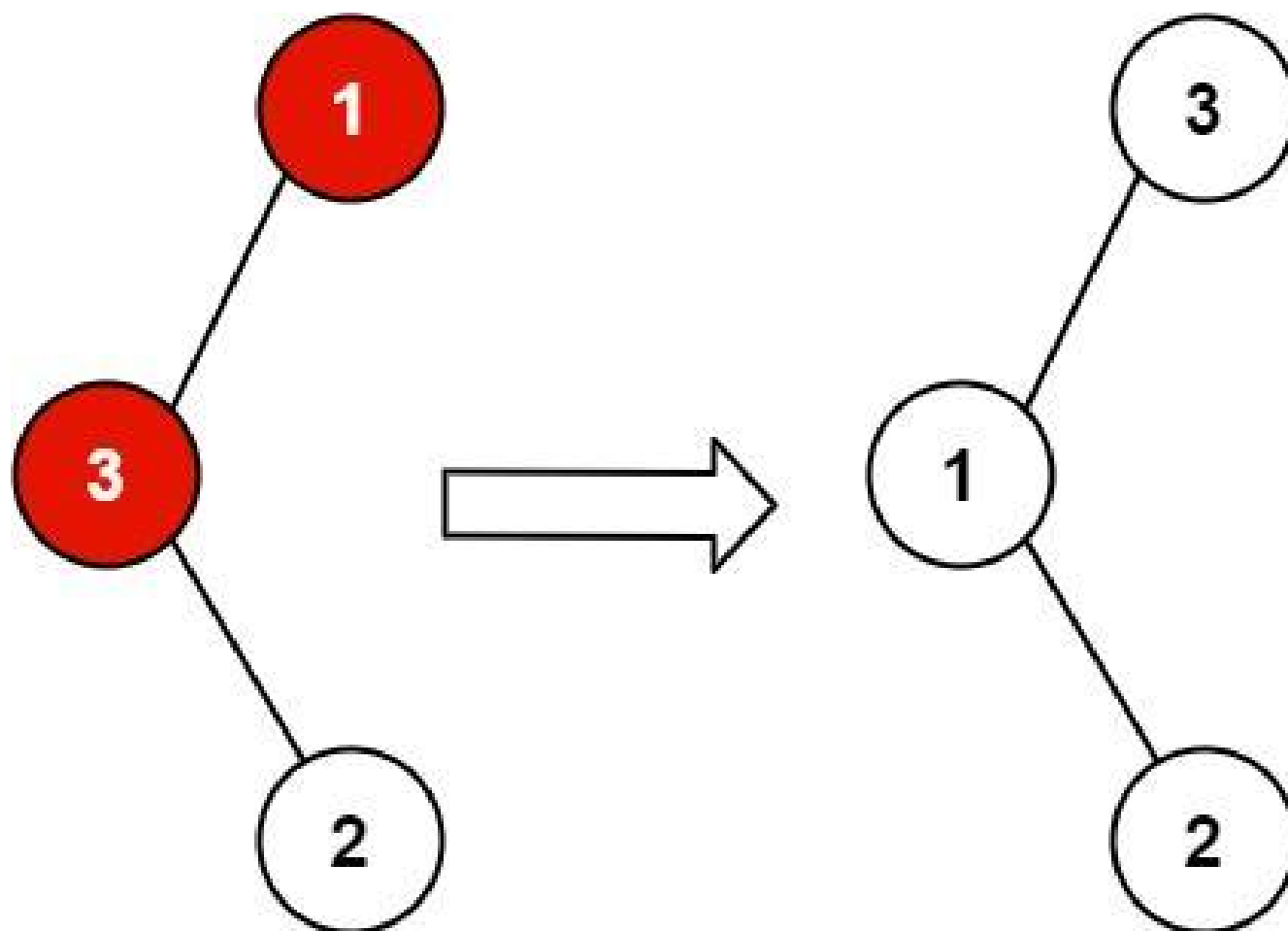
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Binary Search Tree ·
Binary Tree

Lists: AlgoMaster 600

Problem

You are given the root of a binary search tree (BST), where the values of exactly two nodes of the tree were swapped by mistake. Recover the tree without changing its structure.

Example 1:

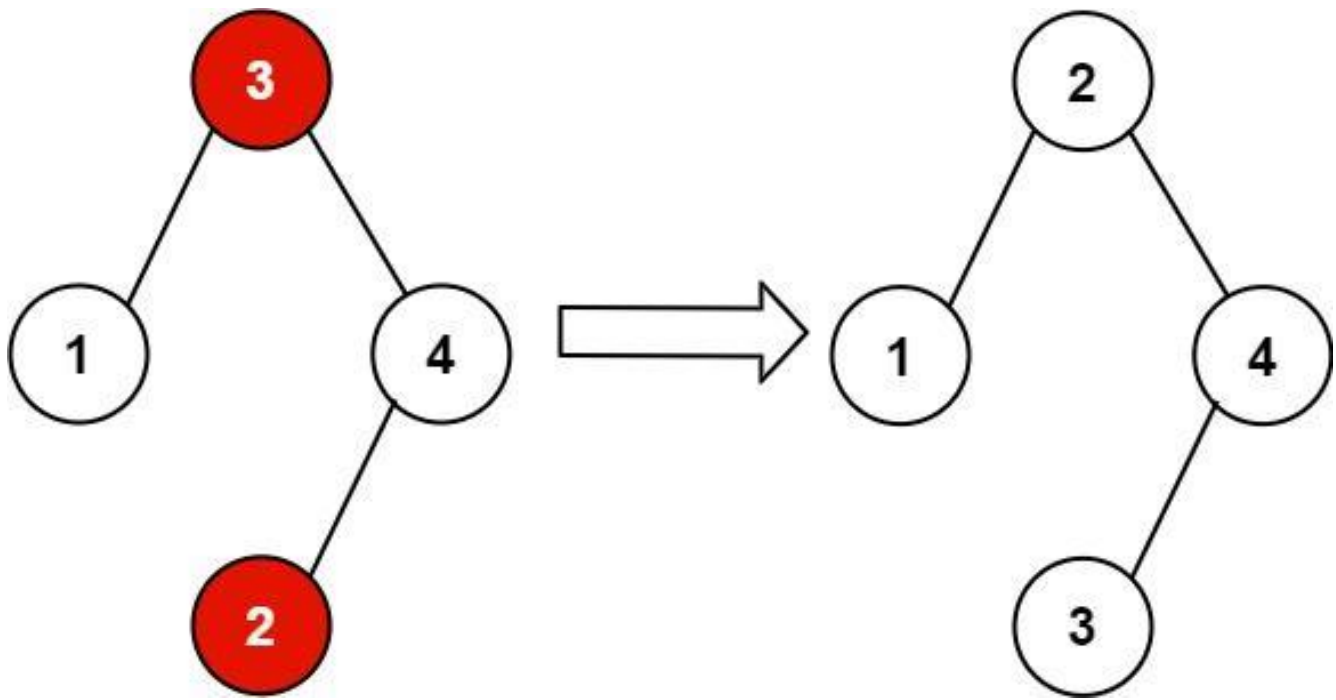


Input: root = [1,3,null,null,2]

Output: [3,1,null,null,2]

Explanation: 3 cannot be a left child of 1 because $3 > 1$. Swapping 1 and 3 makes the BST valid.

Example 2:



Input: root = [3,1,4,null,null,2]

Output: [2,1,4,null,null,3]

Explanation: 2 cannot be in the right subtree of 3 because $2 < 3$. Swapping 2 and 3 makes the BST valid.

Constraints:

- The number of nodes in the tree is in the range [2, 1000].
- $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

Follow up: A solution using $O(n)$ space is pretty straight-forward. Could you devise a constant $O(1)$ space solution?

Community Solutions (Python)

- WalkCC

```

class Solution:
    def recoverTree(self, root: TreeNode | None) -> None:
        def swap(x: TreeNode | None, y: TreeNode | None) -> None:
            temp = x.val
            x.val = y.val
            y.val = temp

        def inorder(root: TreeNode | None) -> None:
            if not root:
                return

```

```

            inorder(root.left)

```

```

            if self.pred and root.val < self.pred.val:
                self.y = root
                if not self.x:
                    self.x = self.pred
                else:
                    return
            self.pred = root

```

```

            inorder(root.right)

```

```

        inorder(root)
        swap(self.x, self.y)

```

```

    pred = None
    x = None # the first wrong node
    y = None # the second wrong node

```

```

class Solution:
    def recoverTree(self, root: TreeNode | None) -> None:
        pred = None
        x = None # the first wrong node
        y = None # the second wrong node
        stack = []

        while root or stack:
            while root:
                stack.append(root)

```

```

        root = root.left
    root = stack.pop()
    if pred and root.val < pred.val:
        y = root
        if not x:
            x = pred
        pred = root
        root = root.right

def swap(x: TreeNode | None, y: TreeNode | None) -> None:

```

```

    temp = x.val
    x.val = y.val
    y.val = temp

swap(x, y)

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def recoverTree(self, root: Optional[TreeNode]) -> None:
        """
        Do not return anything, modify root in-place instead.

        """

    def dfs(self, root):
        if root is None:
            return
        nonlocal prev, first, second
        dfs(root.left)
        if prev and prev.val > root.val:
            if first is None:
                first = prev

```



```

        second = root
        prev = root
        dfs(root.right)

    prev = first = second = None
    dfs(root)
    first.val, second.val = second.val, first.val

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def recoverTree(root):
    # Initialize variables to track the first and second swapped nodes and the
    # previous node in in-order traversal
    first = second = prev = None

    # Helper function to perform in-order traversal
    def inorder(node):
        nonlocal first, second, prev
        if not node:
            return
        # Traverse the left subtree
        inorder(node.left)
        # Check for swapped nodes: if previous node's value is greater than the
        # current node's value, violation occurs
        if prev and prev.val > node.val:

```

```
        if not first:
            # Record the first element when the first violation is
            encountered
            first = prev
            # Update the second element in every violation
            second = node
            # Update the previous node to the current node
            prev = node
            # Traverse the right subtree
            inorder(node.right)

    inorder(root)
    # Swap the values of the two nodes to recover the BST
    first.val, second.val = second.val, first.val
```

◆ Quick Review

Key Insights

- In a valid BST, an in-order traversal produces a sorted sequence.
- A swapped node pair causes a deviation from the sorted order. There can be one or two such violations.
- By tracking the previous node during an in-order traversal, we can identify nodes where the value order is incorrect.
- After finding the two nodes, swap their values to fix the tree.
- While the straightforward recursive in-order traversal uses $O(n)$ space (due to the recursion

stack), the Morris traversal technique can achieve $O(1)$ space.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ in the recursive approach (or $O(1)$ using Morris Traversal)

Solution

The solution involves performing an in-order traversal of the BST to detect the two nodes that have been swapped. During the traversal, we maintain a pointer to the previously visited node. When we encounter a node that is less than its previous node, we record the two nodes that violate the BST invariant. In cases where the swapped nodes are not adjacent, we will see two such violations; otherwise, we see only one. Finally, we swap the values of the two incorrect nodes, thereby restoring the BST.

BST / Ordered Set

□ #426 Convert Binary Search Tree to Sorted Doubly Linked List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Stack · Tree · Depth-First Search
· Binary Search Tree · Binary Tree ·
Doubly-Linked List

Lists: AlgoMaster 600

Problem

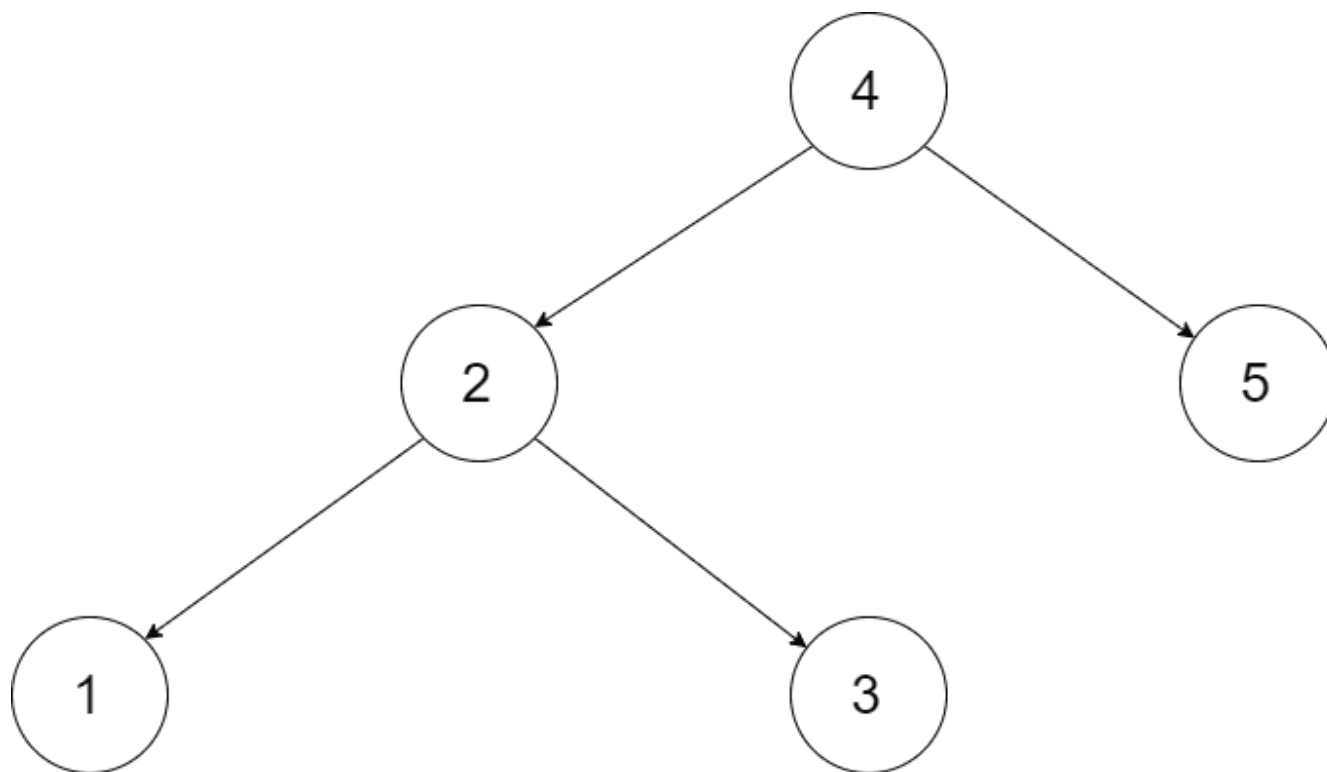
Convert a Binary Search Tree to a sorted Circular Doubly-Linked List in place.

You can think of the left and right pointers as synonymous to the predecessor and successor pointers in a doubly-linked list. For a circular doubly linked list, the predecessor of the first element is the last element, and the successor of the last element is the first element.

We want to do the transformation in place. After the transformation, the left pointer of the tree node should point to its predecessor, and the right pointer should point to its successor. You should return the pointer to the smallest

element of the linked list.

Example 1:



Input: root = [4,2,5,1,3]

Output: [1,2,3,4,5]

Explanation: The figure below shows the transformed BST. The solid line indicates the successor relationship, while the dashed line means the predecessor relationship.

Example 2:

Input: root = [2,1,3]

Output: [1,2,3]

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- $-1000 \leq \text{Node.val} \leq 1000$

- All the values of the tree are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def treeToDoublyList(self, root: 'Node | None') -> 'Node | None':
        if not root:
            return None
        leftHead = self.treeToDoublyList(root.left)
        rightHead = self.treeToDoublyList(root.right)
        root.left = leftHead
        root.right = rightHead
        return self._connect(leftHead, root, rightHead)

    def _connect(self, node1: 'Node | None', node2: 'Node | None') -> 'Node | None':
        if not node1:
            return node2
        if not node2:
            return node1

        tail1 = node1.left
        tail2 = node2.left

        # Connect node1's tail with node2.

        tail1.right = node2
        node2.left = tail1

        # Connect node2's tail with node1.
        tail2.right = node1
        node1.left = tail2
        return node1
```

• LeetDoocs

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
"""

class Solution:
    def treeToDoublyList(self, root: 'Optional[Node]') -> 'Optional[Node]':
        def dfs(root):
            if root is None:
                return
            nonlocal prev, head
            dfs(root.left)
            if prev:
                prev.right = root
                root.left = prev
            else:
                head = root
            prev = root
            dfs(root.right)

        if root is None:
            return None
        head = prev = None
        dfs(root)
        prev.right = head

        head.left = prev
        return head

```

• SimplyLeet

```
# Definition for a Node.
# class Node:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left # Pointer to predecessor in DLL
# self.right = right # Pointer to successor in DLL

def treeToDoublyList(root):
    # If the BST is empty, return None
    if not root:

        return None

    # Initialize the pointers for the previous node and head node of the doubly
    linked list
    prev = None
    head = None

    # Helper function to perform in-order traversal
    def inorder(node):
        nonlocal prev, head
        if not node:

            return

        # Traverse left subtree
        inorder(node.left)

        # Process current node: if prev is None, it means this is the smallest
        element
        if prev is None:
            head = node
        else:
            # Connect the previous node (predecessor) with the current node
            (successor)
```



```
        prev.right = node
        node.left = prev

    # Update prev to current node for the next iteration
    prev = node

    # Traverse right subtree
    inorder(node.right)

# Start in-order traversal starting from the root

inorder(root)

# After traversal, prev points to the last node. Connect head and tail to
# make DLL circular.
head.left = prev
prev.right = head

# Return the head node of the circular doubly linked list.
return head
```

◆ Quick Review

Key Insights

- Use an in-order traversal to process nodes in sorted order.
- Maintain pointers to connect the current node with the previously visited node.
- After traversal, connect the head and tail of the list to make it circular.
- The in-place constraint means no new nodes (or extra data structures for node storage) are created.

Space & Time

Time Complexity: $O(n)$ – Each node is visited once during the in-order traversal.

Space Complexity: $O(n)$ – Due to the recursion stack in the case of an unbalanced tree. For a balanced tree, it can be $O(\log n)$.

Solution

We solve the problem using an in-order depth-first search (DFS) traversal to ensure nodes are accessed in sorted order. While traversing:

- Use a helper recursive function to perform the in-order traversal.
 - Maintain a reference to the previously processed node (prev) and to the head of the list.
 - For each node, connect it with the previous node by setting prev.right to the current node and current node.left to prev.
 - Once traversal is complete, connect the head and tail (last node) to make the list circular.
- The algorithm uses recursion, and the key trick is to update the pointers while unwinding the recursion.

BST / Ordered Set

□ #450 Delete Node in a BST

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Binary Search Tree · Binary Tree
Lists: AlgoMaster 600

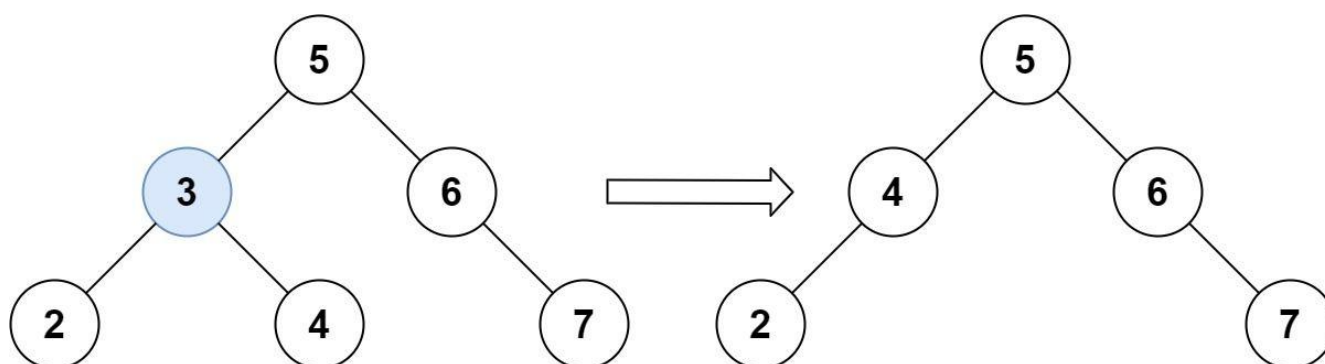
Problem

Given a root node reference of a BST and a key, delete the node with the given key in the BST. Return the root node reference (possibly updated) of the BST.

Basically, the deletion can be divided into two stages:

1. Search for a node to remove.
2. If the node is found, delete the node.

Example 1:



Input: root = [5,3,6,2,4,null,7], key = 3

Output: [5,4,6,2,null,null,7]

Explanation: Given key to delete is 3. So we find the node with value 3 and delete it.

One valid answer is [5,4,6,2,null,null,7], shown in the above BST.

Please notice that another valid answer is [5,2,6,null,4,null,7] and it's also accepted.

Example 2:

Input: root = [5,3,6,2,4,null,7], key = 0

Output: [5,3,6,2,4,null,7]

Explanation: The tree does not contain a node with value = 0.

Example 3:

Input: root = [], key = 0

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-105 \leq \text{Node.val} \leq 105$
- Each node has a unique value.
- root is a valid binary search tree.
- $-105 \leq \text{key} \leq 105$

Follow up: Could you solve it with time complexity $O(\text{height of tree})$?

Community Solutions (Python)

- WalkCC

```

class Solution:
    def deleteNode(self, root: TreeNode | None, key: int) -> TreeNode | None:
        if not root:
            return None
        if root.val == key:
            if not root.left:
                return root.right
            if not root.right:
                return root.left
            minNode = self._getMin(root.right)

            root.right = self.deleteNode(root.right, minNode.val)
            minNode.left = root.left
            minNode.right = root.right
            root = minNode
        elif root.val < key:
            root.right = self.deleteNode(root.right, key)
        else: # root.val > key
            root.left = self.deleteNode(root.left, key)
        return root

    def _getMin(self, node: TreeNode | None) -> TreeNode | None:
        while node.left:
            node = node.left
        return node

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def deleteNode(self, root: Optional[TreeNode], key: int) -> Optional[TreeNode]:
        if root is None:
            return None

```

```

if root.val > key:
    root.left = self.deleteNode(root.left, key)
    return root
if root.val < key:
    root.right = self.deleteNode(root.right, key)
    return root
if root.left is None:
    return root.right
if root.right is None:
    return root.left

```

```

node = root.right
while node.left:
    node = node.left
node.left = root.left
root = root.right
return root

```

• SimplyLeet

Definition for a binary tree node.

```
class TreeNode:
```

```

    def __init__(self, val=0, left=None, right=None):
        self.val = val          # Node's value
        self.left = left        # Pointer to left child
        self.right = right      # Pointer to right child

```

```
class Solution:
```

```

    def deleteNode(self, root: TreeNode, key: int) -> TreeNode:
        if not root:

```

```

            # If the tree is empty, return None.
            return None
        if key < root.val:
            # If the key is less, go to the left subtree.
            root.left = self.deleteNode(root.left, key)
        elif key > root.val:
            # If the key is greater, go to the right subtree.
            root.right = self.deleteNode(root.right, key)
        else:
            # Key found; now perform deletion.

```

```
if not root.left:
    # Node with only right child or no child.
    return root.right
elif not root.right:
    # Node with only left child.
    return root.left
else:
    # Node has two children.
    # Find the inorder successor (smallest in the right subtree).
    temp = root.right

    while temp.left:
        temp = temp.left
    # Replace the value of the node to be deleted.
    root.val = temp.val
    # Delete the inorder successor.
    root.right = self.deleteNode(root.right, temp.val)

return root
```

◆ Quick Review

Key Insights

- If the key is not found in the BST, return the original tree.
- There are three deletion cases:
- The node is a leaf (no children).
- The node has one child (left or right).
- The node has two children.
- For a node with two children, find the inorder successor (smallest node in the right subtree) or the inorder predecessor (largest node in the left subtree) to replace the node's

value.

- Recursively adjust the tree to preserve the BST invariants.

Space & Time

Time Complexity: $O(h)$, where h is the height of the tree (in the worst-case $O(n)$ for a skewed tree).

Space Complexity: $O(h)$, due to the recursion stack.

Solution

We use a recursive approach to traverse the tree to find the node with the given key. When the node is found:

- If it has zero or one child, return the non-null child or null.
- If it has two children, find the inorder successor (the smallest node in the right subtree), copy its value to the current node, and then delete the successor recursively. This ensures that the BST property is maintained after deletion.

BST / Ordered Set

□ #510 Inorder Successor in BST II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Binary Search Tree · Binary Tree
Lists: AlgoMaster 600

Problem

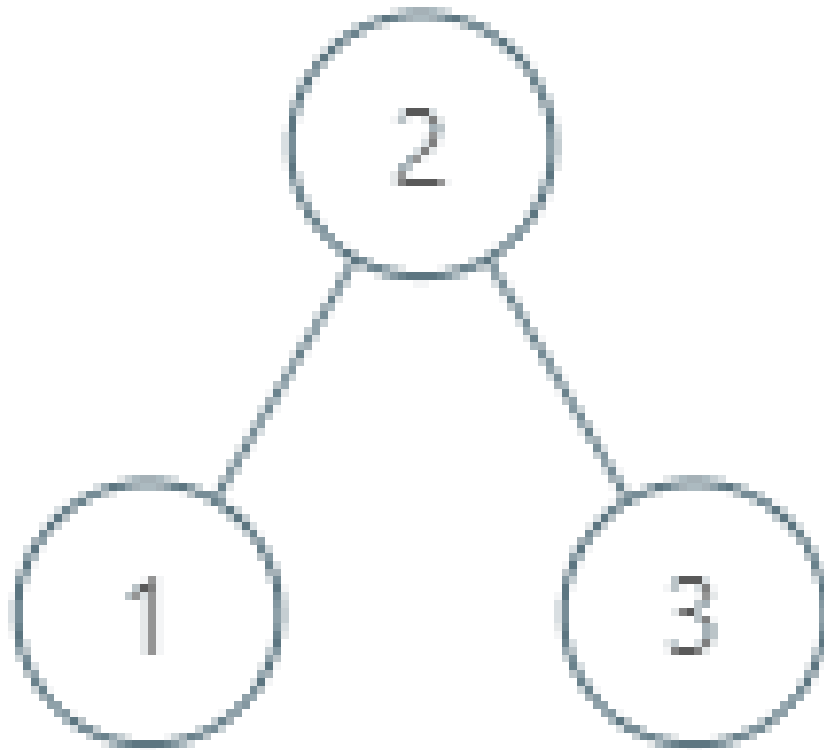
Given a node in a binary search tree, return the in-order successor of that node in the BST. If that node has no in-order successor, return null.

The successor of a node is the node with the smallest key greater than node.val.

You will have direct access to the node but not to the root of the tree. Each node will have a reference to its parent node. Below is the definition for Node:

```
class Node {  
    public int val;  
    public Node left;  
    public Node right;  
    public Node parent;  
}
```

Example 1:

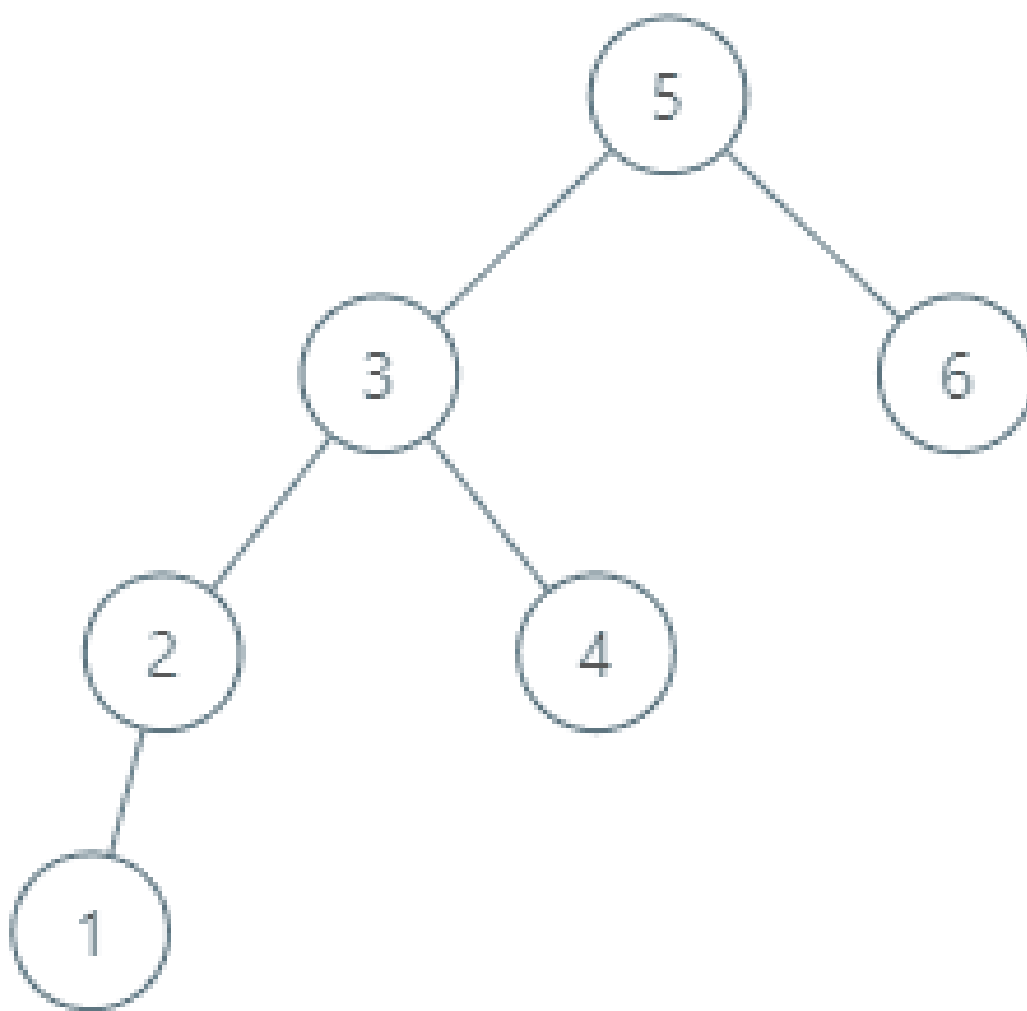


Input: tree = [2,1,3], node = 1

Output: 2

Explanation: 1's in-order successor node is 2. Note that both the node and the return value is of Node type.

Example 2:



Input: tree = [5,3,6,2,4,null,null,1], node = 6

Output: null

Explanation: There is no in-order successor of the current node, so the answer is null.

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-105 \leq \text{Node.val} \leq 105$
- All Nodes will have unique values.

Follow up: Could you solve it without looking up any of the node's values?

Community Solutions (Python)

• LeetCode

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.parent = None
"""

class Solution:
    def inorderSuccessor(self, node: 'Node') -> 'Optional[Node]':
        if node.right:
            node = node.right
            while node.left:
                node = node.left
            return node
        while node.parent and node.parent.right is node:
            node = node.parent
        return node.parent

```

• SimplyLeet

```

# Python solution with line-by-line comments

class Node:
    def __init__(self, val):
        self.val = val          # Node value
        self.left = None       # Left child node
        self.right = None      # Right child node
        self.parent = None     # Parent node

    def inorderSuccessor(self, node):

```

```
# If there is a right subtree, find its leftmost node.
if node.right:
    successor = node.right
    # Loop to find the leftmost child
    while successor.left:
        successor = successor.left
    return successor
# If no right subtree, traverse up using parent pointers.
parent = node.parent
while parent and node == parent.right:

    node = parent
    parent = parent.parent
return parent # This will be the in-order successor or None if not found
```

◆ Quick Review

Key Insights

- If the given node has a right child, the successor is the leftmost node in the right subtree.
- If the node does not have a right child, move up the tree using the parent pointer until you find a node that is a left child of its parent. That parent node is the successor.
- If no suitable parent is found, the given node is the largest element and its successor is null.

Space & Time

Time Complexity: $O(h)$, where h is the height of the tree.

Space Complexity: $O(1)$ using a constant amount of extra space.

Solution

We leverage the properties of a BST along with parent pointers. The algorithm first checks if the node has a right subtree. If yes, it finds the leftmost node in that subtree which is the in-order successor. If there is no right subtree, we traverse up using parent pointers until we find a node that is the left child of its parent, which indicates that the parent's value is the next greater element in the in-order traversal.

BST / Ordered Set

□ #669 Trim a Binary Search Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Binary Search Tree ·
Binary Tree

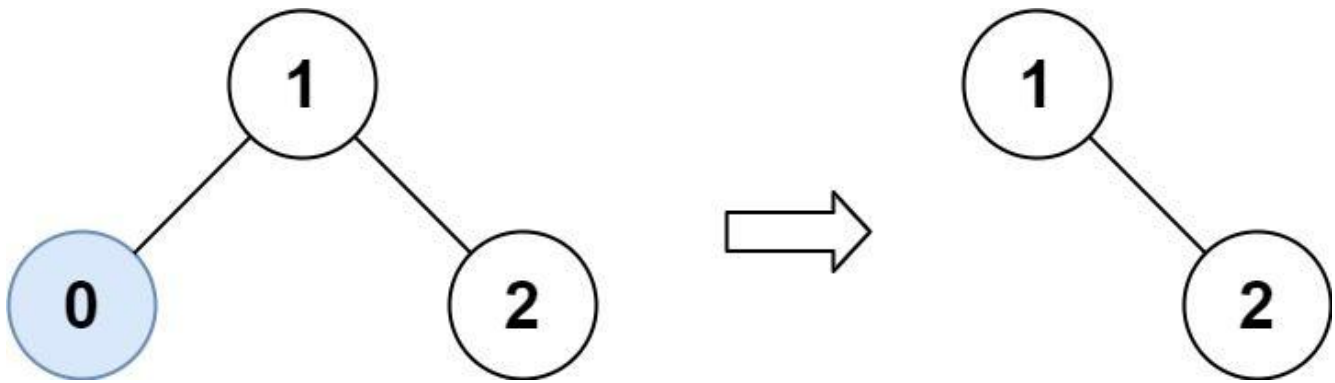
Lists: AlgoMaster 600

Problem

Given the root of a binary search tree and the lowest and highest boundaries as low and high, trim the tree so that all its elements lies in [low, high]. Trimming the tree should not change the relative structure of the elements that will remain in the tree (i.e., any node's descendant should remain a descendant). It can be proven that there is a unique answer.

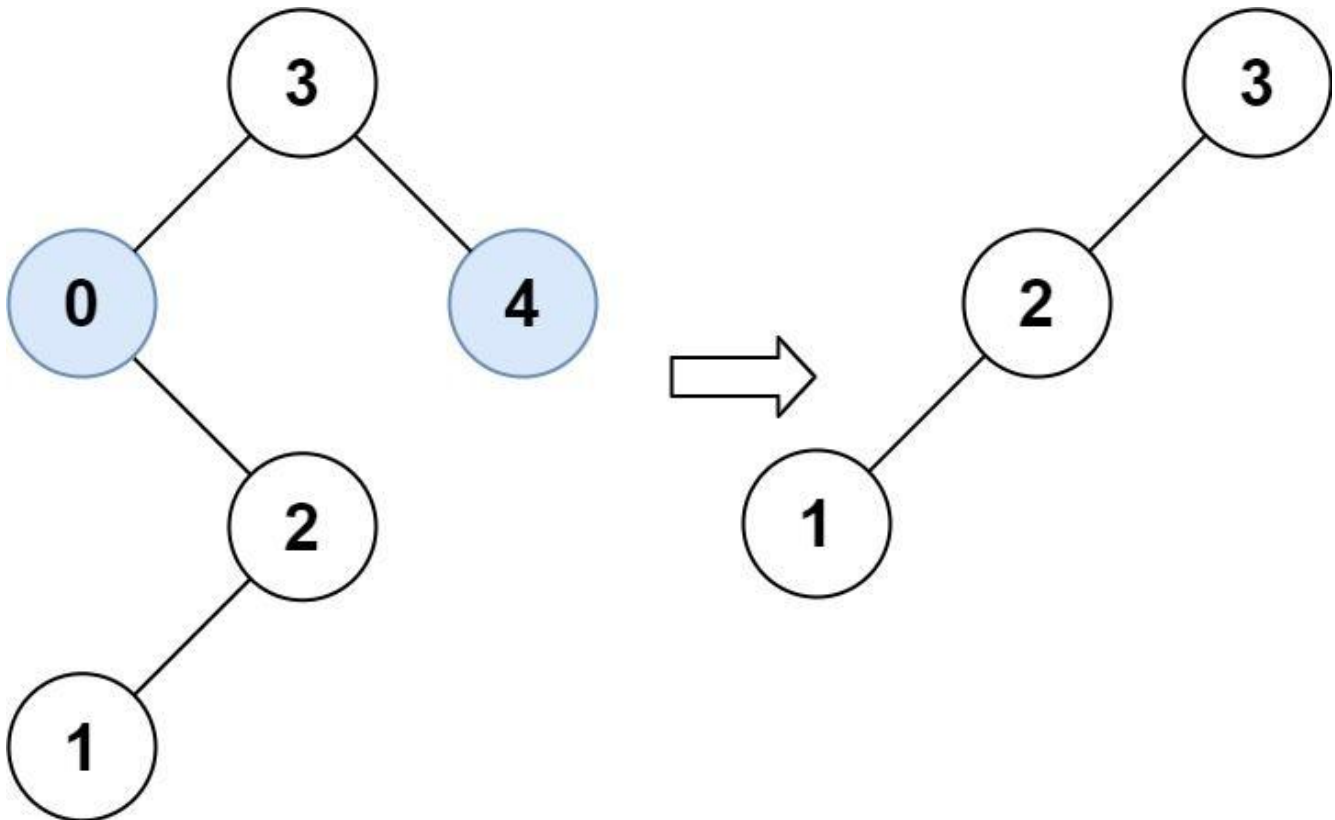
Return the root of the trimmed binary search tree. Note that the root may change depending on the given bounds.

Example 1:



Input: root = [1,0,2], low = 1, high = 2
 Output: [1,null,2]

Example 2:



Input: root = [3,0,4,null,2,null,null,1], low = 1, high = 3
 Output: [3,2,null,1]

Constraints:

- The number of nodes in the tree is in the range [1, 104].

- $0 \leq \text{Node.val} \leq 104$
- The value of each node in the tree is unique.
- root is guaranteed to be a valid binary search tree.
- $0 \leq \text{low} \leq \text{high} \leq 104$

Community Solutions (Python)

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def trimBST(
        self, root: Optional[TreeNode], low: int, high: int
    ) -> Optional[TreeNode]:

        def dfs(root):
            if root is None:
                return root
            if root.val > high:
                return dfs(root.left)
            if root.val < low:
                return dfs(root.right)
            root.left = dfs(root.left)
            root.right = dfs(root.right)
            return root

        return dfs(root)
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # Node's value
        self.left = left        # Left child pointer
        self.right = right      # Right child pointer

class Solution:
    def trimBST(self, root: TreeNode, low: int, high: int) -> TreeNode:
        # Base case: if node is None, return None.

        if not root:
            return None

        # If node's value is less than low, trim right subtree.
        if root.val < low:
            return self.trimBST(root.right, low, high)

        # If node's value is greater than high, trim left subtree.
        if root.val > high:
            return self.trimBST(root.left, low, high)

        # Otherwise, recursively trim left and right subtrees.
        root.left = self.trimBST(root.left, low, high)
        root.right = self.trimBST(root.right, low, high)
        return root
```

◆ Quick Review

Key Insights

- If a node's value is less than low, then its left subtree will also be less than low due to the BST property and can be discarded.
- If a node's value is greater than high, then its right subtree can be discarded.

- Use depth-first search (DFS) recursively to trim both the left and right subtrees.
- Maintain the original tree structure for the retained nodes.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, since each node is visited once.

Space Complexity: $O(n)$ in the worst-case for the recursion call stack ($O(\log n)$ on average for a balanced tree).

Solution

The solution employs a recursive DFS approach on the binary search tree. For each node, we:

- Check if the current node is null; if it is, return null.
- If the node's value is smaller than low, move to its right subtree (since all nodes in its left subtree are automatically out of range).
- If the node's value is greater than high, move to its left subtree.
- Otherwise, recursively trim the left and right subtrees. This approach takes advantage of

the BST properties to efficiently discard entire subtrees that are out of the allowed range without examining every node in those subtrees.

BST / Ordered Set

□ #701 Insert into a Binary Search Tree

MED

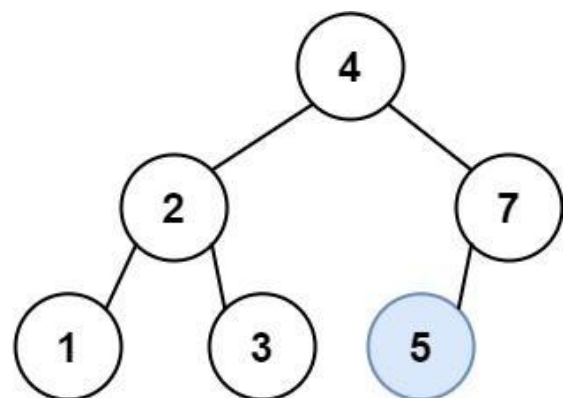
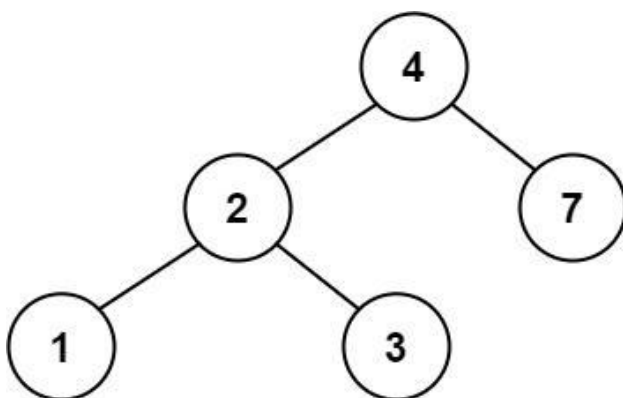
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Binary Search Tree · Binary Tree
Lists: AlgoMaster 600

Problem

You are given the root node of a binary search tree (BST) and a value to insert into the tree. Return the root node of the BST after the insertion. It is guaranteed that the new value does not exist in the original BST.

Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return any of them.

Example 1:



Input: root = [4,2,7,1,3], val = 5
Output: [4,2,7,1,3,5]
Explanation: Another accepted tree is:

Example 2:

Input: root = [40,20,60,10,30,50,70], val = 25
Output: [40,20,60,10,30,50,70,null,null,25]

Example 3:

Input: root = [4,2,7,1,3,null,null,null,null,null,null], val = 5
Output: [4,2,7,1,3,5]

Constraints:

- The number of nodes in the tree will be in the range [0, 104].
- $-108 \leq \text{Node.val} \leq 108$
- All the values Node.val are unique.
- $-108 \leq \text{val} \leq 108$
- It's guaranteed that val does not exist in the original BST.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def insertIntoBST(self, root: TreeNode | None, val: int) -> TreeNode | None:
        if not root:
            return TreeNode(val)
        if root.val > val:
            root.left = self.insertIntoBST(root.left, val)
        else:
            root.right = self.insertIntoBST(root.right, val)
        return root
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def insertIntoBST(self, root: Optional[TreeNode], val: int) -> Optional[TreeNode]:
        if root is None:
            return TreeNode(val)

        if root.val > val:
            root.left = self.insertIntoBST(root.left, val)
        else:
            root.right = self.insertIntoBST(root.right, val)
        return root
```

• SimplyLeet

```
# Define the structure for a tree node
class TreeNode:
    def __init__(self, x):
        self.val = x
        self.left = None
        self.right = None

def insertIntoBST(root, val):
    # If reaching a null node, insert the new node here.
    if root is None:
        return TreeNode(val)
    # If the value to insert is less than the current node's value,
    # move to the left subtree.
    if val < root.val:
        root.left = insertIntoBST(root.left, val)
    # Otherwise, move to the right subtree.
    else:
        root.right = insertIntoBST(root.right, val)
    # Return the unchanged root pointer.
    return root
```

```
# Example usage:
# root = TreeNode(4)
# root.left = TreeNode(2)
# root.right = TreeNode(7)
# root.left.left = TreeNode(1)
# root.left.right = TreeNode(3)
# new_root = insertIntoBST(root, 5)
```

◆ Quick Review

Key Insights

- A BST has the property that for any node, all the nodes in its left subtree are smaller and all the nodes in its right subtree are larger.
- Insertion into a BST involves following the BST property and finding the appropriate null location to attach the new node.
- The process can be implemented recursively or iteratively.
- The problem guarantees uniqueness so no duplicate checks are needed.

Space & Time

Time Complexity: $O(h)$ where h is the height of the BST (in the worst case, for a skewed tree, $O(n)$).

Space Complexity: $O(h)$ for the recursion stack in the recursive approach (or $O(1)$ with an iterative solution).

Solution

We use the BST property to insert the new node. Starting from the root, if the value to insert is less than the current node's value, we proceed to the left subtree; otherwise, we proceed to the right subtree. This continues until we find a null pointer where we can insert the new value as a new node. Key steps:

- Check if the current node is null; if yes, create a new node with the given value.**
- Recurse (or iterate) down the tree until the correct insertion point is found.**
- Update the left or right pointers accordingly.**

This approach uses recursion for clarity, though an iterative implementation is also straightforward.

BST / Ordered Set

□ #729 My Calendar I

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Design · Segment Tree
· Ordered Set

Lists: AlgoMaster 600

Problem

You are implementing a program to use as your calendar. We can add a new event if adding the event will not cause a double booking.

A double booking happens when two events have some non-empty intersection (i.e., some moment is common to both events.).

The event can be represented as a pair of integers `startTime` and `endTime` that represents a booking on the half-open interval $[startTime, endTime)$, the range of real numbers x such that $startTime \leq x < endTime$.

Implement the `MyCalendar` class:

- `MyCalendar()` Initializes the calendar object.

- **boolean book(int startTime, int endTime)**
Returns true if the event can be added to the calendar successfully without causing a double booking. Otherwise, return false and do not add the event to the calendar.

Example 1:

Input

```
["MyCalendar", "book", "book", "book"]  
[[], [10, 20], [15, 25], [20, 30]]
```

Output

```
[null, true, false, true]
```

Explanation

```
MyCalendar myCalendar = new MyCalendar();
```

Constraints:

- $0 \leq \text{start} < \text{end} \leq 109$
- At most 1000 calls will be made to book.

Community Solutions (Python)

• WalkCC

```
class MyCalendar:  
    def __init__(self):  
        self.line = []  
  
    def book(self, startTime: int, endTime: int) -> bool:  
        for s, e in self.line:  
            if max(startTime, s) < min(endTime, e):  
                return False  
        self.line.append((startTime, endTime))  
        return True
```

```
from dataclasses import dataclass

@dataclass
class Node:
    start: int
    end: int
    left = None
    right = None

class Tree:
    def __init__(self):
        self.root = None

    def insert(self, node: Node, root: Node = None) -> bool:
        if not root:
            if not self.root:
                self.root = node
                return True
        else:
            root = self.root

        if node.start >= root.end:
            if not root.right:
                root.right = node
                return True
            return self.insert(node, root.right)
        elif node.end <= root.start:
            if not root.left:
                root.left = node
                return True
            return self.insert(node, root.left)
        else:
            return False

class MyCalendar:
    def __init__(self):
        self.tree = Tree()
```

```
def book(self, start: int, end: int) -> bool:
    return self.tree.insert(Node(start, end))
```

• LeetDoocs

```
class MyCalendar:

    def __init__(self):
        self.sd = SortedDict()

    def book(self, start: int, end: int) -> bool:
        idx = self.sd.bisect_right(start)
        if idx < len(self.sd) and self.sd.values()[idx] < end:
            return False
        self.sd[end] = start

        return True

# Your MyCalendar object will be instantiated and called as such:
# obj = MyCalendar()
# param_1 = obj.book(start,end)
```

• SimplyLeet

```
# Python implementation for My Calendar I

class MyCalendar:
    def __init__(self):
        # List to store booked intervals as (start, end) tuples
        self.bookings = []

    def book(self, start: int, end: int) -> bool:
        # Check each existing booking for an overlap
        for s, e in self.bookings:
```

```
# If new event overlaps with existing event, return False
if not (end <= s or start >= e):
    return False
# Could not find overlap, so add the new event
self.bookings.append((start, end))
return True

# Example usage:
# myCalendar = MyCalendar()
# print(myCalendar.book(10, 20)) # True

# print(myCalendar.book(15, 25)) # False
# print(myCalendar.book(20, 30)) # True
```

◆ Quick Review

Key Insights

- Use a list (or array) to store currently booked time intervals.
- An event [start, end) does not conflict with another if its end time is less than or equal to the other's start time or its start time is greater than or equal to the other's end time.
- With at most 1000 booking attempts, a simple linear scan for overlapping intervals is acceptable.
- Optionally, for efficiency or scalability, maintain the intervals in a sorted order and use binary search, checking only immediate neighbors for overlaps.

Space & Time

Time Complexity: $O(n)$ per booking in the worst-case scenario, where n is the number of existing events.

Space Complexity: $O(n)$, as all events are stored.

Solution

The solution maintains a list of tuples (or pairs) representing booked events. When booking a new event, iterate through the list to check if the new interval overlaps with any already booked interval. The overlap condition is evaluated by ensuring that for every booked interval $[s, e)$, the new event $[start, end)$ satisfies $(end \leq s \text{ or } start \geq e)$. If it does, append the new event and return true. If there is any overlap, return false without adding the event.

BST / Ordered Set

□ #731 My Calendar II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Design · Segment Tree
· Prefix Sum · Ordered Set

Lists: AlgoMaster 600

Problem

You are implementing a program to use as your calendar. We can add a new event if adding the event will not cause a triple booking.

A triple booking happens when three events have some non-empty intersection (i.e., some moment is common to all the three events.).

The event can be represented as a pair of integers `startTime` and `endTime` that represents a booking on the half-open interval $[startTime, endTime)$, the range of real numbers x such that $startTime \leq x < endTime$.

Implement the `MyCalendarTwo` class:

- `MyCalendarTwo()` Initializes the calendar object.

- **boolean book(int startTime, int endTime)**
Returns true if the event can be added to the calendar successfully without causing a triple booking. Otherwise, return false and do not add the event to the calendar.

Example 1:

Input

```
["MyCalendarTwo", "book", "book", "book", "book", "book", "book"]
[[], [10, 20], [50, 60], [10, 40], [5, 15], [5, 10], [25, 55]]
```

Output

```
[null, true, true, true, false, true, true]
```

Explanation

```
MyCalendarTwo myCalendarTwo = new MyCalendarTwo();
```

Constraints:

- $0 \leq \text{start} < \text{end} \leq 109$
- At most 1000 calls will be made to book.

Community Solutions (Python)

• LeetDoocs

```
class MyCalendarTwo:

    def __init__(self):
        self.sd = SortedDict()

    def book(self, startTime: int, endTime: int) -> bool:
        self.sd[startTime] = self.sd.get(startTime, 0) + 1
        self.sd[endTime] = self.sd.get(endTime, 0) - 1
        s = 0
        for v in self.sd.values():
```

```
s += v
if s > 2:
    self.sd[startTime] -= 1
    self.sd[endTime] += 1
    return False
return True
```

```
# Your MyCalendarTwo object will be instantiated and called as such:
# obj = MyCalendarTwo()

# param_1 = obj.book(startTime, endTime)
```

```
class Node:
    def __init__(self, l: int, r: int):
        self.left = None
        self.right = None
        self.l = l
        self.r = r
        self.mid = (l + r) >> 1
        self.v = 0
        self.add = 0
```

```
class SegmentTree:
    def __init__(self):
        self.root = Node(1, 10**9 + 1)

    def modify(self, l: int, r: int, v: int, node: Node = None):
        if l > r:
            return
        if node is None:
            node = self.root
```

```

    if node.l >= l and node.r <= r:
        node.v += v
        node.add += v
        return
    self.pushdown(node)
    if l <= node.mid:
        self.modify(l, r, v, node.left)
    if r > node.mid:
        self.modify(l, r, v, node.right)
    self.pushup(node)

```

```

def query(self, l: int, r: int, node: Node = None) -> int:
    if l > r:
        return 0
    if node is None:
        node = self.root
    if node.l >= l and node.r <= r:
        return node.v
    self.pushdown(node)
    v = 0

```

```

    if l <= node.mid:
        v = max(v, self.query(l, r, node.left))
    if r > node.mid:
        v = max(v, self.query(l, r, node.right))
    return v

```

```

def pushup(self, node: Node):
    node.v = max(node.left.v, node.right.v)

```

```

def pushdown(self, node: Node):

```

```

    if node.left is None:
        node.left = Node(node.l, node.mid)
    if node.right is None:
        node.right = Node(node.mid + 1, node.r)
    if node.add:
        node.left.v += node.add
        node.right.v += node.add
        node.left.add += node.add
        node.right.add += node.add
        node.add = 0

```

```
class MyCalendarTwo:
    def __init__(self):
        self.tree = SegmentTree()

    def book(self, startTime: int, endTime: int) -> bool:
        if self.tree.query(startTime + 1, endTime) >= 2:
            return False
        self.tree.modify(startTime + 1, endTime, 1)

        return True
```

```
# Your MyCalendarTwo object will be instantiated and called as such:
# obj = MyCalendarTwo()
# param_1 = obj.book(startTime,endTime)
```

• SimplyLeet

```
# Python implementation of MyCalendarTwo
class MyCalendarTwo:
    def __init__(self):
        # List to store all the booked events
        self.booked = []
        # List to store all the double-booked intervals (overlaps)
        self.overlaps = []

    def book(self, start: int, end: int) -> bool:
        # Check if new event overlaps with any double-booked interval.
```

```
        for os, oe in self.overlaps:
            # If there's any overlap with a double-booked interval, triple
            booking occurs.
            if start < oe and end > os:
                return False

        # For all previously booked events, check if they overlap with the new
        event.
        for bs, be in self.booked:
            if start < be and end > bs:
                # Calculate the overlapping segment and record it as a double
                booking.
                self.overlaps.append((max(start, bs), min(end, be)))

        # Add the new event to the booked list as it doesn't cause a triple
        booking.
        self.booked.append((start, end))
        return True

# Example usage:
# myCalendar = MyCalendarTwo()
# print(myCalendar.book(10, 20)) # True
# print(myCalendar.book(50, 60)) # True
# print(myCalendar.book(10, 40)) # True

# print(myCalendar.book(5, 15)) # False
```

◆ Quick Review

Key Insights

- Maintain two separate collections: one for all booked events and one for the periods that are already double booked.
- When booking a new event, first check if it conflicts with any of the double booked

intervals. If it does, adding the event would create a triple booking.

- Otherwise, for every existing event, compute and record any new overlap created by the new event with that event. These overlaps form the new double booking intervals.
- Finally, add the new event to the collection of all bookings.

Space & Time

Time Complexity: $O(n)$ per booking call in the worst case, where n is the number of events booked so far.

Space Complexity: $O(n)$ to store all the bookings and overlaps.

Solution

We use a two-list approach:

- A list (or array) to record all booked events.
- A list (or array) to record all intervals where double bookings have occurred.

For each new event:

- First check the new event interval against all intervals in the second list (overlaps). If any overlap exists, then a triple booking would

occur, and we return false.

- Otherwise, for each event already booked, if there is an overlap with the new event, compute that overlapping interval and add it to the overlaps list.

- Finally, add the new event to the booked events list and return true.

This approach leverages the idea of interval intersection and simple linear scans over the events. The challenge lies in correctly computing the overlapping intervals and ensuring that triple overlaps are not introduced.

BST / Ordered Set

□ #1008 Construct Binary Search Tree from Preorder Traversal

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Tree · Binary Search Tree ·
Monotonic Stack · Binary Tree
Lists: AlgoMaster 600

Problem

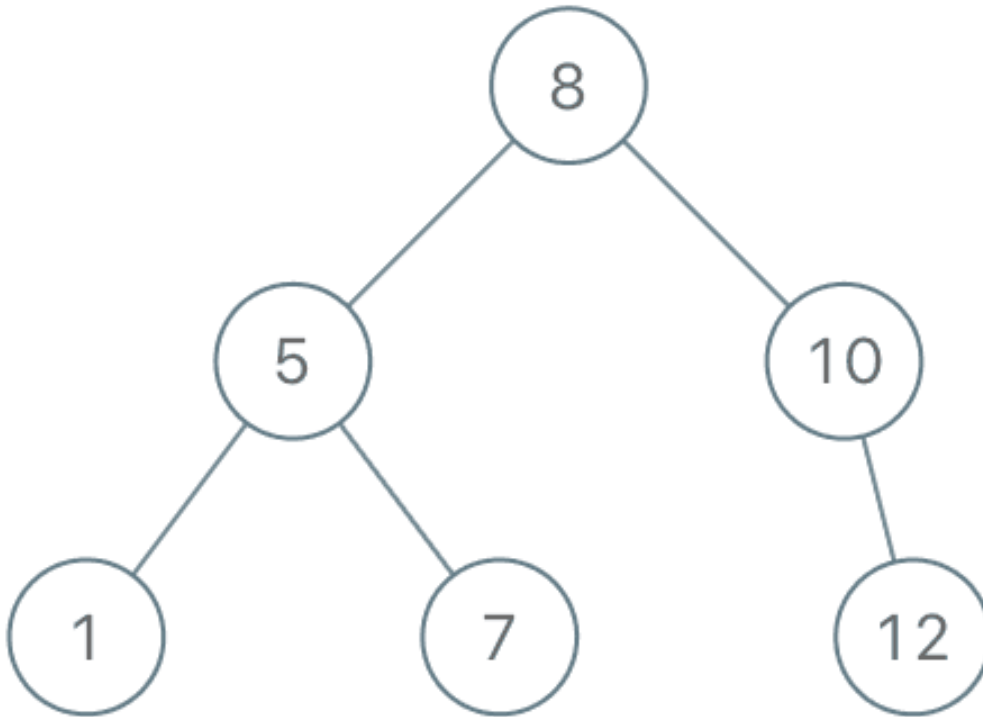
Given an array of integers preorder, which represents the preorder traversal of a BST (i.e., binary search tree), construct the tree and return its root.

It is guaranteed that there is always possible to find a binary search tree with the given requirements for the given test cases.

A binary search tree is a binary tree where for every node, any descendant of Node.left has a value strictly less than Node.val, and any descendant of Node.right has a value strictly greater than Node.val.

A preorder traversal of a binary tree displays the value of the node first, then traverses

Node.left, then traverses Node.right.
Example 1:



Input: preorder = [8,5,1,7,10,12]
Output: [8,5,10,1,7,null,12]

Example 2:

Input: preorder = [1,3]
Output: [1,null,3]

Constraints:

- $1 \leq \text{preorder.length} \leq 100$
- $1 \leq \text{preorder}[i] \leq 1000$
- All the values of preorder are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def bstFromPreorder(self, preorder: list[int]) -> TreeNode | None:
        root = TreeNode(preorder[0])
        stack = [root]

        for i in range(1, len(preorder)):
            parent = stack[-1]
            child = TreeNode(preorder[i])
            # Adjust the parent.
            while stack and stack[-1].val < child.val:

                parent = stack.pop()
            # Create parent-child link according to BST property.
            if parent.val > child.val:
                parent.left = child
            else:
                parent.right = child
            stack.append(child)

        return root
```

• LeetDoocs

```
class Solution:
    def bstFromPreorder(self, preorder: List[int]) -> Optional[TreeNode]:
        def dfs(i: int, j: int) -> Optional[TreeNode]:
            if i > j:
                return None
            root = TreeNode(preorder[i])
            l, r = i + 1, j + 1
            while l < r:
                mid = (l + r) >> 1
                if preorder[mid] > preorder[i]:
```

```

        r = mid
    else:
        l = mid + 1
    root.left = dfs(i + 1, l - 1)
    root.right = dfs(l, j)
    return root

return dfs(0, len(preorder) - 1)

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# Recursive helper function that utilizes bounds to construct the BST.
def bstFromPreorder(preorder):
    # use a list to simulate a pointer that holds the current index reference

    index = [0]

    # Recursive function to build BST within the lower and upper bounds
    def buildBST(lower, upper):
        # If all elements are used, return None.
        if index[0] == len(preorder):
            return None
        # next value in the preorder sequence
        val = preorder[index[0]]
        # If the value does not satisfy BST constraints, return None.

```

```
if val < lower or val > upper:
    return None

# Accept the value and move to the next element
index[0] += 1
# Create a new tree node with the current value.
node = TreeNode(val)
# All values for the left subtree must be less than the current node's
value.
node.left = buildBST(lower, val)
# All values for the right subtree must be greater than the current
node's value.

node.right = buildBST(val, upper)
return node

return buildBST(float('-inf'), float('inf'))

# Example usage:
# preorder = [8,5,1,7,10,12]
# root = bstFromPreorder(preorder)
```

◆ Quick Review

Key Insights

- The preorder sequence always starts with the root of the tree.
- For any node, elements in the left subtree must be less than the node's value, while elements in the right subtree must be greater.
- Use recursion with bounds (or an iterative approach with a stack) to determine where a new value fits in the BST.

- Maintain an index into the preorder array to build the tree sequentially.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes since each node is processed exactly once.

Space Complexity: $O(n)$ in the worst case due to the recursion call stack (for a skewed tree).

Solution

We can solve this problem using a recursive approach. The recursive function will have lower and upper bounds which restrict the valid range for node values at each stage. Starting with the entire range $(-\infty, +\infty)$, we process each value in the preorder list in order. If the next value lies within the bounds, it is accepted as a node. We then recursively construct the left subtree with an updated upper bound (current node's value) and the right subtree with an updated lower bound (current node's value). This ensures that the BST property is maintained throughout.

The key data structure used is the recursion call stack, which inherently manages the

bounds for a valid BST construction. The algorithm processes each element of the preorder sequence exactly once, leading to optimal time complexity.

BST / Ordered Set

□ #2034 Stock Price Fluctuation

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Design · Heap (Priority Queue) ·
Data Stream · Ordered Set
Lists: AlgoMaster 600

Problem

You are given a stream of records about a particular stock. Each record contains a timestamp and the corresponding price of the stock at that timestamp.

Unfortunately due to the volatile nature of the stock market, the records do not come in order. Even worse, some records may be incorrect. Another record with the same timestamp may appear later in the stream correcting the price of the previous wrong record.

Design an algorithm that:

- Updates the price of the stock at a particular timestamp, correcting the price from any previous records at the timestamp.

- Finds the latest price of the stock based on the current records. The latest price is the price at the latest timestamp recorded.
- Finds the maximum price the stock has been based on the current records.
- Finds the minimum price the stock has been based on the current records.

Implement the StockPrice class:

- **StockPrice()** Initializes the object with no price records.
- **void update(int timestamp, int price)**
Updates the price of the stock at the given timestamp.
- **int current()** Returns the latest price of the stock.
- **int maximum()** Returns the maximum price of the stock.
- **int minimum()** Returns the minimum price of the stock.

Example 1:

Input

```
["StockPrice", "update", "update", "current", "maximum", "update", "maximum",
"update", "minimum"]
```

```
[[], [1, 10], [2, 5], [], [], [1, 3], [], [4, 2], []]
```

Output

```
[null, null, null, 5, 10, null, 5, null, 2]
```

Explanation

```
StockPrice stockPrice = new StockPrice();
```

Constraints:

- $1 \leq \text{timestamp}$, $\text{price} \leq 109$
- At most 105 calls will be made in total to update, current, maximum, and minimum.
- current, maximum, and minimum will be called only after update has been called at least once.

Community Solutions (Python)

• WalkCC

```
from sortedcontainers import SortedDict

class StockPrice:
    def __init__(self):
        self.timestampToPrice = SortedDict()
        self.pricesCount = SortedDict()

    def update(self, timestamp: int, price: int) -> None:
        if timestamp in self.timestampToPrice:
```

```

    prevPrice = self.timestampToPrice[timestamp]
    self.pricesCount[prevPrice] -= 1
    if self.pricesCount[prevPrice] == 0:
        del self.pricesCount[prevPrice]
    self.timestampToPrice[timestamp] = price
    self.pricesCount[price] = self.pricesCount.get(price, 0) + 1

def current(self) -> int:
    return self.timestampToPrice.peekitem(-1)[1]

def maximum(self) -> int:
    return self.pricesCount.peekitem(-1)[0]

def minimum(self) -> int:
    return self.pricesCount.peekitem(0)[0]

```

• LeetDoocs

```

class StockPrice:
    def __init__(self):
        self.d = {}
        self.ls = SortedList()
        self.last = 0

    def update(self, timestamp: int, price: int) -> None:
        if timestamp in self.d:
            self.ls.remove(self.d[timestamp])
        self.d[timestamp] = price

        self.ls.add(price)
        self.last = max(self.last, timestamp)

    def current(self) -> int:
        return self.d[self.last]

    def maximum(self) -> int:
        return self.ls[-1]

    def minimum(self) -> int:

```

```
return self.ls[0]
```

```
# Your StockPrice object will be instantiated and called as such:
# obj = StockPrice()
# obj.update(timestamp,price)
# param_2 = obj.current()
# param_3 = obj.maximum()
# param_4 = obj.minimum()
```

• SimplyLeet

```
# Python solution
import heapq

class StockPrice:
    def __init__(self):
        # Dictionary mapping timestamp to price
        self.price_by_timestamp = {}
        # Track the latest timestamp seen so far
        self.latest_timestamp = 0
        # Min heap to obtain the minimum price quickly; stores (price,
        timestamp)

        self.min_heap = []
        # Max heap to obtain the maximum price quickly; stores (-price,
        timestamp)
        self.max_heap = []

    def update(self, timestamp: int, price: int) -> None:
        # Update the dictionary with the new price for the given timestamp
        self.price_by_timestamp[timestamp] = price
        # Update the latest timestamp if necessary
        self.latest_timestamp = max(self.latest_timestamp, timestamp)
        # Push the new price with its timestamp into the min heap
```

```
heapq.heappush(self.min_heap, (price, timestamp))
# Push the negative of price to simulate a max heap into the max heap
heapq.heappush(self.max_heap, (-price, timestamp))

def current(self) -> int:
    # Return the price corresponding to the latest timestamp
    return self.price_by_timestamp[self.latest_timestamp]

def maximum(self) -> int:
    # Remove outdated entries from the max heap

    while self.max_heap:
        # Get the price (as negative) and associated timestamp from the top
        # of the heap
        neg_price, timestamp = self.max_heap[0]
        # If the price in the dictionary doesn't match the current heap
        # entry, it's outdated
        if self.price_by_timestamp[timestamp] != -neg_price:
            heapq.heappop(self.max_heap)
        else:
            return -neg_price
    return -1 # Edge case (should not occur as current is called after an
update)

def minimum(self) -> int:
    # Remove outdated entries from the min heap
    while self.min_heap:
        price, timestamp = self.min_heap[0]
        # If the price in the dictionary doesn't match, pop it as it's
        # outdated
        if self.price_by_timestamp[timestamp] != price:
            heapq.heappop(self.min_heap)
        else:
            return price
    return -1 # Edge case (should not occur as current is called after an
update)
```

◆ Quick Review

Key Insights

- Use a hash table (or dictionary) to map each timestamp to its latest price.
- Maintain two heaps (a max heap and a min heap) to quickly retrieve the maximum and minimum prices.
- Since updates can change historical prices, use lazy deletion in the heaps: when the top element does not match the current price in the hash table, repeatedly pop it.
- Track the latest timestamp separately to efficiently return the current/latest price.

Space & Time

Time Complexity:

- update: $O(\log n)$ due to pushing into two heaps.
- current: $O(1)$.
- maximum/minimum: $O(\log n)$ in the worst-case (when cleaning outdated heap entries).

Space Complexity:

- $O(n)$ to store the hash table and both heaps.

Solution

We use a hash table (or dictionary) to store the mapping from timestamp to price. For maximum and minimum queries, we use a max heap (storing negative prices for a max-oriented comparison) and a min heap. When updating a timestamp, we update the hash table and push the new (price, timestamp) pair into both heaps. For queries, we repeatedly check the top of the heap and compare with the hash table – if it is outdated (i.e. the hash table's price for the timestamp does not match the heap entry), we pop the invalid entry until we find a valid one. We keep track of the latest timestamp in a variable so that the current price query is efficient.

Tries

Round 5 · Mid · Medium · 6 questions

Tries

□ #421 Maximum XOR of Two Numbers in an Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Bit Manipulation · Trie
Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the maximum result of `nums[i] XOR nums[j]`, where $0 \leq i \leq j < n$.

Example 1:

Input: `nums = [3,10,5,25,2,8]`
Output: 28
Explanation: The maximum result is `5 XOR 25 = 28`.

Example 2:

Input: `nums = [14,70,53,83,49,91,36,80,92,51,66,70]`
Output: 127

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^5$
- $0 \leq \text{nums}[i] \leq 2^{31} - 1$

Community Solutions (Python)

- WalkCC


```

class Solution:
    def findMaximumXOR(self, nums: list[int]) -> int:
        maxNum = max(nums)
        if maxNum == 0:
            return 0
        maxBit = int(math.log2(maxNum))
        ans = 0
        prefixMask = 0 # `prefixMask` grows like: 10000 -> 11000 -> ... -> 11111.

        # If ans is 11100 when i = 2, it means that before we reach the last two

        # bits, 11100 is the maximum XOR we have, and we're going to explore if we
        # can get another two 1s and put them into `ans`.
        for i in range(maxBit, -1, -1):
            prefixMask |= 1 << i
            # We only care about the left parts,
            # If i = 2, nums = [1110, 1011, 0111]
            # -> prefixes = [1100, 1000, 0100]
            prefixes = set([num & prefixMask for num in nums])
            # If i = 1 and before this iteration, the ans is 10100, it means that we
            # want to grow the ans to 10100 | 1 << 1 = 10110 and we're looking for

            # XOR of two prefixes = candidate.
            candidate = ans | 1 << i
            for prefix in prefixes:
                if prefix ^ candidate in prefixes:
                    ans = candidate
                    break

        return ans

class TrieNode:
    def __init__(self):
        self.children: list[TrieNode | None] = [None] * 2

class BitTrie:
    def __init__(self, maxBit: int):
        self.maxBit = maxBit
        self.root = TrieNode()

```

```

def insert(self, num: int) -> None:
    node = self.root
    for i in range(self.maxBit, -1, -1):
        bit = num >> i & 1
        if not node.children[bit]:
            node.children[bit] = TrieNode()
        node = node.children[bit]

def getMaxXor(self, num: int) -> int:
    maxXor = 0

    node = self.root
    for i in range(self.maxBit, -1, -1):
        bit = num >> i & 1
        toggleBit = bit ^ 1
        if node.children[toggleBit]:
            maxXor = maxXor | 1 << i
            node = node.children[toggleBit]
        elif node.children[bit]:
            node = node.children[bit]
        else: # There's nothing in the Bit Trie.

    return 0
    return maxXor

```

```

class Solution:
    def findMaximumXOR(self, nums: List[int]) -> int:
        maxNum = max(nums)
        if maxNum == 0:
            return 0
        maxBit = int(math.log2(maxNum))

```

```

    ans = 0
    bitTrie = BitTrie(maxBit)

    for num in nums:
        ans = max(ans, bitTrie.getMaxXor(num))
        bitTrie.insert(num)

    return ans

```

• LeetDoocs

```

class Trie:
    __slots__ = ("children",)

    def __init__(self):
        self.children: List[Trie | None] = [None, None]

    def insert(self, x: int):
        node = self
        for i in range(30, -1, -1):
            v = x >> i & 1

            if node.children[v] is None:
                node.children[v] = Trie()
            node = node.children[v]

    def search(self, x: int) -> int:
        node = self
        ans = 0
        for i in range(30, -1, -1):
            v = x >> i & 1
            if node.children[v ^ 1]:
                ans |= 1 << i
                node = node.children[v ^ 1]
            else:
                node = node.children[v]
        return ans

class Solution:
    def findMaximumXOR(self, nums: List[int]) -> int:
        trie = Trie()

        for x in nums:
            trie.insert(x)
        return max(trie.search(x) for x in nums)

```

• SimplyLeet

```
# Define the node for Trie
class TrieNode:
    def __init__(self):
        self.children = {} # Dictionary to hold child nodes for bits 0 and 1

# Define the Trie data structure
class Trie:
    def __init__(self):
        self.root = TrieNode()

# Insert a number's binary representation into the Trie
def insert(self, num):
    node = self.root
    # Consider 32-bit integers (from bit 31 to 0)
    for i in range(31, -1, -1):
        bit = (num >> i) & 1 # Extract the i-th bit
        if bit not in node.children:
            node.children[bit] = TrieNode()
        node = node.children[bit]

# Query for the maximum XOR partner for the given number
def query(self, num):
    node = self.root
    xor_sum = 0
    for i in range(31, -1, -1):
        bit = (num >> i) & 1
        toggled = 1 - bit # Complement of the current bit
        if toggled in node.children:
            xor_sum |= (1 << i) # Set the i-th bit in the result
            node = node.children[toggled]
        else:
            node = node.children.get(bit, node)
    return xor_sum

class Solution:
    def findMaximumXOR(self, nums):
        trie = Trie()
        max_xor = 0
        # Insert the first number to initialize the Trie
        trie.insert(nums[0])
```

```
for num in nums[1:]:
    # For each number, query the best possible XOR companion
    current_xor = trie.query(num)
    max_xor = max(max_xor, current_xor)
    trie.insert(num)
return max_xor

# Example usage:
# sol = Solution()
# print(sol.findMaximumXOR([3,10,5,25,2,8])) # Expected output: 28
```

◆ Quick Review

Key Insights

- The XOR operation is bitwise, so the maximum result is primarily influenced by the higher-order bits.
- A Trie (prefix tree) can be used to store the binary representation of the numbers enabling efficient bit-by-bit comparison.
- By inserting numbers into the Trie and then checking for the complement bit (i.e., if bit is 0 look for 1 and vice versa), one can greedily maximize the XOR at each bit-level.
- The solution runs in $O(n * L)$ time, where L is the number of bits (typically 32 for standard integers).

Space & Time

Time Complexity: $O(n * L)$ where $L = 32$, so essentially $O(n)$.

Space Complexity: $O(n * L)$ in the worst-case scenario for storing nodes in the Trie.

Solution

We use a Trie data structure to store the binary representations of the numbers in the array. For each number, we traverse the Trie trying to take the opposite branch (complement bit) at each level to maximize the resulting XOR. We then update the maximum XOR seen so far. This approach efficiently checks for the best possible XOR pair for every element in `nums`.

Tries

□ #648 Replace Words

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Trie
Lists: AlgoMaster 600

Problem

In English, we have a concept called root, which can be followed by some other word to form another longer word – let's call this word derivative. For example, when the root "help" is followed by the word "ful", we can form a derivative "helpful".

Given a dictionary consisting of many roots and a sentence consisting of words separated by spaces, replace all the derivatives in the sentence with the root forming it. If a derivative can be replaced by more than one root, replace it with the root that has the shortest length. Return the sentence after the replacement.

Example 1:

```
Input: dictionary = ["cat","bat","rat"], sentence = "the cattle was rattled by the battery"
Output: "the cat was rat by the bat"
```

Example 2:

```
Input: dictionary = ["a","b","c"], sentence = "aadsfasf absbs bbab cadsfafs"
Output: "a a b c"
```

Constraints:

- $1 \leq \text{dictionary.length} \leq 1000$
- $1 \leq \text{dictionary}[i].\text{length} \leq 100$
- `dictionary[i]` consists of only lower-case letters.
- $1 \leq \text{sentence.length} \leq 106$
- `sentence` consists of only lower-case letters and spaces.
- The number of words in `sentence` is in the range $[1, 1000]$
- The length of each word in `sentence` is in the range $[1, 1000]$
- Every two consecutive words in `sentence` will be separated by exactly one space.
- `sentence` does not have leading or trailing spaces.

Community Solutions (Python)

- ☐
- WalkCC


```

class Solution:
    def __init__(self):
        self.root = {}

    def insert(self, word: str) -> None:
        node = self.root
        for c in word:
            if c not in node:
                node[c] = {}
            node = node[c]

        node['word'] = word

    def search(self, word: str) -> str:
        node = self.root
        for c in word:
            if 'word' in node:
                return node['word']
            if c not in node:
                return word
            node = node[c]

        return word

    def replaceWords(self, dictionary: list[str], sentence: str) -> str:
        for word in dictionary:
            self.insert(word)

        words = sentence.split(' ')
        return ' '.join([self.search(word) for word in words])

```

• LeetDoocs

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, w: str) -> None:
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:

```

```

        node.children[idx] = Trie()
        node = node.children[idx]
        node.is_end = True

    def search(self, w: str) -> str:
        node = self
        for i, c in enumerate(w, 1):
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                return w

```

```

            node = node.children[idx]
            if node.is_end:
                return w[:i]
        return w

```

```

class Solution:
    def replaceWords(self, dictionary: List[str], sentence: str) -> str:
        trie = Trie()
        for w in dictionary:
            trie.insert(w)
        return " ".join(trie.search(w) for w in sentence.split())

```

• SimplyLeet

```

# Define a Trie Node class
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

# Define a Trie class to insert roots and search for shortest prefix
class Trie:
    def __init__(self):
        self.root = TrieNode()

```

```
# Insert a word into the Trie
def insert(self, word: str) -> None:
    node = self.root
    for char in word:
        if char not in node.children:
            node.children[char] = TrieNode()
        node = node.children[char]
    node.is_end = True

# Search for the shortest prefix of the word that is in the Trie
def search_shortest_prefix(self, word: str) -> str:
    node = self.root
    prefix = ""
    for char in word:
        if char not in node.children:
            break
        node = node.children[char]
        prefix += char
        if node.is_end:
            return prefix
    return word

# Main function to replace words in the sentence
def replaceWords(dictionary: list, sentence: str) -> str:
    trie = Trie()
    # Insert all roots from dictionary into the Trie
    for root in dictionary:
        trie.insert(root)

    words = sentence.split()
    result = []
    # Process each word in the sentence
    for word in words:
        result.append(trie.search_shortest_prefix(word))
    return " ".join(result)

# Example usage:
# dictionary = ["cat", "bat", "rat"]
# sentence = "the cattle was rattled by the battery"
```

```
# print(replaceWords(dictionary, sentence))
```

◆ Quick Review

Key Insights

- A Trie (prefix tree) is an ideal data structure to quickly find the shortest prefix match.
- Insert all dictionary words into the Trie.
- For each word in the sentence, traverse the Trie character by character until a complete root is found or no further matching branch exists.
- Using a space split for the sentence enables individual word processing.

Space & Time

Time Complexity: $O(m + n * l)$ where m is the total number of characters in all dictionary roots, n is the number of words in the sentence, and l is the average length of a word.

Space Complexity: $O(m)$ for storing the Trie.

Solution

We insert each root from the dictionary into a Trie. For each word in the sentence, we iterate character by character checking for a matching

prefix. Once we find a node that marks the end of a dictionary word (a valid root), we replace the current word with that root. If no such prefix exists for a word, we leave it unchanged. This approach optimally leverages the Trie to quickly determine prefix matches and minimizes unnecessary comparisons.

Tries

□ #1233 Remove Sub-Folders from the Filesystem

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Depth-First Search · Trie
Lists: AlgoMaster 600

Problem

Given a list of folders `folder`, return the folders after removing all sub-folders in those folders. You may return the answer in any order.

If a folder `folder[i]` is located within another folder `folder[j]`, it is called a sub-folder of it. A sub-folder of `folder[j]` must start with `folder[j]`, followed by a `"/`. For example, `"/a/b"` is a sub-folder of `"/a"`, but `"/b"` is not a sub-folder of `"/a/b/c"`.

The format of a path is one or more concatenated strings of the form: `'/'` followed by one or more lowercase English letters.

- For example, `"/leetcode"` and `"/leetcode/problems"` are valid paths while an empty string and `"/"` are not.

Example 1:

Input: folder = ["/a", "/a/b", "/c/d", "/c/d/e", "/c/f"]
Output: ["/a", "/c/d", "/c/f"]
Explanation: Folders "/a/b" is a subfolder of "/a" and "/c/d/e" is inside of folder "/c/d" in our filesystem.

Example 2:

Input: folder = ["/a", "/a/b/c", "/a/b/d"]
Output: ["/a"]
Explanation: Folders "/a/b/c" and "/a/b/d" will be removed because they are subfolders of "/a".

Example 3:

Input: folder = ["/a/b/c", "/a/b/ca", "/a/b/d"]
Output: ["/a/b/c", "/a/b/ca", "/a/b/d"]

Constraints:

- $1 \leq \text{folder.length} \leq 4 * 10^4$
- $2 \leq \text{folder}[i].\text{length} \leq 100$
- `folder[i]` contains only lowercase letters and '/'.
• `folder[i]` always starts with the character '/'.
• Each folder name is unique.

Community Solutions (Python)

- WalkCC

```
class Solution:
    def removeSubfolders(self, folder: list[str]) -> list[str]:
        ans = []
        prev = ""

        folder.sort()

        for f in folder:
            if len(prev) > 0 and f.startswith(prev) and f[len(prev)] == '/':
                continue

            ans.append(f)
            prev = f

        return ans
```

• LeetDoocs

```
class Solution:
    def removeSubfolders(self, folder: List[str]) -> List[str]:
        folder.sort()
        ans = [folder[0]]
        for f in folder[1:]:
            m, n = len(ans[-1]), len(f)
            if m >= n or not (ans[-1] == f[:m] and f[m] == '/'):
                ans.append(f)
        return ans
```

```
class Trie:
    def __init__(self):
        self.children = {}
        self.fid = -1

    def insert(self, i, f):
        node = self
        ps = f.split('/')
        for p in ps[1:]:
            if p not in node.children:
```



```

        node.children[p] = Trie()
        node = node.children[p]
        node.fid = i

    def search(self):
        def dfs(root):
            if root.fid != -1:
                ans.append(root.fid)
                return
            for child in root.children.values():
                dfs(child)

        ans = []
        dfs(self)
        return ans

class Solution:
    def removeSubfolders(self, folder: List[str]) -> List[str]:
        trie = Trie()

        for i, f in enumerate(folder):
            trie.insert(i, f)
        return [folder[i] for i in trie.search()]

```

• SimplyLeet

```

# Python solution with line-by-line comments
class Solution:
    def removeSubfolders(self, folder: List[str]) -> List[str]:
        # Sort folders lexicographically so parent folders come before
        # sub-folders
        folder.sort()
        # This will store the final result with no sub-folders
        result = []
        for f in folder:
            # If result is empty or f is not a sub-folder of the last folder in
            # result,
            # then add f to the result list.

```

```
if not result or not f.startswith(result[-1] + '/'):
    result.append(f)
return result
```

◆ Quick Review

Key Insights

- Sorting the folders lexicographically guarantees that any parent folder comes before its sub-folders.
- While iterating through the sorted list, only add a folder to the result if it is not a sub-folder of the previously added folder.
- Checking if a folder is a sub-folder is done by confirming whether the folder string starts with the previous folder string plus a '/'.

Space & Time

Time Complexity: $O(n \log n * L)$ where n is the number of folders and L is the average length of the folder strings (sorting cost and prefix checking).

Space Complexity: $O(n)$ for storing the result list.

Solution

The approach starts by sorting the folder list lexicographically so that parent folders come before any of their sub-folders. Then, iterate through each folder in the sorted list. For each folder, check if it is a sub-folder of the last folder added to the result. This is done by verifying that the folder does not start with the previous folder path followed by a '/'. If it does not, add the folder to the result. This method avoids unnecessary comparisons and efficiently filters out sub-folders.

Tries

□ #1268 Search Suggestions System

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Binary Search · Trie · Sorting ·
Heap (Priority Queue)
Lists: AlgoMaster 600

Problem

You are given an array of strings `products` and a string `searchWord`.

Design a system that suggests at most three product names from `products` after each character of `searchWord` is typed. Suggested products should have common prefix with `searchWord`. If there are more than three products with a common prefix return the three lexicographically minimums products.

Return a list of lists of the suggested products after each character of `searchWord` is typed.

Example 1:

```
Input: products = ["mobile","mouse","moneypot","monitor","mousepad"],
searchWord = "mouse"
Output: [["mobile","moneypot","monitor"],["mobile","moneypot","monitor"],["mouse","mousepad"],["mouse","mousepad"],["mouse","mousepad"]]
Explanation: products sorted lexicographically =
["mobile","moneypot","monitor","mouse","mousepad"].
After typing m and mo all products match and we show user
["mobile","moneypot","monitor"].
After typing mou, mous and mouse the system suggests ["mouse","mousepad"].
```

Example 2:

```
Input: products = ["havana"], searchWord = "havana"
Output: [["havana"],["havana"],["havana"],["havana"],["havana"],["havana"]]
Explanation: The only word "havana" will be always suggested while typing the search word.
```

Constraints:

- $1 \leq \text{products.length} \leq 1000$
- $1 \leq \text{products}[i].\text{length} \leq 3000$
- $1 \leq \text{sum}(\text{products}[i].\text{length}) \leq 2 * 10^4$
- All the strings of products are unique.
- $\text{products}[i]$ consists of lowercase English letters.
- $1 \leq \text{searchWord.length} \leq 1000$
- searchWord consists of lowercase English letters.

Community Solutions (Python)

- WalkCC

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.word: str | None = None

class Solution:
    def suggestedProducts(
        self,
        products: list[str],

        searchWord: str
    ) -> list[list[str]]:
        ans = []
        root = TrieNode()

        def insert(word: str) -> None:
            node = root
            for c in word:
                node = node.children.setdefault(c, TrieNode())
            node.word = word

        def search(node: TrieNode | None) -> list[str]:
            res: list[str] = []
            dfs(node, res)
            return res

        def dfs(node: TrieNode | None, res: list[str]) -> None:
            if len(res) == 3:
                return
            if not node:
                return
            if node.word:
                res.append(node.word)
            for c in string.ascii_lowercase:
                if c in node.children:
                    dfs(node.children[c], res)

        for product in products:
            insert(product)
```

```

node = root

for c in searchWord:
    if not node or c not in node.children:
        node = None
        ans.append([])
        continue
    node = node.children[c]
    ans.append(search(node))

```

```

return ans

```

• LeetDoocs

```

class Trie:
    def __init__(self):
        self.children: List[Union[Trie, None]] = [None] * 26
        self.v: List[int] = []

    def insert(self, w, i):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
            if len(node.v) < 3:
                node.v.append(i)

    def search(self, w):
        node = self
        ans = [[] for _ in range(len(w))]
        for i, c in enumerate(w):
            idx = ord(c) - ord('a')

```

```

        if node.children[idx] is None:
            break
        node = node.children[idx]
        ans[i] = node.v
    return ans

class Solution:
    def suggestedProducts(
        self, products: List[str], searchWord: str

    ) -> List[List[str]]:
        products.sort()
        trie = Trie()
        for i, w in enumerate(products):
            trie.insert(w, i)
        return [[products[i] for i in v] for v in trie.search(searchWord)]

```

• SimplyLeet

```

# Python solution for Search Suggestions System

import bisect

def suggestedProducts(products, searchWord):
    # Sort products lexicographically
    products.sort()
    result = []
    prefix = ""

    # For each character in searchWord, update the prefix
    for char in searchWord:
        prefix += char
        # Find the start index of matching products using binary search
        start = bisect.bisect_left(products, prefix)
        suggestions = []
        # Check next three products starting from 'start'
        for i in range(start, min(start + 3, len(products))):
            if products[i].startswith(prefix):
                suggestions.append(products[i])

```



```
        else:
            break
        result.append(suggestions)
    return result

# Example usage:
# products = ["mobile", "mouse", "moneypot", "monitor", "mousepad"]
# searchWord = "mouse"
# print(suggestedProducts(products, searchWord))
```

◆ Quick Review

Key Insights

- Sorting the products lexicographically simplifies finding candidates.
- Using binary search helps efficiently locate the starting index for each prefix.
- Limiting suggestions to at most three candidates for every prefix ensures optimal performance in both time and space.
- Alternative methods include Trie-based search which can be efficient for prefix queries, but binary search after sorting is straightforward given the problem constraints.

Space & Time

Time Complexity: $O(n \log n + m \log n)$ where n is the number of products and m is the length of `searchWord`. Sorting takes $O(n \log n)$ and for

each of the m prefixes, binary search takes $O(\log n)$.

Space Complexity: $O(m + k)$ where m is the output size (list of lists for each prefix) and k is the additional space used by the sorting algorithm (which can be $O(n)$ in worst-case depending on implementation).

Solution

The approach starts by sorting the input products lexicographically. For each prefix of `searchWord`, use binary search to quickly find the first product that could match the current prefix. Then, iterate through at most three products from that position and, if they share the prefix, add them to the suggestions for that stage. This process is repeated for every prefix in `searchWord`. This method leverages efficient sorting and binary search to meet the problem's requirements while ensuring the suggestions are lexicographically minimal.

Tries

□ #2452 Words Within Two Edits of Dictionary

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Trie
Lists: AlgoMaster 600

Problem

You are given two string arrays, queries and dictionary. All words in each array comprise of lowercase English letters and have the same length.

In one edit you can take a word from queries, and change any letter in it to any other letter. Find all words from queries that, after a maximum of two edits, equal some word from dictionary.

Return a list of all words from queries, that match with some word from dictionary after a maximum of two edits. Return the words in the same order they appear in queries.

Example 1:

```

Input: queries = ["word","note","ants","wood"], dictionary =
["wood","joke","moat"]
Output: ["word","note","wood"]
Explanation:
- Changing the 'r' in "word" to 'o' allows it to equal the dictionary word
  "wood".
- Changing the 'n' to 'j' and the 't' to 'k' in "note" changes it to "joke".
- It would take more than 2 edits for "ants" to equal a dictionary word.
- "wood" can remain unchanged (0 edits) and match the corresponding dictionary
  word.
Thus, we return ["word","note","wood"].

```

Example 2:

```

Input: queries = ["yes"], dictionary = ["not"]
Output: []
Explanation:
Applying any two edits to "yes" cannot make it equal to "not". Thus, we return
an empty array.

```

Constraints:

- $1 \leq \text{queries.length}, \text{dictionary.length} \leq 100$
- $n == \text{queries}[i].\text{length} == \text{dictionary}[j].\text{length}$
- $1 \leq n \leq 100$
- All $\text{queries}[i]$ and $\text{dictionary}[j]$ are composed of lowercase English letters.

Community Solutions (Python)

- WalkCC

```
class Solution:
    def twoEditWords(
        self,
        queries: list[str],
        dictionary: list[str],
    ) -> list[str]:
        return [query for query in queries
                if any(sum(a != b for a, b in zip(query, word)) < 3
                        for word in dictionary)]
```

• LeetDoocs

```
class Solution:
    def twoEditWords(self, queries: List[str], dictionary: List[str]) -> List[
        str]:
        ans = []
        for s in queries:
            for t in dictionary:
                if sum(a != b for a, b in zip(s, t)) < 3:
                    ans.append(s)
                    break
        return ans
```

• SimplyLeet

```
def two_edits_away(queries, dictionary):
    valid_words = [] # List to hold valid query words
    # Iterate through every query word
    for word in queries:
        # Compare query word with each dictionary word
        for dict_word in dictionary:
            # Count number of positions with different characters
            differences = sum(1 for q_char, d_char in zip(word, dict_word) if
q_char != d_char)
            # If edit count is within two, mark word as valid
            if differences <= 2:
```

```
        valid_words.append(word)
        break # Stop checking other dictionary words for this query
    return valid_words

# Example usage:
queries = ["word", "note", "ants", "wood"]
dictionary = ["wood", "joke", "moat"]
print(two_edits_away(queries, dictionary)) # Output: ["word", "note", "wood"]
```

◆ Quick Review

Key Insights

- For each query word, check every dictionary word to count the number of differing positions.
- A query word is valid if there exists at least one dictionary word with at most two differing characters.
- The problem can be solved using a brute-force approach since the maximum size of queries and dictionary, along with word length, is small.
- Early stopping during letter comparisons can improve performance by breaking out as soon as the edit count exceeds two.

Space & Time

Time Complexity: $O(q * d * n)$, where q is the number of queries, d is the number of

dictionary words, and n is the length of each word.

Space Complexity: $O(1)$ additional space (excluding the space for the output list).

Solution

The approach involves iterating through each word in the queries array. For each query word, we iterate through every word in the dictionary and compare them character by character to count the differences. If the number of differences is two or fewer for any dictionary word, the query word is added to the result list. By using early termination in the character comparison loop, the algorithm improves efficiency when a word exceeds the allowed edit threshold. This brute-force method is acceptable given the constraints.

Tries

□ #3043 Find the Length of the Longest Common Prefix

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Trie
Lists: AlgoMaster 600

Problem

You are given two arrays with positive integers `arr1` and `arr2`.

A prefix of a positive integer is an integer formed by one or more of its digits, starting from its leftmost digit. For example, 123 is a prefix of the integer 12345, while 234 is not.

A common prefix of two integers `a` and `b` is an integer `c`, such that `c` is a prefix of both `a` and `b`. For example, 5655359 and 56554 have common prefixes 565 and 5655 while 1223 and 43456 do not have a common prefix.

You need to find the length of the longest common prefix between all pairs of integers (`x`, `y`) such that `x` belongs to `arr1` and `y` belongs to `arr2`.

Return the length of the longest common prefix among all pairs. If no common prefix exists among them, return 0.

Example 1:

Input: arr1 = [1,10,100], arr2 = [1000]
 Output: 3
 Explanation: There are 3 pairs (arr1[i], arr2[j]):
 - The longest common prefix of (1, 1000) is 1.
 - The longest common prefix of (10, 1000) is 10.
 - The longest common prefix of (100, 1000) is 100.
 The longest common prefix is 100 with a length of 3.

Example 2:

Input: arr1 = [1,2,3], arr2 = [4,4,4]
 Output: 0
 Explanation: There exists no common prefix for any pair (arr1[i], arr2[j]), hence we return 0.
 Note that common prefixes between elements of the same array do not count.

Constraints:

- $1 \leq \text{arr1.length}, \text{arr2.length} \leq 5 * 10^4$
- $1 \leq \text{arr1}[i], \text{arr2}[i] \leq 10^8$

Community Solutions (Python)

• WalkCC

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
```

```

node: TrieNode = self.root
for c in word:
    node = node.children.setdefault(c, TrieNode())
node.isWord = True

def search(self, word: str) -> int:
    prefixLength = 0
    node = self.root
    for c in word:
        if c not in node.children:
            break
        node = node.children[c]
        prefixLength += 1
    return prefixLength

class Solution:
    def longestCommonPrefix(self, arr1: list[int], arr2: list[int]) -> int:
        trie = Trie()

        for num in arr1:
            trie.insert(str(num))

        return max(trie.search(str(num)) for num in arr2)

```

• LeetDoocs

```

class Solution:
    def longestCommonPrefix(self, arr1: List[int], arr2: List[int]) -> int:
        s = set()
        for x in arr1:
            while x:
                s.add(x)
                x //= 10
        mx = 0
        for x in arr2:
            while x:

```

```

        if x in s:
            mx = max(mx, x)
            break
        x //= 10
    return len(str(mx)) if mx else 0

```

• SimplyLeet

Python solution with line-by-line comments

```

def longest_common_prefix_length(arr1, arr2):
    # Helper function to generate all prefixes of a given number string
    def get_prefixes(num_str):
        prefixes = set()
        for i in range(1, len(num_str) + 1):
            prefixes.add(num_str[:i])
        return prefixes

    # Build a set of prefixes from arr1
    prefixes1 = set()
    for num in arr1:
        s = str(num) # Convert the number to a string
        prefixes1.update(get_prefixes(s)) # Add all prefixes of the number

    # Build a set of prefixes from arr2
    prefixes2 = set()
    for num in arr2:
        s = str(num) # Convert the number to a string

        prefixes2.update(get_prefixes(s)) # Add all prefixes of the number

    # Find common prefixes between arr1 and arr2
    common_prefixes = prefixes1.intersection(prefixes2)

    # Determine the longest common prefix length
    max_length = 0
    for prefix in common_prefixes:
        max_length = max(max_length, len(prefix))

```

```
    return max_length

# Example usage:
print(longest_common_prefix_length([1,10,100], [1000])) # Expected output: 3
print(longest_common_prefix_length([1,2,3], [4,4,4]))    # Expected output:
0
```

◆ Quick Review

Key Insights

- Convert each integer into its string representation to easily extract and compare prefixes.
- For any integer, you can derive all possible prefixes (e.g., "12345" gives "1", "12", "123", "1234", "12345").
- Instead of comparing every pair (which would be inefficient), generate a set of all prefixes for arr1 and another set for arr2, then find the intersection of these sets.
- The result is the maximum length among all common prefixes.

Space & Time

Time Complexity: $O((n + m) * d)$, where n and m are the lengths of arr1 and arr2 respectively, and d is the maximum number of digits (at most 9).

Space Complexity: $O((n + m) * d)$ for storing all prefixes in sets.

Solution

The approach involves converting the numbers in each array into strings and generating all possible prefixes for each number. We then store these prefixes in two sets (one for each array) and compute the intersection to obtain the common prefixes. The length of the longest prefix in this intersection is the answer. This method avoids checking every pair individually and leverages set operations for efficiency.

Heaps

Round 5 · Mid · Medium · 4 questions

Heaps

□ #1405 Longest Happy String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Greedy · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

A string s is called happy if it satisfies the following conditions:

- s only contains the letters 'a', 'b', and 'c'.
- s does not contain any of "aaa", "bbb", or "ccc" as a substring.
- s contains at most a occurrences of the letter 'a'.
- s contains at most b occurrences of the letter 'b'.
- s contains at most c occurrences of the letter 'c'.

Given three integers a , b , and c , return the longest possible happy string. If there are multiple longest happy strings, return any of them. If there is no such string, return the empty

string "".

A substring is a contiguous sequence of characters within a string.

Example 1:

Input: a = 1, b = 1, c = 7

Output: "ccaccbcc"

Explanation: "ccbccacc" would also be a correct answer.

Example 2:

Input: a = 7, b = 1, c = 0

Output: "aabaa"

Explanation: It is the only correct answer in this case.

Constraints:

- $0 \leq a, b, c \leq 100$
- $a + b + c > 0$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def longestDiverseString(self, a: int, b: int, c: int) -> str:
        h = []
        if a > 0:
            heappush(h, [-a, 'a'])
        if b > 0:
            heappush(h, [-b, 'b'])
        if c > 0:
            heappush(h, [-c, 'c'])
```



```

ans = []
while len(h) > 0:
    cur = heappop(h)
    if len(ans) >= 2 and ans[-1] == cur[1] and ans[-2] == cur[1]:
        if len(h) == 0:
            break
        nxt = heappop(h)
        ans.append(nxt[1])
        if -nxt[0] > 1:
            nxt[0] += 1

        heappush(h, nxt)
        heappush(h, cur)
    else:
        ans.append(cur[1])
        if -cur[0] > 1:
            cur[0] += 1
            heappush(h, cur)

return ''.join(ans)

```

• SimplyLeet

```

import heapq

def longestDiverseString(a: int, b: int, c: int) -> str:
    # Create a max heap using negative counts (heapq is a min heap by default)
    max_heap = []
    for count, char in [(-a, 'a'), (-b, 'b'), (-c, 'c')]:
        if count < 0:
            heapq.heappush(max_heap, (count, char))

    result = []

```

```

while max_heap:
    # Get the character with the highest remaining count
    count, char = heapq.heappop(max_heap)
    # Check if appending this character causes three consecutive same
    characters
    if len(result) >= 2 and result[-1] == result[-2] == char:
        # If no alternative, break out of the loop
        if not max_heap:
            break
        # Use the next most frequent character instead

    count2, char2 = heapq.heappop(max_heap)
    result.append(char2)
    count2 += 1 # Decrease the count (since counts are negative)
    if count2 < 0:
        heapq.heappush(max_heap, (count2, char2))
    # Push the original character back for future consideration
    heapq.heappush(max_heap, (count, char))
else:
    result.append(char)
    count += 1 # Decrement count (count is negative so adding 1
    increases it)

    if count < 0:
        heapq.heappush(max_heap, (count, char))

return "".join(result)

# Example usage:
print(longestDiverseString(1, 1, 7)) # could output "ccaccbcc" or another
valid answer

```

◆ Quick Review

Key Insights

- Use a greedy strategy to always pick the letter with the highest remaining count unless it would result in three consecutive identical

letters.

- Use a max-heap (or priority queue) to efficiently retrieve the character with the largest remaining frequency.
- If the top element would cause an invalid sequence, select the next best option.
- Continue this process until no valid moves remain.

Space & Time

Time Complexity: $O((a+b+c) * \log 3) \approx O(a+b+c)$ since the heap contains at most 3 elements.

Space Complexity: $O(1)$ additional space, aside from the output string.

Solution

The solution utilizes a greedy algorithm combined with a max-heap (priority queue) to always choose the letter with the highest count. At each step, we check if appending the selected letter causes three consecutive occurrences. If it does, we attempt to use the next most frequent letter instead. This ensures that the longest possible string is constructed without violating the happy string constraints.

Heaps

□ #1642 Furthest Building You Can Reach

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Heap (Priority Queue)
Lists: AlgoMaster 600

Problem

You are given an integer array `heights` representing the heights of buildings, some bricks, and some ladders.

You start your journey from building 0 and move to the next building by possibly using bricks or ladders.

While moving from building i to building $i+1$ (0-indexed),

- If the current building's height is greater than or equal to the next building's height, you do not need a ladder or bricks.
- If the current building's height is less than the next building's height, you can either use one ladder or $(h[i+1] - h[i])$ bricks.

Return the furthest building index (0-indexed) you can reach if you use the given ladders and bricks optimally.

Example 1:



Input: heights = [4,2,7,6,9,14,12], bricks = 5, ladders = 1
Output: 4
Explanation: Starting at building 0, you can follow these steps:
- Go to building 1 without using ladders nor bricks since $4 \geq 2$.
- Go to building 2 using 5 bricks. You must use either bricks or ladders because $2 < 7$.
- Go to building 3 without using ladders nor bricks since $7 \geq 6$.
- Go to building 4 using your only ladder. You must use either bricks or ladders because $6 < 9$.
It is impossible to go beyond building 4 because you do not have any more bricks or ladders.

Example 2:

Input: heights = [4,12,2,7,3,18,20,3,19], bricks = 10, ladders = 2
Output: 7

Example 3:

Input: heights = [14,3,19,3], bricks = 17, ladders = 0
Output: 3

Constraints:

- $1 \leq \text{heights.length} \leq 105$
- $1 \leq \text{heights}[i] \leq 106$
- $0 \leq \text{bricks} \leq 109$
- $0 \leq \text{ladders} \leq \text{heights.length}$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def furthestBuilding(
        self,
        heights: list[int],
        bricks: int,
        ladders: int,
    ) -> int:
        minHeap = []

        for i, (a, b) in enumerate(itertools.pairwise(heights)):

            diff = b - a
            if diff <= 0:
                continue
            heapq.heappush(minHeap, diff)
            # If we run out of ladders, greedily use as less bricks as possible.
            if len(minHeap) > ladders:
                bricks -= heapq.heappop(minHeap)
            if bricks < 0:
                return i

        return len(heights) - 1

```

• LeetDoocs

```

class Solution:
    def furthestBuilding(self, heights: List[int], bricks: int, ladders: int)
    -> int:
        h = []
        for i, a in enumerate(heights[:-1]):
            b = heights[i + 1]
            d = b - a
            if d > 0:
                heappush(h, d)
                if len(h) > ladders:
                    bricks -= heappop(h)

            if bricks < 0:
                return i
        return len(heights) - 1

```

• SimplyLeet

```
import heapq

def furthestBuilding(heights, bricks, ladders):
    # Min-heap to store the positive height differences where resources are
    # used
    min_heap = []

    # Loop through each building
    for i in range(len(heights) - 1):
        climb = heights[i+1] - heights[i]
        # Only consider climbs that require additional resources

        if climb > 0:
            heapq.heappush(min_heap, climb)

        # If we have more climbs than ladders, use bricks for the smallest
        # climb
        if len(min_heap) > ladders:
            bricks -= heapq.heappop(min_heap)

        # If bricks run out, return the current building index
        if bricks < 0:
            return i

    # If we overcame all challenges, return the last building index
    return len(heights) - 1

# Example usage:
#print(furthestBuilding([4,2,7,6,9,14,12], 5, 1))
```

◆ Quick Review

Key Insights

- Use a greedy approach to prioritize ladders for the largest jumps.
- Use bricks for smaller height differences.

- Maintain a min-heap (priority queue) to track the height differences where you chose to use bricks initially.
- When the number of climbs exceeds the available ladders, the smallest jump (from the heap) is replaced by using bricks.
- The simulation stops when the bricks run out before you can cover the required jump.

Space & Time

Time Complexity: $O(n \log n)$ in the worst-case scenario due to the heap operations, where n is the number of buildings.

Space Complexity: $O(n)$ for the worst-case heap storage.

Solution

We solve the problem by simulating the journey from the first building to the last. For each jump, if the next building is taller than the current one, we calculate the height difference. We push this difference into a min-heap. If the number of required climbs (i.e., the heap's size) exceeds the available ladders, we remove the smallest jump from the heap and use bricks to cover it instead. If at

any point the bricks become insufficient (i.e., bricks drop below zero), we cannot progress further and return the current building index. This approach optimally allocates ladders to the largest jumps while using bricks for the smaller ones.

Heaps

□ #1834 Single-Threaded CPU

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting · Heap (Priority Queue)
Lists: AlgoMaster 600

Problem

You are given n tasks labeled from 0 to $n - 1$ represented by a 2D integer array `tasks`, where `tasks[i] = [enqueueTimei, processingTimei]` means that the i th task will be available to process at `enqueueTimei` and will take `processingTimei` to finish processing.

You have a single-threaded CPU that can process at most one task at a time and will act in the following way:

- If the CPU is idle and there are no available tasks to process, the CPU remains idle.
- If the CPU is idle and there are available tasks, the CPU will choose the one with the shortest processing time. If multiple tasks have the same shortest processing time, it will choose the task with the smallest index.

- Once a task is started, the CPU will process the entire task without stopping.
- The CPU can finish a task then start a new one instantly.

Return the order in which the CPU will process the tasks.

Example 1:

Input: tasks = [[1,2],[2,4],[3,2],[4,1]]

Output: [0,2,3,1]

Explanation: The events go as follows:

- At time = 1, task 0 is available to process. Available tasks = {0}.
- Also at time = 1, the idle CPU starts processing task 0. Available tasks = {}.
- At time = 2, task 1 is available to process. Available tasks = {1}.
- At time = 3, task 2 is available to process. Available tasks = {1, 2}.
- Also at time = 3, the CPU finishes task 0 and starts processing task 2 as it is the shortest. Available tasks = {1}.

Example 2:

Input: tasks = [[7,10],[7,12],[7,5],[7,4],[7,2]]

Output: [4,3,2,0,1]

Explanation: The events go as follows:

- At time = 7, all the tasks become available. Available tasks = {0,1,2,3,4}.
- Also at time = 7, the idle CPU starts processing task 4. Available tasks = {0,1,2,3}.
- At time = 9, the CPU finishes task 4 and starts processing task 3. Available tasks = {0,1,2}.
- At time = 13, the CPU finishes task 3 and starts processing task 2. Available tasks = {0,1}.
- At time = 18, the CPU finishes task 2 and starts processing task 0. Available tasks = {1}.

Constraints:

- tasks.length == n
- 1 <= n <= 105

- $1 \leq \text{enqueueTime}_i$, $\text{processingTime}_i \leq 10^9$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def getOrder(self, tasks: list[list[int]]) -> list[int]:
        n = len(tasks)
        A = [[*task, i] for i, task in enumerate(tasks)]
        ans = []
        minHeap = []
        i = 0 # tasks' index
        time = 0 # the current time

        A.sort()

        while i < n or minHeap:
            if not minHeap:
                time = max(time, A[i][0])
            while i < n and time >= A[i][0]:
                heapq.heappush(minHeap, (A[i][1], A[i][2]))
                i += 1
            procTime, index = heapq.heappop(minHeap)
            time += procTime
            ans.append(index)

        return ans
```

• LeetDoocs

```

class Solution:
    def getOrder(self, tasks: List[List[int]]) -> List[int]:
        for i, task in enumerate(tasks):
            task.append(i)
        tasks.sort()
        ans = []
        q = []
        n = len(tasks)
        i = t = 0
        while q or i < n:
            if not q:
                t = max(t, tasks[i][0])
            while i < n and tasks[i][0] <= t:
                heappush(q, (tasks[i][1], tasks[i][2]))
                i += 1
            pt, j = heappop(q)
            ans.append(j)
            t += pt
        return ans

```

• SimplyLeet

```

import heapq

class Solution:
    def getOrder(self, tasks):
        # Augment each task with its original index: (enqueueTime,
        # processingTime, index)
        indexed_tasks = [(task[0], task[1], i) for i, task in enumerate(tasks)]

        # Sort tasks by enqueueTime
        indexed_tasks.sort(key=lambda x: x[0])

        result = [] # This list will store the order of task indices

```

```

min_heap = [] # Heap to select the next task by processing time and
index
time = 0 # Current time
i = 0 # Pointer to tasks in indexed_tasks list
n = len(tasks)

while i < n or min_heap:
    # If no available tasks, jump time to the next task's enqueueTime
    if not min_heap and time < indexed_tasks[i][0]:
        time = indexed_tasks[i][0]
    # Add all tasks that have become available by current time to the
    heap

    while i < n and indexed_tasks[i][0] <= time:
        enqueue_time, processing_time, index = indexed_tasks[i]
        heapq.heappush(min_heap, (processing_time, index))
        i += 1
    # Process the task at the top of the heap (shortest processing time
    then smallest index)
    proc_time, index = heapq.heappop(min_heap)
    time += proc_time # Update the current time by the processing time
    of the task
    result.append(index)
return result

# Example usage:
# sol = Solution()
# print(sol.getOrder([[1,2],[2,4],[3,2],[4,1]]))

```

◆ Quick Review

Key Insights

- Sort tasks by enqueueTime while preserving original indices.
- Use a min-heap (priority queue) to efficiently choose the task with the shortest

processing time.

- If the heap is empty and tasks are yet to arrive, jump the current time to the next task's enqueueTime.
- Process tasks until all tasks have been scheduled.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting and heap operations.

Space Complexity: $O(n)$ for storing tasks and the heap.

Solution

We first augment each task with its original index and sort the tasks based on their enqueue times. We then use a min-heap to keep track of tasks that are available. At each step, we:

- Add all tasks available at the current time to the heap.
- If the heap is not empty, pop the task with the smallest processing time (using the original index as a tie breaker) and update the current time by its processing time.

– If the heap is empty, advance the current time to the next task's enqueue time. This process ensures that the CPU always processes the appropriate task and the order is maintained correctly.

Heaps

□ #1882 Process Tasks Using Servers

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given two 0-indexed integer arrays `servers` and `tasks` of lengths `n` and `m` respectively. `servers[i]` is the weight of the `i`th server, and `tasks[j]` is the time needed to process the `j`th task in seconds.

Tasks are assigned to the servers using a task queue. Initially, all servers are free, and the queue is empty.

At second `j`, the `j`th task is inserted into the queue (starting with the 0th task being inserted at second 0). As long as there are free servers and the queue is not empty, the task in the front of the queue will be assigned to a free server with the smallest weight, and in case of a tie, it is assigned to a free server with the smallest index.

If there are no free servers and the queue is not empty, we wait until a server becomes free and immediately assign the next task. If multiple servers become free at the same time, then multiple tasks from the queue will be assigned in order of insertion following the weight and index priorities above.

A server that is assigned task j at second t will be free again at second $t + \text{tasks}[j]$.

Build an array `ans` of length m , where `ans[j]` is the index of the server the j th task will be assigned to.

Return the array `ans`.

Example 1:

Input: `servers = [3,3,2]`, `tasks = [1,2,3,2,1,2]`

Output: `[2,2,0,2,1,2]`

Explanation: Events in chronological order go as follows:

- At second 0, task 0 is added and processed using server 2 until second 1.
- At second 1, server 2 becomes free. Task 1 is added and processed using server 2 until second 3.
- At second 2, task 2 is added and processed using server 0 until second 5.
- At second 3, server 2 becomes free. Task 3 is added and processed using server 2 until second 5.
- At second 4, task 4 is added and processed using server 1 until second 5.

Example 2:

Input: servers = [5,1,4,3,2], tasks = [2,1,2,4,5,2,1]

Output: [1,4,1,4,1,3,2]

Explanation: Events in chronological order go as follows:

- At second 0, task 0 is added and processed using server 1 until second 2.
- At second 1, task 1 is added and processed using server 4 until second 2.
- At second 2, servers 1 and 4 become free. Task 2 is added and processed using server 1 until second 4.
- At second 3, task 3 is added and processed using server 4 until second 7.
- At second 4, server 1 becomes free. Task 4 is added and processed using server 1 until second 9.

Constraints:

- `servers.length == n`
- `tasks.length == m`
- $1 \leq n, m \leq 2 * 10^5$
- $1 \leq \text{servers}[i], \text{tasks}[j] \leq 2 * 10^5$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def assignTasks(self, servers: list[int], tasks: list[int]) -> list[int]:
        ans = []
        free = [] # (weight, index, freeTime)
        used = [] # (freeTime, weight, index)

        for i, weight in enumerate(servers):
            heapq.heappush(free, (weight, i, 0))

        for i, executionTime in enumerate(tasks): # i := the current time
```

```

# Poll all servers that'll be free at time i.
while used and used[0][0] <= i:
    curr = heapq.heappop(used)
    heapq.heappush(free, (curr[1], curr[2], curr[0]))
if free:
    curr = heapq.heappop(free)
    ans.append(curr[1])
    heapq.heappush(used, (i + executionTime, curr[0], curr[1]))
else:
    curr = heapq.heappop(used)

ans.append(curr[2])
heapq.heappush(used, (curr[0] + executionTime, curr[1], curr[2]))

return ans

```

• LeetDoocs

```

class Solution:
    def assignTasks(self, servers: List[int], tasks: List[int]) -> List[int]:
        idle = [(x, i) for i, x in enumerate(servers)]
        heapify(idle)
        busy = []
        ans = []
        for j, t in enumerate(tasks):
            while busy and busy[0][0] <= j:
                _, s, i = heappop(busy)
                heappush(idle, (s, i))

            if idle:
                s, i = heappop(idle)
                heappush(busy, (j + t, s, i))
            else:
                w, s, i = heappop(busy)
                heappush(busy, (w + t, s, i))
            ans.append(i)
        return ans

```

• SimplyLeet

```

# Python solution using heaps from the heapq module
import heapq

def processTasks(servers, tasks):
    # free_servers heap: (server_weight, server_index)
    free_servers = [(weight, idx) for idx, weight in enumerate(servers)]
    heapq.heapify(free_servers)

    # busy_servers heap: (finish_time, server_weight, server_index)
    busy_servers = []

    n = len(tasks)
    result = [0] * n # To store the server assigned to each task
    time = 0 # Current time

    # Iterate through each task (arrival time is j)
    for j, task_time in enumerate(tasks):
        time = max(time, j) # Ensure current time is at least j

        # Move servers that have completed tasks from busy_servers to
        free_servers

        while busy_servers and busy_servers[0][0] <= time:
            finish_time, weight, server_idx = heapq.heappop(busy_servers)
            heapq.heappush(free_servers, (weight, server_idx))

        # If no free servers, fast-forward time to when the next server becomes
        free

        if not free_servers:
            time = busy_servers[0][0]
            while busy_servers and busy_servers[0][0] <= time:
                finish_time, weight, server_idx = heapq.heappop(busy_servers)
                heapq.heappush(free_servers, (weight, server_idx))

```

```
# Assign the task to the free server with smallest weight (and index)
weight, server_idx = heapq.heappop(free_servers)
result[j] = server_idx

# Add the server into busy_servers with its new finish time
heapq.heappush(busy_servers, (time + task_time, weight, server_idx))

return result
```

```
# Example usage:
# servers = [3,3,2]
# tasks = [1,2,3,2,1,2]
# print(processTasks(servers, tasks)) # Output: [2,2,0,2,1,2]
```

◆ Quick Review

Key Insights

- Use a min-heap to track free servers sorted by weight and index.
- Use another min-heap to manage busy servers, prioritized by when they become free.
- Keep track of the current time, taking into account task arrival times.
- If no server is free at the time of task arrival, fast-forward time to when the next server becomes available.
- Always reassign servers from the busy heap to the free heap when they complete their tasks.

Space & Time

Time Complexity: $O(m \log n)$, where m is the number of tasks and n is the number of servers, since each task assignment may require heap operations.

Space Complexity: $O(n + m)$ primarily for the heaps and the result array.

Solution

We use two heaps:

- **Free Servers Heap:** stores tuples (server weight, server index), sorted so that the server with the smallest weight (and then smallest index) is always at the top.
- **Busy Servers Heap:** stores tuples (finish time, server weight, server index), ordered by finish time. When a server is busy processing a task, it resides in this heap. The algorithm maintains a current time variable. For each task arriving at time j , time is updated to ensure it is at least j . Servers that become free on or before the current time are moved from the busy heap to the free heap. If no free server is available, the current time jumps to the finish time of the next server to be

available. The task is then assigned to the free server at the top of the free heap, and the server's next free time is computed and added to the busy heap. This continues until all tasks are assigned.

Intervals

Round 5 · Mid · Medium · 6 questions

Intervals

□ #452 Minimum Number of Arrows to Burst Balloons

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Sorting
Lists: AlgoMaster 600

Problem

There are some spherical balloons taped onto a flat wall that represents the XY-plane. The balloons are represented as a 2D integer array `points` where `points[i] = [xstart, xend]` denotes a balloon whose horizontal diameter stretches between `xstart` and `xend`. You do not know the exact `y`-coordinates of the balloons.

Arrows can be shot up directly vertically (in the positive `y`-direction) from different points along the `x`-axis. A balloon with `xstart` and `xend` is burst by an arrow shot at `x` if `xstart ≤ x ≤ xend`. There is no limit to the number of arrows that can be shot. A shot arrow keeps traveling up infinitely, bursting any balloons in its path.

Given the array points, return the minimum number of arrows that must be shot to burst all balloons.

Example 1:

Input: points = [[10,16],[2,8],[1,6],[7,12]]

Output: 2

Explanation: The balloons can be burst by 2 arrows:

- Shoot an arrow at x = 6, bursting the balloons [2,8] and [1,6].
- Shoot an arrow at x = 11, bursting the balloons [10,16] and [7,12].

Example 2:

Input: points = [[1,2],[3,4],[5,6],[7,8]]

Output: 4

Explanation: One arrow needs to be shot for each balloon for a total of 4 arrows.

Example 3:

Input: points = [[1,2],[2,3],[3,4],[4,5]]

Output: 2

Explanation: The balloons can be burst by 2 arrows:

- Shoot an arrow at x = 2, bursting the balloons [1,2] and [2,3].
- Shoot an arrow at x = 4, bursting the balloons [3,4] and [4,5].

Constraints:

- $1 \leq \text{points.length} \leq 105$
- $\text{points}[i].\text{length} == 2$
- $-231 \leq \text{xstart} < \text{xend} \leq 231 - 1$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def findMinArrowShots(self, points: list[list[int]]) -> int:
        ans = 0
        arrowX = -math.inf

        for point in sorted(points, key=lambda x: x[1]):
            if point[0] > arrowX:
                ans += 1
                arrowX = point[1]

        return ans

```

• LeetCode

```

class Solution:
    def findMinArrowShots(self, points: List[List[int]]) -> int:
        ans, last = 0, -inf
        for a, b in sorted(points, key=lambda x: x[1]):
            if a > last:
                ans += 1
                last = b
        return ans

```

• SimplyLeet

```

def findMinArrowShots(points):
    # If there are no balloons, return 0
    if not points:
        return 0

    # Sort the balloons by their end coordinate
    points.sort(key=lambda x: x[1])

    # Start with one arrow at the end of the first balloon
    arrows = 1

```

```
current_arrow_position = points[0][1]

# Traverse through each balloon interval
for x_start, x_end in points:
    # Only shoot a new arrow if the current balloon is not covered by the
    last arrow
    if x_start > current_arrow_position:
        arrows += 1 # Increment the arrow count
        current_arrow_position = x_end # Update the arrow position

return arrows

# Example usage:
print(findMinArrowShots([[10,16],[2,8],[1,6],[7,12]])) # Output: 2
```

◆ Quick Review

Key Insights

- Sort the balloon intervals by their ending coordinate.
- Start by shooting an arrow at the end of the first interval.
- Iterate through the sorted intervals and only shoot a new arrow if the next balloon's start is greater than the position where the last arrow was shot.
- This greedy approach minimizes arrows by overlapping as many balloons as possible with one arrow.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(1)$ additional space if sorting is done in-place (ignoring the input storage).

Solution

The solution uses a greedy algorithm combined with sorting. First, sort the array of balloon intervals based on their x_{end} values. Initialize the arrow count to 1 with the first arrow shot at the end of the first interval. Then, for each subsequent balloon, check if its x_{start} is greater than the x position of the last shot arrow. If so, shoot another arrow at the current balloon's end and update the arrow position. This ensures that each arrow covers the maximum number of overlapping balloons.

Intervals

□ #1094 Car Pooling

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting · Heap (Priority Queue) ·
Simulation · Prefix Sum
Lists: AlgoMaster 600

Problem

There is a car with capacity empty seats. The vehicle only drives east (i.e., it cannot turn around and drive west).

You are given the integer capacity and an array trips where $\text{trips}[i] = [\text{numPassengers}_i, \text{from}_i, \text{to}_i]$ indicates that the i th trip has numPassengers_i passengers and the locations to pick them up and drop them off are from_i and to_i respectively. The locations are given as the number of kilometers due east from the car's initial location.

Return true if it is possible to pick up and drop off all passengers for all the given trips, or false otherwise.

Example 1:

Input: trips = [[2,1,5],[3,3,7]], capacity = 4
 Output: false

Example 2:

Input: trips = [[2,1,5],[3,3,7]], capacity = 5
 Output: true

Constraints:

- $1 \leq \text{trips.length} \leq 1000$
- $\text{trips}[i].\text{length} == 3$
- $1 \leq \text{numPassengers}_i \leq 100$
- $0 \leq \text{from}_i < \text{to}_i \leq 1000$
- $1 \leq \text{capacity} \leq 105$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def carPooling(self, trips: List[List[int]], capacity: int) -> bool:
        mx = max(e[2] for e in trips)
        d = [0] * (mx + 1)
        for x, f, t in trips:
            d[f] += x
            d[t] -= x
        return all(s <= capacity for s in accumulate(d))
```

• SimplyLeet

```

# Python solution with detailed comments

def carPooling(trips, capacity):
    # events will store tuples (location, change_in_passengers)
    events = []

    # Create events for each trip: pickup event (+num_passengers) and drop-off
    # event (-num_passengers)
    for num_passengers, start, end in trips:
        events.append((start, num_passengers))    # When passengers get on
        events.append((end, -num_passengers))      # When passengers get off

    # Sort events by location,
    # and in case of ties, process drop-offs (-num_passengers) before pick-ups
    # (+num_passengers)
    events.sort(key=lambda x: (x[0], x[1]))

    current_passengers = 0

    # Process each event in order of location
    for location, change in events:
        current_passengers += change

    # If at any point, the number of passengers exceeds capacity, return
    # False
    if current_passengers > capacity:
        return False

    # If we've processed all events without exceeding capacity, return True
    return True

# Example usage:
print(carPooling([[2,1,5],[3,3,7]], 4)) # Expected output: False
print(carPooling([[2,1,5],[3,3,7]], 5)) # Expected output: True

```

◆ Quick Review

Key Insights

- Use a sweep-line algorithm by converting each trip into two events: one for picking up passengers (adding to the load) and one for dropping them off (subtracting from the load).
- Sort the events by location. For events occurring at the same location, process drop-offs before pick-ups to free up capacity.
- At each event point, update the current passenger count and verify it does not exceed the car's capacity.
- This simulation technique efficiently handles overlapping intervals with a difference array concept.

Space & Time

Time Complexity: $O(N \log N)$ due to sorting the events, where N is the number of trips.

Space Complexity: $O(N)$ for storing the events.

Solution

The idea is to transform each trip into two events. For each trip:

- Create an event at the 'from' location to add passengers.

– Create an event at the 'to' location to subtract passengers. Sort these events by the location value. When events coincide at the same location, process the drop-off events (negative changes) before the pick-up events (positive changes) to ensure capacity is freed before being assigned. Then, iterate over the sorted events, update the cumulative number of passengers, and check against the available capacity. If at any point the current passenger count exceeds the capacity, return false. If the entire journey is simulated without breaching capacity, return true.

Intervals

□ #1229 Meeting Scheduler

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: AlgoMaster 600

Problem

Given the availability time slots arrays `slots1` and `slots2` of two people and a meeting duration `duration`, return the earliest time slot that works for both of them and is of duration `duration`.

If there is no common time slot that satisfies the requirements, return an empty array.

The format of a time slot is an array of two elements `[start, end]` representing an inclusive time range from `start` to `end`.

It is guaranteed that no two availability slots of the same person intersect with each other. That is, for any two time slots `[start1, end1]` and `[start2, end2]` of the same person, either `start1 > end2` or `start2 > end1`.

Example 1:

Input: slots1 = [[10,50],[60,120],[140,210]], slots2 = [[0,15],[60,70]],
 duration = 8
 Output: [60,68]

Example 2:

Input: slots1 = [[10,50],[60,120],[140,210]], slots2 = [[0,15],[60,70]],
 duration = 12
 Output: []

Constraints:

- $1 \leq \text{slots1.length}, \text{slots2.length} \leq 104$
- $\text{slots1}[i].\text{length}, \text{slots2}[i].\text{length} == 2$
- $\text{slots1}[i][0] < \text{slots1}[i][1]$
- $\text{slots2}[i][0] < \text{slots2}[i][1]$
- $0 \leq \text{slots1}[i][j], \text{slots2}[i][j] \leq 109$
- $1 \leq \text{duration} \leq 106$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minAvailableDuration(
        self,
        slots1: list[list[int]],
        slots2: list[list[int]],
        duration: int,
    ) -> list[int]:
        slots1.sort()
        slots2.sort()
```

```

i = 0 # slots1's index
j = 0 # slots2's index

while i < len(slots1) and j < len(slots2):
    start = max(slots1[i][0], slots2[j][0])
    end = min(slots1[i][1], slots2[j][1])
    if start + duration <= end:
        return [start, start + duration]
    if slots1[i][1] < slots2[j][1]:
        i += 1

    else:
        j += 1

return []

```

• LeetCode

```

class Solution:
    def minAvailableDuration(
        self, slots1: List[List[int]], slots2: List[List[int]], duration: int
    ) -> List[int]:
        slots1.sort()
        slots2.sort()
        m, n = len(slots1), len(slots2)
        i = j = 0
        while i < m and j < n:
            start = max(slots1[i][0], slots2[j][0])

            end = min(slots1[i][1], slots2[j][1])
            if end - start >= duration:
                return [start, start + duration]
            if slots1[i][1] < slots2[j][1]:
                i += 1
            else:
                j += 1
        return []

```

• SimplyLeet

```
# Python solution for Meeting Scheduler

def meeting_scheduler(slots1, slots2, duration):
    # Sort both lists based on start time
    slots1.sort(key=lambda x: x[0])
    slots2.sort(key=lambda x: x[0])

    # Initialize two pointers for both lists
    i, j = 0, 0

    # Iterate through both lists until one pointer reaches the end
    while i < len(slots1) and j < len(slots2):
        # Calculate the latest start and earliest end of the two slots
        start = max(slots1[i][0], slots2[j][0])
        end = min(slots1[i][1], slots2[j][1])

        # Check if overlapping duration is enough for the meeting
        if end - start >= duration:
            return [start, start + duration]

        # Move the pointer that has the earlier end time
        if slots1[i][1] < slots2[j][1]:
            i += 1
        else:
            j += 1

    # If no valid meeting slot is found, return an empty list
    return []

# Example usage:

# print(meeting_scheduler([[10,50],[60,120],[140,210]], [[0,15],[60,70]], 8))
```

◆ Quick Review

Key Insights

- Sort both time slot arrays by their start times to simplify the overlap checking process.
- Use a two-pointer approach to traverse the sorted arrays concurrently.
- For each pair of slots examined, compute the intersection by taking the maximum of the start times and the minimum of the end times.
- Check if this overlapping duration is at least as long as the required meeting duration; if so, return the earliest valid meeting interval.
- Advance the pointer corresponding to the slot that ends earlier to look for a potentially better overlapping slot further on.

Space & Time

Time Complexity: $O(n \log n + m \log m)$, where n and m are the number of slots in `slots1` and `slots2`, respectively, due to sorting. The merging process itself runs in $O(n + m)$.

Space Complexity: $O(1)$ additional space (ignoring the space used for sorting if done in-place).

Solution

The solution involves first sorting both `slots1` and `slots2` by their start times. Then, using a

two-pointer approach, iterate over both sorted lists simultaneously. For each pair of slots, calculate the possible overlapping time interval by taking $\max(\text{slot1_start}, \text{slot2_start})$ as the start and $\min(\text{slot1_end}, \text{slot2_end})$ as the end. If the duration of the overlap is at least as long as the required duration, the earliest feasible meeting time is found and we return the interval starting at this overlap start and ending at (overlap start + duration). If the overlap is insufficient, advance the pointer of the slot that ends first since that slot cannot provide a sufficient overlap with any future slots from the other list.

Intervals

□ #1288 Remove Covered Intervals

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting

Lists: AlgoMaster 600

Problem

Given an array `intervals` where `intervals[i] = [li, ri]` represent the interval `[li, ri)`, remove all intervals that are covered by another interval in the list.

The interval `[a, b)` is covered by the interval `[c, d)` if and only if `c ≤ a` and `b ≤ d`.

Return the number of remaining intervals.

Example 1:

```
Input: intervals = [[1,4],[3,6],[2,8]]
Output: 2
Explanation: Interval [3,6] is covered by [2,8], therefore it is removed.
```

Example 2:

```
Input: intervals = [[1,4],[2,3]]
Output: 1
```

Constraints:

- $1 \leq \text{intervals.length} \leq 1000$

- `intervals[i].length == 2`
- `0 ≤ li < ri ≤ 105`
- All the given intervals are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeCoveredIntervals(self, intervals: list[list[int]]) -> int:
        ans = 0
        prevEnd = 0

        for _, end in sorted(intervals, key=lambda x: (x[0], -x[1])):
            # The current interval is not covered by the previous one.
            if prevEnd < end:
                prevEnd = end
                ans += 1

        return ans
```

• LeetDoocs

```
class Solution:
    def removeCoveredIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: (x[0], -x[1]))
        ans = 0
        pre = -inf
        for _, cur in intervals:
            if cur > pre:
                ans += 1
                pre = cur
        return ans
```

• SimplyLeet

```
# Python solution to remove covered intervals with line-by-line comments
class Solution:
    def removeCoveredIntervals(self, intervals):
        # Sort intervals by start ascending and then by end descending.
        intervals.sort(key=lambda x: (x[0], -x[1]))
        count = 0 # Initialize counter for non-covered intervals.
        max_end = 0 # Track the maximum end encountered.

        # Iterate over the sorted intervals.
        for start, end in intervals:

            # If the current interval's end is greater than the maximum end
            # seen,
            # it is not covered by any previous interval.
            if end > max_end:
                count += 1 # Increment count of valid intervals.
                max_end = end # Update the maximum end.
            # Return the count of remaining intervals.
        return count

# Example usage:
sol = Solution()

print(sol.removeCoveredIntervals([[1,4],[3,6],[2,8]]))
```

◆ Quick Review

Key Insights

- Sort intervals by their start points in ascending order and by end points in descending order when starts are equal.
- By sorting this way, if two intervals have the same start, the longer interval comes first, making it easier to detect coverage.

- Keep track of the maximum end encountered so far.
- When iterating, if the current interval's end is less than or equal to the tracked maximum end, it is covered by a previous interval.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting, with an additional $O(n)$ for a single pass through the intervals.

Space Complexity: $O(1)$ extra space if the sort is in-place.

Solution

The solution involves a greedy approach:

- Sort the intervals by start coordinate in ascending order; if two intervals share the same start, sort them by their end coordinate in descending order.
- Initialize a counter for the number of non-covered intervals and a variable to record the farthest end encountered.
- Iterate through the sorted list. For each interval, if its end is greater than the maximum end seen so far, it is not covered and should be counted; update the maximum end.

- Return the count.

This technique ensures each interval is processed only once after sorting, which yields an efficient and optimal solution.

Intervals

□ #1353 Maximum Number of Events That Can Be Attended **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Sorting · Heap (Priority Queue)
Lists: AlgoMaster 600

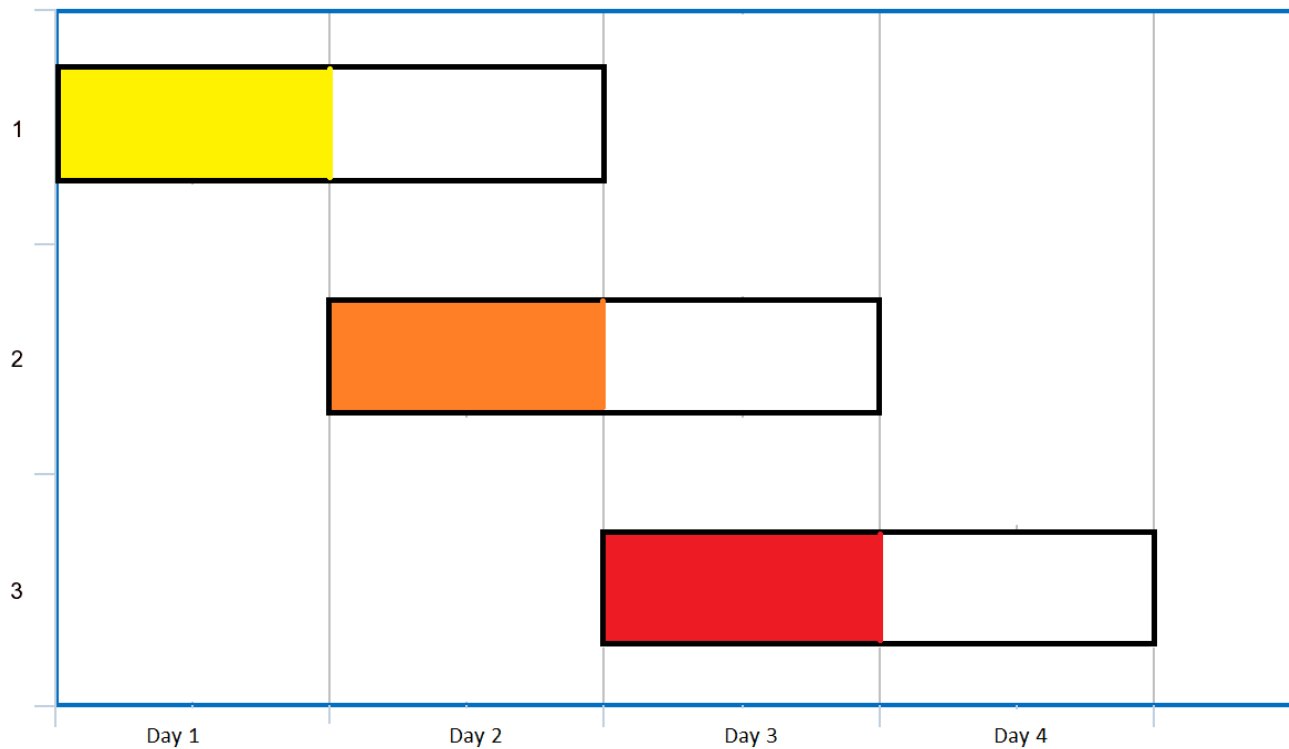
Problem

You are given an array of events where $\text{events}[i] = [\text{startDay}_i, \text{endDay}_i]$. Every event i starts at startDay_i and ends at endDay_i .

You can attend an event i at any day d where $\text{startDay}_i \leq d \leq \text{endDay}_i$. You can only attend one event at any time d .

Return the maximum number of events you can attend.

Example 1:



Input: events = [[1,2],[2,3],[3,4]]

Output: 3

Explanation: You can attend all the three events.

One way to attend them all is as shown.

Attend the first event on day 1.

Attend the second event on day 2.

Attend the third event on day 3.

Example 2:

Input: events= [[1,2],[2,3],[3,4],[1,2]]

Output: 4

Constraints:

- $1 \leq \text{events.length} \leq 105$
- $\text{events}[i].\text{length} == 2$
- $1 \leq \text{startDay}_i \leq \text{endDay}_i \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxEvents(self, events: list[list[int]]) -> int:
        ans = 0
        minHeap = []
        i = 0 # events' index

        events.sort(key=lambda x: x[0])

        while minHeap or i < len(events):
            # If no events are available to attend today, let time flies to the next
            # available event.
            if not minHeap:
                d = events[i][0]
                # All the events starting from today are newly available.
                while i < len(events) and events[i][0] == d:
                    heapq.heappush(minHeap, events[i][1])
                    i += 1
                # Greedily attend the event that'll end the earliest since it has higher
                # chance can't be attended in the future.
                heapq.heappop(minHeap)

            ans += 1
            d += 1
            # Pop the events that can't be attended.
            while minHeap and minHeap[0] < d:
                heapq.heappop(minHeap)

        return ans
```

• LeetDoocs

```

class Solution:
    def maxEvents(self, events: List[List[int]]) -> int:
        g = defaultdict(list)
        l, r = inf, 0
        for s, e in events:
            g[s].append(e)
            l = min(l, s)
            r = max(r, e)
        pq = []
        ans = 0

        for s in range(l, r + 1):
            while pq and pq[0] < s:
                heappop(pq)
            for e in g[s]:
                heappush(pq, e)
            if pq:
                heappop(pq)
                ans += 1
        return ans

```

• SimplyLeet

```

import heapq

def maxEvents(events):
    # Sort events by start day
    events.sort(key=lambda x: x[0])
    # Initialize a min-heap and index for events
    min_heap = []
    i = 0
    total_events = 0
    n = len(events)

```

```
# Determine the last possible day to check (max end day among events)
last_day = max(event[1] for event in events)

# Iterate over each day
for day in range(1, last_day + 1):
    # Add events starting on the current day to the heap with their end
    days
    while i < n and events[i][0] == day:
        heapq.heappush(min_heap, events[i][1])
        i += 1

    # Remove events that have already ended (end day before current day)
    while min_heap and min_heap[0] < day:
        heapq.heappop(min_heap)

    # If there is an available event, attend the one ending earliest
    if min_heap:
        heapq.heappop(min_heap)
        total_events += 1

return total_events

# Example usage:
# events = [[1,2],[2,3],[3,4]]
# print(maxEvents(events)) # Expected Output: 3
```

◆ Quick Review

Key Insights

- Sort events by their start day.
- Use a min-heap (priority queue) to dynamically store events that have started but not yet ended.

- On each day, add all events starting on that day to the heap.
- Remove events from the heap that have already ended.
- Always attend the event that ends the earliest (pop from the heap) to free up future days.
- Iteratively move through each day until all events are processed.

Space & Time

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

Solution

The algorithm starts by sorting the events by their start day. Then, using a min-heap, it processes each day from the earliest start day up to the latest possible day among the events. For each day, it adds all events that begin on that day into the min-heap. The heap is used to keep track of the current events by their end days. Events that have ended before the current day are removed from the heap. On each day, if there is any event in the heap, the one with the smallest end day (which gives the

best chance for future days) is attended and removed from the heap. This greedy strategy maximizes the total number of events attended.

Intervals

□ #2054 Two Best Non-Overlapping Events

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Dynamic Programming ·
Sorting · Heap (Priority Queue)
Lists: AlgoMaster 600

Problem

You are given a 0-indexed 2D integer array of events where $\text{events}[i] = [\text{startTime}_i, \text{endTime}_i, \text{value}_i]$. The i th event starts at startTime_i and ends at endTime_i , and if you attend this event, you will receive a value of value_i . You can choose at most two non-overlapping events to attend such that the sum of their values is maximized.

Return this maximum sum.

Note that the start time and end time is inclusive: that is, you cannot attend two events where one of them starts and the other ends at the same time. More specifically, if you attend an event with end time t , the next event must

start at or after $t + 1$.

Example 1:

Time	1	2	3	4	5
Event 0	2				
Event 1				2	
Event 2		3			

Input: events = `[[1,3,2],[4,5,2],[2,4,3]]`

Output: 4

Explanation: Choose the green events, 0 and 1 for a sum of $2 + 2 = 4$.

Example 2:

Time	1	2	3	4	5
Event 0	2				
Event 1				2	
Event 2	5				

Input: events = `[[1,3,2],[4,5,2],[1,5,5]]`

Output: 5

Explanation: Choose event 2 for a sum of 5.

Example 3:

Time	1	2	3	4	5	6
Event 0	3					
Event 1	1					
Event 2						5

Input: events = `[[1,5,3],[1,5,1],[6,6,5]]`

Output: 8

Explanation: Choose events 0 and 2 for a sum of $3 + 5 = 8$.

Constraints:

- $2 \leq \text{events.length} \leq 105$
- $\text{events}[i].\text{length} == 3$
- $1 \leq \text{startTime}_i \leq \text{endTime}_i \leq 109$
- $1 \leq \text{value}_i \leq 106$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxTwoEvents(self, events: list[list[int]]) -> int:
        ans = 0
        maxValue = 0
        evts = [] # (time, isStart, value)

        for s, e, v in events:
            evts.append((s, 1, v))
            evts.append((e + 1, 0, v))

        # When two events have the same time, the one is not start will be in the front
        evts.sort()

        for _, isStart, value in evts:
            if isStart:
                ans = max(ans, value + maxValue)
            else:
                maxValue = max(maxValue, value)

        return ans
```

• LeetDoocs

```

class Solution:
    def maxTwoEvents(self, events: List[List[int]]) -> int:
        events.sort()
        n = len(events)
        f = [events[-1][2]] * n
        for i in range(n - 2, -1, -1):
            f[i] = max(f[i + 1], events[i][2])
        ans = 0
        for _, e, v in events:
            idx = bisect_right(events, e, key=lambda x: x[0])

            if idx < n:
                v += f[idx]
            ans = max(ans, v)
        return ans

```

• SimplyLeet

```

import bisect
from typing import List

class Solution:
    def maxTwoEvents(self, events: List[List[int]]) -> int:
        # Sort events by their start time
        events.sort(key=lambda x: x[0])
        n = len(events)
        # Extract start times to use for binary search later
        start_times = [event[0] for event in events]

        # Build a suffix max array: suffix_max[i] holds the max event value
        # from events[i] to end
        suffix_max = [0] * n
        suffix_max[-1] = events[-1][2]
        for i in range(n - 2, -1, -1):
            suffix_max[i] = max(events[i][2], suffix_max[i + 1])
        max_value = 0
        # Iterate over each event and use binary search to find the next
        # non-overlapping event
        for i in range(n):
            current_value = events[i][2]
            # Find the first event starting at or after events[i][1] + 1

```

```
next_index = bisect.bisect_left(start_times, events[i][1] + 1)
if next_index < n:
    current_value += suffix_max[next_index]
max_value = max(max_value, current_value)
return max_value
```

◆ Quick Review

Key Insights

- Sort events by start time. This allows for binary search to quickly find the next non-overlapping event.
- Precompute a suffix maximum array where each position stores the maximum event value among all events starting from that index.
- For each event, via binary search, determine the earliest event that can be attended after the current one, and combine the current event's value with the best value from the remainder of the list.
- Consider the case where taking a single event is optimal.

Space & Time

Time Complexity: $O(n \log n)$ – $O(n \log n)$ for sorting and binary search over each event.

Space Complexity: $O(n)$ – for the auxiliary arrays used (start times and suffix maximum values).

Solution

The algorithm sorts the events by their start time. Then, it builds an array that holds the maximum event value from any index to the end (suffix max). For each event, a binary search is used on the sorted start times to find the next event that can be attended (i.e., the event whose start time is at least the current event's end time plus one). The value of the current event is added to the precomputed best value from that next event, and the maximum sum is updated accordingly. This approach ensures that the best combination of two non-overlapping events is found while keeping complexity under control.

K-Way Merge

Round 5 · Mid · Medium · 2 questions

K-Way Merge

□ #373 Find K Pairs with Smallest Sums

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given two integer arrays `nums1` and `nums2` sorted in non-decreasing order and an integer `k`.

Define a pair (u, v) which consists of one element from the first array and one element from the second array.

Return the `k` pairs $(u_1, v_1), (u_2, v_2), \dots, (u_k, v_k)$ with the smallest sums.

Example 1:

```
Input: nums1 = [1,7,11], nums2 = [2,4,6], k = 3
Output: [[1,2],[1,4],[1,6]]
Explanation: The first 3 pairs are returned from the sequence:
[1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6],[11,4],[11,6]
```

Example 2:

```
Input: nums1 = [1,1,2], nums2 = [1,2,3], k = 2
Output: [[1,1],[1,1]]
Explanation: The first 2 pairs are returned from the sequence:
[1,1],[1,1],[1,2],[2,1],[1,2],[2,2],[1,3],[1,3],[2,3]
```

Constraints:

- $1 \leq \text{nums1.length}, \text{nums2.length} \leq 105$
- $-109 \leq \text{nums1}[i], \text{nums2}[i] \leq 109$
- nums1 and nums2 both are sorted in non-decreasing order.
- $1 \leq k \leq 104$
- $k \leq \text{nums1.length} * \text{nums2.length}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def kSmallestPairs(self, nums1: list[int],
                      nums2: list[int],
                      k: int) -> list[list[int]]:

        minHeap = []

        for i in range(min(k, len(nums1))):
            heapq.heappush(minHeap, (nums1[i] + nums2[0], i, 0))

        ans = []

        while minHeap and len(ans) < k:
            _, i, j = heapq.heappop(minHeap)
            ans.append([nums1[i], nums2[j]])
            if j + 1 < len(nums2):
                heapq.heappush(minHeap, (nums1[i] + nums2[j + 1], i, j + 1))

        return ans
```

• LeetDoocs

```

class Solution:
    def kSmallestPairs(
        self, nums1: List[int], nums2: List[int], k: int
    ) -> List[List[int]]:
        q = [[u + nums2[0], i, 0] for i, u in enumerate(nums1[:k])]
        heapify(q)
        ans = []
        while q and k > 0:
            _, i, j = heappop(q)
            ans.append([nums1[i], nums2[j]])

            k -= 1
            if j + 1 < len(nums2):
                heappush(q, [nums1[i] + nums2[j + 1], i, j + 1])
        return ans

```

• SimplyLeet

```

import heapq

def kSmallestPairs(nums1, nums2, k):
    # Edge case: if either array is empty, return an empty list
    if not nums1 or not nums2 or k <= 0:
        return []

    result = []
    minHeap = []

    # Initialize the heap with the first element from nums2 for each element in
    # nums1 (up to k)
    for i in range(min(k, len(nums1))):
        # Each entry: (sum, index in nums1, index in nums2)
        heapq.heappush(minHeap, (nums1[i] + nums2[0], i, 0))

    # Extract the smallest pairs until we found k pairs or the heap is empty
    while k > 0 and minHeap:
        currentSum, i, j = heapq.heappop(minHeap)
        result.append([nums1[i], nums2[j]])
        k -= 1

```



```
# If there's any next element in nums2, push the new pair into the heap
if j + 1 < len(nums2):
    heapq.heappush(minHeap, (nums1[i] + nums2[j + 1], i, j + 1))

return result

# Example usage:
print(kSmallestPairs([1,7,11], [2,4,6], 3))
```

◆ Quick Review

Key Insights

- Both arrays are sorted, making it possible to efficiently traverse and pick the smallest candidate pairs.
- A min-heap (priority queue) can be used to always access the next smallest sum pair.
- Instead of generating all possible pairs (which could be very large), we only generate candidates and push new pairs from `nums2` as needed.

Space & Time

Time Complexity: $O(k \log(\min(k, n1)))$ where $n1$ is the size of `nums1`, since we push at most $\min(k, n1)$ elements into the heap.

Space Complexity: $O(\min(k, n1))$ for storing the heap elements.

Solution

We use a min-heap (priority queue) to efficiently track and extract the pair with the smallest sum. Initially, we push pairs composed of each element in `nums1` (up to `k` elements) paired with the first element of `nums2`. Each element in the heap is a tuple containing (sum, index in `nums1`, index in `nums2`). Every time we extract the smallest pair, we add the next pair from `nums2` (if available) for that index in `nums1`. This approach ensures we always have the next smallest pair available and avoids creating all possible pairs upfront.

K-Way Merge

□ #378 Kth Smallest Element in a Sorted Matrix

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Sorting · Heap (Priority Queue) · Matrix
Lists: AlgoMaster 600

Problem

Given an $n \times n$ matrix where each of the rows and columns is sorted in ascending order, return the k th smallest element in the matrix.

Note that it is the k th smallest element in the sorted order, not the k th distinct element.

You must find a solution with a memory complexity better than $O(n^2)$.

Example 1:

Input: matrix = [[1,5,9],[10,11,13],[12,13,15]], k = 8

Output: 13

Explanation: The elements in the matrix are [1,5,9,10,11,12,13,13,15], and the 8th smallest number is 13

Example 2:

Input: matrix = [[-5]], k = 1

Output: -5

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $1 \leq n \leq 300$
- $-109 \leq \text{matrix}[i][j] \leq 109$
- All the rows and columns of matrix are guaranteed to be sorted in non-decreasing order.
- $1 \leq k \leq n^2$

Follow up:

- Could you solve the problem with a constant memory (i.e., $O(1)$ memory complexity)?
- Could you solve the problem in $O(n)$ time complexity? The solution may be too advanced for an interview but you may find reading this paper fun.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def kthSmallest(self, matrix: list[list[int]], k: int) -> int:
        minHeap = [] # (matrix[i][j], i, j)

        i = 0
        while i < k and i < len(matrix):
            heapq.heappush(minHeap, (matrix[i][0], i, 0))
            i += 1

        while k > 1:
```

```

k -= 1
_, i, j = heapq.heappop(minHeap)
if j + 1 < len(matrix[0]):
    heapq.heappush(minHeap, (matrix[i][j + 1], i, j + 1))

return minHeap[0][0]

```

```

class Solution:
    def kthSmallest(self, matrix: list[list[int]], k: int) -> int:
        def numsNoGreaterThan(m: int) -> int:
            count = 0
            j = len(matrix[0]) - 1
            # For each row, find the first index j s.t. row[j] <= m s.t. the number
            of
            # numbers <= m for this row will be j + 1.
            for row in matrix:
                while j >= 0 and row[j] > m:
                    j -= 1

            count += j + 1
            return count

        l = matrix[0][0]
        r = matrix[-1][-1]
        return bisect.bisect_left(range(l, r), k, key=numsNoGreaterThan) + l

```

• LeetDoocs

```

class Solution:
    def kthSmallest(self, matrix: List[List[int]], k: int) -> int:
        def check(matrix, mid, k, n):
            count = 0
            i, j = n - 1, 0
            while i >= 0 and j < n:
                if matrix[i][j] <= mid:
                    count += i + 1
                    j += 1
                else:

```

```

        i -= 1
        return count >= k

n = len(matrix)
left, right = matrix[0][0], matrix[n - 1][n - 1]
while left < right:
    mid = (left + right) >> 1
    if check(matrix, mid, k, n):
        right = mid
    else:
        left = mid + 1
return left

```

• SimplyLeet

Python solution using binary search

```

def kthSmallest(matrix, k):
    n = len(matrix)
    # Set initial low and high based on the smallest and largest values in the
    matrix
    low, high = matrix[0][0], matrix[-1][-1]

    def count_less_equal(x):
        # Count the number of elements less than or equal to x
        count = 0

        # Start from the bottom-left corner of the matrix
        row, col = n - 1, 0
        while row >= 0 and col < n:
            if matrix[row][col] <= x:
                # All elements in this column up to this row are <= x
                count += row + 1
                col += 1
            else:
                row -= 1
        return count

```

```
# Binary search to find the kth smallest element
while low < high:
    mid = low + (high - low) // 2
    if count_less_equal(mid) < k:
        low = mid + 1 # kth element is greater than mid
    else:
        high = mid # kth element is less than or equal to mid
return low
```

```
# Example usage:
# print(kthSmallest([[1,5,9],[10,11,13],[12,13,15]], 8))
```

◆ Quick Review

Key Insights

- The matrix is sorted both row-wise and column-wise.
- Instead of flattening and sorting the entire matrix (which uses $O(n^2)$ space), take advantage of the sorted properties.
- Binary search can be applied over the range of values in the matrix.
- Count the number of elements less than or equal to a midpoint to determine if the kth element is on the left or right side of the search range.
- Alternatively, a min-heap approach can be used, though it generally leads to higher space

complexity.

Space & Time

Time Complexity: $O(n * \log(\max - \min))$, where max and min represent the range of values in the matrix.

Space Complexity: $O(1)$ extra space (ignoring recursion and input).

Solution

We solve the problem by performing binary search on the value range between the smallest and largest elements in the matrix. For a mid-value during the binary search, we count how many elements in the matrix are less than or equal to mid by scanning from the bottom-left corner. If the count is less than k, we move to the right half of the range; otherwise, we adjust the range to the left. This iterative process continues until the range is narrowed down to a single element, which is the kth smallest.

Data Structure Design

Round 5 · Mid · Medium · 4 questions

Data Structure Design

□ #641 Design Circular Deque

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Linked List · Design · Queue
Lists: AlgoMaster 600

Problem

Design your implementation of the circular double-ended queue (deque).

Implement the MyCircularDeque class:

- **MyCircularDeque(int k)** Initializes the deque with a maximum size of k.
- **boolean insertFront()** Adds an item at the front of Deque. Returns true if the operation is successful, or false otherwise.
- **boolean insertLast()** Adds an item at the rear of Deque. Returns true if the operation is successful, or false otherwise.
- **boolean deleteFront()** Deletes an item from the front of Deque. Returns true if the operation is successful, or false otherwise.

- **boolean deleteLast()** Deletes an item from the rear of Deque. Returns true if the operation is successful, or false otherwise.
- **int getFront()** Returns the front item from the Deque. Returns -1 if the deque is empty.
- **int getRear()** Returns the last item from Deque. Returns -1 if the deque is empty.
- **boolean isEmpty()** Returns true if the deque is empty, or false otherwise.
- **boolean isFull()** Returns true if the deque is full, or false otherwise.

Example 1:

Input

```
["MyCircularDeque", "insertLast", "insertLast", "insertFront", "insertFront",  
"getRear", "isFull", "deleteLast", "insertFront", "getFront"]  
[[3], [1], [2], [3], [4], [], [], [], [4], []]
```

Output

```
[null, true, true, true, false, 2, true, true, true, 4]
```

Explanation

```
MyCircularDeque myCircularDeque = new MyCircularDeque(3);
```

Constraints:

- $1 \leq k \leq 1000$
- $0 \leq \text{value} \leq 1000$
- At most 2000 calls will be made to insertFront, insertLast, deleteFront, deleteLast, getFront, getRear, isEmpty, isFull.

Community Solutions (Python)

• LeetCode

```
class MyCircularDeque:
    def __init__(self, k: int):
        """
        Initialize your data structure here. Set the size of the deque to be k.
        """
        self.q = [0] * k
        self.front = 0
        self.size = 0
        self.capacity = k

    def insertFront(self, value: int) -> bool:
        """
        Adds an item at the front of Deque. Return true if the operation is
        successful.
        """
        if self.isFull():
            return False
        if not self.isEmpty():
            self.front = (self.front - 1 + self.capacity) % self.capacity
        self.q[self.front] = value
        self.size += 1

        return True

    def insertLast(self, value: int) -> bool:
        """
        Adds an item at the rear of Deque. Return true if the operation is
        successful.
        """
        if self.isFull():
            return False
        idx = (self.front + self.size) % self.capacity
        self.q[idx] = value
```

```
        self.size += 1
        return True

    def deleteFront(self) -> bool:
        """
        Deletes an item from the front of Deque. Return true if the operation
        is successful.
        """
        if self.isEmpty():
            return False
        self.front = (self.front + 1) % self.capacity

        self.size -= 1
        return True

    def deleteLast(self) -> bool:
        """
        Deletes an item from the rear of Deque. Return true if the operation is
        successful.
        """
        if self.isEmpty():
            return False
        self.size -= 1

        return True

    def getFront(self) -> int:
        """
        Get the front item from the deque.
        """
        if self.isEmpty():
            return -1
        return self.q[self.front]
```

```
def getRear(self) -> int:
    """
    Get the last item from the deque.
    """
    if self.isEmpty():
        return -1
    idx = (self.front + self.size - 1) % self.capacity
    return self.q[idx]
```

```
def isEmpty(self) -> bool:
```

```
    """
    Checks whether the circular deque is empty or not.
    """
    return self.size == 0
```

```
def isFull(self) -> bool:
```

```
    """
    Checks whether the circular deque is full or not.
    """
    return self.size == self.capacity
```

Your MyCircularDeque object will be instantiated and called as such:

```
# obj = MyCircularDeque(k)
# param_1 = obj.insertFront(value)
# param_2 = obj.insertLast(value)
# param_3 = obj.deleteFront()
# param_4 = obj.deleteLast()
# param_5 = obj.getFront()
# param_6 = obj.getRear()
```

```
# param_7 = obj.isEmpty()
# param_8 = obj.isFull()
```

• SimplyLeet

```

class MyCircularDeque:
    def __init__(self, k: int):
        # Initialize the deque with a fixed capacity of k
        self.capacity = k
        self.deque = [0] * k
        self.size = 0
        self.front = 0           # Points to the current front element
        self.rear = 0           # Points to the next insertion position at
rear

```

```

def insertFront(self, value: int) -> bool:

```

```

    if self.isFull():
        return False
    # Move front pointer backward using modulo arithmetic
    self.front = (self.front - 1 + self.capacity) % self.capacity
    self.deque[self.front] = value
    self.size += 1
    return True

```

```

def insertLast(self, value: int) -> bool:
    if self.isFull():

```

```

        return False
    # Insert element at the rear pointer and update it circularly
    self.deque[self.rear] = value
    self.rear = (self.rear + 1) % self.capacity
    self.size += 1
    return True

```

```

def deleteFront(self) -> bool:
    if self.isEmpty():
        return False

```

```
# Move front pointer forward to delete the current front element
self.front = (self.front + 1) % self.capacity
self.size -= 1
return True

def deleteLast(self) -> bool:
    if self.isEmpty():
        return False
    # Move rear pointer backward to delete the last element
    self.rear = (self.rear - 1 + self.capacity) % self.capacity

    self.size -= 1
    return True

def getFront(self) -> int:
    if self.isEmpty():
        return -1
    return self.deque[self.front]

def getRear(self) -> int:
    if self.isEmpty():
        return -1
    # Since rear pointer indicates next inserting slot, adjust to get last
    # element
    return self.deque[(self.rear - 1 + self.capacity) % self.capacity]

def isEmpty(self) -> bool:
    return self.size == 0

def isFull(self) -> bool:
    return self.size == self.capacity
```

◆ Quick Review

Key Insights

- Use a fixed-size array to simulate a circular buffer.

- Maintain two pointers (front and rear) to track the positions for insertion/deletion.
- Use modulo arithmetic for pointer adjustments to create the circular effect.
- Track the number of elements to easily determine if the deque is full or empty.
- Ensure that operations such as getFront and getRear return -1 when the deque is empty.

Space & Time

Time Complexity: $O(1)$ for all operations

Space Complexity: $O(k)$, where k is the size of the deque

Solution

The solution employs a circular array as the underlying data structure to represent the deque. Two pointers, front and rear, are used to manage the insertion and deletion from both ends of the deque. When inserting at the front, the front pointer is decremented modulo the capacity; and for insertion at the rear, the element is added at the current rear pointer and then the rear pointer is incremented modulo the capacity. A size counter tracks the current number of elements, easing the

detection of full or empty states. Deletion operations adjust the pointers similarly without shifting elements, thereby achieving constant time complexity for all operations.

Data Structure Design

□ #1146 Snapshot Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Binary Search · Design
Lists: AlgoMaster 600

Problem

Implement a `SnapshotArray` that supports the following interface:

- `SnapshotArray(int length)` initializes an array-like data structure with the given length. Initially, each element equals 0.
- `void set(index, val)` sets the element at the given index to be equal to `val`.
- `int snap()` takes a snapshot of the array and returns the `snap_id`: the total number of times we called `snap()` minus 1.
- `int get(index, snap_id)` returns the value at the given index, at the time we took the snapshot with the given `snap_id`

Example 1:

```

Input: ["SnapshotArray","set","snap","set","get"]
[[3],[0,5],[],[0,6],[0,0]]
Output: [null,null,0,null,5]
Explanation:
SnapshotArray snapshotArr = new SnapshotArray(3); // set the length to be 3
snapshotArr.set(0,5); // Set array[0] = 5
snapshotArr.snap(); // Take a snapshot, return snap_id = 0
snapshotArr.set(0,6);

```

Constraints:

- $1 \leq \text{length} \leq 5 * 10^4$
- $0 \leq \text{index} < \text{length}$
- $0 \leq \text{val} \leq 10^9$
- $0 \leq \text{snap_id} < (\text{the total number of times we call snap()})$
- At most $5 * 10^4$ calls will be made to set, snap, and get.

Community Solutions (Python)

• WalkCC

```

class SnapshotArray:
    def __init__(self, length: int):
        self.snaps = [[[0, 0]] for _ in range(length)]
        self.snap_id = 0

    def set(self, index: int, val: int) -> None:
        snap = self.snaps[index][-1]
        if snap[0] == self.snap_id:
            snap[1] = val
        else:

```

```

        self.snaps[index].append([self.snap_id, val])

    def snap(self) -> int:
        self.snap_id += 1
        return self.snap_id - 1

    def get(self, index: int, snap_id: int) -> int:
        i = bisect.bisect_left(self.snaps[index], [snap_id + 1]) - 1
        return self.snaps[index][i][1]

```

• LeetDoocs

```

class SnapshotArray:

    def __init__(self, length: int):
        self.arr = [[] for _ in range(length)]
        self.i = 0

    def set(self, index: int, val: int) -> None:
        self.arr[index].append((self.i, val))

    def snap(self) -> int:

        self.i += 1
        return self.i - 1

    def get(self, index: int, snap_id: int) -> int:
        i = bisect_left(self.arr[index], (snap_id, inf)) - 1
        return 0 if i < 0 else self.arr[index][i][1]

# Your SnapshotArray object will be instantiated and called as such:
# obj = SnapshotArray(length)

# obj.set(index, val)
# param_2 = obj.snap()
# param_3 = obj.get(index, snap_id)

```

• SimplyLeet

```
class SnapshotArray:
    def __init__(self, length: int):
        # Initialize a history for each index with an initial value 0 at
        # snap_id 0.
        self.data = [[(0, 0)] for _ in range(length)]
        # Current snapshot id starts at 0.
        self.snap_id = 0

    def set(self, index: int, val: int) -> None:
        # Get the history list for the index.
        history = self.data[index]

        # If the most recent set was made in the current snapshot, update it.
        if history[-1][0] == self.snap_id:
            history[-1] = (self.snap_id, val)
        else:
            # Otherwise, append a new record for this snapshot.
            history.append((self.snap_id, val))

    def snap(self) -> int:
        # Return the current snap_id then increment for the next snapshot.
        self.snap_id += 1

        return self.snap_id - 1

    def get(self, index: int, snap_id: int) -> int:
        history = self.data[index]
        # Binary search for the right-most record not exceeding snap_id.
        l, r = 0, len(history) - 1
        while l <= r:
            mid = (l + r) // 2
            if history[mid][0] <= snap_id:
                l = mid + 1
```

```
        else:
            r = mid - 1
            # r holds the latest index where the snapshot id is <= snap_id.
            return history[r][1]

# Example usage:
# snapshotArr = SnapshotArray(3)
# snapshotArr.set(0, 5)
# print(snapshotArr.snap()) # returns 0
# snapshotArr.set(0, 6)

# print(snapshotArr.get(0, 0)) # returns 5
```

◆ Quick Review

Key Insights

- Use per-index storage to maintain history of changes; record only when updates occur.
- Each index stores a list of (snap_id, value) pairs, sorted by snap_id.
- Utilize binary search during a get operation to efficiently find the most recent update not exceeding the provided snap_id.
- Keep a current snap_id counter and update it with each snap() call.

Space & Time

Time Complexity:

- set: $O(1)$ (amortized)
- snap: $O(1)$

– get: $O(\log S)$, where S is the number of times an index was set (binary search over updates)
Space Complexity: $O(n + \text{total number of set operations})$, since we store only updates per index.

Solution

We maintain a history for each array index as a list of pairs with each pair containing a snapshot id and its corresponding value. When setting a value, if the latest snapshot id in the history for that index matches the current `snap_id`, update the value. Otherwise, append a new `(snap_id, value)` pair. The `snap()` method will increment the `snap_id` counter and return the previous counter. The `get()` method performs a binary search on the history list for the given index to locate the most recent update where `snap_id` is less than or equal to the queried `snap_id`.

Data Structure Design

□ #1472 Design Browser History

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Linked List · Stack · Design ·
Doubly-Linked List · Data Stream
Lists: AlgoMaster 600

Problem

You have a browser of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

Implement the BrowserHistory class:

- **BrowserHistory(string homepage)** Initializes the object with the homepage of the browser.
- **void visit(string url)** Visits url from the current page. It clears up all the forward history.
- **string back(int steps)** Move steps back in history. If you can only return x steps in the history and $steps > x$, you will return only x steps. Return the current url after moving

back in history at most steps.

- **string forward(int steps)** Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history at most steps.

Example:

Input:

```
["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]
```

```
[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["linkedin.com"],[2],[2],[7]]
```

Output:

```
[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leetcode.com"]
```

Explanation:

```
BrowserHistory browserHistory = new BrowserHistory("leetcode.com");
```

Constraints:

- $1 \leq \text{homepage.length} \leq 20$
- $1 \leq \text{url.length} \leq 20$
- $1 \leq \text{steps} \leq 100$
- homepage and url consist of '.' or lower case English letters.
- At most 5000 calls will be made to visit, back, and forward.

Community Solutions (Python)

- WalkCC

```
class BrowserHistory:
    def __init__(self, homepage: str):
        self.urls = []
        self.index = -1
        self.lastIndex = -1
        self.visit(homepage)

    def visit(self, url: str) -> None:
        self.index += 1
        if self.index < len(self.urls):
            self.urls[self.index] = url
        else:
            self.urls.append(url)
        self.lastIndex = self.index

    def back(self, steps: int) -> str:
        self.index = max(0, self.index - steps)
        return self.urls[self.index]

    def forward(self, steps: int) -> str:
        self.index = min(self.lastIndex, self.index + steps)
        return self.urls[self.index]

class BrowserHistory:
    def __init__(self, homepage: str):
        self.history = []
        self.visit(homepage)

    def visit(self, url: str) -> None:
        self.history.append(url)
        self.future = []

    def back(self, steps: int) -> str:
```

```

while len(self.history) > 1 and steps > 0:
    self.future.append(self.history.pop())
    steps -= 1
return self.history[-1]

def forward(self, steps: int) -> str:
    while self.future and steps > 0:
        self.history.append(self.future.pop())
        steps -= 1
    return self.history[-1]

class Node:
    def __init__(self, url: str):
        self.prev = None
        self.next = None
        self.url = url

class BrowserHistory:
    def __init__(self, homepage: str):
        self.curr = Node(homepage)

    def visit(self, url: str) -> None:
        self.curr.next = Node(url)
        self.curr.next.prev = self.curr
        self.curr = self.curr.next

    def back(self, steps: int) -> str:
        while self.curr.prev and steps > 0:
            self.curr = self.curr.prev
            steps -= 1

        return self.curr.url

    def forward(self, steps: int) -> str:
        while self.curr.next and steps > 0:
            self.curr = self.curr.next
            steps -= 1
        return self.curr.url

```

• LeetDoocs

```
class BrowserHistory:
    def __init__(self, homepage: str):
        self.stk1 = []
        self.stk2 = []
        self.visit(homepage)

    def visit(self, url: str) -> None:
        self.stk1.append(url)
        self.stk2.clear()

    def back(self, steps: int) -> str:
        while steps and len(self.stk1) > 1:
            self.stk2.append(self.stk1.pop())
            steps -= 1
        return self.stk1[-1]

    def forward(self, steps: int) -> str:
        while steps and self.stk2:
            self.stk1.append(self.stk2.pop())
            steps -= 1

        return self.stk1[-1]
```

```
# Your BrowserHistory object will be instantiated and called as such:
# obj = BrowserHistory(homepage)
# obj.visit(url)
# param_2 = obj.back(steps)
# param_3 = obj.forward(steps)
```

• SimplyLeet

```
# Python Implementation of BrowserHistory

class BrowserHistory:
    def __init__(self, homepage: str):
        # Initialize history with the homepage and set current index to 0
        self.history = [homepage]
        self.curr = 0

    def visit(self, url: str) -> None:
        # Clear forward history by slicing the current history up to and
        # including current page

        self.history = self.history[:self.curr + 1]
        # Append the new URL to history
        self.history.append(url)
        # Move current pointer to the end of the history
        self.curr += 1

    def back(self, steps: int) -> str:
        # Move the pointer 'steps' back but don't go below index 0
        self.curr = max(0, self.curr - steps)
        # Return the URL at the new current position

        return self.history[self.curr]

    def forward(self, steps: int) -> str:
        # Move the pointer 'steps' forward but don't exceed the last index in
        # history
        self.curr = min(len(self.history) - 1, self.curr + steps)
        # Return the URL at the new current position
        return self.history[self.curr]
```

◆ Quick Review

Key Insights

- Use a list (or similar data structure) to store the browsing history.

- Maintain an index pointer to track the current page.
- When visiting a URL, remove all forward history beyond the current page.
- For the back operation, decrement the pointer by the given steps, ensuring it does not go below 0.
- For the forward operation, increment the pointer by the given steps, ensuring it does not exceed the last visited page.

Space & Time

Time Complexity:

- visit: $O(1)$ on average (slicing the forward history may take $O(n)$ in worst-case but steps are bounded).
- back and forward: $O(1)$ per operation.

Space Complexity:

- $O(n)$ where n is the number of visited URLs.

Solution

We maintain an array (or list) for the history and an integer pointer to mark the current page.

When visiting a new URL, we remove any forward history, append the new URL, and update the pointer.

For back, we decrement the pointer ensuring it does not go below zero; for forward, we increase the pointer ensuring it does not exceed the available history.

This approach ensures that all operations run efficiently with minimal overhead.

Data Structure Design

□ #2353 Design a Food Rating System

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Design · Heap
(Priority Queue) · Ordered Set
Lists: AlgoMaster 600

Problem

Design a food rating system that can do the following:

- Modify the rating of a food item listed in the system.
- Return the highest-rated food item for a type of cuisine in the system.

Implement the FoodRatings class:

- FoodRatings(String[] foods, String[] cuisines, int[] ratings) Initializes the system. The food items are described by foods, cuisines and ratings, all of which have a length of n. foods[i] is the name of the ith food,
- cuisines[i] is the type of cuisine of the ith food, and

• `ratings[i]` is the initial rating of the *i*th food. Note that a string *x* is lexicographically smaller than string *y* if *x* comes before *y* in dictionary order, that is, either *x* is a prefix of *y*, or if *i* is the first position such that *x*[*i*] $\neq$ *y*[*i*], then *x*[*i*] comes before *y*[*i*] in alphabetic order.

Example 1:

Input

```
["FoodRatings", "highestRated", "highestRated", "changeRating",
"highestRated", "changeRating", "highestRated"]
[[["kimchi", "miso", "sushi", "moussaka", "ramen", "bulgogi"], ["korean",
"japanese", "japanese", "greek", "japanese", "korean"], [9, 12, 8, 15, 14, 7]],
["korean"], ["japanese"], ["sushi", 16], ["japanese"], ["ramen", 16],
["japanese"]]
```

Output

```
[null, "kimchi", "ramen", null, "sushi", null, "ramen"]
```

Explanation

```
FoodRatings foodRatings = new FoodRatings(["kimchi", "miso", "sushi",
"moussaka", "ramen", "bulgogi"], ["korean", "japanese", "japanese", "greek",
"japanese", "korean"], [9, 12, 8, 15, 14, 7]);
```

Constraints:

- $1 \leq n \leq 2 * 10^4$
- $n == \text{foods.length} == \text{cuisines.length} == \text{ratings.length}$
- $1 \leq \text{foods}[i].\text{length}, \text{cuisines}[i].\text{length} \leq 10$
- `foods[i]`, `cuisines[i]` consist of lowercase English letters.
- $1 \leq \text{ratings}[i] \leq 10^8$

- All the strings in foods are distinct.
- food will be the name of a food item in the system across all calls to changeRating.
- cuisine will be a type of cuisine of at least one food item in the system across all calls to highestRated.
- At most $2 * 10^4$ calls in total will be made to changeRating and highestRated.

Community Solutions (Python)

• WalkCC

```
from sortedcontainers import SortedSet

class FoodRatings:
    def __init__(self, foods: list[str], cuisines: list[str], ratings: list[int]):
        self.cuisineToRatingAndFoods = collections.defaultdict(
            lambda: SortedSet(key=lambda x: (-x[0], x[1])))
        self.foodToCuisine = {}
        self.foodToRating = {}

        for food, cuisine, rating in zip(foods, cuisines, ratings):
            self.cuisineToRatingAndFoods[cuisine].add((rating, food))
            self.foodToCuisine[food] = cuisine
            self.foodToRating[food] = rating

        def changeRating(self, food: str, newRating: int) -> None:
            cuisine = self.foodToCuisine[food]
            oldRating = self.foodToRating[food]
            ratingAndFoods = self.cuisineToRatingAndFoods[cuisine]
            ratingAndFoods.remove((oldRating, food))
```

```

ratingAndFoods.add((newRating, food))
self.foodToRating[food] = newRating

def highestRated(self, cuisine: str) -> str:
    return self.cuisineToRatingAndFoods[cuisine][0][1]

```

• LeetDoocs

```

class FoodRatings:

    def __init__(self, foods: List[str], cuisines: List[str], ratings: List[int]):
        self.d = defaultdict(SortedList)
        self.g = {}
        for food, cuisine, rating in zip(foods, cuisines, ratings):
            self.d[cuisine].add((-rating, food))
            self.g[food] = (rating, cuisine)

    def changeRating(self, food: str, newRating: int) -> None:

        oldRating, cuisine = self.g[food]
        self.g[food] = (newRating, cuisine)
        self.d[cuisine].remove((-oldRating, food))
        self.d[cuisine].add((-newRating, food))

    def highestRated(self, cuisine: str) -> str:
        return self.d[cuisine][0][1]

# Your FoodRatings object will be instantiated and called as such:

# obj = FoodRatings(foods, cuisines, ratings)
# obj.changeRating(food,newRating)
# param_2 = obj.highestRated(cuisine)

```

• SimplyLeet

```

import heapq

class FoodRatings:
    def __init__(self, foods, cuisines, ratings):
        # Map food to its current rating.
        self.food_to_rating = {}
        # Map food to its cuisine.
        self.food_to_cuisine = {}
        # Map each cuisine to a heap of (-rating, food) pairs.
        self.cuisine_to_heap = {}

    for food, cuisine, rating in zip(foods, cuisines, ratings):
        self.food_to_rating[food] = rating
        self.food_to_cuisine[food] = cuisine
        if cuisine not in self.cuisine_to_heap:
            self.cuisine_to_heap[cuisine] = []
        # Store negative rating to simulate a max-heap; food name ensures
        # lexicographic order.
        heapq.heappush(self.cuisine_to_heap[cuisine], (-rating, food))

    def changeRating(self, food, newRating):
        # Retrieve corresponding cuisine.
        cuisine = self.food_to_cuisine[food]
        # Update the food's rating.
        self.food_to_rating[food] = newRating
        # Push the new rating into the cuisine heap.
        heapq.heappush(self.cuisine_to_heap[cuisine], (-newRating, food))

    def highestRated(self, cuisine):
        heap = self.cuisine_to_heap[cuisine]
        # Remove stale entries until the top of the heap is valid.

        while heap:
            rating_neg, food = heap[0]
            if -rating_neg == self.food_to_rating[food]:
                return food
            heapq.heappop(heap)
        return ""

```

◆ Quick Review

Key Insights

- Use a hash map to store each food's current rating and cuisine.
- Maintain a max heap for each cuisine to retrieve the highest-rated food quickly.
- Store tuples of $(-rating, food)$ in the heap. The negative rating turns the min-heap (default in many languages) into a max-heap.
- Handle rating updates with lazy deletion: insert the updated rating into the heap and, during retrieval, discard stale entries whose rating does not match the latest value in the hash map.
- Lexicographical order is naturally supported by storing the food name in the tuple and relying on tuple comparison.

Space & Time

Time Complexity:

- `changeRating`: $O(\log n)$ due to heap insertion.
- `highestRated`: $O(\log n)$ amortized as stale entries are lazily removed from the heap.

Space Complexity: $O(n)$ for storing the maps and heaps.

Solution

We create three primary data structures:

- A map from food to its current rating (foodToRating).
- A map from food to its cuisine (foodToCuisine).
- A map from cuisine to a max heap (cuisineToHeap) that stores pairs of (–rating, food).

When a food's rating is updated, we update the foodToRating map and push the new (–rating, food) tuple onto the appropriate heap. When a query for the highest-rated food is made, we repeatedly check the top of the heap for that cuisine. If the top entry's rating (after sign inversion) matches the current rating in foodToRating, it is returned; otherwise, we pop the stale value and continue. This lazy deletion avoids the overhead of directly removing outdated entries from the heap.

Greedy

Round 5 · Mid · Medium · 8 questions

Greedy

□ #406 Queue Reconstruction by Height

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Indexed Tree · Segment Tree ·
Sorting

Lists: AlgoMaster 600

Problem

You are given an array of people, `people`, which are the attributes of some people in a queue (not necessarily in order). Each `people[i] = [hi, ki]` represents the *i*th person of height *hi* with exactly *ki* other people in front who have a height greater than or equal to *hi*.

Reconstruct and return the queue that is represented by the input array `people`. The returned queue should be formatted as an array `queue`, where `queue[j] = [hj, kj]` is the attributes of the *j*th person in the queue (`queue[0]` is the person at the front of the queue).

Example 1:

Input: people = [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]

Output: [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]

Explanation:

Person 0 has height 5 with no other people taller or the same height in front.

Person 1 has height 7 with no other people taller or the same height in front.

Person 2 has height 5 with two persons taller or the same height in front, which is person 0 and 1.

Person 3 has height 6 with one person taller or the same height in front, which is person 1.

Person 4 has height 4 with four people taller or the same height in front, which are people 0, 1, 2, and 3.

Example 2:

Input: people = [[6,0],[5,0],[4,0],[3,2],[2,2],[1,4]]

Output: [[4,0],[5,0],[2,2],[3,2],[1,4],[6,0]]

Constraints:

- $1 \leq \text{people.length} \leq 2000$
- $0 \leq \text{hi} \leq 106$
- $0 \leq \text{ki} < \text{people.length}$
- It is guaranteed that the queue can be reconstructed.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reconstructQueue(self, people: list[list[int]]) -> list[list[int]]:
        ans = []

        people.sort(key=lambda x: (-x[0], x[1]))

        for person in people:
            ans.insert(person[1], person)

        return ans
```

• LeetCode

```
class Solution:
    def reconstructQueue(self, people: List[List[int]]) -> List[List[int]]:
        people.sort(key=lambda x: (-x[0], x[1]))
        ans = []
        for p in people:
            ans.insert(p[1], p)
        return ans
```

• SimplyLeet

Python solution with line-by-line comments

```
class Solution:
    def reconstructQueue(self, people):
        # Sort people: first by descending height; if heights are equal, sort
        # by ascending k value.
        people.sort(key=lambda person: (-person[0], person[1]))

        # Initialize result list
        queue = []

        # Insert each person in the list at the index equal to their k-value.
        for person in people:
            queue.insert(person[1], person) # person[1] is the k value
        indicating the index
        return queue

# Example usage:
# solution = Solution()
# print(solution.reconstructQueue([[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]))
```

◆ Quick Review

Key Insights

- Sort the array in descending order by height; for people with equal height, sort in ascending

order by k .

- When inserting a person into the result list, their k value indicates the index at which they should be placed.
- This greedy approach guarantees that each person is inserted after all taller or equal height persons have already been placed.

Space & Time

Time Complexity: $O(n^2)$ in the worst case due to the insertion process in the list.

Space Complexity: $O(n)$ for storing the output queue.

Solution

The solution leverages a greedy algorithm.

First, sort the input array such that taller people come first and, for the same height, the person with a smaller k comes first. By doing this, when you iterate over the sorted list and insert each person at the index equal to their k value, the relative order of people already inserted (who are taller or of the same height) remains valid, fulfilling the condition of exactly k people in front who are taller or equal.

Greedy

□ #670 Maximum Swap

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Greedy

Lists: AlgoMaster 600

Problem

You are given an integer num. You can swap two digits at most once to get the maximum valued number.

Return the maximum valued number you can get.

Example 1:

Input: num = 2736
Output: 7236
Explanation: Swap the number 2 and the number 7.

Example 2:

Input: num = 9973
Output: 9973
Explanation: No swap.

Constraints:

- $0 \leq \text{num} \leq 10^8$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maximumSwap(self, num: int) -> int:
        s = list(str(num))
        dict = {c: i for i, c in enumerate(s)}

        for i, c in enumerate(s):
            for digit in reversed(string.digits):
                if digit <= c:
                    break
                if digit in dict and dict[digit] > i:

                    s[i], s[dict[digit]] = digit, s[i]
                    return int(''.join(s))

        return num
```

• LeetDoocs

```
class Solution:
    def maximumSwap(self, num: int) -> int:
        s = list(str(num))
        n = len(s)
        d = list(range(n))
        for i in range(n - 2, -1, -1):
            if s[i] <= s[d[i + 1]]:
                d[i] = d[i + 1]
        for i, j in enumerate(d):
            if s[i] < s[j]:
                s[i], s[j] = s[j], s[i]
                break
        return int(''.join(s))
```

• SimplyLeet

```
def maximumSwap(num):
    # Convert number to a list of characters (digits)
    digits = list(str(num))
    # Create a mapping of the last occurrence of each digit
    last = {int(digit): index for index, digit in enumerate(digits)}

    # Iterate over the digits from left to right
    for i, digit in enumerate(digits):
        # Check for a digit larger than the current one from 9 down to
        current+1
        for d in range(9, int(digit), -1):

            # If a larger digit appears later in the number, swap and return
            result
            if last.get(d, -1) > i:
                digits[i], digits[last[d]] = digits[last[d]], digits[i]
                return int("".join(digits))

    return num

# Test example
print(maximumSwap(2736)) # Expected output: 7236
```

◆ Quick Review

Key Insights

- Use a greedy strategy to get the highest value number.
- Record the last occurrence index of each digit (0 through 9) in the number.
- Iterate through the digits. For each digit, check if there is a larger digit available later in the number.
- Swap the current digit with the rightmost occurrence of the larger digit if it makes the

entire number larger.

- Only one swap is allowed, so the first valid swap is the optimal solution.

Space & Time

Time Complexity: $O(n)$ where n is the number of digits (since we scan the digits a constant number of times).

Space Complexity: $O(n)$ for storing the digits and the occurrence mapping.

Solution

We first convert the input number into a list (or equivalent structure) of digits. Then, we create a mapping of each digit (0–9) to its last occurrence index in the list. The key idea is that for each digit (from leftmost to rightmost), we look for a larger digit that occurs later. Starting from 9 down to one greater than the current digit, we check if such a digit exists in the mapping at an index beyond the current position. If found, we swap and return the new number immediately. If no such digit is found for any position, the number is already maximized, so we return the original number.

Greedy

□ #826 Most Profit Assigning Work

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search · Greedy ·
Sorting

Lists: AlgoMaster 600

Problem

You have n jobs and m workers. You are given three arrays: `difficulty`, `profit`, and `worker` where:

- `difficulty[i]` and `profit[i]` are the difficulty and the profit of the i th job, and
- `worker[j]` is the ability of j th worker (i.e., the j th worker can only complete a job with difficulty at most `worker[j]`).

Every worker can be assigned at most one job, but one job can be completed multiple times.

- For example, if three workers attempt the same job that pays \$1, then the total profit will be \$3. If a worker cannot complete any job, their profit is \$0.

Return the maximum profit we can achieve after assigning the workers to the jobs.

Example 1:

Input: difficulty = [2,4,6,8,10], profit = [10,20,30,40,50], worker = [4,5,6,7]
 Output: 100
 Explanation: Workers are assigned jobs of difficulty [4,4,6,6] and they get a profit of [20,20,30,30] separately.

Example 2:

Input: difficulty = [85,47,57], profit = [24,66,99], worker = [40,25,25]
 Output: 0

Constraints:

- $n == \text{difficulty.length}$
- $n == \text{profit.length}$
- $m == \text{worker.length}$
- $1 \leq n, m \leq 104$
- $1 \leq \text{difficulty}[i], \text{profit}[i], \text{worker}[i] \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProfitAssignment(
        self,
        difficulty: list[int],
        profit: list[int],
        worker: list[int],
    ) -> int:
        ans = 0
        jobs = sorted(zip(difficulty, profit))
        worker.sort(reverse=1)
```

```

i = 0
maxProfit = 0

for w in sorted(worker):
    while i < len(jobs) and w >= jobs[i][0]:
        maxProfit = max(maxProfit, jobs[i][1])
        i += 1
    ans += maxProfit

return ans

```

• LeetDoocs

```

class Solution:
    def maxProfitAssignment(
        self, difficulty: List[int], profit: List[int], worker: List[int]
    ) -> int:
        worker.sort()
        jobs = sorted(zip(difficulty, profit))
        ans = mx = i = 0
        for w in worker:
            while i < len(jobs) and jobs[i][0] <= w:
                mx = max(mx, jobs[i][1])
                i += 1
        ans += mx
        return ans

```

• SimplyLeet

```

# Python solution with detailed comments

import bisect

def maxProfitAssignment(difficulty, profit, worker):
    # Pair up difficulty and profit for jobs and sort by difficulty
    jobs = sorted(zip(difficulty, profit))

    max_profit_up_to = []
    current_max = 0

```

```

# Create a list of difficulties and update max profit seen so far.
difficulties = []
for d, p in jobs:
    current_max = max(current_max, p)
    difficulties.append(d)
    max_profit_up_to.append(current_max)

total_profit = 0
# For each worker, binary search for the most difficult job they can work
on
for ability in worker:

    # bisect_right returns insertion point, subtract 1 to get valid index
    idx = bisect.bisect_right(difficulties, ability) - 1
    if idx >= 0:
        total_profit += max_profit_up_to[idx]
return total_profit

# Example usage:
# print(maxProfitAssignment([2,4,6,8,10], [10,20,30,40,50], [4,5,6,7]))

```

◆ Quick Review

Key Insights

- Pair each job's difficulty and profit and sort them by difficulty.
- Compute a running maximum of profit for increasing difficulties; this allows you to quickly determine the best profit available for any worker ability.
- For each worker, use binary search in the sorted job list to find the highest difficulty job they can perform.

- Sum the best profits for all workers, handling workers that cannot perform any job by adding zero.

Space & Time

Time Complexity: $O(n \log n + m \log n)$, where n is the number of jobs and m is the number of workers.

Space Complexity: $O(n)$, used for storing and processing the sorted job pairs and the running max profit list.

Solution

The solution involves first pairing up job difficulties with their respective profits and then sorting these pairs based on difficulty. As you process this sorted list, maintain a running maximum profit to ensure that at any given difficulty level, you know the best profit you can earn from any job up to that difficulty. For each worker, perform a binary search on the sorted job difficulties to quickly determine the best profit that the worker can achieve. By summing these values for all workers, you achieve the maximum total profit. This algorithm leverages sorting, prefix maximum,

and binary search to efficiently compute the result.

Greedy

□ #921 Minimum Add to Make Parentheses Valid **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Greedy
Lists: AlgoMaster 600

Problem

A parentheses string is valid if and only if:

- It is the empty string,
- It can be written as AB (A concatenated with B), where A and B are valid strings, or
- It can be written as (A), where A is a valid string.

You are given a parentheses string *s*. In one move, you can insert a parenthesis at any position of the string.

- For example, if *s* = "())", you can insert an opening parenthesis to be "(())" or a closing parenthesis to be "())".

Return the minimum number of moves required to make *s* valid.

Example 1:

Input: s = "()")
Output: 1

Example 2:

Input: s = "((("
Output: 3

Constraints:

- $1 \leq s.length \leq 1000$
- s[i] is either '(' or ')'

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minAddToMakeValid(self, s: str) -> int:
        l = 0
        r = 0

        for c in s:
            if c == '(':
                l += 1
            else:
                if l == 0:
                    r += 1
                else:
                    l -= 1

        return l + r
```

• LeetDoocs


```
class Solution:
    def minAddToMakeValid(self, s: str) -> int:
        stk = []
        for c in s:
            if c == ')' and stk and stk[-1] == '(':
                stk.pop()
            else:
                stk.append(c)
        return len(stk)
```

```
class Solution:
    def minAddToMakeValid(self, s: str) -> int:
        ans = cnt = 0
        for c in s:
            if c == '(':
                cnt += 1
            elif cnt:
                cnt -= 1
            else:
                ans += 1
```

```
ans += cnt
return ans
```

• SimplyLeet

```
# Function to determine the minimum moves required
def minAddToMakeValid(s):
    moves = 0 # Counts the required insertions for unmatched ')'
    count = 0 # Counts unmatched '(' characters
    for char in s:
        if char == '(':
            count += 1 # Increase count for an open parenthesis
        elif char == ')':
            # If no matching open, it's an error, need one move to balance
            if count == 0:
```

```
        moves += 1
    else:
        count -= 1 # Found a matching pair
# Add moves needed for any leftover unmatched '('
return moves + count

# Example usage:
print(minAddToMakeValid("()")) # Expected output: 1
print(minAddToMakeValid("(((") # Expected output: 3
```

◆ Quick Review

Key Insights

- Use a counter to track the number of unmatched '(' characters.
- When encountering a ')', if there is no preceding unmatched '(', it indicates an imbalance that requires inserting an '('.
- After processing the entire string, any remaining unmatched '(' will require inserting the same number of closing parentheses.
- The total moves is the sum of insertions needed for unmatched ')' and unmatched '('.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(1)$

Solution

The solution iterates through the entire string using a single pass. A counter (named `count`) is incremented for every '(' encountered, representing an imbalance. When a ')' is encountered, if the counter is greater than zero, it implies a matching '(' exists, so the counter is decremented; if not, it indicates an unmatched ')' and requires an insertion, so a moves counter is incremented. After processing all characters, any remaining count of '(' represents unmatched openings, and the same number of moves must be added. This greedy approach ensures we account for all imbalances with a minimum number of moves using constant space.

Greedy

□ #1488 Avoid Flood in The City

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Binary Search · Greedy ·
Heap (Priority Queue)
Lists: AlgoMaster 600

Problem

Your country has 109 lakes. Initially, all the lakes are empty, but when it rains over the n th lake, the n th lake becomes full of water. If it rains over a lake that is full of water, there will be a flood. Your goal is to avoid floods in any lake.

Given an integer array rains where:

- $\text{rains}[i] > 0$ means there will be rains over the $\text{rains}[i]$ lake.
- $\text{rains}[i] == 0$ means there are no rains this day and you must choose one lake this day and dry it.

Return an array ans where:

- $\text{ans.length} == \text{rains.length}$

- $\text{ans}[i] == -1$ if $\text{rains}[i] > 0$.
- $\text{ans}[i]$ is the lake you choose to dry in the i th day if $\text{rains}[i] == 0$.

If there are multiple valid answers return any of them. If it is impossible to avoid flood return an empty array.

Notice that if you chose to dry a full lake, it becomes empty, but if you chose to dry an empty lake, nothing changes.

Example 1:

```
Input: rains = [1,2,3,4]
Output: [-1,-1,-1,-1]
Explanation: After the first day full lakes are [1]
After the second day full lakes are [1,2]
After the third day full lakes are [1,2,3]
After the fourth day full lakes are [1,2,3,4]
There's no day to dry any lake and there is no flood in any lake.
```

Example 2:

```
Input: rains = [1,2,0,0,2,1]
Output: [-1,-1,2,1,-1,-1]
Explanation: After the first day full lakes are [1]
After the second day full lakes are [1,2]
After the third day, we dry lake 2. Full lakes are [1]
After the fourth day, we dry lake 1. There is no full lakes.
After the fifth day, full lakes are [2].
After the sixth day, full lakes are [1,2].
```

Example 3:

```
Input: rains = [1,2,0,1,2]
Output: []
Explanation: After the second day, full lakes are [1,2]. We have to dry one lake in the third day.
After that, it will rain over lakes [1,2]. It's easy to prove that no matter which lake you choose to dry in the 3rd day, the other one will flood.
```

Constraints:

- $1 \leq \text{rains.length} \leq 105$
- $0 \leq \text{rains}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
from sortedcontainers import SortedSet

class Solution:
    def avoidFlood(self, rains: list[int]) -> list[int]:
        ans = [-1] * len(rains)
        lakeIdToFullDay = {}
        emptyDays = SortedSet() # indices of rains[i] == 0

        for i, lakeId in enumerate(rains):

            if lakeId == 0:
                emptyDays.add(i)
                continue
            # The lake was full in a previous day. Greedily find the closest day
            # to make the lake empty.
            if lakeId in lakeIdToFullDay:
                fullDay = lakeIdToFullDay[lakeId]
                emptyDayIndex = emptyDays.bisect_right(fullDay)
                if emptyDayIndex == len(emptyDays): # Not found.
                    return []

            # Empty the lake at this day.
            emptyDay = emptyDays[emptyDayIndex]
            ans[emptyDay] = lakeId
            emptyDays.discard(emptyDay)
            # The lake with `lakeId` becomes full at the day `i`.
            lakeIdToFullDay[lakeId] = i

        # Empty an arbitrary lake if there are remaining empty days.
        for emptyDay in emptyDays:
            ans[emptyDay] = 1
```

```
return ans
```

• LeetCode

```
class Solution:
    def avoidFlood(self, rains: List[int]) -> List[int]:
        n = len(rains)
        ans = [-1] * n
        sunny = SortedList()
        rainy = {}
        for i, v in enumerate(rains):
            if v:
                if v in rainy:
                    idx = sunny.bisect_right(rainy[v])

                    if idx == len(sunny):
                        return []
                    ans[sunny[idx]] = v
                    sunny.discard(sunny[idx])
                rainy[v] = i
            else:
                sunny.add(i)
                ans[i] = 1
        return ans
```

• SimplyLeet

```
# Import bisect for binary search operations on the sorted list of dry day
indices
import bisect

def avoidFlood(rains):
    n = len(rains)
    result = [-1] * n # By default, set rainy day result to -1; dry day will
get updated
    lake_last_rain = {} # Dictionary to map lake to the last day it rained
    dry_days = [] # Sorted list to keep track of indices of available dry days

    for i, lake in enumerate(rains):
```

```

        if lake > 0: # It is raining over the lake
            if lake in lake_last_rain:
                # Find a dry day index that is greater than the last rain day
for 'lake'
                prev_rain_day = lake_last_rain[lake]
                idx = bisect.bisect_right(dry_days, prev_rain_day)
                if idx == len(dry_days):
                    # No available dry day to dry the lake before it rains
again -> flood
                    return []
                # Use the found dry day to dry this lake
                dry_day_index = dry_days[idx]

            result[dry_day_index] = lake # Dry the lake on that dry day
            dry_days.pop(idx) # Remove the used dry day from the available
list

            # Update the last rain day for this lake
            lake_last_rain[lake] = i
            result[i] = -1 # On rainy days, the result is always -1
        else:
            # It's a dry day; add index to the list of dry days (initially set
to arbitrary 1)
            bisect.insort(dry_days, i)
            result[i] = 1 # Default value, may be overridden later if the day
is used to dry a lake

    return result

# Example usage:
#print(avoidFlood([1,2,0,0,2,1]))

```

◆ Quick Review

Key Insights

- When it rains on a lake, mark that lake as full along with the current day index.

- If the same lake is rained upon again without being dried, a flood occurs.
- On a dry day ($\text{rains}[i] == 0$), choose to dry a lake that is scheduled to receive rain in the future and is currently full.
- Use a data structure to keep track of available dry days (and their indices) and search for a dry day that occurs after the previous rain for a given lake.
- A greedy algorithm with binary search (or a balanced tree structure) is a natural fit to make the optimal drying decision.

Space & Time

Time Complexity: $O(n \log n)$ – Each dry day lookup involves a binary search in the sorted list of dry day indices.

Space Complexity: $O(n)$ – To maintain the mapping of lakes to their last rain day and the list of available dry days.

Solution

The solution is based on a greedy strategy combined with binary search. We maintain:

- A dictionary (or hash table) to keep track of the last day each lake was rained upon.

- A sorted list (or tree set) to store indices of days where no rain occurs (dry days).

For each day:

- If it rains ($\text{rains}[i] > 0$):
- If the lake is already full (exists in our dictionary), we try to find the earliest dry day (using binary search) that is after the previous rain day for that lake.
- If such a dry day does not exist, a flood is inevitable, so we return an empty array.
- Otherwise, we assign that dry day to dry the lake and update our data structures.
- If it is a dry day, we simply add the day's index to our sorted list of available dry days and, by default, assign an arbitrary value (e.g., 1) in the result. This value will be overridden if this day gets used to dry a specific lake.

This approach ensures that we always use the best available dry day for a lake that might cause a future flood.

Greedy

□ #1717 Maximum Score From Removing Substrings

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Greedy
Lists: AlgoMaster 600

Problem

You are given a string s and two integers x and y . You can perform two types of operations any number of times.

- Remove substring "ab" and gain x points.
For example, when removing "ab" from "cabxbae" it becomes "cxbae".
- For example, when removing "ba" from "cabxbae" it becomes "cabxe".

Return the maximum points you can gain after applying the above operations on s .

Example 1:

Input: s = "cdbcbbaaabab", x = 4, y = 5

Output: 19

Explanation:

- Remove the "ba" underlined in "cdbcbbaaabab". Now, s = "cdbcbbaaab" and 5 points are added to the score.
 - Remove the "ab" underlined in "cdbcbbaaab". Now, s = "cdbcbbaa" and 4 points are added to the score.
 - Remove the "ba" underlined in "cdbcbbaa". Now, s = "cdbcba" and 5 points are added to the score.
 - Remove the "ba" underlined in "cdbcba". Now, s = "cdcb" and 5 points are added to the score.
- Total score = 5 + 4 + 5 + 5 = 19.

Example 2:

Input: s = "aabbaaxybbaabb", x = 5, y = 4

Output: 20

Constraints:

- $1 \leq s.length \leq 105$
- $1 \leq x, y \leq 104$
- s consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maximumGain(self, s: str, x: int, y: int) -> int:
        # The assumption that gain('ab') > gain('ba') while removing 'ba' first is
        # optimal is contradicted. Only 'b(ab)a' satisfies the condition of
        # preventing two 'ba' removals, but after removing 'ab', we can still
        # remove one 'ba', resulting in a higher gain. Thus, removing 'ba' first is
        # not optimal.
        return (self._gain(s, 'ab', x, 'ba', y) if x > y else
                self._gain(s, 'ba', y, 'ab', x))
```

```

# Returns the points gained by first removing sub1 ('ab' | 'ba') from s with
# point1, then removing sub2 ('ab' | 'ba') from s with point2.
def _gain(self, s: str, sub1: str, point1: int, sub2: str, point2: int) -> int:
    points = 0
    stack1 = []
    stack2 = []

    # Remove 'sub1' from s with point1 gain.
    for c in s:
        if stack1 and stack1[-1] == sub1[0] and c == sub1[1]:

            stack1.pop()
            points += point1
        else:
            stack1.append(c)

    # Remove 'sub2' from s with point2 gain.
    for c in stack1:
        if stack2 and stack2[-1] == sub2[0] and c == sub2[1]:
            stack2.pop()
            points += point2
        else:
            stack2.append(c)

    return points

```

• LeetDoocs

```

class Solution:
    def maximumGain(self, s: str, x: int, y: int) -> int:
        a, b = "a", "b"
        if x < y:
            x, y = y, x
            a, b = b, a
        ans = cnt1 = cnt2 = 0
        for c in s:
            if c == a:
                cnt1 += 1

```

```

        elif c == b:
            if cnt1:
                ans += x
                cnt1 -= 1
            else:
                cnt2 += 1
        else:
            ans += min(cnt1, cnt2) * y
            cnt1 = cnt2 = 0
    ans += min(cnt1, cnt2) * y

    return ans

```

• SimplyLeet

```

# Python solution with detailed comments
def maximumGain(s: str, x: int, y: int) -> int:
    total_points = 0

    # Decide which pair to remove first based on their points
    if x >= y:
        first_pair, first_score = "ab", x
        second_pair, second_score = "ba", y
    else:
        first_pair, first_score = "ba", y

    second_pair, second_score = "ab", x

    # First pass: remove first_pair using a greedy stack approach
    stack = []
    remaining_chars = []
    for char in s:
        if stack and (stack[-1] + char == first_pair):
            stack.pop() # Remove the matching pair from the stack
            total_points += first_score # Add score for this removal
        else:

```

```
        stack.append(char)
# The characters remaining after first pass
remaining_chars = stack

# Second pass: remove second_pair from the remaining characters
stack = []
for char in remaining_chars:
    if stack and (stack[-1] + char == second_pair):
        stack.pop() # removal of second pair
        total_points += second_score
    else:
        stack.append(char)

return total_points

# Example usage:
s = "cdbcbbaaabab"
x = 4
y = 5
print(maximumGain(s, x, y)) # Expected output: 19
```

◆ Quick Review

Key Insights

- The order of removal matters because once a substring is removed, the remaining parts of the string can combine to form new removable pairs.
- It is optimal to first remove the substring with the higher reward. This prevents potentially losing higher points if a higher rewarded pair is formed later.

- Use a greedy approach with a stack to efficiently remove substrings in one pass.
- Process the string twice:
- First pass: Remove the higher-value substring while using a stack to track characters.
- Second pass: On the resulting string, remove the lower-value substring.

Space & Time

Time Complexity: $O(n)$ per pass, hence overall $O(n)$.

Space Complexity: $O(n)$ in worst-case scenario for the stack.

Solution

We use a greedy algorithm with stacks:

- Determine which substring removal gives more points.
- For the substring with higher reward:
- Traverse the string, using a stack to check if the current character and the top of the stack form the high-reward pair.
- If they do, pop from the stack and add the respective points.

- For the remaining substring, repeat a similar process using a new stack.
- Sum the points from both passes to get the maximum score.

This approach ensures that we remove the most valuable pairs first without missing any formation of the less valuable pair later.

Greedy

□ #1871 Jump Game VII

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Sliding
Window · Prefix Sum
Lists: AlgoMaster 600

Problem

You are given a 0-indexed binary string s and two integers minJump and maxJump . In the beginning, you are standing at index 0, which is equal to '0'. You can move from index i to index j if the following conditions are fulfilled:

- $i + \text{minJump} \leq j \leq \min(i + \text{maxJump}, s.\text{length} - 1)$, and
- $s[j] == '0'$.

Return true if you can reach index $s.\text{length} - 1$ in s , or false otherwise.

Example 1:

```
Input: s = "011010", minJump = 2, maxJump = 3
Output: true
Explanation:
In the first step, move from index 0 to index 3.
In the second step, move from index 3 to index 5.
```

Example 2:

Input: s = "01101110", minJump = 2, maxJump = 3
Output: false

Constraints:

- $2 \leq s.length \leq 105$
- s[i] is either '0' or '1'.
- s[0] == '0'
- $1 \leq minJump \leq maxJump < s.length$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canReach(self, s: str, minJump: int, maxJump: int) -> bool:
        count = 0
        dp = [True] + [False] * (len(s) - 1)

        for i in range(minJump, len(s)):
            count += dp[i - minJump]
            if i - maxJump > 0:
                count -= dp[i - maxJump - 1]
            dp[i] = count and s[i] == '0'

        return dp[-1]
```

• LeetDoocs

```

class Solution:
    def canReach(self, s: str, minJump: int, maxJump: int) -> bool:
        n = len(s)
        pre = [0] * (n + 1)
        pre[1] = 1
        f = [True] + [False] * (n - 1)
        for i in range(1, n):
            if s[i] == "0":
                l, r = max(0, i - maxJump), i - minJump
                f[i] = l <= r and pre[r + 1] - pre[l] > 0

        pre[i + 1] = pre[i] + f[i]
        return f[-1]

```

• SimplyLeet

```

def canReach(s: str, minJump: int, maxJump: int) -> bool:
    n = len(s)
    # dp[i] is True if index i is reachable, False otherwise
    dp = [False] * n
    dp[0] = True

    # variable to keep track of the cumulative count of reachable indices in
    # the window
    reachable = 0
    for i in range(1, n):
        # If i is at least minJump steps ahead, add the dp value that enters
        # the window.

        if i >= minJump:
            reachable += dp[i - minJump]
        # If i is beyond maxJump, subtract the dp value that leaves the window.
        if i > maxJump:
            reachable -= dp[i - maxJump - 1]
        # Mark index i as reachable if s[i] is '0' and there is at least one
        # reachable index in the window.
        if s[i] == '0' and reachable > 0:
            dp[i] = True
    return dp[-1]

# Example usage:
print(canReach("011010", 2, 3)) # Expected output: True

```

◆ Quick Review

Key Insights

- A straightforward BFS or DFS would be too slow for large inputs due to overlapping subproblems.
- Use dynamic programming with a sliding window (prefix sum) to efficiently track the cumulative reachability.
- Instead of checking each jump individually, maintain a sliding window sum over the dp values that denote whether previous positions are reachable.
- When the sliding window moves, add the new dp value entering the window and subtract the dp value leaving the window.
- The cell is marked reachable only if it is '0' and there exists at least one previous reachable index within the jump range.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string.

Space Complexity: $O(n)$ for the dp array.

Solution

The solution uses a dynamic programming approach combined with a sliding window technique. We create a boolean array `dp` where `dp[i]` indicates whether index `i` is reachable. We initialize `dp[0]` as true because we start from index 0. A variable (`reachable`) is used to maintain the count of reachable indices in the current window. For each index `i`, if `i` is within `minJump` distance from a previous index, we add `dp[i - minJump]` to `reachable`. If the index is beyond `maxJump`, we subtract `dp[i - maxJump - 1]` from `reachable` as its contribution is no longer valid. If the current character `s[i]` is '0' and `reachable` is greater than zero, we mark `dp[i]` as true. Finally, `dp[s.length - 1]` gives the answer.

Greedy

□ #1975 Maximum Matrix Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Matrix
Lists: AlgoMaster 600

Problem

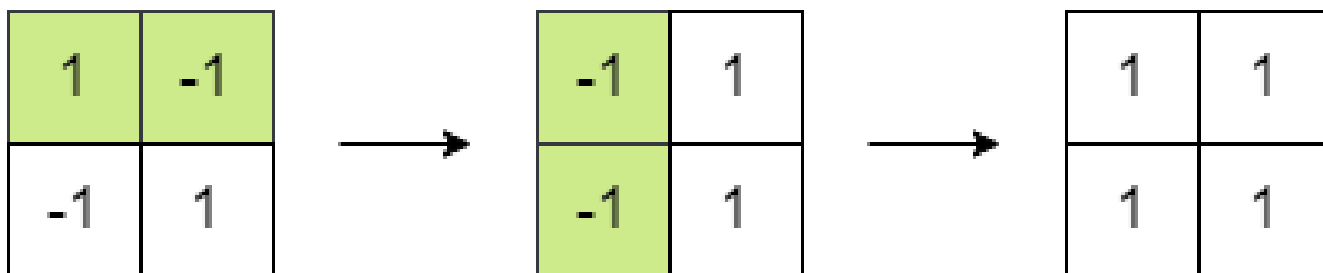
You are given an $n \times n$ integer matrix. You can do the following operation any number of times:

- Choose any two adjacent elements of matrix and multiply each of them by -1 .

Two elements are considered adjacent if and only if they share a border.

Your goal is to maximize the summation of the matrix's elements. Return the maximum sum of the matrix's elements using the operation mentioned above.

Example 1:



Input: matrix = [[1,-1],[-1,1]]

Output: 4

Explanation: We can follow the following steps to reach sum equals 4:

- Multiply the 2 elements in the first row by -1.
- Multiply the 2 elements in the first column by -1.

Example 2:

1	2	3
-1	-2	-3
1	2	3

→

1	2	3
-1	2	3
1	2	3

Input: matrix = [[1,2,3],[-1,-2,-3],[1,2,3]]

Output: 16

Explanation: We can follow the following step to reach sum equals 16:

- Multiply the 2 last elements in the second row by -1.

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $2 \leq n \leq 250$
- $-105 \leq \text{matrix}[i][j] \leq 105$

Community Solutions (Python)

- WalkCC


```

class Solution:
    def maxMatrixSum(self, matrix: list[list[int]]) -> int:
        absSum = 0
        minAbs = math.inf
        # 0 := even number of negatives
        # 1 := odd number of negatives
        oddNeg = 0

        for row in matrix:
            for num in row:

                absSum += abs(num)
                minAbs = min(minAbs, abs(num))
                if num < 0:
                    oddNeg ^= 1

        return absSum - oddNeg * minAbs * 2

```

• LeetCode

```

class Solution:
    def maxMatrixSum(self, matrix: List[List[int]]) -> int:
        mi = inf
        s = cnt = 0
        for row in matrix:
            for x in row:
                cnt += x < 0
                y = abs(x)
                mi = min(mi, y)
                s += y

        return s if cnt % 2 == 0 else s - mi * 2

```

• SimplyLeet

```
# Python solution for Maximum Matrix Sum

class Solution:
    def maxMatrixSum(self, matrix: List[List[int]]) -> int:
        total_sum = 0          # Sum of absolute values of all elements
        min_abs_value = float('inf') # Minimum absolute value in the matrix
        negative_count = 0     # Counter for negative numbers

        # Traverse through the matrix
        for row in matrix:

            for value in row:
                abs_value = abs(value)
                total_sum += abs_value    # Add the absolute value to total_sum
                min_abs_value = min(min_abs_value, abs_value) # Update
minimum absolute value
                if value < 0:
                    negative_count += 1 # Count negatives

            # If the number of negatives is odd, subtract twice the smallest
absolute value
            if negative_count % 2 != 0:
                total_sum -= 2 * min_abs_value

        return total_sum
```

◆ Quick Review

Key Insights

- Flipping the sign of two adjacent numbers does not affect the overall parity (even/odd nature) of the count of negative numbers.
- You can effectively change the signs in pairs, which means if the total count of negative numbers is even, you can transform all values

to positive.

- If the count of negative numbers is odd, one number will remain negative, so you want that to be the smallest (by absolute value) to minimize the reduction in the maximum sum.
- The answer thus becomes the sum of the absolute values of all elements, adjusted by subtracting twice the smallest absolute value if an odd number of negatives exists.

Space & Time

Time Complexity: $O(n^2)$ – We traverse every element in the matrix once.

Space Complexity: $O(1)$ – Only constant extra space is needed (aside from input storage).

Solution

The approach leverages a greedy strategy:

- Iterate through the matrix and compute the sum of the absolute values of each element.
- Count the number of negative numbers and track the smallest absolute value.
- If the count of negatives is even, all negatives can be flipped to positive, so the maximum sum is the sum of absolute values.

- If the count of negatives is odd, one number will remain negative after optimal operations. To maximize the sum, choose the smallest absolute value to be negative. Therefore, subtract twice this minimum value from the total sum of absolute values.
- Use simple loops and arithmetic to achieve the solution.

Topological Sort

Round 5 · Mid · Medium · 3 questions

Topological Sort

□ #802 Find Eventual Safe States

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Depth-First Search · Breadth-First Search ·
Graph Theory · Topological Sort
Lists: AlgoMaster 600

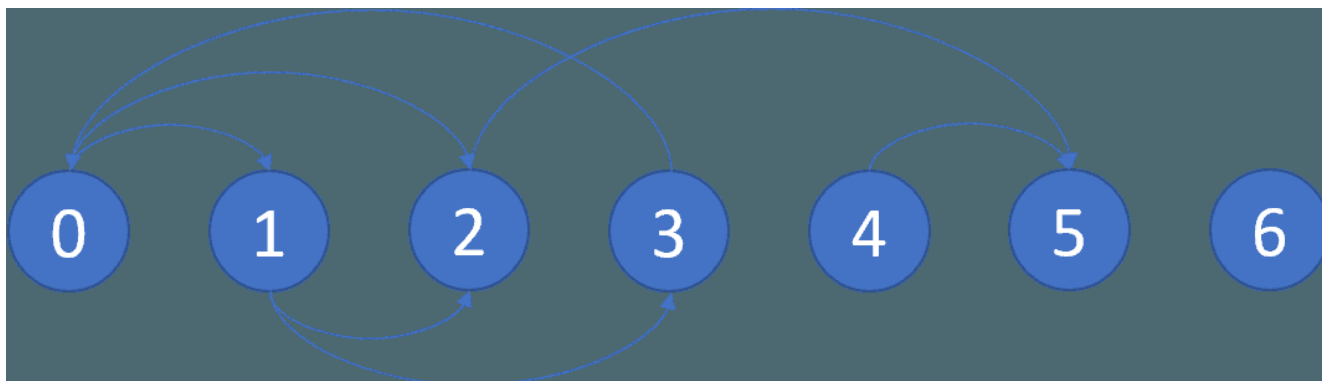
Problem

There is a directed graph of n nodes with each node labeled from 0 to $n - 1$. The graph is represented by a 0-indexed 2D integer array `graph` where `graph[i]` is an integer array of nodes adjacent to node i , meaning there is an edge from node i to each node in `graph[i]`.

A node is a terminal node if there are no outgoing edges. A node is a safe node if every possible path starting from that node leads to a terminal node (or another safe node).

Return an array containing all the safe nodes of the graph. The answer should be sorted in ascending order.

Example 1:



Input: graph = [[1,2],[2,3],[5],[0],[5],[],[[]]]

Output: [2,4,5,6]

Explanation: The given graph is shown above.

Nodes 5 and 6 are terminal nodes as there are no outgoing edges from either of them.

Every path starting at nodes 2, 4, 5, and 6 all lead to either node 5 or 6.

Example 2:

Input: graph = [[1,2,3,4],[1,2],[3,4],[0,4],[[]]]

Output: [4]

Explanation:

Only node 4 is a terminal node, and every path starting at node 4 leads to node 4.

Constraints:

- $n == \text{graph.length}$
- $1 \leq n \leq 104$
- $0 \leq \text{graph}[i].\text{length} \leq n$
- $0 \leq \text{graph}[i][j] \leq n - 1$
- graph[i] is sorted in a strictly increasing order.
- The graph may contain self-loops.
- The number of edges in the graph will be in the range [1, 4 * 104].

Community Solutions (Python)

• WalkCC

```
from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:

    def eventualSafeNodes(self, graph: list[list[int]]) -> list[int]:
        states = [State.INIT] * len(graph)

        def hasCycle(u: int) -> bool:
            if states[u] == State.VISITING:
                return True
            if states[u] == State.VISITED:
                return False
            states[u] = State.VISITING
            if any(hasCycle(v) for v in graph[u]):
                return True
            states[u] = State.VISITED
            return False

        return [i for i in range(len(graph)) if not hasCycle(i)]
```

• LeetDoocs


```

class Solution:
    def eventualSafeNodes(self, graph: List[List[int]]) -> List[int]:
        rg = defaultdict(list)
        indeg = [0] * len(graph)
        for i, vs in enumerate(graph):
            for j in vs:
                rg[j].append(i)
            indeg[i] = len(vs)
        q = deque([i for i, v in enumerate(indeg) if v == 0])
        while q:
            i = q.popleft()
            for j in rg[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return [i for i, v in enumerate(indeg) if v == 0]

```

```

class Solution:
    def eventualSafeNodes(self, graph: List[List[int]]) -> List[int]:
        def dfs(i):
            if color[i]:
                return color[i] == 2
            color[i] = 1
            for j in graph[i]:
                if not dfs(j):
                    return False
            color[i] = 2
        return True

        n = len(graph)
        color = [0] * n
        return [i for i in range(n) if dfs(i)]

```

• SimplyLeet

```
# Python solution using DFS and state tracking.
# States: 0 = unvisited, 1 = visiting, 2 = safe.
class Solution:
    def eventualSafeNodes(self, graph: List[List[int]]) -> List[int]:
        n = len(graph)
        state = [0] * n # Initialize all nodes as unvisited.

        def dfs(node):
            # If node is already processed, return True if safe.
            if state[node] != 0:

                return state[node] == 2
            state[node] = 1 # Mark node as visiting.
            for neighbor in graph[node]:
                # If any neighbor leads to a cycle, node is unsafe.
                if not dfs(neighbor):
                    return False
            state[node] = 2 # All neighbors are safe, mark node as safe.
            return True

        safe_nodes = []

        for i in range(n):
            if dfs(i):
                safe_nodes.append(i)
        return safe_nodes
```

◆ Quick Review

Key Insights

- **Identify cycles:** Nodes involved in cycles are unsafe because there exists a path that does not terminate.
- **DFS with state tracking:** Use three states for each node (unvisited, visiting, and safe) to detect cycles.

- **Memorization:** Once a node's safety is determined, reuse this result to avoid redundant computations.
- **Alternate approach:** Reverse the graph and perform a topological sort, though DFS is more straightforward here.

Space & Time

Time Complexity: $O(V + E)$ where V is the number of nodes and E is the number of edges.

Space Complexity: $O(V)$ due to the recursion stack and additional storage for tracking node states.

Solution

We perform a depth-first search (DFS) and use an array to mark each node with one of three states:

- 0: unvisited
- 1: visiting (currently in recursion stack)
- 2: safe (confirmed safe node)

For each node, if during DFS a node is encountered that is already marked as visiting, a cycle is detected and the node is unsafe. If all

reachable neighbors of a node are safe, then the node itself is safe. Finally, we collect and return all safe nodes sorted in ascending order.

Topological Sort

□ #1462 Course Schedule IV

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Depth-First Search · Breadth-First Search ·
Graph Theory · Topological Sort
Lists: AlgoMaster 600

Problem

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `ai` first if you want to take course `bi`.

- For example, the pair `[0, 1]` indicates that you have to take course 0 before you can take course 1.

Prerequisites can also be indirect. If course `a` is a prerequisite of course `b`, and course `b` is a prerequisite of course `c`, then course `a` is a prerequisite of course `c`.

You are also given an array `queries` where `queries[j] = [uj, vj]`. For the `j`th query, you should answer whether course `uj` is a prerequisite of

course v_j or not.

Return a boolean array `answer`, where `answer[j]` is the answer to the j th query.

Example 1:

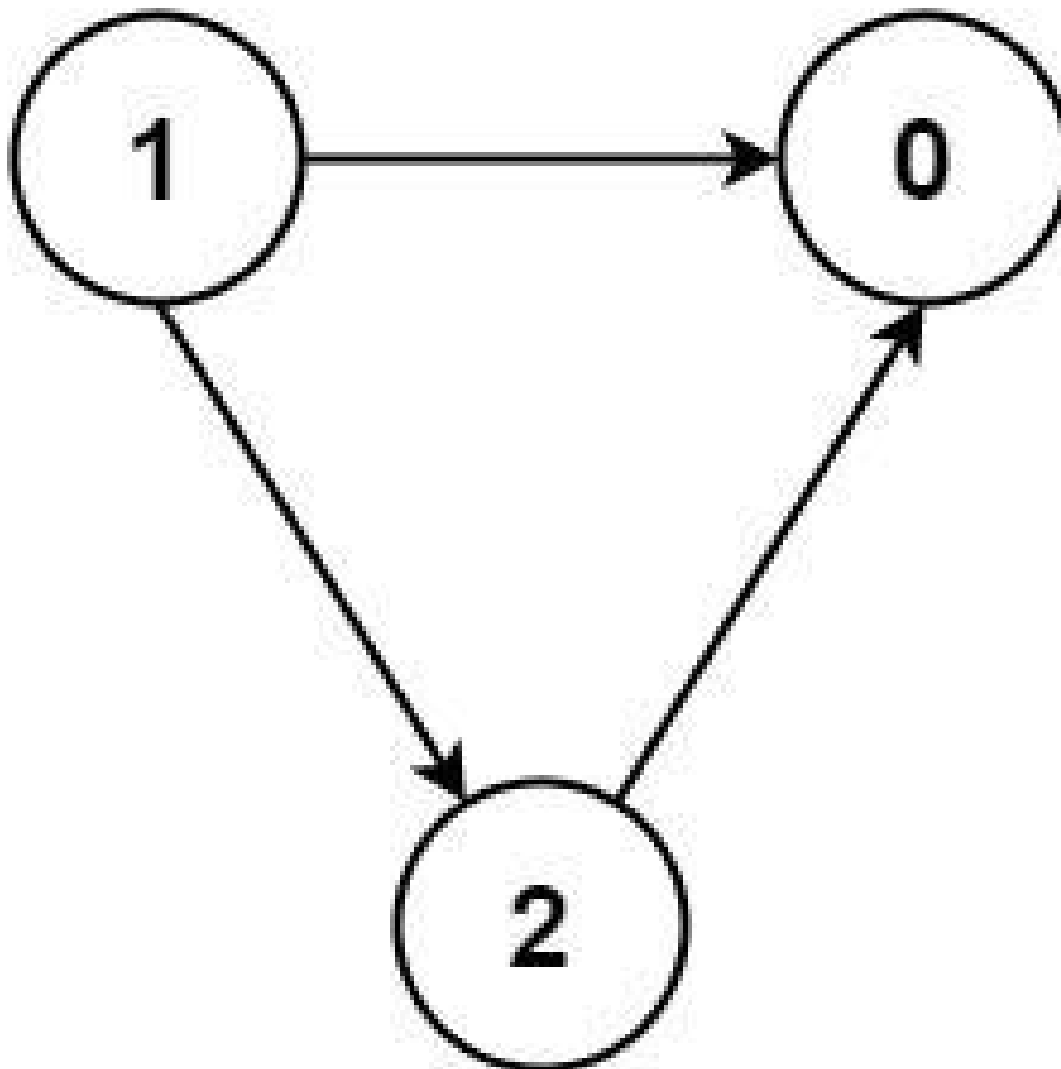


Input: `numCourses = 2, prerequisites = [[1,0]], queries = [[0,1],[1,0]]`
Output: `[false,true]`
Explanation: The pair `[1, 0]` indicates that you have to take course 1 before you can take course 0.
Course 0 is not a prerequisite of course 1, but the opposite is true.

Example 2:

Input: `numCourses = 2, prerequisites = [], queries = [[1,0],[0,1]]`
Output: `[false,false]`
Explanation: There are no prerequisites, and each course is independent.

Example 3:



Input: numCourses = 3, prerequisites = [[1,2],[1,0],[2,0]], queries = [[1,0],[1,2]]
Output: [true,true]

Constraints:

- $2 \leq \text{numCourses} \leq 100$
- $0 \leq \text{prerequisites.length} \leq (\text{numCourses} * (\text{numCourses} - 1) / 2)$
- $\text{prerequisites}[i].\text{length} == 2$
- $0 \leq a_i, b_i \leq \text{numCourses} - 1$
- $a_i \neq b_i$

- All the pairs $[a_i, b_i]$ are unique.
- The prerequisites graph has no cycles.
- $1 \leq \text{queries.length} \leq 104$
- $0 \leq u_i, v_i \leq \text{numCourses} - 1$
- $u_i \neq v_i$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def checkIfPrerequisite(
        self,
        numCourses: int,
        prerequisites: list[list[int]],
        queries: list[list[int]],
    ) -> list[bool]:
        graph = [[] for _ in range(numCourses)]
        # isPrerequisite[i][j] := True if course i is a prerequisite of course j.
        isPrerequisite = [[False] * numCourses for _ in range(numCourses)]

        for u, v in prerequisites:
            graph[u].append(v)

        # DFS from every course.
        for i in range(numCourses):
            self._dfs(graph, i, isPrerequisite[i])

        return [isPrerequisite[u][v] for u, v in queries]

    def _dfs(self, graph: list[list[int]], u: int, used: list[bool]) -> None:
        for v in graph[u]:
            if used[v]:
                continue
            used[v] = True
            self._dfs(graph, v, used)
```


• LeetCode

```
class Solution:
    def checkIfPrerequisite(
        self, n: int, prerequisites: List[List[int]], queries: List[List[int]]
    ) -> List[bool]:
        f = [[False] * n for _ in range(n)]
        for a, b in prerequisites:
            f[a][b] = True
        for k in range(n):
            for i in range(n):
                for j in range(n):
                    if f[i][k] and f[k][j]:
                        f[i][j] = True
        return [f[a][b] for a, b in queries]
```

• SimplyLeet

```
# Python solution using Floyd Warshall algorithm
def check_prerequisites(numCourses, prerequisites, queries):
    # Create a 2D reachability matrix initialized to False
    reachable = [[False] * numCourses for _ in range(numCourses)]

    # Set direct prerequisites in the matrix
    for pre, course in prerequisites:
        reachable[pre][course] = True

    # Compute transitive closure with Floyd Warshall algorithm

    for k in range(numCourses):
        for i in range(numCourses):
            if reachable[i][k]:
                for j in range(numCourses):
                    if reachable[k][j]:
                        reachable[i][j] = True

    # Process each query and return the results
    return [reachable[u][v] for u, v in queries]
```

```
# Example usage:
numCourses = 3
prerequisites = [[1,2],[1,0],[2,0]]
queries = [[1,0],[1,2]]
print(check_prerequisites(numCourses, prerequisites, queries)) # Expected
output: [True, True]
```

◆ Quick Review

Key Insights

- Represent courses and prerequisites as a directed acyclic graph (DAG).
- Use graph traversal techniques (DFS or BFS) or compute the transitive closure.
- The Floyd Warshall algorithm can precompute reachability between all courses.
- After precomputation, each query can be answered in constant time by checking the reachability matrix.

Space & Time

Time Complexity: $O(N^3 + Q)$ where $N = \text{numCourses}$ (Floyd Warshall algorithm) and $Q = \text{number of queries}$

Space Complexity: $O(N^2)$ for storing the reachability matrix

Solution

We solve the problem by converting it into computing reachability in a DAG. The approach is to use the Floyd Warshall algorithm to compute a transitive closure.

Steps:

- Build a boolean reachability matrix and mark direct prerequisites.**
- Use a triple-nested loop (Floyd Warshall) to update the matrix such that if course i can reach course k and course k can reach course j, then course i can reach course j.**
- Answer each query in constant time by checking the precomputed reachability matrix.**

Topological Sort

□ #2115 Find All Possible Recipes from Given Supplies

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Graph Theory ·
Topological Sort
Lists: AlgoMaster 600

Problem

You have information about n different recipes. You are given a string array `recipes` and a 2D string array `ingredients`. The i th recipe has the name `recipes[i]`, and you can create it if you have all the needed ingredients from `ingredients[i]`. A recipe can also be an ingredient for other recipes, i.e., `ingredients[i]` may contain a string that is in `recipes`.

You are also given a string array `supplies` containing all the ingredients that you initially have, and you have an infinite supply of all of them.

Return a list of all the recipes that you can create. You may return the answer in any order.

Note that two recipes may contain each other in their ingredients.

Example 1:

```
Input: recipes = ["bread"], ingredients = [["yeast","flour"]], supplies =  
["yeast","flour","corn"]  
Output: ["bread"]  
Explanation:  
We can create "bread" since we have the ingredients "yeast" and "flour".
```

Example 2:

```
Input: recipes = ["bread","sandwich"], ingredients =  
[["yeast","flour"],["bread","meat"]], supplies = ["yeast","flour","meat"]  
Output: ["bread","sandwich"]  
Explanation:  
We can create "bread" since we have the ingredients "yeast" and "flour".  
We can create "sandwich" since we have the ingredient "meat" and can create the  
ingredient "bread".
```

Example 3:

```
Input: recipes = ["bread","sandwich","burger"], ingredients =  
[["yeast","flour"],["bread","meat"],["sandwich","meat","bread"]], supplies =  
["yeast","flour","meat"]  
Output: ["bread","sandwich","burger"]  
Explanation:  
We can create "bread" since we have the ingredients "yeast" and "flour".  
We can create "sandwich" since we have the ingredient "meat" and can create the  
ingredient "bread".  
We can create "burger" since we have the ingredient "meat" and can create the  
ingredients "bread" and "sandwich".
```

Constraints:

- $n == \text{recipes.length} == \text{ingredients.length}$
- $1 \leq n \leq 100$
- $1 \leq \text{ingredients}[i].\text{length}, \text{supplies.length} \leq 100$

- $1 \leq \text{recipes}[i].\text{length}$, $\text{ingredients}[i][j].\text{length}$, $\text{supplies}[k].\text{length} \leq 10$
- $\text{recipes}[i]$, $\text{ingredients}[i][j]$, and $\text{supplies}[k]$ consist only of lowercase English letters.
- All the values of recipes and supplies combined are unique.
- Each $\text{ingredients}[i]$ does not contain any duplicate values.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findAllRecipes(
        self,
        recipes: list[str],
        ingredients: list[list[str]],
        supplies: list[str],
    ) -> list[str]:
        ans = []
        supplies = set(supplies)
        graph = collections.defaultdict(list)

        inDegrees = collections.Counter()
        q = collections.deque()

        # Build the graph.
        for i, recipe in enumerate(recipes):
            for ingredient in ingredients[i]:
                if ingredient not in supplies:
                    graph[ingredient].append(recipe)
                    inDegrees[recipe] += 1
```

```

# Perform topological sorting.
for recipe in recipes:
    if inDegrees[recipe] == 0:
        q.append(recipe)

while q:
    u = q.popleft()
    ans.append(u)
    for v in graph[u]:
        inDegrees[v] -= 1

    if inDegrees[v] == 0:
        q.append(v)

return ans

```

• LeetDoocs

```

class Solution:
    def findAllRecipes(
        self, recipes: List[str], ingredients: List[List[str]], supplies: List
        [str]
    ) -> List[str]:
        g = defaultdict(list)
        indeg = defaultdict(int)
        for a, b in zip(recipes, ingredients):
            for v in b:
                g[v].append(a)
                indeg[a] += len(b)

        q = supplies
        ans = []
        for i in q:
            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    ans.append(j)
                    q.append(j)

        return ans

```

• SimplyLeet

```
# Python solution using BFS and topological sort.
from collections import deque, defaultdict

def findAllRecipes(recipes, ingredients, supplies):
    # Map each ingredient to the recipes that need it.
    ingredient_to_recipes = defaultdict(list)
    # Dictionary to count required number of ingredients for each recipe.
    recipe_indegree = {}

    # Build graph: for each recipe, iterate over its ingredients.

    for recipe, reqs in zip(recipes, ingredients):
        recipe_indegree[recipe] = len(reqs)
        for ingredient in reqs:
            ingredient_to_recipes[ingredient].append(recipe)

    result = []
    q = deque(supplies) # initialize queue with initial supplies

    # Process available ingredients and recipes.
    while q:

        current = q.popleft()
        # If the current available item is used in any recipe, process it.
        for rec in ingredient_to_recipes[current]:
            # Decrease the recipe's requirement count.
            recipe_indegree[rec] -= 1
            # If all ingredients for the recipe are available, add recipe as an
            ingredient.
            if recipe_indegree[rec] == 0:
                result.append(rec)
                q.append(rec)

    return result

# Example usage:
recipes = ["bread", "sandwich"]
ingredients = [["yeast", "flour"], ["bread", "meat"]]
supplies = ["yeast", "flour", "meat"]
print(findAllRecipes(recipes, ingredients, supplies)) # Output: ["bread",
"sandwich"]
```


◆ Quick Review

Key Insights

- Use a graph (dependency graph) where each recipe has incoming edges from its required ingredients.
- Apply topological sorting: start with the initial supplies and progressively "create" recipes when all their dependencies are met.
- Track in-degrees for recipes. When an ingredient (or recipe) becomes available, decrease the in-degree of all recipes that depend on it.
- When a recipe's in-degree reaches zero, it can be created and added to the result (and used as an ingredient for others).

Space & Time

Time Complexity: $O(R + \text{total_ingredients})$ where R is the number of recipes and total_ingredients sums up all ingredient instances.

Space Complexity: $O(R + I)$ where I is the total unique ingredients used in the dependency mapping.

Solution

We use a topological sort based on Breadth-First Search (BFS). First, we map every ingredient to the list of recipes that need it. We compute the in-degree (number of ingredients required) for each recipe. Initialize the process with the supplied ingredients. For each available ingredient, visit all recipes that depend on it, decrease their in-degree, and add any recipe with an in-degree of zero to the queue. This recipe then becomes an available ingredient for further recipes.

Union Find

Round 5 · Mid · Medium · 3 questions

Union Find

□ #399 Evaluate Division

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Depth-First Search ·
Breadth-First Search · Union-Find · Graph
Theory · Shortest Path
Lists: AlgoMaster 600

Problem

You are given an array of variable pairs equations and an array of real numbers values, where $\text{equations}[i] = [A_i, B_i]$ and $\text{values}[i]$ represent the equation $A_i / B_i = \text{values}[i]$. Each A_i or B_i is a string that represents a single variable.

You are also given some queries, where $\text{queries}[j] = [C_j, D_j]$ represents the j th query where you must find the answer for $C_j / D_j = ?$. Return the answers to all queries. If a single answer cannot be determined, return -1.0 .

Note: The input is always valid. You may assume that evaluating the queries will not result in division by zero and that there is no

contradiction.

Note: The variables that do not occur in the list of equations are undefined, so the answer cannot be determined for them.

Example 1:

```
Input: equations = [["a","b"],["b","c"]], values = [2.0,3.0], queries =
[["a","c"],["b","a"],["a","e"],["a","a"],["x","x"]]
Output: [6.00000,0.50000,-1.00000,1.00000,-1.00000]
Explanation:
Given: a / b = 2.0, b / c = 3.0
queries are: a / c = ?, b / a = ?, a / e = ?, a / a = ?, x / x = ?
return: [6.0, 0.5, -1.0, 1.0, -1.0 ]
note: x is undefined => -1.0
```

Example 2:

```
Input: equations = [["a","b"],["b","c"],["bc","cd"]], values = [1.5,2.5,5.0],
queries = [["a","c"],["c","b"],["bc","cd"],["cd","bc"]]
Output: [3.75000,0.40000,5.00000,0.20000]
```

Example 3:

```
Input: equations = [["a","b"]], values = [0.5], queries =
[["a","b"],["b","a"],["a","c"],["x","y"]]
Output: [0.50000,2.00000,-1.00000,-1.00000]
```

Constraints:

- $1 \leq \text{equations.length} \leq 20$
- $\text{equations}[i].\text{length} == 2$
- $1 \leq \text{Ai.length}, \text{Bi.length} \leq 5$
- $\text{values.length} == \text{equations.length}$
- $0.0 < \text{values}[i] \leq 20.0$
- $1 \leq \text{queries.length} \leq 20$
- $\text{queries}[i].\text{length} == 2$

- $1 \leq C_j.length, D_j.length \leq 5$
- A_i, B_i, C_j, D_j consist of lower case English letters and digits.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def calcEquation(
        self,
        equations: list[list[str]],
        values: list[float],
        queries: list[list[str]],
    ) -> list[float]:
        ans = []
        # graph[A][B] := A / B
        graph = collections.defaultdict(dict)

        for (A, B), value in zip(equations, values):
            graph[A][B] = value
            graph[B][A] = 1 / value

        def devide(A: str, C: str, seen: set[str]) -> float:
            """Returns A / C."""
            if A == C:
                return 1.0

            seen.add(A)

            # value := A / B
            for B, value in graph[A].items():
                if B in seen:
                    continue
                res = devide(B, C, seen) # B / C
                if res > 0: # valid result
                    return value * res # (A / B) * (B / C) = A / C
```

```

        return -1.0 # invalid result

    for A, C in queries:
        if A not in graph or C not in graph:
            ans.append(-1.0)
        else:
            ans.append(devide(A, C, set()))

    return ans

```

• LeetDoocs

```

class Solution:
    def calcEquation(
        self, equations: List[List[str]], values: List[float], queries: List[L
ist[str]]
    ) -> List[float]:
        def find(x):
            if p[x] != x:
                origin = p[x]
                p[x] = find(p[x])
                w[x] *= w[origin]
            return p[x]

        w = defaultdict(lambda: 1)
        p = defaultdict()
        for a, b in equations:
            p[a], p[b] = a, b
        for i, v in enumerate(values):
            a, b = equations[i]
            pa, pb = find(a), find(b)
            if pa == pb:
                continue

            p[pa] = pb
            w[pa] = w[b] * v / w[a]
        return [
            -1 if c not in p or d not in p or find(c) != find(d) else w[c] / w
[d]
            for c, d in queries
        ]

```

• SimplyLeet

Python solution using DFS

```
def calcEquation(equations, values, queries):
    # Build the graph as a dictionary of dictionaries
    graph = {}
    for (dividend, divisor), value in zip(equations, values):
        if dividend not in graph:
            graph[dividend] = {}
        if divisor not in graph:
            graph[divisor] = {}

    graph[dividend][divisor] = value
    graph[divisor][dividend] = 1 / value

    def dfs(current, target, accumulated, visited):
        # If the current node equals the target, return the accumulated product
        if current == target:
            return accumulated
        visited.add(current)
        # Traverse all neighbors
        for neighbor, value in graph[current].items():
            if neighbor in visited:
                continue
            result = dfs(neighbor, target, accumulated * value, visited)
            if result != -1:
                return result
        return -1

    results = []
    for start, end in queries:
        if start not in graph or end not in graph:
```



```

        results.append(-1.0)
    elif start == end:
        results.append(1.0)
    else:
        visited = set()
        result = dfs(start, end, 1, visited)
        results.append(result)
    return results

```

Example usage:

```

if __name__ == "__main__":
    equations = [["a", "b"], ["b", "c"]]
    values = [2.0, 3.0]
    queries = [["a", "c"], ["b", "a"], ["a", "e"], ["a", "a"], ["x", "x"]]
    print(calcEquation(equations, values, queries))

```

◆ Quick Review

Key Insights

- The problem can be modeled as a graph where each variable is a node.
- Each equation $A / B = k$ creates a directed edge from A to B with weight k and from B to A with weight $1/k$.
- To answer a query, perform a DFS or BFS from the source node and try to reach the target node, multiplying the weights along the path.
- If a variable in the query does not exist in the graph or no path exists, return -1.0 .

Space & Time

Time Complexity: $O(V + E)$ per query in the worst case since we might traverse the entire graph.

Space Complexity: $O(V + E)$ for storing the graph and the recursion stack or queue for the search.

Solution

We can solve this problem by first constructing a graph using a hashmap or dictionary where each node points to its neighbors with the corresponding multiplication factors. For each query, we perform a depth-first search (DFS) starting from the dividend. We keep track of the cumulative product along the path. If we eventually reach the divisor, we return the computed product. If not, the query is unsolvable and we return -1.0 . Important points:

- Use a visited set during DFS to prevent cycles.
- Ensure to handle cases where either node in the query is not present in the graph.

– The same approach works with BFS, but DFS may be simpler to implement in these cases.

Union Find

□ #547 Number of Provinces

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Depth-First Search · Breadth-First Search ·
Union-Find · Graph Theory
Lists: AlgoMaster 600

Problem

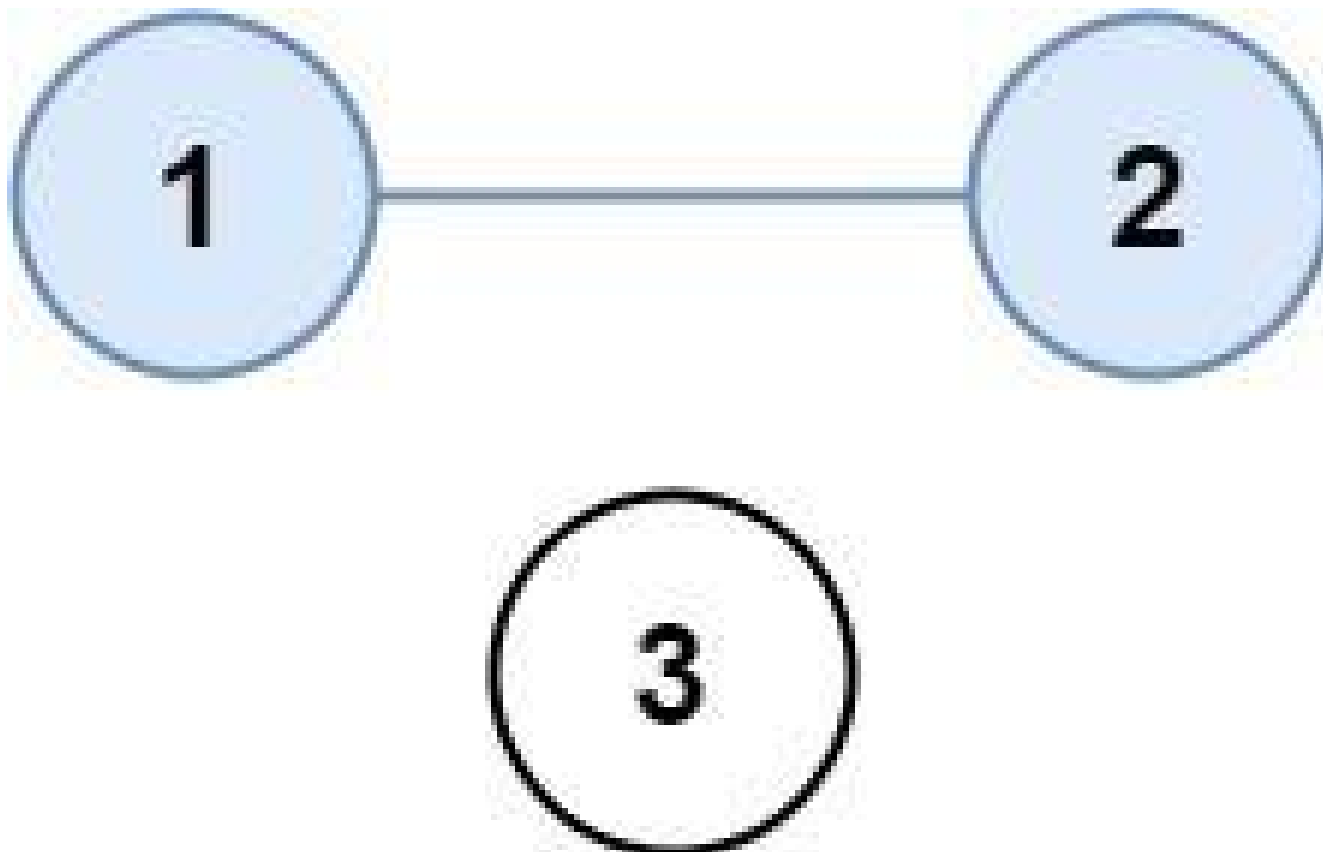
There are n cities. Some of them are connected, while some are not. If city a is connected directly with city b , and city b is connected directly with city c , then city a is connected indirectly with city c .

A province is a group of directly or indirectly connected cities and no other cities outside of the group.

You are given an $n \times n$ matrix `isConnected` where `isConnected[i][j] = 1` if the i th city and the j th city are directly connected, and `isConnected[i][j] = 0` otherwise.

Return the total number of provinces.

Example 1:



Input: `isConnected = [[1,1,0],[1,1,0],[0,0,1]]`
Output: 2

Example 2:



Input: `isConnected = [[1,0,0],[0,1,0],[0,0,1]]`

Output: 3

Constraints:

- $1 \leq n \leq 200$
- $n == \text{isConnected.length}$
- $n == \text{isConnected}[i].\text{length}$
- $\text{isConnected}[i][j]$ is 1 or 0.
- $\text{isConnected}[i][i] == 1$
- $\text{isConnected}[i][j] == \text{isConnected}[j][i]$

Community Solutions (Python)

- WalkCC

```

class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        self.count -= 1

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def findCircleNum(self, isConnected: list[list[int]]) -> int:
        n = len(isConnected)
        uf = UnionFind(n)

        for i in range(n):
            for j in range(i, n):
                if isConnected[i][j] == 1:
                    uf.unionByRank(i, j)

        return uf.count

```

• LeetDoocs

```

class Solution:
    def findCircleNum(self, isConnected: List[List[int]]) -> int:
        def dfs(i: int):
            vis[i] = True
            for j, x in enumerate(isConnected[i]):
                if not vis[j] and x:
                    dfs(j)

        n = len(isConnected)
        vis = [False] * n

        ans = 0
        for i in range(n):
            if not vis[i]:
                dfs(i)
                ans += 1
        return ans

```

```

class Solution:
    def findCircleNum(self, isConnected: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(isConnected)
        p = list(range(n))
        ans = n

        for i in range(n):
            for j in range(i + 1, n):
                if isConnected[i][j]:
                    pa, pb = find(i), find(j)
                    if pa != pb:
                        p[pa] = pb
                        ans -= 1

        return ans

```

☐ • SimplyLeet


```

# Function to count the number of provinces using DFS
def findCircleNum(isConnected):
    n = len(isConnected) # Total number of cities
    visited = [False] * n # Track visited cities

    # Helper function for Depth-First Search
    def dfs(city):
        visited[city] = True # Mark the current city as visited
        # Explore all neighbors of the current city
        for neighbor in range(n):

            # If the neighbor is connected and not visited yet, explore deeper
            if isConnected[city][neighbor] == 1 and not visited[neighbor]:
                dfs(neighbor)

    provinces = 0 # Initialize province counter
    # Iterate over each city
    for city in range(n):
        if not visited[city]:
            dfs(city) # Start DFS for a new province
            provinces += 1 # Increment province count

    return provinces

# Example usage:
if __name__ == "__main__":
    isConnected = [[1, 1, 0], [1, 1, 0], [0, 0, 1]]
    print(findCircleNum(isConnected)) # Expected Output: 2

```

◆ Quick Review

Key Insights

- The matrix represents an undirected graph where each node is a city.
- Direct and indirect connections imply finding connected components in the graph.

- DFS, BFS, or Union–Find can be used to traverse and count connected components.
- Each traversal starting from an unvisited city represents a new province.

Space & Time

Time Complexity: $O(n^2)$ – Every cell of the matrix is inspected.

Space Complexity: $O(n)$ – For the visited array (and recursion stack in DFS worst–case).

Solution

To solve the problem, we use a Depth–First Search (DFS) approach. Start by iterating over each city. When encountering an unvisited city, initiate a DFS to mark all cities within the same province as visited. Increment the province counter for each DFS initiation. This effectively counts the number of connected components (provinces) in the graph.

Union Find

□ #947 Most Stones Removed with Same Row or Column

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Depth-First Search · Union-Find ·
Graph Theory
Lists: AlgoMaster 600

Problem

On a 2D plane, we place n stones at some integer coordinate points. Each coordinate point may have at most one stone.

A stone can be removed if it shares either the same row or the same column as another stone that has not been removed.

Given an array `stones` of length n where `stones[i] = [xi, yi]` represents the location of the i th stone, return the largest possible number of stones that can be removed.

Example 1:

Input: stones = [[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]

Output: 5

Explanation: One way to remove 5 stones is as follows:

1. Remove stone [2,2] because it shares the same row as [2,1].
2. Remove stone [2,1] because it shares the same column as [0,1].
3. Remove stone [1,2] because it shares the same row as [1,0].
4. Remove stone [1,0] because it shares the same column as [0,0].
5. Remove stone [0,1] because it shares the same row as [0,0].

Example 2:

Input: stones = [[0,0],[0,2],[1,1],[2,0],[2,2]]

Output: 3

Explanation: One way to make 3 moves is as follows:

1. Remove stone [2,2] because it shares the same row as [2,0].
2. Remove stone [2,0] because it shares the same column as [0,0].
3. Remove stone [0,2] because it shares the same row as [0,0].

Stones [0,0] and [1,1] cannot be removed since they do not share a row/column with another stone still on the plane.

Example 3:

Input: stones = [[0,0]]

Output: 0

Explanation: [0,0] is the only stone on the plane, so you cannot remove it.

Constraints:

- $1 \leq \text{stones.length} \leq 1000$
- $0 \leq x_i, y_i \leq 104$
- No two stones are at the same coordinate point.

Community Solutions (Python)

- LeetDoocs

```

class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]

        return True

class Solution:
    def removeStones(self, stones: List[List[int]]) -> int:
        uf = UnionFind(len(stones))
        ans = 0
        for i, (x1, y1) in enumerate(stones):
            for j, (x2, y2) in enumerate(stones[:i]):
                if x1 == x2 or y1 == y2:
                    ans += uf.union(i, j)
        return ans

```

• SimplyLeet

```

# Python solution using DFS

def removeStones(stones):
    # Build mappings from row and column to stone indices
    row_map = {}
    col_map = {}
    n = len(stones)
    for i, (x, y) in enumerate(stones):
        row_map.setdefault(x, []).append(i)
        col_map.setdefault(y, []).append(i)

    visited = [False] * n

    # DFS to traverse all stones connected to the current stone
    def dfs(index):
        if visited[index]:
            return
        visited[index] = True
        x, y = stones[index]
        # Traverse stones in the same row
        for neighbor in row_map.get(x, []):
            if not visited[neighbor]:
                dfs(neighbor)
        # Traverse stones in the same column
        for neighbor in col_map.get(y, []):
            if not visited[neighbor]:
                dfs(neighbor)

    components = 0
    # Perform DFS for each stone that hasn't been visited
    for i in range(n):
        if not visited[i]:
            dfs(i)
            components += 1

    # The result is total stones removed = n - number of connected components
    return n - components

# Example usage:
stones = [[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]

```

```
print(removeStones(stones)) # Expected Output: 5
```

◆ Quick Review

Key Insights

- This problem can be modeled as a graph where each stone is a node. An edge exists between two stones if they share the same row or column.
- In each connected component of the graph, you can remove all stones except one.
- Thus, the maximum number of stones removed equals the total number of stones minus the number of connected components.
- Depth-First Search (DFS) or Union Find can be used to count the number of connected components.

Space & Time

Time Complexity: $O(n^2)$ in the worst case when each stone is compared with every other stone (can be optimized with Union Find).

Space Complexity: $O(n)$ for storing visited status and mapping structures.

Solution

We solve the problem using Depth-First Search (DFS). The algorithm first builds mappings from rows and columns to stone indices. Then, it iterates through each stone; if the stone hasn't been visited, it initiates a DFS which marks all stones in the same connected component (those sharing a row or column) as visited. The number of DFS calls equals the number of connected components. Finally, the maximum stones removed is given by the total number of stones minus the number of connected components.

Minimum Spanning Tree

Round 5 · Mid · Medium · 1 question

Minimum Spanning Tree

□ #1135 Connecting Cities With Minimum Cost

MED

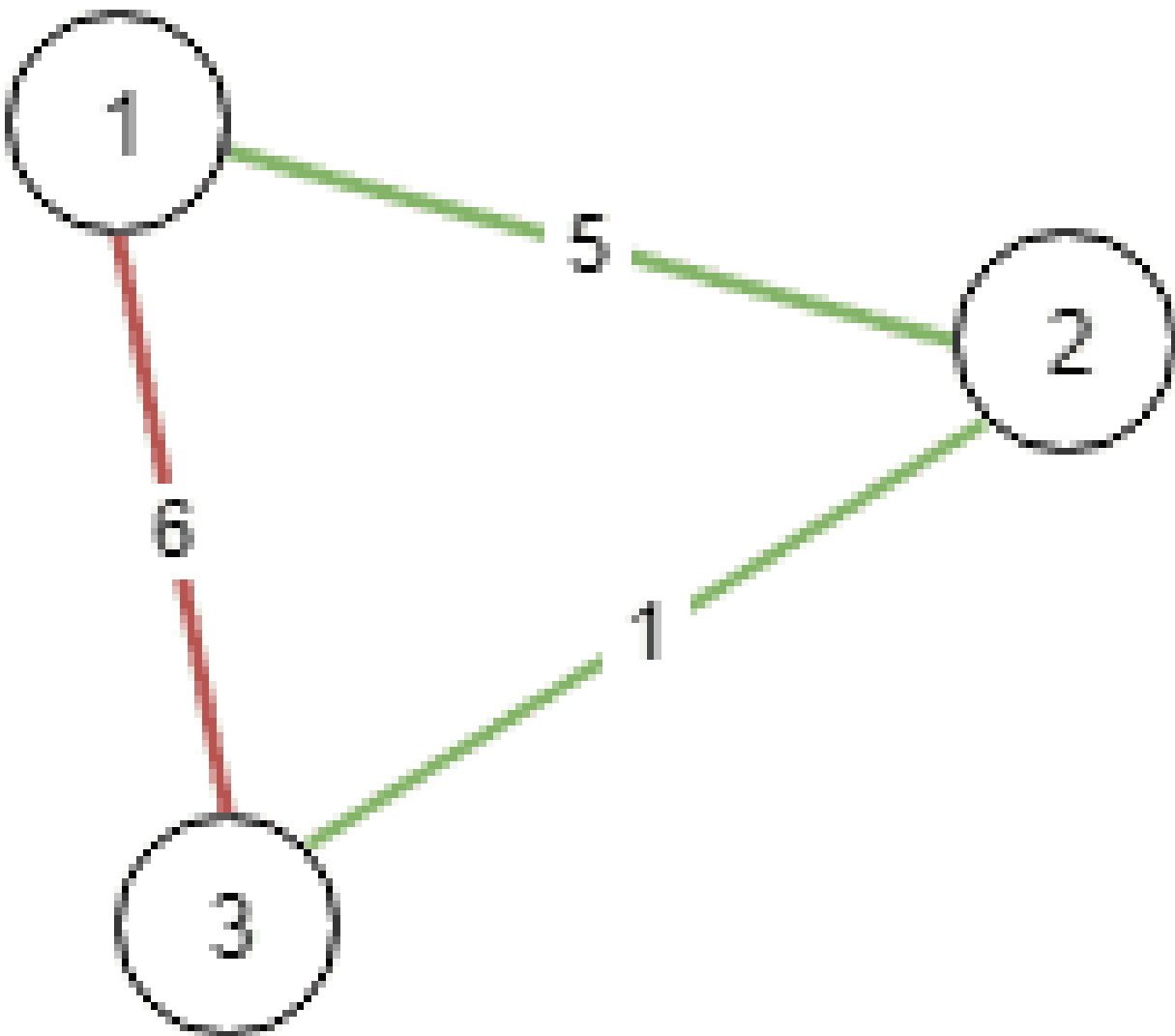
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Union-Find · Graph Theory · Heap (Priority Queue) · Minimum Spanning Tree
Lists: AlgoMaster 600

Problem

There are n cities labeled from 1 to n . You are given the integer n and an array `connections` where `connections[i] = [xi, yi, costi]` indicates that the cost of connecting city x_i and city y_i (bidirectional connection) is $cost_i$.

Return the minimum cost to connect all the n cities such that there is at least one path between each pair of cities. If it is impossible to connect all the n cities, return -1 .
The cost is the sum of the connections' costs used.

Example 1:

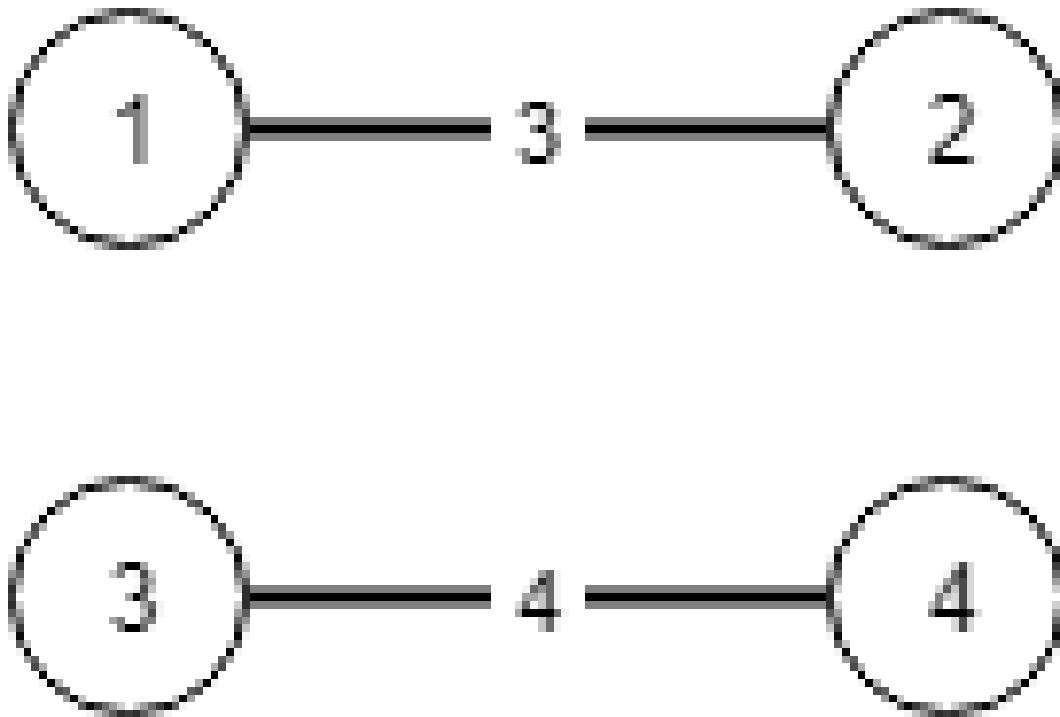


Input: $n = 3$, connections = $[[1,2,5],[1,3,6],[2,3,1]]$

Output: 6

Explanation: Choosing any 2 edges will connect all cities so we choose the minimum 2.

Example 2:



Input: $n = 4$, $\text{connections} = [[1,2,3],[3,4,4]]$

Output: -1

Explanation: There is no way to connect all cities even if all edges are used.

Constraints:

- $1 \leq n \leq 10^4$
- $1 \leq \text{connections.length} \leq 10^4$
- $\text{connections}[i].\text{length} == 3$
- $1 \leq x_i, y_i \leq n$
- $x_i \neq y_i$
- $0 \leq \text{cost}_i \leq 10^5$

Community Solutions (Python)

- WalkCC

```
class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return

        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def minimumCost(self, n: int, connections: list[list[int]]) -> int:
        ans = 0
        uf = UnionFind(n + 1)

        # Sort by cost.
        connections.sort(key=lambda x: x[2])

        for u, v, cost in connections:
            if uf.find(u) == uf.find(v):
                continue
            uf.unionByRank(u, v)
            ans += cost

        root = uf.find(1)
        if any(uf.find(i) != root for i in range(1, n + 1)):
```

```

        return -1

    return ans

```

• LeetCode

```

class Solution:
    def minimumCost(self, n: int, connections: List[List[int]]) -> int:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        connections.sort(key=lambda x: x[2])
        p = list(range(n))
        ans = 0

        for x, y, cost in connections:
            x, y = x - 1, y - 1
            if find(x) == find(y):
                continue
            p[find(x)] = find(y)
            ans += cost
            n -= 1
            if n == 1:
                return ans
        return -1

```

• SimplyLeet

```

# Python solution using Union Find (Kruskal's Algorithm)

# Define the Union find class to manage connected components
class UnionFind:
    def __init__(self, size):
        # Parent list: each node is initially its own parent.
        self.parent = list(range(size))

    def find(self, x):
        # Path Compression: make every node on the path point directly to the
        root

```

```
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        # Union the roots of x and y, if they are different
        rootX = self.find(x)
        rootY = self.find(y)
        if rootX == rootY:
            return False # x and y are already in the same set

        self.parent[rootY] = rootX
        return True

def minimumCost(n, connections):
    # Sort the connections based on cost
    connections.sort(key=lambda x: x[2])

    uf = UnionFind(n)
    total_cost = 0
    edges_used = 0

    # Iterate over each connection in sorted order
    for city1, city2, cost in connections:
        # Adjust city labels to 0-indexed
        if uf.union(city1 - 1, city2 - 1):
            total_cost += cost
            edges_used += 1
            # Once we use n-1 edges, all cities are connected
            if edges_used == n - 1:
                return total_cost

    # If we haven't connected all cities, return -1
    return -1

# Example usage:
# print(minimumCost(3, [[1,2,5],[1,3,6],[2,3,1]]))
```

◆ Quick Review

Key Insights

- This is a Minimum Spanning Tree (MST) problem.
- Kruskal's algorithm can be used by sorting the edges by cost and using a Union-Find (Disjoint Set Union) structure to efficiently check if adding an edge forms a cycle.
- Alternatively, Prim's algorithm with a priority queue can solve this, but Kruskal's is straightforward given edge-based input.
- The key is to add edges one by one in increasing order of cost until all cities are connected.

Space & Time

Time Complexity: $O(E \log E)$ due to sorting the connections, where E is the number of connections.

Space Complexity: $O(n)$ for the Union-Find data structure, plus $O(E)$ space for storing the edges.

Solution

We use Kruskal's algorithm to build a Minimum Spanning Tree. First, sort the connections by cost. Then, iterate through

each edge and use a Union-Find data structure to determine if the current edge connects two previously disconnected components. If it does, add the edge's cost to the total cost and union the two components. Finally, if we have connected all n cities (i.e., used $n-1$ edges), return the total cost; otherwise, return -1 .

Shortest Path

Round 5 · Mid · Medium · 5 questions

Shortest Path

□ #1514 Path with Maximum Probability

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Graph Theory · Heap (Priority Queue) ·
Shortest Path

Lists: AlgoMaster 600

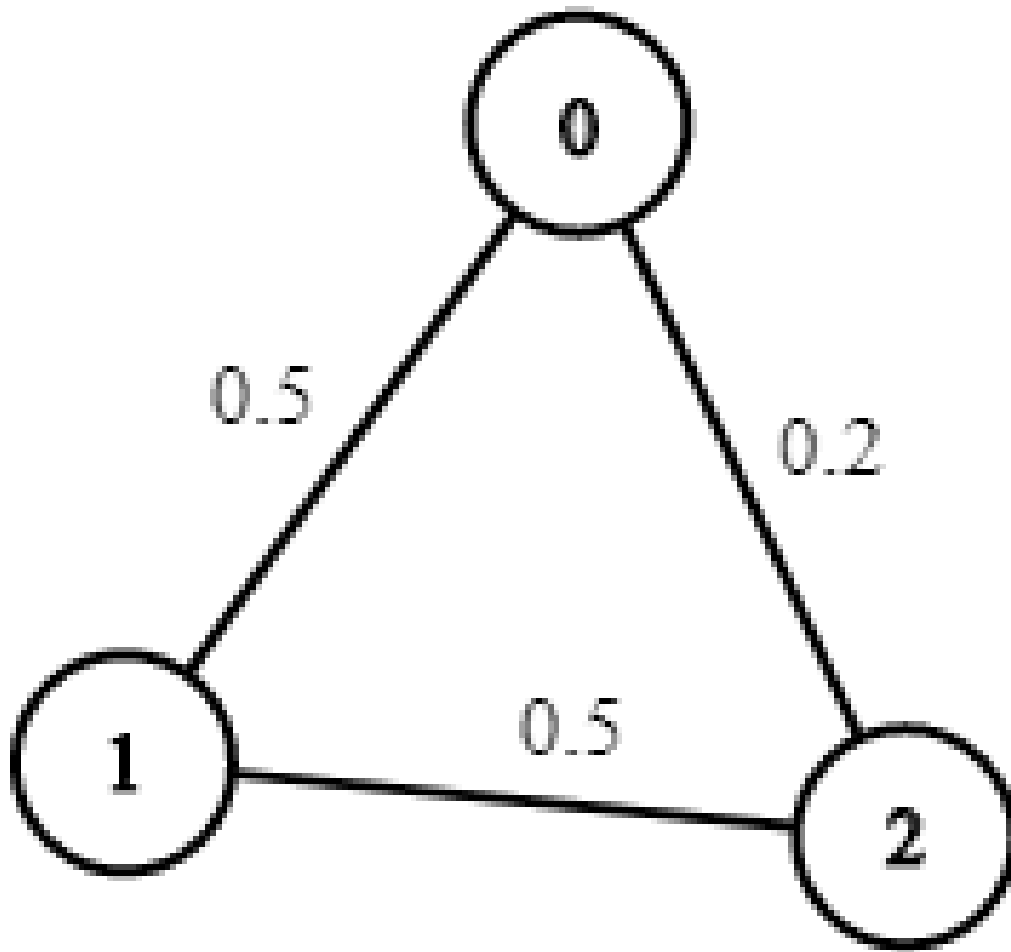
Problem

You are given an undirected weighted graph of n nodes (0-indexed), represented by an edge list where $\text{edges}[i] = [a, b]$ is an undirected edge connecting the nodes a and b with a probability of success of traversing that edge $\text{succProb}[i]$.

Given two nodes start and end , find the path with the maximum probability of success to go from start to end and return its success probability.

If there is no path from start to end , return 0. Your answer will be accepted if it differs from the correct answer by at most $1e-5$.

Example 1:

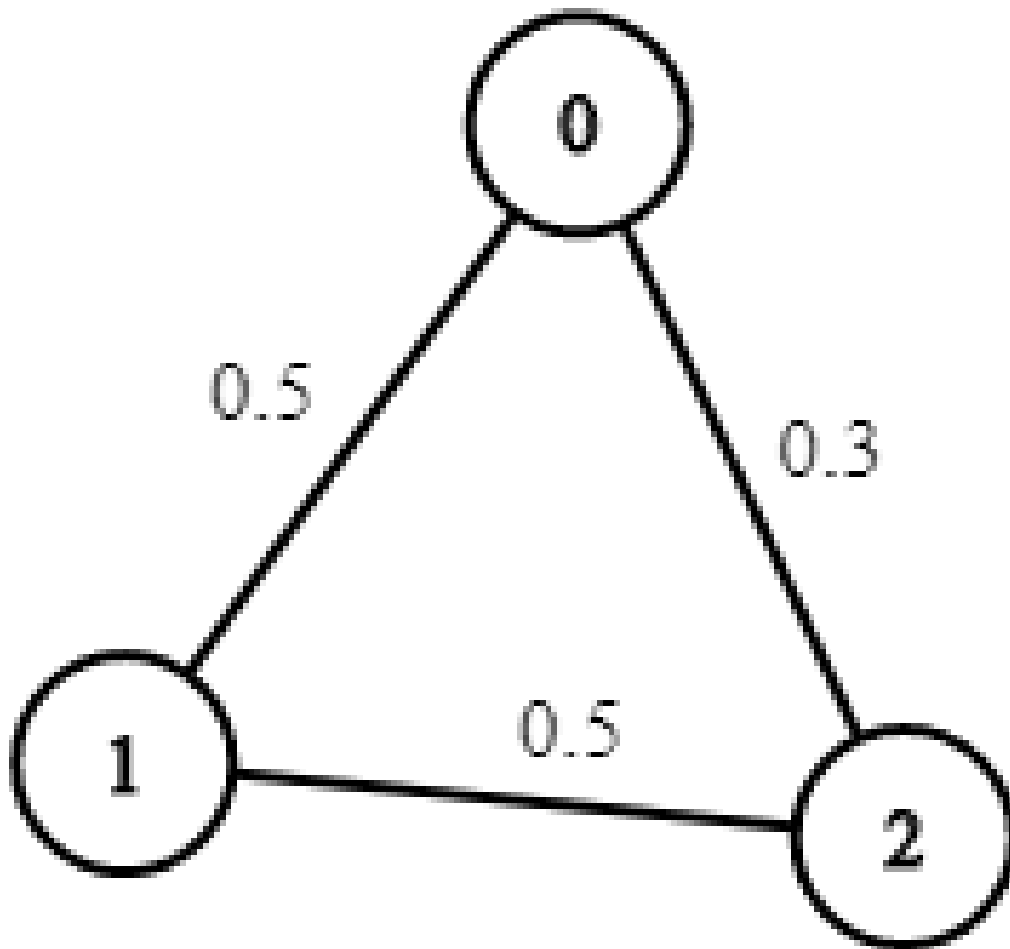


Input: $n = 3$, $edges = [[0,1],[1,2],[0,2]]$, $succProb = [0.5,0.5,0.2]$, $start = 0$, $end = 2$

Output: 0.25000

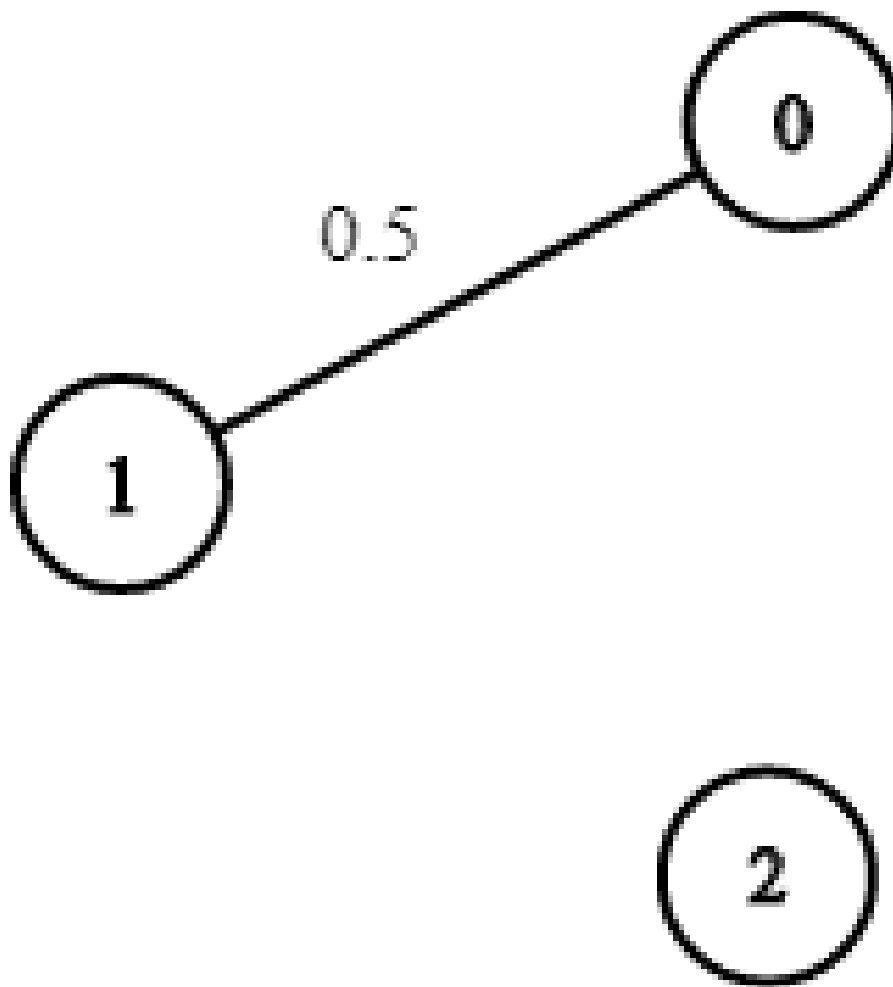
Explanation: There are two paths from start to end, one having a probability of success = 0.2 and the other has $0.5 * 0.5 = 0.25$.

Example 2:



Input: $n = 3$, $\text{edges} = [[0,1],[1,2],[0,2]]$, $\text{succProb} = [0.5,0.5,0.3]$, $\text{start} = 0$,
 $\text{end} = 2$
Output: 0.30000

Example 3:



Input: $n = 3$, $\text{edges} = [[0,1]]$, $\text{succProb} = [0.5]$, $\text{start} = 0$, $\text{end} = 2$

Output: **0.00000**

Explanation: There is no path between 0 and 2.

Constraints:

- $2 \leq n \leq 10^4$
- $0 \leq \text{start}, \text{end} < n$
- $\text{start} \neq \text{end}$
- $0 \leq a, b < n$
- $a \neq b$

- $0 \leq \text{succProb.length} == \text{edges.length} \leq 2 \cdot 10^4$
- $0 \leq \text{succProb}[i] \leq 1$
- There is at most one edge between every two nodes.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProbability(
        self,
        n: int,
        edges: list[list[int]],
        succProb: list[float],
        start: int,
        end: int,
    ) -> float:
        graph = [[] for _ in range(n)] # {a: [(b, probability_ab)]}

        maxHeap = [(-1.0, start)] # (the probability to reach u, u)
        seen = [False] * n

        for i, ((u, v), prob) in enumerate(zip(edges, succProb)):
            graph[u].append((v, prob))
            graph[v].append((u, prob))

        while maxHeap:
            prob, u = heapq.heappop(maxHeap)
            prob *= -1
```

```

    if u == end:
        return prob
    if seen[u]:
        continue
    seen[u] = True
    for nextNode, edgeProb in graph[u]:
        if seen[nextNode]:
            continue
        heapq.heappush(maxHeap, (-prob * edgeProb, nextNode))

return 0

```

• LeetDoocs

```

class Solution:
    def maxProbability(
        self,
        n: int,
        edges: List[List[int]],
        succProb: List[float],
        start_node: int,
        end_node: int,
    ) -> float:
        g: List[List[Tuple[int, float]]] = [[] for _ in range(n)]

        for (a, b), p in zip(edges, succProb):
            g[a].append((b, p))
            g[b].append((a, p))
        pq = [(-1, start_node)]
        dist = [0] * n
        dist[start_node] = 1
        while pq:
            w, a = heappop(pq)
            w = -w
            if dist[a] > w:
                continue
            for b, p in g[a]:
                if (t := w * p) > dist[b]:
                    dist[b] = t
                    heappush(pq, (-t, b))
        return dist[end_node]

```


• SimplyLeet

```
import heapq
from collections import defaultdict

def maxProbability(n, edges, succProb, start, end):
    # Build the graph as an adjacency list
    graph = defaultdict(list)
    for (u, v), prob in zip(edges, succProb):
        graph[u].append((v, prob))
        graph[v].append((u, prob))

    # Create a list to store the maximum probability for reaching each node
    max_probs = [0.0] * n
    max_probs[start] = 1.0

    # Use a max-heap with negative probabilities (since heapq is a min-heap)
    heap = [(-1.0, start)]

    # Process nodes in the order of maximum probability
    while heap:
        # Pop the node with the current highest probability path

        prob, node = heapq.heappop(heap)
        current_prob = -prob

        # If we reached the end node, return the probability
        if node == end:
            return current_prob

        # If we encounter a probability smaller than the recorded one, skip it
        if current_prob < max_probs[node]:
            continue
```

```
# Update the probabilities for neighboring nodes
for neighbor, edge_prob in graph[node]:
    new_prob = current_prob * edge_prob
    if new_prob > max_probs[neighbor]:
        max_probs[neighbor] = new_prob
        heapq.heappush(heap, (-new_prob, neighbor))

# Return 0 if end is unreachable
return 0.0
```

```
# Example usage:
if __name__ == "__main__":
    n = 3
    edges = [[0,1],[1,2],[0,2]]
    succProb = [0.5,0.5,0.2]
    start, end = 0, 2
    print(maxProbability(n, edges, succProb, start, end)) # Expected output:
0.25
```

◆ Quick Review

Key Insights

- The problem can be modeled as a graph traversal where the "distance" is defined in terms of cumulative probability (i.e., product of probabilities).
- Instead of summing edge weights (like in a standard shortest path problem), multiply probabilities.
- Dijkstra's algorithm can be adapted to maximize the product by using a max-heap

(priority queue) to always extend the path with the highest probability so far.

- An alternative trick is to transform the multiplication problem into an addition problem by using logarithms, but here a direct multiplication approach with proper priority update is more natural.
- Maintain an array holding the best probability reached for each node to avoid unnecessary processing.

Space & Time

Time Complexity: $O(E \log V)$, where E is the number of edges and V is the number of vertices.

Space Complexity: $O(V + E)$ for the graph representation and auxiliary data structures.

Solution

We use a modified Dijkstra's algorithm:

- Build an adjacency list representation of the graph.
- Use a max-heap (priority queue) where each element is a pair (current probability, node).

The heap always gives the node with the highest probability path computed so far.

- Initialize the probability of reaching the start node as 1 and 0 for all other nodes.
- While the heap is not empty, pop the node with the highest probability. If it is the end node, return its probability.
- For each neighbor, compute the new probability by multiplying the current probability with the probability of the edge connecting to that neighbor. If this new probability is higher than the best known probability for the neighbor, update it and push the neighbor into the heap.
- If the end node is never reached, return 0.

Shortest Path

□ #1631 Path With Minimum Effort

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Depth-First Search ·
Breadth-First Search · Union-Find · Heap
(Priority Queue) · Matrix
Lists: AlgoMaster 600

Problem

You are a hiker preparing for an upcoming hike. You are given heights, a 2D array of size rows x columns, where heights[row][col] represents the height of cell (row, col). You are situated in the top-left cell, (0, 0), and you hope to travel to the bottom-right cell, (rows-1, columns-1) (i.e., 0-indexed). You can move up, down, left, or right, and you wish to find a route that requires the minimum effort.

A route's effort is the maximum absolute difference in heights between two consecutive cells of the route.

Return the minimum effort required to travel from the top-left cell to the bottom-right cell.

Example 1:

1	2	2
3	8	2
5	3	5

Input: heights = [[1,2,2],[3,8,2],[5,3,5]]

Output: 2

Explanation: The route of [1,3,5,3,5] has a maximum absolute difference of 2 in consecutive cells.

This is better than the route of [1,2,2,2,5], where the maximum absolute difference is 3.

Example 2:

1	2	3
3	8	4
5	3	5

Input: heights = [[1,2,3],[3,8,4],[5,3,5]]

Output: 1

Explanation: The route of [1,2,3,4,5] has a maximum absolute difference of 1 in consecutive cells, which is better than route [1,3,5,3,5].

Example 3:

1	2	1	1	1
1	2	1	2	1
1	2	1	2	1
1	2	1	2	1
1	1	1	2	1

Input: heights = [[1,2,1,1,1],[1,2,1,2,1],[1,2,1,2,1],[1,2,1,2,1],[1,1,1,2,1]]

Output: 0

Explanation: This route does not require any effort.

Constraints:

- rows == heights.length
- columns == heights[i].length
- 1 <= rows, columns <= 100
- 1 <= heights[i][j] <= 106

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minimumEffortPath(self, heights: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(heights)
        n = len(heights[0])
        # diff[i][j] := the maximum absolute difference to reach (i, j)
        diff = [[math.inf] * n for _ in range(m)]
        seen = set()

        minHeap = [(0, 0, 0)]  # (d, i, j)

        diff[0][0] = 0

        while minHeap:
            d, i, j = heapq.heappop(minHeap)
            if i == m - 1 and j == n - 1:
                return d
            seen.add((i, j))
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy

                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if (x, y) in seen:
                    continue
                newDiff = abs(heights[i][j] - heights[x][y])
                maxDiff = max(diff[i][j], newDiff)
                if diff[x][y] > maxDiff:
                    diff[x][y] = maxDiff
                    heapq.heappush(minHeap, (diff[x][y], x, y))
```

• LeetDoocs

```
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]

        return True

    def connected(self, a, b):
        return self.find(a) == self.find(b)

class Solution:
    def minimumEffortPath(self, heights: List[List[int]]) -> int:
        m, n = len(heights), len(heights[0])
        uf = UnionFind(m * n)
```

```

e = []
dirs = (0, 1, 0)
for i in range(m):
    for j in range(n):
        for a, b in pairwise(dirs):
            x, y = i + a, j + b
            if 0 <= x < m and 0 <= y < n:
                e.append(
                    (abs(heights[i][j] - heights[x][y]), i * n + j, x
* n + y)
                )

```

```

e.sort()
for h, a, b in e:
    uf.union(a, b)
    if uf.connected(0, m * n - 1):
        return h
return 0

```

```

class Solution:
    def minimumEffortPath(self, heights: List[List[int]]) -> int:
        def check(h: int) -> bool:
            q = deque([(0, 0)])
            vis = {(0, 0)}
            dirs = (-1, 0, 1, 0, -1)
            while q:
                for _ in range(len(q)):
                    i, j = q.popleft()
                    if i == m - 1 and j == n - 1:
                        return True
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if (
                        0 <= x < m
                        and 0 <= y < n
                        and (x, y) not in vis
                        and abs(heights[i][j] - heights[x][y]) <= h
                    ):
                        q.append((x, y))

```

```

        vis.add((x, y))

    return False

m, n = len(heights), len(heights[0])
return bisect_left(range(10**6), True, key=check)

class Solution:
    def minimumEffortPath(self, heights: List[List[int]]) -> int:
        m, n = len(heights), len(heights[0])
        dist = [[inf] * n for _ in range(m)]
        dist[0][0] = 0
        dirs = (-1, 0, 1, 0, -1)
        q = [(0, 0, 0)]
        while q:
            t, i, j = heappop(q)
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if (
                    0 <= x < m
                    and 0 <= y < n
                    and (d := max(t, abs(heights[i][j] - heights[x][y]))) < di
st[x][y]
                ):
                    dist[x][y] = d
                    heappush(q, (d, x, y))
        return int(dist[-1][-1])

```

• SimplyLeet

```

import heapq

def minimumEffortPath(heights):
    rows, cols = len(heights), len(heights[0])
    # Directions: up, down, left, right
    directions = [(1,0), (-1,0), (0,1), (0,-1)]

    # Distance matrix to track minimum effort found for each cell.
    effort = [[float('inf')] * cols for _ in range(rows)]
    effort[0][0] = 0

```

```

# Priority queue containing (current effort, row, col)
minHeap = [(0, 0, 0)]

while minHeap:
    currentEffort, row, col = heapq.heappop(minHeap)
    if row == rows - 1 and col == cols - 1:
        return currentEffort

    for dr, dc in directions:

        newRow, newCol = row + dr, col + dc
        if 0 <= newRow < rows and 0 <= newCol < cols:
            # The cost to move to the neighbor is the absolute difference
in heights
            stepEffort = abs(heights[newRow][newCol] - heights[row][col])
            # The effort for the new cell is the maximum effort so far
along the path.
            newEffort = max(currentEffort, stepEffort)
            if newEffort < effort[newRow][newCol]:
                effort[newRow][newCol] = newEffort
                heapq.heappush(minHeap, (newEffort, newRow, newCol))

return 0

# Example usage:
heights = [[1,2,2],[3,8,2],[5,3,5]]
print(minimumEffortPath(heights))

```

◆ Quick Review

Key Insights

- The problem asks for a path minimizing the maximum step difference instead of the sum of weights.
- This can be modeled as a graph problem where each cell is a node connected to its

neighbors with edge weights equal to the absolute difference in heights.

- A modified Dijkstra's algorithm fits well since the "cost" of a path is determined by the maximum weight along the path rather than the sum.
- Alternatively, binary search combined with a flood-fill (BFS/DFS) or Union-Find could determine the smallest threshold that connects the start and end.

Space & Time

Time Complexity: $O(m * n * \log(m * n))$ where m and n are the dimensions of the grid (each cell processed and maintained in a priority queue).

Space Complexity: $O(m * n)$ for the distance matrix and priority queue.

Solution

The solution uses a modified Dijkstra's algorithm. The grid is treated as a graph where each cell is a node. The cost to move from one cell to an adjacent cell is the absolute difference in their heights. However, instead of summing up costs, the cost

associated with a path is determined by the maximum single-step cost encountered along that path. Using a min-heap (priority queue), we always expand the least costly (in terms of maximum step encountered so far) path. We update the cost for neighboring cells if a path with a smaller maximum difference is found. The process continues until the destination cell is reached.

Shortest Path

□ #2976 Minimum Cost to Convert String I **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Graph Theory · Shortest Path
Lists: AlgoMaster 600

Problem

You are given two 0-indexed strings `source` and `target`, both of length `n` and consisting of lowercase English letters. You are also given two 0-indexed character arrays `original` and `changed`, and an integer array `cost`, where `cost[i]` represents the cost of changing the character `original[i]` to the character `changed[i]`.

You start with the string `source`. In one operation, you can pick a character `x` from the string and change it to the character `y` at a cost of `z` if there exists any index `j` such that `cost[j] == z`, `original[j] == x`, and `changed[j] == y`.

Return the minimum cost to convert the string `source` to the string `target` using any number of operations. If it is impossible to convert `source` to `target`, return `-1`.

Note that there may exist indices i, j such that $\text{original}[j] == \text{original}[i]$ and $\text{changed}[j] == \text{changed}[i]$.

Example 1:

Input: source = "abcd", target = "acbe", original = ["a","b","c","c","e","d"],
changed = ["b","c","b","e","b","e"], cost = [2,5,5,1,2,20]

Output: 28

Explanation: To convert the string "abcd" to string "acbe":

- Change value at index 1 from 'b' to 'c' at a cost of 5.
- Change value at index 2 from 'c' to 'e' at a cost of 1.
- Change value at index 2 from 'e' to 'b' at a cost of 2.
- Change value at index 3 from 'd' to 'e' at a cost of 20.

The total cost incurred is $5 + 1 + 2 + 20 = 28$.

Example 2:

Input: source = "aaaa", target = "bbbb", original = ["a","c"], changed =
["c","b"], cost = [1,2]

Output: 12

Explanation: To change the character 'a' to 'b' change the character 'a' to 'c' at a cost of 1, followed by changing the character 'c' to 'b' at a cost of 2, for a total cost of $1 + 2 = 3$. To change all occurrences of 'a' to 'b', a total cost of $3 * 4 = 12$ is incurred.

Example 3:

Input: source = "abcd", target = "abce", original = ["a"], changed = ["e"],
cost = [10000]

Output: -1

Explanation: It is impossible to convert source to target because the value at index 3 cannot be changed from 'd' to 'e'.

Constraints:

- $1 \leq \text{source.length} == \text{target.length} \leq 105$
- source, target consist of lowercase English letters.
- $1 \leq \text{cost.length} == \text{original.length} == \text{changed.length} \leq 2000$

- `original[i]`, `changed[i]` are lowercase English letters.
- $1 \leq \text{cost}[i] \leq 106$
- `original[i] != changed[i]`

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minimumCost(
        self,
        source: str,
        target: str,
        original: list[str],
        changed: list[str],
        cost: list[int],
    ) -> int:
        ans = 0

        # dist[u][v] := the minimum distance to change ('a' + u) to ('a' + v)
        dist = [[math.inf] * 26 for _ in range(26)]

        for a, b, c in zip(original, changed, cost):
            u = ord(a) - ord('a')
            v = ord(b) - ord('a')
            dist[u][v] = min(dist[u][v], c)

        for k in range(26):
            for i in range(26):
```

```

        if dist[i][k] < math.inf:
            for j in range(26):
                if dist[k][j] < math.inf:
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

    for s, t in zip(source, target):
        if s == t:
            continue
        u = ord(s) - ord('a')
        v = ord(t) - ord('a')

        if dist[u][v] == math.inf:
            return -1
        ans += dist[u][v]

    return ans

```

• LeetDoocs

```

class Solution:
    def minimumCost(
        self,
        source: str,
        target: str,
        original: List[str],
        changed: List[str],
        cost: List[int],
    ) -> int:
        g = [[inf] * 26 for _ in range(26)]

        for i in range(26):
            g[i][i] = 0
        for x, y, z in zip(original, changed, cost):
            x = ord(x) - ord('a')
            y = ord(y) - ord('a')
            g[x][y] = min(g[x][y], z)
        for k in range(26):
            for i in range(26):
                for j in range(26):
                    g[i][j] = min(g[i][j], g[i][k] + g[k][j])

```

```

ans = 0
for a, b in zip(source, target):
    if a != b:
        x, y = ord(a) - ord('a'), ord(b) - ord('a')
        if g[x][y] >= inf:
            return -1
        ans += g[x][y]
return ans

```

• SimplyLeet

```

# Python solution with line-by-line comments
def minimumCost(source, target, original, changed, cost):
    # There are 26 letters, so we use an indexed list to represent the
    transformation costs
    INF = float('inf')
    n_letters = 26
    # Initialize the cost matrix with INF and 0 cost for same letter
    transformations
    dist = [[INF] * n_letters for _ in range(n_letters)]
    for i in range(n_letters):
        dist[i][i] = 0

    # Update the cost matrix with the provided operations
    for op_orig, op_changed, op_cost in zip(original, changed, cost):
        i = ord(op_orig) - ord('a')
        j = ord(op_changed) - ord('a')
        # Ensure we store the minimum cost if multiple operations exist
        dist[i][j] = min(dist[i][j], op_cost)

    # Apply Floyd Warshall to compute minimum cost for all pairs of letters
    for k in range(n_letters):
        for i in range(n_letters):

```

```

        for j in range(n_letters):
            if dist[i][k] + dist[k][j] < dist[i][j]:
                dist[i][j] = dist[i][k] + dist[k][j]

# Calculate the total conversion cost from source to target
total_cost = 0
for s_char, t_char in zip(source, target):
    # If the characters are the same, no cost is needed
    if s_char == t_char:
        continue

    i = ord(s_char) - ord('a')
    j = ord(t_char) - ord('a')
    # If the conversion is impossible, return -1
    if dist[i][j] == INF:
        return -1
    total_cost += dist[i][j]
return total_cost

# Example usage:
print(minimumCost("abcd", "acbe", ["a","b","c","c","e","d"], ["b","c","b","e",
"b","e"], [2,5,5,1,2,20]))

print(minimumCost("aaaa", "bbbb", ["a","c"], ["c","b"], [1,2]))
print(minimumCost("abcd", "abce", ["a"], ["e"], [10000]))

```

◆ Quick Review

Key Insights

- Model the allowed transformations as a directed weighted graph where nodes represent letters.
- For each allowed transformation, update the edge with the minimum cost if multiple operations exist for the same pair.

- Use the Floyd Warshall algorithm to compute the minimum conversion cost between every pair of letters.
- For each position in the source string, if the minimal cost to convert the source letter to the target letter is not available (infinite), the conversion is impossible.
- Sum up the conversion cost for each character position, applying the transformation cost computed via the graph.

Space & Time

Time Complexity: $O(26^3 + n)$, where n is the length of the strings. The 26^3 part comes from the Floyd Warshall algorithm on 26 English letters.

Space Complexity: $O(26^2)$ for the cost matrix, plus $O(n)$ for input strings.

Solution

The problem can be solved by first modeling the transformations as a graph where each node is a letter ('a'–'z'). Every allowed operation creates a directed edge from letter x to letter y with a weight equal to the operation's cost. If there are multiple

operations between the same pair, we take the one with the smallest cost.

After constructing the graph, we apply the Floyd Warshall algorithm to compute the minimum cost path between every pair of letters. This gives us the cheapest way to transform any letter into any other letter (possibly via intermediate transformations).

Finally, we iterate over each index of the source string. For each character, if the source and target letters are the same, no cost is incurred. Otherwise, we add the minimum cost to transform the source letter to the target letter based on our computed graph. If any transformation is impossible (i.e., remains infinite), we return -1 .

Shortest Path

□ #3341 Find Minimum Time to Reach Last Room I **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Graph Theory · Heap (Priority Queue) ·
Matrix · Shortest Path
Lists: AlgoMaster 600

Problem

There is a dungeon with $n \times m$ rooms arranged as a grid.

You are given a 2D array `moveTime` of size $n \times m$, where `moveTime[i][j]` represents the minimum time in seconds after which the room opens and can be moved to. You start from the room $(0, 0)$ at time $t = 0$ and can move to an adjacent room. Moving between adjacent rooms takes exactly one second.

Return the minimum time to reach the room $(n - 1, m - 1)$.

Two rooms are adjacent if they share a common wall, either horizontally or vertically.

Example 1:

Input: moveTime = [[0,4],[4,4]]

Output: 6

Explanation:

The minimum time required is 6 seconds.

- At time $t == 4$, move from room (0, 0) to room (1, 0) in one second.
- At time $t == 5$, move from room (1, 0) to room (1, 1) in one second.

Example 2:

Input: moveTime = [[0,0,0],[0,0,0]]

Output: 3

Explanation:

The minimum time required is 3 seconds.

- At time $t == 0$, move from room (0, 0) to room (1, 0) in one second.
- At time $t == 1$, move from room (1, 0) to room (1, 1) in one second.
- At time $t == 2$, move from room (1, 1) to room (1, 2) in one second.

Example 3:

Input: moveTime = [[0,1],[1,2]]

Output: 3

Constraints:

- $2 \leq n \leq \text{moveTime.length} \leq 50$
- $2 \leq m \leq \text{moveTime}[i].\text{length} \leq 50$
- $0 \leq \text{moveTime}[i][j] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minTimeToReach(self, moveTime: list[list[int]]) -> int:
        return self._dijkstra(moveTime,
                               (0, 0), (len(moveTime) - 1, len(moveTime[0]) - 1))

    def _dijkstra(
        self,
        moveTime: list[list[int]],
        src: tuple[int, int],
        dst: tuple[int, int]
    ) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(moveTime)
        n = len(moveTime[0])
        dist = [[math.inf] * n for _ in range(m)]

        dist[0][0] = 0
        minHeap = [(0, src)]  # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if u == dst:
                return d
            i, j = u
            if d > dist[i][j]:
                continue
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
```

```

        continue
    newDist = max(moveTime[x][y], d) + 1
    if newDist < dist[x][y]:
        dist[x][y] = newDist
        heapq.heappush(minHeap, (newDist, (x, y)))

return -1

```

• LeetCode

```

class Solution:
    def minTimeToReach(self, moveTime: List[List[int]]) -> int:
        n, m = len(moveTime), len(moveTime[0])
        dist = [[inf] * m for _ in range(n)]
        dist[0][0] = 0
        pq = [(0, 0, 0)]
        dirs = (-1, 0, 1, 0, -1)
        while 1:
            d, i, j = heappop(pq)
            if i == n - 1 and j == m - 1:
                return d
            if d > dist[i][j]:
                continue
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < n and 0 <= y < m:
                    t = max(moveTime[x][y], dist[i][j]) + 1
                    if dist[x][y] > t:
                        dist[x][y] = t
                        heapq.heappush(pq, (t, x, y))

```

• SimplyLeet

```
import heapq

def minTimeToReachLastRoom(moveTime):
    n = len(moveTime)
    m = len(moveTime[0])

    # Directions for adjacent moves: up, right, down, left
    directions = [(-1, 0), (0, 1), (1, 0), (0, -1)]

    # Initialize distance matrix with infinity and set start (0,0)

    dist = [[float('inf')] * m for _ in range(n)]
    dist[0][0] = 0

    # Priority queue holding (current time, row, column)
    heap = [(0, 0, 0)]

    while heap:
        current_time, row, col = heapq.heappop(heap)

        # If destination is reached, return the current time

        if row == n - 1 and col == m - 1:
            return current_time

        # Skip if a better route to this cell is already known
        if current_time > dist[row][col]:
            continue

        # Explore adjacent cells
        for dr, dc in directions:
            new_row, new_col = row + dr, col + dc
```

```

        if 0 <= new_row < n and 0 <= new_col < m:
            # Calculate arrival time: moving takes 1 second and may require
            waiting
            next_time = max(current_time + 1, moveTime[new_row][new_col])
            if next_time < dist[new_row][new_col]:
                dist[new_row][new_col] = next_time
                heapq.heappush(heap, (next_time, new_row, new_col))

    return -1

# Example test case:
print(minTimeToReachLastRoom([[0,4],[4,4]])) # Expected output: 6

```

◆ Quick Review

Key Insights

- Model this as a shortest path problem on a grid where each move costs 1 second, but you might incur waiting time.
- Use Dijkstra's algorithm with a min-heap (priority queue) to efficiently track the earliest arrival times.
- For each move, the effective arrival time at a neighbor is $\max(\text{current time} + 1, \text{moveTime at the neighbor})$.
- Avoid processing outdated routes by checking if a better time for a cell has already been found.

Space & Time

Time Complexity: $O(n * m * \log(n*m))$ due to processing each cell with a min-heap.

Space Complexity: $O(n * m)$ for storing the distance (time) matrix.

Solution

We use a Dijkstra-like algorithm that leverages a min-heap (or priority queue) to always choose the room that can be reached the earliest. For each neighboring room, we calculate the new arrival time using 1 second for movement and possibly waiting until the room's moveTime if necessary. If this new time is better than the previously recorded time, we update and add the neighbor to our heap.

Shortest Path

□ #3650 Minimum Cost Path with Edge Reversals

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Graph Theory · Heap (Priority Queue) · Shortest Path

Lists: AlgoMaster 600

Problem

You are given a directed, weighted graph with n nodes labeled from 0 to $n - 1$, and an array `edges` where `edges[i] = [ui, vi, wi]` represents a directed edge from node ui to node vi with cost wi .

Each node ui has a switch that can be used at most once: when you arrive at ui and have not yet used its switch, you may activate it on one of its incoming edges $vi \rightarrow ui$ reverse that edge to $ui \rightarrow vi$ and immediately traverse it.

The reversal is only valid for that single move, and using a reversed edge costs $2 * wi$.

Return the minimum total cost to travel from node 0 to node $n - 1$. If it is not possible, return

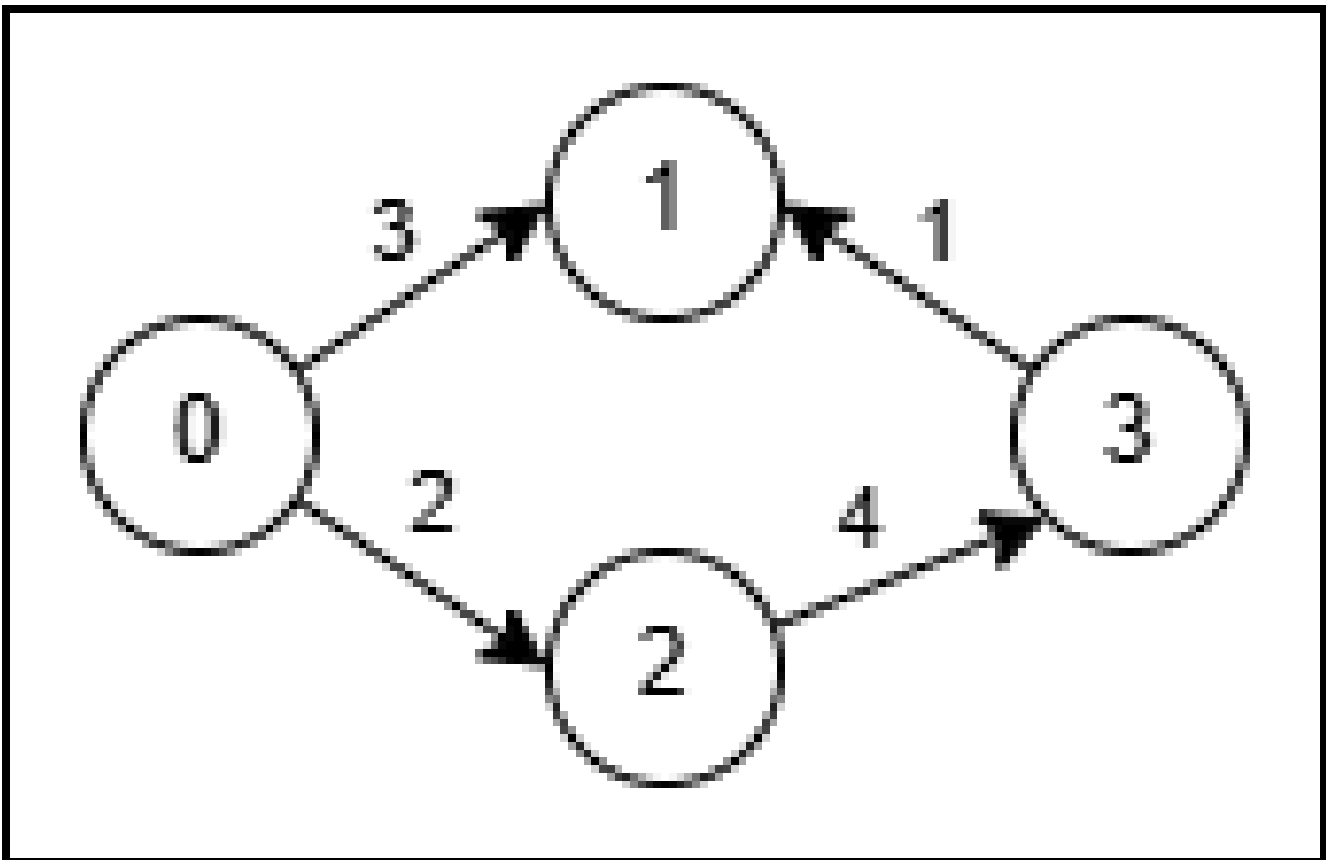
-1.

Example 1:

Input: $n = 4$, $\text{edges} =$
[[0,1,3],[3,1,1],[2,3,4],[0,2,2]]

Output: 5

Explanation:



- Use the path $0 \rightarrow 1$ (cost 3).
- At node 1 reverse the original edge $3 \rightarrow 1$ into $1 \rightarrow 3$ and traverse it at cost $2 * 1 = 2$.
- Total cost is $3 + 2 = 5$.

Example 2:

Input: $n = 4$, $\text{edges} =$
 $[[0,2,1],[2,1,1],[1,3,1],[2,3,3]]$

Output: 3

Explanation:

- No reversal is needed. Take the path $0 \rightarrow 2$ (cost 1), then $2 \rightarrow 1$ (cost 1), then $1 \rightarrow 3$ (cost 1).
- Total cost is $1 + 1 + 1 = 3$.

Constraints:

- $2 \leq n \leq 5 * 10^4$
- $1 \leq \text{edges.length} \leq 10^5$
- $\text{edges}[i] = [\text{ui}, \text{vi}, \text{wi}]$
- $0 \leq \text{ui}, \text{vi} \leq n - 1$
- $1 \leq \text{wi} \leq 1000$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def minCost(self, n: int, edges: List[List[int]]) -> int:
        g = [[] for _ in range(n)]
        for u, v, w in edges:
            g[u].append((v, w))
            g[v].append((u, w * 2))
        pq = [(0, 0)]
        dist = [inf] * n
        dist[0] = 0
        while pq:
```

```
d, u = heappop(pq)
if d > dist[u]:
    continue
if u == n - 1:
    return d
for v, w in g[u]:
    nd = d + w
    if nd < dist[v]:
        dist[v] = nd
        heappush(pq, (nd, v))

return -1
```

Round 6

Mid Priority · Hard

30 questions · 15 patterns

-
- ☐ ● Monotonic Stack #321 #1944 ↗
 - ☐ ● Monotonic Queue #862 #1499 ↗
 - ☐ ● Recursion #273 #761 ↗
 - ☐ ● Merge Sort #327 #493 ↗
 - ☐ ● Backtracking #301 #980 ↗
 - ☐ ● Tries #425 #472 #1032 ↗
 - ☐ ● Heaps #1383 ↗
 - ☐ ● Two Heaps #480 #502 ↗
 - ☐ ● Intervals #2402 ↗
 - ☐ ● Data Structure Design #715 #2296 ↗
 - ☐ ● Greedy #135 #757 #857 #871 ↗
 - ☐ ● Topological Sort #1203 ↗
 - ☐ ● Union Find #924 ↗

Minimum Spanning Tree #1168 #1489 #3600
Shortest Path #1368 #2203

Monotonic Stack

Round 6 · Mid · Hard · 2 questions

Monotonic Stack

□ #321 Create Maximum Number

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Stack · Greedy ·
Monotonic Stack
Lists: AlgoMaster 600

Problem

You are given two integer arrays `nums1` and `nums2` of lengths `m` and `n` respectively. `nums1` and `nums2` represent the digits of two numbers. You are also given an integer `k`.

Create the maximum number of length $k \leq m + n$ from digits of the two numbers. The relative order of the digits from the same array must be preserved.

Return an array of the `k` digits representing the answer.

Example 1:

Input: `nums1 = [3,4,6,5]`, `nums2 = [9,1,2,5,8,3]`, `k = 5`
Output: `[9,8,6,5,3]`

Example 2:

Input: nums1 = [6,7], nums2 = [6,0,4], k = 5
 Output: [6,7,6,0,4]

Example 3:

Input: nums1 = [3,9], nums2 = [8,9], k = 3
 Output: [9,8,9]

Constraints:

- $m == \text{nums1.length}$
- $n == \text{nums2.length}$
- $1 \leq m, n \leq 500$
- $0 \leq \text{nums1}[i], \text{nums2}[i] \leq 9$
- $1 \leq k \leq m + n$
- nums1 and nums2 do not have leading zeros.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxNumber(self, nums1: list[int], nums2: list[int], k: int) -> list[int]:
        :
        def maxArray(nums: list[int], k: int) -> list[int]:
            res = []
            toTop = len(nums) - k
            for num in nums:
                while res and res[-1] < num and toTop > 0:
                    res.pop()
                    toTop -= 1
                res.append(num)
```

```

    return res[:k]

def merge(nums1: List[int], nums2: List[int]) -> List[int]:
    return [max(nums1, nums2).pop(0) for _ in nums1 + nums2]

return max(merge(maxArray(nums1, i), maxArray(nums2, k - i))
           for i in range(k + 1)
           if i <= len(nums1) and k - i <= len(nums2))

```

• LeetDoocs

```

class Solution:
    def maxNumber(self, nums1: List[int], nums2: List[int], k: int) -> List[int]:
        def f(nums: List[int], k: int) -> List[int]:
            n = len(nums)
            stk = [0] * k
            top = -1
            remain = n - k
            for x in nums:
                while top >= 0 and stk[top] < x and remain > 0:
                    top -= 1
                    remain -= 1
                if top + 1 < k:
                    top += 1
                    stk[top] = x
            else:
                remain -= 1
            return stk

        def compare(nums1: List[int], nums2: List[int], i: int, j: int) -> bool:
            if i >= len(nums1):

```



```

        return False
    if j >= len(nums2):
        return True
    if nums1[i] > nums2[j]:
        return True
    if nums1[i] < nums2[j]:
        return False
    return compare(nums1, nums2, i + 1, j + 1)

def merge(nums1: List[int], nums2: List[int]) -> List[int]:

```

```

    m, n = len(nums1), len(nums2)
    i = j = 0
    ans = [0] * (m + n)
    for k in range(m + n):
        if compare(nums1, nums2, i, j):
            ans[k] = nums1[i]
            i += 1
        else:
            ans[k] = nums2[j]
            j += 1

```

```

    return ans

```

```

    m, n = len(nums1), len(nums2)
    l, r = max(0, k - n), min(k, m)
    ans = [0] * k
    for x in range(l, r + 1):
        arr1 = f(nums1, x)
        arr2 = f(nums2, k - x)
        arr = merge(arr1, arr2)
        if ans < arr:

```

```

            ans = arr
    return ans

```

• SimplyLeet

```

def maxNumber(nums1, nums2, k):
    # Helper function to get the maximum subsequence of given length t from
    array nums
    def getMaxSubsequence(nums, t):
        drop = len(nums) - t # Number of elements that can be dropped
        stack = []
        for num in nums:
            # Remove smaller digits when possible to achieve a larger leading
            number
            while drop and stack and stack[-1] < num:
                stack.pop()
                drop -= 1

            stack.append(num)
        # Limit the result to length t
        return stack[:t]

    # Helper function to merge two subsequences into the maximum possible
    number
    def merge(seq1, seq2):
        merged = []
        while seq1 or seq2:
            # Lexicographical comparison of remaining parts
            if seq1 > seq2:
                merged.append(seq1[0])
                seq1 = seq1[1:]
            else:
                merged.append(seq2[0])
                seq2 = seq2[1:]
        return merged

    max_combo = []
    m, n = len(nums1), len(nums2)
    # Iterate over all possible splits: select i digits from nums1 and k-i from
    nums2

```

```

for i in range(max(0, k - n), min(k, m) + 1):
    subseq1 = getMaxSubsequence(nums1, i)
    subseq2 = getMaxSubsequence(nums2, k - i)
    candidate = merge(subseq1, subseq2)
    if candidate > max_combo:
        max_combo = candidate
return max_combo

# Example usage:
print(maxNumber([3,4,6,5], [9,1,2,5,8,3], 5)) # Expected Output: [9,8,6,5,3]

```

◆ Quick Review

Key Insights

- Use a greedy, monotonic stack approach to select the maximum subsequence of a given length from a single array.
- Merge two maximum subsequences by lexicographically comparing their remaining parts.
- Try all possible splits of k digits between the two arrays to ensure the overall maximum is obtained.

Space & Time

Time Complexity: $O((m+n) * k^2)$ in the worst case due to iterating splits and comparing subsequences during merging

Space Complexity: $O(k)$ for storing subsequences and the merged result

Solution

The solution consists of three main steps:

- For each array, utilize a helper function to compute the maximum subsequence of a given length using a monotonic stack. This guarantees that the highest possible digits are chosen while preserving order.**
- Merge two subsequences into the maximum possible result by repeatedly selecting the lexicographically larger sequence among the remaining parts.**
- Iterate over all valid ways to split k between the two arrays (from $\max(0, k - \text{length}(\text{nums2}))$ to $\min(k, \text{length}(\text{nums1}))$) and record the best merged result.**

Data structures employed include arrays and stacks. The algorithmic approach is greedy and leverages lexicographical comparison to merge sequences optimally.

Monotonic Stack

□ #1944 Number of Visible People in a Queue

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: AlgoMaster 600

Problem

There are n people standing in a queue, and they numbered from 0 to $n - 1$ in left to right order. You are given an array `heights` of distinct integers where `heights[i]` represents the height of the i th person.

A person can see another person to their right in the queue if everybody in between is shorter than both of them. More formally, the i th person can see the j th person if $i < j$ and $\min(\text{heights}[i], \text{heights}[j]) > \max(\text{heights}[i+1], \text{heights}[i+2], \dots, \text{heights}[j-1])$.

Return an array `answer` of length n where `answer[i]` is the number of people the i th person can see to their right in the queue.

Example 1:



Input: heights = [10,6,8,5,11,9]

Output: [3,1,2,1,1,0]

Explanation:

Person 0 can see person 1, 2, and 4.

Person 1 can see person 2.

Person 2 can see person 3 and 4.

Person 3 can see person 4.

Person 4 can see person 5.

Example 2:

Input: heights = [5,1,2,3,10]

Output: [4,1,1,1,0]

Constraints:

- $n == \text{heights.length}$
- $1 \leq n \leq 105$
- $1 \leq \text{heights}[i] \leq 105$
- All the values of heights are unique.

Community Solutions (Python)

- WalkCC

```

class Solution:
    def canSeePersonsCount(self, heights: list[int]) -> list[int]:
        ans = [0] * len(heights)
        stack = []

        for i, height in enumerate(heights):
            while stack and heights[stack[-1]] <= height:
                ans[stack.pop()] += 1
            if stack:
                ans[stack[-1]] += 1

            stack.append(i)

        return ans

```

• LeetCode

```

class Solution:
    def canSeePersonsCount(self, heights: List[int]) -> List[int]:
        n = len(heights)
        ans = [0] * n
        stk = []
        for i in range(n - 1, -1, -1):
            while stk and stk[-1] < heights[i]:
                ans[i] += 1
                stk.pop()
            if stk:
                ans[i] += 1
            stk.append(heights[i])
        return ans

```

• SimplyLeet

```
# Python solution using a monotonic stack

def canSeePersonsCount(heights):
    n = len(heights) # Total number of people in the queue
    answer = [0] * n # Initialize the answer array with zeros
    stack = [] # Stack to keep track of heights

    # Iterate from rightmost person to leftmost person
    for i in range(n - 1, -1, -1):
        count = 0 # Count for the number of people the current person can see

        # Pop all people shorter than the current person as they are visible
        while stack and heights[i] > stack[-1]:
            stack.pop()
            count += 1 # Each popped person is visible from the current
person's view

        # If the stack is not empty, the current person can see one more person
(taller than them)
        if stack:
            count += 1

    answer[i] = count # Save the computed count for the current person
    stack.append(heights[i]) # Push the current person's height onto the
stack

    return answer

# Example usage:
# heights = [10,6,8,5,11,9] should output [3,1,2,1,1,0]
print(canSeePersonsCount([10,6,8,5,11,9]))
```

◆ Quick Review

Key Insights

- The problem requires determining the visibility count for each person based on

height comparisons.

- A monotonic stack is an effective tool to “simulate” the view from each person.
- By iterating from right-to-left, we can efficiently maintain a stack containing potential visible persons.
- For each current person, pop all shorter persons from the stack (they are visible) and if the stack is not empty afterward, it means the next taller person is still visible.
- The use of a monotonic stack allows the solution to run in $O(n)$ time by processing each element only once.

Space & Time

Time Complexity: $O(n)$ – each element is pushed and popped at most once.

Space Complexity: $O(n)$ – in the worst-case scenario, the stack can hold all n elements.

Solution

We solve this problem by iterating through the queue from right to left while maintaining a monotonic stack. The stack stores the heights of people in a decreasing order (from top to bottom). For each person, we perform the

following steps:

- Initialize a count for visible persons.**
- While the stack is not empty and the current person's height is greater than the height on the top of the stack, we pop the stack. Each popped element represents a person that the current person can see.**
- If the stack remains non-empty after the popping process, the current person can also see the next taller person (the one currently at the top of the stack).**
- Store the computed count in the result array and push the current person's height onto the stack. This approach guarantees that each person's visibility count is computed efficiently in a single pass using a monotonic stack.**

Monotonic Queue

Round 6 · Mid · Hard · 2 questions

Monotonic Queue

□ #862 Shortest Subarray with Sum at Least K

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Queue · Sliding Window
· Heap (Priority Queue) · Prefix Sum ·
Monotonic Queue

Lists: AlgoMaster 600

Problem

Given an integer array `nums` and an integer `k`, return the length of the shortest non-empty subarray of `nums` with a sum of at least `k`. If there is no such subarray, return `-1`.

A subarray is a contiguous part of an array.

Example 1:

Input: `nums = [1]`, `k = 1`
Output: `1`

Example 2:

Input: `nums = [1,2]`, `k = 4`
Output: `-1`

Example 3:

Input: `nums = [2,-1,2]`, `k = 3`
Output: `3`

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-105 \leq \text{nums}[i] \leq 105$
- $1 \leq k \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def shortestSubarray(self, nums: list[int], k: int) -> int:
        n = len(nums)
        ans = n + 1
        dq = collections.deque()
        prefix = list(itertools.accumulate(nums, initial=0))

        for i in range(n + 1):
            while dq and prefix[i] - prefix[dq[0]] >= k:
                ans = min(ans, i - dq.popleft())

            while dq and prefix[i] <= prefix[dq[-1]]:
                dq.pop()
            dq.append(i)

        return ans if ans <= n else -1
```

• LeetDoocs

```
class Solution:
    def shortestSubarray(self, nums: List[int], k: int) -> int:
        s = list(accumulate(nums, initial=0))
        q = deque()
        ans = inf
        for i, v in enumerate(s):
            while q and v - s[q[0]] >= k:
                ans = min(ans, i - q.popleft())
            while q and s[q[-1]] >= v:
                q.pop()
            q.append(i)
```

```

        q.append(i)
    return -1 if ans == inf else ans

```

• SimplyLeet

```

# Python solution for Shortest Subarray with Sum at Least K
from collections import deque

def shortestSubarray(nums, k):
    # Build the prefix sums, starting with an initial 0
    prefix_sums = [0]
    for num in nums:
        prefix_sums.append(prefix_sums[-1] + num)

    # Deque to store indices of prefix_sums in increasing order

    d = deque()
    res = float('inf')

    # Iterate over prefix sums
    for i, current_sum in enumerate(prefix_sums):
        # If the difference between current and oldest prefix sum is at least
        k,

        # update the result and remove the element from deque front.
        while d and current_sum - prefix_sums[d[0]] >= k:
            res = min(res, i - d[0])
            d.popleft()

        # Maintain increasing order of prefix sums in deque
        while d and prefix_sums[d[-1]] >= current_sum:
            d.pop()

        # Add current index
        d.append(i)

    return res if res != float('inf') else -1

# Example usage
if __name__ == "__main__":
    print(shortestSubarray([2, -1, 2], 3)) # Expected output: 3

```

◆ Quick Review

Key Insights

- Use a prefix-sum array to quickly compute the sum of any subarray.
- The difference between two prefix sums gives the sum of the subarray between their indices.
- Maintain a monotonic deque that stores indices of the prefix-sum array in increasing order.
- As you iterate through the prefix sums, check and update the answer when the difference meets or exceeds k .
- Remove indices from the deque that are no longer potential candidates (either because they yield a subarray that is too long or their prefix sum is not optimal).

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in `nums`.

Space Complexity: $O(n)$ due to storing the prefix sum array and the deque.

Solution

We solve the problem by first constructing a prefix-sum array where each element $p[i+1]$ equals $p[i]$ plus $nums[i]$. For any two indices i and j (with $i < j$), the sum of the subarray $nums[i]$ to $nums[j-1]$ is $p[j] - p[i]$. We need this difference to be at least k . By using a deque, we can efficiently track and check for the shortest valid subarray.

The deque is maintained in increasing order of prefix sums:

- As we iterate through the prefix sums, if the current prefix sum minus the smallest prefix sum from the deque is at least k , we update the minimum length.
- We remove indices from the back if they offer no benefit (i.e., having a larger prefix sum when a smaller one has already been seen).
- This ensures that we always consider the best candidate indices for forming a valid subarray.

Monotonic Queue

□ #1499 Max Value of Equation

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Queue · Sliding Window · Heap (Priority Queue) · Monotonic Queue
Lists: AlgoMaster 600

Problem

You are given an array `points` containing the coordinates of points on a 2D plane, sorted by the `x`-values, where `points[i] = [xi, yi]` such that $x_i < x_j$ for all $1 \leq i < j \leq \text{points.length}$. You are also given an integer `k`.

Return the maximum value of the equation $y_i + y_j + |x_i - x_j|$ where $|x_i - x_j| \leq k$ and $1 \leq i < j \leq \text{points.length}$.

It is guaranteed that there exists at least one pair of points that satisfy the constraint $|x_i - x_j| \leq k$.

Example 1:

Input: points = [[1,3],[2,0],[5,10],[6,-10]], k = 1

Output: 4

Explanation: The first two points satisfy the condition $|x_i - x_j| \leq 1$ and if we calculate the equation we get $3 + 0 + |1 - 2| = 4$. Third and fourth points also satisfy the condition and give a value of $10 + -10 + |5 - 6| = 1$. No other pairs satisfy the condition, so we return the max of 4 and 1.

Example 2:

Input: points = [[0,0],[3,0],[9,2]], k = 3

Output: 3

Explanation: Only the first two points have an absolute difference of 3 or less in the x-values, and give the value of $0 + 0 + |0 - 3| = 3$.

Constraints:

- $2 \leq \text{points.length} \leq 105$
- $\text{points}[i].\text{length} == 2$
- $-108 \leq x_i, y_i \leq 108$
- $0 \leq k \leq 2 * 108$
- $x_i < x_j$ for all $1 \leq i < j \leq \text{points.length}$
- x_i form a strictly increasing sequence.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMaxValueOfEquation(self, points: list[list[int]], k: int) -> int:
        ans = -math.inf
        maxHeap = [] # (y - x, x)

        for x, y in points:
            while maxHeap and x + maxHeap[0][1] > k:
                heapq.heappop(maxHeap)
            if maxHeap:
                ans = max(ans, x + y - maxHeap[0][0])
```

```
heapq.heappush(maxHeap, (x - y, -x))
```

```
return ans
```

• LeetCode

```
class Solution:
    def findMaxValueOfEquation(self, points: List[List[int]], k: int) -> int:
        ans = -inf
        pq = []
        for x, y in points:
            while pq and x - pq[0][1] > k:
                heapq.heappop(pq)
            if pq:
                ans = max(ans, x + y - pq[0][0])
                heapq.heappush(pq, (x - y, x))

        return ans
```

• SimplyLeet

```
# Python solution implementation with detailed comments.
from collections import deque

def findMaxValueOfEquation(points, k):
    # Initialize deque to store tuples of (x, y+x) in a monotonic decreasing
    # order for y+x values.
    mono_deque = deque()
    max_value = float('-inf')

    # Iterate over each point in the sorted list
    for x, y in points:
```

```

    # Remove points from the front that are out of the window (x difference
    > k)
    while mono_deque and x - mono_deque[0][0] > k:
        mono_deque.popleft()

    # If deque is not empty, a candidate exists to maximize the equation.
    if mono_deque:
        # The candidate is the current point's (y-x) plus the maximum (y+x)
        from the deque front.
        max_value = max(max_value, y - x + mono_deque[0][1])

    # Prepare the current value for future comparisons

    current_value = y + x

    # Before appending the current point, pop from the back if
    current_value is greater
    while mono_deque and mono_deque[-1][1] <= current_value:
        mono_deque.pop()

    # Append the current point (store x and current value)
    mono_deque.append((x, current_value))

    return max_value

# Example usage:
# points = [[1,3],[2,0],[5,10],[6,-10]]
# k = 1
# print(findMaxValueOfEquation(points, k)) # Expected output: 4

```

◆ Quick Review

Key Insights

- The equation can be rewritten as $(y_i + x_i) + (y_j - x_j)$ since $x_i \leq x_j$.
- For every current point j , we are looking to pair it with a previous point i that maximizes

$(y_i + x_i)$, provided that $x_j - x_i \leq k$.

- Use a sliding window approach based on x -values since points are sorted.
- A deque (monotonic queue) is well-suited to maintain potential candidates for $(y_i + x_i)$ in decreasing order while ensuring the window constraint.
- As you iterate through points, remove points that are outside the valid window and update the maximum answer accordingly.

Space & Time

Time Complexity: $O(n)$ – Each point is processed once and each is added/removed from the deque at most once.

Space Complexity: $O(n)$ – In the worst-case scenario, the deque can contain all points.

Solution

The solution involves the following steps:

- Transform the equation into a form where for each point j , you want to maximize $(y_i + x_i)$ from a previous point i that satisfies $x_j - x_i \leq k$.

- Use a deque to store indices (or the computed value along with x_i) such that the values $(y_i + x_i)$ are maintained in decreasing order.
- For the current point j , first remove any points from the front of the deque that are no longer within the valid x -window (i.e., where $x_j - x_i > k$).
- If the deque is not empty, compute a candidate answer as $(\text{current } y_j - \text{current } x_j) + (\text{front value in deque})$ and update the maximum answer.
- Before moving to the next point, maintain the monotonicity of the deque by removing from the back any point whose $(y_i + x_i)$ value is less than or equal to the current point's value $(y_j + x_j)$ as they will never be optimal for future points.
- Append the current point to the deque.
- Return the maximum answer found.

Recursion

Round 6 · Mid · Hard · 2 questions

Recursion

□ #273 Integer to English Words

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Recursion
Lists: AlgoMaster 600

Problem

Convert a non-negative integer num to its English words representation.

Example 1:

Input: num = 123
Output: "One Hundred Twenty Three"

Example 2:

Input: num = 12345
Output: "Twelve Thousand Three Hundred Forty Five"

Example 3:

Input: num = 1234567
Output: "One Million Two Hundred Thirty Four Thousand Five Hundred Sixty Seven"

Constraints:

- $0 \leq \text{num} \leq 2^{31} - 1$

Community Solutions (Python)

- WalkCC


```

class Solution:
    def numberToWords(self, num: int) -> str:
        if num == 0:
            return 'Zero'

        belowTwenty = ['', 'One', 'Two', 'Three',
                        'Four', 'Five', 'Six', 'Seven',
                        'Eight', 'Nine', 'Ten', 'Eleven',
                        'Twelve', 'Thirteen', 'Fourteen', 'Fifteen',
                        'Sixteen', 'Seventeen', 'Eighteen', 'Nineteen']

        tens = ['', 'Ten', 'Twenty', 'Thirty', 'Forty',
                'Fifty', 'Sixty', 'Seventy', 'Eighty', 'Ninety']

        def helper(num: int) -> str:
            if num < 20:
                s = belowTwenty[num]
            elif num < 100:
                s = tens[num // 10] + ' ' + belowTwenty[num % 10]
            elif num < 1000:
                s = helper(num // 100) + ' Hundred ' + helper(num % 100)

            elif num < 1000000:
                s = helper(num // 1000) + ' Thousand ' + helper(num % 1000)
            elif num < 1000000000:
                s = helper(num // 1000000) + ' Million ' + helper(num % 1000000)
            else:
                s = helper(num // 1000000000) + ' Billion ' + helper(num % 1000000000)
            return s.strip()

        return helper(num)

```

• LeetDoocs

```
class Solution:
    def numberToWords(self, num: int) -> str:
        if num == 0:
            return 'Zero'

        lt20 = [
            '',
            'One',
            'Two',
            'Three',

            'Four',
            'Five',
            'Six',
            'Seven',
            'Eight',
            'Nine',
            'Ten',
            'Eleven',
            'Twelve',
            'Thirteen',

            'Fourteen',
            'Fifteen',
            'Sixteen',
            'Seventeen',
            'Eighteen',
            'Nineteen',
        ]
        tens = [
            '',
            'Ten',

            'Twenty',
            'Thirty',
            'Forty',
            'Fifty',
            'Sixty',
            'Seventy',
            'Eighty',
            'Ninety',
        ]
        thousands = ['Billion', 'Million', 'Thousand', '']
```

```
def transfer(num):
    if num == 0:
        return ''
    if num < 20:
        return lt20[num] + ' '
    if num < 100:
        return tens[num // 10] + ' ' + transfer(num % 10)
    return lt20[num // 100] + ' Hundred ' + transfer(num % 100)
```

```
res = []
i, j = 1000000000, 0
while i > 0:
    if num // i != 0:
        res.append(transfer(num // i))
        res.append(thousands[j])
        res.append(' ')
        num %= i
    j += 1
    i //= 1000
```

```
return ''.join(res).strip()
```

• SimplyLeet

```
# Define mappings for number words
ones = ['', "One", "Two", "Three", "Four", "Five", "Six", "Seven", "Eight", "Nine"]
teens = ["Ten", "Eleven", "Twelve", "Thirteen", "Fourteen", "Fifteen", "Sixteen", "Seventeen", "Eighteen", "Nineteen"]
tens = ['', '', "Twenty", "Thirty", "Forty", "Fifty", "Sixty", "Seventy", "Eighty", "Ninety"]
scales = ['', "Thousand", "Million", "Billion"]

def three_digit_to_words(num):
    # Convert a number less than 1000 to words
    if num == 0:
        return ""
```

```

result = ""
# Handle hundreds place
if num >= 100:
    result += ones[num // 100] + " Hundred "
    num %= 100
# Handle tens and ones
if num >= 20:
    result += tens[num // 10] + " "
    num %= 10
elif num >= 10: # Handle teen numbers specifically

    result += teens[num - 10] + " "
    num = 0
# Handle remaining ones (if any)
if num > 0:
    result += ones[num] + " "
return result

def number_to_words(num):
    # Special case: if number is 0, return "Zero"
    if num == 0:

        return "Zero"

    words = ""
    scale_index = 0
    # Process each group of 3 digits
    while num > 0:
        current_chunk = num % 1000 # take last three digits
        if current_chunk != 0:
            # Convert the current chunk and add the corresponding scale
            words = three_digit_to_words(current_chunk) + scales[scale_index]
        + " " + words

```

```
num //= 1000
scale_index += 1

# Return the cleaned up result
return words.strip()

# Example Usage:
print(number_to_words(123))      # Output: "One Hundred Twenty Three"
print(number_to_words(12345))   # Output: "Twelve Thousand Three Hundred
Forty Five"
print(number_to_words(1234567)) # Output: "One Million Two Hundred Thirty
Four Thousand Five Hundred Sixty Seven"
```

◆ Quick Review

Key Insights

- The number is processed in groups of three digits (hundreds, tens, and ones), mapping these groups to their word equivalents.
- Special handling is required for numbers between 10 and 19 (teens) because they have unique names.
- Larger groups beyond the basic hundred (thousand, million, billion) need to be appended with the appropriate scale.
- The solution is recursive or iterative in nature, processing each three-digit block separately and then combining them.
- Edge case: when the number is 0, the output should simply be "Zero".

Space & Time

Time Complexity: $O(1)$ — Although it appears to depend on the number of digits, the maximum input size is fixed (at most 10 digits for $2^{31} - 1$).

Space Complexity: $O(1)$ — Only a constant amount of extra space is used.

Solution

The solution involves breaking the integer into segments of three digits starting from the least significant digits. For each segment:

- Convert the hundreds, tens, and ones place into words.
- Handle the special case for numbers between 10 and 19.
- Append the appropriate scale (thousand, million, billion) based on the segment's position. The final result is constructed by concatenating the word representations of these segments (ignoring empty segments) in reverse order (most significant segment first).

Data Structures used:

- Arrays or lists for mapping digits, teens, tens and scales.

Algorithmic Approach:

- Define helper functions that convert a three-digit number into words.**
- Loop through each segment (every three digits) and convert if non-zero.**
- Concatenate segments with proper scale values.**
- Trim any extra spaces from the final output.**

Recursion

□ #761 Special Binary String

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Divide and Conquer · Sorting
Lists: AlgoMaster 600

Problem

Special binary strings are binary strings with the following two properties:

- The number of 0's is equal to the number of 1's.
- Every prefix of the binary string has at least as many 1's as 0's.

You are given a special binary string s .

A move consists of choosing two consecutive, non-empty, special substrings of s , and swapping them. Two strings are consecutive if the last character of the first string is exactly one index before the first character of the second string.

Return the lexicographically largest resulting string possible after applying the mentioned

operations on the string.

Example 1:

Input: `s = "11011000"`

Output: `"11100100"`

Explanation: The strings "10" [occurring at `s[1]`] and "1100" [at `s[3]`] are swapped.

This is the lexicographically largest string possible after some number of swaps.

Example 2:

Input: `s = "10"`

Output: `"10"`

Constraints:

- $1 \leq s.length \leq 50$
- `s[i]` is either '0' or '1'.
- `s` is a special binary string.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def makeLargestSpecial(self, s: str) -> str:
        specials = []
        count = 0

        i = 0
        for j, c in enumerate(s):
            count += 1 if c == '1' else -1
            if count == 0:
                specials.append(
                    '1' + self.makeLargestSpecial(s[i + 1:j]) + '0'
                )
                i = j + 1

        return ''.join(sorted(specials)[::-1])
```

• LeetCode

```
class Solution:
    def makeLargestSpecial(self, s: str) -> str:
        if s == '':
            return ''
        ans = []
        cnt = 0
        i = j = 0
        while i < len(s):
            cnt += 1 if s[i] == '1' else -1
            if cnt == 0:
                ans.append('1' + self.makeLargestSpecial(s[j + 1 : i]) + '0')
                j = i + 1
                i += 1
        ans.sort(reverse=True)
        return ''.join(ans)
```

• SimplyLeet

Python solution with detailed comments

```
def makeLargestSpecial(s: str) -> str:
    # Base case: if s is empty, return empty string
    if not s:
        return s

    segments = [] # List to hold valid special substrings
    count = 0 # Counter to track the balance of '1's and '0's
    start = 0 # Starting index for a potential special substring

    # Iterate over each character in string s
    for i, char in enumerate(s):
        # Increase counter for '1' and decrease for '0'
        if char == '1':
            count += 1
        else:
            count -= 1

        # When the count becomes 0, we found a valid special substring
```

```
    if count == 0:
        # Recursively process the inner part (excluding the outer '1' and
        # '0')
        inner = makeLargestSpecial(s[start+1:i])
        # Wrap the processed inner substring with outer characters '1' and
        # '0'
        segments.append("1" + inner + "0")
        start = i + 1 # Update start for the next segment

    # Sort segments in reverse lexicographical order to maximize value
    segments.sort(reverse=True)
    # Join sorted segments and return as final answer

    return "".join(segments)

# Example usage:
print(makeLargestSpecial("11011000"))
```

◆ Quick Review

Key Insights

- The problem has an inherent recursive structure where each valid special string can be broken down into smaller special strings.
- The inner structure between a matching pair of 1 and 0 (i.e. a segment "1" + sub-string + "0") is itself a special binary string.
- Once you recursively transform the inner segments, sorting these segments in descending order helps maximize the lexicographical order of the overall string.

- The crucial trick is recognizing that by recursively processing the inner parts and then reassembling them after sorting, you capture all possible valid swaps.

Space & Time

Time Complexity: $O(n^2 * \log n)$ in the worst-case scenario due to the recursive partitioning and sorting of segments.

Space Complexity: $O(n)$ for recursion depth and storage of sub-segments.

Solution

We solve the problem using a recursive approach. The algorithm works as follows:

- Initialize a counter and traverse string s .
- For every substring that forms a valid special binary string (when the counter reaches zero), recursively process the inner substring (after removing the leading '1' and trailing '0').
- Wrap the processed substring with '1' and '0' and add it to a list of candidate segments.
- Sort these segments in descending (lexicographically largest) order.

- Concatenate the sorted segments and return the result.

Key data structures used include:

- A list (or array) to hold candidate special substrings.
- Recursive function calls to process inner special strings.

This recursive decomposition ensures we explore all possible swaps implicitly, and sorting the segments yields the lexicographically largest result possible.

Merge Sort

Round 6 · Mid · Hard · 2 questions

Merge Sort

□ #327 Count of Range Sum

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Divide and Conquer ·
Binary Indexed Tree · Segment Tree · Merge
Sort · Ordered Set

Lists: AlgoMaster 600

Problem

Given an integer array `nums` and two integers `lower` and `upper`, return the number of range sums that lie in `[lower, upper]` inclusive.

Range sum $S(i, j)$ is defined as the sum of the elements in `nums` between indices `i` and `j` inclusive, where $i \leq j$.

Example 1:

Input: `nums = [-2,5,-1]`, `lower = -2`, `upper = 2`

Output: 3

Explanation: The three ranges are: `[0,0]`, `[2,2]`, and `[0,2]` and their respective sums are: -2, -1, 2.

Example 2:

Input: `nums = [0]`, `lower = 0`, `upper = 0`

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-231 \leq \text{nums}[i] \leq 231 - 1$
- $-105 \leq \text{lower} \leq \text{upper} \leq 105$
- The answer is guaranteed to fit in a 32-bit integer.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countRangeSum(self, nums: list[int], lower: int, upper: int) -> int:
        n = len(nums)
        self.ans = 0
        prefix = list(itertools.accumulate(nums, initial=0))

        self._mergeSort(prefix, 0, n, lower, upper)
        return self.ans

    def _mergeSort(
        self,
        prefix: list[int],
        l: int,
        r: int,
        lower: int,
        upper: int,
    ) -> None:
        if l >= r:
            return
```



```

m = (l + r) // 2
self._mergeSort(prefix, l, m, lower, upper)
self._mergeSort(prefix, m + 1, r, lower, upper)
self._merge(prefix, l, m, r, lower, upper)

def _merge(
    self,
    prefix: list[int],
    l: int,
    m: int,

    r: int,
    lower: int,
    upper: int,
) -> None:
    lo = m + 1 # the first index s.t. prefix[lo] - prefix[i] >= lower
    hi = m + 1 # the first index s.t. prefix[hi] - prefix[i] > upper

    # For each index i in range [l, m], add hi - lo to `ans`.
    for i in range(l, m + 1):
        while lo <= r and prefix[lo] - prefix[i] < lower:

            lo += 1
        while hi <= r and prefix[hi] - prefix[i] <= upper:
            hi += 1
        self.ans += hi - lo

    sorted = [0] * (r - l + 1)
    k = 0 # sorted's index
    i = l # left's index
    j = m + 1 # right's index

    while i <= m and j <= r:
        if prefix[i] < prefix[j]:
            sorted[k] = prefix[i]
            k += 1
            i += 1
        else:
            sorted[k] = prefix[j]
            k += 1
            j += 1

```

```

# Put the possible remaining left part into the sorted array.
while i <= m:
    sorted[k] = prefix[i]
    k += 1
    i += 1

# Put the possible remaining right part into the sorted array.
while j <= r:
    sorted[k] = prefix[j]
    k += 1

j += 1

prefix[l:l + len(sorted)] = sorted

```

• LeetCode

```

class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x, v):
        while x <= self.n:
            self.c[x] += v
            x += x & -x

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= x & -x
        return s

class Solution:
    def countRangeSum(self, nums: List[int], lower: int, upper: int) -> int:

```

```

s = list(accumulate(nums, initial=0))
arr = sorted(set(v for x in s for v in (x, x - lower, x - upper)))
tree = BinaryIndexedTree(len(arr))
ans = 0
for x in s:
    l = bisect_left(arr, x - upper) + 1
    r = bisect_left(arr, x - lower) + 1
    ans += tree.query(r) - tree.query(l - 1)
    tree.update(bisect_left(arr, x) + 1, 1)
return ans

```

• SimplyLeet

Python implementation using merge sort style divide and conquer

```

def countRangeSum(nums, lower, upper):
    # Build prefix sum array - prefix[i] is the sum of nums[0] up to nums[i-1].
    prefix = [0]
    for num in nums:
        prefix.append(prefix[-1] + num)

```

Helper function to count valid range sums during merge sort.

```

def merge_sort(start, end):

```

```

    if end - start <= 1: # Base case: one element is already sorted.
        return 0
    mid = (start + end) // 2
    count = merge_sort(start, mid) + merge_sort(mid, end)
    # Two pointers for the merging process
    j = k = mid
    temp = []
    r = mid
    # Iterate over the left part
    for left in prefix[start:mid]:

```

```

        # Increase k while the condition is not satisfied.
        while k < end and prefix[k] - left < lower:
            k += 1
        # Increase j while the condition is met.
        while j < end and prefix[j] - left <= upper:
            j += 1
        count += j - k

    # Standard merge sort merge operation.
    l, r = start, mid

```

```

    sorted_list = []
    while l < mid and r < end:
        if prefix[l] <= prefix[r]:
            sorted_list.append(prefix[l])
            l += 1
        else:
            sorted_list.append(prefix[r])
            r += 1
    # Append remaining elements.
    sorted_list.extend(prefix[l:mid])

```

```

    sorted_list.extend(prefix[r:end])
    # Write back sorted list into prefix.
    prefix[start:end] = sorted_list
    return count

```

```

return merge_sort(0, len(prefix))

```

```

# Example usage:

```

```

print(countRangeSum([-2,5,-1], -2, 2)) # Output: 3

```

◆ Quick Review

Key Insights

- Transform the problem using prefix sums. Define $\text{prefix}[i]$ as the sum of elements $\text{nums}[0]$ to $\text{nums}[i-1]$ which simplifies $S(i, j)$ to

$\text{prefix}[j+1] - \text{prefix}[i]$.

- The problem then reduces to counting, for every $\text{prefix}[j+1]$, how many earlier $\text{prefix}[i]$ satisfy: $\text{lower} \leq \text{prefix}[j+1] - \text{prefix}[i] \leq \text{upper}$.
- Use a modified merge sort (or other data structures like Binary Indexed Tree or Segment Tree) to count the number of valid pairs while sorting the prefix sum array.
- The merge step cleverly counts the number of valid prefix differences before merging two sorted halves.

Space & Time

Time Complexity: $O(n \log n)$ due to merge sort style divide and conquer.

Space Complexity: $O(n)$ for the prefix sum and temporary arrays used in merging.

Solution

We transform the input array `nums` into a prefix sum array. For each new prefix sum, we search the sorted list of previous prefix sums to count how many fall into the range $[\text{current} - \text{upper}, \text{current} - \text{lower}]$. This is efficiently done during the merge sort process where each merge step counts the valid pairs

between the two sorted subarrays. The merge sort guarantees that the array segments are sorted, which allows binary-counting of valid ranges by maintaining two pointers. This approach eliminates the need for nested loops and achieves $O(n \log n)$ complexity.

Merge Sort

□ #493 Reverse Pairs

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Divide and Conquer ·
Binary Indexed Tree · Segment Tree · Merge
Sort · Ordered Set

Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the number of reverse pairs in the array.

A reverse pair is a pair (i, j) where:

- $0 \leq i < j < \text{nums.length}$ and
- $\text{nums}[i] > 2 * \text{nums}[j]$.

Example 1:

Input: `nums = [1,3,2,3,1]`

Output: 2

Explanation: The reverse pairs are:

$(1, 4) \rightarrow \text{nums}[1] = 3, \text{nums}[4] = 1, 3 > 2 * 1$

$(3, 4) \rightarrow \text{nums}[3] = 3, \text{nums}[4] = 1, 3 > 2 * 1$

Example 2:

Input: nums = [2,4,3,5,1]

Output: 3

Explanation: The reverse pairs are:

(1, 4) --> nums[1] = 4, nums[4] = 1, 4 > 2 * 1

(2, 4) --> nums[2] = 3, nums[4] = 1, 3 > 2 * 1

(3, 4) --> nums[3] = 5, nums[4] = 1, 5 > 2 * 1

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def reversePairs(self, nums: List[int]) -> int:
        def merge_sort(l, r):
            if l >= r:
                return 0
            mid = (l + r) >> 1
            ans = merge_sort(l, mid) + merge_sort(mid + 1, r)
            t = []
            i, j = l, mid + 1
            while i <= mid and j <= r:
                if nums[i] <= 2 * nums[j]:
                    i += 1
                else:
                    ans += mid - i + 1
                    j += 1
            i, j = l, mid + 1
            while i <= mid and j <= r:
                if nums[i] <= nums[j]:
                    t.append(nums[i])
                    i += 1
```



```

        else:
            t.append(nums[j])
            j += 1
        t.extend(nums[i : mid + 1])
        t.extend(nums[j : r + 1])
        nums[l : r + 1] = t
        return ans

    return merge_sort(0, len(nums) - 1)

```

```

class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):

```

```

        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s

```

```

class Solution:
    def reversePairs(self, nums: List[int]) -> int:
        s = set()
        for num in nums:
            s.add(num)
            s.add(num * 2)
        alls = sorted(s)
        m = {v: i for i, v in enumerate(alls, 1)}

```

```

ans = 0
tree = BinaryIndexedTree(len(m))
for num in nums[::-1]:
    ans += tree.query(m[num] - 1)
    tree.update(m[num * 2], 1)
return ans

```

```

class Node:
    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0

class SegmentTree:
    def __init__(self, n):
        self.tr = [Node() for _ in range(4 * n)]

```

```

        self.build(1, 1, n)

```

```

    def build(self, u, l, r):
        self.tr[u].l = l
        self.tr[u].r = r
        if l == r:
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)

```

```

    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].v += 1
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)

```

```

        self.pushup(u)

    def pushup(self, u):
        self.tr[u].v = self.tr[u << 1].v + self.tr[u << 1 | 1].v

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        v = 0
        if l <= mid:
            v += self.query(u << 1, l, r)
        if r > mid:
            v += self.query(u << 1 | 1, l, r)
        return v

class Solution:
    def reversePairs(self, nums: List[int]) -> int:
        s = set()

        for num in nums:
            s.add(num)
            s.add(num * 2)
        alls = sorted(s)
        m = {v: i for i, v in enumerate(alls, 1)}
        tree = SegmentTree(len(m))
        ans = 0
        for v in nums[::-1]:
            x = m[v]
            ans += tree.query(1, 1, x - 1)

            tree.modify(1, m[v * 2], 1)
        return ans

```

• SimplyLeet

```
# Define the Solution class
class Solution:
    def reversePairs(self, nums):
        # Helper function that uses merge sort to count reverse pairs
        def merge_sort(left, right):
            # Base case: if the segment has one or no elements, no reverse
            # pairs exist
            if left >= right:
                return 0
            mid = (left + right) // 2
            # Count reverse pairs in the left and right halves

            count = merge_sort(left, mid) + merge_sort(mid+1, right)
            j = mid + 1
            # Count the reverse pairs: for every element in left half,
            # move pointer j in right half until condition fails
            for i in range(left, mid+1):
                while j <= right and nums[i] > 2 * nums[j]:
                    j += 1
                count += j - (mid + 1)
            # Merge the two sorted halves
            temp = []

            p1, p2 = left, mid+1
            while p1 <= mid and p2 <= right:
                if nums[p1] <= nums[p2]:
                    temp.append(nums[p1])
                    p1 += 1
                else:
                    temp.append(nums[p2])
                    p2 += 1
            while p1 <= mid:
                temp.append(nums[p1])
                p1 += 1
            while p2 <= right:
                temp.append(nums[p2])
                p2 += 1
            # Copy the sorted elements back into the original array
            nums[left:right+1] = temp
            return count

        return merge_sort(0, len(nums)-1)
```

◆ Quick Review

Key Insights

- The brute force solution checks every possible pair, resulting in $O(n^2)$ time complexity, which is too slow for the given constraints.
- A modified merge sort can be used to both sort the array and count reverse pairs efficiently.
- During the merge process, since the two halves are sorted, a two-pointer technique can efficiently count how many numbers in the right half satisfy the reverse pair condition for each number in the left half.
- Alternative approaches include Binary Indexed Trees or Segment Trees, but merge sort provides a clean divide and conquer solution.

Space & Time

Time Complexity: $O(n \log n)$ – Each level of the merge sort takes $O(n)$ time and there are $O(\log n)$ levels.

Space Complexity: $O(n)$ – Additional space is required for the temporary array during merging.

Solution

We solve the problem using a modified merge sort. The idea is to recursively split the array into halves, count reverse pairs in each half, and then count reverse pairs across the two halves before merging them. When merging two sorted halves, a two–pointer approach can be used to count the reverse pairs quickly. After counting, the two halves are merged in sorted order.

Backtracking

Round 6 · Mid · Hard · 2 questions

Backtracking

□ #301 Remove Invalid Parentheses

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Backtracking · Breadth-First Search
Lists: AlgoMaster 600

Problem

Given a string *s* that contains parentheses and letters, remove the minimum number of invalid parentheses to make the input string valid.

Return a list of unique strings that are valid with the minimum number of removals. You may return the answer in any order.

Example 1:

```
Input: s = "()())()"
Output: ["()()()", "()()())"]
```

Example 2:

```
Input: s = "(a())()"
Output: ["(a())()", "(a())()"]
```

Example 3:

```
Input: s = ")("
Output: [""]
```

Constraints:

- $1 \leq s.length \leq 25$
- s consists of lowercase English letters and parentheses '(' and ')'.
 - There will be at most 20 parentheses in s .

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeInvalidParentheses(self, s: str) -> list[str]:
        # Similar to 921. Minimum Add to Make Parentheses Valid
        def getLeftAndRightCounts(s: str) -> tuple[int, int]:
            """Returns how many '(' and ')' need to be deleted."""
            l = 0
            r = 0

            for c in s:
                if c == '(':
                    l += 1
                elif c == ')':
                    if l == 0:
                        r += 1
                    else:
                        l -= 1

            return l, r

        def isValid(s: str):
```

```

opened = 0 # the number of '(' - # of ')'
for c in s:
    if c == '(':
        opened += 1
    elif c == ')':
        opened -= 1
    if opened < 0:
        return False
return True # opened == 0

```

```

ans = []

def dfs(s: str, start: int, l: int, r: int) -> None:
    if l == 0 and r == 0 and isValid(s):
        ans.append(s)
        return

    for i in range(start, len(s)):
        if i > start and s[i] == s[i - 1]:
            continue

        if r > 0 and s[i] == ')': # Delete s[i]
            dfs(s[:i] + s[i + 1:], i, l, r - 1)
        elif l > 0 and s[i] == '(': # Delete s[i]
            dfs(s[:i] + s[i + 1:], i, l - 1, r)

l, r = getLeftAndRightCounts(s)
dfs(s, 0, l, r)
return ans

```

• LeetDoocs

```

class Solution:
    def removeInvalidParentheses(self, s: str) -> List[str]:
        def dfs(i, l, r, lcnt, rcnt, t):
            if i == n:
                if l == 0 and r == 0:
                    ans.add(t)
                return
            if n - i < l + r or lcnt < rcnt:
                return
            if s[i] == '(' and l:

```

```

        dfs(i + 1, l - 1, r, lcnt, rcnt, t)
    elif s[i] == ')' and r:
        dfs(i + 1, l, r - 1, lcnt, rcnt, t)
    dfs(i + 1, l, r, lcnt + (s[i] == '('), rcnt + (s[i] == ')'), t + s
[i])

l = r = 0
for c in s:
    if c == '(':
        l += 1
    elif c == ')':
        if l:
            l -= 1
        else:
            r += 1
ans = set()
n = len(s)
dfs(0, l, r, 0, 0, '')
return list(ans)

```

• SimplyLeet

```

# Python solution using BFS
from collections import deque

def removeInvalidParentheses(s):
    # Helper function to check if a string is valid.
    def isValid(string):
        count = 0
        for char in string:
            if char == '(':
                count += 1

```

```

        elif char == ')':
            count -= 1
            if count < 0:
                return False
        return count == 0

result = []
visited = set([s]) # Set to avoid re-processing the same string
queue = deque([s])
found = False # Flag indicating if we've found a valid expression at
current level

while queue:
    current = queue.popleft()
    if isValid(current):
        result.append(current)
        found = True
    # Continue exploring only if a valid expression hasn't been found at
    this BFS level.
    if found:
        continue
    for i in range(len(current)):

        # Skip non-parenthesis characters.
        if current[i] != '(' and current[i] != ')':
            continue
        next_str = current[:i] + current[i+1:]
        if next_str not in visited:
            visited.add(next_str)
            queue.append(next_str)
    return result

# Example usage:

# print(removeInvalidParentheses("()())()"))

```

◆ Quick Review

Key Insights

- Use BFS to generate all possible strings by removing one parenthesis at a time.
- Stop the BFS once valid strings are found at the current level (to ensure minimum removals).
- Use a set to avoid processing the same string multiple times.
- Use a helper function to check if a string has valid parentheses.

Space & Time

Time Complexity: $O(n * 2^n)$ in the worst-case, as each character could be removed leading to exploring 2^n possibilities.

Space Complexity: $O(2^n)$ for storing all unique intermediate strings in the worst-case.

Solution

We solve this problem using a Breadth-First Search (BFS) approach. Start with the original string and generate all possible strings by removing one parenthesis at each step. For every generated string, check if it is valid using a helper function that traverses the string and maintains a balance counter. As soon as valid expressions are found at a

particular level of BFS, add them to the result list and do not proceed to deeper levels. This ensures we remove the minimal number of parentheses. Data structures used include a queue for BFS and a set for visited strings to avoid duplicates.

Backtracking

□ #980 Unique Paths III

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking · Bit Manipulation · Matrix
Lists: AlgoMaster 600

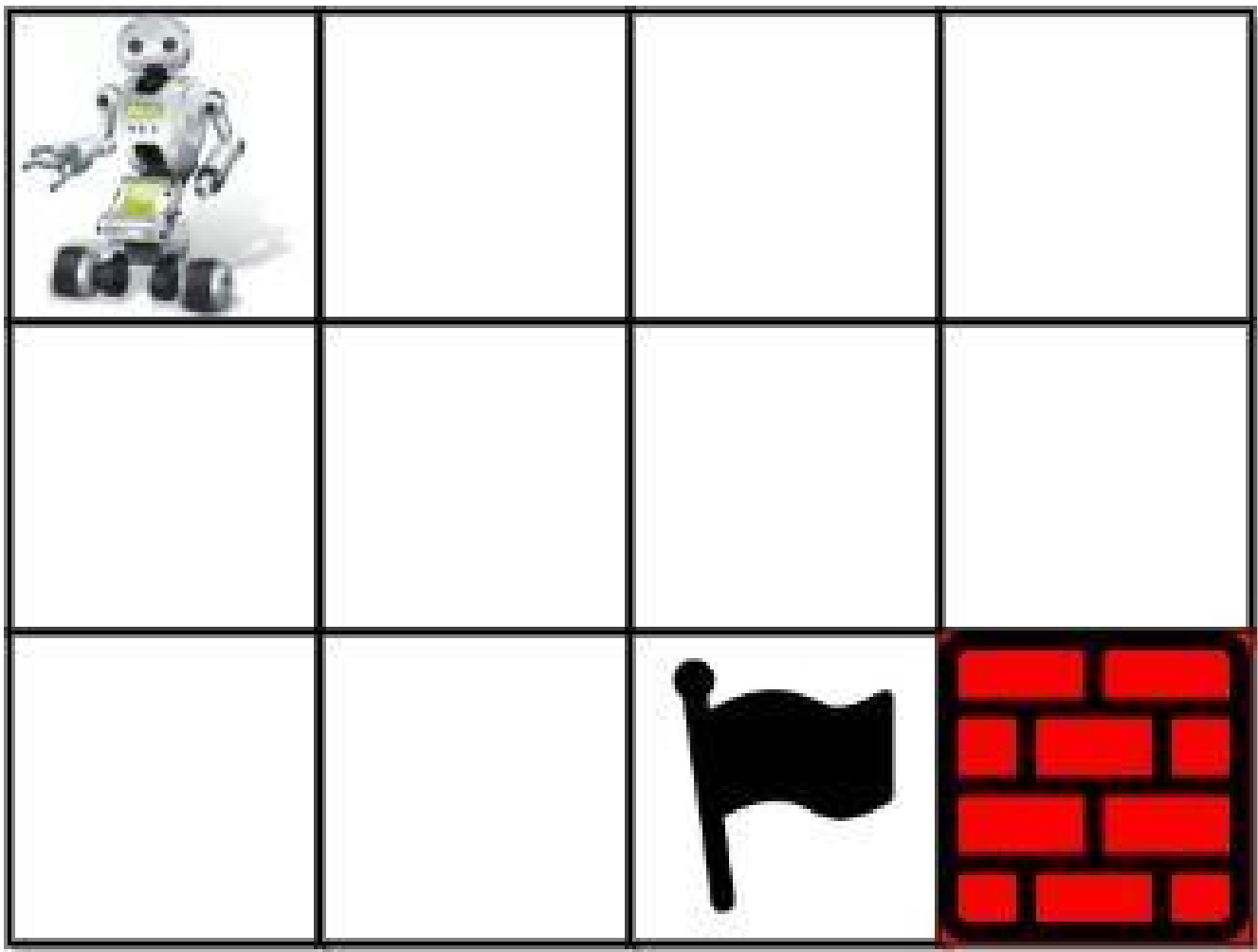
Problem

You are given an $m \times n$ integer array grid where $\text{grid}[i][j]$ could be:

- 1 representing the starting square. There is exactly one starting square.
- 2 representing the ending square. There is exactly one ending square.
- 0 representing empty squares we can walk over.
- -1 representing obstacles that we cannot walk over.

Return the number of 4-directional walks from the starting square to the ending square, that walk over every non-obstacle square exactly once.

Example 1:



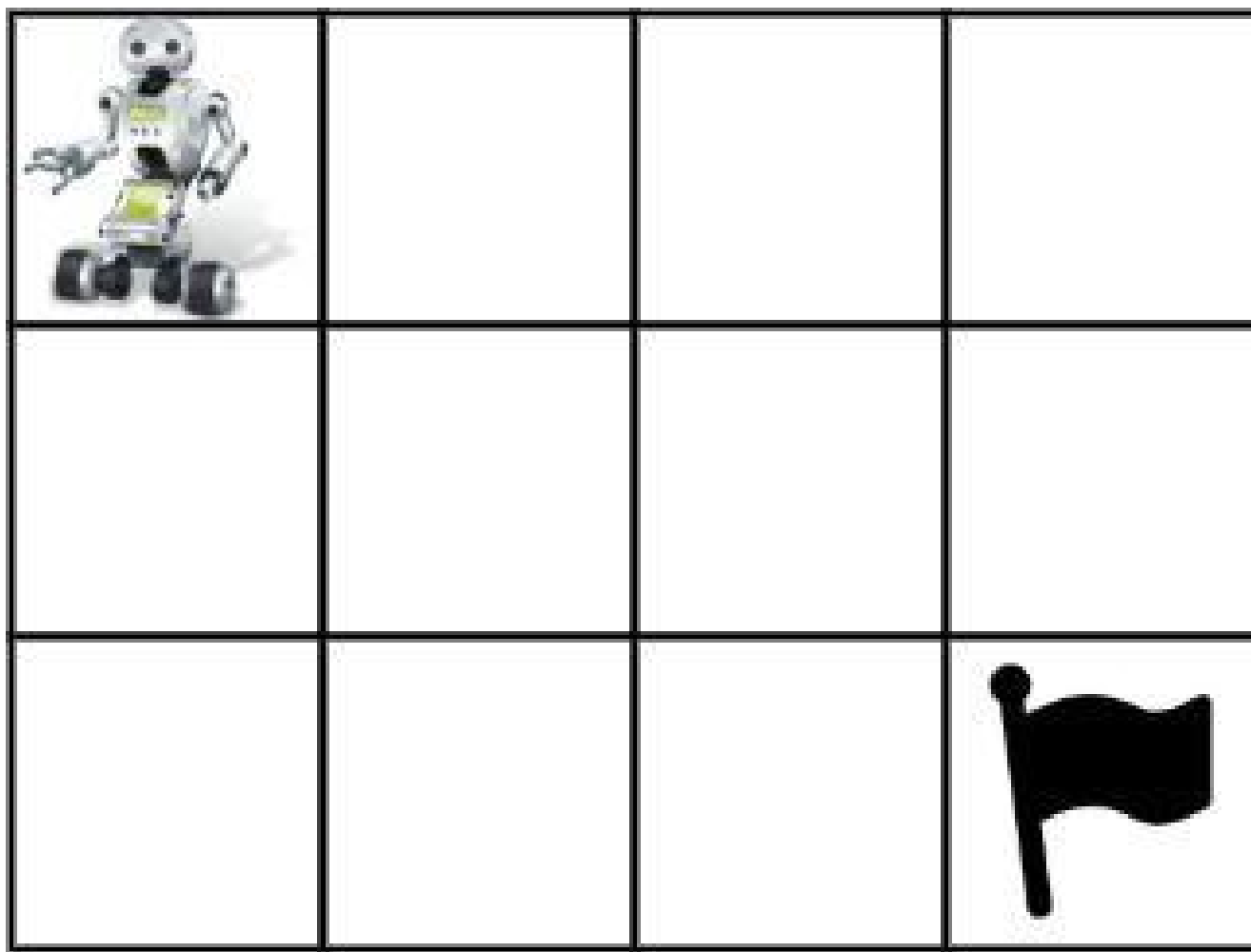
Input: grid = [[1,0,0,0],[0,0,0,0],[0,0,2,-1]]

Output: 2

Explanation: We have the following two paths:

1. (0,0),(0,1),(0,2),(0,3),(1,3),(1,2),(1,1),(1,0),(2,0),(2,1),(2,2)
2. (0,0),(1,0),(2,0),(2,1),(1,1),(0,1),(0,2),(0,3),(1,3),(1,2),(2,2)

Example 2:



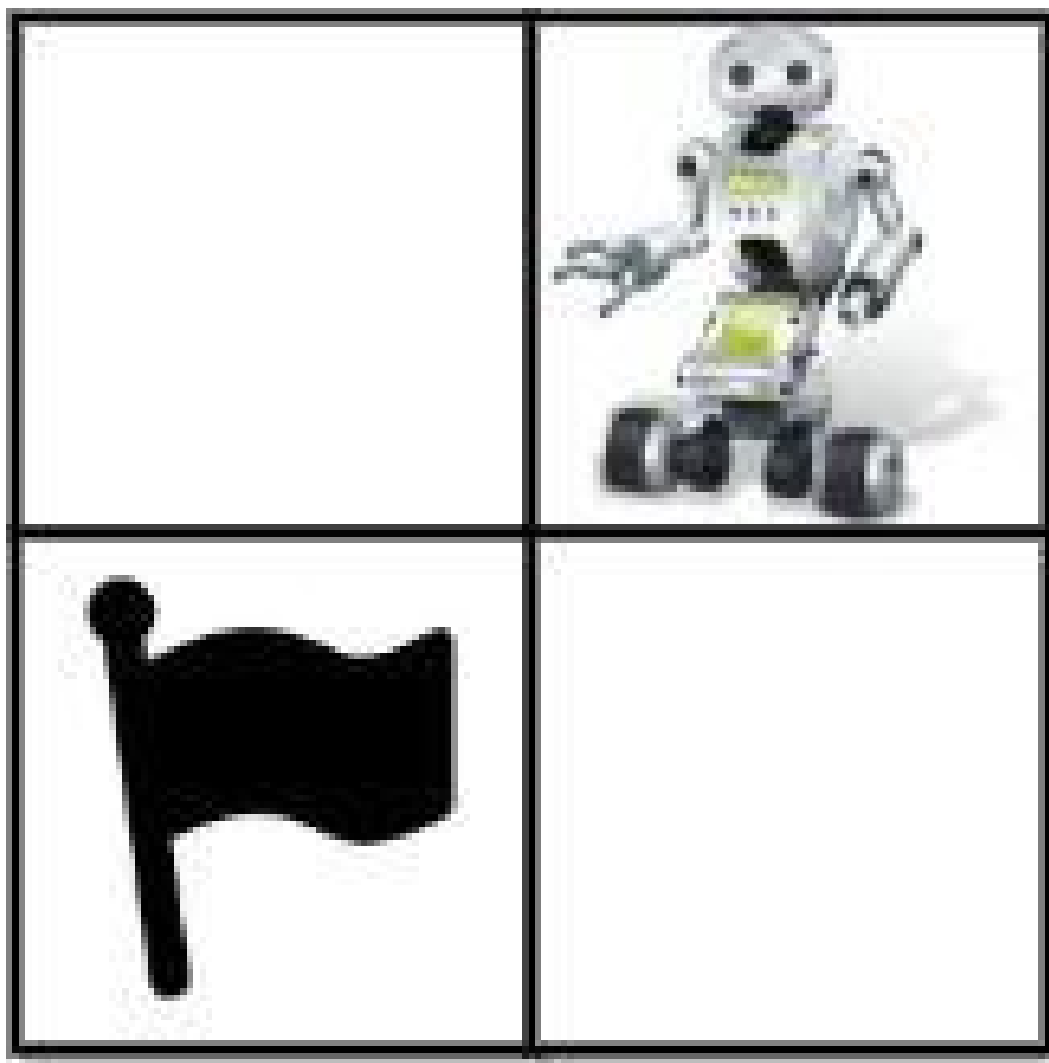
Input: grid = [[1,0,0,0],[0,0,0,0],[0,0,0,2]]

Output: 4

Explanation: We have the following four paths:

1. (0,0),(0,1),(0,2),(0,3),(1,3),(1,2),(1,1),(1,0),(2,0),(2,1),(2,2),(2,3)
2. (0,0),(0,1),(1,1),(1,0),(2,0),(2,1),(2,2),(1,2),(0,2),(0,3),(1,3),(2,3)
3. (0,0),(1,0),(2,0),(2,1),(2,2),(1,2),(1,1),(0,1),(0,2),(0,3),(1,3),(2,3)
4. (0,0),(1,0),(2,0),(2,1),(1,1),(0,1),(0,2),(0,3),(1,3),(1,2),(2,2),(2,3)

Example 3:



Input: grid = [[0,1],[2,0]]

Output: 0

Explanation: There is no path that walks over every empty square exactly once.
Note that the starting and ending square can be anywhere in the grid.

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 20$
- $1 \leq m * n \leq 20$
- $-1 \leq \text{grid}[i][j] \leq 2$

- There is exactly one starting cell and one ending cell.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def uniquePathsIII(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int, k: int) -> int:
            if grid[i][j] == 2:
                return int(k == cnt + 1)
            ans = 0
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and (x, y) not in vis and grid[x]
[y] != -1:
                    vis.add((x, y))

                    ans += dfs(x, y, k + 1)
                    vis.remove((x, y))
            return ans

        m, n = len(grid), len(grid[0])
        start = next((i, j) for i in range(m) for j in range(n) if grid[i][j]
== 1)
        dirs = (-1, 0, 1, 0, -1)
        cnt = sum(row.count(0) for row in grid)
        vis = {start}
        return dfs(*start, 0)
```

• SimplyLeet

```
def uniquePathsIII(grid):
    # Get the dimensions of the grid
    rows, cols = len(grid), len(grid[0])
    empty = 0
    start = None

    # Count the number of non-obstacle cells and locate the starting cell
    for i in range(rows):
        for j in range(cols):
            if grid[i][j] != -1:

                empty += 1
                if grid[i][j] == 1:
                    start = (i, j)

    paths = 0
    # DFS helper to explore the grid
    def dfs(i, j, count):
        nonlocal paths
        # When the ending cell is reached, check if all non-obstacle cells are
        visited
        if grid[i][j] == 2:

            if count == empty:
                paths += 1
            return

        # Mark the current cell as visited by setting it as an obstacle
        temp = grid[i][j]
        grid[i][j] = -1

        # Explore all four directions
        for di, dj in [(1,0), (-1,0), (0,1), (0,-1)]:
```

```

        new_i, new_j = i + di, j + dj
        if 0 <= new_i < rows and 0 <= new_j < cols and grid[new_i][new_j]
!= -1:
            dfs(new_i, new_j, count + 1)

    # Backtrack: restore the original cell value
    grid[i][j] = temp

    # Start the DFS from the starting point with a count of 1
    dfs(start[0], start[1], 1)
    return paths

# Example usage:
if __name__ == '__main__':
    grid = [[1,0,0,0],[0,0,0,0],[0,0,2,-1]]
    print(uniquePathsIII(grid)) # Expected output: 2

```

◆ Quick Review

Key Insights

- Count all non-obstacle cells (including start and end) to know the route length required.
- Use Depth First Search (DFS) to explore all possible paths from the starting cell.
- Mark cells as visited during recursion to avoid revisiting and backtrack afterwards.
- Only count a path when reaching the ending cell and the number of visited cells equals the total non-obstacle count.

Space & Time

Time Complexity: $O(3^{(mn)})$ in the worst-case scenario due to branching factor in DFS.

Space Complexity: $O(mn)$ due to recursion call stack and in-place modifications of the grid.

Solution

We solve the problem by performing a DFS starting from the unique starting cell. Before DFS, count the total number of non-obstacle cells. As DFS explores neighbors recursively, mark the cell as visited (by setting it to -1) to prevent cycles and restore its value after exploring (backtracking). When the ending cell is reached, check if we have visited all non-obstacle cells. If yes, increment the valid path count.

Data structures and algorithms used:

- Recursion (DFS) for exploring all valid paths.
- In-place grid marking for tracking visited cells.
- Backtracking for restoring grid state. This approach ensures that every possible valid route is considered while correctly handling obstacles and ensuring each non-obstacle cell is visited exactly once.

Tries

Round 6 · Mid · Hard · 3 questions

Tries

□ #425 Word Squares

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Backtracking · Trie
Lists: AlgoMaster 600

Problem

Given an array of unique strings words, return all the word squares you can build from words. The same word from words can be used multiple times. You can return the answer in any order.

A sequence of strings forms a valid word square if the kth row and column read the same string, where $0 \leq k < \max(\text{numRows}, \text{numColumns})$.

- For example, the word sequence ["ball", "area", "lead", "lady"] forms a word square because each word reads the same both horizontally and vertically.

Example 1:

Input: words = ["area", "lead", "wall", "lady", "ball"]
Output: [["ball", "area", "lead", "lady"], ["wall", "area", "lead", "lady"]]
Explanation:
The output consists of two word squares. The order of output does not matter (just the order of words in each word square matters).

Example 2:

Input: words = ["abat", "baba", "atan", "atal"]
Output: [["baba", "abat", "baba", "atal"], ["baba", "abat", "baba", "atan"]]
Explanation:
The output consists of two word squares. The order of output does not matter (just the order of words in each word square matters).

Constraints:

- $1 \leq \text{words.length} \leq 1000$
- $1 \leq \text{words}[i].\text{length} \leq 4$
- All words[i] have the same length.
- words[i] consists of only lowercase English letters.
- All words[i] are unique.

Community Solutions (Python)

• WalkCC

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.startsWith: list[str] = []

class Trie:
    def __init__(self, words: list[str]):
        self.root = TrieNode()
        for word in words:
```

```

        self._insert(word)

def findBy(self, prefix: str) -> list[str]:
    node = self.root
    for c in prefix:
        if c not in node.children:
            return []
        node = node.children[c]
    return node.startsWith

def _insert(self, word: str) -> None:
    node = self.root
    for c in word:
        node = node.children.setdefault(c, TrieNode())
        node.startsWith.append(word)

class Solution:
    def wordSquares(self, words: list[str]) -> list[list[str]]:
        if not words:
            return []

        n = len(words[0])
        ans = []
        path = []
        trie = Trie(words)

        for word in words:
            path.append(word)
            self._dfs(trie, n, path, ans)

        path.pop()

        return ans

    def _dfs(self, trie: Trie, n: int, path: list[str], ans: list[list[str]]):
        if len(path) == n:
            ans.append(path.copy())
            return

        prefix = self._getPrefix(path)

```

```

for s in trie.findBy(prefix):
    path.append(s)
    self._dfs(trie, n, path, ans)
    path.pop()

def _getPrefix(self, path: list[str]) -> str:
    """
    e.g. path = ["wall",
                  "area"]

    prefix = "le.."
    """
    prefix = []
    index = len(path)
    for s in path:
        prefix.append(s[index])
    return ''.join(prefix)

```

• LeetDoocs

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.v = []

    def insert(self, w, i):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
            node.v.append(i)

    def search(self, w):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return []

```

```

        node = node.children[idx]
        return node.v

class Solution:
    def wordSquares(self, words: List[str]) -> List[List[str]]:
        def dfs(t):
            if len(t) == len(words[0]):
                ans.append(t[:])
                return

            idx = len(t)
            pref = [v[idx] for v in t]
            indexes = trie.search(''.join(pref))
            for i in indexes:
                t.append(words[i])
                dfs(t)
                t.pop()

        trie = Trie()
        ans = []

        for i, w in enumerate(words):
            trie.insert(w, i)
        for w in words:
            dfs([w])
        return ans

```

• SimplyLeet

Python solution for Word Squares

```

class WordSquares:
    def __init__(self, words):
        self.N = len(words[0])
        self.words = words
        self.prefix_map = {}
        # Build prefix mapping: for every prefix, map to list of words with
        # that prefix.
        for word in words:
            for i in range(len(word) + 1):

```

```

        prefix = word[:i]
        if prefix in self.prefix_map:
            self.prefix_map[prefix].append(word)
        else:
            self.prefix_map[prefix] = [word]

def find_word_squares(self):
    results = []
    square = []
    # Begin backtracking starting with each word.

    for word in self.words:
        square = [word]
        self.backtrack(1, square, results)
    return results

def backtrack(self, step, square, results):
    # If the current square is complete, add a copy to results
    if step == self.N:
        results.append(list(square))
        return

    # Build the prefix for the next word in the square using the kth letter
    # of each current word.
    prefix = ''.join([word[step] for word in square])
    for candidate in self.prefix_map.get(prefix, []):
        square.append(candidate)
        self.backtrack(step + 1, square, results)
        square.pop() # backtrack if no valid square found

# Example usage:
# words = ["area", "lead", "wall", "lady", "ball"]

# ws = WordSquares(words)
# print(ws.find_word_squares())

```

◆ Quick Review

Key Insights

- Use backtracking to construct candidate word squares one row at a time.
- For each new row, determine the required prefix (formed from the kth letters of previously chosen words) that the candidate word must match.
- Precompute a mapping (or Trie) from prefixes to the list of words that share that prefix. This allows efficient lookup and pruning.
- The problem leverages the fact that all words have the same length and the board is square.

Space & Time

Time Complexity: In the worst-case, it can be exponential due to backtracking. However, using prefix lookups significantly prunes the search. With L as the word length and N as the number of words, the effective time complexity is approximately $O(N * L * (\text{number of candidates per prefix})^L)$.

Space Complexity: $O(N * L)$ for the prefix mapping, plus $O(L)$ for the recursion stack and building the current square.

Solution

This solution uses backtracking combined with a prefix mapping technique. First, we precompute a dictionary that maps every possible prefix to the list of words with that prefix. Then we start forming a word square row-by-row. At each recursive call, we derive the prefix for the next word based on the current square (by taking the i th character from each of the already chosen words where i is the current depth). We then only consider words that match this prefix. This drastically reduces the number of possibilities, making the backtracking efficient even for multiple words. The algorithm uses recursive DFS (depth-first search) and backtracking to explore valid combinations and build complete word squares.

Tries

□ #472 Concatenated Words

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Dynamic Programming ·
Depth-First Search · Trie · Sorting
Lists: AlgoMaster 600

Problem

Given an array of strings words (without duplicates), return all the concatenated words in the given list of words.

A concatenated word is defined as a string that is comprised entirely of at least two shorter words (not necessarily distinct) in the given array.

Example 1:

```
Input: words = ["cat","cats","catsdogcats","dog","dogcatsdog","hippopotamuses",  
               "rat","ratcatdogcat"]  
Output: ["catsdogcats","dogcatsdog","ratcatdogcat"]  
Explanation: "catsdogcats" can be concatenated by "cats", "dog" and "cats";  
              "dogcatsdog" can be concatenated by "dog", "cats" and "dog";  
              "ratcatdogcat" can be concatenated by "rat", "cat", "dog" and "cat".
```

Example 2:

```
Input: words = ["cat","dog","catdog"]  
Output: ["catdog"]
```


Constraints:

- $1 \leq \text{words.length} \leq 104$
- $1 \leq \text{words}[i].\text{length} \leq 30$
- $\text{words}[i]$ consists of only lowercase English letters.
- All the strings of words are unique.
- $1 \leq \sum(\text{words}[i].\text{length}) \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findAllConcatenatedWordsInADict(self, words: list[str]) -> list[str]:
        wordSet = set(words)

        @functools.lru_cache(None)
        def isConcat(word: str) -> bool:
            for i in range(1, len(word)):
                prefix = word[:i]
                suffix = word[i:]
                if prefix in wordSet and (suffix in wordSet or isConcat(suffix)):

                    return True

            return False

        return [word for word in words if isConcat(word)]
```

• LeetDoocs

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, w):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

class Solution:
    def findAllConcatenatedWordsInADict(self, words: List[str]) -> List[str]:
        def dfs(w):
            if not w:
                return True

            node = trie
            for i, c in enumerate(w):
                idx = ord(c) - ord('a')
                if node.children[idx] is None:
                    return False
                node = node.children[idx]
                if node.is_end and dfs(w[i + 1 :]):
                    return True
            return False

        trie = Trie()
        ans = []
        words.sort(key=lambda x: len(x))
        for w in words:
            if dfs(w):
                ans.append(w)
            else:
                trie.insert(w)
        return ans
```

• SimplyLeet

```
def findAllConcatenatedWordsInADict(words):
    # Sort the words by length to ensure that smaller words are processed
    # first.
    words.sort(key=len)
    concatenated = []
    word_set = set()

    # Function to check if the word can be formed by words in the set.
    def can_form(word):
        if not word:
            return False

        dp = [False] * (len(word) + 1)
        dp[0] = True
        # Try all possible splits.
        for i in range(1, len(word) + 1):
            # Limit inner loop based on maximum word length of 30.
            for j in range(max(0, i - 30), i):
                if dp[j] and word[j:i] in word_set:
                    dp[i] = True
                    break
        return dp[len(word)]

    # Process each word.
    for word in words:
        if not word:
            continue
        if can_form(word):
            concatenated.append(word)
            word_set.add(word)

    return concatenated

# Example usage:
words = ["cat", "cats", "catsdogcats", "dog", "dogcatsdog", "hippopotamuses",
"rat", "ratcatdogcat"]
print(findAllConcatenatedWordsInADict(words))
```

◆ Quick Review

Key Insights

- Sort the words by length so that smaller words, which can be used to build larger ones, are processed first.
- Use dynamic programming to determine if a given word can be segmented into two or more words from a pre-built dictionary (set).
- Ensure that when checking a word, you do not consider the word itself; only use previously seen (shorter) words.
- Optimize by limiting the inner loop based on the maximum possible word length (given constraint is 30).

Space & Time

Time Complexity: $O(n * l^2)$ where n is the number of words and l is the average length of the words.

Space Complexity: $O(n * l)$ for storing words and the DP array for each word.

Solution

The solution involves first sorting the list of words by their length. For each word, use a DP

array where $dp[i]$ indicates whether the substring $word[0:i]$ can be constructed from words in a set of previously seen words. If a word can be segmented into at least two parts based on the dictionary, it qualifies as a concatenated word. After processing each word, add it to the set so that it can be used to form longer words. This approach effectively builds up from the smallest words to the largest.

Tries

□ #1032 Stream of Characters

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Design · Trie · Data Stream
Lists: AlgoMaster 600

Problem

Design an algorithm that accepts a stream of characters and checks if a suffix of these characters is a string of a given array of strings words.

For example, if words = ["abc", "xyz"] and the stream added the four characters (one by one) 'a', 'x', 'y', and 'z', your algorithm should detect that the suffix "xyz" of the characters "axyz" matches "xyz" from words.

Implement the StreamChecker class:

- **StreamChecker(String[] words)** Initializes the object with the strings array words.
- **boolean query(char letter)** Accepts a new character from the stream and returns true if any non-empty suffix from the stream forms

a word that is in words.

Example 1:

Input

```
["StreamChecker", "query", "query", "query", "query", "query", "query",
"query", "query", "query", "query", "query", "query"]
[[["cd", "f", "kl"]], ["a"], ["b"], ["c"], ["d"], ["e"], ["f"], ["g"], ["h"],
["i"], ["j"], ["k"], ["l"]]
```

Output

```
[null, false, false, false, true, false, true, false, false, false, false,
false, true]
```

Explanation

```
StreamChecker streamChecker = new StreamChecker(["cd", "f", "kl"]);
```

Constraints:

- $1 \leq \text{words.length} \leq 2000$
- $1 \leq \text{words}[i].\text{length} \leq 200$
- $\text{words}[i]$ consists of lowercase English letters.
- letter is a lowercase English letter.
- At most $4 * 10^4$ calls will be made to query.

Community Solutions (Python)

• WalkCC

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class StreamChecker:
    def __init__(self, words: list[str]):
        self.root = TrieNode()
        self.letters = []
```

```

    for word in words:
        self._insert(word)

def query(self, letter: str) -> bool:
    self.letters.append(letter)
    node = self.root
    for c in reversed(self.letters):
        if c not in node.children:
            return False
        node = node.children[c]

    if node.isWord:
        return True
    return False

def _insert(self, word: str) -> None:
    node = self.root
    for c in reversed(word):
        node = node.children.setdefault(c, TrieNode())
    node.isWord = True

```

• LeetDoocs

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, w: str):
        node = self
        for c in w[::-1]:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:

```



```

        node.children[idx] = Trie()
        node = node.children[idx]
        node.is_end = True

    def search(self, w: List[str]) -> bool:
        node = self
        for c in w[::-1]:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return False

            node = node.children[idx]
            if node.is_end:
                return True
        return False

class StreamChecker:
    def __init__(self, words: List[str]):
        self.trie = Trie()
        self.cs = []

        self.limit = 201
        for w in words:
            self.trie.insert(w)

    def query(self, letter: str) -> bool:
        self.cs.append(letter)
        return self.trie.search(self.cs[-self.limit :])

# Your StreamChecker object will be instantiated and called as such:

# obj = StreamChecker(words)
# param_1 = obj.query(letter)

```

• SimplyLeet

```
# Python implementation of StreamChecker
class StreamChecker:
    def __init__(self, words: list[str]):
        # Build a reverse Trie, where each node is a dictionary
        self.trie = {}
        self.max_length = 0 # maximum length of a word in words
        for word in words:
            self.max_length = max(self.max_length, len(word))
            node = self.trie
            # Insert word in reverse order

            for char in reversed(word):
                if char not in node:
                    node[char] = {}
                node = node[char]
            # Mark the end of a word
            node['#'] = True
        self.stream = [] # buffer to store characters from query

    def query(self, letter: str) -> bool:
        # Append the new letter to our stream buffer

        self.stream.append(letter)
        # Keep the stream buffer size within the maximum word length
        if len(self.stream) > self.max_length:
            self.stream.pop(0)

        node = self.trie
        # Traverse the stream backwards (i.e., the most recent characters)
        for ch in reversed(self.stream):
            if ch not in node:
                return False # no further matching path in Trie

            node = node[ch]
            # Check if we have reached the end of a word
            if '#' in node:
                return True
        return False

# Example usage:
# streamChecker = StreamChecker(["cd", "f", "kl"])
# print(streamChecker.query("a"))
# print(streamChecker.query("b"))
```

```
# print(streamChecker.query("c"))  
# print(streamChecker.query("d"))
```

◆ Quick Review

Key Insights

- Instead of checking all possible suffixes against every word, we store words in a Trie data structure.
- Since we are matching suffixes, it is efficient to insert the words in reversed order. This way, when a new character is queried, we traverse the stream backward and the Trie forward.
- Limit the size of the stored stream to the length of the longest word.
- The worst-case query cost is proportional to the maximum length of words.

Space & Time

Time Complexity: $O(M)$ per query, where M is the maximum length of the words.

Space Complexity: $O(\text{Total characters in all words})$ for the Trie plus $O(M)$ for the stream buffer.

Solution

We construct a reverse Trie where each word from the list is stored in reverse order. With every query, append the new character to a stream buffer and traverse this list backwards while matching against the Trie. If at any point you hit a Trie node marked as the end of a word, return true. The stream buffer length is capped at the length of the longest word to ensure efficient memory usage. This method drastically reduces unnecessary comparisons and quickly identifies valid suffixes.

Heaps

Round 6 · Mid · Hard · 1 question

Heaps

□ #1383 Maximum Performance of a Team **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Sorting · Heap (Priority Queue)
Lists: AlgoMaster 600

Problem

You are given two integers n and k and two integer arrays `speed` and `efficiency` both of length n . There are n engineers numbered from 1 to n . `speed[i]` and `efficiency[i]` represent the speed and efficiency of the i th engineer respectively.

Choose at most k different engineers out of the n engineers to form a team with the maximum performance.

The performance of a team is the sum of its engineers' speeds multiplied by the minimum efficiency among its engineers.

Return the maximum performance of this team. Since the answer can be a huge number, return it modulo $10^9 + 7$.

Example 1:

Input: $n = 6$, $\text{speed} = [2, 10, 3, 1, 5, 8]$, $\text{efficiency} = [5, 4, 3, 9, 7, 2]$, $k = 2$

Output: 60

Explanation:

We have the maximum performance of the team by selecting engineer 2 (with $\text{speed}=10$ and $\text{efficiency}=4$) and engineer 5 (with $\text{speed}=5$ and $\text{efficiency}=7$). That is, $\text{performance} = (10 + 5) * \min(4, 7) = 60$.

Example 2:

Input: $n = 6$, $\text{speed} = [2, 10, 3, 1, 5, 8]$, $\text{efficiency} = [5, 4, 3, 9, 7, 2]$, $k = 3$

Output: 68

Explanation:

This is the same example as the first but $k = 3$. We can select engineer 1, engineer 2 and engineer 5 to get the maximum performance of the team. That is, $\text{performance} = (2 + 10 + 5) * \min(5, 4, 7) = 68$.

Example 3:

Input: $n = 6$, $\text{speed} = [2, 10, 3, 1, 5, 8]$, $\text{efficiency} = [5, 4, 3, 9, 7, 2]$, $k = 4$

Output: 72

Constraints:

- $1 \leq k \leq n \leq 105$
- $\text{speed.length} == n$
- $\text{efficiency.length} == n$
- $1 \leq \text{speed}[i] \leq 105$
- $1 \leq \text{efficiency}[i] \leq 108$

Community Solutions (Python)

- WalkCC

```

class Solution:
    # Similar to 857. Minimum Cost to Hire K Workers
    def maxPerformance(
        self,
        n: int,
        speed: list[int],
        efficiency: list[int],
        k: int,
    ) -> int:
        MOD = 1_000_000_007

        ans = 0
        speedSum = 0
        # (efficiency[i], speed[i]) sorted by efficiency[i] in descending order
        A = sorted([(e, s) for s, e in zip(speed, efficiency)], reverse=True)
        minHeap = []

        for e, s in A:
            heapq.heappush(minHeap, s)
            speedSum += s
            if len(minHeap) > k:

                speedSum -= heapq.heappop(minHeap)
            ans = max(ans, speedSum * e)

        return ans % MOD

```

• LeetDoocs

```

class Solution:
    def maxPerformance(
        self, n: int, speed: List[int], efficiency: List[int], k: int
    ) -> int:
        t = sorted(zip(speed, efficiency), key=lambda x: -x[1])
        ans = tot = 0
        mod = 10**9 + 7
        h = []
        for s, e in t:
            tot += s

```



```

        ans = max(ans, tot * e)
        heappush(h, s)
        if len(h) == k:
            tot -= heappop(h)
    return ans % mod

```

• SimplyLeet

```

import heapq

def maxPerformance(n, speed, efficiency, k):
    MOD = 10**9 + 7
    # Create pairs of (efficiency, speed) and sort by efficiency descending
    engineers = sorted(zip(efficiency, speed), key=lambda x: -x[0])

    max_perf = 0      # Store maximum performance encountered
    speed_sum = 0     # Running sum of speeds in the current team
    speed_heap = []   # Min-heap for the speeds in the team

    # Iterate through each engineer sorted by descending efficiency
    for curr_eff, curr_speed in engineers:
        # Add current engineer's speed to the heap
        heapq.heappush(speed_heap, curr_speed)
        speed_sum += curr_speed

        # If there are more than k engineers, remove the one with the smallest
        # speed
        if len(speed_heap) > k:
            speed_sum -= heapq.heappop(speed_heap)

        # Calculate team performance with current engineer as minimum
        # efficiency
        curr_perf = speed_sum * curr_eff
        max_perf = max(max_perf, curr_perf)

    return max_perf % MOD

# Example usage:
print(maxPerformance(6, [2,10,3,1,5,8], [5,4,3,9,7,2], 2))

```

◆ Quick Review

Key Insights

- Sort the engineers by efficiency in descending order so that when processing an engineer, their efficiency is the minimum for the current selection.
- Use a min-heap to maintain the top speeds among the selected engineers. This helps in efficiently removing the engineer with the smallest speed when the team exceeds k members.
- Keep a running sum of the speeds currently in the team.
- At each iteration, calculate the potential performance as the current speed sum multiplied by the current engineer's efficiency (which is the minimum efficiency for the team so far) and update the maximum performance if it is higher.

Space & Time

Time Complexity: $O(n \log k)$

Space Complexity: $O(k)$

Solution

The solution involves the following steps:

- Pair each engineer's speed and efficiency then sort these pairs in descending order based on efficiency.**
- Initialize a min-heap (or priority queue) to store speeds and a variable to keep the running sum of speeds.**
- Iterate through the sorted engineers. For each engineer:**
 - Add the engineer's speed to the min-heap and update the running sum.**
 - If the size of the heap exceeds k , remove the smallest speed from the heap and subtract it from the running sum.**
 - Compute the current performance as the running sum multiplied by the current engineer's efficiency.**
 - Update the maximum performance if the current performance is greater.**
 - Return the maximum performance modulo $10^9 + 7$.**

Using a min-heap ensures that when the team size exceeds k , the smallest speed can be removed in $O(\log k)$ time, keeping the overall

complexity efficient.

Two Heaps

Round 6 · Mid · Hard · 2 questions

Two Heaps

□ #480 Sliding Window Median

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sliding Window · Heap
(Priority Queue)
Lists: AlgoMaster 600

Problem

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value. So the median is the mean of the two middle values.

- For examples, if $\text{arr} = [2,3,4]$, the median is 3.
- For examples, if $\text{arr} = [1,2,3,4]$, the median is $(2 + 3) / 2 = 2.5$.

You are given an integer array `nums` and an integer `k`. There is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return the median array for each window in the original array. Answers within 10^{-5} of the actual value will be accepted.

Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [1.00000,-1.00000,-1.00000,3.00000,5.00000,6.00000]
Explanation:
Window position Median
-----
[1 3 -1] -3 5 3 6 7 1
1 [3 -1 -3] 5 3 6 7 -1
1 3 [-1 -3 5] 3 6 7 -1
```

Example 2:

```
Input: nums = [1,2,3,4,2,3,1,4,2], k = 3
Output: [2.00000,3.00000,3.00000,3.00000,2.00000,3.00000,2.00000]
```

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $-231 \leq \text{nums}[i] \leq 231 - 1$

Community Solutions (Python)

• LeetDoocs

```
class MedianFinder:
    def __init__(self, k: int):
        self.k = k
        self.small = []
        self.large = []
        self.delayed = defaultdict(int)
        self.small_size = 0
        self.large_size = 0

    def add_num(self, num: int):
```

```

        if not self.small or num <= -self.small[0]:
            heappush(self.small, -num)
            self.small_size += 1
        else:
            heappush(self.large, num)
            self.large_size += 1
        self.rebalance()

    def find_median(self) -> float:
        return -self.small[0] if self.k & 1 else (-self.small[0] + self.large[
0]) / 2

```

```

    def remove_num(self, num: int):
        self.delayed[num] += 1
        if num <= -self.small[0]:
            self.small_size -= 1
            if num == -self.small[0]:
                self.prune(self.small)
        else:
            self.large_size -= 1
            if num == self.large[0]:

```

```

                self.prune(self.large)
            self.rebalance()

```

```

    def prune(self, pq: List[int]):
        sign = -1 if pq is self.small else 1
        while pq and sign * pq[0] in self.delayed:
            self.delayed[sign * pq[0]] -= 1
            if self.delayed[sign * pq[0]] == 0:
                self.delayed.pop(sign * pq[0])
            heappop(pq)

```



```
def rebalance(self):
    if self.small_size > self.large_size + 1:
        heappush(self.large, -heappop(self.small))
        self.small_size -= 1
        self.large_size += 1
        self.prune(self.small)
    elif self.small_size < self.large_size:
        heappush(self.small, -heappop(self.large))
        self.large_size -= 1
```

```
self.small_size += 1
self.prune(self.large)
```

```
class Solution:
    def medianSlidingWindow(self, nums: List[int], k: int) -> List[float]:
        finder = MedianFinder(k)
        for x in nums[:k]:
            finder.add_num(x)
        ans = [finder.find_median()]
```

```
for i in range(k, len(nums)):
    finder.add_num(nums[i])
    finder.remove_num(nums[i - k])
    ans.append(finder.find_median())
return ans
```

```
class Solution:
    def medianSlidingWindow(self, nums: List[int], k: int) -> List[float]:
        l = SortedList()
        r = SortedList()
        ans = []
        for i, x in enumerate(nums):
            r.add(x)
            l.add(r.pop(0))
            while len(l) - len(r) > 1:
                r.add(l.pop())
```

```

        j = i - k + 1
        if j >= 0:
            ans.append(l[-1] if k & 1 else (l[-1] + r[0]) / 2)
            if nums[j] in l:
                l.remove(nums[j])
            else:
                r.remove(nums[j])
    return ans

```

• SimplyLeet

```

import heapq

class DualHeap:
    def __init__(self, k):
        self.small = [] # max heap (store negatives)
        self.large = [] # min heap
        self.k = k
        self.delayed = {} # counts for lazy removal
        self.smallSize = 0 # effective size of small heap
        self.largeSize = 0 # effective size of large heap

    def prune(self, heap):
        # Remove elements from the top that are marked for lazy deletion.
        while heap:
            num = heap[0] if heap is self.large else -heap[0]
            if num in self.delayed and self.delayed[num] > 0:
                heapq.heappop(heap)
                self.delayed[num] -= 1
                if self.delayed[num] == 0:
                    del self.delayed[num]
            else:
                break

    def balance(self):
        # Ensure the two heaps' sizes differ by at most one.
        if self.smallSize > self.largeSize + 1:
            val = -heapq.heappop(self.small)
            self.smallSize -= 1
            heapq.heappush(self.large, val)
            self.largeSize += 1

```

```

        self.prune(self.small)
    elif self.smallSize < self.largeSize:
        val = heapq.heappop(self.large)
        self.largeSize -= 1
        heapq.heappush(self.small, -val)
        self.smallSize += 1
        self.prune(self.large)

def add(self, num):
    if not self.small or num <= -self.small[0]:

        heapq.heappush(self.small, -num)
        self.smallSize += 1
    else:
        heapq.heappush(self.large, num)
        self.largeSize += 1
    self.balance()

def remove(self, num):
    # Mark the element for lazy removal.
    self.delayed[num] = self.delayed.get(num, 0) + 1

    if self.small and num <= -self.small[0]:
        self.smallSize -= 1
        if num == -self.small[0]:
            self.prune(self.small)
    else:
        self.largeSize -= 1
        if self.large and num == self.large[0]:
            self.prune(self.large)
    self.balance()

def median(self):
    if self.k % 2:
        return float(-self.small[0])
    else:
        return (-self.small[0] + self.large[0]) / 2.0

def slidingWindowMedian(nums, k):
    result = []
    dh = DualHeap(k)
    for i, num in enumerate(nums):

```

```
        dh.add(num)
        if i >= k - 1:
            result.append(dh.median())
            dh.remove(nums[i - k + 1])
    return result

# Example usage:
nums = [1, 3, -1, -3, 5, 3, 6, 7]
k = 3
print(slidingWindowMedian(nums, k))
```

◆ Quick Review

Key Insights

- Use the sliding window technique to sequentially add and remove elements.
- Maintain two heaps:
- A max heap (implemented via negatives) for the lower half of the data.
- A min heap for the higher half of the data.
- The two heaps allow efficient median retrieval in $O(1)$ time.
- Lazy removal and balancing of heaps are crucial to ensure the correctness when elements exit the window.

Space & Time

Time Complexity: $O(n \log k)$ because each insertion and removal in the heaps takes $O(\log$

k).

Space Complexity: $O(k)$ to maintain the two heaps.

Solution

We solve the problem using a DualHeap approach. We maintain two heaps to divide the sliding window into lower and upper halves. When a new element is added, it is placed into one of the heaps based on its value relative to the current median. When an element falls out of the window, we mark it for lazy removal in the appropriate heap. We then balance the heaps to ensure the sizes differ by at most one, allowing quick computation of the median.

Two Heaps

□ #502 IPO

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Sorting · Heap (Priority Queue)
Lists: AlgoMaster 600

Problem

Suppose LeetCode will start its IPO soon. In order to sell a good price of its shares to Venture Capital, LeetCode would like to work on some projects to increase its capital before the IPO. Since it has limited resources, it can only finish at most k distinct projects before the IPO. Help LeetCode design the best way to maximize its total capital after finishing at most k distinct projects.

You are given n projects where the i th project has a pure profit $profits[i]$ and a minimum capital of $capital[i]$ is needed to start it.

Initially, you have w capital. When you finish a project, you will obtain its pure profit and the profit will be added to your total capital.

Pick a list of at most k distinct projects from given projects to maximize your final capital, and return the final maximized capital.

The answer is guaranteed to fit in a 32-bit signed integer.

Example 1:

Input: $k = 2$, $w = 0$, profits = [1,2,3], capital = [0,1,1]

Output: 4

Explanation: Since your initial capital is 0, you can only start the project indexed 0.

After finishing it you will obtain profit 1 and your capital becomes 1.

With capital 1, you can either start the project indexed 1 or the project indexed 2.

Since you can choose at most 2 projects, you need to finish the project indexed 2 to get the maximum capital.

Therefore, output the final maximized capital, which is $0 + 1 + 3 = 4$.

Example 2:

Input: $k = 3$, $w = 0$, profits = [1,2,3], capital = [0,1,2]

Output: 6

Constraints:

- $1 \leq k \leq 105$
- $0 \leq w \leq 109$
- $n == \text{profits.length}$
- $n == \text{capital.length}$
- $1 \leq n \leq 105$
- $0 \leq \text{profits}[i] \leq 104$
- $0 \leq \text{capital}[i] \leq 109$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def findMaximizedCapital(
        self, k: int, w: int, profits: List[int], capital: List[int]
    ) -> int:
        h1 = [(c, p) for c, p in zip(capital, profits)]
        heapify(h1)
        h2 = []
        while k:
            while h1 and h1[0][0] <= w:
                heappush(h2, -heappop(h1)[1])

            if not h2:
                break
            w -= heappop(h2)
            k -= 1
        return w
```

• SimplyLeet

```
import heapq

def findMaximizedCapital(k, w, profits, capital):
    # Create project pairs (required capital, profit)
    projects = sorted(zip(capital, profits), key=lambda x: x[0])
    max_heap = [] # max heap (simulate by pushing negative profits)
    idx = 0
    n = len(projects)

    # Iterate up to k projects

    for _ in range(k):
        # Add all projects that are affordable with current capital w
        while idx < n and projects[idx][0] <= w:
            # Push negative profit to simulate max-heap since heapq is a
            min-heap
            heapq.heappush(max_heap, -projects[idx][1])
            idx += 1

        # If no project is affordable, break out early
        if not max_heap:
            break
```



```
# Select project with maximum profit and update capital
w += -heapq.heappop(max_heap)

return w

# Example usage:
# k = 2, w = 0, profits = [1,2,3], capital = [0,1,1]
print(findMaximizedCapital(2, 0, [1, 2, 3], [0, 1, 1])) # Output: 4
```

◆ Quick Review

Key Insights

- Sort the projects by their required capital so that you can easily know which projects are feasible given current capital.
- Use a max heap (priority queue) to always select the feasible project with the highest profit.
- Iterate up to k times, each time adding all newly feasible projects (those whose capital requirement is at most the current capital) to the max heap.
- Terminate early if there are no projects available in the heap.

Space & Time

Time Complexity: $O(n \log n + k \log n)$ where n is the number of projects.

Space Complexity: $O(n)$ for storing project information and the heap.

Solution

The approach starts by pairing each project's required capital with its profit, then sorting these pairs based on the capital requirement. For each of the k rounds, add all projects that are affordable (given the current capital) to a max heap that prioritizes profit. Then select the project with the maximum profit, update the available capital and continue. This greedy selection process ensures that the best immediate profit boost is always taken, eventually maximizing the total capital.

Intervals

Round 6 · Mid · Hard · 1 question

Intervals

□ #2402 Meeting Rooms III

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sorting · Heap (Priority Queue) · Simulation
Lists: AlgoMaster 600

Problem

You are given an integer n . There are n rooms numbered from 0 to $n - 1$.

You are given a 2D integer array `meetings` where `meetings[i] = [starti, endi]` means that a meeting will be held during the half-closed time interval `[starti, endi)`. All the values of `starti` are unique.

Meetings are allocated to rooms in the following manner:

1. Each meeting will take place in the unused room with the lowest number.
2. If there are no available rooms, the meeting will be delayed until a room becomes free. The delayed meeting should have the same

duration as the original meeting.

3. When a room becomes unused, meetings that have an earlier original start time should be given the room.

Return the number of the room that held the most meetings. If there are multiple rooms, return the room with the lowest number.

A half-closed interval $[a, b)$ is the interval between a and b including a and not including b .

Example 1:

Input: $n = 2$, meetings = $[[0,10],[1,5],[2,7],[3,4]]$

Output: 0

Explanation:

- At time 0, both rooms are not being used. The first meeting starts in room 0.
- At time 1, only room 1 is not being used. The second meeting starts in room 1.
- At time 2, both rooms are being used. The third meeting is delayed.
- At time 3, both rooms are being used. The fourth meeting is delayed.
- At time 5, the meeting in room 1 finishes. The third meeting starts in room 1 for the time period $[5,10)$.

Example 2:

Input: $n = 3$, meetings = $[[1,20],[2,10],[3,5],[4,9],[6,8]]$

Output: 1

Explanation:

- At time 1, all three rooms are not being used. The first meeting starts in room 0.
- At time 2, rooms 1 and 2 are not being used. The second meeting starts in room 1.
- At time 3, only room 2 is not being used. The third meeting starts in room 2.
- At time 4, all three rooms are being used. The fourth meeting is delayed.
- At time 5, the meeting in room 2 finishes. The fourth meeting starts in room 2 for the time period $[5,10)$.

Constraints:

- $1 \leq n \leq 100$
- $1 \leq \text{meetings.length} \leq 105$
- $\text{meetings}[i].\text{length} == 2$
- $0 \leq \text{start}_i < \text{end}_i \leq 5 * 105$
- All the values of start_i are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mostBooked(self, n: int, meetings: list[list[int]]) -> int:
        count = [0] * n

        meetings.sort()

        occupied = [] # (endTime, roomId)
        availableRoomIds = [i for i in range(n)]
        heapq.heapify(availableRoomIds)

        for start, end in meetings:
            # Push meetings ending before this `meeting` in occupied to the
            # `availableRoomIds`.
            while occupied and occupied[0][0] <= start:
                heapq.heappush(availableRoomIds, heapq.heappop(occupied)[1])
            if availableRoomIds:
                roomId = heapq.heappop(availableRoomIds)
                count[roomId] += 1
                heapq.heappush(occupied, (end, roomId))
            else:
                newStart, roomId = heapq.heappop(occupied)
                count[roomId] += 1
                heapq.heappush(occupied, (newStart + (end - start), roomId))

        return count.index(max(count))
```

• LeetCode

```
class Solution:
    def mostBooked(self, n: int, meetings: List[List[int]]) -> int:
        meetings.sort(key=lambda x: x[0])
        busy = []
        idle = list(range(n))
        heapify(idle)
        cnt = [0] * n
        for s, e in meetings:
            while busy and busy[0][0] <= s:
                heappush(idle, heappop(busy)[1])

            i = 0
            if idle:
                i = heappop(idle)
                heappush(busy, (e, i))
            else:
                time_end, i = heappop(busy)
                heappush(busy, (time_end + e - s, i))
            cnt[i] += 1
        ans = 0
        for i in range(n):
            if cnt[ans] < cnt[i]:
                ans = i
        return ans
```

• SimplyLeet

```
import heapq

def mostBooked(n, meetings):
    # Sort meetings by their original start times.
    meetings.sort(key=lambda x: x[0])

    # Initialize available rooms heap and ongoing meetings heap.
    available_rooms = list(range(n))
    heapq.heapify(available_rooms)
    busy_rooms = [] # Each element is (end_time, room)
```

```

# Count of meetings held in each room.
meeting_count = [0] * n

for start, end in meetings:
    # Free up all rooms that have finished before the current meeting
    start.
    while busy_rooms and busy_rooms[0][0] <= start:
        end_time, room = heapq.heappop(busy_rooms)
        heapq.heappush(available_rooms, room)

    duration = end - start

    if available_rooms:
        # If a room is available, assign the meeting immediately.
        room = heapq.heappop(available_rooms)
        meeting_start = start
        meeting_end = start + duration
    else:
        # Delay the meeting: get the room that will be free the earliest.
        earliest_end, room = heapq.heappop(busy_rooms)

        meeting_start = earliest_end
        meeting_end = earliest_end + duration

    # Update the room meeting count and busy heap.
    meeting_count[room] += 1
    heapq.heappush(busy_rooms, (meeting_end, room))

    # Return the room with the maximum meeting count (lowest numbered in case
    of tie).
    return meeting_count.index(max(meeting_count))

# Example usage:
print(mostBooked(2, [[0,10],[1,5],[2,7],[3,4]])) # Expected output: 0

```

◆ Quick Review

Key Insights

- Use a min-heap to track currently available rooms by room number.
- Use another min-heap to manage ongoing meetings, keyed by their finish times.
- Process the meetings in order of their start times since they are unique.
- When no room is available at the meeting's start time, delay the meeting to the earliest finish time available and update its finish time by adding its original duration.
- Count how many meetings are allocated per room and return the one with the maximum count (resolving ties by room number).

Space & Time

Time Complexity: $O(m \log n)$ where m is the number of meetings, since each meeting may require heap operations on a heap of size at most n .

Space Complexity: $O(m + n)$ due to storing meetings for sorting and maintaining the heaps (the busy heap is $O(n)$ and the sort uses $O(m)$).

Solution

We simulate the meeting room allocation with two heaps:

- Heap for available rooms (storing just the room numbers so that the smallest numbered available room is taken first).**
- Heap for ongoing meetings (storing pairs of [end_time, room_number]). For each meeting (processed in order by start time):**
 - Free any rooms that have finished their meetings by comparing the meeting's start time with the top of the ongoing meetings heap.**
 - If there is an available room, assign the meeting immediately.**
 - Otherwise, delay the meeting until the earliest room is free, update the meeting's start and finish times accordingly.**
 - Update the count for each room's usage.**
 - Finally, select the room with the maximum meeting count (choosing the smallest room number in case of ties).**

Data Structure Design

Round 6 · Mid · Hard · 2 questions

Data Structure Design

□ #715 Range Module

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Design · Segment Tree · Ordered Set
Lists: AlgoMaster 600

Problem

A Range Module is a module that tracks ranges of numbers. Design a data structure to track the ranges represented as half-open intervals and query about them.

A half-open interval $[left, right)$ denotes all the real numbers x where $left \leq x < right$.

Implement the RangeModule class:

- **RangeModule()** Initializes the object of the data structure.
- **void addRange(int left, int right)** Adds the half-open interval $[left, right)$, tracking every real number in that interval. Adding an interval that partially overlaps with currently tracked numbers should add any numbers in the interval $[left, right)$ that are not already

tracked.

- **boolean queryRange(int left, int right)**
Returns true if every real number in the interval [left, right) is currently being tracked, and false otherwise.
- **void removeRange(int left, int right)** Stops tracking every real number currently being tracked in the half-open interval [left, right).

Example 1:

```
Input
["RangeModule", "addRange", "removeRange", "queryRange", "queryRange",
"queryRange"]
[[], [10, 20], [14, 16], [10, 14], [13, 15], [16, 17]]
Output
[null, null, null, true, false, true]
```

```
Explanation
RangeModule rangeModule = new RangeModule();
```

Constraints:

- $1 \leq \text{left} < \text{right} \leq 109$
- At most 104 calls will be made to addRange, queryRange, and removeRange.

Community Solutions (Python)

- WalkCC

```

class RangeModule:
    def __init__(self):
        self.A = []

    def addRange(self, left: int, right: int) -> None:
        i = bisect_left(self.A, left)
        j = bisect_right(self.A, right)
        self.A[i:j] = [left] * (i % 2 == 0) + [right] * (j % 2 == 0)

    def queryRange(self, left: int, right: int) -> bool:

        i = bisect_right(self.A, left)
        j = bisect_left(self.A, right)
        return i == j and i % 2 == 1

    def removeRange(self, left: int, right: int) -> None:
        i = bisect_left(self.A, left)
        j = bisect_right(self.A, right)
        self.A[i:j] = [left] * (i % 2 == 1) + [right] * (j % 2 == 1)

```

• LeetDoocs

```

class Node:
    __slots__ = ['left', 'right', 'add', 'v']

    def __init__(self):
        self.left = None
        self.right = None
        self.add = 0
        self.v = False

class SegmentTree:
    __slots__ = ['root']

    def __init__(self):
        self.root = Node()

    def modify(self, left, right, v, l=1, r=int(1e9), node=None):
        if node is None:
            node = self.root
        if l >= left and r <= right:

```

```
        if v == 1:
            node.add = 1
            node.v = True
        else:
            node.add = -1
            node.v = False
        return
    self.pushdown(node)
    mid = (l + r) >> 1
    if left <= mid:
        self.modify(left, right, v, l, mid, node.left)
    if right > mid:
        self.modify(left, right, v, mid + 1, r, node.right)
    self.pushup(node)

def query(self, left, right, l=1, r=int(1e9), node=None):
    if node is None:
        node = self.root
    if l >= left and r <= right:
        return node.v

    self.pushdown(node)
    mid = (l + r) >> 1
    v = True
    if left <= mid:
        v = v and self.query(left, right, l, mid, node.left)
    if right > mid:
        v = v and self.query(left, right, mid + 1, r, node.right)
    return v

def pushup(self, node):
```

```

        node.v = bool(node.left and node.left.v and node.right and node.right.
v)

def pushdown(self, node):
    if node.left is None:
        node.left = Node()
    if node.right is None:
        node.right = Node()
    if node.add:
        node.left.add = node.right.add = node.add
        node.left.v = node.add == 1

        node.right.v = node.add == 1
        node.add = 0

class RangeModule:
    def __init__(self):
        self.tree = SegmentTree()

    def addRange(self, left: int, right: int) -> None:
        self.tree.modify(left, right - 1, 1)

    def queryRange(self, left: int, right: int) -> bool:
        return self.tree.query(left, right - 1)

    def removeRange(self, left: int, right: int) -> None:
        self.tree.modify(left, right - 1, -1)

# Your RangeModule object will be instantiated and called as such:
# obj = RangeModule()

# obj.addRange(left,right)
# param_2 = obj.queryRange(left,right)
# obj.removeRange(left,right)

```

• SimplyLeet


```
# Python implementation with detailed line-by-line comments
class RangeModule:
    def __init__(self):
        # List to hold the non-overlapping intervals sorted by start
        self.intervals = []

    def addRange(self, left: int, right: int) -> None:
        new_intervals = []
        i = 0
        # Add all intervals that end before 'left'

        while i < len(self.intervals) and self.intervals[i][1] < left:
            new_intervals.append(self.intervals[i])
            i += 1

        # Merge all overlapping intervals with [left, right)
        while i < len(self.intervals) and self.intervals[i][0] <= right:
            left = min(left, self.intervals[i][0])
            right = max(right, self.intervals[i][1])
            i += 1

        # Add the merged interval
        new_intervals.append([left, right])

        # Append the remaining intervals that start after 'right'
        while i < len(self.intervals):
            new_intervals.append(self.intervals[i])
            i += 1

        self.intervals = new_intervals

    def queryRange(self, left: int, right: int) -> bool:
        # Binary search to find the position where 'left' would be inserted
        lo, hi = 0, len(self.intervals)
        while lo < hi:
            mid = (lo + hi) // 2
            if self.intervals[mid][1] < left:
                lo = mid + 1
            else:
                hi = mid
        # If no interval covers [left, right), return false
```

```

        if lo < len(self.intervals) and self.intervals[lo][0] <= left and self
.intervals[lo][1] >= right:
            return True
        return False

def removeRange(self, left: int, right: int) -> None:
    new_intervals = []
    i = 0
    # Add all intervals that end before 'left'
    while i < len(self.intervals) and self.intervals[i][1] <= left:
        new_intervals.append(self.intervals[i])

        i += 1

    # Process intervals that overlap with [left, right)
    while i < len(self.intervals) and self.intervals[i][0] < right:
        # If part of the interval is left of 'left', keep it
        if self.intervals[i][0] < left:
            new_intervals.append([self.intervals[i][0], left])
        # If part of the interval is right of 'right', keep it
        if self.intervals[i][1] > right:
            new_intervals.append([right, self.intervals[i][1]])

        i += 1

    # Append the remaining intervals that start after 'right'
    while i < len(self.intervals):
        new_intervals.append(self.intervals[i])
        i += 1

    self.intervals = new_intervals

```

◆ Quick Review

Key Insights

- Use a sorted data structure (like an ordered list or balanced BST) to maintain non-overlapping intervals.

- When adding an interval, merge it with overlapping intervals.
- Query by checking if an interval is fully covered by one of the existing intervals.
- Removing an interval requires splitting overlapping intervals accordingly.
- Binary search can be used to quickly locate intervals that might overlap with the query range.

Space & Time

Time Complexity: Each `addRange`, `queryRange`, and `removeRange` operation can take $O(n)$ in the worst-case as merging or splitting intervals might require scanning through intervals.

Space Complexity: $O(n)$ where n is the number of disjoint intervals stored.

Solution

We maintain a sorted list of intervals, where each interval is a pair `[start, end)` and the intervals are non-overlapping. For the `addRange` operation, we locate all intervals that overlap with `[left, right)` and merge them together. For `queryRange`, we use binary

search to check if an interval exists that completely covers [left, right). For removeRange, overlapping intervals are modified or split to remove the range [left, right).

The key is to update the intervals list while ensuring it remains sorted and non-overlapping. This data structure avoids tracking every single number and is efficient in handling intervals up to large input sizes.

Data Structure Design

□ #2296 Design a Text Editor

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · String · Stack · Design ·
Simulation · Doubly-Linked List
Lists: AlgoMaster 600

Problem

Design a text editor with a cursor that can do the following:

- Add text to where the cursor is.
- Delete text from where the cursor is (simulating the backspace key).
- Move the cursor either left or right.

When deleting text, only characters to the left of the cursor will be deleted. The cursor will also remain within the actual text and cannot be moved beyond it. More formally, we have that $0 \leq \text{cursor.position} \leq \text{currentText.length}$ always holds.

Implement the TextEditor class:

- **TextEditor()** Initializes the object with empty text.
- **void addText(string text)** Appends text to where the cursor is. The cursor ends to the right of text.
- **int deleteText(int k)** Deletes k characters to the left of the cursor. Returns the number of characters actually deleted.
- **string cursorLeft(int k)** Moves the cursor to the left k times. Returns the last min(10, len) characters to the left of the cursor, where len is the number of characters to the left of the cursor.
- **string cursorRight(int k)** Moves the cursor to the right k times. Returns the last min(10, len) characters to the left of the cursor, where len is the number of characters to the left of the cursor.

Example 1:

Input

```
["TextEditor", "addText", "deleteText", "addText", "cursorRight", "cursorLeft",  
"deleteText", "cursorLeft", "cursorRight"]
```

```
[[], ["leetcode"], [4], ["practice"], [3], [8], [10], [2], [6]]
```

Output

```
[null, null, 4, null, "etpractice", "leet", 4, "", "practi"]
```

Explanation

```
TextEditor textEditor = new TextEditor(); // The current text is "|". (The '|' character represents the cursor)
```

Constraints:

- $1 \leq \text{text.length}$, $k \leq 40$
- text consists of lowercase English letters.
- At most $2 * 10^4$ calls in total will be made to addText, deleteText, cursorLeft and cursorRight.

Follow-up: Could you find a solution with time complexity of $O(k)$ per call?

Community Solutions (Python)

• WalkCC

```
class TextEditor:
    def __init__(self):
        self.s = []
        self.stack = []

    def addText(self, text: str) -> None:
        for c in text:
            self.s.append(c)

    def deleteText(self, k: int) -> int:

        numDeleted = min(k, len(self.s))
        for _ in range(numDeleted):
            self.s.pop()
        return numDeleted

    def cursorLeft(self, k: int) -> str:
        while self.s and k > 0:
            self.stack.append(self.s.pop())
            k -= 1
        return self._getString()
```

```

def cursorRight(self, k: int) -> str:
    while self.stack and k > 0:
        self.s.append(self.stack.pop())
        k -= 1
    return self._getString()

def _getString(self) -> str:
    if len(self.s) < 10:
        return ''.join(self.s)

    return ''.join(self.s[-10:])

```

• LeetDoocs

```

class TextEditor:
    def __init__(self):
        self.left = []
        self.right = []

    def addText(self, text: str) -> None:
        self.left.extend(list(text))

    def deleteText(self, k: int) -> int:
        k = min(k, len(self.left))

        for _ in range(k):
            self.left.pop()
        return k

    def cursorLeft(self, k: int) -> str:
        k = min(k, len(self.left))
        for _ in range(k):
            self.right.append(self.left.pop())
        return ''.join(self.left[-10:])

```



```
def cursorRight(self, k: int) -> str:
    k = min(k, len(self.right))
    for _ in range(k):
        self.left.append(self.right.pop())
    return ''.join(self.left[-10:])
```

```
# Your TextEditor object will be instantiated and called as such:
# obj = TextEditor()
# obj.addText(text)
```

```
# param_2 = obj.deleteText(k)
# param_3 = obj.cursorLeft(k)
# param_4 = obj.cursorRight(k)
```

• SimplyLeet

```
class TextEditor:
    def __init__(self):
        # left_stack contains text left to the cursor; right_stack contains
        # text right to the cursor.
        self.left_stack = []
        self.right_stack = []

    def addText(self, text: str) -> None:
        # Append each character to left_stack.
        for ch in text:
            self.left_stack.append(ch)

    def deleteText(self, k: int) -> int:
        # Remove up to k characters from the left_stack.
        count = 0
        while self.left_stack and count < k:
            self.left_stack.pop()
            count += 1
        return count

    def cursorLeft(self, k: int) -> str:
```

```

        # Move at most k characters from left_stack to right_stack.
        count = 0
        while self.left_stack and count < k:
            self.right_stack.append(self.left_stack.pop())
            count += 1
        # Get last min(10, len(left_stack)) characters and return them as a
        string.
        return "".join(self.left_stack[-10:])

    def cursorRight(self, k: int) -> str:
        # Move at most k characters from right_stack back into left_stack.

        count = 0
        while self.right_stack and count < k:
            self.left_stack.append(self.right_stack.pop())
            count += 1
        # Get last min(10, len(left_stack)) characters and return them as a
        string.
        return "".join(self.left_stack[-10:])

# Example usage:
# textEditor = TextEditor()
# textEditor.addText("leetcode")

# print(textEditor.deleteText(4))
# textEditor.addText("practice")
# print(textEditor.cursorRight(3))
# print(textEditor.cursorLeft(8))
# print(textEditor.deleteText(10))
# print(textEditor.cursorLeft(2))
# print(textEditor.cursorRight(6))

```

◆ Quick Review

Key Insights

- Use two separate data structures (stacks) to represent the text to the left and right of the cursor.

- The left stack holds characters before the cursor, while the right stack holds characters after the cursor.
- Each operation is executed in $O(k)$ time where k is the number of characters to process in that operation.
- Cursor movement is simulated by transferring characters between the left and right stacks.
- For retrieving the last $\min(10, \text{len})$ characters left of the cursor, simply extract from the left stack.

Space & Time

Time Complexity: $O(k)$ per call for `addText`, `deleteText`, `cursorLeft`, and `cursorRight` where k is the number of characters processed.

Space Complexity: $O(n)$ overall, where n is the total number of characters maintained in both stacks.

Solution

We simulate a text editor using two stacks. One stack, `leftStack`, contains characters to the left of the cursor; the other, `rightStack`, contains characters to the right. When adding

text, push characters onto leftStack. For deleting text, pop characters from leftStack. Moving the cursor left pops characters from leftStack into rightStack (up to k times or until leftStack is empty), and moving right pops from rightStack back into leftStack. After each cursor move, the solution returns the last 10 characters from leftStack (or whatever is available). This approach guarantees that each operation processes only the relevant k characters.

Greedy

Round 6 · Mid · Hard · 4 questions

Greedy

□ #135 Candy

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy

Lists: AlgoMaster 600

Problem

There are n children standing in a line. Each child is assigned a rating value given in the integer array `ratings`.

You are giving candies to these children subjected to the following requirements:

- Each child must have at least one candy.
- Children with a higher rating get more candies than their neighbors.

Return the minimum number of candies you need to have to distribute the candies to the children.

Example 1:

Input: `ratings = [1,0,2]`

Output: 5

Explanation: You can allocate to the first, second and third child with 2, 1, 2 candies respectively.

Example 2:

Input: ratings = [1,2,2]

Output: 4

Explanation: You can allocate to the first, second and third child with 1, 2, 1 candies respectively.

The third child gets 1 candy because it satisfies the above two conditions.

Constraints:

- $n == \text{ratings.length}$
- $1 \leq n \leq 2 * 10^4$
- $0 \leq \text{ratings}[i] \leq 2 * 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def candy(self, ratings: list[int]) -> int:
        n = len(ratings)

        ans = 0
        l = [1] * n
        r = [1] * n

        for i in range(1, n):
            if ratings[i] > ratings[i - 1]:
                l[i] = l[i - 1] + 1

        for i in range(n - 2, -1, -1):
            if ratings[i] > ratings[i + 1]:
                r[i] = r[i + 1] + 1

        for a, b in zip(l, r):
            ans += max(a, b)

        return ans
```

• LeetDoocs

```
class Solution:
    def candy(self, ratings: List[int]) -> int:
        n = len(ratings)
        left = [1] * n
        right = [1] * n
        for i in range(1, n):
            if ratings[i] > ratings[i - 1]:
                left[i] = left[i - 1] + 1
        for i in range(n - 2, -1, -1):
            if ratings[i] > ratings[i + 1]:
                right[i] = right[i + 1] + 1
        return sum(max(a, b) for a, b in zip(left, right))
```

• SimplyLeet

```
# Function to distribute candies according to ratings.
def candy(ratings):
    n = len(ratings)
    candies = [1] * n # Initialize with 1 candy for each child.

    # Left-to-right pass: ensure right neighbor condition.
    for i in range(1, n):
        if ratings[i] > ratings[i - 1]:
            candies[i] = candies[i - 1] + 1

    # Right-to-left pass: ensure left neighbor condition.
    for i in range(n - 2, -1, -1):
        if ratings[i] > ratings[i + 1]:
            candies[i] = max(candies[i], candies[i + 1] + 1)

    # Sum up all distributed candies.
    return sum(candies)

# Example usage:
ratings = [1, 0, 2]

print(candy(ratings)) # Expected Output: 5
```

◆ Quick Review

Key Insights

- Every child must get at least one candy.
- A left-to-right pass ensures that each child has more candies than the left neighbor if their rating is higher.
- A right-to-left pass ensures that each child has more candies than the right neighbor if their rating is higher.
- The final candy distribution for each child is determined by taking the maximum candies assigned by the two passes.

Space & Time

Time Complexity: $O(n)$ – Two passes over the array, each in linear time.

Space Complexity: $O(n)$ – An auxiliary array is used to store the candy counts for each child.

Solution

We start by initializing an array with 1 candy for each child, since each child must receive at least one candy. Then, perform a left-to-right pass: if a child's rating is higher than the previous child, assign one more candy than the previous child. Next, perform a right-to-left pass: if a child's rating is higher

than the next child, update the candy count to be the maximum of its current value or one more than the next child's candy count. The answer is the sum of the adjusted candy counts.

Greedy

□ #757 Set Intersection Size At Least Two

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Sorting
Lists: AlgoMaster 600

Problem

You are given a 2D integer array `intervals` where `intervals[i] = [starti, endi]` represents all the integers from `starti` to `endi` inclusively.

A containing set is an array `nums` where each interval from `intervals` has at least two integers in `nums`.

- For example, if `intervals = [[1,3], [3,7], [8,9]]`, then `[1,2,4,7,8,9]` and `[2,3,4,8,9]` are containing sets.

Return the minimum possible size of a containing set.

Example 1:

Input: `intervals = [[1,3],[3,7],[8,9]]`

Output: 5

Explanation: let `nums = [2, 3, 4, 8, 9]`.

It can be shown that there cannot be any containing array of size 4.

Example 2:

Input: intervals = [[1,3],[1,4],[2,5],[3,5]]

Output: 3

Explanation: let nums = [2, 3, 4].

It can be shown that there cannot be any containing array of size 2.

Example 3:

Input: intervals = [[1,2],[2,3],[2,4],[4,5]]

Output: 5

Explanation: let nums = [1, 2, 3, 4, 5].

It can be shown that there cannot be any containing array of size 4.

Constraints:

- $1 \leq \text{intervals.length} \leq 3000$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i < \text{end}_i \leq 108$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def intersectionSizeTwo(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: (x[1], -x[0]))
        s = e = -1
        ans = 0
        for a, b in intervals:
            if a <= s:
                continue
            if a > e:
                ans += 2
                s, e = b - 1, b
            else:
                ans += 1
                s, e = e, b
        return ans
```

• SimplyLeet

```
# Python code solution
from typing import List

class Solution:
    def intersectionSizeTwo(self, intervals: List[List[int]]) -> int:
        # sort intervals by end ascending, start descending when ends are equal
        intervals.sort(key=lambda x: (x[1], -x[0]))

        result = 0 # count of numbers in the chosen set
        # last two selected numbers; initialize them to values that would not
        # interfere with any interval.

        # They represent the two largest numbers added so far that are used to
        # cover intervals.
        # We initialize them to values that are "unused".
        p, q = -1, -1

        for start, end in intervals:
            # Three cases depending on how many of {p, q} fall into the current
            # interval [start, end]
            # Case 1: Both numbers are already in the interval, no addition
            # required
            if start <= p:
                continue
            # Case 2: Only one of the previous numbers is in the interval

            if start <= q:
                # We need to add one number to cover interval, choose end as it
                # is the largest possible in this interval.
                result += 1
                p = q
                q = end
            else:
                # Case 3: None of the previous selected numbers are in the
                # interval.
                # We add two numbers: end-1 and end.
                result += 2
                p = end - 1
```

```
        q = end

    return result

# Example usage:
# sol = Solution()
# print(sol.intersectionSizeTwo([[1,3],[3,7],[8,9]])) # Expected output: 5
```

◆ Quick Review

Key Insights

- Use a greedy strategy by processing intervals in order of increasing end points.
- Sort the intervals by end point (and in case of tie, by start in descending order) so that choosing numbers near the end maximizes coverage for future intervals.
- For each interval, count the numbers already chosen that fall into the interval.
- If fewer than two numbers are present, add the necessary number(s) (always choosing the largest possible values within the interval to maximize overlap with future intervals).

Space & Time

Time Complexity: $O(n \log n)$ due to the sorting step where n is the number of intervals.

Space Complexity: $O(n)$ for storing the chosen numbers (in worst-case scenarios) plus auxiliary space for sorting.

Solution

The solution uses a greedy approach after sorting the intervals. First, the intervals are sorted by their end boundary, with a tie-breaker on the start boundary (in descending order) to ensure that when intervals share an end, the one with the larger start is handled last. For each interval, we check how many selected numbers (already added in our result set) fall inside the interval. If fewer than two, we add additional numbers at the end of the interval. This ensures that the numbers we add are as large as possible within the current interval, thereby maximizing the chance they will cover future intervals as well. The key idea is that by always “covering” an interval from the end, we are potentially covering as many subsequent intervals as possible.

Greedy

□ #857 Minimum Cost to Hire K Workers

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Sorting · Heap (Priority Queue)
Lists: AlgoMaster 600

Problem

There are n workers. You are given two integer arrays `quality` and `wage` where `quality[i]` is the quality of the i th worker and `wage[i]` is the minimum wage expectation for the i th worker. We want to hire exactly k workers to form a paid group. To hire a group of k workers, we must pay them according to the following rules:

1. Every worker in the paid group must be paid at least their minimum wage expectation.
2. In the group, each worker's pay must be directly proportional to their quality. This means if a worker's quality is double that of another worker in the group, then they must be paid twice as much as the other worker.

Given the integer k , return the least amount of money needed to form a paid group satisfying the above conditions. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

Input: `quality = [10,20,5]`, `wage = [70,50,30]`, `k = 2`
 Output: `105.00000`
 Explanation: We pay 70 to 0th worker and 35 to 2nd worker.

Example 2:

Input: `quality = [3,1,10,10,1]`, `wage = [4,8,2,2,7]`, `k = 3`
 Output: `30.66667`
 Explanation: We pay 4 to 0th worker, 13.33333 to 2nd and 3rd workers separately.

Constraints:

- $n == \text{quality.length} == \text{wage.length}$
- $1 \leq k \leq n \leq 104$
- $1 \leq \text{quality}[i], \text{wage}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mincostToHireWorkers(
        self,
        quality: list[int],
        wage: list[int],
        k: int,
    ) -> float:
        ans = math.inf
        qualitySum = 0
        # (wagePerQuality, quality) sorted by wagePerQuality
```

```

workers = sorted((w / q, q) for q, w in zip(quality, wage))
maxHeap = []

for wagePerQuality, q in workers:
    heapq.heappush(maxHeap, -q)
    qualitySum += q
    if len(maxHeap) > k:
        qualitySum += heapq.heappop(maxHeap)
    if len(maxHeap) == k:
        ans = min(ans, qualitySum * wagePerQuality)

return ans

```

• LeetCode

```

class Solution:
    def mincostToHireWorkers(
        self, quality: List[int], wage: List[int], k: int
    ) -> float:
        t = sorted(zip(quality, wage), key=lambda x: x[1] / x[0])
        ans, tot = inf, 0
        h = []
        for q, w in t:
            tot += q
            heappush(h, -q)

            if len(h) == k:
                ans = min(ans, w / q * tot)
                tot -= heappop(h)
        return ans

```

• SimplyLeet

```
# Python solution using heapq as a max-heap by pushing negative values.
import heapq

def mincostToHireWorkers(quality, wage, k):
    # Create a list of tuples (ratio, quality) for each worker.
    workers = []
    for q, w in zip(quality, wage):
        ratio = w / q # Minimum acceptable wage per unit quality.
        workers.append((ratio, q))

    # Sort workers by their ratio in ascending order.
    workers.sort(key=lambda x: x[0])

    quality_sum = 0 # The running sum of qualities in the current group.
    max_quality_heap = [] # Max-heap to maintain the largest quality values
    (using negatives).
    min_cost = float('inf') # Initialize the minimum cost as infinity.

    # Process each worker in order of increasing ratio.
    for ratio, q in workers:
        # Add the current worker's quality to the heap (invert sign for
        max-heap simulation).

        heapq.heappush(max_quality_heap, -q)
        quality_sum += q

        # If we have more than k workers, remove the one with the highest
        quality.
        if len(max_quality_heap) > k:
            removed_quality = -heapq.heappop(max_quality_heap)
            quality_sum -= removed_quality

        # When the group size is exactly k, calculate the cost.
        if len(max_quality_heap) == k:
```

```
        current_cost = quality_sum * ratio
        min_cost = min(min_cost, current_cost)

    return min_cost

# Example usage:
# quality = [10,20,5]
# wage = [70,50,30]
# k = 2
# Expected output: 105.00000

#print(mincostToHireWorkers([10,20,5], [70,50,30], 2))
```

◆ Quick Review

Key Insights

- Determine the wage-to-quality ratio for each worker, which represents the minimum acceptable rate per unit quality.
- By sorting workers by their ratio, you can ensure that if a worker is the highest ratio in the chosen group, then paying every worker according to this ratio meets the minimum wage requirement.
- Use a max-heap (priority queue) to maintain the smallest total quality sum among the chosen workers. If the group exceeds size k , remove the worker with the highest quality (thus the highest contribution to cost) to minimize the total cost.

- At each step (when heap size reaches k), compute the total cost as the current sum of qualities multiplied by the current worker's ratio and update the minimum cost answer.

Space & Time

Time Complexity: $O(n \log n + n \log k)$

Space Complexity: $O(n)$

Solution

This solution calculates the effective wage-to-quality ratio for each worker and sorts the workers by this ratio. The sorted order guarantees that any chosen group has a maximum ratio that enforces the proportional payment rule without violating any minimal wage constraints.

A max-heap is then used to keep track of the largest qualities among the selected workers. As we traverse through the sorted list, we add each worker's quality into the heap and update a running sum of qualities. If the heap size exceeds k , we remove the worker with the highest quality to keep only the k smallest qualities (thus minimizing cost). The current total cost candidate is then computed as the

running sum multiplied by the current worker's ratio. The minimum such candidate is the solution.

The key data structures used are:

- Array for storing worker data (ratios and quality).
- Sorting algorithm to order workers by the computed ratio.
- Max-heap (implemented using a min-heap of negative values in languages like Python) to efficiently maintain the current group's total quality.

Greedy

□ #871 Minimum Number of Refueling Stops

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy · Heap
(Priority Queue)
Lists: AlgoMaster 600

Problem

A car travels from a starting position to a destination which is target miles east of the starting position.

There are gas stations along the way. The gas stations are represented as an array stations where $\text{stations}[i] = [\text{position}_i, \text{fuel}_i]$ indicates that the i th gas station is position_i miles east of the starting position and has fuel_i liters of gas. The car starts with an infinite tank of gas, which initially has startFuel liters of fuel in it. It uses one liter of gas per one mile that it drives. When the car reaches a gas station, it may stop and refuel, transferring all the gas from the station into the car.

Return the minimum number of refueling stops the car must make in order to reach its destination. If it cannot reach the destination, return -1 .

Note that if the car reaches a gas station with 0 fuel left, the car can still refuel there. If the car reaches the destination with 0 fuel left, it is still considered to have arrived.

Example 1:

```
Input: target = 1, startFuel = 1, stations = []  
Output: 0  
Explanation: We can reach the target without refueling.
```

Example 2:

```
Input: target = 100, startFuel = 1, stations = [[10,100]]  
Output: -1  
Explanation: We can not reach the target (or even the first gas station).
```

Example 3:

```
Input: target = 100, startFuel = 10, stations =  
[[10,60],[20,30],[30,30],[60,40]]  
Output: 2  
Explanation: We start with 10 liters of fuel.  
We drive to position 10, expending 10 liters of fuel. We refuel from 0 liters  
to 60 liters of gas.  
Then, we drive from position 10 to position 60 (expending 50 liters of fuel),  
and refuel from 10 liters to 50 liters of gas. We then drive to and reach the  
target.  
We made 2 refueling stops along the way, so we return 2.
```

Constraints:

- $1 \leq \text{target}$, $\text{startFuel} \leq 109$
- $0 \leq \text{stations.length} \leq 500$

- $1 \leq \text{position}_i < \text{position}_{i+1} < \text{target}$
- $1 \leq \text{fuel}_i < 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minRefuelStops(
        self,
        target: int,
        startFuel: int,
        stations: list[list[int]],
    ) -> int:
        # dp[i] := the farthest position we can reach w / i refuels
        dp = [startFuel] + [0] * len(stations)

        for i, station in enumerate(stations):
            for j in range(i + 1, 0, -1):
                if dp[j - 1] >= station[0]:
                    dp[j] = max(dp[j], dp[j - 1] + station[1])

        for i, d in enumerate(dp):
            if d >= target:
                return i

        return -1
```

• LeetDoocs

```
class Solution:
    def minRefuelStops(
        self, target: int, startFuel: int, stations: List[List[int]]
    ) -> int:
        pq = []
        ans = pre = 0
        stations.append([target, 0])
        for pos, fuel in stations:
            dist = pos - pre
            startFuel -= dist
```

```

        while startFuel < 0 and pq:
            startFuel -= heappop(pq)
            ans += 1
        if startFuel < 0:
            return -1
        heappush(pq, -fuel)
        pre = pos
    return ans

```

• SimplyLeet

```

import heapq

def minRefuelStops(target, startFuel, stations):
    # Max-heap to store the fuels available at the stations we've passed.
    max_heap = []
    numRefuels = 0
    prev_position = 0
    fuel = startFuel

    # Append target as a final station to unify logic

    stations.append([target, 0])

    for position, station_fuel in stations:
        # Calculate distance from previous station (or start) to current
        station.
        distance = position - prev_position

        # While we don't have enough fuel to reach the current station,
        # refuel from the stations previously passed (using max fuel from
        heap).
        while fuel < distance and max_heap:
            added_fuel = -heapq.heappop(max_heap) # Retrieve max fuel
            available

```

```
        fuel += added_fuel
        numRefuels += 1

    # If we still can't reach the station, return -1.
    if fuel < distance:
        return -1

    # Subtract the fuel used to travel to this station.
    fuel -= distance
    # Store the fuel available at this station into the max-heap.

    heapq.heappush(max_heap, -station_fuel)
    # Update prev_position for next iteration.
    prev_position = position

    return numRefuels

# Example usage:
print(minRefuelStops(100, 10, [[10,60],[20,30],[30,30],[60,40]])) # Expected
output: 2
```

◆ Quick Review

Key Insights

- The car's fuel depletes as it travels, so fuel management is critical.
- Using a greedy strategy with a max-heap (priority queue) helps decide from which station to refuel.
- At each station, if the current fuel is insufficient to reach the next station (or the destination), refuel from the station with the maximum fuel among all previously passed

stations.

- Dynamic programming can also be used, but a greedy approach with a heap usually yields a simpler solution.
- Edge cases: no stations available, or fuel exactly reaches a station/destination with 0 fuel remaining.

Space & Time

Time Complexity: $O(n \log n)$ where n is the number of stations. Each station may be pushed or popped from the heap.

Space Complexity: $O(n)$ due to the storage used by the heap for up to n elements.

Solution

The solution uses a greedy approach combined with a max-heap. As the car moves towards the target, it subtracts the distance traveled from its current fuel. Whenever the fuel is insufficient to travel to the next station or destination, the algorithm refuels using the station that offered the most fuel among those passed (available in the max-heap). If no such station exists and the car cannot proceed further, the destination is unreachable (return

–1). This approach minimizes the number of stops since we always choose the best available fuel option when needed.

Topological Sort

Round 6 · Mid · Hard · 1 question

Topological Sort

□ #1203 Sort Items by Groups Respecting Dependencies

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Depth-First Search · Breadth-First Search ·
Graph Theory · Topological Sort
Lists: AlgoMaster 600

Problem

There are n items each belonging to zero or one of m groups where $\text{group}[i]$ is the group that the i -th item belongs to and it's equal to -1 if the i -th item belongs to no group. The items and the groups are zero indexed. A group can have no item belonging to it.

Return a sorted list of the items such that:

- The items that belong to the same group are next to each other in the sorted list.
- There are some relations between these items where $\text{beforeItems}[i]$ is a list containing all the items that should come before the i -th item in the sorted array (to the left of the i -th item).

Return any solution if there is more than one solution and return an empty list if there is no solution.

Example 1:

Item	Group	Before
0	-1	
1	-1	6
2	1	5
3	0	6
4	0	3, 6
5	1	
6	0	
7	-1	

Input: n = 8, m = 2, group = [-1,-1,1,0,0,1,0,-1], beforeItems =
[[],[6],[5],[6],[3,6],[],[],[]]
Output: [6,3,4,1,5,2,0,7]

Example 2:

Input: $n = 8$, $m = 2$, $group = [-1, -1, 1, 0, 0, 1, 0, -1]$, $beforeItems = [[], [6], [5], [6], [3], [], [4], []]$

Output: $[]$

Explanation: This is the same as example 1 except that 4 needs to be before 6 in the sorted list.

Constraints:

- $1 \leq m \leq n \leq 3 * 10^4$
- $group.length == beforeItems.length == n$
- $-1 \leq group[i] \leq m - 1$
- $0 \leq beforeItems[i].length \leq n - 1$
- $0 \leq beforeItems[i][j] \leq n - 1$
- $i \neq beforeItems[i][j]$
- $beforeItems[i]$ does not contain duplicates elements.

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def sortItems(
        self, n: int, m: int, group: List[int], beforeItems: List[List[int]]
    ) -> List[int]:
        def topo_sort(degree, graph, items):
            q = deque(i for _, i in enumerate(items) if degree[i] == 0)
            res = []
            while q:
                i = q.popleft()
                res.append(i)
```

```

        for j in graph[i]:
            degree[j] -= 1
            if degree[j] == 0:
                q.append(j)
        return res if len(res) == len(items) else []

idx = m
group_items = [[] for _ in range(n + m)]
for i, g in enumerate(group):
    if g == -1:

        group[i] = idx
        idx += 1
        group_items[group[i]].append(i)

item_degree = [0] * n
group_degree = [0] * (n + m)
item_graph = [[] for _ in range(n)]
group_graph = [[] for _ in range(n + m)]
for i, gi in enumerate(group):
    for j in beforeItems[i]:

        gj = group[j]
        if gi == gj:
            item_degree[i] += 1
            item_graph[j].append(i)
        else:
            group_degree[gi] += 1
            group_graph[gj].append(gi)

group_order = topo_sort(group_degree, group_graph, range(n + m))
if not group_order:

    return []
ans = []
for gi in group_order:
    items = group_items[gi]
    item_order = topo_sort(item_degree, item_graph, items)
    if len(items) != len(item_order):
        return []
    ans.extend(item_order)
return ans

```

• SimplyLeet

```
# Python solution for "Sort Items by Groups Respecting Dependencies"

from collections import defaultdict, deque

def sortItems(n, m, group, beforeItems):
    # Step 1: Reassign group ids for ungrouped items (-1) starting from m
    new_group = m
    for i in range(n):
        if group[i] == -1:
            group[i] = new_group

    new_group += 1

    total_groups = new_group

    # Graphs and indegree arrays for groups and items
    group_graph = defaultdict(list)
    group_indegree = [0] * total_groups
    item_graph = defaultdict(list)
    item_indegree = [0] * n

    # Mapping from group id to items in that group
    group_to_items = defaultdict(list)
    for i in range(n):
        group_to_items[group[i]].append(i)

    # Build graphs for items and groups based on dependencies
    for i in range(n):
        for prev in beforeItems[i]:
            # if dependency is within the same group, link in item graph
            if group[prev] == group[i]:
```

```

        item_graph[prev].append(i)
        item_indegree[i] += 1
    else:
        # dependency across groups: add edge from group[prev] to
group[i]

        group_graph[group[prev]].append(group[i])
        group_indegree[group[i]] += 1
        # Also build the item graph for dependency within items across
groups

        item_graph[prev].append(i)
        item_indegree[i] += 1

# Helper: topological sort function
def topologicalSort(nodes, graph, indegree):
    order = []
    dq = deque([node for node in nodes if indegree[node] == 0])
    while dq:
        node = dq.popleft()
        order.append(node)
        for neighbor in graph[node]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                dq.append(neighbor)
    if len(order) == len(nodes):
        return order
    else:
        return [] # cycle detected

# Topologically sort groups
groups = list(range(total_groups))
sorted_groups = topologicalSort(groups, group_graph, group_indegree[:])
if not sorted_groups:

```

```

        return [] # return empty list if cycle in groups detected

# Topologically sort items
items = list(range(n))
sorted_items = topologicalSort(items, item_graph, item_indegree[:])
if not sorted_items:
    return [] # return empty list if cycle in items detected

# Map each item to its order in the sorted_items list
item_order = {item: i for i, item in enumerate(sorted_items)}

# For each group, sort items according to the global item order
for grp in group_to_items:
    group_to_items[grp].sort(key=lambda x: item_order[x])

# Build final result following the order of groups from sorted_groups
result = []
for grp in sorted_groups:
    result.extend(group_to_items[grp])
return result

# Example usage:
if __name__ == "__main__":
    n = 8
    m = 2
    group = [-1,-1,1,0,0,1,0,-1]
    beforeItems = [[],[6],[5],[6],[3,6],[],[],[ ]]
    print(sortItems(n, m, group, beforeItems))

```

◆ Quick Review

Key Insights

- Items belonging to no group (`group[i] == -1`) should be assigned unique group ids.
- Build two graphs: one for items (to handle intra-group ordering) and one for groups (to

handle inter-group ordering).

- For every dependency ($j \rightarrow i$): if items j and i belong to the same group, add an edge in the item graph; if they belong to different groups, add an edge in the group graph.
- Use topological sort (Kahn's algorithm or DFS based) to determine a valid order for both groups and items.
- Combine the ordered items as per the sorted order of groups to get the final answer.

Space & Time

Time Complexity: $O(n + E)$ where n is the number of items and E is the total number of dependency edges.

Space Complexity: $O(n + E)$ for storing graphs, indegree arrays, and auxiliary data structures.

Solution

We first reassign group ids for ungrouped items ($\text{group}[i] == -1$) to ensure every item is assigned a group. Then, we build two graphs:

- A graph for items, connecting nodes within the same group using dependency constraints.

- A graph for groups, connecting different groups based on dependency relations across items.

Using topological sort, we obtain an ordering of groups. Then, for each group, we topologically sort the items in that group. Finally, we concatenate the sorted items in the order given by the group sort.

If any cycle is detected (either at the group level or within any group), the function returns an empty list.

Union Find

Round 6 · Mid · Hard · 1 question

Union Find

□ #924 Minimize Malware Spread

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Depth-First Search ·
Breadth-First Search · Union-Find · Graph
Theory

Lists: AlgoMaster 600

Problem

You are given a network of n nodes represented as an $n \times n$ adjacency matrix graph, where the i th node is directly connected to the j th node if $\text{graph}[i][j] == 1$.

Some nodes initial are initially infected by malware. Whenever two nodes are directly connected, and at least one of those two nodes is infected by malware, both nodes will be infected by malware. This spread of malware will continue until no more nodes can be infected in this manner.

Suppose $M(\text{initial})$ is the final number of nodes infected with malware in the entire network after the spread of malware stops. We will

remove exactly one node from initial.

Return the node that, if removed, would minimize $M(\text{initial})$. If multiple nodes could be removed to minimize $M(\text{initial})$, return such a node with the smallest index.

Note that if a node was removed from the initial list of infected nodes, it might still be infected later due to the malware spread.

Example 1:

```
Input: graph = [[1,1,0],[1,1,0],[0,0,1]], initial = [0,1]
Output: 0
```

Example 2:

```
Input: graph = [[1,0,0],[0,1,0],[0,0,1]], initial = [0,2]
Output: 0
```

Example 3:

```
Input: graph = [[1,1,1],[1,1,1],[1,1,1]], initial = [1,2]
Output: 1
```

Constraints:

- $n == \text{graph.length}$
- $n == \text{graph}[i].\text{length}$
- $2 \leq n \leq 300$
- $\text{graph}[i][j]$ is 0 or 1.
- $\text{graph}[i][j] == \text{graph}[j][i]$
- $\text{graph}[i][i] == 1$
- $1 \leq \text{initial.length} \leq n$

- $0 \leq \text{initial}[i] \leq n - 1$
- All the integers in initial are unique.

Community Solutions (Python)

• LeetDoocs

```
class UnionFind:
    __slots__ = "p", "size"

    def __init__(self, n: int):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])

        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True

    def get_size(self, root: int) -> int:
        return self.size[root]

class Solution:
    def minMalwareSpread(self, graph: List[List[int]], initial: List[int]) -> int:
```

```

n = len(graph)
uf = UnionFind(n)
for i in range(n):
    for j in range(i + 1, n):
        graph[i][j] and uf.union(i, j)
cnt = Counter(uf.find(x) for x in initial)
ans, mx = n, 0
for x in initial:
    root = uf.find(x)
    if cnt[root] > 1:
        continue
    sz = uf.get_size(root)
    if sz > mx or (sz == mx and x < ans):
        ans = x
        mx = sz
return min(initial) if ans == n else ans

```

• SimplyLeet

Python Implementation

```

def minMalwareSpread(graph, initial):
    n = len(graph)
    # visited array to track if a node has been processed.
    visited = [False] * n
    # component mapping: node -> component id, and component size mapping.
    comp_id = [-1] * n
    comp_size = {}
    current_component = 0

```

DFS function to traverse nodes in the same component.

```

def dfs(node):
    stack = [node]
    nodes_in_component = []
    while stack:
        current = stack.pop()
        if not visited[current]:
            visited[current] = True
            comp_id[current] = current_component

```

```

        nodes_in_component.append(current)
        # Check all neighbors.
        for neighbor in range(n):
            if graph[current][neighbor] == 1 and not visited[neighbor]
:
                stack.append(neighbor)
        return nodes_in_component

# Identify components.
for i in range(n):
    if not visited[i]:

        nodes = dfs(i)
        comp_size[current_component] = len(nodes)
        current_component += 1

# Count initial infected nodes in each component.
init_count = [0] * current_component
for node in initial:
    init_count[comp_id[node]] += 1

# Determine the best candidate for removal.

result = None
max_saved = -1
for node in sorted(initial):
    cid = comp_id[node]
    # Only one infected in the component, meaning removal saves component
size nodes.
    if init_count[cid] == 1:
        if comp_size[cid] > max_saved:
            max_saved = comp_size[cid]
            result = node

# If no such node, return the smallest index.
if result is None:
    result = min(initial)
return result

# Example usage:
# print(minMalwareSpread([[1,1,0],[1,1,0],[0,0,1]], [0,1]))

```

◆ Quick Review

Key Insights

- The graph is undirected and fully connected in the sense that if two nodes are in the same connected component, an infection in one will eventually infect all nodes in that component.
- By removing a node from the initial list, the remaining initially infected nodes will still infect entire connected components.
- Use connected components detection (via DFS or Union-Find) to group nodes.
- For each component, count its size and the number of initially infected nodes within it.
- Only if a component has exactly one initially infected node, removal of that node would save the rest of the component from infection.
- The final answer is the node whose removal saves the largest number of nodes (largest component size), with ties broken by smallest index. If no such node exists, return the smallest index from the initial list.

Space & Time

Time Complexity: $O(n^2)$ – We traverse the entire $n \times n$ graph to find connected

components.

Space Complexity: $O(n)$ – We store component information and visited flags for up to n nodes.

Solution

We solve the problem by first using DFS (or Union-Find) to find all connected components in the graph. For each component, we compute its total number of nodes and the count of initially infected nodes. If a component has exactly one initially infected node, then removing that particular node prevents the spread to the rest of the component. We track, for each initially infected node, how many nodes can be saved by its removal (specifically, the component size of its unique component). Finally, we return the candidate with the maximum saving; if no candidate exists (i.e., every component has multiple infections), we return the smallest index from the initial list.

Minimum Spanning Tree

Round 6 · Mid · Hard · 3 questions

Minimum Spanning Tree

□ #1168 Optimize Water Distribution in a Village

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Union-Find · Graph Theory · Heap (Priority Queue) · Minimum Spanning Tree
Lists: AlgoMaster 600

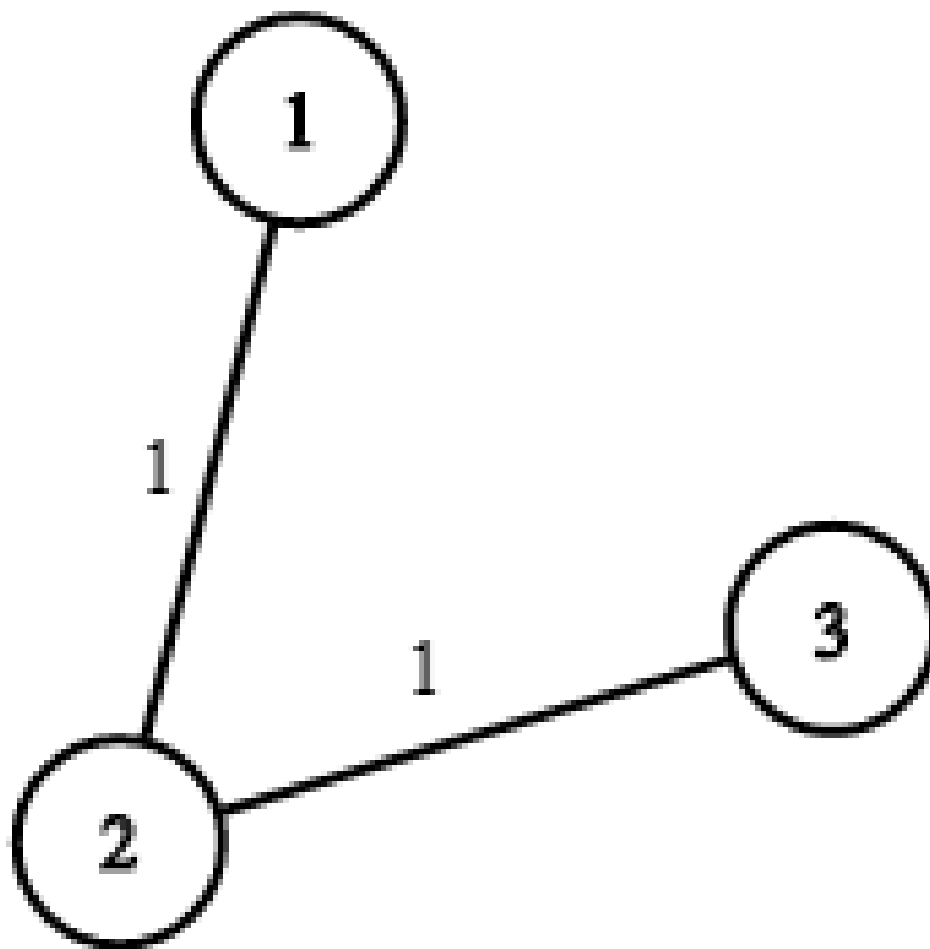
Problem

There are n houses in a village. We want to supply water for all the houses by building wells and laying pipes.

For each house i , we can either build a well inside it directly with cost $\text{wells}[i - 1]$ (note the -1 due to 0-indexing), or pipe in water from another well to it. The costs to lay pipes between houses are given by the array pipes where each $\text{pipes}[j] = [\text{house1j}, \text{house2j}, \text{costj}]$ represents the cost to connect house1j and house2j together using a pipe. Connections are bidirectional, and there could be multiple valid connections between the same two houses with different costs.

Return the minimum total cost to supply water to all houses.

Example 1:



Input: $n = 3$, wells = [1,2,2], pipes = [[1,2,1],[2,3,1]]

Output: 3

Explanation: The image shows the costs of connecting houses using pipes.

The best strategy is to build a well in the first house with cost 1 and connect the other houses to it with cost 2 so the total cost is 3.

Example 2:

Input: $n = 2$, wells = [1,1], pipes = [[1,2,1],[1,2,2]]

Output: 2

Explanation: We can supply water with cost two using one of the three options:

Option 1:

- Build a well inside house 1 with cost 1.
- Build a well inside house 2 with cost 1.

The total cost will be 2.

Option 2:

Constraints:

- $2 \leq n \leq 104$
- `wells.length == n`
- $0 \leq \text{wells}[i] \leq 105$
- $1 \leq \text{pipes.length} \leq 104$
- `pipes[j].length == 3`
- $1 \leq \text{house1j}, \text{house2j} \leq n$
- $0 \leq \text{costj} \leq 105$
- `house1j != house2j`

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minCostToSupplyWater(
        self,
        n: int,
        wells: list[int],
        pipes: list[list[int]],
    ) -> int:
        ans = 0
        graph = [[] for _ in range(n + 1)]
        minHeap = [] # (d, u)
```

```

for u, v, w in pipes:
    graph[u].append((v, w))
    graph[v].append((u, w))

# Connect virtual 0 with nodes 1 to n.
for i, well in enumerate(wells):
    graph[0].append((i + 1, well))
    heapq.heappush(minHeap, (well, i + 1))

```

```

mst = {0}

while len(mst) < n + 1:
    d, u = heapq.heappop(minHeap)
    if u in mst:
        continue
    # Add the new vertex.
    mst.add(u)
    ans += d
    # Expand if possible.

    for v, w in graph[u]:
        if v not in mst:
            heapq.heappush(minHeap, (w, v))

return ans

```

• LeetDoocs

```

class Solution:
    def minCostToSupplyWater(
        self, n: int, wells: List[int], pipes: List[List[int]]
    ) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        for i, w in enumerate(wells, 1):

```

```
        pipes.append([0, i, w])
    pipes.sort(key=lambda x: x[2])
    p = list(range(n + 1))
    ans = 0
    for a, b, c in pipes:
        pa, pb = find(a), find(b)
        if pa != pb:
            p[pa] = pb
            n -= 1
            ans += c
```

```
    if n == 0:
        return ans
```

```
class UnionFind:
    __slots__ = ("p", "size")

    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])

        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
```

```

        self.p[pa] = pb
        self.size[pb] += self.size[pa]
        return True

class Solution:
    def minCostToSupplyWater(
        self, n: int, wells: List[int], pipes: List[List[int]]
    ) -> int:
        for i, w in enumerate(wells, 1):

            pipes.append([0, i, w])
            pipes.sort(key=lambda x: x[2])
            uf = UnionFind(n + 1)
            ans = 0
            for a, b, c in pipes:
                if uf.union(a, b):
                    ans += c
                    n -= 1
                    if n == 0:
                        return ans

```

• SimplyLeet

```

# Python solution using Kruskal's algorithm and Union-Find

# Define a class to perform Union-Find (Disjoint Set Union)
class UnionFind:
    def __init__(self, size):
        # Initialize parent pointers and rank array for union by rank
        self.parent = list(range(size))
        self.rank = [0] * size

    def find(self, x):

```

```

    # Path compression to find root of x
    if self.parent[x] != x:
        self.parent[x] = self.find(self.parent[x])
    return self.parent[x]

def union(self, x, y):
    # Union two sets if roots are different; returns True if union
    performed
    rootX = self.find(x)
    rootY = self.find(y)
    if rootX == rootY:

        return False # They are already connected
    # Attach the tree with lower rank to the root of the tree with higher
    rank
    if self.rank[rootX] < self.rank[rootY]:
        self.parent[rootX] = rootY
    elif self.rank[rootX] > self.rank[rootY]:
        self.parent[rootY] = rootX
    else:
        self.parent[rootY] = rootX
        self.rank[rootX] += 1
    return True

def minCostToSupplyWater(n, wells, pipes):
    # Build the edge list, starting with edges from a virtual node 0 to every
    house
    edges = []
    for i in range(1, n + 1):
        # (cost, house, virtual node 0)
        edges.append((wells[i - 1], i, 0))
    # Add all pipe edges to the list – note these are bidirectional
    for house1, house2, cost in pipes:
        edges.append((cost, house1, house2))

```

```
# Sort edges based on cost
edges.sort()

uf = UnionFind(n + 1)
total_cost = 0

# Process each edge: if it connects two disjoint sets, add it to the MST
for cost, house1, house2 in edges:
    if uf.union(house1, house2):
        total_cost += cost

return total_cost

# Example usage:
n = 3
wells = [1, 2, 2]
pipes = [[1, 2, 1], [2, 3, 1]]
print(minCostToSupplyWater(n, wells, pipes))
```

◆ Quick Review

Key Insights

- Transform the problem by introducing a virtual node (node 0) where connecting house i to this node costs $\text{wells}[i - 1]$.
- Build a graph where the edges are both the actual pipes between houses and edges from the virtual node to each house representing the well cost.
- Use a Minimum Spanning Tree (MST) approach (either Kruskal's or Prim's algorithm) to connect all houses with the minimum cost.

- The MST will automatically choose the cheaper option between building a well and laying a connecting pipe.

Space & Time

Time Complexity: $O((n + p) \log(n + p))$, where n is the number of houses and p is the number of pipes (for sorting the edges in the MST algorithm).

Space Complexity: $O(n + p)$ to store the union–find data structures and the edge list.

Solution

We solve the problem by modeling it as an MST problem. First, we add a virtual node (0) to represent a water source. Then, we connect this node to each house i with an edge that has a cost equal to `wells[i - 1]`. Next, we add all the given pipe connections between houses to the graph. With this graph setup, the problem reduces to finding the minimum spanning tree that connects all nodes (including the virtual node). We can then use Kruskal's algorithm with a union–find data structure to build this MST. The union–find data structure helps efficiently determine

whether adding an edge will create a cycle, while sorting the edges ensures that we always add the smallest available edge that connects two components of the graph.

Minimum Spanning Tree

□ #1489 Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Union-Find · Graph Theory · Sorting ·
Minimum Spanning Tree · Strongly Connected
Component

Lists: AlgoMaster 600

Problem

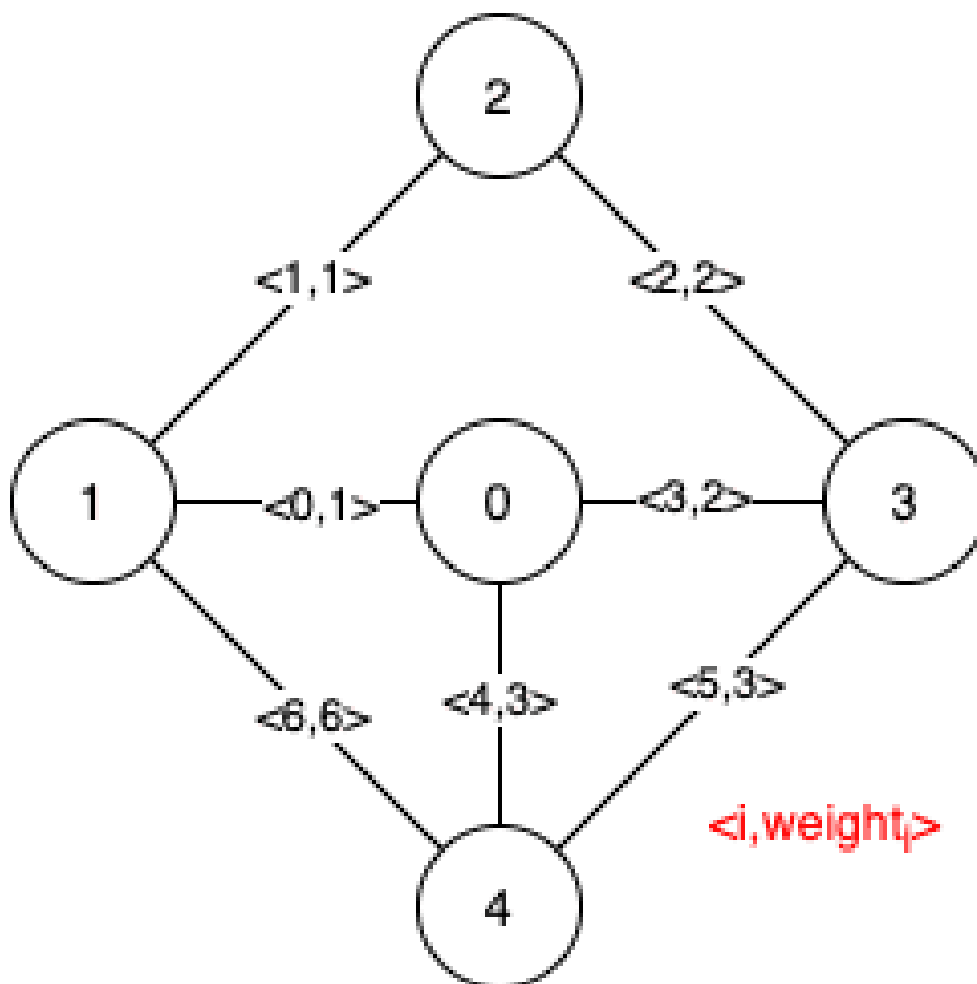
Given a weighted undirected connected graph with n vertices numbered from 0 to $n - 1$, and an array `edges` where `edges[i] = [ai, bi, weighti]` represents a bidirectional and weighted edge between nodes a_i and b_i . A minimum spanning tree (MST) is a subset of the graph's edges that connects all vertices without cycles and with the minimum possible total edge weight.

Find all the critical and pseudo-critical edges in the given graph's minimum spanning tree (MST). An MST edge whose deletion from the graph would cause the MST weight to increase is called a critical edge. On the other hand, a

pseudo-critical edge is that which can appear in some MSTs but not all.

Note that you can return the indices of the edges in any order.

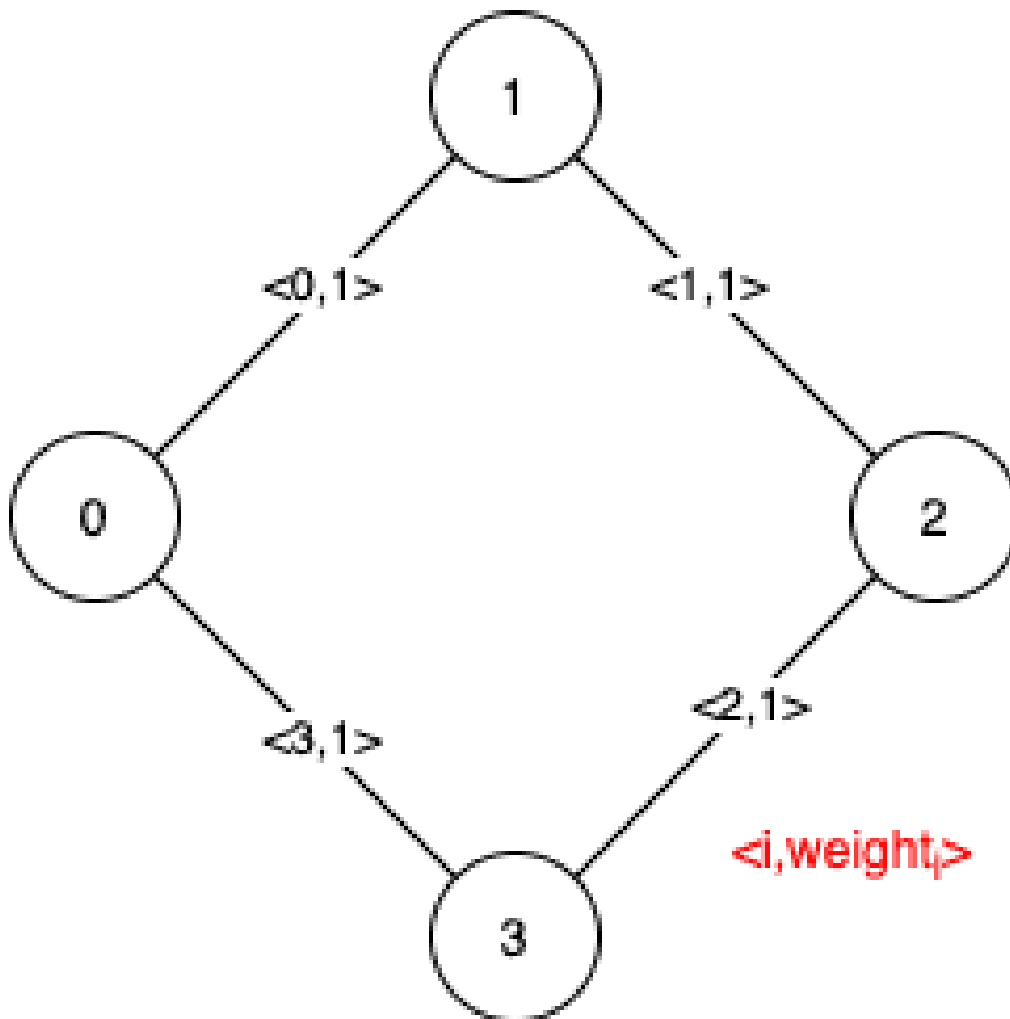
Example 1:



Input: $n = 5$, edges = $[[0,1,1],[1,2,1],[2,3,2],[0,3,2],[0,4,3],[3,4,3],[1,4,6]]$
 Output: $[[0,1],[2,3,4,5]]$
 Explanation: The figure above describes the graph.
 The following figure shows all the possible MSTs:

Notice that the two edges 0 and 1 appear in all MSTs, therefore they are critical edges, so we return them in the first list of the output.
 The edges 2, 3, 4, and 5 are only part of some MSTs, therefore they are considered pseudo-critical edges. We add them to the second list of the output.

Example 2:



Input: $n = 4$, edges = $[[0,1,1],[1,2,1],[2,3,1],[0,3,1]]$
 Output: $[[],[0,1,2,3]]$
 Explanation: We can observe that since all 4 edges have equal weight, choosing any 3 edges from the given 4 will yield an MST. Therefore all 4 edges are pseudo-critical.

Constraints:

- $2 \leq n \leq 100$
- $1 \leq \text{edges.length} \leq \min(200, n * (n - 1) / 2)$
- $\text{edges}[i].\text{length} == 3$
- $0 \leq a_i < b_i < n$
- $1 \leq \text{weight}_i \leq 1000$
- All pairs (a_i, b_i) are distinct.

Community Solutions (Python)

• WalkCC

```
class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return

        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
```

```
        self.id[u] = self.find(self.id[u])
    return self.id[u]

class Solution:
    def findCriticalAndPseudoCriticalEdges(self, n: int, edges: list[list[int]])
    -> list[list[int]]:
        criticalEdges = []
        pseudoCriticalEdges = []

        # Record the index information, so edges[i] := (u, v, weight, index).

        for i in range(len(edges)):
            edges[i].append(i)

        # Sort by the weight.
        edges.sort(key=lambda x: x[2])

        def getMSTWeight(
            firstEdge: list[int],
            deletedEdgeIndex: int) -> int | float:
            mstWeight = 0

            uf = UnionFind(n)

            if firstEdge:
                uf.unionByRank(firstEdge[0], firstEdge[1])
                mstWeight += firstEdge[2]

            for u, v, weight, index in edges:
                if index == deletedEdgeIndex:
                    continue
                if uf.find(u) == uf.find(v):
```

```

        continue
    uf.unionByRank(u, v)
    mstWeight += weight

    root = uf.find(0)
    if any(uf.find(i) != root for i in range(n)):
        return math.inf

    return mstWeight

```

```

mstWeight = getMSTWeight([], -1)

for edge in edges:
    index = edge[3]
    # Deleting the `edge` increases the weight of the MST or makes the MST
    # invalid.
    if getMSTWeight([], index) > mstWeight:
        criticalEdges.append(index)
    # If an edge can be in any MST, we can always add `edge` to the edge set.
    elif getMSTWeight(edge, -1) == mstWeight:

        pseudoCriticalEdges.append(index)

return [criticalEdges, pseudoCriticalEdges]

```

• LeetDoocs

```

class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.n = n

    def union(self, a, b):
        if self.find(a) == self.find(b):
            return False
        self.p[self.find(a)] = self.find(b)
        self.n -= 1

```



```

        return True

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

class Solution:
    def findCriticalAndPseudoCriticalEdges(
        self, n: int, edges: List[List[int]]
    ) -> List[List[int]]:
        for i, e in enumerate(edges):
            e.append(i)
        edges.sort(key=lambda x: x[2])
        uf = UnionFind(n)
        v = sum(w for f, t, w, _ in edges if uf.union(f, t))
        ans = [[], []]
        for f, t, w, i in edges:
            uf = UnionFind(n)

            k = sum(z for x, y, z, j in edges if j != i and uf.union(x, y))
            if uf.n > 1 or (uf.n == 1 and k > v):
                ans[0].append(i)
                continue

            uf = UnionFind(n)
            uf.union(f, t)
            k = w + sum(z for x, y, z, j in edges if j != i and uf.union(x, y))

        if k == v:
            ans[1].append(i)

        return ans

```

• SimplyLeet

```
# Python solution with detailed comments

class UnionFind:
    def __init__(self, n):
        # Each node is initially its own parent
        self.parent = list(range(n))
        # Rank for union-by-rank optimization
        self.rank = [0] * n

    def find(self, x):
        # Path compression optimization
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        # Union two sets; return True if they were separate
        xr = self.find(x)
        yr = self.find(y)
        if xr == yr:
            return False
        if self.rank[xr] < self.rank[yr]:
            self.parent[xr] = yr
        elif self.rank[xr] > self.rank[yr]:
            self.parent[yr] = xr
        else:
            self.parent[yr] = xr
            self.rank[xr] += 1
        return True

def buildMST(n, edges, skip_edge=None, force_edge=None):
    uf = UnionFind(n)
    weight = 0
    count = 0
    # If force_edge is provided, include it prior to processing other edges
    if force_edge is not None:
        u, v, w, idx = force_edge
        if uf.union(u, v):
            weight += w
            count += 1
```

```

# Process each edge in order of increasing weight
for u, v, w, idx in edges:
    if skip_edge is not None and idx == skip_edge:
        continue
    if uf.union(u, v):
        weight += w
        count += 1
    if count == n - 1:
        break
return weight if count == n - 1 else float('inf')

```

```

def findCriticalAndPseudoCriticalEdges(n, edges_input):
    edges = []
    # Append original indices to edges
    for idx, (u, v, w) in enumerate(edges_input):
        edges.append((u, v, w, idx))
    edges.sort(key=lambda x: x[2])
    original_weight = buildMST(n, edges)
    critical = []
    pseudo_critical = []

```

```

# Check each edge
for u, v, w, idx in edges:
    # Exclude the edge and compute MST weight
    if buildMST(n, edges, skip_edge=idx) > original_weight:
        critical.append(idx)
    else:
        # Force include this edge and compute MST weight
        if buildMST(n, edges, force_edge=(u, v, w, idx)) == original_weight:
            pseudo_critical.append(idx)
return [critical, pseudo_critical]

```

```

# Example usage:
if __name__ == "__main__":
    n = 5
    edges = [[0,1,1],[1,2,1],[2,3,2],[0,3,2],[0,4,3],[3,4,3],[1,4,6]]
    result = findCriticalAndPseudoCriticalEdges(n, edges)
    print(result)

```

◆ Quick Review

Key Insights

- Use Kruskal's algorithm to compute the MST.
- Sort the edges by weight and use Union–Find to efficiently detect cycles.
- Compute the baseline MST weight.
- For each edge, test if excluding it increases the MST weight (critical edge).
- For each edge, test if forcing its inclusion can still produce an MST with the same weight (pseudo–critical edge).

Space & Time

Time Complexity: $O(E^2 * \alpha(N))$ where E is the number of edges and α is the inverse Ackermann function.

Space Complexity: $O(N + E)$ for the Union–Find data structure and edge storage.

Solution

The solution uses Kruskal's algorithm along with a Union–Find (Disjoint Set Union) data structure:

- Compute the baseline MST weight by sorting the edges and applying Kruskal's algorithm.

- For every edge:
- To check if it is critical, recompute the MST without the edge. If the resulting weight increases or the MST cannot be formed, the edge is critical.
- To check if it is pseudo-critical, force the inclusion of the edge and then compute the MST. If the resulting total weight equals the baseline MST weight, the edge is pseudo-critical. This requires careful manipulation of the Union-Find structure and multiple MST computations.

Minimum Spanning Tree

□ #3600 Maximize Spanning Tree Stability with Upgrades **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Binary Search · Greedy · Union-Find · Graph
Theory · Minimum Spanning Tree
Lists: AlgoMaster 600

Problem

You are given an integer n , representing n nodes numbered from 0 to $n - 1$ and a list of edges, where $\text{edges}[i] = [u_i, v_i, s_i, \text{must}_i]$:

- u_i and v_i indicates an undirected edge between nodes u_i and v_i .
- s_i is the strength of the edge.
- must_i is an integer (0 or 1). If $\text{must}_i == 1$, the edge must be included in the spanning tree. These edges cannot be upgraded.

You are also given an integer k , the maximum number of upgrades you can perform. Each upgrade doubles the strength of an edge, and each eligible edge (with $\text{must}_i == 0$) can be upgraded at most once.

The stability of a spanning tree is defined as the minimum strength score among all edges included in it.

Return the maximum possible stability of any valid spanning tree. If it is impossible to connect all nodes, return -1 .

Note: A spanning tree of a graph with n nodes is a subset of the edges that connects all nodes together (i.e. the graph is connected) without forming any cycles, and uses exactly $n - 1$ edges.

Example 1:

Input: $n = 3$, $\text{edges} = [[0,1,2,1],[1,2,3,0]]$, $k = 1$

Output: 2

Explanation:

- Edge $[0,1]$ with strength = 2 must be included in the spanning tree.
- Edge $[1,2]$ is optional and can be upgraded from 3 to 6 using one upgrade.
- The resulting spanning tree includes these two edges with strengths 2 and 6.
- The minimum strength in the spanning tree is 2, which is the maximum possible stability.

Example 2:

Input: $n = 3$, $edges = [[0,1,4,0],[1,2,3,0],[0,2,1,0]]$,
 $k = 2$

Output: 6

Explanation:

- Since all edges are optional and up to $k = 2$ upgrades are allowed.
- Upgrade edges $[0,1]$ from 4 to 8 and $[1,2]$ from 3 to 6.
- The resulting spanning tree includes these two edges with strengths 8 and 6.
- The minimum strength in the tree is 6, which is the maximum possible stability.

Example 3:

Input: $n = 3$, $edges = [[0,1,1,1],[1,2,1,1],[2,0,1,1]]$,
 $k = 0$

Output: -1

Explanation:

- All edges are mandatory and form a cycle, which violates the spanning tree property of acyclicity. Thus, the answer is -1.

Constraints:

- $2 \leq n \leq 105$
- $1 \leq edges.length \leq 105$
- $edges[i] = [ui, vi, si, musti]$

- $0 \leq u_i, v_i < n$
- $u_i \neq v_i$
- $1 \leq s_i \leq 105$
- $must_i$ is either 0 or 1.
- $0 \leq k \leq n$
- There are no duplicate edges.

Community Solutions (Python)

• LeetDoocs

```
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n
        self.cnt = n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
```

```

        self.size[pb] += self.size[pa]
    self.cnt -= 1
    return True

```

```

class Solution:

```

```

    def maxStability(self, n: int, edges: List[List[int]], k: int) -> int:
        def check(lim: int) -> bool:
            uf = UnionFind(n)
            for u, v, s, _ in edges:

```

```

                if s >= lim:
                    uf.union(u, v)
            rem = k
            for u, v, s, _ in edges:
                if s * 2 >= lim and rem > 0:
                    if uf.union(u, v):
                        rem -= 1
            return uf.cnt == 1

```

```

    uf = UnionFind(n)

```

```

    mn = 10**6
    for u, v, s, must in edges:
        if must:
            mn = min(mn, s)
            if not uf.union(u, v):
                return -1
    for u, v, _, _ in edges:
        uf.union(u, v)
    if uf.cnt > 1:
        return -1

```

```

    l, r = 1, mn
    while l < r:
        mid = (l + r + 1) >> 1
        if check(mid):
            l = mid
        else:
            r = mid - 1
    return l

```

Shortest Path

Round 6 · Mid · Hard · 2 questions

Shortest Path

□ #1368 Minimum Cost to Make at Least One Valid Path in a Grid

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Breadth-First Search · Graph Theory ·
Heap (Priority Queue) · Matrix · Shortest Path
Lists: AlgoMaster 600

Problem

Given an $m \times n$ grid. Each cell of the grid has a sign pointing to the next cell you should visit if you are currently in this cell. The sign of $\text{grid}[i][j]$ can be:

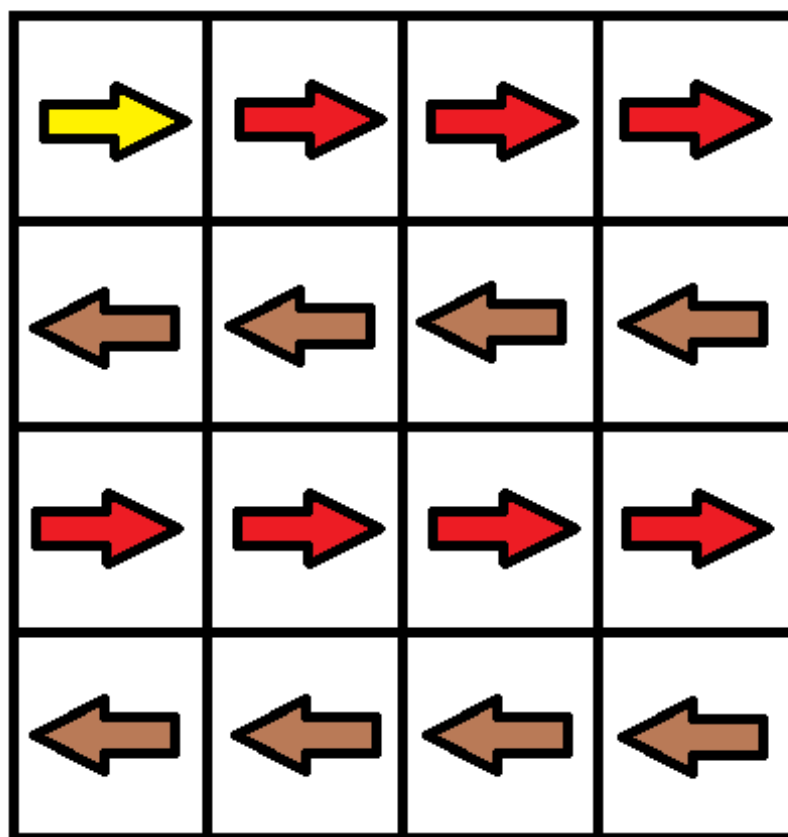
- 1 which means go to the cell to the right. (i.e go from $\text{grid}[i][j]$ to $\text{grid}[i][j + 1]$)
- 2 which means go to the cell to the left. (i.e go from $\text{grid}[i][j]$ to $\text{grid}[i][j - 1]$)
- 3 which means go to the lower cell. (i.e go from $\text{grid}[i][j]$ to $\text{grid}[i + 1][j]$)
- 4 which means go to the upper cell. (i.e go from $\text{grid}[i][j]$ to $\text{grid}[i - 1][j]$)

Notice that there could be some signs on the cells of the grid that point outside the grid.

You will initially start at the upper left cell $(0, 0)$. A valid path in the grid is a path that starts from the upper left cell $(0, 0)$ and ends at the bottom-right cell $(m - 1, n - 1)$ following the signs on the grid. The valid path does not have to be the shortest.

You can modify the sign on a cell with cost = 1. You can modify the sign on a cell one time only. Return the minimum cost to make the grid have at least one valid path.

Example 1:



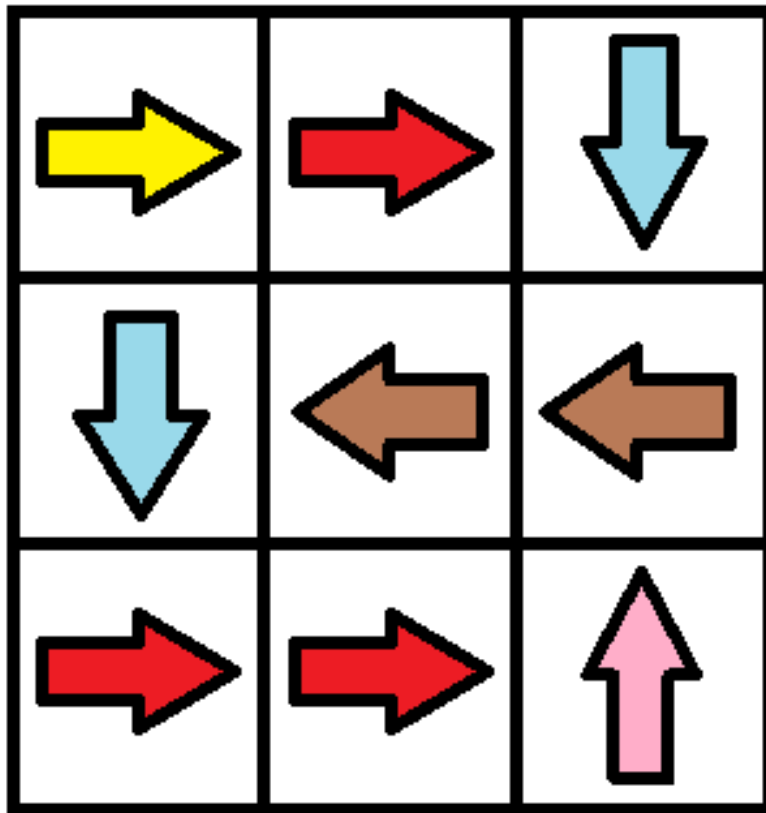
Input: grid = [[1,1,1,1],[2,2,2,2],[1,1,1,1],[2,2,2,2]]

Output: 3

Explanation: You will start at point (0, 0).

The path to (3, 3) is as follows. (0, 0) --> (0, 1) --> (0, 2) --> (0, 3)
 change the arrow to down with cost = 1 --> (1, 3) --> (1, 2) --> (1, 1) --> (1, 0)
 change the arrow to down with cost = 1 --> (2, 0) --> (2, 1) --> (2, 2) -->
 (2, 3) change the arrow to down with cost = 1 --> (3, 3)
 The total cost = 3.

Example 2:

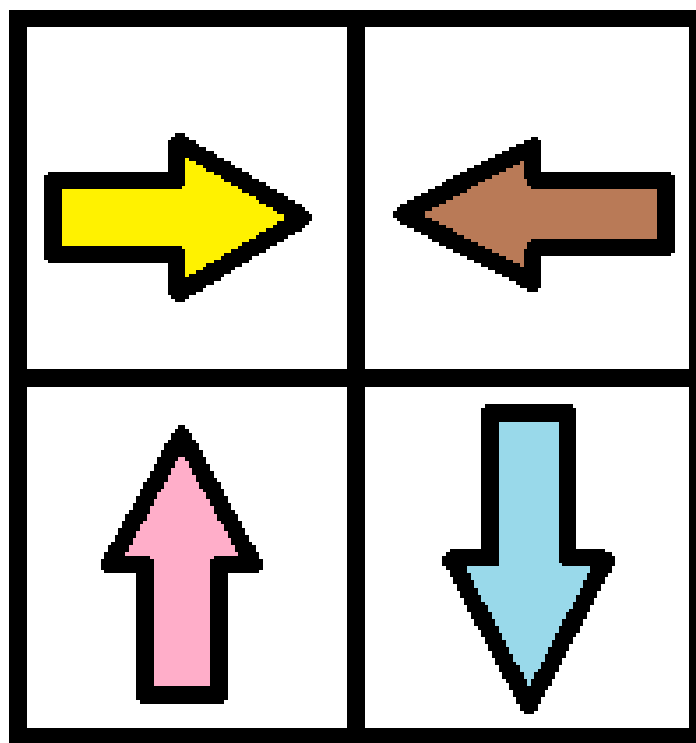


Input: grid = [[1,1,3],[3,2,2],[1,1,4]]

Output: 0

Explanation: You can follow the path from (0, 0) to (2, 2).

Example 3:



Input: `grid = [[1,2],[4,3]]`

Output: 1

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 100`
- `1 <= grid[i][j] <= 4`

Community Solutions (Python)

- WalkCC


```

class Solution:
    def minCost(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        DIRS = ((0, 1), (0, -1), (1, 0), (-1, 0))
        dp = [[-1] * n for _ in range(m)]
        q = collections.deque()

        def dfs(i: int, j: int, cost: int) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return
            if dp[i][j] != -1:
                return
            dp[i][j] = cost
            q.append((i, j))
            dx, dy = DIRS[grid[i][j] - 1]
            dfs(i + dx, j + dy, cost)

        dfs(0, 0, 0)

        cost = 0
        while q:
            cost += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for dx, dy in DIRS:
                    dfs(i + dx, j + dy, cost)

        return dp[-1][-1]

```

• LeetDoocs

```

class Solution:
    def minCost(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        dirs = [[0, 0], [0, 1], [0, -1], [1, 0], [-1, 0]]
        q = deque([(0, 0, 0)])
        vis = set()
        while q:
            i, j, d = q.popleft()
            if (i, j) in vis:
                continue

            vis.add((i, j))
            if i == m - 1 and j == n - 1:
                return d
            for k in range(1, 5):
                x, y = i + dirs[k][0], j + dirs[k][1]
                if 0 <= x < m and 0 <= y < n:
                    if grid[i][j] == k:
                        q.appendleft((x, y, d))
                    else:
                        q.append((x, y, d + 1))

        return -1

```

• SimplyLeet

```

from collections import deque

def minCost(grid):
    m, n = len(grid), len(grid[0])
    # Directions: right, left, down, up correspond to 0,1,2,3 in our 0-indexed
    list.
    directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]
    # cost matrix initialized with infinity
    cost = [[float('inf')] * n for _ in range(m)]
    cost[0][0] = 0
    dq = deque()

```

```

dq.append((0, 0))

while dq:
    i, j = dq.popleft()
    # If reached the destination, return the cost immediately.
    if i == m - 1 and j == n - 1:
        return cost[i][j]
    for k, (di, dj) in enumerate(directions):
        ni, nj = i + di, j + dj
        if 0 <= ni < m and 0 <= nj < n:
            # If the current direction (grid[i][j]) matches the direction
            # index k+1 then costChange is 0; otherwise 1.
            cost_change = 0 if grid[i][j] == k + 1 else 1
            if cost[i][j] + cost_change < cost[ni][nj]:
                cost[ni][nj] = cost[i][j] + cost_change
                if cost_change == 0:
                    dq.appendleft((ni, nj))
                else:
                    dq.append((ni, nj))
    return cost[m-1][n-1]

# Example usage:
grid = [[1,1,1,1],[2,2,2,2],[1,1,1,1],[2,2,2,2]]
print(minCost(grid))

```

◆ Quick Review

Key Insights

- The problem can be viewed as finding the shortest path in a grid where moving in the designated arrow direction costs 0 and any change (i.e., moving in any other direction) costs 1.

- This is a weighted graph problem with two edge costs: 0 and 1, making it a natural candidate for a 0–1 Breadth First Search (BFS) algorithm.
- With 0–1 BFS, we use a deque (double-ended queue) to efficiently process nodes ensuring that 0-cost moves are prioritized.
- The grid dimensions are limited ($m, n \leq 100$), so using a BFS-based algorithm leads to a manageable number of states.

Space & Time

Time Complexity: $O(m * n)$ – Each cell is processed, and there can be at most a constant number of operations per cell with 0–1 BFS.

Space Complexity: $O(m * n)$ – The auxiliary space for the cost matrix and the deque is proportional to the number of grid cells.

Solution

We use a 0–1 BFS approach to solve the problem. We view each cell as a node in a graph. For a given cell (i, j) :

- Moving in the direction indicated by the cell is free (cost 0).

– Moving in any other direction requires modifying the cell's direction (cost 1). A deque is used to store the cells to be processed. When a move has 0 cost, we add that cell to the front of the deque so that it is processed sooner; moves with cost 1 are added to the back. This guarantees that we always expand the current least-cost path. We maintain a cost matrix to keep track of the minimum cost achieved to reach each cell, stopping when the target cell $(m-1, n-1)$ is reached.

Shortest Path

□ #2203 Minimum Weighted Subgraph With the Required Paths

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Graph Theory · Heap (Priority Queue) · Shortest Path

Lists: AlgoMaster 600

Problem

You are given an integer n denoting the number of nodes of a weighted directed graph. The nodes are numbered from 0 to $n - 1$.

You are also given a 2D integer array `edges` where `edges[i] = [fromi, toi, weighti]` denotes that there exists a directed edge from `fromi` to `toi` with weight `weighti`.

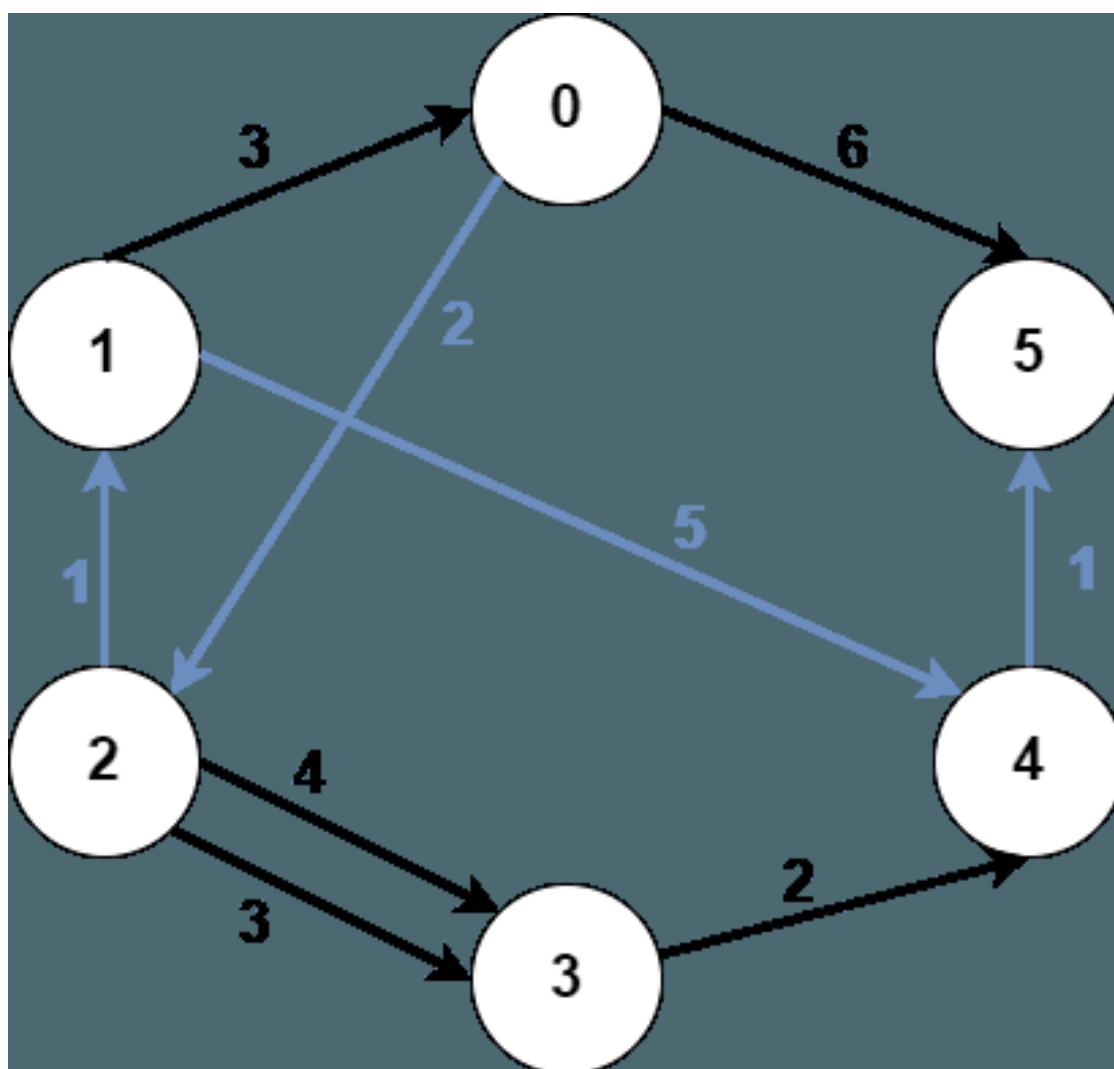
Lastly, you are given three distinct integers `src1`, `src2`, and `dest` denoting three distinct nodes of the graph.

Return the minimum weight of a subgraph of the graph such that it is possible to reach `dest` from both `src1` and `src2` via a set of edges of this subgraph. In case such a subgraph does not

exist, return -1 .

A subgraph is a graph whose vertices and edges are subsets of the original graph. The weight of a subgraph is the sum of weights of its constituent edges.

Example 1:



Input: $n = 6$, $edges =$
 $[[0,2,2],[0,5,6],[1,0,3],[1,4,5],[2,1,1],[2,3,3],[2,3,4],[3,4,2],[4,5,1]],$
 $src1 = 0$, $src2 = 1$, $dest = 5$
 Output: 9
 Explanation:
 The above figure represents the input graph.
 The blue edges represent one of the subgraphs that yield the optimal answer.
 Note that the subgraph $[[1,0,3],[0,5,6]]$ also yields the optimal answer. It is not possible to get a subgraph with less weight satisfying all the constraints.

Example 2:



Input: $n = 3$, $edges = [[0,1,1],[2,1,1]],$ $src1 = 0$, $src2 = 1$, $dest = 2$
 Output: -1
 Explanation:
 The above figure represents the input graph.
 It can be seen that there does not exist any path from node 1 to node 2, hence there are no subgraphs satisfying all the constraints.

Constraints:

- $3 \leq n \leq 105$
- $0 \leq edges.length \leq 105$
- $edges[i].length == 3$
- $0 \leq from_i, to_i, src1, src2, dest \leq n - 1$
- $from_i \neq to_i$
- $src1, src2,$ and $dest$ are pairwise distinct.
- $1 \leq weight[i] \leq 105$

Community Solutions (Python)

- WalkCC


```
class Solution:
    def minimumWeight(
        self,
        n: int,
        edges: list[list[int]],
        src1: int,
        src2: int,
        dest: int,
    ) -> int:
        graph = [[] for _ in range(n)]

        reversedGraph = [[] for _ in range(n)]

        for u, v, w in edges:
            graph[u].append((v, w))
            reversedGraph[v].append((u, w))

        fromSrc1 = self._dijkstra(graph, src1)
        fromSrc2 = self._dijkstra(graph, src2)
        fromDest = self._dijkstra(reversedGraph, dest)
        minWeight = min(a + b + c for a, b, c in zip(fromSrc1, fromSrc2, fromDest))

        return -1 if minWeight == math.inf else minWeight

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
    ) -> list[int]:
        dist = [math.inf] * len(graph)

        dist[src] = 0
```

```

minHeap = [(dist[src], src)] # (d, u)

while minHeap:
    d, u = heapq.heappop(minHeap)
    if d > dist[u]:
        continue
    for v, w in graph[u]:
        if d + w < dist[v]:
            dist[v] = d + w
            heapq.heappush(minHeap, (dist[v], v))

return dist

```

• LeetDoocs

```

class Solution:
    def minimumWeight(
        self, n: int, edges: List[List[int]], src1: int, src2: int, dest: int
    ) -> int:
        def dijkstra(g, u):
            dist = [inf] * n
            dist[u] = 0
            q = [(0, u)]
            while q:
                d, u = heapq.heappop(q)

                if d > dist[u]:
                    continue
                for v, w in g[u]:
                    if dist[v] > dist[u] + w:
                        dist[v] = dist[u] + w
                        heapq.heappush(q, (dist[v], v))
            return dist

        g = defaultdict(list)
        rg = defaultdict(list)

```

```

for f, t, w in edges:
    g[f].append((t, w))
    rg[t].append((f, w))
d1 = dijkstra(g, src1)
d2 = dijkstra(g, src2)
d3 = dijkstra(rg, dest)
ans = min(sum(v) for v in zip(d1, d2, d3))
return -1 if ans >= inf else ans

```

• SimplyLeet

```

import heapq

def minimumWeight(n, edges, src1, src2, dest):
    # Build graph in normal direction and reversed direction
    graph = [[] for _ in range(n)]
    reverse_graph = [[] for _ in range(n)]

    # Build both graphs according to the given directed weighted edges
    for u, v, w in edges:
        graph[u].append((v, w))

        reverse_graph[v].append((u, w))

    # Function to perform Dijkstra's algorithm from a source node
    def dijkstra(start, graph):
        dist = [float('inf')] * n
        dist[start] = 0
        minheap = [(0, start)] # (distance, node)
        while minheap:
            d, node = heapq.heappop(minheap)
            # If we have already found a better path, skip this entry

            if d > dist[node]:
                continue
            # Relax edges out of current node
            for neighbor, weight in graph[node]:
                new_d = d + weight
                if new_d < dist[neighbor]:
                    dist[neighbor] = new_d
                    heapq.heappush(minheap, (new_d, neighbor))
        return dist

```

```

# Compute distances from src1 and src2 on the original graph.
dist_from_src1 = dijkstra(src1, graph)
dist_from_src2 = dijkstra(src2, graph)

# Compute distances from dest on the reversed graph (equivalent to
distances from every node to dest)
dist_to_dest = dijkstra(dest, reverse_graph)

# Compute the minimum combined weight
result = float('inf')
for node in range(n):

    total = dist_from_src1[node] + dist_from_src2[node] + dist_to_dest[nod
e]

    if total < result:
        result = total

# If result remains infinity, then there's no valid subgraph meeting
requirements.
return -1 if result == float('inf') else result

# Example usage:
# n = 6, edges =
[[0,2,2],[0,5,6],[1,0,3],[1,4,5],[2,1,1],[2,3,3],[2,3,4],[3,4,2],[4,5,1]],
src1 = 0, src2 = 1, dest = 5
# Expected output: 9

print(minimumWeight(6, [[0,2,2],[0,5,6],[1,0,3],[1,4,5],[2,1,1],[2,3,3],[2,3,4
],[3,4,2],[4,5,1]], 0, 1, 5))

```

◆ Quick Review

Key Insights

- The problem essentially requires two paths: one from src1 to dest and one from src2 to dest.

- Instead of independently finding two paths, we can leverage the observation that there exists an intermediate node v such that:
- There is a shortest path from $src1$ to v .
- There is a shortest path from $src2$ to v .
- And there is a shortest path from v to $dest$.
- By reversing the graph, the path from any node v to $dest$ in the original graph becomes a path from $dest$ to v in the reversed graph.
- Running three instances of Dijkstra's algorithm (from $src1$, from $src2$, and from $dest$ on the reversed graph) gives the minimum distance from the respective source nodes to every other node.
- The answer is the minimum over all nodes v of $(dist(src1, v) + dist(src2, v) + dist(v, dest))$. If any part is unreachable for a given v , skip that candidate.

Space & Time

Time Complexity: $O(E \log N)$, where E is the number of edges and N is the number of nodes because Dijkstra's algorithm is run three times.

Space Complexity: $O(N + E)$ for the graph representation and distance arrays.

Solution

To solve the problem, we use Dijkstra's algorithm three times:

- From src1 on the original graph to get distances from src1 to every node.**
- From src2 on the original graph to get distances from src2 to every node.**
- From dest on the reversed graph to get distances from every node to dest. For each node v , we calculate $\text{totalCost} = \text{dist1}[v] + \text{dist2}[v] + \text{dist3}[v]$ (where $\text{dist3}[v]$ is the distance from v to dest calculated on the reversed graph). The answer is the minimum totalCost among all nodes v that are reachable from the respective sources. If no such candidate exists, we return -1 .**

Round 7 · Low · Easy

Round 7

Low Priority · Easy

6 questions · 4 patterns

- ☐ ● 1-D DP #509 ↗
- ☐ ● 2D Grid DP #118 ↗
- ☐ ● Maths / Geometry #168 #263 #1071 ↗
- ☐ ● String Matching #459 ↗

1-D DP

Round 7 · Low · Easy · 1 question

1-D DP

□ #509 Fibonacci Number

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Recursion ·
Memoization

Lists: AlgoMaster 600

Problem

The Fibonacci numbers, commonly denoted $F(n)$ form a sequence, called the Fibonacci sequence, such that each number is the sum of the two preceding ones, starting from 0 and 1. That is,

$$F(0) = 0, F(1) = 1$$
$$F(n) = F(n - 1) + F(n - 2), \text{ for } n > 1.$$

Given n , calculate $F(n)$.

Example 1:

Input: $n = 2$
Output: 1
Explanation: $F(2) = F(1) + F(0) = 1 + 0 = 1$.

Example 2:

Input: $n = 3$
Output: 2
Explanation: $F(3) = F(2) + F(1) = 1 + 1 = 2$.

Example 3:

Input: $n = 4$

Output: 3

Explanation: $F(4) = F(3) + F(2) = 2 + 1 = 3$.

Constraints:

- $0 \leq n \leq 30$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def fib(self, n: int) -> int:
        if n < 2:
            return n

        dp = [0, 0, 1]

        for i in range(2, n + 1):
            dp[0] = dp[1]
            dp[1] = dp[2]

            dp[2] = dp[0] + dp[1]

        return dp[2]
```

• LeetDoocs

```
class Solution:
    def fib(self, n: int) -> int:
        a, b = 0, 1
        for _ in range(n):
            a, b = b, a + b
        return a
```

```
import numpy as np

class Solution:
    def fib(self, n: int) -> int:
        factor = np.asmatrix([(1, 1), (1, 0)], np.dtype("0"))
        res = np.asmatrix([(1, 0)], np.dtype("0"))
        while n:
            if n & 1:
                res = res * factor

            factor = factor * factor
            n >>= 1
        return res[0, 1]
```

• SimplyLeet

```
# Function to compute the nth Fibonacci number
def fibonacci(n):
    # If n is 0, return 0 directly
    if n == 0:
        return 0
    # Initialize first two Fibonacci numbers
    a, b = 0, 1
    # Iterate from 2 up to n (inclusive)
    for i in range(2, n + 1):
        # Compute the next Fibonacci number by summing the last two

        a, b = b, a + b # Update a and b for the next iteration
    # For n=1, b already equals 1; for other n, result stored in b
    return b

# Example usage:
print(fibonacci(4)) # Expected output: 3
```

◆ Quick Review

Key Insights

- The Fibonacci sequence is defined recursively.
- A direct recursive solution can lead to redundant calculations.
- An iterative or dynamic programming (DP) approach can efficiently compute the result.
- For n in the range $[0, 30]$, an iterative solution with $O(n)$ time complexity is both simple and effective.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We solve the problem using an iterative approach which computes the Fibonacci numbers starting from 0 and 1 and then iteratively builds up to $F(n)$. This method uses constant space by only keeping track of the last two computed Fibonacci values. The algorithm is efficient and avoids the pitfalls of recursive stack overflow or repeated calculations that would result from a naive recursion.

2D Grid DP

Round 7 · Low · Easy · 1 question

2D Grid DP

□ #118 Pascal's Triangle

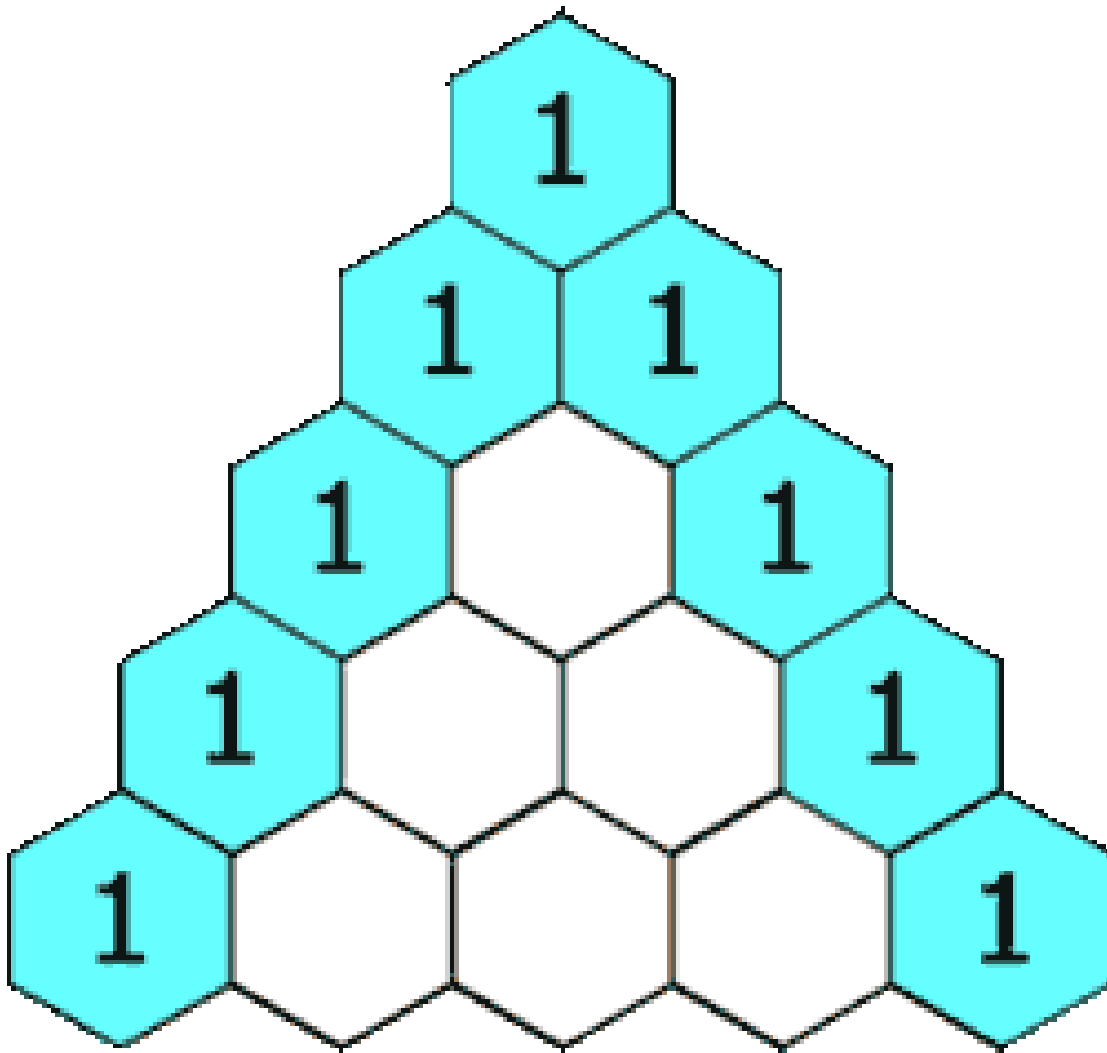
EAS

**LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: AlgoMaster 600**

Problem

Given an integer numRows, return the first numRows of Pascal's triangle.

In Pascal's triangle, each number is the sum of the two numbers directly above it as shown:



Example 1:

Input: numRows = 5

Output: [[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]

Example 2:

Input: numRows = 1

Output: [[1]]

Constraints:

- $1 \leq \text{numRows} \leq 30$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def generate(self, numRows: int) -> list[list[int]]:
        ans = []

        for i in range(numRows):
            ans.append([1] * (i + 1))

        for i in range(2, numRows):
            for j in range(1, len(ans[i]) - 1):
                ans[i][j] = ans[i - 1][j - 1] + ans[i - 1][j]

        return ans
```

• LeetCode

```
class Solution:
    def generate(self, numRows: int) -> List[List[int]]:
        f = [[1]]
        for i in range(numRows - 1):
            g = [1] + [a + b for a, b in pairwise(f[-1])] + [1]
            f.append(g)
        return f
```

• SimplyLeet

```
def generate_pascal_triangle(numRows):
    # Initialize the triangle with the first row
    triangle = [[1]]

    # Generate each row from the second to numRows
    for row_number in range(1, numRows):
        # Start the new row with the number 1
        row = [1]
        # Get the previous row to compute the current row values
        previous_row = triangle[row_number - 1]
```



```
# Compute the inner elements of the row
for j in range(1, row_number):
    # Sum of two adjacent numbers from the previous row
    row.append(previous_row[j - 1] + previous_row[j])
# End the row with the number 1
row.append(1)
# Add the constructed row to the triangle
triangle.append(row)

return triangle
```

```
# Example usage:
print(generate_pascal_triangle(5))
```

◆ Quick Review

Key Insights

- Each row begins and ends with the number 1.
- Every inner element is the sum of the two elements from the previous row.
- Only two nested loops are required: one for iterating through the rows and one for computing the inner elements.
- The algorithm steadily builds the triangle row by row.

Space & Time

Time Complexity: $O(n^2)$ – We construct each row by iterating through all previous rows.

Space Complexity: $O(n^2)$ – We store all the numbers in the triangle.

Solution

This problem is solved by iterative construction of each row. We start with a triangle containing only the first row. For each new row, we set the first and last element to 1. For the inner elements, sum the two adjacent numbers from the previous row. The primary data structure is a list (or array) of lists (or arrays) that holds the triangle. This algorithm uses nested loops: the outer loop to iterate through each row, and the inner loop to compute the inner cells of each row.

Maths / Geometry

Round 7 · Low · Easy · 3 questions

Maths / Geometry

□ #168 Excel Sheet Column Title

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String

Lists: AlgoMaster 600

Problem

Given an integer `columnNumber`, return its corresponding column title as it appears in an Excel sheet.

For example:

```
A -> 1
B -> 2
C -> 3
...
Z -> 26
AA -> 27
AB -> 28
...
```

Example 1:

```
Input: columnNumber = 1
Output: "A"
```

Example 2:

```
Input: columnNumber = 28
Output: "AB"
```

Example 3:

Input: columnNumber = 701
Output: "ZY"

Constraints:

- $1 \leq \text{columnNumber} \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def convertToTitle(self, n: int) -> str:
        return (self.convertToTitle((n - 1) // 26) + chr(ord('A') + (n - 1) % 26)
                if n
                else '')
```

• LeetDoocs

```
class Solution:
    def convertToTitle(self, columnNumber: int) -> str:
        res = []
        while columnNumber:
            columnNumber -= 1
            res.append(chr(ord('A') + columnNumber % 26))
            columnNumber //= 26
        return ''.join(res[::-1])
```

• SimplyLeet

```
# Python solution for converting a column number to Excel sheet column title
def convertToTitle(columnNumber: int) -> str:
    result = [] # list to collect column letters
    while columnNumber > 0:
        columnNumber -= 1 # adjust to 0-indexed
        remainder = columnNumber % 26 # determine current letter index
        # Convert remainder to corresponding uppercase letter
        letter = chr(ord('A') + remainder)
        result.append(letter) # add the letter to the result list
        columnNumber //= 26 # update columnNumber for next iteration
```

```
# Since letters are computed in reverse, reverse them before joining
return ''.join(reversed(result))

# Example usage:
print(convertToTitle(1))    # Output: "A"
print(convertToTitle(28))   # Output: "AB"
print(convertToTitle(701))  # Output: "ZY"
```

◆ Quick Review

Key Insights

- This problem is analogous to converting a number to a base-26 numeral system.
- The column title is like a number in base 26 but without a zero digit; letters A-Z correspond to values 1-26.
- A common trick is to subtract 1 from `columnNumber` before modulo operations to properly map the "digit" to a character.
- Repeated division and modulo will generate letters from the least significant one to the most significant, thus they need to be reversed at the end.

Space & Time

Time Complexity: $O(\log_{26}(n))$ — The number of iterations is proportional to the number of letters in the resulting string.

Space Complexity: $O(1)$ — The extra space used does not depend on the size of `columnNumber`.

Solution

The solution uses a greedy algorithm approach:

- While `columnNumber` > 0 , subtract 1 from it to adjust for the 1-indexed letter mapping.
- Compute the remainder (`columnNumber % 26`) to find the corresponding letter.
- Convert the remainder to a character by adding it to the ASCII value of 'A'.
- Append or prepend the letter to a result string.
- Divide `columnNumber` by 26 and continue until it becomes zero. The resulting string, after reversing (if built in reverse order) or by inserting at the front, gives the final Excel sheet column title.

Maths / Geometry

□ #263 Ugly Number

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math

Lists: AlgoMaster 600

Problem

An ugly number is a positive integer which does not have a prime factor other than 2, 3, and 5. Given an integer n , return true if n is an ugly number.

Example 1:

Input: $n = 6$
Output: true
Explanation: $6 = 2 \times 3$

Example 2:

Input: $n = 1$
Output: true
Explanation: 1 has no prime factors.

Example 3:

Input: $n = 14$
Output: false
Explanation: 14 is not ugly since it includes the prime factor 7.

Constraints:

- $-231 \leq n \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isUgly(self, n: int) -> bool:
        if n == 0:
            return False

        for prime in 2, 3, 5:
            while n % prime == 0:
                n //= prime

        return n == 1
```

• LeetDoocs

```
class Solution:
    def isUgly(self, n: int) -> bool:
        if n < 1:
            return False
        for x in [2, 3, 5]:
            while n % x == 0:
                n //= x
        return n == 1
```

• SimplyLeet

```
# Function to check if a number is ugly.
def is_ugly(n):
    # Only positive numbers can be ugly.
    if n <= 0:
        return False
    # Divide out factor 2.
    while n % 2 == 0:
        n //= 2
    # Divide out factor 3.
    while n % 3 == 0:
```

```
    n //= 3
    # Divide out factor 5.
    while n % 5 == 0:
        n //= 5
    # If n is reduced to 1, then it's an ugly number.
    return n == 1

# Example usage.
print(is_ugly(6))    # Output: True
print(is_ugly(14))   # Output: False
```

◆ Quick Review

Key Insights

- If n is not positive, it cannot be an ugly number.
- Continuously divide n by 2, 3, and 5 to factor out valid prime factors.
- If after removing all factors 2, 3, and 5, n becomes 1, then n is an ugly number.
- If n is not 1 after removal, then n contains other prime factors hence is not ugly.

Space & Time

Time Complexity: $O(\log n)$ since the number is divided by constant factors.

Space Complexity: $O(1)$ constant extra space.

Solution

The solution involves an iterative division process. We repeatedly divide the number by 2, 3, and 5 if it is divisible by any of them. The idea is that if n is composed solely of these factors, then after continuous division, n should reduce to 1. If n is reduced to any number other than 1, it means there are other prime factors involved, and the number is not ugly. This approach uses only basic arithmetic operations and conditional checks, which makes it efficient in terms of both time and space.

Maths / Geometry

□ #1071 Greatest Common Divisor of Strings

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String

Lists: AlgoMaster 600

Problem

For two strings s and t , we say " t divides s " if and only if $s = t + t + t + \dots + t + t$ (i.e., t is concatenated with itself one or more times).

Given two strings $str1$ and $str2$, return the largest string x such that x divides both $str1$ and $str2$.

Example 1:

Input: $str1 = \text{"ABCABC"}$, $str2 = \text{"ABC"}$

Output: "ABC"

Example 2:

Input: $str1 = \text{"ABABAB"}$, $str2 = \text{"ABAB"}$

Output: "AB"

Example 3:

Input: $str1 = \text{"LEET"}$, $str2 = \text{"CODE"}$

Output: ""

Example 4:

Input: str1 = "AAAAAB", str2 = "AAA"

Output: ""

Constraints:

- $1 \leq \text{str1.length}, \text{str2.length} \leq 1000$
- str1 and str2 consist of English uppercase letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def gcdOfStrings(self, str1: str, str2: str) -> str:
        for sz in range(min(len(str1), len(str2)), 0, -1):
            if self._isDivisible(str1, str2, sz):
                return str1[:sz]
        return ''

    def _isDivisible(self, str1: str, str2: str, sz: int) -> bool:
        """Returns True if str1 and str2 are divisible by str1[0..sz)."""
        if len(str1) % sz > 0 or len(str2) % sz > 0:
            return False
        gcd = str1[:sz]
        return str1.replace(gcd, '') == '' and str2.replace(gcd, '') == ''

class Solution:
    def gcdOfStrings(self, str1: str, str2: str) -> str:
        if str1 + str2 != str2 + str1:
            return ''
        g = math.gcd(len(str1), len(str2))
        return str1[:g]
```

• LeetCode

```
class Solution:
    def gcdOfStrings(self, str1: str, str2: str) -> str:
        def check(a, b):
            c = ""
            while len(c) < len(b):
                c += a
            return c == b

        for i in range(min(len(str1), len(str2)), 0, -1):
            t = str1[:i]

            if check(t, str1) and check(t, str2):
                return t
        return ''
```

```
class Solution:
    def gcdOfStrings(self, str1: str, str2: str) -> str:
        if str1 + str2 != str2 + str1:
            return ''
        n = gcd(len(str1), len(str2))
        return str1[:n]
```

• SimplyLeet

```
def gcd(a, b):
    # Compute the greatest common divisor using the Euclidean algorithm
    while b:
        a, b = b, a % b
    return a

def gcdOfStrings(str1, str2):
    # Check if concatenating the strings in different orders gives the same
    # result
    if str1 + str2 != str2 + str1:
        return ''
```

```
# Compute the gcd of the lengths of str1 and str2
gcd_length = gcd(len(str1), len(str2))
# The substring of str1 with length equal to gcd_length is the greatest
common divisor string
return str1[:gcd_length]

# Example usage:
print(gcdOfStrings("ABCABC", "ABC")) # Output: "ABC"
```

◆ Quick Review

Key Insights

- The necessary condition for a common divisor string is that $\text{str1} + \text{str2}$ must equal $\text{str2} + \text{str1}$.
- The length of the candidate divisor string must be a divisor of both string lengths.
- Using the greatest common divisor (gcd) of the lengths of str1 and str2 gives the length of the largest possible candidate.
- Once a candidate is found, it can be verified by checking if repeating it forms the original strings.

Space & Time

Time Complexity: $O(n + m)$, where n and m are the lengths of str1 and str2 respectively.

Space Complexity: $O(n + m)$, primarily for storage of the input strings and the resulting substring.

Solution

The solution first checks if $\text{str1} + \text{str2}$ equals $\text{str2} + \text{str1}$; if not, no common divisor exists and an empty string is returned. Otherwise, the algorithm computes the gcd of the lengths of str1 and str2 . The substring of str1 up to this gcd length is the common divisor string since repeating it forms both str1 and str2 .

String Matching

Round 7 · Low · Easy · 1 question

String Matching

□ #459 Repeated Substring Pattern

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · String Matching
Lists: AlgoMaster 600

Problem

Given a string *s*, check if it can be constructed by taking a substring of it and appending multiple copies of the substring together.

Example 1:

```
Input: s = "abab"  
Output: true  
Explanation: It is the substring "ab" twice.
```

Example 2:

```
Input: s = "aba"  
Output: false
```

Example 3:

```
Input: s = "abcabcabcabc"  
Output: true  
Explanation: It is the substring "abc" four times or the substring "abcabc" twice.
```

Constraints:

- $1 \leq s.length \leq 104$
- *s* consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def repeatedSubstringPattern(self, s: str) -> bool:
        return s in (s + s)[1:-1]
```

• LeetDoocs

```
class Solution:
    def repeatedSubstringPattern(self, s: str) -> bool:
        return (s + s).index(s, 1) < len(s)
```

• SimplyLeet

```
# Function to check if the string can be constructed by repeated substring
pattern
def repeatedSubstringPattern(s: str) -> bool:
    # Create a new string by concatenating s with itself and removing first and
    last characters
    doubled_s = (s + s)[1:-1]
    # Check if the original string s is present in the modified string
    return s in doubled_s

# Example usage:
if __name__ == "__main__":
    test_string = "abcabc"

    print(repeatedSubstringPattern(test_string)) # Expected output: True
```

◆ Quick Review

Key Insights

- The repeated substring must be a divisor of the entire string's length.

- A trick uses string manipulation by checking if the string exists in $(s+s)$ with its first and last characters removed.
- Alternatively, iterate through all possible substring lengths (that are divisors of the total length) and check if repeating the substring yields the original string.

Space & Time

Time Complexity: $O(n^2)$ in the worst case if checking each possible candidate manually, but the trick method runs in $O(n)$.

Space Complexity: $O(n)$ due to the extra temporary string created in the trick method.

Solution

We can solve the problem using one of two approaches:

- Brute-force: Try each possible substring whose length divides the original string length. For each candidate, repeat it and compare with the original string.
- Trick approach: Form a new string by concatenating the original string with itself and remove the first and last characters. Then check if the original string exists in this

modified string. If it does, the original string is composed of repeated substrings.

The trick approach is more elegant and efficient. It leverages the fact that a repeated pattern will appear at least twice in the overlapped string.

Round 8 · Low · Medium

Round 8

Low Priority · Medium

42 questions · 15 patterns

Bucket Sort #164 #451

1-D DP #343 #646 #740 #1262

0/1 Knapsack #474 #1049

Unbounded Knapsack #279 #983

**Longest Increasing Subsequence (LIS) #673
#1027 #1048**

2D Grid DP #63 #120 #931 #1277 #1937

String DP #1653

Tree / Graph DP #95 #337 #1976

Bitmask DP #698 #1066 #1986 #2305

Digit DP #357

Probability DP #688 #808 #837

State Machine DP #714

Round 7 · Low · Easy

p.1722

**Maths / Geometry #172 #204 #264 #593 #611
#939 #963 #1922**

String Matching #686 #1698

Binary Indexed Tree / Segment Tree #308

Bucket Sort

Round 8 · Low · Medium · 2 questions

Bucket Sort

□ #164 Maximum Gap

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting · Bucket Sort · Radix Sort
Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the maximum difference between two successive elements in its sorted form. If the array contains less than two elements, return 0.

You must write an algorithm that runs in linear time and uses linear extra space.

Example 1:

Input: `nums = [3,6,9,1]`

Output: 3

Explanation: The sorted form of the array is `[1,3,6,9]`, either `(3,6)` or `(6,9)` has the maximum difference 3.

Example 2:

Input: `nums = [10]`

Output: 0

Explanation: The array contains less than 2 elements, therefore return 0.

Constraints:

- $1 \leq \text{nums.length} \leq 105$

- $0 \leq \text{nums}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Bucket:
    def __init__(self, mn: int, mx: int):
        self.mn = mn
        self.mx = mx

class Solution:
    def maximumGap(self, nums: list[int]) -> int:
        if len(nums) < 2:
            return 0

        mn = min(nums)
        mx = max(nums)
        if mn == mx:
            return 0

        gap = math.ceil((mx - mn) / (len(nums) - 1))
        bucketSize = (mx - mn) // gap + 1
        buckets = [Bucket(math.inf, -math.inf) for _ in range(bucketSize)]

        for num in nums:
            i = (num - mn) // gap
            buckets[i].mn = min(buckets[i].mn, num)
            buckets[i].mx = max(buckets[i].mx, num)

        ans = 0
        prevMax = mn

        for bucket in buckets:
            if bucket.mn == math.inf:
```

```

        continue # empty bucket
    ans = max(ans, bucket.mn - prevMax)
    prevMax = bucket.mx

    return ans

```

• LeetDoocs

```

class Solution:
    def maximumGap(self, nums: List[int]) -> int:
        n = len(nums)
        if n < 2:
            return 0
        mi, mx = min(nums), max(nums)
        bucket_size = max(1, (mx - mi) // (n - 1))
        bucket_count = (mx - mi) // bucket_size + 1
        buckets = [[inf, -inf] for _ in range(bucket_count)]
        for v in nums:
            i = (v - mi) // bucket_size
            buckets[i][0] = min(buckets[i][0], v)
            buckets[i][1] = max(buckets[i][1], v)
        ans = 0
        prev = inf
        for curmin, curmax in buckets:
            if curmin > curmax:
                continue
            ans = max(ans, curmin - prev)
            prev = curmax

        return ans

```

• SimplyLeet

```
# Python solution for Maximum Gap
def maximum_gap(nums):
    # If there are fewer than 2 elements, return 0
    if len(nums) < 2:
        return 0

    # Find the minimum and maximum values in the array
    min_val, max_val = min(nums), max(nums)
    if min_val == max_val:
        return 0 # All numbers are the same

    # Calculate the bucket size using a ceiling division method, ensuring at
    # least 1
    n = len(nums)
    bucket_size = max(1, (max_val - min_val) // (n - 1))
    bucket_count = (max_val - min_val) // bucket_size + 1

    # Initialize buckets where each bucket holds [min_value, max_value]
    buckets = [[None, None] for _ in range(bucket_count)]

    # Distribute the numbers into the buckets

    for num in nums:
        # Identify the index for the bucket this number belongs to
        idx = (num - min_val) // bucket_size
        # Update bucket's min and max values
        if buckets[idx][0] is None:
            buckets[idx][0] = num
            buckets[idx][1] = num
        else:
            buckets[idx][0] = min(buckets[idx][0], num)
            buckets[idx][1] = max(buckets[idx][1], num)
```

```
# Compute the maximum gap from the buckets.
max_gap = 0
previous_max = None

for b in buckets:
    # Ignore empty buckets
    if b[0] is None:
        continue
    if previous_max is not None:
        max_gap = max(max_gap, b[0] - previous_max)
        previous_max = b[1]

return max_gap

# Example usage:
print(maximum_gap([3, 6, 9, 1])) # Output: 3
print(maximum_gap([10]))        # Output: 0
```

◆ Quick Review

Key Insights

- Sorting the array explicitly would take $O(n \log n)$ time, which is not acceptable; instead, we can use an optimal bucket sort-based approach.
- The idea is to determine the minimum and maximum values in the array.
- The maximum gap will occur not within a bucket but between adjacent buckets.
- Compute appropriate bucket size (gap) and then distribute numbers across these buckets.

- Track the minimum and maximum value within each bucket.
- The maximum gap is the maximum difference between the minimum value of the current non-empty bucket and the maximum value of the previous non-empty bucket.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution involves these steps:

- Find the minimum and maximum value in the array. If they are equal, return 0 because all numbers are the same.
- Compute the bucket size using the formula:
 $\text{bucket_size} = \max(1, (\text{max_val} - \text{min_val}) / (n - 1))$.
- Calculate the number of buckets required:
 $\text{bucket_count} = (\text{max_val} - \text{min_val}) / \text{bucket_size} + 1$.
- Initialize each bucket to store its minimum and maximum values.

- Iterate over the array and place each element in its corresponding bucket.
- Finally, iterate through the buckets and compute the maximum gap by comparing the minimum value of the current non-empty bucket with the maximum value of the previous non-empty bucket.

This bucket-based strategy ensures that the maximum gap is found without performing a full sort, maintaining linear time complexity.

Bucket Sort

□ #451 Sort Characters By Frequency

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sorting · Heap (Priority Queue) · Bucket Sort · Counting
Lists: AlgoMaster 600

Problem

Given a string *s*, sort it in decreasing order based on the frequency of the characters. The frequency of a character is the number of times it appears in the string.

Return the sorted string. If there are multiple answers, return any of them.

Example 1:

Input: *s* = "tree"
Output: "eert"
Explanation: 'e' appears twice while 'r' and 't' both appear once.
So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.

Example 2:

Input: *s* = "cccaaa"
Output: "aaaccc"
Explanation: Both 'c' and 'a' appear three times, so both "cccaaa" and "aaaccc" are valid answers.
Note that "cacaca" is incorrect, as the same characters must be together.

Example 3:

Input: s = "Aabb"

Output: "bbAa"

Explanation: "bbaA" is also a valid answer, but "Aabb" is incorrect.

Note that 'A' and 'a' are treated as two different characters.

Constraints:

- $1 \leq s.length \leq 5 * 10^5$
- s consists of uppercase and lowercase English letters and digits.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def frequencySort(self, s: str) -> str:
        ans = []
        buckets = [[] for _ in range(len(s) + 1)]

        for c, freq in collections.Counter(s).items():
            buckets[freq].append(c)

        for freq in reversed(range(len(buckets))):
            for c in buckets[freq]:
                ans.append(c * freq)

        return ''.join(ans)
```

• LeetDoocs

```
class Solution:
    def frequencySort(self, s: str) -> str:
        cnt = Counter(s)
        return ''.join(c * v for c, v in sorted(cnt.items(), key=lambda x: -x[1]))
```

• SimplyLeet

```
# Count the frequency of each character using a dictionary
def frequencySort(s: str) -> str:
    char_frequency = {}
    # Build frequency dictionary
    for char in s:
        char_frequency[char] = char_frequency.get(char, 0) + 1

    # Sort characters based on frequency in descending order
    sorted_chars = sorted(char_frequency.items(), key=lambda item: item[1], reverse=True)

    # Form the result string by repeating characters
    result = []
    for char, freq in sorted_chars:
        result.append(char * freq)

    return "".join(result)

# Example usage:
print(frequencySort("tree")) # Output could be "eert" or "eetr"
```

◆ Quick Review

Key Insights

- Use a hash map/dictionary to count the frequency of each character.
- Sort the characters by their frequency in descending order.
- Reconstruct the final string by repeating characters based on their frequency.
- The problem allows any valid ordering when frequencies are equal.

Space & Time

Time Complexity: $O(n \log n)$ in the worst case due to sorting (or $O(n)$ if bucket sort is applied).

Space Complexity: $O(n)$ for storing the frequency counts and the final output.

Solution

The solution involves:

- Iterating through the string to compute the frequency of each character using a hash table/dictionary.
- Sorting the unique characters based on their frequency in descending order.
- Building the result string by repeating each character based on its frequency. A key trick here is that the problem guarantees that the order among characters with equal frequency can be arbitrary, so a simple sort is sufficient.

1-D DP

Round 8 · Low · Medium · 4 questions

1-D DP

□ #343 Integer Break

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming
Lists: AlgoMaster 600

Problem

Given an integer n , break it into the sum of k positive integers, where $k \geq 2$, and maximize the product of those integers.

Return the maximum product you can get.

Example 1:

Input: $n = 2$
Output: 1
Explanation: $2 = 1 + 1$, $1 \times 1 = 1$.

Example 2:

Input: $n = 10$
Output: 36
Explanation: $10 = 3 + 3 + 4$, $3 \times 3 \times 4 = 36$.

Constraints:

- $2 \leq n \leq 58$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def integerBreak(self, n: int) -> int:
        # If an optimal product contains a factor f >= 4, then we can replace it
        # with 2 and f - 2 without losing optimality. As 2(f - 2) = 2f - 4 >= f,
        # we never need a factor >= 4, meaning we only need factors 1, 2, and 3
        # (and 1 is wasteful).
        # Also, 3 * 3 is better than 2 * 2 * 2, so we never use 2 more than twice.
        if n == 2: # 1 * 1
            return 1
        if n == 3: # 1 * 2

            return 2

        ans = 1

        while n > 4:
            n -= 3
            ans *= 3
        ans *= n

        return ans

```

• LeetDoocs

```

class Solution:
    def integerBreak(self, n: int) -> int:
        f = [1] * (n + 1)
        for i in range(2, n + 1):
            for j in range(1, i):
                f[i] = max(f[i], f[i - j] * j, (i - j) * j)
        return f[n]

```

```

class Solution:
    def integerBreak(self, n: int) -> int:
        if n < 4:
            return n - 1
        if n % 3 == 0:
            return pow(3, n // 3)
        if n % 3 == 1:
            return pow(3, n // 3 - 1) * 4
        return pow(3, n // 3) * 2

```

• SimplyLeet

```
def integerBreak(n: int) -> int:
    # dp[i] holds the maximum product obtainable for integer i
    dp = [0] * (n + 1)
    # Base cases: for n=2 and n=3, the maximum product is defined by their
    # splits (1*1 and 1*2).
    dp[1] = 1
    dp[2] = 1 # 2 = 1+1, product = 1 (special handling)

    for i in range(3, n + 1):
        # Compute dp[i] by trying all splits j and i-j
        for j in range(1, i):

            # Option 1: Do not split further the part (i - j)
            product1 = j * (i - j)
            # Option 2: Use optimum splitting for the part (i - j)
            product2 = j * dp[i - j]
            dp[i] = max(dp[i], product1, product2)

    return dp[n]

# Example usage:
print(integerBreak(10)) # Expected output: 36
```

◆ Quick Review

Key Insights

- For maximizing product, breaking the integer into smaller parts can yield a higher product than not splitting it.
- Greedy insight: Breaking n into as many 3's as possible generally gives the optimal solution, with adjustment for remainders (e.g. when a remainder of 1 occurs, it is beneficial to convert $3+1$ into $2+2$).

- A dynamic programming approach can also be used: compute maximum products for smaller integers and build up to n by considering all partitions.
- Be cautious with base cases ($n = 2$, $n = 3$) to ensure correctness.

Space & Time

Time Complexity: $O(n^2)$ in the dynamic programming approach since for each integer from 2 to n , we consider all possible splits.

Space Complexity: $O(n)$ because we use an array dp of size $n+1$ to store intermediate results.

Solution

We can solve this problem using a dynamic programming approach. Define $dp[i]$ to be the maximum product obtainable by breaking the integer i . For each i from 3 to n , iterate through possible first splits and update $dp[i]$ by taking the maximum of:

- The product of the two parts ($j * (i - j)$)
- The product of j and the optimal product for $(i - j)$ stored in $dp[i - j]$

This covers the scenario where further breaking the second part yields a higher product.

An alternative greedy approach is based on the mathematical insight that partitioning n into as many 3's as possible (with a check on the remainder) yields the maximum product. However, the DP approach is simple and direct, especially given the constraint that n is relatively small.

1-D DP

□ #646 Maximum Length of Pair Chain

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy ·
Sorting

Lists: AlgoMaster 600

Problem

You are given an array of n pairs `pairs` where `pairs[i] = [lefti, righti]` and $lefti < righti$.

A pair $p2 = [c, d]$ follows a pair $p1 = [a, b]$ if $b < c$. A chain of pairs can be formed in this fashion. Return the length longest chain which can be formed.

You do not need to use up all the given intervals. You can select pairs in any order.

Example 1:

Input: `pairs = [[1,2],[2,3],[3,4]]`

Output: 2

Explanation: The longest chain is `[1,2] -> [3,4]`.

Example 2:

Input: `pairs = [[1,2],[7,8],[4,5]]`

Output: 3

Explanation: The longest chain is `[1,2] -> [4,5] -> [7,8]`.

Constraints:

- $n == \text{pairs.length}$
- $1 \leq n \leq 1000$
- $-1000 \leq \text{lefti} < \text{righti} \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findLongestChain(self, pairs: list[list[int]]) -> int:
        ans = 0
        prevEnd = -math.inf

        for s, e in sorted(pairs, key=lambda x: x[1]):
            if s > prevEnd:
                ans += 1
                prevEnd = e

        return ans
```

• LeetDoocs

```
class Solution:
    def findLongestChain(self, pairs: List[List[int]]) -> int:
        pairs.sort(key=lambda x: x[1])
        ans, pre = 0, -inf
        for a, b in pairs:
            if pre < a:
                ans += 1
                pre = b
        return ans
```

• SimplyLeet

```
# Function to compute the maximum length of a pair chain
def findLongestChain(pairs):
    # Sort pairs based on their second element (end value)
    pairs.sort(key=lambda x: x[1])

    # Initialize the number of pairs in the longest chain and the current end
    # marker
    chain_length = 0
    current_end = float('-inf')

    # Iterate over each pair in the sorted array

    for pair in pairs:
        # If the current pair's start is greater than the current end, it can
        # be added to the chain
        if pair[0] > current_end:
            chain_length += 1
            current_end = pair[1] # Update the current end to the end of the
            # current pair

    return chain_length

# Example usage:
pairs = [[1,2], [7,8], [4,5]]

print(findLongestChain(pairs)) # Expected Output: 3
```

◆ Quick Review

Key Insights

- Sort the pairs based on their second (right) element to make a greedy selection.
- The greedy approach ensures that by choosing the pair with the smallest end, you leave maximum space for subsequent pairs.

- After sorting, iterate through the list and select a pair if its start is greater than the end value of the last selected pair.
- This strategy results in the optimal solution.

Space & Time

Time Complexity: $O(n \log n)$ due to the sorting step.

Space Complexity: $O(1)$ if we ignore the space used to store the input.

Solution

The solution involves sorting the pairs by their right endpoints. Start with a variable to keep track of the current end of the chain (initialized to negative infinity) and a counter for the chain length. Iterate over the sorted pairs; if the current pair's start is greater than the current end, it can be added to the chain. Update the counter and the current end accordingly. This greedy method ensures the longest possible chain is built by always choosing the next available pair that leaves room for as many subsequent pairs as possible.

1-D DP

□ #740 Delete and Earn

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Dynamic Programming
Lists: AlgoMaster 600

Problem

You are given an integer array `nums`. You want to maximize the number of points you get by performing the following operation any number of times:

- Pick any `nums[i]` and delete it to earn `nums[i]` points. Afterwards, you must delete every element equal to `nums[i] - 1` and every element equal to `nums[i] + 1`.

Return the maximum number of points you can earn by applying the above operation some number of times.

Example 1:

Input: `nums = [3,4,2]`

Output: 6

Explanation: You can perform the following operations:

- Delete 4 to earn 4 points. Consequently, 3 is also deleted. `nums = [2]`.
- Delete 2 to earn 2 points. `nums = []`.

You earn a total of 6 points.

Example 2:

Input: nums = [2,2,3,3,3,4]

Output: 9

Explanation: You can perform the following operations:

- Delete a 3 to earn 3 points. All 2's and 4's are also deleted. nums = [3,3].
- Delete a 3 again to earn 3 points. nums = [3].
- Delete a 3 once more to earn 3 points. nums = [].

You earn a total of 9 points.

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $1 \leq \text{nums}[i] \leq 10^4$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def deleteAndEarn(self, nums: List[int]) -> int:
        mx = -inf
        for num in nums:
            mx = max(mx, num)
        total = [0] * (mx + 1)
        for num in nums:
            total[num] += num
        first = total[0]
        second = max(total[0], total[1])

        for i in range(2, mx + 1):
            cur = max(first + total[i], second)
            first = second
            second = cur
        return second
```

• SimplyLeet

```
# Python solution for Delete and Earn
```

```
def deleteAndEarn(nums):  
    # Step 1: Build frequency dictionary  
    points = {}  
    for num in nums:  
        if num in points:  
            points[num] += num  
        else:  
            points[num] = num
```

```
    # Step 2: Find the maximum number in nums to define the dp array size  
    max_num = max(nums)
```

```
    # Step 3: Initialize dp array where dp[i] represents maximum points up to i  
    dp = [0] * (max_num + 1)  
    dp[0] = 0  
    dp[1] = points.get(1, 0)
```

```
    # Step 4: Fill dp using recurrence relation
```

```
    for i in range(2, max_num + 1):  
        dp[i] = max(dp[i-1], dp[i-2] + points.get(i, 0))
```

```
    # Final answer is maximum points at the maximum number  
    return dp[max_num]
```

```
# Example usage:
```

```
print(deleteAndEarn([3,4,2])) # Output: 6
```

◆ Quick Review

Key Insights

- Transform the problem by aggregating points for each unique number, similar to grouping.

- Realize that if you earn points from number x , you cannot earn points from $x-1$ and $x+1$.
- The problem resembles the House Robber problem where adjacent houses (numbers) cannot be robbed (taken).
- Use dynamic programming to decide whether to take or skip each unique number.

Space & Time

Time Complexity: $O(\text{max} + n)$ where max is the maximum number in nums (since we iterate through a frequency array up to max) and n is the length of nums .

Space Complexity: $O(\text{max})$ for the frequency and dp arrays.

Solution

The solution begins by using a hash table (or frequency array) to compute the total points that can be earned from each unique number (i.e., $\text{points} = \text{number} * \text{frequency}$). Then, we convert the problem to one similar to the House Robber problem: for each number i , decide to either take $\text{dp}[i-2]$ plus the current total points at i or skip i and take $\text{dp}[i-1]$. This decision is captured in the recurrence relation

$dp[i] = \max(dp[i-1], dp[i-2] + total[i])$. Finally, return $dp[max_num]$ as the result.

1-D DP

□ #1262 Greatest Sum Divisible by Three

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy ·
Sorting

Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the maximum possible sum of elements of the array such that it is divisible by three.

Example 1:

```
Input: nums = [3,6,5,1,8]
Output: 18
Explanation: Pick numbers 3, 6, 1 and 8 their sum is 18 (maximum sum divisible by 3).
```

Example 2:

```
Input: nums = [4]
Output: 0
Explanation: Since 4 is not divisible by 3, do not pick any number.
```

Example 3:

```
Input: nums = [1,2,3,4,4]
Output: 12
Explanation: Pick numbers 1, 3, 4 and 4 their sum is 12 (maximum sum divisible by 3).
```

Constraints:

- $1 \leq \text{nums.length} \leq 4 * 10^4$
- $1 \leq \text{nums}[i] \leq 10^4$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def maxSumDivThree(self, nums: List[int]) -> int:
        n = len(nums)
        f = [[-inf] * 3 for _ in range(n + 1)]
        f[0][0] = 0
        for i, x in enumerate(nums, 1):
            for j in range(3):
                f[i][j] = max(f[i - 1][j], f[i - 1][(j - x) % 3] + x)
        return f[n][0]
```

```
class Solution:
    def maxSumDivThree(self, nums: List[int]) -> int:
        f = [0, -inf, -inf]
        for x in nums:
            g = f[:]
            for j in range(3):
                g[j] = max(f[j], f[(j - x) % 3] + x)
            f = g
        return f[0]
```

• SimplyLeet

```
# Python Solution
def maxSumDivThree(nums):
    # Initialize dp array where:
    # dp[0]: maximum sum with remainder 0 (divisible by three)
    # dp[1]: maximum sum with remainder 1
    # dp[2]: maximum sum with remainder 2
    dp = [0, float('-inf'), float('-inf')]

    # Process each number in the list
    for num in nums:
```

```
# Create a copy of current dp to update new states
current = dp.copy()
# Iterate through each remainder state in the current dp
for i in range(3):
    # Calculate new remainder when adding the current number
    new_remainder = (i + num) % 3
    # Update dp for the new remainder with the maximum possible sum
    dp[new_remainder] = max(dp[new_remainder], current[i] + num)
# dp[0] holds the maximum sum divisible by three
return dp[0]
```

```
# Example usage:
# print(maxSumDivThree([3,6,5,1,8])) # Expected output: 18
```

◆ Quick Review

Key Insights

- The problem focuses on maximizing a sum that is divisible by three.
- Instead of trying all subsets which is inefficient, we can use dynamic programming to store the best possible sums for each remainder modulo 3.
- The idea is to maintain a state for each remainder (0, 1, 2).
- When processing each number, update the potential maximum sum for each remainder by considering the number added to previous state values.

- Use a temporary copy for state update to avoid interfering with the ongoing iteration.

Space & Time

Time Complexity: $O(n)$, where n is the length of the `nums` array.

Space Complexity: $O(1)$, since we only maintain a fixed-size array (size 3) for the dynamic programming state.

Solution

We use a dynamic programming approach where we maintain an array, `dp`, of size 3. Each index i in `dp` represents the maximum sum we can achieve that has a remainder i when divided by three. Initially, `dp[0]` is 0 (since a sum of 0 is divisible by three) and the other elements are set to a very low number to indicate that those remainders have not been achieved. For each number in the input list, we compute new possible sums for each remainder by adding this number to the existing sums. We update our `dp` array accordingly while ensuring that we do not override the results of the current iteration with the updated values immediately. Finally,

dp[0] will contain the maximum sum divisible by three.

0/1 Knapsack

Round 8 · Low · Medium · 2 questions

0/1 Knapsack

□ #474 Ones and Zeroes

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Dynamic Programming
Lists: AlgoMaster 600

Problem

You are given an array of binary strings `strs` and two integers `m` and `n`.

Return the size of the largest subset of `strs` such that there are at most `m` 0's and `n` 1's in the subset.

A set `x` is a subset of a set `y` if all elements of `x` are also elements of `y`.

Example 1:

```
Input: strs = ["10","0001","111001","1","0"], m = 5, n = 3
Output: 4
Explanation: The largest subset with at most 5 0's and 3 1's is {"10", "0001", "1", "0"}, so the answer is 4.
Other valid but smaller subsets include {"0001", "1"} and {"10", "1", "0"}.
{"111001"} is an invalid subset because it contains 4 1's, greater than the maximum of 3.
```

Example 2:

```
Input: strs = ["10","0","1"], m = 1, n = 1
Output: 2
Explanation: The largest subset is {"0", "1"}, so the answer is 2.
```

Constraints:

- $1 \leq \text{strs.length} \leq 600$
- $1 \leq \text{strs}[i].\text{length} \leq 100$
- $\text{strs}[i]$ consists only of digits '0' and '1'.
- $1 \leq m, n \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMaxForm(self, strs: list[str], m: int, n: int) -> int:
        # dp[i][j] := the maximum size of the subset given i 0s and j 1s are
        # available
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for s in strs:
            zeros = s.count('0')
            ones = len(s) - zeros
            for i in range(m, zeros - 1, -1):

                for j in range(n, ones - 1, -1):
                    dp[i][j] = max(dp[i][j], dp[i - zeros][j - ones] + 1)

        return dp[m][n]
```

• LeetDoocs

```
class Solution:
    def findMaxForm(self, strs: List[str], m: int, n: int) -> int:
        sz = len(strs)
        f = [[[0] * (n + 1) for _ in range(m + 1)] for _ in range(sz + 1)]
        for i, s in enumerate(strs, 1):
            a, b = s.count("0"), s.count("1")
            for j in range(m + 1):
                for k in range(n + 1):
                    f[i][j][k] = f[i - 1][j][k]
                    if j >= a and k >= b:
```

```

        f[i][j][k] = max(f[i][j][k], f[i - 1][j - a][k - b] +
1)
        return f[sz][m][n]

class Solution:
    def findMaxForm(self, strs: List[str], m: int, n: int) -> int:
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for s in strs:
            a, b = s.count("0"), s.count("1")
            for i in range(m, a - 1, -1):
                for j in range(n, b - 1, -1):
                    f[i][j] = max(f[i][j], f[i - a][j - b] + 1)
        return f[m][n]

```

• SimplyLeet

Python solution for Ones and Zeroes problem

```

def findMaxForm(strs, m, n):
    # Create a 2D DP array with dimensions (m+1) x (n+1)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Process each binary string in the list
    for binary_str in strs:
        count0 = binary_str.count('0') # count zeros in the string
        count1 = binary_str.count('1') # count ones in the string

        # Traverse the dp array in reverse order to avoid using the same string
        twice
        for zeros in range(m, count0 - 1, -1):
            for ones in range(n, count1 - 1, -1):
                # Decision: take max between not choosing and choosing the
                current string
                dp[zeros][ones] = max(dp[zeros][ones], dp[zeros - count0][ones
- count1] + 1)

        return dp[m][n]

# Example usage:

```

```
if __name__ == '__main__':  
    strs = ["10", "0001", "111001", "1", "0"]  
    m = 5  
    n = 3  
    print(findMaxForm(strs, m, n)) # Expected output: 4
```

◆ Quick Review

Key Insights

- Recognize this problem as a 0/1 knapsack variant with two dimensions (number of zeros and ones acting as weight limits).
- Use dynamic programming with a 2D dp array where $dp[i][j]$ represents the maximum subset size achievable with i zeros and j ones.
- Process each binary string by counting its zeros and ones, and update the dp array in reverse to avoid interference.
- Using reverse iteration ensures that each string is considered only once (mimics the 0/1 knapsack pattern).

Space & Time

Time Complexity: $O(l * m * n)$ where l is the number of strings, and m, n are the limits for zeros and ones respectively.

Space Complexity: $O(m * n)$ due to the 2D dp array used for dynamic programming.

Solution

We use a dynamic programming (DP) approach with a two-dimensional dp array of size $(m+1) \times (n+1)$. The entry $dp[i][j]$ holds the maximum number of strings that can be included in the subset with at most i zeros and j ones. For each string in the array, count the number of zeros and ones. Then, update the dp state in reverse order to ensure that each string is used only once. This reverse update is crucial to prevent overcounting since each state depends on previous states before the string was considered. The algorithm ensures that by the end, $dp[m][n]$ holds the size of the largest possible subset.

0/1 Knapsack

□ #1049 Last Stone Weight II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: AlgoMaster 600

Problem

You are given an array of integers `stones` where `stones[i]` is the weight of the *i*th stone.

We are playing a game with the stones. On each turn, we choose any two stones and smash them together. Suppose the stones have weights x and y with $x \leq y$. The result of this smash is:

- If $x == y$, both stones are destroyed, and
- If $x \neq y$, the stone of weight x is destroyed, and the stone of weight y has new weight $y - x$.

At the end of the game, there is at most one stone left.

Return the smallest possible weight of the left stone. If there are no stones left, return 0.

Example 1:

Input: stones = [2,7,4,1,8,1]

Output: 1

Explanation:

We can combine 2 and 4 to get 2, so the array converts to [2,7,1,8,1] then, we can combine 7 and 8 to get 1, so the array converts to [2,1,1,1] then, we can combine 2 and 1 to get 1, so the array converts to [1,1,1] then, we can combine 1 and 1 to get 0, so the array converts to [1], then that's the optimal value.

Example 2:

Input: stones = [31,26,33,21,40]

Output: 5

Constraints:

- $1 \leq \text{stones.length} \leq 30$
- $1 \leq \text{stones}[i] \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lastStoneWeightII(self, stones: list[int]) -> int:
        summ = sum(stones)
        s = 0
        dp = [True] + [False] * summ

        for stone in stones:
            for w in range(summ // 2 + 1)[::-1]:
                if w >= stone:
                    dp[w] = dp[w] or dp[w - stone]

        if dp[w]:
            s = max(s, w)

        return summ - 2 * s
```

• LeetDoocs

```

class Solution:
    def lastStoneWeightII(self, stones: List[int]) -> int:
        s = sum(stones)
        m, n = len(stones), s >> 1
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(n + 1):
                dp[i][j] = dp[i - 1][j]
                if stones[i - 1] <= j:
                    dp[i][j] = max(
                        dp[i][j], dp[i - 1][j - stones[i - 1]] + stones[i - 1]
                    )
        return s - 2 * dp[-1][-1]

```

• SimplyLeet

Python solution for Last Stone Weight II

```

def lastStoneWeightII(stones):
    total_sum = sum(stones) # Total sum of all stones
    half_sum = total_sum // 2 # Maximum sum we target for one of the groups
    dp = [False] * (half_sum + 1) # DP array where dp[j] indicates if sum j is
    achievable
    dp[0] = True # Base case: sum 0 is always achievable

    # Process each stone
    for stone in stones:

        # Traverse backwards to avoid using the same stone twice in one
        iteration
        for j in range(half_sum, stone - 1, -1):
            if dp[j - stone]:
                dp[j] = True

    # Find the best achievable subset sum (s1) closest to half_sum
    for s1 in range(half_sum, -1, -1):
        if dp[s1]:
            # The minimal possible remaining stone weight
            return total_sum - 2 * s1

```



```
    return 0 # Fallback, though this line should never be reached

# Example usage
print(lastStoneWeightII([2,7,4,1,8,1])) # Expected output: 1
```

◆ Quick Review

Key Insights

- The problem can be reinterpreted as a partition problem where we divide the stones into two groups, and the result is the difference between the sums of these two groups.
- The optimal result is achieved when the difference between the two group sums is minimized.
- By framing the problem this way, it becomes a variant of the classic subset sum / knapsack problem.
- Use Dynamic Programming (DP) to find the best achievable subset sum that does not exceed half of the total sum of all stones.

Space & Time

Time Complexity: $O(n * S)$ where n is the number of stones and S is the total sum of

stones.

Space Complexity: $O(S)$, where S is the total sum of stones (DP array space).

Solution

We convert the problem into a subset sum problem:

- Compute the total sum of the stones.
- Use a DP array to record which subset sums are possible.
- Iterate through each stone and update the DP array in reverse order to ensure each stone is used only once.
- Find the largest subset sum ($s1$) that does not exceed half of the total sum.
- The answer is the difference between the total sum and twice this subset sum ($\text{total} - 2 * s1$), representing the minimum remaining weight.

Unbounded Knapsack

Round 8 · Low · Medium · 2 questions

Unbounded Knapsack

□ #279 Perfect Squares

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Breadth-First
Search

Lists: AlgoMaster 600

Problem

Given an integer n , return the least number of perfect square numbers that sum to n .

A perfect square is an integer that is the square of an integer; in other words, it is the product of some integer with itself. For example, 1, 4, 9, and 16 are perfect squares while 3 and 11 are not.

Example 1:

Input: $n = 12$
Output: 3
Explanation: $12 = 4 + 4 + 4$.

Example 2:

Input: $n = 13$
Output: 2
Explanation: $13 = 4 + 9$.

Constraints:

- $1 \leq n \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numSquares(self, n: int) -> int:
        dp = [n] * (n + 1) # 1^2 x n
        dp[0] = 0 # no way
        dp[1] = 1 # 1^2

        for i in range(2, n + 1):
            j = 1
            while j * j <= i:
                dp[i] = min(dp[i], dp[i - j * j] + 1)

                j += 1

        return dp[n]
```

• LeetDoocs

```
class Solution:
    def numSquares(self, n: int) -> int:
        m = int(sqrt(n))
        f = [[inf] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 0
        for i in range(1, m + 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= i * i:
                    f[i][j] = min(f[i][j], f[i][j - i * i] + 1)

        return f[m][n]
```

```
class Solution:
    def numSquares(self, n: int) -> int:
        m = int(sqrt(n))
        f = [0] + [inf] * n
        for i in range(1, m + 1):
            for j in range(i * i, n + 1):
                f[j] = min(f[j], f[j - i * i] + 1)
        return f[n]
```

• SimplyLeet

Python solution using dynamic programming

```
def numSquares(n: int) -> int:
    # Initialize dp array where dp[i] indicates the least number of perfect
    # squares summing to i.
    dp = [float('inf')] * (n + 1)
    dp[0] = 0 # Base case: 0 perfect squares sum to 0

    # Iterate through each number from 1 to n
    for i in range(1, n + 1):
        # Try every perfect square number j*j less than or equal to i

        j = 1
        while j * j <= i:
            # Update dp[i] with the minimum count found so far
            dp[i] = min(dp[i], dp[i - j*j] + 1)
            j += 1

    return dp[n]

# Example usage:
# print(numSquares(12)) # Output: 3

# print(numSquares(13)) # Output: 2
```

◆ Quick Review

Key Insights

- The problem is similar to making change with coins, where the "coins" are the perfect squares.
- Dynamic Programming (DP) is an effective approach by building up the solution for each number from 1 to n .
- Breadth-First Search (BFS) can also be used to treat each sum as a state and find the shortest path.
- The constraint $1 \leq n \leq 10^4$ allows an $O(n * \sqrt{n})$ solution to be efficient.

Space & Time

Time Complexity: $O(n * \sqrt{n})$

Space Complexity: $O(n)$

Solution

The solution uses dynamic programming. We construct an array dp where $dp[i]$ represents the minimum number of perfect squares that sum up to i . For each number i from 1 to n , we iterate over all perfect squares jj that are less than or equal to i and update $dp[i]$ to be the minimum of its current value or $1 + dp[i - jj]$. This builds up the solution from the base case $dp[0] = 0$.

Unbounded Knapsack

□ #983 Minimum Cost For Tickets

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: AlgoMaster 600

Problem

You have planned some train traveling one year in advance. The days of the year in which you will travel are given as an integer array `days`. Each day is an integer from 1 to 365.

Train tickets are sold in three different ways:

- a 1-day pass is sold for `costs[0]` dollars,
- a 7-day pass is sold for `costs[1]` dollars, and
- a 30-day pass is sold for `costs[2]` dollars.

The passes allow that many days of consecutive travel.

- For example, if we get a 7-day pass on day 2, then we can travel for 7 days: 2, 3, 4, 5, 6, 7, and 8.

Return the minimum number of dollars you need to travel every day in the given list of days.

Example 1:

Input: days = [1,4,6,7,8,20], costs = [2,7,15]

Output: 11

Explanation: For example, here is one way to buy passes that lets you travel your travel plan:

On day 1, you bought a 1-day pass for costs[0] = \$2, which covered day 1.

On day 3, you bought a 7-day pass for costs[1] = \$7, which covered days 3, 4, ..., 9.

On day 20, you bought a 1-day pass for costs[0] = \$2, which covered day 20.

In total, you spent \$11 and covered all the days of your travel.

Example 2:

Input: days = [1,2,3,4,5,6,7,8,9,10,30,31], costs = [2,7,15]

Output: 17

Explanation: For example, here is one way to buy passes that lets you travel your travel plan:

On day 1, you bought a 30-day pass for costs[2] = \$15 which covered days 1, 2, ..., 30.

On day 31, you bought a 1-day pass for costs[0] = \$2 which covered day 31.

In total, you spent \$17 and covered all the days of your travel.

Constraints:

- $1 \leq \text{days.length} \leq 365$
- $1 \leq \text{days}[i] \leq 365$
- days is in strictly increasing order.
- $\text{costs.length} == 3$
- $1 \leq \text{costs}[i] \leq 1000$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def mincostTickets(self, days: list[int], costs: list[int]) -> int:
        ans = 0
        last7 = collections.deque()
        last30 = collections.deque()

        for day in days:
            while last7 and last7[0][0] + 7 <= day:
                last7.popleft()
            while last30 and last30[0][0] + 30 <= day:

                last30.popleft()
            last7.append([day, ans + costs[1]])
            last30.append([day, ans + costs[2]])
            ans = min(ans + costs[0], last7[0][1], last30[0][1])

        return ans

```

• LeetDoocs

```

class Solution:
    def mincostTickets(self, days: List[int], costs: List[int]) -> int:
        @cache
        def dfs(i: int) -> int:
            if i >= n:
                return 0
            ans = inf
            for c, v in zip(costs, valid):
                j = bisect_left(days, days[i] + v)
                ans = min(ans, c + dfs(j))

            return ans

        n = len(days)
        valid = [1, 7, 30]
        return dfs(0)

```

```

class Solution:
    def mincostTickets(self, days: List[int], costs: List[int]) -> int:
        m = days[-1]
        f = [0] * (m + 1)
        valid = [1, 7, 30]
        j = 0
        for i in range(1, m + 1):
            if i == days[j]:
                f[i] = inf
                for c, v in zip(costs, valid):
                    f[i] = min(f[i], f[max(0, i - v)] + c)
                j += 1
            else:
                f[i] = f[i - 1]
        return f[m]

```

• SimplyLeet

```

# Python solution with detailed comments
def mincostTickets(days, costs):
    # Create an array dp with size up to the last travel day + 1
    last_day = days[-1]
    dp = [0] * (last_day + 1)
    travel_days = set(days) # Using a set for O(1) look-up

    for day in range(1, last_day + 1):
        if day not in travel_days:
            # If it's not a travel day, cost remains as previous day's cost
            dp[day] = dp[day - 1]
        else:
            # Calculate cost if we buy a 1-day, 7-day, or 30-day pass
            cost1 = dp[day - 1] + costs[0]
            cost7 = dp[max(0, day - 7)] + costs[1]
            cost30 = dp[max(0, day - 30)] + costs[2]
            # Choose the minimum cost option
            dp[day] = min(cost1, cost7, cost30)
    return dp[last_day]

# Example usage:
print(mincostTickets([1,4,6,7,8,20], [2,7,15]))

```

◆ Quick Review

Key Insights

- Use dynamic programming to build up a solution using the minimum cost required up to each day.
- For each travel day, consider purchasing a 1-day, 7-day, or 30-day pass and choose the one that minimizes the total cost.
- Skip days without travel since they do not affect the overall cost.
- Use a DP array keyed by day or simulate the process using pointers with the travel days list.

Space & Time

Time Complexity: $O(n + \text{last_day})$ where n is the number of travel days and last_day is the maximum day value. In practice, $O(365)$ due to the year constraint.

Space Complexity: $O(\text{last_day})$ which is $O(365)$ in the worst-case scenario.

Solution

The solution uses dynamic programming. We maintain a DP array (or dictionary) where $dp[d]$ represents the minimum cost to cover all travel days up to day d . For each day d from 1 to the last travel day:

- If d is not a travel day, then $dp[d] = dp[d-1]$ because no additional cost is incurred.
- If d is a travel day, then we calculate:
 - The cost if a 1-day pass is purchased today: $dp[d-1] + costs[0]$.
 - The cost if a 7-day pass is used: $dp[\max(0, d-7)] + costs[1]$.
 - The cost if a 30-day pass is used: $dp[\max(0, d-30)] + costs[2]$.
- We then set $dp[d]$ as the minimum of these three computed values.

This DP approach ensures that we account for the overlapping intervals provided by the different passes and choose the optimal cost at each travel day.

Longest Increasing Subsequence (LIS)

Round 8 · Low · Medium · 3 questions

Longest Increasing Subsequence (LIS)

□ #673 Number of Longest Increasing Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Binary Indexed
Tree · Segment Tree
Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the number of longest increasing subsequences.

Notice that the sequence has to be strictly increasing.

Example 1:

Input: `nums = [1,3,5,4,7]`

Output: 2

Explanation: The two longest increasing subsequences are `[1, 3, 4, 7]` and `[1, 3, 5, 7]`.

Example 2:

Input: `nums = [2,2,2,2,2]`

Output: 5

Explanation: The length of the longest increasing subsequence is 1, and there are 5 increasing subsequences of length 1, so output 5.

Constraints:

- $1 \leq \text{nums.length} \leq 2000$

- $-106 \leq \text{nums}[i] \leq 106$
- The answer is guaranteed to fit inside a 32-bit integer.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findNumberOfLIS(self, nums: list[int]) -> int:
        ans = 0
        maxLength = 0
        # length[i] := the length of the LIS ending in nums[i]
        length = [1] * len(nums)
        # count[i] := the number of LIS's ending in nums[i]
        count = [1] * len(nums)

        # Calculate the `length` and `count` arrays.

        for i, num in enumerate(nums):
            for j in range(i):
                if nums[j] < num:
                    if length[i] < length[j] + 1:
                        length[i] = length[j] + 1
                        count[i] = count[j]
                    elif length[i] == length[j] + 1:
                        count[i] += count[j]

        # Get the number of LIS.

        for i, l in enumerate(length):
            if l > maxLength:
                maxLength = l
                ans = count[i]
            elif l == maxLength:
                ans += count[i]

        return ans
```

• LeetDoocs


```

class Solution:
    def findNumberOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        f = [1] * n
        cnt = [1] * n
        mx = 0
        for i in range(n):
            for j in range(i):
                if nums[j] < nums[i]:
                    if f[i] < f[j] + 1:
                        f[i] = f[j] + 1
                        cnt[i] = cnt[j]
                    elif f[i] == f[j] + 1:
                        cnt[i] += cnt[j]
            if mx < f[i]:
                mx = f[i]
                ans = cnt[i]
            elif mx == f[i]:
                ans += cnt[i]
        return ans

```

```

class BinaryIndexedTree:
    __slots__ = ["n", "c", "d"]

    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)
        self.d = [0] * (n + 1)

    def update(self, x, v, cnt):
        while x <= self.n:
            if self.c[x] < v:
                self.c[x] = v
                self.d[x] = cnt
            elif self.c[x] == v:
                self.d[x] += cnt
            x += x & -x

    def query(self, x):
        v = cnt = 0
        while x:

```

```

        if self.c[x] > v:
            v = self.c[x]
            cnt = self.d[x]
        elif self.c[x] == v:
            cnt += self.d[x]
        x -= x & -x
    return v, cnt

```

```
class Solution:
```

```

    def findNumberOfLIS(self, nums: List[int]) -> int:
        arr = sorted(set(nums))
        m = len(arr)
        tree = BinaryIndexedTree(m)
        for x in nums:
            i = bisect_left(arr, x) + 1
            v, cnt = tree.query(i - 1)
            tree.update(i, v + 1, max(cnt, 1))
        return tree.query(m)[1]

```

• SimplyLeet

```
# Python solution for Number of Longest Increasing Subsequence
```

```

def findNumberOfLIS(nums):
    # Get the length of the input array
    n = len(nums)

    # If the list is empty, return 0 (edge-case)
    if n == 0:
        return 0

```

```

# dp[i] stores the length of the longest increasing subsequence ending at i
dp = [1] * n
# count[i] stores the number of longest increasing subsequences ending at i
count = [1] * n

# Iterate over each element in nums
for i in range(n):
    # Check all previous elements to update dp[i] and count[i]
    for j in range(i):
        if nums[j] < nums[i]:

            # Found a potential sequence ending at i
            if dp[j] + 1 > dp[i]:
                # A longer subsequence is found, update length and count
                dp[i] = dp[j] + 1
                count[i] = count[j]
            elif dp[j] + 1 == dp[i]:
                # Found another subsequence of the same maximum length
                # ending at i, add count
                count[i] += count[j]

    # Find the length of the overall longest increasing subsequence in the
    # array

longest = max(dp)
# Sum the number of ways for all indices that achieve this longest length
ans = sum(c for i, c in enumerate(count) if dp[i] == longest)

return ans

# Example usage
# nums = [1,3,5,4,7]
# print(findNumberOfLIS(nums)) # Output should be 2

```

◆ Quick Review

Key Insights

- Use dynamic programming to keep track of both the length of the longest increasing

subsequence (LIS) ending at each index and the number of ways to achieve that length.

- Maintain two arrays: one (dp) for the length of the LIS ending at the current index, and another (count) for the number of such LIS ending at that index.
- For each element, iterate over all previous elements; if a previous element is less than the current one, update the dp and count arrays.
- After processing, the result is the sum of counts for indices where the dp value equals the maximum length found.

Space & Time

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

Solution

The solution uses dynamic programming with two auxiliary arrays. The dp array stores the length of the longest increasing subsequence ending at each index. The count array stores the number of ways to achieve that length at each index.

For every index i , loop through all previous indices j . If $nums[j]$ is less than $nums[i]$,

meaning an increasing sequence can be formed:

- If extending the sequence from j gives a longer subsequence than previously recorded for i (i.e., $dp[j] + 1 > dp[i]$), update $dp[i]$ and set $count[i]$ equal to $count[j]$.
- If it equals the already found length (i.e., $dp[j] + 1 == dp[i]$), then add $count[j]$ to $count[i]$ because another sequence of the same length is found.

Finally, the answer is computed by summing all $count[i]$ for which $dp[i]$ equals the maximum value in dp .

Longest Increasing Subsequence (LIS)

□ #1027 Longest Arithmetic Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Binary Search · Dynamic
Programming

Lists: AlgoMaster 600

Problem

Given an array `nums` of integers, return the length of the longest arithmetic subsequence in `nums`.

Note that:

- A subsequence is an array that can be derived from another array by deleting some or no elements without changing the order of the remaining elements.
- A sequence `seq` is arithmetic if `seq[i + 1] - seq[i]` are all the same value (for $0 \leq i < \text{seq.length} - 1$).

Example 1:

Input: `nums = [3,6,9,12]`

Output: 4

Explanation: The whole array is an arithmetic sequence with steps of length = 3.

Example 2:

Input: `nums = [9,4,7,2,10]`

Output: 3

Explanation: The longest arithmetic subsequence is `[4,7,10]`.

Example 3:

Input: `nums = [20,1,15,3,10,5,8]`

Output: 4

Explanation: The longest arithmetic subsequence is `[20,15,10,5]`.

Constraints:

- $2 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] \leq 500$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestArithSeqLength(self, nums: list[int]) -> int:
        n = len(nums)
        ans = 0
        # dp[i][k] := the length of the longest arithmetic subsequence of
        # nums[0..i]
        # with k = diff + 500
        dp = [[0] * 1001 for _ in range(n)]

        for i in range(n):
            for j in range(i):
                k = nums[i] - nums[j] + 500
                dp[i][k] = max(2, dp[j][k] + 1)
                ans = max(ans, dp[i][k])

        return ans
```

• LeetDoocs

```

class Solution:
    def longestArithSeqLength(self, nums: List[int]) -> int:
        n = len(nums)
        f = [[1] * 1001 for _ in range(n)]
        ans = 0
        for i in range(1, n):
            for k in range(i):
                j = nums[i] - nums[k] + 500
                f[i][j] = max(f[i][j], f[k][j] + 1)
                ans = max(ans, f[i][j])

        return ans

```

• SimplyLeet

```

# Python solution for Longest Arithmetic Subsequence
def longestArithSeqLength(nums):
    # n is the length of the input array
    n = len(nums)
    # dp will be a list of dictionaries.
    # dp[i] will store {diff: length} for subsequences ending at index i
    dp = [{}, for _ in range(n)]
    max_length = 0 # initialize max length

    # iterate over each pair (j, i) where j < i

    for i in range(n):
        for j in range(i):
            # current difference between nums[i] and nums[j]
            diff = nums[i] - nums[j]
            # get the length for this difference from dp[j], default is 1 (only
            # nums[j])
            dp[i][diff] = dp[j].get(diff, 1) + 1
            # update the maximum length found so far
            max_length = max(max_length, dp[i][diff])

    return max_length

# Example usage:
# print(longestArithSeqLength([3,6,9,12])) -> Output: 4

```


◆ Quick Review

Key Insights

- Use dynamic programming where $dp[i][diff]$ represents the length of the longest arithmetic subsequence ending at index i with difference $diff$.
- For every pair of indices (j, i) with $j \leq i$, compute the difference $diff = nums[i] - nums[j]$.
- If there exists a subsequence ending at j with the same difference, extend it; otherwise, start a new subsequence with length 2.
- Finally, keep track of the maximum subsequence length encountered.

Space & Time

Time Complexity: $O(n^2)$ where n is the number of elements in the array.

Space Complexity: $O(n^2)$ in the worst case due to storing differences for each index.

Solution

We iterate over the array with a nested loop. For each pair (j, i) , we calculate the difference $diff = nums[i] - nums[j]$ and update our dp

array: if there is an entry $dp[j][diff]$, then $dp[i][diff] = dp[j][diff] + 1$; otherwise, $dp[i][diff]$ is set to 2 (since any two numbers form an arithmetic sequence). The answer is the maximum value in our dp structure. We use a hash table (dictionary) for each index to store the counts of arithmetic subsequences by their common difference.

Longest Increasing Subsequence (LIS)

□ #1048 Longest String Chain

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Two Pointers · String ·
Dynamic Programming · Sorting
Lists: AlgoMaster 600

Problem

You are given an array of words where each word consists of lowercase English letters. wordA is a predecessor of wordB if and only if we can insert exactly one letter anywhere in wordA without changing the order of the other characters to make it equal to wordB.

- For example, "abc" is a predecessor of "abac", while "cba" is not a predecessor of "bcad".

A word chain is a sequence of words [word1, word2, ..., wordk] with $k \geq 1$, where word1 is a predecessor of word2, word2 is a predecessor of word3, and so on. A single word is trivially a word chain with $k == 1$.

Return the length of the longest possible word chain with words chosen from the given list of words.

Example 1:

Input: words = ["a","b","ba","bca","bda","bdca"]

Output: 4

Explanation: One of the longest word chains is ["a","ba","bda","bdca"].

Example 2:

Input: words = ["xbc","pcxbcf","xb","cxbc","pcxbc"]

Output: 5

Explanation: All the words can be put in a word chain ["xb", "xbc", "cxbc", "pcxbc", "pcxbcf"].

Example 3:

Input: words = ["abcd","dbqca"]

Output: 1

Explanation: The trivial word chain ["abcd"] is one of the longest word chains. ["abcd","dbqca"] is not a valid word chain because the ordering of the letters is changed.

Constraints:

- $1 \leq \text{words.length} \leq 1000$
- $1 \leq \text{words}[i].\text{length} \leq 16$
- $\text{words}[i]$ only consists of lowercase English letters.

Community Solutions (Python)

- WalkCC

```
class Solution:
    def longestStrChain(self, words: list[str]) -> int:
        wordsSet = set(words)

        @functools.lru_cache(None)
        def dp(s: str) -> int:
            """Returns the longest chain where s is the last word."""
            ans = 1
            for i in range(len(s)):
                pred = s[:i] + s[i + 1:]

                if pred in wordsSet:
                    ans = max(ans, dp(pred) + 1)
            return ans

        return max(dp(word) for word in words)
```

```
class Solution:
    def longestStrChain(self, words: list[str]) -> int:
        dp = {}

        for word in sorted(words, key=len):
            dp[word] = max(dp.get(word[:i] + word[i + 1:], 0) +
                          1 for i in range(len(word)))

        return max(dp.values())
```

• LeetDoocs

```
class Solution:
    def longestStrChain(self, words: List[str]) -> int:
        def check(w1, w2):
            if len(w2) - len(w1) != 1:
                return False
            i = j = cnt = 0
            while i < len(w1) and j < len(w2):
                if w1[i] != w2[j]:
                    cnt += 1
            else:
                return False
```

```

        i += 1
        j += 1
        return cnt < 2 and i == len(w1)

n = len(words)
dp = [1] * (n + 1)
words.sort(key=lambda x: len(x))
res = 1
for i in range(1, n):
    for j in range(i):
        if check(words[j], words[i]):
            dp[i] = max(dp[i], dp[j] + 1)
    res = max(res, dp[i])
return res

```

```

class Solution:
    def longestStrChain(self, words: List[str]) -> int:
        words.sort(key=lambda x: len(x))
        res = 0
        mp = {}
        for word in words:
            x = 1
            for i in range(len(word)):
                pre = word[:i] + word[i + 1 :]
                x = max(x, mp.get(pre, 0) + 1)

            mp[word] = x
            res = max(res, x)
        return res

```

• SimplyLeet

```

# Function to compute the longest string chain using dynamic programming
def longestStrChain(words):
    # Sort words based on their length to ensure potential predecessors come
    # first
    words.sort(key=len)
    # Dictionary to store the longest chain length ending with the word
    dp = {}
    max_chain = 1 # Minimum chain length is 1

    # Process each word in sorted order
    for word in words:

        current_length = 1 # Each word by itself is a chain of length 1
        # Try removing one letter at a time to form a potential predecessor
        for i in range(len(word)):
            # Form the predecessor by skipping the i-th letter
            predecessor = word[:i] + word[i+1:]
            # If predecessor exists, update current chain length
            if predecessor in dp:
                current_length = max(current_length, dp[predecessor] + 1)
        # Save the computed chain length for the current word
        dp[word] = current_length

    # Update the overall maximum chain length
    max_chain = max(max_chain, current_length)

    return max_chain

# Example usage:
words = ["a", "b", "ba", "bca", "bda", "bdca"]
print(longestStrChain(words)) # Expected output: 4

```

◆ Quick Review

Key Insights

- Sort the words by length so that any potential predecessor appears before its longer successor.

- Use dynamic programming where for each word, you calculate the maximum chain ending with that word by checking all possible words formed by deleting one letter.
- A hash table (map/dictionary) allows you to quickly store and update the chain lengths for each word.
- The final answer is the maximum chain length found over all words.

Space & Time

Time Complexity: $O(N * L^2)$ where N is the number of words and L is the maximum word length.

Space Complexity: $O(N * L)$ due to storage in the hash table for chain lengths and intermediate strings.

Solution

The approach is to use dynamic programming. First, sort the words by their lengths so that any possible predecessor comes before the word itself. Then, initialize a hash table where the key is the word and the value is the longest chain ending at that word. For each word, generate all possible predecessors by

deleting one character at a time, and if the predecessor exists in our hash table, update the current word's chain length by taking the maximum chain value from its predecessors plus one. Finally, the answer is the maximum value stored in the hash table.

2D Grid DP

Round 8 · Low · Medium · 5 questions

2D Grid DP

□ #63 Unique Paths II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: AlgoMaster 600

Problem

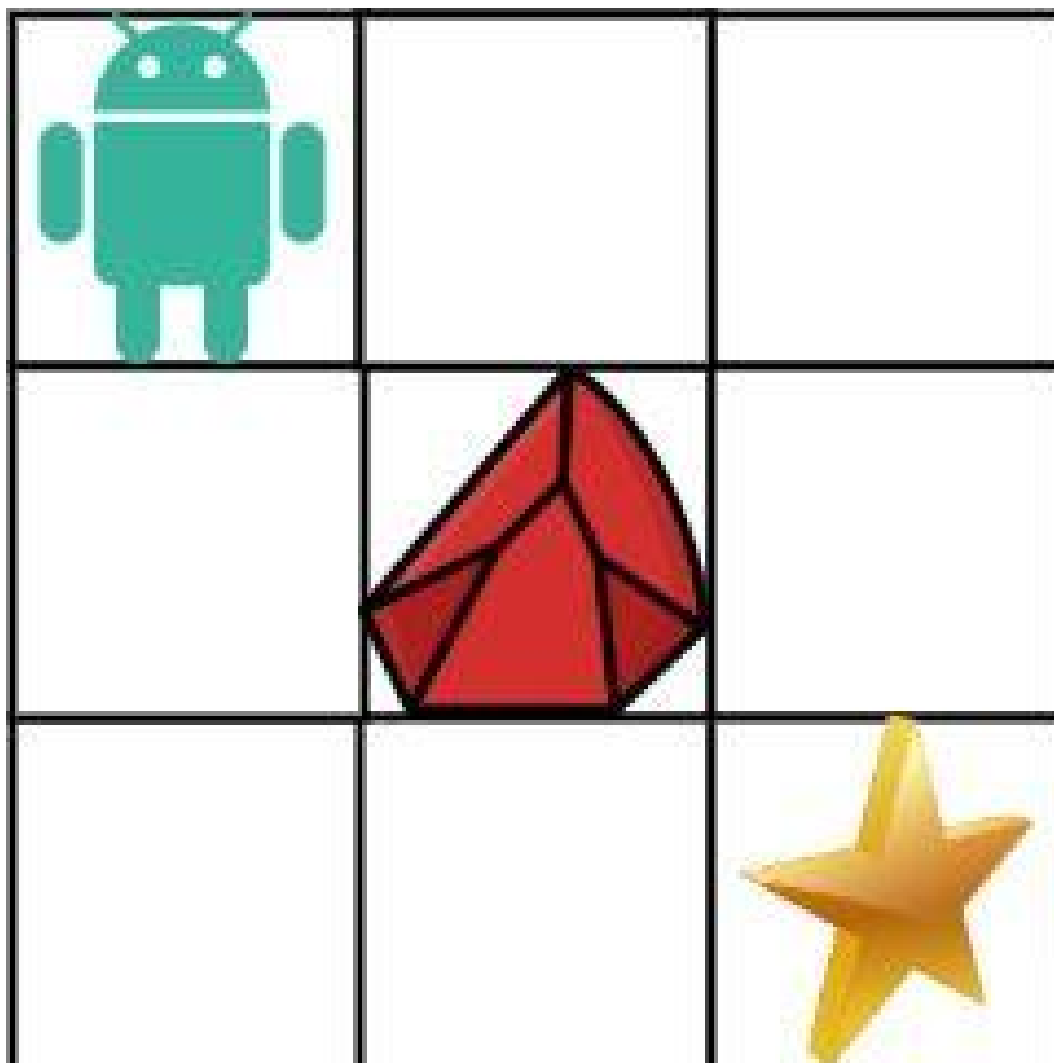
You are given an $m \times n$ integer array `grid`. There is a robot initially located at the top-left corner (i.e., `grid[0][0]`). The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

An obstacle and space are marked as 1 or 0 respectively in `grid`. A path that the robot takes cannot include any square that is an obstacle.

Return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The testcases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:



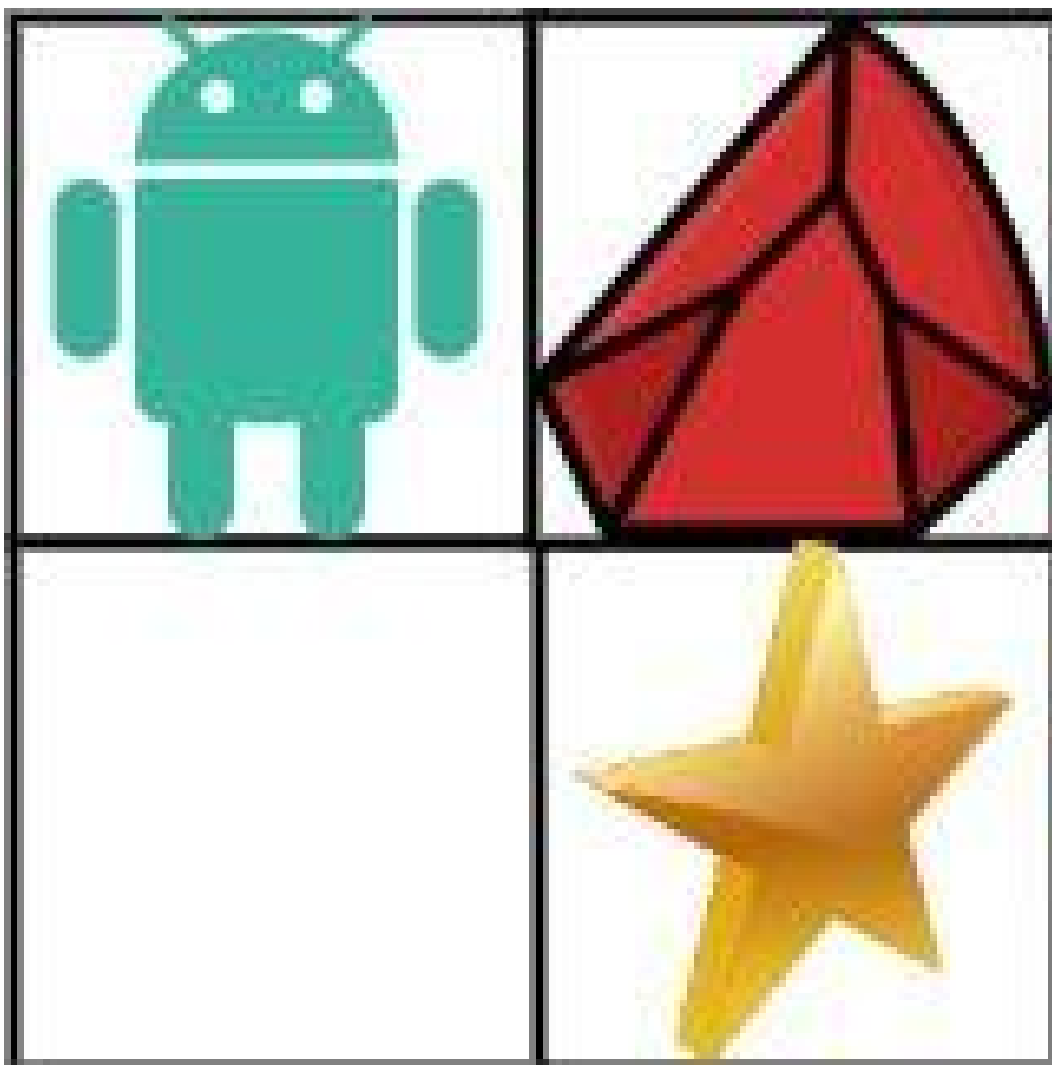
Input: obstacleGrid = `[[0,0,0],[0,1,0],[0,0,0]]`

Output: 2

Explanation: There is one obstacle in the middle of the 3x3 grid above. There are two ways to reach the bottom-right corner:

1. Right -> Right -> Down -> Down
2. Down -> Down -> Right -> Right

Example 2:



Input: `obstacleGrid = [[0,1],[0,0]]`

Output: 1

Constraints:

- `m == obstacleGrid.length`
- `n == obstacleGrid[i].length`
- `1 <= m, n <= 100`
- `obstacleGrid[i][j]` is 0 or 1.

Community Solutions (Python)

- WalkCC

```

class Solution:
    def uniquePathsWithObstacles(self, obstacleGrid: list[list[int]]) -> int:
        m = len(obstacleGrid)
        n = len(obstacleGrid[0])
        # dp[i][j] := the number of unique paths from (0, 0) to (i, j)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        dp[0][1] = 1 # Can also set dp[1][0] = 1.

        for i in range(1, m + 1):
            for j in range(1, n + 1):

                if obstacleGrid[i - 1][j - 1] == 0:
                    dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

        return dp[m][n]

```

• LeetDoocs

```

class Solution:
    def uniquePathsWithObstacles(self, obstacleGrid: List[List[int]]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            if i >= m or j >= n or obstacleGrid[i][j]:
                return 0
            if i == m - 1 and j == n - 1:
                return 1
            return dfs(i + 1, j) + dfs(i, j + 1)

        m, n = len(obstacleGrid), len(obstacleGrid[0])
        return dfs(0, 0)

```

• SimplyLeet

```

# Function to calculate unique paths with obstacles in a grid
def uniquePathsWithObstacles(obstacleGrid):
    # If the starting cell has an obstacle return 0 immediately
    if not obstacleGrid or obstacleGrid[0][0] == 1:
        return 0
    m, n = len(obstacleGrid), len(obstacleGrid[0])

    # Create a dp grid with same dimensions initialized to 0
    dp = [[0] * n for _ in range(m)]
    dp[0][0] = 1 # start position

    # Fill the first column considering obstacles
    for i in range(1, m):
        dp[i][0] = 0 if obstacleGrid[i][0] == 1 else dp[i-1][0]

    # Fill the first row considering obstacles
    for j in range(1, n):
        dp[0][j] = 0 if obstacleGrid[0][j] == 1 else dp[0][j-1]

    # Fill the rest dp table

    for i in range(1, m):
        for j in range(1, n):
            if obstacleGrid[i][j] == 1:
                dp[i][j] = 0 # obstacle cell
            else:
                dp[i][j] = dp[i-1][j] + dp[i][j-1] # paths from top and left
    return dp[m-1][n-1]

# Example usage:
grid = [[0,0,0],[0,1,0],[0,0,0]]

print(uniquePathsWithObstacles(grid)) # Output: 2

```

◆ Quick Review

Key Insights

- Use Dynamic Programming (DP) to build up the solution by calculating the number of paths to reach each cell.
- For any cell, if it is not an obstacle, the number of paths to reach it equals the sum of the paths from the cell directly above and the cell to the left.
- Cells containing obstacles are set to 0, meaning no path can go through them.
- Special handling is required for the first row and first column since the robot can only come from one direction for those cells.
- If the starting cell has an obstacle, the answer is immediately 0.

Space & Time

Time Complexity: $O(m * n)$ – Each cell is processed once.

Space Complexity: $O(m * n)$ – Using an auxiliary grid for DP. An optimization to $O(n)$ space is possible by reusing a single row.

Solution

We use a 2D dynamic programming approach where a dp array is maintained with the same dimensions as the grid. Initially, $dp[0][0]$ is set

to 1 if the starting cell is free, otherwise 0. The first row and first column are filled based on the absence of obstacles until a blockage is met. Then, for every other cell, if it is free, the value in $dp[i][j]$ is computed as the sum of $dp[i-1][j]$ (from above) and $dp[i][j-1]$ (from left). If the cell is an obstacle, the value is 0. This continues until the bottom-right cell is computed.

2D Grid DP

□ #120 Triangle

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: AlgoMaster 600

Problem

Given a triangle array, return the minimum path sum from top to bottom.

For each step, you may move to an adjacent number of the row below. More formally, if you are on index i on the current row, you may move to either index i or index $i + 1$ on the next row.

Example 1:

```
Input: triangle = [[2],[3,4],[6,5,7],[4,1,8,3]]
```

```
Output: 11
```

```
Explanation: The triangle looks like:
```

```
2
```

```
3 4
```

```
6 5 7
```

```
4 1 8 3
```

```
The minimum path sum from top to bottom is 2 + 3 + 5 + 1 = 11 (underlined above).
```

Example 2:

```
Input: triangle = [[-10]]
```

```
Output: -10
```

Constraints:

- $1 \leq \text{triangle.length} \leq 200$
- $\text{triangle}[0].\text{length} == 1$
- $\text{triangle}[i].\text{length} == \text{triangle}[i - 1].\text{length} + 1$
- $-104 \leq \text{triangle}[i][j] \leq 104$

Follow up: Could you do this using only $O(n)$ extra space, where n is the total number of rows in the triangle?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minimumTotal(self, triangle: list[list[int]]) -> int:
        for i in range(len(triangle) - 2, -1, -1):
            for j in range(i + 1):
                triangle[i][j] += min(triangle[i + 1][j],
                                      triangle[i + 1][j + 1])

        return triangle[0][0]
```

• LeetDoocs

```
class Solution:
    def minimumTotal(self, triangle: List[List[int]]) -> int:
        n = len(triangle)
        f = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 1):
                f[i][j] = min(f[i + 1][j], f[i + 1][j + 1]) + triangle[i][j]
        return f[0][0]
```

• SimplyLeet

```
def minimumTotal(triangle):
    # If the triangle is empty, return 0
    if not triangle:
        return 0
    # Start with a copy of the last row which acts as our dp array
    dp = triangle[-1][:]
    # Process the triangle from the second last row up to the top row
    for row in range(len(triangle) - 2, -1, -1):
        for col in range(len(triangle[row])):
            # Update dp array by adding the current triangle element
            # to the minimum of its two children in the dp array
            dp[col] = triangle[row][col] + min(dp[col], dp[col + 1])
    return dp[0]

# Example usage:
triangle1 = [[2], [3, 4], [6, 5, 7], [4, 1, 8, 3]]
print(minimumTotal(triangle1)) # Output: 11
```

◆ Quick Review

Key Insights

- Use dynamic programming to reduce overlapping subproblems.
- A bottom-up approach is most efficient by calculating the minimum path sums from the bottom row upward.
- This method reduces extra space usage to $O(n)$ where n is the number of rows.
- The solution iteratively updates a dp array that eventually holds the minimum path sum at its first index.

Space & Time

Time Complexity: $O(n^2)$ where n is the number of rows (each element is processed once).

Space Complexity: $O(n)$ extra space is used for the dp array storing one row of results.

Solution

A bottom-up dynamic programming approach is used to handle this problem. First, initialize a dp array as a copy of the last row of the triangle. Then, starting from the second-to-last row, update each element in the dp array by adding the current triangle element to the minimum of the two adjacent numbers from the row below (already stored in dp). By the time you reach the top row, $dp[0]$ holds the minimum total path sum. The dp array is updated in place to ensure an $O(n)$ space solution.

2D Grid DP

□ #931 Minimum Falling Path Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: AlgoMaster 600

Problem

Given an $n \times n$ array of integers matrix, return the minimum sum of any falling path through matrix.

A falling path starts at any element in the first row and chooses the element in the next row that is either directly below or diagonally left/right. Specifically, the next element from position (row, col) will be (row + 1, col - 1), (row + 1, col), or (row + 1, col + 1).

Example 1:

2	1	3
6	5	4
7	8	9

2	1	3
6	5	4
7	8	9

2	1	3
6	5	4
7	8	9

Input: matrix = [[2,1,3],[6,5,4],[7,8,9]]

Output: 13

Explanation: There are two falling paths with a minimum sum as shown.

Example 2:

-19	57
-40	-5

-19	57
-40	-5

Input: matrix = [[-19,57],[-40,-5]]

Output: -59

Explanation: The falling path with a minimum sum is shown.

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $1 \leq n \leq 100$
- $-100 \leq \text{matrix}[i][j] \leq 100$

Community Solutions (Python)

- WalkCC


```
class Solution:
    def minFallingPathSum(self, A: list[list[int]]) -> int:
        n = len(A)

        for i in range(1, n):
            for j in range(n):
                mn = math.inf
                for k in range(max(0, j - 1), min(n, j + 2)):
                    mn = min(mn, A[i - 1][k])
                A[i][j] += mn

        return min(A[-1])
```

• LeetCode

```
class Solution:
    def minFallingPathSum(self, matrix: List[List[int]]) -> int:
        n = len(matrix)
        f = [0] * n
        for row in matrix:
            g = [0] * n
            for j, x in enumerate(row):
                l, r = max(0, j - 1), min(n, j + 2)
                g[j] = min(f[l:r]) + x
            f = g

        return min(f)
```

• SimplyLeet

```
# Python solution with line-by-line comments

def minFallingPathSum(matrix):
    # Get the dimension of the matrix
    n = len(matrix)

    # Process the matrix from second last row to the first row
    for row in range(n - 2, -1, -1):
        for col in range(n):
            # Initialize the minimum sum from the next row as a large number.
```

```
best_below = matrix[row + 1][col]
# Check the diagonal left if it is within bounds
if col - 1 >= 0:
    best_below = min(best_below, matrix[row + 1][col - 1])
# Check the diagonal right if it is within bounds
if col + 1 < n:
    best_below = min(best_below, matrix[row + 1][col + 1])
# Update the current cell with the sum of its value and the best
move below
matrix[row][col] += best_below

# The result is the minimum value in the first row after processing
return min(matrix[0])

# Example usage:
matrix = [[2,1,3],[6,5,4],[7,8,9]]
print(minFallingPathSum(matrix)) # Expected output: 13
```

◆ Quick Review

Key Insights

- Use dynamic programming to calculate the minimum path sum from bottom to top.
- For each cell, only consider the three possible moves in the next row (down, down-left, down-right), making sure to handle boundary conditions.
- A bottom-up approach allows us to build the solution by using the already computed results of the subproblems.

- The final result is the minimum value in the computed first row.

Space & Time

Time Complexity: $O(n^2)$ because each of the $n \times n$ elements is processed once.

Space Complexity: $O(n^2)$ using an auxiliary DP matrix, though the solution can be optimized to $O(1)$ extra space by updating the matrix in-place.

Solution

We solve this problem using dynamic programming with a bottom-up approach. We initialize a DP structure (or update the matrix in-place) that stores the minimum falling path sum from each cell to the bottom row. Starting from the second last row, we iterate upward. For each cell (i, j) , we add the current cell value with the minimum of the allowed moves from the row below (directly below, diagonally left, and diagonally right). Consider boundary conditions for the first and last columns. The answer is the minimum value found in the top row after processing.

2D Grid DP

□ #1277 Count Square Submatrices with All Ones

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: AlgoMaster 600

Problem

Given a $m * n$ matrix of ones and zeros, return how many square submatrices have all ones.

Example 1:

```
Input: matrix =  
[  
  [0,1,1,1],  
  [1,1,1,1],  
  [0,1,1,1]  
]  
Output: 15  
Explanation:
```

Example 2:

```
Input: matrix =  
[  
  [1,0,1],  
  [1,1,0],  
  [1,1,0]  
]  
Output: 7  
Explanation:
```

Constraints:

- $1 \leq \text{arr.length} \leq 300$

- $1 \leq \text{arr}[0].\text{length} \leq 300$
- $0 \leq \text{arr}[i][j] \leq 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countSquares(self, matrix: list[list[int]]) -> int:
        for i in range(len(matrix)):
            for j in range(len(matrix[0])):
                if matrix[i][j] == 1 and i > 0 and j > 0:
                    matrix[i][j] += min(matrix[i - 1][j - 1],
                                         matrix[i - 1][j], matrix[i][j - 1])
        return sum(map(sum, matrix))
```

• LeetDoocs

```
class Solution:
    def countSquares(self, matrix: List[List[int]]) -> int:
        m, n = len(matrix), len(matrix[0])
        f = [[0] * n for _ in range(m)]
        ans = 0
        for i, row in enumerate(matrix):
            for j, v in enumerate(row):
                if v == 0:
                    continue
                if i == 0 or j == 0:
                    f[i][j] = 1
                else:
                    f[i][j] = min(f[i - 1][j - 1], f[i - 1][j], f[i][j - 1]) + 1
                ans += f[i][j]
        return ans
```

• SimplyLeet

```
# Function to count square submatrices with all ones
def countSquares(matrix):
    # Get dimensions of matrix
    rows = len(matrix)
    cols = len(matrix[0])
    total_squares = 0

    # Iterate over each cell in the matrix
    for i in range(rows):
        for j in range(cols):

            # If the cell is not in the first row or first column and is a 1
            if matrix[i][j] == 1 and i > 0 and j > 0:
                # Determine the size of the largest square ending at (i, j)
                matrix[i][j] = min(matrix[i-1][j], matrix[i][j-1], matrix[i-1]
[j-1]) + 1

            # Add current value to the total count of squares
            total_squares += matrix[i][j]
    return total_squares

# Example usage
example_matrix = [

    [0, 1, 1, 1],
    [1, 1, 1, 1],
    [0, 1, 1, 1]
]
print("Total squares:", countSquares(example_matrix))
```

◆ Quick Review

Key Insights

- Use dynamic programming to solve the problem.
- Create a dp table where each $dp[i][j]$ represents the size of the largest square that ends at cell (i, j) .

- For a cell containing one, the potential square size is determined by taking the minimum of its top, left, and top-left neighbors, then adding one.
- Accumulate the value of $dp[i][j]$ to get the total count of square submatrices with all ones.
- Handling boundary conditions where the cell is in the first row or column is essential.

Space & Time

Time Complexity: $O(m * n)$

Space Complexity: $O(m * n)$ (this can be optimized to $O(n)$ if modifying the original matrix is allowed)

Solution

We solve the problem using dynamic programming. For each cell in the matrix that contains a one, we determine the size of the maximal square submatrix ending at that cell by looking at its top, left, and top-left neighbors in the dp table. The cell's dp value will be one plus the minimum of these three values. The final answer is the sum of all values in the dp table, which represent the

count of squares ending at each position.

2D Grid DP

□ #1937 Maximum Number of Points with Cost

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: AlgoMaster 600

Problem

You are given an $m \times n$ integer matrix `points` (0-indexed). Starting with 0 points, you want to maximize the number of points you can get from the matrix.

To gain points, you must pick one cell in each row. Picking the cell at coordinates (r, c) will add `points[r][c]` to your score.

However, you will lose points if you pick a cell too far from the cell that you picked in the previous row. For every two adjacent rows r and $r + 1$ (where $0 \leq r < m - 1$), picking cells at coordinates $(r, c1)$ and $(r + 1, c2)$ will subtract $\text{abs}(c1 - c2)$ from your score.

Return the maximum number of points you can achieve.

abs(x) is defined as:

- x for $x \geq 0$.
- $-x$ for $x < 0$.

Example 1:

1	2	3
1	5	1
3	1	1

Input: `points = [[1,2,3],[1,5,1],[3,1,1]]`

Output: 9

Explanation:

The blue cells denote the optimal cells to pick, which have coordinates (0, 2), (1, 1), and (2, 0).

You add $3 + 5 + 3 = 11$ to your score.

However, you must subtract $\text{abs}(2 - 1) + \text{abs}(1 - 0) = 2$ from your score.

Your final score is $11 - 2 = 9$.

Example 2:

1	5
2	3
4	2

Input: `points = [[1,5],[2,3],[4,2]]`

Output: 11

Explanation:

The blue cells denote the optimal cells to pick, which have coordinates (0, 1), (1, 1), and (2, 0).

You add $5 + 3 + 4 = 12$ to your score.

However, you must subtract $\text{abs}(1 - 1) + \text{abs}(1 - 0) = 1$ from your score.

Your final score is $12 - 1 = 11$.

Constraints:

- `m == points.length`
- `n == points[r].length`

- $1 \leq m, n \leq 105$
- $1 \leq m * n \leq 105$
- $0 \leq \text{points}[r][c] \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxPoints(self, points: list[list[int]]) -> int:
        n = len(points[0])
        # dp[j] := the maximum number of points you can have if points[i][j] is the
        # most recent cell you picked
        dp = [0] * n

        for row in points:
            leftToRight = [0] * n
            runningMax = 0

            for j in range(n):
                runningMax = max(runningMax - 1, dp[j])
                leftToRight[j] = runningMax

            rightToLeft = [0] * n
            runningMax = 0
            for j in range(n - 1, -1, -1):
                runningMax = max(runningMax - 1, dp[j])
                rightToLeft[j] = runningMax

            for j in range(n):
                dp[j] = max(leftToRight[j], rightToLeft[j]) + row[j]

        return max(dp)
```

• LeetDoocs

```

class Solution:
    def maxPoints(self, points: List[List[int]]) -> int:
        n = len(points[0])
        f = points[0][:]
        for p in points[1:]:
            g = [0] * n
            lmx = -inf
            for j in range(n):
                lmx = max(lmx, f[j] + j)
                g[j] = max(g[j], p[j] + lmx - j)

            rmx = -inf
            for j in range(n - 1, -1, -1):
                rmx = max(rmx, f[j] - j)
                g[j] = max(g[j], p[j] + rmx + j)
            f = g
        return max(f)

```

• SimplyLeet

```

# Function to calculate the maximum number of points
def maxPoints(points):
    m = len(points)
    n = len(points[0])
    # dp will track the maximum score ending in each column for the previous
    row
    dp = points[0][:] # Copy the first row as initial state

    # Process each row from the second row onward
    for i in range(1, m):
        # Create helper arrays to capture best previous scores adjusting for
        movement costs

        left = [0] * n
        right = [0] * n

        # Build the left helper array from left to right
        left[0] = dp[0]
        for j in range(1, n):
            left[j] = max(left[j - 1] - 1, dp[j])

        # Build the right helper array from right to left
        right[n - 1] = dp[n - 1]

```

```

        for j in range(n - 2, -1, -1):
            right[j] = max(right[j + 1] - 1, dp[j])

        # Update dp for the current row using the best scores from left and
        # right arrays
        new_dp = [0] * n
        for j in range(n):
            new_dp[j] = points[i][j] + max(left[j], right[j])
        dp = new_dp  # Move to the next row

    # The answer is the maximum value in the final dp row

    return max(dp)

# Example usage:
if __name__ == '__main__':
    points_example = [[1,2,3],[1,5,1],[3,1,1]]
    print(maxPoints(points_example))  # Expected output: 9

```

◆ Quick Review

Key Insights

- Use dynamic programming to store the best achievable score ending at each column.
- Transition between rows must incorporate the cost of changing columns, given by $\text{abs}(c1 - c2)$.
- Precompute two helper arrays (left and right) for each row to capture the maximum score obtainable from left-to-right and right-to-left, effectively handling the absolute difference cost in $O(n)$ time per row.

- This optimized transition allows for solving the problem efficiently without using a nested loop for cost computation.

Space & Time

Time Complexity: $O(m * n)$, where m is the number of rows and n is the number of columns.

Space Complexity: $O(n)$, since only one-dimensional arrays (dp , $left$, $right$) are maintained for computations.

Solution

The solution involves dynamic programming by tracking the maximum score achievable at each cell of the current row. For each row beyond the first, two helper arrays, $left$ and $right$, are computed:

- The $left$ array is built by iterating from left to right, adjusting the score by subtracting 1 for each move to the right.
- The $right$ array is built by iterating from right to left, similarly subtracting 1 for each move to the left. Then, for every cell in the new row, the best possible previous score is added from either the $left$ or $right$ helper array along

with the current cell's points. This trick avoids the need for an inner loop to compute the absolute difference cost for every pair of columns, thereby optimizing the transition.

String DP

Round 8 · Low · Medium · 1 question

String DP

□ #1653 Minimum Deletions to Make String Balanced

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Stack
Lists: AlgoMaster 600

Problem

You are given a string s consisting only of characters 'a' and 'b'.

You can delete any number of characters in s to make s balanced. s is balanced if there is no pair of indices (i, j) such that $i < j$ and $s[i] = 'b'$ and $s[j] = 'a'$.

Return the minimum number of deletions needed to make s balanced.

Example 1:

Input: $s = \text{"aababbab"}$

Output: 2

Explanation: You can either:

Delete the characters at 0-indexed positions 2 and 6 ($\text{"aababbab"} \rightarrow \text{"aaabbb"}$),
or

Delete the characters at 0-indexed positions 3 and 6 ($\text{"aababbab"} \rightarrow \text{"aabbbb"}$).

Example 2:

Input: s = "bbaaaaabb"

Output: 2

Explanation: The only solution is to delete the first two characters.

Constraints:

- $1 \leq s.length \leq 105$
- s[i] is 'a' or 'b' .

Community Solutions (Python)

• WalkCC

```
class Solution:
    # Same as 926. Flip String to Monotone Increasing
    def minimumDeletions(self, s: str) -> int:
        dp = 0 # the number of characters to be deleted to make subso far balanced
        countB = 0

        for c in s:
            if c == 'a':
                # 1. Delete 'a'.
                # 2. Keep 'a' and delete the previous 'b's.

                dp = min(dp + 1, countB)
            else:
                countB += 1

        return dp
```

• LeetDoocs

```

class Solution:
    def minimumDeletions(self, s: str) -> int:
        n = len(s)
        f = [0] * (n + 1)
        b = 0
        for i, c in enumerate(s, 1):
            if c == 'b':
                f[i] = f[i - 1]
                b += 1
            else:
                f[i] = min(f[i - 1] + 1, b)
        return f[n]

```

```

class Solution:
    def minimumDeletions(self, s: str) -> int:
        lb, ra = 0, s.count('a')
        ans = len(s)
        for c in s:
            ra -= c == 'a'
            ans = min(ans, lb + ra)
            lb += c == 'b'
        return ans

```

• SimplyLeet

```

# Function to compute minimum deletions needed to make a string balanced
def minimum_deletions(s):
    deletion_count = 0 # Tracks the minimum number of deletions required so far
    count_b = 0 # Counts the number of 'b' characters encountered
    # Iterate over each character in the string
    for char in s:
        if char == 'b':
            count_b += 1 # Increment count for each 'b'
        else: # char is 'a'
            # Either delete the current 'a' (deletion_count + 1) or remove all previous 'b's (count_b)

```

```
        deletion_count = min(deletion_count + 1, count_b)
    return deletion_count

# Example usage
s = "aababbab"
print(minimum_deletions(s)) # Expected output: 2
```

◆ Quick Review

Key Insights

- A balanced string means every 'a' must come before every 'b'.
- When scanning the string, conflicts occur when an 'a' appears after a 'b'.
- At each 'a', we face a choice: delete the current 'a' or delete all preceding 'b's.
- By maintaining a counter for 'b's and tracking the minimum deletions required so far, we can decide optimally at each step.
- This approach leverages a greedy/dynamic programming method with a single pass.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string.

Space Complexity: $O(1)$ since only a few variables are used.

Solution

The solution iterates through the string while keeping track of two variables: `countB` (the number of 'b's encountered so far) and `deletionCount` (the minimum deletions required up to the current point). For each character:

- If the character is 'b', increment `countB`.
- If the character is 'a', update `deletionCount` to be the minimum of either deleting this 'a' ($\text{deletionCount} + 1$) or deleting all previous 'b's (which is `countB`). This strategy ensures that the final `deletionCount` reflects the minimum number of deletions needed to achieve a balanced string.

Tree / Graph DP

Round 8 · Low · Medium · 3 questions

Tree / Graph DP

□ #95 Unique Binary Search Trees II

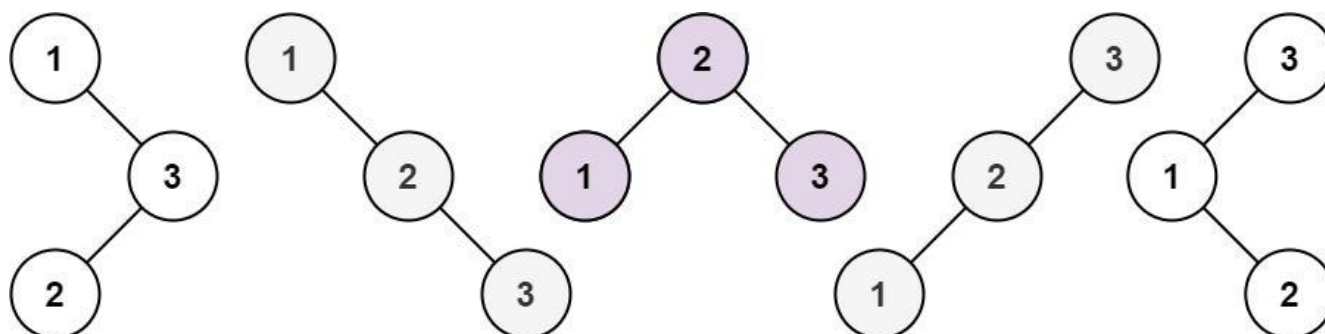
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Backtracking · Tree ·
Binary Search Tree · Binary Tree
Lists: AlgoMaster 600

Problem

Given an integer n , return all the structurally unique BST's (binary search trees), which has exactly n nodes of unique values from 1 to n . Return the answer in any order.

Example 1:



Input: $n = 3$

Output:

`[[1,null,2,null,3],[1,null,3,2],[2,1,3],[3,1,null,null,2],[3,2,null,1]]`

Example 2:

Input: $n = 1$
Output: $[[1]]$

Constraints:

- $1 \leq n \leq 8$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def generateTrees(self, n: int) -> list[TreeNode]:
        if n == 0:
            return []

        def generateTrees(mn: int, mx: int) -> list[int | None]:
            if mn > mx:
                return [None]

            ans = []

            for i in range(mn, mx + 1):
                for left in generateTrees(mn, i - 1):
                    for right in generateTrees(i + 1, mx):
                        ans.append(TreeNode(i))
                        ans[-1].left = left
                        ans[-1].right = right

            return ans

        return generateTrees(1, n)
```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def generateTrees(self, n: int) -> List[Optional[TreeNode]]:
        def dfs(i: int, j: int) -> List[Optional[TreeNode]]:
            if i > j:
                return [None]
            ans = []
            for v in range(i, j + 1):
                left = dfs(i, v - 1)
                right = dfs(v + 1, j)
                for l in left:
                    for r in right:
                        ans.append(TreeNode(v, l, r))
            return ans

        return dfs(1, n)

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val      # Initialize the node's value
        self.left = left     # Reference to the left subtree
        self.right = right   # Reference to the right subtree

class Solution:
    def generateTrees(self, n: int):
        # Edge-case: if no numbers, return empty list

```

```
if n == 0:
    return []

# Recursive helper function to generate BSTs for a given range
def generate_trees(start, end):
    trees = []
    # Base case: represent an empty tree
    if start > end:
        trees.append(None)
        return trees

    # Loop through each number as a potential root
    for root_val in range(start, end + 1):
        # All possible left subtrees if root_val is chosen
        left_trees = generate_trees(start, root_val - 1)
        # All possible right subtrees if root_val is chosen
        right_trees = generate_trees(root_val + 1, end)
        # Combine each left and right subtree with the root
        for left in left_trees:
            for right in right_trees:
                root = TreeNode(root_val)

                root.left = left
                root.right = right
                trees.append(root)

    return trees

return generate_trees(1, n)

# Example usage:
# sol = Solution()
# trees = sol.generateTrees(3)

# This will generate all unique BSTs for n = 3.
```

◆ Quick Review

Key Insights

- Utilize a recursive (divide and conquer) approach where each number in the range can be chosen as the root.
- For a chosen root, all numbers less than the root become part of the left subtree, and all numbers greater become part of the right subtree.
- Recursively generate all valid left and right subtrees then combine them to form unique BSTs.
- Base case: When the starting index exceeds the ending index, return a list containing None (or null in other languages) to represent an empty tree.
- The problem essentially generates trees corresponding to Catalan numbers.

Space & Time

Time Complexity: $O(n * \text{Catalan}(n))$

Space Complexity: $O(n * \text{Catalan}(n))$ (space used to store all generated trees and recursion stack)

Solution

We use a recursive function that accepts a range (start, end) and returns all possible BSTs

constructed from numbers within that range. For every possible root value within the range, we recursively generate all possible left subtrees (from start to value-1) and right subtrees (from value+1 to end). We then iterate over every combination of left and right subtree and create a new tree node with the selected root value. This tree is appended to our list of unique BSTs. Key details include handling the base case where the start index exceeds the end index (which represents an empty subtree) and combining results appropriately.

Tree / Graph DP

□ #337 House Robber III

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Tree · Depth-First
Search · Binary Tree
Lists: AlgoMaster 600

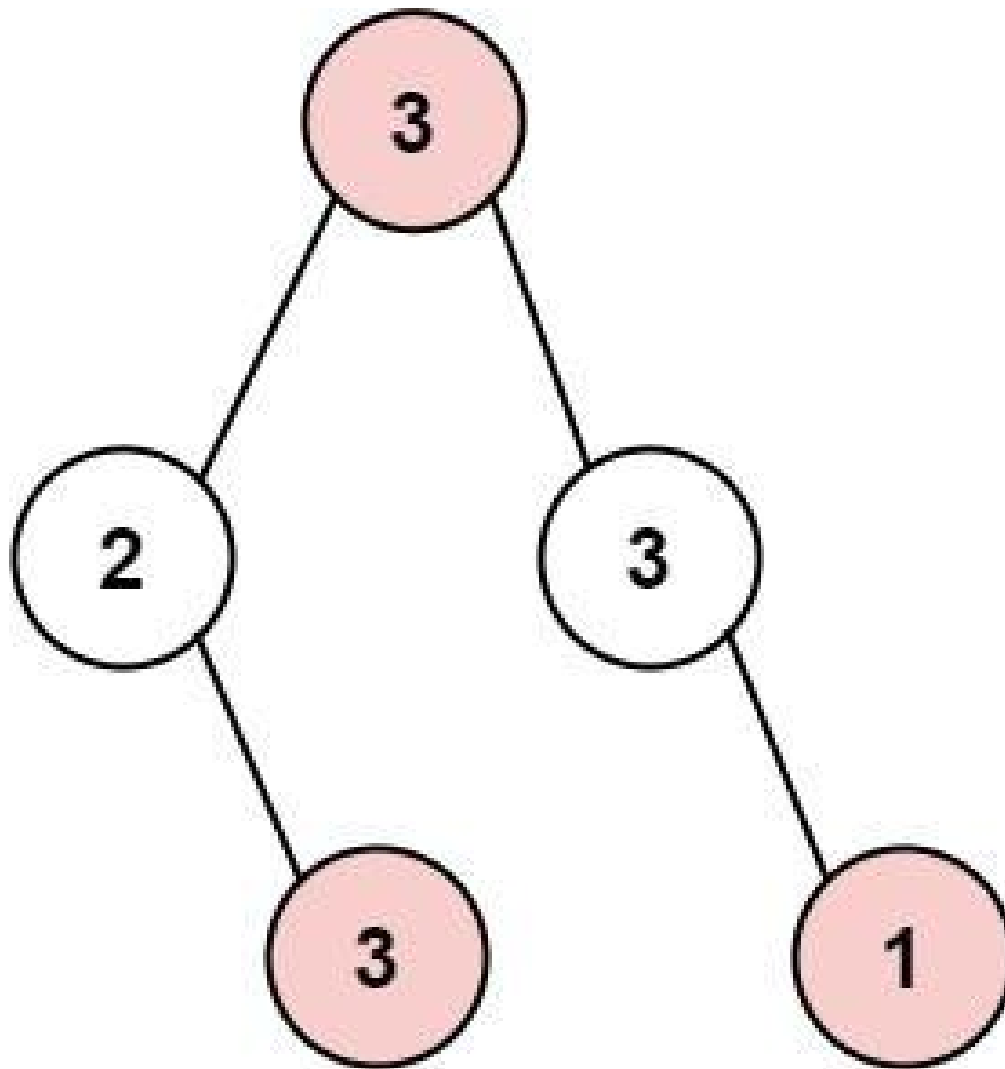
Problem

The thief has found himself a new place for his thievery again. There is only one entrance to this area, called root.

Besides the root, each house has one and only one parent house. After a tour, the smart thief realized that all houses in this place form a binary tree. It will automatically contact the police if two directly-linked houses were broken into on the same night.

Given the root of the binary tree, return the maximum amount of money the thief can rob without alerting the police.

Example 1:

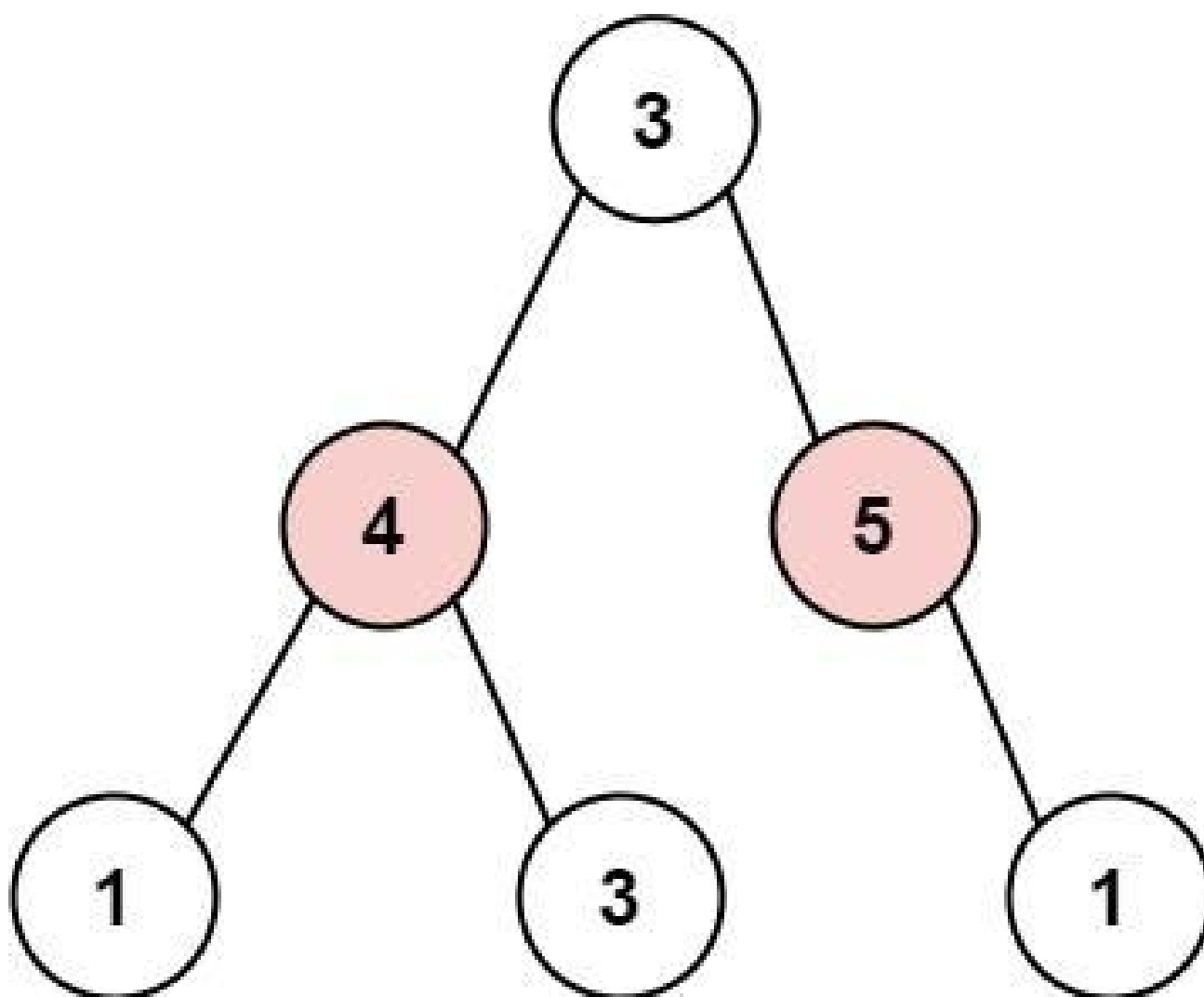


Input: root = [3,2,3,null,3,null,1]

Output: 7

Explanation: Maximum amount of money the thief can rob = 3 + 3 + 1 = 7.

Example 2:



Input: root = [3,4,5,1,3,null,1]

Output: 9

Explanation: Maximum amount of money the thief can rob = 4 + 5 = 9.

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $0 \leq \text{Node.val} \leq 104$

Community Solutions (Python)

- WalkCC


```

class Solution:
    def rob(self, root: TreeNode | None) -> int:
        def robOrNot(root: TreeNode | None) -> tuple:
            if not root:
                return (0, 0)

            robLeft, notRobLeft = robOrNot(root.left)
            robRight, notRobRight = robOrNot(root.right)

            return (root.val + notRobLeft + notRobRight,

                    max(robLeft, notRobLeft) + max(robRight, notRobRight))

        return max(robOrNot(root))

```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rob(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> (int, int):
            if root is None:
                return 0, 0
            la, lb = dfs(root.left)
            ra, rb = dfs(root.right)
            return root.val + lb + rb, max(la, lb) + max(ra, rb)

        return max(dfs(root))

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val      # value of the node (money in the house)
        self.left = left    # left child
        self.right = right   # right child

def rob(root: TreeNode) -> int:
    # Returns a tuple (rob, notRob) representing:
    # rob -> maximum money when this node is robbed

    # notRob -> maximum money when this node is not robbed
    def dfs(node):
        if not node:
            return (0, 0)
        left = dfs(node.left)
        right = dfs(node.right)
        # If we rob this node, we cannot rob its children
        rob_current = node.val + left[1] + right[1]
        # If we do not rob this node, choose the max for each child
        # independently
        not_rob_current = max(left) + max(right)

        return (rob_current, not_rob_current)

    return max(dfs(root))

# Example usage:
# root = TreeNode(3, TreeNode(2, None, TreeNode(3)), TreeNode(3, None,
# TreeNode(1)))
# print(rob(root)) # Output should be 7

```

◆ Quick Review

Key Insights

- The problem is modeled on a binary tree where each node represents a house.

- If a node is robbed, its immediate children cannot be robbed.
- Using DFS (Depth First Search) allows us to explore the tree and decide for each node whether to rob it or not.
- At each node, compute two values: one when the node is robbed and one when it is not.
- The optimal solution at each node is derived by considering the maximum money collected from its subtrees, taking the constraints into account.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes, as each node is visited once.

Space Complexity: $O(n)$ due to the recursion call stack in the worst-case skewed tree.

Solution

We use a DFS-based recursive strategy to solve the problem. For each node, compute two scenarios:

- Robbing the current node: In this case, we cannot rob its children. So, the value is `node.val` plus the sum of values from not robbing its left and right children.

– **Not robbing the current node:** In this case, we can rob or not rob its children based on which option yields more money from each subtree. By returning these two values (rob, notRob) for each node and combining them from the bottom-up, the solution picks the optimal strategy at the root. Key data structures include recursion (function call stack) and tuples/pairs to maintain the two states per node.

Tree / Graph DP

□ #1976 Number of Ways to Arrive at Destination

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Graph Theory ·
Topological Sort · Shortest Path
Lists: AlgoMaster 600

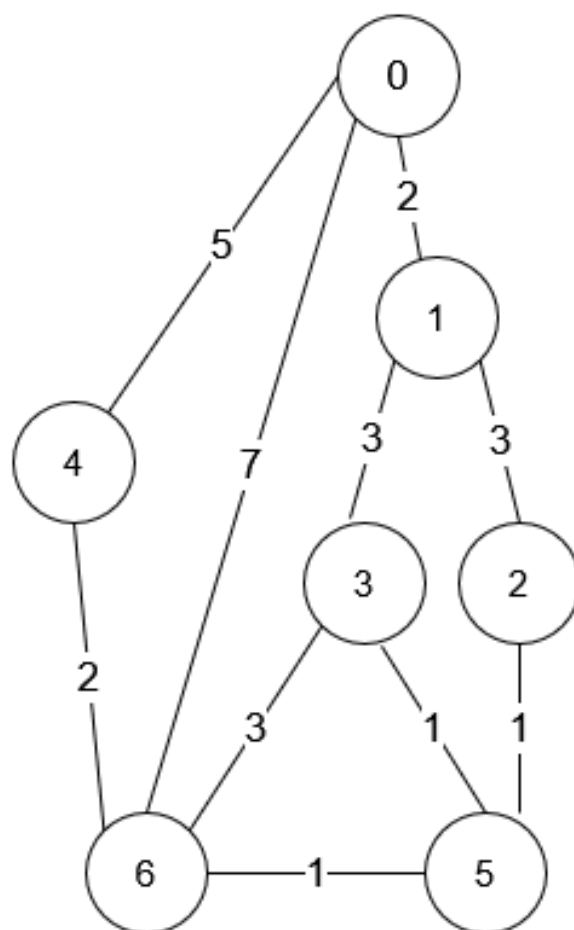
Problem

You are in a city that consists of n intersections numbered from 0 to $n - 1$ with bi-directional roads between some intersections. The inputs are generated such that you can reach any intersection from any other intersection and that there is at most one road between any two intersections.

You are given an integer n and a 2D integer array `roads` where `roads[i] = [ui, vi, timei]` means that there is a road between intersections ui and vi that takes $timei$ minutes to travel. You want to know in how many ways you can travel from intersection 0 to intersection $n - 1$ in the shortest amount of time.

Return the number of ways you can arrive at your destination in the shortest amount of time. Since the answer may be large, return it modulo $10^9 + 7$.

Example 1:



Input: $n = 7$, $roads = [[0,6,7],[0,1,2],[1,2,3],[1,3,3],[6,3,3],[3,5,1],[6,5,1],[2,5,1],[0,4,5],[4,6,2]]$

Output: 4

Explanation: The shortest amount of time it takes to go from intersection 0 to intersection 6 is 7 minutes.

The four ways to get there in 7 minutes are:

- $0 \rightarrow 6$
- $0 \rightarrow 4 \rightarrow 6$
- $0 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 6$
- $0 \rightarrow 1 \rightarrow 3 \rightarrow 5 \rightarrow 6$

Example 2:

Input: $n = 2$, $roads = [[1,0,10]]$

Output: 1

Explanation: There is only one way to go from intersection 0 to intersection 1, and it takes 10 minutes.

Constraints:

- $1 \leq n \leq 200$
- $n - 1 \leq roads.length \leq n * (n - 1) / 2$
- $roads[i].length == 3$
- $0 \leq ui, vi \leq n - 1$
- $1 \leq time_i \leq 109$
- $ui \neq vi$
- There is at most one road connecting any two intersections.
- You can reach any intersection from any other intersection.

Community Solutions (Python)

- WalkCC

```

class Solution:
    def countPaths(self, n: int, roads: list[list[int]]) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in roads:
            graph[u].append((v, w))
            graph[v].append((u, w))

        return self._dijkstra(graph, 0, n - 1)

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
        dst: int,
    ) -> int:
        MOD = 10**9 + 7
        ways = [0] * len(graph)
        dist = [math.inf] * len(graph)

        ways[src] = 1
        dist[src] = 0
        minHeap = [(dist[src], src)]

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    ways[v] = ways[u]
                    heapq.heappush(minHeap, (dist[v], v))
                elif d + w == dist[v]:
                    ways[v] += ways[u]
                    ways[v] %= MOD

        return ways[dst]

```

• LeetDoocs


```

class Solution:
    def countPaths(self, n: int, roads: List[List[int]]) -> int:
        g = [[inf] * n for _ in range(n)]
        for u, v, t in roads:
            g[u][v] = g[v][u] = t
        g[0][0] = 0
        dist = [inf] * n
        dist[0] = 0
        f = [0] * n
        f[0] = 1

        vis = [False] * n
        for _ in range(n):
            t = -1
            for j in range(n):
                if not vis[j] and (t == -1 or dist[j] < dist[t]):
                    t = j
            vis[t] = True
            for j in range(n):
                if j == t:
                    continue

                ne = dist[t] + g[t][j]
                if dist[j] > ne:
                    dist[j] = ne
                    f[j] = f[t]
                elif dist[j] == ne:
                    f[j] += f[t]

        mod = 10**9 + 7
        return f[-1] % mod

```

• SimplyLeet

```

import heapq

def countPaths(n, roads):
    MOD = 10**9 + 7
    # Build the graph as an adjacency list
    graph = [[] for _ in range(n)]
    for u, v, time in roads:
        graph[u].append((v, time))
        graph[v].append((u, time))

```

```
# Initialize distance and ways arrays
dist = [float('inf')] * n
ways = [0] * n
dist[0] = 0
ways[0] = 1

# Min-heap: (distance, node)
heap = [(0, 0)]

# Modified Dijkstra's algorithm

while heap:
    current_dist, u = heapq.heappop(heap)
    if current_dist > dist[u]:
        continue
    for v, time in graph[u]:
        next_dist = current_dist + time
        if next_dist < dist[v]:
            dist[v] = next_dist
            ways[v] = ways[u]
            heapq.heappush(heap, (next_dist, v))

        elif next_dist == dist[v]:
            ways[v] = (ways[v] + ways[u]) % MOD
return ways[n - 1]
```

◆ Quick Review

Key Insights

- The problem requires computing the shortest distance from the source (0) to the destination (n-1) in a weighted graph.
- Dijkstra's algorithm is a natural choice due to non-negative edge weights.

- While computing the shortest distances, maintain a count of how many ways each node can be reached using the minimum time.
- Update the count when a new shorter path is found or when another path leading to the same shortest distance is discovered.
- Always perform modulus operations to handle large numbers.

Space & Time

Time Complexity: $O(E \log V)$, where E is the number of roads and V is the number of intersections.

Space Complexity: $O(V + E)$, which accounts for the distance array, ways array, and graph representation.

Solution

We modify Dijkstra's algorithm to not only compute the shortest path but also track the number of ways to reach each intersection using shortest possible travel times. We use:

- A graph represented as an adjacency list.
- A distance array (or vector) initialized with infinity for all nodes except the starting node (set to 0).

- A ways array to count the number of ways to achieve the recorded shortest distance.
- A priority queue (or min-heap) to efficiently fetch the next node with the smallest temporary distance. During the relaxation step for each neighbor, if a shorter path is found, update both the distance and the ways count; if an equally short path is found, add the number of new ways to the count modulo $10^9 + 7$.

Bitmask DP

Round 8 · Low · Medium · 4 questions

Bitmask DP

□ #698 Partition to K Equal Sum Subsets

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Backtracking ·
Bit Manipulation · Memoization · Bitmask
Lists: AlgoMaster 600

Problem

Given an integer array `nums` and an integer `k`, return `true` if it is possible to divide this array into `k` non-empty subsets whose sums are all equal.

Example 1:

```
Input: nums = [4,3,2,3,5,2,1], k = 4  
Output: true  
Explanation: It is possible to divide it into 4 subsets (5), (1, 4), (2,3),  
(2,3) with equal sums.
```

Example 2:

```
Input: nums = [1,2,3,4], k = 3  
Output: false
```

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 16$
- $1 \leq \text{nums}[i] \leq 104$

- The frequency of each element is in the range [1, 4].

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canPartitionKSubsets(self, nums: list[int], k: int) -> bool:
        summ = sum(nums)
        if summ % k != 0:
            return False

        target = summ // k # the target sum of each subset
        if any(num > target for num in nums):
            return False

        def dfs(s: int, remainingGroups: int, currSum: int, used: int) -> bool:
            if remainingGroups == 0:
                return True
            if currSum > target:
                return False
            if currSum == target: # Find a valid group, so fresh start.
                return dfs(0, remainingGroups - 1, 0, used)

            for i in range(s, len(nums)):
                if used >> i & 1:
                    continue
                if dfs(i + 1, remainingGroups, currSum + nums[i], used | 1 << i):
                    return True

            return False

        nums.sort(reverse=True)
        return dfs(0, k, 0, 0)
```

• LeetDoocs

```

class Solution:
    def canPartitionKSubsets(self, nums: List[int], k: int) -> bool:
        def dfs(i: int) -> bool:
            if i == len(nums):
                return True
            for j in range(k):
                if j and cur[j] == cur[j - 1]:
                    continue
                cur[j] += nums[i]
                if cur[j] <= s and dfs(i + 1):
                    return True
                cur[j] -= nums[i]
            return False

        s, mod = divmod(sum(nums), k)
        if mod:
            return False
        cur = [0] * k
        nums.sort(reverse=True)
        return dfs(0)

```

```

class Solution:
    def canPartitionKSubsets(self, nums: List[int], k: int) -> bool:
        @cache
        def dfs(state, t):
            if state == mask:
                return True
            for i, v in enumerate(nums):
                if (state >> i) & 1:
                    continue
                if t + v > s:
                    break
                if dfs(state | 1 << i, (t + v) % s):
                    return True
            return False

        s, mod = divmod(sum(nums), k)
        if mod:
            return False
        nums.sort()
        mask = (1 << len(nums)) - 1

```



```
return dfs(0, 0)
```

```
class Solution:
```

```
    def canPartitionKSubsets(self, nums: List[int], k: int) -> bool:
```

```
        s = sum(nums)
```

```
        if s % k:
```

```
            return False
```

```
        s //= k
```

```
        nums.sort()
```

```
        n = len(nums)
```

```
        f = [False] * (1 << n)
```

```
        cur = [0] * (1 << n)
```

```
        f[0] = True
```

```
        for i in range(1 << n):
```

```
            if not f[i]:
```

```
                continue
```

```
            for j in range(n):
```

```
                if cur[i] + nums[j] > s:
```

```
                    break
```

```
                if (i >> j & 1) == 0:
```

```
                    if not f[i | 1 << j]:
```

```
                        cur[i | 1 << j] = (cur[i] + nums[j]) % s
```

```
        f[i | 1 << j] = True
```

```
return f[-1]
```

• SimplyLeet

```
# Define the class with the solution method
```

```
class Solution:
```

```
    def canPartitionKSubsets(self, nums, k):
```

```
        # Calculate the total sum and check divisibility by k
```

```
        total_sum = sum(nums)
```

```
        if total_sum % k != 0:
```

```
            return False
```

```
        target = total_sum // k
```

```

    # Sort the numbers in descending order for optimization
    nums.sort(reverse=True)
    # If the largest number is greater than the target, partition is
    impossible
    if nums[0] > target:
        return False

    n = len(nums)
    used = [False] * n

    # Backtracking function: start_index is where to continue searching,

    # curr_sum is the current subset sum, count is how many subsets have
    been formed so far.
    def backtrack(start_index, curr_sum, count):
        # If we've correctly formed k-1 subsets, the rest of the numbers
        form the last subset.
        if count == k - 1:
            return True
        # When current subset sum equals target, start forming a new
        subset.
        if curr_sum == target:
            return backtrack(0, 0, count + 1)
        # Try to assign next numbers to current subset.
        for i in range(start_index, n):
            if not used[i] and curr_sum + nums[i] <= target:
                used[i] = True
                if backtrack(i + 1, curr_sum + nums[i], count):
                    return True
                used[i] = False
            # Skip duplicate elements to prevent redundant work.
            while i + 1 < n and nums[i] == nums[i + 1]:
                i += 1
        return False

    return backtrack(0, 0, 0)

# Example usage:
# sol = Solution()
# print(sol.canPartitionKSubsets([4,3,2,3,5,2,1], 4)) # Output: True

```

◆ Quick Review

Key Insights

- The total sum of the array must be divisible by k ; otherwise, it's impossible to partition it into subsets of equal sum.
- The target sum for each subset is the total sum divided by k .
- Sorting the array in descending order helps in pruning branches early in the backtracking process.
- A backtracking (DFS) approach, optionally enhanced with memoization or bitmasking, can be used to explore possible subset partitions.
- Handling duplicate numbers carefully reduces redundant work during recursion.

Space & Time

Time Complexity: $O(k * 2^n)$ in the worst-case scenario (where n is the number of elements) due to exploring subset combinations via backtracking.

Space Complexity: $O(n)$ for recursion stack and additional data structures (or $O(2^n)$ if using memoization with bitmasking).

Solution

We first check if the total sum of the array is divisible by k . Then determine the target sum for each subset. By sorting the array in descending order, we ensure that the larger numbers are placed first so that invalid configurations are pruned quickly. We use a backtracking (DFS) approach to try and assign numbers to each subset while ensuring the running sum does not exceed the target. When a subset reaches the target sum, we recursively try to fill the next subset. Care is taken to skip duplicates in order to avoid redundant recursion. This approach efficiently determines if a valid k -partition exists.

Bitmask DP

□ #1066 Campus Bikes II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Backtracking ·
Bit Manipulation · Bitmask
Lists: AlgoMaster 600

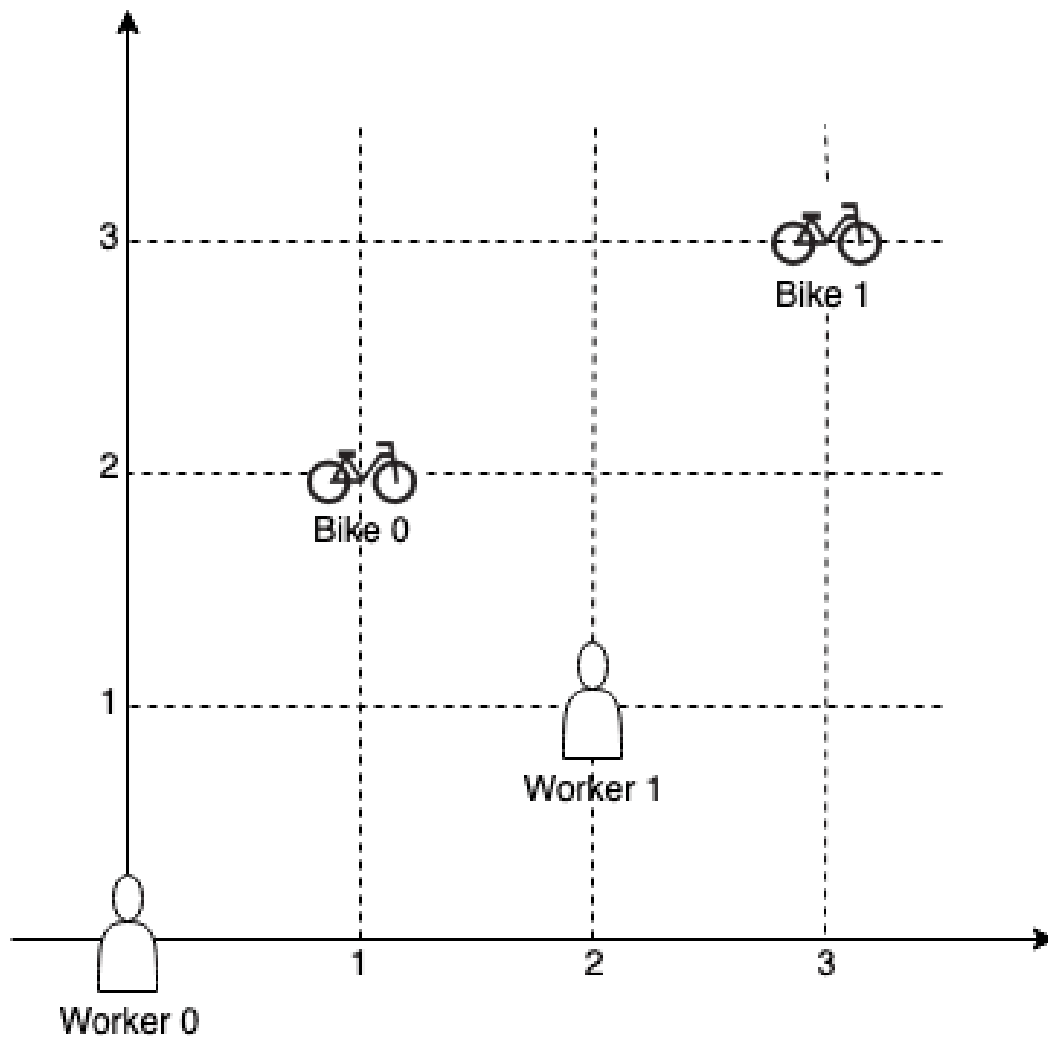
Problem

On a campus represented as a 2D grid, there are n workers and m bikes, with $n \leq m$. Each worker and bike is a 2D coordinate on this grid. We assign one unique bike to each worker so that the sum of the Manhattan distances between each worker and their assigned bike is minimized.

Return the minimum possible sum of Manhattan distances between each worker and their assigned bike.

The Manhattan distance between two points p_1 and p_2 is $\text{Manhattan}(p_1, p_2) = |p_1.x - p_2.x| + |p_1.y - p_2.y|$.

Example 1:



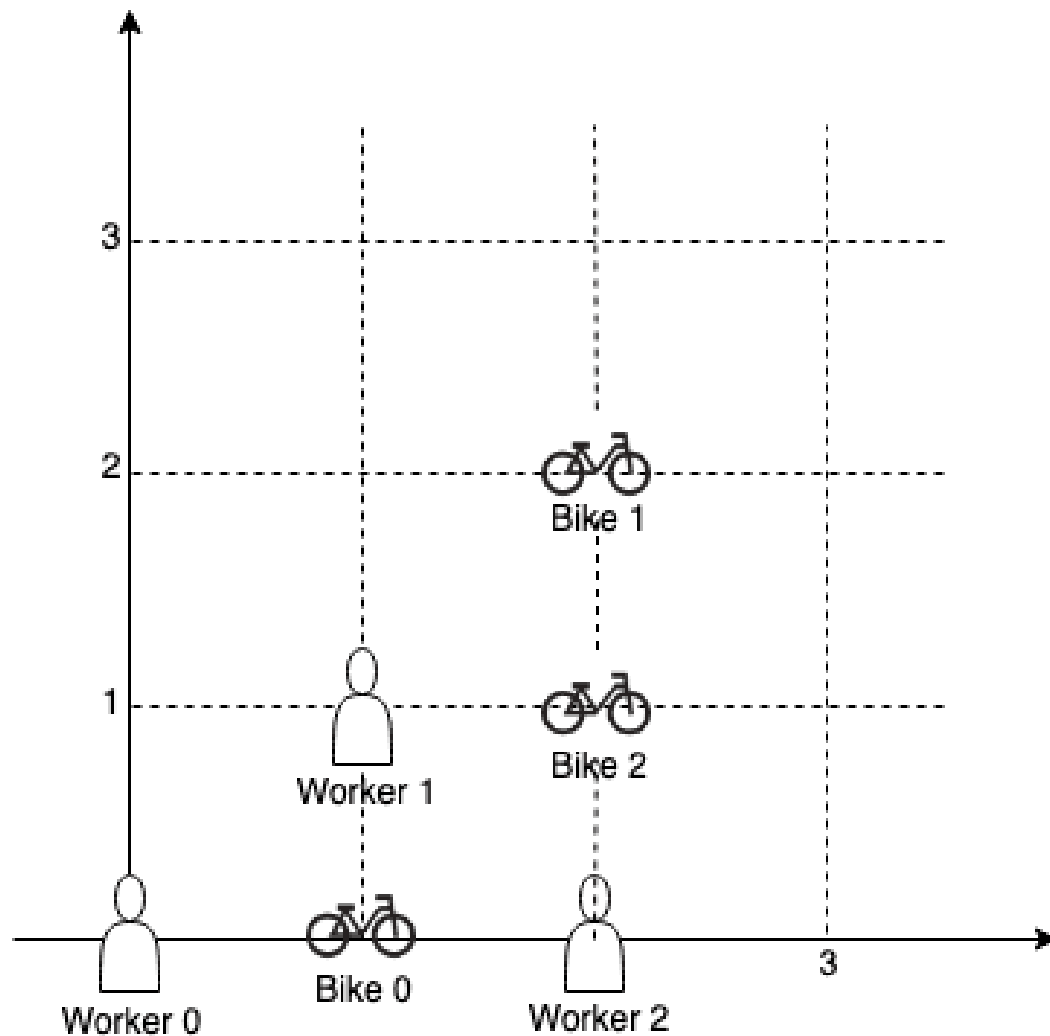
Input: workers = `[[0,0],[2,1]]`, bikes = `[[1,2],[3,3]]`

Output: 6

Explanation:

We assign bike 0 to worker 0, bike 1 to worker 1. The Manhattan distance of both assignments is 3, so the output is 6.

Example 2:



Input: workers = `[[0,0],[1,1],[2,0]]`, bikes = `[[1,0],[2,2],[2,1]]`

Output: 4

Explanation:

We first assign bike 0 to worker 0, then assign bike 1 to worker 1 or worker 2, bike 2 to worker 2 or worker 1. Both assignments lead to sum of the Manhattan distances as 4.

Example 3:

Input: workers = `[[0,0],[1,0],[2,0],[3,0],[4,0]]`, bikes = `[[0,999],[1,999],[2,999],[3,999],[4,999]]`

Output: 4995

Constraints:

- $n == \text{workers.length}$

- $m == \text{bikes.length}$
- $1 \leq n \leq m \leq 10$
- $\text{workers}[i].\text{length} == 2$
- $\text{bikes}[i].\text{length} == 2$
- $0 \leq \text{workers}[i][0], \text{workers}[i][1], \text{bikes}[i][0], \text{bikes}[i][1] < 1000$
- All the workers and the bikes locations are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def assignBikes(
        self,
        workers: list[list[int]],
        bikes: list[list[int]],
    ) -> int:
        def dist(p1: list[int], p2: list[int]) -> int:
            return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])

        @functools.lru_cache(None)

        def dp(workerIndex: int, used: int) -> int:
            """
            Returns the minimum Manhattan distances to assign bikes to
            workers[workerIndex..n), where `used` is the bitmask of the used bikes.
            """
            if workerIndex == len(workers):
                return 0
            return min(
                (dist(workers[workerIndex],
                    bike) + dp(workerIndex + 1, used | 1 << i) for i,
```



```

        bike in enumerate(bikes) if not used >> i & 1),
        default=math.inf)

    return dp(0, 0)

```

• LeetDoocs

```

class Solution:
    def assignBikes(self, workers: List[List[int]], bikes: List[List[int]]) ->
    int:
        n, m = len(workers), len(bikes)
        f = [[inf] * (1 << m) for _ in range(n + 1)]
        f[0][0] = 0
        for i, (x1, y1) in enumerate(workers, 1):
            for j in range(1 << m):
                for k, (x2, y2) in enumerate(bikes):
                    if j >> k & 1:
                        f[i][j] = min(
                            f[i][j],
                            f[i - 1][j ^ (1 << k)] + abs(x1 - x2) + abs(y1 - y
2),
                        )
        return min(f[n])

```

• SimplyLeet

```

from functools import lru_cache

def assignBikes(workers, bikes):
    # Helper function to compute Manhattan distance between two points
    def manhattan(p1, p2):
        return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])

    n = len(workers)
    m = len(bikes)

```

```

    # dp(worker, mask): minimum total Manhattan distance starting from worker
    index 'worker'
    # with bikes represented by 'mask' (1 bit per bike, 1 means assigned)
    @lru_cache(maxsize=None)
    def dp(worker, mask):
        # Base case: if all workers are assigned, return 0
        if worker == n:
            return 0

        res = float('inf')
        # Iterate over all bikes

        for j in range(m):
            # Check if bike j is free
            if not (mask & (1 << j)):
                # Calculate new mask with bike j assigned and compute total
                cost = manhattan(workers[worker], bikes[j]) + dp(worker + 1, mask | (1 << j))
                res = min(res, cost)

        return res

    # Start with the first worker and no bikes assigned (mask = 0)

    return dp(0, 0)

# Example usage:
# workers = [[0,0], [2,1]]
# bikes = [[1,2], [3,3]]
# print(assignBikes(workers, bikes))

```

◆ Quick Review

Key Insights

- The problem is a variant of the assignment problem which can be solved using Dynamic Programming (DP) with bitmasking due to the

small constraints ($n, m \leq 10$).

- A bitmask can represent which bikes have already been assigned. For example, if $m = 3$, then $\text{mask} = 101$ means bikes 0 and 2 have been taken.
- The state of the DP can be represented using two parameters: the current worker index and the bitmask representing the used bikes.
- Recursively assign bikes to workers by iterating over available bikes and take the minimum cumulative Manhattan distance.
- Memoization is key to avoid duplicate computations in overlapping subproblems.

Space & Time

Time Complexity: $O(m * 2^m)$ in the worst case, since for each of the 2^m states, we try assigning up to m bikes.

Space Complexity: $O(2^m)$, for storing the DP results (memoization cache).

Solution

The solution uses recursion with memoization (DP with bitmasking). We define a helper function $\text{dp}(\text{worker}, \text{mask})$ where worker is the index of the current worker to assign a bike

and mask is an integer representing the set of bikes already taken. For each worker, we iterate through all bikes and if a bike is not already assigned (checked using the bitmask), we calculate the Manhattan distance from the current worker to this bike and recursively assign the remaining workers. The result for each state is memoized to avoid recomputation. The base case occurs when all workers have been assigned bikes.

Bitmask DP

□ #1986 Minimum Number of Work Sessions to Finish the Tasks

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Backtracking ·
Bit Manipulation · Bitmask
Lists: AlgoMaster 600

Problem

There are n tasks assigned to you. The task times are represented as an integer array `tasks` of length n , where the i th task takes `tasks[i]` hours to finish. A work session is when you work for at most `sessionTime` consecutive hours and then take a break.

You should finish the given tasks in a way that satisfies the following conditions:

- If you start a task in a work session, you must complete it in the same work session.
- You can start a new task immediately after finishing the previous one.
- You may complete the tasks in any order.

Given tasks and sessionTime, return the minimum number of work sessions needed to finish all the tasks following the conditions above.

The tests are generated such that sessionTime is greater than or equal to the maximum element in tasks[i].

Example 1:

Input: tasks = [1,2,3], sessionTime = 3

Output: 2

Explanation: You can finish the tasks in two work sessions.

- First work session: finish the first and the second tasks in $1 + 2 = 3$ hours.
- Second work session: finish the third task in 3 hours.

Example 2:

Input: tasks = [3,1,3,1,1], sessionTime = 8

Output: 2

Explanation: You can finish the tasks in two work sessions.

- First work session: finish all the tasks except the last one in $3 + 1 + 3 + 1 = 8$ hours.
- Second work session: finish the last task in 1 hour.

Example 3:

Input: tasks = [1,2,3,4,5], sessionTime = 15

Output: 1

Explanation: You can finish all the tasks in one work session.

Constraints:

- $n == \text{tasks.length}$
- $1 \leq n \leq 14$
- $1 \leq \text{tasks}[i] \leq 10$
- $\max(\text{tasks}[i]) \leq \text{sessionTime} \leq 15$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minSessions(self, tasks: list[int], sessionTime: int) -> int:
        # Returns True if we can assign tasks[s..n) to `sessions`. Note that
        # `sessions`
        # may be occupied by some tasks.
        def dfs(s: int, sessions: list[int]) -> bool:
            if s == len(tasks):
                return True

            for i, session in enumerate(sessions):
                # Can't assign the tasks[s] to this session.

                if session + tasks[s] > sessionTime:
                    continue
                # Assign the tasks[s] to this session.
                sessions[i] += tasks[s]
                if dfs(s + 1, sessions):
                    return True
                # Backtracking.
                sessions[i] -= tasks[s]
                # If it's the first time we assign the tasks[s] to this session, then
                # `session`s can't satisfy either.

            if sessions[i] == 0:
                return False

            return False

        for numSessions in range(1, len(tasks) + 1):
            if dfs(0, [0] * numSessions):
                return numSessions
```

• LeetDoocs

```

class Solution:
    def minSessions(self, tasks: List[int], sessionTime: int) -> int:
        n = len(tasks)
        ok = [False] * (1 << n)
        for i in range(1, 1 << n):
            t = sum(tasks[j] for j in range(n) if i >> j & 1)
            ok[i] = t <= sessionTime
        f = [inf] * (1 << n)
        f[0] = 0
        for i in range(1, 1 << n):
            j = i
            while j:
                if ok[j]:
                    f[i] = min(f[i], f[i ^ j] + 1)
                j = (j - 1) & i
        return f[-1]

```

• SimplyLeet

```

def minSessions(tasks, sessionTime):
    n = len(tasks)
    from functools import lru_cache

    @lru_cache(maxsize=None)
    def dp(mask, time_used):
        # Base condition: if all tasks are completed.
        if mask == (1 << n) - 1:
            return 0
        best = float('inf')

        for i in range(n):
            # If task i is not yet completed.
            if not (mask & (1 << i)):
                if time_used + tasks[i] <= sessionTime:
                    best = min(best, dp(mask | (1 << i), time_used + tasks[i]))
                else:
                    best = min(best, 1 + dp(mask | (1 << i), tasks[i]))
        return best

    # We start with one session and 0 time used.

```



```
    return 1 + dp(0, 0)

# Example Usage
print(minSessions([1,2,3], 3))      # Output: 2
print(minSessions([3,1,3,1,1], 8)) # Output: 2
print(minSessions([1,2,3,4,5], 15)) # Output: 1
```

◆ Quick Review

Key Insights

- Use bitmasking to represent the state of completed tasks since $n \leq 14$.
- Apply recursion with memoization (DP) to explore task assignments.
- Each DP state is represented by a bitmask (indicating tasks done) and the current session's used time.
- When a task does not fit in the current session, start a new session and update the count.
- The problem can be solved with backtracking combining DP over subsets of tasks.

Space & Time

Time Complexity: $O(2^n * n)$, where n is the number of tasks.

Space Complexity: $O(2^n)$ due to memoization storage.

Solution

We use a dynamic programming approach with bitmasking to represent the subsets of tasks completed. The recursive function $dp(mask, timeUsed)$ returns the number of extra sessions required (beyond an initial session) for a given state. For every state, we try assigning each incomplete task: if it fits into the current session ($timeUsed + task[i] \leq sessionTime$), we continue; otherwise, we start a new session and add 1 to our count. By caching results of each state $(mask, timeUsed)$, we prevent redundant calculations, which makes the approach feasible given the constraints.

Bitmask DP

□ #2305 Fair Distribution of Cookies

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Backtracking ·
Bit Manipulation · Bitmask

Lists: AlgoMaster 600

Problem

You are given an integer array `cookies`, where `cookies[i]` denotes the number of cookies in the *i*th bag. You are also given an integer *k* that denotes the number of children to distribute all the bags of cookies to. All the cookies in the same bag must go to the same child and cannot be split up.

The unfairness of a distribution is defined as the maximum total cookies obtained by a single child in the distribution.

Return the minimum unfairness of all distributions.

Example 1:

Input: cookies = [8,15,10,20,8], k = 2

Output: 31

Explanation: One optimal distribution is [8,15,8] and [10,20]

- The 1st child receives [8,15,8] which has a total of $8 + 15 + 8 = 31$ cookies.

- The 2nd child receives [10,20] which has a total of $10 + 20 = 30$ cookies.

The unfairness of the distribution is $\max(31,30) = 31$.

It can be shown that there is no distribution with an unfairness less than 31.

Example 2:

Input: cookies = [6,1,3,2,2,4,1,2], k = 3

Output: 7

Explanation: One optimal distribution is [6,1], [3,2,2], and [4,1,2]

- The 1st child receives [6,1] which has a total of $6 + 1 = 7$ cookies.

- The 2nd child receives [3,2,2] which has a total of $3 + 2 + 2 = 7$ cookies.

- The 3rd child receives [4,1,2] which has a total of $4 + 1 + 2 = 7$ cookies.

The unfairness of the distribution is $\max(7,7,7) = 7$.

It can be shown that there is no distribution with an unfairness less than 7.

Constraints:

- $2 \leq \text{cookies.length} \leq 8$
- $1 \leq \text{cookies}[i] \leq 105$
- $2 \leq k \leq \text{cookies.length}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def distributeCookies(self, cookies: list[int], k: int) -> int:
        ans = math.inf

        def dfs(s: int, children: list[int]) -> None:
            nonlocal ans
            if s == len(cookies):
                ans = min(ans, max(children))
            return
```

```

    for i in range(k):
        children[i] += cookies[s]
        dfs(s + 1, children)
        children[i] -= cookies[s]

dfs(0, [0] * k)
return ans

```

• LeetCode

```

class Solution:
    def distributeCookies(self, cookies: List[int], k: int) -> int:
        def dfs(i):
            if i >= len(cookies):
                nonlocal ans
                ans = max(cnt)
                return
            for j in range(k):
                if cnt[j] + cookies[i] >= ans or (j and cnt[j] == cnt[j - 1]):
                    continue

                cnt[j] += cookies[i]
                dfs(i + 1)
                cnt[j] -= cookies[i]

        ans = inf
        cnt = [0] * k
        cookies.sort(reverse=True)
        dfs(0)
        return ans

```

• SimplyLeet

```
# Python solution using recursion and backtracking.
def distributeCookies(cookies, k):
    # Initialize the minimum unfairness as a large number.
    min_unfairness = float('inf')
    # Initialize a list to store the running sum of cookies for each child.
    children = [0] * k

    def backtrack(index):
        nonlocal min_unfairness
        # Base case: if we've assigned all bags.

        if index == len(cookies):
            # Update the minimum unfairness with the current maximum value
            # among children.
            min_unfairness = min(min_unfairness, max(children))
            return

        # Try giving the current cookie bag to each child.
        for i in range(k):
            children[i] += cookies[index]
            # Pruning: only proceed if the current child's sum does not exceed
            # known min_unfairness.
            if max(children) < min_unfairness:

                backtrack(index + 1)
                children[i] -= cookies[index]
                # Optimization: if this child hasn't been given a cookie bag yet,
                # break to avoid redundant permutations.
                if children[i] == 0:
                    break

        backtrack(0)
        return min_unfairness

# Example usage:
print(distributeCookies([8,15,10,20,8], 2)) # Output: 31
print(distributeCookies([6,1,3,2,2,4,1,2], 3)) # Output: 7
```

◆ Quick Review

Key Insights

- Use a backtracking approach to try every possible distribution since the number of cookie bags (n) is small ($n \leq 8$).
- Maintain an array representing the total cookies each child has received.
- Prune paths early if the current maximum exceeds the current best solution.
- The goal is to minimize the maximum sum among children across all distributions.

Space & Time

Time Complexity: $O(k^n)$ in the worst case, since we assign n bags to k children.

Space Complexity: $O(n)$ due to recursion depth and auxiliary data structures.

Solution

We can solve the problem using a recursive backtracking algorithm. At each recursive call, we consider giving the current cookie bag to each child and update their totals accordingly. After processing all bags, we compute the unfairness (maximum among all children's totals) and update the global minimum if the current distribution is better. The small input

size allows an exponential approach. Key structures include an array (or vector) to store each child's accumulated cookies and a recursion mechanism to explore all assignments.

Digit DP

Round 8 · Low · Medium · 1 question

Digit DP

□ #357 Count Numbers with Unique Digits **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Backtracking
Lists: AlgoMaster 600

Problem

Given an integer n , return the count of all numbers with unique digits, x , where $0 \leq x < 10^n$.

Example 1:

Input: $n = 2$
Output: 91
Explanation: The answer should be the total numbers in the range of $0 \leq x < 100$, excluding 11,22,33,44,55,66,77,88,99

Example 2:

Input: $n = 0$
Output: 1

Constraints:

- $0 \leq n \leq 8$

Community Solutions (Python)

- • WalkCC

```

class Solution:
    def countNumbersWithUniqueDigits(self, n: int) -> int:
        if n == 0:
            return 1

        ans = 10
        uniqueDigits = 9
        availableNum = 9

        while n > 1 and availableNum > 0:

            uniqueDigits *= availableNum
            ans += uniqueDigits
            n -= 1
            availableNum -= 1

        return ans

```

• LeetCode

```

class Solution:
    def countNumbersWithUniqueDigits(self, n: int) -> int:
        @cache
        def dfs(i: int, mask: int, lead: bool) -> int:
            if i < 0:
                return 1
            ans = 0
            for j in range(10):
                if mask >> j & 1:
                    continue

                if lead and j == 0:
                    ans += dfs(i - 1, mask, True)
                else:
                    ans += dfs(i - 1, mask | 1 << j, False)
            return ans

        return dfs(n - 1, 0, True)

```

• SimplyLeet

```
# Define the function to count numbers with unique digits
def countNumbersWithUniqueDigits(n):
    # For n = 0, only number valid is 0.
    if n == 0:
        return 1

    # Initialize result with one count for '0'
    result = 1

    # For numbers with more than 0 digits, iterate through digit lengths 1 to n
    # (max 10 digits)

    for k in range(1, min(n, 10) + 1):
        # Start with 9 options for the first digit (1 through 9)
        unique_count = 9
        # For remaining digits, each digit's options reduce as digits are used
        for j in range(1, k):
            unique_count *= (10 - j)
        # Add count for k-digit numbers to result
        result += unique_count

    return result

# Example usage:
n = 2
print(countNumbersWithUniqueDigits(n)) # Expected output: 91

# You can test other cases by calling countNumbersWithUniqueDigits with
different n values.
```

◆ Quick Review

Key Insights

- Use combinatorial counting to determine the number of valid numbers of a given digit length.

- For numbers with more than one digit, the first digit cannot be zero.
- For each additional digit, the choices decrease because digits must remain unique.
- The solution adds up valid counts for each possible digit length from 1 to n , plus the number 0.

Space & Time

Time Complexity: $O(n)$ per computation since we iterate at most 10 times ($n \leq 8$).

Space Complexity: $O(1)$ as only a few variables are used.

Solution

The solution uses a combinatorial approach.

For $n = 0$, the answer is 1 (only number 0). For $n \geq 1$, the solution counts valid numbers for each digit length k ($1 \leq k \leq n$):

- For 1-digit numbers, there are 9 possibilities (1–9).
- For k -digit numbers ($k > 1$), the first digit has 9 possibilities (1–9), and each subsequent digit has a decreasing number of available digits (from 10 minus the number already used). Specifically, for the second digit there

are 9 choices, for the third digit there are 8, and so forth.

- Sum the counts for each valid digit length and add 1 for the number 0. This approach avoids generating all numbers and leverages basic combinatorial principles for an efficient count.

Probability DP

Round 8 · Low · Medium · 3 questions

Probability DP

□ #688 Knight Probability in Chessboard

MED

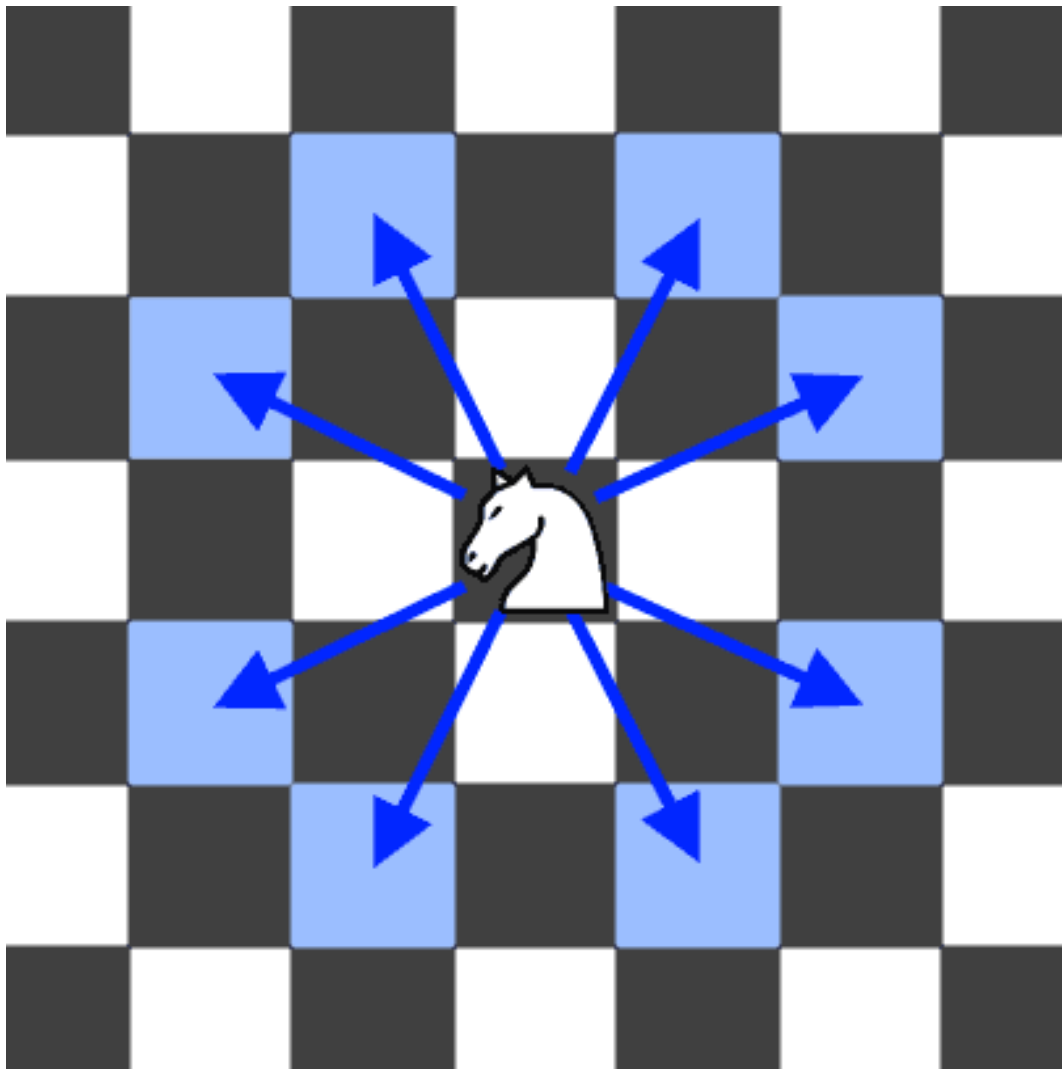
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming

Lists: AlgoMaster 600

Problem

On an $n \times n$ chessboard, a knight starts at the cell (row, column) and attempts to make exactly k moves. The rows and columns are 0-indexed, so the top-left cell is (0, 0), and the bottom-right cell is ($n - 1$, $n - 1$).

A chess knight has eight possible moves it can make, as illustrated below. Each move is two cells in a cardinal direction, then one cell in an orthogonal direction.



Each time the knight is to move, it chooses one of eight possible moves uniformly at random (even if the piece would go off the chessboard) and moves there.

The knight continues moving until it has made exactly k moves or has moved off the chessboard.

Return the probability that the knight remains on the board after it has stopped moving.

Example 1:

Input: $n = 3, k = 2, \text{row} = 0, \text{column} = 0$

Output: 0.06250

Explanation: There are two moves (to (1,2), (2,1)) that will keep the knight on the board.

From each of those positions, there are also two moves that will keep the knight on the board.

The total probability the knight stays on the board is 0.0625.

Example 2:

Input: $n = 1, k = 0, \text{row} = 0, \text{column} = 0$

Output: 1.00000

Constraints:

- $1 \leq n \leq 25$
- $0 \leq k \leq 100$
- $0 \leq \text{row}, \text{column} \leq n - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def knightProbability(self, n: int, k: int, row: int, column: int) -> float:
        DIRS = ((1, 2), (2, 1), (2, -1), (1, -2),
                (-1, -2), (-2, -1), (-2, 1), (-1, 2))
        # dp[i][j] := the probability to stand on (i, j)
        dp = [[0] * n for _ in range(n)]
        dp[row][column] = 1.0

        for _ in range(k):
            newDp = [[0] * n for _ in range(n)]
```

```

for i in range(n):
    for j in range(n):
        for dx, dy in DIRS:
            x = i + dx
            y = j + dy
            if 0 <= x < n and 0 <= y < n:
                newDp[i][j] += dp[x][y]
dp = newDp

return sum(map(sum, dp)) / 8**k

```

• LeetCode

```

class Solution:
    def knightProbability(self, n: int, k: int, row: int, column: int) -> float:
        f = [[[0] * n for _ in range(n)] for _ in range(k + 1)]
        for i in range(n):
            for j in range(n):
                f[0][i][j] = 1
        for h in range(1, k + 1):
            for i in range(n):
                for j in range(n):
                    for a, b in pairwise((-2, -1, 2, 1, -2, 1, 2, -1, -2)):
                        x, y = i + a, j + b
                        if 0 <= x < n and 0 <= y < n:
                            f[h][i][j] += f[h - 1][x][y] / 8
        return f[k][row][column]

```

• SimplyLeet

```

# Python solution using iterative DP

def knightProbability(n, k, row, column):
    # Define the 8 possible moves for a knight
    moves = [(2, 1), (1, 2), (-1, 2), (-2, 1),
              (-2, -1), (-1, -2), (1, -2), (2, -1)]

    # Initialize dp with probability 0 for each cell, and set starting cell to
    1
    dp = [[0 for _ in range(n)] for _ in range(n)]
    dp[row][column] = 1.0

    # Iterate for each move
    for move_num in range(k):
        # Create temporary dp for the next move
        dp_next = [[0 for _ in range(n)] for _ in range(n)]
        # Process each cell
        for i in range(n):
            for j in range(n):
                if dp[i][j] > 0:
                    # Try each knight move

                    for dx, dy in moves:
                        ni, nj = i + dx, j + dy
                        # Check board boundaries
                        if 0 <= ni < n and 0 <= nj < n:
                            dp_next[ni][nj] += dp[i][j] / 8.0

        # Update dp for the current move
        dp = dp_next

    # Sum probabilities of all cells for the final answer
    total_probability = sum(map(sum, dp))

    return total_probability

# Example usage:
#print(knightProbability(3, 2, 0, 0))

```

◆ Quick Review

Key Insights

- Use Dynamic Programming since the state (i.e., cell and number of moves remaining) overlaps.
- A recursive approach with memoization or iterative bottom-up DP can be applied.
- Each knight move occurs with a probability of $1/8$.
- Base cases include $k=0$ (the knight is on the board) and moves leading off the board contribute 0 probability.
- Transition: For each valid cell at a certain move, accumulate the probability from all 8 possible moves from that cell.

Space & Time

Time Complexity: $O(k * n^2)$ as we iterate over k moves and each cell of the board.

Space Complexity: $O(n^2)$ if using iterative DP with two 2D arrays (or $O(k * n^2)$ for a memoization approach).

Solution

We solve the problem using a bottom-up dynamic programming approach. We maintain

a 2D DP array that represents the probability of the knight being at each cell after a given number of moves. We start by initializing the starting position with a probability of 1. For each move, we compute a new probability distribution over the board by iterating through each cell and distributing its probability over the 8 knight moves (only if the move remains within bounds). After k moves, the sum of the probabilities in the DP table gives the final answer. This iterative updating avoids repeated computations and elegantly manages the probability propagation.

Probability DP

□ #808 Soup Servings

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Probability and
Statistics

Lists: AlgoMaster 600

Problem

You have two soups, A and B, each starting with n mL. On every turn, one of the following four serving operations is chosen at random, each with probability 0.25 independent of all previous turns:

- pour 100 mL from type A and 0 mL from type B
- pour 75 mL from type A and 25 mL from type B
- pour 50 mL from type A and 50 mL from type B
- pour 25 mL from type A and 75 mL from type B

Note:

- There is no operation that pours 0 mL from A and 100 mL from B.
- The amounts from A and B are poured simultaneously during the turn.
- If an operation asks you to pour more than you have left of a soup, pour all that remains of that soup.

The process stops immediately after any turn in which one of the soups is used up.

Return the probability that A is used up before B, plus half the probability that both soups are used up in the same turn. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

Input: $n = 50$

Output: 0.62500

Explanation:

If we perform either of the first two serving operations, soup A will become empty first.

If we perform the third operation, A and B will become empty at the same time.

If we perform the fourth operation, B will become empty first.

So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is $0.25 * (1 + 1 + 0.5 + 0) = 0.625$.

Example 2:

Input: $n = 100$

Output: 0.71875

Explanation:

If we perform the first serving operation, soup A will become empty first.

If we perform the second serving operations, A will become empty on performing operation [1, 2, 3], and both A and B become empty on performing operation 4.

If we perform the third operation, A will become empty on performing operation [1, 2], and both A and B become empty on performing operation 3.

If we perform the fourth operation, A will become empty on performing operation 1, and both A and B become empty on performing operation 2.

So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is 0.71875.

Constraints:

- $0 \leq n \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def soupServings(self, n: int) -> float:
        @functools.lru_cache(None)
        def dfs(a: int, b: int) -> float:
            if a <= 0 and b <= 0:
                return 0.5
            if a <= 0:
                return 1.0
            if b <= 0:
                return 0.0

            return 0.25 * (dfs(a - 4, b) +
                          dfs(a - 3, b - 1) +
                          dfs(a - 2, b - 2) +
                          dfs(a - 1, b - 3))

        return 1 if n >= 4800 else dfs((n + 24) // 25, (n + 24) // 25)
```

• LeetDoocs

```

class Solution:
    def soupServings(self, n: int) -> float:
        @cache
        def dfs(i: int, j: int) -> float:
            if i <= 0 and j <= 0:
                return 0.5
            if i <= 0:
                return 1
            if j <= 0:
                return 0

            return 0.25 * (
                dfs(i - 4, j)
                + dfs(i - 3, j - 1)
                + dfs(i - 2, j - 2)
                + dfs(i - 1, j - 3)
            )

        return 1 if n > 4800 else dfs((n + 24) // 25, (n + 24) // 25)

```

• SimplyLeet

```

import math
from functools import lru_cache

def soupServings(n):
    # For large n, the probability tends to 1 directly.
    if n > 4800:
        return 1.0
    # Scale down n by 25 using ceiling division.
    N = (n + 24) // 25

    @lru_cache(maxsize=None)
    def dp(a, b):
        # Both soups are depleted simultaneously.
        if a <= 0 and b <= 0:
            return 0.5
        # If only soup A is empty, it counts as a valid case.
        if a <= 0:
            return 1.0
        # If only soup B is empty, it's not a favorable outcome.
        if b <= 0:

```

```

        return 0.0
    # Calculate probability by averaging outcomes of the four operations.
    return 0.25 * (dp(a - 4, b) + dp(a - 3, b - 1) +
                  dp(a - 2, b - 2) + dp(a - 1, b - 3))

    return dp(N, N)

# Example usage:
#print(soupServings(50)) # Expected output: 0.62500

```

◆ Quick Review

Key Insights

- The problem is inherently probabilistic and can be solved using recursion with memoization (DP).
- Transform the problem by scaling down the soup amounts (dividing n by 25 and rounding up), converting the operations to unit decrements.
- Use a recursive function $dp(a, b)$ to compute the probability starting with a units of soup A and b units of soup B.
- Base cases:
 - Both soups empty ($a \leq 0$ and $b \leq 0$): return 0.5.
 - Only soup A empty ($a \leq 0$): return 1.
 - Only soup B empty ($b \leq 0$): return 0.

- For large n ($n > 4800$), the answer approaches 1, so return it early for performance reasons.

Space & Time

Time Complexity: $O((n/25)^2)$ in the worst-case because the recursive state space is roughly bounded by the transformed soup amount.

Space Complexity: $O((n/25)^2)$ due to the memoization cache storing each state.

Solution

The solution employs recursion with memoization to avoid redundant calculations. By scaling down the amount of soup (dividing by 25) the problem works in a smaller state space. The recursive function handles:

- Base conditions where one or both soups are empty.
- A summation of the outcomes from the four possible operations. To improve efficiency for very large soup amounts, an early return is added when n exceeds 4800, as the probability converges to 1.

Probability DP

□ #837 New 21 Game

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Sliding Window
· Probability and Statistics

Lists: AlgoMaster 600

Problem

Alice plays the following game, loosely based on the card game "21".

Alice starts with 0 points and draws numbers while she has less than k points. During each draw, she gains an integer number of points randomly from the range $[1, \text{maxPts}]$, where maxPts is an integer. Each draw is independent and the outcomes have equal probabilities.

Alice stops drawing numbers when she gets k or more points.

Return the probability that Alice has n or fewer points.

Answers within 10^{-5} of the actual answer are considered accepted.

Example 1:

Input: $n = 10$, $k = 1$, $\text{maxPts} = 10$

Output: 1.00000

Explanation: Alice gets a single card, then stops.

Example 2:

Input: $n = 6$, $k = 1$, $\text{maxPts} = 10$

Output: 0.60000

Explanation: Alice gets a single card, then stops.

In 6 out of 10 possibilities, she is at or below 6 points.

Example 3:

Input: $n = 21$, $k = 17$, $\text{maxPts} = 10$

Output: 0.73278

Constraints:

- $0 \leq k \leq n \leq 10^4$
- $1 \leq \text{maxPts} \leq 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def new21Game(self, n: int, k: int, maxPts: int) -> float:
        # When the game ends, the point is in [k..k - 1 + maxPts].
        # P = 1, if n >= k - 1 + maxPts
        # P = 0, if n < k (note that the constraints already have k <= n)
        if k == 0 or n >= k - 1 + maxPts:
            return 1.0

        ans = 0.0
        dp = [1.0] + [0] * n # dp[i] := the probability to have i points
```

```

windowSum = dp[0] # P(i - 1) + P(i - 2) + ... + P(i - maxPts)

for i in range(1, n + 1):
    # The probability to get i points is
    # P(i) = [P(i - 1) + P(i - 2) + ... + P(i - maxPts)] / maxPts
    dp[i] = windowSum / maxPts
    if i < k:
        windowSum += dp[i]
    else: # The game ends.
        ans += dp[i]

        if i - maxPts >= 0:
            windowSum -= dp[i - maxPts]

return ans

```

• LeetDoocs

```

class Solution:
    def new21Game(self, n: int, k: int, maxPts: int) -> float:
        @cache
        def dfs(i: int) -> float:
            if i >= k:
                return int(i <= n)
            if i == k - 1:
                return min(n - k + 1, maxPts) / maxPts
            return dfs(i + 1) + (dfs(i + 1) - dfs(i + maxPts + 1)) / maxPts

        return dfs(0)

```

```

class Solution:
    def new21Game(self, n: int, k: int, maxPts: int) -> float:
        f = [0] * (k + maxPts)
        for i in range(k, min(n + 1, k + maxPts)):
            f[i] = 1
        f[k - 1] = min(n - k + 1, maxPts) / maxPts
        for i in range(k - 2, -1, -1):
            f[i] = f[i + 1] + (f[i + 1] - f[i + maxPts + 1]) / maxPts
        return f[0]

```

• SimplyLeet

```
# Python solution for New 21 Game
class Solution:
    def new21Game(self, n: int, k: int, maxPts: int) -> float:
        # Edge case: If k is 0 then Alice doesn't draw any card, always count
        # as 1.
        if k == 0:
            return 1.0

        # dp[i] will represent the probability of reaching exactly i points.
        dp = [0.0] * (n + 1)
        dp[0] = 1.0 # start with 0 points, probability 1

        # W is the cumulative probability for states that can transition into
        # the current state.
        w = 1.0
        result = 0.0

        # Calculate probabilities for each score from 1 to n.
        for i in range(1, n + 1):
            dp[i] = w / maxPts
            # Only add probabilities for states less than k to the window
            # because game ends when score >= k.
            if i < k:
                w += dp[i]
            else:
                # For i >= k, these states represent a terminal state where
                # Alice stops.
                result += dp[i]
                # If the window has exceeded the size maxPts, subtract the oldest
                # probability.
                if i - maxPts >= 0:
                    w -= dp[i - maxPts]

        return result

# Example usage:
# solution = Solution()
# print(solution.new21Game(21, 17, 10))
```


◆ Quick Review

Key Insights

- If k is 0, Alice never draws a card, so the answer is 1.
- If n is large enough ($n \geq k - 1 + \text{maxPts}$), Alice will always win.
- Use dynamic programming to compute the probability of reaching every score from 0 to n .
- Use a sliding window sum to efficiently compute the probability recurrence relation.
- The state transition relates $\text{dp}[x]$ to the sum of probabilities $\text{dp}[x - 1] \dots \text{dp}[x - \text{maxPts}]$ divided by maxPts .

Space & Time

Time Complexity: $O(n)$, because we calculate each dp state at most once.

Space Complexity: $O(n)$, for storing the dp array.

Solution

We solve this problem using dynamic programming with a sliding window approach:

- Create an array dp where $dp[i]$ represents the probability of reaching exactly i points.
- Base case: $dp[0] = 1.0$ (starting with 0 points).
- Use a sliding window sum to maintain the sum of dp values for states from $i - \text{maxPts}$ to $i - 1$.
- For each score i from 1 to n :
- The probability $dp[i]$ is the average of the probabilities in the window.
- If i is less than k , add $dp[i]$ to the window sum.
- If $i - \text{maxPts}$ is within range, subtract $dp[i - \text{maxPts}]$ from the window sum.
- The final answer is the sum of $dp[i]$ for i from k to n , as these are the terminal states where Alice stops.

State Machine DP

Round 8 · Low · Medium · 1 question

State Machine DP

□ #714 Best Time to Buy and Sell Stock with Transaction Fee

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy
Lists: AlgoMaster 600

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the *i*th day, and an integer `fee` representing a transaction fee.

Find the maximum profit you can achieve. You may complete as many transactions as you like, but you need to pay the transaction fee for each transaction.

Note:

- You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).
- The transaction fee is only charged once for each stock purchase and sale.

Example 1:

Input: prices = [1,3,2,8,4,9], fee = 2
 Output: 8
 Explanation: The maximum profit can be achieved by:
 - Buying at prices[0] = 1
 - Selling at prices[3] = 8
 - Buying at prices[4] = 4
 - Selling at prices[5] = 9
 The total profit is $((8 - 1) - 2) + ((9 - 4) - 2) = 8$.

Example 2:

Input: prices = [1,3,7,5,10,3], fee = 3
 Output: 6

Constraints:

- $1 \leq \text{prices.length} \leq 5 * 10^4$
- $1 \leq \text{prices}[i] < 5 * 10^4$
- $0 \leq \text{fee} < 5 * 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProfit(self, prices: list[int], fee: int) -> int:
        sell = 0
        hold = -math.inf

        for price in prices:
            sell = max(sell, hold + price)
            hold = max(hold, sell - price - fee)

        return sell
```

• LeetDoocs

```

class Solution:
    def maxProfit(self, prices: List[int], fee: int) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            if i >= len(prices):
                return 0
            ans = dfs(i + 1, j)
            if j:
                ans = max(ans, prices[i] + dfs(i + 1, 0) - fee)
            else:
                ans = max(ans, -prices[i] + dfs(i + 1, 1))
            return ans

        return dfs(0, 0)

```

```

class Solution:
    def maxProfit(self, prices: List[int], fee: int) -> int:
        n = len(prices)
        f = [[0] * 2 for _ in range(n)]
        f[0][1] = -prices[0]
        for i in range(1, n):
            f[i][0] = max(f[i - 1][0], f[i - 1][1] + prices[i] - fee)
            f[i][1] = max(f[i - 1][1], f[i - 1][0] - prices[i])
        return f[n - 1][0]

```

```

class Solution:
    def maxProfit(self, prices: List[int], fee: int) -> int:
        f0, f1 = 0, -prices[0]
        for x in prices[1:]:
            f0, f1 = max(f0, f1 + x - fee), max(f1, f0 - x)
        return f0

```

• SimplyLeet

```
# Define a function to calculate maximum profit.
def maxProfit(prices, fee):
    # Base case: if prices is empty, profit is 0
    if not prices:
        return 0

    # Initialize cash (profit without stock) and hold (profit with stock)
    cash = 0
    hold = -prices[0]

    # Loop through each day's price starting from the second day
    for price in prices[1:]:
        # Temporary variable to hold previous cash value
        prev_cash = cash
        # Update cash. Either you keep your previous cash or sell the stock
        # today.
        cash = max(cash, hold + price - fee)
        # Update hold. Either you keep holding or buy the stock today.
        hold = max(hold, prev_cash - price)

    # The maximum profit is when you are not holding any stock.

    return cash

# Example usage:
prices = [1, 3, 2, 8, 4, 9]
fee = 2
print(maxProfit(prices, fee)) # Output: 8
```

◆ Quick Review

Key Insights

- Use dynamic programming to track profits when holding and not holding the stock.
- Define two states:

- cash: maximum profit when you do not hold any stock.
- hold: maximum profit when you hold a stock.
- When buying, you update the hold state by subtracting the current price from cash.
- When selling, add the current price to hold and subtract the fee.
- Update both states iteratively as you traverse the price list.

Space & Time

Time Complexity: $O(n)$, where n is the number of days (`prices.length`).

Space Complexity: $O(1)$, as only two variables (states) are maintained at each iteration.

Solution

We use a dynamic programming approach using two variables: cash and hold.

– cash represents the maximum profit when no stock is held.

– hold represents the maximum profit when a stock is held.

Initialize `cash = 0` and `hold = -prices[0]`. Then, for each day, update:

- cash as the maximum of the previous cash or the profit from selling the stock held (hold + current price – fee).
- hold as the maximum of the previous hold or the profit from buying the stock (cash – current price).

After processing all days, the maximum profit will be in cash because you must have sold any held stock to realize the profit.

Maths / Geometry

Round 8 · Low · Medium · 8 questions

Maths / Geometry

□ #172 Factorial Trailing Zeroes

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math

Lists: AlgoMaster 600

Problem

Given an integer n , return the number of trailing zeroes in $n!$.

Note that $n! = n * (n - 1) * (n - 2) * ... * 3 * 2 * 1$.

Example 1:

Input: $n = 3$
Output: 0
Explanation: $3! = 6$, no trailing zero.

Example 2:

Input: $n = 5$
Output: 1
Explanation: $5! = 120$, one trailing zero.

Example 3:

Input: $n = 0$
Output: 0

Constraints:

- $0 \leq n \leq 104$

Follow up: Could you write a solution that works in logarithmic time complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def trailingZeroes(self, n: int) -> int:
        return 0 if n == 0 else n // 5 + self.trailingZeroes(n // 5)
```

• LeetDoocs

```
class Solution:
    def trailingZeroes(self, n: int) -> int:
        ans = 0
        while n:
            n //= 5
            ans += n
        return ans
```

• SimplyLeet

```
# Python solution to count trailing zeroes in n!

def trailingZeroes(n):
    # Initialize count of trailing zeroes
    count = 0
    # Loop until n becomes zero
    while n > 0:
        # Divide n by 5 to count multiples of 5
        n //= 5
        # Add the result to the overall count

        count += n
    # Return the total count of trailing zeroes
    return count

# Example usage:
print(trailingZeroes(5)) # Expected output: 1
```

◆ Quick Review

Key Insights

- Trailing zeroes are produced by factors of 10 ($2 * 5$) in the factorial.
- There are more 2s than 5s in $n!$ so counting the number of 5 factors will suffice.
- Count how many multiples of 5, 25, 125, etc., since each contributes additional factors of 5.
- This approach runs in logarithmic time relative to n .

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

We count the factors of 5 in $n!$ by repeatedly dividing n by 5. The intuition is that every 5th number contributes at least one factor of 5, every 25th contributes an extra factor because $25 = 5 * 5$, and so on. We continue this division until n becomes 0. This avoids computing the factorial directly, providing a very efficient solution. The algorithm uses a simple loop and integer division, resulting in logarithmic time

complexity and constant space usage.

Maths / Geometry

□ #204 Count Primes

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Enumeration · Number Theory
Lists: AlgoMaster 600

Problem

Given an integer n , return the number of prime numbers that are strictly less than n .

Example 1:

Input: $n = 10$
Output: 4
Explanation: There are 4 prime numbers less than 10, they are 2, 3, 5, 7.

Example 2:

Input: $n = 0$
Output: 0

Example 3:

Input: $n = 1$
Output: 0

Constraints:

- $0 \leq n \leq 5 * 10^6$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def countPrimes(self, n: int) -> int:
        if n <= 2:
            return 0
        return sum(self._sieveEratosthenes(n))

    def _sieveEratosthenes(self, n: int) -> list[bool]:
        isPrime = [True] * n
        isPrime[0] = False
        isPrime[1] = False

        for i in range(2, int(n**0.5) + 1):
            if isPrime[i]:
                for j in range(i * i, n, i):
                    isPrime[j] = False
        return isPrime
```

• LeetCode

```
class Solution:
    def countPrimes(self, n: int) -> int:
        primes = [True] * n
        ans = 0
        for i in range(2, n):
            if primes[i]:
                ans += 1
                for j in range(i + i, n, i):
                    primes[j] = False
        return ans
```

• SimplyLeet

```
def countPrimes(n):
    # If n is less than 2, there are no prime numbers
    if n < 2:
        return 0

    # Initialize a boolean array indicating prime status for each number
    is_prime = [True] * n
    # 0 and 1 are not prime numbers
    is_prime[0] = is_prime[1] = False
    # Iterate over each number up to the square root of n
    p = 2
```



```
while p * p < n:
    if is_prime[p]:
        # Mark multiples of p starting from p*p as non-prime
        for i in range(p * p, n, p):
            is_prime[i] = False
        p += 1
# Sum up the True values to get the total count of primes
return sum(is_prime)

# Example usage:

print(countPrimes(10)) # Expected output: 4
```

◆ Quick Review

Key Insights

- The problem can be efficiently solved using the Sieve of Eratosthenes.
- Only numbers less than n are considered.
- Iterating only up to the square root of n will mark all non-prime numbers effectively.
- An auxiliary boolean array can represent whether each number is prime.

Space & Time

Time Complexity: $O(n \log \log n)$

Space Complexity: $O(n)$

Solution

The Sieve of Eratosthenes algorithm is used to solve this problem. We create a boolean array

of size n where each index initially assumes the number is prime. We then mark 0 and 1 as non-prime. For each number p starting from 2 up to the square root of n , if p is still marked as prime, we mark all multiples of p (starting at $p \cdot p$) as non-prime. Finally, the count of true values in the boolean array gives the number of primes less than n .

Maths / Geometry

□ #264 Ugly Number II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Math · Dynamic Programming ·
Heap (Priority Queue)
Lists: AlgoMaster 600

Problem

An ugly number is a positive integer whose prime factors are limited to 2, 3, and 5.

Given an integer n , return the n th ugly number.

Example 1:

Input: $n = 10$
Output: 12
Explanation: [1, 2, 3, 4, 5, 6, 8, 9, 10, 12] is the sequence of the first 10 ugly numbers.

Example 2:

Input: $n = 1$
Output: 1
Explanation: 1 has no prime factors, therefore all of its prime factors are limited to 2, 3, and 5.

Constraints:

- $1 \leq n \leq 1690$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def nthUglyNumber(self, n: int) -> int:
        nums = [1]
        i2 = 0
        i3 = 0
        i5 = 0

        while len(nums) < n:
            next2 = nums[i2] * 2
            next3 = nums[i3] * 3

            next5 = nums[i5] * 5
            next = min(next2, next3, next5)
            if next == next2:
                i2 += 1
            if next == next3:
                i3 += 1
            if next == next5:
                i5 += 1
            nums.append(next)

        return nums[-1]
```

• LeetDoocs

```
class Solution:
    def nthUglyNumber(self, n: int) -> int:
        h = [1]
        vis = {1}
        ans = 1
        for _ in range(n):
            ans = heappop(h)
            for v in [2, 3, 5]:
                nxt = ans * v
                if nxt not in vis:
                    vis.add(nxt)
                    heappush(h, nxt)
        return ans
```

```

class Solution:
    def nthUglyNumber(self, n: int) -> int:
        dp = [1] * n
        p2 = p3 = p5 = 0
        for i in range(1, n):
            next2, next3, next5 = dp[p2] * 2, dp[p3] * 3, dp[p5] * 5
            dp[i] = min(next2, next3, next5)
            if dp[i] == next2:
                p2 += 1
            if dp[i] == next3:
                p3 += 1
            if dp[i] == next5:
                p5 += 1
        return dp[-1]

```

• SimplyLeet

```

# Function to return the nth ugly number
def nthUglyNumber(n: int) -> int:
    # dp array to store ugly numbers, starting with 1 as the first ugly number
    ugly_numbers = [0] * n
    ugly_numbers[0] = 1

    # pointers for multiples of 2, 3, and 5
    i2 = i3 = i5 = 0

    # iterate to generate ugly numbers up to the nth

    for i in range(1, n):
        # Next possible ugly numbers from each prime factor
        next_multiple_of_2 = ugly_numbers[i2] * 2
        next_multiple_of_3 = ugly_numbers[i3] * 3
        next_multiple_of_5 = ugly_numbers[i5] * 5

        # Choose the smallest one as the next ugly number
        next_ugly = min(next_multiple_of_2, next_multiple_of_3, next_multiple_
of_5)
        ugly_numbers[i] = next_ugly

```

```
# Move pointers if they were used
if next_ugly == next_multiple_of_2:
    i2 += 1
if next_ugly == next_multiple_of_3:
    i3 += 1
if next_ugly == next_multiple_of_5:
    i5 += 1

# The last element in the list is the nth ugly number
return ugly_numbers[-1]
```

```
# Example usage:
print(nthUglyNumber(10)) # Output: 12
```

◆ Quick Review

Key Insights

- Ugly numbers are generated by multiplying smaller ugly numbers by 2, 3, or 5.
- Use dynamic programming to build a sequence of ugly numbers starting from 1.
- Maintain three pointers (for factors 2, 3, and 5) that determine the next candidate ugly number.
- Avoid duplicate entries by checking and advancing all pointers that match the current minimum candidate.
- The approach runs in $O(n)$ time and uses $O(n)$ space.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution uses a dynamic programming approach. We maintain an array (or list) that stores the ugly numbers in ascending order, starting with 1. Three pointers are used—each corresponding to the multipliers 2, 3, and 5.

For each iteration:

- Calculate the next possible ugly numbers by multiplying the number pointed to by each pointer with 2, 3, and 5.
- Select the minimum of these candidates as the next ugly number.
- Increment the corresponding pointer(s) to avoid duplicate calculations.
- Continue until the n th ugly number is generated. This process ensures that each ugly number is generated in order and without duplicates.

Maths / Geometry

□ #593 Valid Square

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Geometry

Lists: AlgoMaster 600

Problem

Given the coordinates of four points in 2D space p_1 , p_2 , p_3 and p_4 , return true if the four points construct a square.

The coordinate of a point p_i is represented as $[x_i, y_i]$. The input is not given in any order.

A valid square has four equal sides with positive length and four equal angles (90-degree angles).

Example 1:

Input: $p_1 = [0,0]$, $p_2 = [1,1]$, $p_3 = [1,0]$, $p_4 = [0,1]$
Output: true

Example 2:

Input: $p_1 = [0,0]$, $p_2 = [1,1]$, $p_3 = [1,0]$, $p_4 = [0,12]$
Output: false

Example 3:

Input: p1 = [1,0], p2 = [-1,0], p3 = [0,1], p4 = [0,-1]
 Output: true

Constraints:

- p1.length == p2.length == p3.length == p4.length == 2
- $-104 \leq x_i, y_i \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def validSquare(
        self,
        p1: list[int],
        p2: list[int],
        p3: list[int],
        p4: list[int],
    ) -> bool:
        def dist(p1: list[int], p2: list[int]) -> int:
            return (p1[0] - p2[0])**2 + (p1[1] - p2[1])**2

        distSet = set([dist(*pair)
                        for pair in list(
                            itertools.combinations([p1, p2, p3, p4], 2))]
        return 0 not in distSet and len(distSet) == 2
```

• LeetDoocs

```

class Solution:
    def validSquare(
        self, p1: List[int], p2: List[int], p3: List[int], p4: List[int]
    ) -> bool:
        def check(a, b, c):
            (x1, y1), (x2, y2), (x3, y3) = a, b, c
            d1 = (x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2)
            d2 = (x1 - x3) * (x1 - x3) + (y1 - y3) * (y1 - y3)
            d3 = (x2 - x3) * (x2 - x3) + (y2 - y3) * (y2 - y3)
            return any(
                [
                    d1 == d2 and d1 + d2 == d3 and d1,
                    d2 == d3 and d2 + d3 == d1 and d2,
                    d1 == d3 and d1 + d3 == d2 and d1,
                ]
            )

        return (
            check(p1, p2, p3)
            and check(p2, p3, p4)

            and check(p1, p3, p4)
            and check(p1, p2, p4)
        )

```

• SimplyLeet

```

# Define a function to calculate squared distance between two points
def squared_distance(p, q):
    # Calculate the squared difference in x and y coordinates
    return (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2

def validSquare(p1, p2, p3, p4):
    # List of all points
    points = [p1, p2, p3, p4]
    distances = []

```

```

    # Compute squared distances for each unique pair among 4 points (6 pairs in total)
    for i in range(4):
        for j in range(i+1, 4):
            dist = squared_distance(points[i], points[j])
            if dist == 0:
                # If two points are the same, it's not a square
                return False
            distances.append(dist)

    # Use a dictionary to count frequency of each unique distance

    freq = {}
    for d in distances:
        freq[d] = freq.get(d, 0) + 1

    # There must be exactly 2 distinct distances
    if len(freq) != 2:
        return False

    # Identify side and diagonal distances based on frequency count
    side, diag = sorted(freq, key=lambda x: (freq[x], x))

    # For a valid square, the side should appear 4 times and diagonal 2 times
    return freq[side] == 4 and freq[diag] == 2

# Example usage:
print(validSquare([0,0], [1,1], [1,0], [0,1])) # Expected output: True

```

◆ Quick Review

Key Insights

- Calculate all six pairwise squared distances between the four points.
- For a valid square, there should be exactly two unique distances: the smaller one representing the sides (appearing exactly 4

times) and the larger one representing the diagonals (appearing exactly 2 times).

- Duplicate points or a zero distance immediately disqualify the formation of a square.

Space & Time

Time Complexity: $O(1)$ (since the number of points is fixed)

Space Complexity: $O(1)$ (constant extra space used for computations)

Solution

We start by calculating the squared Euclidean distance between every pair of points to avoid the complications of floating point precision. We then count the frequency of these distances. A valid square must show exactly two distinct distances: one (side length squared) should appear 4 times and the other (diagonal length squared) should appear 2 times. This provides a robust check for both equal sides and the correct geometric property (diagonals in a square are longer than sides and appear exactly twice). The solution uses simple iteration over the fixed set of point

pairs to tally frequencies and then verifies the required counts.

Maths / Geometry

□ #611 Valid Triangle Number

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search · Greedy ·
Sorting

Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the number of triplets chosen from the array that can make triangles if we take them as side lengths of a triangle.

Example 1:

```
Input: nums = [2,2,3,4]
Output: 3
Explanation: Valid combinations are:
2,3,4 (using the first 2)
2,3,4 (using the second 2)
2,2,3
```

Example 2:

```
Input: nums = [4,2,3,4]
Output: 4
```

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def triangleNumber(self, nums: list[int]) -> int:
        ans = 0

        nums.sort()

        for k in range(len(nums) - 1, 1, -1):
            i = 0
            j = k - 1
            while i < j:
                if nums[i] + nums[j] > nums[k]:
                    ans += j - i
                    j -= 1
                else:
                    i += 1

        return ans
```

• LeetDoocs

```
class Solution:
    def triangleNumber(self, nums: List[int]) -> int:
        nums.sort()
        ans, n = 0, len(nums)
        for i in range(n - 2):
            for j in range(i + 1, n - 1):
                k = bisect_left(nums, nums[i] + nums[j], lo=j + 1) - 1
                ans += k - j
        return ans
```

• SimplyLeet

```
# Python solution for Valid Triangle Number

def triangleNumber(nums):
    # First, sort the array to facilitate the two-pointer approach
    nums.sort()
    count = 0
    n = len(nums)

    # Iterate from the end of the array towards the beginning, selecting the
    # largest side at position k
    for k in range(n - 1, 1, -1):
        left = 0          # Start pointer for the smallest side
        right = k - 1     # End pointer for the middle side

        # While the left pointer is less than the right pointer
        while left < right:
            # Check if the sum of the two smaller sides is greater than the
            # largest side
            if nums[left] + nums[right] > nums[k]:
                # All indices from left to right-1 form a valid triangle with
                # nums[right] and nums[k]
                count += right - left
                # Move the right pointer one step left to find a new pair

                right -= 1
            else:
                # Move the left pointer one step right to try and increase the
                # sum
                left += 1
        return count

# Example usage:
# input_nums = [2,2,3,4]
# print(triangleNumber(input_nums)) # Expected output: 3
```

◆ Quick Review

Key Insights

- Sorting the array simplifies meeting the triangle inequality condition.
- Once sorted, if $\text{nums}[\text{left}] + \text{nums}[\text{right}] > \text{nums}[\text{k}]$ for some triplet, then all elements between left and right with the current right will also satisfy the condition.
- Using a two-pointer approach inside a loop iterating the largest side allows us to efficiently count valid pairs.
- The innermost loop operates in $O(n)$ time leading to a quadratic overall time complexity.

Space & Time

Time Complexity: $O(n^2)$ due to the two nested loops iterating through the array.

Space Complexity: $O(1)$ or $O(\log n)$ if considering the sorting space overhead.

Solution

The solution begins by sorting the array. For each potential largest side (considered in reverse order), use two pointers (one starting at the beginning and another just before the largest side index) to find pairs of sides whose sum is greater than the largest side. When a valid pair is found, all numbers between the

two pointers with the current right pointer also form valid triangles because the array is sorted. Increment the count accordingly and adjust the pointers until all possibilities are exhausted. This greedy two-pointer approach reduces the need for a three-nested loop, ensuring an efficient solution.

Maths / Geometry

□ #939 Minimum Area Rectangle

MED

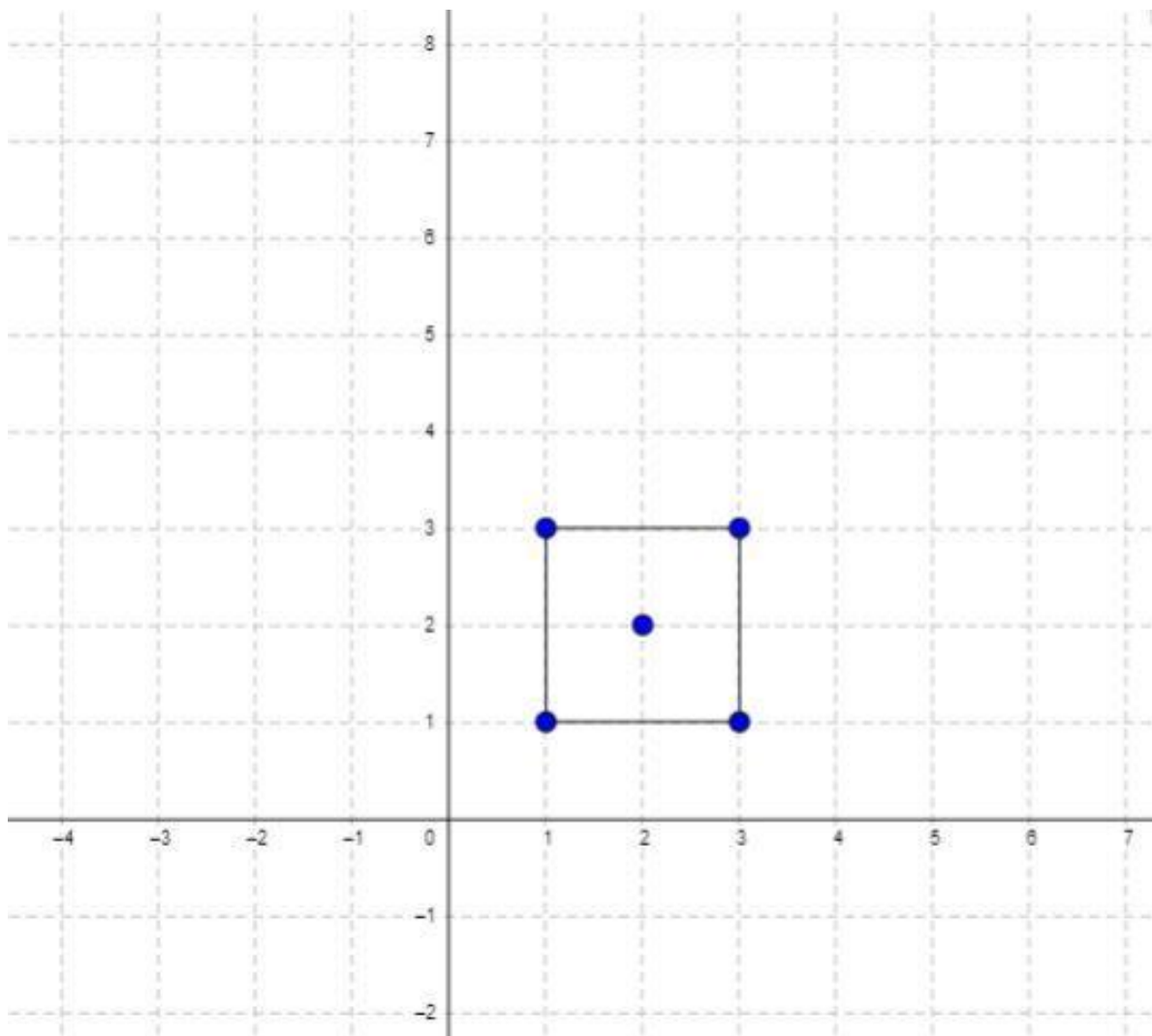
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Geometry · Sorting
Lists: AlgoMaster 600

Problem

You are given an array of points in the X-Y plane points[i] = [xi, yi].

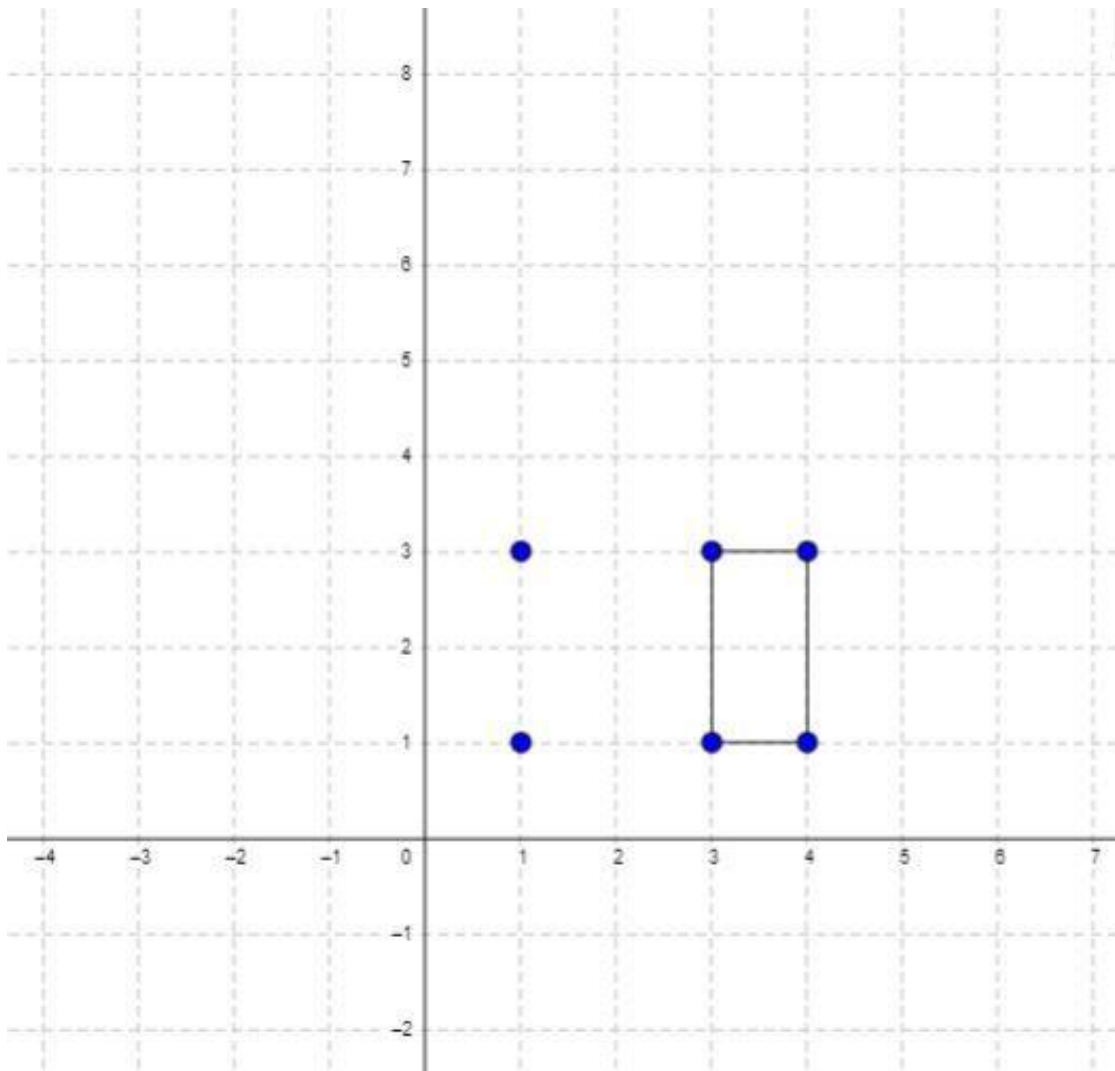
Return the minimum area of a rectangle formed from these points, with sides parallel to the X and Y axes. If there is not any such rectangle, return 0.

Example 1:



Input: points = [[1,1],[1,3],[3,1],[3,3],[2,2]]
Output: 4

Example 2:



Input: `points = [[1,1],[1,3],[3,1],[3,3],[4,1],[4,3]]`

Output: 2

Constraints:

- $1 \leq \text{points.length} \leq 500$
- $\text{points}[i].\text{length} == 2$
- $0 \leq x_i, y_i \leq 4 * 10^4$
- All the given points are unique.

Community Solutions (Python)

- WalkCC

```

class Solution:
    def minAreaRect(self, points: List[List[int]]) -> int:
        ans = math.inf
        xToYs = collections.defaultdict(set)

        for x, y in points:
            xToYs[x].add(y)

        for i in range(len(points)):
            for j in range(i):

                x1, y1 = points[i]
                x2, y2 = points[j]
                if x1 == x2 or y1 == y2:
                    continue
                if y2 in xToYs[x1] and y1 in xToYs[x2]:
                    ans = min(ans, abs(x1 - x2) * abs(y1 - y2))

        return ans if ans < math.inf else 0

```

• LeetCode

```

class Solution:
    def minAreaRect(self, points: List[List[int]]) -> int:
        d = defaultdict(list)
        for x, y in points:
            d[x].append(y)
        pos = {}
        ans = inf
        for x in sorted(d):
            ys = d[x]
            ys.sort()

            n = len(ys)
            for i, y1 in enumerate(ys):
                for y2 in ys[i + 1 :]:
                    if (y1, y2) in pos:
                        ans = min(ans, (x - pos[(y1, y2)]) * (y2 - y1))
                        pos[(y1, y2)] = x
            return 0 if ans == inf else ans

```

• SimplyLeet

```
# Python solution with step-by-step comments
class Solution:
    def minAreaRect(self, points: List[List[int]]) -> int:
        # Create a set of tuples for O(1) point lookup
        point_set = {(x, y) for x, y in points}
        n = len(points)
        min_area = float('inf')

        # Iterate through each pair of points to consider as potential
        # diagonals
        for i in range(n):
            x1, y1 = points[i]
            for j in range(i + 1, n):
                x2, y2 = points[j]
                # Check if the points are diagonally opposite (distinct x and
                # y)
                if x1 != x2 and y1 != y2:
                    # Check if the other two required corners exist
                    if (x1, y2) in point_set and (x2, y1) in point_set:
                        # Calculate the area of the rectangle
                        area = abs(x2 - x1) * abs(y2 - y1)
                        # Update the minimum area if this rectangle is smaller

                        min_area = min(min_area, area)

        # If no rectangle was found, return 0; otherwise, return the minimum
        # area
        return 0 if min_area == float('inf') else min_area
```

◆ Quick Review

Key Insights

- Use a hash set (or equivalent) to store points for O(1) lookup.
- Iterate over all pairs of points that could form opposite corners of a rectangle (ensuring

they have different x and y values).

- For each such pair, check if the other two required points exist.
- Keep track of the smallest area found; if no rectangle is detected, return 0.

Space & Time

Time Complexity: $O(n^2)$ where n is the number of points (each pair is inspected).

Space Complexity: $O(n)$ for storing the points in a hash set.

Solution

The solution involves iterating through every pair of points. For each pair with distinct x and y coordinates (potential diagonal corners of a rectangle), check if the other two corners (with swapped y and x values) exist in the set. If they do, calculate the area and update the minimum area if it is smaller than the current minimum. If no valid rectangle is found after checking all pairs, return 0.

Maths / Geometry

□ #963 Minimum Area Rectangle II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Geometry
Lists: AlgoMaster 600

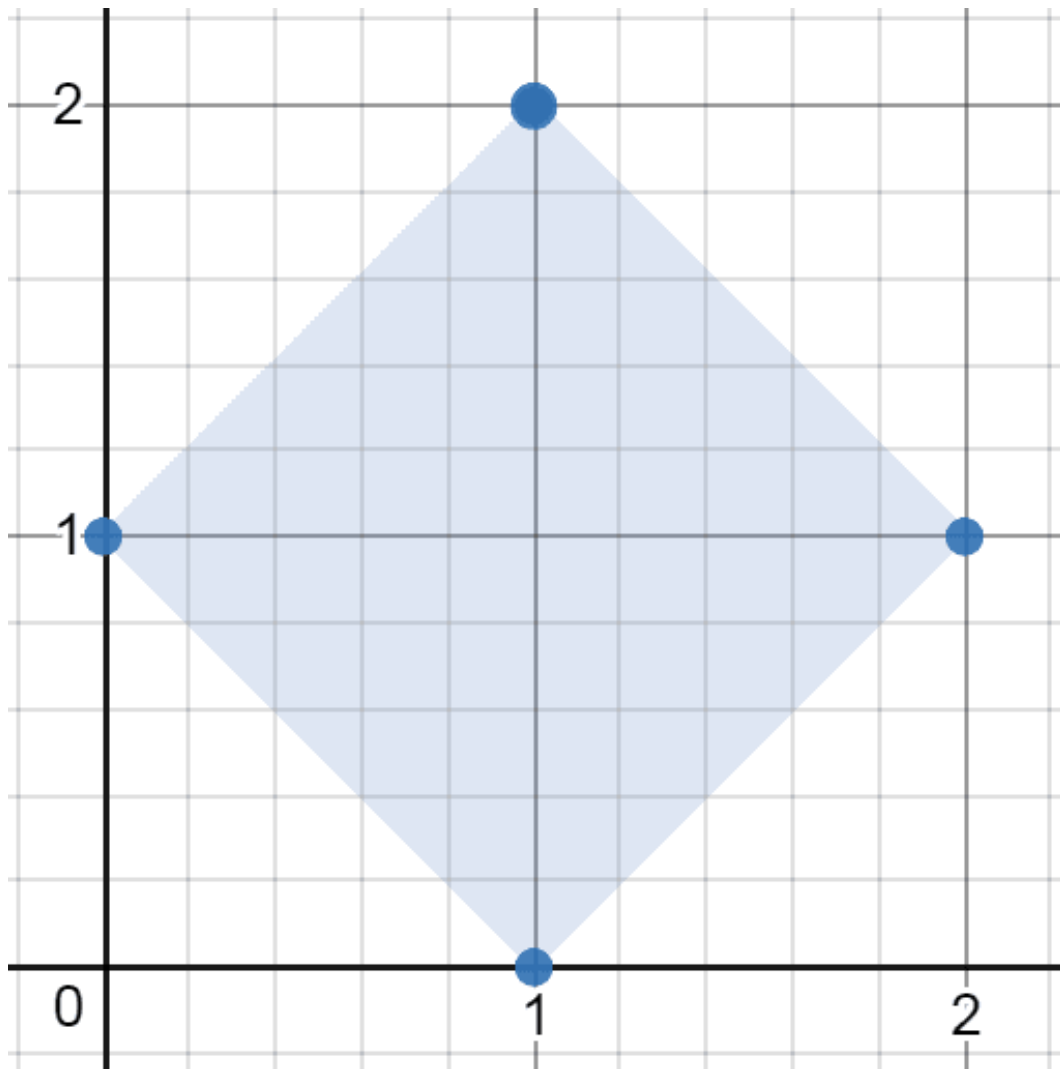
Problem

You are given an array of points in the X-Y plane points where $\text{points}[i] = [x_i, y_i]$.

Return the minimum area of any rectangle formed from these points, with sides not necessarily parallel to the X and Y axes. If there is not any such rectangle, return 0.

Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

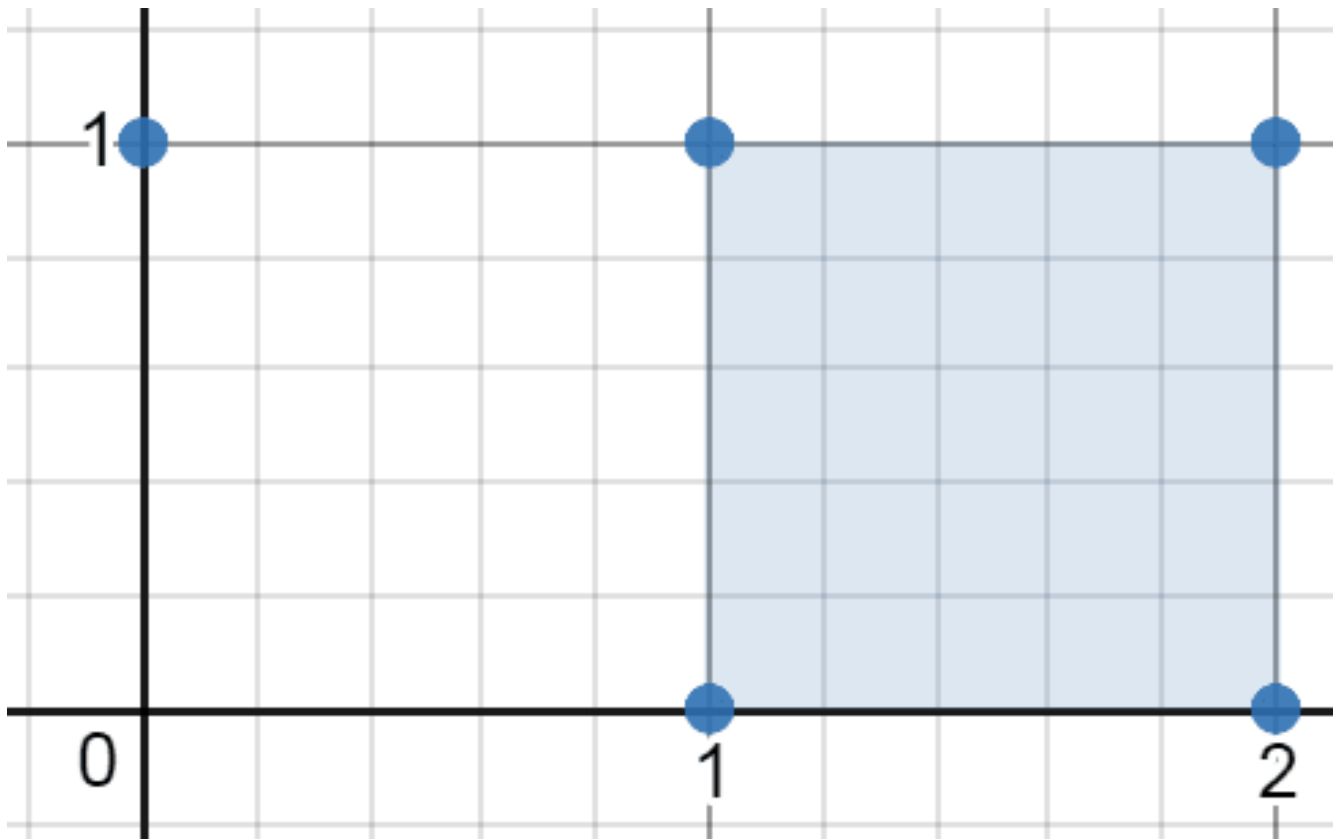


Input: points = [[1,2],[2,1],[1,0],[0,1]]

Output: 2.00000

Explanation: The minimum area rectangle occurs at [1,2],[2,1],[1,0],[0,1], with an area of 2.

Example 2:

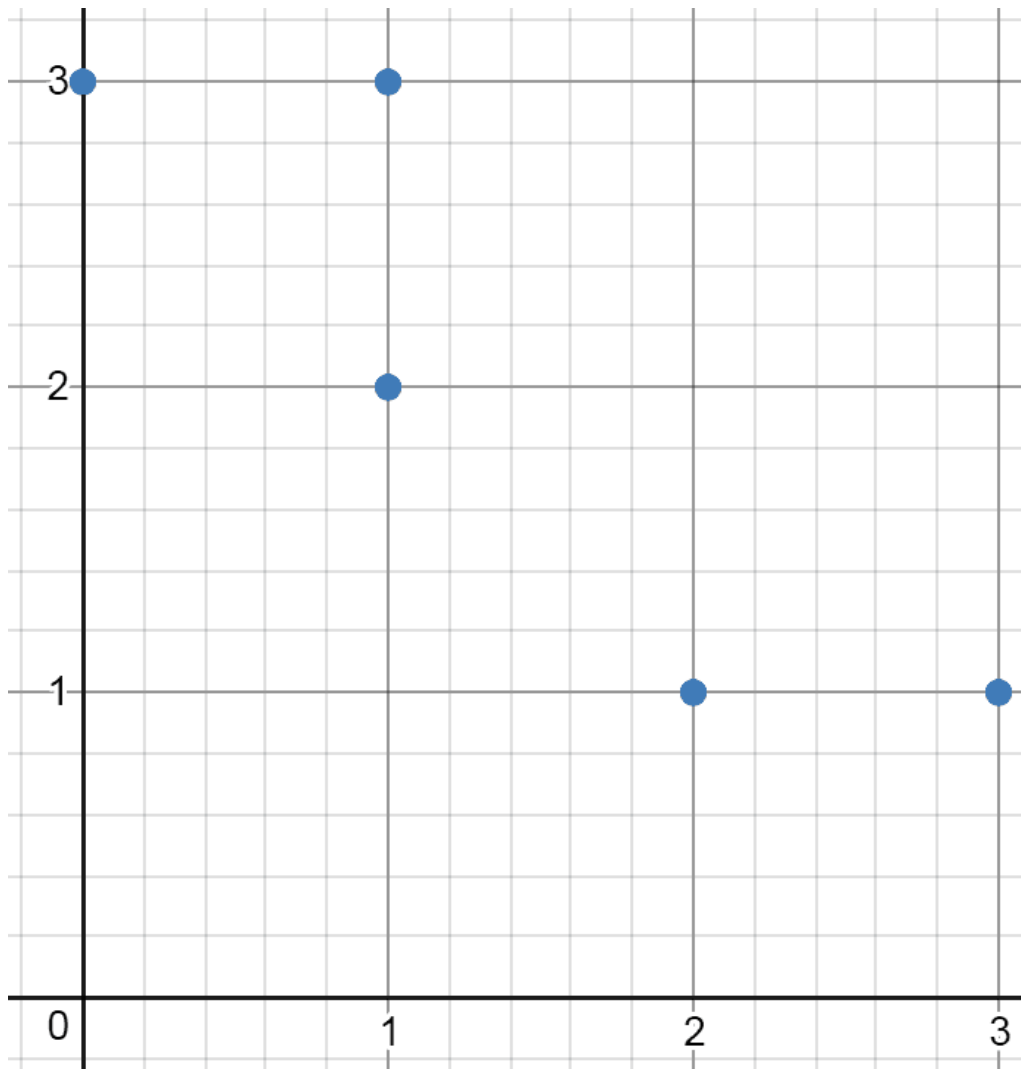


Input: points = [[0,1],[2,1],[1,1],[1,0],[2,0]]

Output: 1.00000

Explanation: The minimum area rectangle occurs at [1,0],[1,1],[2,1],[2,0], with an area of 1.

Example 3:



Input: points = [[0,3],[1,2],[3,1],[1,3],[2,1]]

Output: 0

Explanation: There is no possible rectangle to form from these points.

Constraints:

- $1 \leq \text{points.length} \leq 50$
- $\text{points}[i].\text{length} == 2$
- $0 \leq x_i, y_i \leq 4 * 10^4$
- All the given points are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minAreaFreeRect(self, points: list[list[int]]) -> float:
        ans = math.inf
        # For each A, B pair points, {hash(A, B): (ax, ay, bx, by)}.
        centerToPoints = collections.defaultdict(list)

        for ax, ay in points:
            for bx, by in points:
                center = ((ax + bx) / 2, (ay + by) / 2)
                centerToPoints[center].append((ax, ay, bx, by))

        def dist(px: int, py: int, qx: int, qy: int) -> float:
            return (px - qx)**2 + (py - qy)**2

        # For all pair points "that share the same center".
        for points in centerToPoints.values():
            for ax, ay, _, _ in points:
                for cx, cy, dx, dy in points:
                    # AC is perpendicular to AD.
                    # AC dot AD = (cx - ax, cy - ay) dot (dx - ax, dy - ay) == 0.

                    if (cx - ax) * (dx - ax) + (cy - ay) * (dy - ay) == 0:
                        squaredArea = dist(ax, ay, cx, cy) * dist(ax, ay, dx, dy)
                        if squaredArea > 0:
                            ans = min(ans, squaredArea)

        return 0 if ans == math.inf else math.sqrt(ans)
```

• LeetDoocs

```
class Solution:
    def minAreaFreeRect(self, points: List[List[int]]) -> float:
        s = {(x, y) for x, y in points}
        n = len(points)
        ans = inf
        for i in range(n):
            x1, y1 = points[i]
            for j in range(n):
                if j != i:
                    x2, y2 = points[j]
```

```

        for k in range(j + 1, n):
            if k != i:
                x3, y3 = points[k]
                x4 = x2 - x1 + x3
                y4 = y2 - y1 + y3
                if (x4, y4) in s:
                    v21 = (x2 - x1, y2 - y1)
                    v31 = (x3 - x1, y3 - y1)
                    if v21[0] * v31[0] + v21[1] * v31[1] == 0:
                        w = sqrt(v21[0] ** 2 + v21[1] ** 2)

                    h = sqrt(v31[0] ** 2 + v31[1] ** 2)
                    ans = min(ans, w * h)

    return 0 if ans == inf else ans

```

• SimplyLeet

```

import math
from collections import defaultdict

def minAreaFreeRect(points):
    # Number of points
    n = len(points)
    # Dictionary to group pairs by (center, squared distance)
    diag_groups = defaultdict(list)

    # Iterate over each unique pair of points
    for i in range(n):
        for j in range(i + 1, n):
            # Compute the center as the sum of coordinates to avoid fractions
            center = (points[i][0] + points[j][0], points[i][1] + points[j][1])

            # Compute the squared distance between the two points (diagonal
            # length squared)
            dx = points[i][0] - points[j][0]
            dy = points[i][1] - points[j][1]
            dist2 = dx*dx + dy*dy
            # Append the pair (points[i], points[j]) to the corresponding group
            key = (center, dist2)
            diag_groups[key].append((points[i], points[j]))

```

```

min_area = float('inf')
# For every group of pairs sharing same center and squared distance, they
# form potential rectangles
for pairs in diag_groups.values():
    if len(pairs) < 2:
        continue
    # Compare every combination of two pairs from this group
    m = len(pairs)
    for i in range(m):
        for j in range(i + 1, m):

            p, q = pairs[i]
            r, s = pairs[j]
            # For rectangle, choose p as one vertex.
            # Determine vectors from p to r and p to s.
            vec_pr = (r[0] - p[0], r[1] - p[1])
            vec_ps = (s[0] - p[0], s[1] - p[1])
            # Check perpendicularity: (vec_pr) dot (vec_ps) should be zero.
            if vec_pr[0] * vec_ps[0] + vec_pr[1] * vec_ps[1] != 0:
                # If not perpendicular, try swapping r and s.
                vec_pr = (s[0] - p[0], s[1] - p[1])

            vec_ps = (r[0] - p[0], r[1] - p[1])
            if vec_pr[0] * vec_ps[0] + vec_pr[1] * vec_ps[1] != 0:
                continue # Not perpendicular; skip
            # Compute lengths of the two sides
            side1 = math.sqrt(vec_pr[0]*vec_pr[0] + vec_pr[1]*vec_pr[1])
            side2 = math.sqrt(vec_ps[0]*vec_ps[0] + vec_ps[1]*vec_ps[1])
            area = side1 * side2
            # Update the minimum area if a smaller valid rectangle is found
            if area < min_area:
                min_area = area

return 0 if min_area == float('inf') else min_area

# Example usage:
# print(minAreaFreeRect([[1,2],[2,1],[1,0],[0,1]]))

```

◆ Quick Review

Key Insights

- A rectangle, even when rotated, has the property that its diagonals have the same midpoint and equal lengths.
- By iterating over every pair of points (which can be potential diagonal endpoints), we can group pairs by their midpoint (represented in a way to avoid floating point issues) and their squared distance.
- For each group of pairs sharing the same center and squared distance, any two pairs can potentially form a rectangle.
- Given two candidate diagonal pairs (p, q) and (r, s) with the same midpoint, the rectangle's area can be computed by taking one vertex p and calculating:
 - $\text{side1} = \text{distance from } p \text{ to } r$
 - $\text{side2} = \text{distance from } p \text{ to } s$
 - $\text{area} = \text{side1} * \text{side2}$, provided that the vectors $(r - p)$ and $(s - p)$ are perpendicular (i.e. their dot product is zero).
- Iterate through all valid pair combinations and keep track of the minimum area found.

Space & Time

Time Complexity: $O(n^2 * k^2)$ where n is the number of points and k is the number of pairs in each grouping (in worst-case, overall it is $O(n^4)$, but since $n \leq 50$, it is acceptable).

Space Complexity: $O(n^2)$ for storing all pairs in the dictionary.

Solution

The solution involves:

- Iterating over all pairs of points and computing a key based on the pair's midpoint (using the sum of coordinates to avoid floating point precision issues) and their squared distance.
- Storing these pairs in a hash map (dictionary) keyed by (center, squared distance).
- For each list in the dictionary with more than one pair, checking every combination of two pairs:
 - Choose one endpoint from the first pair and check with both endpoints of the second pair to form two vectors.
 - Verify that these two vectors are perpendicular (dot product equals 0). This

confirms the points form a rectangle.

- Compute the area of the rectangle as the product of the lengths of the two vectors.**
- Return the minimum area among all valid rectangles, or 0 if none exist.**

Below are code solutions in multiple languages with detailed line-by-line comments.

Maths / Geometry

□ #1922 Count Good Numbers

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Recursion

Lists: AlgoMaster 600

Problem

A digit string is good if the digits (0-indexed) at even indices are even and the digits at odd indices are prime (2, 3, 5, or 7).

- For example, "2582" is good because the digits (2 and 8) at even positions are even and the digits (5 and 2) at odd positions are prime. However, "3245" is not good because 3 is at an even index but is not even.

Given an integer n , return the total number of good digit strings of length n . Since the answer may be large, return it modulo $10^9 + 7$.

A digit string is a string consisting of digits 0 through 9 that may contain leading zeros.

Example 1:

Input: n = 1

Output: 5

Explanation: The good numbers of length 1 are "0", "2", "4", "6", "8".

Example 2:

Input: n = 4

Output: 400

Example 3:

Input: n = 50

Output: 564908303

Constraints:

- $1 \leq n \leq 1015$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countGoodNumbers(self, n: int) -> int:
        MOD = 1_000_000_007

        def modPow(x: int, n: int) -> int:
            if n == 0:
                return 1
            if n % 2 == 1:
                return x * modPow(x, n - 1) % MOD
            return modPow(x * x % MOD, n // 2)

        return modPow(4 * 5, n // 2) * (1 if n % 2 == 0 else 5) % MOD
```

• LeetDoocs

```
class Solution:
    def countGoodNumbers(self, n: int) -> int:
        mod = 10**9 + 7
        return pow(5, (n + 1) >> 1, mod) * pow(4, n >> 1, mod) % mod
```

• SimplyLeet

```
# Function to perform fast modular exponentiation
def mod_pow(base, exp, mod):
    result = 1
    while exp > 0:
        # If the current exponent is odd, multiply the result by base mod mod
        if exp % 2:
            result = (result * base) % mod
        # Square the base and reduce the exponent by half
        base = (base * base) % mod
        exp //= 2

    return result

def countGoodNumbers(n):
    mod = 10**9 + 7
    # Calculate the count of indices for even and odd positions
    even_count = (n + 1) // 2 # ceil(n/2)
    odd_count = n // 2        # floor(n/2)
    # Compute the result using modular exponentiation for both parts
    result = (mod_pow(5, even_count, mod) * mod_pow(4, odd_count, mod)) % mod
    return result

# For example: n = 4 should output 400
print(countGoodNumbers(4))
```

◆ Quick Review

Key Insights

- The string has n positions, where positions are 0-indexed.
- Even indices must contain one of 5 even digits {0, 2, 4, 6, 8}.

- Odd indices must contain one of 4 prime digits {2, 3, 5, 7}.
- Count the number of even index positions as $\text{ceil}(n / 2)$ and odd index positions as $\text{floor}(n / 2)$.
- The total good numbers equals $5^{\text{ceil}(n / 2)} * 4^{\text{floor}(n / 2)} \text{ modulo } 10^9 + 7$.
- Use fast modular exponentiation due to the large constraint on n (up to 10^{15}).

Space & Time

Time Complexity: $O(\log n)$ due to fast modular exponentiation.

Space Complexity: $O(1)$ as only a fixed amount of memory is used.

Solution

We determine the count of good digit strings by dividing the positions into even and odd index positions. Even indexes must be one of the 5 even digits, so they contribute 5 possibilities each; odd indexes must be one of the 4 prime digits, and they contribute 4 possibilities each. If n is the length, then:

- Number of even indexed positions = $\text{ceil}(n / 2)$

– Number of odd indexed positions = $\text{floor}(n / 2)$

Thus, the total number of good strings is:
 $5^{(\text{ceil}(n / 2))} * 4^{(\text{floor}(n / 2))} \text{ modulo } (10^9 + 7)$.

To compute these large exponents efficiently, we use fast modular exponentiation (also known as modular power). This ensures that even for n as large as 10^{15} , the exponents are calculated quickly.

The provided code solutions in Python, JavaScript, C++, and Java include detailed inline comments to clarify each step.

String Matching

Round 8 · Low · Medium · 2 questions

String Matching

□ #686 Repeated String Match

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · String Matching
Lists: AlgoMaster 600

Problem

Given two strings *a* and *b*, return the minimum number of times you should repeat string *a* so that string *b* is a substring of it. If it is impossible for *b* to be a substring of *a* after repeating it, return -1 .

Notice: string "abc" repeated 0 times is "", repeated 1 time is "abc" and repeated 2 times is "abcabc".

Example 1:

```
Input: a = "abcd", b = "cdabcdab"  
Output: 3  
Explanation: We return 3 because by repeating a three times "abcdabcdabcd", b  
is a substring of it.
```

Example 2:

```
Input: a = "a", b = "aa"  
Output: 2
```

Constraints:

- $1 \leq a.length, b.length \leq 104$
- a and b consist of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def repeatedStringMatch(self, a: str, b: str) -> int:
        n = math.ceil(len(b) / len(a))
        s = a * n
        if b in s:
            return n
        if b in s + a:
            return n + 1
        return -1
```

• LeetDoocs

```
class Solution:
    def repeatedStringMatch(self, a: str, b: str) -> int:
        m, n = len(a), len(b)
        ans = ceil(n / m)
        t = [a] * ans
        for _ in range(3):
            if b in ''.join(t):
                return ans
            ans += 1
            t.append(a)

        return -1
```

• SimplyLeet

```
# Function to determine the minimum number of times 'a' must be repeated
# so that 'b' is a substring of the repeated string.
def repeatedStringMatch(a, b):
    # Calculate the minimum repeats required using ceiling division
    repeat_count = -(-len(b) // len(a)) # This is equivalent to
    math.ceil(len(b) / len(a))
    # Build the repeated string
    repeated = a * repeat_count
    # Check if b is a substring of the repeated string
    if b in repeated:
        return repeat_count

    # Check with one additional repeat to handle potential overlap between
    # repeats
    if b in (repeated + a):
        return repeat_count + 1
    return -1

# Example usage:
print(repeatedStringMatch("abcd", "cdabcdab")) # Expected output: 3
```

◆ Quick Review

Key Insights

- The key observation is that b will be found within the repeated string a after a sufficient number of repeats, possibly spanning the junction between two copies of a .
- The initial number of repeats can be approximated by $\text{ceiling}(|b| / |a|)$. To handle potential overlaps at the boundaries, one additional repetition might be necessary.

- A simple check using built-in substring methods is sufficient to verify if b is contained in the repeated string.

Space & Time

Time Complexity: $O(n * k)$, where n is the length of a and k is the minimum number of repeats (usually a small constant in practice).

Space Complexity: $O(n * k)$, which is the space required to construct the repeated string.

Solution

The solution calculates an initial repeat count using the ceiling of the division of b 's length by a 's length. It then constructs a repeated version of a and checks if b is a substring. Because b could span across the boundary of two adjacent a 's, the solution appends one more repetition of a if necessary. If b is still not found, return -1 .

String Matching

□ #1698 Number of Distinct Substrings in a String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Trie · Rolling Hash · Suffix Array ·
Hash Function

Lists: AlgoMaster 600

Problem

Given a string s , return the number of distinct substrings of s .

A substring of a string is obtained by deleting any number of characters (possibly zero) from the front of the string and any number (possibly zero) from the back of the string.

Example 1:

Input: $s = \text{"aabbaba"}$

Output: 21

Explanation: The set of distinct strings is $[\text{"a"}, \text{"b"}, \text{"aa"}, \text{"bb"}, \text{"ab"}, \text{"ba"}, \text{"aab"}, \text{"abb"}, \text{"bab"}, \text{"bba"}, \text{"aba"}, \text{"aabb"}, \text{"abba"}, \text{"bbab"}, \text{"baba"}, \text{"aabba"}, \text{"abbab"}, \text{"bbaba"}, \text{"aabbab"}, \text{"abbaba"}, \text{"aabbaba"}]$

Example 2:

Input: $s = \text{"abcdefg"}$

Output: 28

Constraints:

- $1 \leq s.length \leq 500$
- s consists of lowercase English letters.

Follow up: Can you solve this problem in $O(n)$ time complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countDistinct(self, s: str) -> int:
        BASE = 26
        HASH = 1_000_000_007

        n = len(s)
        ans = 0
        pow = [1] + [0] * n      # pow[i] := BASE^i
        hashes = [0] * (n + 1)  # hashes[i] := the hash of s[0..i]

        def val(c: str) -> int:
            return ord(c) - ord('a')

        for i in range(1, n + 1):
            pow[i] = pow[i - 1] * BASE % HASH
            hashes[i] = (hashes[i - 1] * BASE + val(s[i - 1])) % HASH

        def getHash(l: int, r: int) -> int:
            """Returns the hash of s[l..r)."""
            hash = (hashes[r] - hashes[l] * pow[r - l]) % HASH

            return hash + HASH if hash < 0 else hash

        for length in range(1, n + 1):
            seen = set()
            for i in range(n - length + 1):
                seen.add(getHash(i, i + length))
            ans += len(seen)

        return ans
```

• LeetDoocs

```
class Solution:
    def countDistinct(self, s: str) -> int:
        n = len(s)
        return len({s[i:j] for i in range(n) for j in range(i + 1, n + 1)})
```

```
class Solution:
    def countDistinct(self, s: str) -> int:
        base = 131
        n = len(s)
        p = [0] * (n + 10)
        h = [0] * (n + 10)
        p[0] = 1
        for i, c in enumerate(s):
            p[i + 1] = p[i] * base
            h[i + 1] = h[i] * base + ord(c)
```

```
ss = set()
for i in range(1, n + 1):
    for j in range(i, n + 1):
        t = h[j] - h[i - 1] * p[j - i + 1]
        ss.add(t)
return len(ss)
```

• SimplyLeet

```
# Function to count distinct substrings in a string
def countDistinctSubstrings(s):
    # Initialize a set to store unique substrings
    unique_subs = set()
    n = len(s)
    # Loop over all possible starting indices
    for i in range(n):
        # Loop over all possible ending indices (non-inclusive)
        for j in range(i + 1, n + 1):
            # Add the current substring to the set
```

```
        unique_subs.add(s[i:j])
    # Return the count of unique substrings
    return len(unique_subs)

# Example usage:
s = "aabbaba"
print(countDistinctSubstrings(s)) # Expected output: 21
```

◆ Quick Review

Key Insights

- All possible substrings can be generated by considering each possible start and end index.
- Utilizing a set (or hash set) ensures that duplicates are automatically filtered.
- Even though generating all substrings has a nested loop, the maximum length of s (500) keeps the solution efficient for the problem constraints.
- Advanced methods (like suffix trees, suffix arrays, or rolling hash) can optimize the process closer to $O(n)$ time but are more complex to implement.

Space & Time

Time Complexity: $O(n^2)$ in the context of iterating over all substring intervals (n is the length of s).

Space Complexity: $O(n^2)$ in the worst case for storing all distinct substrings.

Solution

The solution involves iterating through every possible substring of s by using two nested loops: one for the start index and another for the end index. Each generated substring is added to a set which handles duplicate removals automatically. After the loops, the size of the set gives the number of distinct substrings.

Key Data Structures and Techniques:

- Set/Hash Set: to store distinct substrings.**
- Nested Loop: to generate all possible substrings.**
- (Optional) Advanced techniques like rolling hash and suffix arrays can optimize the solution, but the simple approach is sufficient given the constraints.**

Binary Indexed Tree / Segment Tree

Round 8 · Low · Medium · 1 question

Binary Indexed Tree / Segment Tree

□ #308 Range Sum Query 2D – Mutable

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Design · Binary Indexed Tree · Segment
Tree · Matrix

Lists: AlgoMaster 600

Problem

Given a 2D matrix `matrix`, handle multiple queries of the following types:

1. Update the value of a cell in matrix.
2. Calculate the sum of the elements of matrix inside the rectangle defined by its upper left corner `(row1, col1)` and lower right corner `(row2, col2)`.

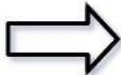
Implement the `NumMatrix` class:

- `NumMatrix(int[][] matrix)` Initializes the object with the integer matrix `matrix`.
- `void update(int row, int col, int val)` Updates the value of `matrix[row][col]` to be `val`.
- `int sumRegion(int row1, int col1, int row2, int col2)` Returns the sum of the elements of

matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

Example 1:

3	0	1	4	2
5	6	3	2	1
1	2	0	1	5
4	1	0	1	7
1	0	3	0	5



3	0	1	4	2
5	6	3	2	1
1	2	0	1	5
4	1	2	1	7
1	0	3	0	5

Input

```
["NumMatrix", "sumRegion", "update", "sumRegion"]
[[[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]], [2, 1, 4, 3], [3, 2, 2], [2, 1, 4, 3]]
```

Output

```
[null, 8, null, 10]
```

Explanation

```
NumMatrix numMatrix = new NumMatrix([[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]);
```

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 200$
- $-1000 \leq \text{matrix}[i][j] \leq 1000$
- $0 \leq \text{row} < m$

- $0 \leq \text{col} < n$
- $-1000 \leq \text{val} \leq 1000$
- $0 \leq \text{row1} \leq \text{row2} < m$
- $0 \leq \text{col1} \leq \text{col2} < n$
- At most 5000 calls will be made to `sumRegion` and `update`.

Community Solutions (Python)

• WalkCC

```
class FenwickTree:
    def __init__(self, m: int, n: int):
        self.sums = [[0] * (n + 1) for _ in range(m + 1)]

    def add(self, row: int, col: int, delta: int) -> None:
        i = row
        while i < len(self.sums):
            j = col
            while j < len(self.sums[0]):
                self.sums[i][j] += delta

                j += FenwickTree.lowbit(j)
            i += FenwickTree.lowbit(i)

    def get(self, row: int, col: int) -> int:
        summ = 0
        i = row
        while i > 0:
            j = col
            while j > 0:
                summ += self.sums[i][j]
```

```

        j -= FenwickTree.lowbit(j)
        i -= FenwickTree.lowbit(i)
        return summ

    @staticmethod
    def lowbit(i: int) -> int:
        return i & -i

class NumMatrix:

    def __init__(self, matrix: list[list[int]]):
        self.matrix = matrix
        self.tree = FenwickTree(len(matrix), len(matrix[0]))

        for i in range(len(matrix)):
            for j, val in enumerate(matrix[i]):
                self.tree.add(i + 1, j + 1, val)

    def update(self, row: int, col: int, val: int) -> None:
        self.tree.add(row + 1, col + 1, val - self.matrix[row][col])

        self.matrix[row][col] = val

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
        return (self.tree.get(row2 + 1, col2 + 1) - self.tree.get(row1, col2 + 1)
-           self.tree.get(row2 + 1, col1) + self.tree.get(row1, col1))

```

• LeetDoocs

```

class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):

```

```

        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s

```

```

class NumMatrix:
    def __init__(self, matrix: List[List[int]]):
        self.trees = []
        n = len(matrix[0])
        for row in matrix:
            tree = BinaryIndexedTree(n)
            for j, v in enumerate(row):
                tree.update(j + 1, v)

            self.trees.append(tree)

    def update(self, row: int, col: int, val: int) -> None:
        tree = self.trees[row]
        prev = tree.query(col + 1) - tree.query(col)
        tree.update(col + 1, val - prev)

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
        return sum(
            tree.query(col2 + 1) - tree.query(col1)

            for tree in self.trees[row1 : row2 + 1]
        )

```

```

# Your NumMatrix object will be instantiated and called as such:
# obj = NumMatrix(matrix)
# obj.update(row,col,val)
# param_2 = obj.sumRegion(row1,col1,row2,col2)

```

```
class Node:
    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0

class SegmentTree:
    def __init__(self, nums):
        n = len(nums)

        self.nums = nums
        self.tr = [Node() for _ in range(4 * n)]
        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l = l
        self.tr[u].r = r
        if l == r:
            self.tr[u].v = self.nums[l - 1]
            return

        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)
        self.pushup(u)

    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].v = v
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1

        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)
        self.pushup(u)

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
```



```

        v = 0
        if l <= mid:
            v += self.query(u << 1, l, r)
        if r > mid:
            v += self.query(u << 1 | 1, l, r)
        return v

    def pushup(self, u):
        self.tr[u].v = self.tr[u << 1].v + self.tr[u << 1 | 1].v

```

```

class NumMatrix:
    def __init__(self, matrix: List[List[int]]):
        self.trees = [SegmentTree(row) for row in matrix]

    def update(self, row: int, col: int, val: int) -> None:
        tree = self.trees[row]
        tree.modify(1, col + 1, val)

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:

        return sum(
            self.trees[row].query(1, col1 + 1, col2 + 1)
            for row in range(row1, row2 + 1)
        )

```

```

# Your NumMatrix object will be instantiated and called as such:
# obj = NumMatrix(matrix)
# obj.update(row,col,val)
# param_2 = obj.sumRegion(row1,col1,row2,col2)

```

• SimplyLeet

```

class NumMatrix:
    def __init__(self, matrix):
        # initialize dimensions and store given matrix copy
        if not matrix or not matrix[0]:
            return
        self.m = len(matrix)
        self.n = len(matrix[0])
        self.matrix = [[0] * self.n for _ in range(self.m)]
        # BIT is 1-indexed so allocate with extra row and col
        self.bit = [[0] * (self.n + 1) for _ in range(self.m + 1)]

        # Initialize BIT with the matrix values
        for i in range(self.m):
            for j in range(self.n):
                self.update(i, j, matrix[i][j])

    def update(self, row, col, val):
        # find the difference from the current value
        if self.m == 0 or self.n == 0:
            return
        diff = val - self.matrix[row][col]

        self.matrix[row][col] = val
        # update BIT for all indexes that include cell (row, col)
        i = row + 1
        while i <= self.m:
            j = col + 1
            while j <= self.n:
                self.bit[i][j] += diff
                # j moves to next index responsible for including col
                j += (j & -j)
            i += (i & -i)

    def _bitSum(self, row, col):
        # Compute prefix sum from BIT for submatrix (0,0) to (row, col)
        s = 0
        i = row
        while i > 0:
            j = col
            while j > 0:
                s += self.bit[i][j]
                j -= (j & -j)

```

```

        i -= (i & -i)
    return s

def sumRegion(self, row1, col1, row2, col2):
    # Use inclusion-exclusion principle to get region sum
    sum_total = self._bitSum(row2 + 1, col2 + 1)
    sum_top = self._bitSum(row1, col2 + 1)
    sum_left = self._bitSum(row2 + 1, col1)
    sum_corner = self._bitSum(row1, col1)
    return sum_total - sum_top - sum_left + sum_corner

```

◆ Quick Review

Key Insights

- This is a 2D range sum query problem with updates.
- A brute-force approach may be too slow due to frequent updates.
- The Binary Indexed Tree (Fenwick Tree) or 2D Segment Tree can support both update and sumRegion queries efficiently.
- Use an auxiliary BIT structure to maintain cumulative information to quickly answer sum queries.
- Maintain a separate copy of the original matrix to compute differences during updates.

Space & Time

Time Complexity:

- update: $O(M \cdot \log(M) \cdot \log(N))$ in worst-case per update, where M and N are the dimensions of the matrix.
 - sumRegion: $O(\log(M) \cdot \log(N))$ per query.
- Space Complexity:
- $O(M \cdot N)$ for storing the BIT and the matrix.

Solution

We implement a 2D Binary Indexed Tree. When performing an update, compute the difference between the new value and the current value of the cell, and update the BIT accordingly. For sumRegion queries, use the inclusion-exclusion principle on prefix sums from the BIT. This allows both updates and sum queries to be done in logarithmic time relative to the dimensions of the matrix. The BIT is a 2D array where each cell $BIT[i][j]$ contributes to the cumulative sum over a specific range determined by the least significant bit. By iterating properly within the BIT update and query functions, we can ensure that all necessary BIT cells are updated or summed.

Round 9

Low Priority · Hard

36 questions · 15 patterns

- ☐ ● Bucket Sort #220 ↗
- ☐ ● Eulerian Circuit #753 #2097 ↗
- ☐ ● 1-D DP #1411 ↗
- ☐ ● 0/1 Knapsack #879 ↗
- ☐ ● Longest Increasing Subsequence (LIS) #354 ↗
- ☐ ● 2D Grid DP #85 #174 #403 #465 #546 #741 ↗
- ☐ ● #1335 #1547 #1931 #3651 ↗
- ☐ ● String DP #44 #140 #664 #1092 ↗
- ☐ ● Tree / Graph DP #834 #968 ↗
- ☐ ● Bitmask DP #847 ↗
- ☐ ● Digit DP #233 #902 ↗
- ☐ ● State Machine DP #188 ↗
- ☐ ● Maths / Geometry #149 ↗

- ☐ ● String Matching #214 #1044 #1392 ↗
- ☐ ● Binary Indexed Tree / Segment Tree #699 #2426 ↗
- ☐ ● #3161 #3721 ↗
- ☐ ● Line Sweep #391 #850 ↗

Bucket Sort

Round 9 · Low · Hard · 1 question

Bucket Sort

□ #220 Contains Duplicate III

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sliding Window · Sorting · Bucket Sort ·
Ordered Set

Lists: AlgoMaster 600

Problem

You are given an integer array `nums` and two integers `indexDiff` and `valueDiff`.

Find a pair of indices (i, j) such that:

- $i \neq j$,
- $\text{abs}(i - j) \leq \text{indexDiff}$.
- $\text{abs}(\text{nums}[i] - \text{nums}[j]) \leq \text{valueDiff}$, and

Return `true` if such pair exists or `false` otherwise.

Example 1:

```
Input: nums = [1,2,3,1], indexDiff = 3, valueDiff = 0
Output: true
Explanation: We can choose (i, j) = (0, 3).
We satisfy the three conditions:
i != j --> 0 != 3
abs(i - j) <= indexDiff --> abs(0 - 3) <= 3
abs(nums[i] - nums[j]) <= valueDiff --> abs(1 - 1) <= 0
```

Example 2:

Input: nums = [1,5,9,1,5,9], indexDiff = 2, valueDiff = 3

Output: false

Explanation: After trying all the possible pairs (i, j), we cannot satisfy the three conditions, so we return false.

Constraints:

- $2 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$
- $1 \leq \text{indexDiff} \leq \text{nums.length}$
- $0 \leq \text{valueDiff} \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def containsNearbyAlmostDuplicate(
        self,
        nums: list[int],
        indexDiff: int,
        valueDiff: int,
    ) -> bool:
        if not nums or indexDiff <= 0 or valueDiff < 0:
            return False

        mn = min(nums)
        diff = valueDiff + 1 # In case that `valueDiff` equals 0.
        bucket = {}

        def getKey(num: int) -> int:
            return (num - mn) // diff

        for i, num in enumerate(nums):
            key = getKey(num)
            if key in bucket: # the current bucket
```

```

        return True
    # the left adjacent bucket
    if key - 1 in bucket and num - bucket[key - 1] < diff:
        return True
    # the right adjacent bucket
    if key + 1 in bucket and bucket[key + 1] - num < diff:
        return True
    bucket[key] = num
    if i >= indexDiff:
        del bucket[getKey(nums[i - indexDiff])]

return False

```

• LeetCode

```

class Solution:
    def containsNearbyAlmostDuplicate(
        self, nums: List[int], indexDiff: int, valueDiff: int
    ) -> bool:
        s = SortedSet()
        for i, v in enumerate(nums):
            j = s.bisect_left(v - valueDiff)
            if j < len(s) and s[j] <= v + valueDiff:
                return True
            s.add(v)

        if i >= indexDiff:
            s.remove(nums[i - indexDiff])
        return False

```

• SimplyLeet

```
class Solution:
    def containsNearbyAlmostDuplicate(self, nums, indexDiff, valueDiff):
        # Early return; bucket size is valueDiff + 1 to avoid division by zero
        # when valueDiff is 0
        if valueDiff < 0:
            return False

        bucketSize = valueDiff + 1 # Bucket size ensures closer numbers fall
        # in same or adjacent buckets
        buckets = {} # Map bucket id to the number in that bucket

        for i, num in enumerate(nums):

            # Compute bucket id. For negative values, adjust bucket
            # calculation.
            bucketId = num // bucketSize if num >= 0 else ((num + 1) // bucket
            # Size - 1)

            # Check if current bucket already contains an element
            if bucketId in buckets:
                return True

            # Check the left adjacent bucket
            if (bucketId - 1 in buckets and abs(num - buckets[bucketId - 1]) <
            bucketSize):
                return True

            # Check the right adjacent bucket
            if (bucketId + 1 in buckets and abs(num - buckets[bucketId + 1]) <
            bucketSize):
                return True

            # Insert the current number into its bucket
            buckets[bucketId] = num

            # Maintain the sliding window of size indexDiff
            if i >= indexDiff:
```

```
# Compute bucket id for the element to be removed
removeNum = nums[i - indexDiff]
removeBucketId = removeNum // bucketSize if removeNum >= 0 else
e ((removeNum + 1) // bucketSize - 1)
del buckets[removeBucketId]

return False
```

◆ Quick Review

Key Insights

- Use a sliding window of size `indexDiff` to ensure the index condition.
- To efficiently check the value condition, maintain a data structure with near-ordering.
- Bucket sort (or using an ordered set / BST) can help check for any element within `[num - valueDiff, num + valueDiff]` in $O(1)$ average time.
- Use a bucket size of `(valueDiff + 1)` to map numbers to buckets. Two numbers in the same or adjacent buckets may satisfy the condition.

Space & Time

Time Complexity: $O(n)$ on average (each element is processed once and bucket operations take constant time on average).

Space Complexity: $O(\min(n, \text{indexDiff}))$ since we only store the current sliding window's elements.

Solution

The solution leverages bucket sort along with a sliding window. We use a hash map to maintain buckets, where each bucket size is defined as $(\text{valueDiff} + 1)$. For each element:

- Compute its bucket ID.
- Check if this bucket already has a number (this guarantees the value difference is within valueDiff).
- Check the adjacent buckets ($\text{bucket}-1$ and $\text{bucket}+1$) for potential candidates.
- Insert the current number into its bucket.
- Once the sliding window exceeds the size indexDiff , remove the element that falls outside the window. This approach ensures both conditions (index and value constraints) are met efficiently.

Eulerian Circuit

Round 9 · Low · Hard · 2 questions

Eulerian Circuit

□ #753 Cracking the Safe

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Depth-First Search · Graph Theory ·
Eulerian Circuit

Lists: AlgoMaster 600

Problem

There is a safe protected by a password. The password is a sequence of n digits where each digit can be in the range $[0, k - 1]$.

The safe has a peculiar way of checking the password. When you enter in a sequence, it checks the most recent n digits that were entered each time you type a digit.

- For example, the correct password is "345" and you enter in "012345": After typing 0, the most recent 3 digits is "0", which is incorrect.
- After typing 1, the most recent 3 digits is "01", which is incorrect.
- After typing 2, the most recent 3 digits is "012", which is incorrect.

- After typing 3, the most recent 3 digits is "123", which is incorrect.
- After typing 4, the most recent 3 digits is "234", which is incorrect.
- After typing 5, the most recent 3 digits is "345", which is correct and the safe unlocks.

Return any string of minimum length that will unlock the safe at some point of entering it.

Example 1:

Input: $n = 1, k = 2$

Output: "10"

Explanation: The password is a single digit, so enter each digit. "01" would also unlock the safe.

Example 2:

Input: $n = 2, k = 2$

Output: "01100"

Explanation: For each possible password:

- "00" is typed in starting from the 4th digit.
- "01" is typed in starting from the 1st digit.
- "10" is typed in starting from the 3rd digit.
- "11" is typed in starting from the 2nd digit.

Thus "01100" will unlock the safe. "10011", and "11001" would also unlock the safe.

Constraints:

- $1 \leq n \leq 4$
- $1 \leq k \leq 10$
- $1 \leq kn \leq 4096$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def crackSafe(self, n: int, k: int) -> str:
        passwordSize = k**n
        path = '0' * n
        seen = set()
        seen.add(path)

        def dfs(path: str) -> str:
            if len(seen) == passwordSize:
                return path

            for c in map(str, range(k)):
                node = path[-n + 1:] + c if n > 1 else c
                if node not in seen:
                    seen.add(node)
                    res = dfs(path + c)
                    if res:
                        return res
                    seen.remove(node)

        return dfs(path)
```

• LeetDoocs

```
class Solution:
    def crackSafe(self, n: int, k: int) -> str:
        def dfs(u):
            for x in range(k):
                e = u * 10 + x
                if e not in vis:
                    vis.add(e)
                    v = e % mod
                    dfs(v)
                    ans.append(str(x))
```

```

mod = 10 ** (n - 1)
vis = set()
ans = []
dfs(0)
ans.append("0" * (n - 1))
return "".join(ans)

```

• SimplyLeet

Python solution using DFS to generate a de Bruijn sequence

```

def crackSafe(n, k):
    # Initialize the set for visited combinations (edges)
    visited = set()
    result = []

    # Depth-first search helper function
    def dfs(node):
        # Try every digit in the alphabet [0, k-1]
        for digit in map(str, range(k)):

            # Form the new combination
            edge = node + digit
            # If the edge is not visited, continue DFS
            if edge not in visited:
                visited.add(edge)
                # Recursively visit the next node given by the last (n-1)
                # characters of edge
                dfs(edge[1:])
                # Append the digit to the result (backtracking)
                result.append(digit)

    # Start with an initial node of (n-1) zeros (or empty if n=1)
    start = "0" * (n - 1)
    dfs(start)
    # The final safe sequence is the starting node appended with the result
    # reversed
    return start + "".join(result[::-1])

# Example usage:
print(crackSafe(2, 2))

```

◆ Quick Review

Key Insights

- The problem is equivalent to generating a de Bruijn sequence for n -length strings over an alphabet of size k .
- Every possible n -digit combination appears exactly once as a substring in the de Bruijn sequence.
- The idea is to view the problem through the lens of graph theory where nodes represent the $(n-1)$ digit suffixes and edges correspond to possible n -digit combinations.
- An Eulerian circuit (or DFS based approach) can be used to traverse every edge exactly once ensuring that all combinations are covered.

Space & Time

Time Complexity: $O(k^n)$ – The algorithm essentially explores all possible n -length combinations.

Space Complexity: $O(k^n)$ – The space is primarily used to store visited edges and recursion stack.

Solution

We can solve this problem using a DFS approach to generate a de Bruijn sequence.

The steps are:

- Create a starting node with $(n-1)$ zeros.
- Use DFS to traverse every possible edge from the current node. An edge is represented by appending a digit to the current node.
- Mark each combination (edge) as visited when you traverse it.
- Append the digit to the result once the DFS call for that branch completes (backtracking).
- Finally, append the starting node to the result to complete the sequence. This method guarantees that every possible combination appears exactly once.

Eulerian Circuit

□ #2097 Valid Arrangement of Pairs

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Depth-First Search · Graph Theory ·
Eulerian Circuit

Lists: AlgoMaster 600

Problem

You are given a 0-indexed 2D integer array `pairs` where `pairs[i] = [starti, endi]`. An arrangement of pairs is valid if for every index `i` where $1 \leq i < \text{pairs.length}$, we have `endi-1 == starti`.

Return any valid arrangement of pairs.

Note: The inputs will be generated such that there exists a valid arrangement of pairs.

Example 1:

```
Input: pairs = [[5,1],[4,5],[11,9],[9,4]]
Output: [[11,9],[9,4],[4,5],[5,1]]
Explanation:
This is a valid arrangement since endi-1 always equals starti.
end0 = 9 == 9 = start1
end1 = 4 == 4 = start2
end2 = 5 == 5 = start3
```

Example 2:

Input: pairs = [[1,3],[3,2],[2,1]]

Output: [[1,3],[3,2],[2,1]]

Explanation:

This is a valid arrangement since $\text{endi}-1$ always equals start_i .

$\text{end}_0 = 3 == 3 = \text{start}_1$

$\text{end}_1 = 2 == 2 = \text{start}_2$

The arrangements [[2,1],[1,3],[3,2]] and [[3,2],[2,1],[1,3]] are also valid.

Example 3:

Input: pairs = [[1,2],[1,3],[2,1]]

Output: [[1,2],[2,1],[1,3]]

Explanation:

This is a valid arrangement since $\text{endi}-1$ always equals start_i .

$\text{end}_0 = 2 == 2 = \text{start}_1$

$\text{end}_1 = 1 == 1 = \text{start}_2$

Constraints:

- $1 \leq \text{pairs.length} \leq 105$
- $\text{pairs}[i].\text{length} == 2$
- $0 \leq \text{start}_i, \text{end}_i \leq 109$
- $\text{start}_i \neq \text{end}_i$
- No two pairs are exactly the same.
- There exists a valid arrangement of pairs.

Community Solutions (Python)

- WalkCC

```

class Solution:
    def validArrangement(self, pairs: list[list[int]]) -> list[list[int]]:
        ans = []
        graph = collections.defaultdict(list)
        outDegree = collections.Counter()
        inDegrees = collections.Counter()

        for start, end in pairs:
            graph[start].append(end)
            outDegree[start] += 1

            inDegrees[end] += 1

        def getStartNode() -> int:
            for u in graph.keys():
                if outDegree[u] - inDegrees[u] == 1:
                    return u
            return pairs[0][0] # Arbitrarily choose a node.

        def euler(u: int) -> None:
            stack = graph[u]

            while stack:
                v = stack.pop()
                euler(v)
                ans.append([u, v])

        euler(getStartNode())
        return ans[::-1]

```

• SimplyLeet

```

# Build the graph, find the Eulerian path, and convert the path to the valid
arrangement of pairs.
from collections import defaultdict, deque

def validArrangement(pairs):
    graph = defaultdict(list)
    in_degree = defaultdict(int)
    out_degree = defaultdict(int)

    # Build graph and degree count
    for start, end in pairs:

```

```
graph[start].append(end)
out_degree[start] += 1
in_degree[end] += 1

# Sort each list in reverse order for efficient pop
for start in graph:
    graph[start].sort(reverse=True)

# Find start node: node with out-degree = in-degree + 1, else any node with
# outgoing edge.
start_node = pairs[0][0]

for node in out_degree:
    if out_degree[node] - in_degree[node] == 1:
        start_node = node
        break

path = []
def dfs(node):
    while graph[node]:
        next_node = graph[node].pop()
        dfs(next_node)

    path.append(node)

dfs(start_node)
path.reverse()

# Convert vertex path to pairs. Since path is a list of vertices, each
# consecutive pair forms an edge.
arrangement = []
for i in range(1, len(path)):
    arrangement.append([path[i-1], path[i]])
return arrangement

# Example usage:
pairs_example = [[5,1],[4,5],[11,9],[9,4]]
print(validArrangement(pairs_example))
```

◆ Quick Review

Key Insights

- Interpret each pair as a directed edge from start to end.
- The problem reduces to finding an Eulerian path (or circuit) in a directed graph.
- If a vertex exists with out-degree exactly one greater than in-degree, it should be the starting vertex; otherwise, any vertex can serve as a valid starting node.
- Hierholzer's algorithm is ideal to reconstruct an Eulerian path by performing a modified DFS that “peels off” cycles and ensures all edges get used exactly once.

Space & Time

Time Complexity: $O(N)$, where N is the number of pairs since every edge is visited exactly once.

Space Complexity: $O(N)$ for storing the graph and the recursion/stack space for DFS.

Solution

We first build a graph mapping each start node to a list of its end nodes. To efficiently obtain and remove edges, sort each list in reverse order so that we can “pop” from the end. Next,

determine a valid starting node: if there is a vertex whose out-degree is exactly one more than its in-degree, start from there. Otherwise, we pick any vertex that exists in the graph.

Using Hierholzer's algorithm, perform a DFS:

- While the current vertex has outgoing edges, pop the last edge (which is efficient thanks to our reverse-sorted order) and recursively visit the end node.**
- Append the current vertex to the path once all its edges are visited.**
- Finally, reverse the recorded path to obtain the Eulerian path.**

This path will be a list of vertices. To convert it back into a list of pairs, form pairs from consecutive vertices. Since each pair is exactly the edge used in the Eulerian path and all pairs are unique, this fulfills the problem's requirements.

1-D DP

Round 9 · Low · Hard · 1 question

1-D DP

□ #1411 Number of Ways to Paint $N \times 3$ Grid

HAR

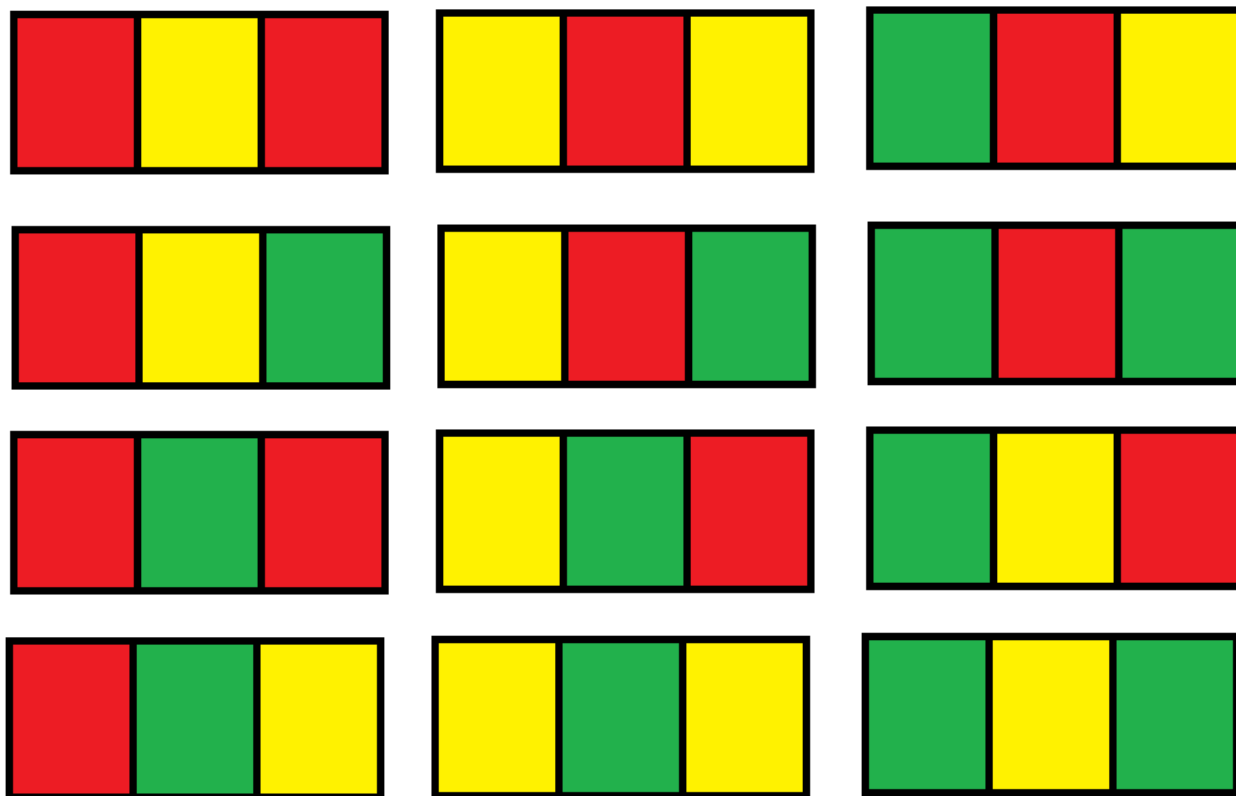
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming
Lists: AlgoMaster 600

Problem

You have a grid of size $n \times 3$ and you want to paint each cell of the grid with exactly one of the three colors: Red, Yellow, or Green while making sure that no two adjacent cells have the same color (i.e., no two cells that share vertical or horizontal sides have the same color).

Given n the number of rows of the grid, return the number of ways you can paint this grid. As the answer may grow large, the answer must be computed modulo $10^9 + 7$.

Example 1:



Input: $n = 1$

Output: 12

Explanation: There are 12 possible way to paint the grid as shown.

Example 2:

Input: $n = 5000$

Output: 30228214

Constraints:

- $n == \text{grid.length}$
- $1 \leq n \leq 5000$

Community Solutions (Python)

- LeetCode

```

class Solution:
    def numOfWays(self, n: int) -> int:
        mod = 10**9 + 7
        f0 = f1 = 6
        for _ in range(n - 1):
            g0 = (3 * f0 + 2 * f1) % mod
            g1 = (2 * f0 + 2 * f1) % mod
            f0, f1 = g0, g1
        return (f0 + f1) % mod

```

```

class Solution:
    def numOfWays(self, n: int) -> int:
        def f1(x: int) -> bool:
            last = -1
            for _ in range(3):
                if x % 3 == last:
                    return False
                last = x % 3
                x //= 3
            return True

```

```

    def f2(x: int, y: int) -> bool:
        for _ in range(3):
            if x % 3 == y % 3:
                return False
            x //= 3
            y //= 3
        return True

```

```

mod = 10**9 + 7

```

```

m = 27
valid = {i for i in range(m) if f1(i)}
d = defaultdict(list)
for i in valid:
    for j in valid:
        if f2(i, j):
            d[i].append(j)
f = [int(i in valid) for i in range(m)]
for _ in range(n - 1):
    g = [0] * m

```

```

        for i in valid:
            for j in d[i]:
                g[j] = (g[j] + f[i]) % mod
        f = g
    return sum(f) % mod

```

• SimplyLeet

```
MOD = 10**9 + 7
```

```

def numOfWays(n: int) -> int:
    # Base case: for row 1, there are 6 patterns of Type A and 6 patterns of
    # Type B.
    dpA, dpB = 6, 6
    # Process rows 2 through n.
    for i in range(2, n + 1):
        nextA = (3 * dpA + 2 * dpB) % MOD # From Type A and Type B transitions
        # to Type A.
        nextB = (2 * dpA + 2 * dpB) % MOD # From Type A and Type B transitions
        # to Type B.
        dpA, dpB = nextA, nextB # Update current states.

    return (dpA + dpB) % MOD

# Example usage:
print(numOfWays(1)) # Expected output: 12
print(numOfWays(5000)) # Outputs the computed answer modulo 10^9 + 7

```

◆ Quick Review

Key Insights

- Each row of 3 cells (with the horizontal-adjacency restriction) can be painted in 12 valid ways.
- These 12 colorings fall into two categories:

- **Type A: Patterns in which all 3 cells have distinct colors (6 possibilities).**
- **Type B: Patterns in which the first and third cell share the same color (6 possibilities).**
- **The valid painting for the next row depends on the pattern category of the previous row. By carefully analyzing the vertical constraints (avoiding same color in the same column) alongside the horizontal condition within the row, we can derive transition rules between the two types:**
 - **From a Type A row:**
 - **Next row can be Type A in 3 ways.**
 - **Next row can be Type B in 2 ways.**
 - **From a Type B row:**
 - **Next row can be Type A in 2 ways.**
 - **Next row can be Type B in 2 ways.**
 - **Use dynamic programming to update the number of ways row by row based on these transitions.**

Space & Time

Time Complexity: $O(n)$ – we iterate once through the n rows.

Space Complexity: $O(1)$ – only a constant amount of memory is used to store the counts for the two groups.

Solution

We solve the problem using a dynamic programming approach with two state variables:

- **dpA**: number of ways to paint the current row with a Type A (all different) pattern.
- **dpB**: number of ways to paint the current row with a Type B (first and third same) pattern.

For the first row, both dpA and dpB are 6 (total 12 valid patterns). For each subsequent row, we calculate:

- $\text{nextA} = 3 * \text{dpA}$ (transitions from a previous Type A) + $2 * \text{dpB}$ (transitions from a previous Type B).
- $\text{nextB} = 2 * \text{dpA}$ (transitions from a previous Type A) + $2 * \text{dpB}$ (transitions from a previous Type B).

After updating these values for each row, the final answer is $(\text{dpA} + \text{dpB}) \bmod 10^9 + 7$. This solution uses constant space and

processes each row in constant time.

0/1 Knapsack

Round 9 · Low · Hard · 1 question

0/1 Knapsack

□ #879 Profitable Schemes

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: AlgoMaster 600

Problem

There is a group of n members, and a list of various crimes they could commit. The i th crime generates a $profit[i]$ and requires $group[i]$ members to participate in it. If a member participates in one crime, that member can't participate in another crime.

Let's call a profitable scheme any subset of these crimes that generates at least $minProfit$ profit, and the total number of members participating in that subset of crimes is at most n .

Return the number of schemes that can be chosen. Since the answer may be very large, return it modulo $10^9 + 7$.

Example 1:

Input: $n = 5$, $\text{minProfit} = 3$, $\text{group} = [2,2]$, $\text{profit} = [2,3]$

Output: 2

Explanation: To make a profit of at least 3, the group could either commit crimes 0 and 1, or just crime 1.

In total, there are 2 schemes.

Example 2:

Input: $n = 10$, $\text{minProfit} = 5$, $\text{group} = [2,3,5]$, $\text{profit} = [6,7,8]$

Output: 7

Explanation: To make a profit of at least 5, the group could commit any crimes, as long as they commit one.

There are 7 possible schemes: (0), (1), (2), (0,1), (0,2), (1,2), and (0,1,2).

Constraints:

- $1 \leq n \leq 100$
- $0 \leq \text{minProfit} \leq 100$
- $1 \leq \text{group.length} \leq 100$
- $1 \leq \text{group}[i] \leq 100$
- $\text{profit.length} == \text{group.length}$
- $0 \leq \text{profit}[i] \leq 100$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def profitableSchemes(
        self, n: int, minProfit: int, group: List[int], profit: List[int]
    ) -> int:
        @cache
        def dfs(i: int, j: int, k: int) -> int:
            if i >= len(group):
                return 1 if k == minProfit else 0
            ans = dfs(i + 1, j, k)
            if j + group[i] <= n:
```

```

        ans += dfs(i + 1, j + group[i], min(k + profit[i], minProfit))
    return ans % (10**9 + 7)

return dfs(0, 0, 0)

```

• SimplyLeet

```

# Python solution
MOD = 10**9 + 7

def profitableSchemes(n, minProfit, group, profit):
    crimeCount = len(group)
    # dp[people][p] = number of schemes achieving profit 'p' with used members
    # 'people'
    dp = [[0] * (minProfit + 1) for _ in range(n + 1)]
    dp[0][0] = 1 # one way to achieve 0 profit with 0 members

    # Process each crime
    for idx in range(crimeCount):
        membersNeeded = group[idx]
        profitEarned = profit[idx]
        # iterate backwards for members to avoid reusing the same crime
        for people in range(n, membersNeeded - 1, -1):
            for p in range(minProfit, -1, -1):
                # New profit, not exceeding minProfit as cap
                newProfit = min(minProfit, p + profitEarned)
                # Update dp: add ways by including this crime
                dp[people][newProfit] = (dp[people][newProfit] + dp[people - m
membersNeeded][p]) % MOD

    # Sum over all ways where profit achieved is at least minProfit (which is
    dp[...][minProfit])
    totalSchemes = sum(dp[people][minProfit] for people in range(n + 1)) % MOD
    return totalSchemes

# Example usage:
print(profitableSchemes(5, 3, [2,2], [2,3])) # Output: 2

```

◆ Quick Review

Key Insights

- Use dynamic programming to model the state as (members used, current profit).
- Since profit can exceed minProfit, cap the profit dimension at minProfit (any excess is treated as meeting the requirement).
- Iterate over the list of crimes in a reverse order (to avoid overwriting needed previous values).
- The DP state update involves including or excluding the current crime from the scheme.
- The answer is the sum over all states where profit is at least minProfit with any number of members.

Space & Time

Time Complexity: $O(\text{number_of_crimes} * n * \text{minProfit})$

Space Complexity: $O(n * \text{minProfit})$

Solution

We use a 2D DP approach where $\text{dp}[\text{member}][\text{profit}]$ represents the number of schemes using exactly 'member' people and achieving exactly 'profit' (capped at minProfit).

For each crime with required people and profit, we update the dp table in reverse order with respect to the number of people (to avoid double counting usages within the same iteration). The profit index is updated as: $\text{newProfit} = \min(\text{minProfit}, \text{current profit} + \text{profit from crime})$ so that any profit above the threshold is treated the same as exactly minProfit. Finally, summing over $\text{dp}[i][\text{minProfit}]$ for all valid i (i is range $[0, n]$) gives the final result.

Longest Increasing Subsequence (LIS)

Round 9 · Low · Hard · 1 question

Longest Increasing Subsequence (LIS)

□ #354 Russian Doll Envelopes

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Dynamic Programming ·
Sorting

Lists: AlgoMaster 600

Problem

You are given a 2D array of integers `envelopes` where `envelopes[i] = [wi, hi]` represents the width and the height of an envelope.

One envelope can fit into another if and only if both the width and height of one envelope are greater than the other envelope's width and height.

Return the maximum number of envelopes you can Russian doll (i.e., put one inside the other).

Note: You cannot rotate an envelope.

Example 1:

Input: `envelopes = [[5,4],[6,4],[6,7],[2,3]]`

Output: 3

Explanation: The maximum number of envelopes you can Russian doll is 3 (`[2,3]` => `[5,4]` => `[6,7]`).

Example 2:

Input: envelopes = [[1,1],[1,1],[1,1]]

Output: 1

Constraints:

- $1 \leq \text{envelopes.length} \leq 105$
- $\text{envelopes}[i].\text{length} == 2$
- $1 \leq w_i, h_i \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxEnvelopes(self, envelopes: list[list[int]]) -> int:
        envelopes.sort(key=lambda x: (x[0], -x[1]))
        return self._lengthOfLIS(envelopes)

    def _lengthOfLIS(self, envelopes: list[list[int]]) -> int:
        # tails[i] := the minimum tails of all the increasing subsequences having
        # length i + 1
        tails = []

        for _, h in envelopes:
            if not tails or h > tails[-1]:
                tails.append(h)
            else:
                tails[bisect.bisect_left(tails, h)] = h

        return len(tails)
```

• LeetDoocs

```
class Solution:
    def maxEnvelopes(self, envelopes: List[List[int]]) -> int:
        envelopes.sort(key=lambda x: (x[0], -x[1]))
        d = [envelopes[0][1]]
        for _, h in envelopes[1:]:
            if h > d[-1]:
                d.append(h)
            else:
                idx = bisect_left(d, h)
                d[idx] = h

        return len(d)
```

• SimplyLeet

```
# Python solution with detailed comments
def maxEnvelopes(envelopes):
    # Sort envelopes primarily by width in ascending order and
    # by height in descending order if widths are equal.
    envelopes.sort(key=lambda x: (x[0], -x[1]))

    lis = [] # List to track the Longest Increasing Subsequence (LIS) of
    heights
    import bisect
    for _, height in envelopes:
        # Find the position where 'height' can be inserted to maintain lis in
        sorted order.

        pos = bisect.bisect_left(lis, height)
        if pos == len(lis):
            # 'height' is greater than all elements in lis; extend lis.
            lis.append(height)
        else:
            # Replace element to keep the potential of a longer increasing
            sequence.
            lis[pos] = height
    return len(lis)

# Example usage

print(maxEnvelopes([[5,4],[6,4],[6,7],[2,3]]))
```

◆ Quick Review

Key Insights

- Sort envelopes by width in ascending order and, if the widths are equal, by height in descending order.
- Sorting in this manner allows us to avoid conflicts when envelopes share the same width.
- After sorting, the problem reduces to finding the Longest Increasing Subsequence (LIS) based solely on heights.
- Utilize binary search within the LIS algorithm to achieve an efficient $O(n \log n)$ solution.

Space & Time

Time Complexity: $O(n \log n)$, where n is the number of envelopes.

Space Complexity: $O(n)$, required for the dynamic list used in the LIS solution.

Solution

The algorithm first sorts the envelopes by their width in increasing order. For envelopes with the same width, sorting by height in descending order ensures that only one

envelope of the same width can be considered in the nesting sequence. Then, we extract the heights and use a binary search-based approach to compute the longest increasing subsequence (LIS) of heights, which represents the maximum number of envelopes we can nest. The list maintained during the LIS calculation stores the smallest possible tail for increasing subsequences of different lengths, allowing efficient updating and extension of potential sequences.

2D Grid DP

Round 9 · Low · Hard · 10 questions

2D Grid DP

□ #85 Maximal Rectangle

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Stack · Matrix
· Monotonic Stack

Lists: AlgoMaster 600

Problem

Given a rows x cols binary matrix filled with 0's and 1's, find the largest rectangle containing only 1's and return its area.

Example 1:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

Input: matrix = `[["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]`

Output: 6

Explanation: The maximal rectangle is shown in the above picture.

Example 2:

Input: matrix = `[["0"]]`

Output: 0

Example 3:

Input: matrix = `[["1"]]`

Output: 1

Constraints:

- `rows == matrix.length`
- `cols == matrix[i].length`
- `1 <= rows, cols <= 200`
- `matrix[i][j]` is '0' or '1'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maximalRectangle(self, matrix: list[list[str]]) -> int:
        if not matrix:
            return 0

        ans = 0
        hist = [0] * len(matrix[0])

        def largestRectangleArea(heights: list[int]) -> int:
            ans = 0

            stack = []

            for i in range(len(heights) + 1):
                while stack and (i == len(heights) or heights[stack[-1]] > heights[i]):
                    h = heights[stack.pop()]
                    w = i - stack[-1] - 1 if stack else i
                    ans = max(ans, h * w)
                    stack.append(i)

            return ans

        for row in matrix:
            for i, num in enumerate(row):
                hist[i] = 0 if num == '0' else hist[i] + 1
            ans = max(ans, largestRectangleArea(hist))

        return ans
```

• LeetCode

```
class Solution:
    def maximalRectangle(self, matrix: List[List[str]]) -> int:
        heights = [0] * len(matrix[0])
        ans = 0
        for row in matrix:
            for j, v in enumerate(row):
                if v == "1":
                    heights[j] += 1
                else:
                    heights[j] = 0

            ans = max(ans, self.largestRectangleArea(heights))
        return ans

    def largestRectangleArea(self, heights: List[int]) -> int:
        n = len(heights)
        stk = []
        left = [-1] * n
        right = [n] * n
        for i, h in enumerate(heights):
            while stk and heights[stk[-1]] >= h:
                stk.pop()
            if stk:
                left[i] = stk[-1]
            stk.append(i)
        stk = []
        for i in range(n - 1, -1, -1):
            h = heights[i]
            while stk and heights[stk[-1]] >= h:
                stk.pop()
            if stk:
                right[i] = stk[-1]
            stk.append(i)
        return max(h * (right[i] - left[i] - 1) for i, h in enumerate(heights))
)
```

• SimplyLeet

```
# Python solution for Maximal Rectangle

class Solution:
    def maximalRectangle(self, matrix: List[List[str]]) -> int:
        if not matrix or not matrix[0]:
            return 0

        # Number of columns
        n = len(matrix[0])
        # Initialize histogram heights with 0's

        heights = [0] * n
        max_area = 0

        # Process each row in the matrix
        for row in matrix:
            for i in range(n):
                # Update histogram: increase height if '1' else reset to 0
                if row[i] == '1':
                    heights[i] += 1
                else:
                    heights[i] = 0

            # Calculate maximal area for the current histogram row
            max_area = max(max_area, self.largestRectangleArea(heights))

        return max_area

    def largestRectangleArea(self, heights: List[int]) -> int:
        # Append a zero at the end for flush-out
        heights.append(0)

        stack = []
        max_area = 0

        # Process every bar in the histogram
        for i, h in enumerate(heights):
            # Maintain stack in non-decreasing order of heights
            while stack and heights[stack[-1]] > h:
                height = heights[stack.pop()]
                # Determine width of the rectangle
                width = i if not stack else i - stack[-1] - 1
```

```
        max_area = max(max_area, height * width)
        stack.append(i)

    # Remove the appended 0
    heights.pop()
    return max_area
```

◆ Quick Review

Key Insights

- Transform each row into a histogram, where the height at each column is the count of consecutive '1's encountered so far.
- Use a monotonic stack algorithm to compute the largest rectangle area within this histogram.
- Iterate row by row, updating histogram heights and computing the maximal area based on the updated histogram.
- This reduces a 2D problem into repeatedly solving the Largest Rectangle in Histogram problem.

Space & Time

Time Complexity: $O(m \cdot n)$, where m is the number of rows and n is the number of columns, since each histogram is processed in $O(n)$ for each row.

Space Complexity: $O(n)$, needed for the histogram and the stack used in the algorithm.

Solution

The algorithm works by processing each row of the matrix and updating a histogram that represents the number of consecutive '1's for each column up to that row. For every row, we compute the largest rectangle area in the histogram by using a monotonic stack:

- Iterate through each column index in the histogram.**
- Use a stack to store column indices such that the heights are in non-decreasing order.**
- When a height less than the current height is encountered, pop indices from the stack and calculate the area using the height at the popped index and the current index as the right boundary.**
- After processing, ensure remaining indices in the stack are handled.**
- The maximum area found over all rows is returned as the answer.**

2D Grid DP

□ #174 Dungeon Game

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: AlgoMaster 600

Problem

The demons had captured the princess and imprisoned her in the bottom-right corner of a dungeon. The dungeon consists of $m \times n$ rooms laid out in a 2D grid. Our valiant knight was initially positioned in the top-left room and must fight his way through dungeon to rescue the princess.

The knight has an initial health point represented by a positive integer. If at any point his health point drops to 0 or below, he dies immediately.

Some of the rooms are guarded by demons (represented by negative integers), so the knight loses health upon entering these rooms; other rooms are either empty (represented as 0) or contain magic orbs that increase the knight's

health (represented by positive integers).

To reach the princess as quickly as possible, the knight decides to move only rightward or downward in each step.

Return the knight's minimum initial health so that he can rescue the princess.

Note that any room can contain threats or power-ups, even the first room the knight enters and the bottom-right room where the princess is imprisoned.

Example 1:

-2	-3	3
-5	-10	1
10	30	-5

Input: `dungeon = [[-2,-3,3],[-5,-10,1],[10,30,-5]]`

Output: 7

Explanation: The initial health of the knight must be at least 7 if he follows the optimal path: RIGHT-> RIGHT -> DOWN -> DOWN.

Example 2:

Input: `dungeon = [[0]]`

Output: 1

Constraints:

- `m == dungeon.length`
- `n == dungeon[i].length`
- `1 <= m, n <= 200`

- $-1000 \leq \text{dungeon}[i][j] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def calculateMinimumHP(self, dungeon: list[list[int]]) -> int:
        m = len(dungeon)
        n = len(dungeon[0])
        dp = [math.inf] * (n + 1)
        dp[n - 1] = 1

        for i in reversed(range(m)):
            for j in reversed(range(n)):
                dp[j] = min(dp[j], dp[j + 1]) - dungeon[i][j]

            dp[j] = max(dp[j], 1)

        return dp[0]
```

• LeetDoocs

```
class Solution:
    def calculateMinimumHP(self, dungeon: List[List[int]]) -> int:
        m, n = len(dungeon), len(dungeon[0])
        dp = [[inf] * (n + 1) for _ in range(m + 1)]
        dp[m][n - 1] = dp[m - 1][n] = 1
        for i in range(m - 1, -1, -1):
            for j in range(n - 1, -1, -1):
                dp[i][j] = max(1, min(dp[i + 1][j], dp[i][j + 1]) - dungeon[i][j])

        return dp[0][0]
```

• SimplyLeet

```
def calculateMinimumHP(dungeon):
    # Determine the dimensions of the dungeon grid.
    m, n = len(dungeon), len(dungeon[0])

    # Create a DP table with an extra row and column for boundary conditions.
    dp = [[float('inf')] * (n + 1) for _ in range(m + 1)]

    # Set the cells adjacent to the princess's cell to 1.
    dp[m][n - 1] = 1
    dp[m - 1][n] = 1

    # Fill the DP table starting from the bottom-right corner moving backwards.
    for i in range(m - 1, -1, -1):
        for j in range(n - 1, -1, -1):
            # Min health required from right and bottom cells.
            min_next = min(dp[i + 1][j], dp[i][j + 1])
            # Calculate health needed at the current cell.
            dp[i][j] = max(min_next - dungeon[i][j], 1)

    return dp[0][0]

# Example usage:
dungeon = [[-2, -3, 3],
            [-5, -10, 1],
            [10, 30, -5]]
print(calculateMinimumHP(dungeon))
```

◆ Quick Review

Key Insights

- Use Dynamic Programming with a reverse (bottom-up) approach starting from the princess's cell.
- At each cell, compute the minimum health needed to ensure survival until reaching the

princess.

- The required health at any cell is $\max(\text{minimum necessary health from the right or down cell minus the current cell value}, 1)$.
- Boundary initialization is key; set extra row and column cells to a high value (or infinity) except the positions adjacent to the princess's cell set to 1.

Space & Time

Time Complexity: $O(m * n)$

Space Complexity: $O(m * n)$

Solution

We solve the problem by initializing a DP table where $dp[i][j]$ represents the minimum health needed at cell (i, j) to eventually reach the princess. We initialize the dp table for boundaries to simulate an “exit” from the dungeon cell with a minimum requirement of 1 health. Then we iterate from the bottom-right cell (princess's cell) backwards to the top-left cell. For each cell, compute the minimum health needed based on the minimum of the health requirements of the right and bottom neighbors, ensuring that the value is at least 1.

This approach guarantees that the knight's health never falls to 0 or below at any point.

2D Grid DP

□ #403 Frog Jump

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: AlgoMaster 600

Problem

A frog is crossing a river. The river is divided into some number of units, and at each unit, there may or may not exist a stone. The frog can jump on a stone, but it must not jump into the water.

Given a list of stones positions (in units) in sorted ascending order, determine if the frog can cross the river by landing on the last stone. Initially, the frog is on the first stone and assumes the first jump must be 1 unit.

If the frog's last jump was k units, its next jump must be either $k - 1$, k , or $k + 1$ units. The frog can only jump in the forward direction.

Example 1:

Input: stones = [0,1,3,5,6,8,12,17]

Output: true

Explanation: The frog can jump to the last stone by jumping 1 unit to the 2nd stone, then 2 units to the 3rd stone, then 2 units to the 4th stone, then 3 units to the 6th stone, 4 units to the 7th stone, and 5 units to the 8th stone.

Example 2:

Input: stones = [0,1,2,3,4,8,9,11]

Output: false

Explanation: There is no way to jump to the last stone as the gap between the 5th and 6th stone is too large.

Constraints:

- $2 \leq \text{stones.length} \leq 2000$
- $0 \leq \text{stones}[i] \leq 231 - 1$
- $\text{stones}[0] == 0$
- stones is sorted in a strictly increasing order.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canCross(self, stones: list[int]) -> bool:
        n = len(stones)
        # dp[i][j] := True if a frog can make a size j jump to stones[i]
        dp = [[False] * (n + 1) for _ in range(n)]
        dp[0][0] = True

        for i in range(1, n):
            for j in range(i):
                k = stones[i] - stones[j]
```

```

    if k > n:
        continue
    for x in (k - 1, k, k + 1):
        if 0 <= x <= n:
            dp[i][k] |= dp[j][x]

return any(dp[-1])

```

• LeetDoocs

```

class Solution:
    def canCross(self, stones: List[int]) -> bool:
        @cache
        def dfs(i, k):
            if i == n - 1:
                return True
            for j in range(k - 1, k + 2):
                if j > 0 and stones[i] + j in pos and dfs(pos[stones[i] + j],
j):
                    return True
            return False

```

```

n = len(stones)
pos = {s: i for i, s in enumerate(stones)}
return dfs(0, 0)

```

```

class Solution:
    def canCross(self, stones: List[int]) -> bool:
        n = len(stones)
        f = [[False] * n for _ in range(n)]
        f[0][0] = True
        for i in range(1, n):
            for j in range(i - 1, -1, -1):
                k = stones[i] - stones[j]
                if k - 1 > j:
                    break

                f[i][k] = f[j][k - 1] or f[j][k] or f[j][k + 1]
                if i == n - 1 and f[i][k]:
                    return True
        return False

```


• SimplyLeet

```
# Define the class with the canCross method.
class Solution:
    def canCross(self, stones):
        # Create a dictionary mapping each stone position to a set of jump
        # sizes that can reach that stone.
        stone_jumps = {stone: set() for stone in stones}
        # The frog starts at the first stone with a jump of 0 (this enables the
        # first jump of 1).
        stone_jumps[0].add(0)

        # Iterate over each stone.
        for stone in stones:

            # For each jump that reaches the current stone.
            for jump in stone_jumps[stone]:
                # Evaluate possible new jump sizes: k-1, k, k+1.
                for next_jump in (jump - 1, jump, jump + 1):
                    # The next jump must be positive.
                    if next_jump > 0:
                        next_stone = stone + next_jump
                        # If the calculated stone exists in the stones list.
                        if next_stone in stones:
                            stone_jumps[next_stone].add(next_jump)

            # If the last stone is reached, return True.
            if next_stone == stones[-1]:
                return True

        return False

# Example usage:
# sol = Solution()
# print(sol.canCross([0,1,3,5,6,8,12,17])) # Expected output: True
```

◆ Quick Review

Key Insights

- Use dynamic programming to track possible jump sizes that can land the frog on each

stone.

- Leverage a hash-map (or dictionary) to map each stone's position to a set of possible jump sizes that can arrive there.
- The valid next jump sizes from a stone reached using jump k are $(k-1)$, k , and $(k+1)$, ensuring the jump is >0 .
- Early termination is possible as soon as the last stone is reached.

Space & Time

Time Complexity: $O(n^2)$ in the worst-case scenario, where n is the number of stones, because for each stone we may explore up to three possible jump sizes.

Space Complexity: $O(n^2)$ in the worst-case scenario, due to the storage of potential jump sizes for each stone in the hash-map.

Solution

The solution uses a dynamic programming approach by keeping track of all possible jump lengths that can reach every stone using a hash-map. Starting from the first stone with a known jump size (0 initially, to allow the first jump of 1 unit), for each stone, the algorithm

computes the next possible stones that can be reached by jumps of size $k-1$, k , and $k+1$. If a jump results in landing on a valid stone, that jump size is stored for that stone. The process continues until the last stone is reached, in which case the function returns true. If the entire process finishes without reaching the last stone, then the answer is false.

2D Grid DP

□ #465 Optimal Account Balancing

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Backtracking ·
Bit Manipulation · Bitmask

Lists: AlgoMaster 600

Problem

You are given an array of transactions
transactions where $\text{transactions}[i] = [\text{from}_i, \text{to}_i, \text{amount}_i]$ indicates that the person with ID =
 from_i gave amount_i \$ to the person with ID =
 to_i .

Return the minimum number of transactions
required to settle the debt.

Example 1:

Input: transactions = [[0,1,10],[2,0,5]]

Output: 2

Explanation:

Person #0 gave person #1 \$10.

Person #2 gave person #0 \$5.

Two transactions are needed. One way to settle the debt is person #1 pays
person #0 and #2 \$5 each.

Example 2:

Input: transactions = [[0,1,10],[1,0,1],[1,2,5],[2,0,5]]

Output: 1

Explanation:

Person #0 gave person #1 \$10.

Person #1 gave person #0 \$1.

Person #1 gave person #2 \$5.

Person #2 gave person #0 \$5.

Therefore, person #1 only need to give person #0 \$4, and all debt is settled.

Constraints:

- $1 \leq \text{transactions.length} \leq 8$
- $\text{transactions}[i].\text{length} == 3$
- $0 \leq \text{from}_i, \text{to}_i < 12$
- $\text{from}_i \neq \text{to}_i$
- $1 \leq \text{amount}_i \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minTransfers(self, transactions: list[list[int]]) -> int:
        balance = [0] * 21

        for u, v, amount in transactions:
            balance[u] -= amount
            balance[v] += amount

        debts = [b for b in balance if b]
```

```
def dfs(s: int) -> int:
    while s < len(debts) and not debts[s]:
        s += 1
    if s == len(debts):
        return 0

    ans = math.inf

    for i in range(s + 1, len(debts)):
        if debts[i] * debts[s] < 0:

            debts[i] += debts[s] # `debts[s]` is settled.
            ans = min(ans, 1 + dfs(s + 1))
            debts[i] -= debts[s] # Backtrack.

    return ans

return dfs(0)
```

• LeetDoocs

```
class Solution:
    def minTransfers(self, transactions: List[List[int]]) -> int:
        g = defaultdict(int)
        for f, t, x in transactions:
            g[f] -= x
            g[t] += x
        nums = [x for x in g.values() if x]
        m = len(nums)
        f = [inf] * (1 << m)
        f[0] = 0

        for i in range(1, 1 << m):
            s = 0
            for j, x in enumerate(nums):
                if i >> j & 1:
                    s += x
            if s == 0:
                f[i] = i.bit_count() - 1
                j = (i - 1) & i
                while j > 0:
                    f[i] = min(f[i], f[j] + f[i ^ j])
```

```

        j = (j - 1) & i
    return f[-1]

```

• SimplyLeet

Python solution for Optimal Account Balancing

```

def minTransfers(transactions):
    from collections import defaultdict
    # Compute net balance for each person.
    balance = defaultdict(int)
    for frm, to, amount in transactions:
        balance[frm] -= amount # person gives money
        balance[to] += amount # person receives money

```

```

# Filter out persons with zero balance.
debts = [amount for amount in balance.values() if amount != 0]

```

```

def dfs(start):
    # Skip settled debts.
    while start < len(debts) and debts[start] == 0:
        start += 1
    # All debts settled.
    if start == len(debts):
        return 0

```

```

min_trans = float('inf')
for i in range(start + 1, len(debts)):
    # Only attempt to settle if debts[start] and debts[i] are opposite
    in sign.
    if debts[start] * debts[i] < 0:
        # Settle debts[start] with debts[i].
        debts[i] += debts[start]
        # Recurse: 1 transaction is used.
        min_trans = min(min_trans, 1 + dfs(start + 1))
        # Backtrack: restore debts[i].
        debts[i] -= debts[start]

```

```
        # Optimization: if debts[i] becomes zero after settlement,
        # no need to try other similar values.
        if debts[i] + debts[start] == 0:
            break
    return min_trans

    return dfs(0)

# Example usage
transactions = [[0,1,10],[2,0,5]]

print(minTransfers(transactions)) # Output: 2
```

◆ Quick Review

Key Insights

- Compute the net balance for each person.
- Only consider persons with non-zero balance, since persons with a zero net balance do not affect the result.
- Use backtracking (DFS) to try settling debts between pairs of people.
- Prune redundant operations by skipping similar debt values in subsequent recursive calls.
- The problem size is small (max 12 persons and 8 transactions) so a DFS/backtracking solution is practical.

Space & Time

Time Complexity: $O(2^n)$ in the worst-case scenario, where n is the number of non-zero balances.

Space Complexity: $O(n)$ due to recursion and the balance array.

Solution

We first compute each person's net balance. Then, we use a depth-first search (DFS) approach to try and settle each outstanding balance with minimal transactions. In the DFS, we attempt to settle the debt of one person with another by transferring money and recursively exploring the outcome. If a debt is completely settled, we move on to the next balance. To optimize, we skip over redundant operations when we have already tried to settle with a similar debt value. We employ backtracking to restore the original state after trying each settlement possibility. This recursive strategy allows us to explore all combinations while keeping track of the minimum number of transactions required.

2D Grid DP

□ #546 Remove Boxes

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Memoization
Lists: AlgoMaster 600

Problem

You are given several boxes with different colors represented by different positive numbers.

You may experience several rounds to remove boxes until there is no box left. Each time you can choose some continuous boxes with the same color (i.e., composed of k boxes, $k \geq 1$), remove them and get $k * k$ points.

Return the maximum points you can get.

Example 1:

```
Input: boxes = [1,3,2,2,2,3,4,3,1]
Output: 23
Explanation:
[1, 3, 2, 2, 2, 3, 4, 3, 1]
----> [1, 3, 3, 4, 3, 1] (3*3=9 points)
----> [1, 3, 3, 3, 1] (1*1=1 points)
----> [1, 1] (3*3=9 points)
----> [] (2*2=4 points)
```

Example 2:

Input: boxes = [1,1,1]
Output: 9

Example 3:

Input: boxes = [1]
Output: 1

Constraints:

- $1 \leq \text{boxes.length} \leq 100$
- $1 \leq \text{boxes}[i] \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeBoxes(self, boxes: list[int]) -> int:
        @functools.lru_cache(None)
        def dp(i: int, j: int, k: int) -> int:
            """
            Returns the maximum score of boxes[i..j] if k boxes equal to boxes[j].
            """
            if i > j:
                return 0

            r = j
            sameBoxes = k + 1
            while r > 0 and boxes[r - 1] == boxes[r]:
                r -= 1
                sameBoxes += 1
            res = dp(i, r - 1, 0) + sameBoxes * sameBoxes

            for p in range(i, r):
                if boxes[p] == boxes[r]:
                    res = max(res, dp(i, p, sameBoxes) + dp(p + 1, r - 1, 0))
```

```

        return res

    return dp(0, len(boxes) - 1, 0)

```

• LeetCode

```

class Solution:
    def removeBoxes(self, boxes: List[int]) -> int:
        @cache
        def dfs(i, j, k):
            if i > j:
                return 0
            while i < j and boxes[j] == boxes[j - 1]:
                j, k = j - 1, k + 1
            ans = dfs(i, j - 1, 0) + (k + 1) * (k + 1)
            for h in range(i, j):
                if boxes[h] == boxes[j]:
                    ans = max(ans, dfs(h + 1, j - 1, 0) + dfs(i, h, k + 1))
            return ans

        n = len(boxes)
        ans = dfs(0, n - 1, 0)
        dfs.cache_clear()
        return ans

```

• SimplyLeet

```

# Python solution using recursion with memoization
class Solution:
    def removeBoxes(self, boxes):
        # memo dictionary to store computed values for state (l, r, k)
        memo = {}

        def dp(l, r, k):
            # Base case: no boxes left
            if l > r:
                return 0

```

```

        # Check memoized result
        if (l, r, k) in memo:
            return memo[(l, r, k)]

        # Increase l and k if consecutive boxes are the same as boxes[l]
        orig_l, orig_k = l, k
        while l + 1 <= r and boxes[l+1] == boxes[l]:
            l += 1
            k += 1

        # Remove the boxes[l] block, get (k+1)^2 points, then solve for the
        rest
        result = (k+1)**2 + dp(l+1, r, 0)

        # Try to merge non-adjacent boxes that are the same as boxes[l]
        for m in range(l+1, r+1):
            if boxes[m] == boxes[l]:
                # Combine boxes[l] with boxes[m] after clearing boxes in
                between

                temp = dp(l+1, m-1, 0) + dp(m, r, k+1)
                if temp > result:
                    result = temp

        memo[(orig_l, r, orig_k)] = result
        return result

    return dp(0, len(boxes)-1, 0)

```

◆ Quick Review

Key Insights

- This is a hard optimization problem with overlapping sub-problems.
- The heart of the solution is dynamic programming with memoization.

- The key is to consider not just the boxes between indices l and r , but also how many same-colored boxes adjacent to the segment can be combined (parameter k).
- Use a 3-dimensional DP state $dp(l, r, k)$ representing the maximum points achievable from range $[l, r]$ with k extra boxes (of the same color as box l) appended from the left.
- When the first box has consecutive boxes of the same color, they can be aggregated to simplify the state.
- Properly splitting the array at optimal positions where a future same-colored box appears is essential to combine points.

Space & Time

Time Complexity: $O(n^3)$ in the worst-case scenario due to the three nested parameters (l , r , and k), where n is the length of boxes.

Space Complexity: $O(n^3)$ for memoization storage.

Solution

The solution uses recursion with memoization (a top-down dynamic programming approach). For a subarray from index l to r with k extra

boxes of the same color as boxes[l] appended, calculate the maximum points as follows:

- First, merge all consecutive boxes at the beginning that are the same as boxes[l] by increasing k and moving l forward.
- Remove the merged block immediately and calculate points as $(k+1)^2$ plus the result of solving the remaining subarray.
- Try to find any index m in [l+1, r] where boxes[m] equals boxes[l]. By not removing boxes[l] immediately, we first remove the boxes in between (l+1 to m-1) and then merge boxes[l] with boxes[m]. This may yield a higher score.
- Use memoization to avoid repeated calculations for the same state (l, r, k).

The trick is in the merge step that optimally leverages same-colored boxes that are separated by other boxes.

2D Grid DP

□ #741 Cherry Pickup

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: AlgoMaster 600

Problem

You are given an $n \times n$ grid representing a field of cherries, each cell is one of three possible integers.

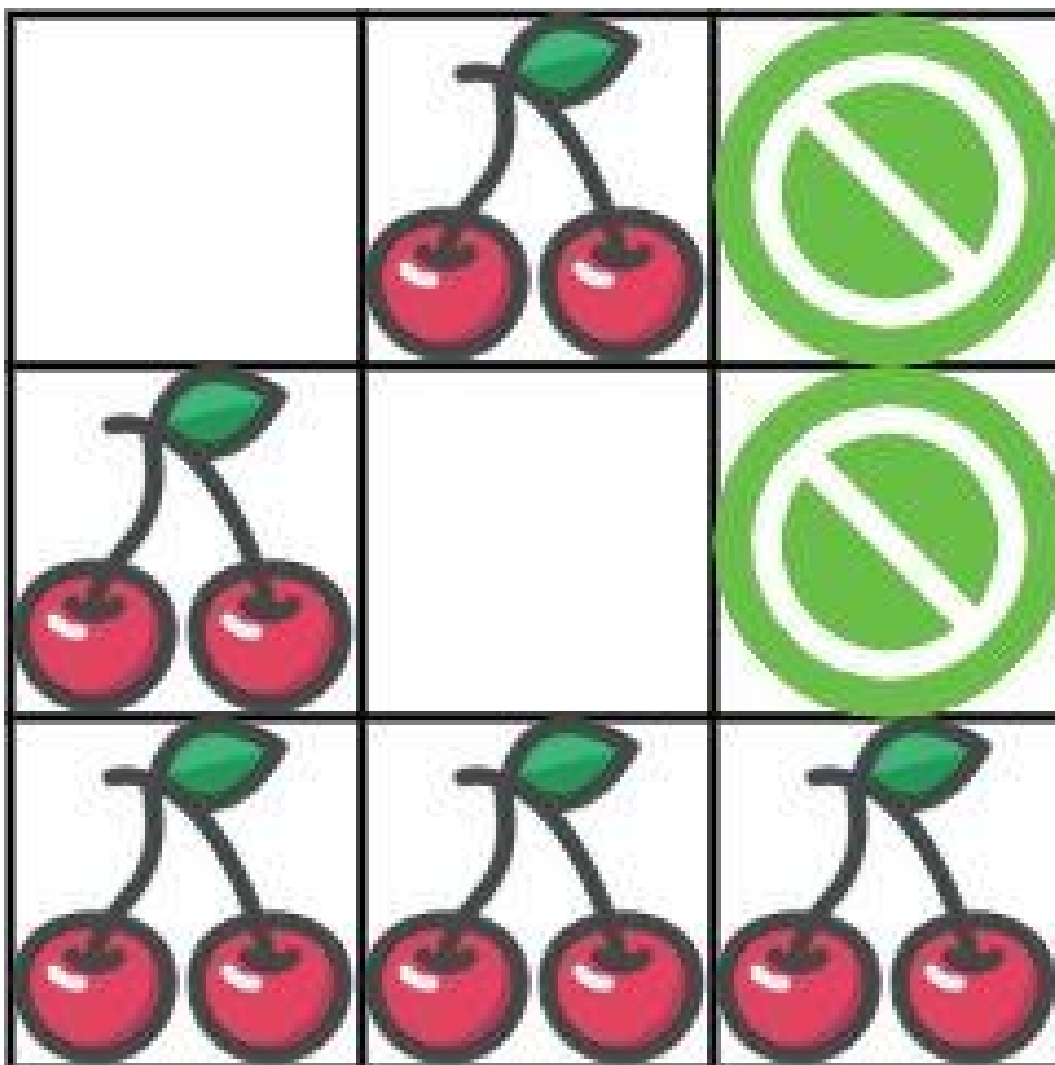
- 0 means the cell is empty, so you can pass through,
- 1 means the cell contains a cherry that you can pick up and pass through, or
- -1 means the cell contains a thorn that blocks your way.

Return the maximum number of cherries you can collect by following the rules below:

- Starting at the position (0, 0) and reaching (n - 1, n - 1) by moving right or down through valid path cells (cells with value 0 or 1).

- After reaching $(n - 1, n - 1)$, returning to $(0, 0)$ by moving left or up through valid path cells.
- When passing through a path cell containing a cherry, you pick it up, and the cell becomes an empty cell 0.
- If there is no valid path between $(0, 0)$ and $(n - 1, n - 1)$, then no cherries can be collected.

Example 1:



Input: grid = `[[0,1,-1],[1,0,-1],[1,1,1]]`

Output: 5

Explanation: The player started at (0, 0) and went down, down, right right to reach (2, 2).

4 cherries were picked up during this single trip, and the matrix becomes

`[[0,1,-1],[0,0,-1],[0,0,0]]`.

Then, the player went left, up, up, left to return home, picking up one more cherry.

The total number of cherries picked up is 5, and this is the maximum possible.

Example 2:

Input: grid = `[[1,1,-1],[1,-1,1],[-1,1,1]]`

Output: 0

Constraints:

- `n == grid.length`

- $n == \text{grid}[i].\text{length}$
- $1 \leq n \leq 50$
- $\text{grid}[i][j]$ is -1 , 0 , or 1 .
- $\text{grid}[0][0] \neq -1$
- $\text{grid}[n - 1][n - 1] \neq -1$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def cherryPickup(self, grid: List[List[int]]) -> int:
        n = len(grid)
        f = [[[-inf] * n for _ in range(n)] for _ in range((n << 1) - 1)]
        f[0][0][0] = grid[0][0]
        for k in range(1, (n << 1) - 1):
            for i1 in range(n):
                for i2 in range(n):
                    j1, j2 = k - i1, k - i2
                    if (
                        not 0 <= j1 < n
                        or not 0 <= j2 < n
                        or grid[i1][j1] == -1
                        or grid[i2][j2] == -1
                    ):
                        continue
                    t = grid[i1][j1]
                    if i1 != i2:
                        t += grid[i2][j2]
                    for x1 in range(i1 - 1, i1 + 1):
                        for x2 in range(i2 - 1, i2 + 1):
                            if x1 >= 0 and x2 >= 0:
                                f[k][i1][i2] = max(f[k][i1][i2], f[k - 1][x1][
x2] + t)
                    return max(0, f[-1][-1][-1])
```

• SimplyLeet

```
def cherryPickup(grid):
    n = len(grid)
    memo = {}

    def dp(r1, c1, r2):
        c2 = r1 + c1 - r2 # Because steps taken are the same: r1+c1 = r2+c2.
        # Check for out-of-bound indices or thorns.
        if r1 >= n or c1 >= n or r2 >= n or c2 >= n or grid[r1][c1] == -1 or grid[r2][c2] == -1:
            return float('-inf')
        # If reached destination.

        if r1 == n - 1 and c1 == n - 1:
            return grid[r1][c1]
        # Check memoization table.
        if (r1, c1, r2) in memo:
            return memo[(r1, c1, r2)]
        result = grid[r1][c1]
        # Avoid double counting if both agents are on the same cell.
        if r1 != r2 or c1 != c2:
            result += grid[r2][c2]
        # Explore next moves: down/down, right/right, down/right, right/down.

        temp = max(
            dp(r1 + 1, c1, r2 + 1),
            dp(r1, c1 + 1, r2),
            dp(r1 + 1, c1, r2),
            dp(r1, c1 + 1, r2 + 1)
        )
        result += temp
        memo[(r1, c1, r2)] = result
        return result

    ans = dp(0, 0, 0)
    return max(ans, 0)

# Example usage:
grid = [[0, 1, -1], [1, 0, -1], [1, 1, 1]]
print(cherryPickup(grid)) # Expected output: 5
```

◆ Quick Review

Key Insights

- Transforming the two trips (out and back) into one simultaneous traversal by having two agents start at $(0,0)$ and move to $(n-1,n-1)$ concurrently.
- Since both agents make the same number of moves, their positions satisfy $r1+c1 = r2+c2$, which allows reducing the state dimension.
- Use dynamic programming with a 3-dimensional state $(r1, c1, r2)$ while computing $c2$ as $(r1+c1-r2)$.
- Avoid double counting the cherry if both agents land on the same cell.
- Return 0 if no valid path exists (i.e. if the computed maximum is negative).

Space & Time

Time Complexity: $O(n^3)$ where n is the grid dimension.

Space Complexity: $O(n^3)$ for the DP memoization table.

Solution

The solution employs a dynamic programming approach by simulating two agents moving simultaneously from the start to the destination. Each state is defined by $(r1, c1, r2)$ with $c2$ computed from the invariant $r1 + c1 = r2 + c2$. The algorithm explores all the valid combinations of right and down moves for both agents. When both agents are on the same cell, its cherry (if present) is only counted once. If any move leads to an invalid cell or a thorn, that path is considered as $-\infty$. A memoization table caches computed states to avoid redundant calculations. The final answer is the maximum cherries collected along the optimal path from start to finish, or 0 if no valid path exists.

2D Grid DP

□ #1335 Minimum Difficulty of a Job Schedule

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: AlgoMaster 600

Problem

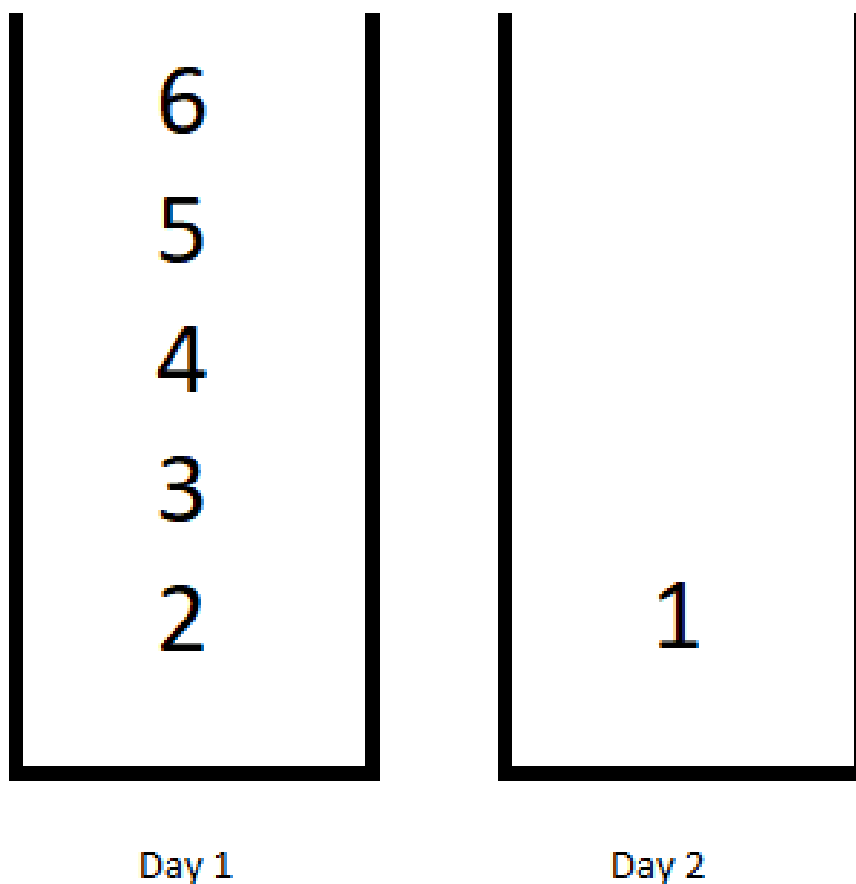
You want to schedule a list of jobs in d days. Jobs are dependent (i.e To work on the i th job, you have to finish all the jobs j where $0 \leq j < i$).

You have to finish at least one task every day. The difficulty of a job schedule is the sum of difficulties of each day of the d days. The difficulty of a day is the maximum difficulty of a job done on that day.

You are given an integer array `jobDifficulty` and an integer d . The difficulty of the i th job is `jobDifficulty[i]`.

Return the minimum difficulty of a job schedule. If you cannot find a schedule for the jobs return -1 .

Example 1:



Input: `jobDifficulty = [6,5,4,3,2,1]`, `d = 2`

Output: 7

Explanation: First day you can finish the first 5 jobs, total difficulty = 6.

Second day you can finish the last job, total difficulty = 1.

The difficulty of the schedule = $6 + 1 = 7$

Example 2:

Input: `jobDifficulty = [9,9,9]`, `d = 4`

Output: -1

Explanation: If you finish a job per day you will still have a free day. you cannot find a schedule for the given jobs.

Example 3:

Input: jobDifficulty = [1,1,1], d = 3

Output: 3

Explanation: The schedule is one job per day. total difficulty will be 3.

Constraints:

- $1 \leq \text{jobDifficulty.length} \leq 300$
- $0 \leq \text{jobDifficulty}[i] \leq 1000$
- $1 \leq d \leq 10$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minDifficulty(self, jobDifficulty: list[int], d: int) -> int:
        n = len(jobDifficulty)
        if d > n:
            return -1

        # dp[i][k] := the minimum difficulty to schedule the first i jobs in k days
        dp = [[math.inf] * (d + 1) for _ in range(n + 1)]
        dp[0][0] = 0

        for i in range(1, n + 1):
            for k in range(1, d + 1):
                maxDifficulty = 0 # max(job[j + 1..i])
                for j in range(i - 1, k - 2, -1): # 1-based
                    maxDifficulty = max(maxDifficulty, jobDifficulty[j]) # 0-based
                dp[i][k] = min(dp[i][k], dp[j][k - 1] + maxDifficulty)

        return dp[n][d]
```

• LeetDoocs

```

class Solution:
    def minDifficulty(self, jobDifficulty: List[int], d: int) -> int:
        n = len(jobDifficulty)
        f = [[inf] * (d + 1) for _ in range(n + 1)]
        f[0][0] = 0
        for i in range(1, n + 1):
            for j in range(1, min(d + 1, i + 1)):
                mx = 0
                for k in range(i, 0, -1):
                    mx = max(mx, jobDifficulty[k - 1])
                f[i][j] = min(f[i][j], f[k - 1][j - 1] + mx)
        return -1 if f[n][d] >= inf else f[n][d]

```

• SimplyLeet

```

def minDifficulty(jobDifficulty, d):
    n = len(jobDifficulty)
    # If there are fewer jobs than days, it's not possible to schedule
    if n < d:
        return -1

    # DP table: dp[i][day] represents the minimum difficulty for the first i
    # jobs in "day" days.
    dp = [[float('inf')] * (d + 1) for _ in range(n + 1)]
    dp[0][0] = 0

    # For each day, decide the partition of jobs.
    for day in range(1, d + 1):
        # i loops from "day" because at least one job must be done per day.
        for i in range(day, n + 1):
            max_difficulty = 0
            # Evaluate possible partitions by going backward.
            for j in range(i - 1, day - 2, -1):
                max_difficulty = max(max_difficulty, jobDifficulty[j])
                dp[i][day] = min(dp[i][day], dp[j][day - 1] + max_difficulty)

    return dp[n][d]

# Example usage:
print(minDifficulty([6,5,4,3,2,1], 2)) # Expected output: 7

```

◆ Quick Review

Key Insights

- Use dynamic programming to break the problem into subproblems by scheduling the first i jobs over a specific number of days.
- Define a DP state $dp[i][day]$ representing the minimum total difficulty for scheduling the first i jobs in "day" days.
- Transition by iterating backwards to form the current day's block, updating the maximum difficulty as you include more jobs.
- The answer is $dp[n][d]$, where n is the total number of jobs.
- Edge case: if the number of jobs is less than d , it's impossible to schedule, return -1 .

Space & Time

Time Complexity: $O(n^2 * d)$ – Iterating over days and for each, iterating over all possible partitions.

Space Complexity: $O(n * d)$ – For maintaining the DP table.

Solution

We utilize a 2D DP array to store our intermediate results. For every day (from 1 to d) and every job index, we update the DP table by considering all valid partitions where the current day receives a block of consecutive jobs. As we backtrack, we maintain the maximum difficulty encountered for the current day, and combine it with the previous optimal schedule stored in the DP table. This ensures that each partition minimizes the total difficulty across all days.

2D Grid DP

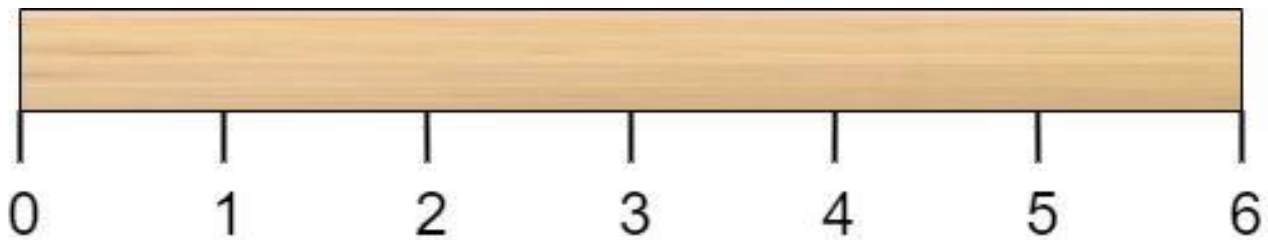
□ #1547 Minimum Cost to Cut a Stick

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Sorting
Lists: AlgoMaster 600

Problem

Given a wooden stick of length n units. The stick is labelled from 0 to n . For example, a stick of length 6 is labelled as follows:



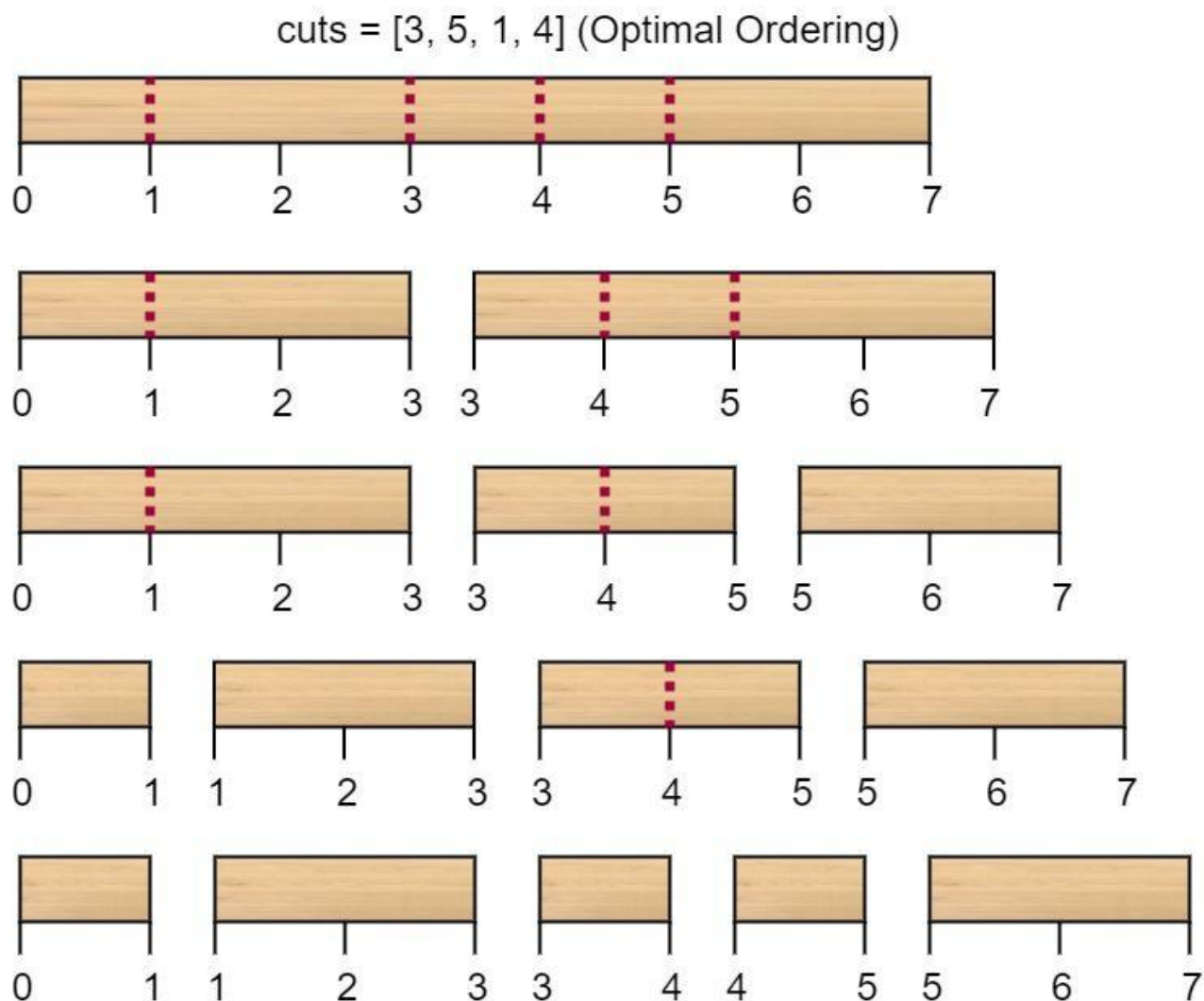
Given an integer array `cuts` where `cuts[i]` denotes a position you should perform a cut at. You should perform the cuts in order, you can change the order of the cuts as you wish.

The cost of one cut is the length of the stick to be cut, the total cost is the sum of costs of all cuts. When you cut a stick, it will be split into two smaller sticks (i.e. the sum of their lengths

is the length of the stick before the cut). Please refer to the first example for a better explanation.

Return the minimum total cost of the cuts.

Example 1:



Input: $n = 7$, cuts = [1,3,4,5]

Output: 16

Explanation: Using cuts order = [1, 3, 4, 5] as in the input leads to the following scenario:

The first cut is done to a rod of length 7 so the cost is 7. The second cut is done to a rod of length 6 (i.e. the second part of the first cut), the third is done to a rod of length 4 and the last cut is to a rod of length 3. The total cost is $7 + 6 + 4 + 3 = 20$.

Rearranging the cuts to be [3, 5, 1, 4] for example will lead to a scenario with total cost = 16 (as shown in the example photo $7 + 4 + 3 + 2 = 16$).

Example 2:

Input: $n = 9$, cuts = [5,6,1,4,2]

Output: 22

Explanation: If you try the given cuts ordering the cost will be 25.

There are much ordering with total cost ≤ 25 , for example, the order [4, 6, 5, 2, 1] has total cost = 22 which is the minimum possible.

Constraints:

- $2 \leq n \leq 106$
- $1 \leq \text{cuts.length} \leq \min(n - 1, 100)$
- $1 \leq \text{cuts}[i] \leq n - 1$
- All the integers in cuts array are distinct.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minCost(self, n: int, cuts: list[int]) -> int:
        A = sorted([0] + cuts + [n])

        @functools.lru_cache(None)
        def dp(i, j):
            if j - i <= 1:
                return 0

            return min(A[j] - A[i] + dp(i, k) + dp(k, j) for k in range(i + 1, j))
```

```
return dp(0, len(A) - 1)
```

```
class Solution:
    def minCost(self, n: int, cuts: list[int]) -> int:
        arr = sorted([0] + cuts + [n])

        dp = [[0] * len(arr) for _ in range(len(arr))]

        for d in range(2, len(arr)):
            for i in range(len(arr) - d):
                j = i + d
                dp[i][j] = math.inf

                for k in range(i + 1, j):
                    dp[i][j] = min(dp[i][j], arr[j] - arr[i] + dp[i][k] + dp[k][j])

        return dp[0][len(arr) - 1]
```

• LeetCode

```
class Solution:
    def minCost(self, n: int, cuts: List[int]) -> int:
        cuts.extend([0, n])
        cuts.sort()
        m = len(cuts)
        f = [[0] * m for _ in range(m)]
        for l in range(2, m):
            for i in range(m - l):
                j = i + l
                f[i][j] = inf

                for k in range(i + 1, j):
                    f[i][j] = min(f[i][j], f[i][k] + f[k][j] + cuts[j] - cuts[i])

        return f[0][-1]
```

• SimplyLeet


```
# Python solution for Minimum Cost to Cut a Stick

def minCost(n, cuts):
    # Sort the cuts and add boundaries 0 and n.
    positions = [0] + sorted(cuts) + [n]
    m = len(positions)

    # Initialize dp table with 0's.
    dp = [[0] * m for _ in range(m)]

    # Build the dp table from smaller segments (gap) to larger ones.
    for gap in range(2, m): # gap is the window size between i and j.
        for i in range(m - gap):
            j = i + gap
            min_cost = float('inf')
            # Try every possible cut between positions[i] and positions[j]
            for k in range(i + 1, j):
                cost = dp[i][k] + dp[k][j] + positions[j] - positions[i]
                min_cost = min(min_cost, cost)
            dp[i][j] = min_cost

    return dp[0][m - 1]

# Example usage:
print(minCost(7, [1, 3, 4, 5])) # Output: 16
```

◆ Quick Review

Key Insights

- Sort the list of cuts and include the two boundaries (0 and n) to enable easier segment computation.
- Use dynamic programming where $dp[i][j]$ represents the minimum cost to cut the stick between $positions[i]$ and $positions[j]$.

- The cost to perform a cut is the length of the current segment ($\text{positions}[j] - \text{positions}[i]$).
- For each segment, try each cut between the boundaries to determine the optimal cost using the recurrence:
 - $\text{dp}[i][j] = \min(\text{dp}[i][k] + \text{dp}[k][j] + (\text{positions}[j] - \text{positions}[i]))$ for every k such that $i \leq k \leq j$.
- The problem exhibits optimal substructure, making it suitable for a bottom-up DP approach.

Space & Time

Time Complexity: $O(m^3)$, where m is the number of cut positions plus two (boundaries). Given that m is at most 102, this cubic time is acceptable.

Space Complexity: $O(m^2)$ to store the dynamic programming table.

Solution

We approach this problem using a bottom-up dynamic programming algorithm. First, sort the cuts array and add the boundaries 0 and n . This results in an array `positions` where $\text{positions}[0] = 0$ and $\text{positions}[m-1] = n$. Then,

define $dp[i][j]$ as the minimum cost to cut the segment from $positions[i]$ to $positions[j]$. For each interval, try every possible intermediate cut position k (where $i \leq k \leq j$) to determine the cost of cutting that segment.

The recurrence relation is:

$dp[i][j] = \min(dp[i][k] + dp[k][j] + (positions[j] - positions[i]))$, for every valid k

Initialize $dp[i][j] = 0$ for segments that have no cuts between them (i.e. $j = i + 1$). Build the solution from smaller segments up to the entire stick. Finally, the answer is $dp[0][m - 1]$.

2D Grid DP

□ #1931 Painting a Grid With Three Different Colors

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming
Lists: AlgoMaster 600

Problem

You are given two integers m and n . Consider an $m \times n$ grid where each cell is initially white. You can paint each cell red, green, or blue. All cells must be painted.

Return the number of ways to color the grid with no two adjacent cells having the same color. Since the answer can be very large, return it modulo $10^9 + 7$.

Example 1:

Input: $m = 1, n = 1$

Output: 3

Explanation: The three possible colorings are shown in the image above.

Example 2:

Input: $m = 1, n = 2$

Output: 6

Explanation: The six possible colorings are shown in the image above.

Example 3:

Input: m = 5, n = 5

Output: 580986

Constraints:

- $1 \leq m \leq 5$
- $1 \leq n \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def colorTheGrid(self, m: int, n: int) -> int:
        def getColor(mask: int, r: int) -> int:
            return mask >> r * 2 & 3

        def setColor(mask: int, r: int, color: int) -> int:
            return mask | color << r * 2

        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def dp(r: int, c: int, prevColMask: int, currColMask: int) -> int:
            if c == n:
                return 1
            if r == m:
                return dp(0, c + 1, currColMask, 0)

            ans = 0

            # 1 := red, 2 := green, 3 := blue
```

```

for color in range(1, 4):
    if getColor(prevColMask, r) == color:
        continue
    if r > 0 and getColor(currColMask, r - 1) == color:
        continue
    ans += dp(r + 1, c, prevColMask, setColor(currColMask, r, color))
    ans %= MOD

```

```

return ans

```

```

return dp(0, 0, 0, 0)

```

• LeetDoocs

```

class Solution:
    def colorTheGrid(self, m: int, n: int) -> int:
        def f1(x: int) -> bool:
            last = -1
            for _ in range(m):
                if x % 3 == last:
                    return False
                last = x % 3
                x //= 3
            return True

```

```

        def f2(x: int, y: int) -> bool:
            for _ in range(m):
                if x % 3 == y % 3:
                    return False
                x, y = x // 3, y // 3
            return True

```

```

mod = 10**9 + 7
mx = 3**m

```

```

valid = {i for i in range(mx) if f1(i)}
d = defaultdict(list)
for x in valid:
    for y in valid:
        if f2(x, y):
            d[x].append(y)
f = [int(i in valid) for i in range(mx)]
for _ in range(n - 1):
    g = [0] * mx
    for i in valid:
        for j in d[i]:
            g[i] = (g[i] + f[j]) % mod
    f = g
return sum(f) % mod

```

• SimplyLeet

```

MOD = 10**9 + 7

# Recursively generate all valid states for one column
def generate_valid_states(m, colors=3):
    states = []
    def backtrack(row, current):
        if row == m:
            states.append(tuple(current))
            return
        for color in range(colors):
            if row > 0 and current[-1] == color:
                continue
            current.append(color)
            backtrack(row + 1, current)
            current.pop()
    backtrack(0, [])
    return states

# Check if two states are compatible (i.e., horizontal adjacent cells have
different colors)
def are_compatible(state1, state2):

```

```

    return all(c1 != c2 for c1, c2 in zip(state1, state2))

def colorTheGrid(m, n):
    valid_states = generate_valid_states(m)
    state_count = len(valid_states)

    # Precompute valid transitions between states for adjacent columns
    transitions = [[False] * state_count for _ in range(state_count)]
    for i in range(state_count):
        for j in range(state_count):
            if are_compatible(valid_states[i], valid_states[j]):
                transitions[i][j] = True

    dp = [1] * state_count # Initialize dp for the first column
    for _ in range(1, n):
        new_dp = [0] * state_count
        for i in range(state_count):
            for j in range(state_count):
                if transitions[i][j]:
                    new_dp[j] = (new_dp[j] + dp[i]) % MOD

        dp = new_dp
    return sum(dp) % MOD

# Example usage:
print(colorTheGrid(1, 1)) # Expected Output: 3
print(colorTheGrid(1, 2)) # Expected Output: 6
print(colorTheGrid(5, 5)) # Expected Output: 580986

```

◆ Quick Review

Key Insights

- Since m is small ($m \leq 5$) but n can be large ($n \leq 1000$), it is efficient to treat each column as a state.

- Precompute all valid color configurations for a single column (ensuring vertical adjacent cells differ).
- For any two consecutive columns, ensure that colors in corresponding rows (horizontal adjacent cells) are different.
- Use dynamic programming where $dp[i]$ represents the number of ways to paint up to the current column ending with the i -th valid state.
- Precompute valid transitions between states to avoid repeated checks.

Space & Time

Time Complexity: $O(n * S^2)$ where S is the number of valid states (at most $3 * 2^{(m-1)}$).

Space Complexity: $O(S^2 + S)$ for storing state transition information and DP arrays.

Solution

We solve the problem using a dynamic programming (DP) approach on columns:

- Generate all valid states for a column of height m ensuring vertical adjacent cells are painted with different colors.

- Precompute a 2D boolean matrix capturing whether state i is compatible with state j for adjacent columns (i.e., for every row, the color in state i is different from the color in state j).
- Initialize the DP array with 1 for each valid state as the first column can be painted in any valid configuration.
- For each subsequent column, update the DP counts using the precomputed transitions.
- Finally, return the sum of the DP array values modulo $10^9 + 7$.

Below are solutions in Python, JavaScript, C++, and Java with detailed comments.

2D Grid DP

□ #3651 Minimum Cost Path with Teleportations

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: AlgoMaster 600

Problem

You are given a $m \times n$ 2D integer array `grid` and an integer k . You start at the top-left cell $(0, 0)$ and your goal is to reach the bottom-right cell $(m - 1, n - 1)$.

There are two types of moves available:

- **Normal move:** You can move right or down from your current cell (i, j) , i.e. you can move to $(i, j + 1)$ (right) or $(i + 1, j)$ (down). The cost is the value of the destination cell.
- **Teleportation:** You can teleport from any cell (i, j) , to any cell (x, y) such that $\text{grid}[x][y] \leq \text{grid}[i][j]$; the cost of this move is 0. You may teleport at most k times.

Return the minimum total cost to reach cell $(m - 1, n - 1)$ from $(0, 0)$.

Example 1:

Input: `grid = [[1,3,3],[2,5,4],[4,3,5]]`, `k = 2`

Output: 7

Explanation:

Initially we are at (0, 0) and cost is 0.

The minimum cost to reach bottom-right cell is 7.

Example 2:

Input: `grid = [[1,2],[2,3],[3,4]]`, `k = 1`

Output: 9

Explanation:

Initially we are at (0, 0) and cost is 0.

The minimum cost to reach bottom-right cell is 9.

Constraints:

- $2 \leq m, n \leq 80$
- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $0 \leq \text{grid}[i][j] \leq 104$
- $0 \leq k \leq 10$

Community Solutions (Python)

- LeetDoocs

```

class Solution:
    def minCost(self, grid: List[List[int]], k: int) -> int:
        m, n = len(grid), len(grid[0])
        f = [[[inf] * n for _ in range(m)] for _ in range(k + 1)]
        f[0][0][0] = 0
        for i in range(m):
            for j in range(n):
                if i:
                    f[0][i][j] = min(f[0][i][j], f[0][i - 1][j] + grid[i][j])
                if j:
                    f[0][i][j] = min(f[0][i][j], f[0][i][j - 1] + grid[i][j])
        g = defaultdict(list)
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                g[x].append((i, j))
        keys = sorted(g, reverse=True)
        for t in range(1, k + 1):
            mn = inf
            for key in keys:
                pos = g[key]

                for i, j in pos:
                    mn = min(mn, f[t - 1][i][j])
                for i, j in pos:
                    f[t][i][j] = mn
            for i in range(m):
                for j in range(n):
                    if i:
                        f[t][i][j] = min(f[t][i][j], f[t][i - 1][j] + grid[i][j])
                    if j:
                        f[t][i][j] = min(f[t][i][j], f[t][i][j - 1] + grid[i][j])

        return min(f[t][m - 1][n - 1] for t in range(k + 1))

```

String DP

Round 9 · Low · Hard · 4 questions

String DP

□ #44 Wildcard Matching

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Greedy ·
Recursion

Lists: AlgoMaster 600

Problem

Given an input string (s) and a pattern (p), implement wildcard pattern matching with support for '?' and '*' where:

- '?' Matches any single character.
- '*' Matches any sequence of characters (including the empty sequence).

The matching should cover the entire input string (not partial).

Example 1:

Input: s = "aa", p = "a"
Output: false
Explanation: "a" does not match the entire string "aa".

Example 2:

Input: s = "aa", p = "*"
Output: true
Explanation: '*' matches any sequence.

Example 3:

Input: s = "cb", p = "?a"

Output: false

Explanation: '?' matches 'c', but the second letter is 'a', which does not match 'b'.

Constraints:

- $0 \leq s.length, p.length \leq 2000$
- s contains only lowercase English letters.
- p contains only lowercase English letters, '?' or '*'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m = len(s)
        n = len(p)
        # dp[i][j] := True if s[0..i) matches p[0..j)
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True

        def isMatch(i: int, j: int) -> bool:
            return i >= 0 and p[j] == '?' or s[i] == p[j]

        for j, c in enumerate(p):
            if c == '*':
                dp[0][j + 1] = dp[0][j]

        for i in range(m):
            for j in range(n):
                if p[j] == '*':
                    matchEmpty = dp[i + 1][j]
                    matchSome = dp[i][j + 1]
```



```

        dp[i + 1][j + 1] = matchEmpty or matchSome
    elif isMatch(i, j):
        dp[i + 1][j + 1] = dp[i][j]

    return dp[m][n]

```

• LeetDoocs

```

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        @cache
        def dfs(i: int, j: int) -> bool:
            if i >= len(s):
                return j >= len(p) or (p[j] == "*" and dfs(i, j + 1))
            if j >= len(p):
                return False
            if p[j] == "*":
                return dfs(i + 1, j) or dfs(i + 1, j + 1) or dfs(i, j + 1)

            return (p[j] == "?" or s[i] == p[j]) and dfs(i + 1, j + 1)

        return dfs(0, 0)

```

```

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for j in range(1, n + 1):
            if p[j - 1] == "*":
                f[0][j] = f[0][j - 1]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if p[j - 1] == "*":
                    f[i][j] = f[i - 1][j] or f[i][j - 1] or f[i - 1][j - 1]
                else:
                    f[i][j] = f[i - 1][j - 1] and (
                        p[j - 1] == "?" or s[i - 1] == p[j - 1]
                    )
        return f[m][n]

```

• SimplyLeet

```
def isMatch(s: str, p: str) -> bool:
    n, m = len(s), len(p)
    # Create a DP table with (n+1) x (m+1) initialized to False
    dp = [[False] * (m + 1) for _ in range(n + 1)]
    # Empty string matches empty pattern
    dp[0][0] = True

    # Initialize first row for patterns that have '*' which can match empty
    string
    for j in range(1, m + 1):
        if p[j - 1] == '*':
            dp[0][j] = dp[0][j - 1]

    # Build the DP table
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if p[j - 1] == '?' or p[j - 1] == s[i - 1]:
                # Characters match or pattern has '?' so inherit the previous
                state
                dp[i][j] = dp[i - 1][j - 1]
            elif p[j - 1] == '*':
                # '*' matches empty sequence: dp[i][j-1] or one/more
                characters: dp[i-1][j]
                dp[i][j] = dp[i][j - 1] or dp[i - 1][j]
            else:
                dp[i][j] = False

    return dp[n][m]

# Example usage:
print(isMatch("aa", "a"))    # False
print(isMatch("aa", "*"))    # True
print(isMatch("cb", "?a"))   # False
```

◆ Quick Review

Key Insights

- Use dynamic programming to track matching status between prefixes of s and p .
- When $p[j]$ is a regular character or '?', check for character match (or any character match for '?').
- When $p[j]$ is '*', it can match an empty sequence ($dp[i][j-1]$) or extend a previous match ($dp[i-1][j]$).
- A greedy two-pointer approach can also be considered, but the DP solution is clearer for handling worst-case scenarios.

Space & Time

Time Complexity: $O(n * m)$, where n is the length of s and m is the length of p .

Space Complexity: $O(n * m)$, due to the DP table used to store matching results.

Solution

We solve the problem using a dynamic programming approach by constructing a boolean table dp , where $dp[i][j]$ indicates whether the first i characters of s match the first j characters of p . The recurrence is as follows:

- If $p[j-1]$ is a normal character or '?', then $dp[i][j] = dp[i-1][j-1]$ if the current characters match (or $p[j-1]$ is '?').
- If $p[j-1]$ is '*', then $dp[i][j]$ is true if either the star matches no characters ($dp[i][j-1]$) or if it matches one more character ($dp[i-1][j]$). A careful initialization of the dp table is required, especially for patterns that start with '*' which can match empty strings.

String DP

□ #140 Word Break II

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Dynamic
Programming · Backtracking · Trie ·
Memoization

Lists: AlgoMaster 600

Problem

Given a string *s* and a dictionary of strings *wordDict*, add spaces in *s* to construct a sentence where each word is a valid dictionary word. Return all such possible sentences in any order.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

```
Input: s = "catsanddog", wordDict = ["cat","cats","and","sand","dog"]  
Output: ["cats and dog","cat sand dog"]
```

Example 2:

```
Input: s = "pineapplepenapple", wordDict =  
["apple","pen","applepen","pine","pineapple"]  
Output: ["pine apple pen apple","pineapple pen apple","pine applepen apple"]  
Explanation: Note that you are allowed to reuse a dictionary word.
```

Example 3:

Input: s = "catsanddog", wordDict = ["cats","dog","sand","and","cat"]
 Output: []

Constraints:

- $1 \leq s.length \leq 20$
- $1 \leq wordDict.length \leq 1000$
- $1 \leq wordDict[i].length \leq 10$
- s and wordDict[i] consist of only lowercase English letters.
- All the strings of wordDict are unique.
- Input is generated in a way that the length of the answer doesn't exceed 105.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> list[str]:
        wordSet = set(wordDict)

        @functools.lru_cache(None)
        def wordBreak(s: str) -> list[str]:
            ans = []

            # 1 <= len(prefix) < len(s)
            for i in range(1, len(s)):
                prefix = s[:i]
                suffix = s[i:]
                if prefix in wordSet:
                    ans.append(prefix + wordBreak(suffix))
            return ans
```

```

    prefix = s[:i]
    suffix = s[i:]
    if prefix in wordSet:
        for word in wordBreak(suffix):
            ans.append(prefix + ' ' + word)

    # `wordSet` contains the whole string s, so don't add any space.
    if s in wordSet:
        ans.append(s)

    return ans

return wordBreak(s)

```

• LeetDoocs

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word):
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, word):
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return False

```

```

        node = node.children[idx]
        return node.is_end

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> List[str]:
        def dfs(s):
            if not s:
                return [[]]
            res = []
            for i in range(1, len(s) + 1):
                if trie.search(s[:i]):
                    for v in dfs(s[i:]):
                        res.append([s[:i]] + v)
            return res

        trie = Trie()
        for w in wordDict:
            trie.insert(w)
        ans = dfs(s)

        return [' '.join(v) for v in ans]
```

• SimplyLeet

```

# Python solution using memoized recursion (DFS)

def wordBreak(s, wordDict):
    word_set = set(wordDict)    # Use a set for O(1) lookups
    memo = {}                   # dictionary to memoize results from each start index

    def dfs(start):
        # if reached the end of the string, return a list with empty string
        if start == len(s):
            return [""]

        for end in range(start + 1, len(s) + 1):
            if s[start:end] in word_set:
                for res in dfs(end):
                    memo[start] = [s[start:end] + ' ' + res]
```



```
    if start in memo:
        return memo[start]

    sentences = []
    for end in range(start + 1, len(s) + 1):
        word = s[start:end]
        if word in word_set:
            # recursive call to get sentences for the remaining substring
            sub_sentences = dfs(end)

            # build sentences with proper spacing
            for sub in sub_sentences:
                sentence = word + (" " + sub if sub else "")
                sentences.append(sentence)

    memo[start] = sentences # cache result
    return sentences

return dfs(0)

# Example usage:
if __name__ == "__main__":
    s = "catsanddog"
    wordDict = ["cat", "cats", "and", "sand", "dog"]
    result = wordBreak(s, wordDict)
    print(result) # Expected: ["cats and dog", "cat sand dog"]
```

◆ Quick Review

Key Insights

- Use backtracking (DFS) to try splitting the string at every possible index.
- Utilize memoization to store the list of valid sentences starting from a given index to avoid redundant computations.

- The problem is naturally solved through recursion with memoization (a top-down dynamic programming approach).

Space & Time

Time Complexity: $O(n^2 * m)$ in the worst case, where n is the length of the string and m is the average number of words found per recursion; memoization helps to reduce repeated work.

Space Complexity: $O(n * m)$ for the recursion stack and memoization storage, where m represents the average length of the sentence list from each index.

Solution

We use a depth-first search (DFS) that tries every possible break point in the string. At each index, if the current substring is in the dictionary, we recursively solve for the remainder of the string. Memoization is used to cache results for each index to avoid redundant computations. The solution builds the final sentences by concatenating the current word with the sentence fragments returned by the recursive call.

String DP

□ #664 Strange Printer

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: AlgoMaster 600

Problem

There is a strange printer with the following two special properties:

- The printer can only print a sequence of the same character each time.
- At each turn, the printer can print new characters starting from and ending at any place and will cover the original existing characters.

Given a string s , return the minimum number of turns the printer needed to print it.

Example 1:

Input: $s = \text{"aaabbb"}$

Output: 2

Explanation: Print "aaa" first and then print "bbb".

Example 2:

Input: s = "aba"

Output: 2

Explanation: Print "aaa" first and then print "b" from the second place of the string, which will cover the existing character 'a'.

Constraints:

- $1 \leq s.length \leq 100$
- s consists of lowercase English letters.

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def strangePrinter(self, s: str) -> int:
        n = len(s)
        f = [[inf] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            f[i][i] = 1
            for j in range(i + 1, n):
                if s[i] == s[j]:
                    f[i][j] = f[i][j - 1]
                else:
                    for k in range(i, j):
                        f[i][j] = min(f[i][j], f[i][k] + f[k + 1][j])
        return f[0][-1]
```

• SimplyLeet

```
# Python solution for Strange Printer

def strangePrinter(s: str) -> int:
    n = len(s)
    # Initialize a 2D dp table with zeros.
    dp = [[0] * n for _ in range(n)]

    # Base case: each single character requires 1 turn.
    for i in range(n):
        dp[i][i] = 1
```

```

# Process all substrings of increasing lengths.
for length in range(2, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1
        # Assume printing s[j] separately after s[i...j-1].
        dp[i][j] = dp[i][j - 1] + 1
        # Try merging operations where s[k] equals s[j].
        for k in range(i, j):
            if s[k] == s[j]:
                # When merging, cost is dp[i][k] plus dp[k+1][j-1], if any.
                cost = dp[i][k] + (dp[k + 1][j - 1] if k + 1 <= j - 1 else
0)
                dp[i][j] = min(dp[i][j], cost)
return dp[0][n - 1]

# Example usage:
s = "aba"
print(strangePrinter(s)) # Output: 2

```

◆ Quick Review

Key Insights

- Use dynamic programming where $dp[i][j]$ represents the minimum turns needed to print the substring $s[i...j]$.
- Base case: $dp[i][i] = 1$, since a single character takes one turn.
- If $s[i]$ and $s[j]$ are the same, the last print can cover both, potentially reducing the turn count.

- For every substring, try all possible partitions and combine results, taking advantage of matching characters to merge operations.

Space & Time

Time Complexity: $O(n^3)$ in the worst case because of three nested loops to fill in the dp table.

Space Complexity: $O(n^2)$ due to the dp table storing solutions for each substring.

Solution

We use a 2D dynamic programming table `dp` where `dp[i][j]` indicates the minimum turns needed to print substring `s[i...j]`. Initialize `dp[i][i] = 1` for all `i`. For each substring of length ≥ 2 , set `dp[i][j] = dp[i][j-1] + 1` by default (printing `s[j]` separately). Then, iterate through every index `k` from `i` to `j - 1`; if `s[k]` equals `s[j]`, merge the operation by calculating `dp[i][j] = min(dp[i][j], dp[i][k] + dp[k+1][j-1])` while handling edge cases. This approach systematically breaks down the problem into smaller overlapping subproblems.

String DP

□ #1092 Shortest Common Supersequence **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: AlgoMaster 600

Problem

Given two strings `str1` and `str2`, return the shortest string that has both `str1` and `str2` as subsequences. If there are multiple valid strings, return any of them.

A string `s` is a subsequence of string `t` if deleting some number of characters from `t` (possibly 0) results in the string `s`.

Example 1:

Input: `str1 = "abac", str2 = "cab"`

Output: `"cabac"`

Explanation:

`str1 = "abac"` is a subsequence of `"cabac"` because we can delete the first `"c"`.

`str2 = "cab"` is a subsequence of `"cabac"` because we can delete the last `"ac"`.

The answer provided is the shortest such string that satisfies these properties.

Example 2:

Input: `str1 = "aaaaaaaa", str2 = "aaaaaaaa"`

Output: `"aaaaaaaa"`

Constraints:

- $1 \leq \text{str1.length}, \text{str2.length} \leq 1000$
- str1 and str2 consist of lowercase English letters.

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def shortestCommonSupersequence(self, str1: str, str2: str) -> str:
        m, n = len(str1), len(str2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if str1[i - 1] == str2[j - 1]:
                    f[i][j] = f[i - 1][j - 1] + 1
                else:
                    f[i][j] = max(f[i - 1][j], f[i][j - 1])

        ans = []
        i, j = m, n
        while i > 0 or j > 0:
            if i == 0:
                j -= 1
                ans.append(str2[j])
            elif j == 0:
                i -= 1
                ans.append(str1[i])
            else:
                if f[i][j] == f[i - 1][j]:
                    i -= 1
                    ans.append(str1[i])
                elif f[i][j] == f[i][j - 1]:
                    j -= 1
                    ans.append(str2[j])
                else:
                    i, j = i - 1, j - 1
                    ans.append(str1[i])
        return ''.join(ans[::-1])
```


• SimplyLeet

Python solution using DP for LCS and reconstruction for SCS.

```
def shortestCommonSupersequence(str1: str, str2: str) -> str:
    m, n = len(str1), len(str2)
    # DP table initialization: dp[i][j] will hold LCS for str1[:i] and str2[:j]
    dp = ["" for _ in range(n + 1)] for _ in range(m + 1)]

    # Build the dp table for LCS string reconstruction.
    for i in range(1, m + 1):
        for j in range(1, n + 1):

            if str1[i - 1] == str2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + str1[i - 1]
            else:
                # Choose the longer LCS when characters do not match
                if len(dp[i - 1][j]) > len(dp[i][j - 1]):
                    dp[i][j] = dp[i - 1][j]
                else:
                    dp[i][j] = dp[i][j - 1]

    # Reconstruct the shortest common supersequence using the LCS.

    lcs = dp[m][n]
    i = j = 0
    scs = []
    # Append non-LCS characters from str1 and str2 until the LCS characters
    appear.
    for c in lcs:
        # Append all characters from str1 until we hit the LCS character.
        while str1[i] != c:
            scs.append(str1[i])
            i += 1
        # Append all characters from str2 until we hit the LCS character.
```

```
while str2[j] != c:
    scs.append(str2[j])
    j += 1
# Now append the LCS character.
scs.append(c)
i += 1
j += 1

# Append any remaining characters after processing the LCS.
scs.extend(list(str1[i:]))

scs.extend(list(str2[j:]))
return "".join(scs)

# Example test cases:
print(shortestCommonSupersequence("abac", "cab")) # Expected output: "cabac"
(or any valid SCS)
print(shortestCommonSupersequence("aaaaaaaa", "aaaaaaaa")) # Expected output:
"aaaaaaaa"
```

◆ Quick Review

Key Insights

- The problem can be solved by leveraging the concept of the Longest Common Subsequence (LCS).
- By first finding the LCS of str1 and str2, we can merge the two strings while ensuring that common parts (LCS) are not repeated unnecessarily.
- Dynamic Programming is used to compute the LCS.

- Once the LCS is available, the final result is built by interleaving characters from str1 and str2 while aligning with the LCS.

Space & Time

Time Complexity: $O(m * n)$, where m and n are the lengths of str1 and str2 respectively.

Space Complexity: $O(m * n)$ for the DP table to store LCS lengths, with additional space for the output string.

Solution

The solution is divided into two main steps:

- Compute the LCS of the two strings using a Dynamic Programming table. This table $dp[i][j]$ represents the length of the LCS of $str1[0...i-1]$ and $str2[0...j-1]$.
- Reconstruct the shortest common supersequence by traversing the two strings in parallel and using the LCS as a guide. For every character in the LCS, append all non-LCS characters from str1 and str2 until that common character is reached, then add the LCS character. Append any remaining characters after processing the LCS.

The key observation is that by including the non-matching parts between matched LCS characters, we ensure both strings are subsequences of the constructed result, and the result is the shortest possible.

Tree / Graph DP

Round 9 · Low · Hard · 2 questions

Tree / Graph DP

□ #834 Sum of Distances in Tree

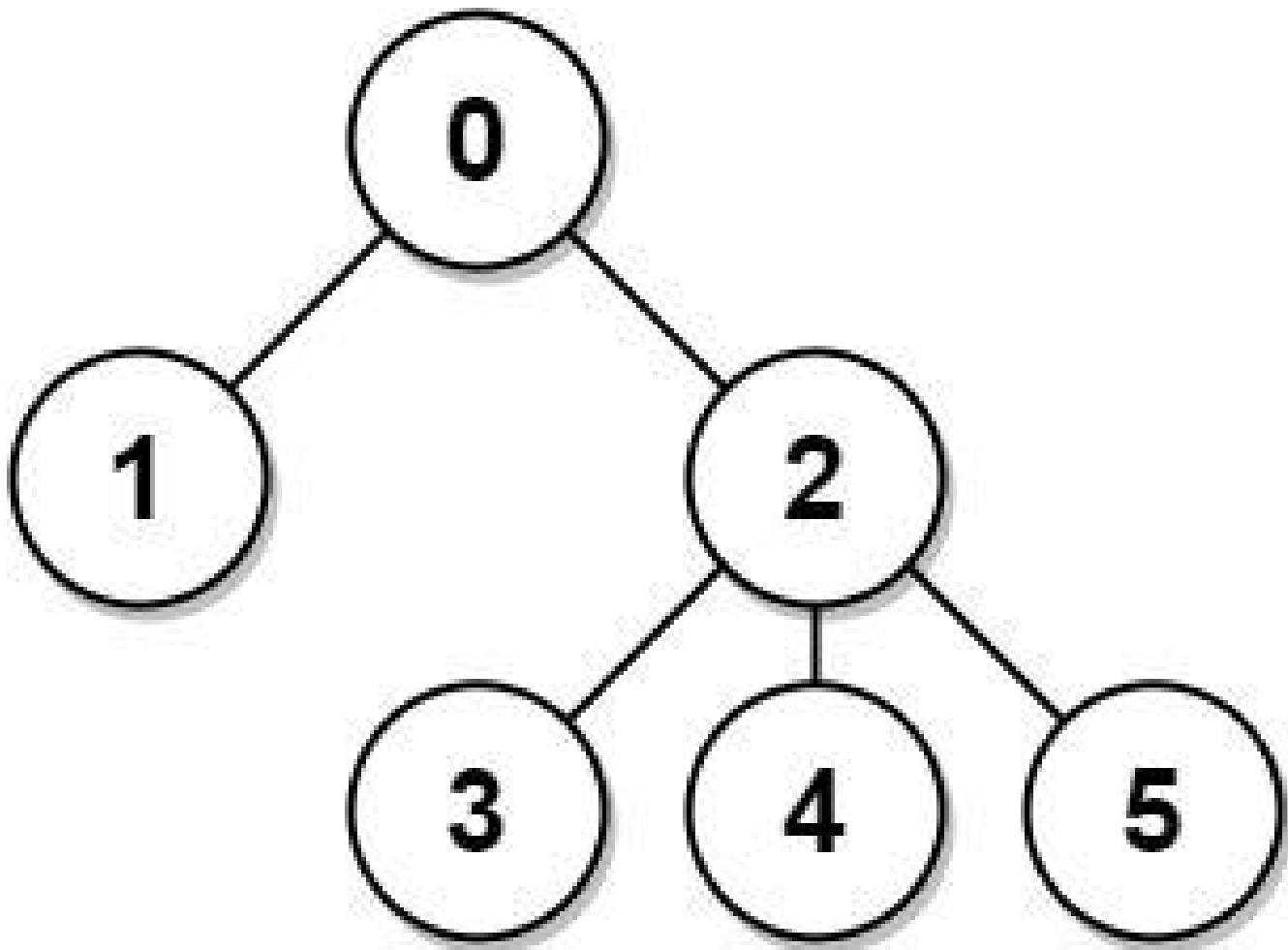
HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Tree · Depth-First
Search · Graph Theory
Lists: AlgoMaster 600

Problem

There is an undirected connected tree with n nodes labeled from 0 to $n - 1$ and $n - 1$ edges. You are given the integer n and the array `edges` where `edges[i] = [ai, bi]` indicates that there is an edge between nodes a_i and b_i in the tree. Return an array `answer` of length n where `answer[i]` is the sum of the distances between the i th node in the tree and all other nodes.

Example 1:



Input: $n = 6$, $edges = [[0,1],[0,2],[2,3],[2,4],[2,5]]$

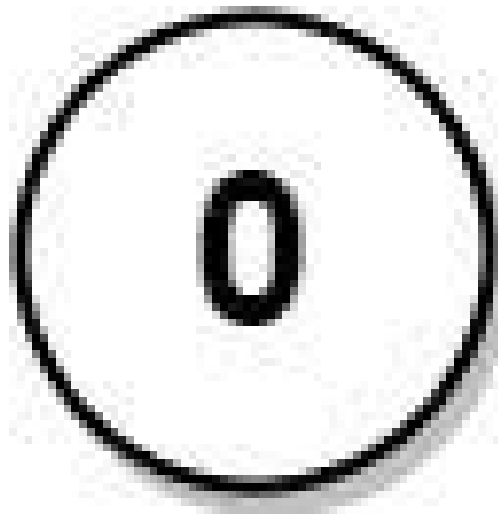
Output: $[8,12,6,10,10,10]$

Explanation: The tree is shown above.

We can see that $dist(0,1) + dist(0,2) + dist(0,3) + dist(0,4) + dist(0,5)$ equals $1 + 1 + 2 + 2 + 2 = 8$.

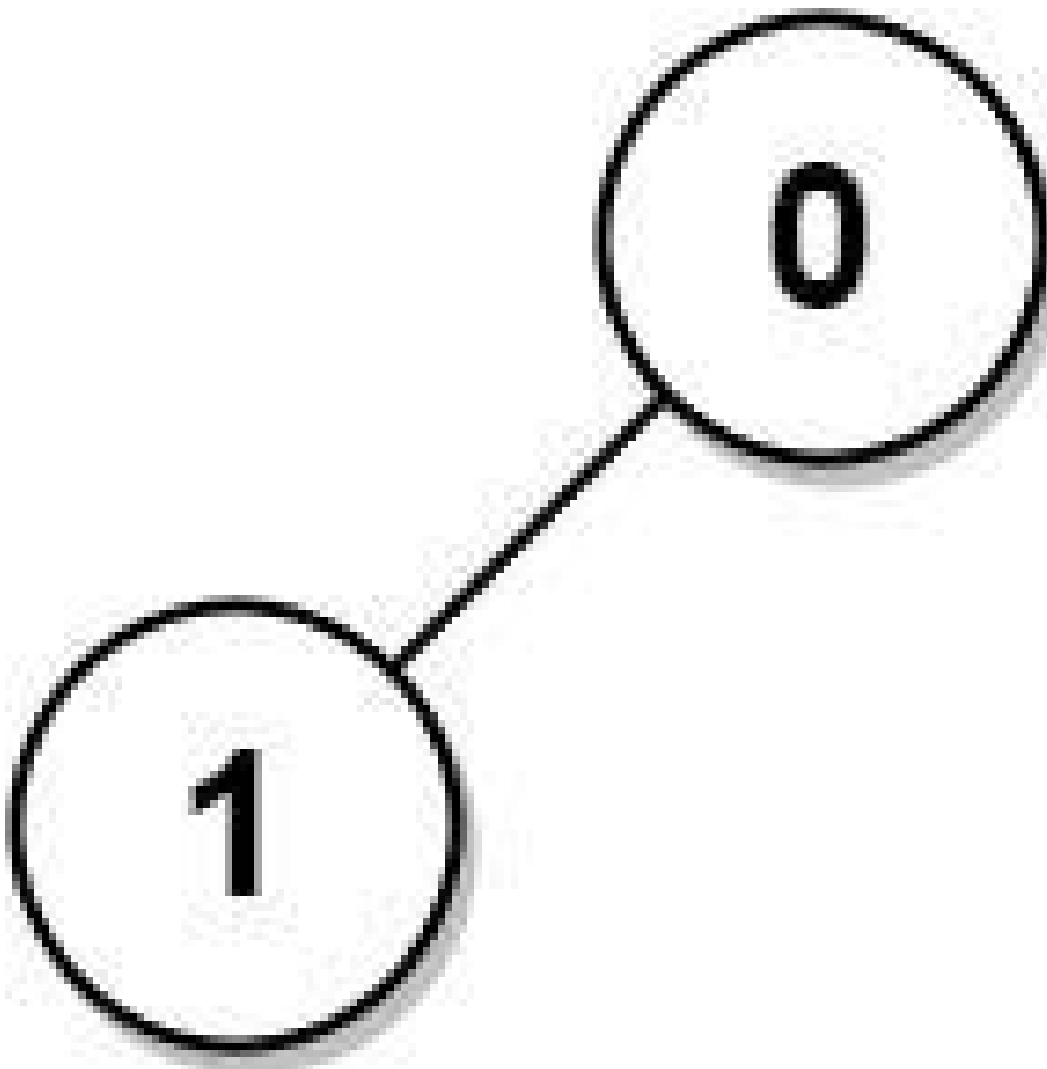
Hence, $answer[0] = 8$, and so on.

Example 2:



Input: $n = 1$, $edges = []$
Output: $[0]$

Example 3:



Input: $n = 2$, $edges = [[1,0]]$

Output: $[1,1]$

Constraints:

- $1 \leq n \leq 3 * 10^4$
- $edges.length == n - 1$
- $edges[i].length == 2$
- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- The given input represents a valid tree.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sumOfDistancesInTree(self, n: int, edges: list[list[int]]) -> list[int]:
        ans = [0] * n
        count = [1] * n
        tree = [set() for _ in range(n)]

        for u, v in edges:
            tree[u].add(v)
            tree[v].add(u)

        def postorder(u: int, prev: int) -> None:
            for v in tree[u]:
                if v == prev:
                    continue
                postorder(v, u)
                count[u] += count[v]
                ans[u] += ans[v] + count[v]

        def preorder(u: int, prev: int) -> None:
            for v in tree[u]:
                if v == prev:
                    continue
                # count[v] us are 1 step closer from v than prev.
                # (n - count[v]) us are 1 step farther from v than prev.
                ans[v] = ans[u] - count[v] + (n - count[v])
                preorder(v, u)

        postorder(0, -1)
        preorder(0, -1)
        return ans
```

• LeetDoocs

```

class Solution:
    def sumOfDistancesInTree(self, n: int, edges: List[List[int]]) -> List[int]:
        def dfs1(i: int, fa: int, d: int):
            ans[0] += d
            size[i] = 1
            for j in g[i]:
                if j != fa:
                    dfs1(j, i, d + 1)
                    size[i] += size[j]

        def dfs2(i: int, fa: int, t: int):
            ans[i] = t
            for j in g[i]:
                if j != fa:
                    dfs2(j, i, t - size[j] + n - size[j])

        g = defaultdict(list)
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)

        ans = [0] * n
        size = [0] * n
        dfs1(0, -1, 0)
        dfs2(0, -1, ans[0])
        return ans

```

• SimplyLeet

```

# Python solution
class Solution:
    def sumOfDistancesInTree(self, n: int, edges: List[List[int]]) -> List[int]:
        # Build tree as an adjacency list
        tree = [[] for _ in range(n)]
        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        # Initialize result and count arrays

```

```
res = [0] * n
count = [1] * n # each node counts as 1

# Postorder DFS: compute count and res for subtrees
def dfs(node: int, parent: int):
    for child in tree[node]:
        if child == parent:
            continue
        dfs(child, node)
        count[node] += count[child]

        res[node] += res[child] + count[child]
dfs(0, -1)

# Preorder DFS: update results using parent's information
def dfs2(node: int, parent: int):
    for child in tree[node]:
        if child == parent:
            continue
        # Update result for child using parent's result formula
        res[child] = res[node] - count[child] + (n - count[child])

        dfs2(child, node)
dfs2(0, -1)

return res
```

◆ Quick Review

Key Insights

- Use tree dynamic programming with two DFS traversals.
- The first DFS computes, for each node, the sum of distances from that node's subtree and the count of nodes in that subtree.

- The second DFS propagates the computed distances from the root to all children using the relation that adjusts the distances based on subtree sizes.
- This method avoids recomputation, leading to an efficient $O(n)$ solution.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ (due to tree adjacency list and recursion stack)

Solution

We use a two-step DFS approach:

- In the first DFS (postorder), for each node we compute:
 - $\text{count}[\text{node}]$: number of nodes in the subtree (including itself).
 - $\text{res}[\text{node}]$: sum of distances from node to all nodes in its subtree.
- In the second DFS (preorder), we use the parent's result to update each child's result:
 - When moving from parent to child, update $\text{res}[\text{child}] = \text{res}[\text{parent}] - \text{count}[\text{child}] + (n - \text{count}[\text{child}])$.

- This transformation works because moving to a child subtracts the distance for all nodes in the child's subtree (as they get closer by 1) and adds the distance for the rest (as they get farther by 1).

The main data structures used are an adjacency list for the tree and two arrays (or vectors) to store the count and results. The algorithm leverages recursion to do DFS traversals.

Tree / Graph DP

□ #968 Binary Tree Cameras

HAR

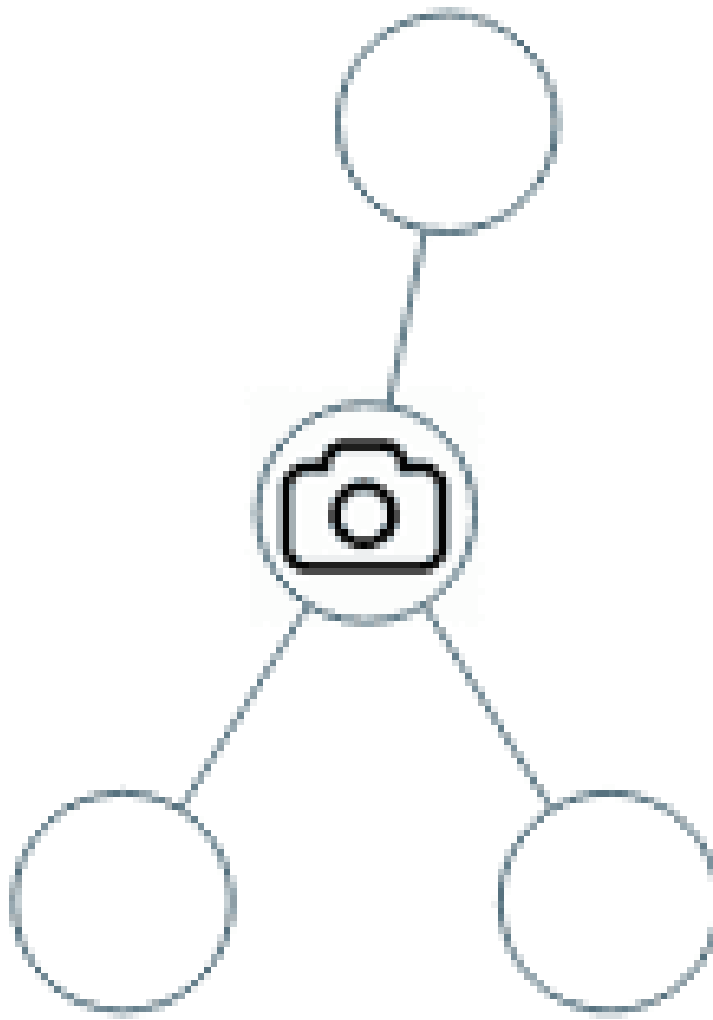
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Tree · Depth-First
Search · Binary Tree
Lists: AlgoMaster 600

Problem

You are given the root of a binary tree. We install cameras on the tree nodes where each camera at a node can monitor its parent, itself, and its immediate children.

Return the minimum number of cameras needed to monitor all nodes of the tree.

Example 1:

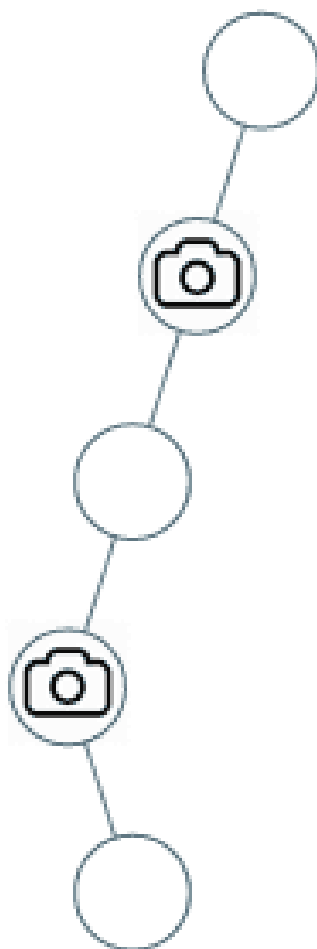


Input: root = [0,0,null,0,0]

Output: 1

Explanation: One camera is enough to monitor all nodes if placed as shown.

Example 2:



Input: root = [0,0,null,0,null,0,null,null,0]

Output: 2

Explanation: At least two cameras are needed to monitor all nodes of the tree.
The above image shows one of the valid configurations of camera placement.

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- Node.val == 0

Community Solutions (Python)

- LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def minCameraCover(self, root: Optional[TreeNode]) -> int:
        def dfs(root):
            if root is None:
                return inf, 0, 0
            la, lb, lc = dfs(root.left)
            ra, rb, rc = dfs(root.right)
            a = min(la, lb, lc) + min(ra, rb, rc) + 1
            b = min(la + rb, lb + ra, la + ra)
            c = lb + rb
            return a, b, c

        a, b, _ = dfs(root)
        return min(a, b)
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def minCameraCover(self, root: TreeNode) -> int:
        # Define states:
```

```
# 0: Node has a camera.
# 1: Node is covered, but has no camera.
# 2: Node is not covered.

self.camera_count = 0

def dfs(node):
    if not node:
        # Null nodes are considered covered.
        return 1

    left = dfs(node.left)
    right = dfs(node.right)

    # If any child is not covered, current node must have a camera.
    if left == 2 or right == 2:
        self.camera_count += 1
        return 0 # Place camera at current node.

    # If any child has a camera, current node is covered.

    if left == 0 or right == 0:
        return 1 # Covered but no camera.

    # Else, node is not covered.
    return 2

if dfs(root) == 2:
    self.camera_count += 1
return self.camera_count
```

◆ Quick Review

Key Insights

- Use a DFS (depth-first search) to traverse the tree in postorder.

- For each node, classify it with one of three states:
- State 0: Node has a camera.
- State 1: Node is covered by a camera (child or parent) but has no camera.
- State 2: Node is not covered and needs a camera.
- Place a camera at the current node if any child is in state 2.
- If at least one child has a camera, mark the current node as covered.
- For leaf nodes (or null nodes), consider them as covered for the sake of simplifying the DFS recurrence.

Space & Time

Time Complexity: $O(N)$ where N is the number of nodes in the tree since each node is visited once.

Space Complexity: $O(H)$ due to the recursion stack, where H is the height of the tree.

Solution

We solve the problem using a DFS based on postorder traversal. At every node, inspect the

state of its children. Based on that, decide if the current node should have a camera, be considered covered, or require a camera from its parent. The main challenge is to correctly propagate the state information upward to minimize the number of cameras while ensuring every node is covered. This strategy uses a greedy approach on the tree structure with three states.

Bitmask DP

Round 9 · Low · Hard · 1 question

Bitmask DP

□ #847 Shortest Path Visiting All Nodes

HAR

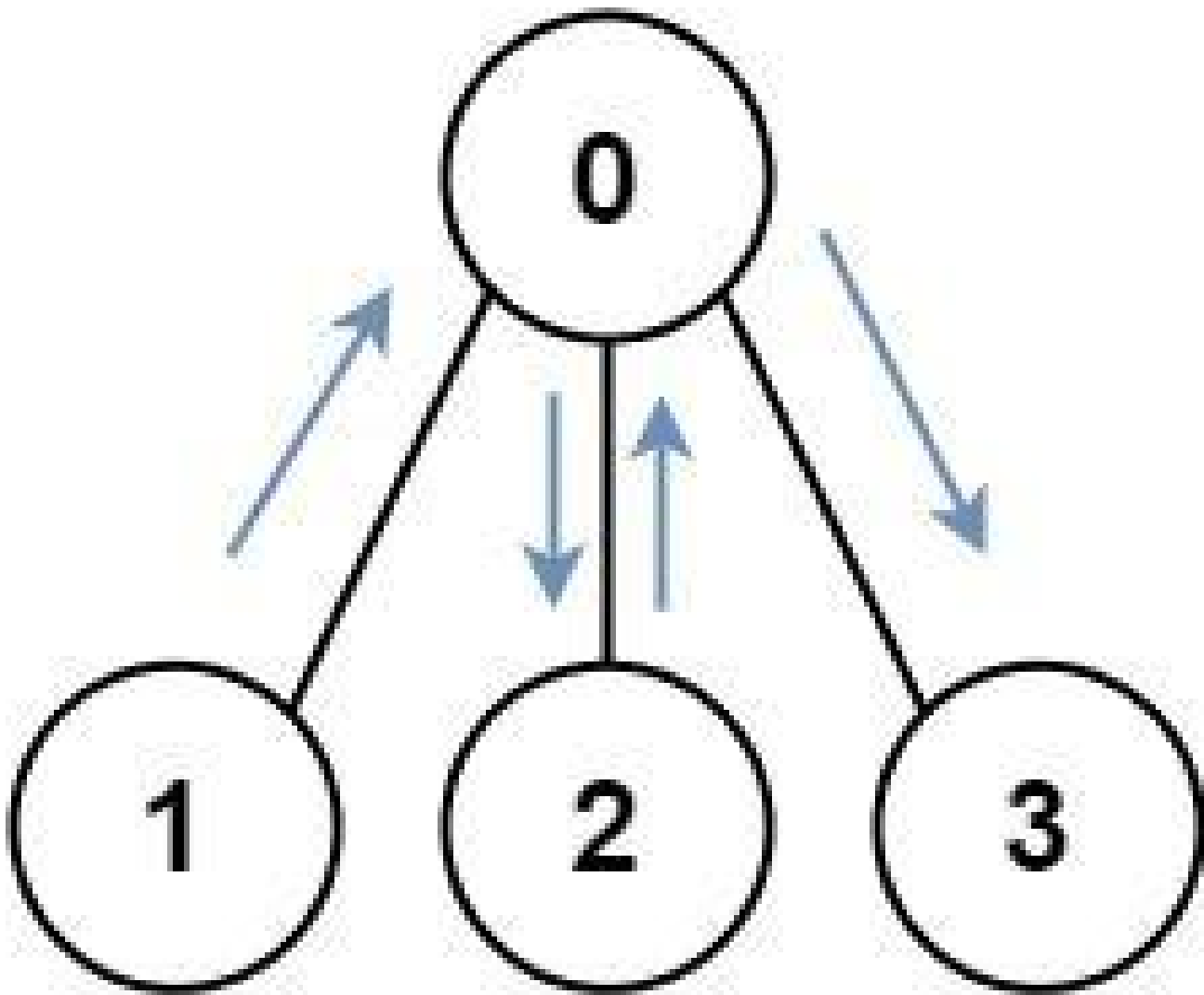
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Bit Manipulation ·
Breadth-First Search · Graph Theory · Bitmask
Lists: AlgoMaster 600

Problem

You have an undirected, connected graph of n nodes labeled from 0 to $n - 1$. You are given an array `graph` where `graph[i]` is a list of all the nodes connected with node i by an edge.

Return the length of the shortest path that visits every node. You may start and stop at any node, you may revisit nodes multiple times, and you may reuse edges.

Example 1:

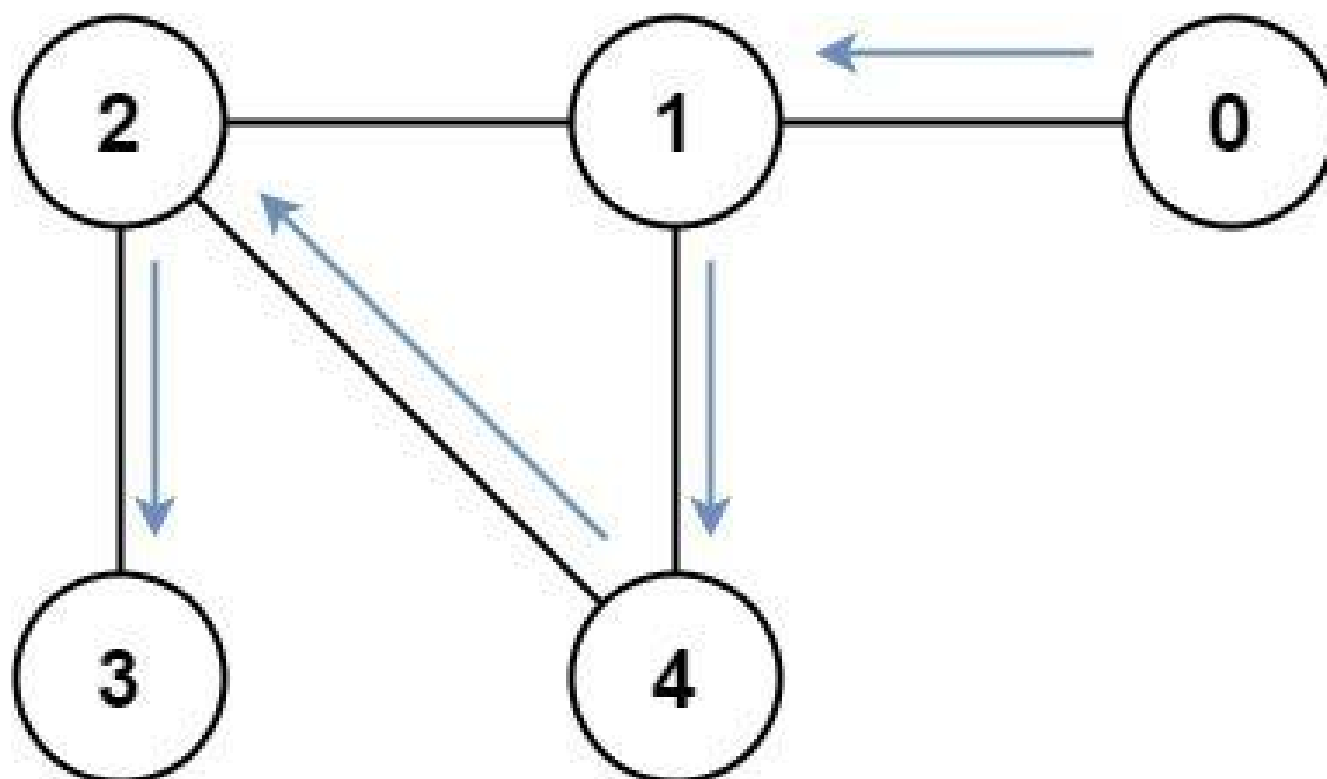


Input: graph = [[1,2,3],[0],[0],[0]]

Output: 4

Explanation: One possible path is [1,0,2,0,3]

Example 2:



Input: graph = [[1],[0,2,4],[1,3,4],[2],[1,2]]

Output: 4

Explanation: One possible path is [0,1,4,2,3]

Constraints:

- $n == \text{graph.length}$
- $1 \leq n \leq 12$
- $0 \leq \text{graph}[i].\text{length} < n$
- $\text{graph}[i]$ does not contain i .
- If $\text{graph}[a]$ contains b , then $\text{graph}[b]$ contains a .
- The input graph is always connected.

Community Solutions (Python)

- WalkCC

```

class Solution:
    def shortestPathLength(self, graph: list[list[int]]) -> int:
        n = len(graph)
        goal = (1 << n) - 1
        q = collections.deque()  # (u, state)
        seen = set()

        for i in range(n):
            q.append((i, 1 << i))

        step = 0
        while q:
            for _ in range(len(q)):
                u, state = q.popleft()
                if state == goal:
                    return step
                if (u, state) in seen:
                    continue
                seen.add((u, state))
                for v in graph[u]:
                    q.append((v, state | 1 << v))
            step += 1

        return -1

```

• LeetDoocs

```

class Solution:
    def shortestPathLength(self, graph: List[List[int]]) -> int:
        n = len(graph)
        q = deque()
        vis = set()
        for i in range(n):
            q.append((i, 1 << i))
            vis.add((i, 1 << i))
        ans = 0
        while 1:

```

```

    for _ in range(len(q)):
        i, st = q.popleft()
        if st == (1 << n) - 1:
            return ans
        for j in graph[i]:
            nst = st | 1 << j
            if (j, nst) not in vis:
                vis.add((j, nst))
                q.append((j, nst))

    ans += 1

```

• SimplyLeet

```

# Import required collections for the queue
from collections import deque

def shortestPathLength(graph):
    n = len(graph)
    # If there is only one node, we're already done
    if n == 1:
        return 0

    # All nodes visited bitmask (e.g., for n=3, all_visited = 0b111)

    all_visited = (1 << n) - 1

    # Creating a queue for BFS where each element is (current_node,
    visited_mask)
    queue = deque()
    # Use a set to record visited (node, mask) states
    seen = set()

    # Multi-source initialization: start from every node.
    for i in range(n):
        # visited_mask with only the i-th bit set

```

```

mask = 1 << i
queue.append((i, mask, 0)) # (node, visited_mask, distance)
seen.add((i, mask))

# BFS loop
while queue:
    node, mask, dist = queue.popleft()
    # Check all adjacent nodes
    for neighbor in graph[node]:
        # Update the visited mask with the neighbor visited

        next_mask = mask | (1 << neighbor)
        # Return result if all nodes have been visited
        if next_mask == all_visited:
            return dist + 1
        # If this state is not seen, add it to the queue
        if (neighbor, next_mask) not in seen:
            seen.add((neighbor, next_mask))
            queue.append((neighbor, next_mask, dist + 1))

# Fallback (should not be reached since graph is connected)

return -1

# Example usage:
# graph = [[1,2,3],[0],[0],[0]]
# print(shortestPathLength(graph))

```

◆ Quick Review

Key Insights

- The problem is a variant of the Traveling Salesman Problem, but the small constraint ($n \leq 12$) permits a solution using bitmasking.
- Each state can be represented as a combination of the current node and a bitmask

indicating which nodes have been visited.

- Breadth-first search (BFS) helps guarantee that the first time we encounter a state with all nodes visited, it is via the shortest path.
- Using multi-source BFS from each node as a starting point simplifies the logic because we can start from any node.

Space & Time

Time Complexity: $O(n * 2^n)$ where n is the number of nodes since we potentially visit each state (node, visited-mask) once.

Space Complexity: $O(n * 2^n)$ for storing the states in the visited set and the BFS queue.

Solution

The solution uses a BFS that expands states of the form (current node, visited mask). Initially, every node is added to the queue with its corresponding bitmask. At each BFS step, we iterate over all adjacent nodes and update the visited mask by setting the bit corresponding to the neighbor. When the visited mask equals $(1 \ll n) - 1$ (all nodes visited), we return the number of steps taken. This BFS expansion ensures that we always find the

shortest path.

Digit DP

Round 9 · Low · Hard · 2 questions

Digit DP

□ #233 Number of Digit One

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Recursion
Lists: AlgoMaster 600

Problem

Given an integer n , count the total number of digit 1 appearing in all non-negative integers less than or equal to n .

Example 1:

Input: $n = 13$
Output: 6

Example 2:

Input: $n = 0$
Output: 0

Constraints:

- $0 \leq n \leq 10^9$

Community Solutions (Python)

- WalkCC


```

class Solution:
    def countDigitOne(self, n: int) -> int:
        ans = 0

        pow10 = 1
        while pow10 <= n:
            divisor = pow10 * 10
            quotient = n // divisor
            remainder = n % divisor
            if quotient > 0:

                ans += quotient * pow10
            if remainder >= pow10:
                ans += min(remainder - pow10 + 1, pow10)
            pow10 *= 10

        return ans

```

• LeetCode

```

class Solution:
    def countDigitOne(self, n: int) -> int:
        @cache
        def dfs(i: int, cnt: int, limit: bool) -> int:
            if i >= len(s):
                return cnt
            up = int(s[i]) if limit else 9
            ans = 0
            for j in range(up + 1):
                ans += dfs(i + 1, cnt + (j == 1), limit and j == up)

            return ans

        s = str(n)
        return dfs(0, 0, True)

```

• SimplyLeet

```
# Python solution for counting the number of digit one in range [0, n]

def countDigitOne(n: int) -> int:
    count = 0
    i = 1 # digit position (1, 10, 100, ...)
    while i <= n:
        # Divide n into parts based on the current digit's place value.
        divider = i * 10
        high = n // divider # higher part of n
        current = (n // i) % 10 # digit at current position

        low = n % i # lower part of n

        # Accumulate count from high digits contributions.
        if current == 0:
            count += high * i
        elif current == 1:
            count += high * i + low + 1
        else:
            count += (high + 1) * i

        # Move to the next position.
        i *= 10

    return count

# Example usage:
print(countDigitOne(13)) # Output: 6
```

◆ Quick Review

Key Insights

- Analyze each digit position in n (ones, tens, hundreds, etc.) independently.
- For each digit position, decompose n into three parts: high (digits left of the current

position), current (the digit at the current position), and low (digits right of the current position).

- Depending on the value of the current digit:
- If it is 0, its contribution is solely defined by the higher digits.
- If it is 1, count both the higher digits and the remainder (low) plus one.
- If it is greater than 1, the contribution is fully determined by the higher digits incremented by one.
- This mathematical method avoids inefficient iterations and leverages place value analysis.

Space & Time

Time Complexity: $O(\log n)$ because the number of digits in n determines the loop iterations.

Space Complexity: $O(1)$ as only a few integer variables are used.

Solution

We use a mathematical approach by iterating over each digit position. At each position, calculate:

- **high**: n divided by the current digit place multiplied by 10.
- **current**: the digit at the current position from n .
- **low**: the remainder when n is divided by the current digit place. Based on the value of the current digit, determine its contribution to the total count of 1's using specific formulae:
 - If $\text{current} == 0$: add $\text{high} * i$.
 - If $\text{current} == 1$: add $\text{high} * i + (\text{low} + 1)$.
 - Else: add $(\text{high} + 1) * i$. This algorithm efficiently counts the digit 1 occurrences without iterating through every number from 0 to n .

Digit DP

□ #902 Numbers At Most N Given Digit Set **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · String · Binary Search · Dynamic Programming

Lists: AlgoMaster 600

Problem

Given an array of digits which is sorted in non-decreasing order. You can write numbers using each `digits[i]` as many times as we want. For example, if `digits = ['1','3','5']`, we may write numbers such as `'13'`, `'551'`, and `'1351315'`.

Return the number of positive integers that can be generated that are less than or equal to a given integer `n`.

Example 1:

Input: `digits = ["1","3","5","7"], n = 100`

Output: 20

Explanation:

The 20 numbers that can be written are:

1, 3, 5, 7, 11, 13, 15, 17, 31, 33, 35, 37, 51, 53, 55, 57, 71, 73, 75, 77.

Example 2:

Input: digits = ["1","4","9"], n = 1000000000

Output: 29523

Explanation:

We can write 3 one digit numbers, 9 two digit numbers, 27 three digit numbers, 81 four digit numbers, 243 five digit numbers, 729 six digit numbers, 2187 seven digit numbers, 6561 eight digit numbers, and 19683 nine digit numbers.

In total, this is 29523 integers that can be written using the digits array.

Example 3:

Input: digits = ["7"], n = 8

Output: 1

Constraints:

- $1 \leq \text{digits.length} \leq 9$
- $\text{digits}[i].\text{length} == 1$
- $\text{digits}[i]$ is a digit from '1' to '9'.
- All the values in digits are unique.
- digits is sorted in non-decreasing order.
- $1 \leq n \leq 10^9$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def atMostNGivenDigitSet(self, digits: list[str], n: int) -> int:
        ans = 0
        num = str(n)

        for i in range(1, len(num)):
            ans += pow(len(digits), i)

        for i, c in enumerate(num):
            dHasSameNum = False
```

```

for digit in digits:
    if digit[0] < c:
        ans += pow(len(digits), len(num) - i - 1)
    elif digit[0] == c:
        dHasSameNum = True
    if not dHasSameNum:
        return ans

return ans + 1

```

• LeetDoocs

```

class Solution:
    def atMostNGivenDigitSet(self, digits: List[str], n: int) -> int:
        @cache
        def dfs(i: int, lead: int, limit: bool) -> int:
            if i >= len(s):
                return lead ^ 1

            up = int(s[i]) if limit else 9
            ans = 0
            for j in range(up + 1):
                if j == 0 and lead:
                    ans += dfs(i + 1, 1, limit and j == up)
                elif j in nums:
                    ans += dfs(i + 1, 0, limit and j == up)
            return ans

        s = str(n)
        nums = {int(x) for x in digits}
        return dfs(0, 1, True)

```

• SimplyLeet

```

# Python Solution
def atMostNGivenDigitSet(digits, n):
    # Convert n to string for easy digit comparison
    s = str(n)
    n_len = len(s)
    k = len(digits)
    count = 0

    # Count numbers with length less than n
    for i in range(1, n_len):

        count += k ** i

    # Process numbers with length equal to n
    for i, ch in enumerate(s):
        # Count valid digits less than current digit ch in n
        smaller_count = 0
        for d in digits:
            if d < ch:
                smaller_count += 1
            else:
                break # since digits is sorted, no further digit is smaller

        # Options for the current digit position multiplied by choices for
        # remaining positions
        count += smaller_count * (k ** (n_len - i - 1))

        # If the current digit ch is not in digits, no need to continue further
        if ch not in digits:
            return count # early termination

    # If the loop completes, n itself can be formed using digits

    return count + 1

# Example usage:
#print(atMostNGivenDigitSet(["1", "3", "5", "7"], 100))

```

◆ Quick Review

Key Insights

- Count numbers with a digit-length strictly less than the length of n ; every combination is valid.
- Handle numbers with the same digit-length as n digit by digit.
- For each digit in the same-length number, consider:
 - The count of available digits less than the corresponding digit in n multiplied by the number of completions with any digits thereafter.
 - If the digit equal to the current digit in n exists, continue to the next digit.
 - If a chosen digit does not equal the current digit in n and is not less than it, break the loop.
 - This approach is similar to digit dynamic programming but done greedily with careful counting.

Space & Time

Time Complexity: $O(m * k)$ where m is the number of digits in n and k is the number of given digits.

Space Complexity: $O(1)$ or $O(m)$ if considering the string representation conversion space.

Solution

The solution counts valid numbers in two parts:

- Count numbers with digit-length strictly smaller than that of n . For each d -digit number where $d \leq \text{len}(n)$, the count is $(\text{number of given digits})^d$.**
- Count numbers with the same digit-length as n . Process each digit from left to right; for the current digit, count digits in the given array that are less than the corresponding digit in n and multiply by the number of completions possible for the remaining positions. If a matching digit is found, continue; if not, break. Finally, if we completed all digits matching, include the number n itself.**

Key data structures include simple variables for counting and arithmetic operations; no elaborate structure is required. The algorithm leverages greedy decisions at each digit position.

State Machine DP

Round 9 · Low · Hard · 1 question

State Machine DP

□ #188 Best Time to Buy and Sell Stock IV

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: AlgoMaster 600

Problem

You are given an integer array `prices` where `prices[i]` is the price of a given stock on the *i*th day, and an integer *k*.

Find the maximum profit you can achieve. You may complete at most *k* transactions: i.e. you may buy at most *k* times and sell at most *k* times.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

Input: *k* = 2, `prices` = [2,4,1]

Output: 2

Explanation: Buy on day 1 (price = 2) and sell on day 2 (price = 4), profit = 4-2 = 2.

Example 2:

Input: $k = 2$, $prices = [3, 2, 6, 5, 0, 3]$

Output: 7

Explanation: Buy on day 2 (price = 2) and sell on day 3 (price = 6), profit = $6 - 2 = 4$. Then buy on day 5 (price = 0) and sell on day 6 (price = 3), profit = $3 - 0 = 3$.

Constraints:

- $1 \leq k \leq 100$
- $1 \leq prices.length \leq 1000$
- $0 \leq prices[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProfit(self, k: int, prices: list[int]) -> int:
        if k >= len(prices) // 2:
            sell = 0
            hold = -math.inf

            for price in prices:
                sell = max(sell, hold + price)
                hold = max(hold, sell - price)

            return sell

        sell = [0] * (k + 1)
        hold = [-math.inf] * (k + 1)

        for price in prices:
            for i in range(k, 0, -1):
                sell[i] = max(sell[i], hold[i] + price)
                hold[i] = max(hold[i], sell[i - 1] - price)

        return sell[k]
```

• LeetCode

```
class Solution:
    def maxProfit(self, k: int, prices: List[int]) -> int:
        @cache
        def dfs(i: int, j: int, k: int) -> int:
            if i >= len(prices):
                return 0
            ans = dfs(i + 1, j, k)
            if k:
                ans = max(ans, prices[i] + dfs(i + 1, j, 0))
            elif j:
                ans = max(ans, -prices[i] + dfs(i + 1, j - 1, 1))
            return ans

        return dfs(0, k, 0)
```

```
class Solution:
    def maxProfit(self, k: int, prices: List[int]) -> int:
        n = len(prices)
        f = [[[0] * 2 for _ in range(k + 1)] for _ in range(n)]
        for j in range(1, k + 1):
            f[0][j][1] = -prices[0]
        for i, x in enumerate(prices[1:], 1):
            for j in range(1, k + 1):
                f[i][j][0] = max(f[i - 1][j][1] + x, f[i - 1][j][0])
                f[i][j][1] = max(f[i - 1][j - 1][0] - x, f[i - 1][j][1])

        return f[n - 1][k][0]
```

```
class Solution:
    def maxProfit(self, k: int, prices: List[int]) -> int:
        f = [[0] * 2 for _ in range(k + 1)]
        for j in range(1, k + 1):
            f[j][1] = -prices[0]
        for x in prices[1:]:
            for j in range(k, 0, -1):
                f[j][0] = max(f[j][1] + x, f[j][0])
                f[j][1] = max(f[j - 1][0] - x, f[j][1])
        return f[k][0]
```

• SimplyLeet

```

# Python solution for Best Time to Buy and Sell Stock IV

def maxProfit(k, prices):
    # Edge case: if there are no prices, no profit can be made.
    n = len(prices)
    if n == 0 or k == 0:
        return 0

    # If k is large enough, perform as many transactions as possible.
    if k >= n // 2:

        total_profit = 0
        for i in range(1, n):
            # Add profit if today's price is higher than yesterday's.
            if prices[i] > prices[i - 1]:
                total_profit += prices[i] - prices[i - 1]
        return total_profit

    # Initialize DP table: dp[t][i] is the max profit for at most t
    transactions until day i.
    dp = [[0] * n for _ in range(k + 1)]

    # Fill the DP table for each transaction from 1 to k.
    for t in range(1, k + 1):
        maxDiff = -prices[0] # Initialize maxDiff as maximum value of
        dp[t-1][0] - prices[0].
        for i in range(1, n):
            # Either do not trade on day i or sell on day i using best buy
            option.
            dp[t][i] = max(dp[t][i - 1], prices[i] + maxDiff)
            # Update maxDiff to incorporate the profit from previous
            transaction.
            maxDiff = max(maxDiff, dp[t - 1][i] - prices[i])
        return dp[k][n - 1]

# Example usage:
print(maxProfit(2, [3,2,6,5,0,3])) # Expected output: 7

```

◆ Quick Review

Key Insights

- For a high value of k ($k \geq n/2$ where n is the number of days), the problem reduces to the unlimited transactions case.
- When k is small, a dynamic programming approach is necessary to track the optimal profit at each transaction and day.
- Use a DP recurrence: $dp[t][i] = \max(dp[t][i-1], \text{prices}[i] + \text{maxDiff})$, where maxDiff is updated as $\max(\text{maxDiff}, dp[t-1][i] - \text{prices}[i])$.
- The idea is to iteratively build the solution for each transaction, using previously computed results efficiently.

Space & Time

Time Complexity: $O(k * n)$ where n is the number of days.

Space Complexity: $O(k * n)$ for the DP table (can be optimized to $O(n)$ per transaction if needed).

Solution

We use a dynamic programming solution. First, check if k is large enough to allow unlimited transactions. If yes, simply sum up all positive

differences between consecutive days. Otherwise, create a 2D DP table where $dp[t][i]$ represents the maximum profit using at most t transactions until day i . For each transaction from 1 to k and for each day we maintain a variable $maxDiff$ representing the maximum value of $(dp[t-1][j] - prices[j])$ for j from 0 to the current day. This helps to update the dp state for a transaction with a sell on the current day using the optimal previous transaction state.

Maths / Geometry

Round 9 · Low · Hard · 1 question

Maths / Geometry

□ #149 Max Points on a Line

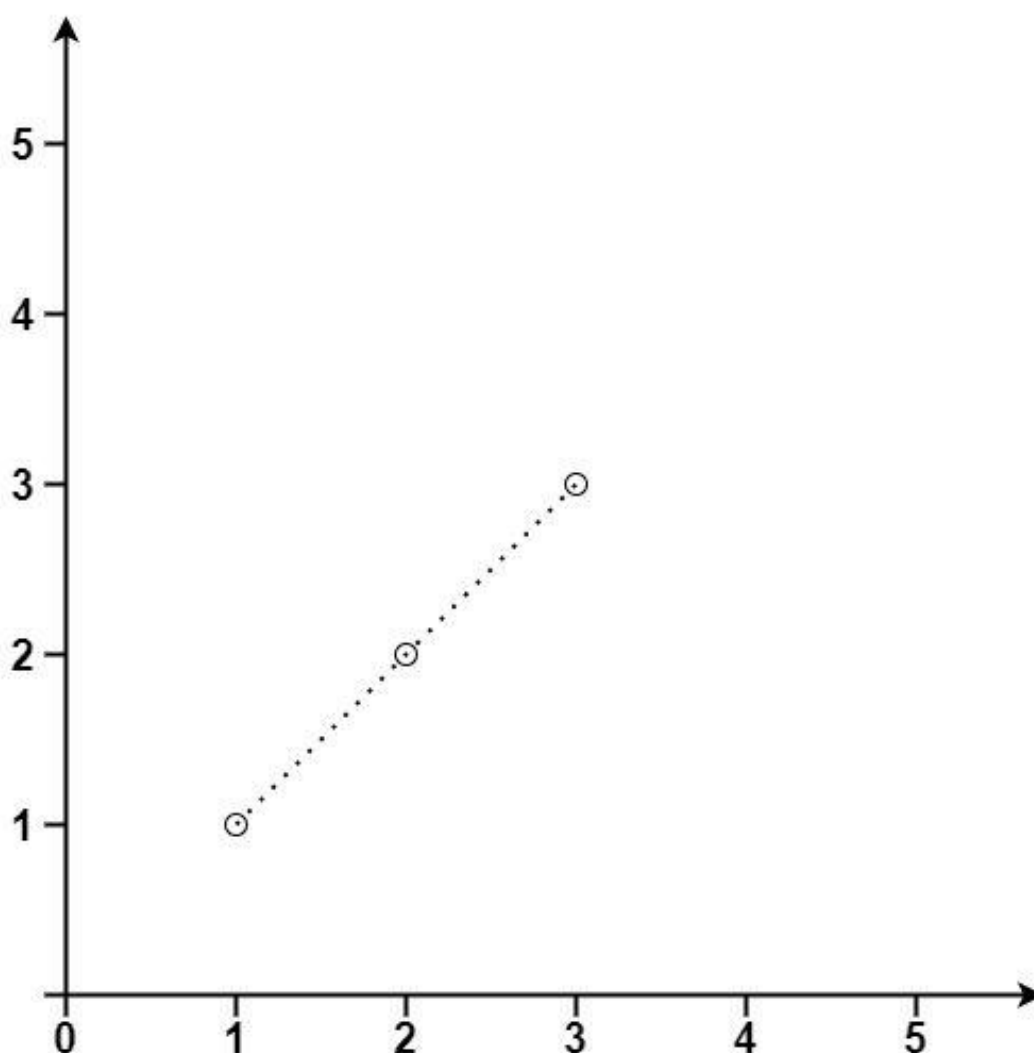
HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Geometry
Lists: AlgoMaster 600

Problem

Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane, return the maximum number of points that lie on the same straight line.

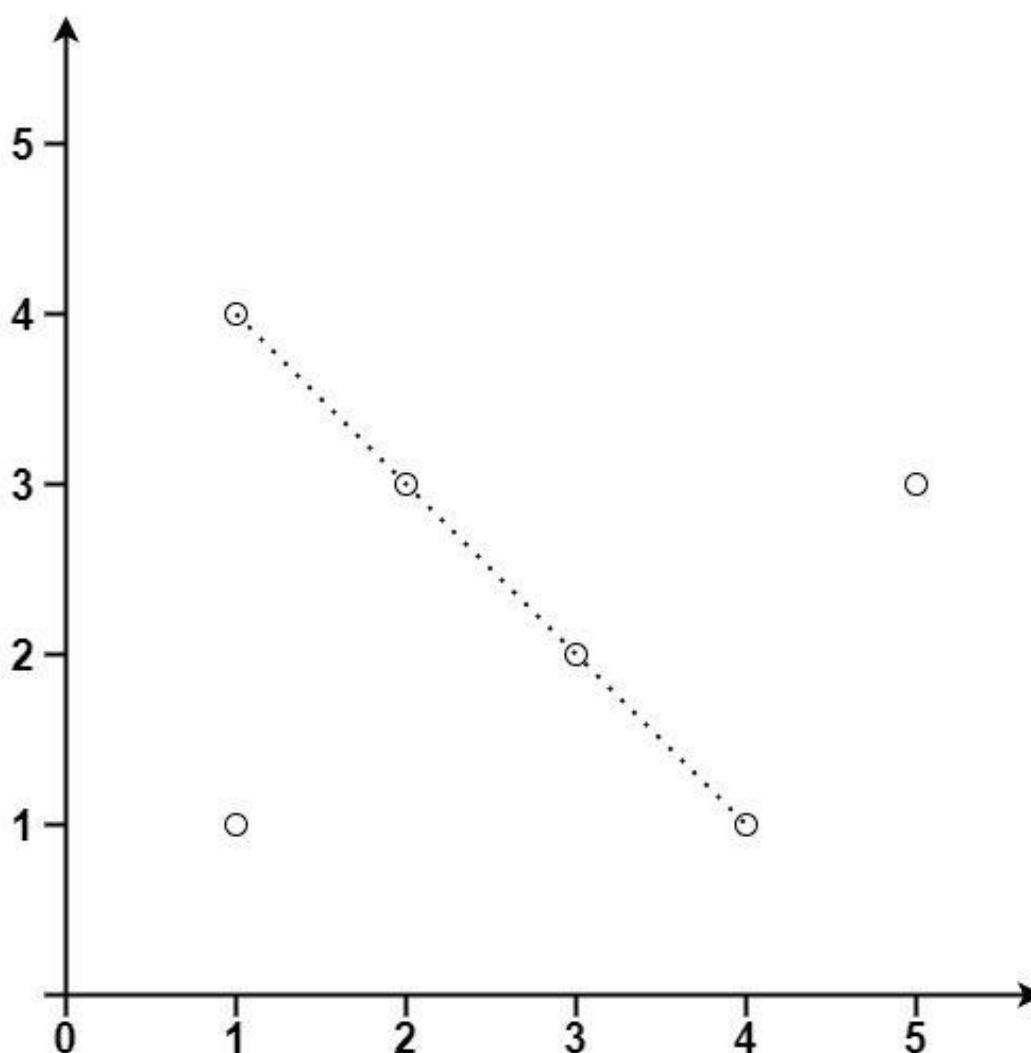
Example 1:



Input: points = [[1,1],[2,2],[3,3]]

Output: 3

Example 2:



Input: points = [[1,1],[3,2],[5,3],[4,1],[2,3],[1,4]]

Output: 4

Constraints:

- $1 \leq \text{points.length} \leq 300$
- $\text{points}[i].\text{length} == 2$
- $-104 \leq x_i, y_i \leq 104$
- All the points are unique.

Community Solutions (Python)

- WalkCC

```

class Solution:
    def maxPoints(self, points: list[list[int]]) -> int:
        ans = 0

        def gcd(a: int, b: int) -> int:
            return a if b == 0 else gcd(b, a % b)

        def getSlope(p: list[int], q: list[int]) -> tuple[int, int]:
            dx = p[0] - q[0]
            dy = p[1] - q[1]

            if dx == 0:
                return (0, p[0])
            if dy == 0:
                return (p[1], 0)
            d = gcd(dx, dy)
            return (dx // d, dy // d)

        for i, p in enumerate(points):
            slopeCount = collections.defaultdict(int)
            samePoints = 1

            maxPoints = 0 # the maximum number of points with the same slope
            for j in range(i + 1, len(points)):
                q = points[j]
                if p == q:
                    samePoints += 1
                else:
                    slope = getSlope(p, q)
                    slopeCount[slope] += 1
                    maxPoints = max(maxPoints, slopeCount[slope])
            ans = max(ans, samePoints + maxPoints)

        return ans

```

• LeetDoocs

```

class Solution:
    def maxPoints(self, points: List[List[int]]) -> int:
        n = len(points)
        ans = 1
        for i in range(n):
            x1, y1 = points[i]
            for j in range(i + 1, n):
                x2, y2 = points[j]
                cnt = 2
                for k in range(j + 1, n):
                    x3, y3 = points[k]
                    a = (y2 - y1) * (x3 - x1)
                    b = (y3 - y1) * (x2 - x1)
                    cnt += a == b
                ans = max(ans, cnt)
        return ans

```

```

class Solution:
    def maxPoints(self, points: List[List[int]]) -> int:
        def gcd(a, b):
            return a if b == 0 else gcd(b, a % b)

        n = len(points)
        ans = 1
        for i in range(n):
            x1, y1 = points[i]
            cnt = Counter()

            for j in range(i + 1, n):
                x2, y2 = points[j]
                dx, dy = x2 - x1, y2 - y1
                g = gcd(dx, dy)
                k = (dx // g, dy // g)
                cnt[k] += 1
            ans = max(ans, cnt[k] + 1)
        return ans

```

• SimplyLeet

```
import math
from collections import defaultdict

class Solution:
    def maxPoints(self, points):
        # If there are less than 3 points, return the count directly
        if len(points) < 3:
            return len(points)

        max_points = 0

        # Iterate over each point treating it as an anchor
        for i in range(len(points)):
            slopes = defaultdict(int)
            same = 1 # Count the anchor point itself
            curr_max = 0

            # Compare with every other point after i
            for j in range(i + 1, len(points)):
                dx = points[j][0] - points[i][0]

                dy = points[j][1] - points[i][1]

                # Deal with the (theoretical) duplicate point case.
                if dx == 0 and dy == 0:
                    same += 1
                else:
                    # Handle vertical lines
                    if dx == 0:
                        slope = ('inf', 0)
                    else:
                        gcd = math.gcd(dx, dy)
                        dx_norm = dx // gcd
                        dy_norm = dy // gcd
                        # Normalize representation: ensure dx_norm is positive
                        if dx_norm < 0:
                            dx_norm *= -1
                            dy_norm *= -1
                        slope = (dy_norm, dx_norm)

                    slopes[slope] += 1
```



```
curr_max = max(curr_max, slopes[slope])
max_points = max(max_points, curr_max + same)
return max_points
```

◆ Quick Review

Key Insights

- Use a nested loop to treat each point as an anchor and calculate slopes to all other points.
- Represent slopes as a reduced fraction (dy/dx) using the greatest common divisor (gcd) to avoid floating-point imprecision.
- Specially handle vertical lines (when dx is 0) by using a unique identifier.
- The problem naturally leads to an $O(n^2)$ solution by processing each pair of points.
- Use a hash table (or dictionary) to count how many points form the same slope with the anchor point.

Space & Time

Time Complexity: $O(n^2)$ where n is the number of points, since every pair of points is compared.

Space Complexity: $O(n)$ for the hash table used for storing slope counts for each anchor point.

Solution

The solution iterates through each point and treats it as an anchor. For every other point, we calculate the slope between the anchor and that point. The slope is computed as a pair (dy, dx) reduced to its simplest form via gcd. For vertical lines ($dx == 0$), we use a special key. A hash table is used to count occurrences of each slope. The maximum number of points with the same slope (plus the anchor) is recorded. The final result is the maximum count from all anchors.

String Matching

Round 9 · Low · Hard · 3 questions

String Matching

□ #214 Shortest Palindrome

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Rolling Hash · String Matching · Hash
Function

Lists: AlgoMaster 600

Problem

You are given a string s . You can convert s to a palindrome by adding characters in front of it. Return the shortest palindrome you can find by performing this transformation.

Example 1:

Input: $s = \text{"aacecaaa"}$
Output: "aaacecaaa"

Example 2:

Input: $s = \text{"abcd"}$
Output: "dcbabcd"

Constraints:

- $0 \leq s.length \leq 5 * 10^4$
- s consists of lowercase English letters only.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def shortestPalindrome(self, s: str) -> str:
        t = s[::-1]

        for i in range(len(t)):
            if s.startswith(t[i:]):
                return t[:i] + s

        return t + s
```

• LeetCode

```
class Solution:
    def shortestPalindrome(self, s: str) -> str:
        base = 131
        mod = 10**9 + 7
        n = len(s)
        prefix = suffix = 0
        mul = 1
        idx = 0
        for i, c in enumerate(s):
            prefix = (prefix * base + (ord(c) - ord('a') + 1)) % mod

            suffix = (suffix + (ord(c) - ord('a') + 1) * mul) % mod
            mul = (mul * base) % mod
            if prefix == suffix:
                idx = i + 1
        return s if idx == n else s[idx:][::-1] + s
```

```
typedef unsigned long long ull;
```

```
class Solution {
public:
    string shortestPalindrome(string s) {
        int base = 131;
        ull mul = 1;
        ull prefix = 0;
        ull suffix = 0;
        int idx = 0, n = s.size();
```

```

    for (int i = 0; i < n; ++i) {
        int t = s[i] - 'a' + 1;
        prefix = prefix * base + t;
        suffix = suffix + mul * t;
        mul *= base;
        if (prefix == suffix) idx = i + 1;
    }
    if (idx == n) return s;
    string x = s.substr(idx, n - idx);
    reverse(x.begin(), x.end());

    return x + s;
}
};

```

```

class Solution:
    def shortestPalindrome(self, s: str) -> str:
        t = s + "#" + s[::-1] + "$"
        n = len(t)
        next = [0] * n
        next[0] = -1
        i, j = 2, 0
        while i < n:
            if t[i - 1] == t[j]:
                j += 1
            next[i] = j
            i += 1
        elif j:
            j = next[j]
        else:
            next[i] = 0
            i += 1
        return s[::-1][:-next[-1]] + s

```

• SimplyLeet

```

# Python solution using KMP prefix function
def shortestPalindrome(s: str) -> str:
    if not s:
        return s
    # Generate combined string with a separator
    rev_s = s[::-1]
    combined = s + "#" + rev_s
    n = len(combined)

    # Build KMP table (lps) for combined string

    lps = [0] * n
    for i in range(1, n):
        j = lps[i - 1]
        while j > 0 and combined[i] != combined[j]:
            j = lps[j - 1]
        if combined[i] == combined[j]:
            j += 1
        lps[i] = j

    # lps[-1] gives length of longest prefix of s which is palindrome

    longest_pal_pref = lps[-1]
    add_on = rev_s[:len(s) - longest_pal_pref]
    return add_on + s

# Example usage:
print(shortestPalindrome("aacecaaa")) # Output: "aaacecaaa"
print(shortestPalindrome("abcd"))    # Output: "dcbabcd"

```

◆ Quick Review

Key Insights

- The main idea is to identify the longest prefix of s that is already a palindrome.
- Once this prefix is identified, the remaining suffix can be reversed and prepended to s to

form a complete palindrome.

- A robust way to find the longest palindromic prefix is to use the KMP (Knuth–Morris–Pratt) algorithm on a constructed string made by concatenating s , a separator (such as '#'), and the reverse of s .
- The prefix table computed from KMP helps determine the maximum matching prefix which is also a palindrome.

Space & Time

Time Complexity: $O(n)$, where n is the length of s , because building and processing the combined string takes linear time.

Space Complexity: $O(n)$, needed for the combined string and the prefix (LPS) table.

Solution

The solution builds a new string $\text{combined} = s + \text{'\#'} + \text{reverse}(s)$. The KMP prefix function is then computed for this string. The last value of the prefix table indicates the length of the longest prefix of s which is also a palindrome. The characters not included in this palindrome (the suffix) are then reversed and prepended to s to form the shortest palindrome. This

approach leverages string matching techniques to efficiently determine the required palindrome structure.

String Matching

□ #1044 Longest Duplicate Substring

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Binary Search · Sliding Window ·
Rolling Hash · Suffix Array · Hash Function
Lists: AlgoMaster 600

Problem

Given a string s , consider all duplicated substrings: (contiguous) substrings of s that occur 2 or more times. The occurrences may overlap.

Return any duplicated substring that has the longest possible length. If s does not have a duplicated substring, the answer is "".

Example 1:

Input: $s = \text{"banana"}$
Output: "ana"

Example 2:

Input: $s = \text{"abcd"}$
Output: ""

Constraints:

- $2 \leq s.length \leq 3 * 10^4$

- s consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestDupSubstring(self, s: str) -> str:
        BASE = 26
        HASH = 1_000_000_007
        bestStart = -1
        l = 1
        r = len(s)

        def val(c: str) -> int:
            return ord(c) - ord('a')

        # k := the length of the substring to be hashed
        def getStart(k: int) -> int | None:
            maxPow = pow(BASE, k - 1, HASH)
            hashToStart = collections.defaultdict(list)
            h = 0

            # Compute the hash value of s[:k].
            for i in range(k):
                h = (h * BASE + val(s[i])) % HASH

            hashToStart[h].append(0)

            # Compute the rolling hash by Rabin Karp.
            for i in range(k, len(s)):
                startIndex = i - k + 1
                h = (h - maxPow * val(s[startIndex])) % HASH
                h = (h * BASE + val(s[i])) % HASH
                if h in hashToStart:
                    currSub = s[startIndex:startIndex + k]
                    for start in hashToStart[h]:
```

```

        if s[start:start + k] == currSub:
            return startIndex
        hashToStart[h].append(startIndex)

while l < r:
    m = (l + r) // 2
    start: int | None = getStart(m)
    if start:
        bestStart = start
        l = m + 1

    else:
        r = m

if bestStart == -1:
    return ''
if getStart(l):
    return s[bestStart:bestStart + l]
return s[bestStart:bestStart + l - 1]

```

• LeetDoocs

```

class Solution:
    def longestDupSubstring(self, s: str) -> str:
        def check(l):
            vis = set()
            for i in range(n - l + 1):
                t = s[i : i + l]
                if t in vis:
                    return t
                vis.add(t)
            return ''

        n = len(s)
        left, right = 0, n
        ans = ''
        while left < right:
            mid = (left + right + 1) >> 1
            t = check(mid)
            ans = t or ans
            if t:
                left = mid

```

```
        else:
            right = mid - 1
    return ans
```

```
typedef unsigned long long ULL;
```

```
class Solution {
public:
    ULL p[30010];
    ULL h[30010];
    string longestDupSubstring(string s) {
        int base = 131, n = s.size();
        p[0] = 1;
        for (int i = 0; i < n; ++i) {

            p[i + 1] = p[i] * base;
            h[i + 1] = h[i] * base + s[i];
        }
        int left = 0, right = n;
        string ans = "";
        while (left < right) {
            int mid = (left + right + 1) >> 1;
            string t = check(s, mid);
            if (t.empty())
                right = mid - 1;

            else {
                left = mid;
                ans = t;
            }
        }
        return ans;
    }

    string check(string& s, int len) {
        int n = s.size();
```

```

unordered_set<ULL> vis;
for (int i = 1; i + len - 1 <= n; ++i) {
    int j = i + len - 1;
    ULL t = h[j] - h[i - 1] * p[j - i + 1];
    if (vis.count(t)) return s.substr(i - 1, len);
    vis.insert(t);
}
return "";
}
};

```

• SimplyLeet

```

# Define the Solution class with the function longestDupSubstring
class Solution:
    def longestDupSubstring(self, s: str) -> str:
        n = len(s)
        # Convert characters to integers: 'a' -> 0, 'b' -> 1, ..., 'z' -> 25
        nums = [ord(c) - ord('a') for c in s]
        mod = 2**63 - 1 # a large modulus to minimize hash collisions
        base = 26 # base value for characters

        # search function returns the starting index of a duplicate substring
        # of length L if it exists, else -1

    def search(L):
        h = 0
        # Compute the hash for the first substring of length L
        for i in range(L):
            h = (h * base + nums[i]) % mod
        seen = {h}
        # Precompute base^L % mod for use in rolling hash
        baseL = pow(base, L, mod)
        for i in range(1, n - L + 1):
            # Compute rolling hash in O(1) time

```

```

        h = (h * base - nums[i - 1] * baseL + nums[i + L - 1]) % mod
        if h in seen:
            return i # found duplicate, return starting index
        seen.add(h)
    return -1

left, right = 1, n
start = -1
# Binary search for the maximum length L that has a duplicate substring
while left <= right:

    mid = left + (right - left) // 2
    idx = search(mid)
    if idx != -1:
        left = mid + 1 # duplicate found, try longer substring
        start = idx
    else:
        right = mid - 1 # no duplicate found, decrease length
return s[start:start+left-1] if start != -1 else ""

```

◆ Quick Review

Key Insights

- Use binary search on the possible lengths of duplicated substrings.
- For each candidate length, use a rolling hash (polynomial rolling hash) to efficiently hash every substring of that length.
- Use a hash set to check whether a substring hash has been seen before.
- Adjust the binary search range based on whether a duplicate was found, ultimately retrieving the longest duplicate substring.

Space & Time

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

Solution

The solution leverages binary search to determine the maximum length L for which a duplicate substring exists. For each candidate L , we use a rolling hash technique to compute hash values for all substrings of length L in $O(n)$ time. A hash set tracks previously seen substrings by their hash values; if a duplicate exists, we update our answer and attempt a larger L . This approach is efficient for the problem's constraints and handles overlapping duplicates. Key details include careful management of the modulo arithmetic in the rolling hash to avoid collisions and overflow, and possibly using large mod values or double hashing in competitive environments.

String Matching

□ #1392 Longest Happy Prefix

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Rolling Hash · String Matching · Hash
Function

Lists: AlgoMaster 600

Problem

A string is called a happy prefix if is a non-empty prefix which is also a suffix (excluding itself).

Given a string s , return the longest happy prefix of s . Return an empty string "" if no such prefix exists.

Example 1:

Input: $s = \text{"level"}$

Output: "l"

Explanation: s contains 4 prefix excluding itself ("l", "le", "lev", "leve"), and suffix ("l", "el", "vel", "evel"). The largest prefix which is also suffix is given by "l".

Example 2:

Input: $s = \text{"ababab"}$

Output: "abab"

Explanation: "abab" is the largest prefix which is also suffix. They can overlap in the original string.

Constraints:

- $1 \leq s.length \leq 105$
- s contains only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestPrefix(self, s: str) -> str:
        BASE = 26
        HASH = 8_417_508_174_513
        n = len(s)
        maxLength = 0
        pow = 1
        prefixHash = 0 # the hash of s[0..i]
        suffixHash = 0 # the hash of s[j..n)

        def val(c: str) -> int:
            return ord(c) - ord('a')

        j = n - 1
        for i in range(n - 1):
            prefixHash = (prefixHash * BASE + val(s[i])) % HASH
            suffixHash = (val(s[j]) * pow + suffixHash) % HASH
            pow = pow * BASE % HASH
            if prefixHash == suffixHash:
                maxLength = i + 1

            j -= 1

        return s[:maxLength]
```

• LeetDoocs

```
class Solution:
    def longestPrefix(self, s: str) -> str:
        for i in range(1, len(s)):
            if s[:-i] == s[i:]:
                return s[i:]
        return ''
```

```

typedef unsigned long long ULL;

class Solution {
public:
    string longestPrefix(string s) {
        int base = 131;
        int n = s.size();
        ULL p[n + 10];
        ULL h[n + 10];
        p[0] = 1;

        h[0] = 0;
        for (int i = 0; i < n; ++i) {
            p[i + 1] = p[i] * base;
            h[i + 1] = h[i] * base + s[i];
        }
        for (int l = n - 1; l > 0; --l) {
            ULL prefix = h[l];
            ULL suffix = h[n] - h[n - l] * p[l];
            if (prefix == suffix) return s.substr(0, l);
        }

        return "";
    }
};

```

```

class Solution:
    def longestPrefix(self, s: str) -> str:
        s += "#"
        n = len(s)
        next = [0] * n
        next[0] = -1
        i, j = 2, 0
        while i < n:
            if s[i - 1] == s[j]:
                j += 1

```

```

        next[i] = j
        i += 1
    elif j:
        j = next[j]
    else:
        next[i] = 0
        i += 1
    return s[: next[-1]]

```

• SimplyLeet

```

# Function to compute the longest happy prefix using KMP prefix function
def longestHappyPrefix(s: str) -> str:
    n = len(s)
    # pi[i] will hold the length of the longest prefix which is also suffix
    # ending at i
    pi = [0] * n
    # k: length of current longest prefix-suffix
    k = 0
    # iterate from second character to build the pi array
    for i in range(1, n):
        # if the current character doesn't match, fallback using pi array

        while k > 0 and s[i] != s[k]:
            k = pi[k - 1]
        # if there is a match, extend the prefix-suffix length
        if s[i] == s[k]:
            k += 1
            pi[i] = k
        else:
            pi[i] = 0
    # pi[n-1] gives the length of the longest happy prefix
    return s[:pi[-1]]

```

```

# Example usage:
print(longestHappyPrefix("level"))    # Output: "l"
print(longestHappyPrefix("ababab"))   # Output: "abab"

```

◆ Quick Review

Key Insights

- The problem can be solved efficiently by leveraging techniques used in string matching algorithms.
- A key method is to compute the prefix function (also known as pi array) from the KMP (Knuth–Morris–Pratt) algorithm.
- The value at the last index of the pi array corresponds to the length of the longest prefix that is also a suffix (excluding the whole string).
- An alternative approach is to use a rolling hash technique, but the KMP prefix function is more straightforward and has a linear time complexity.

Space & Time

Time Complexity: $O(n)$, where n is the length of s

Space Complexity: $O(n)$, for storing the prefix function

Solution

We use the KMP prefix function to solve this problem:

- Initialize an array pi of size n (length of s) with zeros.
- Iterate through the string from the second character, and update the pi array which keeps track of the longest proper prefix which is also a suffix for each substring ending at that index.
- The value at $pi[n-1]$ gives the length of the longest happy prefix.
- Return the substring from the beginning of s up to that length. The algorithm ensures that we only traverse the string once with extra space proportional to n .

Binary Indexed Tree / Segment Tree

Round 9 · Low · Hard · 4 questions

Binary Indexed Tree / Segment Tree

□ #699 Falling Squares

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Segment Tree · Ordered Set
Lists: AlgoMaster 600

Problem

There are several squares being dropped onto the X-axis of a 2D plane.

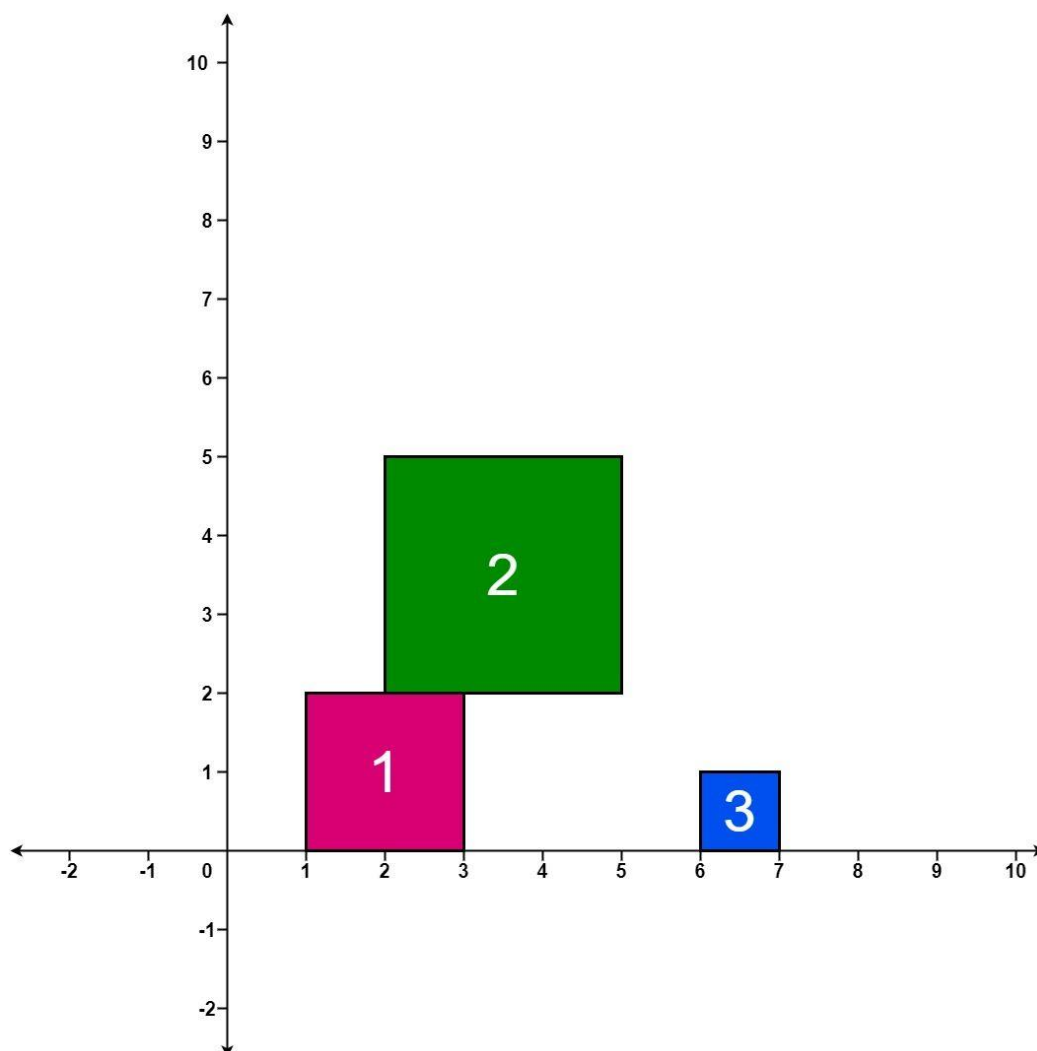
You are given a 2D integer array `positions` where `positions[i] = [lefti, sideLengthi]` represents the *i*th square with a side length of `sideLengthi` that is dropped with its left edge aligned with X-coordinate `lefti`.

Each square is dropped one at a time from a height above any landed squares. It then falls downward (negative Y direction) until it either lands on the top side of another square or on the X-axis. A square brushing the left/right side of another square does not count as landing on it. Once it lands, it freezes in place and cannot be moved.

After each square is dropped, you must record the height of the current tallest stack of squares.

Return an integer array `ans` where `ans[i]` represents the height described above after dropping the i th square.

Example 1:



Input: positions = [[1,2],[2,3],[6,1]]

Output: [2,5,5]

Explanation:

After the first drop, the tallest stack is square 1 with a height of 2.

After the second drop, the tallest stack is squares 1 and 2 with a height of 5.

After the third drop, the tallest stack is still squares 1 and 2 with a height of 5.

Thus, we return an answer of [2, 5, 5].

Example 2:

Input: positions = [[100,100],[200,100]]

Output: [100,100]

Explanation:

After the first drop, the tallest stack is square 1 with a height of 100.

After the second drop, the tallest stack is either square 1 or square 2, both with heights of 100.

Thus, we return an answer of [100, 100].

Note that square 2 only brushes the right side of square 1, which does not count as landing on it.

Constraints:

- $1 \leq \text{positions.length} \leq 1000$
- $1 \leq \text{lefti} \leq 108$
- $1 \leq \text{sideLengthi} \leq 106$

Community Solutions (Python)

• LeetDoocs

```
class Node:
    def __init__(self, l, r):
        self.left = None
        self.right = None
        self.l = l
        self.r = r
        self.mid = (l + r) >> 1
        self.v = 0
        self.add = 0
```

```
class SegmentTree:
    def __init__(self):
        self.root = Node(1, int(1e9))

    def modify(self, l, r, v, node=None):
        if l > r:
            return
        if node is None:
            node = self.root

        if node.l >= l and node.r <= r:
            node.v = v
            node.add = v
            return
        self.pushdown(node)
        if l <= node.mid:
            self.modify(l, r, v, node.left)
        if r > node.mid:
            self.modify(l, r, v, node.right)
        self.pushup(node)

    def query(self, l, r, node=None):
        if l > r:
            return 0
        if node is None:
            node = self.root
        if node.l >= l and node.r <= r:
            return node.v
        self.pushdown(node)
        v = 0

        if l <= node.mid:
            v = max(v, self.query(l, r, node.left))
        if r > node.mid:
            v = max(v, self.query(l, r, node.right))
        return v

    def pushup(self, node):
        node.v = max(node.left.v, node.right.v)

    def pushdown(self, node):
```

```

if node.left is None:
    node.left = Node(node.l, node.mid)
if node.right is None:
    node.right = Node(node.mid + 1, node.r)
if node.add:
    node.left.v = node.add
    node.right.v = node.add
    node.left.add = node.add
    node.right.add = node.add
    node.add = 0

```

```

class Solution:
    def fallingSquares(self, positions: List[List[int]]) -> List[int]:
        ans = []
        mx = 0
        tree = SegmentTree()
        for l, w in positions:
            r = l + w - 1
            h = tree.query(l, r) + w

            mx = max(mx, h)
            ans.append(mx)
            tree.modify(l, r, h)
        return ans

```

• SimplyLeet

```

# Python implementation for Falling Squares
def fallingSquares(positions):
    # List to record the results after each drop
    ans = []
    # List to keep each square's (left, right, height)
    squares = []
    # The current global maximum height so far
    current_max = 0

    # Iterate through each position in the input list

```

```

    for left, side in positions:
        new_left, new_right = left, left + side # calculate boundaries for
the new square
        base_height = 0 # start with height 0 for the landing base

        # Check all previous squares for overlapping intervals
        for prev_left, prev_right, prev_height in squares:
            # If the horizontal intervals overlap (not just touching)
            if not (new_right <= prev_left or new_left >= prev_right):
                # Update base_height to the maximum height this new square can
land on
                base_height = max(base_height, prev_height)

        # The height of the current square is the base height plus its side
length
        cur_square_height = base_height + side
        # Add the current square info to the squares list
        squares.append((new_left, new_right, cur_square_height))
        # Update the overall maximum height
        current_max = max(current_max, cur_square_height)
        # Append the current maximum height to the results list
        ans.append(current_max)

    # Return the results after dropping each square

    return ans

# Example usage:
#print(fallingSquares([[1,2],[2,3],[6,1]])) # Expected output: [2, 5, 5]

```

◆ Quick Review

Key Insights

- Each square's final height is determined by checking for overlap with all previously placed squares.

- Two squares overlap if the interval $[left, left + sideLength)$ of the current square and $[prevLeft, prevLeft + prevSideLength)$ of a previous square intersect.
- The height at which a new square lands is equal to its side length plus the maximum height of any square that overlaps its horizontal interval.
- While a brute-force $O(n^2)$ solution is feasible for up to 1000 positions, more efficient approaches can be implemented using segment trees or ordered sets when scaling to larger inputs.
- Coordinate compression might be used along with advanced data structures if deciding for performance optimizations.

Space & Time

Time Complexity: $O(n^2)$ in the brute-force approach since for each square we scan all previously dropped squares

Space Complexity: $O(n)$ for storing the square information and the answer array

Solution

The idea is to iterate over each square in the order provided. For the current square, determine its horizontal range using its left coordinate and side length. Check all previous squares to see if their horizontal range overlaps with the current square. If there is an overlap, the current square will land on top of that square, and its height becomes the height of that square plus its own side length. The final height for the current drop is the maximum height determined by all overlapping squares, or simply its side length if there is no overlap. Update the answer list with the maximum height seen so far.

This approach uses an array (or list) to store the details (left coordinate, right coordinate, and height) of each dropped square. The brute force check of overlapping intervals is intuitive and sufficient given the problem constraints.

Binary Indexed Tree / Segment Tree

□ #2426 Number of Pairs Satisfying Inequality

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Divide and Conquer ·
Binary Indexed Tree · Segment Tree · Merge
Sort · Ordered Set

Lists: AlgoMaster 600

Problem

You are given two 0-indexed integer arrays `nums1` and `nums2`, each of size `n`, and an integer `diff`. Find the number of pairs (i, j) such that:

- $0 \leq i < j \leq n - 1$ and
- $\text{nums1}[i] - \text{nums1}[j] \leq \text{nums2}[i] - \text{nums2}[j] + \text{diff}$.

Return the number of pairs that satisfy the conditions.

Example 1:

Input: nums1 = [3,2,5], nums2 = [2,2,1], diff = 1

Output: 3

Explanation:

There are 3 pairs that satisfy the conditions:

1. $i = 0, j = 1$: $3 - 2 \leq 2 - 2 + 1$. Since $i < j$ and $1 \leq 1$, this pair satisfies the conditions.

2. $i = 0, j = 2$: $3 - 5 \leq 2 - 1 + 1$. Since $i < j$ and $-2 \leq 2$, this pair satisfies the conditions.

3. $i = 1, j = 2$: $2 - 5 \leq 2 - 1 + 1$. Since $i < j$ and $-3 \leq 2$, this pair satisfies the conditions.

Therefore, we return 3.

Example 2:

Input: nums1 = [3,-1], nums2 = [-2,2], diff = -1

Output: 0

Explanation:

Since there does not exist any pair that satisfies the conditions, we return 0.

Constraints:

- $n == \text{nums1.length} == \text{nums2.length}$
- $2 \leq n \leq 105$
- $-104 \leq \text{nums1}[i], \text{nums2}[i] \leq 104$
- $-104 \leq \text{diff} \leq 104$

Community Solutions (Python)

• LeetDoocs

```
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):
```

```

        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s

```

```

class Solution:
    def numberOfPairs(self, nums1: List[int], nums2: List[int], diff: int) ->
int:
        tree = BinaryIndexedTree(10**5)
        ans = 0
        for a, b in zip(nums1, nums2):
            v = a - b
            ans += tree.query(v + diff + 40000)
            tree.update(v + 40000, 1)

        return ans

```

• SimplyLeet

Python solution using a Binary Indexed Tree (BIT)

```

class BIT:
    # Initialize BIT with given size
    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    # Update BIT at index i (1-indexed) with value delta
    def update(self, i, delta):

```

```
        while i <= self.size:
            self.tree[i] += delta
            i += i & -i

# Query BIT: get the prefix sum for index i (1-indexed)
def query(self, i):
    result = 0
    while i > 0:
        result += self.tree[i]
        i -= i & -i

    return result

def numberOfPairs(nums1, nums2, diff):
    n = len(nums1)
    # Compute the difference array where each element is nums1[i] - nums2[i]
    diff_arr = [nums1[i] - nums2[i] for i in range(n)]

    # Prepare a sorted list of all values that we will query/update:
    # Include each diff_arr[j] and diff_arr[j] + diff for proper
    discretization.
    all_values = []

    for val in diff_arr:
        all_values.append(val)
        all_values.append(val + diff)
    sorted_values = sorted(set(all_values))

    # Map each value to its discretized index (1-indexed for BIT)
    value_to_index = {value: i+1 for i, value in enumerate(sorted_values)}

    # Initialize BIT based on the number of unique values
    bit = BIT(len(sorted_values))
```

```

result = 0

# Iterate over diff_arr elements
for val in diff_arr:
    # Query BIT: count number of previous indices where stored value <=
    current val + diff
    index_query = value_to_index[val + diff]
    result += bit.query(index_query)
    # Add current diff_arr value into BIT for later queries
    index_update = value_to_index[val]
    bit.update(index_update, 1)

return result

# Example usage:
nums1 = [3, 2, 5]
nums2 = [2, 2, 1]
diff = 1
print(numberOfPairs(nums1, nums2, diff)) # Output should be 3

```

◆ Quick Review

Key Insights

- Transform the problem by defining a new array value: $\text{diff_arr}[i] = \text{nums1}[i] - \text{nums2}[i]$.
- The inequality becomes: $\text{diff_arr}[i] \leq \text{diff_arr}[j] + \text{diff}$ for $i \leq j$.
- For every index j , count how many earlier indices i have a diff_arr value $\leq \text{diff_arr}[j] + \text{diff}$.
- Use data structures such as Binary Indexed Tree (Fenwick Tree) or balanced BST to count

the number of valid indices efficiently.

- Discretize the values since `nums1` and `nums2` lie in a moderate range but the computed differences can be shifted by `diff`.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting for discretization and BIT operations for each element.

Space Complexity: $O(n)$ for the BIT and the discretized mapping.

Solution

The idea is to pre-compute an array `diff_arr` where each element is computed as `nums1[i] - nums2[i]`. The problem then reduces to counting, for each index `j`, the number of indices `i < j` that satisfy: `diff_arr[i] <= diff_arr[j] + diff`. A Binary Indexed Tree (BIT) is used to maintain counts of previously seen `diff_arr` values (after discretization). We discretize the possible values (both `diff_arr` and `diff_arr[j] + diff`) to map them onto indices in the BIT. As we iterate through `diff_arr` from left to right, we query the BIT for how many indices have values less

than or equal to $\text{diff_arr}[j] + \text{diff}$. Then update the BIT with the current $\text{diff_arr}[j]$. This counts all valid pairs in an efficient manner.

Binary Indexed Tree / Segment Tree

□ #3161 Block Placement Queries

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Binary Indexed Tree ·
Segment Tree · Ordered Set
Lists: AlgoMaster 600

Problem

There exists an infinite number line, with its origin at 0 and extending towards the positive x -axis.

You are given a 2D array queries, which contains two types of queries:

1. For a query of type 1, queries[i] = [1, x]. Build an obstacle at distance x from the origin. It is guaranteed that there is no obstacle at distance x when the query is asked.
2. For a query of type 2, queries[i] = [2, x, sz]. Check if it is possible to place a block of size sz anywhere in the range $[0, x]$ on the line, such that the block entirely lies in the range $[0, x]$. A block cannot be placed if it intersects with any obstacle, but it may touch it. Note

that you do not actually place the block.

Queries are separate.

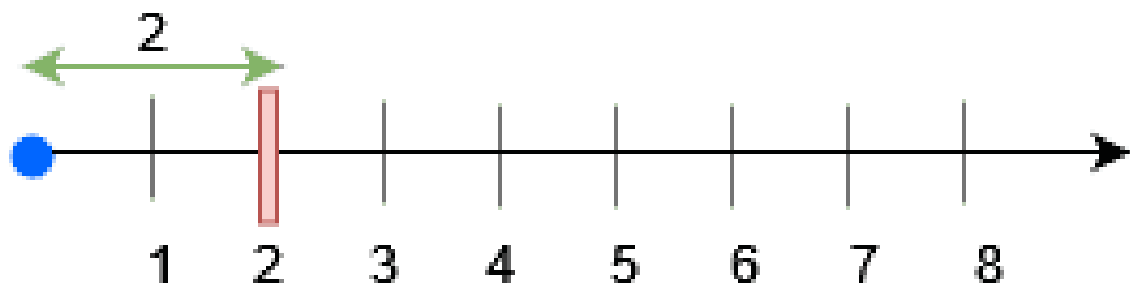
Return a boolean array results, where results[i] is true if you can place the block specified in the ith query of type 2, and false otherwise.

Example 1:

Input: queries = [[1,2],[2,3,3],[2,3,1],[2,2,2]]

Output: [false,true,true]

Explanation:



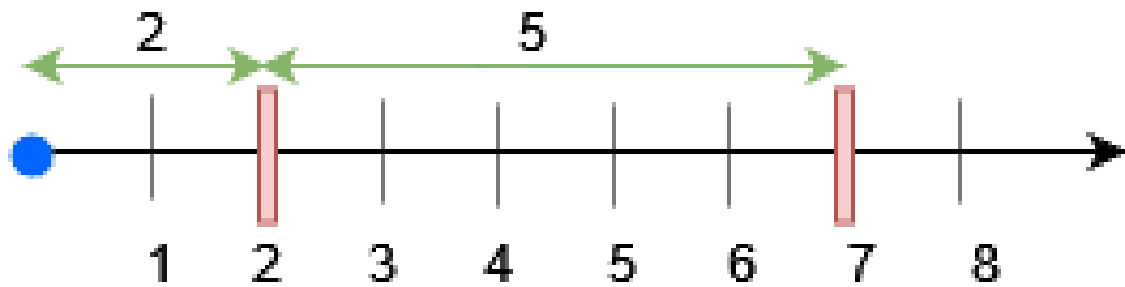
For query 0, place an obstacle at $x = 2$. A block of size at most 2 can be placed before $x = 3$.

Example 2:

Input: queries = [[1,7],[2,7,6],[1,2],[2,7,5],[2,7,6]]

Output: [true,true,false]

Explanation:



- Place an obstacle at $x = 7$ for query 0. A block of size at most 7 can be placed before $x = 7$.
- Place an obstacle at $x = 2$ for query 2. Now, a block of size at most 5 can be placed before $x = 7$, and a block of size at most 2 before $x = 2$.

Constraints:

- $1 \leq \text{queries.length} \leq 15 * 10^4$
- $2 \leq \text{queries}[i].\text{length} \leq 3$
- $1 \leq \text{queries}[i][0] \leq 2$
- $1 \leq x, sz \leq \min(5 * 10^4, 3 * \text{queries.length})$
- The input is generated such that for queries of type 1, no obstacle exists at distance x when the query is asked.

- The input is generated such that there is at least one query of type 2.

Community Solutions (Python)

• WalkCC

```
from sortedcontainers import SortedList

class FenwickTree:
    def __init__(self, n: int):
        self.vals = [0] * (n + 1)

    def maximize(self, i: int, val: int) -> None:
        while i < len(self.vals):
            self.vals[i] = max(self.vals[i], val)

            i += FenwickTree.lowtree(i)

    def get(self, i: int) -> int:
        res = 0
        while i > 0:
            res = max(res, self.vals[i])
            i -= FenwickTree.lowtree(i)
        return res

    @staticmethod
    def lowtree(i: int) -> int:
        return i & -i

class Solution:
    def getResults(self, queries: list[list[int]]) -> list[bool]:
        n = min(50000, len(queries) * 3)
        ans = []
        tree = FenwickTree(n + 1)
        obstacles = SortedList([0, n]) # sentinel values
```

```

for query in queries:
    type = query[0]
    if type == 1:
        x = query[1]
        obstacles.add(x)

for x1, x2 in itertools.pairwise(obstacles):
    tree.maximize(x2, x2 - x1)

for query in reversed(queries):
    type = query[0]
    x = query[1]
    if type == 1:
        i = obstacles.index(x)
        next = obstacles[i + 1]
        prev = obstacles[i - 1]
        obstacles.remove(x)
        tree.maximize(next, next - prev)
    else:
        sz = query[2]
        i = obstacles.bisect_right(x)
        prev = obstacles[i - 1]
        ans.append(tree.get(prev) >= sz or x - prev >= sz)

return ans[::-1]

```

• SimplyLeet

Python solution using Segment Tree

A segment tree node which holds the computed free lengths.

class SegmentTreeNode:

def __init__(self, start, end):

self.start = start

left index of the segment

self.end = end

right index of the segment

self.total = end - start + 1

total number of positions in the

segment

self.prefix = self.total

contiguous free length from the start

self.suffix = self.total

contiguous free length up to the end

```
        self.best = self.total          # maximum contiguous free length in
this segment
        self.left = None                # left child
        self.right = None               # right child

# Segment Tree class that supports update and query.
class SegmentTree:
    def __init__(self, start, end):
        self.root = self.build(start, end)

    def build(self, start, end):

        node = SegmentTreeNode(start, end)
        if start == end:
            return node
        mid = (start + end) // 2
        node.left = self.build(start, mid)
        node.right = self.build(mid + 1, end)
        return node

    def update(self, node, idx, value):
        # Update the node corresponding to idx with value (1 for free, 0 for
        blocked)

        if node.start == node.end:
            node.prefix = value
            node.suffix = value
            node.best = value
            return
        if idx <= node.left.end:
            self.update(node.left, idx, value)
        else:
            self.update(node.right, idx, value)
        # Merge left and right children
```

```

node.prefix = node.left.prefix
if node.left.prefix == node.left.total:
    node.prefix = node.left.total + node.right.prefix
node.suffix = node.right.suffix
if node.right.suffix == node.right.total:
    node.suffix = node.right.total + node.left.suffix
node.best = max(node.left.best, node.right.best, node.left.suffix + node.right.prefix)

```

```

def query(self, node, l, r):
    # Return a tuple: (prefix, suffix, best, total) for the query interval [l, r]

```

```

if node.start > r or node.end < l:
    return (0, 0, 0, 0)
if l <= node.start and node.end <= r:
    return (node.prefix, node.suffix, node.best, node.total)
left_res = self.query(node.left, l, r)
right_res = self.query(node.right, l, r)
total = left_res[3] + right_res[3]
prefix = left_res[0]
if left_res[0] == left_res[3]:
    prefix = left_res[3] + right_res[0]

```

```

suffix = right_res[1]
if right_res[1] == right_res[3]:
    suffix = right_res[3] + left_res[1]
best = max(left_res[2], right_res[2], left_res[1] + right_res[0])
return (prefix, suffix, best, total)

```

Class to handle block placement queries.

```

class BlockPlacement:
    def __init__(self, max_x):
        self.max_x = max_x

```

```

        self.segment_tree = SegmentTree(0, max_x)

    def build_obstacle(self, x):
        # When an obstacle is built, mark the position as blocked (0)
        self.segment_tree.update(self.segment_tree.root, x, 0)

    def can_place_block(self, x, sz):
        # Query the segment tree for range [0, x] and check if best free
        segment >= sz.
        res = self.segment_tree.query(self.segment_tree.root, 0, x)
        return res[2] >= sz

def blockPlacementQueries(queries):
    # Find maximum x in queries to initialize the segment tree.
    M = 0
    for q in queries:
        if q[0] == 1 or q[0] == 2:
            M = max(M, q[1])
    bp = BlockPlacement(M)
    results = []
    for q in queries:
        if q[0] == 1:
            bp.build_obstacle(q[1])
        else:
            results.append(bp.can_place_block(q[1], q[2]))
    return results

# Example:
# print(blockPlacementQueries([[1,2],[2,3,3],[2,3,1],[2,2,2]]))

```

◆ Quick Review

Key Insights

- Placing obstacles on the number line splits $[0, x]$ into free intervals.

- A block of length sz can be placed if at least one free interval (or a truncated free interval at the boundary x) has length at least sz .
- Using a segment tree that “remembers” the longest contiguous free segment is a great way to support both dynamic updates (when an obstacle is added) and range queries (to check for a large enough free interval).
- In the segment tree, each node will store:
 - prefix length: the contiguous free segment starting at the left end,
 - suffix length: the contiguous free segment ending at the right end,
 - best: the maximum free contiguous interval within the segment,
 - total: the length (number of positions) of the segment.
- For an update (type 1 query), we mark the position as blocked (i.e. “0”) and update the tree.
- For a query (type 2), we query the segment tree for the range $[0, x]$ and check if the “best” free length is at least sz .

Space & Time

Time Complexity:

- Update (placing an obstacle): $O(\log M)$ per update, where M is the maximum possible position (here, $M \leq 50000$).
- Query (checking free interval): $O(\log M)$ per query.

Space Complexity: $O(M)$ to store the segment tree.

Solution

We solve the problem by modeling the positions $[0, M]$ (with M equal to the maximum x encountered in the queries) in a segment tree. Initially, all positions are free. Each leaf node of the segment tree represents a single position, and its values (prefix, suffix, best) are initialized to 1. When an obstacle is placed at a position x , we update that leaf to 0 (blocked). The segment tree nodes merge their children's values so that each node accurately reflects:

- prefix: if the entire left child is free, then the free length extends into the right child's prefix,
- suffix: similarly using the right child,
- best: the larger of the left child's best, the right child's best, or a combination of the left

child's suffix and the right child's prefix. For a query, we fetch the node information for the interval $[0, x]$ and check if the best free contiguous segment length is at least sz .

Binary Indexed Tree / Segment Tree

□ #3721 Longest Balanced Subarray II

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Divide and Conquer ·
Segment Tree · Prefix Sum
Lists: AlgoMaster 600

Problem

You are given an integer array `nums`.

A subarray is called balanced if the number of distinct even numbers in the subarray is equal to the number of distinct odd numbers.

Return the length of the longest balanced subarray.

Example 1:

Input: `nums = [2,5,4,3]`

Output: 4

Explanation:

- The longest balanced subarray is `[2, 5, 4, 3]`.
- It has 2 distinct even numbers `[2, 4]` and 2 distinct odd numbers `[5, 3]`. Thus, the answer is 4.

Example 2:

Input: `nums = [3,2,2,5,4]`

Output: 5

Explanation:

- The longest balanced subarray is [3, 2, 2, 5, 4].
- It has 2 distinct even numbers [2, 4] and 2 distinct odd numbers [3, 5]. Thus, the answer is 5.

Example 3:

Input: `nums = [1,2,3,2]`

Output: 3

Explanation:

- The longest balanced subarray is [2, 3, 2].
- It has 1 distinct even number [2] and 1 distinct odd number [3]. Thus, the answer is 3.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i] \leq 105$

Community Solutions (Python)

- LeetDoocs

```

class Solution:
    def longestBalanced(self, nums: List[int]) -> int:
        n = len(nums)

        class Node:
            __slots__ = ("l", "r", "mn", "mx", "lazy")
            def __init__(self):
                self.l = self.r = 0
                self.mn = self.mx = 0
                self.lazy = 0

        tr = [Node() for _ in range((n + 1) * 4)]

        def build(u: int, l: int, r: int):
            tr[u].l, tr[u].r = l, r
            tr[u].mn = tr[u].mx = tr[u].lazy = 0
            if l == r:
                return
            mid = (l + r) >> 1
            build(u << 1, l, mid)

            build(u << 1 | 1, mid + 1, r)

        def apply(u: int, v: int):
            tr[u].mn += v
            tr[u].mx += v
            tr[u].lazy += v

        def pushdown(u: int):
            if tr[u].lazy:
                apply(u << 1, tr[u].lazy)

            apply(u << 1 | 1, tr[u].lazy)
            tr[u].lazy = 0

        def pushup(u: int):
            tr[u].mn = min(tr[u << 1].mn, tr[u << 1 | 1].mn)
            tr[u].mx = max(tr[u << 1].mx, tr[u << 1 | 1].mx)

        def modify(u: int, l: int, r: int, v: int):
            if tr[u].l >= l and tr[u].r <= r:
                apply(u, v)

```

```

        return
    pushdown(u)
    mid = (tr[u].l + tr[u].r) >> 1
    if l <= mid:
        modify(u << 1, l, r, v)
    if r > mid:
        modify(u << 1 | 1, l, r, v)
    pushup(u)

def query(u: int, target: int) -> int:

    if tr[u].l == tr[u].r:
        return tr[u].l
    pushdown(u)
    if tr[u << 1].mn <= target <= tr[u << 1].mx:
        return query(u << 1, target)
    return query(u << 1 | 1, target)

build(1, 0, n)

last = {}

now = ans = 0

for i, x in enumerate(nums, start=1):
    det = 1 if (x & 1) else -1
    if x in last:
        modify(1, last[x], n, -det)
        now -= det
    last[x] = i
    modify(1, i, n, det)
    now += det

    pos = query(1, now)
    ans = max(ans, i - pos)

return ans

```

Line Sweep

Round 9 · Low · Hard · 2 questions

Line Sweep

□ #391 Perfect Rectangle

HAR

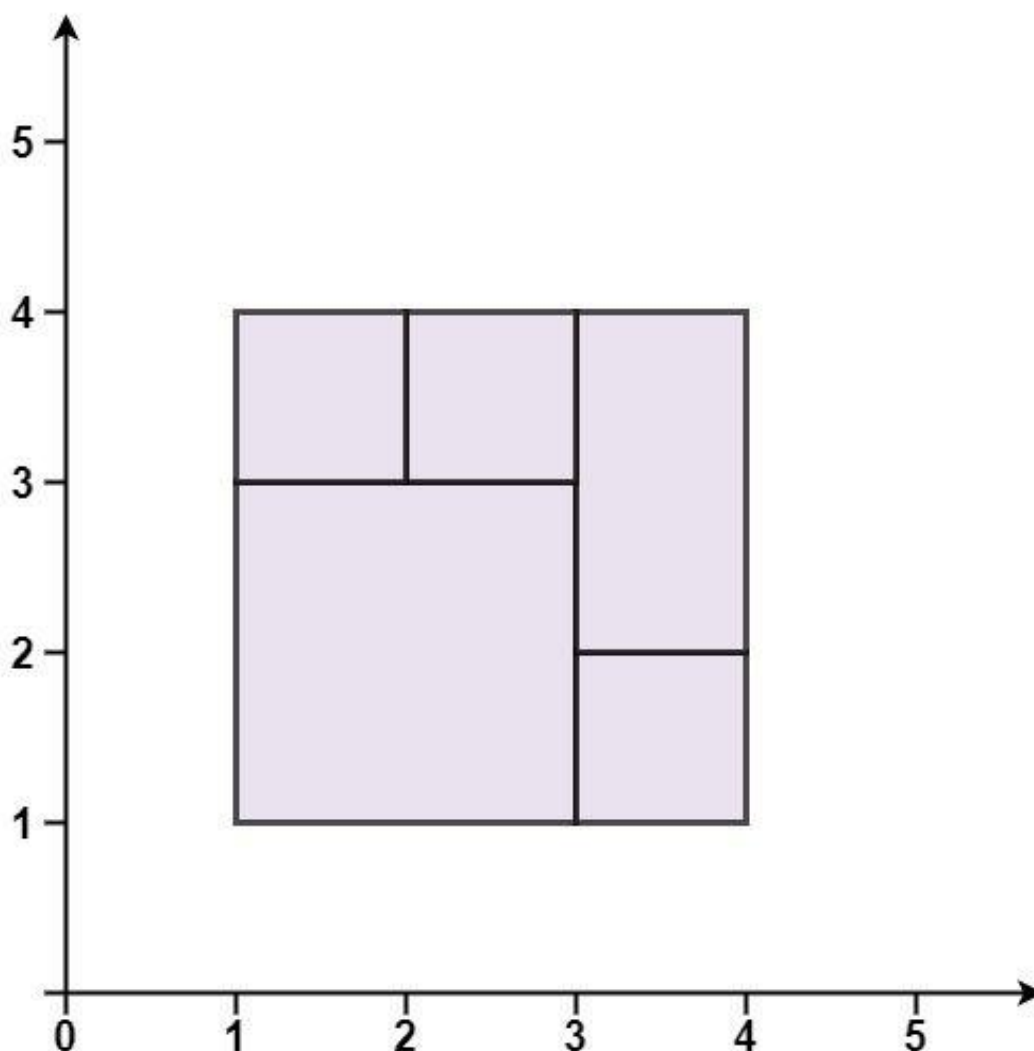
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Geometry · Sweep
Line

Lists: AlgoMaster 600

Problem

Given an array `rectangles` where `rectangles[i] = [xi, yi, ai, bi]` represents an axis-aligned rectangle. The bottom-left point of the rectangle is (xi, yi) and the top-right point of it is (ai, bi) . Return `true` if all the rectangles together form an exact cover of a rectangular region.

Example 1:

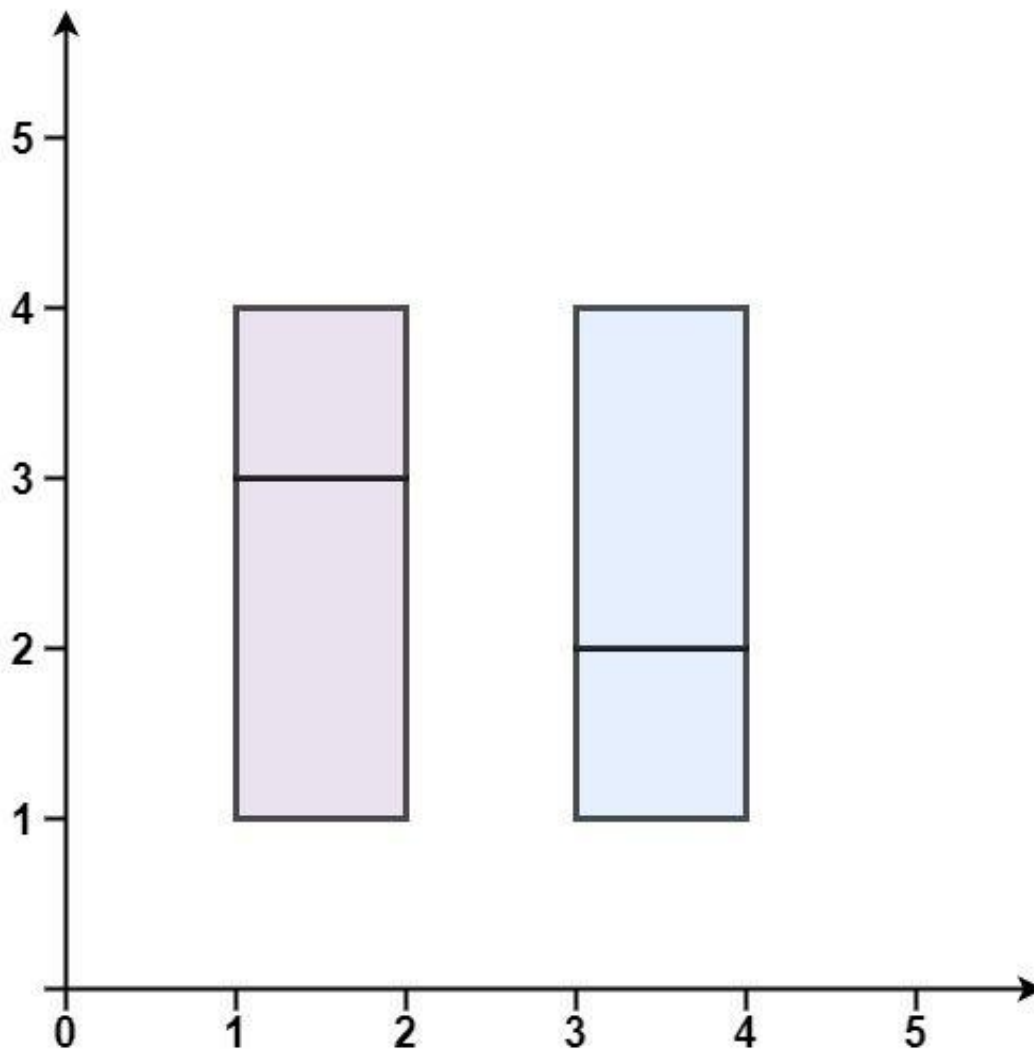


Input: rectangles = [[1,1,3,3],[3,1,4,2],[3,2,4,4],[1,3,2,4],[2,3,3,4]]

Output: true

Explanation: All 5 rectangles together form an exact cover of a rectangular region.

Example 2:

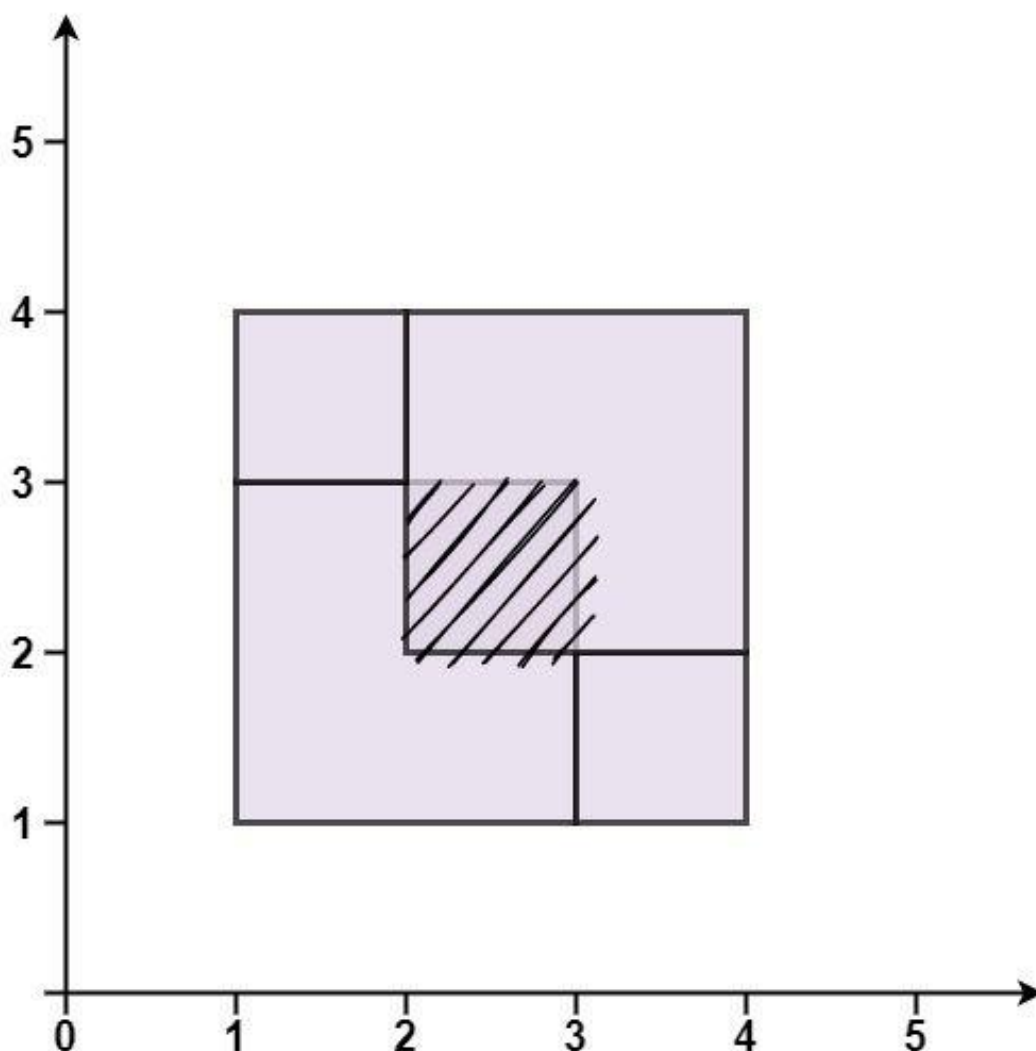


Input: rectangles = [[1,1,2,3],[1,3,2,4],[3,1,4,2],[3,2,4,4]]

Output: false

Explanation: Because there is a gap between the two rectangular regions.

Example 3:



Input: rectangles = [[1,1,3,3],[3,1,4,2],[1,3,2,4],[2,2,4,4]]

Output: false

Explanation: Because two of the rectangles overlap with each other.

Constraints:

- $1 \leq \text{rectangles.length} \leq 2 * 10^4$
- $\text{rectangles}[i].\text{length} == 4$
- $-105 \leq x_i < a_i \leq 105$
- $-105 \leq y_i < b_i \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isRectangleCover(self, rectangles: list[list[int]]) -> bool:
        area = 0
        x1 = math.inf
        y1 = math.inf
        x2 = -math.inf
        y2 = -math.inf
        corners: set[tuple[int, int]] = set()

        for x, y, a, b in rectangles:

            area += (a - x) * (b - y)
            x1 = min(x1, x)
            y1 = min(y1, y)
            x2 = max(x2, a)
            y2 = max(y2, b)

            # the four points of the current rectangle
            for point in [(x, y), (x, b), (a, y), (a, b)]:
                if point in corners:
                    corners.remove(point)

            else:
                corners.add(point)

        if len(corners) != 4:
            return False
        if ((x1, y1) not in corners or
            (x1, y2) not in corners or
            (x2, y1) not in corners or
            (x2, y2) not in corners):
            return False

        return area == (x2 - x1) * (y2 - y1)
```

• LeetCode

```

class Solution:
    def isRectangleCover(self, rectangles: List[List[int]]) -> bool:
        area = 0
        minX, minY = rectangles[0][0], rectangles[0][1]
        maxX, maxY = rectangles[0][2], rectangles[0][3]
        cnt = defaultdict(int)

        for r in rectangles:
            area += (r[2] - r[0]) * (r[3] - r[1])

            minX = min(minX, r[0])
            minY = min(minY, r[1])
            maxX = max(maxX, r[2])
            maxY = max(maxY, r[3])

            cnt[(r[0], r[1])] += 1
            cnt[(r[0], r[3])] += 1
            cnt[(r[2], r[3])] += 1
            cnt[(r[2], r[1])] += 1

        if (
            area != (maxX - minX) * (maxY - minY)
            or cnt[(minX, minY)] != 1
            or cnt[(minX, maxY)] != 1
            or cnt[(maxX, maxY)] != 1
            or cnt[(maxX, minY)] != 1
        ):
            return False

        del cnt[(minX, minY)], cnt[(minX, maxY)], cnt[(maxX, maxY)], cnt[(maxX,
, minY)]

        return all(c == 2 or c == 4 for c in cnt.values())

```

• SimplyLeet

```
def isRectangleCover(rectangles):
    # Initialize bounding rectangle coordinates
    min_x = float('inf')
    min_y = float('inf')
    max_x = float('-inf')
    max_y = float('-inf')
    total_area = 0
    # Set to track all corners
    corners = set()

    for rect in rectangles:
        x, y, a, b = rect[0], rect[1], rect[2], rect[3]
        # Update overall bounding box
        min_x = min(min_x, x)
        min_y = min(min_y, y)
        max_x = max(max_x, a)
        max_y = max(max_y, b)
        # Calculate area of the current rectangle and add to total area
        total_area += (a - x) * (b - y)

    # List of current rectangle's corners
    current_corners = [(x, y), (x, b), (a, y), (a, b)]
    # Add or remove corners from the set
    for point in current_corners:
        if point in corners:
            corners.remove(point)
        else:
            corners.add(point)

    # Check if total area matches the area of the overall bounding rectangle
```

```
expected_area = (max_x - min_x) * (max_y - min_y)
if total_area != expected_area:
    return False

# The set must contain exactly the four corners of the overall rectangle
expected_corners = {(min_x, min_y), (min_x, max_y), (max_x, min_y), (max_x,
max_y)}
return corners == expected_corners

# Example usage:
rectangles = [[1,1,3,3],[3,1,4,2],[3,2,4,4],[1,3,2,4],[2,3,3,4]]

print(isRectangleCover(rectangles)) # Output: True
```

◆ Quick Review

Key Insights

- Compute the overall bounding rectangle by taking the minimum x, minimum y, maximum a, and maximum b from all rectangles.
- Sum the areas of all given rectangles; this should equal the area of the overall bounding rectangle.
- Use a set to track all four corners of each rectangle. For each rectangle, add its four corners if they are seen for the first time and remove them if they appear again.
- At the end, the set should contain exactly the four corners of the overall bounding rectangle. Any extra points indicate either overlapping or

gaps.

Space & Time

Time Complexity: $O(n)$, where n is the number of rectangles, because we iterate through each rectangle exactly once.

Space Complexity: $O(n)$ in the worst-case scenario for storing corner points in the set.

Solution

The solution is based on two main observations. First, the total area of all small rectangles must equal the area of the overall bounding rectangle defined by the extreme coordinates. Second, by using a set to add and remove corner points of every rectangle, all internal points (which occur more than once) cancel out, leaving only the four corner points of the bounding rectangle. If the set does not exactly match these four corners, then the rectangles do not form a perfect cover.

Line Sweep

□ #850 Rectangle Area II

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Segment Tree · Sweep Line · Ordered
Set

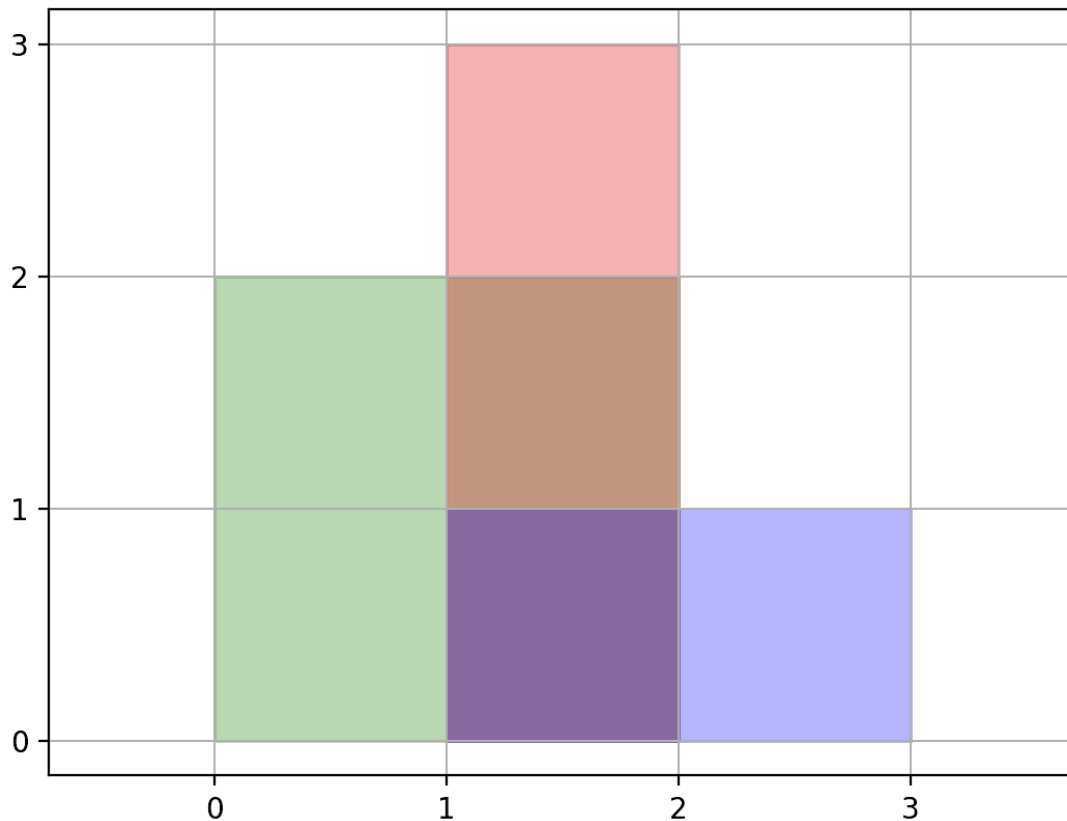
Lists: AlgoMaster 600

Problem

You are given a 2D array of axis-aligned rectangles. Each $\text{rectangle}[i] = [x_{i1}, y_{i1}, x_{i2}, y_{i2}]$ denotes the i th rectangle where (x_{i1}, y_{i1}) are the coordinates of the bottom-left corner, and (x_{i2}, y_{i2}) are the coordinates of the top-right corner. Calculate the total area covered by all rectangles in the plane. Any area covered by two or more rectangles should only be counted once.

Return the total area. Since the answer may be too large, return it modulo $10^9 + 7$.

Example 1:



Input: rectangles = `[[0,0,2,2],[1,0,2,3],[1,0,3,1]]`

Output: 6

Explanation: A total area of 6 is covered by all three rectangles, as illustrated in the picture.

From (1,1) to (2,2), the green and red rectangles overlap.

From (1,0) to (2,3), all three rectangles overlap.

Example 2:

Input: rectangles = `[[0,0,1000000000,1000000000]]`

Output: 49

Explanation: The answer is 1018 modulo (109 + 7), which is 49.

Constraints:

- $1 \leq \text{rectangles.length} \leq 200$
- $\text{rectangles}[i].\text{length} == 4$

- $0 \leq x_{i1}, y_{i1}, x_{i2}, y_{i2} \leq 109$
- $x_{i1} \leq x_{i2}$
- $y_{i1} \leq y_{i2}$
- All rectangles have non zero area.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rectangleArea(self, rectangles: list[list[int]]) -> int:
        events = []

        for x1, y1, x2, y2 in rectangles:
            events.append((x1, y1, y2, 's'))
            events.append((x2, y1, y2, 'e'))

        events.sort(key=lambda x: x[0])

        ans = 0
        prevX = 0
        yPairs = []

        def getHeight(yPairs: list[tuple[int, int]]) -> int:
            height = 0
            prevY = 0

            for y1, y2 in yPairs:
                prevY = max(prevY, y1)

            return height - prevY
```

```

        if y2 > prevY:
            height += y2 - prevY
            prevY = y2

    return height

for currX, y1, y2, type in events:
    if currX > prevX:
        width = currX - prevX
        ans += width * getHeight(yPairs)

    prevX = currX
    if type == 's':
        yPairs.append((y1, y2))
        yPairs.sort()
    else: # type == 'e'
        yPairs.remove((y1, y2))

return ans % (10**9 + 7)

```

• LeetDoocs

```

class Node:
    def __init__(self):
        self.l = self.r = 0
        self.cnt = self.length = 0

class SegmentTree:
    def __init__(self, nums):
        n = len(nums) - 1
        self.nums = nums

        self.tr = [Node() for _ in range(n << 2)]
        self.build(1, 0, n - 1)

    def build(self, u, l, r):
        self.tr[u].l, self.tr[u].r = l, r
        if l != r:
            mid = (l + r) >> 1
            self.build(u << 1, l, mid)
            self.build(u << 1 | 1, mid + 1, r)

```

```

def modify(self, u, l, r, k):
    if self.tr[u].l >= l and self.tr[u].r <= r:
        self.tr[u].cnt += k
    else:
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if l <= mid:
            self.modify(u << 1, l, r, k)
        if r > mid:
            self.modify(u << 1 | 1, l, r, k)
    self.pushup(u)

```

```

def pushup(self, u):
    if self.tr[u].cnt:
        self.tr[u].length = self.nums[self.tr[u].r + 1] - self.nums[self.t
r[u].l]
    elif self.tr[u].l == self.tr[u].r:
        self.tr[u].length = 0
    else:
        self.tr[u].length = self.tr[u << 1].length + self.tr[u << 1 | 1].l
length

```

```
@property
```

```

def length(self):
    return self.tr[1].length

```

```

class Solution:
    def rectangleArea(self, rectangles: List[List[int]]) -> int:
        segs = []
        alls = set()
        for x1, y1, x2, y2 in rectangles:
            segs.append((x1, y1, y2, 1))

```

```

        segs.append((x2, y1, y2, -1))
        alls.update([y1, y2])

    segs.sort()
    alls = sorted(alls)
    tree = SegmentTree(alls)
    m = {v: i for i, v in enumerate(alls)}
    ans = 0
    for i, (x, y1, y2, k) in enumerate(segs):
        if i:
            ans += tree.length * (x - segs[i - 1][0])
            tree.modify(1, m[y1], m[y2] - 1, k)
    ans %= int(1e9 + 7)
    return ans

```

• SimplyLeet

Python solution using segment tree with coordinate compression

MOD = 10**9 + 7

Helper function to build and update the segment tree

class SegmentTree:

def __init__(self, ys):

self.ys = ys # sorted unique y values

self.n = len(ys) - 1 # intervals count

count: how many times a segment is covered

length: the total covered length in that segment

self.count = [0] * (4 * self.n)

self.length = [0] * (4 * self.n)

def update(self, idx, left, right, ql, qr, val):

if ql >= right or qr <= left: # no overlap

return

if ql <= left and right <= qr: # total overlap

self.count[idx] += val

else:

```

        mid = (left + right) // 2
        self.update(idx * 2, left, mid, ql, qr, val)
        self.update(idx * 2 + 1, mid, right, ql, qr, val)

    if self.count[idx] > 0:
        # If count > 0, entire interval is covered
        self.length[idx] = self.ys[right] - self.ys[left]
    else:
        if right - left == 1:
            self.length[idx] = 0

        else:
            self.length[idx] = self.length[idx * 2] + self.length[idx * 2
+ 1]

def rectangleArea(rectangles):
    events = []
    ys = set()
    # Prepare events and collect y values for compression
    for x1, y1, x2, y2 in rectangles:
        events.append((x1, y1, y2, 1)) # entering event
        events.append((x2, y1, y2, -1)) # leaving event

    ys.add(y1)
    ys.add(y2)

    # Sort unique y coordinates and events by x-coordinate
    ys = sorted(ys)
    events.sort()

    st = SegmentTree(ys)
    prev_x = events[0][0]
    area = 0

```

```

for x, y1, y2, typ in events:
    dx = x - prev_x
    # st.length[1] holds the total covered length in the y direction
    area = (area + dx * st.length[1]) % MOD
    # Find indices for y1 and y2 in the compressed list
    left = ys.index(y1)
    right = ys.index(y2)
    st.update(1, 0, st.n, left, right, typ)
    prev_x = x
return area

```

```

# Example usage:
# print(rectangleArea([[0,0,2,2],[1,0,2,3],[1,0,3,1]]))

```

◆ Quick Review

Key Insights

- Use a line sweep algorithm along the x-axis to break the problem into vertical slices.
- For each vertical slice, compute the union of the active y-intervals.
- Apply coordinate compression on the y-values since they can be very large.
- Utilize a segment tree (or an equivalent data structure) to efficiently track and update the length of y-intervals that are currently active.
- Process two event types per rectangle: one when entering (adding a y-interval) and one when leaving (removing a y-interval).

Space & Time

Time Complexity: $O(n \log n)$ – sorting events and processing them using the segment tree.

Space Complexity: $O(n)$ – for storing events, the list of unique y -coordinates, and the segment tree structure.

Solution

The solution involves sweeping a vertical line across the x -axis and handling events where a rectangle starts or ends. For each rectangle, generate two events: one for adding the corresponding y -interval when the sweep line hits the left edge, and one for removing the interval when hitting the right edge. The y -coordinates are compressed to allow efficient segment tree operations. The segment tree keeps track of the coverage count and the total covered length on the y -axis. For each segment between consecutive x -values, multiply the width of the segment (delta in x) by the current union length of y -intervals to accumulate the area. Finally, return the accumulated area modulo 10^9+7 .